O'REILLY®

AI Systems Performance Engineering

Optimizing Model Training and Inference Workloads with GPUs, CUDA, and PyTorch

Chris Fregly

对《*AI Systems Performance Engineering*》的赞誉

AI 系统正通过密集的研究与开发飞速演进，并充满多层复杂性。Chris 将这种复杂性逐层拆解，以所有人都能从中受益的方式加以阐释，提供了一份将在未来多年树立标准的独特参考资料。

—Chris Lattner，Modular 首席执行官

理解 LLM 如何工作只是开始。通过模型和硬件优化充分发挥它们的能力，才是优秀实践者与卓越实践者的分野。Chris 的书正是缺失的参考手册，告诉你如何做到这一点。

—Sebastian Raschka，博士，机器学习/AI 研究员，畅销书《*从零构建大语言模型*》（Manning，2024）作者

这里的覆盖范围令人惊叹——CUDA 内核、分布式训练、PyTorch 编译器内部机制、解耦式推理，全都汇集于一处。Chris 把通常分散在无数博客文章和论文中的内容整合起来。这是一部出色的 ML 系统百科全书。

—Mark Saroufim，Meta 的 PyTorch 工程师、GPU MODE 社区创始人

驾驭大规模 AI 系统是我们这个时代的重大挑战之一。Chris Fregly 为这一新前沿写下了权威的实战指南。他巧妙地连接从芯片到应用的各个环节，提供每位 AI 工程师将原始算力转化为高效高性能模型所需的全栈智慧。

—Harsh Banwait，Coreweave 产品总监

对于希望深入现代 ML 系统工程的任何人，本书都是卓越资源。它聚焦 vLLM 和 llm-d 等先进项目的细节，体现出对推理优化技术以及开源软件变革力量的深刻理解。

—Michael Goin，vLLM 维护者、Red Hat 首席工程师

对于不甘于默认性能的 AI 工程师，本书是一份权威指南。无论你在调优 CUDA 内核、扩展 LLM 推理，还是跨 GPU 编排 AI 智能体，本书都提供精密工具和系统层面的清晰认识，助你高效运行现代 AI 工作负载。阅读并使用它，你将重新定义构建、扩展和思考现代 AI 系统的方式。《AI Systems Performance Engineering》是释放 AI 性能的一把万能钥匙。

—Arpitha Srinivas，全球领先的高性能、高能效 AI 服务器制造商的 AI 系统性能工程师

《AI Systems Performance Engineering》出色地连接了 GPU 硬件架构与现代 AI 工作负载优化，将严谨的底层调优与高层系统设计相结合。它涵盖 GPU 内存带宽优化、KV 缓存管理、批处理策略和多 GPU 推理，提供了在真实生产环境中淬炼出的洞见。Chris 让 Nsight 剖析、Transformer 注意力调优和分布式扩展等复杂主题变得清晰，是连接硬件、系统与 AI 的性能工程师必读之作。

—Amer Ather，Netflix 云与机器学习性能工程师

这是构建当代 AI 系统最全面、最新的指南。每位 AI/机器学习开发者和从业者都应阅读。

—Chaim Rand，AI/机器学习算法工程师

这是所有 AI 性能相关主题的首选参考资料。Chris 的最新著作内容丰富，在我日常优化和调优 AI 工作负载时大有帮助。他涵盖了当今所有 AI 系统性能问题，并给出了对每家试图将 AI 投入生产的公司都极具价值的解决方案。

—Antje Barth，Amazon AGI 技术人员

在这一领域，没有其他书籍能接近《AI Systems Performance Engineering》。每一章都展现了深厚专长，完全可以独立成书。内容既新鲜又经久不衰，也非常易于理解。

—Suman Debnath，Anyscale 机器学习系统工程师

这是一部力作，是从事最新 AI 应用的性能工程师必读之书。

—Adrian Cockcroft，OrionX.net（曾任 Netflix 云基础设施负责人及 AWS 工程副总裁）

这正是我一直等待的书。它将分散、庞大且快速变化的 AI 系统性能工程世界，汇聚成一份清晰而现代的资源。

—Madison Kanna，Baseten AI 工程师

随着 AI 成为现代计算的基础，理解加速器架构及其生态系统已不再是可选项，而是一项战略要务。Chris 将非凡的技术深度提炼为清晰易懂的叙述。对于任何在超大规模环境中从事工程或管理工作的人，本书都是不可或缺的起点。

—Omer Zaki，Oracle AI 基础设施副总裁

今天的 AI 是一个系统问题：必须针对硬件互连协同优化软件，才能获得峰值性能。Chris 揭开帷幕，展示现代 AI 工作负载所运行的 AI 软件与硬件栈的不同层次。

—Abdul Dakkak，Modular 的 GenAI 负责人

AI 系统性能工程

使用 GPU、CUDA 和 PyTorch 优化模型训练与推理工作负载

Chris Fregly

AI 系统性能工程

作者： Chris Fregly

版权所有 © 2026 Flux Capacitor, LLC。保留所有权利。

出版方：O’Reilly Media, Inc.，141 Stony Circle, Suite 195, Santa Rosa, CA 95401。

O’Reilly 图书可购买用于教育、商业或销售推广用途。大多数图书也提供在线版（https://oreilly.com）。欲了解更多信息，请联系企业/机构销售部门：800-998-9938 或 *corporate@oreilly.com*。

选题编辑：Nicole Butterfield

开发编辑：Angela Rufino

制作编辑：Kristen Brown

文字编辑：nSight, Inc.

校对：Piper Content Partners

索引编制：Sue Klefstad

封面设计：Susan Brown

封面插画：José Marzan Jr.

内页设计：David Futato

内页插画：Kate Dullea

2025 年 11 月： 第一版

第一版修订历史

- 2025-11-11: 首次发布
- 2026-01-16: 第二次发布
- 2026-03-13: 第三次发布

有关发布详情，请参阅 *http://oreilly.com/catalog/errata.csp?isbn=9798341627789* 。

O’Reilly 徽标是 O’Reilly Media, Inc. 的注册商标。*AI 系统性能工程*、封面图像及相关商业外观均为 O’Reilly Media, Inc. 的商标。

本书表达的观点仅代表作者本人，并不代表出版方观点。尽管出版方和作者已本着诚信尽力确保本书所载信息和说明准确，但对于错误或遗漏，以及因使用或依赖本书所导致的损害（包括但不限于此），出版方和作者概不负责。使用本书所载信息和说明，风险由读者自

行承担。若本书包含或描述的任何代码示例或其他技术受开源许可证或他人知识产权约束，读者有责任确保其使用符合相应许可证和/或权利要求。

979-8-341-62778-9

[LSI]

前言

在旧金山充满活力的街道上，创新如同美国 101 号公路上的自动驾驶车辆一样常见；我们正身处一个令人惊叹的人工智能世界。AI 的快速进步正在从各方面重新定义日常生活。过去 20 年间，我们经历了推荐引擎（2000 年代）、AI 助手（2010 年代）和完全自动驾驶汽车（2020 年代）。随着 AI 以极快速度发展并产生巨大社会影响，2030 年代将更加令人兴奋。

我进入快速发展的 AI 系统性能工程领域，源于希望理解前沿硬件、高度优化的软件和巧妙算法之间微妙平衡与协同设计的好奇心；它们共同支撑着如此复杂的系统和有影响力的应用。这一认识激励我深入“全栈”AI 性能工程领域。我想了解处理器、内存架构、网络互连、操作系统和软件框架等多个组件如何协同工作。这些交互的复杂性既带来挑战，也带来机遇，促使我深入探索这种独特的技术组合。

本书凝聚了我多年来作为一线 ML 和 AI 性能工程师的探索成果。我为渴望理解各层级 AI 系统性能基础的工程师、研究人员、实践者和爱好者创作了本书。读者可能正在构建 AI 应用、优化神经网络训练策略、设计和管理可扩展的推理服务器，也可能只是着迷于现代 AI 系统的运作机制。总之，本书提供跨越多个学科、连接理论与实践的洞见。

本书读者很可能已具备神经网络基础知识，并对 Python 和 ML 有基本了解。不过，即使没有这些基础，好奇的读者仍可跟随以硬件、软件和算法第一性原理为基础的多维协同设计性能叙事。我相信本书能为每类读者提供一些收获，并确信每位读者都能从中学到新东西。

在各章中，我们将考察硬件架构的演进，深入软件优化的细节，并探索现实案例，突出构建高性能且经济高效的 AI 系统所需的模式和最佳实践。每一节都建立在前一节基础上，覆盖从基本概念到高级应用的全部内容。

本书使用的约定

本书使用以下排版约定：

Italic

表示新术语、URL、电子邮件地址、文件名和文件扩展名。

`Constant width`

用于程序清单，也用于段落中指代变量名、函数名、数据库、数据类型、环境变量、语句和关键字等程序元素。

`Constant width bold`

表示用户应原样键入的命令或其他文本。

`Constant width italic`

表示应替换为用户提供的值或由上下文确定的值的文本。

该元素表示提示或建议。

该元素表示一般性说明。

使用代码示例

补充材料（代码示例、练习等）可从 *https://github.com/cfregly/ai-performance-engineering* 下载。

如有技术问题或使用代码示例时遇到问题，请发送电子邮件至 *support@oreilly.com*。

本书旨在帮助你完成工作。一般而言，如本书提供示例代码，你可在自己的程序和文档中使用它们；除非复制了大量代码，否则无需联系我们取得许可。例如，编写使用本书若干代码片段的程序不需要许可；出售或分发 O’Reilly 图书中的示例则需要许可。引用本书和示例代码来回答问题不需要许可；将本书大量示例代码纳入产品文档则需要许可。

我们感谢但通常不要求署名。署名通常包括书名、作者、出版商和 ISBN。例如：“Chris Fregly 著《*AI Systems Performance Engineering*》（O’Reilly）。Copyright 2026 Flux Capacitor, LLC, 979-8-341-62778-9。”

如认为你对代码示例的使用超出合理使用或上述许可范围，请通过 *permissions@oreilly.com* 联系我们。

O’Reilly 在线学习

40 多年来，*O’Reilly Media* 一直提供技术与商业培训、知识和洞见，帮助企业取得成功。

我们独特的专家与创新者网络通过图书、文章和在线学习平台分享知识与专长。O’Reilly 在线学习平台按需提供直播培训课程、深入学习路径、交互式编程环境，以及来自 O’Reilly 和 200 多家其他出版商的大量文字和视频资料。欲了解更多信息，请访问 *https://oreilly.com*。

如何联系我们

有关本书的意见和问题，请联系出版商：

O’Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969（美国或加拿大境内）

707-827-7019（国际或本地）

707-829-0104（传真）

support@oreilly.com

https://oreilly.com/about/contact.html

我们为本书设有网页，列出勘误和其他补充信息。可在 *https://oreil.ly/AI-SysPerfEng* 访问该页面。

有关图书和课程的新闻与信息，请访问 *https://oreilly.com*。

在 LinkedIn 上关注我们：*https://linkedin.com/company/oreilly-media*。

在 YouTube 上观看我们的视频：*https://youtube.com/oreillymedia*。

致谢

尽管我是本书唯一作者，但这项工作建立在许多献身于这些主题的实践者和学者的成果之上。我感谢那些以研究和创新为本书涉及的主题铺平道路的杰出人士；他们的贡献塑造了本书的内容和深度。我要感谢全程审阅者 Adrian Cockroft、Antje Barth、Arpitha Moorthy、Chaim Rand、Fabrizio Milo、Simon Zamarin、Suneeta Mall 和 Todd Bezenek，以及众多显著提升本书整体质量的兼职审阅者。你们挑战了我以及我对这些复杂主题的理解；你们的洞见融入了本书的每一页。

我尤其要感谢 Meta 的 Mark Saroufim。他和成功的 GPU Mode community 让我认识到：需要这类覆盖 PyTorch、CUDA 和 GPU 软硬件协同设计的深入技术内容。我深知维护和发展一个面向此类高级主题的大型社群需要投入多少工作。

我还要向亲爱的家人 Ann Marie、Meredith、Tommy 和 Conor 致以衷心感谢。你们持续的支持让我能够彻夜写作直到清晨。若过去一年探望你们时我并非总是“在场”，还请原谅……我当时在地下室研究、编程和写作！

还有一种颇为特别的感谢：我要感谢我的汽车 Little Red Whiting Hood（图 P-1）。你是一个由高度调优的硬件和软件构成的复杂跨学科系统。自十五年多前我开启旧金山之旅时起，我就拥有了你。每次开你，都让我想起自己为何成为工程师。若按相对的 GPU 世代来比，你就是 NVIDIA Ampere A100——由跨学科工程师采用当时可获得的最高性能组件打造的永恒经典。尽管你最初是为风冷（Luftkühlung）设计的，但你明白要获得最大性能，必须采用水冷（Wasserkühlung）。你的创造者一代又一代固执而不懈地推动着你的

实体工程极限。你的涡轮就像 Tensor Core：被逼得太狠时既会闹情绪又会发热。我永远不会丢弃你。

正如 Phish 经典歌曲《Contact》的歌词所说：

我在十一月的一个清晨醒来

意识到我爱你

并不是你前方的车灯

你的尾管，或你上方的天窗

而是你紧贴道路的方式

当风试图将你推开

我永不会驾车离去

又在没有你的情况下回家

Little Red，感谢你鼓励我拥抱机械同理心！

图 P-1　一辆名为“Little Red Whiting Hood”的高性能、讲究机械同理心的 Porsche 911 Turbo 仪表盘（白色外观、红色内饰）

随着我们持续拓展人工智能与超级计算的前沿，本书的知识旨在启发、教育并赋能读者。对 *AI Systems Performance Engineering* 复杂性的探索不仅是技术探索，也提醒我们人类的创造力、理解周边环境的必要性，以及通过技术与创新不断改进的愿望。世界上真正理解如何协同设计硬件、软件和算法以获得最高性能与效率的人并不多；读完本书后，你将成为其中之一！

Chris Fregly
加利福尼亚州旧金山

第 1 章　引言与 AI 系统概览

2024 年末，中国一家名为 DeepSeek.AI 的小型初创公司震动了整个 AI 社区：他们在无法使用当时最新、最先进的 NVIDIA GPU 的情况下，训练出了一个*前沿*大语言模型（LLM）。受出口管制限制，DeepSeek 的工程师无法获得顶级的 NVIDIA Blackwell（B200、B300 等）或 Hopper（H100、H200 等）GPU，因而转而采用当时在本地可以买到且符合出口规定的替代产品，其中包括 NVIDIA H800 GPU。他们利用自定义内核以及模型蒸馏等高级优化技术，从这些能力相对有限的 GPU 中榨取出最大性能。

尽管存在这些限制，DeepSeek.AI 仍然训练出了 DeepSeek-R1 模型，其推理能力接近当时使用 NVIDIA 最强芯片训练的领先前沿模型。这一案例说明，精通 AI 系统性能工程的从业者和研究人员，无论受到何种约束，都能够充分发挥现有硬件的潜力。

例如，DeepSeek 的工程师把通信带宽视为一种*稀缺资源*，对在线路上传输的每一个字节都进行优化，从而在这套基础设施上实现了许多人认为不可能做到的事情。他们通过新颖的软件和算法优化克服限制，将数千块能力受限、且仅由有限带宽互连连接的 GPU 横向扩展为大规模系统。

与 DeepSeek 的做法相比，美国和欧洲最大的 AI 前沿实验室走的是“蛮力”路线。这些实验室继续追求更大的计算集群和更大的模型。模型规模已经从数百万参数激增至数十亿，如今又达到数万亿参数。尽管规模每扩大 10 倍都会解锁性质上全新的能力，但这也需要付出巨大的成本并消耗大量资源。

例如，据报道，训练 OpenAI 的 GPT-4（2023 年）估计耗资约 1 亿美元，而训练 Google 的 Gemini Ultra（2023 年末）估计耗资高达约 1.91 亿美元。这表明，随着模型规模和成本持续增长，提高资源效率将是今后的必然要求。

DeepSeek 声称，其模型训练所耗费的计算成本不到 600 万美元，比 GPT-4 和 Gemini Ultra 等模型低一个数量级。与此同时，在 DeepSeek 所报告的基准测试中，其模型性能可与成本高得多的闭源模型系统相媲美。

尽管有人质疑这项 600 万美元成本声明是否可信，也质疑该数字究竟包含了什么（例如是否仅指单次训练运行）、排除了什么（例如实验和模型开发流水线），但这一消息仍一度震动美国金融市场。受此消息影响，NVIDIA 股价在一天之内下跌约 17%。市场担忧 DeepSeek 的效率创新会使未来所需的 NVIDIA 硬件减少。虽然这一市场反应有些过度——NVIDIA 股价也在随后的交易日中回升——但它表明，AI 效率方面的突破可能对全球金融市场产生重大的经济影响。

除模型训练之外，DeepSeek 还通过具有硬件感知能力的新颖算法来改进 Transformer 架构，从而显著提升推理效率；大多数现代前沿 LLM 都以该架构为基础。DeepSeek 清楚地证明，巧妙的 AI 系统性能工程优化能够彻底改变超大规模 AI 模型训练与推理的经济性。本书余下各章将介绍这些优化。

由此得到的深刻认识是：在这样的规模下，从系统中榨出的每一点性能，都可能转化为数百万乃至数十亿美元的成本节省。消除任何一个瓶颈，都可能对训练吞吐量和推理延迟产生超乎比例的影响，进而降低成本并提升最终用户的整体满意度。简而言之，AI 系统性能

工程并不只是追求速度，而是要让过去不可能实现的事情既成为可能，又在成本上可承受。

在第 1 章中，我们将深入探讨 AI 系统性能工程师这一职业。在大规模人工智能时代，这一角色已经变得至关重要。本章将全面介绍这项职业所承担的多方面职责，以及它对现代 AI 系统产生的关键影响。

我们首先回顾 AI 工作负载的演进，重点说明计算方式如何从传统范式转向满足当代 AI 应用的需求。这一背景有助于我们理解为什么 AI 领域需要专业的性能工程。

随后，本章将深入介绍 AI 系统性能工程师所需的核心能力。我们会考察这一岗位必备的技术素养，包括对硬件架构、软件优化技术和系统级集成的深刻理解。此外，我们还会讨论解决问题、沟通与协作等软技能的重要性；面对 AI 项目跨学科的特点，这些能力不可或缺。

本章有很大一部分篇幅聚焦于这一岗位的实践工作。我们将探讨性能工程师如何分析系统瓶颈、实施优化策略，并保障 AI 系统的可扩展性与可靠性。书中还会通过真实场景和案例研究来阐释这些概念，为实际工作中遇到的挑战及其解决方案提供具体示例。

此外，我们还会讨论性能工程师常用的工具和方法，介绍性能测试、监控及基准测试实践，其中包括业界标准工具概览，以及如何使用这些工具评估并提升系统性能。

读完第 1 章后，读者将全面理解 AI 系统性能工程师的职责、胜任这一岗位所需的技能，以及性能工程对于 AI 系统成功部署和运行的关键意义。这些基础知识将为后续章节做好铺垫；届时我们会进一步探讨成就卓越 AI 性能工程的具体技术、相关科技和最佳实践。

AI 系统性能工程师

AI 系统性能工程师是一种专业岗位，重点优化 AI 模型*以及*承载模型运行的底层系统的性能。这类工程师要确保 AI 训练和推理流水线不仅速度快、成本效益高、性能出色，而且可靠并具有高可用性。规模越大，AI 系统性能工程师就越重要。

AI 系统性能工程师往往能获得顶尖薪酬，而且理由十分充分：我们的工作会直接影响企业利润。我们融合了硬件、软件和算法等领域的专业知识，必须理解操作系统底层问题、内存层次结构、网络基础，以及 Python、C++ 等多种语言；同时还要熟悉 PyTorch、OpenAI Triton 和 NVIDIA 统一计算设备架构（CUDA）等不同的 AI 框架与库。

在任何一个工作日，AI 系统性能工程师都可能需要检查底层 GPU 内核效率、优化操作系统线程调度、分析内存访问模式、提高网络吞吐效率，或调试分布式训练算法。其主要职责包括基准测试、性能分析、调试、优化、扩展和高效管理资源。性能工程师可能专攻硬件、软件和算法中的某些组合，但关键在于这些专业领域需要协同设计（见图 1-1）。因此，理解它们之间的权衡以及彼此如何影响很有必要。

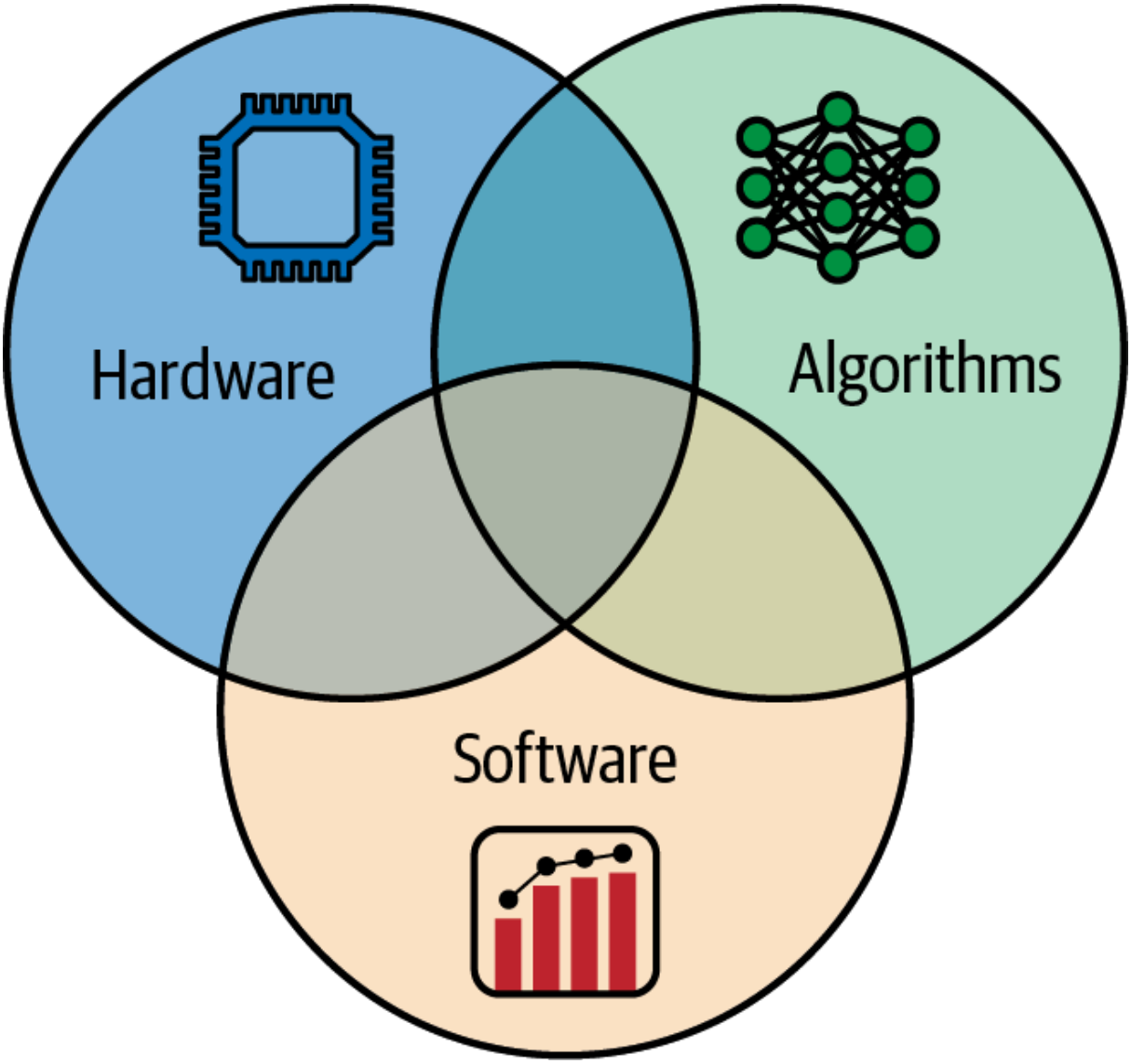


图 1-1　协同设计硬件、软件与算法

基准测试与性能分析

基准测试与性能分析是指测量 AI 模型在训练、推理等不同工作负载下的延迟、吞吐量、内存用量及其他性能指标。为了识别瓶颈，我们必须反复结合使用 NVIDIA Nsight Systems、NVIDIA Nsight Compute 和 PyTorch Profiler。随着我们不断提升 AI 系统的整体性能，这些工具相互配合，能够在技术栈的不同层级准确定位瓶颈，并持续跟踪性能变化。

> 务必建立自动化性能测试，以便在开发周期早期发现性能回归，也就是性能下降。

调试和优化性能问题时，需要追踪问题的根本原因，无论它是未达到最优的 CUDA 内核、不必要的通信开销，还是训练或推理工作负载不均衡。

在一种情况下，我们可能希望采用效率更高的矩阵运算，利用最新的 NVIDIA Transformer Engine（TE）硬件；这种硬件针对采用 Transformer 架构的现代 LLM 进行了优化。在另一种情况下，我们可以为“易并行”推理工作负载配置更高的并行度，从而改进软件框架。还有一种情况是，我们可以通过改进内存管理，并相对于所需 GPU 计算量减少进出 GPU 显存的数据量，来改进 Transformer 的注意力算法。

即使很小的代码调整也可能带来巨大收益。例如，某个用 Python 编写的数据预处理步骤可能拖慢整条训练流水线。可以用 C++ 重新实现这段代码来消除瓶颈，也可以使用 NVIDIA cuPyNumeric——一种可直接替换 NumPy 的方案，它能把数组运算工作负载分布到 CPU 和 GPU 上。

扩展分布式训练与推理

把小型研究工作负载扩展为超大规模集群上的大型生产工作负载时，要确保系统从 8 块 GPU 扩展到 80,000 块 GPU 仍只产生极少的额外开销和效率损失。这要求使用 NVIDIA 集合通信库（NCCL，读作“nickel”）优化训练中常见的 all-reduce 等分布式集合通信。此外，NVIDIA Inference Xfer Library（NIXL）能为分布式推理提供高吞吐量、低延迟的点对点数据移动，跨越 GPU 内存和不同存储层级。这类通信既可以发生在单个节点内的 GPU 之间，也可以跨越数千个节点。它们还会优化模型训练和推理中大量使用的通信及集合聚合操作，例如 all-reduce、all-to-all 和 all-gather。

你可能需要利用数据并行、张量并行和流水线并行，在节点之间巧妙地放置数据。也可能由于模型大到无法装入单块 GPU，而必须重新设计工作负载来采用张量并行或流水线并行。如果使用的是*混合专家*（MoE）模型，则还可以利用专家并行。

高效管理资源

优化模型使用 CPU 核心、GPU 显存、互连带宽和存储 I/O 等资源的方式十分重要。这可能涉及多方面工作，例如确保以最大速率向 GPU 供给数据、将线程绑定到特定 CPU 核心、减少上下文切换开销、协调内存使用，以及使用大型模型进行训练和推理时避免 GPU 出现内存不足（OOM）错误。当某项作业不需要 GPU 的全部算力时，可以使用 GPU 虚拟化技术（例如 NVIDIA Multi-Instance GPU，即 MIG）划分 GPU 资源，从而提高整体利用率。

跨团队协作

跨团队协作对 AI 系统性能工程师至关重要。我们需要与研究人员、数据科学家、应用开发人员密切合作，也要与网络、存储等基础设施团队并肩工作。

提升性能可能需要修改模型代码，这就涉及与研究人员协调；也可能需要部署新的 GPU 驱动程序来提高效率，这则需要基础设施团队参与。

性能改进往往横跨多个团队。例如，为修复错误和提高效率而更新 CUDA 驱动程序或 CUDA 版本，需要与 DevOps、基础设施和支持团队仔细协调；为提高性能而改进模型代码，则需要与研究人员密切合作。性能工程师处于这些跨学科领域的交汇点，需要同时掌握 AI、计算机科学和系统工程的专业语言。

透明度与可复现性

在性能工程中，测量一切并相信数据而非假设至关重要。公开自己的工作，能让其他人学习、复现并在你的成果之上继续发展。

DeepSeek 的故事中有一点尤其值得关注：他们非常开放地分享了基础设施优化成果。在 2025 年 2 月的 Open-Source Week 期间，他们发布了一系列开源 GitHub 仓库，包括 FlashMLA、DeepGEMM、DeepEP、专家并行负载均衡器（EPLB）、DualPipe 和 Fire-Flyer File System（3FS）。每个项目都经过生产环境检验，旨在从硬件中榨取尽可能高的性能。《DeepSeek-V3 技术报告》对这些项目进行了说明。

FlashMLA 是他们用 CUDA C++ 编写的优化注意力内核。DeepGEMM 提供针对 FP8 优化的矩阵乘法库；据称，它在稠密和稀疏运算上的性能都超过许多厂商内核。Deep Experts Parallelism（DeepEP）是他们为混合专家（MoE）模型高度调优的通信库。EPLB 实现了冗余专家策略，通过复制负载很高的专家来承接额外负载。DualPipe 是一种双向流水线并行算法，它让前向/反向计算与通信阶段相互重叠，以减少流水线气泡。3FS 则是他们的高性能分布式文件系统；它提醒我们，要让 AI 系统发挥最大性能，每一层都需要优化，文件系统也不例外。

DeepSeek 在 2025 年 2 月的“Open-Source Week”期间，将这些项目开源到 GitHub。这不仅让他人能够复现结果，从而证明其主张的可信度，也回馈了社区。这种透明度使其他开发者能够对其方法进行基准测试、复现和学习，其中包括利用 DeepEP/DualPipe 流水线并行让通信相互重叠，以及用 3FS 充分占满 NVMe SSD/RDMA 带宽。

DeepSeek 的 Open Infra Index 等开放项目提供了宝贵的基线和工具。它们给出各种 AI 硬件配置的实际性能测量数据，并鼓励进行条件一致的横向比较和可复现研究。同样，MLPerf 开放基准测试套件提供了一套标准，用于在不同软硬件配置之间以可复现方式比较训练和推理性能。

MLPerf 等业界基准测试量化了此类协同设计优化在不同硬件世代之间带来的效果。在 MLPerf Training v5.0（2025）中，基于 Blackwell 的 NVIDIA GB200 NVL72 系统，其单 GPU 训练吞吐量最高达到同类 Hopper 系统的 2.6 倍，如图 1-2所示。在 MLPerf Inference v5.0（2025）中，由于单 GPU 性能更高且 NVLink 域大得多，Blackwell NVL72 的单 GPU 推理吞吐量约为同类 Hopper 集群的 3.4 倍。这些结果由 NVIDIA 报告，如图 1-3所示。

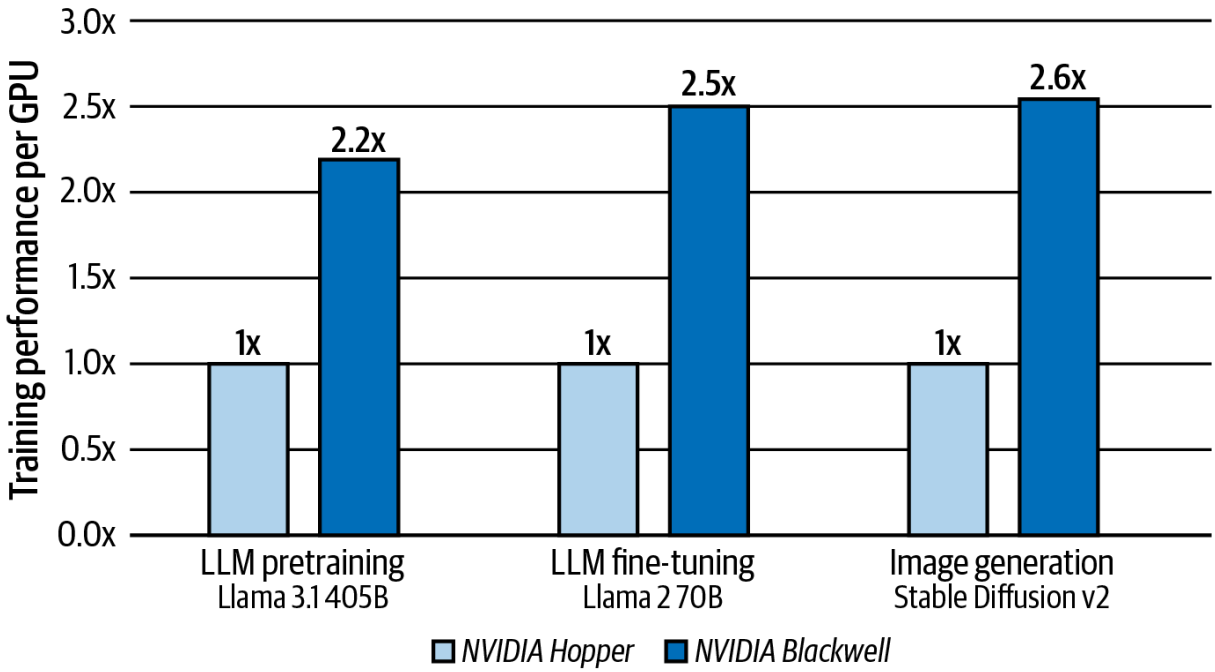


图 1-2　根据已发布的 GB200 NVL72 与 Hopper 提交结果，由 NVIDIA 整理的单 GPU MLPerf Training v5.0 对比（来源：https://oreil.ly/Ao1l8）

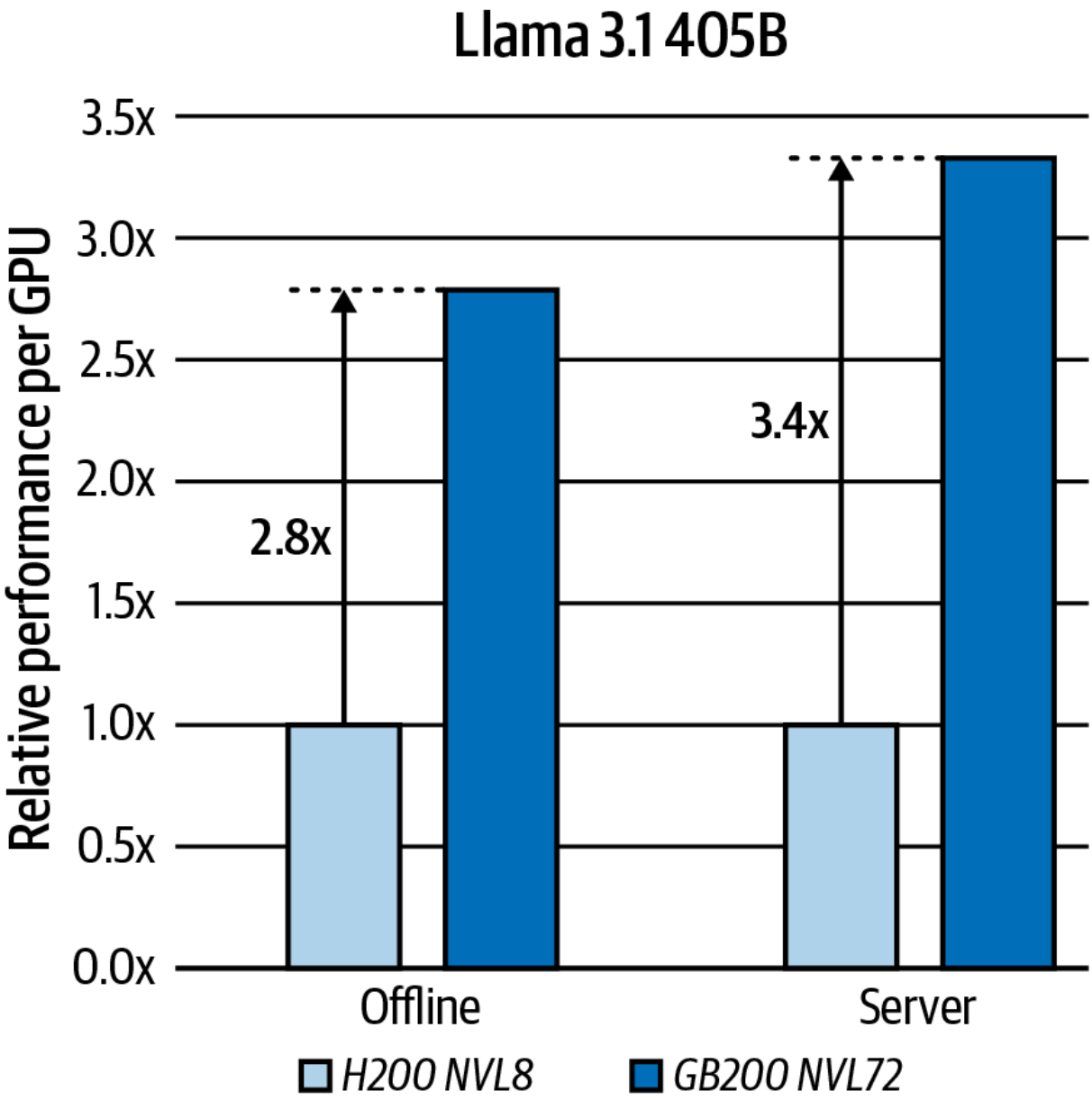


图 1-3　根据已发布的 GB200 NVL72 与 Hopper 提交结果，由 NVIDIA 整理的单 GPU MLPerf Inference v5.0 对比（来源：https://oreil.ly/V-jze）

在后续章节中，我们会引用其中一些开放基准测试来支持各种性能调优概念。例如，在讨论 GPU 内核优化时，我们会引用 DeepSeek 已公开的性能分析结果，以说明其自定义内核如何在 NVIDIA GPU 上实现接近峰值的内存带宽利用率。

> 引用 MLPerf 结果时，请记住 MLPerf 的提醒：跨平台比较时，单 GPU 结果并不是主要指标。正确的比较依据应是系统级端到端吞吐量；单 GPU 数字只能视为组件级参考指标。

实验透明度和可复现性对于推动 AI 性能工程领域发展至关重要。人们很容易落入依靠轶事和“感觉”的优化陷阱（“我们做了 X，感觉系统变快了”）。我主张采用严谨的科学方法：

提出假设，用可复现的基准测试测量结果，进行调整以改善结果，重新运行基准测试，并在每一个步骤分享全部结果。

在美国对华硬件出口限制下，DeepSeek 仍扩展至约 6800 亿参数模型

有时，系统性能创新源于现实所迫。如前所述，由于美国出口管制，DeepSeek 只能使用 NVIDIA H800 GPU。H800 是符合出口规定的 Hopper GPU 变体。与 H100 相比，它降低了 NVLink 互连带宽和 FP64 性能，而 HBM 容量与带宽大体相近。

作为参考，NVIDIA H100 每块 GPU 可提供约 900 GB/s 的 NVLink 互连带宽，而 H800 每块 GPU 约为 400 GB/s。这使 GPU 之间的数据传输变慢，最终限制了多 GPU 系统的可扩展性。H100 的 HBM 显存带宽约为 3.35 TB/s；H800 的 HBM 带宽则大体相近。因此，这里的主要限制是较低的 GPU 间互连吞吐量，而不是 HBM 带宽。互连受限可能成为分布式训练作业的瓶颈，并降低扩展效率。译注

DeepSeek 着手在这种限制重重的环境中训练一个约有 6800 亿参数的大型 MoE 语言模型，即 DeepSeek-V3。对于每个输入 token，这个 MoE 模型只激活约 370 亿参数，而不是一次使用全部约 6800 亿参数。

采用这种架构后，任何时刻都只会激活模型的一小部分。即使采用规模相对紧凑的 H800 配置，这也有助于控制计算负载。具体来说，对于每个 token，DeepSeek-V3 会使用 1 个共享专家，再从 256 个专家中由路由器选出 8 个，因此每个 token 共激活 9 个专家，如图 1-4所示。

后续章节将更详细地介绍 MoE。这里先了解一点：为了绕过环境限制，DeepSeek 实现了一种新颖的 DualPipe 并行算法，通过精心重叠计算与通信来掩盖 H800 的固有弱点。

DeepSeek 设计了自定义 CUDA 内核，绕过一部分默认的 NCCL 集合通信操作，使数据传输能够与正在进行的计算协调并行。这样一来，即使互连带宽有所降低，GPU 仍能保持较高的利用效率。通信与计算重叠是本书余下部分将反复讨论的主题。

这些创新工程工作取得了回报：DeepSeek-V3 完成训练所用的 GPU 时间（以及成本），仅为 OpenAI、Meta、DeepMind 等机构同等规模前沿模型的一小部分；与许多人原先认为在性能更强的集群上训练这种规模模型所必需的资源相比，也只占一小部分。

译注：原文末两句把 NVLink 互连吞吐量与 HBM 显存带宽混在了一起。NVIDIA H100 规格资料将 HBM 带宽与 NVLink 带宽列为不同指标；本段上一段也指出 H800 与 H100 的 HBM 容量和带宽大体相近。DeepSeek-V3 技术报告所强调的通信优化，针对的是节点内 NVLink 和跨节点 InfiniBand。因此译文按数据通路分别表述，并将瓶颈归于 GPU 间互连带宽。

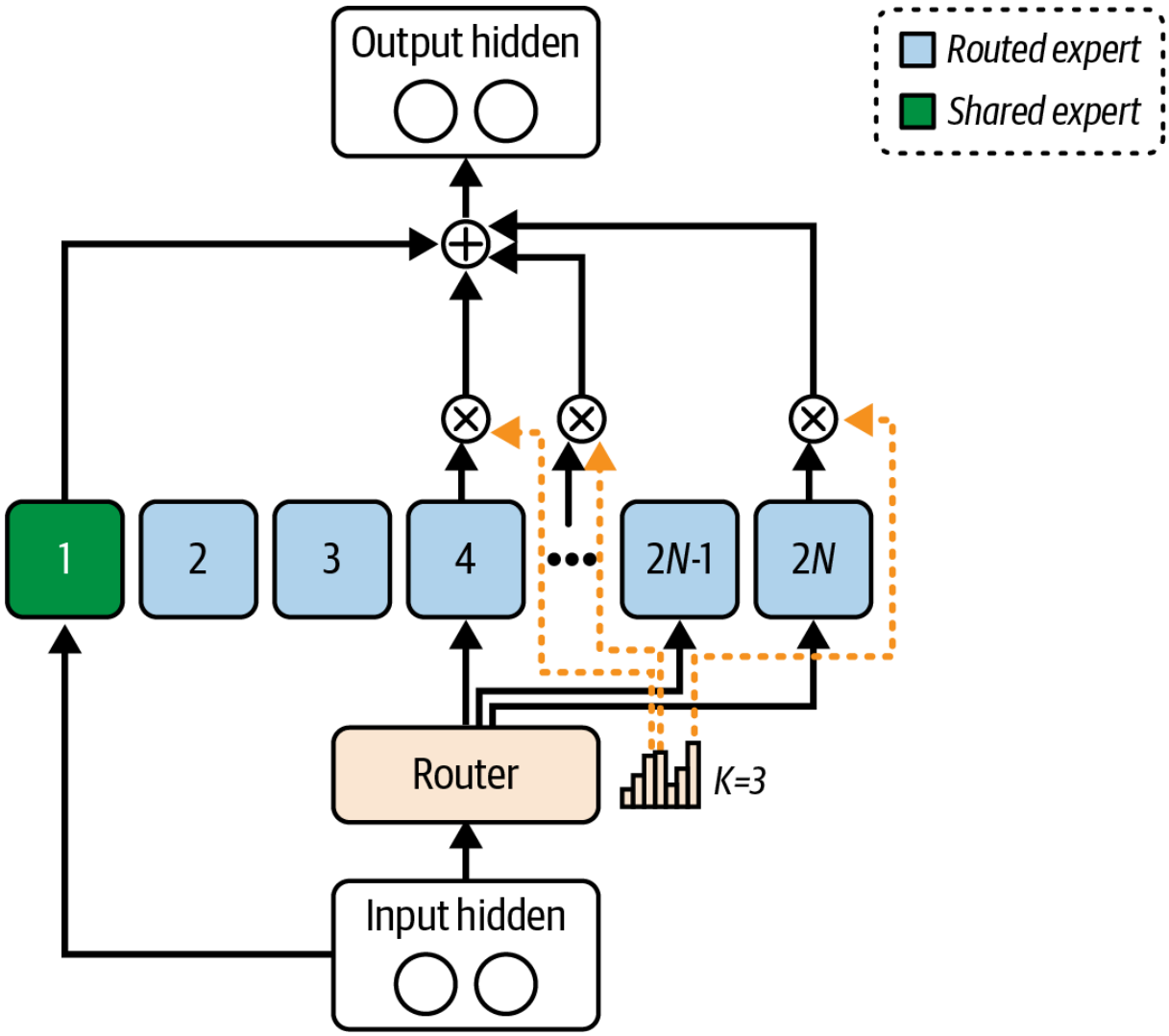


图 1-4　DeepSeek-V3 的专家路由

DeepSeek 报告称，在其公布的语言理解、推理和编程类任务基准测试中，旗下开源模型可与 OpenAI 和 Anthropic 的闭源前沿模型相媲美。这支持了一个核心观点：即使受到硬件限制，开放模型也能够与闭源模型竞争。

在 DeepSeek-V3 模型的基础上，团队随后创建了 DeepSeek-R1。这是一种专用推理模型，其构建方式与 OpenAI 的 o1 和 o3 系列推理模型相似。这些模型采用传统的分阶段流水线训练方法，包括冷启动数据、监督微调（SFT）、拒绝采样和强化学习（RL）。

此外，DeepSeek 还创建了 DeepSeek-R1-Zero，它采用一种新颖的训练方法，相比 SFT 更强调 RL。该技术只使用极少量监督数据，转而侧重结合思维链推理数据开展 RL。这种方法降低了训练成本和时间，并再次说明巧妙的软件与算法设计能够克服硬件瓶颈。

从中得到的经验是：即使内存和计算预算有限，大型稀疏激活 MoE 模型依然可以有效扩展。新颖的训练调度，以及底层通信与计算重叠优化能够克服硬件限制；DeepSeek 的工作所取得的巨大投资回报率（ROI）正说明了这一点。

这种投资回报十分明显：DeepSeek 的非常规方法大幅提高了效率，以低得多的训练成本和时间打造出一系列强大模型。即使受到通信带宽降低的限制，团队仍从 NVIDIA H800 GPU 中榨取出每一分性能，以少数百万美元的成本实现了 GPT-4 级别的模型性能。此外，R1 的推理微调阶段不需要那么多人工标注数据，又进一步节省了成本。

简而言之，巧妙的软件与算法设计战胜了依靠蛮力堆叠硬件所面临的限制，使 DeepSeek 能够在成本和硬件预算紧张的情况下开发大规模 AI 模型。

迈向 100 万亿参数模型

100 万亿参数模型是 AI 领域一个令人向往的里程碑，人们常把它与人类新皮层中估计约 100 万亿个突触连接相比较。在这种粗略类比中，一个突触连接对应一个模型参数。理论上可以构建如此规模的模型，但它需要极其庞大的资源和资金。除了最富有的少数组织，其他机构都不可能依靠蛮力把模型扩展到 100 万亿参数。

作为数量级上的粗略估算，训练一个稠密的 100 万亿参数模型时，若使用约 29 万亿个 token，通常可以用稠密 Transformer 的经验公式 $C \approx 6ND$ 估算，结果约为 1.74×10^{28}（$6 \times 1e14 \times 2.9e13$）FLOPs。token 数量更大时，计算量会随之线性增长。尽管稀疏激活（MoE）能够降低有效计算量，但仅凭这一点还不足以训练超大规模模型。与本书的主旨一致，要让巨型模型的训练真正可行，必须有效协同设计硬件、软件和算法。

> 尽管本书讨论的优化也适用于较小的模型和集群，我仍会不断回到 100 万亿参数模型这个例子，以强化一个观念：不能只是向扩展问题堆砌硬件。我们需要让巧妙的软件和算法创新与计算、网络、内存及存储等硬件协同设计。

训练成本如此爆炸式增长，促使人们寻找新的 AI 系统和软件工程技术，以提高性能、降低成本，并在计算资源有限、功耗受限且资金有限的条件下使极端规模 AI 成为可能。研究人员一直在探索降低有效计算需求的新技术。

一种重要思路是利用稀疏性，具体来说就是采用 MoE 模型。MoE 之类的稀疏模型与传统稠密模型形成对照，后者包括由 OpenAI 推广开来的常见 GPT 系列大语言模型（LLM）。事实上，外界公开推测，OpenAI 专有的 GPT 系列和 o 系列推理模型等部分前沿模型采用了 MoE 架构。

对于每个输入 token，MoE 等稀疏模型只激活模型的一部分。每个输入 token 只会被路由到众多内部“专家”中的一部分，因此即使总参数量增长，每 token 所需的 FLOPs 仍大致保持不变。这类稀疏模型证明，模型可以扩展至数万亿参数，而计算成本不必出现同等幅度的爆炸式增长。此外，由于这类模型在推理期间激活的参数更少，MoE 模型的请求—响应延迟通常远低于规模相当的稠密模型。这些认识对于训练和服务 100 万亿参数级模型至关重要。

DeepSeek-V3（基础模型）和 DeepSeek-R1（基于强化学习的推理变体）是体现 MoE 效率的优秀案例。它们共有约 6800 亿参数，但每个输入 token 只激活约 370 亿参数。DeepSeek 的技术报告和公开文章描述了一种 MoE：每个 token 使用 1 个共享专家，并从 256 个专家中选出 8 个，因而共激活约 9 个专家、约 370 亿参数。这使 DeepSeek-V3 和 DeepSeek-R1 的资源效率远高于规模相当的稠密大语言模型。另一个例子是 Google 于 2021 年推出的 Switch Transformer MoE。这个 1.6 万亿参数的 MoE 模型只使用稠密模型一小部分的计算量，便达到了同等准确率；其训练速度是同类稠密方法的 7 倍。

除了巨大的计算需求，内存也是主要瓶颈。例如，仅加载一个 100 万亿参数模型的 FP16 权重，就需要约 200 TB GPU 显存（100 万亿参数 × 2 字节）。这大约比单块 NVIDIA

Blackwell B200 GPU 可寻址的 180 GB HBM 容量高三个数量级。

仅仅为了加载权重，一个 100 万亿参数模型就需要约 1,110 块 B200 GPU（每块 180 GB），或约 695 块 B300 GPU（每块 288 GB）。这还没有包括存储激活值、优化器状态、KV 缓存以及服务/训练工作空间开销所需的内存；这些都会进一步增加总内存需求。

作为参考，一个典型的 B200 GPU 计算节点只有 8 块 B200 GPU。若使用这些节点，仅仅加载模型就需要约 139 个 B200 GPU 节点；若采用每节点 8 块 B300 GPU 的 Blackwell Ultra B300 计算节点集群，则需要约 87 个节点。译注

此外，加载训练数据也会变得极其困难，因为要以足够快的速度向这种模型供给数据、让所有 GPU 持续忙碌并非易事。特别是当 100 万亿参数模型被划分到约 1,000 块 B200 GPU 或 700 块 B300 GPU 上时，GPU 之间的通信开销会显著增加。训练单个模型可能消耗数百万 GPU 小时，并耗费以兆瓦时计的大量能源。为了扩展到足以训练和服务 100 万亿参数模型的规模，需要付出巨大的成本和能耗。

100 万亿参数 AI 的时代将迫使我们彻底重新思考系统设计，才能让这种规模的训练和部署切实可行。硬件、算法和软件都需要协同演进，以迎接这一新前沿。

译注：原文给出的节点数与同段采用的容量不一致。按其前文数值，1,110 块 B200、每节点 8 块，需要向上取整为 139 个节点，而不是约 125 个；695 块 B300、每节点 8 块，需要 87 个节点，而不是约 86 个。译文采用算术复核后的节点数。

NVIDIA 的“机架级 AI 超级计算机”

为了应对超大规模计算的挑战，NVIDIA 构建了一类专门面向万亿参数级工作负载的新型 AI 超级计算机。例如 2024 年的 NVIDIA Grace Blackwell GB200 NVL72，以及 GB300 NVL72 Ultra；后者采用 Blackwell Ultra GPU，每块 GPU 配备 288 GB HBM3e，并维持一个由 72 块 GPU 组成、聚合带宽约 130 TB/s 的 NVLink 域。

Vera Rubin VR200（2026 年）和 Feynman（2028 年）系统延续了这一趋势：把 百亿亿次级（exascale）超级计算机浓缩进数据中心的单个机架。事实上，NVIDIA 把这些 NVL 机架系统（如 NVL72、NVL144、NVL576）称为“机架级 AI 超级计算机”，理由十分充分。

每个 GB200/GB300 NVL72 机架集成 36 个 Grace Blackwell，并通过 NVLink 与 NVSwitch 相连，由此提供机架级交换结构。每个 Grace Blackwell Superchip 由 1 个 NVIDIA Grace CPU（含 72 个 CPU 核心）和 2 块 NVIDIA Blackwell GPU 组成，因此整套系统共有 36 个 Grace CPU 和 72 块 Blackwell GPU——“NVL72”名称中的“72”正是由此而来。

> 如果你还没有猜到，NVL72 中的 NVL 代表 *NVLink*。这个名称提醒我们，系统中的 GPU 是通过 NVLink 互连的——免得我们忘记！

每块 Grace Blackwell 板卡都通过 NVSwitch（机架内 NVLink 交换网络）与其他板卡相连。这样，全部 72 块 GPU 都能以完整的 NVLink 5 带宽相互通信：每块 GPU 的双向带

宽为 1.8 TB/s（18 条 × 100 GB/s 链路）。在整个机架范围内，NVLink Switch System 可在一个 NVL72 域中提供约 130 TB/s 的 GPU 间聚合带宽。

实际上，NVL72 的内部互连结构把全部 72 块 Grace Blackwell GPU 统一为一个高速集群。因此，用于训练的 PyTorch 和用于推理的 vLLM 等框架，可以利用单一 NVLink 域高效实施数据并行、张量并行、流水线并行和专家并行。CUDA Unified Memory 可以在需要时通过 NVLink 迁移页面或远程访问页面。不过，远程内存具有不同的延迟和带宽，应按非一致内存对待。若跨 Grace 和 Blackwell 使用托管内存分配，应优先通过 `cudaMemPrefetchAsync` 和 `cudaMemAdvise` 显式预取，以减少缺页停顿。

后续章节将更详细地介绍这台超级计算机的计算、内存与互连硬件。现在，我们先在现代生成式 AI 模型的背景下，分析这台 AI 超级计算机作为整体的性能规格。

在采用 2:1 结构化稀疏的情况下，一整套 GB200 NVL72 机架理论上可达到约 1.44 exaFLOPS 的 FP4 算力和约 720 petaFLOPS 的 FP8 算力。72 块 GPU 合计提供约 13.5 TB HBM3e（13,824 GB = 每块 GPU 192 GB × 72 块 GPU）；若把同一 NVLink 域中的 Grace CPU 内存也计算在内，总容量约为 30 TB。

NVLink 为 72 块 GPU 提供池化访问能力，CUDA Unified Memory 也能够通过 NVLink 迁移或远程访问页面；但远程访问的性能不同，因此整个机架并不会表现为一台统一、均匀的内存设备。

简而言之，GB200 NVL72 是一套自包含系统，拥有 72 块 GPU、1.44 exaFLOPS 算力和 30 TB 内存；机架功耗约为 120～132 kW，具体取决于厂商配置和冷却方案。它确实是一台 AI 超级计算机，能够在单个机架内训练和服务数万亿参数模型。

把这些机架组合成超大规模集群，就能支持庞大的数万亿参数模型。更方便的是，通过 Amazon Web Services（AWS）、Google Cloud Platform（GCP）、Microsoft Azure、CoreWeave、Lambda Labs 等云服务商，只需点击几下（当然还要花上一大笔钱！），便可配置这些机架和机架集群。

> 尽管本书着重介绍 NVIDIA Grace Blackwell 这一代芯片，但书中讨论的优化原则源自此前多代 NVIDIA 硬件。这些优化也会继续适用于未来多代 NVIDIA 芯片并随之演进，包括 Vera Rubin（2026 年）、Feynman（2028 年）及更远的世代。这份路线图延续了一种发展模式：每一代新 GPU 的性能、内存和集成度都实现翻倍。

本书将带你了解每一代产品在计算、内存、网络和存储方面的创新，如何推动 AI 进一步扩展，形成超大规模集群、数万亿参数模型、高吞吐量训练作业和超低延迟模型推理服务器。硬件感知算法为这些创新提供了动力，并贯彻下一节将讨论的机械同理心和软硬件协同设计原则。

机械同理心：软硬件协同设计

*机械同理心（mechanical sympathy）*一词最初由软件工程师 Martin Thompson 提出。他借用了英国一级方程式冠军 Jackie Stewart 的类比；Stewart 对赛车的机械细节了如指掌。在计算领域，这个词是指编写能够深入感知其运行硬件的软件。在 AI 语境中，它意味着让算法与硬件能力携手协同设计，从而最大限度提升性能。

实践经验表明，即使只是略微调整 GPU 内核或内存访问模式，也可能带来远超预期的收益。一个经典例子是 FlashAttention：这种新颖算法以硬件感知方式重新实现了 Transformer 注意力机制。

FlashAttention 对 GPU 计算进行“分块（tiling）”，从而尽量减少对 GPU 显存发起的读写次数。它显著减少数据移动，并加速注意力计算。对于长序列，用 FlashAttention 替换默认的 Transformer 注意力机制/算法，可以让训练和推理提速 2～4 倍，同时降低总体内存占用。

这种改变把原先的主要瓶颈——注意力计算——压缩到总运行时间的一小部分。FlashAttention 几乎一夜之间就成为许多库的默认方案，因为它能让模型以更快、更高效的方式处理更长序列。自 FlashAttention 问世以来，又出现了许多新的注意力算法，其中包括 DeepSeek 的多头潜在注意力（Multi-Head Latent Attention，MLA）。

DeepSeek 的 MLA 算法以 NVIDIA GPU 内核实现，并于 2025 年开源；这是软硬件协同设计，也就是机械同理心的又一案例。与 FlashAttention 类似，MLA 重构了注意力计算，以便更充分地利用 NVIDIA 的内存层次结构和专用 GPU“张量核心（Tensor Core）”。这些优化让 MLA 能够发挥受限 H800 GPU 架构的潜力，以一小部分成本获得更高吞吐量，甚至在相同 H800 系统上超过 FlashAttention 的性能。

从本质上说，本书整本都是对机械同理心的研究。我们将看到无数案例：新的硬件特性，或者像 DeepSeek 所面临的那种硬件限制，会催生新颖的软件与算法技术；反过来，新的软件算法也会推动硬件创新。

例如，Transformer 模型和低精度量化（如 FP8/FP4）的兴起，促使 NVIDIA 增加 Transformer Engine、专用低精度张量核心等硬件，以加快矩阵数学计算。反过来，这些硬件创新又让研究人员能够探索新颖的数值优化器和神经网络架构；它们继而推动硬件设计者更进一步，并解锁更新的算法，如此往复。这是一个良性循环！

现代 GPU 采用支持 FP4 和微缩放（microscaling）的最新 Transformer Engine。结合新一代 NVLink，这些特性主要通过更快的张量核心数学运算、更高的内存带宽和改进的特殊函数单元（SFU）来提高注意力吞吐量。例如，指数计算单元专门用于加速 Transformer 注意力算法大量使用的 softmax 运算；注意力算法是当今 LLM 的关键组成部分。随着 Transformer 普及，由于 softmax 对注意力机制至关重要，它的延迟已经成为此前 GPU 上的一项瓶颈。

> NVIDIA 通过现代 GPU 提高特殊函数吞吐量并改进快速数学流水线，以最充分、最纯粹的形式体现了机械同理心以及硬件—软件—算法协同设计。他们直接与研究人员和从业者合作，解决并缓解现代 LLM 算法（如注意力）在其硬件上运行时遇到的瓶颈。

GPU 与 AI 算法协同演进的这种紧密互动，正是 AI 领域机械同理心的核心。只有硬件公司（如 NVIDIA 和 Advanced RISC Machine，即 ARM）、AI 研究实验室（如 OpenAI 和 Anthropic）以及 AI 系统性能工程师（比如我们！）密切合作，才能实现这些协同设计创新。

衡量“Goodput”：有效吞吐量

运行由数百、数千乃至数百万块 GPU 组成的集群时，务必要弄清楚：理论硬件能力中，究竟有多少真正用于执行有效工作。FLOPS 和设备利用率等传统吞吐量指标可能高得具有误导性，因为大量时间很可能耗费在通信停顿、计算单元空闲或失败作业重启上。这时就需要引入“goodput（有效吞吐量）”概念；Meta 在一篇 2025 年论文中将其描述为“有效训练时间比率”。

> NVIDIA 把硬件的理论上限称为*光速（speed of light）*，你可能在 NVIDIA 的博客、文档、网络研讨会和会议演讲中见过这种说法。

简单来说，goodput 衡量单位时间内完成的有效工作吞吐量，例如处理的 token 数或完成的推理请求数；凡是没有直接促进模型训练或推理的工作都要扣除。从富有成效的模型训练或推理角度看，它实际上就是系统的端到端效率。还可以用集群的最大可能吞吐量对 goodput 进行归一化，得到百分比形式的效率值。

例如，假设一个配有 8 块 GPU 的节点能在 10 秒内处理 100,000 个 token，那么它的 goodput 就是每秒 10,000 个 token。如果节点中每块 GPU 的理论峰值吞吐量为每秒 1,500 个 token，8 块 GPU 合计为每秒 12,000 个 token，则该节点的效率为 83.3%（0.833 = 实际吞吐量 10,000 ÷ 峰值吞吐量 12,000）。

Meta 的 AI 基础设施团队在《Revisiting Reliability》论文中强调了 goodput 的重要性。该论文提出*有效训练时间比率指标*，说明即使表面利用率很高，抢占、资源碎片和故障仍会减少实际训练时间。论文还分析了抢占、硬件故障和网络拥塞如何降低实际吞吐量。

换句话说，虽然集群看起来达到了 100% 利用率，但由于通信延迟、欠佳的并行化、数据延迟或故障恢复等开销，70%～75% 的计算能力并未转化为有效训练。Meta 的分析还表明，在大规模环境中，作业抢占、网络热点和不可恢复故障等问题，是 goodput 损失的主要来源。

例如，设想一个训练作业在理想硬件上理论上每秒可以处理 1,000 个样本，但由于输入流水线设计不佳且同步过多，实际训练吞吐量只有每秒 300 个样本。我们会说该作业的 goodput 为 30%，其余 70% 的能力基本被浪费了。

识别并弥合这些差距，是我们工作的核心组成部分。例如，如果 GPU 正在等待从存储加载数据，我们可以引入缓存或异步预取。如果 GPU 在模型训练的梯度同步步骤中处于空闲状态，我们通常应让 GPU 计算（如计算梯度）与 GPU 间通信（如同步梯度）相互重叠。我们的目标是把浪费掉的低效周期转化为有效工作。

理论性能与实际性能之间的差距，正体现了 AI 系统性能工程师这一岗位的价值。我们的使命是从硬件、软件和算法等技术栈各个层级消除低效并降低成本，从而尽可能提高 goodput，理想情况下使其逼近 100%。

投资 AI 系统性能工程师通常能够持续产生远高于投入的回报。假设一名性能工程师帮助集群效率提升 20%，那么在大规模 AI 环境中，这可以节省数百万美元的硬件成本。

关注 goodput，就是在优化真正重要的事情：每投入一美元成本、每消耗一焦耳能量，究竟完成了多少有效训练。与原始 FLOPS 或设备利用率相比，goodput 更接近衡量成功的终极指标，因为它概括了硬件、软件和算法为实现“更快、更低成本地训练 AI 模型”这一最终目标而协同配合的程度。

提高 goodput，需要深入理解硬件、软件和算法之间的相互作用。硬件包括 CPU、GPU、网络拓扑、内存层次结构和存储布局；软件包括操作系统配置、分页内存和 I/O 利用率；算法则包括 Transformer 架构变体、不同注意力机制，以及各种缓存和批处理策略。

正是因为需要对硬件、软件和算法等多个学科拥有既广又深的理解，AI 系统性能工程师如今才如此稀缺。这也是我写作本书的原因！接下来将介绍本书其余内容的路线图和方法论。

全书路线图与方法论

我们将如何着手优化 100 万亿参数 AI 系统？本书从硬件基础出发，逐步深入软件栈和算法技术，并强调在每个层级开展动手分析。下面是本书其余内容的安排。

第 2 章深入介绍 NVIDIA AI 系统硬件，其中包括 GB200/GB300 NVL72“机架级 AI 超级计算机”。它把 Grace Blackwell Superchip 设计与 NVLink 网络结合起来，形成兼具超级计算机性能和功耗特征的系统。

随后，第 3～5 章介绍基于 GPU 的 AI 系统在操作系统、网络和存储层面的优化。这些优化包括 CPU 与内存绑定，以及 Docker 容器和 Kubernetes 编排方面的注意事项，其中涵盖 GPU 环境的网络 I/O 与存储配置。

第 6～12 章讨论 NVIDIA CUDA 编程基础和 CUDA 内核优化；这些知识对于开发新颖的硬件感知算法至关重要。广为使用的此类算法包括 FlashAttention 和 DeepSeek MLA。它们以 Transformer 架构中资源密集的注意力机制为优化对象，而该架构主导着当今的生成式 AI 工作负载。

掌握 NVIDIA 硬件和 CUDA 软件背景后，我们将深入研究分布式通信优化，包括如何高效训练和服务超大型模型。我们会考察尽量减少通信的策略，例如让计算与通信重叠；这种模式适用于 AI 系统栈的许多层级。

第 13、14 章讨论 PyTorch 专项优化，其中包括 PyTorch 编译器栈，以及 OpenAI 基于 Python、用于编写自定义 GPU 内核的 Triton 语言与编译器。这些编译器降低了开发新型 CUDA 内核的门槛，因为使用它们不必像传统 CUDA 内核开发那样深入掌握 C++。这两章还会讨论模型训练的分布式并行技术，包括数据并行（DP）、完全分片数据并行（FSDP）、张量并行（TP）、流水线并行（PP）、上下文并行（CP）和混合专家（MoE）。我们将展示如何把数万亿参数模型拆分到许多 GPU 上并进行高效训练，还会讨论超大规模模型训练期间的内存优化技术，包括激活检查点、优化器状态分片，以及将数据卸载到容量更大的 CPU 内存。当模型规模超过 GPU 硬件的物理限制时，这些技术至关重要。

第 15～19 章聚焦高吞吐量、低延迟模型推理和智能体 AI 系统的软件及算法创新。其中包括解耦式预填充与解码；NVIDIA Dynamo 和社区技术栈现已支持这种架构，并可通过采用 GPUDirect RDMA 的 UCX、NCCL 点对点通信或框架提供的传输机制来移动键值（KV）缓存。我们还会讨论广泛使用的模型服务引擎，包括 vLLM、SGLang 和 NVIDIA TensorRT LLM；随后介绍 NVIDIA Dynamo 分布式推理框架。Dynamo 可与这些引擎集成，并包含 NIXL，用于在解耦式预填充—解码配置中低延迟传输 KV 缓存。

我们还将考察如何利用 NVL72 中的 Grace CPU 进行预处理，如何为推测解码等高性能推理算法同时运行较小的“草稿”模型，以及如何通过高效的请求路由和批处理最大化推理系统的整体吞吐量。此外，我们会探索模型压缩与加速技术，例如 4 位量化、让更有能力的“教师”模型教导较小“学生”模型的知识蒸馏、稀疏化与剪枝，以及使用专用 TensorRT 内核。本书将重点讨论解耦式预填充—解码，以及在运行时动态调优系统的自适应技术。

第 20 章介绍利用 AI 辅助工具优化内核和 AI 系统性能的最新工作。本章还包含高效训练与服务数十亿、数万亿参数模型的案例研究，并讨论自我改进的 AI 系统优化和智能体等新兴趋势。由此，我们可以描绘 100 万亿参数级 AI 系统的发展方向，并思考如何为 AI 系统性能工程的未来做好准备。

附录提供了一份常用性能优化与成本节省技巧清单，可应用到你自己的 AI 系统中；它汇总了全书讨论的可操作优化和效率提升措施。

简而言之，本书采用重视动手实践和实证的方法论来应用性能优化。我们将经常分析实际运行、案例研究、基准测试结果和性能分析数据，以理解瓶颈并验证改进。读完全书后，你应当能够掌握优化超大型 AI 系统的原则，并获得一些实际工具使用经验，把这些优化应用到超大规模、多 GPU、多节点和多机架 AI 系统中，例如 NVIDIA GB200/GB300 NVL72 机架级 AI 超级计算机，以及现在和未来的类似 AI 系统。

关键要点

以下特质共同定义了 AI 系统性能工程师这一角色。这类工程师把深厚的技术知识与策略性、由性能分析驱动的优化相结合，从而把原始硬件转化为兼具成本效益和高性能的 AI 解决方案：

衡量 goodput

> 不要只看原始 FLOPS 或利用率，而要测量 GPU 执行有效工作（如前向传播/反向传播计算）的时间，与等待数据或处理其他开销的时间之比。使用 NVIDIA Nsight Systems/Compute 或 PyTorch Profiler 测量这一比率，并努力将其提高，因为 goodput 关注的是有效且有用的 GPU 利用。

优先采用高水平工程优化，而不是靠蛮力烧钱

> 增加硬件并非万能药。硬件受限时，巧妙的软件和系统优化能够弥合差距，实现原本需要昂贵得多的基础设施才能取得的成果。DeepSeek 的成就正是如此。其工程师通过优化通信和硬件使用，以一小部分成本在受限的 H800 GPU（互连带宽有限）上训练出前沿模型。DeepSeek 模型达到了使用强大得多的硬件训练的前沿模型性能。换句话说，高水平工程胜过了依靠蛮力投入资金。

用渐进式优化寻求数量级影响

在大规模环境中，即使效率只提高几个百分点，也能节省数百万美元。反过来说，冗余计算和缓慢数据流水线等微小低效，会随着系统扩展而悄然推高成本。

以性能分析驱动的思维开展调优

使用数据和性能分析工具指导优化。借助分析器识别真正的瓶颈，无论它是计算利用率、内存带宽、内存延迟、缓存未命中，还是通信/网络延迟；然后针对该瓶颈实施定向优化。

保持全局视角

提升 AI 系统性能横跨 GPU、CPU、内存和网络等硬件，也涉及算法和库等软件。任何一层的弱点都可能成为整体瓶颈。最优秀的性能工程师会考虑软硬件协同设计：有时改变算法能够缓解硬件限制，有时新的硬件特性又会成就新的算法。

持续了解最新硬件、软件和算法

现代 AI 硬件和软件发展迅速。统一 CPU—GPU 内存、更快互连和新型数值精度格式等能力，都可能改变最优策略。优秀的性能工程师会持续关注这些进展，并相应更新自己的思维模型，以迅速消除瓶颈。此外，MLPerf 基准测试套件是了解不同模型上 AI 硬件性能的绝佳资源。

结论

这篇入门分析强调，在大规模环境中，优化不是可选项，而是绝对必要。它决定了一个系统是能够工作，还是完全不切实际。无论硬件还是算法，传统方法在这种规模下都会失效。要继续向前推进，我们既需要先进硬件，也需要巧妙的软件技术。

显然，AI 模型正在逼近物理资源极限。硬件竞相追赶新的模型架构和算法，而性能工程师坐在驾驶席上，负责确保所有这些昂贵设备真正产生成果。

我们已经说明，AI 系统性能工程师这一角色正变得愈发重要。仅仅向问题投入资金和硬件远远不够；我们需要协同优化模型架构、算法、硬件和系统设计等一切要素，推动 AI 能力实现下一次飞跃。

作为 AI 系统性能工程师，你的工作跨越多个学科，既复杂又不断变化。今天你可能深入研究 GPU 性能分析，明天转向网络拓扑，后天又钻研算法复杂度。这个岗位适合热衷于从硬件和软件中榨取每一分可用性能的“全栈”性能极客。

从本质上说，AI 系统性能工程师的信条是“机械同理心”。我们深入理解硬件和软件这部机器，从而量身定制高效解决方案，充分发挥整个技术栈的性能潜力。

如前文所示，100 万亿参数模型远远超出当今硬件的能力范围。按照本章前述约 1.74×10^{28} FLOPs 的粗略估算，即使持续获得 1 exaFLOP 的有效算力，单次训练的理想下限也约为 552 年；实际 goodput 较低时还会更久。这显然不切实际，也说明要达到这种规模，先进硬件与巧妙的软件优化缺一不可。译注

在接下来的章节中，我们将展示如何拆解 AI 系统的各个组成部分——从处理器、内存、互连一直到软件框架——并学习如何以系统化原则优化每个组件。我们会研究具体案例，了解微小改动如何带来巨大的性能提升和成本改善，从而建立一个可以从多个维度推理性能优化的思维模型。

完成这段旅程后，作为读者和实践者，你将掌握当今的最佳实践，也会具备用工程思维应对明日挑战的能力。无论现在还是未来，你都会拥有一整套把 AI 系统推向极限的技术。AI 系统性能工程师肩负的使命十分明确：我们必须从这些创新中学习，并准备在技术栈的每一个层级实施积极优化。

现在，背景已经交代清楚，让我们深入了解现代 AI 系统的硬件组件，包括 CPU、GPU、内存技术、网络结构和存储机制。通过研究支撑当代 AI 超级计算机的各个组件，你将掌握基础知识，为后续章节深入探讨优化技术打下根基。

译注：原文称“即使百亿亿次级系统也需要数千年 GPU 时间”，但未说明假设。按本章前述训练量 1.74×10^{28} FLOPs 和持续 1×10^{18} FLOP/s 计算，时间约为 1.74×10^{10} 秒，即约 552 年。只有有效持续算力显著低于 1 exaFLOP（例如约 20% goodput）时，时间才会上升到数千年。译文给出可由本章数字直接推出的理想下限，并说明实际时间会更长。

第 2 章　AI 系统硬件概览

设想把相当于一台超级计算机的 AI 硬件压缩进一个机架。NVIDIA 的最新架构正是如此。本章将探讨 NVIDIA 如何将 CPU 与 GPU 融合为强大的*超级芯片*，再用超高速互连连接数十个此类模块，构成“机架中的 AI 超级计算机”。我们将介绍 Grace CPU 和 Blackwell GPU 这两个基础硬件构件，并了解其紧密集成和巨大内存池如何简化 AI 工程工作。

随后，我们将扩展到网络结构：它把 72 块 GPU 像一台机器一样连接起来。一路上会重点说明计算性能、内存容量和效率上的跃升如何赋予该系统强大能力。读完本章，你将理解这些前沿硬件如何使训练和服务过去看来不可能的多万亿参数模型成为可能。

CPU 与 GPU 超级芯片

NVIDIA 的扩展方法从单个结合 CPU 与 GPU 的超级芯片模块开始。自 Hopper 一代起，NVIDIA 开始将基于 ARM 的 CPU 与一块或多块 GPU 封装在同一单元内，并通过高速接口紧密连接。结果是一个表现如统一计算引擎的模块。

超级芯片的首个实现是 Grace Hopper（GH200），它将一颗 Grace CPU 与一块 Hopper GPU 配对。随后出现的 Grace Blackwell（GB200）超级芯片，则在同一封装中配对一颗 Grace CPU 与两块 Blackwell GPU。如图 2-1所示，Grace CPU 位于模块中央，周围是两颗 Blackwell GPU 裸片。

图 2-1. NVIDIA Grace Blackwell 超级芯片模块：单个模块内包含一颗 Grace CPU（中央）和两块 Blackwell B200 GPU（左上和右上），具有共享的统一内存空间，并通过名为 NVLink-C2C（chip-to-chip）的定制高速链路连接

在传统系统中，CPU 和 GPU 分别拥有内存池，并通过相对较慢的总线（如 PCIe）通信，因此数据必须来回复制。NVIDIA 的超级芯片通过名为 NVLink-C2C（chip-to-chip）的定制高速链路连接 CPU 和 GPU，从而消除了这一障碍。

在 GB200 超级芯片中，NVLink-C2C 在 Grace CPU 和 Blackwell GPU 之间可提供最高约 900 GB/s 带宽。相比之下，PCIe Gen5 x16（Blackwell B200）单向约为 64 GB/s，PCIe Gen6 x16（Blackwell Ultra B300）单向约为 128 GB/s。NVLink-C2C 的互连速度比典型 PCIe 高一个数量级。更重要的是，它具有缓存一致性。

缓存一致性意味着 CPU 和 GPU共享一个一致的统一内存架构，因此始终看到相同的值。实际中，超级芯片上的 Grace CPU 和 Blackwell GPU 都可像访问一个巨大内存池一样直接访问彼此内存。GPU 可读取或写入 CPU 内存中存储的数据，反之亦然，无需显式复制。NVIDIA 常将这种架构称为统一 CPU-GPU 内存（Unified CPU-GPU Memory）或

扩展 GPU 内存（Extended GPU Memory，EGM）；它有效模糊了 CPU 内存与 GPU内存之间的边界。

每个 Grace Blackwell 超级芯片都携带极大的内存容量。Grace CPU 配有数百 GB 的 LPDDR5X DRAM，而每块 Blackwell GPU 都有自身的高速高带宽内存（HBM）堆叠。

在 GB200 超级芯片中，Grace CPU 提供最高约 480 GB、带宽最高约 500 GB/s 的 LPDDR5X；两块 Blackwell GPU 合计提供最高约 384 GB HBM3e 内存（每块 GPU 共 192 GB）。总计，GB200 超级芯片在统一地址空间中向 GPU 和 CPU 暴露约 900 GB 可访问的一致性统一内存。

简单说，每个超级芯片都拥有接近 1 TB 的高速统一内存可供使用。这对巨大 AI 模型是一次重大变化。在旧系统中，单块 GPU 的内存可能不足 100 GB，更大的模型必须切分或卸载到更慢的存储。这里，GPU 可将 CPU 内存无缝作为扩展使用。

若神经网络层或大型嵌入表放不进 GPU 本地 HBM，它可以位于 CPU 内存中，GPU 仍可通过 NVLink-C2C 使用它。从编程角度，统一虚拟地址空间和一致性简化了正确性处理。但出于性能考虑，仍应使用异步预取和分阶段流水线等方法显式管理放置和内存移动。通过 NVLink-C2C 访问 LPDDR5X 的延迟更高、带宽约比直接访问 HBM 低一个数量级。

GPU 内存仍比 CPU 内存快得多，也更接近 GPU 核心；可以把 CPU 内存视为容量大但稍慢的扩展。访问 LPDDR5X 不如访问 GPU 上的 HBM 快，其带宽约低一个数量级、延迟更高。智能运行时会将最常用数据保留在 HBM 中，将 CPU 的 LPDDR5X 用于溢出数据或对速度不那么敏感的数据。关键在于，溢出数据不再需要移往 NVMe SSD 或经由网络。

GPU 可按约 900 GB/s（每方向 450 GB/s）从 CPU RAM 取数；虽慢于 HBM，却远快于从 NVMe SSD 存储取数。这种灵活性很关键：例如，一个 500 GB 的模型（大到放不进单块 GPU 的 HBM）仍可完全置于单个超级芯片模块中，访问合计 192 GB（可用 180 GB）HBM 和约 500 GB CPU 内存。该模型无需跨多 GPU 切分即可运行；GPU 只需在需要时透明地从 CPU 内存拉取额外数据。

归根结底，只要整个模型能放进超级芯片的 CPU 与 GPU 合计内存，内存大小就不再是容纳超大模型的硬性限制。许多研究者都遇到过模型放不进 GPU 时令人沮丧的“内存不足”（out of memory，OOM）错误；这种架构旨在大幅推开这一边界。

NVIDIA Grace CPU

Grace CPU 本身并不迟缓。它是 NVIDIA 为带宽和效率定制的 ARM Neoverse V2 CPU。在超级芯片中，它负责通用任务、预处理并向 GPU 供给数据，以及管理所连接的大量内存。其时钟频率适中，却通过巨大的内存带宽——对 LPDDR5X 最高约 500 GB/s——以及包括超过 100 MB L3 缓存的大量缓存加以弥补。

其设计理念是：向 GPU 输送数据时，CPU 不应成为瓶颈。它可从存储流式读取数据，或执行分词、数据增强等即时数据变换，再经由 NVLink-C2C 高效供给 GPU。若工作负载的一部分更适合 CPU，Grace 核心可处理它，并让 GPU 立即访问结果。

这是一种协调的耦合：CPU 在随机内存访问、控制密集型代码等 GPU 较弱的领域扩展 GPU 的能力；GPU 则加速 CPU 无法跟上的数值计算。

CPU 与 GPU 之间的低延迟链路意味着它们可在没有通常开销的情况下交换任务。例如，CPU 启动 GPU 内核会比传统系统快得多，因为命令不必穿越缓慢的 PCIe 总线。CPU 与

GPU 实质上位于同一块板上，这类似调用快速的本地函数而非较慢的远程函数。接下来讨论超级芯片的强力计算引擎：Blackwell GPU。

NVIDIA Blackwell“双裸片”GPU

Blackwell 是 NVIDIA 对这一代 GPU 的代号。无论计算能力还是内存，它都较前代 Hopper（H100）GPU 有显著跃升。Blackwell B200 和 B300“Ultra”GPU 不是单芯片，而是采用多芯片模块（multichip module，MCM）设计，在一个模块中放置两颗 GPU 裸片。因此，Blackwell 被称为*双裸片* GPU（见图 2-2）。

> 本节会深入介绍双裸片架构；本书其余部分将把 Blackwell 的两颗组合 GPU 裸片统称为“Blackwell GPU”。

这种*小芯片*（chiplet）方法将原本会是一颗巨大 GPU 的设计拆分为较小 GPU 裸片，再用超高速封装内裸片互连将它们连接起来。为什么这样做？因为单片式裸片受制造限制，硅片上可制造的芯片尺寸有上限。将两颗物理 GPU 裸片组合到单个模块，可使 NVIDIA 将该模块的总晶体管预算翻倍。

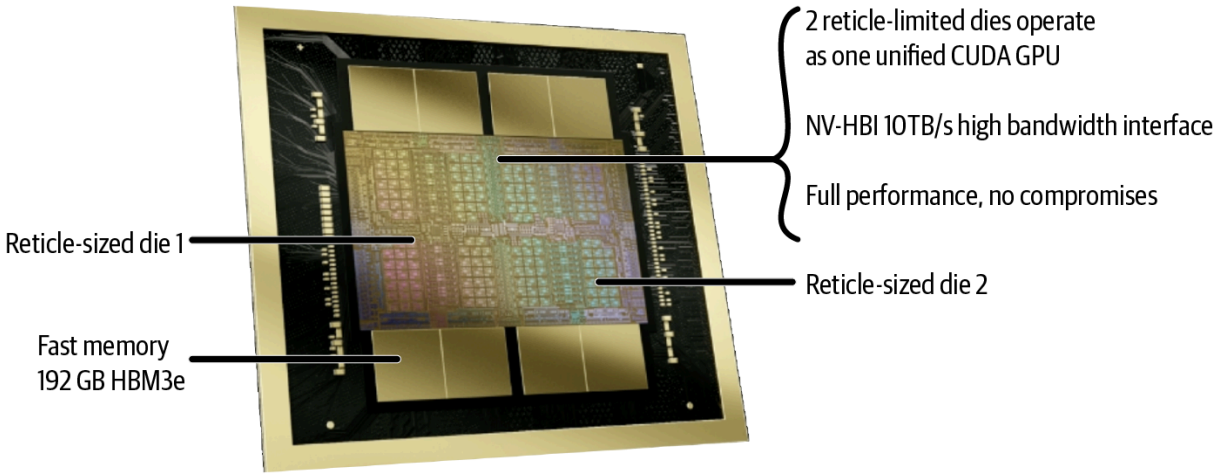


图 2-2. Blackwell 双裸片多芯片模块（MCM）设计

对于 Blackwell B200 MCM，每颗 GPU 裸片约有 1,040 亿个晶体管和 96 GB HBM3e 内存。组合后的 GPU 模块每块 B200 约有 2,080 亿个晶体管和 192 GB（可用 180 GB）总内存。相比之下，Hopper H100 GPU 约有 800 亿个晶体管和 80 GB HBM3（Blackwell 使用 HBM3e）内存。因此，Blackwell B200 的晶体管数量超过两倍，内存容量约增加 2.4 倍。

Blackwell 的两颗 GPU 裸片通过称为 NV-HBI（High-Bandwidth Interface）的专用高速 10 TB/s 裸片间互连通信。这使模块中的两颗 GPU 裸片作为一个统一 GPU 工作；上层软件仅看到一个 GPU。

从系统角度看，Blackwell GPU 是一个单独模块或设备，具有大型内存池（192 GB HBM3e，其中 180 GB 可用）和大量执行单元；但底层实际是两颗芯片协同工作。NVIDIA 的软件与调度会确保工作在两颗 GPU 裸片间平衡，并使内存访问保持一致性。因

此开发者可以在很大程度上忽略这种复杂性，因为它们按 NVIDIA 的设计呈现为一个 GPU。

每个 Blackwell B200 GPU 模块具有横跨两颗 GPU 裸片（各 96 GB）的 192 GB（可用 180 GB）HBM3e 内存，并划分为 8-Hi 堆叠。一个 8-Hi HBM3e 堆叠由八颗 DRAM 裸片垂直堆叠构成，每颗 3 GB，故每个堆叠总计 24 GB。

B200 GPU 使用八个此类堆叠（每颗裸片四个），提供 192 GB（可用 180 GB）的封装内内存（192 GB = 8 个堆叠 × 每堆叠 24 GB）。相较上一代 Hopper GPU，这增加了每 GPU 的堆叠数和容量，为模型参数、激活值、梯度和输入数据留出更多空间。

> 由于纠错码（error correcting code，ECC）、系统固件占用、制造限制及其他导致芯片无法暴露完整 192 GB 的因素，每块 B200 的 192 GB HBM3e 中只有 180 GB 可用。因此，下文提及 Blackwell B200 可用内存时使用 180 GB 而非完整的 192 GB。

内存也更快：Blackwell B200 的 HBM3e 每块 GPU 聚合带宽最高约 8 TB/s。相比之下，Hopper 使用上一代 HBM3，每块 GPU 约为 3.35 TB/s。因此，Blackwell 的内存带宽吞吐量约为 Hopper 的 2.4 倍。

以每秒 8 TB 的速度供给数据，可让 Blackwell GPU 核心忙于大矩阵运算，较少因等待数据而停顿。NVIDIA 还加强了片上缓存：Blackwell 的 L2 缓存总量为 126 MB（每颗裸片 63 MB）。这种缓存是 GPU 上保存近期使用数据的小型超高速内存。

与 Hopper 的 50 MB L2 缓存相比，L2 容量增加超过 2.5 倍，使 Blackwell 能在芯片上保留更多神经网络权重或中间结果，避免额外访问 HBM。这同样有助于确保 GPU 计算单元很少因缺数据而闲置。

接下来说明 Blackwell GPU 如何配备专用的低精度 Tensor Cores，以及 NVIDIA 针对 Transformer 优化、称为 Transformer Engine 的硬件与软件 API。PyTorch 等框架和 vLLM 等推理引擎可通过 CUDA、CUTLASS 和 OpenAI Triton 等库支持这些优化，后续章节将讨论它们。

> 请记住，本书其余部分会将 Blackwell 的双裸片 GPU 简称为“Blackwell GPU”。

NVIDIA GPU Tensor Cores 与 Transformer Engine

说到计算单元，Blackwell 引入了专门面向 AI 工作负载的增强。其中核心是 NVIDIA 的 Tensor Core 技术和 Transformer Engine（TE）。Tensor Cores 位于 GPU 的每个流式多处理器（streaming multiprocessor，SM）中，是可高速执行矩阵乘法的专用单元。

Tensor Cores 在此前各代已存在，但 Blackwell Tensor Cores 支持更多数值格式，包括 8 位、4 位浮点等极低精度格式。低精度背后的想法很简单：使用更少位表示数字，便可同时执行更多运算；同一数据占用的位数也更少，内存利用更充分。当然，这要求算法能容忍一定的数值精度损失。如今，许多 AI 算法在设计时就考虑了低精度数值格式。

NVIDIA 率先推出 TE，用于在深度学习中自动调整并使用混合精度：关键层使用较高精度（FP16 或 BF16），较不关键的层使用 FP8。TE 会自动优化精度平衡，目标是在较低精度下维持模型准确性。

Hopper 一代的 TE 首次引入 FP8 支持，相较 FP16 可使理论吞吐量翻倍。Blackwell 更进一步，引入 NVIDIA FP4（NVFP4）：这是一种仅使用 FP8 一半位数的 4 位浮点格式。FP4 足够小，在适用工作负载上可能使计算吞吐量相对 FP8 再翻倍。图 2-3显示 FP8 和 FP4 相对 FP16 的加速比。

一整个 NVL72 机架（72 块 GPU）在 4 位精度下的理论 Tensor Core 吞吐量超过 1.4 exaFLOPS（即 1.4×10^{18} FLOPS）。这个数字令人惊叹，使单个机架跻身世界最快超级计算机的量级——尽管是在低精度 FP4 下。真实工作负载未必总能达到峰值，但该能力的确存在。

现代 GPU 使用的 TE 增加了 NVFP4 支持，并改进了缩放与校准。实践中，可在 PyTorch 等框架内使用其内核和模块来采用 TE；这样，FP8 与 NVFP4 会在能够保持精度时应用。并非所有框架都会完全自动地为每层作出这一决定。

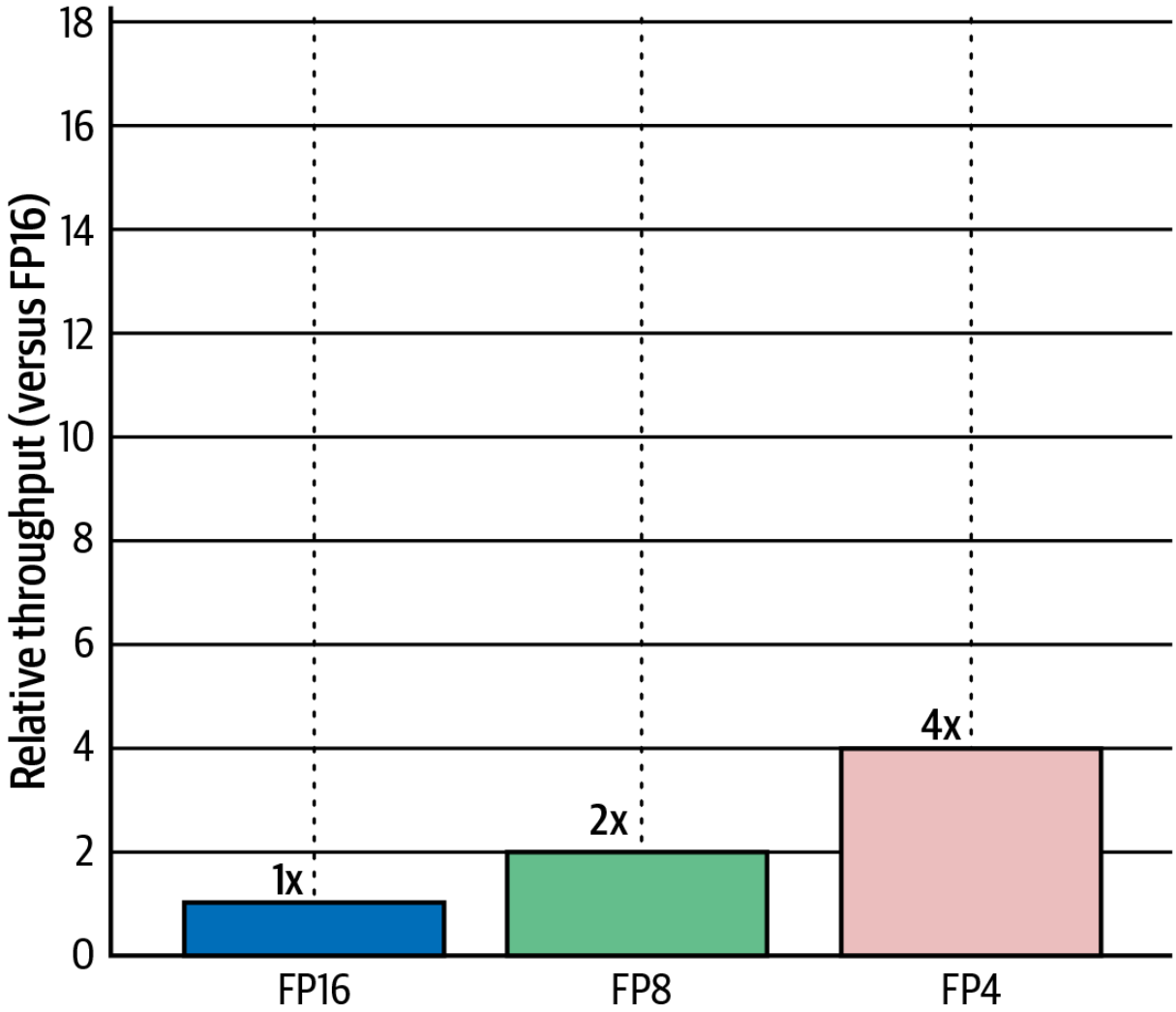


图 2-3. FP8 和 FP4 相比 FP16 的相对加速比

高级技术包括在训练和推理期间动态改变神经网络各层精度。目标是为每层使用仍能保持模型准确性的最低精度。例如，TE 可能将神经网络前几层保留为 FP16，因为早期层可能

对噪声敏感；而基于启发式规则，它可能为容忍度更高的后续层，或高精度并非那么关键的巨大嵌入矩阵使用 FP8 或 FP4。

所有这些可在 NVIDIA 库和 PyTorch 等 AI 框架内部完成。用户只需启用混合精度，就可能获得近似“免费”的大幅加速。第 9 章将讨论混合精度；如今许多 LLM 正因此使用它。相对 FP16 和 FP32，这些低精度可提高训练速度并减少精度损失。Blackwell 的设计让 FP8 与 FP4 更易用、更高效。

这些低精度格式也能降低内存使用量。与 FP8 相比，FP4 将每个参数所需内存减半（FP8 又将 FP16 的内存使用量减半），因此可在 GPU 内存中容纳更大的模型。

NVIDIA 显然押注于 AI 的未来在于低精度算术，并赋予 Blackwell 擅长此类计算的能力。这对大模型推理服务尤为关键，因为吞吐量（每秒 token 数）和延迟至关重要。

为说明 Hopper 到 Blackwell 的代际跃升，NVIDIA 报告称：基于 H100 的系统在一个 1.8 万亿参数 MoE 模型上，每 GPU 每秒只能生成约 3.4 个 token，且首 token 延迟超过 5 秒，难以用于交互式场景。

基于 Blackwell 的系统（NVL72）在相同模型上达到每 GPU 每秒约 150 个 token，首 token 延迟低至约 50 毫秒。按该厂商测试配置计算，这相对 Hopper 一代的实时吞吐量提升约 30 倍。NVL72 因此能让这一大模型提供实时响应，打开更多低延迟应用场景；实际结果取决于模型、精度和服务配置。

这一加速来自原始 FLOPS、更快 GPU、低精度（FP4）的结合，以及通过 NVLink 互连持续向 GPU 供给数据。它说明跨计算与通信的整体设计如何转化为实际性能提升。

归根结底，Blackwell GPU 比前代更强大、更智能，也获得更充足的数据供给。借助 Tensor Cores、TE 和低精度，它们可更快完成数值计算；巨大的内存带宽、大缓存和 NVLink 则确保数据能迅速可用。

继续之前，先快速讨论 GPU 内部层级，这有助于理解后续的性能调优。

流式多处理器、线程与 Warp

每块 Blackwell GPU 与前代一样，由许多流式多处理器（SM）组成。可将其视作 GPU 的“核心”，如图 2-4所示。

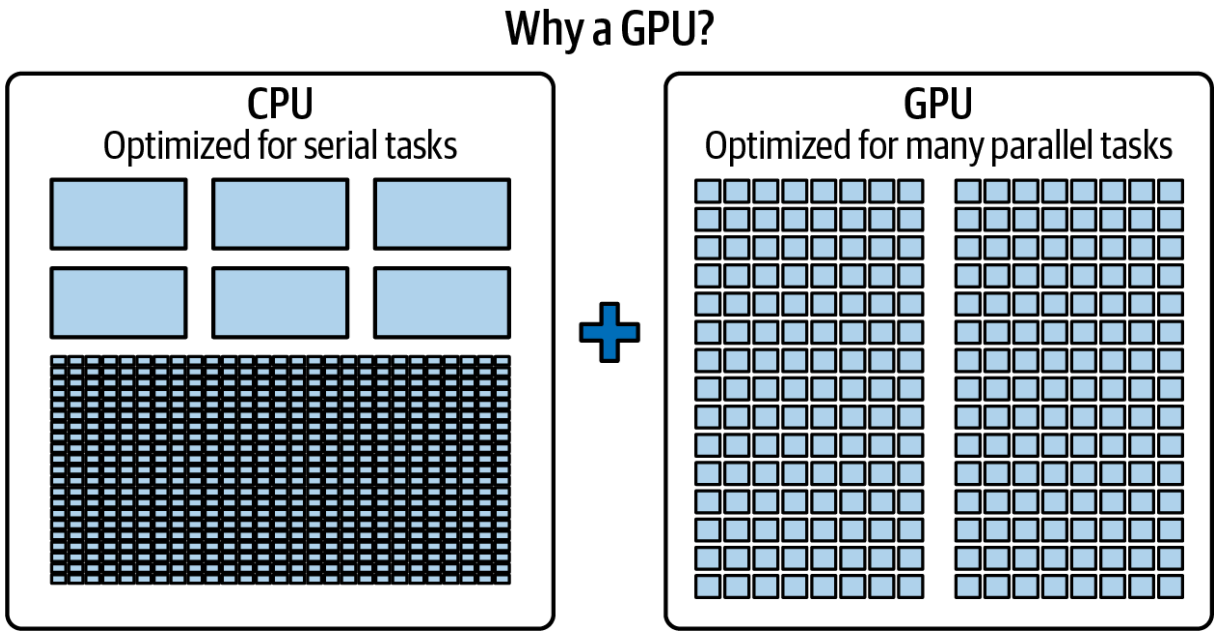


图 2-4. CPU 核心与 GPU 核心的比较（来源：https://oreil.ly/003EH、https://oreil.ly/Z25Tf）

每个 SM 包含多个算术单元（用于 FP32、INT32 等）、用于矩阵计算的 Tensor Cores、用于内存操作的加载/存储单元，以及处理超越函数等的某些特殊函数单元。GPU 还有自己的小型超高速内存池，包括寄存器、共享内存和 L1 缓存。

SM 以称为 *warp* 的固定大小分组执行线程。每个 warp 恰含 32 个线程，以锁步方式执行完全相同的指令。这称为单指令多线程（single instruction, multiple threads，SIMT）执行模型。

SM 可并行执行许多活跃 warp，以帮助掩盖线程等待从全局内存取数的延迟。设想一个 SM 同时有数十个 warp（数百个线程）在飞行：若一个 warp 等待内存读取，另一个 warp 可以运行。这称为*延迟隐藏*（latency hiding）。本书会反复讨论它；它是性能调优工具箱中非常重要的工具。

Blackwell 等高端 GPU 会有数百个 SM，每个 SM 都能同时运行数千线程，因此单块 GPU 上可拥有数万个活跃线程。如前所述，所有这些 SM 共享 126 MB L2 缓存，以及连接 HBM 的内存控制器。内存层级包括：寄存器（每线程）→ 共享内存（每个 SM 上、每线程块）→ L1 缓存（每 SM）→ L2 缓存（GPU 上所有 SM 共享）→ HBM 内存（片外），如图 2-5所示。

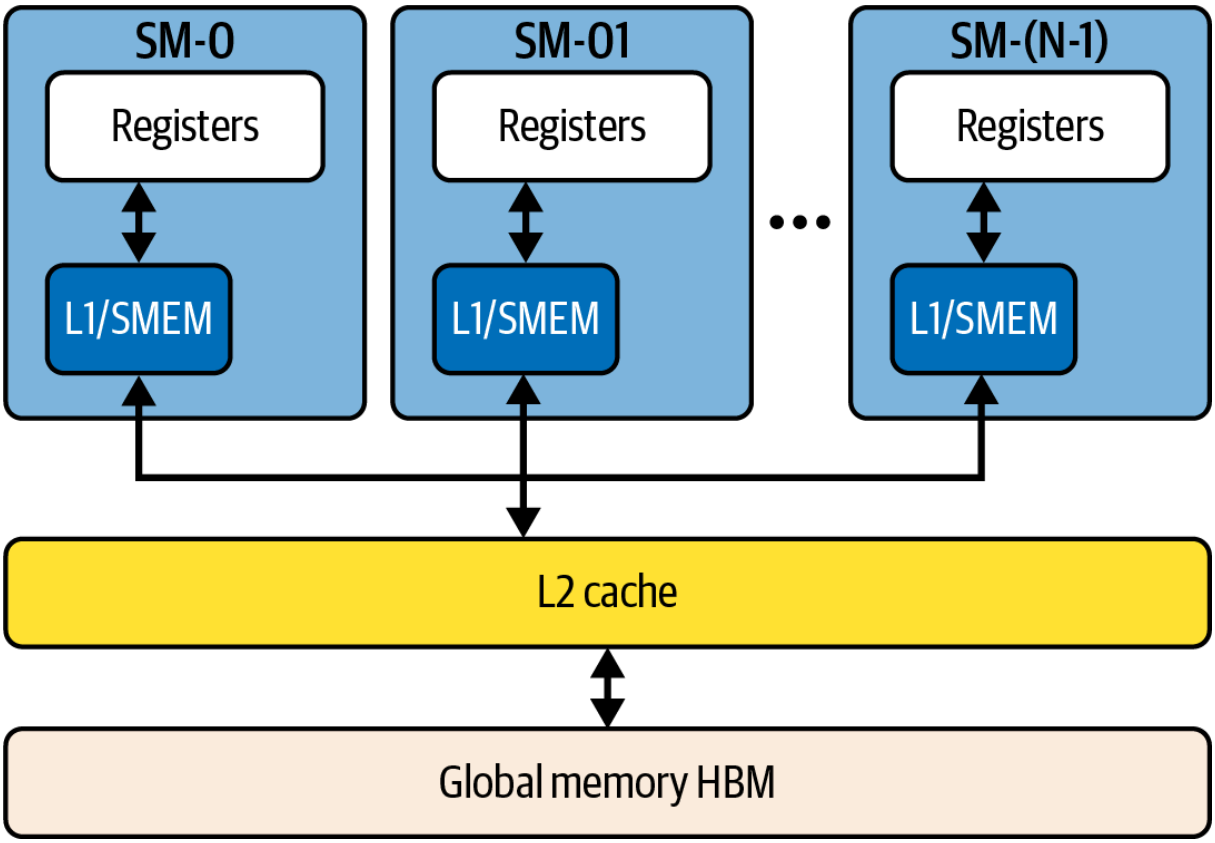


图 2-5. GPU 内存层级

为了获得最佳性能，数据应尽可能停留在该层级的高位。即便 HBM 带宽达到 8 TB/s，如果每次操作都访问 HBM，GPU 仍会因片外内存访问延迟增加而频繁停顿。将可复用数据保留在 SM 本地内存或 L2 缓存中，GPU 才能实现极高吞吐量。Blackwell 将缓存和带宽翻倍，正是为了持续供给 GPU。

作为性能工程师，我们会看到许多内核的性能既受计算能力，也受内存流量与吞吐量限制。NVIDIA 显然将 Blackwell 设计为使许多 AI 工作负载的 FLOPS 与内存带宽相匹配。

Blackwell 的设计平衡了计算与内存，因此对于许多 AI 内核，GPU 能以较少停顿持续计算。实践中，经过良好优化的稠密数学运算可复用片上内存中的数据，接近峰值 FLOPS，而不会严重受内存限制。

这些特性意味着，在代码经过良好优化时，GPU 通常会忙于计算而非等待数据。请注意，巨大归约、随机内存访问等操作仍可能受内存限制，但更新后的 GPU、内存和互连硬件会在一定程度上缓解这一问题。

将多块 GPU 视作一体的超大规模网络

将两块 GPU 和一颗 CPU封装进超级芯片，可得到极强大的节点。下一项挑战是连接许多此类超级芯片，以扩展到更大模型的训练。

NVIDIA 提供一种使用 GB200/GB300 超级芯片的大型机架配置，称为 NVL72 系统。NVL72 指拥有 72 块 Blackwell GPU 和 36 颗 Grace CPU、全部通过 NVLink 互连的系统。本质上，它是一整个机架中的 AI 超级计算机。

如图 2-6所示，GB200/GB300 NVL72 由 18 个计算节点构建；每个节点包含两个 GB200/GB300 超级芯片，因此每个计算节点合计为四块 Blackwell GPU 和两颗 Grace CPU。

图 2-6. GB200/GB300 NVL72 机架内配备两颗 Grace Blackwell 超级芯片的 1U 计算托盘（来源：developer.nvidia.com）

这里，每个超级芯片模块包含一颗 Grace CPU 和两块 Blackwell GPU（每块 B200 都是双裸片 MCM）。NVL72 将 18 个这样的托盘连接在一起。通过互连这 18 个计算节点，GB200/GB300 NVL72 将 72 块 Blackwell GPU（18 节点 × 每节点 4 块 GPU）和 36 颗 Grace CPU（18 节点 × 每节点 2 颗 CPU）组成强大的统一 CPU-GPU 集群。

NVL72 的特别之处在于：在单个 NVLink 域内，每块 GPU 都能通过 NVLink Switch 网络结构以极高速度与任意其他 GPU 通信。NVIDIA 通过 GPU 上的 NVLink 5 连接和名为 NVSwitch 的专用交换芯片实现这一点。

NVLink 与 NVSwitch

每块 Blackwell GPU 提供18 个 NVLink 5 端口。NVL72 将所有端口连接至 NVLink Switch System，每 GPU 的 NVLink 聚合双向带宽为 1.8 TB/s（18 条 NVLink 链路 × 每条双向 100 GB/s）。每个 NVLink 交换托盘提供 144 个、速率 100 GB/s 的 NVLink 端

口。九个托盘中，每块 GPU 的 18 条 NVLink 5 链路分别连接到一个 NVSwitch 芯片，使 72 块 GPU 以全双向带宽完全互连。

这相当于 Hopper GPU 所用上一代每 GPU NVLink 带宽的两倍。Hopper H100 也使用 18 个 NVLink 4 端口，但其速度为 NVLink 5 的一半。经 NVLink 的 GPU 间延迟处于个位数微秒范围。

GPU 通过 NVSwitch 芯片以网络方式连接。NVSwitch 本质上类似网络交换机，却专为 NVLink 构建。因此，任一 GPU 都可在 NVLink Switch System 中经过一级交换，以全双向带宽到达另一 GPU。在单个 NVL72 机架内，这种单级特性成立，因为每块 GPU 用其 18 条 NVLink 链路连接到 18 颗 NVSwitch 芯片，从而形成经过单个交换机的路径。图 2-7展示 NVL72 所用的 NVLink Switch 托盘。

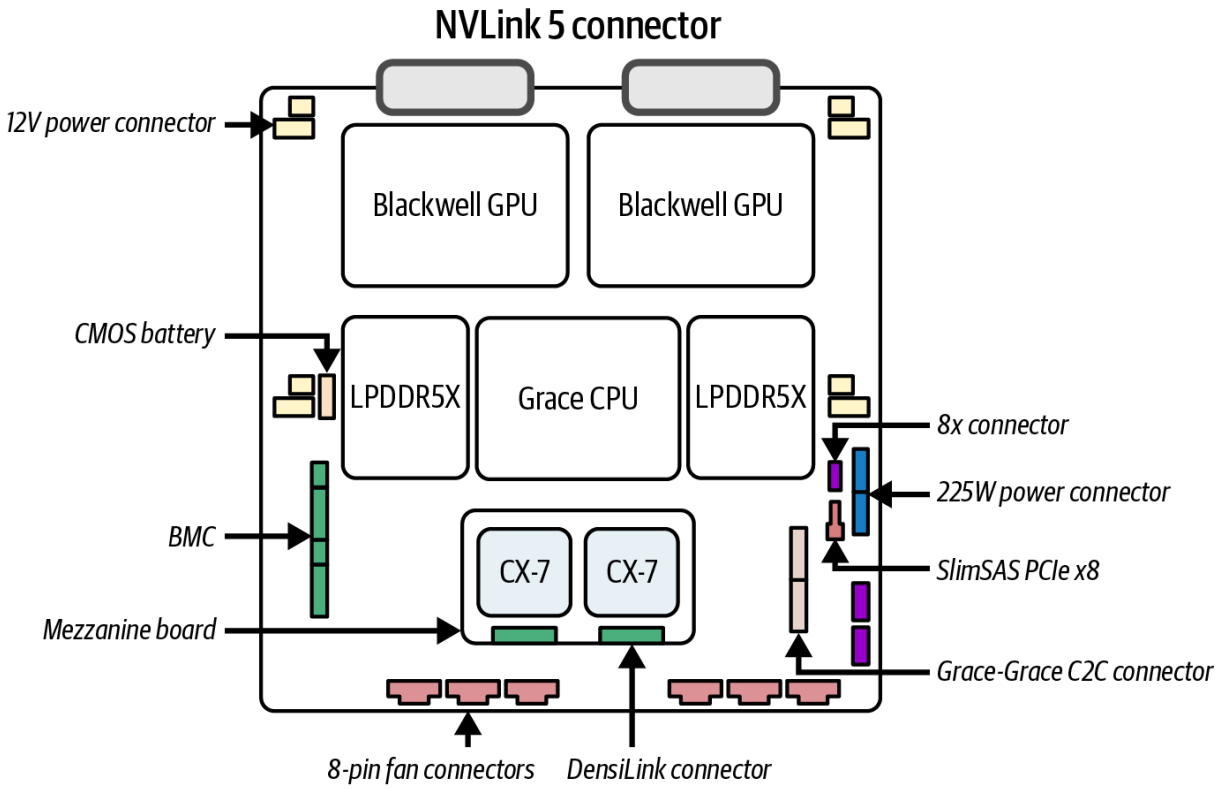


图 2-7. NVL72 内的一个 NVLink Switch 托盘（来源：https://oreil.ly/h7seG）

每个交换托盘包含两颗 NVSwitch 芯片和多个高速端口。如图 2-8所示，NVL72 机架由 9 个此类交换托盘和 18 个计算托盘组成。

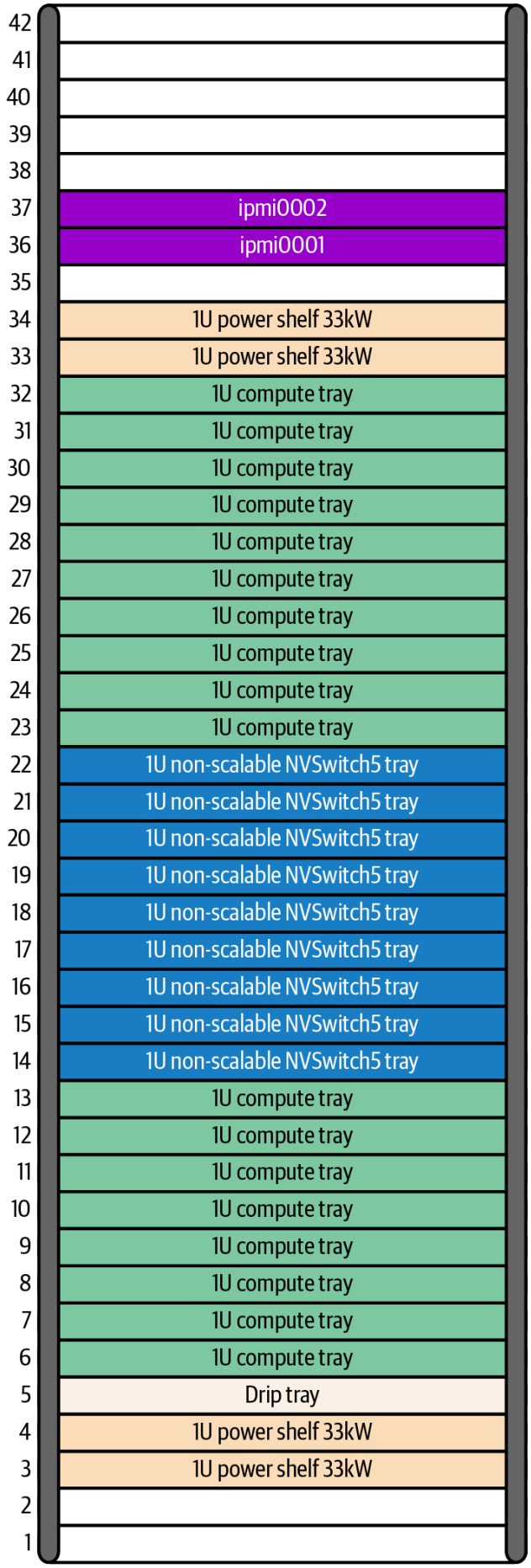

42
41
40
39
38
37
ipmi0002
36
ipmi0001
35
34
1U power shelf 33kW
33
1U power shelf 33kW
32
1U compute tray
31
1U compute tray
30
1U compute tray
29
1U compute tray
28
1U compute tray
27
1U compute tray
26
1U compute tray
25
1U compute tray
24
1U compute tray
23
1U compute tray
22
1U non-scalable NVSwitch5 tray
21
1U non-scalable NVSwitch5 tray
20
1U non-scalable NVSwitch5 tray
19
1U non-scalable NVSwitch5 tray
18
1U non-scalable NVSwitch5 tray
17
1U non-scalable NVSwitch5 tray
16
1U non-scalable NVSwitch5 tray
15
1U non-scalable NVSwitch5 tray
14
1U non-scalable NVSwitch5 tray
13
1U compute tray
12
1U compute tray
11
1U compute tray
10
1U compute tray
9
1U compute tray
8
1U compute tray
7
1U compute tray
6
1U compute tray
5
Drip tray
4
1U power shelf 33kW
3
1U power shelf 33kW
2
1

图 2-8. NVL72 机架内由九个托盘构成的 NVSwitch 系统（来源：https://oreil.ly/h7seG）

由于 9 个交换托盘各含两颗 NVSwitch 芯片，NVL72 系统总计有 18 颗 NVSwitch 芯片。网络按全交叉开关布置：每块 GPU 连接到每颗 NVSwitch，每颗 NVSwitch 也连接到每块 GPU，从而在任意 GPU 对之间提供高带宽路径。

每个交换托盘暴露 144 个 NVLink 端口，以完全连接每块 GPU 的 18 条 NVLink 链路。具体而言，每块 GPU 使用其 18 条 NVLink 链路连接到 18 颗 NVSwitch 芯片（每个交换机一条）。这意味着任何 GPU 都可经过一跳（GPU → NVSwitch → GPU）到达任意其他 GPU，同时获得极大带宽。图 2-9展示完整的 NVL72 架构：72 块全互连 GPU（36 个 GB200 超级芯片）和 18 颗 NVSwitch。

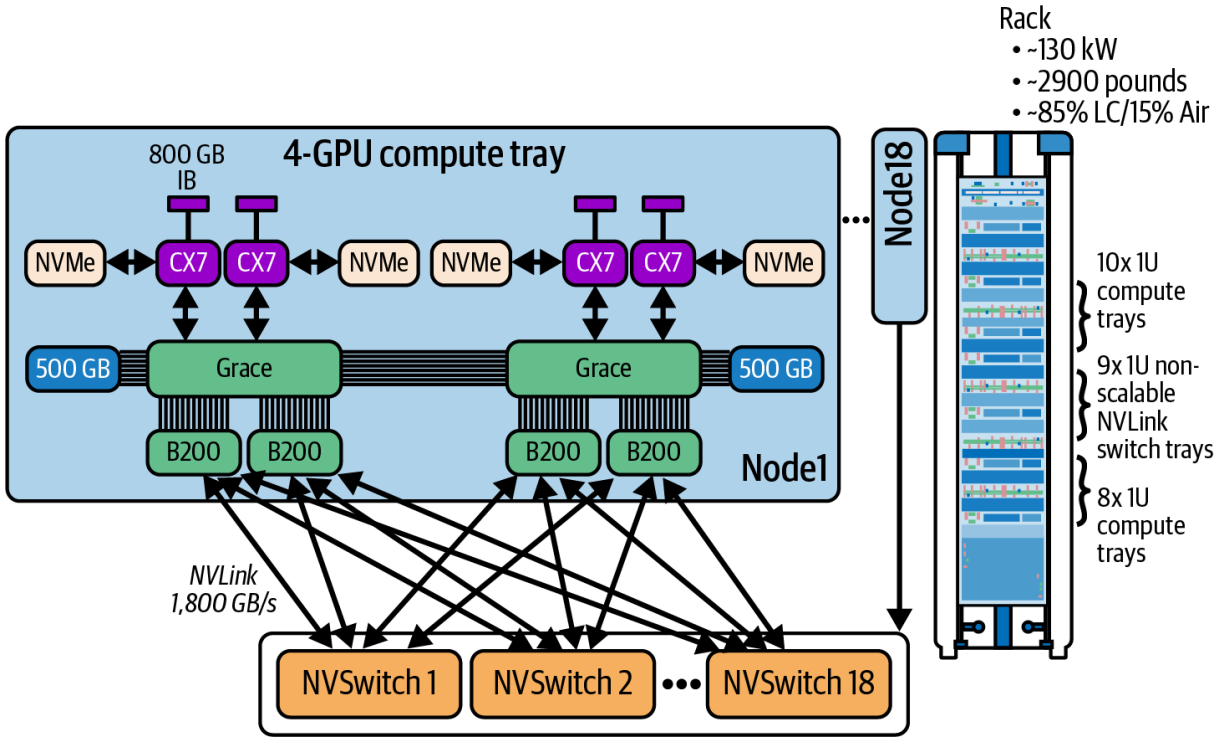


图 2-9. 每块 GPU 连接到每颗 NVSwitch（每个交换机一条链路）

在 NVL72 机架内，整个 72-GPU 网络的聚合双向带宽约为 130 TB/s。作为参照，这比同等规模的顶级 InfiniBand 集群高出许多倍。该设计提供全互连高带宽网络结构和跨 GPU 的全局地址空间，使集合操作和单边操作得以高效执行，同时仍由软件显式控制同步和一致性。

多 GPU 编程

从编程模型的角度看，一块 GPU 可以通过 NVLink 使用点对点和分区全局地址空间（PGAS）模型，例如 NVIDIA SHMEM（NVSHMEM），直接访问另一块 GPU 的内存。各 GPU 缓存并不具备全局一致性。只有 NVLink-C2C 的 CPU–GPU 路径具备缓存一致性。NCCL 和 NVSHMEM 提供同步与顺序保证。

远程直接内存访问（RDMA）是一种网络技术，可在 InfiniBand 和 RoCE 传输中实现主机之间的直接、零拷贝内存传输。InfiniBand Trade Association (IBTA)

可选的远程原子操作由 InfiniBand Trade Association（IBTA）为 InfiniBand 和 RoCE 定义。GPUDirect RDMA 是 NVIDIA 对 RDMA 协议的实现，允许网络接口控制器（NIC）注册 GPU 内存，并通过 nvidia-peermem 驱动直接执行 RDMA。GPU 可以在不经过 CPU 的情况下跨节点交换数据并执行原子操作。GPUDirect RDMAnetwork interface controllers (NICs)nvidia-peermem

节点之间的远程原子操作和单向操作由 NVSHMEM 等上层库提供，它们在 RDMA 传输上实现这些语义。GPUDirect RDMA 提供的是直接数据路径，而不是原子 API 本身。分布式训练和推理工作负载需要在许多 GPU 之间频繁同步并交换信息。

传统上，GPU 分布在不同的计算节点和机架中。同步只能通过 InfiniBand 和以太网等相对较慢的网络链路完成，这往往成为大型 AI 模型在多 GPU 上扩展时的瓶颈。

在 NVL72 系统中，这些交换可以以极高速率通过 NVLink 和 NVSwitch 完成。训练作业或推理集群可以扩展到 72 块 GPU，同时仅产生极小的通信开销。

相比之下，将相同任务扩展到规模相近的 72-GPU H100 集群，需要使用九台独立计算服务器，每台配备八块 Hopper H100 GPU。这种配置依赖 InfiniBand，网络瓶颈会显著降低集群的扩展效率。

下面用具体数字比较 NVL72 与 72-GPU H100 集群。在单个 NVL72 机架内，GPU 到 GPU 的带宽最高可达每 GPU 1.8 TB/s（双向聚合）；对于数 KB 的小消息，延迟约为 1–2 微秒。大消息耗时更长，通常受带宽限制。

NVL72 网络的每 GPU 带宽明显高于基于主机 NIC 的网络，延迟也更低。NVLink 5 为每块 GPU 提供约 1.8 TB/s 的聚合带宽，而现代主机 NIC 在 400–800 Gb/s 链路速率下每端口约为 50–100 GB/s。

实际而言，与传统节点到节点网络相比，基于 NVLink 的 NVL72 系统中集合操作开销要低得多。NVIDIA 报告称，一个 1.8 万亿参数的 MoE 模型在 H100 上约为每 GPU 每秒 3.4 个 token，而 GB200 NVL72 上约为 150 个 token，首 token 时间从超过 5 秒降至约 50 毫秒。reported

在单个 NVL72 机架内，通信速度极快，通信瓶颈几乎被消除；而在传统 InfiniBand 和以太网集群中，通信通常是主要瓶颈。

应尽量将工作负载的通信保留在机架内（“机架内”），以利用高速 NVLink 和 NVSwitch。只有在需要扩展到超出 NVL72 计算和内存资源时，才使用机架间的 InfiniBand 或以太网通信。

使用 NVIDIA SHARP 进行网络内聚合

另一种由硬件支持的优化是网络内集合操作卸载，涉及 NVLink 和 InfiniBand 可扩展分层聚合与归约协议（Scalable Hierarchical Aggregation and Reduction Protocol, SHARP）。这些路径的启用方式、拓扑约束和支持的集合操作行为各不相同。

NVLink SHARP（NVLS）为启用 SHARP 的 NVSwitch 域提供集合操作卸载支持。对于这类 NVSwitch 系统，支持的集合操作可以在交换机硬件中完成，而不必让 GPU 执行全部聚合计算或传输所有中间结果。

图 2-10 展示了 NVLink Switch System 机架如何使用直接集成在 NVSwitch ASIC 中的 SHARP 引擎，在网络内执行归约，从而卸载归约及其他集合操作。图 2-10

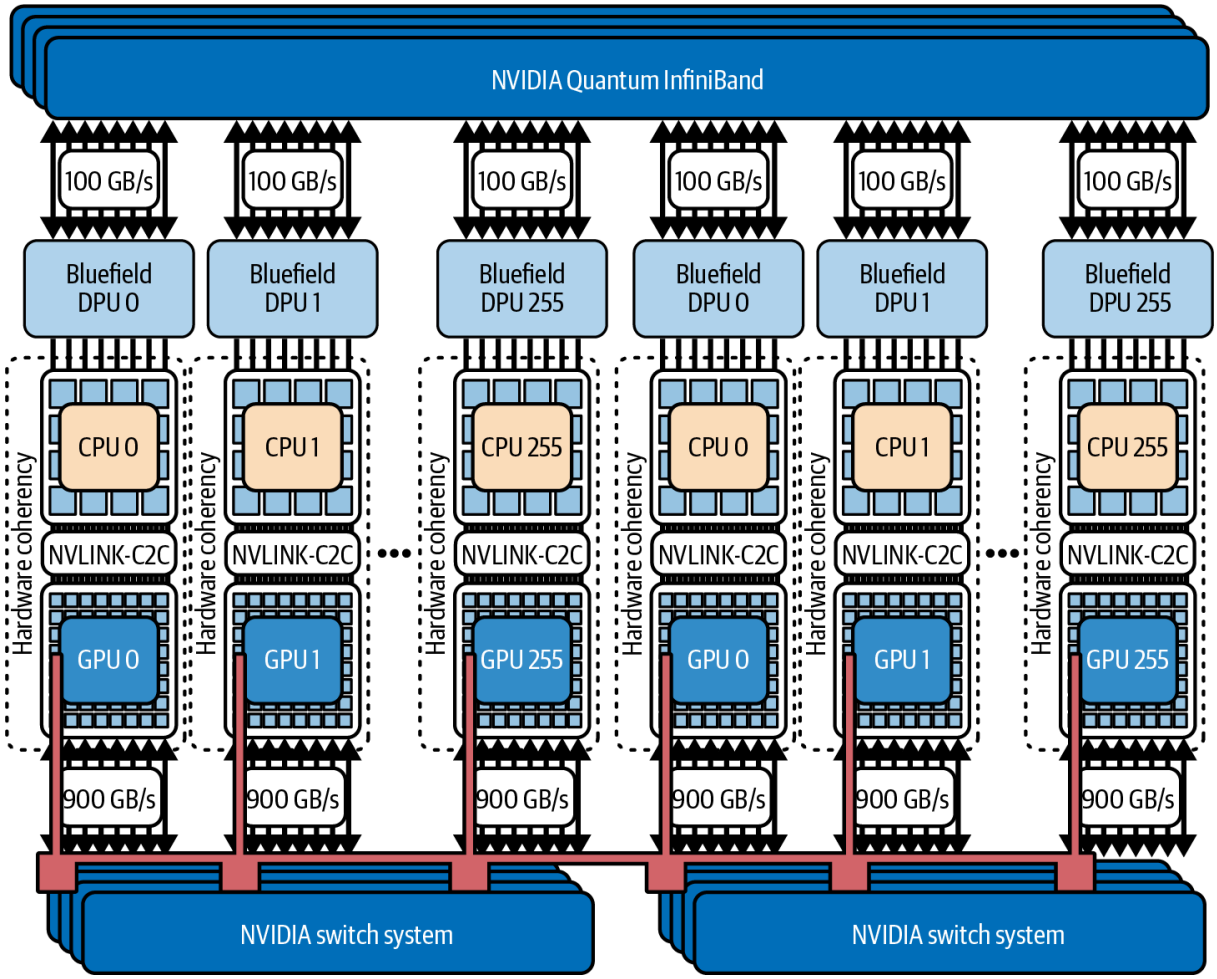


图 2-10： NVLink SHARP（NVLS）归约引擎在 NVSwitch 域内执行交换机内集合操作卸载

图 2-10：NVLink SHARP（NVLS）归约引擎在 NVSwitch 域内执行交换机内集合操作卸载。

NVSwitch 网络可以在不让数据汇聚经过 GPU 的情况下合并部分结果。SHARP 将集合计算从 GPU 卸载到交换机硬件本身，使 GPU 能专注于更复杂的计算，同时降低集合操作延迟、减少网络传输的数据总量并提高系统效率。

SHARP 的高效率意味着，在分布式训练期间，梯度聚合或参数同步等繁重工作由 NVSwitch 专用的 SHARP 引擎处理。因此，无论是机架内还是机架间配置，都能获得更高效的扩展。

使用 SHARP 后，即使 GPU 数量增加，性能也能接近线性提升。这种网络内计算能力对于超大规模模型训练尤其重要，因为集合操作节省的每一个微秒，都可能转化为显著的整体加速。

SHARP 是 NVIDIA 在 2019–2020 年收购 Mellanox 时获得的最具影响力的创新之一。如果目前尚未使用 SHARP，值得进行评估。SHARP 可以显著降低集合操作的延迟和流量，并且通常能提高通信受限训练的扩展效率。

多机架与存储通信

接下来讨论一个 NVL72 机架如何与另一个 NVL72 机架或共享文件系统等外部存储系统通信。如前所述，NVL72 机架内部的所有 GPU 到 GPU 流量都由 NVLink 承载；机架之外则依赖更传统的网络硬件。

NVL72 的每个计算节点都配备高速网络接口卡和数据处理单元（DPU）。DPU 从主机 CPU 卸载、加速并隔离网络、存储、安全和管理任务。通过直接在 NIC 上运行这些操作，DPU 可以降低 CPU 开销和延迟。

在 NVL72 设计中，BlueField-3 DPU 负责线速数据包处理、RDMA 和 NVMe over Fabrics（NVMe-oF）操作。NVMe-oF 是 NVMe 的协议变体，可将存储扩展到网络结构上。因此，DPU 无需 CPU 参与，就能在网络、存储和 GPU 内存之间直接移动数据，从而最大限度地提高系统整体吞吐量和效率。

GB200/GB300 NVL72 机架可与 Quantum-X800 InfiniBand 或 Spectrum-X800 以太网网络集成。计算托盘通常在每个节点使用四块 ConnectX-8 800 Gb/s NIC，以获得较高的外部带宽。需要为存储、安全和控制平面任务提供网络内加速或卸载时，则使用 BlueField-3 DPU。

四块 800 Gb/s NIC 可为每个计算节点提供 3.2 Tbit/s，总计每机架约 57.6 Tbit/s（57.6 Tbit/s = 3.2 Tbit/s/节点 × 18 个节点）。尽管这一吞吐量十分惊人，但离开机架后仍需要超高速网络，才能避免多机架扩展在机架边界处形成瓶颈。NVIDIA 将这类多机架部署称为*AI 工厂*，并通过每节点四块 NIC 使 NVL72 能够接入更大规模的网络结构。

每个节点中的 BlueField-3 DPU 可帮助卸载 RDMA、TCP/IP 和 NVMe SSD 存储访问等网络任务，避免 Grace CPU 因处理网络中断而承受过大负担。DPU 实际上充当智能网络控制器，使用 NVIDIA GPUDirect RDMA 软件在 NIC 与 GPU 内存之间直接移动数据，无需经由主机内存暂存，也不消耗 CPU 周期。

BlueField DPU 可避免 CPU 参与，这在大型训练作业从存储服务器流式读取大规模数据集时尤其有用。具体而言，DPU 可以处理传输并将数据直接写入 GPU 内存，同时让 CPU 专注于数据预处理等其他任务。

除提供性能卸载能力外，DPU 还支持安全多租户。它可以隔离不同作业和用户的网络流量，在节点上充当智能防火墙和交换机。

扩展到多个 NVL72 机架时，NVIDIA 使用 Quantum 系列 InfiniBand 交换机。多个 NVL72 机架可以通过这些 InfiniBand 交换机互连，形成大型 NVL72 机架集群。

例如，由 8 个 NVL72 机架、总计 576 块 GPU 构成的系统，可通过 NVLink Switch System 连接为一个 NVLink 5 域。随后使用 InfiniBand 或以太网将该 NVLink 域连接到其他域（例如其他 NVL72 机架）或外部存储；不过，跨机架 InfiniBand 或以太网通信的性能低于机架内 NVLink/NVSwitch 通信。

简而言之，InfiniBand 和以太网 NIC（例如 NVIDIA ConnectX）以及 BlueField DPU 通常与 NVLink 配合使用。它们既提供机架之间的高带宽连接，也借助 DPU 上的网络内计算卸载协议处理。

预集成机架设备

由于 NVL72 是非常复杂的系统，NVIDIA 将其作为单机柜预集成“设备”交付。机柜出厂时已装配全部 18 个计算节点、9 个 NVSwitch 单元、内部 NVLink 线缆、配电和冷却系统。用户可以整机订购，设备到货即可使用：只需接入设施电源、连接水冷接口和 InfiniBand 网络线缆，然后开机即可。

该系统开箱后基本即可使用，只需少量设置便能运行 AI 工作负载。无需为 72 块 GPU 单独连接 NVLink，因为 NVIDIA 已在机架内部完成布线。液冷系统也采用自包含设计，后文将对此进行讨论。

这种设备化交付方式可以加快部署，并确保系统由 NVIDIA 正确构建和验证。机架还包括 NVIDIA Base Command Manager 集群管理软件，以及用于集群作业调度和编排的 Simple Linux Utility for Resource Management（SLURM）和 Kubernetes。

简而言之，NVL72 机架旨在直接部署到用户环境中，开箱即可运行生产 AI 工作负载。它不需要手动安装或复杂配置。

共封装光学器件：网络硬件的未来

随着网络数据吞吐率提升到 800 Gbit/s、1.6 Tbit/s 甚至更高，NVIDIA 开始将硅光子和共封装光学器件（CPO）集成到网络硬件中。这包括 Quantum-X800 InfiniBand 和 Spectrum-X800 Ethernet 平台。两者提供 800 Gb/s 端到端连接及网络内计算功能（例如 SHARP）。采用 CPO 后，光学发射器可以紧邻交换机芯片集成，从而大幅缩短电气路径，支持机架之间更高带宽的链路、降低功耗并提高整体通信效率。

从实际效果看，CPO 等技术正在为将数百乃至数千个机架（AI 工厂）连接到统一网络结构铺平道路，使机架间带宽不再成为瓶颈。这类光网络进步对于超大规模 GPU 系统所需的高性能机架间带宽至关重要。

总结来说，在 NVL72 机架内部，NVIDIA 使用 NVLink 和 NVSwitch 为 72 块 GPU 构建极高速的全互连网络。这些互连速度快且特性一致，使 GPU 在许多集合操作中实际上表现得像一个整体。机架之外，高速 NIC（例如 InfiniBand 或以太网）将机架连接到其他机架或存储系统，DPU 则负责高效管理数据移动。

NVL72 既是功能强大的独立系统，也是构建更大 AI 超级计算机或*AI 工厂*的基础单元。由多个此类机架组成的大规模 AI 数据中心正在成为现实。NVIDIA 正与 HPE、Supermicro 等 OEM 和系统厂商合作供应 GB200 NVL72 系统。NVIDIA 的硬件和网络路线图都明确面向 AI 工厂愿景。总之，NVL72 展示了软硬件协同设计的潜力：GPU、网络和机架硬件协同构建，以尽可能无缝、高效地扩展到数千乃至数百万块 GPU。

计算密度与功耗要求

NVL72 机架的计算密度极高，因此单个机架的功耗也非常高。满载 NVL72 在最大负载下最多可消耗约 130 kW，比 NVIDIA 上一代约 50–60 kW 的 AI 机架高出两倍以上。将 72 块前沿 GPU 和全部配套硬件装入一个机架，已经逼近数据中心基础设施的承载极限。

要为 NVL72 机架提供 130 kW，不能只使用一条标准供电线路。数据中心通常需要配置多条大容量回路。例如，可以部署两条完全独立的供电线路，并让每条线路都具备在另一条

故障时承载整个机架负载的能力。

如果一条供电线路离线，剩余回路仍可支持完整的 130 kW 负载，从而避免电路过载跳闸。这种冗余是重要的保护措施，否则断电可能会中止持续数月的训练作业。

在机架内部，电力分配到每个 1U 计算节点的电源。电力先从交流转换为直流，供本地电子元件使用。NVL72 的每个计算节点包含两块 Grace Blackwell Superchip，合计功耗约 6 kW。18 个计算节点总计约 110 kW；NVSwitch 托盘、网络交换机、风冷设备和水冷泵约消耗 20 kW，因此整个 NVL72 机架总功耗为 130 kW。

典型数据中心在 415 V 三相交流电等电压下需要使用极大的电流，因此所有设备都必须按高电流设计。运营人员必须谨慎规划机架部署，通常需要专用配电单元（PDU）和严密监控。还必须考虑功率瞬变：72 块 GPU 从空闲升至满功耗时，可能在几毫秒内迅速吸收数十千瓦功率。良好的设计会加入电容或采用分时启动，以避免大幅电压下降。

系统可能会让 GPU 的升压时钟以极小的时间间隔错开，使它们不会在完全相同的微秒内同时达到峰值，从而平滑功率浪涌。正是这些电气工程细节，使 130 kW 机架变得可管理。

将这款处于高密度计算前沿的 NVL72 机架称为迷你电力变电站并不夸张。8 个机架、共 576 块 GPU 将消耗接近 1 MW 的功率（8 个机架 × 每机架 130 kW），这相当于一个小型数据中心的全部容量！值得注意的是，虽然单机架 130 kW 功耗很高，但单位功率能够完成的工作量也非常大。

如果一台 NVL72 能替代数个旧设备机架，整体效率会更高。但你确实需要相应基础设施来支撑如此集中的功耗。部署 NVL72 机架的任何设施都必须确保具备充足的供电能力和冷却能力，下一节将讨论这一点。

液冷与风冷对比

在单个机架中冷却 130 kW 的热量，已经超出传统风冷的能力范围。让气流通过 72 块 GPU（每块约可耗散 1,200 W）需要飓风般的风量，噪声极大且效率低下，更不用说排出的热空气会非常难以处理。因此，对于这种功率密度的 NVL72 机架，液冷是唯一可行的方案。

NVL72 是完全液冷的系统。每个 Grace Blackwell Superchip 模块和每个 NVSwitch 芯片都安装有冷板。冷板是带有内部管路、直接贴合在组件上的金属板。水基冷却液流经管路带走热量，所有冷板通过软管、歧管和泵连接起来，使冷却液在整个系统中循环。

通常，机架会为每个节点配备快速断开接头，使服务器可以滑入或移出而不会洒出冷却液。机架还会连接到外部设施冷冻水系统的供水和回水管路。热交换器通常称为冷却液分配单元（CDU），可以集成在机架内部或安装在机架旁边。CDU 将机架内部冷却回路的热量传递到数据中心的水回路。

设施提供温度为 20–30°C 的冷冻水，水通过热交换器吸收热量，然后被泵回冷水机或冷却塔重新冷却。在现代设计中，甚至可以采用温水冷却：冷冻水以 30°C 进入系统，以 45°C 流出，随后通过蒸发式冷却塔冷却，而无需主动制冷，从而提高整体效率。关键在于，水或液体冷却剂单位流量所能携带的热量远高于空气，因此在小空间内处理高功率时，液冷的效果要好得多。

液冷能使 GPU 和 CPU 的温度远低于风冷条件，从而减少 GPU 的热降频。GPU 可以在不触及温度上限的情况下维持最高时钟频率。此外，芯片在较低温度下运行时功率泄漏更低，因此较低温度还能提高可靠性，甚至改善能效。

NVL72 在负载下可将 GPU 温度保持在 50–70°C，这对于此类高功耗设备来说非常出色。冷板和冷却回路经过精心设计，使每块 GPU 能向系统排出 1,000 W、每个 CPU 排出 500 W 的热量。冷却液流量也必须足够大，才能快速带走热量。粗略估计，要耗散约 130 kW，需要每分钟约 150–200 升的流量，并对应 10–12°C 的水温升高。

系统无疑配备了冷却液温度、压力和泄漏检测传感器及控制器。如果滴漏传感器或压力损失传感器检测到泄漏，系统可以迅速关机或隔离相应区域。建议使用自密封接头，并考虑设置二级防漏托盘，以降低液体泄漏风险。

这种级别的机架液冷过去很不寻常，但如今已成为大规模 AI 集群的标准。Meta、xAI 和 Google 等公司都在其 AI 集群中采用液冷，因为风冷无法支撑这些系统消耗的大量功率。

因此，尽管 NVL72 需要更复杂的设施（包括液冷回路），许多数据中心如今已按液冷进行建设。NVL72 机架内置液冷系统，可以直接接入设施冷却回路。

内部液冷的一个副作用是机架重量。装满硬件和冷却液后，NVL72 机架重约 3,000 磅（1.3–1.4 公吨）。这对机架来说非常重，大致相当于一辆小汽车的重量，却集中在几平方英尺的地面上。采用架空地板的数据中心必须检查地板能否承受以磅/平方英尺计量的负载。高密度机架通常放置在加固楼板上，或由额外支柱支撑。移动这种机架需要叉车等专用设备。这些都是部署时必须考虑的因素，因为安装 AI 超级计算机会带来独特的物理和物流挑战。

NVIDIA 还通过机架管理控制器集成管理和安全功能，负责监控冷却泵、阀门位置、功耗以及每个节点的状态。管理员可以通过它在所有节点上更新固件，或安全地关闭系统。

所有这些考虑都说明，NVL72 是围绕数据中心基础设施进行协同设计的。NVIDIA 与系统工程师共同设计计算架构、电力供应和冷却方案，并与设施工程师协作确定安装和运行方式。这不仅关乎高速芯片，也关乎交付一个均衡且可用的系统。

这种复杂性带来的回报非常可观。通过突破供电和冷却极限，大量计算能力被集中到单个机架中，从而实现很高的单位功率计算量。130 kW 的功耗确实很高，但按每块 GPU 或每万亿 FLOP（TFLOP）计算，与将相同 GPU 分散到多个、冷却效率较低的机架相比，其能效实际上更高。

实际性能监控与利用率

面对如此强大的机器，必须确保充分发挥其能力。高效运行 NVL72 需要仔细监控性能、利用率和功耗。NVIDIA 提供 Data Center GPU Manager（DCGM）等工具，可跟踪每块 GPU 的利用率、内存使用量、温度和 NVLink 吞吐量。

作为性能工程师，你会在训练运行和推理工作负载期间持续关注这些指标。理想情况下，训练作业期间 GPU 大部分时间都应接近 100% 利用率。如果看到 GPU 利用率只有 50%，说明某些因素让它们一半时间处于空闲状态，可能是数据加载瓶颈或同步问题。

同样，也可以监控 NVLink 的使用情况。如果 NVLink 链路频繁达到饱和，问题很可能出在通信上。BlueField DPU 和 NIC 也有各自的统计信息，可以监控这些指标，以确保读取

数据时存储链路没有饱和。NVL72 等现代系统会公开这些遥测数据。

功耗监控同样至关重要。在约 130 kW 的功耗水平下，即使很小的低效或配置错误，也可能浪费大量电力和资金。系统通常允许按节点或按 GPU 监控功耗。若不需要满性能，管理员可以限制 GPU 的功耗或时钟频率以节省能源。

NVIDIA GPU 支持设置功耗上限。例如，运行较小作业且不需要全部性能时，可以降低 GPU 时钟频率，以提高以每瓦性能衡量的能效，同时仍满足吞吐量要求。这样可能节省数千瓦功率，经过数周训练后可转化为显著的节能和成本效率。

共享与调度

另一个方面是 NVL72 上工作负载的共享与调度。很少有单个作业需要全部 72 块 GPU。多个团队或实验可能同时使用 GPU 子集运行任务。使用 SLURM 或 Kubernetes 等集群调度器及 NVIDIA 插件，可以在同一机架内为一个用户分配 8 块 GPU，为另一个用户分配 16 块，为第三个用户分配 48 块。

此外，NVIDIA 的多实例 GPU（MIG）功能可以在硬件层面将一块物理 GPU 划分为多个更小的 GPU。例如，一块拥有 180 GB GPU 内存的 Blackwell GPU 可以拆分为多个较小分区，同时运行许多小型推理作业。

每块 Blackwell GPU 最多支持七个完全隔离的 MIG 实例。这样，一块物理 GPU 可以划分为最多七个更小的 GPU，每个实例拥有专用内存和 SM。MIG 分区大小由产品代际固定。下一章将详细介绍 MIG 分区。

实际中，对于如此大型的 GPU，MIG 可用于在一块 GPU 上服务多个模型的推理场景。BlueField DPU 还支持安全多租户，因为 DPU 可以充当防火墙和虚拟交换机，隔离不同作业和用户的网络流量。这样，组织就能安全地让不同部门甚至外部客户使用系统分区，彼此互不干扰，类似云服务商将大型服务器划分给多个客户并提供安全的多租户隔离。

从成本角度看，NVL72 这类系统是价值数百万美元的资产，每月可能消耗数万美元电费。因此，必须尽可能完成更多有用工作，即提高有效吞吐量（goodput）。如果系统闲置，大量资本成本和运营成本都会被浪费。这就是持续监控利用率很重要的原因，可以跟踪 GPU 已使用小时数与可用小时数的对比。

如果发现系统利用率不足，可以整合工作负载，或向更多团队开放以承接更多项目。一些组织采用内部计费模式，由各团队使用自己的预算按 GPU 小时付费。这能鼓励高效使用，并计入电费和折旧成本。这样的透明度有助于让使用者认识到资源的价值。

升级硬件的投资回报率（ROI）

有人可能会问，投资这些前沿硬件是否值得。分析投资回报率（ROI）时，关键通常是每美元带来的性能。如果一台 NVL72 能完成四个旧一代机架的工作，那么从长期看，它可能同时节省硬件和电力成本。本章前面提到，就吞吐量而言，一块 Blackwell GPU 可以替代 2–3 块 Hopper GPU。因此，升级后完成相同工作可能只需更少的 GPU。

下面看一个简短案例。假设当前使用 100 块 H100 GPU 处理工作负载，那么可能只需 50 块 Blackwell GPU，因为每块 Blackwell 的速度超过 H100 的两倍（使用 FP8/FP4 时可

能更高）。因此只需购买 50 块而不是 100 块 GPU。即使每块 Blackwell 的价格高于 H100，购买数量减半也可能使总成本持平甚至更低。在功耗方面，100 块 H100 可能消耗 70 kW，而完成相同工作量的 50 块 Blackwell 可能消耗 50 kW，节能效果显著。

一年下来，这一功耗差异可节省数万美元。此外，GPU 数量减少意味着需要维护的服务器更少，服务器所需的 CPU、RAM 和网络资源开销也更低，从而进一步节省成本。综合来看，在某些情况下，升级新硬件可在 1–2 年内收回成本，尤其是在有足够工作负载让设备全天候运行时。

具体计算当然取决于价格和使用模式，但重点是：对于大规模部署，采用最新 AI 硬件的 ROI 可能非常高。除了可量化的回报外，使用一套强大系统代替许多小系统，还能简化系统架构。这种简化可通过降低功耗和网络复杂度来提高运维效率。

例如，不必因为内存限制而将模型拆分到多块旧 GPU 上，可以简化软件并降低工程复杂度。采用最新硬件还可以利用最新的软件优化，并跟上同样升级硬件的竞争对手。没有人愿意以竞争对手一半的速度训练和服务模型。升级既能提高性能，也能同时支持更大模型、更快迭代和更迅速的响应。

高效运行 NVL72 既是软件和管理挑战，也是硬件工程挑战。硬件提供了惊人的潜力，但能否充分发挥，取决于工程师是否监控性能、保持高利用率并智能调度作业。

好消息是，NVIDIA 提供了丰富的软件栈来监控和改善性能，包括驱动程序、分析器、容器运行时和集群编排工具。本书后续将介绍如何优化软件，以充分利用 GB200/GB300 NVL72 等系统。现在需要记住的是：当一个机箱提供 exaFLOPS 级性能时，也必须采用同样先进的策略，让每个 FLOP 和每个字节都发挥价值。

展望未来：NVIDIA 路线图

在本书写作时，Grace Blackwell NVL72 平台代表了 AI 硬件的前沿水平，但 NVIDIA 已在准备下一轮跃迁。简要了解 NVIDIA 未来几年的硬件路线图很有价值，因为它展现了清晰的扩展模式：NVIDIA 将继续加强性能、内存和集成度。

Blackwell Ultra 与 Grace Blackwell Ultra

NVIDIA 的 Blackwell Ultra（B300）及其对应的 Grace Blackwell Ultra 超级芯片（GB300）是 NVL72 架构的直接升级。每块 B300 GPU 的内存容量（288 GB）比 B200（180 GB）多约 50%，AI 计算性能约高 1.5 倍，并配备更大的片上加速器，专门面向注意力操作和低精度（如 NVFP4）。这意味着 B300 相比 B200 可获得 45%–50% 更高的推理吞吐量；这些为厂商特定指标，实际表现须按工作负载验证。

一个包含 72 块 GB300 GPU 的机架由 36 个 Grace Blackwell Ultra 模块组成（每个模块包含 2 块 GPU 和 1 个 CPU），拥有约 20.7 TB HBM（72 × 288 GB）和约 18 TB DDR（36 × 500 GB），合计每个 GB300 NVL72 机架约 38 TB 高速内存。GB300 NVL72 Ultra 的机架内 NVLink 和 NVSwitch 网络与 GB200 NVL72 一样使用 NVLink 5。

简而言之，GB300 是基于相同架构对 GB200 的渐进式升级，但各方面规格都更高，包括更多 SM、更大内存和更高时钟频率。

Vera Rubin 超级芯片（2026）

Vera Rubin Superchip（VR200）以那位为暗物质证据作出重要贡献的女性天文学家命名，是下一代重要架构。Vera 是基于 ARM 的 Grace CPU 后继产品，Rubin 是 Blackwell GPU 架构的后继产品。NVIDIA 在单个 VR200 模块中组合一颗 Vera CPU 与两颗 Rubin GPU，延续 Grace Blackwell（GB200/GB300）的超级芯片配置。

Vera CPU 采用 TSMC 3 nm 工艺，拥有更多 CPU 核心，并使用带宽约 1 TB/s 的 LPDDR6 内存。Rubin GPU 提供更高计算能力和更大 HBM 容量，预计每块 GPU 配备 288 GB HBM。

NVLink 预计升级到第六代 NVLink 6，使 CPU–GPU 和 GPU–GPU 链路带宽翻倍。Vera Rubin 也可能允许每机架容纳更多节点，或每个 NVLink 域包含更多机架，从而突破八机架 GB200/GB300 NVL72 集群的 576-GPU 限制。

Vera Rubin 这一代在大多数指标上又实现约 2 倍跃升，包括核心数、内存、带宽和 TFLOPS。Rubin GPU 将每个裸片的 SM 数量提升到约 200 个；第二代 FP4 甚至实验性的 2-bit 精度目前仍只是推测。

由于 Rubin 的 288 GB HBM 仍可能成为大型 AI 模型的瓶颈，NVIDIA 或许会加入第二级内存，例如 Vera CPU 使用的 256 GB LPDDR6。LPDDR 带宽低于 HBM，但容量更大且功耗更低。

如果实现，单个 GPU 模块可能拥有约 550 GB 总内存（288 GB HBM + 256 GB LPDDR），并具备缓存一致性和统一地址空间。这将进一步模糊 CPU 内存与 GPU 内存的界限。

总体而言，Vera Rubin 和 Vera Rubin Ultra 机架的性能可能达到 GB200/GB300 NVL72 的 5 倍，功耗也约为 5 倍，即每机架接近 600 kW。VR200/VR300 NVL 系统将在 Rubin GPU 上提供每 GPU 288 GB HBM 和数十 TB CPU 内存；机架内 NVLink 6 的通信开销低于 NVLink 5。

Rubin Ultra and Vera Rubin Ultra (2027)

按照这一演进模式，Rubin（R300）和 Vera Rubin 的 Ultra 版本将在原版发布一年后推出。届时 NVIDIA 可能转向四裸片 GPU 模块：将两个双裸片 Rubin 封装组合成四裸片 Rubin GPU。R300 Rubin Ultra 模块在一个封装中包含四个 GPU 裸片和 16 个 HBM 堆栈，单个模块拥有 1 TB HBM；四个裸片使核心数相比双裸片 B300 模块翻倍。

Vera Rubin NVL144 系统将在机架中包含 144 个此类裸片，即 36 个每个含四裸片的超级芯片模块。此外，Vera Rubin NVL576 配置将通过多裸片封装把完整系统的 GPU 数量提升至 4 倍。

到 2027 年，每个机架的计算性能可能达到 3–4 exaFLOPS，并拥有合计 165 TB GPU HBM（288 GB/每块 Rubin GPU × 576 块 GPU）。这些数字仍属于路线图预测，实际产品规格可能变化。

Feynman GPU（2028）与每年翻倍的路线

NVIDIA 将 Rubin 之后的一代命名为 Feynman，计划于 2028 年发布。虽然细节有限，但 Feynman GPU 可能转向更先进的 TSMC 2 nm 工艺，采用 HBM5，并在模块内集成更多 DDR 内存；裸片数量也可能从四个增加到八个。

预计到 2028 年，推理需求将主导 AI 工作负载，尤其是随着模型推理能力持续发展。与过去的非推理模型相比，推理模型在推理时需要多出数百乃至数千倍的计算量。因此，芯片设计很可能会针对大规模推理效率进行优化，包括采用更新的数值精度、增加片上内存，以及使用封装内光学链路进一步提升 NVLink 吞吐量。

NVIDIA 似乎在每一代、每一年都尽可能让某项指标翻倍。某一年翻倍内存，下一年翻倍裸片数量，再下一年翻倍互连带宽，以此类推。几年后，这种翻倍的复合效应非常巨大。NVIDIA 的激进路线可以从各代产品不断将重要指标翻倍看出：例如 Blackwell 引入双 GPU 裸片（每个模块两个裸片而非一个），NVLink 单链路双向带宽从约 900 GB/s 翻倍到约 1.8 TB/s，单 GPU 内存也从 Blackwell 的 180 GB 增加到 Blackwell Ultra 的约 288 GB。Rubin 和 Feynman 还将进一步提升计算、内存和带宽。

NVIDIA 反复谈到 AI 工厂：机架就是 AI 模型的生产线。NVIDIA 设想通过合作伙伴提供“机架即服务”，让公司租用超级计算机的一部分，而不必自行构建全部基础设施。随着前沿硬件以可直接部署的集成 Pod 形式交付，这一趋势很可能继续。每一代产品都可以通过替换新 Pod 来翻倍容量、提高性能并降低成本。

对性能工程师而言，重要的是硬件将持续解锁新的扩展规模。如今无法实现的模型，几年后可能会变得稀松平常。这也意味着我们必须持续调整软件，以利用新的精度格式、更大的内存池和更先进的互连。这是令人兴奋的时代，因为前沿模型的发展与这些硬件创新密切相关。

关键要点

以下创新共同使 NVIDIA 硬件能够以前所未有的速度、效率和可扩展性处理超大规模 AI 模型：

集成式超级芯片架构

> NVIDIA 将基于 ARM 的 CPU（Grace）与 GPU（Hopper/Blackwell）融合到单个超级芯片中，创建统一内存空间。这种设计无需在 CPU 与 GPU 之间手动传输数据，从而简化数据管理。

统一内存架构

> 统一内存架构和一致性互连降低了编程复杂度。开发者无需操心显式数据移动即可编写代码，从而加快开发并专注于改进 AI 算法。

超高速互连

> 通过使用 NVLink（包括 NVLink-C2C 和 NVLink 5）及 NVSwitch，系统实现极高的机架内带宽和极低延迟。这使 GPU 几乎可以像一个大型处理器的组成部分那样通信，对扩展 AI 训练和推理至关重要。

高密度、超大规模系统（NVL72）

NVL72 机架在紧凑系统中集成 72 块 GPU。该整合设计将高计算性能与巨大的统一内存池结合起来，支持大规模模型，并实现传统架构难以完成的任务。

先进的冷却与功耗管理

NVL72 依靠复杂的液冷和可靠的配电系统运行，每个机架功耗约 130 kW（130 kW = 18 个节点 × 每节点 6 kW + 约 20 kW 的 NVSwitch/冷却/其他开销）。如此规模的冷却和供电对于管理高密度、高性能组件并确保可靠运行至关重要。

显著的性能与效率提升

与 Hopper H100 等前代产品相比，Blackwell GPU 的计算能力和内存带宽提升约 2–2.5 倍。这会显著提高训练和推理速度——在使用 Blackwell FP4 Tensor Core 和 Transformer Engine 的某些情况下，推理速度最高可提升 30 倍——并可能通过减少 GPU 数量节省成本。

现代软件栈支持

NVIDIA 的软件和框架持续演进，以充分利用最新硬件并支持最新的协同设计系统优化，包括统一内存管理以及原生 FP8/FP4 精度支持。因此，工程师只需进行少量代码修改，就能利用系统的全部性能。

面向未来的路线图

NVIDIA 的发展路线图（包括 Blackwell Ultra、Vera Rubin、Vera Rubin Ultra 和 Feynman）承诺持续让计算吞吐量、内存带宽等关键参数翻倍。这一路线旨在支持未来越来越大的 AI 模型和越来越复杂的工作负载。

结论

NVIDIA NVL72 系统凭借 Grace Blackwell Superchip、NVLink 网络和先进冷却，代表了 AI 硬件设计的前沿。本章展示了每个组件如何通过协同设计服务于加速 AI 工作负载这一核心目标。CPU 与 GPU 融合为一个单元，消除数据传输瓶颈并提供巨大的统一内存。

数十块 GPU 通过超高速网络连接起来，表现得像一块通信延迟极低的巨型 GPU。内存子系统也得到扩展和加速，以满足 GPU 核心对数据的巨大需求。为实现这种计算密度，供电和热管理同样被提升到新的水平。

结果是：单个机架即可提供过去只有多机架超级计算机才能达到的性能。NVIDIA 对整个计算栈——芯片、板卡、网络和冷却——进行端到端优化，使其能够在超大规模上训练和服务大型 AI 模型。

但这类硬件创新也带来挑战：需要专用设施、周密的供电和冷却规划，以及复杂的软件才能充分利用它们。回报同样巨大。研究人员如今可以直接试验前所未有规模和复杂度的模型，而不必等待数周或数月才能得到结果。过去基础设施可能需要一个月训练的模型，在 NVL72 上可能几天就能完成。原本几乎无法交互（每次查询需要数秒）的推理任务，如今

可以实时完成（仅需毫秒）。这为过去难以实现的应用打开了大门，例如数万亿参数的交互式 AI 助手和智能体。

NVIDIA 快速推进的路线图表明，这还只是开始。Grace Blackwell 架构将演进为 Vera Rubin、Feynman 及其后续架构。正如 NVIDIA CEO Jensen Huang所说：“AI 正以光速发展，各家公司都在竞相建设能够扩展的 AI 工厂，以满足推理 AI 和推理时扩展的处理需求。”

NVL72 及其后继产品是 AI 工厂的核心，是处理海量数据、产出强大 AI 能力的重型设备。作为性能工程师，我们站在这些硬件创新的肩膀上。它们提供了巨大的原始能力，而我们的职责是开发软件和算法，充分发挥硬件潜力。

下一章将从硬件转向软件。我们将探讨如何优化 NVL72 等系统上的操作系统、驱动程序和库，确保这些强大硬件不会被闲置。后续章节还会讨论与软件架构相配合的内存管理，以及分布式训练/推理算法。

本书主题是协同设计（codesign）。正如硬件是为 AI 协同设计的，我们的软件和方法也必须协同设计以利用硬件。现在已经清楚硬件基础，我们可以深入研究提升 AI 系统性能的软件策略。AI 超级计算时代已经到来，充分发挥其能力将是一段令人振奋的旅程。让我们开始吧！

第 3 章。 面向 GPU 环境的 OS、Docker 与 Kubernetes 调优

即使 GPU 代码和库已经高度优化，系统级瓶颈仍可能限制大规模 AI 训练的性能。GPU 的有效性能取决于周边环境能否及时供给数据和指令。本章介绍如何调优操作系统与容器运行时，使 GPU 在实际工作负载和系统约束下发挥更高性能。

我们先介绍基础 GPU 软件栈，再讨论 NUMA 亲和性和大页等关键 CPU 与内存优化，使数据能够更高效地从存储经 CPU 流向 GPU。同时还会讨论 GPU 持久化模式、多进程服务（MPS）和多实例 GPU（MIG）等驱动与运行时功能。这些机制用途各不相同：它们可能减少初始化或共享开销、改善资源隔离与并发，但不能单独保证最大 GPU 利用率。

借助 NVIDIA Container Toolkit、NVIDIA Container Runtime、Kubernetes Topology Manager 和 Kubernetes GPU Operator 等组件，可以为 GPU 环境构建较一致的软件栈，并在单节点与多节点环境中支持资源发现、分配和工作负载调度。是否真正提高 GPU 有效利用率，仍取决于拓扑、调度策略、数据路径和应用本身，并应通过端到端测量验证。

在此过程中，你将理解这些优化为何重要。它们的本质是降低延迟、最大化吞吐量，并确保 GPU 始终获得充足数据、以峰值性能运行。结果是一个稳健且可扩展的系统，能够为训练和推理工作负载带来显著性能提升和较高的有效吞吐量（goodput）比例。

操作系统

操作系统（OS）是所有程序运行的基础。GPU 服务器通常运行 Ubuntu Server LTS 或 Red Hat 等 Linux 发行版，并使用支持最新 GPU 硬件的更新内核。NVIDIA 驱动会安装内核模块，为每块 GPU 创建 `/dev/nvidia0`、`/dev/nvidia1` 和 `/dev/nvidia2` 等设备文件。驱动还会创建用于驱动控制操作的 `/dev/nvidiactl`、用于统一虚拟内存的 `/dev/nvidia-uvm`，以及用于模式设置和缓冲区管理的 `/dev/nvidia-modeset`。

OS 负责 CPU 调度、内存、网络和存储，这些都应针对高 GPU 吞吐量进行调优。因此，应配置 OS，避免干扰 GPU 任务。例如，GPU 节点应禁用交换，或将 `vm.swappiness` 设置为 0，避免 OS 发起的内存交换干扰 GPU 工作负载。性能工程师的一项职责，就是调整这些 OS 设置，为 GPU 的最大性能做好准备。

面向 GPU 的服务器可能需要运行额外的守护进程或后台进程，例如 NVIDIA Persistence Daemon，即使没有 GPU 作业运行，也能保持 GPU 驱动和硬件上下文处于加载、就绪状态。此外，Fabric Manager 负责管理 GPU 互连拓扑，NVIDIA Data Center GPU Manager（DCGM）则监控 GPU 系统健康指标。

NVIDIA 软件栈

运行多 petaFLOPS GPU 集群，不仅仅是编写 PyTorch、TensorFlow 或 JAX 等高级代码。GPU 操作由完整的软件栈支撑，每一层都可能影响性能。图 3-1 展示了开发和产品化现代 LLM 工作负载时常用的一组框架、库、编译器、运行时和工具，包括 PyTorch、cuDNN、cuBLAS、CUTLASS、CUDA C++、`nvcc` 以及 CUDA Runtime API（例如 CUDA 工具和驱动等）。

NVIDIA GPU 和 CUDA 生态系统还包含 Python 库，允许使用 OpenAI 的Triton领域特定语言（DSL）、NVIDIA 的Warp框架，以及 NVIDIA 的CUDA Python、cuTile 和CUTLASS库在 Python 中创建 CUDA 内核。

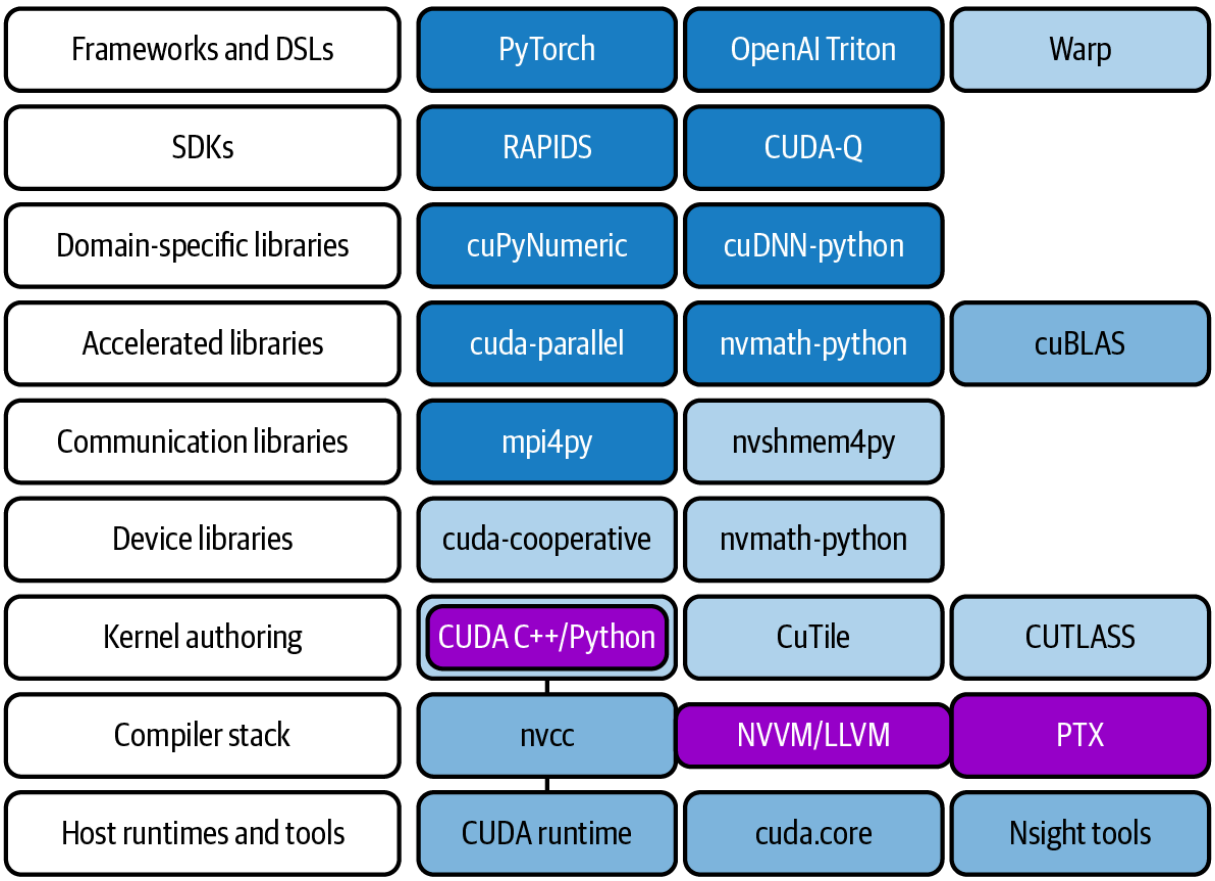


图 3-1：开发和产品化现代 LLM 工作负载时常用的框架、库、编译器、运行时和工具

GPU 驱动

软件栈的底层是 NVIDIA GPU 驱动，它在 Linux OS 与 GPU 硬件之间提供接口。驱动负责低层 GPU 操作，包括设备内存分配、GPU 核心上的任务调度，以及为多租户使用划分 GPU。

GPU 驱动会启用 GPU 功能，并持续为硬件提供工作。保持 NVIDIA 驱动为最新版本非常重要。新的驱动版本通常会带来性能改进，并支持最新 GPU 架构和 CUDA 功能。

`nvidia-smi` 等工具随驱动提供，可用于监控温度、测量利用率、查询纠错码（ECC）内存状态，以及启用持久化模式等不同 GPU 模式。

CUDA 工具包与运行时

驱动之上是 CUDA Runtime 和库组成的 CUDA Toolkit。工具包包含用于编译 CUDA C++ 内核的 CUDA 编译器 `nvcc`。编译后，CUDA 程序会链接 CUDA 运行时（`cudart`）。CUDA 运行时直接与 NVIDIA 驱动通信，以启动工作并在 GPU 上分配内存。

此外，CUDA Toolkit 还提供许多优化库：cuDNN 用于神经网络原语，cuBLAS 用于线性代数，NCCL 用于多 GPU 通信等。因此，必须使用支持 GPU 计算能力（CC）的最新 CUDA Toolkit 版本，因为最新工具包包含针对 GPU 的最新编译器优化和专用库。后续章节将更详细介绍 CUDA 编译器、编程模型以及 CUDA（和 PyTorch）优化。

CUDA 跨 GPU 硬件代际的向前与向后兼容性

NVIDIA GPU 编程模型的一个重要特性是跨硬件代际的兼容性。编译 CUDA 代码时，生成的二进制文件既包含虚拟或中间 PTX 代码，也包含物理设备代码（例如 ARM、x86 和 GPU 指令），如图 3-2所示。

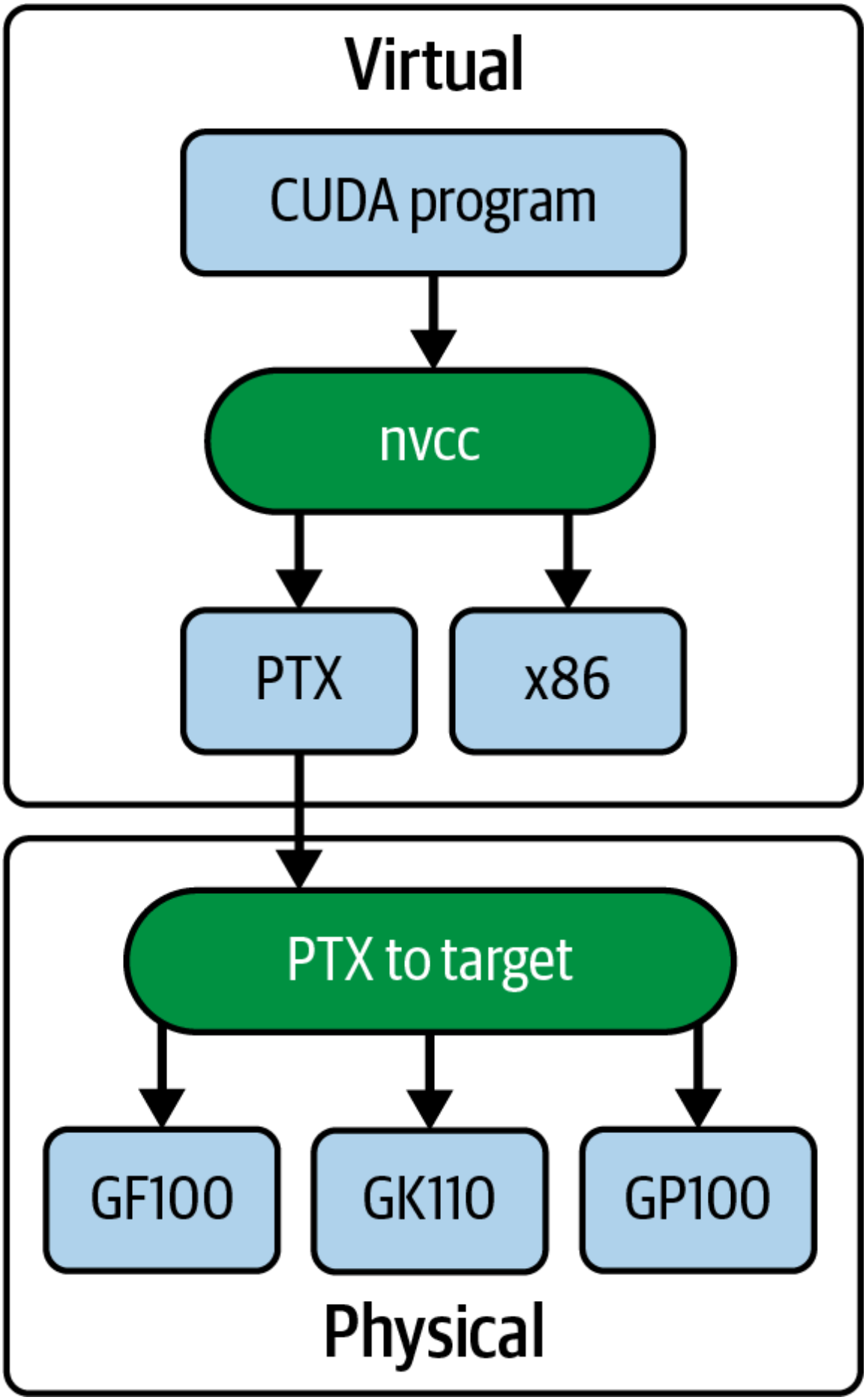

Virtual
CUDA program
nvcc
PTX
x86
PTX to target
GF100
GK110
GP100
Physical

图 3-2：使用 nvcc 将 CUDA 程序编译为 PTX，并最终生成目标 GPU 的底层指令

这使较新的 GPU 可以即时（JIT）编译 PTX，从而让程序运行在未来架构上；同时，较新的 GPU 也可以执行面向旧架构的旧二进制代码。这种兼容性通过 NVIDIA 的 *fatbinary* 模型实现：其中包含面向未来的 PTX，以及面向已知架构的 *CUBIN*（架构专用 CUDA 设备代码二进制文件）。

CUBIN 是 `nvcc` 使用 `-cubin` 选项为特定 SM 目标生成的架构专用二进制文件。它包含针对特定 NVIDIA 架构编译出的 GPU 流式汇编（SASS）指令，并被打包进 fatbinary（也称 fatbin），供 CUDA 驱动在运行时加载。PTX 是用于在新架构上通过 JIT 编译实现向前兼容的虚拟 ISA 表示；CUBIN 二进制文件则可在已知 GPU 架构上直接执行。实际应用中，应在 fatbinary 中同时包含 PTX 和相关原生 CUBIN：PTX 提供向前兼容性，CUBIN 则能在已知 SM 目标上快速原生加载。

C++ 与 Python CUDA 库

虽然大多数 CUDA Toolkit 库使用 C++，但 NVIDIA 面向 Python 的当前选项包括 CUDA Python（例如低层驱动和运行时访问）；用于数组编程的 cuPyNumeric、CuTe DSL、cuTile 和 CuPy；以及用于在 Python 中编写 GPU 内核的 NVIDIA Warp。CUTLASS 是 C++ 模板库，供 cuBLAS 等库在底层使用，而不是 Python 库。

虽然 CUDA Toolkit 库大多基于 C++，但 NVIDIA 正推出越来越多以“Cu”开头、构建于 C++ 工具包之上的 Python 库。例如，cuTile 和 cuPyNumeric 是 2025 年初推出的 Python 库，旨在降低 Python 开发者使用 CUDA 为 NVIDIA GPU 构建应用的门槛。

cuTile 是一种 Python 库，旨在通过将大型矩阵拆分为称为 *tile* 的较小子矩阵，简化 GPU 上的大矩阵处理。它提供基于 tile 的高层抽象，使分块计算、内存访问优化和 GPU 内核调度更加容易。

通过将大型矩阵划分为 tile，cuTile 帮助开发者充分利用 GPU 并行性，而无需手动管理底层细节。对于需要密集矩阵计算的应用，这种方法可以改善缓存利用率并提升整体性能。

cuPyNumeric 旨在为常用 Python `numpy` 库提供较高兼容度的 GPU 加速替代方案（`import cupynumeric as np`）。它实现了 NumPy API 的较大子集，使部分应用只需少量修改即可迁移。底层运行时可利用 GPU 并行执行操作，从而加速适合并行化的大规模数值计算、矩阵运算和数据分析；但它并非覆盖 NumPy 的所有语义和工作负载，性能收益也取决于数据规模、操作支持程度及数据移动开销。

通过将工作卸载到 GPU，cuPyNumeric 可加速计算并提高处理海量数据集的应用效率。它旨在降低 Python 开发者使用 GPU 的门槛，无需学习全新接口，因此成为高性能计算中强大的 NumPy 即插即用替代品。

另一个值得关注的 Python 编程模型是开源 Triton 语言与编译器。Triton 是一种 Python DSL，允许开发者用 Python 风格的代码编写自定义 GPU 内核，由编译器生成底层 GPU 代码。它不是 NVIDIA CUDA 库，但可作为 CUDA 生态中的内核开发工具，用于实现和调优高性能算子。

后续章节将介绍 Triton 及其优化；目前只需知道，在许多情况下 Triton 减少了手写 CUDA C++ 的需要。它还集成到 PyTorch 编译器后端中，可自动优化和融合 GPU 操作以提高性能。下面转向 PyTorch。

PyTorch 与高级 AI 框架

构建于 CUDA 之上的常用 Python 框架包括 PyTorch、TensorFlow、JAX 和 Keras。这些框架利用 NVIDIA GPU 的能力，为深度学习提供高层接口。本书主要关注 PyTorch 的编译和图优化功能，包括 `torch.compile` 栈。

PyTorch 编译器栈由 TorchDynamo、AOT Autograd 以及 TorchInductor 或 Accelerated Linear Algebra（XLA）等后端组成，可自动捕获并优化模型。TorchInductor 是常用后端之一，在 NVIDIA GPU 上通常使用 Triton 生成内核。Triton 可进行内核融合，并针对特定 GPU 和系统环境执行自动调优，详见第 14 章。

使用 GPU 对 PyTorch 张量执行操作时，张量看似通过一次 Python 调用就从 CPU 移动到了 GPU。但这次调用实际上会被转换为一系列 CUDA Runtime 调用，并使用各种 CUDA 库，如图 3-3所示。

图 3-3：从 PyTorch 代码到 GPU 设备的执行流程

例如，执行矩阵乘法时，PyTorch 会把相应操作分派给 cuBLAS 等库。cuBLAS 是 CUDA Toolkit 的组成部分，针对 GPU 执行进行了优化。PyTorch 的前向和反向传播最终由底层 CUDA 函数、生成的内核及优化库共同执行；具体路径取决于算子、数据类型、设备和编译模式。

简而言之，PyTorch 隐藏了直接使用 CUDA 编程的许多复杂细节，让开发者用较直观的 Python 接口调用优化后的 CUDA 算子。不过，高层抽象并不保证每个模型都获得最佳性能，仍需分析图中断、内核选择、形状和数据移动。本书将在第 4 章和第 5 章讨论 CUDA 相关优化，并在第 9 章讨论 PyTorch 优化。

所有这些组件——OS、GPU 驱动、CUDA Toolkit、CUDA 库和 PyTorch——必须协同工作，才能创建理想的 GPU 开发环境。研究人员提交训练作业后，调度器预留节点，OS 通过 NVIDIA 驱动提供 GPU 设备和内存分配，容器则提供正确的软件环境（包括经过优化且了解硬件特性的 CUDA 库）。用户代码（例如 PyTorch、TensorFlow、JAX）使用这些 CUDA 库，最终与驱动和硬件通信。

本章介绍的优化旨在让软件栈的每一层都尽可能高效。它们将帮助 GPU 持续执行有用的训练和推理工作，而不是等待 CPU、内存或磁盘 I/O，或等待其他 GPU 完成同步。

经过良好调优的系统可以确保跨数十块 GPU 拆分的模型不会受 I/O 或 OS 开销限制。人们往往更关注模型优化而忽视系统级调优，但系统级优化也能带来可观性能提升。在某些情况下，只需对 OS 配置进行小幅调整，就能获得两位数百分比的性能提升。在大型 AI 项目规模下，这可以节省数万甚至数十万美元的计算时间成本。

为 GPU 环境配置 CPU 与操作系统

GPU 无法达到满利用率的最常见原因之一，是 CPU 没有持续为 GPU 提供有用工作。在典型训练循环中，CPU 负责准备下一批数据，包括从磁盘加载、分词和转换数据等。此外，CPU 还负责分发 GPU 内核，并协调线程和进程。

如果这些主机侧任务速度缓慢，或 OS 调度不当，昂贵的 GPU 就可能处于空闲状态，等待下一项任务或下一批数据。为避免这种情况，需要优化 CPU 和 OS 处理 GPU 工作负载的方式。

这些优化包括设置 CPU 亲和性，避免跨 NUMA 节点流量，让正确的核心处理正确的数据；采用内存分配策略避免 NUMA 惩罚；以及进行 OS 层面的修改，消除不必要的延迟。这样 GPU 就不会因缺少数据而挨饿。其中一项措施是将后台守护进程和 OS 任务隔离到专用核心上，远离为 GPU 提供数据的核心，下面将讨论这一点。

NUMA 感知与 CPU 绑核

现代服务器 CPU 往往拥有数十个核心，并划分为多个 NUMA 节点。*NUMA 节点*是由物理位置相近的 CPU、GPU、网络接口控制器（NIC）和内存组成的逻辑分组。了解系统的 NUMA 架构对于性能调优十分重要；访问本地 NUMA 节点内的资源通常比访问其他节点的资源更快。

例如，NUMA 节点 0 上某 CPU 运行的进程若要访问 NUMA 节点 1 上的 GPU，数据就需要经过节点间互连，因而产生更高延迟。在某些平台和访问模式下，跨节点内存访问延迟可能接近本地访问的两倍；具体差异取决于处理器拓扑、互连和负载。

> 在 GH200 和 GB200 等基于 Grace 的超级芯片中，CPU 与 GPU 通过 NVLink-C2C 连接。Grace 与其配对加速器之间可提供最高约 900 GB/s 的一致性互连带宽。Linux 仍将 CPU DRAM 作为 CPU NUMA 内存、将 GPU HBM 作为设备内存。因此，即使一致性机制降低了软件开销，仍应把 CPU 线程绑定到本地 Grace CPU，并保持数据局部性。这里的 900 GB/s 是特定平台的双向聚合互连带宽口径，并不等同于任意应用都能达到的有效传输吞吐。

在许多双路系统中，远端内存访问延迟可能明显高于本地访问。在一项实验中，本地 NUMA 节点内存访问延迟约为 80 ns，而远端（跨节点）访问约为 139 ns，增幅约 74%。这些数值是特定测试平台和方法下的结果，不应直接视为所有双路系统的固定比例。

把进程绑定到与其 GPU 位于同一 NUMA 节点的 CPU 上，可以减少这类额外开销。例如，可使用 `numactl --cpunodebind=<node> --membind=<node>`，将 CPU 线程和内存分配约束到 GPU 所在的本地 NUMA 节点。核心原则是让为某块 GPU 服务的 CPU 执行和内存访问尽量保持在本地。

> Linux 虽然提供自动 NUMA balancing，但它不一定满足性能敏感型 AI 工作负载对可预测性的要求。调度器可能迁移线程，自动 NUMA balancing 也可能迁移内存页；若线程、内存与 GPU 的位置不匹配，就会产生远端访问延迟。因此，可以根据实测结果，使用 `numactl`、`taskset`、cpuset cgroup 等机制显式约束进程和内存位置。

要显式指定 NUMA 亲和性，需要把进程或线程“固定”到与 GPU 同属一个 NUMA 节点的特定 CPU 上。这类 CPU 亲和性设置称为 *CPU 绑核*。假设一个节点中有 8 块 GPU，其中 4 块连接到 NUMA 节点 0，另外 4 块连接到 NUMA 节点 1。

如果启动 8 个训练进程、每块 GPU 对应一个进程，就应把每个进程绑定到与该 GPU 同属一个 NUMA 节点的 CPU 核心或核心集合。本例中，GPU 0–3 连接到 NUMA 节点 0，GPU 4–7 连接到 NUMA 节点 1，如图 3-4所示。

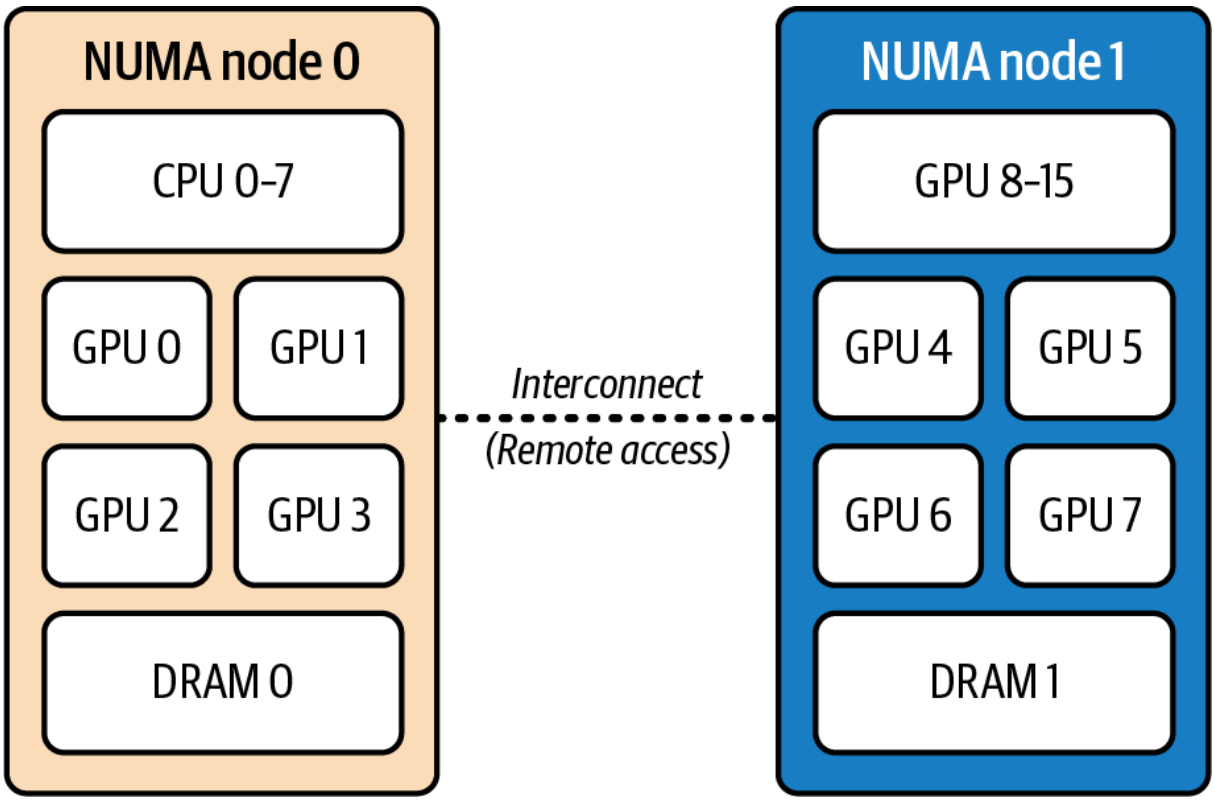


图 3-4。一个节点中的 8 块 GPU：4 块连接到 NUMA 节点 0，另外 4 块连接到 NUMA 节点 1

这样，当 CPU 进程向 GPU 4 供给数据时，应在 NUMA 节点 1 所连接的 CPU 上运行，因为 GPU 4 也连接到 NUMA 节点 1。Linux 提供了相应工具，例如 `numactl --cpunodebind=<node> --membind=<node>` 可在指定 NUMA 节点的 CPU 与内存策略下启动进程。

也可以用 `taskset` 把进程绑定到特定 CPU ID。下面使用 `numactl`，将 `train.py` 脚本约束在与 GPU 4 同属 NUMA 节点 1 的 CPU 和内存上：

```
numactl --cpunodebind=1 --membind=1 \
    python train.py --gpu 4
```

上例假定已知 NUMA 节点 ID，且脚本只使用一块 GPU。如果要为多块 GPU 自动选择本地 NUMA 节点，处理会更复杂。下面的示例尝试使用 `nvidia-smi topo` 查询拓扑，再按 GPU 所在的本地 NUMA 节点启动脚本：

```
#!/bin/bash
for GPU in 0 1 2 3; do
  # Query NUMA node for this GPU
  NODE=$(nvidia-smi topo -m -i $GPU \
         | awk '/NUMA Affinity/ {print $NF}')

  # Launch the training process pinned to that NUMA node
  numactl --cpunodebind=$NODE --membind=$NODE \
```

```
    bash -c "CUDA_VISIBLE_DEVICES=$GPU python train.py --gpu $GPU"
done
```

这里用 `topo -m` 查看 CPU 与 NUMA 亲和性，然后从输出中提取单个 NUMA 节点 ID，最后把 `--cpunodebind` 和 `--membind` 都设置为该节点，使进程线程和后续内存分配尽量位于 GPU 的本地 NUMA 域。需要注意，`nvidia-smi topo` 的选项和输出格式随驱动版本、GPU 类型及虚拟化环境而异，上述 shell 解析只是示意；部署前应在目标系统验证，生产工具更适合通过 NVML 或 sysfs 获取结构化拓扑信息。

许多深度学习框架也允许以编程方式设置线程亲和性。例如，PyTorch DataLoader 提供 `worker_init_fn`，可在每个 worker 进程初始化时设置其 CPU 亲和性，如下所示：

```
import os
import re
import glob
import subprocess
import psutil
import ctypes
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, Dataset
from functools import partial

# Optional: NVML is preferred for GPU↔NUMA mapping
try:
    import pynvml as nvml  # pip install nvidia-ml-py3
    _HAS_NVML = True
except Exception:
    _HAS_NVML = False

# --- libnuma for memory binding
_libnuma = ctypes.CDLL("libnuma.so")
if _libnuma.numa_available() < 0:
    raise RuntimeError("NUMA not available on this system")
_libnuma.numa_run_on_node.argtypes = [ctypes.c_int]
_libnuma.numa_set_preferred.argtypes = [ctypes.c_int]

def parse_physical_cpu_list(phys_str: str):
    """Parse '0-3,8-11' -> [0,1,2,3,8,9,10,11]."""
    cpus = []
    if not phys_str:
        return cpus
    for part in phys_str.split(','):
        part = part.strip()
        if not part:
            continue
        if '-' in part:
            start, end = map(int, part.split('-'))
            cpus.extend(range(start, end + 1))
        else:
            cpus.append(int(part))
    return cpus

def get_numa_cpus_for_node(node: int):
    """Read /sys/devices/system/node/node{node}/cpulist."""
    path = f"/sys/devices/system/node/node{node}/cpulist"
    with open(path, "r") as f:
        return parse_physical_cpu_list(f.read().strip())
```

```
def get_numa_cpus_and_memory():
    """Return (current_cpu_mask, preferred_node) from numactl --show."""
    out = subprocess.run(["numactl", "--show"],
        capture_output=True, text=True).stdout
    phys = re.search(r"physcpubind:\s*([\d,\-\s]+)", out).group(1)
    cpus = parse_physical_cpu_list(phys)
    node = int(re.search(r"preferred node:\s*(-?\d+)", out).group(1))
    return cpus, node

def get_gpu_numa_node(device: int) -> int:
    """
    Determine NUMA node for a GPU (prefer NVML; fall back to sysfs;
    final fallback to current preferred node).
    """
    # NVML path (preferred)
    if _HAS_NVML:
        try:
            nvml.nvmlInit()
            props = torch.cuda.get_device_properties(device)
            pci = props.pci_bus_id  # '0000:03:00.0' or '00000000:03:00.0'
            # Normalize to 8-hex-digit domain if needed for NVML
            try:
                domain, bus, devfn = pci.split(':')
                if len(domain) < 8:
                    domain = domain.rjust(8, '0')
                pci8 = f"{domain}:{bus}:{devfn}"
            except ValueError:
                pci8 = pci
            try:
                handle = nvml.nvmlDeviceGetHandleByPciBusId_v2(pci8)
            except AttributeError:
                handle = nvml.nvmlDeviceGetHandleByPciBusId(pci8)

            # Direct NUMA ID if driver exposes it
            try:
                numa_id = nvml.nvmlDeviceGetNUMANodeId(handle)
                if isinstance(numa_id, int) and numa_id >= 0:
                    return numa_id
            except Exception:
                pass

            # Derive from NVML CPU affinity
            cpu_count = psutil.cpu_count(logical=True)
            elems = (cpu_count + 63) // 64
            mask = nvml.nvmlDeviceGetCpuAffinity(handle, elems)
            cpus = []
            for i, m in enumerate(mask):
                m = int(m)
                for b in range(64):
                    if m & (1 << b):
                        cpu_id = i * 64 + b
                        if cpu_id < cpu_count:
                            cpus.append(cpu_id)
            # Build CPU→NUMA map from sysfs and choose majority node
            cpu2node = {}
            for node_path in sorted(glob.glob("/sys/devices/system/node/node*")):
                node_id = int(os.path.basename(node_path).replace("node", ""))
                with open(os.path.join(node_path, "cpulist"), "r") as f:
                    for c in parse_physical_cpu_list(f.read().strip()):
                        cpu2node[c] = node_id
            counts = {}
            for c in cpus:
                n = cpu2node.get(c)
                if n is not None:
```

```
                    counts[n] = counts.get(n, 0) + 1
            if counts:
                return max(counts.items(), key=lambda kv: kv[1])[0]
        except Exception:
            pass

    # sysfs fallback
    try:
        props = torch.cuda.get_device_properties(device)
        pci = props.pci_bus_id
        sysfs_path = f"/sys/bus/pci/devices/{pci}/numa_node"
        with open(sysfs_path, "r") as f:
            val = int(f.read().strip())
            return val if val >= 0 else 0
    except Exception:
        pass

    # last resort: current preferred node
    _, node = get_numa_cpus_and_memory()
    return node if node >= 0 else 0

def set_numa_affinity(node: int):
    """Bind current process to CPUs and memory of the given NUMA node."""
    cpus = get_numa_cpus_for_node(node)  # IMPORTANT: CPUs of target node
    psutil.Process(os.getpid()).cpu_affinity(cpus)
    _libnuma.numa_run_on_node(node)
    _libnuma.numa_set_preferred(node)
    print(f"PID={os.getpid()} bound to NUMA node {node} (CPUs={cpus})")
    return cpus

def _worker_init_fn(worker_id: int, node: int, cpus: list):
    """Reapply binding in each DataLoader worker (no CUDA calls here)."""
    psutil.Process(os.getpid()).cpu_affinity(cpus)
    _libnuma.numa_run_on_node(node)
    _libnuma.numa_set_preferred(node)
    print(f"Worker {worker_id} (PID={os.getpid()}) bound to NUMA node {node}")

 # ----- Example usage below -----
class MyDataset(Dataset):
    def __len__(self): return 1024
    def __getitem__(self, idx): return torch.randn(224*224*3, device='cpu')

def main():
    # DDP setup
    dist.init_process_group(backend="nccl", init_method="env://")
    device = torch.cuda.current_device()

    # Determine GPU's NUMA node and bind this process
    gpu_node = get_gpu_numa_node(device)
    cpus = set_numa_affinity(gpu_node)

    # Build dataloader with closure-based worker_init_fn
    dataset = MyDataset()
    init_fn = partial(_worker_init_fn, node=gpu_node, cpus=cpus)
    dataloader = DataLoader(
        dataset,
        batch_size=32,
        num_workers=4,
        pin_memory=True,
        persistent_workers=True,  # reduces worker respawn churn
        worker_init_fn=init_fn,
        prefetch_factor=2,
    )
```

```
        # Model and DDP
        model = torch.nn.Linear(224*224*3, 10, bias=True).to("cuda")
        ddp_model = DDP(model, device_ids=[device], static_graph=True)

        for batch in dataloader:
            batch = batch.to("cuda", non_blocking=True)
            out = ddp_model(batch)
            # ... loss, backward, optimizer ...

if __name__ == "__main__":
    main()
```

该脚本把主训练进程和各个 DataLoader worker 约束到 GPU 的本地 NUMA 节点，以减少跨 NUMA 访问。在 DataLoader 中，通过闭包形式的 `worker_init_fn`，在每个 worker 内重新应用预先计算的 NUMA 绑定；worker 中不调用任何 CUDA API。

启动时，进程使用 NVML 将当前 GPU 映射到其 NUMA 节点和 CPU 亲和性掩码。若所用 NVML 版本提供相应接口，可直接读取节点；否则可通过 GPU 的 CPU 亲和性掩码（`nvmlDeviceGetCpuAffinity`）推导。若 NVML 不可用或无法给出节点，则回退到内核 sysfs 条目 `/sys/bus/pci/devices/<PCI_ID>/numa_node`；最后才采用进程当前允许或首选的节点。不同 NVML 版本及语言绑定暴露的 API 名称并不完全一致，使用前应核对实际接口。

随后，从 `/sys/devices/system/node/node<N>/cpulist` 读取该节点的 CPU 列表，并使用 `psutil` 设置 CPU 亲和性。脚本还通过 `libnuma`（`numa_run_on_node` 与 `numa_set_preferred`）影响后续内存分配。严格来说，`numa_set_preferred` 设置的是“优先”策略，目标节点内存不足时允许回退；若需要强制约束，应使用适当的 bind 策略，并处理分配失败风险。

Linux 的 CPU 亲和性和 NUMA 内存策略通常会由子进程继承，并在 `exec` 后保留；但框架、启动器、容器配置或 worker 初始化代码可能再次修改这些设置。因此，对框架管理的 worker 显式检查并按需重新应用绑定，可以避免把继承行为当作未经验证的假设。

可评估设置 `pin_memory=True`，并在 H2D 复制中使用 `non_blocking=True`，为页锁定主机缓冲区和异步复制创造条件。页锁定操作本身并不保证物理页位于期望的 NUMA 节点，仍应确保首次分配和触页发生在正确策略下。可使用 `persistent_workers=True` 避免每个 epoch 重建 worker。不要在 `worker_init_fn` 中调用 `torch.cuda.*`；可通过闭包或环境变量传入 GPU 索引。

理想情况下，数据准备和批次加载会主要使用本地内存，从而减少 GPU 等待远端 NUMA 访问的时间。不过，这段代码不能保证所有分配都在本地，也不能保证 GPU 始终繁忙；页迁移、共享内存、I/O、框架线程池和容器 cpuset 都会影响结果。应在安装了 `libnuma` 和 `numactl` 的目标 Linux 系统上检查实际亲和性，并通过性能计数器和端到端基准验证收益。

默认情况下，`numactl` 为目标进程设置 CPU 亲和性和 NUMA 内存策略；这些属性通常由 `fork` 创建的子进程继承，并跨 `exec` 保留。Python 的 `spawn` 会启动新解释器，但操作系统层面的亲和性与内存策略通常仍来自其父进程。实际风险主要是框架、运行时或管理员配置随后覆盖策略，因此使用框架管理的 worker 时，应在 worker 内验证有效 CPU 集合和 NUMA 策略，必要时再显式设置。

在 Grace Blackwell（以及后续同类平台）这样的超级芯片架构中，CPU 与 GPU 通过 NVLink-C2C 提供一致性访问。不过，Linux 和 CUDA 软件栈仍区分 CPU DRAM 与 GPU HBM 等不同内存池；把 CPU 线程放在本地 CPU NUMA 节点上通常仍有助于保持局部性。

绑核可以减少由线程跨 NUMA 节点迁移引起的调度变化。例如，可避免 GPU 数据加载线程在训练或推理过程中被迁移到远端节点的核心。消除跨 NUMA 流量和不必要的核心迁移有时能改善训练吞吐并降低性能抖动，但原文所称 5%–10% 并非通用结果；收益取决于瓶颈位置、拓扑和负载，应通过 A/B 基准测量。

许多高性能 AI 系统会评估 CPU 同时多线程（SMT，Intel 平台常称为超线程）；有时为获得更可预测的单核性能而禁用 SMT，但效果取决于工作负载。系统也可为操作系统后台任务预留少量核心，例如结合 `isolcpus`、cpuset、IRQ 亲和性和服务管理配置。Kubernetes 可通过 CPU Manager 的 static policy、reserved CPUs 等节点级设置隔离系统守护进程与工作负载 CPU。单独设置 `isolcpus` 并不能保证其余核心只执行训练或推理线程，仍需处理内核线程、中断及共享资源。

对于 NVIDIA Grace Blackwell 这类集成式 CPU-GPU 超级芯片，NVLink-C2C 提供一致的共享虚拟地址访问，而 CPU DRAM 与 GPU HBM 仍是不同的物理内存池。与传统 PCIe 分离式系统相比，这可降低一部分 CPU-GPU 数据移动的软件与互连开销，但不会自动消除所有 NUMA、放置或带宽瓶颈。

NVIDIA 通过 Grace Blackwell 等超级芯片把 CPU 与 GPU 紧密连接，以缓解 CPU-GPU 通路瓶颈。NVLink-C2C 为两个处理器提供一致性内存访问，特定平台标称双向聚合带宽最高约 900 GB/s，从而减少部分显式数据搬运和同步开销；但这并不意味着 CPU DRAM 与 GPU HBM 合并为单一等速内存，也不代表应用可持续获得该峰值带宽。此类软硬件协同设计预计仍会是系统优化的重要方向。

即便采用紧耦合 CPU-GPU 超级芯片，仍需正确配置硬件和软件，并通过测量优化整个技术栈。目标是减少不必要的数据处理延迟，使 GPU 在实际瓶颈允许的范围内保持较高有效利用率。后续小节将讨论大页、预取和锁页内存；这些选项都应按工作负载验证，而不应默认全部启用。

NUMA 友好的内存分配与锁页内存

默认情况下，Linux 通常采用 first-touch 策略：物理页在首次触及时从执行该触页操作的 CPU 所在 NUMA 节点分配。因此，把进程固定到 NUMA 节点 0，并在该节点上首次触页，通常会使内存来自节点 0 的本地 RAM。但若调度器迁移线程，或内存在绑核前已由其他节点触页，节点 0 上运行的进程仍可能访问节点 1 的内存，从而削弱 CPU 绑核带来的局部性收益。

为避免这种情况，可使用 `numactl --membind`，将内存分配约束到指定 NUMA 节点；程序也可通过 NUMA API 设置策略。一般原则是让内存靠近使用它的 CPU，并让该 CPU 靠近目标 GPU，使内存、CPU 与 GPU 的数据路径尽量保持在同一 NUMA 域。下面沿用前

例，通过 `--membind=1` 强制从 NUMA 节点 1 分配；如果该节点内存不足，严格绑定可能导致分配失败，而不是自动回退：

```
numactl --cpunodebind=1 --membind=1 python train.py --gpu 5 &
```

通过 `numactl` 启动进程时，CPU 约束（`--cpunodebind`）和内存策略（`--membind`）会应用于目标进程。Linux 通常让子进程继承这些属性，而且 NUMA 内存策略通常跨 `exec` 保留；原文声称只有 `fork` 子进程继承、`spawn` 或 `exec` 不继承，并不准确。框架或容器运行时仍可能主动覆盖设置，因此应在实际 worker 中验证有效策略。

此外，锁页内存（*page-locked memory*，也称 pinned memory）不会被换出，其物理页可在 DMA 操作期间保持稳定。CUDA 可直接从已注册的锁页主机缓冲区发起 DMA；对于普通 pageable 内存，运行时通常需要先复制到内部锁页 staging 缓冲区。锁页内存因而可能提高 H2D/D2H 带宽并便于异步重叠，但原文所称“快 2–3 倍”不是通用保证，实际收益取决于传输大小、互连、并发和系统内存压力。

> 可以使用 CUDA Samples 中的 `bandwidthTest`比较 pageable 与 pinned 主机内存的 CPU-GPU 传输带宽。具体命令行选项取决于所安装 CUDA Samples 的版本；应先运行 `bandwidthTest --help`，不能假定所有版本都支持原文给出的 `--memory=<pinned or pageable>` 语法。

NVIDIA GPUDirect RDMA 可在受支持的 GPU、NIC、PCIe 拓扑、内核模块和驱动配置下，使 InfiniBand/RDMA NIC 与 GPU 内存直接交换数据。类似地，GPUDirect Storage（GDS）可让兼容存储路径在 GPU 内存与存储之间进行直接 DMA，减少经由 CPU bounce buffer 的数据复制。两者仍需要 CPU 提交和管理 I/O，也可能有控制路径开销，因此“没有额外 CPU 开销”并不准确。

深度学习框架通常允许数据加载器使用锁页内存。例如，PyTorch `DataLoader` 的 `pin_memory=True` 会让返回批次经过锁页内存处理，如图 3-5所示。具体由哪个线程执行 pinning、内存落在哪个 NUMA 节点，会受到 PyTorch 版本、worker 模式和首次触页位置影响。

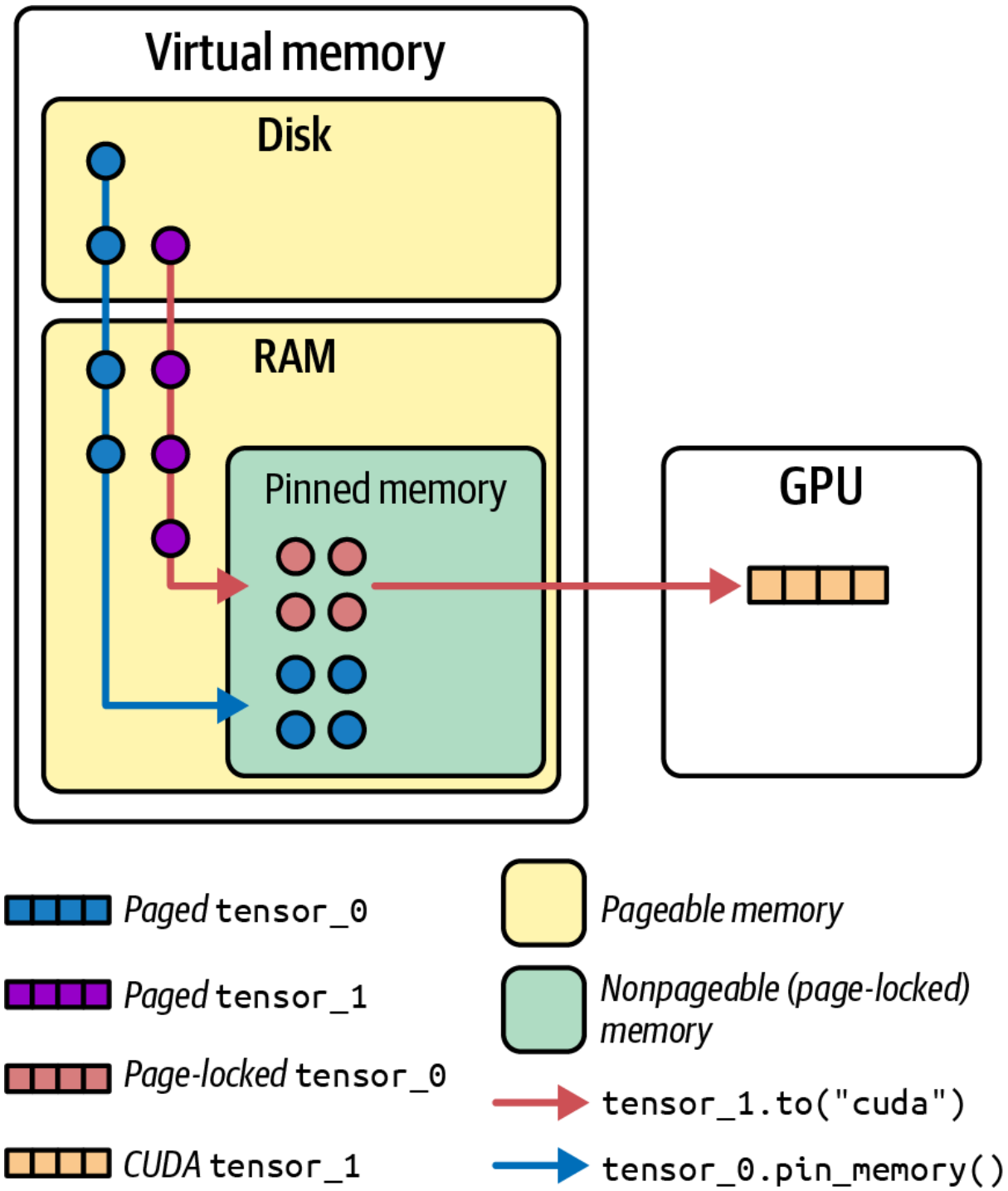


图 3-5。 锁页内存（也称 pinned memory 或不可分页内存）不能被换出到磁盘

锁页内存可能加快 `tensor.to(device)`，主要因为页锁定主机内存支持直接 DMA，并为主机到设备复制与 GPU 工作重叠创造条件。pageable 内存通常需要额外 staging 或同步，从而减少可重叠空间。要真正异步重叠，还需要合适的 CUDA stream、`non_blocking=True`、独立 copy engine 以及不存在阻塞依赖。

实践中，`pin_memory=True` 经常能改善吞吐，但收益取决于工作负载、批大小、传输粒度和主机内存行为。当 H2D 复制位于关键路径时，锁页内存通常更有价值；锁页过多也会减少操作系统可分页内存并增加系统压力。

总之，应根据基准测试决定数据加载器是否使用锁页内存，例如 PyTorch `DataLoader` 的 `pin_memory=True`。GPUDirect RDMA 和 GDS 只应在硬件、驱动、文件系统或网络路径均受支持且验证通过时启用；这些技术可能降低数据路径开销，但不会在所有负载中必然降低延迟。

操作系统会限制进程可以锁定的内存量。shell 中可用 `ulimit -l` 查看或设置 `RLIMIT_MEMLOCK`；数值单位和硬上限取决于 shell 与系统配置。在容器环境中，可配置 Docker `--ulimit memlock` 或相应运行时设置，但容器内的 capability、宿主机硬限制和 cgroup/运行时策略仍可能约束锁页能力。

> 如果需要大型锁页缓冲区，应根据实测峰值和并发量设置足够的 `RLIMIT_MEMLOCK`，否则注册或分配可能失败。不应无条件设为 `unlimited`：锁页过多会削弱系统回收内存的能力，并扩大资源耗尽风险。在受控 HPC 节点上可以提高甚至取消该限制，但应同时设置作业级资源边界、监控和容量余量。

透明大页（Transparent Hugepages）

除了锁定内存并将其绑定到 NUMA 节点之外，我们还应讨论透明大页（THP）。Linux 内存管理通常使用 4 KB 页面；但当进程使用数十乃至数百 GB 内存时（例如容纳深度学习数据集、预取批次和模型参数等），管理数百万个小页面的效率很低。

大页（2 MB 甚至 1 GB 的页面）通过增大内存块来降低虚拟内存管理开销。其主要优势是减少缺页异常，并减轻地址转换后备缓冲器（translation lookaside buffer，TLB）的压力。

TLB 是 CPU 用来缓存虚拟地址到物理地址映射关系的高速缓存。页面数量更少、尺寸更大，意味着相同数量的 TLB 表项可以覆盖更多内存，从而减少 TLB 未命中。

大页通常带来的收益不算很高，吞吐量提升往往约为 3%–5%。这是通过降低缺页异常开销和 TLB 压力实现的。在大多数系统上，启用 THP 是一种简单有效的优化，因为内核会自动使用 2 MB 页面为大型内存分配提供后备存储。在内存池非常大的场景中（例如为 I/O 预分配锁页缓冲区），还可以使用 `vm.nr_hugepages` 或 `hugetlbfs` 显式配置大页，以获得更加确定的性能。

> 请记住，使用大块锁页内存区域时，应将 `ulimit -l`（最大锁定内存）设置为较高的值或 `unlimited`。如果此限制过低，锁定内存可能会失败，导致退回到可交换内存，或者发生内存不足（OOM）错误。

需要特别注意，THP 的后台内存规整可能引入不可预测的停顿，这对延迟敏感的 LLM 推理工作负载可能造成严重影响。Linux 默认配置为尽可能通过 THP 自动分配 2 MB 页面。这通常已经足够，但仍值得针对具体工作负载进行测试。

可以禁用 THP，但之后需要手动分配和控制大页。这会增加复杂度，却可能是推理等低延迟工作负载所必需的。禁用 THP 后，系统可以避免由内核执行内存规整所造成的停顿。

目前普遍的做法是：对于大多数重视吞吐量、基于 GPU 的训练工作负载，启用 THP；对于推理等重视延迟的工作负载，则完全禁用 THP（`transparent_hugepage=never`），或使用 `madvise`。当多个 rank（GPU）同时分配内存时，分布式训练工作负载也需要考虑这一取舍。

除 CPU/内存绑定和大页之外，还有一些值得说明的操作系统级调优措施，包括线程调度、虚拟内存管理、文件系统缓存和 CPU 频率设置。接下来几节将逐一介绍。

调度器与中断亲和性

在繁忙的系统上，应确保数据流水线线程等重要线程不会频繁受到中断。Linux 默认使用完全公平调度器（Completely Fair Scheduler，CFS），它适用于大多数场景。

但是，如果某个负责向 GPU 提供数据的线程对延迟极为敏感，可以考虑为该线程使用实时先进先出（FIFO）或轮转（RR）优先级调度。这样可以确保高优先级线程运行时不会被普通优先级线程抢占。

不过，使用这种方式需要谨慎，因为实时线程若管理不当，可能使其他进程得不到运行机会。实际使用中，如果已经将线程绑定到专用核心，通常不必调整实时线程优先级，但仍应留意这一选项。

另一种选择是隔离核心或创建独立的 CPU 分区，从而进一步减少这些专用计算资源受到的干扰。可以使用 `cset`、`isolcpus` 和 `nohz_full` 等内核参数，或者使用 cgroup 的 `cpuset` 隔离。完成隔离后，操作系统调度器会将这些 CPU 核心留给指定的工作负载使用。

在生产环境中，强烈建议使用 cgroup 配置 CPU 和内存亲和性。这样可以将每个 AI 工作负载隔离到各自的物理核心和内存区域，防止不同工作负载之间发生资源争用，并避免 NUMA 访问惩罚。应使用 `cpuset` cgroup 或容器运行时选项（`docker --cpuset-cpus`）强制实施这种隔离。

可以将每个设备的硬件中断分配给同一 NUMA 节点上的核心。这样可以避免跨节点处理中断，否则不仅会增加延迟，还会逐出远端节点中有用的缓存行。例如，当 NUMA 节点 0 上的 GPU 或 NIC 触发中断时，可将其中断绑定到节点 0 上的某个核心，避免由其他节点处理。如果不进行这种绑定，另一个 NUMA 节点上的 CPU 可能会处理中断，从而引发缓存一致性流量和跨节点通信。

实际使用中，对性能敏感的系统通常会禁用默认的 `irqbalance` 守护进程，或通过定制规则运行它。另一种方法是使用 `/proc/irq/*/smp_affinity` 手动设置每个中断的亲和性掩码。将每个 GPU 和 NIC 中断绑定到距离最近的核心，可确保这些设备中断始终由最合适的 NUMA 节点处理。

简而言之，结合使用专用核心、适当的调度优先级以及 NUMA 感知的硬件中断绑定，有助于最大限度地降低向 GPU 提供数据的数据加载线程所产生的抖动。

虚拟内存与交换

不言而喻，应始终尽量避免内存交换。如果进程的部分内存被换出到磁盘，性能可能下降若干个数量级，后果十分严重。GPU 程序往往会分配大量主机内存来缓存数据。如果操作系统将部分数据从内存换出到磁盘，那么 GPU 后续需要访问这些数据时将遭遇很长的延迟。

建议设置 `vm.swappiness=0`，它会使 Linux 尽可能延后主动换出匿名页，但并不会禁用交换。还应为训练作业显式配置 cgroup 内存及 swap 限制；仅设置 swappiness 或创建 cgroup，都不能保证完全不发生交换。

> 应通过 Docker 或 Kubernetes 使用 cgroup v2，并显式配置 cpuset CPU/NUMA 节点及 swap 限制。正确配置这些控制项后，才能在容器化环境中实施 NUMA 亲和性和禁止交换策略。

还可以使用 `sudo swapoff -a` 禁用当前所有交换设备和交换文件；若未修改持久化配置，重启后它们可能再次启用。务必确保工作负载拥有足够的 RAM，或设置资源限制以避免过量承诺；否则，OOM killer 可能终止进程。可使用 `vmstat` 或 `free -m` 监控交换空间使用情况，确认其保持为零。

另一个相关设置是前文在锁页内存部分提到的 `ulimit -l`。它规定进程可以锁定、因而不会被换出的内存上限；调高此限制只是允许应用锁定更多内存，并不会自动锁定普通内存。对于使用大量锁页内存的 AI 工作负载，通常会将此限制设置为较高值或 unlimited，同时结合应用的锁页行为和 cgroup swap 策略。

文件系统缓存与回写

大型训练作业的一项最佳实践是频繁将检查点写入磁盘，以便作业失败后从已知完好的检查点重新启动。不过，保存检查点时突发写入的大量数据可能占用大量操作系统页缓存，并引发停顿。

在存储方面，可以调整 `vm.dirty_ratio` 和 `vm.dirty_background_ratio`，改变脏页开始后台回写以及写入进程自身受到节流的阈值；它们并不直接设置页缓存总大小。例如，对于数 GB 的检查点，提高脏页阈值可让操作系统在回写磁盘之前先在 RAM 中聚合更多数据，从而吸收写入突发。但阈值过高也可能积累更多脏页，造成更长的集中回写停顿，因此应根据内存容量、存储带宽和实测延迟进行调优。

另一种选择是在独立线程中保存检查点。PyTorch 较新的方案支持由集群中的各个节点分别写入分布式检查点分片；作业失败并重新启动后，加载检查点时会组合这些分片。

对于延迟敏感的训练工作流，可以考虑绕过页缓存。例如，可使用 `O_DIRECT` 打开检查点文件；也可使用 Linux 的 `io_uring` 实现异步 I/O，但只有配合 direct-I/O 选项时才会绕过页缓存。写完检查点并确保脏页已完成回写后，可调用 `posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED)`，提示内核回收相应缓存页，以减轻后续迭代的内存压力；该调用只是建议，并不保证页面立即被丢弃。

CPU 频率与 C-states

许多计算节点默认会让 CPU 采用节能模式，在空闲时降低频率或进入睡眠状态。这有助于节省能源、减少发热并降低成本。在模型训练期间，当 GPU 处理数据集的最后几批数据时，CPU 不一定始终处于满负载。不过，新任务到来、系统恢复 CPU 活跃状态时，这些电源管理功能可能增加额外延迟。

为了获得较高且较稳定的性能，AI 系统常将 CPU 频率 governor 配置为“performance”模式。该模式倾向于请求策略范围内的最高性能，但实际频率仍可能受硬件调频、睿频、功耗和温度限制影响，并非始终固定在最大频率。可以使用 `cpupower frequency-set -g performance` 配置，也可以通过基本输入/输出系统（BIOS）设置。

同样，限制或禁用深层 C-state 可以防止核心进入唤醒延迟较高的低功耗空闲状态。CPU C-state 是系统 ACPI 规范定义的处理器电源状态：C0 表示处理器正在执行指令，C1 及后续状态表示不同深度的空闲状态。通常状态越深，节能越多，但新任务到来时唤醒所需时间也可能越长。限制深层 C-state 有助于减少过高的延迟尖峰。

> 实际使用中，许多服务器 BIOS/UEFI（统一可扩展固件接口）提供高性能配置文件，可设置偏重性能的 CPU 电源策略，并限制深层 C-state；具体行为因平台和固件而异。

本质上，这是以增加一部分功耗来换取响应更快的 CPU 行为。在 GPU 是主要耗电设备的训练场景中，如果增加少量 CPU 功耗能够持续为 GPU 供给数据，通常是可以接受的。例如，当数据加载器线程在等待数据期间休眠、CPU 进入较深的 C6 状态时，处理器的很大一部分会断电，以最大限度节能。

CPU 进入较深的睡眠状态后，唤醒可能需要数微秒。单次时间虽短，累积起来却可能在管理不当时造成 GPU 气泡。所谓*气泡*，是指 GPU 等待 CPU 恢复数据处理的空闲时段。让 CPU 保持随时可用，有助于减少此类停顿。许多服务器 BIOS 都提供禁用 C-state 或限制其最大深度的设置。

> 对于延迟敏感且经测量确认受这些因素影响的系统，应减少会引入不可预测延迟的机制，例如过多的上下文切换、频率状态转换和内存换出。目标是让 CPU 以 GPU 所能消费的速度持续供给数据，避免操作系统将关键任务调度到不合适的核心，或在不恰当的时机占用 CPU 周期。是否禁用节能功能应依据延迟、吞吐量、功耗和散热的实测权衡决定。

调优主机 CPU 内存分配器

在调优良好的 GPU 服务器上，由于大部分计算由 GPU 承担，CPU 利用率未必很高。不过，CPU 活动应当保持稳定，并与 GPU 活动协调一致：GPU 处理当前批次时，CPU 需要持续准备下一批输入。

CPU 到 GPU 的顺畅交接对于维持较高的 GPU 利用率至关重要。调优主机内存分配器（`jemalloc` 或 `tcmalloc`）可以减少由内存分配和回收引起的数据准备停顿，但无法消除所有不可预测的延迟。实际收益取决于应用的分配模式以及其他 CPU、I/O 和同步瓶颈。

调优后，应通过性能分析确认 GPU 因 CPU 端延迟而等待数据的时间有所下降，而不能仅以利用率接近 100% 作为保证。使用 `jemalloc` 时，可以通过 `narenas` 调整 arena 数量，以降低多线程分配时的锁争用；这与 jemalloc 的 per-CPU arena 模式并非同一设置。还可启用 `background_thread`，在后台执行页面回收，并延长 `dirty_decay_ms`/`muzzy_decay_ms`，避免过快地回收已释放页面。这些设置可能降低回收开销，却也可能增加常驻内存，且 arena 过多会加剧内存占用和碎片，因此必须测量后取舍。

可以通过 `MALLOC_CONF` 环境变量调优 `jemalloc`。以下配置仅是示例起点：

```
export MALLOC_CONF="narenas:8,dirty_decay_ms:10000,muzzy_decay_ms:10000
,background_thread:true"
```

类似地，可以调节 `tcmalloc` 的 `TCMALLOC_MAX_TOTAL_THREAD_CACHE_BYTES` 和 `TCMALLOC_RELEASE_RATE` 环境变量。增大线程缓存容量可能让更多小型分配避开中央分配路径和系统调用，但也会增加内存占用；释放速率的具体含义和可用性还取决于所使用的 TCMalloc 实现与版本。可以从以下配置开始测试：

```
export TCMALLOC_MAX_TOTAL_THREAD_CACHE_BYTES=$((512*1024*1024))
export TCMALLOC_RELEASE_RATE=16
```

简而言之，优化内存分配器可以降低分配开销，并在合适的配置下改善碎片问题，从而减少 CPU 向 GPU 供给数据时的一类停顿。应针对具体工作负载和运行环境测试这些环境变量，同时监控吞吐量、尾延迟、常驻内存和碎片情况。

GPU 驱动与运行时性能设置

我们已经完成了 CPU 端的优化，但 GPU 驱动和运行时中也有一些会影响性能的重要设置，尤其是在多 GPU 和多用户场景中。NVIDIA GPU 提供了若干调节选项，合理配置后可以降低开销，并改善多个工作负载共享 GPU 的方式。

接下来将介绍 GPU 持久化模式、MPS、MIG 分区，以及时钟设置、ECC 内存和内存不足行为等其他注意事项。

GPU 持久化模式

默认情况下，当没有应用程序使用某块 GPU 时，驱动可能释放与该 GPU 相关的状态，硬件也可进入较低功耗状态。下一个应用程序使用 GPU 时，需要重新初始化相应状态。根据平台和配置，这项初始化可能带来明显的启动延迟，有时可达到秒级。

对于周期性释放并重新获取 GPU 的工作负载，GPU 初始化开销会影响性能。例如，训练集群中的作业可能频繁启动和停止；低流量推理集群也可能在新请求到来时重新初始化 GPU。在这些场景中，冷启动开销会降低整体工作负载性能。

可以运行 `nvidia-persistenced` 守护进程来启用持久化模式。它会在没有活跃应用程序时保留 GPU 的驱动状态，避免最后一个客户端退出后完全销毁这些状态。持久化模式可以减少下一个作业的初始化延迟，但并不保证 GPU 保持完全唤醒、禁止所有电源门控或实现零

启动延迟。它通常适合频繁启动作业或对冷启动延迟敏感的环境。可使用以下命令配置持久化守护进程在开机时启用；若要立即启动，还需另行执行启动操作：

```
systemctl enable nvidia-persistenced
```

> 在 Kubernetes 环境中，可以配置 NVIDIA GPU Operator，自动为各 GPU 管理持久化模式。

AI 集群通常会在服务器启动时为 GPU 启用持久化模式。这样可以提前保留驱动状态，在作业开始时减少初始化工作。它不会加速实际的数学运算，但可以缩短作业启动时间，并降低一部分冷启动延迟。

GPU 持久化模式也有助于交互式使用。未启用持久化时，系统空闲一段时间后的第一次 CUDA 调用可能因重新初始化 GPU 状态而停顿；启用后通常可以减少这部分延迟，但具体效果取决于驱动、GPU 和系统电源管理状态。

持久化模式可能使 GPU 在空闲时保留更多状态，并带来一定的资源或功耗开销；具体功耗影响依平台而异。对许多数据中心场景而言，这可以换取更一致的启动性能。GPU 持久化模式通常由具有 `sudo` 权限的管理员统一配置。

MPS

通常，当多个进程共享一块 GPU 时，GPU 调度器会在这些进程之间进行时间复用。例如，两个 Python 进程都要在同一块 GPU 上运行内核时，GPU 可能先执行一个进程的内核，再执行另一个进程的内核，如此交替。如果这些内核很短、两次提交之间又存在空闲间隙，那么上下文切换和无法重叠执行可能导致 GPU 利用不足。

NVIDIA 多进程服务（Multi-Process Service，MPS）允许多个进程更高效地并发使用 GPU。资源允许时，MPS 客户端提交的内核可以并发执行，从而减少独立 CUDA 上下文之间切换及串行调度所造成的开销。MPS 服务器为客户端协调共享连接和执行资源，但不能简单理解为把所有客户端的 CUDA 上下文或虚拟地址空间合并成一个。

MPS 何时有用？模型训练通常采用每块 GPU 一个进程，此时未必需要 MPS。若在一块大型 GPU 上运行多个无法单独占满设备的推理作业，MPS 则可能提高资源利用率。例如，假设一块 40 GB GPU 上运行四个独立推理作业，每个占用 5–10 GB 显存，单独运行时仅利用约 30% 的计算能力。如果这些作业主要按时间片串行运行，设备将留下大量空闲执行资源。这里的比例只是示例，实际调度还取决于 CUDA 上下文、内核并发条件和驱动行为。

为这些推理作业启用 MPS 后，一个作业受延迟或资源依赖阻塞时，另一个作业的内核可能利用剩余执行资源，从而提高总体利用率。例如，两个单独运行时各使用约 40% GPU 资源的进程，在资源需求互补的情况下，合并利用率可能达到 80%–90%；这不是 MPS 的性能保证。

类似地，两个单独执行各需一小时的训练进程，可以尝试通过 MPS 在同一块 GPU 上并发运行。当并发客户端的内核和内存带宽需求互补、且彼此争用较少时，总完成时间可能接近一小时多一点，而不是串行的两小时；若资源争用严重，则不会获得这种近 2 倍的收

益。为便于理解，假设进程 A 和进程 B 在未启用 MPS 时周期性启动内核，GPU 可能按 A-B-A-B 的方式调度，并在二者等待期间出现间隙，如图 3-6 所示。

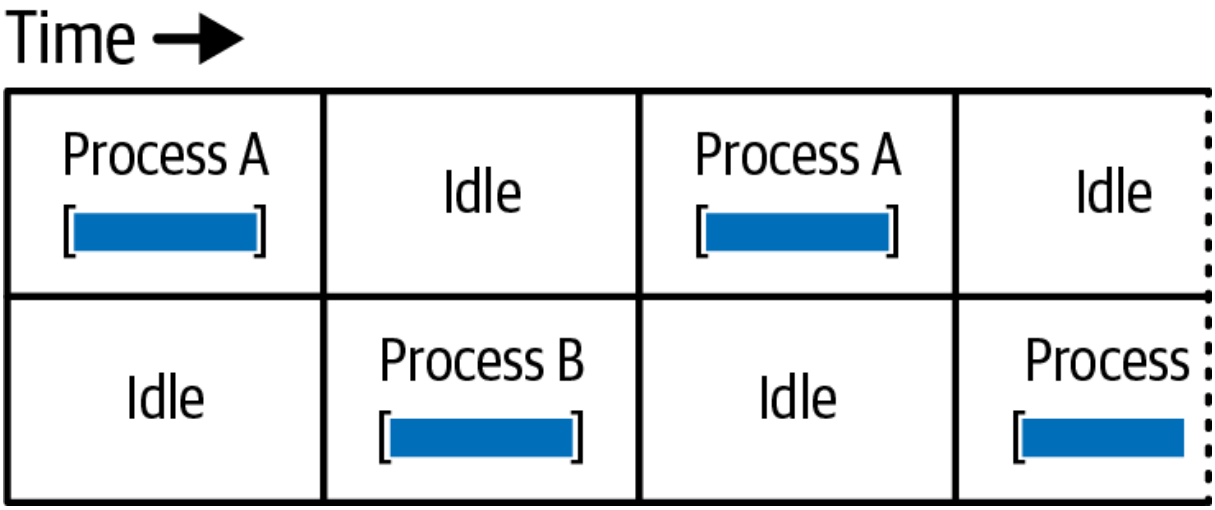


图 3-6。GPU 交替运行进程 A 和进程 B 的内核；一个进程等待、另一个进程活动时会产生空闲间隙

启用 MPS 后，调度可以重叠 A 与 B：当 A 未使用 GPU 的部分执行资源时，B 的工作可以同时利用这些资源，反之亦然。这种重叠有机会减少空闲间隙，如图 3-7 所示。

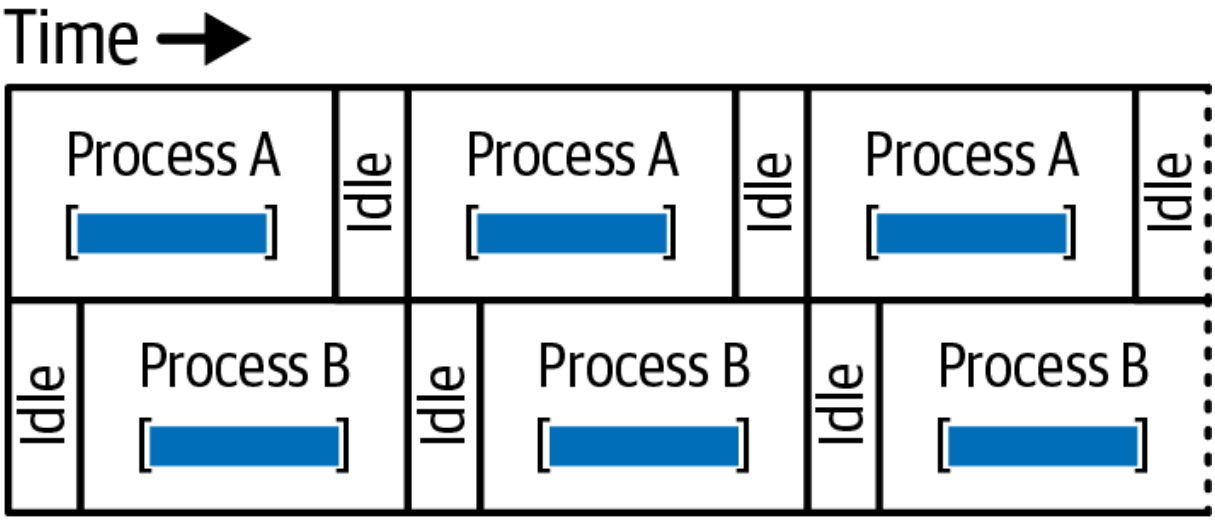


图 3-7。使用 MPS 减少进程 A 和进程 B 的空闲间隙

配置 MPS 需要运行 MPS 控制守护进程（`nvidia-cuda-mps-control`），由它按需启动 MPS 服务器进程来协调 GPU 访问。在较新的 GPU 架构上，MPS 客户端可通过更直接的提交路径与硬件交互，从而减少服务器进程参与关键执行路径的开销。

通常会在节点上启动 MPS 控制服务，并按 GPU、用户或调度策略管理相应服务器；随后设置 MPS 所需的环境变量，再启动 GPU 作业。连接到同一 MPS 环境的作业便可以并发共享 GPU。

MPS 还可以为客户端设置 active thread percentage，用于限制该客户端可用的并发执行资源上限。它控制可供客户端使用的线程/SM 资源比例，但不是专用 SM 的硬分区，也不能单独保证 QoS。例如，可以设置 `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=50`，将客户端可用执行资源上限限制在约 50%；若未设置，各作业将竞争可用的 GPU 资源。

需要注意，MPS 不像 MIG 那样硬性划分 GPU 显存容量。各客户端拥有独立的 GPU 虚拟地址空间，但共享同一物理显存池和其他设备资源。某个进程的大量显存分配或严重 GPU 故障可能影响同一 MPS 环境中的其他客户端，具体故障传播行为取决于 GPU 架构和错误类型。此外，如果一个程序已经独占并饱和 GPU，MPS 不会使它变得更快；只有单个作业留有其他作业可填补的执行空隙时，MPS 才可能带来收益。

MPS 的另一个限制与多用户部署有关。传统的 MPS 部署通常要求连接同一服务器的客户端使用同一 Unix 用户；多用户集群因此常由调度器协调，使同一时间只有一个用户的作业通过 MPS 共享 GPU。系统级共享部署必须仔细评估权限、监控、故障传播和资源隔离风险。

较新的 NVIDIA 驱动和受支持的架构提供多用户 MPS，允许不同 Unix 用户的进程连接 MPS 服务。这提高了易用性，但不提供 MIG 式的显存容量、带宽或故障硬隔离；需要强隔离时应优先考虑 MIG。Kubernetes GPU time-slicing 是另一种共享方式：设备插件可以将一块 GPU以多个逻辑副本暴露给不同 Pod。例如，将复制因子设为 4 后，调度器可以把四个 Pod 放到同一块 GPU 上，由底层驱动对其 CUDA 上下文进行时间复用。

Kubernetes time-slicing 不需要 MPS，也不会提供 MPS 的跨进程并发执行协调。它主要增加可被调度到同一设备的工作负载数量，并不保证更快切换，也不提供显存或故障隔离。它可用于允许一定性能波动的交互式或低利用率工作负载；高吞吐场景是否更适合 MPS，强隔离场景是否更适合 MIG，都应通过实际工作负载测试决定，下一节将继续讨论 MIG。

MIG

支持 MIG（Multi-Instance GPU，多实例 GPU）的现代 NVIDIA GPU 可以在硬件层划分为多个 GPU 实例。MIG 属于硬件辅助的资源分区机制，可以提供可预测的专用资源和较强隔离，但会牺牲动态共享资源的灵活性。其性能开销取决于工作负载、profile 和资源利用情况，不能统一概括为固定的几个百分点。

由于资源是硬划分的，一个实例空闲时通常不能把自己的分区资源借给另一个实例。部分支持 MIG 的 GPU 最多可划分为七个较小的逻辑 GPU；每个实例拥有专用的显存分区和计算资源（包括一定数量的 SM），如图 3-8 所示。可用实例数量与 profile 组合取决于具体 GPU 型号。

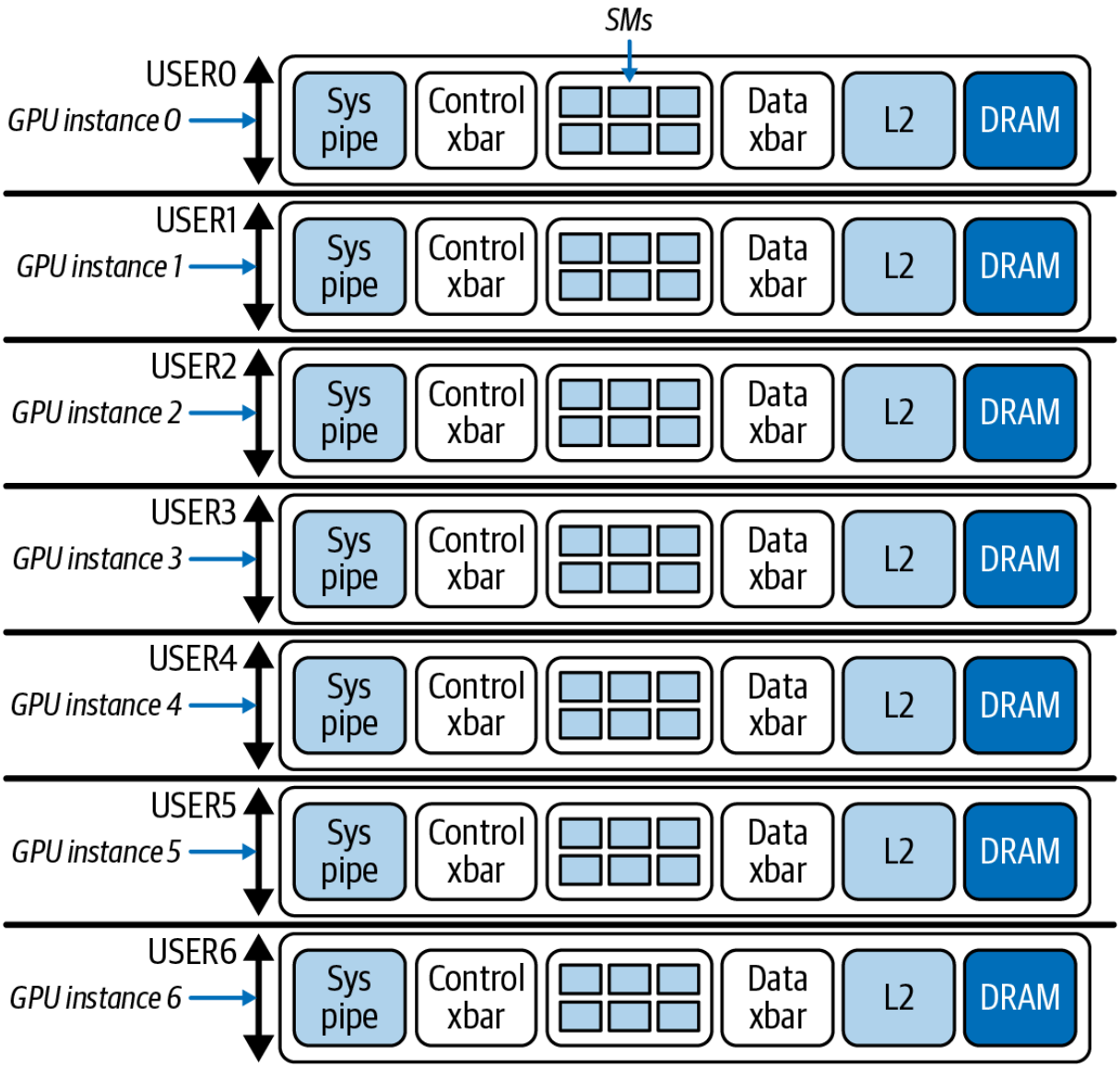


图 3-8。一块物理 GPU 被划分为七个 MIG 实例（改编自 NVIDIA MIG User Guide）

按照 NVIDIA 的 MIG profile 命名约定，前缀 `<X>g` 表示分配给实例的 GPU slice 数量。例如，在采用七个 GPU slice 的架构上，`1g`、`2g` 和 `7g` 分别表示一、二和七个 slice。slice 是硬件定义的分区单位，不能仅用 GPU 公布的总 SM 数除以 7 来推导某个 profile 的实际可用 SM 数；应以对应 GPU 型号的官方 profile 表为准。

因此，原文以 132 个 SM 直接计算出每个 `1g` 约含 19 个 SM，只能作为粗略算术示意，并不适用于所有 MIG GPU，也不适用于紧接着列出的 B200 profile。实际 profile 会按硬件 GPC/SM 分组进行分区，并可能保留一部分资源。

后缀 `<Y>gb` 则表示该 profile 保留的 HBM 显存容量。MIG profile 的具体取值由 GPU 代际和型号决定，可查阅 NVIDIA 文档。Blackwell B200 的部分 MIG profile 如表 3-1 所示。

表 3-1。Blackwell B200 的 MIG profile（来源：https://oreil.ly/FsPEx）

Profile 名称	显存比例	SM 比例	L2 缓存比例	复制引擎数	实例数量
MIG 1g.23gb	1/8	1/7	1/8	2	7
MIG 1g.45gb	2/8	1/7	2/8	2	4
MIG 2g.45gb	2/8	2/7	2/8	3	3
MIG 3g.90gb	4/8	3/7	4/8	6	2
MIG 4g.90gb	4/8	4/7	4/8	8	1
MIG 7g.180gb	全部	全部	全部	16	1

该表还列出了每种 profile 的显存、SM、L2 缓存比例、复制引擎数量和可创建实例数。这些固定 profile 与 GPU 的底层分区拓扑相匹配，显存 slice 对应特定的 HBM 资源。

这种两段式名称分别表达计算资源规模（GPU slice 数量）和显存容量（GB）。管理员可以选择硬件支持的 MIG profile 组合；由于底层放置和分区约束，已分配资源之和不一定恰好填满整块 GPU，也不能任意创建新的 slice 大小。

管理员只能在每块 GPU 上启用受支持的 MIG profile，例如 `1g.23gb`、`2g.45gb` 和 `4g.90gb`。可以使用 `nvidia-smi -mig` 管理 MIG 模式，或通过 NVIDIA Kubernetes GPU Operator 的 `nvidia.com/mig.config` 配置。重新配置前通常需要清空节点上的工作负载，再按平台支持的流程应用实例配置。

GPU 进入 MIG 模式后，可以在不重启整个系统的情况下创建和销毁 GPU instance 与 compute instance；具体操作是否需要 GPU reset 取决于 GPU 代际、驱动和当前状态。启用或禁用 MIG 模式在部分平台上需要 GPU reset，而 Ampere 之后某些 GPU 的行为有所不同，因此应遵循对应版本的 NVIDIA MIG User Guide，并在重新配置前清空工作负载。

从软件视角看，每个 MIG 设备近似一块独立 GPU，拥有专用显存容量、SM、L2 cache slice 和相关引擎资源。MIG 的优势是为作业提供硬件级资源隔离和更可预测的服务质量。例如，可以把只需要较小显存和计算配额的多个独立服务部署到同一块物理 GPU；它们对显存容量和计算资源的直接争用会受到分区边界限制。不过，主机接口和部分系统资源仍可能共享，因此不应理解为绝对无干扰。

作业可以明确请求 MIG 设备，但调度时需要尽量填满可用实例。例如，在七个 1g 实例的配置中，如果一个作业只占用一个实例，应将其他合适的作业安排到剩余实例，否则会留

下大量闲置资源。集群可以让部分节点使用 MIG 承载小型推理作业，另一些节点则关闭 MIG，供大型训练作业使用完整 GPU。

MIG 通常在系统或节点层面配置：管理员启用 MIG 模式，并创建所需的 GPU instance 和 compute instance。完成后，这些 MIG 设备会以各自的设备标识符呈现给 CUDA 和管理工具。启用模式、实例创建和 reset 的准确顺序因 GPU 架构与驱动版本而异。

如果 profile 规划与作业需求不匹配，未使用的实例或无法组合利用的剩余 slice 会造成碎片和资源浪费。因此应根据工作负载预先规划分区大小，并在需求变化时重新配置。调整实例本身不一定总要 reset GPU，但必须先处理仍在使用相关设备的工作负载，并遵循平台要求。

对于跨多块 GPU 的大型模型训练和推理服务，MIG 往往并非首选，因为这些工作负载通常需要完整设备资源和高速 GPU 间通信。相反，对于能够在较小 GPU 分区中运行的多租户小模型推理，MIG 的资源隔离可能很有价值。

> 按照当前 MIG 部署指南（包括 R570 时期的行为），跨 GPU 的 MIG P2P 仍不受支持。跨 GPU instance 的 CUDA IPC 也存在限制。NCCL 对 MIG 的支持范围随 GPU 架构、驱动和 NCCL 版本变化，不能笼统断言全部不受支持；部署前应核对当前兼容性矩阵，并确认训练或推理拓扑是否依赖跨 GPU peer communication。
>
> 这些限制可能降低分布式训练吞吐量。大型训练作业以及稀疏 MoE 专家推理系统通常需要大量 GPU 间通信，因此一般不适合采用 MIG。在不支持直接 peer path 的拓扑中，MIG 设备之间的通信需要经过主机或网络互连。

简而言之，当需要在同一块 GPU 上运行多个相互独立、并要求较强资源隔离的作业时，可以考虑启用 MIG。对于依赖完整 GPU 资源和高速互连的大规模分布式训练或跨 GPU 推理，应先验证通信支持和性能，通常更适合使用完整 GPU。

在本书讨论的大型 Transformer 模型训练与推理场景中，我们将保持 MIG 关闭。不过，集群可以根据作业组合切换运行模式：例如白天使用 MIG 承载大量小型训练或推理实验，夜间在清空作业并重新配置后关闭 MIG，运行使用完整 GPU 的大型训练任务。

> Kubernetes device plugin 可以按所选 MIG strategy 暴露 MIG 资源。例如，B200 的 1g、23 GB profile 可显示为 `nvidia.com/mig-1g.23gb`；实际资源名称取决于 GPU 型号、profile 和插件配置。

GPU 时钟频率与 ECC

NVIDIA GPU 提供 GPU Boost，可在功耗、温度和其他硬件限制范围内自动调整核心时钟。大多数情况下，应保留这种自动管理。为了提高测试的一致性，有些用户会设置允许的时钟范围或 application clocks；但这不能保证 GPU 始终运行在固定最大频率，实际频率仍会受到功耗和温度限制。

执行基准测试时，控制并记录时钟、功耗和温度状态非常重要。连续运行后，GPU 可能因为温度上升或触及功耗限制而降低频率。如果没有预热设备并监控这些状态，就可能把后续运行的较差结果误判为代码或配置变化所致。

GPU Boost 会上下调整核心时钟，以遵守功耗与温度限制。可以在受支持的设备上使用 `nvidia-smi -lgc` 设置 GPU 时钟范围；`-ac` 则设置受支持的显存/图形 application clocks，并非所有新设备都支持，也不能简单等同于通用的显存锁频接口。这些设置有助于减少动态频率变化，但不会绕过热保护、功耗限制或保证恒定实际频率。

> 这主要用于提高基准测试的可重复性，但不能使整个系统变成确定性执行。日常训练和推理通常保留自动 Boost；只有在测量到显著性能波动并确认与频率或限频相关时，才考虑调整时钟策略。

追求最后一部分测试一致性时，可以评估时钟控制；但通常保留 GPU 默认的自动 Boost 模式即可，并应通过遥测确认性能变化的真实原因。

有些团队会有意降低 GPU 时钟或功耗上限，以减少热量和能耗，尤其是在长期运行且机架受功率或散热约束时。配置良好的数据中心 GPU 通常有相应冷却能力，但是否降频仍应依据持续性能、能效和温度实测决定。

另一种方法是使用 `nvidia-smi -pl` 设置板卡功耗上限。TDP 是用于设计供电与散热能力的热设计指标，不应定义为 GPU 在持续负载下必然产生的“最大热量”；`-pl` 所设置的则是设备支持范围内的功耗限制。

降低功耗上限后，GPU Boost 会相应调整时钟，以遵守新的功率预算。这可能降低峰值功耗和热负载，在某些工作负载上还能避免更剧烈的限频；性能影响从很小到明显都有可能，取决于工作负载是否受计算频率、显存带宽或功率限制，因此必须实测。

GPU 显存的 ECC（纠错码）是另一项重要考虑。ECC 可以纠正某些可纠正错误（常见示例是单比特错误），并检测不可纠正错误；实际错误分类和处理行为取决于显存架构与 GPU。NVIDIA 数据中心 GPU 通常默认启用相应的 ECC 保护。

在某些较早的 GPU 架构上，禁用 ECC 可能改变可用显存容量或性能；但现代数据中心 GPU 常使用带专用 ECC 位的 HBM，容量和性能影响随型号而异，不能统一认为关闭 ECC 会释放一部分用户显存并提升几个百分点。关闭 ECC 会失去关键的错误保护，增加作业失败或数据损坏风险。

对于 Hopper、Blackwell 等 NVIDIA 数据中心 GPU，应按厂商支持配置保留 ECC，以保障长时间训练和推理的数据完整性。单次不可纠正的显存错误可能终止作业；缺乏错误检测还可能使损坏的数据继续传播。

因此，生产 AI 工作负载通常应保持 ECC 开启。只有在明确了解具体 GPU 的支持状态、实际容量或性能收益，并能接受可靠性风险的受控实验中，才应评估关闭 ECC；不应把它作为常规的显存扩容手段。

在支持切换 ECC 模式的 GPU 上，更改通常需要 GPU reset，并会影响正在运行的作业；某些型号可能不允许用户关闭 ECC。它不是应频繁切换的选项。对于长期 AI 作业，可靠性通常远比未经验证的微小性能收益重要。

GPU 显存超额分配、碎片与内存不足处理

与 CPU RAM 不同，普通 CUDA device allocation 默认没有透明的 GPU“交换空间”。申请超过当前可用显存的 device memory 通常会返回 OOM 错误，框架随后可能终止当前操作或进程。缓解方法包括按需增长分配、在适用场景使用 CUDA Unified Memory、采用内存池或缓存分配器，以及从模型和批次层面降低峰值显存需求。

某些框架（例如 TensorFlow）默认可能在启动时预留大部分可见 GPU 显存，以减少后续分配开销和碎片。在共享 GPU 的场景中，如果不了解这种行为，一个进程可能占据其他进程需要的显存。PyTorch 的缓存分配器通常会按需求向 CUDA 请求内存，再保留已获得的块供后续复用。

TensorFlow 可通过 `TF_FORCE_GPU_ALLOW_GROWTH=true` 从较小的分配开始，再按需增长显存使用量。普通 TensorFlow 或 PyTorch device allocation 仍不能超过可用物理显存；但 CUDA Unified Memory 等机制可以在受支持的系统上超额订阅。按需增长更适合某些多租户场景，因为多个进程不会在启动时都尝试预留最大可用显存，不过它本身不提供容量隔离或公平性保证。

CUDA Unified Memory 提供统一虚拟地址和托管内存抽象，应用不必始终手动决定每个分配驻留在 CPU 还是 GPU。CUDA 驱动可根据处理器访问、预取与放置提示、系统拓扑和内存压力迁移或映射页面。较新的 NVIDIA GPU 支持按需分页和页面迁移所需的硬件机制；“Page Migration Engine”是常见概括，但实际迁移涉及 GPU MMU、页面故障处理和驱动策略。

托管内存页面可能因访问位置、显式预取或内存超额订阅而在 GPU 显存与主机内存之间迁移，也可能根据平台能力采用直接访问。它提供了容量与编程灵活性，但页面故障、迁移和远程访问通常比工作集完全驻留在 GPU 显存中开销更高。

这种 GPU 与 CPU 内存之间的数据移动可能很慢，因为经 CPU-GPU 互连访问或迁移主机内存，通常无法达到本地 GPU HBM 的带宽与延迟，正如第 2 章所述。Unified Memory 可帮助运行无法完全装入显存的工作集，也可简化数据管理，但并非免费的显存扩展。

对于性能敏感的工作负载，应尽量避免依赖 Unified Memory 的持续超额订阅，并通过预取、访问提示和工作集设计减少反复迁移。它可以避免某些分配立即失败，但显存被超额订阅时，性能下降幅度取决于访问局部性、迁移频率和 CPU-GPU 互连。

PyTorch 等库使用缓存分配器：张量释放后，分配器不会立即把对应显存块交还给 CUDA 驱动，而是保留它们以供后续分配复用。这样可以减少重复调用底层分配器的同步与管理开销，也可能缓解某些分配模式造成的碎片；但缓存分配器自身仍可能产生碎片。

可以通过 `PYTORCH_ALLOC_CONF`（`PYTORCH_CUDA_ALLOC_CONF` 是兼容别名）配置 PyTorch CUDA allocator 的 backend、分块、垃圾回收阈值和可扩展 segment 等行为。它没有一个适用于所有 backend 的通用“最大内存池大小”选项；可用选项应以当前 PyTorch 文档为准。后续章节将继续讨论 PyTorch 内存分配机制的优化。

遇到 GPU OOM 时，应先区分真实峰值需求、其他进程占用、张量生命周期或泄漏、以及 allocator 碎片。`torch.cuda.empty_cache()` 只释放当前未被张量占用的缓存块，让其他进程或 CUDA 分配可使用；它不会释放仍被活跃张量占用的显存，也不会增加 PyTorch 自身可用的物理容量。

PyTorch 还提供 `torch.cuda.memory_stats()` 和 `torch.cuda.memory_summary()` 等工具，通过 allocated、reserved、inactive split 等统计信息帮助分析显存使用和碎片。NVIDIA Nsight Systems 可展示 GPU 内存活动、CPU-GPU 互连活动以及 GPUDirect Storage 时间线，结合应用标记有助于定位异常长生命周期分配；Nsight Compute 则提供内核级 occupancy、吞吐量和相关硬件指标分析。后续章节将介绍这些工具。

Docker 的 `--gpus` 参数用于选择并向容器暴露 GPU，但不会为普通整卡共享设置显存上限。如果需要显存容量与计算资源的硬分区，可在支持的 GPU 上使用 MIG，并在 Kubernetes 中请求 `nvidia.com/mig-2g.45gb` 等 MIG 资源。MPS 的 active thread percentage 只能限制一部分执行资源上限，不能提供显存容量硬隔离或完整的公平共享保证。

在多租户节点中，MIG 可用于隔离合适的小型作业；若每块 GPU 只运行一个作业，通常让作业使用可用显存更为直接。是否进行分区应依据服务隔离、调度粒度和利用率需求决定。

一般而言，可在应用层降低 OOM 风险，例如减小批量大小、使用更低精度的模型权重和优化器状态、采用 activation checkpointing、参数或优化器分片、CPU/NVMe offload，或在可行时缩小模型规模。

最佳实践是在训练和推理期间使用 `nvidia-smi` 或 NVML API 监控 GPU 显存使用情况。还应记录峰值、每进程占用和 allocator 统计。接近容量上限时，可减小 batch size、在训练中使用 activation checkpointing，或采用其他降低峰值显存的技术。

还应避免关键主机内存被换出，因为 GPU 或 CPU 数据流水线访问已换出页面时会受到磁盘 I/O 延迟影响，进而降低 GPU 利用率和 goodput。应结合容量规划、cgroup swap 策略和必要的锁页内存配置；调高 `ulimit` 只允许应用锁定更多内存，降低 swappiness 也不等于彻底禁用交换。

在常驻 GPU 节点上，通常会让 NVIDIA 驱动模块保持加载，而不是在作业之间频繁卸载。持久化模式保留的是设备驱动状态；内核模块是否加载是更底层但不同的问题。卸载驱动后，下一个作业需要重新加载模块并初始化设备，MIG 配置的持久性也取决于 GPU 代际与驱动行为。

> 建议在专用 GPU 节点上按运维策略保持驱动和所需 MIG 配置，避免不必要的冷启动与重新配置。驱动升级、内核维护、故障恢复或安全处置时仍可能需要卸载或重载模块。因此，集群管理员通常会在机器启动后加载 NVIDIA 驱动模块，并通过受控维护流程变更。

GPU 容器运行时优化

许多 AI 系统使用编排工具和容器运行时管理软件环境，Kubernetes 与 Docker 在 AI 基础设施中尤为常见。容器可以固定 CUDA、框架和库版本，提高依赖环境的一致性，减少“在我的机器上可以运行”一类问题。容器会增加镜像、存储、网络、权限和编排方面的复杂度；经过合理配置，GPU 计算路径通常可以接近裸机性能，但整体开销仍取决于 I/O、网络与资源隔离配置。

节点上运行的容器并不是传统虚拟机（VM）。容器与主机共享操作系统内核，因此 CPU 和内存操作通常接近原生性能。NVIDIA Container Toolkit 将主机 GPU 设备和所需驱动组件提供给容器，CUDA 内核仍在物理 GPU 上直接执行。这条计算路径通常没有传统设备仿真开销，但容器的存储、网络、cgroup、NUMA 和编排配置仍可能影响端到端性能。

> 对于使用较新 NVIDIA Container Toolkit 并正确配置的现代 GPU，纯 GPU 计算性能通常可接近裸机；原文给出的“差异小于 2%”只能视为特定软硬件和基准下的经验值，不是通用保证。MLPerf Inference v5.0 中存在使用 Red Hat OpenShift 和 Kubernetes 的提交结果，说明经过调优的容器化方案能够取得有竞争力的效率和延迟，但不能据此推断任意容器或编排配置都没有开销。

NVIDIA Container Toolkit 与 CUDA 兼容性

在容器中使用 GPU 的一项挑战是确保容器内 CUDA 用户态库与主机驱动兼容。NVIDIA Container Toolkit 和官方基础镜像为此提供标准部署方式。与内核及硬件紧密集成的 NVIDIA 驱动位于主机；容器镜像中通常包含特定版本的 CUDA runtime 和其他用户态库。

一般规则是：主机 NVIDIA 驱动必须达到容器内 CUDA 版本要求的最低版本。CUDA 13.x 的 Linux minor-version compatibility 基线为 R580 或更新分支；CUDA 12.x 的基线为 R525 或更新分支。不过，特定 Toolkit 发行版、功能、GPU 架构或安全修复可能要求更高版本，必须查阅对应 release notes。若主机驱动低于所需版本且没有适用的兼容机制，CUDA 初始化或特定功能将失败。

> 每个 CUDA Toolkit 版本都有最低驱动要求。更新 Toolkit 或容器镜像时，应查阅 NVIDIA 的官方兼容性矩阵和 release notes，并在需要时升级主机驱动，而不是只根据 CUDA 主版本推断。

在 Docker 和 Kubernetes 环境中，可以优先使用 NVIDIA GPU Cloud（NGC）或官方镜像仓库提供的基础镜像。诸如 `nvcr.io/nvidia/pytorch` 的框架镜像会组合经过测试的 CUDA runtime、cuDNN、NCCL 等版本，并在镜像说明中列出最低驱动要求。仍应固定具体镜像 tag 或 digest、检查支持周期和漏洞更新，而不要依赖可变的 latest 标签；这样才能在获得新硬件支持的同时降低依赖冲突。

NVIDIA 容器运行时

NVIDIA Container Toolkit 的运行时集成可在创建容器时向其提供选定的 GPU 设备以及主机驱动的用户态组件，因此镜像中不应打包主机 NVIDIA 内核驱动。容器依赖主机已安装的驱动，并通过设备挂载、OCI hook 或 CDI 等机制获得 GPU 访问权限。与传统 VM 不同，普通 Linux 容器共享主机内核，但仍受到 namespace、cgroup 和安全策略约束。

容器内应用通常使用镜像提供的 CUDA runtime 库，例如 `libcudart.so`；NVIDIA Container Toolkit 则在容器启动时按配置挂载与主机驱动匹配的 `libcuda.so`、

`libnvidia-ml.so` 等组件。它们被映射到容器进程的文件系统与地址空间，并通过 GPU 设备节点同主机内核驱动交互，而不是把库函数“转到主机进程中执行”。

只要主机驱动满足镜像内 CUDA runtime 和所用功能的最低要求，这种“容器内用户态 runtime + 主机驱动”拆分就是受支持的。若容器使用的 CUDA 版本或功能超出旧主机驱动的兼容范围，可能在初始化或调用特定 API 时失败。因此应按镜像说明和 NVIDIA 兼容性矩阵验证版本，而不是要求二者版本号完全相同。

关键在于：普通容器部署通常不包含硬件 VM 的 hypervisor 或 GPU 设备仿真层。容器进程共享主机内核，并通过主机 NVIDIA 驱动向物理 GPU 提交工作，所以 GPU kernel 的执行路径接近主机原生进程。不过，容器隔离、设备注入、调度、I/O 和网络层依然存在。

因此，正确配置的 Docker/containerd 与 NVIDIA Container Toolkit 通常不会因 GPU 指令虚拟化本身造成明显损失，但端到端性能不应无条件视为等同裸机。VMware、vGPU、SR-IOV、机密容器或其他虚拟化部署具有不同的数据路径和隔离机制，需要单独评估与调优。

> NVIDIA Container Toolkit 不仅支持 Docker，也可与 containerd 和 Podman 集成。这对于使用 containerd 或兼容 CRI 实现的现代 Kubernetes 环境尤其重要；具体安装方式应遵循当前 Toolkit 文档。

避免容器 OverlayFS 开销

Docker 容器与直接在主机上运行相比，较明显的性能差异之一可能来自 I/O。容器镜像常使用联合或叠加文件系统，将多个只读镜像层与容器可写层组合成统一视图；这些层最终仍存放在主机文件系统上，并不是独立的“容器文件系统设备”。

在 OverlayFS 等联合文件系统中，来自 lowerdir 和 upperdir 的文件与目录会呈现为一个合并后的文件系统视图。这非常适合容器：基础镜像的只读层可以与每个容器自己的可写层组合。

叠加文件系统可能产生额外开销。路径解析和元数据操作需要根据 upper 与 lower 层的状态确定可见对象；白化文件、目录合并和大量小文件访问尤其可能增加元数据成本。对于只读、已缓存或顺序读取的负载，额外开销可能很小，因此应以实际 I/O 模式测量。

OverlayFS 的 copy-up 机制还会影响写入：首次修改只存在于只读 lower 层的文件时，需要先把文件复制到可写 upper 层，再在副本上写入。大文件首次修改、元数据密集型操作以及容器可写层的存储驱动配置都可能放大这项成本；已经位于 upper 层的后续写入不必每次复制整个 lower 文件。

模型训练在读取数据集、加载模型和写入检查点时可能产生大量 I/O。可以使用 bind mount、Docker volume 或 Kubernetes volume，把主机目录、本地 NVMe 或网络文件系统挂载到容器中，使大规模数据路径避开镜像的可写 overlay 层。

bind mount 通常绕过容器 root filesystem 的 OverlayFS 数据路径，因此性能可接近直接访问同一主机路径。但它仍受到主机文件系统、挂载选项、页缓存、网络协议、权限映射、NUMA 拓扑和共享存储争用的影响，不能保证获得底层设备的“全部性能”。多 TB 数据集通常不应打包进镜像，而应通过合适的存储卷提供。

例如，若训练数据位于主机的 `/data/dataset`，可使用 `-v /data/dataset:/mnt/dataset:ro` 挂载，其中 *ro* 表示容器端只读。训练脚本随后从 `/mnt/dataset` 读取数据。该路径直接映射到主机目录，不经过容器镜像的可写层；仍应根据安全需求设置只读、SELinux 标签和传播选项。

最佳实践是避免在容器可写层中执行大量数据读写，尤其是写入大型检查点或修改来自镜像 lower 层的大文件。应将数据目录、缓存目录和输出目录放在经过容量与性能规划的 volume 中，并通过测量确认 I/O 没有受 copy-up、元数据路径或底层存储限制。

缩小镜像以加快容器启动

如果镜像很大并且节点需要通过网络拉取，首次启动可能明显变慢，时间主要花在下载、校验、解压和准备 snapshot 上。镜像层已缓存后，镜像总大小通常不再直接决定容器进程的启动时间。对于持续数小时、数天甚至数月的训练任务，几分钟冷启动可能占比很小；但在节点自动扩缩容、批处理队列和短任务场景中仍然重要。应通过多阶段构建、清理包管理器缓存，并排除不必要的构建工具和临时文件，使运行镜像保持合理精简，从而节省网络、磁盘和启动准备时间。

一些 HPC 中心更偏好 Apptainer（由 Singularity 社区项目演进而来），因为它适合共享 HPC 环境，可在不依赖常驻 root 守护进程的情况下运行容器。Apptainer 使用 SIF 镜像、mount、namespace 和 bind path 将镜像环境与主机资源组合，并非简单地“直接使用主机文件系统”；性能取决于镜像格式、挂载、网络和 I/O 路径。

无论使用 Docker 还是 Apptainer，正确配置时，GPU 计算吞吐通常可以接近主机原生运行；部分研究和基准中的差异只有几个百分点。但这个数字依赖具体工作负载、存储、网络、cgroup、安全隔离和测量方法，不能作为通用上限。仅凭 GPU 利用率与吞吐日志有时难以判断作业是否运行在容器中，却不能据此排除容器化对其他指标的影响。

Kubernetes 拓扑感知容器编排与网络

Kubernetes（又称 K8s）是广泛用于 AI 训练与推理的容器编排系统。NVIDIA device plugin for Kubernetes 在各 GPU 节点上与 kubelet device-plugin 管理器交互，把 GPU 作为 `nvidia.com/gpu` 等扩展资源注册到节点，并在 Pod 获得设备后配置相应设备访问。它不是把 `/dev/nvidia0`、`/dev/nvidiactl` 等设备节点逐个直接发布给 scheduler。GPU 扩展资源通常在 `resources.limits` 中声明；若同时写 `resources.requests`，两者必须相等，而省略 requests 时 Kubernetes 会以 limits 值作为请求量。插件可向 kubelet 提供设备的 NUMA topology hint，但同一 NUMA 或 NVLink 域内的多 GPU 选择是否得到保证，还取决于 kubelet Topology Manager、插件分配策略与平台拓扑支持。

NVIDIA GPU Operator 可自动管理 NVIDIA 驱动、device plugin、Container Toolkit、DCGM 监控组件等软件的部署与生命周期，具体组件取决于 Operator 配置。GPU Feature Discovery（GFD） 会根据节点上检测到的 GPU 特征生成节点级标签，例如产品、显存、MIG 能力及部分拓扑属性；它并非默认为每块 GPU 创建可供 scheduler 直接选择的独立 NUMA 或 NVLink/NVSwitch ID 标签。节点标签可帮助 scheduler 选择合适节点，但节点内具体设备选择仍由 kubelet、device plugin 和拓扑策略完成。

使用 Kubernetes 编排 GPU 容器时，希望资源分配考虑 NUMA、PCIe/NVLink 拓扑和节点间网络。Kubernetes 并非完全“不感知拓扑”：scheduler 支持节点、可用区等拓扑约束，kubelet Topology Manager 可协调节点内 NUMA hint；但默认 scheduler 对设备内部 NVLink 图和每块 GPU 的精细互连关系缺乏通用建模。因此，仅请求若干 `nvidia.com/gpu` 并不自动保证得到最优的 GPU 组合。

例如，一台 8-GPU 服务器可能包含两个各有 4 块 GPU 的高速互连域。如果某个 Pod 请求 4 块 GPU，理想情况是分配同一互连域中的四块设备，以获得更好的 peer-to-peer 通信；若设备选择策略没有理解这一拓扑，则可能从两个域各分配两块，迫使部分通信经过较慢的节点内路径。

节点内缺乏拓扑感知可能使 GPU 间流量从 NVLink 路径退化到 PCIe，并跨越 CPU socket 或 NUMA interconnect；具体带宽下降取决于硬件，不存在统一“减半”规律。InfiniBand 或 Ethernet 通常属于节点间路径：标准 Pod 的 GPU 设备都来自该 Pod 所在的同一节点，不会让单个 Pod 的四块 GPU 分布到不同机架。跨节点和跨机架放置是分布式作业多个 Pod 或 worker 的调度问题，需要另行采用 gang scheduling、拓扑约束和网络感知策略。

以基于 NVLink 5 的 NVIDIA NVL72 机架为例，它把 72 块 GPU 连接在一个高带宽 NVLink 域内。按每 GPU 约 1.8 TB/s 双向端口带宽相加，聚合口径约为 130 TB/s（72 × 1.8 TB/s）；这不是单个作业或任意通信模式可获得的端到端吞吐。对于跨多个 Pod 的分布式作业，如果调度器不了解 NVL72 边界，就可能把 worker 放到不同 NVLink 域甚至域外节点，从而使部分流量经过较慢的网络 fabric，无法充分利用机架内互连。

为减少资源争用，可根据工作负载选择独占节点、完整 GPU 请求、资源配额或受控共享。对于需要 CPU 与 GPU 紧密协作的 Pod，应结合 CPU Manager、Memory Manager 和 Kubernetes Topology Manager，尽量让 CPU、内存和设备的 NUMA hint 对齐；Topology Manager 本身负责协调 hint 和准入策略，并不独自完成所有 CPU 绑定。下一节将详细讨论。

使用 Kubernetes Topology Manager 编排容器

Kubernetes Topology Manager 是 kubelet 组件，负责收集 CPU Manager、Memory Manager 和 device plugin 等 hint provider 提供的 NUMA 拓扑提示，并根据节点策略在 Pod 准入时协调资源对齐。GPU device plugin 可以为设备报告 NUMA 节点 hint，但 Topology Manager 本身不会探测“NVLink 域 A”或“PCIe 总线 Z”，也不会把节点内设备图反馈给 Kubernetes scheduler 以进行全局 GPU 选择。

拓扑感知 GPU 调度仍需要多层配合。管理员可以用节点标签、node affinity、taint/toleration 或自定义调度插件描述节点级 GPU 与互连能力，使多 GPU Pod 被调度到合适类型的服务器；但节点标签只能约束节点选择，不能单独保证节点内分配的 GPU 共享同一 NVLink 域或 NUMA 节点。后者还取决于设备插件的 topology hint、分配策略和 kubelet Topology Manager。

运行多 GPU 或 CPU+GPU 工作负载时，应根据隔离与可用性要求配置 kubelet，例如 `--topology-manager-policy=best-effort`、`--topology-manager-policy=restricted` 或 `--topology-manager-policy=single-numa-node`。`best-effort` 尽力对齐但不会因无法对齐而拒绝 Pod；`restricted` 和 `single-numa-node` 会施加更严格的准入条件。若要获得独占且可绑定的 CPU，还需配合 CPU Manager static policy 和 Guaranteed QoS 等配置。这

些机制可以减少远端 NUMA 访问，但不会自动优化 NVLink 图，并且受单 NUMA 节点容量约束。

应使用与 Kubernetes、驱动和 GPU Operator 版本兼容的 NVIDIA device plugin，而不是笼统追求“最新”。插件可以提供设备 NUMA topology hint，GPU Operator 则负责部署相关组件；二者本身不保证将多 GPU Pod 打包到同一 NVLink 域。应在目标服务器上验证实际设备选择，并结合自定义资源、调度扩展或整节点分配满足更复杂的互连要求。节点内 NUMA 对齐主要降低主机 CPU/内存到 GPU 的远端访问开销，与多节点网络延迟是不同问题。

在 NVLink 5 NVL72 系统中，一个机架级 NVLink 域的聚合双向 GPU 端口带宽可按约 130 TB/s 表述，相当于每块 GPU 约 1.8 TB/s 的端口带宽总和。该聚合值不是单次 collective 的保证吞吐。调度通信密集型训练时，应尽量把相关 worker 保留在同一高速 NVLink 域内，再通过较慢的节点间网络 fabric 扩展；这需要 scheduler 层的拓扑建模，而不只是 kubelet Topology Manager。

Kubernetes 与 Slurm 作业调度

在多节点部署中，作业调度器负责排队、资源匹配和放置，对于提高跨节点利用率至关重要。Slurm 广泛用于 HPC 与训练集群，Kubernetes 常用于云原生训练和在线推理，但这不是固定分工，两者都可服务多种 AI 工作负载。开源 Slinky 项目提供在 Kubernetes 上部署、管理和集成 Slurm 的组件，以改善 Slurm 集群的云原生运维；它不是把 Slurm 与 Kubernetes scheduler 简单合并为一个统一调度器。

这些系统把节点和 GPU 分配给作业，并由相应的作业控制器、Slurm step、MPI launcher 或训练 operator 协调跨节点进程启动。若训练作业请求 8 个节点、每节点 8 块 GPU，调度器先选择满足条件的节点；随后可由 `srun`、`mpirun`、Kubernetes operator 或容器运行时协作启动进程，并注入 rank、地址和设备可见性信息。NGC 等经过测试的镜像可以统一 CUDA Toolkit、框架、NCCL 和 Python 包等用户态环境；主机 NVIDIA 内核驱动仍由各节点提供，不应打包进普通容器镜像。

Slurm 同样需要正确描述 GPU 拓扑。它使用 GRES（generic resources）管理 GPU，并可结合 `gres.conf`、节点拓扑、CPU affinity、`--gpu-bind` 和 `--gres-flags=enforce-binding` 等配置，使作业获得与 CPU/NUMA 更合适的设备。Slurm 可以记录 NVLink 连接信息并影响设备选择，但“请求同一 NVLink/NVSwitch 域”并不是所有版本和集群都自动提供的统一保证，应根据实际配置验证。

如果调度器或节点设备配置把 GPU 视为完全同质资源，多 GPU 作业可能得到不理想的节点或设备组合。正确的 GRES、NUMA、网络 topology、约束与绑定策略可以减少不必要的跨 socket、PCIe 层级和跨网络域通信；是否避免所有跨 NVLink 域通信取决于具体硬件与调度扩展。

Slurm 也可以把 MIG 设备作为 GRES 进行调度，用于将多个独立作业装入同一块物理 GPU。Kubernetes 则可通过 NVIDIA device plugin 按 MIG strategy 暴露对应扩展资源。两者都要求管理员预先配置 MIG 实例和资源发现，且资源命名与可计费粒度依 GPU、profile 和插件版本而定。下一节将介绍 Kubernetes 中的 MIG 资源请求。

使用 MIG 切分 GPU

启用前文介绍的 NVIDIA MIG 模式后，一块受支持的物理 GPU 可以按官方 profile 划分为具有硬件资源隔离的 MIG 设备。以下 Kubernetes Pod 资源片段请求两个 `nvidia.com/mig-2g.45gb` 设备；它假设节点已经配置相应 MIG 实例，并且 NVIDIA Kubernetes device plugin 采用能暴露该 profile 名称的 MIG strategy：

```
resources:
  limits:
    nvidia.com/mig-2g.45gb: "2"
```

配置中的值 `"2"` 表示 Pod 需要两个独立的 `2g.45gb` MIG 设备。每个设备标称 45 GB 显存，因此总标称容量是 90 GB，而不是原文所称的 45 GB。`2g` 表示两个 GPU slice；实际 SM 数由具体 GPU 和 profile 决定，不能用假设的 132 个总 SM 直接按 `2/7` 外推。

Kubernetes 只会在某一个节点能同时提供两个该扩展资源时调度此 Pod，因为单个 Pod 不能跨节点。但标准资源请求并不保证两个 MIG 设备来自同一块物理 GPU；它们可能位于同一节点的不同 GPU 上，具体取决于 device plugin 分配与拓扑策略。若没有任何单节点拥有两个空闲的 `2g.45gb` 资源，Pod 将保持 `Pending`，即使集群内多个节点合计容量足够。

这一约束说明 MIG profile 与资源数量需要按常见作业需求规划。例如，若许多作业请求两个 `2g.45gb` 设备，应确保足够多的节点能同时提供两个该资源。B200 的某些放置方式允许一块 GPU 创建三个 `2g.45gb` 实例，但是否要求同一物理 GPU 共置，需要额外的资源建模或分配策略，不能仅靠上述 YAML 保证。

> 单节点约束可能使 Pod 长期无法调度，即使所需 MIG 资源分散在集群的多个节点上。只有一个节点同时拥有全部请求资源时，该 Pod 才能被调度。

MIG 的运维代价之一是模式切换与实例重配置需要先清空相关工作负载，并可能要求 GPU reset；是否需要 reset 或节点重启取决于 GPU 代际和驱动。scheduler 通常不会按每个作业即时改变硬件分区，管理员会预先创建 profile，并在一段时间内保持该配置。

在 Kubernetes 中，NVIDIA GPU Operator 的 MIG Manager 可根据节点标签和配置策略应用期望的 MIG 状态。重启或驱动重载后，它可以重新协调配置，但这不等于所有 MIG 实例在硬件层天然跨这些事件保持不变。应把 MIG 分区视为受控生命周期操作，并在启动、升级和故障恢复后验证实际状态，再允许工作负载调度。

可以让一组 K8s 节点运行 MIG profile，另一组保持整卡模式，并通过标签、taint 和 affinity 放置相应 Pod。MIG profile 是显式配置的硬件分区，而不是普通 scheduler 自动生成的资源；实例可以通过管理流程动态创建或销毁，因此也不应称为绝对“静态”。

在适用的平台上，可结合持久化守护进程减少没有客户端时驱动状态被卸载的开销。不过，persistence mode 不负责创建 MIG 实例，也不能单独保证 MIG 配置跨 reset、重启或驱动重载持久化；MIG Manager 的期望状态协调和启动后验证仍然必要。

优化 Kubernetes 网络通信

在 Kubernetes 容器中运行多节点 GPU 工作负载时，各 Pod 需要相互通信。Kubernetes 网络模型要求每个 Pod 拥有可通信的 IP，但具体实现可能是原生路由、overlay、eBPF datapath 或其他 CNI 方案；Pod 间是否经过封装或 NAT 取决于 CNI、服务路径和集群网络设计，并非 Kubernetes 默认必然如此。封装、连接跟踪、MTU 不匹配或额外转发路径都可能增加开销。

对于部分性能敏感的 GPU 集群，host networking 是一种较直接的选择：Pod 共享节点的网络 namespace。在 Kubernetes Pod 规范中设置 `hostNetwork: true`；Docker 中可使用 `--network=host`。这种方式减少一层 Pod 网络虚拟化，但会增加端口冲突风险，改变 DNS、NetworkPolicy 和安全隔离行为，也不应作为所有集群的默认方案。

host networking 让容器使用主机的 IP 地址与网络接口，但不会自动向 Pod 提供 InfiniBand verbs/RDMA 设备，也不会绕过主机防火墙、路由、qdisc 或安全策略。MPI 作业可能因此减少显式端口映射和 overlay 路径；若要使用 RDMA，还必须为 Pod 分配相应字符设备、HCA/VF、驱动库和权限。

若不能或不应使用 host networking，需要确认 CNI datapath、MTU、NetworkPolicy、防火墙和服务发现能够承载 NCCL/MPI 的控制与数据流量。`net.ipv4.ip_local_port_range` 只设置 Linux 自动选择临时源端口的范围，并不等同于开放 NCCL 端口；应结合应用使用的监听端口和集群安全策略配置防火墙。`NCCL_SOCKET_IFNAME`用于选择 NCCL socket transport 所使用的网络接口，不控制防火墙，也不直接选择 RDMA HCA。应分别验证 bootstrap、socket 与 RDMA 数据路径。

使用 overlay 网络时，应测量延迟、吞吐、CPU 消耗、MTU 和丢包，并检查是否存在不必要的用户态代理、service mesh sidecar、加密或连接跟踪瓶颈。内核态或 eBPF datapath 可能减少一部分开销，但“在内核空间运行”本身并不保证低延迟。

在 Kubernetes 中启用 RDMA 时，可以部署 Kubernetes RDMA shared device plugin 或 SR-IOV Network Device Plugin，把 RDMA 设备资源暴露给 kubelet；再通过 Multus、SR-IOV CNI 或合适的网络方案向 Pod 附加接口。device plugin 本身不会创建 Pod 网络接口，也不会自动启用 GPUDirect RDMA 或保证零拷贝。

若使用 InfiniBand 或 RoCE，并且 GPU、NIC、PCIe 拓扑、驱动、内核模块和通信库均受支持，可配置 GPUDirect RDMA，使 NIC 直接 DMA 到 GPU 显存，避免数据经 CPU 主存中转。CPU 仍参与控制路径，且 Pod 需要获得相应设备与权限。该机制往往对多节点性能很重要，但是否启用以及收益大小必须通过拓扑检查和 NCCL/RDMA 测试验证。

减少 Kubernetes 编排抖动

运行 Kubernetes 这样的编排系统时，每个工作节点上都会有 kubelet、容器运行时、CNI、设备插件、日志与监控代理等后台进程。这些服务消耗 CPU、内存和 I/O，但不能统一估算为“单个核心的几个百分点”，也不能假定它们对 GPU 训练没有明显影响。实际开销取决于 Pod 创建/销毁频率、镜像操作、日志量、监控采样、网络 datapath 和故障事件。应测量节点级尾延迟和 CPU/I/O 抢占，并通过 `systemReserved`、`kubeReserved`、reserved CPUs、CPU Manager 与中断亲和性等机制为系统服务和工作负载划定资源。

若训练与推理工作负载在同一节点混部，执行时序与吞吐可能出现抖动；任何多租户组合都可能发生这种情况。另一个容器突发占用 CPU、内存带宽、存储、网络或共享 GPU 资源时，会影响训练或推理容器。应设置合理的 requests/limits、QoS、cpuset、I/O 与网络策略，并根据 SLA 使用 taint、affinity 或专用节点池隔离延迟敏感负载。

> 从系统调试与调优角度看，全训练或全推理等相对同质的节点通常比训练与推理混部更容易建立稳定基线。不过，同质并不自动消除资源争用；是否混部应根据利用率、成本、故障域和服务等级目标决定。

改善资源保证

为降低资源争用，Kubernetes 允许为容器定义 CPU 和内存 requests/limits，并把 Pod 中各容器的请求汇总用于调度。例如，可以声明训练作业请求 16 个 CPU 和 64 GiB 内存。CPU request 提供调度容量与相对权重，不会自动获得独占核心；独占 CPU 通常要求 kubelet CPU Manager 使用 `static` policy，Pod 满足 `Guaranteed` QoS，并为容器请求整数 CPU 且 CPU request 等于 limit。内存 request 用于调度，但不是专用物理内存分区。

Linux cgroup 按资源类型实施约束：容器持续超过 CPU limit 时通常会受到 CFS bandwidth throttling；超过 memory limit 且无法回收时，可能触发 cgroup OOM 并终止进程。requests 本身通常不是强制上限。对于性能敏感作业，应设置准确的 requests/limits，并在需要时配合 CPU Manager、Topology Manager、reserved CPUs 与 IRQ affinity；即使获得 exclusive CPUs，内核线程、中断或未正确隔离的系统任务仍可能占用这些核心。

后台内核线程和中断也是抖动来源，正如本章“调度器与中断亲和性”所述。如果其他 Pod 共享网络、磁盘或 PCIe 路径，它们可能产生大量中断、回写或内核工作，影响同一节点上的作业。CPU cpuset 只能解决其中一部分问题，还应设置 IRQ affinity、I/O QoS、网络策略并监控共享带宽。

若作业需要最稳定的尾延迟或重复性，可以使用 taint/toleration、node affinity 和调度策略为其提供独占 GPU 节点。若必须混部，应使用 CPU、memory、I/O、PID 等 cgroup 控制器和设备分区机制谨慎划分资源。标准 Kubernetes GPU extended resource 通常分配整数设备，不提供单块整卡内部的显存或算力 limit；这类硬分区需要 MIG 等机制。

Kubernetes 可以在满足上述前提时提供独占 CPU 分配，但不会因为 Pod 声明了 CPU 与内存 request 就保证“专用核心和专用内存”，也不能完全阻止主机级系统任务干扰。应检

查实际 cpuset、NUMA placement、cgroup 配置和节点保留资源，并通过压力测试验证隔离效果。

> 对性能和一致性要求极高的 Kubernetes 作业，申请整节点或使用专用节点池通常最容易控制干扰，但并非所有作业都应占用节点的全部 CPU 与 GPU。即使整节点分配，kubelet、runtime、DaemonSet 和系统服务仍可能运行。应根据 SLA、利用率和成本，在整节点隔离与精细资源分区之间取舍。

内存隔离与避免 OOM Killer

如果没有正确配置，内存也会发生资源干扰。Kubernetes 通过 Linux cgroup 实施容器内存约束，并使用 requests 进行调度。没有 limit 的容器可能占用大量节点内存。之后系统是回收页缓存、发生匿名页交换、触发 kubelet eviction、cgroup OOM 还是全局 OOM，取决于可回收内存、swap/NodeSwap 配置、cgroup v1/v2 策略和节点压力；不能简单断言一定会换出到磁盘。

无上限容器过度占用节点内存时，可能引发节点级内存压力。kubelet 可根据 eviction threshold 和 Pod QoS/用量驱逐 Pod；若某个 cgroup 达到 memory limit，内核可能在该 cgroup 内触发 OOM；若整个节点耗尽内存，还可能发生全局 OOM。受影响的进程不一定就是最初增长最快的容器，因此必须通过容量规划和隔离避免到达这一状态。

Linux OOM 选择依据包括 `oom_score_adj`、进程内存占用与其他因素；Kubernetes 会根据 Pod QoS 和相对 request 的用量设置相应保护倾向，并不只是“杀掉最大的 Pod”。故意不给训练或推理容器设置 memory request/limit 并不是可靠的保护方法：这样的 Pod 可能成为 `BestEffort`，在节点压力下最先被驱逐，同时也可能损害其他工作负载。

应根据实际峰值与测量结果设置准确的 memory request，并为不可避免的波动留出经过验证的余量。limit 过紧会在长作业峰值时触发 cgroup OOM；limit 过高或缺失则可能让节点暴露于更大风险。还应设置节点 system/kube reserved、监控 working set 与 memory events、建立告警，并为大型长作业考虑专用节点或 admission policy。

> 在传统容器级资源规则下，没有设置 CPU/内存 requests 或 limits 的 Pod 属于 `BestEffort`，在节点资源压力下驱逐优先级最低。要获得 `Guaranteed` QoS，Pod 中每个容器通常都必须为 CPU 和内存同时设置 request 与 limit，且对应值相等。只设置较高 limit 通常得到 `Burstable`，而不是 `Guaranteed`。启用 Pod-level resources 等新特性的集群可能有补充规则，应以当前 Kubernetes 版本文档为准。

处理 I/O 隔离

截至本书写作时，Kubernetes 没有像 CPU 和内存 requests/limits 那样通用、稳定的原生 Pod 块 I/O 带宽或 IOPS API。Linux cgroup v2 提供 `io.max`、`io.weight` 等控制，但 Kubernetes 核心 API 不会自动把普通 Pod 资源声明转换为这些限制。PersistentVolume、CSI 驱动或云盘 StorageClass 可能提供存储系统级 IOPS/吞吐配置，那属于具体驱动与后端能力。

若需要避免同一 GPU 节点上的重 I/O 工作负载相互干扰，可以使用专用本地盘或节点、存储后端 QoS、不同 StorageClass，以及由容器运行时/systemd 或受控节点代理配置 cgroup v2 I/O controller。直接手工修改 kubelet 管理的 cgroup 层级可能在 Pod 重建后失效，并非可靠的声明式方案。还要注意，Kubernetes 的 CPU scheduling 和 requests 只能管理一部分 CPU 争用，并不能“防止”所有 CPU 或 I/O 干扰。

容器与主机共享内核和硬件电源管理策略。例如，主机 CPU governor 设为 performance 后，同一主机上的容器观察到的硬件行为也受该策略影响；这不是把配置值复制或“继承”到容器。云 VM、托管 Kubernetes 或受限容器中，租户可能没有权限修改宿主机 governor、块设备调度或其他节点级设置。

因此，应在节点层完成并验证关键主机调优。部分 namespaced `sysctl` 可通过 Pod securityContext 设置；unsafe sysctl 需要 kubelet 显式允许。HugeTLB 资源可由 Kubernetes 请求，但大页预留、CPU governor 和许多设备设置仍需节点管理员管理。通常可通过基础 OS 镜像、systemd、MachineConfig、Node Tuning Operator 或专用 DaemonSet/operator 实施。NVIDIA GPU Operator 可管理驱动、Toolkit、device plugin、DCGM 与部分 GPU 配置，但不是通用的节点 `sysctl` 管理器。

关键要点

以下是本章的关键要点，涵盖操作系统、驱动、GPU、CPU 和容器层的优化：

数据与计算局部性至关重要

> 尽量让数据在靠近计算单元的位置存储和处理。可使用本地 NVMe SSD 缓存等高速存储来降低延迟，并减少对远程文件系统或网络 I/O 的依赖；同时要根据容量、一致性和数据复用方式设计缓存。

实施 NUMA 感知配置与 CPU 亲和性

> 让关键进程、CPU 核心、主机内存和 GPU 尽量位于同一 NUMA 节点，以优化 CPU 到 GPU 的数据流。可使用 `numactl` 和 `taskset` 等工具设置亲和性。正确绑定能减少远端内存访问，但不能保证完全消除；应通过拓扑检查和性能测量验证延迟与吞吐收益。

提高 GPU 驱动与运行时效率

> 根据工作负载调节 GPU 驱动与运行时设置。例如，可使用 persistence daemon 保留设备驱动状态，减少冷启动开销；可评估多进程服务（MPS），让一块 GPU 上资源需求互补的多个进程并发执行；多租户环境若需要硬件资源隔离，则可评估 MIG。三者用途不同，不能相互替代。

有效预取并批量处理数据

> 提前预取数据，并把小型 I/O 操作合并为更大、更高效的读取，以持续向 GPU 供给数据。可结合 PyTorch DataLoader 的 `prefetch_factor` 与 `num_workers` 预先准备多个批次；参数过大也会增加主机内存、进程和 I/O 压力，因此应实测调优。

数据加载时使用锁页内存

PyTorch DataLoader 设置 `pin_memory=True` 后，会尝试把返回批次放入锁页主机内存，使后续主机到 GPU 复制具备更高效传输和异步执行的条件。要让数据加载、复制与模型执行真正重叠，还需使用 non-blocking copy、合适的 CUDA stream 和足够的流水线并行；锁页内存也受系统上限和主机容量约束。

优化内存传输

可使用锁页内存减少主机到 GPU 传输的额外 staging，并评估大页以降低 CPU 侧 TLB 与页面管理开销。大页本身不会直接提高 PCIe/NVLink 复制带宽；异步传输能否与计算重叠，还取决于硬件 copy engine、stream、依赖关系和工作负载。

重叠通信与计算

尽可能将梯度同步、数据 staging 等通信或内存操作与仍可独立执行的 GPU 计算重叠，以减少等待时间。收益受依赖关系、通信库、stream 调度和共享带宽竞争限制，应通过时间线分析确认实际重叠程度。

调优并扩展网络栈

在多节点环境中，可使用支持 RDMA 的 InfiniBand 或 RoCE 网络，并根据实际 transport 调节 MTU、拥塞控制、接口选择、中断亲和性和相应缓冲区。TCP 缓冲区只影响 socket 路径，不应与 RDMA 参数混为一谈。应使用 NCCL/RDMA 基准和生产 collective 验证训练与推理吞吐。

利用容器与编排保持一致性

可以使用 Docker/containerd、NVIDIA Container Toolkit，以及配有 GPU Operator 和 device plugin 的 Kubernetes，统一容器内 CUDA 用户态库、框架和应用代码。主机内核驱动仍由节点管理，必须满足容器兼容性要求。这些组件可提供资源发现和拓扑 hint，但 CPU-GPU 亲和性与硬件拓扑放置仍需 CPU/Topology Manager、调度策略和节点配置协同完成。

控制容器运行时开销

容器能够提高可复现性并简化部署，但应正确配置 CPU/GPU 亲和性、存储、网络和资源隔离。host networking 不是默认最佳方案；应根据 CNI 性能、安全与端口冲突权衡，并测量端到端开销。

采用编排与调度最佳实践

Kubernetes 等编排系统可管理资源、故障恢复和工作负载生命周期。Topology Manager 是 kubelet 侧的 NUMA hint 协调器，不能保证把具有高速 NVLink 互连的 GPU 聚在一起；这需要 device plugin、节点标签、自定义 scheduler 或整节点分配等机制共同实现。

通过自适应与扩缩容保持灵活性

编排层可以分配工作、管理副本并协调跨节点作业生命周期。这种灵活性有助于扩展训练任务，也能应对数据负载和请求模式变化显著的推理场景；分布式 gang

scheduling、数据分片和模型并行仍需专门的 operator、训练框架或作业控制器支持。

持续、渐进地调优

系统级优化并非一次完成。应持续监控吞吐、尾延迟、利用率、功耗与故障指标；随着工作负载变化，重新调整 CPU 亲和性、批量大小和预取设置。逐项基准测试并保留回退方案，避免把同时发生的变化误认为累积收益。

降低整个技术栈中的瓶颈

最终目标是让操作系统、CPU、存储、网络、GPU 驱动与运行时协调工作。消除 CPU 内存分配或驱动初始化等某一层瓶颈，只有在该层确实位于关键路径时才会改善端到端性能；应以性能分析数据决定优化优先级，并用训练时间、服务 SLO 和总成本验证结果。

综合运用这些策略，可以减少不必要的数据移动和等待，并在可靠性、隔离、功耗与成本约束下提高硬件的有效利用率，从而改善训练和推理效率。

结论

本章说明，即使使用最先进的 GPU，周边环境的低效仍可能限制整体性能。经过合理调优的操作系统、容器运行时、集群编排器和软件栈，是高性能 AI 系统的重要基础。通过 NUMA 感知的 CPU/内存亲和性与合适的本地存储改善数据和计算局部性，让可并行的通信与计算重叠，并针对主机系统和 GPU 驱动进行测量驱动的调优，可以降低部分延迟并提高端到端吞吐量。

应把整个系统视为相互依赖的整体：CPU、内存、GPU、存储、网络、容器、编排器和编程栈都可能位于关键路径。启用持久化模式或优化 CPU 调度等调整单独看似细微，在大型 GPU 集群上累积后可能节省时间和成本；但前提是它们确实解决了已测量的瓶颈。对大型 Transformer 训练和复杂推理流水线而言，目标不是追求一个抽象的 100% 利用率，而是在可靠性、服务等级、功耗和成本约束下提高有效工作比例。

随着模型与系统规模持续增长，系统级调优会变得更加重要。本章介绍的技术帮助性能工程师和系统架构师识别数据移动、资源放置、初始化、隔离和调度方面的限制。正确应用这些方法，可以缩短迭代周期并改善 AI 部署的成本效率；是否获得收益仍需使用真实工作负载、代表性数据和端到端指标验证。

最后，硬件与软件栈虽然包含大量相互关联的配置，但不应一次性随意调整所有参数。应持续监控性能与可靠性指标，建立可重复的基线，每次改变少量因素，并逐层验证吞吐、尾延迟、goodput、功耗和成本。让数据决定优化优先级，保留回退路径，才能把真正的瓶颈转化为可验证的效率收益。

第 4 章。 分布式网络通信调优

现代 AI 系统需要在 GPU、存储和网络接口之间高效、低延迟地移动数据。本章介绍用于训练的 NVIDIA Magnum IO 组件（例如 NCCL、GPUDirect RDMA 和 GDS），以及用于解耦式推理的 NIXL，并结合现代 GPU 与 NVL72 等集群讨论其应用。你将了解这些库及其支持的底层硬件如何共同构成超大规模 AI 系统的数据传输基础。

在大规模系统中，即使 GPU 本身很快，低效的通信以及内存或磁盘数据传输仍会限制整体性能。本章将讨论加速数据传输、合理划分数据、直接利用高速存储子系统，以及在 GPU 上重叠通信与计算的高级模式。通信与计算重叠是 AI 系统性能工程中反复出现的重要方法。

我们将借助 NVIDIA I/O 加速平台 Magnum IO 的组件来说明通信与计算重叠，其中包括 NCCL、GPUDirect RDMA 和 GPUDirect Storage（GDS）。本章将展示如何使用这些库，在受支持的硬件和正确拓扑配置下，降低通信延迟、减少 CPU 数据搬运开销，并提高多节点、多 GPU AI 系统各层的有效吞吐量。

PyTorch 等高层 AI 框架可以调用这些底层库，让部分计算与通信并行执行。把这些技术纳入 AI 系统，需要从应用、运行时、网络和存储整体考虑，可用于加速超大模型训练的数据流水线，也可支持扩展分布式推理服务；具体规模和收益仍取决于工作负载与部署条件。

这些优化并不能自动确保每个组件都达到峰值性能。性能工程师需要测量并配置网络与存储 fabric，在吞吐、尾延迟、可靠性和资源竞争等约束下，提高 GPU 有效利用率和 goodput（有效吞吐量）。

重叠通信与计算（流水线）

重叠通信与计算，也称为*流水线化*，是构建高效大规模训练和推理系统的重要方法。其目标是让 GPU 持续执行有用工作，减少等待数据的时间。

核心思想是在依赖关系允许时，让数据传输与正在进行的计算并发执行，使一个任务完成时，下一阶段需要的结果已经在传输或已经到达。PyTorch 等现代框架支持异步操作，因此梯度 all-reduce 等集合通信有机会与计算任务并行。如图 4-1所示，这可以减少 GPU 空闲时间并提高系统吞吐；但只有当通信和计算使用的资源、stream 与依赖关系允许并发时，时间线才会真正重叠。

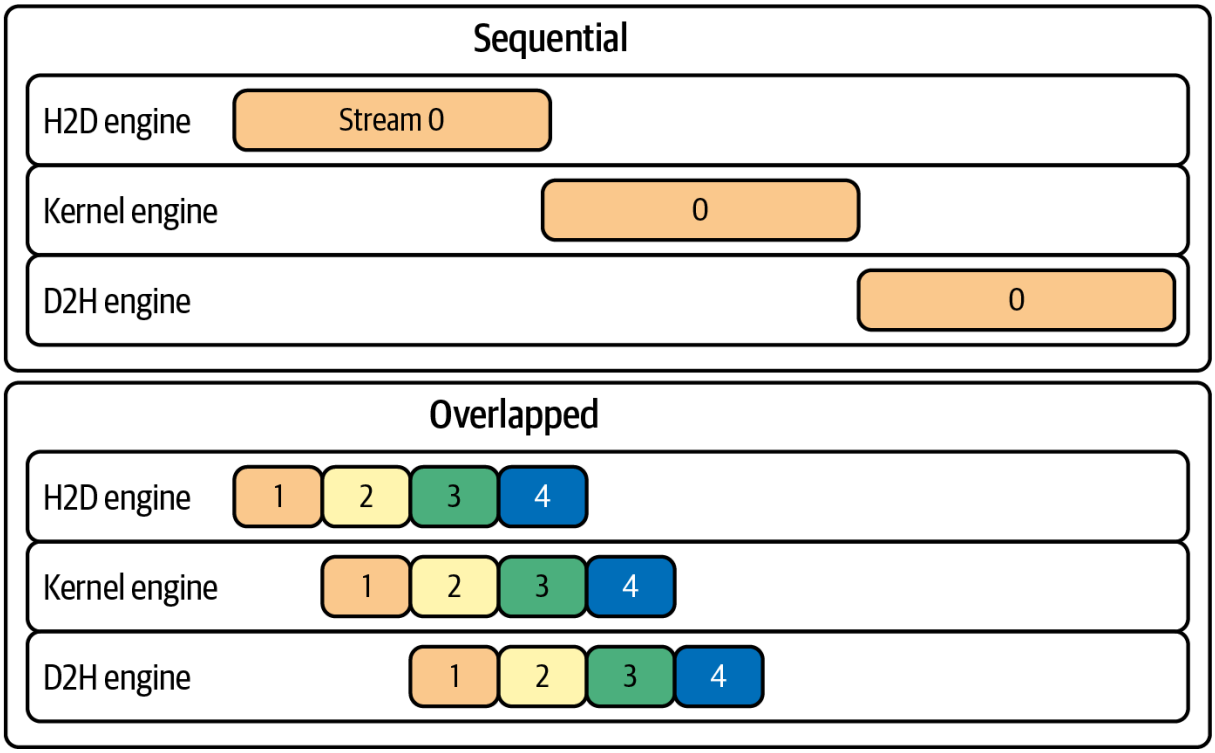


图 4-1。 在多个 CUDA stream 0–3 上，将主机到设备（H2D）及设备到主机（D2H）通信与计算重叠

基于 CUDA 的库可以利用多个 CUDA stream。例如，一个 stream 执行计算密集型矩阵乘法，另一个 stream 处理梯度聚合等通信任务。神经网络各层完成计算后，已就绪的梯度或输出可立即进入聚合或后续处理。若硬件支持并发、操作位于不同 stream 且同步依赖正确，这种重叠就能减少不必要的等待并维持较稳定的数据流。

增加两次通信事件之间的计算量，可以降低通信开销在总时间中的相对占比。较大的批次通常意味着同步交换之前能完成更多计算，但也会增加显存占用，并可能影响收敛、延迟和最大可用并行度。

在分布式训练中，常见做法是梯度累积：先处理多个微批次，再执行一次参数更新或梯度同步。降低通信频率可以摊薄每次交换的固定开销并提高吞吐，但通信数据总量是否降低取决于实现，累积步数也会改变有效批大小和优化行为。

压缩也可缩短通信占用的时间窗口。比如，发送梯度前对其量化、稀疏化或编码，可以减少网络传输的数据量，从而降低链路占用并缓解拥塞。

不过，压缩本身并不会产生重叠，而且还会引入编解码计算、额外缓冲和潜在数值误差。只有节省的传输时间超过这些成本，并且压缩操作能够正确调度时，端到端性能才会改善。

现代深度学习框架还会把大型张量通信拆分为较小的 bucket，以促进重叠。例如，PyTorch DistributedDataParallel 通常按 bucket 聚合梯度；某个 bucket 的梯度全部就绪后即可启动 all-reduce，而不必等所有参数的梯度都计算完成。具体就绪顺序与 bucket 行为取决于模型反向图和框架配置。

合理调整 bucket 大小和传输调度，可以提高重叠程度，减少通信阻塞计算流水线的情况。bucket 过小会增加启动和调度开销，过大则会推迟通信开始。PyTorch profiler 和 NVIDIA Nsight Systems 可用于查看计算、复制与集合通信是否实际重叠，从而依据时间线调节参数。

较大批次、梯度累积、异步传输、压缩和 bucket 化可以组合使用，以减少同步次数和空闲时间。但这些方法不能消除物理网络上限，也不保证最优 GPU 利用率；需要同时测量训练收敛、显存、带宽、尾延迟和 goodput。

在适用的工作负载中，这些优化可以缩短训练或推理时间，并提高硬件资源的有效使用率。成熟的通信库和框架让工程师无需重复实现底层网络协议，但拓扑选择、正确性验证和高层参数调优仍是系统工程的一部分。

使用 stream 异步执行

实现重叠的基础是异步执行。GPU 支持多个 stream，即有序操作队列。不同 stream 中的操作在依赖关系允许、且硬件执行资源或 copy engine 具备并发能力时，才可能并发或重叠。例如，一个 stream 可执行矩阵乘法等计算内核，另一个 stream 可排入数据复制或 all-reduce 等通信操作；放入不同 stream 本身并不保证并行。

把工作分配到不同 stream 并使用异步 API，可以让主机在通信完成前继续提交其他工作。例如，可把 all-reduce 排入通信 stream，而不在主机端立即等待其 GPU 执行完成。这里的“后台”是指设备时间线上的异步进展，并非通信不消耗 GPU、网络或内存带宽。

与此同时，默认 stream 可继续处理与该通信没有数据依赖的计算。NCCL 等库的集合操作通常会把工作排入给定 CUDA stream，并在操作实际完成前把控制权返回主机；这与某些 API 所称的 nonblocking communicator 初始化或错误处理模式不是同一个概念。程序必须用 CUDA event、stream wait 或框架依赖跟踪，在消费者真正使用通信结果前建立正确同步。

实践中，AI 框架会隐藏其中许多细节。PyTorch `DistributedDataParallel`（DDP）在反向传播中注册 autograd hook，把参数梯度组织为 bucket；当某个 bucket 中的梯度就绪时，reducer 会安排相应的异步集合通信，使其有机会与其余梯度计算重叠。具体 stream、bucket 重建、通信 hook 和同步行为会随 PyTorch 版本及后端实现而变化，不应假定所有 all-reduce 都固定运行在一个公开可控的“专用通信 stream”上。

> 后续章节将深入介绍 CUDA stream。目前只需记住：stream 可用于表达独立工作及其依赖，为通信与计算重叠创造条件，并帮助避免代码中不必要的同步点。

使用 CUDA stream 交错安排计算与通信，可以形成级联流水线，并在关键路径之外隐藏一部分通信延迟；它不能保证 GPU 始终繁忙，因为执行仍可能受依赖、带宽竞争、负载不均或网络延迟限制。应避免在热路径中不必要地调用 `torch.cuda.synchronize()`。`torch.Tensor.item()` 把标量结果返回 CPU 时，也会等待生成该值所需的 GPU 工作，因此可能打断流水线，但不应笼统描述为所有情况下都执行完整设备同步。性能测量优先使用正确记录在 stream 上的 CUDA event；若测量端到端墙钟时间，则可在测量边界同步，而不是在生产训练的每次迭代中无条件加入全局同步。

降低通信频率与数据量

如前所述，在每次通信之间完成更多工作，有时可以提高计算/通信比和整体效率。梯度累积是一种常见方法：先在本地累积多个微批次的梯度，再执行一次跨 rank 同步。对于 PyTorch DDP，需要在非同步微批次使用 no_sync() 或等效机制；仅多次调用 backward 而不抑制 DDP hook，通常仍会在每个微批次触发梯度通信。

例如，每 4 个微批次同步一次，理论上可把 all-reduce 调用频率降为原来的四分之一，使两次同步之间执行更多计算。但有效全局批大小通常会增大，并可能影响优化器超参数、收敛、激活/梯度内存和故障恢复粒度。是否值得采用，应在通信时间、显存、吞吐与模型质量之间测量权衡。

另一种减少通信量的方法是压缩或量化交换的数据。梯度量化、低精度通信或稀疏化可能减少每次发送的字节数，从而缩短链路占用时间，为重叠留下更多空间。但压缩/解压也需要计算和缓冲，误差可能累积；稀疏表示还会产生索引及不规则通信开销。通常需要误差反馈等算法设计，并通过最终模型质量验证，不能默认“不会显著影响质量”。

PyTorch DDP 的 bucket 机制把多个参数梯度组合为较大的通信缓冲区，以摊薄每次 collective 的启动开销。bucket 大小需要权衡：较大 bucket 通常更容易获得较高链路效率，却会等待更多梯度就绪后才启动；较小 bucket 可以更早通信，但会产生更多 NCCL 调用和调度开销。

撰写本书时，PyTorch DDP 的 bucket_cap_mb 默认值为 25 MiB（API 参数名虽含 mb，文档按 MiB 描述），而不是严格的十进制 25 MB。这个默认值只是通用起点，并不保证多数模型都能得到最佳重叠。大型层、多小层、动态图、梯度就绪顺序和网络特性都会改变最优值；应使用 profiler 比较不同 bucket 配置的迭代时间、通信带宽和显存占用。

在实践中实现最大程度的重叠

为了说明通信与计算重叠可能带来的收益，下面比较两种情形：一种是在全部计算结束后同步执行梯度通信，二者不重叠；另一种使用 DDP，让部分梯度通信有机会与反向计算重叠。示例以两块 GPU 模拟一个简单的分布式训练步骤，但它主要用于说明时间线概念，并不是完整基准。

第一种情形不使用 DDP 内置的重叠机制。每个进程先在本地计算全部梯度，再在反向传播结束后对各参数梯度执行 all-reduce。可以使用 PyTorch distributed primitive，在 `loss.backward()` 之后显式调用 `dist.all_reduce`，形成计算完成后才开始通信的串行时间线。

下面的代码在单个节点上尝试启动两个进程，分别对应 GPU/rank 0 和 GPU/rank 1。它运行前向与反向传播，再手动对两个进程的梯度求和并除以 world size。代码保持原文不变；运行时仍需通过环境或启动器提供 MASTER_ADDR、MASTER_PORT 等 env:// rendezvous 参数，且单 GPU fallback 也需要有效的进程组初始化环境。

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.distributed as dist
import torch.multiprocessing as mp
```

```python
# A simple model with multiple layers to produce multiple gradients
class MultiLayerNet(nn.Module):
    def __init__(self, size):
        super().__init__()
        self.fc1 = nn.Linear(size, size)
        self.fc2 = nn.Linear(size, size)
        self.fc3 = nn.Linear(size, 1)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

def train_no_overlap(rank, world_size):
    dist.init_process_group("nccl", init_method="env://",
                            world_size=world_size, rank=rank)
    torch.cuda.set_device(rank)

    # Each rank synthesizes its own data
    # (avoid sending big tensors via spawn)
    batch_size = 256
    data = torch.randn(batch_size, 1024, device=rank)
    target = torch.randn(batch_size, 1, device=rank)

    model = MultiLayerNet(data.size(1)).to(rank)
    optimizer = optim.SGD(model.parameters(), lr=0.01)

    # Forward + backward (manual, no overlap)
    output = model(data)
    loss = nn.functional.mse_loss(output, target)
    loss.backward()

    # Synchronous gradient all-reduce after backward
    for p in model.parameters():
        dist.all_reduce(p.grad, op=dist.ReduceOp.SUM)
        p.grad /= world_size

    optimizer.step()
    dist.destroy_process_group()

if __name__ == "__main__":
    import torch.multiprocessing as mp
    mp.set_start_method("spawn", force=True)
    world_size = min(2, torch.cuda.device_count() or 1)
    if world_size > 1:
        mp.spawn(train_no_overlap, args=(world_size,), nprocs=world_size,
                 join=True)
    else:
        train_no_overlap(0, 1)
```

每个进程独立计算 `MultiLayerNet` 的梯度。`loss.backward()` 返回后，代码逐个参数执行 all-reduce，再除以 world size 得到平均梯度。DDP 内部同样需要归约梯度，但会按 bucket 组织通信并在反向传播过程中尽早启动；这里则等全部反向计算完成后才逐个归约，因此没有计算/通信重叠，而且多次小 collective 本身也可能增加启动开销。

假设前向加反向计算耗时 10 ms，所有梯度通信单独耗时 12 ms，在完全串行且忽略其他开销时，总时间约为 22 ms。理论上的完全重叠下界接近两者最大值 12 ms，但这个下界通常不可直接达到：梯度只有在反向传播逐步生成后才能通信，早期计算之前没有可用梯度，最后一个 bucket 之后也可能留下通信尾部，计算与 NCCL 还会争用 GPU 和内存带宽。因此必须从实际时间线测量可隐藏的比例。

用 Nsight Systems 分析无重叠示例，预期会先看到反向计算内核，再看到显式 NCCL all-reduce；两阶段之间较清晰。在通信阶段，SM 并非严格“空闲”，因为 NCCL 的 GPU kernel 也可能使用 SM、copy engine 和内存系统，但此时没有模型计算与之并发。

下面改用 PyTorch DDP。DDP 在反向传播中跟踪梯度就绪状态，并按 bucket 安排归约，使通信有机会与其余反向计算重叠。代码通过 `DistributedDataParallel` 包装模型来处理同步。原文行内标识符中的排版用零宽字符已清除；此外，该程序清单引用 os.environ 却缺少 import os，实际运行前必须补充。

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.distributed as dist
import torch.multiprocessing as mp

class MultiLayerNet(nn.Module):
    def __init__(self, size):
        super().__init__()
        self.fc1 = nn.Linear(size, size)
        self.fc2 = nn.Linear(size, size)
        self.fc3 = nn.Linear(size, 1)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

def train_ddp(rank, world_size):
    rank = int(os.environ.get("LOCAL_RANK", rank))
    torch.cuda.set_device(rank)
    dist.init_process_group("nccl", init_method="env://",
                            world_size=world_size, rank=rank)
    torch.cuda.set_device(rank)

    model = MultiLayerNet(1024).to(rank)
    ddp_model = nn.parallel.DistributedDataParallel(model, device_ids=[rank])
    optimizer = optim.SGD(ddp_model.parameters(), lr=0.01)

    # Each rank makes its own data
    batch_size = 256
    data = torch.randn(batch_size, 1024, device=rank)
    target = torch.randn(batch_size, 1, device=rank)

    output = ddp_model(data)
    loss = nn.functional.mse_loss(output, target)
    loss.backward()  # DDP overlaps gradient all-reduce on background stream
    optimizer.step()
    dist.destroy_process_group()

def main():
    world_size = min(2, torch.cuda.device_count() or 1)
    mp.set_start_method("spawn", force=True)
    if world_size > 1:
        mp.spawn(train_ddp, args=(world_size,), nprocs=world_size, join=True)
    else:
        print("Only one GPU present; running DDP demo with world_size=1")
        train_ddp(0, 1)

if __name__ == "__main__":
    main()
```

使用 DDP 后，梯度同步由 `DistributedDataParallel` 处理，用户代码更简单。调用 `loss.backward()` 时，DDP reducer 跟踪 bucket 中梯度的就绪状态，并在整个 bucket 就绪后安排 all-reduce。对该顺序网络而言，反向传播通常先计算输出侧 `fc3` 的梯度，再依次处理 `fc2` 和 `fc1`；原文称 `fc3` 最后计算、而 `fc1`/`fc2` 已先完成归约，顺序写反了。实际 bucket 边界不一定按单层划分，第一轮还可能根据梯度就绪顺序重建 bucket。

如果模型很小，全部梯度可能只落入一个 bucket，通信接近反向末尾才开始，重叠空间有限。更大模型可能产生多个 bucket，但更大 batch 并不会自动增加 bucket 数；它通常增加每层计算量，从而可能扩大隐藏固定梯度通信的计算窗口。是否获得显著收益仍取决于 bucket、网络、计算/通信比例和资源竞争。

分析 DDP 时间线时，可能看到 NCCL all-reduce 与反向计算内核交错或并发，而不是严格的先计算、后通信两阶段。理想时间线中的通信应尽量位于其余反向计算之下；仅看到锯齿状交替并不等同于真正重叠，还需检查时间区间是否并发以及关键路径是否缩短。

较好的重叠通常表现为：迭代总时间小于独立测得的计算与通信时间之和，通信等待较少，且 collective 与可并行计算在时间线上实际并发。表 4-1给出概念性比较，其中数值不是这两段代码实测所得。

表 4-1。 重叠带来的潜在收益（示意）

指标	无重叠（手动同步）	重叠（DDP）	说明
反向传播与通信总时间	100%（基线）	约为基线的 70%	例如，重叠使每次迭代快约 30%（仅作示意）
通信开始时间	反向传播完成后	反向传播期间	DDP 可在部分 bucket 就绪后开始通信
通信阶段的模型计算空档	存在：反向结束后只执行 all-reduce	可能较少：通信与其他层计算并发	DDP 可能隐藏部分通信延迟
SM（GPU）利用情况	可能较低（通信期间没有模型计算）	可能较高（活动更连续）	NCCL 自身也可能占用 SM；需以 profiler 为准
实现的重叠比例（被计算覆盖的通信比例）	0%（串行执行）	约 50%（或更高）	粗略示意；实际比例取决于模型、bucket 和系统

注意：所有指标表中的数值都用于说明概念，并非上述代码的实测结果。不同 GPU 架构上的基准结果见本书 GitHub 仓库。

表中假设重叠可使迭代时间缩短约 30%，但这不是示例代码测得的结果，也不能推出更大训练作业必然获得更大收益。更大模型既可能提供更多重叠机会，也可能加重网络、显存和负载不均瓶颈。PyTorch `DistributedDataParallel` 的 `bucket_cap_mb` 默认值为 25 MiB。调整该参数可能改善特定模型的重叠；较大 bucket 通常推迟通信启动，不只是增加“最后一个 bucket”的延迟，而较小 bucket 会增加 collective 启动开销。

核心结论是：经过分析和调优的 DDP 可以隐藏一部分梯度通信。通常至少存在通信尾部，因为最后就绪的 bucket 之后没有剩余反向计算可用于覆盖；此外，较早 bucket 的通信也未必都能隐藏。研究者还在探索将优化器步骤与通信重叠、张量分区等方法。DDP 的基本策略与*无等待反向传播*（wait-free backpropagation，WFBP）思想相近：对梯度分组，并在 bucket 就绪后尽早启动归约。相关研究给出了更系统的分析。

某些编码模式会无意间削弱重叠。例如，在反向传播和下一次迭代之间读取尚未就绪的设备结果，会让主机等待相应 GPU 工作；是否阻塞后续设备工作取决于调用位置、依赖与提交方式。

> 应减少在热路径中把未就绪张量从 GPU 物化到 CPU 的操作，例如频繁调用 `.item()` 后立即打印或记录。该调用必须等待产生目标值所需的设备工作，可能使主机提交路径停顿并破坏流水线。调试日志的影响取决于频率和位置，不应一概而论；可降低采样频率、批量记录，或在明确同步边界后读取。

不能简单地把 `.item()` 这类需要返回 CPU 值的操作“移到另一个 CUDA stream”来消除等待；主机最终仍需等数据可用。更实用的做法是减少同步读取、延迟消费、批量日志，并在需要异步 D2H 时使用锁页缓冲区、独立 stream、event 和正确的缓冲区生命周期管理。应尽量减少手动调用 `torch.cuda.synchronize()`，只在正确性或明确的测量边界需要时使用。PyTorch 通常会跟踪算子依赖，但与外部库、自定义 stream、主机读取及缓冲区复用交互时，用户仍可能需要显式同步。

总之，重叠计算与通信是分布式性能工程中的重要技术。正确使用时，它能把部分通信延迟隐藏在有用计算之后，从而缩短关键路径；实际收益必须通过时间线和端到端迭代时间验证。

后续小节将介绍支持这些机制的软件和硬件基础设施，以及如何在现代系统上评估实际重叠程度。下一节概览 NVIDIA Magnum IO 栈，其中包含支持计算、网络与存储数据路径协同的技术。

NVIDIA Magnum IO 优化栈

NVIDIA 的综合 I/O 加速平台 Magnum IO汇集多种技术，用于改善 GPU、CPU、存储与网络接口之间的数据移动、访问和管理。Magnum IO 架构可概括为存储 I/O、网络 I/O、在网计算与 I/O 管理四个组成部分，如图 4-2所示。

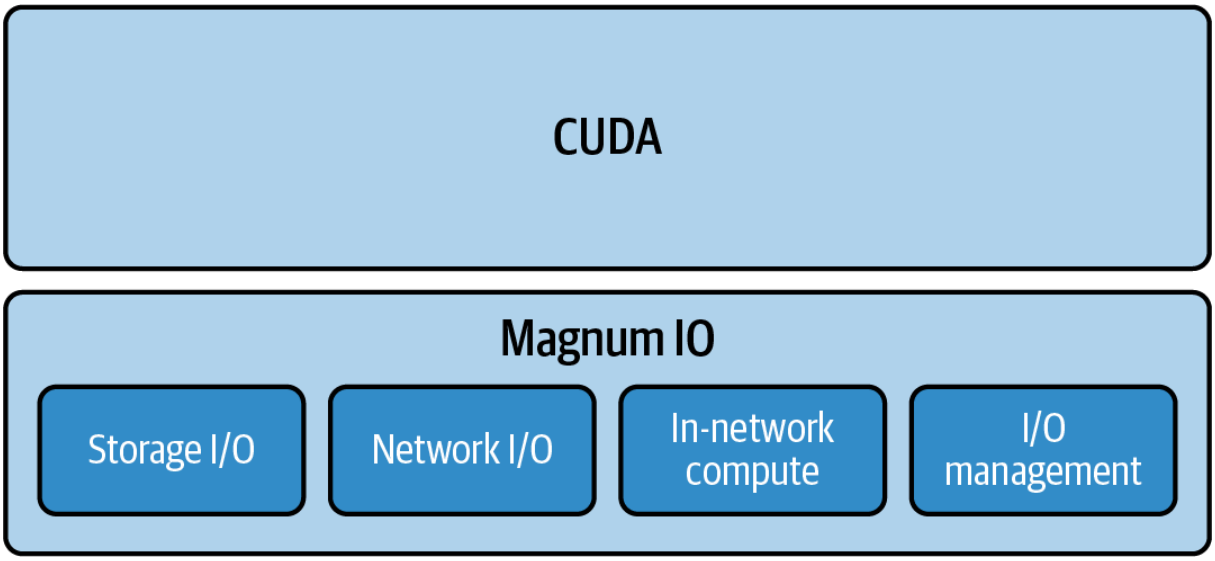


图 4-2。 NVIDIA Magnum IO 加速平台的四个组成部分

图 4-2中的四个组成部分如下：

存储 I/O

相关技术包括 NVIDIA GPUDirect Storage（GDS）和 BlueField SNAP。在受支持的存储、文件系统、驱动与拓扑上，GDS 可在 NVMe SSD 等存储与 GPU 内存之间建立直接 DMA 数据路径，减少经主机 CPU 内存 bounce buffer 的不必要复制；CPU 仍参与 I/O 提交与控制路径。第 5 章将深入讨论 GDS。

网络 I/O

相关技术包括 GPUDirect RDMA、NCCL、NVSHMEM、UCX 和 HPC-X（MPI/SHMEM 软件包）。它们分别在不同抽象层支持跨节点 GPU 通信。GPUDirect RDMA 可在兼容配置下让 NIC 与 GPU 内存直接进行 DMA，绕过主机内存 staging；这并不意味着 CPU 完全不参与节点间通信的控制、连接建立和进度管理。

在网计算

SHARP 可在 Quantum 系列 InfiniBand 交换机中执行在网归约，部分归约运算由交换机芯片完成。BlueField DPU 可卸载网络工作，并可承载 Subnet Manager、SHARP Aggregation Manager 等控制服务。当启用 NCCL RDMA SHARP 插件，fabric 具备兼容的 SHARP 固件和活动 Aggregation Manager，而且集合操作符合支持条件时，部分归约可卸载到 IB 交换机，从而减少主机或 GPU 承担的通信处理。后文将进一步介绍 NVIDIA SHARP。

> 以太网 GPU 集群可通过 RoCEv2 等技术提供 RDMA，但不具备 InfiniBand SHARP 这一特定交换机内归约机制。InfiniBand 与 Ethernet/RoCE 在拥塞控制、运维生态、成本、可扩展性和在网能力上各有取舍；不能仅根据是否支持 SHARP 判断哪一种互连更适合超大规模 AI。SHARP 对受支持的 collective 和拓扑可能带来收益，但应在目标 fabric 上验证，并考虑资源共享、公平性与回退路径。

I/O 管理

> NVIDIA NetQ 和 Unified Fabric Manager（UFM）等工具属于这一类别，为数据中心 I/O fabric 提供遥测、诊断与生命周期管理能力。具体支持范围分别面向相应的 Ethernet 或 InfiniBand 产品与版本。

这些组件通过 API 和 SDK 协同工作，隐藏一部分底层复杂性。Magnum IO 的目标是改善大规模 AI 与数据分析工作负载的端到端数据路径和吞吐量；它不是单一运行时，也不能自动保证所有组件达到峰值性能，其功能会随硬件与软件版本演进。

Magnum IO 生态还支持 NVLink Switch 网络域，用于机架内或 NVLink fabric 域中的 GPU 通信，并利用 InfiniBand（Quantum-2、Quantum-X800 系列）和 Ethernet（Spectrum-X）平台的能力。实际通信开销取决于系统代际、拓扑、路由、软件配置和工作负载。

本章后续将深入介绍用于网络传输的 RDMA、用于 collective 的 NCCL、用于推理数据移动的 NIXL，以及用于数据加载的 GDS，并讨论如何分析和调优这些组件。

使用 RDMA 实现高速、低开销数据传输

RDMA（远程直接内存访问）面向低延迟、高吞吐数据传输。它允许 RDMA NIC 直接读写已注册的应用内存，绕过传统内核网络栈中的大部分逐包处理和中间复制，从而减少上下文切换、缓冲区复制和 CPU 数据路径开销。CPU 仍负责建立队列、注册内存、提交 work request 和处理 completion。应只在受支持且已验证的环境中选用 RDMA，并持续通过日志、计数器和微基准确认实际数据路径。

在 Docker 和 Kubernetes 等容器环境中，容器需要获得相应 RDMA 设备（例如 `/dev/infiniband`）、网络接口、驱动库和权限。否则 NCCL 可能回退到 socket transport，或使用非 GPUDirect 的主机 staging 路径。回退通常可在 NCCL 日志中发现，但应用未开启日志时可能只表现为吞吐下降。原文所举“数十 GB/s 降至数 Gb/s”不是通用幅度，应以目标链路基准为准。

另一个容器常见问题是 RoCE GID、网络 namespace、接口索引或 RDMA 资源配置与宿主机不一致，某些共享 RDMA 镜像尤其容易出现。GID 不匹配更常导致寻址、建链或选择错误端口；它并不必然直接“阻止 GPUDirect 内存注册”。应分别验证 RDMA 连接、GPU 内存注册和实际传输路径，避免把所有回退都归因于 GID。

> 应验证实际使用的是 GPUDirect RDMA。采用 legacy peer-memory 路径时，可用 `lsmod | grep nvidia_peermem` 并检查 `dmesg`；采用较新的 DMA-BUF 路径时，不一定加载 `nvidia_peermem`。端到端验证可启用 `NCCL_DEBUG=INFO` 检查实际 network plugin/transport，并在兼容版本中使用支持 CUDA 缓冲区的 RDMA perftest。还应结合 NIC/GPU 计数器与拓扑验证，避免隐蔽回退。

传输路径回退可能在 profiler 或通信基准中表现为吞吐下降、主机内存复制增加或 CPU 占用上升，但不一定达到一个数量级。监控应覆盖 NCCL transport 日志、链路吞吐、重传/拥塞指标、CPU 使用率及 GPU 通信时间。

NVIDIA GPU 的相关能力称为 GPUDirect RDMA。它允许 InfiniBand 或 RDMA over Converged Ethernet（RoCE）等 RDMA NIC 在两台服务器之间直接 DMA 访问 GPU 设备内存，绕过主机系统内存中的数据 staging，而不是“完全绕过 CPU”。RoCE 上的 GPU 间直接数据路径如图 4-3所示。

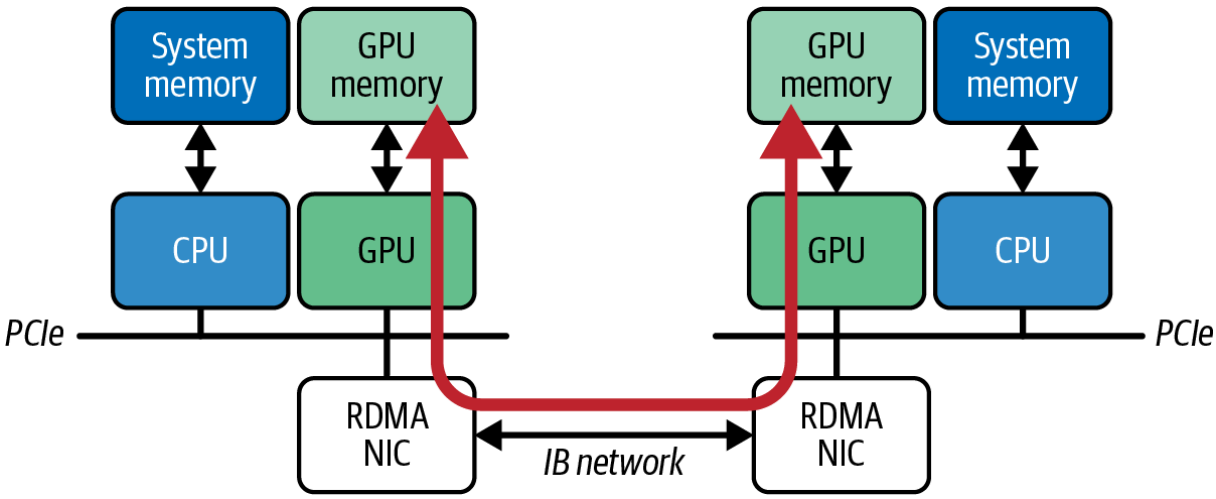


图 4-3。 使用 RoCE 的 GPU 到 GPU 直接数据传输

通过向 RDMA 子系统注册 GPU 缓冲区，GPUDirect RDMA 可支持 NIC 对 GPU 内存执行 RDMA read/write 等操作，并为通信库构建跨节点 GPU 数据路径。具体是否暴露单边语义取决于上层 API；NCCL collective 本身并不是用户直接发起的通用单边 RDMA 接口。该路径可降低多节点训练的数据复制和 CPU 开销，但收益依消息大小、拓扑和网络状态而定。

InfiniBand 原生提供 RDMA 传输，RoCE 则在 Ethernet 上承载 RDMA transport。要实现 GPU 直接路径，需要 NIC、交换网络、GPU、PCIe 拓扑、内核/用户态驱动和通信库共同支持并正确配置。驱动栈可以是发行版 inbox RDMA core、NVIDIA DOCA-OFED/MLNX_OFED 等受支持组合，不能笼统认为所有 InfiniBand/RoCE 都必须安装某一个“ NVIDIA OFED”。

RDMA 与传统 kernel TCP socket 路径之间可能存在显著性能差异。例如，现代 InfiniBand 在特定点对点小消息基准中可达到数微秒量级延迟，而 TCP over Ethernet 在相同硬件、消息和测量口径下可能更高。原文所称 5–10 倍不是通用比例；内核 bypass、NIC、交换跳数、拥塞、CPU 与软件栈都会影响结果。

对受网络带宽限制的大消息，现代 InfiniBand 或 RoCE 链路可提供数百 Gb/s 的标称速率，并在调优后获得较高有效吞吐。TCP 同样可以运行在 200/400 GbE 甚至更高速率上，并可利用多队列、offload、零拷贝等机制；它的有效性能不能由“是否 RDMA”单独决定。比较时应使用相同线速、消息大小、并发、NUMA 位置和端到端协议栈。

在分布式深度学习中，大型梯度或激活通信经常更关注带宽，但小 collective、控制消息、pipeline stage 和延迟敏感型推理仍可能由启动延迟主导。高带宽与低延迟都重要，其相对权重取决于消息大小分布和并行策略。

如果使用 Ethernet，应根据应用需求选择足够的链路带宽、低延迟拓扑和合适 transport。例如，在其他条件相近时，正确配置的 200 Gb/s 以上 RoCE 网络通常比 10–25 Gb/s TCP 网络更适合大规模 all-reduce，但这是线速和协议栈共同造成的差异，不能作为公平的 RDMA/TCP 对照。较大 MTU 可减少大流量下的包数和逐包开销，但 MTU

9000 不是无条件最低要求；端点、交换机、VLAN/隧道和整个路径必须一致，否则会产生丢包或连接问题。应按 fabric 建议和基准选择 MTU。

对实际使用 TCP socket transport 的路径，还应检查 TCP 自动调优和 socket buffer 上限，例如 Linux `net.core.rmem_max/wmem_max` 与 `net.ipv4.tcp_rmem/tcp_wmem`。所需窗口取决于带宽时延积、并发连接和内存预算；盲目调大可能增加内存占用。若 NCCL 已使用 InfiniBand/RoCE RDMA transport，这些 TCP 参数不会调优其 RDMA 数据路径。

对 TCP 路径，还需根据 RTT、丢包、队列与网络管理策略选择拥塞控制算法。在经过良好设计的专用集群网络中，默认 CUBIC 可能已经足够，也可能需要数据中心专用参数；CUBIC 的目标是在广泛网络条件下探测可用带宽并响应拥塞，并非因为网络“没有外部流量”就一定最优。

> 对带宽时延积较高的 TCP 路径，可以评估 Bottleneck Bandwidth and Round-trip propagation time（BBR）等算法，并按测得的 RTT、吞吐和公平性调整缓冲区。BBR 不保证在所有数据中心 fabric、队列管理和混合流量中优于 CUBIC，也不作用于 RoCE 拥塞控制。可用 `sysctl net.ipv4.tcp_congestion_control` 查看当前 TCP 设置，并通过可重复基准验证变更。

在云或混合环境中，需要确认实际网络路径、实例类型、placement 与配额。AWS EC2 Elastic Fabric Adapter（EFA）通过 libfabric 暴露 OS-bypass 能力，并使用 AWS Scalable Reliable Datagram（SRD）等传输机制；它不是 InfiniBand，也不应笼统称为标准 RDMA。cluster placement group 可改善实例间网络局部性，但可达带宽和 GPUDirect 支持仍取决于实例与 EFA 配置。若训练跨越本地数据中心与云端，应使用 Direct Connect、VPN 或其他明确的专用/受控连接；否则路径、延迟和拥塞可能不可预测。

> 多节点部署应使用经过验证、容量充足且可监控拥塞的网络。需要与云服务商或网络团队确认实例到实例、机架、可用区及混合连接的实际路径与限制。

即使使用 RDMA，CPU 仍负责控制路径、work queue 提交和 completion 处理，因此 CPU/NUMA 亲和性依然重要。可以根据实际进度模型，把 polling/progress 线程、应用线程和相关 IRQ 放在靠近 NIC 的 NUMA 节点，并尽量兼顾目标 GPU 的拓扑位置。例如，InfiniBand HCA 位于 NUMA 节点 0 时，可评估把其 IRQ 与通信线程约束到节点 0 的 CPU。不要把所有中断集中到单一核心；应结合 irqbalance、RSS/队列数量和 CPU 负载分散策略，通过延迟与吞吐测量验证。

多节点连接调优

在 GPU 分布式多节点训练中，需要避免网络成为关键瓶颈。这既要求选择适合的通信库和互连技术，也要求正确配置拓扑、transport、路由和资源亲和性。下面列出若干实践建议和常见风险：

了解拓扑

可使用 `nvidia-smi topo -m` 查看 GPU、NIC、CPU affinity 与互连的基础矩阵。对于基于 NVSwitch/NVLink 的系统，还应根据驱动版本使用 `nvidia-smi nvlink`、NVML/DCGM、fabric 管理工具和 Nsight Systems，了解链路状态、路由及多跳 fabric 连接。Nsight Systems 主要展示运行时通信时间线，并不单独提供完整物理拓扑清单。

利用可用的 NVLink Switch 域

NVIDIA GB200/GB300 NVL72 机架系统通过 NVLink Switch 将最多 72 块 GPU 组成一个 NVLink fabric 域，并提供很低的节点内/机架内互连延迟。原文所称约 130 TB/s 来自 72 块 GPU 各自约 1.8 TB/s 双向 NVLink 端口带宽的聚合（约 129.6 TB/s），不是任意 all-to-all 模式下单个作业可获得的吞吐量；“亚微秒”也取决于测量端点、消息和跳数。如果集群提供这种基础设施，应让需要高带宽互联的 rank 尽量位于同一 NVLink 域，同时仍需使用 InfiniBand/Ethernet 等 scale-out 网络与域外 GPU、存储和服务通信。Quantum 系列 InfiniBand 的具体端口速率与在网功能取决于交换机代际和配置，不能简单用 NVLink 与网络端口峰值直接比较。

在验证收益后使用 RDMA

在 InfiniBand 或支持 RoCE 的硬件上，应确认 NCCL 等通信库确实选择了预期 RDMA transport，并进一步确认是否启用了 GPU Direct 路径。NCCL 会根据插件、设备、拓扑和配置自动选择 transport，但“硬件可用”不代表必然使用 GPUDirect RDMA。配置错误时可能回退到 socket 或主机 staging。all-reduce 期间 GPU 利用率下降、CPU 占用上升只是线索，也可能来自负载不均、同步等待或协议线程；应结合 `NCCL_DEBUG`、NIC/GPU 计数器和微基准确认。

使用多 NIC 聚合可用带宽

部分服务器配有多个 NIC 或多个端口。NCCL 可以在多个 rail 上分配流量，以提高聚合带宽和拓扑局部性。`NCCL_NSOCKS_PERTHREAD` 和 `NCCL_SOCKET_NTHREADS` 只调节 socket transport 的 helper thread/socket 并行度，并不是 RDMA multirail 开关。多 NIC 也不必机械地位于不同子网；寻址和 rail 设计应符合 IB fabric、RoCE VLAN/路由及 NCCL 拓扑发现要求。两条 800 Gb/s 链路的标称聚合线速为 1.6 Tb/s，四条为 3.2 Tb/s，但 collective 有效吞吐还受 PCIe、GPU/NIC 比例、方向、协议和网络竞争限制。

利用受支持的直接 NIC 路径

应根据 GPU 数量和目标通信带宽配置足够的高带宽 NIC/rail，并让 GPU 尽量使用拓扑邻近的 NIC。NIC 通常经 PCIe 连接到主机 I/O 子系统，也可能与 DPU 协同。现代受支持系统中，NCCL 可利用 InfiniBand GPUDirect Async（IBGDA）等 GPU 发起的网络路径，如图 4-4所示。IBGDA 可减少 CPU proxy 在稳态数据路径中的参与和延迟，但初始化、内存注册、控制与错误处理仍需要主机软件，也不能单独保证达到链路线速。

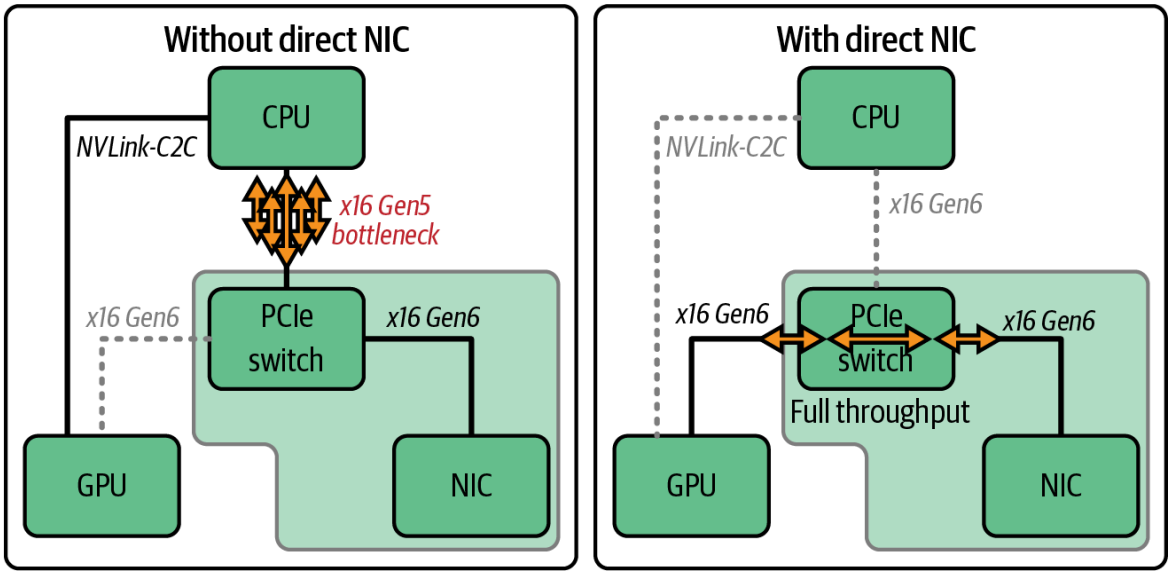


图 4-4。 通过 GPU 与 NIC 的直接数据路径减少 CPU 数据路径瓶颈

检查错误配置

网络配置不一致可能导致通信回退到更慢路径。例如，RDMA 不可用时，NCCL 可能改用 100 GbE 上的 socket transport；如果高性能接口选择错误，流量甚至可能进入 10 GbE 管理网络。可结合 NCCL 调试输出、路由/接口配置和实际端口计数器确认路径。`ibstat` 主要显示 InfiniBand CA/端口状态，`ifstat` 可显示接口流量；更细的 IB/RoCE 计数通常需要 `perfquery`、sysfs、`ethtool -S` 或厂商遥测。对预期 200–400 Gb/s 的数据路径，误用 10 Gb/s 管理网会形成严重瓶颈。

多节点通信常见陷阱

把训练扩展到集群中的多个节点会引入新的故障模式。下面介绍若干常见问题，并通过具体示例说明如何识别和处理。

陷阱 1：使用面向 CPU 的 Gloo backend，而非 NCCL

PyTorch distributed 支持多种通信 backend。对 NVIDIA GPU tensor 的多 GPU 训练，NCCL 通常是首选；Gloo 主要用于 CPU tensor，并通常通过 TCP 通信。显式创建 Gloo process group 后再期望执行 CUDA collective，可能不受支持、报错，或在某些构建/调用路径中涉及主机侧处理，不能假定总会正确地自动 staging。NCCL 初始化失败也通常会报错或挂起，而不会由 PyTorch 普遍、无声地自动切换到 Gloo。

如果应用实际对 CPU tensor 执行 Gloo collective，代码可能正常运行，却没有测到 GPU 通信路径。对于基于 CUDA 的 NVIDIA 多 GPU 训练，应明确验证 process group、tensor device 和 transport；显式指定 NCCL 有助于复现和诊断。现代 PyTorch 在省略 backend 时可根据 device type 注册或选择相应 backend，但具体行为随版本和 `device_id` 参数而变化，不应只凭默认值推断实际路径。性能差异也不一定固定为一个数量级。

下面的代码示意两个 rank 在不同节点上执行大 tensor all-reduce，并假设节点间有标称 800 Gb/s（100 GB/s）InfiniBand 互连。第一版故意使用 Gloo。不过代码实际分配的是

CPU tensor，因此它展示的是 CPU/Gloo collective，并不使用 GPU，也不能直接与 GPU/NCCL 的链路线速比较：

```python
# dist_allreduce.py

#!/usr/bin/env python

import os
import argparse
import torch
import torch.distributed as dist

def main():
    parser = argparse.ArgumentParser(description="Multi-node Gloo all-reduce")
    parser.add_argument(
        "--data-size",
        type=int,
        default=1024 * 1024 * 100,  # 100M floats ≈ 400 MB
        help="Number of elements in the tensor",
    )
    args = parser.parse_args()

    # Initialize the default ProcessGroup over env://
    # (uses MASTER_ADDR, MASTER_PORT, etc.)
    dist.init_process_group(backend="gloo", init_method="env://")

    rank = dist.get_rank()
    world_size = dist.get_world_size()

    # Allocate a large CPU tensor (Gloo is CPU-bound)
    tensor = torch.ones(args.data_size,
       dtype=torch.float32, device="cpu")

    # Warm up and barrier
    dist.barrier()

    # All-reduce (sum) across all ranks
    dist.all_reduce(tensor, op=dist.ReduceOp.SUM)

    # Barrier
    dist.barrier()

    ...

    dist.destroy_process_group()

if __name__ == "__main__":
    main()
```

代码显式选择 `backend="gloo"`，并在每个 rank 的 CPU 上分配约 400 MiB tensor（100 × 1024 × 1024 个 FP32 元素）。Gloo 对该 CPU tensor 执行 collective，数据路径通常是 TCP。若改成 CUDA tensor，是否支持或如何处理取决于 PyTorch/Gloo 构建与版本，不能笼统断言一定经 CPU staging。与 NCCL/GPUDirect RDMA 的公平比较需要相同数据语义、world size、计时边界和有效带宽公式。

因此，这个 Gloo 示例可用于验证 CPU process group，却不能作为 GPU collective 基准。若目标是比较 backend，应分别确认 tensor 位于何处、实际网络接口、是否发生 staging，并使用适合各 collective 的性能指标。

原文给出下面的示意输出，但程序清单省略了计时代码，因而无法由所示脚本复现：

```
Rank0: All-reduce of 400.0 MB took 200.00 ms (2 GB/s)
```

按单个约 400 MiB payload 除以 200 ms，简单 payload rate 约为 2.1 GB/s（若按十进制 400 MB 则为 2 GB/s）。但 all-reduce 的 bus bandwidth 还需考虑算法和 world size，不能把这个数字直接与 100 GB/s 单向端口线速比较。CPU 使用率接近 100% 也只是原文假设，示例没有 profiler 输出；应实际测量 CPU、NIC 和 collective 时间。

要测试 NCCL，不能只把 backend 改为 `nccl`；还必须把 tensor 放到每个 rank 对应的 CUDA device，正确设置 device/process group，并确认 NCCL transport 和 GPUDirect 条件。满足这些前提后，NCCL 可使用 GPU collective。原文给出另一条示意输出：

```
Rank0: All-reduce of 400.0 MB took 4.00 ms (100 GB/s)
```

400 MB / 4 ms 的简单 payload rate 为 100 GB/s，但这不证明 all-reduce 达到了 800 Gb/s 链路线速：collective 会在链路上传输与 payload 不同的总字节数，且 GB/GiB、单向/双向与算法带宽口径必须统一。该数字也不是所示代码实测。正确结论是：NVIDIA CUDA tensor collective 通常应使用 NCCL，并通过 `nccl-tests` 或等效基准报告 algorithm bandwidth 与 bus bandwidth，再确认是否使用 GPUDirect RDMA。

> 可调用 `torch.distributed.get_backend()` 查看指定 process group 的 backend；还需结合 tensor device、NCCL 日志和网络计数器验证实际 transport。

NCCL all-reduce 会使用 GPU kernel、copy engine、NVLink/PCIe 和网络 transport 的不同组合；跨节点 GPUDirect RDMA 并不是 GPU 直接寻址“另一块远端 GPU 内存”的单一硬件动作，而是由 NCCL 协议、NIC DMA 和 GPU 缓冲协同完成。示例中的 Gloo tensor 本来就在 CPU 上，所以 GPU 不参与该 collective；这不等同于证明 Gloo 自动把 GPU 数据经 CPU 缓冲转发。

> 在多 NIC 生产集群中，可以按网络设计设置 `NCCL_SOCKET_IFNAME`，选择 NCCL socket/bootstrap 使用的 IP 接口。InfiniBand HCA 的 IPoIB 接口不一定名为 `ib0`，RoCE 接口也常使用普通 Ethernet 名称；该变量只选择 socket 接口，并不直接选择 RDMA HCA 或保证随后使用 GPUDirect RDMA。应确保所有 rank 可通过所选 bootstrap 地址互达，并分别用 NCCL 日志验证 bootstrap 与数据 transport。

陷阱 2：NCCL 版本不一致

PyTorch wheel/conda 环境可能依赖随包分发的 NCCL，也可能因构建方式和动态链接搜索路径加载系统 `libnccl.so`。单个进程最终加载的是某个具体库，不是让“bundled NCCL 与 system NCCL 同时对抗”；真正风险是容器/节点之间加载版本不同、ABI 或 network plugin 不兼容、以及 `LD_LIBRARY_PATH` 覆盖预期依赖。这些问题可能导致初始化

错误、hang 或性能路径变化，但不保证无声回退。可用 `torch.cuda.nccl.version()` 查看 PyTorch 报告的运行时版本（原文示例中的空元组不是正常版本值），并用动态链接诊断、NCCL 日志和各 rank 环境清单确认实际库。应固定一致的 `nvidia-nccl-cu*`/框架包或按受支持方式针对系统 NCCL 构建，避免这些兼容性问题。

陷阱 3：NCCL bootstrap 期间 TCP 端口耗尽

NCCL 的 out-of-band bootstrap 和 socket transport 会使用 TCP 连接。在并发作业和连接数很高时，若本地临时端口范围过窄、存在大量 TIME_WAIT 连接，或防火墙/NAT 施加额外限制，可能出现连接失败或超时。可通过 `sysctl net.ipv4.ip_local_port_range` 查看范围，并根据节点并发、其他服务、安全策略和可观测数据扩展；原文示例 `50000 51000` 只有约 1001 个端口，不能称为普遍适用的“宽范围”。端口范围也不控制远端监听端口或绕过防火墙。应结合 socket 状态、NCCL 日志和编排系统诊断bootstrap 失败，而非预防性地在所有集群套用同一数值。

陷阱 4：网络带宽不足或 NIC 配置错误

另一个多节点问题是：同步数据量超过可用网络容量，或作业没有使用所有预期接口。随着 GPU 数量和单卡计算吞吐增加，这类瓶颈更容易暴露。Blackwell 系统能否饱和单节点 400 Gb/s 链路取决于模型、并行策略、消息大小和计算/通信比，不是单凭 GPU 型号即可保证。

在网络受限时分析工作负载，通常会看到跨节点扩展效率下降、每 GPU 吞吐降低、collective 时间增长或计算等待通信。还应排除数据加载、负载不均和同步尾部等其他原因，再检查链路利用率、错误、拥塞与路由。

`nvidia-smi dmon` 可采集 GPU 利用率、功耗、温度、显存活动及版本支持的部分 PCIe 指标，但不是通用 NIC/NVLink fabric 吞吐监控器。NIC 字节、包、错误和队列计数可查看 `ethtool -S <iface>`、`ip -s link show <iface>`、RDMA/IB 端口计数器或厂商遥测；`iftop`、`nload` 等可观察部分实时接口流量。应确认工具是否能看到 RDMA offload 流量以及所用 namespace。

如果单条链路已饱和且服务器具备额外 NIC/rail，可评估 NCCL 多 NIC 路径，同时确认 GPU-NIC 拓扑、PCIe 容量和 fabric 路由。`NCCL_NSOCKS_PERTHREAD` 与 `NCCL_SOCKET_NTHREADS` 调节的是 NCCL socket/TCP transport，而非 InfiniBand/RDMA 多 rail。

在 socket transport 中，从平台默认值适度增加这些变量有时能提高单接口或多接口利用率。例如，可以测试 `NCCL_NSOCKS_PERTHREAD=2` 与 `NCCL_SOCKET_NTHREADS=2`，即每个连接方向使用相应 helper 并行度；这并不意味着“两块 NIC 因此各获两个连接”。不要任意增大。NVIDIA 文档对支持版本规定两者乘积上限为 64，更多线程和 socket 也会增加 CPU、内存与调度开销；应核对当前 NCCL 文档和平台默认值。参考资料

`NCCL_NSOCKS_PERTHREAD` 和 `NCCL_SOCKET_NTHREADS` 仅在 socket transport 中相关。对 IB/RDMA 路径，应首先检查 fabric 健康、GID/端口、GPU Direct、拓扑与 NCCL network plugin，再按需要配置 `NCCL_IB_HCA`、`NCCL_IB_GID_INDEX` 等版本相关策略。现代 NCCL 可自动选择 GID；手动固定错误索引反而会破坏连接。

如果确认使用 socket transport，可逐步测试这些线程相关设置（例如 2、4、8），持续测量吞吐、CPU 和尾延迟。过多线程会争用资源并产生收益递减。所有变更都应通过 NCCL 调试日志和通信基准验证。

陷阱 5：慢节点或慢进程

在同步式多节点训练中，collective 和 step 边界往往需要等待所有参与 rank，因此关键路径常由最慢节点、GPU 或进程决定。一台机器的链路降速、热降频、数据加载变慢或被其他任务争用，都可能拖慢整个作业；异步或流水并行系统的传播方式不同，但慢 rank 仍会形成背压。

为减少慢 rank，可尽量使用同构 GPU、CPU、NIC 和交换路径，并为关键训练作业提供明确的资源保证。云环境中混用实例类型、网络能力或 placement 可能增加差异，但同构硬件也不能消除共享网络、温度、纠错、存储和软件状态造成的抖动；需要持续测量每个 rank 的阶段耗时。

可使用 DCGM、IB/RoCE 端口计数器、应用阶段时间和系统遥测，定位 NIC link flap、重传、GPU 热/功耗降频、CPU/I/O 争用等问题。PyTorch `torch.distributed.monitored_barrier` 可帮助报告未按时到达的 rank，但其文档要求 Gloo process group，不能直接当作 NCCL collective 监控器；可为诊断创建单独 Gloo group。NCCL 异步错误检测的环境变量和语义随版本变化，应使用当前 PyTorch/NCCL 文档支持的配置、timeout 和 watchdog，并避免把错误检测误认为性能慢节点检测。

陷阱 6：UCX/RDMA 环境中的 GPU 内存碎片

PyTorch caching allocator 会保留已释放的 GPU block 供后续复用，因此 `memory_reserved()` 可能明显高于 `memory_allocated()`。这可能与真实峰值、碎片或其他进程共同造成 OOM，但不能仅凭差值推断 UCX/RDMA registration pool 已耗尽。UCX registration cache、NIC memory key 和 PyTorch allocator 是不同层的状态；需要分别查看 allocator snapshot、UCX 日志/统计和通信路径。下面示例只观察 PyTorch allocator，并未初始化或调用 UCX，因此不能证明标题所述因果关系。相关资料

```
import torch
import torch.distributed as dist
import os
import time

def log_mem(iteration):
    reserved = torch.cuda.memory_reserved()
    allocated = torch.cuda.memory_allocated()
    print(f"[Iter {iteration:02d}] Reserved: {reserved/1e9:.3f} GB, "
          f"Allocated: {allocated/1e9:.3f} GB")

def run(rank, world_size):
    # Standard DDP / UCX init
    dist.init_process_group(backend="nccl", init_method="env://")
    rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(rank)

    # Pre-allocate a big buffer that UCX will register once and hold
    big_buffer = torch.empty(int(2e8), device=rank)  # ~0.8 GB
    log_mem(0)
```

```
    for i in range(1, 11):
        # Simulate per-iteration tensor allocations of varying size
        small = torch.randn(int(1e7), device=rank)  # ~40 MB
        medium = torch.randn(int(5e7), device=rank) # ~200 MB

        # Free them explicitly to return to allocator cache
        del small, medium
        torch.cuda.synchronize()

        # Log memory after freeing
        log_mem(i)

        # Barrier so all ranks print in sync
        dist.barrier()
        time.sleep(0.1)

    dist.destroy_process_group()

if __name__ == "__main__":
    run(0, 1)
```

示例意在说明 allocator 缓存释放后的 block，使 reserved memory 在达到适合复用的容量后保持，而 allocated memory 随临时 tensor 删除而下降。不过输出与代码不一致：`big_buffer` 在整个函数中始终存活并占用约 0.8 GB，所以 `memory_allocated()` 不可能在各次迭代后降为 0。代码也没有证据表明该缓冲区被 UCX 注册。以下输出按原文保留，仅作示意，不能视为实际运行结果：

```
[Iter 00] Reserved: 0.800 GB, Allocated: 0.800 GB
[Iter 01] Reserved: 1.040 GB, Allocated: 0.000 GB
[Iter 02] Reserved: 1.240 GB, Allocated: 0.000 GB
...
[Iter 10] Reserved: 1.240 GB, Allocated: 0.000 GB
```

处理这类问题时，不应把“升级到最新 CUDA”视为通用修复；框架、驱动、CUDA、UCX 和 network plugin 必须使用经过验证的兼容组合。`torch.cuda.empty_cache()` 只把 allocator 中未占用的缓存 block 释放给其他 GPU 应用可用，不能释放仍存活的 tensor，也不能保证整理碎片或清空 UCX registration cache。应先用 memory snapshot/summary 区分真实活跃内存、缓存、泄漏和碎片，再按 allocator backend 调整 split/rounding/garbage collection 等受支持选项，并单独诊断 UCX 注册资源。

总结这些多节点陷阱：对 NVIDIA CUDA collective 通常选择 NCCL；验证预期的 RDMA/高速网络路径；按拓扑利用所需 NIC，而不是盲目使用全部接口；固定经过兼容性验证的 NCCL/CUDA/驱动/plugin 版本；监控 transport 回退、慢 rank 和内存状态。下一节将聚焦 NCCL 及节点内、节点间 GPU 通信优化。

使用 NCCL 进行分布式多 GPU 通信

NVIDIA Collective Communications Library（NCCL）为一组 rank/GPU 提供 collective 与点对点通信原语，是 NVIDIA 生态中许多多 GPU 训练框架的底层通信库。

NCCL 提供 all-reduce、all-gather、broadcast、reduce-scatter 等 collective 的优化实现，可用于从少量 GPU 到大型集群。原文所称“未来数百万 GPU”是扩展愿景，不是

当前已验证的单一 communicator 规模保证。多 GPU 训练与推理需要交换模型权重、梯度、激活等数据；NCCL 根据 collective、拓扑、消息大小和可用 transport 组织这些交换。

在典型数据并行训练中，每个 rank 对自己的数据分片计算梯度，再通过 NCCL all-reduce 聚合梯度，使各 rank 使用一致的归约结果更新模型。并非所有分布式训练都执行完整梯度 all-reduce；ZeRO/FSDP、张量并行和流水并行会使用 reduce-scatter、all-gather 或点对点通信等不同模式。

分布式推理中，GPU 也可能需要交换激活、KV cache 分片、路由结果和其他中间数据，具体通信模式取决于张量并行、流水并行、专家并行或解耦式服务架构。

部分推理框架使用 NCCL `send()`/`recv()` 或 collective，另一些系统通过 UCX、libfabric 或 NIXL 等库移动点对点数据。哪种方案具有更低尾延迟或更好重叠，取决于消息模式、连接管理、backend、内存注册和框架集成，不能作为普遍排序。

NCCL 针对 NVIDIA GPU 优化，可通过 PCIe、NVLink/NVSwitch、InfiniBand/RoCE network plugin 和 TCP socket 等路径通信。它会探测拓扑，并依据 collective、消息大小、协议和 cost model 选择 channel、算法与 transport；这不保证任意两块 GPU 的每次通信都自动采用物理“最快路径”，错误拓扑、容器可见性、插件或环境变量都可能改变选择。

较新的 NVIDIA Inference Xfer Library（NIXL）补充了面向推理数据移动的 API，尤其适用于解耦式推理中的 KV cache 等点对点传输。

NIXL 使用可插拔 backend 接入不同传输和存储层，包括与 POSIX 文件、GPUDirect Storage（GDS）相关的路径；Amazon S3 等对象存储支持取决于相应插件、版本和部署。这些 backend 可在内存层级、远端节点与存储之间移动 KV cache 分片。后文及推理调优章节将进一步介绍 NIXL。

> NIXL 面向异构内存/存储 backend 与解耦式推理数据移动，NCCL `send()`/`recv()` 则与 NCCL communicator、CUDA stream 和其 network transport 集成。两者抽象和适用场景不同。NIXL 在特定 KV cache 工作流中可能减少集成开销或改善尾延迟，但不能无条件断言它比 NCCL P2P 延迟更低；应在实际消息大小、并发、backend 与故障语义下测量。

NCCL 的拓扑感知

拓扑感知对 NCCL 性能十分重要。NCCL 探测 GPU、CPU、NIC、PCIe、NVLink/NVSwitch 等连接关系，并据此构建通信图。在 NVLink Switch fabric 中，GPU 间流量可使用 NVLink/NVSwitch 路径；具体路由、channel 和有效带宽仍由系统拓扑与 NCCL 版本决定。

NCCL 可使用 ring、tree、NVLS 等算法和协议，并会根据拓扑与消息大小选择通信图。某些多节点或多拓扑域 collective 会采用分层方式，例如先在节点内归约，再跨节点交换，最后在节点内分发。原文把这一过程描述为必然自动选择的单一“hierarchy algorithm”过于简化；实际选择由 NCCL tuner/cost model、硬件支持和配置共同决定，目标是减少慢链路流量并利用高带宽路径。

例如，GPU 0–1 通过 NVLink 相连，GPU 2–3 通过另一组 NVLink 相连，而两组之间只能经过较慢的 PCIe 路径。分层 all-reduce 可以先在各 NVLink 组内归约，再由代表 GPU 跨 PCIe 交换归约结果，最后在组内分发，从而减少跨组链路承载的数据。NCCL 是否恰好采用这种步骤，还取决于检测到的拓扑、算法实现和消息大小。

NCCL 的自动选择通常是合理起点，但不保证对每个模型、消息分布和拥塞状态都最优。可用 `NCCL_ALGO` 限制允许的算法，例如 `NCCL_ALGO=NVLS,NVLSTree,Tree,Ring,PAT`；可用值与 collective、NCCL 版本及硬件支持有关。手动覆盖适合故障诊断、可重复实验或经基准证实的特定部署，并应保留自动选择作为对照。

再考虑 4 块 GPU 分属两个 PCIe switch 的系统：GPU 0–1 位于一侧，GPU 2–3 位于另一侧。跨全部 GPU 的朴素单 ring 可能反复经过较窄的 switch 间路径；两组内先通信、再由代表 GPU 跨组交换的分层方案可能降低该路径压力。这只是概念示例，不能据此断言 NCCL 必然生成两个 ring；应查看 NCCL 拓扑/图日志和实际链路计数。

Nsight Systems、NCCL 日志和链路计数可用于判断通信阶段、算法和 transport。看到多个 NCCL CUDA kernel 并不能单独证明采用了分层拓扑算法，因为 channel、chunk 和协议也会产生多个 kernel。拓扑优化可能把每 GPU 有效带宽从几十 GB/s 提升到更高水平，但原文“数百 GB/s”的示例依 NVLink 代际、collective 口径、消息大小和 GPU 数量而定，必须使用 nccl-tests 等工具实测。

原文用一个示意场景比较两种配置：朴素方案的 SM busy 为 60%、迭代时间 100 ms；拓扑感知并重叠通信后，SM busy 为 90%、迭代时间降至 70 ms。表中没有基准代码或硬件说明，这些数值不能视为实测结论。通信等待也不一定表现为“warp 因 PCIe 数据传输而 memory stall”；GPU 可能在 collective kernel、同步依赖或空档中呈现不同 stall reason。表 4-2仅用于说明应比较的指标。

表 4-2。 应用 NCCL 拓扑感知及计算/通信重叠前后的 GPU 指标（示意值）

指标	优化前（无重叠）	优化后（有重叠）
SM busy	60%	90%
发生内存 stall 的 warp	多	显著减少
迭代时间	100 ms（无重叠）	70 ms（有重叠）

总之，拓扑可能决定多 GPU 扩展效率。一般应让高流量 collective 尽量使用可用的高带宽、低争用路径，例如节点内 NVLink/NVSwitch，并减少不必要的跨 PCIe switch、跨 NUMA 或跨较窄网络链路传输。但“最快路径”需结合并发、共享带宽和算法，而不是只比较标称峰值。

如果多 GPU 作业在单节点或单 fabric 域内扩展不佳，应先检查通信是否经过意外的慢路径。GB200/GB300 NVL72 中每块 Blackwell GPU 的 NVLink 5 端口聚合双向带宽标称约 1.8 TB/s，通常描述为 18 条约 100 GB/s 的链路；上一代相应聚合口径约 900 GB/s。这里是每 GPU 所有 NVLink 端口的双向聚合峰值，不是单个 peer、单方向或任意 collective 的持续有效带宽。

在非全连接 NVLink 拓扑中，某些设备对可能需要经过较少链路、NVSwitch 或 PCIe；跨 NUMA 的主机路径也可能降低吞吐。NVL72 的 72 块 GPU 位于一个 NVLink Switch fabric 域，提供面向全体 GPU 的高带宽连接和 NVLS 等 collective 支持。官方所称 full-bisection/all-to-all 是 fabric 级拓扑能力，不代表每一对 GPU 在所有并发模式下都独享每 GPU 1.8 TB/s；仍需考虑 switch 路由、collective、方向和共享资源。

NCCL 会根据拓扑与 cost model选择通信图，但应通过日志、时间线和链路计数确认。如果仍受拓扑瓶颈限制，可评估把作业放在连接更紧密的 GPU 子集上，或调整 rank/NIC 映射、算法和并行策略。

例如，4 块位于同一 NUMA 邻域或 NVSwitch island 的 GPU 可能比跨慢链路使用 8 块 GPU 获得更高的单 GPU 效率；但总吞吐是否更高取决于额外计算能力能否抵消通信开销，不能一概断言少用 GPU 更好。应同时比较 step time、总样本吞吐和成本。

NVIDIA Grace Hopper/Blackwell 超级芯片通过 NVLink-C2C 提供 CPU-GPU 一致性连接。特定平台标称 CPU-GPU 双向聚合互连带宽约 900 GB/s，使 GPU 能以一致性语义访问 CPU 内存，并简化部分数据移动。CPU LPDDR/DRAM 与 GPU HBM 仍是不同物理内存池，延迟和带宽不同，因此不能把 CPU 内存描述为与本地 HBM 等速的“高速 GPU 内存扩展”。

即使 CPU-GPU 互连很快，让 all-reduce 经 CPU 或系统内存也不自动达到旧 GPU-GPU 链路性能；还要考虑内存控制器、数据复制、协议、方向和竞争。关键仍是根据实际拓扑选择合适路径，并用 collective 基准和应用时间线验证。

> NVIDIA Nsight Systems、`NCCL_DEBUG=INFO`、当前版本支持的拓扑/图 dump 机制，以及 NVLink/DCGM 计数器可共同验证路径。`NCCL_TOPO_DUMP_FILE=<path>` 是否可用及具体语义取决于 NCCL 版本；日志或拓扑文件本身不能证明链路已“充分利用”，仍需测量吞吐与计数。

NCCL 通信算法

在内部，NCCL 会根据数据大小、GPU 数量和拓扑选择不同的通信算法。NCCL 用于集合通信的主要算法包括 Ring、Tree、CollNet 和 Parallel Aggregated Tree（PAT）等。下面逐一介绍：

Ring（环形）

在环形 all-reduce 中，GPU 在逻辑上排列成环。每个 GPU 以流水线方式向一侧的相邻 GPU 发送数据，并从另一侧接收数据。执行 all-reduce 时，数据的各个分块沿环传递，同时累积局部归约结果。环形算法能够均衡网络负载，使每个 GPU 发送和接收相同的数据量。对于给定数据大小和参与 GPU 数量的 all-reduce，每个 rank 传输的总数据量为 2 × `data_size` ×（`num_gpus` − 1）÷ `num_gpus` 字节；在经典分块模型中，每一步传输 `data_size` ÷ `num_gpus` 字节。因此，它在理想链路模型下具有良好的带宽效率。代价是启动步数随 GPU 数量线性增长，因为数据必须经过整个环。对于大消息，实际搬运字节的时间远大于启动各次传输的开销，所以 Ring 往往表现很好。这类负载称为*带宽主导型负载*。

Tree 和 NVLSTree

Tree 算法使用树形结构完成归约和广播。树形 all-reduce 通常由向根节点归约，再从根节点广播结果这两个阶段组成。对于 N 个 GPU，平衡树所需的通信阶段数为 O(log N)，而环形算法为 O(N)，因此 Tree 通常能为小消息提供更低延迟。但在处理大消息时，它未必能让所有链路始终保持繁忙，例如叶节点在某个阶段完成发送后便不再参与该方向的传输。NCCL 对 Tree 算法做了优化，并经常将其用于总耗时主要由传输启动延迟决定的小消息。这类负载称为*延迟主导型负载*，与 Ring 面向大消息的带宽主导型场景相对。`NVLSTree` 是使用 NVLink SHARP（NVLS）能力的树形算法选项；能否实际启用交换机内归约取决于 NCCL 版本、GPU/NVSwitch 平台及运行时拓扑支持。

CollNet（跨节点分层集合通信）

CollNet 是 NCCL 的分层集合通信路径，将高速本地域聚合与跨组交换结合起来。它组合两类集合通信策略，以优化不同规模下的通信。CollNet 面向网络能够协助执行集合操作的环境。在部署支持 NVIDIA SHARP 等技术时，部分归约可在网络交换结构中执行，从而减少必须穿越网络的数据量并降低端到端通信延迟。

它首先依据高速本地互连划分 GPU，例如将单个节点或一个 NVSwitch 域内的 GPU 归为一组，再通过本地通信路径聚合各组数据。随后，各组的代表端点参与跨组通信；具体的数据流和层次结构由 NCCL、CollNet 变体以及网络插件根据拓扑共同决定，并不一定表现为固定的“每组一个 leader GPU 加一棵树”。这种分层方法旨在兼顾节点内带宽和节点间扩展性，尤其适合大型多节点 GPU 集群，但实际收益取决于网络卸载支持和拓扑。

NCCL 提供多个 CollNet 变体，包括 CollNet Direct 和 CollNet Chain。它们采用不同的数据交换结构和流水线方式；实际可用性取决于 NCCL 网络插件及底层网络能力。

Parallel Aggregated Tree（PAT，并行聚合树）

PAT 是 NCCL 的并行聚合树算法。它以并行、流水线化的树形交换降低扩展到更多 rank 时的启动步数，最初主要用于 all-gather 和 reduce-scatter；可用的 collective 与性能特征会随 NCCL 版本演进。把 PAT 简单描述为“Ring 与 Tree 的混合体”，或声称它必然同时达到 Ring 的带宽和 Tree 的延迟，都过于绝对。更准确地说，PAT 通过切分消息并调度多个并行交换，在特定拓扑和消息规模下改善传统算法的延迟与带宽折中。

与其他通信算法一样，NCCL 的选择通常取决于 collective 类型、消息大小和拓扑。小消息往往更看重较少的通信阶段和较低的启动延迟，大消息则更看重持续带宽利用率。不过，“几十 MB”并不是固定分界线，阈值会随硬件、rank 数量、协议和 NCCL 版本变化。

默认情况下，NCCL 使用内部调优模型为给定的集合操作、消息大小和拓扑选择算法与协议。NCCL 还支持对称内存优化和低延迟内核，可降低 NVLink 互联系统上部分中小消息操作的延迟。在特定测试配置中，此类改进曾测得最高约 7.6 倍的加速，但这不是所有 collective、消息大小或系统都能达到的通用结果。这些优化与 PAT 等算法可以进一步降低 NVL72 等系统上的通信开销。

NCCL 的通信策略也在持续增强拓扑感知能力。例如，在受支持的平台和操作中，它可以利用 NVSwitch/NVLink 域的硬件多播能力更高效地分发相同数据。具体路径仍取决于硬

件代际、NCCL 版本和所选算法。

NCCL 也能利用较新的硬件卸载能力。在受支持的 InfiniBand 部署中，它可通过 NCCL RDMA SHARP 插件使用网络内 SHARP；在受支持的 NVSwitch 系统上，则可使用 NVLink SHARP（NVLS）执行交换机内集合通信卸载。两者是不同作用域的技术，也都需要匹配的软件栈和平台支持。

如前所述，PAT 通过对消息分块并并行调度聚合树上的数据交换，改善某些 collective 在较大 rank 规模下的通信效率。它并不意味着应用显式创建多个 CUDA stream，也不能概括为在所有硬件上都同时取得 Tree 延迟和 Ring 带宽；应以目标系统上的 NCCL 测试结果为准。

NCCL 在 communicator 初始化期间发现互连拓扑和 GPU 等信息，并在每次 collective 调用时结合消息大小等条件，通过调优模型选择算法与协议。这个选择旨在获得良好性能，但并不构成“始终选中最快组合”的保证。如果分析结果显示通信性能不理想，例如跨节点延迟异常偏高，可以通过 `NCCL_ALGO` 环境变量限制或覆盖候选算法。例如可设置 `NCCL_ALGO=NVLSTree,PAT`，但算法名称和支持范围具有版本依赖性，使用前应查阅对应 NCCL 版本文档并进行基准测试。环境变量必须在 NCCL 初始化之前设置；如果直接使用 NCCL API，则应在调用 `ncclCommInitRank()` 等 communicator 初始化函数前完成设置。

分布式数据并行策略

在实践中，对数十亿乃至数万亿参数模型进行大规模训练和推理，通常需要组合使用数据并行、张量模型并行、流水线并行、专家并行和上下文并行等策略。合理组合这些策略，有助于提高训练集群的扩展效率，避免通信、同步和调度开销浪费 GPU 资源；但线性扩展并非自动保证。

关键目标是在各个层次尽可能重叠通信与计算。例如，使用 NCCL 执行 all-reduce 等集合通信，并在适用的推理数据移动场景中使用 NIXL。此类机制可以支持大规模 GPU 集群，但可扩展规模和效率仍受模型分解方式、网络拓扑、负载均衡、故障域及软件栈限制；原文所称“高效扩展到数百万 GPU”并无普适依据。

> 在超大规模训练中，梯度累积和激活检查点等技术也常用于控制内存占用。不过它们并非没有代价：梯度累积会改变有效 batch 和同步频率，激活检查点则以额外重计算换取显存，因此应同时评估吞吐量和收敛行为。

在单节点上扩展到多个 GPU 时，PyTorch 在框架层提供数据并行（划分数据）和模型并行（划分模型）等方式。后续章节会进一步讨论这些方法；这里先从系统性能角度比较两种基础的数据并行 API：`nn.DataParallel`（DP）与 `torch.distributed.DistributedDataParallel`（DDP）。理解二者的差异很重要，因为选择不当可能显著影响性能：

数据并行（DP）

DP 是一个易于使用的 API，由一个进程控制多个 GPU。该模块把每个输入 batch 自动拆分到可用 GPU，在各副本上执行前向传播，再把输出收集到主 GPU。反向传播

时，各副本产生的梯度被归约到原始模块所在设备，供优化器更新主副本参数；下一次前向传播时又会重新复制模块。原文所称“平均梯度后再广播回其他 GPU”并不是 PyTorch `DataParallel` 每步更新的准确描述。

`DataParallel` 的训练循环运行在单个进程中，因此无需启动多个训练进程，接入较简单。它使用线程在不同设备上执行副本，但 Python 调度、scatter/gather、每次前向复制模块以及主设备上的输出和梯度归约都会形成额外开销。GIL 可能影响 Python 侧调度，但不能单独解释全部扩展瓶颈。“超过 2–4 个 GPU 就不能很好扩展”也不是严格边界；更准确的结论是，PyTorch 官方通常建议多 GPU 训练使用 DDP，即使只在单节点上运行。

全分片数据并行（FSDP）

FSDP 通过在各 rank 之间分片参数、梯度和优化器状态，避免每个 GPU 长期保存全部模型状态，从而显著降低内存开销。激活内存通常需通过激活检查点、选择性重计算或其他并行策略另行管理，并不是 FSDP 默认分片对象。对于超大规模模型，FSDP 常与张量并行、流水线并行等策略组合使用。

> 我们将在第 13 章讨论 FSDP 以及专家并行等其他超大规模训练技术。本节聚焦 DP 和 DDP，为后续章节中更复杂的内容打下基础。

分布式数据并行（DDP）

使用 CUDA 时，DDP 的推荐配置是每个 GPU 对应一个进程，并通过 NCCL 等后端同步梯度。与普通复制式数据并行类似，每个进程各自持有完整模型副本；FSDP 等分片方法则不同。

在反向传播期间，DDP 按梯度 bucket 触发归约，使各 rank 得到一致的梯度。bucket 的通信可以与后续梯度的反向计算重叠，但实际重叠程度取决于模型图、bucket 划分、梯度就绪顺序、后端和资源竞争，并非自动达到完全重叠。

DDP 使用独立进程，避免了多 GPU 副本共享同一 Python 解释器所带来的调度限制，并可通过 NCCL 执行高效通信。因此，多 GPU 训练中 DDP 通常优于 DP。PyTorch 开发者也建议使用 `DistributedDataParallel` 而不是 `DataParallel`。不过，性能差异来自进程模型、通信方式、复制与 gather 开销等多项因素，不能只归因于 GIL 这一瓶颈。

下面回到之前的例子，在单节点上比较 DP 和 DDP。我们将使用一个简单模型，分别执行一次训练步骤。

首先使用 DataParallel 包装模型，让 PyTorch 拆分输入 batch 并使用两个 GPU。下面是单次训练迭代的示例：

```
# before_dataparallel.py

import torch
import torch.nn as nn
import torch.optim as optim
```

```python
# Dummy model and dataset
class SimpleNet(nn.Module):
    def __init__(self, input_size, hidden_size):
        super(SimpleNet, self).__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.linear2 = nn.Linear(hidden_size, 1)
    def forward(self, x):
        return self.linear2(self.relu(self.linear1(x)))

# Setup model and data
input_size = 1024
hidden_size = 256

model = SimpleNet(input_size, hidden_size)

model.cuda()  # move model to GPU 0, it will also replicate to GPU 1

model = nn.DataParallel(model)    # utilize 2 GPUs (0 and 1 by default)

optimizer = optim.SGD(model.parameters(), lr=0.01)
data = torch.randn(512, input_size).cuda()   # batch of 512 on GPU0
target = torch.randn(512, 1).cuda()          # target on GPU0

# Run a single training step
output = model(data)                         # forward (DP splits data internally)
loss = nn.functional.mse_loss(output, target)
loss.backward()                              # backward (DP gathers grads to GPU0)
optimizer.step()
```

在这里，`nn.DataParallel` 的前向传播把形状为 [512,1024] 的输入张量（矩阵）拆成两个大小为 [256,1024] 的张量，一个送往 GPU 0，另一个送往 GPU 1。执行前向传播时，GPU 0 和 GPU 1 上会有同一模型的副本。原文写成“GPU 1 和 GPU 2”，与代码使用默认设备 0、1 的情形不一致。

> 严格来说，调用 `model.cuda()` 后，基础模块最初只位于 GPU 0；每次执行 `forward` 时，DP 会把模块复制到参与计算的 GPU（包括 GPU 1），再运行各副本的计算。它并不是只在第一次前向传播时复制后永久复用所有副本。

随后，`DataParallel` 使用线程在 GPU 0 和 GPU 1 上执行各模型副本，并把操作排入相应设备。Python 侧的复制、scatter、线程调度和 gather 会产生开销；实际 GPU kernel 在依赖允许时仍可并发执行。原文断言所有 kernel launch 都因 GIL 在 Python 中严格串行，过度简化了 PyTorch 调用进入 C++、释放 GIL以及异步 CUDA 入队的实际行为。

前向传播结束后，`DataParallel` 将各 GPU 的输出收集到主设备 GPU 0，以便计算 loss。反向传播会把各副本的参数梯度归约到 GPU 0 上基础模块的参数中，优化器随后更新这些基础参数。其他设备上的副本会在下一次前向传播时由更新后的基础模块重新创建；原文所称每步把聚合梯度从 GPU 0 广播回其他 GPU 并不准确。

这个 `DataParallel` 示例有几个值得注意之处。GPU 0 还承担输入分发、输出收集和梯度归约等额外工作，单进程调度与每次复制模块也会增加 CPU 和设备间传输开销。梯度归约路

径不像 DDP 的 bucket all-reduce 那样设计为随反向传播逐步重叠，因此 GPU 0 更容易成为瓶颈。下面比较 `DistributedDataParallel` 如何处理这些问题。

切换到 `DistributedDataParallel` 后，通常通过 torchrun 或 `torch.multiprocessing.spawn` 等方式为每个 GPU 启动一个进程。每个进程持有完整模型副本，并处理各自的一份 batch。下例中每个进程的本地 batch size 为 256；两个进程合计的全局 batch size 为 512，与前面的 DataParallel 设置相同：

```
# after_ddp.py

import os, time
import torch
import torch.nn as nn
import torch.optim as optim
import torch.distributed as dist
import torch.multiprocessing as mp

class SimpleNet(nn.Module):
    def __init__(self, input_size, hidden_size):
        super(SimpleNet, self).__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.linear2 = nn.Linear(hidden_size, 1)
    def forward(self, x):
        return self.linear2(self.relu(self.linear1(x)))

def train_ddp(rank, world_size):
    rank = int(os.environ.get("LOCAL_RANK", rank))
    torch.cuda.set_device(rank)
    dist.init_process_group("nccl",
       init_method="env://",
       world_size=world_size, rank=rank)
    model = SimpleNet(input_size=1024,
       hidden_size=256)

    model.cuda(rank)

    ddp_model =
       nn.parallel.DistributedDataParallel(model,
         device_ids=[rank])
    optimizer = optim.SGD(ddp_model.parameters(),
       lr=0.01)

    # Each process gets its own portion of data
    batch_size = 256
    data = torch.randn(batch_size, 1024).cuda(rank)
    target = torch.randn(batch_size, 1).cuda(rank)

    # Run one training iteration
    output = ddp_model(data)
    loss = nn.functional.mse_loss(output, target)
    loss.backward()
    optimizer.step()

    dist.destroy_process_group()

if __name__ == "__main__":
    main()
```

原则上，可以使用 `torchrun` 启动器（或集群的 MPI/SLURM/Kubernetes 集成）运行 DDP，并由启动器提供 rendezvous 所需的环境变量。对于下面的 torchrun --standalone 单节点命令，启动器会设置必要的 rank 和 rendezvous 信息，无需手工导出 `MASTER_ADDR` 与 `MASTER_PORT`。不过，上面的 Python 清单不能直接产生下述结果：它调用了未定义的 main()，没有计时代码或打印语句，并把传给 process group 的全局 rank 覆盖为 LOCAL_RANK，这在多节点上会造成 rank 重复。以下命令和输出仅作为原书示意保留；结果依工作负载与硬件而异。

```
# Environment (common gotchas)
export NCCL_DEBUG=INFO
export NCCL_ASYNC_ERROR_HANDLING=1
export NCCL_SOCKET_IFNAME=ib0      # use your HCA (e.g., ib0, ib1)
# Optional: for multi-rail IB, set NIC ordering
# so ranks use distinct rails.

# Local bring-up, 2 GPUs
torchrun --standalone --nproc-per-node=2 after_ddp.py

# SLURM (example)
srun --ntasks=$WORLD_SIZE --gpus-per-task=1 --nodes=$NNODES \
     --cpus-per-task=8 python after_ddp.py

### Output: ###

DDP step took 30.00 ms
```

在匹配良好的实现中，DDP 迭代通常会比 DP 更快。但原书给出的 DDP 30 ms 与 DP 45 ms 并非由所示清单测得，因为两个清单都没有计时或输出代码；因此“快 33%”只是无法从清单复现的示意数值，不能作为预期性能。

DDP 的优势来自多个方面。每个进程处理自己的本地 batch，避免多个 GPU 副本共享一个 Python 解释器；梯度按 bucket 通过 NCCL all-reduce 归约，并可与部分反向计算重叠；通信负载也分布在参与 collective 的各 GPU 上，而不是把输出和梯度集中到一块主 GPU。归约后的梯度在各 rank 上保持一致，不过是否“原地平均”取决于 DDP 的梯度除法语义、通信 hook 和后端实现，不宜把它当成固定底层步骤。

原书示例称 DDP 用时 30 ms、DP 用时 45 ms，并推断模型越大、GPU 越多，差距必然越大。实际差距取决于模型计算量、参数规模、batch、bucket、互连、输入流水线和 CPU 调度。DDP 本身也会受通信延迟与带宽、慢 rank、网络拓扑以及 host 开销限制，并不能仅凭改用 DDP 就扩展到数千 GPU。应在预热后用 CUDA event 或同步边界明确的墙钟计时，报告多次迭代统计和全局 batch 口径。

总的来说，多 GPU 训练通常应优先采用 `DistributedDataParallel` 而不是 `DataParallel`；PyTorch 团队也明确给出这一建议。DP 仍可用于快速原型或兼容旧代码，但是否可接受应由实现复杂度与实测性能共同决定，而不是固定限定为两块 GPU。

即使只在单节点上使用多个 GPU，也可以采用 `DistributedDataParallel`。通常使用 torchrun 为每个 GPU 启动一个进程；也可自行使用 `torch.multiprocessing.spawn`，但此时必须正确建立全局 rank、local rank、world size 和 rendezvous 配置。每个进程持有自己的模型副本和数据分片。

DDP 在反向传播过程中按 bucket 调度 NCCL all-reduce，从而为梯度通信与计算重叠创造条件，并避开 DP 的单进程多设备调度和主设备 gather 路径。这通常能改善 GPU 利用率与端到端迭代时间，因此重视性能的多 GPU 训练一般优先选择 `DistributedDataParallel`。

NCCL communicator 生命周期与环境配置陷阱

尽管 NCCL 抽象了大多数底层细节，在代码中使用 NCCL 的方式仍会影响性能。此外，NCCL 提供许多用于控制其行为的环境变量。错误配置可能降低性能，甚至造成通信 hang。

本节介绍 NCCL communicator 和环境设置中的常见陷阱，以及诊断和规避方法。

陷阱 1：过于频繁地创建 NCCL communicator

NCCL communicator 表示一组能够参与集合通信的 GPU 或 rank。通过 C++ 的 `ncclCommInitRank` 或 PyTorch 的 `torch.distributed.init_process_group` 初始化通信组通常代价较高。初始化过程需要交换 unique ID、地址等 bootstrap 信息，发现拓扑、建立 channel/算法所需状态并分配相关资源；具体步骤依 NCCL 版本和 transport 而异。

如果代码反复初始化 communicator，每次都会承担这部分成本。不过，“32 个 GPU 分别创建 32 个 communicator”并不等同于正常的一组 32-rank communicator：每个 rank 都会持有同一通信组的本地 communicator 句柄。原文给出的 2–3 分钟与 2–3 秒没有测试条件，不能视为通用值。初始化时间可能随 rank 数量、网络和 bootstrap 协调方式出现非线性增长，应在目标集群上测量。

在 PyTorch DDP 中，通常在程序开始时调用一次 `init_process_group`，再复用默认 process group。底层可能按设备、backend、子组或实现需要惰性建立一个或多个 NCCL communicator，因此不应把它严格理解为整个程序永远只有一个 communicator。

下面的 PyTorch 示例在每次训练迭代中初始化并销毁 NCCL process group，用来说明这种反模式的成本。清单按原书保留，但它缺少 import os，并错误地把 local rank 当成可传给 process group 的全局 rank；因此多节点上不能直接使用：

```
import torch
import torch.distributed as dist
import torch.multiprocessing as mp

def run(rank, world_size):
    rank = int(os.environ.get("LOCAL_RANK", rank))
    device = torch.cuda.device(rank)
    for i in range(5):  # simulate 5 iterations
        # This naive approach re-initializes NCCL each
        # iteration. THIS IS EXTREMELY SLOW AND NOT RECOMMENDED!!!
        dist.init_process_group("nccl", init_method="env://",
                                world_size=world_size, rank=rank)
        # do a tiny all-reduce to simulate some work
        tensor = torch.ones(1).cuda(rank)
        dist.all_reduce(tensor)
        if rank == 0:
            print(f"Iter {i} done")
        dist.destroy_process_group()
```

即使 all-reduce 很小，这段代码也会把大量时间花在 `init_process_group` 和 `destroy_process_group` 上；rank 越多，协调成本通常越明显。所有 rank 还必须以一致顺序进入初始化、collective 和销毁，否则可能 hang。

训练主 process group 通常应在启动阶段调用 `init_process_group`，在训练循环结束后再销毁。下面把初始化和清理移到循环之外。清单仍把 local rank 复用于全局 rank，但在所示单节点两进程场景中两者数值恰好相同；生产代码应分别维护二者：

```
import torch
import torch.distributed as dist
import torch.multiprocessing as mp
import os

def run(rank, world_size):
    # Pin GPU
    rank = int(os.environ.get("LOCAL_RANK", rank))
    device = torch.cuda.device(rank)

    # Initialize NCCL communicator once
    dist.init_process_group(
        backend="nccl",
        init_method="tcp://127.0.0.1:45678",
        world_size=world_size,
        rank=rank
    )

    # Simulate 5 training iterations
    for i in range(5):
        tensor = torch.ones(1, device=rank)
        dist.all_reduce(tensor)

    # Cleanup once at the end
    dist.destroy_process_group()

if __name__ == "__main__":
    world_size = 2
    mp.spawn(run, args=(world_size,), nprocs=world_size)
```

把 communicator 的建立和清理移出循环，可以消除每次迭代重复初始化的成本。表 4-3 给出的 48 ms、0.5 ms 和超过 98% 的降幅并非由上述清单测得，因为清单没有计时代码；这些数值只能作为示意。

表 4-3。避免重复初始化/销毁 communicator 对每次迭代时间的影响（示意）

指标	之前（每次迭代）	之后（每次迭代）
init_process_group + destroy（初始化并销毁进程组）	48.0 ms	0 ms
dist.all_reduce（单元素张量）	0.5 ms	0.5 ms
迭代总时间	48.5 ms	0.5 ms

表中微小 all-reduce 的示意时间仍为 0.5 ms，而重复初始化成本完全主导了迭代时间。真实多节点作业中的绝对数值和收益取决于规模与环境，但复用长生命周期 process group 通常是明确的性能实践。

陷阱 2：不要在每次迭代中创建和销毁 NCCL communicator

由于 NCCL communicator 初始化成本较高，在为模型并行或流水线并行定义进程子组时，应避免无意中反复创建新组。通常在初始化阶段调用 PyTorch 的 `torch.distributed.new_group()` 创建所需子组，并在后续迭代中复用。所有 rank 必须按照全局一致的顺序创建重叠的 process group，否则可能出现死锁。

如果确实需要初始化多个 communicator，例如进行分阶段初始化，NCCL C++ API 可在 `ncclGroupStart()` 与 `ncclGroupEnd()` 之间批量发起多个 `ncclCommInitRank(...)` 调用。分组调用可并行推进初始化并避免某些顺序依赖，但收益依拓扑和版本而定，不能保证总是“大幅”降低开销。动态成员变更还需要更高层的容错与 rendezvous 协议，不能仅靠 grouped init 解决。

> PyTorch 的常规 process group 不支持在现有组内任意增删成员。弹性训练通常通过作业重启或重新 rendezvous 建立新的 worker group，而不是就地修改已有 NCCL communicator。创建与 collective 调用必须在相关 rank 间保持一致顺序；销毁语义则还取决于 PyTorch/NCCL 版本和异步工作的完成状态，不能简单概括为所有 rank 必须严格同时调用。

陷阱 3：避免通过环境变量过度调优或禁用 NCCL 功能

NCCL 提供许多环境变量，例如 `NCCL_BUFFSIZE`、`NCCL_NSOCKS_PERTHREAD`、`NCCL_P2P_LEVEL` 和 `NCCL_SHM_DISABLE`。除非有明确原因和测量证据，通常应保留默认行为。原文建议把当前默认值显式写入配置，但这会在升级时冻结旧行为；更稳妥的做法是只记录必要的覆盖项，并在升级硬件、驱动、插件或 NCCL 后根据 release notes 重新验证。

> `NCCL_BUFFSIZE` 控制 NCCL 每个 channel 使用的缓冲区大小，改变它可能影响部分消息模式的吞吐和显存占用，但增大并不保证改善大消息 all-reduce。原文的 4 MB 是特定版本的默认口径，不应作为所有版本的固定起点；应查询当前 NCCL 文档，并同时测量通信性能与显存峰值。

一个常见错误是在调试时禁用某项功能，随后忘记清除生产配置。P2P 或共享内存 transport 是否应启用取决于平台支持；重点是不要把用于隔离故障的覆盖项未经验证带入生产环境。

设置 `NCCL_P2P_DISABLE=1` 禁用直接 GPU P2P 传输，有助于在排障时隔离 P2P 路径问题，但在本应使用 NVLink 或 PCIe P2P 的节点上通常会降低性能。NCCL 会选择其他可用 transport，例如 SHM 或网络；并非所有流量都必然经过同一种“CPU host-staged”路径。原文给出的延迟从几微秒升至几十微秒、带宽从数百降至数十 GB/s 只是可能的示例，实际差异依拓扑和消息而定。

只在诊断需要时使用 `NCCL_P2P_DISABLE=1`。完成调试后，应删除该环境变量以恢复版本默认行为；显式设置 `NCCL_P2P_DISABLE=0` 也可允许 P2P，但仍应验证 NCCL 最终选择的实际路径。

同样，设置 `NCCL_SHM_DISABLE=1` 会禁止 NCCL 使用 SHM transport。当 P2P 不可用时，这可能迫使 NCCL 改走网络等路径并增加延迟。这里的 SHM 指 NCCL 的进程间主机共享内存 transport，不是 CUDA kernel 的片上 shared memory。

用于调试的性能关键环境变量应有明确作用域和失效机制。恢复正常运行前，删除临时覆盖项并通过 NCCL 日志与基准确认生产 transport。

另一个变量是 `NCCL_DEBUG`。设置 `NCCL_DEBUG=INFO` 可记录初始化、算法与 transport 选择等信息。NCCL 文档中的常用级别包括 VERSION、WARN、INFO 和 TRACE；原文所写的 `DEBUG` 不是常规级别名称。详细日志会产生 I/O 和格式化开销，因此生产环境通常使用默认 WARN，排障时再临时提高级别，并可用 NCCL_DEBUG_SUBSYS 缩小范围。下面介绍一些调优变量：

NCCL_NSOCKS_PERTHREAD 和 NCCL_SOCKET_NTHREADS（Socket/TCP transport）

这两个变量调节 NCCL 的 Socket/TCP transport，因此只在实际使用 Socket 路径时相关。多 NIC 环境应先确认 NCCL 使用的是 Socket 还是 RDMA，再调整对应 transport 的控制项。例如，`NCCL_NSOCKS_PERTHREAD=2` 表示每个 socket helper thread 使用两个 socket，但连接总数还取决于 `NCCL_SOCKET_NTHREADS`、channel、peer 和接口选择，不能由“两块 NIC”直接推导为固定四条连接。默认值随平台而异；根据 NVIDIA 文档，两变量的乘积不得超过 64。

64 是 `NCCL_SOCKET_NTHREADS * NCCL_NSOCKS_PERTHREAD` 这一配置乘积的上限，不是“每个 communicator 的 TCP 连接总数”上限。原文混淆了每个连接路径的 helper 并行参数与整个 communicator 跨 peer/channel 的连接数量。

NCCL_MIN_NCHANNELS 和 NCCL_MAX_NCHANNELS

这些变量限制 NCCL 可使用的 channel 数量。NCCL 可以把数据分布到多个 channel，以提高并行度；channel 不等同于一条物理 NVLink，也不总是简单对应“subring”。其 GPU 执行通常会占用相应 thread block 和其他资源，过多 channel 可能与应用计算争用。通常应让 NCCL 根据拓扑、collective、协议和消息大小选择，只有在 profiling 证明有收益时才覆盖。

NCCL_TOPO_FILE

可以通过该变量指定拓扑 XML 文件，覆盖或补充 NCCL 的拓扑发现。这适用于已确认自动发现结果不正确或需要仿真/定制的环境，但错误或过期文件可能产生更差路径。用于转储运行时拓扑的变量和 XML 格式具有版本依赖性；使用 `NCCL_TOPO_DUMP_FILE` 前应核对当前 NCCL 文档，并检查日志确认文件是否生效。

NCCL_MNNVL_ENABLE

该变量控制 Multi-Node NVLink（MNNVL）支持，适用于具有跨计算节点 NVLink fabric 的受支持系统。它不是普通多节点 InfiniBand/Ethernet 作业的通用加速开关；还要求兼容的 GPU fabric、Fabric Manager/相关服务、统一的 fabric UUID/安全域以及匹配的 NCCL 版本。NVL72 的具体节点边界和支持方式应以平台文档为准。

NCCL_SHARP_DISABLE

该设置用于控制 SHARP 网络内聚合。SHARP 通常通过 NCCL network plugin 集成，变量名称、默认行为和支持范围可能随插件版本变化，并非仅由 NCCL core 决定。在对应软件栈确实支持该变量时，可设置 `NCCL_SHARP_DISABLE=1` 做 A/B 测试或排障；是否实际启用应以插件日志、SHARP 管理面和性能计数为准。

总之，除非证据表明某个可调项有益，否则优先采用默认值。任何覆盖项都应记录适用硬件、软件版本、基准和回滚条件，并在升级后重新验证。

陷阱 4：验证 NCCL 线程的 CPU-GPU NUMA 亲和性

NCCL 及其 network transport/plugin 可能启动后台 CPU 线程，用于 socket/RDMA progress、proxy 工作和操作入队等任务；并非所有 GPU kernel 都由这些线程“dispatch”。通过 `torch.multiprocessing` 或 MPI 启动多个进程时，每个进程会继承 CPU affinity mask；若使用 `taskset`、`numactl`、MPI 或调度器绑定，它可能只包含部分核心。

如果进程被限制在过少核心上，通信辅助线程可能相互争用并降低吞吐。`NCCL_IGNORE_CPU_AFFINITY=1` 会让 NCCL 忽略作业提供的 CPU affinity，并依据 GPU affinity 自行设置线程亲和性；它不保证只在原进程允许的“本地 NUMA 核心子集”内自由分布。若调度器已为每个 rank 分配并隔离了正确 `cpuset`，盲目忽略 affinity 反而可能破坏资源隔离。因此应先设计每个 rank 的 CPU/GPU/NIC 局部性，再通过线程与性能观测决定是否设置该变量。

例如，一台节点有两个 NUMA 域和八个 GPU。若拓扑工具确认 GPU 0–3 更接近第一个 CPU socket、GPU 4–7 更接近第二个，可把对应 rank 绑定到各自 NUMA 域的 CPU 核心，同时纳入 NIC/HCA 的 PCIe 位置。是否再设置 `NCCL_IGNORE_CPU_AFFINITY=1` 必须通过测试决定，而不是固定的下一步。

> `numactl`、调度器 cpuset 或 MPI binding 可控制 CPU/内存亲和性；`CUDA_DEVICE_ORDER` 和 `CUDA_VISIBLE_DEVICES` 则控制 GPU 枚举与可见性，不会自行绑定 CPU 核心。`torchrun` 负责 rank 启动和环境变量，但通常不自动完成 NUMA 最优 CPU 绑定，仍需调度器或启动脚本配置并验证。

在自动拓扑发现有误时，也可以指定显式拓扑文件，但它不会替代 CPU binding，且必须与实际硬件一致。可以使用 MPI runtime binding 或 Slurm 的 `--cpu-bind` 等选项分配核心，再结合 `nvidia-smi topo -m`、PCI sysfs 和 NIC 拓扑确认每个 rank 的 CPU-GPU-NIC 路径。

陷阱 5：不要忽略 NCCL 警告和错误

适当启用日志后，NCCL 会输出初始化、拓扑和错误信息。其中可能显示 P2P 不可用、transport/plugin 加载失败或接口选择变化。日志中的 INFO 路径说明不一定都是“警告”，但任何与预期拓扑不一致的选择都应调查。

如果看到类似“unable to enable P2P, falling back to copy”的日志，应确认是哪一对 GPU、P2P 能力与预期是否一致。原因可能是跨 PCIe root complex、ACS/IOMMU 设置、容器设备可见性、虚拟化或硬件不支持。回退路径可能是 SHM 或网络等可用 transport，不一定全部经主机 CPU 内存，也不能仅凭一行日志断言性能必然“大幅”下降。

根据诊断结果，可以重新选择需要频繁通信的 GPU 组合，优先保持 NVLink/PCIe/NUMA 局部性。日志 `NCCL INFO NET/Socket: using Ethernet interface eth0` 表示 Socket 路径选择了 eth0；它是 INFO 信息，不必然是 warning。如果这不符合网络设计，可设置 `NCCL_SOCKET_IFNAME` 选择 bootstrap/Socket 接口，例如在确有 IPoIB 接口名 ib0 时使用 `NCCL_SOCKET_IFNAME=ib0`。该变量不选择 RDMA HCA，也不保证数据面使用最快 GPUDirect RDMA 路径；应分别检查 NET/Plugin、IB/RoCE 日志和链路计数。

陷阱 6：NCCL communicator hang、报错或完全关闭

如果某个进程崩溃、GPU rank 出错，或者不同 rank 进入了不一致的 collective，其他 rank 可能等待无法完成的操作，直到 watchdog 或 process-group timeout 触发。大规模集群中硬件、网络和软件中断不可忽视；表 4-4摘录了 Meta 对 Llama 3 405B 预训练中意外中断的分类。

表 4-4。Llama 3 405B 预训练 54 天期间意外中断的根因分类（来源：https://oreil.ly/z8QKu；原表中 72/17.2% 的比例与其他行推导的总数不一致）

组件	类别	中断次数	中断占比
GPU 故障	GPU	148	30.1%
GPU HBM3 显存	GPU	72	17.2%
软件缺陷	依赖项	54	12.9%
网络交换机/线缆	网络	35	8.4%
主机维护	计划外维护	32	7.6%
GPU SRAM	GPU	19	4.5%
GPU 系统处理器	GPU	17	4.1%
NIC	主机	7	1.7%
NCCL watchdog 超时	未知	7	1.7%
静默数据损坏	GPU	6	1.4%
GPU 导热界面与传感器	GPU	6	1.4%
SSD	主机	3	0.7%
电源	主机	3	0.7%
服务器机箱	主机	2	0.5%
I/O 扩展板	主机	2	0.5%
依赖项	依赖项	2	0.5%
CPU	主机	2	0.5%
系统内存	主机	2	0.5%

原文建议设置 `NCCL_ASYNC_ERROR_HANDLING=1`，并称新版 PyTorch 默认设置它。但现代 PyTorch 文档使用的是 `TORCH_NCCL_ASYNC_ERROR_HANDLING`，用于控制 ProcessGroupNCCL watchdog 检测到异步错误后的处理方式；它不是同名 NCCL core 开关。具体取值也不只是布尔语义，不同值决定是否中止 communicator 和/或进程。应以部署所用 PyTorch 版本文档为准。

在 PyTorch 部署中，如需覆盖默认行为，应配置 `TORCH_NCCL_ASYNC_ERROR_HANDLING` 并同时设置合理的 process-group timeout、监控失败传播和作业重启策略。直接使用 NCCL C/C++ API 时，应通过 `ncclCommGetAsyncError`、nonblocking communicator 配置及对应版本 API 处理异步错误；不要假定存在通用的 `NCCL_ASYNC_ERROR_HANDLING` 环境变量。

> 不要把“显式配置”理解为写死所有默认值。应在 runbook 中记录软件版本、采用默认值的决策、必要覆盖项及其验证依据。对安全性或故障语义关键的设置可以显式固定；性能调优默认值则通常应随版本演进，并在升级测试中检测变化。

NCCL 是高性能通信引擎，但正确性和性能仍取决于调用顺序、communicator 生命周期、设备映射、CPU/GPU/NIC 拓扑以及配置。通常复用长生命周期通信组，合理分配 CPU 核心，并只在测量证明需要时调整 affinity 或 transport 环境变量。

升级时应审阅 NCCL、framework 和 network plugin 的 release notes，并重新测试正确性、启动时间、collective 性能和故障处理。新版本经常包含优化和新硬件支持，但不保证每个工作负载都更快；默认值、算法和兼容性变化都可能需要重新调优，而把所有旧默认值固定下来同样可能阻止改进。

分析与调试 NCCL

NCCL 提供 nonblocking communicator 和异步错误查询 API，框架也可在网络或 GPU 错误后中止作业并重新调度，但 NCCL 本身不会透明地把 collective “fail over” 到另一条路径后继续保证应用状态。原文建议的 `NCCL_ASYNC_ERROR_HANDLING=1` 不是现代 NCCL core 的通用标准变量；PyTorch 应参考对应版本的 `TORCH_NCCL_ASYNC_ERROR_HANDLING`。调试时可设置 `NCCL_DEBUG=WARN` 或 `INFO`，检查 rank 调用不匹配、接口或 transport 配置错误等问题。

NCCL 还提供用于调试和分析的 profiler plugin API。支持该 API 的插件可以接收 NCCL 内部通信事件及其层级关系，帮助构建时间线并关联 collective、P2P 和 proxy 活动。它提供诊断数据，但不会自动判定所有慢设备或系统瓶颈，仍需结合 GPU、CPU、NIC 计数和应用阶段分析。

支持的 profiler plugin 可以通过 NCCL 的插件加载机制动态载入，并由分析工具实现相应回调。`NCCL_PROFILER_PLUGIN` 的确切取值、库命名和发现规则具有 NCCL 版本依赖性，应按当前 profiler plugin 文档和加载日志配置，而不是假定只设置变量即可获得数据。

该 API 便于第三方分析工具与 NCCL 集成，以层级化方式记录复杂通信活动。PyTorch Kineto 等工具也可借助 CUPTI、NVTX 和框架侧标记观察 NCCL kernel 与 API 活动，但这不等同于获得 profiler plugin API 暴露的全部内部事件。支持矩阵和运行时开销应以工具版本为准。

不要假定 NCCL profiler plugin 已随所有 NVIDIA 工具或 PyTorch/Kineto 环境打包。应确认具体 profiler 是否实现兼容插件、NCCL 是否成功加载它，以及时间线显示的是框架/CUPTI 事件还是 NCCL 内部 profiler 事件。

插件加载后，会通过 event activation mask 选择希望接收的事件类型。原文将它固定描述为 32 位整数；该类型与事件位定义属于 API 版本契约，应以当前头文件为准。事件可包括 group、collective、点对点以及多类 proxy 操作，并以父子关系组织，便于工具呈现详细通信活动。

本书所对应的 NCCL profiler plugin API 定义五类主要回调。`init` 回调初始化插件，建立 opaque context 并协商需要分析的事件；`startEvent` 接收 NCCL 提供的事件描述符，创建事件对象并返回供后续操作使用的 opaque handle。具体 ABI 版本和结构字段必须与运行时 NCCL 匹配。

`stopEvent` 回调标记事件结束，使插件能够回收相关资源；`recordEventState` 在事件经过不同状态时更新记录；`finalize` 则在分析结束时释放 profiler context 关联的资源。插件实现必须控制回调开销和线程安全，避免分析过程本身明显扰动通信。

SHARP 网络内聚合

使用支持网络内计算的高级网络硬件时，可以把部分受支持的集合操作卸载到网络中，从而获得额外性能。例如 NVIDIA 的 Scalable Hierarchical Aggregation and Reduction Protocol（SHARP，可扩展分层聚合与归约协议）。

InfiniBand SHARP 是用于 NVIDIA Quantum 系列 InfiniBand 交换结构的网络内归约技术。NCCL 可通过兼容的 RDMA SHARP network plugin 使用它，但 SHARP 也服务于 MPI/UCX 等其他通信栈，并非 NCCL 专属。

与之相关但并不相同的是 NVLink SHARP（NVLS）：它是特定 NVLink/NVSwitch 系统中的集合通信卸载机制，作用域位于 NVSwitch fabric，而不是 InfiniBand scale-out 网络。NVLS 可加速受支持的 reduce-scatter、all-reduce 等操作；NVSwitch fabric 本身也支持高效连接和多播，但不能因此把 NVLS 称为通用 all-to-all 算术卸载引擎。NVL72 等系统的具体能力取决于硬件代际和 NCCL 版本。

从概念上说，SHARP 让受支持网络硬件在数据流经交换结构时执行部分归约，例如求和，再向上层或端点传递聚合结果。这样可以减少端点之间传递的中间归约数据和网络热点，并降低部分大规模 MPI/NCCL collective 的延迟。收益取决于 collective、消息大小、并发作业、树资源以及端点和 fabric 带宽。

在经典环形 reduce-scatter 的简化模型中，若每个 rank 的输入大小为 B、共有 n 个 rank，则每个 rank 在 $n-1$ 个步骤中发送并接收的总数据量均为 $B\,(n-1)/n$。网络内归约可改变端点和链路上的数据流，但原文将其概括为“交换机只向每个 GPU 返回 B/n，因此端点接收量变为 $1/(n-1)$”并非通用 SHARP/NVLS 流量公式：它混合了最终输出 shard 大小与协议实际收发量。应使用目标算法和 profiler/计数器核对。

对于 all-gather，硬件多播可以让 fabric 复制数据，减少源端重复发送相同分片的需要。但实际算法可能采用多棵树、channel 和分块流水线，因此不能把发送量普遍写成相对 full ring 的 $1/(n-1)$。如果 reduce-scatter 与后续 all-gather 在算法、数据依赖和资源上

允许重叠，端到端阶段时间可能接近两者耗时的最大值，而不是简单相加；但 collective 通常存在依赖，且共享交换机、链路和 GPU 资源，原文“带宽主导时约减半”只是理想化上界。

总之，NVLS 可以利用 NVSwitch 内归约与多播能力减少部分端点工作和串行通信阶段，从而在适合的分片训练或推理通信中提高有效带宽、缩短停顿。具体效果必须按 collective 和拓扑测量。

> All-gather 不执行算术归约，因此相关硬件加速主要来自多播复制和数据路径优化。其收益取决于拓扑、消息大小和 NCCL 算法；不能普遍断言一定小于 NVLS 对 all-reduce 或 reduce-scatter 的收益。

NCCL 可以通过 NCCL RDMA SHARP plugin，把符合条件的 collective 卸载到启用 SHARP 的 InfiniBand fabric。部署通常需要兼容的 Quantum 交换机固件、Subnet Manager/SHARP Aggregation Manager、主机 OFED/UCX 或 RDMA 软件栈、network plugin 以及正确的权限和资源配置。GPU Direct RDMA 可使用 legacy peer-memory 或受支持的 DMA-BUF 路径，不是每台主机都必须加载同一个特定内核模块。配置完成且插件被实际选中后，NCCL 才能为受支持的 collective 使用 SHARP。NVIDIA 在特定大规模 AI 系统上报告过 all-reduce 约 2–5 倍加速，但这不是任意集群、消息或基线算法的保证。

当 GPU 和节点数量增加、scale-out 网络成为瓶颈时，SHARP 的收益通常更明显。在两到四个节点的小型集群上，端点处理、PCIe/NVLink 或启动延迟可能占主导，收益可能较小；但“32 个节点必然显著降低延迟”也不是固定门槛，仍取决于 collective、拓扑与 SHARP 资源。

> SHARP 是否默认启用取决于 network plugin、fabric 配置和策略。对于 InfiniBand SHARP，`NCCL_COLLNET_ENABLE=1` 可允许 CollNet 候选路径，但它本身不会安装或启用 SHARP，仍需兼容插件与 fabric。受支持 NVSwitch 系统上的 NVLS 可通过 `NCCL_NVLS_ENABLE=1` 显式允许，但 NVLS 与 InfiniBand SHARP 是不同机制。`NCCL_SHARP_DISABLE=1`. 中的句点属于原文代码标签且不应成为变量值；该禁用变量也可能由 SHARP plugin 定义而非 NCCL core，需核对插件文档。原文末尾的 “SHARE” 是 “SHARP” 的拼写错误。生产环境是否启用应由兼容性与 A/B 测试决定。

在已集成的环境中，应用通常无需改写 collective 调用即可使用 SHARP。可以查看 NCCL 和 network plugin 日志（`NCCL_DEBUG=INFO`）确认是否选择了 SHARP 路径。`ibv_devinfo` 主要报告 RDMA 设备与端口能力，不能单独证明交换 fabric 已配置 SHARP；还应检查 Aggregation Manager、插件状态、作业资源和 SHARP 计数器。

概括而言，SHARP 把部分归约计算移入网络，体现了现代系统中计算与通信边界的融合。如果集群使用兼容的 Quantum InfiniBand fabric，性能工程师应确认 SHARP 是否可用、是否被插件选择以及是否确有收益。它可以补充 NCCL 的 GPU 侧优化，但不是适用于所有 collective 的“加速按钮”。

本书所述 SHARP 主要是 NVIDIA Quantum InfiniBand 技术。Spectrum-X Ethernet 通过拥塞控制、adaptive routing、telemetry 及端到端软件栈改善 NCCL 性能；截至本

书所依据的公开资料，它并未暴露与 Quantum SHARP 相同的交换机驻留归约接口。不过产品能力会演进，应以当前官方文档为准，也不应仅凭是否具备 SHARP 判定 Ethernet fabric 是否适合大规模 AI。

SHARP 会消耗交换结构中的 aggregation tree、上下文和缓冲资源。这些资源有限，但大消息通常可分块和流水线处理；是否回退不只由“消息达到数 MB 或 GB”决定，还与并发组数、数据类型、操作、配额、插件和 fabric 状态有关。

> 建议监控 NCCL/plugin 日志、Aggregation Manager 和 fabric telemetry，对 SHARP 资源分配失败或路径回退告警。不要仅凭 GPU 内存压力推断交换机 aggregation 资源耗尽；两者是不同资源域。

持久化 NCCL 用户缓冲区与零拷贝注册

NCCL 支持用户缓冲区注册。注册后，受支持的算法和 transport 可以直接使用应用缓冲区，减少某些内部 staging、注册和复制开销。但注册本身不保证每个 collective 都采用零拷贝路径，也不必然减少 NCCL channel 数量；实际优化取决于 NCCL 版本、拓扑、内存类型和算法。

> 满足对称内存、地址/offset 和平台等前提时，持久化 NCCL 用户缓冲区注册可启用部分 CollNet/SHARP 或 NVLS 优化路径。零拷贝或直接路径可能减少复制、注册开销以及某些 GPU kernel 资源占用，但不能笼统保证降低 SM/channel 使用量；应通过 NCCL 日志和 profiler 验证。

可以使用 `ncclCommRegister()` 和 `ncclCommDeregister()` 显式注册与注销 communicator 上的持久化用户缓冲区。注册句柄、缓冲区生命周期、未完成操作与注销顺序必须遵循对应 NCCL 版本的 API 契约。某些需要对称注册的算法要求所有 rank 都提供兼容的已注册缓冲区，并要求相对注册区起始地址的 offset 在各 rank 间一致；原文把这一条件写成适用于所有通信的绝对规则，范围过宽。

NVIDIA NIXL 与解耦推理

NCCL 擅长模型训练中常见的多对多组通信，而现代大规模 AI 推理带来了不同的数据移动需求。NVIDIA 的 NIXL 是 2025 年初发布的开源数据传输库，面向高吞吐、低延迟的点对点及相关异步传输场景。

NIXL 旨在加速大规模 LLM 的分布式与解耦推理。解耦推理把不同推理阶段放到不同 worker 上，并可通过 NIXL 在 GPU、主机内存和存储层之间交换阶段数据。延迟与开销取决于实际 backend、拓扑、内存注册和调度。

解耦推理与 NIXL 是服务大型模型的可选架构手段，而非所有集群的无条件最佳实践。只有当阶段资源需求、负载与传输成本匹配时，解耦才可能优于共置部署。

NIXL 是 NVIDIA 开源 Dynamo 推理平台的数据移动组件之一，可用于一对一或一对少数端点的传输，例如在解耦阶段间移动 key-value（KV）cache。它补充而非取代主要用于集合通信的 NCCL。

NIXL 提供一致的异步 API，用于在 GPU、CPU、SSD 和共享网络存储之间移动数据。它通过可用插件/backend、注册元数据和传输属性选择可行路径，但不能保证“总是最快”；策略还可能由上层 cache manager 决定。图 4-5以 NVIDIA Dynamo KV Cache Manager 为例展示这一存储层次。

扩展 LLM 推理时，需要在 GPU、CPU、节点和机架间高效移动大型缓存，例如 Transformer attention KV cache。推理引擎可借助 NIXL backend，通过 NVLink 或 InfiniBand 等路径转移大型缓存。原文以 100 GB 为例，但实际大小由层数、KV head、head dimension、序列长度、batch 和数据类型决定；迁移也会占用链路并可能与计算争用，不能概括为“开销极小”。释放的 GPU 容量可用于处理新请求，这对长上下文服务尤其重要。

在满足支持矩阵时，NIXL 可利用 GPUDirect RDMA 在跨节点 GPU 内存之间直接 DMA，避免把 payload 暂存到主机内存。RDMA NIC 或 DPU 执行主要数据搬运，但 CPU 仍参与元数据交换、内存注册、队列提交、completion 和错误处理；低延迟也取决于拓扑、消息大小、拥塞与软件路径。

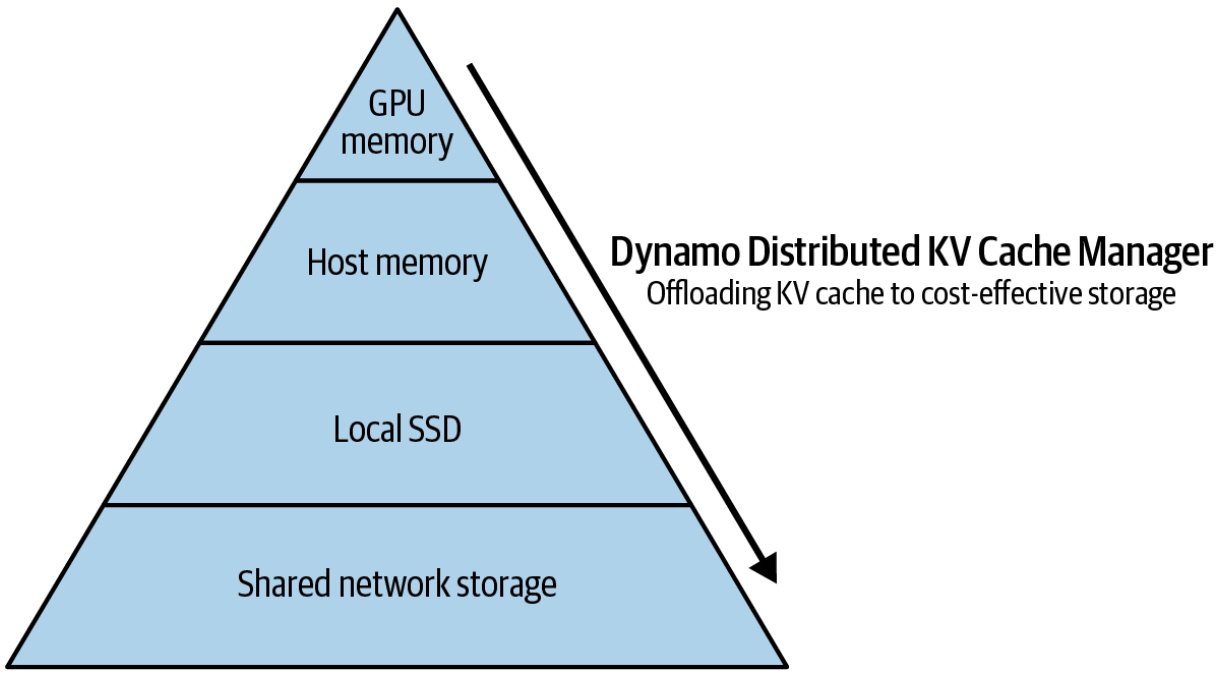


图 4-5。NVIDIA Dynamo Distributed KV Cache Manager 将较少访问的 KV cache 卸载到成本更低的存储层级（来源：https://oreil.ly/nsxdl）

NCCL 仍用于需要同步的集合操作，而 NIXL 侧重 LLM 推理中常见的一对一或一对少数端点的数据移动。下面介绍 Dynamo 如何在 prefill 与 decode 解耦流程中使用 NIXL。

NCCL 仍是大规模训练中 all-reduce 等集合操作的常用库；NIXL 则针对 KV cache 移动等推理数据传输。两者的能力有重叠边界，具体选择应按通信语义和框架集成决定。

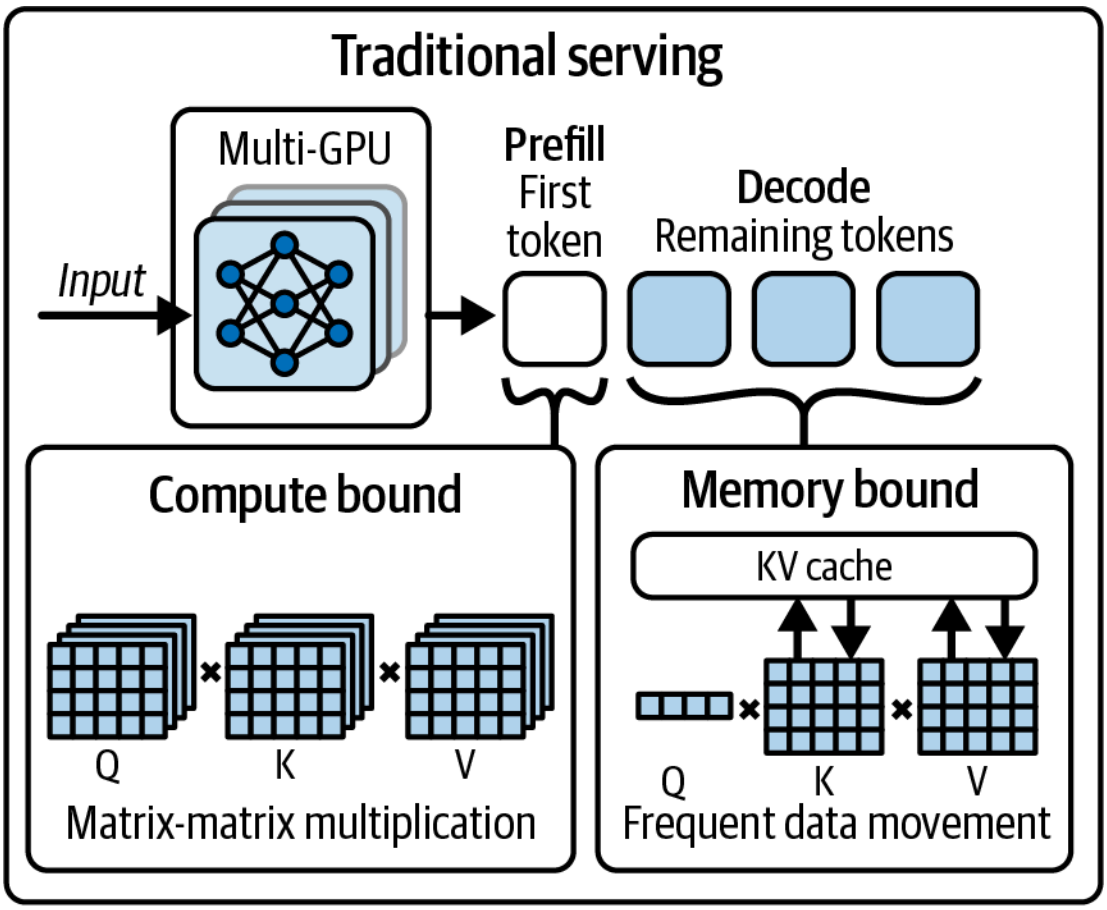
NVIDIA Dynamo 的相关资料展示了 NIXL 在多节点 LLM 服务中的应用；是否提高吞吐及幅度取决于部署。NIXL 在高性能推理流水线中补充而非取代 NCCL。

分离 prefill 与 decode 推理阶段

后续章节将深入讨论高调优推理系统的性能；在继续介绍 NIXL 前，先理解 Transformer 推理的两个阶段：prefill 与 decode。

第一阶段 prefill 对输入请求（即 *prompt*）中的多个 token 做并行计算并构建 KV cache，通常具有较高算术强度。第二阶段 decode 自回归地生成后续 token（即 *completion* 或 *response*），每步并行度较低，常受模型权重与 KV cache 的内存带宽和容量限制。不过两阶段是否 compute-bound 或 memory-bound 仍取决于 batch、序列长度、模型、量化和硬件。

vLLM、SGLang、NVIDIA Dynamo 和 TensorRT-LLM 等推理系统都区分 prefill/decode 阶段，但是否支持把它们部署到不同 worker、以及如何传输 KV cache，取决于版本和集成。*prefill*（prompt ingestion）创建 KV cache，*decode*（generation）使用该缓存。NIXL 可加速此工作流中的节点间 KV cache 传输。图 4-6 比较共置服务与把两个阶段放在不同 GPU worker 池中的解耦服务；后者旨在独立扩缩容和调优，而不是自动保证最大吞吐。


Traditional serving
Multi-GPU
Input
Prefill
First token
Decode
Remaining tokens
Compute bound
Q
K
V
Matrix-matrix multiplication
Memory bound
KV cache
Q
K
V
Frequent data movement

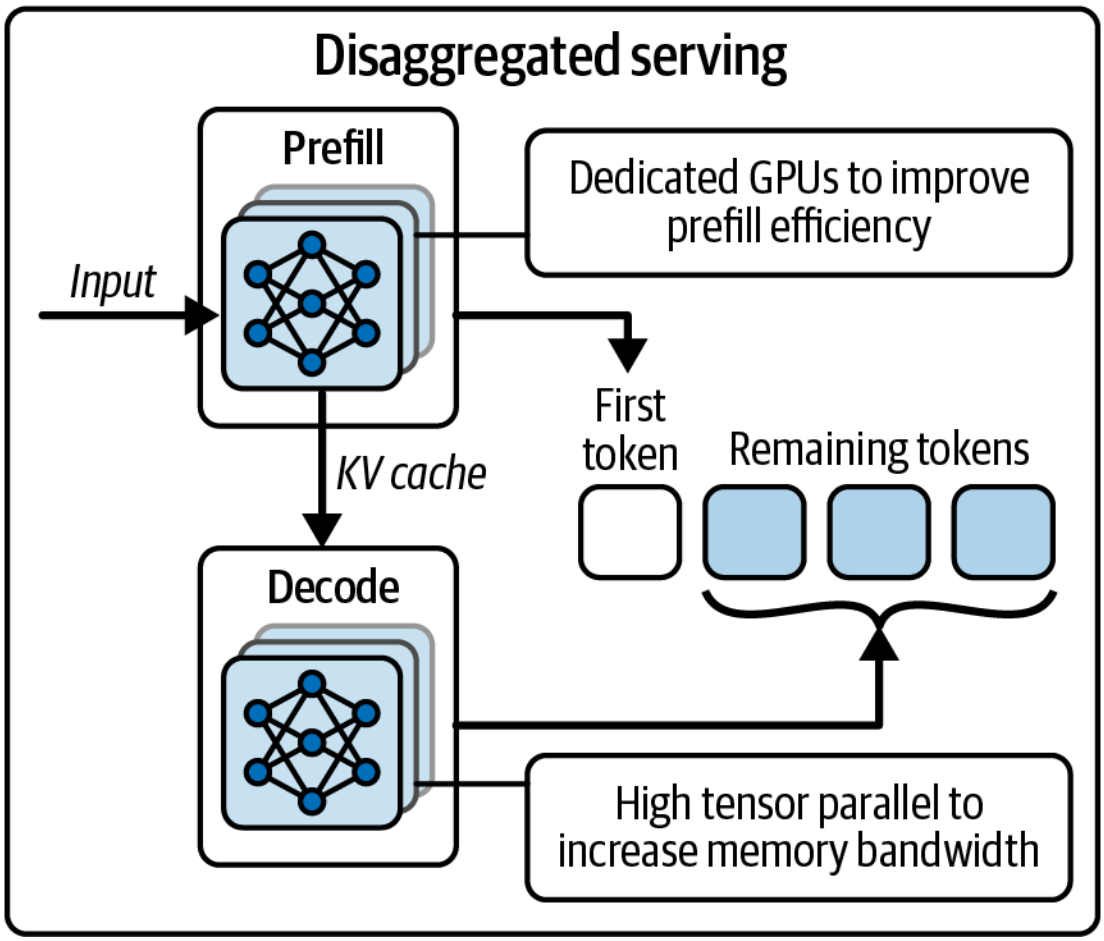

Disaggregated serving
Prefill
Input
Dedicated GPUs to improve prefill efficiency
First token
Remaining tokens
KV cache
Decode
High tensor parallel to increase memory bandwidth

图 4-6。解耦服务将 prefill 和 decode 阶段部署到不同 GPU worker 集群（来源：https://oreil.ly/nsxdl）

图上方的传统配置让每个 GPU 节点同时处理 prefill 和 decode；图下方的解耦配置把 prefill worker 与 decode worker 放入不同 GPU 池。这里所谓 decode 的 “I/O-bound” 主要指设备内存数据移动，而不是通常意义上的磁盘 I/O。

prefill 池中的 GPU 为输入序列生成 KV cache，再通过 NIXL 集成把它传给 decode 池中的目标 GPU。阶段专用化允许分别配置 batch、并行度与副本数，但额外传输和调度也可能抵消收益，需要按服务目标测量。

对于长 prompt，单请求或一批请求的 KV cache 可能达到数 GB 乃至更高，必须在首 token 延迟目标内从 prefill worker 传到 decode worker。传输延迟直接影响 time to first token；“用户无感”取决于产品 SLO，不能由 NIXL 单独保证。

把数据经 CPU DRAM staging 或持久化存储中转，可能增加复制和延迟；但主机内存或存储层也可能是容量分层的有意选择。NIXL 旨在为这些层级提供统一且高效的数据移动接口，但不能消除互连带宽这一基本瓶颈。

理想情况下，组件间使用高带宽 GPU-to-GPU 直接传输，并尽可能与其他请求或其他 buffer 的计算重叠。目标 GPU 不能在所需 KV cache 尚未到达时计算依赖它的 token；重叠通常来自分块流水线或跨请求并发，而不是违反数据依赖。

NIXL 可通过相应 backend 在同节点、跨节点乃至跨机架端点间传输数据。它依据已加载插件、可达性、内存类型和元数据选择路径，但无法预知所有动态拥塞，也不保证每次都选择最快路径。

这种 backend/path 选择与 NCCL 的拓扑选择有相似之处，但面向异步数据移动和推理模式。以 GB200/GB300 NVL72 环境为例，同一 NVLink fabric 域内可使用 CUDA/NVLink 相关路径；跨 fabric 域则可能使用 InfiniBand、RoCE 等 RDMA backend。实际选择不是简单固定的“先 NVSwitch、跨机架自动切换”，还取决于部署边界、插件配置和地址注册。

总之，NIXL 通过插件抽象 NVLink/NVLink-C2C、NVSwitch 可达路径、InfiniBand/RoCE 以及 NVMe/存储 backend 等数据移动能力。它为上层提供选择与异步操作机制，但“最快”仍需基准、telemetry 和策略验证。

KV cache 传输的智能互连路由

传统方案可能把数据从源 GPU 复制到 CPU DRAM，再复制到目标 GPU，这会增加 staging、内存带宽占用和延迟。把源与目标限制在同一节点可利用局部互连，但会限制调度和扩展灵活性。NIXL 面向大型 payload（如 KV cache）的异步数据移动，可通过相应 backend 跨 GPU、节点和机架传输。

NIXL 的目标是利用可用路径获得高带宽，并尽可能把传输与其他工作重叠。目标 GPU 不能在所依赖的 KV cache 数据到达前生成对应 token；实际重叠通常通过分块传输、双缓冲或处理其他请求实现。

NIXL 通过插件抽象多种互连与存储 backend，包括 CUDA/NVLink 可达路径、NVSwitch fabric、PCIe、跨节点 InfiniBand/RoCE RDMA，以及 NVMe 或共享存储。路径是否可用取决于内存类型、拓扑、插件、注册与部署配置；NIXL 不保证在动态拥塞下

始终选中最快互连。它的统一 API 可以在 GPU HBM、CPU DRAM 和 NVMe SSD 等内存/存储层之间传输数据，但不同层级的持久性、延迟和一致性语义仍然不同。

NIXL 异步 API 与完成通知

从开发者角度看，NIXL 提供统一的数据传输抽象。应用注册内存或存储区域，交换远端 metadata，准备源/目标描述符并提交异步请求。目标可以是 GPU、CPU 或由相应 backend 支持的存储；性能由 backend 和系统条件决定，并不保证“尽可能快”。

例如，NIXL 请求可以把 KV cache 分片传到另一块 GPU、CPU 主机缓冲区，或由插件支持的对象/文件存储。不同目标共享上层 API 模型，但地址、metadata、完成语义和性能并不相同，如图 4-7所示。

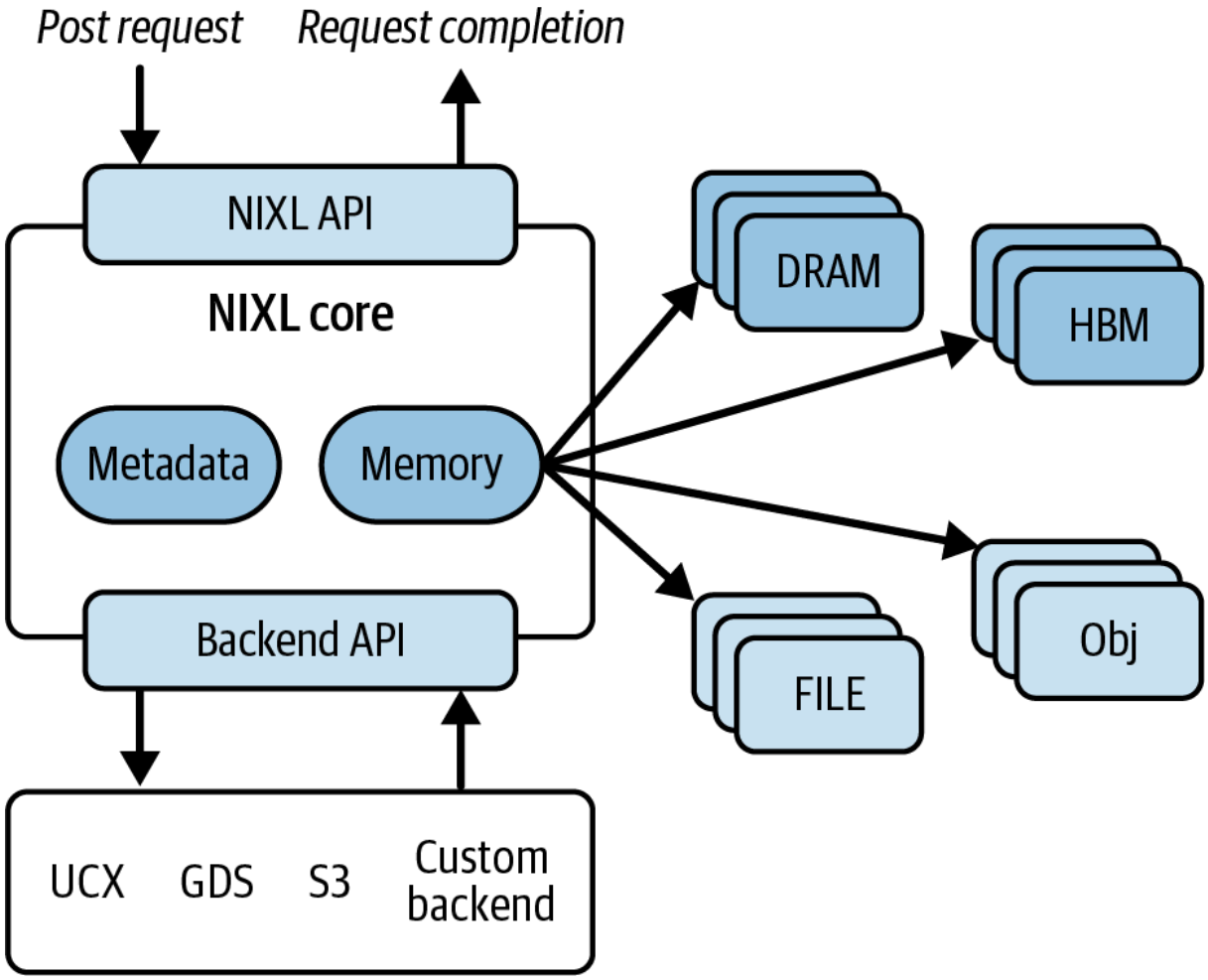


图 4-7。NIXL 架构（改编自 NVIDIA Dynamo 架构概览）

模块化 backend 设计便于 NIXL 增加新的 transport 或存储类型，同时尽量维持上层 API。不过新增 backend 仍可能带来新的能力查询、配置和语义约束；NIXL 根据已加载且兼容的 backend 协调数据移动。

NIXL 可在 GPU HBM、CPU DRAM、NVMe 文件存储和对象存储等层级间移动数据。统一 API 可以从可用 backend 中选择路径，例如 GPU 间 CUDA/UCX 路径或经 GDS 的 GPU-NVMe 路径。但路径选择、对齐、注册、文件系统和设备拓扑都会影响吞吐，不能保证 KV cache 卸载总能接近线速。

本书所对应的 API 流程使用 `registerMem` 注册区域，通过 `trim` 获得传输描述符，以 `prepXfer` 准备非阻塞请求，再用 `postXfer` 提交。实际 API 签名随 NIXL 快速演进，应用还需交换远端 agent metadata 和内存描述符。NIXL 根据 backend 能力使用 CUDA/PCIe/NVLink 可达路径、RDMA 或 GPUDirect Storage 等存储路径。

异步传输返回请求句柄，可通过 `checkXfer` 等版本对应机制查询完成状态。非阻塞提交为通信计算重叠创造条件，但轮询、progress 和 completion 仍可能消耗 CPU。下游 kernel 只能在对应数据范围完成传输并建立正确 CUDA stream/event 或应用同步后使用它；分块完成可以让已到达的 KV cache chunk 与后续 chunk 在途传输形成流水线。

`nixlAgent` 是 NIXL 的核心对象之一，封装本地 endpoint 配置、内存注册、backend、metadata 和异步请求管理。与远端 agent 通信前，通常需要交换可序列化 metadata，而不是直接共享另一个进程中的 C++ 对象。

跨 endpoint 传输涉及源和目标 agent 的信息。源侧 `agentSrc` 管理源内存与 backend，目标侧 `agentDst` 管理接收区域；但应用不一定需要在同一进程中构造两个 agent，真实跨节点部署通常是每个进程拥有自己的本地 agent，再经控制面交换 metadata。

每个 endpoint 的 agent 维护独立资源和请求生命周期，NIXL 根据双方 metadata 和 backend 能力准备传输路径。下面的代码把 `agentSrc` 与 `agentDst` 都放在同一进程、同一 GPU 上，且没有展示跨进程 metadata 交换，因此只能视为版本相关的 API 示意，并不能演示真实的源/目标 GPU 跨节点传输：

```
// NIXL 0.5.x style example: nonblocking VRAM->VRAM transfer between two agents
#include <nixl.h>
#include <nixl_types.h>

#include <cuda_runtime.h>
#include <iostream>
#include <thread>
#include <vector>
#include <cassert>
#include <cstdint>

int main() {
    // 1) Configure agents. Prefer UCX for GPU<->GPU,
    // allow GDS if you later target storage.
    nixl_agent_config cfg{};
    cfg.backends = {"UCX"};  // {"UCX","GDS"} if you also plan storage transfers
    cfg.thread_safe = true;  // thread-safe mode added in early 0.2.x

    // 2) Create source and destination agents
    nixlAgent agentSrc("srcAgent", cfg);
    nixlAgent agentDst("dstAgent", cfg);

    // 3) Allocate simple test buffers on the same GPU for illustration
    int deviceId = 0;
    cudaSetDevice(deviceId);
    const size_t bytes = 1 << 20; // 1 MiB

    void* d_src = nullptr;
    void* d_dst = nullptr;
    cudaMalloc(&d_src, bytes);
    cudaMalloc(&d_dst, bytes);

    // 4) Build registration descriptors for VRAM
    //    Each descriptor uses an address, length, and associated device
```

```
nixl_desc_t srcDesc{};
srcDesc.addr      = reinterpret_cast<uintptr_t>(d_src);
srcDesc.len       = bytes;
srcDesc.devId     = deviceId;
srcDesc.seg       = VRAM_SEG;

nixl_desc_t dstDesc{};
dstDesc.addr      = reinterpret_cast<uintptr_t>(d_dst);
dstDesc.len       = bytes;
dstDesc.devId     = deviceId;
dstDesc.seg       = VRAM_SEG;

std::vector<nixl_desc_t> srcList{srcDesc};
std::vector<nixl_desc_t> dstList{dstDesc};

// 5) Register memory with each agent and trim to xfer descriptors
auto srcRegs = agentSrc.registerMem(srcList);
auto dstRegs = agentDst.registerMem(dstList);

auto srcXfer = srcRegs.trim();  // metadata-free descriptors used for xfer
auto dstXfer = dstRegs.trim();

// 6) Prepare a WRITE from srcAgent->dstAgent, then post it (nonblocking)
nixlReqH reqHandle = nullptr;

// prepare + post
if (agentSrc.prepXfer(NIXL_WRITE, srcXfer, dstXfer, "dstAgent", reqHandle)
  != NIXL_SUCCESS) {
    std::cerr << "prepXfer failed\n";
    return 1;
}
if (agentSrc.postXfer(NIXL_WRITE, srcXfer, dstXfer, "dstAgent", reqHandle)
  != NIXL_SUCCESS) {
    std::cerr << "postXfer failed\n";
    return 1;
}

std::cout << "Transfer posted — doing other work...\n";

// 7) Poll for completion (replaces deprecated getNotifs/poll map)
nixl_status_t st;
do {
    st = agentSrc.checkXfer(reqHandle);
    if (st == NIXL_INPROGRESS) std::this_thread::yield();
} while (st == NIXL_INPROGRESS);

if (st != NIXL_SUCCESS) {
    std::cerr << "Transfer completed with error: " << st << "\n";
    agentSrc.releaseReqH(reqHandle);
    agentSrc.deregisterMem(srcRegs);
    agentDst.deregisterMem(dstRegs);
    cudaFree(d_src);
    cudaFree(d_dst);
    return 1;
}

std::cout << "Transfer completed!\n";

// 8) Cleanup
agentSrc.releaseReqH(reqHandle);
agentSrc.deregisterMem(srcRegs);
agentDst.deregisterMem(dstRegs);

cudaFree(d_src);
```

```
    cudaFree(d_dst);
    return 0;
}
```

示例以名称和配置初始化两个 agent，通过 `registerMem` 注册内存，以 `trim` 生成传输描述符，再用 `prepXfer`/`postXfer` 准备和提交非阻塞写入。它用 `checkXfer` 轮询请求，并在进行中时 yield；成功后通过 `releaseReqH` 释放句柄并注销内存。标题所称“callbacks”并未出现在代码中。清单还未初始化源数据、验证目标数据或检查 CUDA API 返回值，且 API/类型是否能在 NIXL 0.5.x 直接编译应以对应源码和示例为准。

NIXL 的 UCX backend 使用 Unified Communication X（UCX）提供 InfiniBand、RoCE/TCP、共享内存及 GPU memory 支持等低层传输能力；其他 backend 则不一定经过 UCX。在兼容环境中，NIXL 还可利用 GPUDirect RDMA 和 InfiniBand GPUDirect Async（IBGDA）减少稳态 progress 路径中的 CPU proxy 工作。IBGDA 不是“CPU 完全不参与”：初始化、注册、控制、异常和资源管理仍需要主机软件，延迟收益依硬件与消息模式而定。

NIXL backend 会尽量避免不必要的 staging copy，但 pageable CPU memory 不能简单通过“自动 pin”原地变成长期安全的 registered memory；可能需要注册、锁页、bounce buffer 或由应用提供受支持内存，具体取决于 backend 与操作系统限制。GPU memory 也只有在设备、拓扑、驱动和 transport 支持时才能直接传输。应用应检查注册结果与实际路径，而不是假定始终绕过中间主机缓冲区后到达目标。

使用 NIXL 卸载 KV cache

NIXL 的设计动机之一，是管理 LLM 推理中的大型数据。如果 GPU HBM 无法容纳长序列、多轮会话或高并发请求的全部 KV cache，NVIDIA Dynamo 等推理服务器可以借助 NIXL backend 将较冷数据卸载到 CPU 内存甚至 NVMe SSD，并在需要时取回。是否值得卸载取决于重用概率、传输成本与重新计算成本。

例如，NIXL 可与 Dynamo KV Cache Manager 配合管理这一层次。NVIDIA Grace Hopper 与 Grace Blackwell Superchip 通过 NVLink-C2C 提供 CPU-GPU coherent 地址访问和高带宽连接，但 CPU LPDDR 与 GPU HBM 仍是不同物理内存池，延迟和带宽并不相同，参见图 4-8。

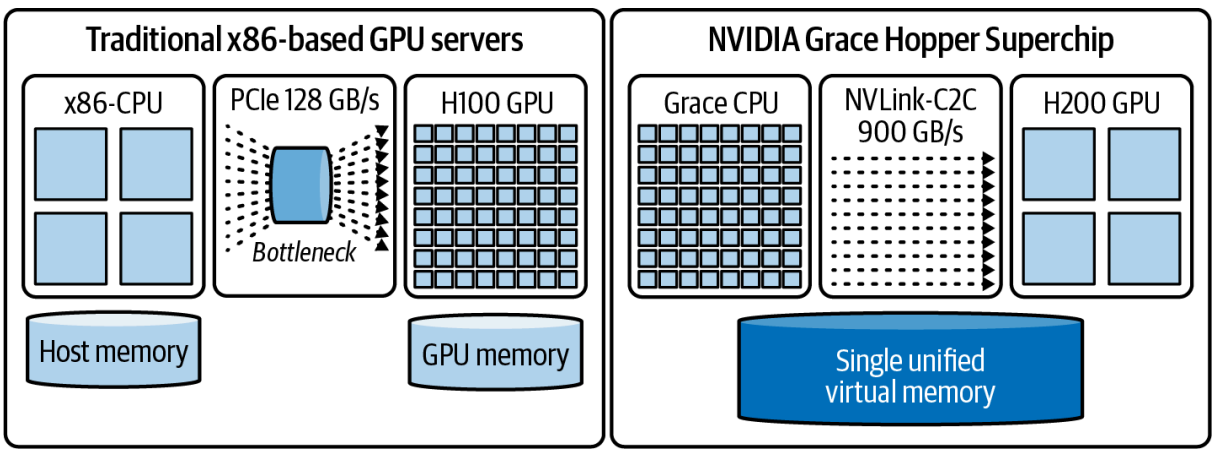


图 4-8。基于 Arm 的 Grace Hopper Superchip 使用双向聚合 900 GB/s 的 NVLink-C2C，缓解传统 PCIe 路径瓶颈（来源：https://oreil.ly/zf6rF）

推理服务器可以把 KV cache 卸载到容量更大的 CPU 内存，从而释放 GPU HBM。缓存命中后重新载入可能比从 prompt 重新计算更快，但也可能受 PCIe/NVLink-C2C 与主机内存带宽限制。来源中的特定 PCIe x86 + H100 基准在长输入序列上，相对“从头重算 cache”报告了最高约 14 倍的 time to first token（TTFT）改善；该结果见图 4-9，不能外推为一般推理性能倍数。

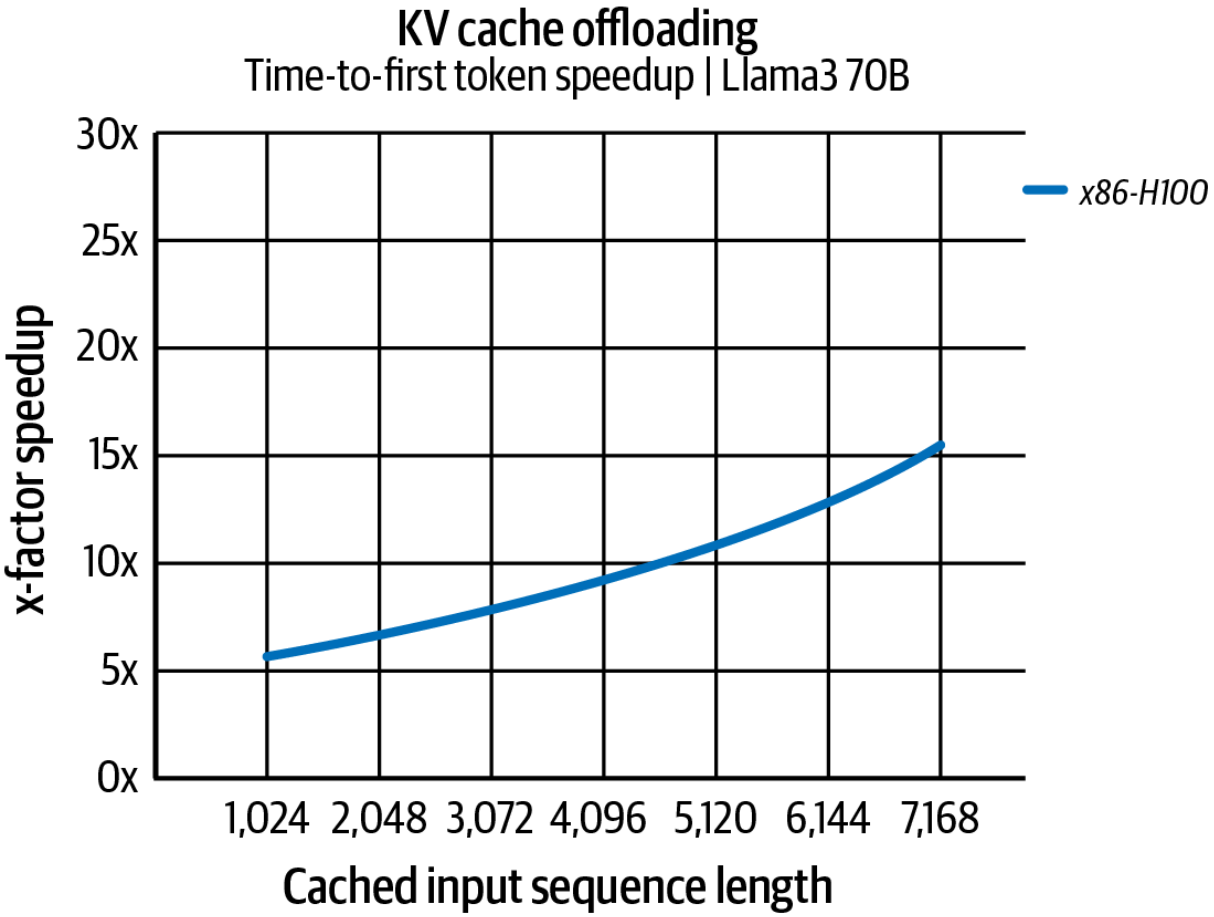


图 4-9。在特定长输入序列测试中，基于 x86 的 NVIDIA H100 系统通过 KV cache 卸载，相对从头重算测得最高 14 倍 TTFT 加速（来源：https://oreil.ly/zf6rF）

来源还报告：使用双向聚合 900 GB/s NVLink-C2C 的 Arm Grace Hopper Superchip，在该 KV cache 卸载基准中，TTFT 相比前述基于 x86 的 H100 平台最高约快 2 倍，参见资料和图 4-10。该对比同时改变了 CPU、内存系统和互连，不能把全部差异单独归因于 NIXL 或链路线速。

这些结果展示了软件 cache 管理与 Superchip 内存互连协同设计的潜力。NIXL 可降低部分传输路径成本，使 KV cache 卸载在容量受限的大模型服务中成为可行选项；但它不是所有大规模推理部署的必要步骤，仍需权衡命中率、迁移、重算、队列和 SLO。

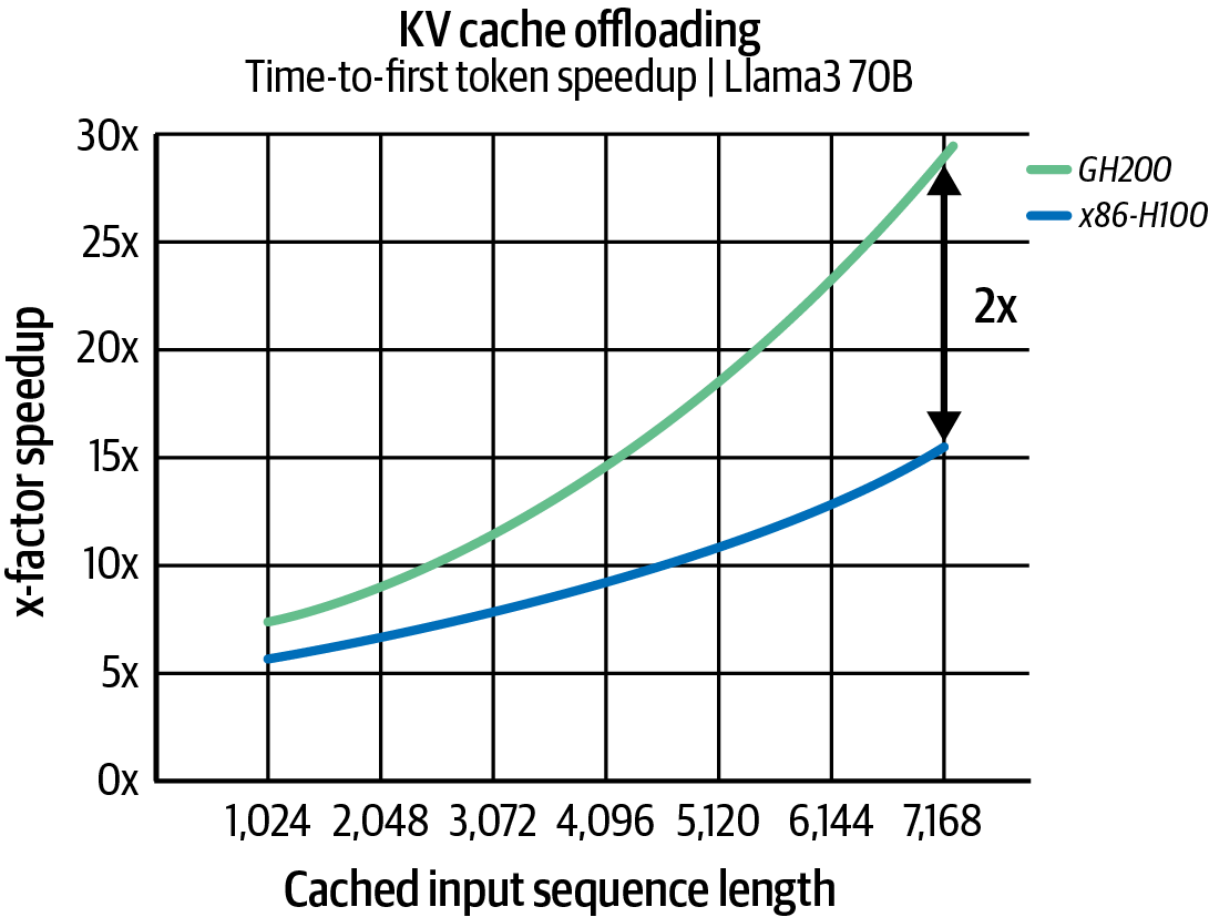


图 4-10。在来源所示 KV cache 卸载测试中，Arm Grace Hopper Superchip 相对 x86 H100 系统测得最高约 2 倍 TTFT 加速（NVLink-C2C 为双向聚合 900 GB/s；来源：https://oreil.ly/zf6rF）

随着模型和工作负载增大，高效异步移动大型数据对象变得重要。性能工程师应先明确通信语义：模型并行的同步 collective/点对点操作通常由 NCCL 等库承担，而跨推理阶段、GPU/CPU/存储层的持久数据移动可能适合 NIXL。应根据框架集成、正确性、传输路径和实测结果选择，而不是把二者视为可任意互换。

NIXL 与 NVIDIA Dynamo 等高性能推理系统

NIXL 可以影响 NVIDIA Dynamo 等分布式推理系统的数据移动效率；Dynamo 也于 2025 年初发布。NVIDIA 的特定内部测试在 72-GPU Blackwell NVL72 上服务约 680B 参数模型时报告最高约 30 倍推理吞吐提升。该结果比较的是完整 Dynamo/部署配置与特定基线，包含调度、并行、解耦、批处理和硬件等变化，不能解释为 NIXL 库单独带来 30 倍加速。

NIXL 可把多 GB 上下文/KV 数据的跨节点移动作为异步操作集成到推理流水线中，但传输仍受链路、拥塞、注册和队列限制。后续章节将深入介绍 NVIDIA Dynamo、TensorRT、vLLM 及其模型推理优化。

NCCL 与 NIXL 对比

NIXL不是 NCCL 的替代品，而是补充。NCCL 处理紧耦合 GPU rank 间的 collective 和点对点通信，例如多 GPU all-reduce；NIXL 面向任务/阶段之间或 GPU、CPU、存储组

件之间的异步数据移动。表 4-5比较二者的典型定位，但边界并非绝对。

表 4-5。NCCL 与 NIXL 对比

方面	NCCL（collective 与点对点通信）	NIXL（异步数据移动）
主要使用场景	训练与推理中紧耦合 GPU 组的 collective（如 all-reduce、all-gather）及 point-to-point。	分布式推理、cache 分层或流水线中的一对一/一对少数端点数据移动（如大型张量或 cache）。
通信模式	各 rank 必须以一致顺序参与 collective；操作可异步入队，但 collective completion 不等同于通用 barrier。	异步读/写式数据传输，一个发起端与一个或多个目标；支持单向数据移动及多种内存/存储类型。
与计算重叠	在依赖和资源允许时可重叠，例如 DDP bucket all-reduce 与部分反向计算重叠；具体 stream 行为依框架实现。	异步提交与轮询/通知为重叠创造条件；是否与计算并行取决于依赖、progress、链路及 GPU/CPU 资源竞争。
拓扑感知	发现拓扑并通过调优模型选择 ring/tree/NVLS 等算法与 transport；不保证对每个负载都达到最优。	通过 backend/plugin 和 endpoint metadata 使用 CUDA、UCX/RDMA、GDS 等可用路径；选择依配置和支持矩阵。
数据范围	从小控制消息到大型梯度、activation、参数分片等多种大小；关键是 collective/P2P 语义，而非固定尺寸。	常用于大型 KV cache、模型分片等数据对象，但 backend 也可处理其他大小；最佳粒度需基准确定。
集成方式	集成于 PyTorch DDP、Horovod 等训练/推理框架，或由应用直接调用 NCCL。	独立开源项目并被 NVIDIA Dynamo 等系统使用；框架或应用调用 NIXL API，管理 metadata、内存注册与请求生命周期。
示例	在 8 个 GPU 上对 100 MB 梯度执行 all-reduce。	在推理流水线中把 1 GB KV cache 从 GPU 0 移到 GPU 1、CPU 内存或 NVMe SSD。

NCCL 支持 point-to-point `send()`/`recv()`，并非只适合“同步训练”。NIXL 的差异在于它提供跨多种内存/存储 backend 的注册、metadata 与异步数据移动抽象，适合大规模推理 cache 层次和阶段解耦。后续章节介绍的 NVIDIA Dynamo 使用 NIXL 在 GPU、CPU 与 SSD 等组件间协调 KV cache 移动。

概括而言，NCCL 面向单机和跨节点多 GPU collective/P2P 通信，通过拓扑发现和调优选择算法与 transport，但不保证“完全跑满”所有 PCIe、NVLink、InfiniBand 或 Ethernet 链路。NIXL 则以异步、可插拔 backend 的数据移动原则连接 GPU、CPU 和存

储设备，主要服务大规模分布式推理中的 KV cache 等数据。两者可在同一系统中共同使用。

要点总结

通过细致工程实践和本章介绍的技术，可以逐步接近目标硬件与协议在给定工作负载下的可达上限。这里的上限不是单一“光速”数字，而取决于拓扑、消息、并发、协议开销和资源竞争。调优网络层时应记住以下要点：

拓扑很重要

> 节点间互连（例如 InfiniBand/RoCE/Ethernet）与节点内互连（例如 PCIe、NVLink/NVSwitch）会影响合适的通信策略。多节点、多 GPU 配置可考虑分层 collective。应通过日志、拓扑和计数器确认实际路径，避免因配置或默认选择误走慢路径；在 fabric、插件和 collective 均受支持时，再评估 SHARP 等网络内聚合/归约能力。

调优环境与系统

> 环境变量或 OS 设置有时会改善吞吐，也可能造成回归。例如，可按实际 transport 调整 NIC/socket buffer、临时启用 NCCL 日志、控制特定功能，并正确配置 CPU/IRQ 亲和性。所有 OS 与驱动级优化都应有基线、A/B 测试和回滚方案，参见第 3 章。

合理利用新硬件能力

> NVIDIA Grace Hopper 与 Grace Blackwell Superchip 等平台提供较大 CPU 内存和高速、一致性 CPU-GPU 互连，可用于数据分层、模型分区和向 CPU 内存卸载 KV cache。SHARP 等网络内计算在特定大规模测试中曾报告 2–5 倍 collective 加速，但并非普适结果。新一代计算与网络硬件会改变容量、拓扑和最佳配置，应持续核对支持矩阵并重新测量。

目标不是让 GPU、网络和磁盘在每一时刻都同时达到 100%，因为它们会共享功耗、内存、PCIe 和调度资源，而且负载阶段不同。更有意义的是提高端到端 goodput，缩短关键路径和尾延迟，同时确保高利用率来自有效工作，而不是重试、排队或无用搬运。

这需要迭代测量和验证，并在额外内存、代码复杂度、可靠性与可维护性之间权衡。经过验证的调优可以缩短训练与推理时间，提高昂贵基础设施的整体利用率。

结论

高性能分布式、多 GPU 通信与存储系统，是调优大型复杂 AI 系统的基础。针对正确语义使用 NCCL collective、NIXL 推理数据移动和 RDMA transport，可以减少特定通信瓶颈；收益必须在完整应用中测量。

NVSwitch 和支持 SHARP 的 InfiniBand 交换结构提供新的数据路径与卸载能力，但只有在拓扑、插件、collective 和配置匹配时才会转化为训练或推理性能。更新 CUDA、PyTorch 或 NCCL 也不等于自动启用 SHARP，因为它还依赖 fabric、管理服务与

network plugin。NVIDIA Dynamo、vLLM 等服务框架可以集成部分优化，但仍需按版本和部署验证。

本章的核心结论是：任何单一组件都无法独自提供峰值性能。只有协调高速通信与高效数据处理、正确性、容错和全系统调优，才能构建可扩展、稳健的 AI 系统。

对性能工程师而言，*数据移动*与原始计算能力同样关键。如果 GPU 持续等待 CPU、存储、网络或其他 GPU，再高的峰值算力也难以转化为 *goodput*。

下一章将讨论基于 GPU 的存储策略和优化。除 RDMA、NCCL、NIXL 等网络协议与库外，GDS 和高效输入流水线也是持续为 GPU 提供有效工作的系统方法之一。

第 5 章 面向 GPU 的存储 I/O 优化

对 AI 工作负载而言，持续向 GPU 提供数据与计算本身同样重要。设想一个在数千块 GPU 上训练的 100 万亿参数模型，它可能处理数十亿个训练样本，包括 token、图像、音频和视频等。

这意味着必须从存储系统读取海量数据，并及时送入 GPU。如果存储与输入流水线过慢，GPU 会因等待数据而空闲；即使采用前文介绍的通信优化，整体利用率仍会很低。

本章讨论存储与输入流水线优化，重点介绍如何高效读取本地或远端存储、预处理数据，并在依赖与资源允许时将 I/O 和预处理与 GPU 计算重叠。

高速存储与数据局部性

大型模型训练通常需要读取庞大数据集。大语言模型的数据单位可能达到数十亿乃至数万亿 token 或样本记录；文本语料可达 TB 级，视觉或多模态数据可达 PB 级。具体容量取决于编码、去重、压缩和 epoch 重用方式。

在超大规模训练中，存储系统必须为可能持续数月运行的数千块 GPU 稳定提供足够吞吐。原文进一步写到“数百万 GPU”，但没有可复核的现实部署依据。将 NVMe SSD 部署在计算节点或机架附近，或使用具备机架局部拓扑的 NVMe over Fabrics（NVMe-oF），可以减少部分网络跳数并改善性能一致性；实际效果仍取决于共享比例、交换网络、控制器和故障域。

如果数据位于 NFS 或 Amazon S3 等网络存储，需要确认所有计算节点的聚合读取带宽以及元数据/请求速率均足够。假设根据模型和 batch 实测，每块 GPU 需要 200 MB/s 的输入数据，8 块 GPU 合计约需 1.6 GB/s。Blackwell、Rubin 等新 GPU 可能提高处理能力，但输入带宽需求不是由 GPU 型号单独决定，而取决于模型、样本、压缩、缓存命中和预处理。应以目标 workload 的 bytes/sample 与 samples/s 计算，而不是只追求让设备饱和。

NVIDIA Grace Blackwell GB200/GB300 NVL72 在一个 NVLink fabric 域内连接 72 块 Blackwell GPU。按每块 GPU 200 MB/s 计算，聚合需求为 72 × 200 MB/s = 14.4 GB/s；原文给出的 14–20 GB/s 中，上限 20 GB/s 需要额外余量或更高的单 GPU 假设。容量规划还应计入协议开销、突发、检查点、shuffle、元数据和降级冗余。

> 如果工作负载流式处理体积更大的媒体或多模态样本，应使用实测 bytes/sample 与 samples/s 校准。还要分别统计存储读取字节、解压后字节和送入 GPU 的字节，因为它们可能相差很大。

一种方案是使用节点或机架内的 NVMe SSD，或设计合适的 NVMe-oF 拓扑；另一种是使用 Lustre、IBM Storage Scale（原 GPFS）等并行文件系统。并行文件系统本身不等同于“把数据缓存到本地 SSD”，本地缓存/分层通常还需客户端缓存、burst buffer、数据

预置或专门的数据加速层。存储具备足够吞吐后，还需配置合适的数据加载进程、线程、预取和解码并行度。Python GIL 只影响执行 Python bytecode 的线程；C/C++ I/O、解码库或多进程 loader 的瓶颈需要分别分析。

在成本、容量和可靠性允许时，应提高数据局部性。“近”可以是同一节点的本地 NVMe SSD，也可以是同一机架或低跳数高速 fabric 上的 NVMe-oF/存储服务。标准名称为 NVMe over Fabrics（NVMe-oF），不是原文写法 “NVME-oF”。局部副本还会增加预置时间、容量占用和一致性管理成本。

在分布式多节点训练中，可以把数据集预分片或缓存到各节点，让节点优先读取本地子集。例如，100 TB 数据分布到 10 个容量足够的节点时，可让每节点持有约 10 TB 主分片。不过还需考虑 epoch shuffle、数据均衡、节点故障、弹性扩缩容和副本；严格“只读本地 10 TB”可能降低全局随机性或使故障后数据不可达。

PyTorch 的 `DistributedSampler` 按 rank 划分 dataset 索引，并可按 epoch 确定性 shuffle，以减少同一 epoch 内重复处理；它不知道文件实际位于哪个节点，也不会自动实现存储局部性。要让逻辑样本分片与本地文件分片对齐，需要定制 dataset/sampler、清单或缓存路由，并正确调用 set_epoch()。

顺序读取与随机读取模式

GPU 处理数据很快，输入流水线通常也能从较大、连续的传输中获得更高效率。存储设备和文件系统在大型顺序读取时通常比 4 KB 级随机读取获得更高吞吐，但现代 NVMe 的随机 IOPS 也很高，实际差距取决于 queue depth、并发、缓存、压缩和文件系统。因此，数据布局应尽量把常见访问组合成适当大小的连续区间，而不是机械取消训练所需的随机性。

例如，图像训练若保存数百万个独立小文件，瓶颈往往来自 open/stat、目录查找、权限和远端元数据请求，而不只是机械磁盘的随机 seek。可以把样本打包为大小适中的 shard，例如 TFRecord、WebDataset tar、数据库/record 文件，或在模式适配时使用 Arrow/Parquet。格式选择应考虑压缩、随机访问索引、局部损坏、并行 worker、重写成本和生态工具；“少数几个超大文件”并不总是理想布局。

> 随着 GPU 处理速度提高，把小文件组合为较大 shard 往往更重要，因为元数据和小请求更容易暴露为瓶颈。现代并行文件系统与对象存储能处理一定程度的并发小访问，但能力依元数据服务、分区键、请求配额和客户端而异，必须显式测试。

从较大 shard 读取一个连续区间，可以一次取得多个样本并摊薄请求成本。Amazon S3 等对象存储中也常预先合并小对象，但 shard 不宜过大：过大的对象会降低细粒度并行、增加重试与更新成本，并可能导致 worker 负载不均。通常需要按目标请求大小、range GET、缓存和失败恢复选择 shard 大小。

还应调节读取大小。1 MB 读取通常比 4 KB 读取更容易获得高吞吐，因为每次系统调用、协议和设备命令的固定成本被摊薄，但最佳值取决于样本、cache、direct I/O 对齐、设备和并发。Python `open()` 提供用户态 buffered I/O；内核 readahead 是独立机制，不能

把二者混为同一个“OS read-ahead buffer”。随机访问也可能从应用缓存、合并读取或异步并发受益。

可以把相邻请求合并为较大连续区间，或使用高级 dataset API，例如 `TFRecordDataset`，以及 PyTorch 的 `IterableDataset`/`DataLoader`。这些 API 并不自动保证顺序读取；访问模式由 dataset 实现、sampler、worker 分片和 shuffle buffer 决定。应共同调节 buffer、prefetch、worker 数量和内存上限。

如果访问必须随机，可通过线程调用 `pread()`，或使用 Linux `io_uring` 等异步接口并发提交多个读取。`io_uring` 支持批量提交、固定文件/缓冲区和可选 polling，可减少部分系统调用与注册开销；但文件系统、内核、direct/buffered I/O 模式和设备仍决定是否真正异步，以及 polling 是否值得占用 CPU。提高 queue depth 有助于隐藏延迟，但过高并发会增加队列、cache 污染和尾延迟。

文件系统和挂载配置应匹配大型并发 I/O。XFS 常用于 Linux NVMe 服务器；`noatime` 可避免读取时更新访问时间，但现代 Linux 默认的 relatime 已减少许多写入，实际收益需测量。Amazon EFS 的 General Purpose 与 Max I/O 是 performance mode；Max I/O 面向高度并行、可容忍更高操作延迟的旧/特定工作负载，并非所有场景的“最高性能”。throughput mode 还包括 Elastic、Provisioned 等当前选项，选择与可否变更应以文件系统创建配置和最新 AWS 文档为准。任何模式都不能自动保证满足大规模 AI I/O，应通过 CloudWatch 与端到端基准验证。

面向吞吐量调优 NVMe 与文件系统

现代 Linux 使用多队列块层 `blk-mq`，把软件/硬件队列映射到多个 CPU。`blk-mq` 是块层架构，不是某个具体 I/O scheduler。高速 NVMe 的有效 queue depth 和硬件队列数由驱动、控制器与 workload 共同决定，通常先采用默认值；对低开销设备可测试 none，对需要延迟控制或公平性的场景可测试 mq-deadline 等策略，不能仅凭“顺序读取”决定。

旧式 completely fair queueing（CFQ）scheduler不属于 blk-mq 路径；现代系统可能为 NVMe 使用 `none`、`mq-deadline` 或发行版/设备选择的其他 scheduler。可通过 `/sys/block/<device>/queue/scheduler` 查看当前与可选策略。none 的低软件开销常适合 NVMe，但并非所有低延迟或多租户负载的固定最佳项；部分设备还可见 BFQ 或 kyber 等策略。

> 高性能 NVMe 可先比较 `none` 与 `mq-deadline`，但选择应同时考虑吞吐、尾延迟、公平性和 CPU 开销。可在对应 namespace/block device 的 `/sys/block/nvme*/queue/scheduler` 检查设置；修改前应确认 device-mapper、RAID 或容器看到的实际块设备层级，并通过可重复基准验证。

另一个可调项是 readahead。内核在检测到顺序读取时会预取后续页面，可在 `/sys/block/<device>/queue/read_ahead_kb` 查看块设备设置。默认值并不保证是 128 KB；大型顺序流可能从更大窗口受益，随机读取则可能浪费带宽并污染 page cache。readahead 主要合并/提前底层读取，不会直接减少应用发出的 read syscall 数量。可通

过 `blockdev --setra` 设置以 512-byte sector 为单位的值，必须注意它与 sysfs 中 KB 单位不同。

使用 NVMe SSD 时，应核对 PCIe generation、lane width、NUMA/PCIe switch 拓扑和实际链路协商状态，避免控制器或共享上行成为瓶颈。多个 SSD 可通过 RAID 0/软件 striping 提高聚合吞吐，但 RAID 0 没有冗余，任一设备故障都会影响整个阵列；训练数据必须可重建或另有可靠副本。也应确认 CPU、PCIe switch 和文件系统能支撑聚合带宽。

Linux page cache 会用可用 RAM 缓存文件页面。数据集远大于内存且访问缺少局部性时，可能出现较低命中率和回收开销；这不一定是严格意义的“thrash”，应结合 cache hit、major fault、reclaim 与设备 I/O 判断。能重复命中的 warm cache 可以显著减少后续存储读取，但基准必须区分 cold-cache 与 warm-cache 结果。

如果数据或高频子集能放入 CPU RAM，可考虑预热 page cache、应用 cache 或 RAM-backed 数据层，以减少训练期间磁盘 I/O。但预加载会增加启动时间、内存压力和多进程副本风险。Grace Blackwell 的 CPU LPDDR 与 GPU HBM 通过 NVLink-C2C 提供 coherent 访问，但仍是不同物理内存池；不能把容量简单相加后称为无差别的“CPU + GPU unified memory”缓存。PB 级数据通常需要流式读取、分层缓存和可控预取。

数据加载通常需要多个 worker，例如 PyTorch `DataLoader(num_workers=N)`。当 num_workers 大于 0 时，DataLoader 使用独立进程而不是“CPU threads”；每个进程可并行读取和预处理，但会带来进程启动、IPC、内存复制/共享和文件描述符开销。最佳 worker 数需结合每个训练进程、CPU affinity、NUMA、存储和解码库实测。

第 13 章和第 14 章将深入讨论 PyTorch 调优。这里先说明：`pin_memory=True` 让 DataLoader 尝试把返回 tensor 放入 pinned host memory，`non_blocking=True` 在源为 pinned memory、设备和 stream 支持时允许 H2D copy 异步入队；真正与计算重叠还需独立 stream、正确依赖和硬件 copy engine。`persistent_workers=True` 可避免每个 epoch 重启 worker，但会保留其内存和状态。`prefetch_factor` 应按 workload 调节；`num_workers` 大于 0 时常见默认值为每 worker 2 个 batch，但语义和默认值应以当前 PyTorch 版本为准。

worker 太少可能让 GPU 等待数据；太多则会争用 CPU 核心、内存、文件描述符和 I/O 带宽。应同时监控 batch ready time、GPU idle、CPU/NUMA、存储吞吐、IOPS 与尾延迟。目标不是让磁盘长期接近 100% busy，因为队列饱和可能显著恶化延迟；应以满足训练 goodput 且保留恢复/检查点余量为准。

> 高核心数 CPU（例如部分 GB200/GB300 Superchip 配置中的 Grace CPU）可能支持更多 loader worker，但可用核心数、SMT、容器 cpuset 和具体 SKU 必须从实际系统确认，不能把 “72-core” 当成所有 GB200/GB300 节点的统一配置。还应注意 I/O、内存带宽和 IPC 争用带来的边际收益递减。

使用 NVIDIA GDS

GPUDirect Storage（GDS）允许兼容的存储路径与 GPU 内存直接进行 DMA，从而避免在 CPU 内存中建立额外的数据副本。传统路径通常先把 NVMe SSD 数据读入主机内存，

再通过 CUDA copy 将其传至 GPU 内存。

> GDS 与 GPUDirect RDMA 相互补充：前者面向存储到 GPU 的 DMA 路径，后者面向网络设备到 GPU 的 RDMA 路径。两者都不会消除 CPU 的配置和编排工作，其关键收益是避免主机内存 bounce buffer。

使用 GDS 时，存储设备或 NIC 的 DMA engine 可执行直接内存访问（DMA），把数据传入 GPU HBM，而不经过额外的主机内存副本。原文称“GPU 发起 DMA”，容易误解：应用和 CPU 仍负责提交与控制 I/O。GDS 可支持本地 NVMe，也可在兼容的软件、文件系统和网络栈上支持 NVMe-oF 等远端存储。

实际部署中，GDS 在存储与 GPU 内存之间建立绕过主机内存 bounce buffer 的 DMA 路径。具备专门集成的集群文件系统以及部分对象存储接入方案也可使用这一路径，但不能仅凭存储类型推定兼容；CPU 仍负责配置、提交、完成处理和错误管理。

启用 GDS 需要受支持的 NVIDIA GPU、驱动和 CUDA/GDS 组件，以及兼容直接 DMA 的存储栈。本地 NVMe SSD 或 RAID volume 是常见方案，但具体支持范围随 GDS、内核、文件系统和厂商插件版本变化。原文列举了在 XFS/EXT4 上结合 `O_DIRECT` 的本地 NVMe 与 NVMe-oF、NFS over RDMA，以及 BeeGFS、WekaFS、VAST、IBM Storage Scale 等集成 `nvidia-fs` 或相应接口的系统；部署时应以当前 NVIDIA 支持矩阵和存储厂商文档为准。

应用还必须使用相应 API，例如通过 CUDA 的 `cuFile` 库执行 GDS 文件 I/O。`cuFile` 提供缓冲区注册、对齐处理以及与受支持文件系统的集成，但并不意味着任意普通文件都会自动走直接路径。

如果 GDS 已正确配置，并且读取路径使用 `cuFileRead`，数据便可从存储直接进入 GPU 内存。这通常能降低 CPU 利用率，并在主机副本是瓶颈时提高吞吐。`cuFileRead` 使用 Linux 文件描述符接入文件系统；`cuFile` 的异步 API `cuFileReadAsync` 和 `cuFileWriteAsync` 可把存储 I/O 与 CUDA stream 协调起来，在依赖、对齐、后端和硬件允许时实现流水化或重叠，第 11 章还会讨论 stream。

> 在后端要求且访问模式合适时使用 `O_DIRECT`，可以绕过 OS page cache，并满足某些直接 DMA 路径的条件。现代 GDS 版本的 `cuFile` 也可能接受非 `O_DIRECT` 文件描述符，但会依配置使用兼容路径；缓冲区、offset 或长度未对齐可能引入额外复制、失败或性能下降。

WekaIO、DDN、VAST、Cloudian 等存储厂商提供了支持 GDS 的产品或插件，使其系统能在兼容 RDMA 路径上把数据传入 GPU 内存。不过，这并不表示所有企业 NAS 或并行文件系统都能“开箱即用”；仍需匹配产品版本、客户端、驱动、网络、文件系统协议和许可配置。

VAST Data 的厂商报告称，在其特定 AI workload 中，单块 A100 采用 GDS 后顺序读取吞吐提高约 20%，在适用配置上更接近每 NIC 100 Gb/s 的链路容量。该结果不是 GDS 的

通用保证，且链路速率不能直接等同于应用有效吞吐。图 5-1 对比了使用和不使用 GDS 的架构。

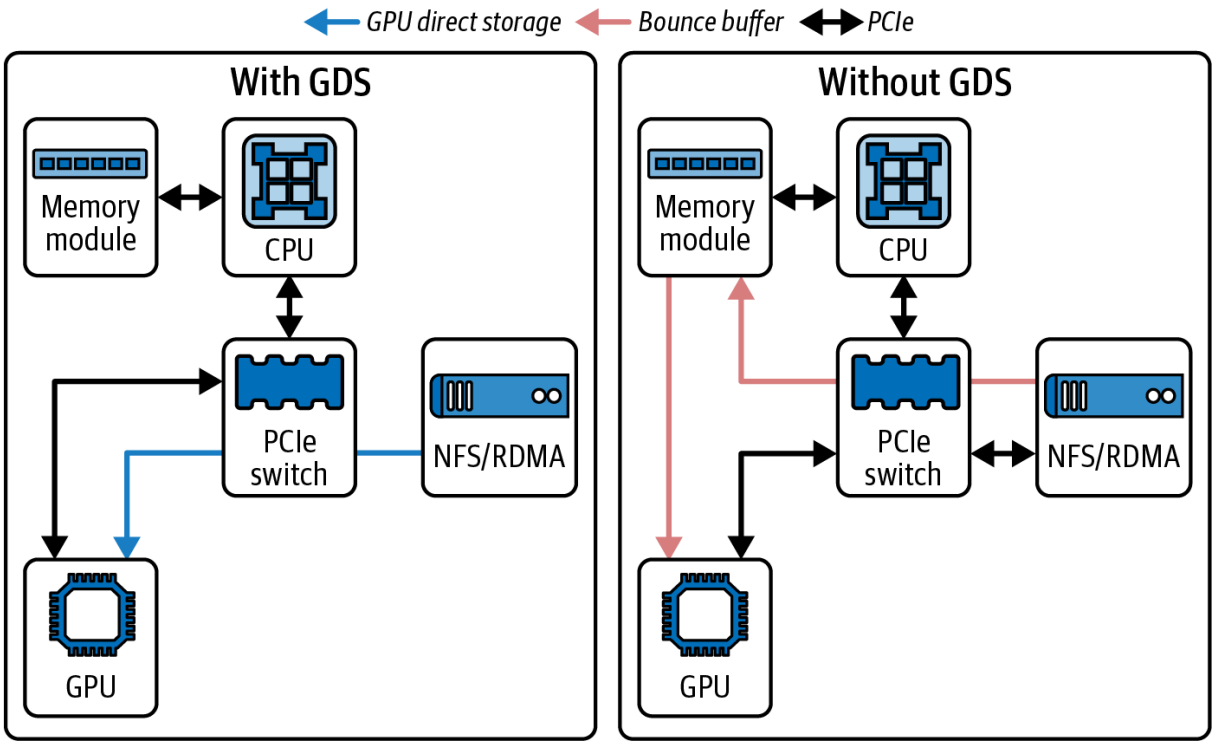


图 5-1. VAST Data 使用 GDS 与不使用 GDS 时的网络架构

图左侧是经过主机内存复制的传统 staged DMA，右侧是借助 GDS 绕过主机内存副本的直接路径，可降低 CPU 利用率。VAST 的报告称，Ampere A100 的读取吞吐提高约 20%，Hopper H100 提高 30% 以上；原文把差异归因于更高 NIC 带宽和更重 CPU 负担，但这属于特定厂商配置下的测量，不能仅由 GPU 架构推导。

> 应在目标 workload 与 fabric 上验证收益，因为结果会随 I/O 大小、queue depth、NIC 代际、拓扑、文件系统实现和缓存状态而变化。

GDS 往往需要调优，而且并非所有 workload 都会显著提速。如果 CPU 原本就能轻松处理传输，吞吐可能变化不大；CPU 占用也通常会下降，但降幅取决于实际是否走直接路径。若主机因大量 `memcpy` 饱和，GDS 的潜在收益更大，仍应以端到端训练 goodput 验证。

使用 GDS 时，必须正确满足 `O_DIRECT` 语义和对齐要求。直接存储到 GPU 的路径不使用 host pinned memory 作为 staging buffer。`cuFile` 注册 GPU device buffer，`nvidia-fs` 或当前受支持机制协调存储设备/RDMA NIC 与 GPU 内存之间的 DMA。它以 POSIX 文件描述符接入，因此 `cuFile` 可用于受支持的普通文件；网络文件系统仅在协议、客户端和厂商实现明确支持 GDS/RDMA 时可用，并非“支持 RDMA”这一条件本身就足够。

例如，每个训练 batch 为 1 MB、每秒向 GPU 提供 1,000 个 batch，对应约 1,000 MB/s 的逻辑输入率。原文断言 CPU copy 会轻易占用几个核心，但实际 CPU 成本取决于 copy

次数、内存带宽、NUMA、并行度和预处理，不能从 1 GB/s 单独推导。GDS 可消除相应 staging copy 并释放部分 CPU；速率更高或扩展到数千块 GPU 时，这一聚合收益可能更明显，但也会受到存储和网络瓶颈限制。

训练数据路径通常以读取为主，因此多数 GDS 性能结果聚焦存储读取；快速写入 checkpoint 同样重要。要让写入走 RDMA 加速的 GDS 路径，文件系统、客户端、网络和 GDS 版本必须共同支持相应 write path。

WekaFS 是面向大规模 AI 训练的并行文件系统之一，并提供支持 GDS 的读取和写入集成。是否真正通过 RDMA 直接访问 GPU 内存，应按其当前兼容矩阵、部署配置和运行时遥测确认。

使用 cuda-checkpoint 检查点保存 GPU 状态

在 Linux 上，可以把 NVIDIA 的 `cuda-checkpoint` 工具与 Checkpoint/Restore in Userspace（CRIU）等 CPU 进程检查点工具配合使用，以保存 GPU 状态。`cuda-checkpoint` 在运行中的进程内挂起 CUDA，等待已提交工作完成，把 device memory 复制到由驱动管理的 host allocation，并释放 GPU 资源；随后，CPU 侧检查点工具便可对进程制作快照。

挂起路径会锁定 CUDA driver entry point，排空未完成工作，将 device memory 复制到主机并释放 GPU 资源。估算挂起时间时，需要同时考虑实际占用的 device memory 容量，以及挂起期间可用的 GPU 到主机互连带宽。

由于驱动会在挂起阶段把 device memory 复制到 host allocation，有效挂起时间至少受到内存映像大小和平台互连的约束，还包括排空 kernel、同步、分配及驱动管理等开销。应在 lock 和 checkpoint 调用周围设置标记，并使用 Nsight Systems 或相应时间测量来确认挂起阶段的实际耗时。

恢复进程时，驱动重新取得 GPU，把 device memory 映射回原虚拟地址，恢复 stream、context 等受支持的 CUDA object，然后解除 driver/process lock，使 CUDA 调用可以继续执行。可恢复的对象和限制应以所用 CUDA 与驱动版本的兼容文档为准。

具体而言，CUDA Driver API 提供 `cuCheckpointProcessLock`、`cuCheckpointProcessCheckpoint`、`cuCheckpointProcessRestore` 和 `cuCheckpointProcessUnlock`。恢复前需要启用 persistence mode，或调用 `cuInit` 完成相应初始化；在受支持的条件下，可以重新映射到相同 chip type 的其他物理 GPU，但这不表示任意 GPU、拓扑或节点之间都可无条件迁移。

需要注意，这条路径与框架级模型 checkpoint（例如 PyTorch checkpoint）解决的问题不同。CUDA 进程 checkpoint 可用于长时间训练或推理作业的抢占、迁移与某些故障恢复流程，但其可靠性仍取决于 CRIU、驱动、CUDA 状态、外部资源和分布式协调是否都受支持。

与使用 GDS 输入数据不同，这条 checkpoint 路径不会把 GPU memory 直接 DMA 到存储。驱动先在挂起期间把 device memory 映像移入 host memory，再由 CRIU 将进程内存持久化为 checkpoint image。因此，它应作为框架 state dict 或 sharded checkpoint 文件的补充，而不是替代品；后者仍负责可移植的模型、优化器、数据迭代与分布式训练状态。

使用 gdsio 测量 GDS

NVIDIA 提供了 `gdsio` 工具，默认安装在 `/usr/local/cuda/gds/tools` 下，用于测试存储与 GPU 之间的 GDS I/O。它很适合验证路径是否生效并建立受控基准。

原文称 GDS 的吞吐提升常达到 10%–20% 甚至更多，尤其是在 CPU 受限场景；这只是经验范围，不是性能保证。下面使用 NVIDIA 的 `gdsio` 工具，示意比较 CPU-mediated read（“之前”）和直接 GDS read（“之后”）。以下是 CLI 命令及 throughput/latency 输出：

```
# Before (Storage → CPU Memory only)

# CPU path, host memory, async copies (-x 2)
$ /usr/local/cuda/gds/tools/gdsio \
    -f /mnt/data/large_file \
    -d 0 -w 4 -s 10G -i 1M -I 0 -x 2

Total Throughput: 8.0 GB/s
Average Latency: 1.25 ms
```

第一个调用在 read mode（-I 0）下使用带 pinned host memory 和异步复制的 CPU 路径（-x 2）建立 baseline。下面的第二个调用保持其他参数相同，并在 read mode（-I 0）下启用 GDS 路径（-x 0）。比较时必须一致地使用 transfer selector；在这里，`gdsio` 的 `-x 2` 测量 CPU-mediated transfer，`-x 0` 测量 GDS 路径。参数语义仍应以所安装版本的帮助和文档为准：

```
# After (Storage → GPU Memory using GPUDirect Storage)
#   - same config, GDS path (-x 0)
$ /usr/local/cuda/gds/tools/gdsio \
    -f /mnt/data/large_file \
    -d 0 -w 4 -s 10G -i 1M -I 0 -x 0

Total Throughput: 9.6 GB/s
Average Latency: 1.00 ms
```

在这组示意输出中，GDS 直接路径的吞吐比 CPU 路径高 20%，平均 I/O latency 也相应降低，见表 5-1。绕过 host buffer staging 还可能减少 CPU copy 工作。不过，书中没有给出硬件、文件系统、cache 状态、重复次数或统计分布，因此这些数字不能视为可复现实验；实际验证应控制文件、I/O size、并发、对齐、warm-up 和 cache 条件，并同时记录 CPU 利用率。

表 5-1. 本示例使用 GDS 前后的吞吐与延迟

路径	吞吐	延迟
存储 → CPU（不使用 GDS）	8.0 GB/s	1.25 ms
存储 → GPU（使用 GDS）	9.6 GB/s (+20%)	1.00 ms (–20%)

就本示例而言，使用 GDS（存储 → GPU）后，读取吞吐从 8.0 GB/s 增至 9.6 GB/s，延迟从 1.25 ms 降至 1.00 ms；按点估计计算，吞吐提高 20%，延迟降低 20%。这只能描述该组输出，不能替代多次测量、误差范围和端到端训练 goodput 对比。

DeepSeek 的 Fire-Flyer 文件系统

DeepSeek 从头开发了名为 Fire-Flyer File System（3FS） 的定制开源文件系统。其设计动机之一，是 AI workload 会产生大量随机读取。

对于工作集远大于内存、重复访问较少的 LLM 训练或推理 workload，这类随机读取可能降低传统 read cache 的收益，甚至让 cache 管理成为负担。3FS 采用 direct I/O 风格的数据路径，尽量减少 kernel page cache 参与及主机侧复制，类似一些强调直接存储访问的 HPC 文件系统。但原文所称“每个请求都直达 NVMe SSD”过于绝对：请求仍可能经过客户端、网络、storage service、设备内部 cache 和副本/纠删码路径。

> 3FS 体现了围绕 AI workload 协同设计存储栈的趋势。它与 NVIDIA GDS 都试图减少传统主机数据路径开销，但二者不是同一层：3FS 是分布式文件系统，GDS 是让受支持存储路径直接访问 GPU memory 的软件/驱动机制。

3FS 包含四类关键组件：cluster manager、metadata service、storage service 和 client。它们通过 InfiniBand 或 RoCE 等支持 RDMA 的 fabric 互连，以减少数据面 CPU 处理与部分 host-side copy；CPU 仍参与请求提交、metadata、控制与错误处理。各组件及连接见图 5-2。

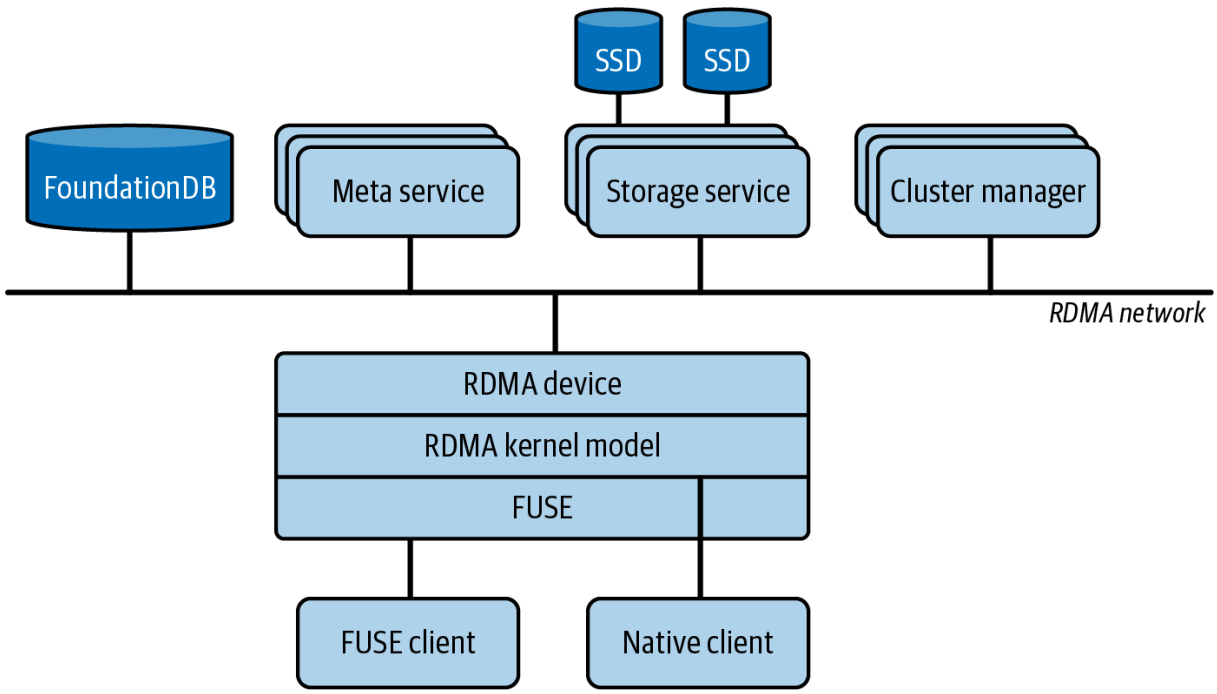


图 5-2. DeepSeek Fire-Flyer File System（3FS）的组件（来源：https://oreil.ly/xD3id）

3FS 面向 Linux，并通过客户端接口兼容已有应用，同时利用 RDMA 加速 client 与 storage service 间的数据传输。metadata 跨节点分片并复制，以支持横向扩展。其主数据路径旨在绕过 OS page cache，但这不应等同于所有应用、metadata 和异常路径都完全不使用 kernel cache，也不自动意味着数据直接进入 GPU memory。

> 仅凭文件系统使用用户态 FUSE 接口，不能笼统断言它永远无法提供 GDS 路径。能否直接传入 GPU memory 取决于具体 GDS 版本、kernel/client integration、厂商插件、文件描述符路径及 `O_DIRECT`/对齐支持；普通 FUSE read/write 路径本身通常不等于 GDS direct path。

3FS 集成了基于 RDMA 的传输，可高效把数据送入 AI pipeline；但 RDMA 到 client host memory 与 GDS/GPUDirect 路径直接进入 GPU device memory 是不同能力。若确实需要 GDS，应选用当前兼容矩阵明确支持的本地 NVMe、NVMe-oF 或 BeeGFS、WekaFS、IBM Storage Scale、VAST 等文件系统集成，并验证运行时数据路径。即使受支持，“异步、zero-copy、极低开销”也仍受 buffer registration、对齐、拓扑和 fallback 影响。

3FS 可以配合本章的 I/O 与计算重叠技术，使 data prefetch 和 transfer 有机会与 GPU kernel 并发。能否实际重叠仍取决于应用流水线、buffer 生命周期、CPU/GPU/网络资源竞争和同步依赖。可把它理解为把第 4 章讨论的级联 pipeline/wave 思路扩展到存储层。

DeepSeek 公开报告称，大规模 3FS 集群的聚合读取吞吐可达数 TB/s，其环境中的结果最高为 7.3 TB/s。另一项基准使用 68 节点 AI-HPC 集群，每节点 10 块 16 TB NVMe SSD，并配置双 100 Gb/s 网络，报告聚合读取约 6.6 TB/s，同时还服务约 1.4 TB/s 的后台 workload。原文进一步将其与约 1.1 TB/s 的 Ceph 结果比较；这些都是特定测试的聚合口径，只有在副本、数据布局、请求模式、网络、节点数和软件配置相当时才可公平比较。

3FS 通过跨节点协调 I/O 获得这种聚合性能。在相匹配的 workload 中，较高持续带宽可以降低 data staging 成为瓶颈的概率，并帮助训练和推理维持较高 GPU 利用率；但 metadata、small I/O、尾延迟或计算瓶颈仍可能限制端到端 goodput。

DeepSeek 构建面向随机读取的文件系统，并将其与 RDMA-first 数据路径结合，说明大规模 AI 系统需要端到端的 full-stack 性能工程，其中也包括存储设计。

3FS 展示了重新设计存储层如何缓解部分 I/O 瓶颈，但不能据此称其消除了“最后一点”瓶颈。自研文件系统需要大量前期投入以及持续的正确性、可靠性、升级和运维工作；多数团队更可能先从已有分布式文件系统或对象存储开始。下一节将讨论这些方案。

分布式、并行文件系统与对象存储

多节点训练常使用 NFS 等共享文件系统，或 Lustre、IBM Storage Scale（原 GPFS）、Ceph 等分布式/并行存储，使所有节点访问同一数据集。这很方便，但如果容量、metadata、网络、客户端并发和数据布局配置不当，存储仍会成为瓶颈。

单个 NFS endpoint 在大量节点并发读取时容易受服务器、后端盘、NIC 或协议处理能力限制。应按目标 workload 测量，而不是只检查是否有多块高速 NIC；多服务器分片、

NFS cluster 或 scale-out NAS 可以分散负载，但还需处理 namespace、数据放置、故障切换和热点。

原文建议 NFS 只用于少数节点，但可扩展上限不能由协议名称固定决定：高端 scale-out NFS 与单机服务器的能力差别很大，需求也随 bytes/sample、metadata rate 和 cache 命中变化。大型训练集群常选择并行文件系统或 Amazon FSx for Lustre 等托管数据存储/缓存方案，是因为它们能横向扩展吞吐与 metadata，而不是因为 NFS 在某个固定节点数后必然失效。

> 对于 Amazon FSx for Lustre 等云端存储/缓存方案，应验证端到端性能提升能否抵偿额外成本。若结果不符合预期，应检查数据导入/懒加载状态、容量与吞吐配置、客户端、网络和 workload，并按需与云服务商共同核验架构。

NFS 提供 rsize/wsize（读写请求大小）等调优参数。较大值（例如协商出的 1 MiB）可能提高大块顺序吞吐，但最大值并非所有 small/random I/O 的固定最佳项。还要核对后端存储、NFS server/client、网络和 mount option。NVMe 与 striping 可提高带宽；若采用 RAID 0，必须另行保证数据可重建和故障恢复。

例如，可以测试 `rsize=1048576,wsize=1048576,noatime,async`，其中 rsize/wsize 请求上限为 1 MiB，`noatime` 避免更新访问时间。不过 Linux NFS client 的 `async` 语义和默认行为依版本而定，且写入持久性风险不能为吞吐而忽略。还可按一致性需求测试 `actimeo=60,lookupcache=pos`；属性缓存会造成最长一段时间的 metadata 可见性延迟，`lookupcache=pos` 也有明确的正/负目录项语义，不能笼统称为“缓存目录项 60 秒”。

Amazon S3 等对象存储不是传统 POSIX 文件系统，但在 AI workload 中很常见。训练期间直接进行大量小对象请求可能受请求 latency、rate、解码和网络限制；常见方案包括预置到本地 NVMe，或使用与对象存储集成的缓存/文件系统层。`s5cmd` 和 `aws s3 cp` 等工具可以在训练前下载数据，但预置时间、增量更新和本地容量也应计入作业生命周期。

> 可使用 AWS SDK 或 `s5cmd` 等支持并行传输的工具，但并发度应与对象大小、连接池、CPU、网络和服务端配额匹配；过高并发可能增加 throttling、重试和尾延迟。

也可以使用支持 range request 与 cache 的 streaming library 读取 S3 object。请求应足够大以摊薄固定开销，但不是“越大越好”：还需权衡首 batch latency、并行度、无用字节、重试粒度和内存。多线程 range `Get` 只有在对象布局、SDK 和网络允许时才会提高有效吞吐。

Lustre 与 IBM Storage Scale 等并行文件系统面向高并发和高吞吐。例如，Lustre 使用多个 Object Storage Target（OST）提供文件数据。把大型文件 stripe 到多个 OST 可以聚合带宽，但 stripe count/size 应匹配文件大小、并发 reader 和 OST 负载；对小文件或大量并发文件，盲目扩大 stripe count 会增加 metadata 与锁开销。

跨 OST 对文件进行条带化，意味着文件 chunk 分布在不同 target 上，从而提供并行读取的可能。例如，可把 Arrow、TFRecord 或 Parquet 文件 stripe 到 4 个 OST。若每个

OST 在该访问模式下可独立提供 500 MB/s，且客户端、网络和其他组件不受限，理论聚合上限为 2 GB/s；协议开销、负载不均和共享竞争会使实测值更低。

数据调优、复制与压缩

调优这些文件系统时，应查阅对应版本的文档。例如，Lustre 可用 `lfs setstripe` 设置大文件或目录的 stripe count/size，使数据分布到多个 OST 并聚合带宽；4 或 8 个 OST 只是示例，最佳值取决于文件大小、并发和集群负载。

训练期间应监控文件系统 I/O，例如使用 Lustre Monitoring Tool（`lmt`）或厂商工具，检查 OST、metadata server、client 和网络是否出现热点。热点可能来自 striping/sharding 不均，也可能来自少量热门文件、metadata、重建、设备降速、client 拓扑或其他 workload，不能预先归结为一种原因。

在某些场景中，可以把数据集复制到每个计算节点，从而让 steady-state 训练主要读取本地存储。这要求每节点有足够容量，还需付出预置、版本一致性、epoch shuffle、故障恢复和弹性扩缩容成本；首次填充与 cache miss 仍可能产生网络读取。因此，全量复制不保证立即提速，应按作业持续时间和重用率核算。

另一种方案是把数据以压缩形式存入文件系统或对象存储，并在读取时解压，例如 JPEG 图像，或在 Arrow IPC/Parquet 容器中使用受支持的 column/page compression codec。Arrow 和 Parquet 本身不是“压缩文本”格式。压缩可用 CPU/GPU 解码成本换取更少 I/O 字节；当存储或网络受限且计算资源有余量时，这可能是合理权衡。

许多 data pipeline 已经使用压缩，但并不需要在每一步都重复压缩。应测量压缩率、encode/decode 时间、随机访问粒度、CPU/GPU 占用和端到端 samples/s，避免转码、重复压缩或 decompression 成为新瓶颈。

nvJPEG 等库可以在 GPU 上解码图像。部分 NVIDIA 数据中心 GPU/平台还提供硬件 decompression engine，可在相应 CUDA API、数据路径和 codec 支持下加速 LZ4、Snappy、Deflate 等格式。Blackwell 平台可在 pipeline 中利用这类能力，并减少某些解压任务占用 SM；但支持范围依具体 GPU/SKU 和软件版本而定，不能仅因 workload I/O-bound 就无条件偏好这些格式。

这提供了把部分 decode/decompression 从 CPU 卸载到 GPU 或专用引擎的途径，并可能与训练计算重叠。Grace Hopper/Blackwell 等特定平台的 NVLink-C2C 宣称最高约 900 GB/s 双向聚合带宽，但这不是任意 CPU-GPU 传输可获得的单向应用吞吐，也不能自动消除 CPU-assisted stage 的瓶颈；实际结果仍受内存池、访问方向、并发和软件流水线限制。

合理使用这些软件与硬件 offload 能减少部分 CPU 工作，并帮助平衡 input pipeline。最终仍应比较压缩后的存储/网络节省与 decode、数据移动、内存占用及资源竞争成本；如果 decompression 取代 I/O 成为关键瓶颈，整体方案就需要重新选择 codec、batch/layout 或执行位置。

监控存储 I/O

与所有性能工程任务一样，测量是关键。与监控网络通信类似，应组合使用不同层次的工具，观察从存储设备、文件系统和网络到 loader 与 GPU 的完整 data pipeline。

可用工具包括 Linux `iostat`、`iotop`、`nvme-cli`、`perf` 和 `eBPF`，以及厂商工具与 dashboard，用来观察 queue depth、吞吐、latency、readahead、cache hit 和错误/重试。不同工具覆盖的层次不同：本地 NVMe 指标不能单独证明 NAS 或对象存储的网络链路已饱和，还需结合 NIC、RDMA、客户端和服务端计数器。

还可以使用 Nsight Systems 跟踪 I/O API 活动，并观察它与 GPU kernel 的时间关系。受支持版本可使用 `--trace=gds` 在 timeline 中捕获 `cuFile` API 事件；也可依据当前 GDS/Nsight 文档启用 GDS `cuFile` tracepoint，通过 `/etc/cufile.json` 配置相应 `cuFile` tracing。API timeline 不等同于完整设备数据路径：NVMe peer-to-peer DMA 的 kernel/device counter 可能不在 Nsight Systems 中暴露，也并非所有 GDS stack 都提供相同可见性。

NVIDIA Data Center GPU Manager（DCGM）可报告 GPU 利用率、内存、PCIe/NVLink 等受支持的设备指标。它不是通用文件系统 I/O 监控器，应与 host OS、存储、NIC/RDMA 和应用级指标结合，判断 GPU idle 是否确由数据供应不足造成。

在 PyTorch 中，对 `next(data_iterator)` 计时，测到的是主线程取得下一 batch 的 wall time，而不是 GPU 总空闲时间。后台 worker/prefetch 已经完成多少工作会影响该值；H2D copy 是否包含在内则取决于 dataset/training loop 的边界。应结合 GPU timeline 和 iteration critical path，才能判断 GPU 是否在等待数据。

若要观察同步 dataset/transform 路径，可临时设置 `num_workers=0`，从而移除 DataLoader worker 进程及其后台预取，再测量 iterator pull。不过这会改变执行模型，不能直接代表正常多 worker 配置的成本。可另行测量 `.to("cuda")` 和 pinned-memory staging；异步 GPU copy 应使用 CUDA event 或 profiler，并在正确依赖边界读取结果。

瓶颈可能位于 Python/data pipeline，*也可能*位于进入 GPU memory 的 `memcpy`、同步或上游存储。可以进行以下分解测量，并与 iteration timeline 中实际的 GPU idle interval 比较：

DataLoader 与 Python 路径成本

> 使用 `num_workers=0` 测量主进程中的 Python loop、读取与 transform。这会移除后台 worker 进程及 prefetch，而不是原文所称的“后台线程调度”；结果应与正常配置分别解释。

Host → Device copy 成本

> 查看 Nsight Systems 的 “Copy” lane，测量 device transfer 的执行时间及其是否位于 critical path。也可记录 `torch.cuda.Event`，把 event 放在 `.to("cuda")` 调用前后；event 测量 GPU timeline 上的 elapsed time，但只有结合 stream dependency 才能判断 copy 是否真的导致 GPU stall。

把这些测量与整体 iteration critical path 对照，可以判断应优化 Python pipeline（例如调节 worker、简化 transform）、H2D 路径（例如合理使用 pinned memory 和异步 copy），还是底层存储/网络。GDS 只解决兼容存储到 GPU memory 的特定数据路径，不是任意 DataLoader 或 H2D 瓶颈的直接替代方案。

例如，监控可能显示每个 iteration 有 30% 的时间在等待数据，调优后降至 5%。如果每步有效计算量不变，原总时间分别按 `compute / 0.70` 与 `compute / 0.95` 表示，理想吞吐加速约为 `0.95 / 0.70 ≈ 1.36×`，而不是原文把等待比例相除得到的 6×。真实收益还取决于等待与计算是否重叠、batch 和模型是否变化，以及剩余瓶颈。

存储与 I/O 优化常常需要消除累积的小额开销，例如额外 5 ms latency，或与请求模式不匹配的 10 MB buffer；但参数是否“太小”必须通过测量判断。在规模化系统中，应监控并分析从 disk、filesystem、network、loader 和 H2D copy 到 GPU memory 的各环节，目标是提高端到端 goodput 并保留延迟与恢复余量，而不是让每项资源无条件 100% 饱和。

调优数据流水线

除了原始存储 I/O，CPU 或 GPU 上的预处理与数据加载流水线也是 AI workload 整体性能的关键。调优目标是让数据按 critical path 所需速率到达 GPU，并合理安排并行 CPU 工作，同时避免为了追求零空闲而过度预取或争用资源。

现代深度学习框架提供加载和预处理数据的高级 API，其配置应随 workload 测量和调节。下面讨论一般策略，以及 NVIDIA DALI、NeMo 等面向高级数据流水线管理的工具。

高效加载与预处理数据

典型训练数据加载流程包括从存储读取，解析 JSON、解码 JPEG 等反序列化/解码，执行文本 tokenization、图像 crop 等 transform，再整理为 batch。这些步骤可能消耗大量 CPU；在算子、数据路径和 GPU 余量合适时，也可将部分工作移至 GPU。可采用以下技术维持吞吐：

使用多个 worker 进程或线程

> PyTorch `DataLoader` 等 API 允许设置 `num_workers`。每个 worker 并行获取和预处理数据；在 PyTorch map/iterable dataset 的多 worker 模式下，它们通常是独立进程，主进程通过队列接收结果。这样可绕开部分 Python GIL 限制，但会增加进程、IPC、内存副本、文件描述符和 dataset 分片管理成本。

避免 Python 瓶颈

> 若加载逻辑包含大量 Python-level 处理，应通过 profiler 判断逐元素 loop、对象创建或序列化是否占用 critical path。可行时使用批处理/向量化操作，或调用 Rust/C/C++ 实现，例如 Hugging Face Tokenizers 等库。不能仅凭具有 Python binding 就判断性能；数据转换、跨语言边界、线程池和小调用粒度仍需测量。TorchText 的组件与维护状态也随版本变化，应按当前生态选择。

重叠 CPU 与 GPU 工作

目标是在 GPU 处理 batch N 时，让 CPU 加载并预处理 batch N+1，必要时放入 pinned memory，同时开始准备 batch N+2。不过 multiple worker 通常只提供后台加载/预取；H2D copy 与 GPU compute 的重叠还需要 pinned source、nonblocking enqueue、独立 stream、正确依赖和可用 copy engine。框架不会仅因增加 worker 就自动保证完整重叠，因此必须在 timeline 中确认。

整理 tensor 并按 batch 执行操作

适合时，可用向量化操作一次处理整个 tensor batch，而不是逐 sample 调用。可以通过自定义 `collate_fn()` 整理 batch，也可在 training loop 中把适合的操作移至 GPU。batch 化并非总是更好：可变长度数据、随机 augmentation、内存峰值、padding 浪费和 load balancing 都会影响最佳边界。

在数据加载中使用锁页内存

在 PyTorch DataLoader 中启用 `pin_memory=True`，会尝试把返回 tensor 放入 pinned host memory；配合 `.to(..., non_blocking=True)` 可让兼容 H2D copy 异步入队。pinned memory 避免驱动临时 staging，并满足 DMA 的页稳定条件，但锁页/注册、pinning thread、RAM 压力和 NUMA 放置也有成本，并非几乎总是有利。应按并发 buffer 容量设置合理的 `ulimit -l` 或容器 `--ulimit memlock`，而不是无条件设得极高。

预取 batch

部分框架允许设置 prefetch queue depth。PyTorch DataLoader 在 num_workers > 0 时常见默认 `prefetch_factor=2`，语义是每个 worker 预取的 batch 数；总体 in-flight 数量大致与 `num_workers * prefetch_factor` 成比例，但内部队列、消费时序和版本实现会影响瞬时状态。I/O 有突发或 GPU 偶发饥饿时可测试 4 或 8，也要监控 host/pinned memory 和 cache 压力。下面的 PyTorch 示例同时使用 `DataLoader`、`pin_memory` 与 `prefetch_factor=4`：

```
import torch
from torch.utils.data import Dataset, DataLoader

# Create a Dataset and DataLoader that prefetches 4 batches
# per worker into pinned CPU memory.
class Synthetic(Dataset):
    def __init__(self, n, shape): self.n, self.shape = n, shape
    def __len__(self): return self.n
    def __getitem__(self, i):
        # Cheap CPU-side work; replace with real parse/decode
        return torch.ones(self.shape,  dtype=torch.float32)

B, C, H, W = 32, 3, 224, 224
dataset = Synthetic(n=100_000, shape=(B, C, H, W))

loader = DataLoader(
    dataset,
    batch_size=B,
    num_workers=8,
```

```
        pin_memory=True,
        persistent_workers=True,
        prefetch_factor=4,
    )

copy_stream = torch.cuda.Stream()
compute_stream = torch.cuda.current_stream()

for batch in loader:
    with torch.cuda.stream(copy_stream):
        batch_gpu = batch.to(device, non_blocking=True)
    # ensure pending H2D completes before compute uses it
    with torch.cuda.stream(compute_stream):
        torch.cuda.current_stream().wait_stream(copy_stream)
        outputs = model(batch_gpu)
```

该示例配置 8 个 worker，每个 worker 的 `prefetch_factor` 为 4；这表示每 worker 预取 4 个 batch，不是原文所称“batch size 为 4”。实际 batch_size 是 32。更严重的是，Synthetic dataset 的单个 sample 已返回 `(B,C,H,W)`，DataLoader 又按 B 个 sample collate，因此结果形状会成为 `(B,B,C,H,W)`，与普通图像 batch 不符。代码按原书保留，仅用于展示 API 位置。pinned host memory 使 `.to(device, non_blocking=True)` 有条件异步使用 DMA，但不保证高吞吐。

DataLoader worker 可以在 GPU 处理当前 batch N 时准备后续 batch；示例还使用独立 copy stream 并建立 wait dependency。是否真正实现 batch N 计算与 batch N+1 H2D overlap，取决于循环的 enqueue 顺序、数据依赖、copy engine 和计算/带宽竞争，不能从代码表面直接断言。没有预先 pinned 的 source 时，驱动可能需要 staging 或同步处理，但 pinned memory 也不能保证最大吞吐。

还可启用 `persistent_workers=True`，让 worker在一个 *epoch* 的 dataset 消费结束后继续存活。这对短 epoch、频繁重复 dataset，或 import/open/init 成本较高的 worker 尤其有用，可减少跨 epoch 的进程重建。但它不会无条件让队列跨 epoch 持续填充，并会保留 worker dataset 状态、内存、文件句柄和随机数状态，生命周期与 reseeding/sharding 必须正确管理。

debug logging、昂贵 CPU transform 或共享资源竞争都可能引入隐蔽瓶颈，并且只在负载下显现。可以先隔离 profile DataLoader，测量产出 100 个 batch 的 warm steady-state 时间，同时关闭下游 GPU 工作；还应单独记录 startup、首 batch、cache 状态与方差。然后将 loader service rate 与目标 iteration rate、正常训练 timeline 和 GPU idle interval 比较。

若 isolated DataLoader 过慢，可减少逐元素日志、简化 transform 或调节 worker。若隔离测试与真实运行差距很大，不能只归因于 H2D 或 kernel-launch overhead；CPU/GPU 争用、NUMA、page cache、存储并发、内存压力、队列 backpressure 和同步都可能造成差异。

> 关闭 GPU kernel 会减少 CPU launch 工作，也会消除训练对 CPU、内存、PCIe 和存储的部分争用；因此 isolated loader throughput 通常可能高于真实训练，而不是原文所称“通常更低”。另一方面，真实训练的 queue backpressure 可能让 producer 表观速率下降。隔离测试仍有价值，但必须与集成 timeline 配对解释。

随 GPU 数量扩展 worker

增加 GPU 时，也应按目标 aggregate samples/s 和 bytes/sample规划 data pipeline，否则设备可能等待输入。可调资源包括每 rank 的 DataLoader worker、CPU 核心、内存、存储/网络带宽、解码器和 prefetch；并非总要机械增加 worker。GPU 数量增加也不必然要求提高 global batch size，batch 策略还受优化收敛、显存、并行方式和 gradient accumulation 影响。

若只扩展计算而不扩展 ingestion 资源，瓶颈可能转移到数据加载。在多节点 data parallel 配置中，正确配置的 sampler/dataset 通常让各 rank 处理互斥或尽量不重复的样本子集，但原文所称“每个 rank 读取不同物理 shard”并非自动保证；逻辑索引划分、文件布局与本地 cache 需要分别协调。聚合输入需求通常随并发 rank 增长，但也受 per-rank batch、模型吞吐和数据复用影响。

> 应持续测量 CPU 利用率，同时观察 per-core/NUMA、内存带宽、loader queue、存储与 GPU idle。随着 GPU 训练加速，input pipeline 更可能暴露为瓶颈，但并非必然；瓶颈也可能在通信、模型计算或同步。

为维持所需吞吐，大规模集群通常需要可并行扩展的高带宽存储或分层 cache，并正确划分数据请求。若采用每节点本地 shard/cache，应确认每节点容量与吞吐能承担其份额，同时规划 shuffle、replication、节点故障、再平衡和远端回退；并非所有节点都必须保存固定独占的数据分片。

使用 NVIDIA DALI 处理多模态数据

对于复杂或繁重的数据预处理，NVIDIA 提供 Data Loading Library（DALI）。DALI 通过 GPU operator 或优化的 CPU 实现执行数据处理，尤其适合可从硬件/并行解码与 augmentation 获益的图像、视频等数据。收益取决于格式、operator placement 和现有瓶颈。

例如，DALI 可以通过受支持的 mixed/GPU decoder 路径解码 JPEG，并在 GPU 上执行 random crop、resize、normalization 等 operator。它可能比 CPU pipeline 更快并减少部分 CPU worker，但 JPEG 硬件路径、fallback、batch、GPU 余量和与训练计算的资源竞争都会影响结果。

DALI pipeline 以 operator graph 声明。原文介绍的 class-based API 是继承 `nvidia.dali.pipeline.Pipeline` 并在 `define_graph()` 中声明 data source 与 CPU/GPU operator；现代 DALI 还提供 functional API 和 pipeline definition decorator。DALI 使用自身的 executor、prefetch queue 和 thread pool 管理执行，但具体 API 与行为应以所用版本为准。

若 workload 确实 input-bound，集成 DALI 可能显著提高吞吐；若 GPU compute 已饱和，把更多预处理放到 GPU 反而可能争用 SM、memory bandwidth 或 decoder。DALI 还需与 framework iterator、sharding、random seed、last-batch policy 和训练 loop 正确集成，会增加一定复杂度。

对于分类、目标检测、分割等常见 workload，DALI 提供 readers、decoders、augmentation operator 和示例 pipeline，可组合图像/视频 GPU 路径。并非所有 operator 或 codec 都使用专用 media hardware，也不能保证“完全利用”硬件；运行时路径应通过 DALI/Nsight 指标确认。

设想一个读取图像和视频、执行 augmentation 并训练目标检测模型的 data pipeline。监控可能显示 CPU utilization 为 800%，即按常见工具口径相当于 8 个逻辑 CPU 满载，而 GPU 仍偶尔等待输入。这个数字只是示例，还应查看 per-core、NUMA、I/O wait 和实际 queue。

在合适的示例中，DALI 可能把 CPU utilization 降到约 200%，让 CPU 主要承担读取与控制，而由 GPU/专用 decoder 执行部分图像视频解码。这些不是通用比例。存储读取通常仍由 CPU/client/runtime 提交；数据传输、decode 与模型计算是否并发，取决于 DALI executor、queue、stream、硬件引擎和资源竞争，不能概括为“GPU 同时读取和计算”。

实际加速很大程度上取决于 DALI 在 pipeline 中的位置。如果只在 GPU 上解码 JPEG，随后立即把 raw pixel 传回 CPU 做 augmentation 和 collation，就会增加 device-to-host 及后续 host-to-device 数据移动，可能抵消收益。应尽量让相邻算子保持在合适的同一处理域，同时满足框架的数据格式与生命周期要求。

> 另一种思路是识别适合 GPU 的预处理操作，并尽可能在 GPU computation graph 或连续 operator 中执行，以减少 CPU/GPU 往返。TorchVision 同时包含 CPU 与部分 CUDA operator，并非整体都是 CUDA-based；TensorRT 主要用于神经网络 inference 优化，也不是通用数据预处理库。可以比较 DALI、TorchVision/Kornia 等适用库、framework operator 或定制 CUDA kernel，但 fusion、正确性和维护成本都需评估。端到端比较比按产品名称推断更可靠。

仍应在真实条件下对端到端系统进行 benchmark。比较 CPU-only pipeline、DALI pipeline 和适用的 GPU graph/融合实现，并同时衡量 samples/s、CPU/GPU 利用率、latency、显存、功耗、模型正确性与集成复杂度，选择适合目标模型和数据集的方案。

使用 NVIDIA NeMo Curator 创建高质量 LLM 数据集

NVIDIA NeMo 是一组面向生成式 AI 模型开发、训练和部署的组件；其生态中包括开源数据处理项目 NeMo Curator。NeMo、NeMo Framework 与 NeMo Curator 的包边界和名称会随版本演进，使用时应以当前项目文档为准。

Curator 用于准备大型文本和多模态训练数据，适合处理来自不同来源的 TB 级乃至更大数据集。可用步骤包括读取/转换、清洗、过滤、去重、分类和部分 shuffle/分片等；tokenization 等具体能力与推荐 pipeline 依版本和下游训练格式而定。

NeMo Curator 可利用分布式执行后端，把适合的预处理扩展到多个 GPU 或节点。并非每个步骤都能或应该使用 accelerator：文本解析、I/O、shuffle、GPU embedding/inference 等算子的瓶颈不同。多节点还需计算调度、网络、shuffle 和中间数据写入成本，规模增加不保证线性提速。

Curator pipeline 可以把处理结果写为 sharded JSONL、Parquet 等格式，并可配合后续工具执行压缩、packing 或转换为训练所需 binary/indexed dataset。它也可用于合成数据生成/处理流程，但 synthetic data 不是天然高质量补充：必须评估来源授权、隐私、偏差、重复、污染、模型自训练退化和任务有效性。

把昂贵且可复用的预处理放在训练前离线完成，可以简化在线 training pipeline，使其主要读取已准备的数据并执行轻量 last-mile shuffle/augmentation。不过 offline artifact 需要版本、lineage、schema、tokenizer 与 reproducibility 管理；数据变更时还要考虑增量重建成本。

NeMo Curator 还可通过 exact/fuzzy/semantic dedup 与多种 classifier/filter 辅助数据质量处理。过滤规则不能自动“保证”质量，阈值会影响覆盖率、代表性与误删；敏感/问题内容也需要领域政策和人工评估。预先完成清洗、tokenization 或 sequence packing 可以减少训练时 string processing，但把所有样本 padding 到固定长度可能浪费 token compute，应比较 dynamic batching、packing 和长度 bucketing。

使用 Curator 等工具预先减少训练期文本处理和海量随机小文件访问，通常有助于让训练资源集中在 forward/backward。部分 NeMo/Megatron 训练流程使用 memory-mappable *.bin* data file 和 *.idx* index file；Curator 的 `DocumentDataset` 或对应版本的数据抽象可读取/写入 sharded JSONL、Parquet 等。把它们转换为 *.bin/.idx* indexed dataset 是独立的下游构建步骤，格式选择应与具体训练框架版本匹配。

> 可以考虑预生成若干不同 shuffle order，但为 N 个 epoch 保存 N 份完整数据通常会把存储、写入、校验和 cache footprint 放大到 N 倍。更常见的选择包括保存索引/permutation、deterministic sampler、shard-level shuffle 和有限 shuffle buffer；应在随机性质量、运行时开销、故障恢复与容量之间权衡。

一般而言，应在训练前完成可复用、确定性且昂贵的数据准备。原文所称“几乎绝不应使用 raw text 训练”过于绝对；研究迭代、streaming corpus、在线 augmentation 或动态 tokenizer 场景可能有不同权衡。离线预处理能否回本，取决于数据重用次数、变更频率和 artifact 构建成本，并可能带来更稳定的扩展表现。

这些技术旨在减少 data loading 位于 critical path 的时间。昂贵 GPU 若长期等待输入会降低 goodput，因此需要同时分析 storage、network、CPU、GPU 与应用队列。不过短暂空闲可能来自同步、阶段边界或合理资源余量，不应把“绝不空闲”当成独立目标。

> 不能把 NeMo 的所有 data loading 笼统归为 CPU-only；reader、transform 和 training stack 的执行位置依组件与版本而定。GDS 可以在兼容条件下绕过 host-memory staging copy，但不会绕过 CPU 的 I/O submission/control，也不会自动加速 JSON parsing、tokenization 或其他 CPU transform。

当 input pipeline 是主瓶颈时，调优数据路径可能比单个 kernel 或算法微调带来更大收益；若输入已被隐藏，则收益会较小。原文所举浪费 50% GPU 时间、算法优化仅几个百分点没有通用依据，应以目标 workload 的 profiler、质量指标和端到端 cost/time-to-train 决策。

持续性能分析与调优工作流

性能工程是一个迭代过程。为让分布式训练或推理应用在扩展和修改后仍保持效率，应持续收集可比较的性能数据、提出瓶颈假设、实施优化，再重新测量。

硬件、driver、library、compiler、数据与模型更新都会改变最佳配置。性能团队通常维护 dashboard，长期跟踪 samples/sec、latency percentile、goodput、错误率、功耗和成本等指标，并记录软硬件与 workload fingerprint。

可以设置自动化 nightly/周期性 benchmark，使用固定数据、seed、warm-up、运行时长和资源隔离，并重复测量。这样才能区分真实回归/改进与共享集群噪声，再把变化关联到代码、环境或数据版本。

下面给出一个可广泛用于 profiling 和 debugging 的典型工作流，而不限于本章主题：

建立基线

> 可从单 GPU 或最小配置开始，测量训练 samples/s、time-to-train/quality，或推理 latency percentile、throughput 与 SLO，再扩展到单节点多 GPU 和多节点。简单 data parallel 在保持每 GPU workload 的弱扩展中，N 块 GPU 的理想吞吐接近 N 倍；强扩展、固定 global batch 或同步开销下并不期望严格线性。若 8 GPU 相对 1 GPU 只有 5 倍吞吐，按该特定 speedup efficiency 口径为 5/8 = 62.5%，但是否可接受取决于规模、成本、质量和延迟目标。

分析多 GPU 运行中的瓶颈

> 使用 Nsight Systems（`nsys`）等 system profiler 观察多 GPU 作业的代表性窗口，检查 GPU/CPU timeline、collective、I/O、launch 和 synchronization。GPU 在 all-reduce 附近出现空隙只能说明该区间位于 critical path，原因还可能是迟到 rank、依赖或 enqueue 延迟；iteration 开头空隙也可能来自 data loading、CPU、同步或 kernel launch。应结合 rank timeline 与计数器验证，而不是仅凭空隙命名瓶颈。

必要时深入具体 kernel

> 如果某个 compute kernel 运行慢于预期，可用 Nsight Compute（`ncu`）检查 occupancy、SM issue、memory traffic、stall sampling 等指标。对 NCCL/device communication kernel，Nsight Systems、NCCL log 和 fabric/PCIe/NVLink counter 往往同样重要。原文给出的 SM busy 从 60% 升至 90% 是未提供实验配置的示例，且 SM busy 高低不能单独证明 PCIe/NVLink 原因。kernel 还可能同时受 dependency、latency、instruction、memory、network、launch 或资源竞争限制，并非 network/memory/compute 三选一。

识别原因

> 发现瓶颈现象后，应列出可证伪假设并逐项做对照实验。网络受限可能来自 transport、message size、拓扑、拥塞或 overlap；rank 等待可能来自数据不均、硬件降速、OS noise 或不同步输入；CPU 受限可能来自 loader、通信 progress、logging 或内存；GPU memory-bound 也需区分 HBM traffic、cache、occupancy

和算子形状。减小 batch 或 kernel fusion 不是通用答案，它们可能改变效率、显存、通信比例和模型收敛。

应用修复或优化

确认原因后再选择修复。通信问题应先验证实际 transport、GPU/NIC 拓扑、RDMA/GPU Direct、message 与 collective。`NCCL_NSOCKS_PERTHREAD` 只调节 socket transport 的每 helper thread socket 数，不会因为存在多 NIC 就启用 RDMA multirail。gradient compression 会引入 encode/decode、误差与模型质量权衡；NUMA/层次算法也应由真实拓扑和 collective benchmark 驱动。

对于节点内拓扑问题，应按 GPU-GPU、GPU-NIC、PCIe switch 和 NUMA 路径选择设备与算法；把作业限制到单一 NUMA domain 可能改善局部性，也可能减少可用 GPU，需比较总 goodput。CPU/data 问题可测试 worker、DALI 或离线预处理，但增加并发也可能加剧争用。若单 rank 因 validation/logging 变慢，可减少频率、分离职责或把安全的工作移出 critical path；异步操作仍需正确依赖、buffer 生命周期和错误处理。

若同步位于关键路径，可移除经验证不必要的 `torch.cuda.synchronize()` 或 barrier，第 13 章会进一步讨论；不能删除保证正确性的同步。`NCCL_IGNORE_CPU_AFFINITY=1` 会让 NCCL 忽略进程 CPU affinity 并采用 GPU affinity，只有 launcher/cpuset 与期望拓扑冲突且实测改善时才应使用。CPU pinning 同样需考虑 scheduler、container cpuset、IRQ 和 NUMA。少量修改有时收益很大，但“maximum utilization”不是独立目标：

每次修改后重新测量

每轮尽量只改变一个可解释因素，并重复测量；若必须同时修改多个耦合配置，应采用明确的实验设计。保存 workload、环境、分布统计和置信范围，而不只记录一个“好数值”，这样扩展时才能判断变化是否超过噪声。

更新软件，但始终验证

新版 NCCL、CUDA 或 PyTorch可能改进 algorithm、protocol、compiler 或 framework overhead，也可能改变默认值、compatibility、数值结果和最佳参数。升级后应重新运行 correctness 与 performance suite，并与 last-known-good 系统配置按相同 workload 比较；不能假设新版会自动启用特定 hierarchy 或 overlap。

利用现代硬件能力

使用新硬件时，先确认 memory capacity、memory bandwidth、interconnect、media engine 和支持的软件矩阵。原文笼统称“更多 unified memory”，但许多平台仍有不同物理内存池。更大容量可能容纳更大模型或 batch，却不表示更大 batch 在优化上合适；应逐步扩展并监控新的瓶颈。

在生产环境自动化监控

对经常运行的大型训练和推理 workload，应统一采集 GPU、network、memory、storage、latency、error 与 application goodput，并关联 job、node、版本和数据。Kubernetes 等 scheduler 可集成监控，但 alert 不应只依赖 utilization 低于固

定阈值；需要结合 workload phase、SLO、历史基线、持续时间和多指标，减少误报。

记录并共享知识

性能调优常包含环境变量覆盖、版本缺陷和拓扑假设等隐性知识。应在代码、配置或 runbook 中记录结论、适用环境、benchmark 方法、日期与回滚条件。例如，“在该集群和该 workload 中，`NCCL_SOCKET_NTHREADS=2` 使多节点吞吐提高 10%”；不要把单次结果变成无条件默认值。

持续执行这一流程，就形成 运行 → 测量 → 调优 → 再运行的反馈环。随着 GPU 规模、模型、数据和软件变化，持续回归测试比性能长期退化后再恢复更容易。目标是保持可解释、可重复的性能，而不是让所有组件始终处于最高利用率。

总之，应把 performance 当作需要测试和验证的产品属性，同时与 correctness、quality、reliability 和 cost 共同设门槛。GPU 翻倍时吞吐是否接近翻倍，要先明确强/弱扩展和 batch 口径。Nsight Systems 提供系统 timeline，Nsight Compute 深入适用的 GPU kernel，NCCL/PyTorch log 说明 framework 与 communication 行为；还需结合 CPU、storage、network 和 application telemetry 才能完成归因。

完成一轮流程后，可以得到针对当前 workload 与环境调优的系统；代码、数据或硬件更新后仍需迭代。性能调优没有永久终点，但稳定的基线、自动化实验和可复核记录会让后续工作更高效。

诊断通信受限与计算受限 workload

为了判断模型训练 workload 主要受计算还是通信限制，可以设计实验改变 compute/communication ratio，并同时观察 iteration critical path 与网络指标。假设某次 backward profile 显示 gradient all-reduce 在标称 100 GB/s NIC 上达到 60 GB/s；首先要确认这是 NIC port rate、NCCL algorithm bandwidth、bus bandwidth 还是 payload/time，单位口径不能混用。

一种实验是保持模型、参数 dtype、world size、optimizer/parallelism 与 communication hook 不变，改变 per-rank batch size 来改变计算量。对标准 dense data-parallel all-reduce，每次 optimizer step 的 gradient tensor 总元素数主要由参数决定，通常不随 microbatch size 改变；但这并非“完美固定通信”：gradient accumulation、unused/sparse parameter、compression、bucket rebuild、mixed precision、ZeRO/FSDP 和算法选择都可能改变字节数或调用模式，batch 还会改变 gradient ready time 与 overlap。

在确认 collective payload 与配置确实不变后，可把 batch size 减半并重复测量。若 all-reduce 仍为 60 GB/s，这只说明该通信阶段的 measured rate 对本次 compute 变化不敏感；要判定 network 是总 iteration limiter，还需确认通信位于 critical path、没有被 overlap 隐藏，并检查 collective latency、queue、fabric counter 与 rank skew。

如果 measured rate 从 60 降到 40 GB/s，也不能直接断言“GPU 太慢、network 等待数据”。更小 batch 可能缩短/改变 kernel、bucket readiness、collective concurrency、message scheduling 和 measurement window，导致链路利用率下

降；host enqueue、rank skew 或 algorithm/protocol 切换也可能造成相同现象。应对齐单次 collective 的 payload、duration 和启动时刻，再分析 compute 是否位于关键路径。

还可反向增大 batch size 进行交叉验证。在 dense DDP 前提成立时，all-reduce payload 可能保持近似不变，但 compute duration 和 overlap window 会增长。communication 占总 iteration 的比例下降，既可能是更多计算稀释固定通信时间，也可能是 overlap 改善；它不独立证明 compute 是原瓶颈。应比较 exposed communication time、iteration time 与模型吞吐，而不只看百分比。

联合观察 NIC/fabric GB/s、collective algorithm/bus bandwidth、通信绝对时间、exposed time、compute time、rank skew 和 end-to-end samples/s，比单一指标更可靠。随 batch size 绘制这些量可以显示趋势，但这不是严格的 roofline，也不能“精确”把 workload 二分为 network 或 GPU SM bound；memory bandwidth、CPU、launch、storage、同步和混合瓶颈仍可能主导。更可靠的实验还包括固定 compute 的 collective microbenchmark、禁用/替换通信的上界测试，以及受控 artificial delay。

> 使用 Nsight Systems 获取端到端 timeline。compute kernel 之间与 NCCL 相关的长 gap 表明 communication/dependency 可能暴露在关键路径，但仍需区分 host wait、迟到 rank 和 collective 本身。GPU busy 但 FLOPS 低可能来自 memory、instruction mix、occupancy、latency 或通信 kernel，而不是简单的“memory 或 compute”。Nsight Compute、PyTorch profiler、NCCL log 与 fabric counter 应联合使用。

关键要点

分布式 AI 的高性能来自全栈协同优化，包括 GPU kernel、网络传输、CPU thread、数据流水线和存储；任一环节位于 critical path 都可能限制整体 goodput。调优存储层时，应牢记以下要点：

随计算规模扩展输入数据流水线

扩展 GPU 时不能忽略存储和数据加载。应按目标 samples/s、bytes/sample、metadata rate 和故障余量提供足够资源，但不必追求存储长期 100% 饱和。DataLoader 并行度也不应机械地与 GPU 数线性增长；CPU、内存、I/O、NUMA、cache 和 queue depth 都可能先达到最佳点。若 input pipeline 无法跟上，继续增加 GPU 的边际收益会下降。

针对任务选择合适工具

NCCL 面向 GPU collective 和 point-to-point communication，常用于分布式训练；NIXL 面向异构 memory/storage 的高吞吐 transfer abstraction，常用于推理中的 KV cache 与 inference-state movement，并不等同于 token streaming protocol。NVSHMEM 提供 one-sided communication 与 symmetric heap programming model，不是 NCCL collective 的简单同类替代。GPUDirect RDMA 和 GDS 分别减少兼容网络/存储路径中的 host-memory bounce buffer，CPU 仍负责控制与提交。PyTorch 多进程单 GPU 训练通常优先 `DistributedDataParallel`，而

`DataParallel` 的单进程多线程模型常扩展较差；但“始终”选择某一 API 仍需结合模型并行、FSDP、单机原型和框架约束。

进行端到端性能分析

瓶颈通常不能凭单一 utilization 推断。可结合 Nsight Systems、Nsight Compute、PyTorch profiler、NCCL log，以及 CPU、storage 和 fabric counter 分析时间与资源路径。workload 可能同时受到 compute、memory、communication、I/O、CPU、launch 或 synchronization 限制，而不是三选一。应验证 NCCL kernel 与 compute 的实际 overlap、GPU idle 的依赖来源，并以端到端 throughput/latency/quality 确认优化收益。

结论

高性能分布式存储是调优大型复杂 AI 系统的重要基础。正确集成 NVMe SSD、GDS 及相应文件系统与数据格式，可以改善 data loading pipeline、缩短训练时间并提高实验迭代速度；收益取决于 workload 是否受这些路径限制以及部署是否兼容。

通过离线预处理、合适的 cache/数据布局，以及在依赖允许时采用异步 I/O 与传输，可以在模型与数据集扩展时维持较高吞吐。每项技术都有容量、一致性、CPU/GPU 资源、尾延迟和故障恢复代价，应以端到端 goodput 衡量。

实践者通常不需要从零自研 I/O 栈。NVIDIA、云服务商、存储厂商和开源社区提供了针对特定问题优化的 library 与工具；应优先复用成熟组件，同时核对版本、compatibility、正确性和可观测性，而不是假设它们开箱即达到最佳性能。

对性能工程师而言，数据移动与原始计算能力同样重要。高性能 GPU 若经常位于 input wait 的关键路径，就无法发挥价值；反过来，单纯让存储或 GPU 满载也不保证更高模型 goodput。

GDS 和高级 input pipeline 是 full-stack 方法的一部分。持续 profiling、受控实验与调优能让分布式 AI 系统接近其在当前 workload、质量和成本约束下的可实现上界，但不能把理论硬件峰值当成所有端到端应用的预期结果。

后续章节将在此基础上讨论 CUDA、PyTorch 优化和更多系统调优主题。本章的原则仍适用于各层：识别 critical path，在正确依赖下重叠通信与计算，选择合适的数据路径，并用统一口径测量实际收益。最终目标是缩短 time-to-insight，提高资源效率与可靠性，并降低满足业务或研究目标的总成本。

第 6 章 GPU 架构、CUDA 编程与最大化占用率

本章首先回顾 single instruction, multiple threads（SIMT）执行模型，以及 warp、thread block 和 grid 如何把 GPU 算法映射到 streaming multiprocessor（SM）。

随后将深入 CUDA 编程模式，讨论 register file、shared memory/L1、L2 与 HBM3e 等 memory hierarchy，并介绍 GPU 异步数据传输能力，包括 Tensor Memory Accelerator（TMA），以及在受支持 Blackwell Tensor Core 路径中用作 accumulator storage 的 Tensor Memory（TMEM）。具体能力依 GPU 架构和指令/API 而定。

本章还将介绍 roofline analysis，用于分析 kernel 的 compute 与 memory traffic 上界。Roofline 是诊断模型而非二元分类器，其目标是帮助识别可实现性能与当前瓶颈，而不是保证达到理论峰值。

理解 GPU 架构

通用 CPU 通常强调低延迟与强单线程性能，GPU 则以大量并行线程获得高吞吐；这是一种概括，现代 CPU/GPU 都包含多种 latency/throughput 优化机制。图 6-1 展示了简化的 CPU-GPU CUDA 编程流程。

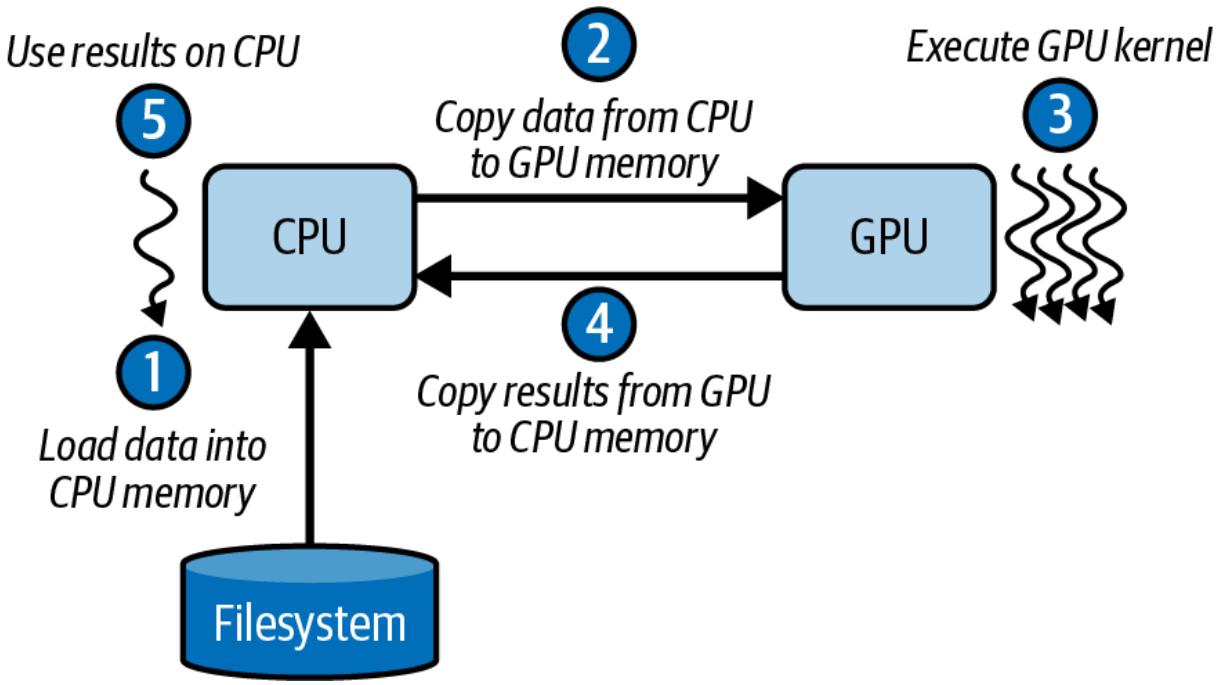


图 6-1. 简化的 CUDA 编程流程

在图示的传统显式复制流程中，host 先把数据载入 CPU memory，再复制到 GPU memory，随后启动 GPU kernel，并在需要 CPU 消费结果时把数据复制回 host。这只

是常见基本路径；unified/managed memory、zero-copy、GDS、peer access、异步 copy 和 device-resident pipeline 可能采用不同的数据移动方式。

GPU 主要通过在 warp 之间切换来隐藏 instruction、memory 和部分依赖 latency，而不是用 SM 内并行直接“隐藏”图 6-1 中整段 CPU-GPU transfer。GPU 包含多个 SM；它们与 CPU core 不能一一类比。对书中讨论的 Blackwell compute capability，单 SM 的最大 resident warp 数可为 64，每 warp 32 个 thread；具体限制应以目标 device attribute 和 tuning guide 为准。

每块 GPU 包含多个 SM，每个 SM 管理 warp scheduler、execution pipeline、register file、shared memory/L1 等资源。原文给出 Blackwell 每 SM 最多 64 个 resident warp（2,048 thread）、64K 个 32-bit register（256 KiB），以及合计 256 KiB 的 L1/shared-memory SRAM；其中最多约 228 KiB 可配置为 shared memory，单 block 可用动态 shared memory 约 227 KiB。此类上限依具体 architecture、SKU、carveout、static allocation 和 opt-in API 而定，不能把所有 Blackwell 或“现代 GPU”视为完全相同。

Blackwell SM 包含多个 warp scheduler/processing block，可从 ready warp 向相应 execution pipeline 发出指令。原文把它概括为四个独立 scheduler 每 cycle 最多各选择一个 warp，并进一步断言每个 scheduler 可从同一 warp dual-issue 一条 arithmetic 与一条 memory instruction。具体 issue width、配对和 pipeline mapping 属于 architecture-dependent 微架构细节，CUDA programming model 不保证所有指令组合或每 cycle 都能如此发射。

在有足够 ready warp、无 dependency/structural hazard 且指令组合可发射的理想条件下，多个 scheduler partition 可以并行推进 warp。图 6-2 是概念示意，不应把“发出指令”直接等同于四个 warp 的全部 thread 在同一 cycle 完成执行。

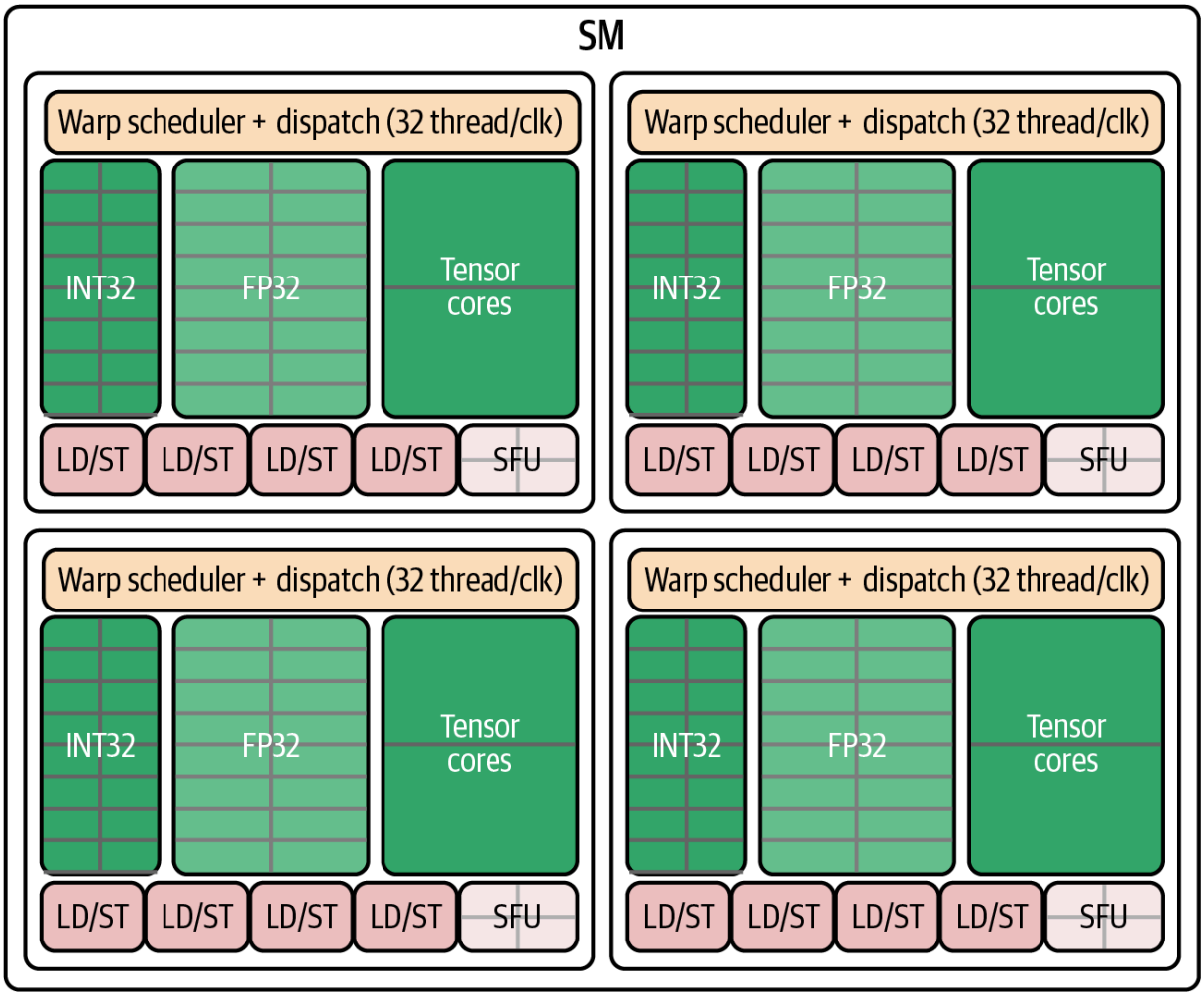


图 6-2. Blackwell SM warp scheduler partition 与 math/memory dual-issue 的概念示意

图中把 SM 画成四个 scheduling partition，每个包含 warp scheduler 与 dispatch logic，并共享部分 on-chip resource。“四个 mini-SM”只是帮助理解的比喻；实际 register、cache/shared memory、execution unit 和 crossbar 的归属及共享方式依架构而定。

所谓 *dual-issue*，是特定独立指令在满足配对规则时可由相应 dispatch path 同 cycle 发出。原文固定写成同一 warp 的一条 INT32/FP32/Tensor Core arithmetic 加一条 load/store，并不是 CUDA 对所有 Blackwell 指令的通用保证。表 6-1 保留原书的概念数字，但实际行为应由 architecture documentation 与 profiler counter 验证。

表 6-1. SM scheduler 与 instruction issue 的概念性上限（每 clock cycle）

指标	概念值
scheduler 数量	4
最大发射 warp 数	4（每 scheduler 1 个）
最大 math instruction issue 数	4（每 scheduler 一个 arithmetic issue）
最大 memory instruction issue 数	4（每 scheduler 一个 load/store issue）

> 注意：所有指标表中的数字均用于说明概念。不同 GPU 架构的实际 benchmark 结果见本书 GitHub 仓库；微架构上限仍应以 NVIDIA 官方文档与目标设备实测为准。

在原表的理想化模型中，四个 partition 可合计发出四条 math 与四条 memory instruction。但达到 issue 上限不代表同时达到 compute 与 memory 峰值：instruction latency、execution width、operand collector、dependency、cache miss、memory transaction 和 pipeline contention 都会限制吞吐。该模型来自 SM 的多 partition 组织方式，只能作为分析起点。

Special Function Unit（SFU）执行部分 transcendental/approximate operation，例如 reciprocal、square root，以及由编译器/library 映射的部分 sine/cosine 路径。具体指令可能由 SFU、multi-instruction sequence 或 library 实现；不能把所有这些数学函数都固定映射到同一独立 pipeline。

SFU 与其他 execution resource 的并行性可以增加 instruction-level parallelism，但 dependency、scheduler issue、operand 和 shared resource 仍可能让复杂数学操作位于关键路径。“独立 pipeline”不意味着主 math/memory 指令永远无需等待，也不保证 mixed-operation kernel 自动加速。

当存在足够独立 ready work 时，多个 scheduler 可让多个 warp 在同一 cycle 取得进展。原文进一步给出每 SM 合计 16 条 LD/ST pipeline、每 scheduler 4 条的固定数字；这类 unit count、吞吐与 pairing 不属于 CUDA programming model 保证，且 load/store instruction 会经过 address generation、coalescing、cache 与 memory hierarchy，不能由 pipeline 数直接推导 transaction throughput。

> LD/ST unit 数量、issue pairing 与吞吐不受跨架构保证。应使用 profiler counter 判断 kernel 是否受 memory issue、dependency、cache/memory bandwidth 或 compute 限制，并查阅目标架构的 NVIDIA 文档；可从 Blackwell tuning guide 开始。

概括而言，GPU 擅长具有大量并行工作和足够算术/内存并发度的 workload，例如矩阵乘、卷积及许多 elementwise/reduction 操作。开发者可以直接编写 CUDA C++ kernel，也可通过 PyTorch 等框架或 OpenAI Triton 等 domain-specific GPU language 间接生成 kernel。

在深入 kernel 开发和 memory-access 优化之前，先回顾 CUDA thread hierarchy 及支撑这些实践的关键术语。

线程、warp、block 与 grid

CUDA 把并行工作组织为thread、thread block（也称 *cooperative thread array*，CTA）和 grid 层级。每个 logical thread 执行 kernel program 的一个实例；thread 组成 block，kernel launch 的所有 block 构成 grid。许多现代 NVIDIA GPU 的单 block 上限为 1,024 个 thread，但还受每维上限、register/shared memory 和目标 device attribute 限制，如图 6-3 所示。

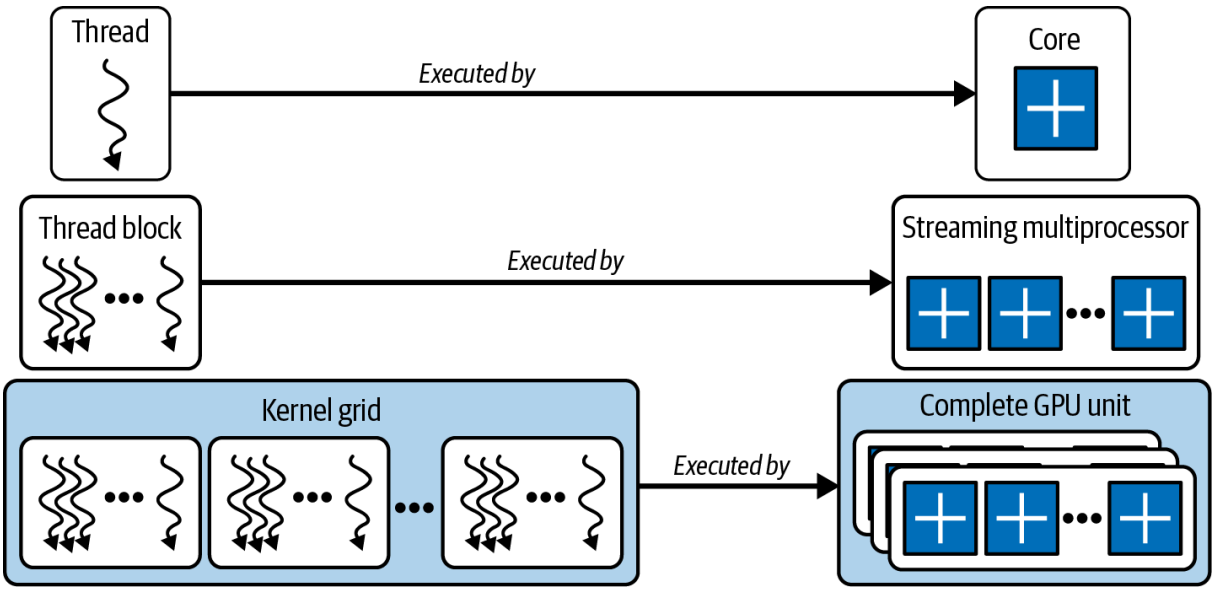


图 6-3. thread、thread block（CTA）与 grid

通过设置足够大的 grid，同一 kernel 逻辑可以覆盖大量 logical thread。CUDA runtime 把 block 调度到可用 SM；PyTorch 等 framework 通常生成并启动相应 kernel，但并不直接规定每个 block 的硬件调度顺序。图 6-4 从另一角度展示 thread hierarchy，包括发起 device kernel 的 CPU host。

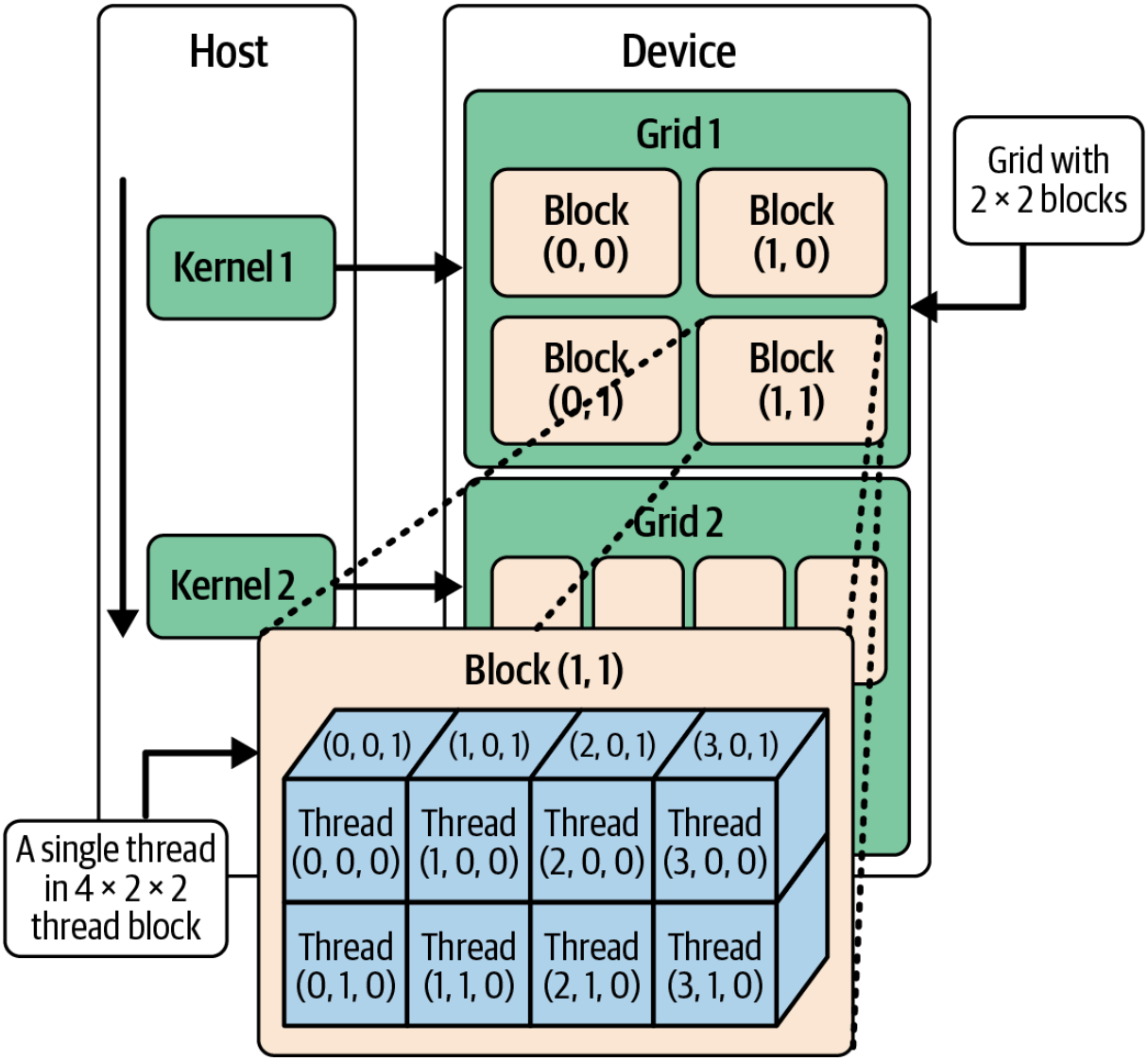


图 6-4. 包含 CPU host 与 GPU kernel launch 的 thread hierarchy

经典 CUDA programming model 中，不同 block 必须可独立执行，不能依赖普通 block 间同步。Hopper 及后续受支持架构引入 thread block cluster：同一 cluster 的 block 会被协同调度到同一 GPU Processing Cluster 范围，并可使用 cluster-scoped synchronization 与 distributed shared memory。它不是所有“现代 GPU”的通用能力。

在 thread block cluster 内，一个 block 可通过 cluster address mapping 访问其他 block 的 shared memory，并使用 cluster barrier。DSMEM 为 cluster 中各 block 的 shared-memory segment 提供分布式地址空间，可支持跨 block tile、histogram 或部分矩阵算法；是否有利取决于访问模式、cluster size 与 occupancy，如图 6-5。

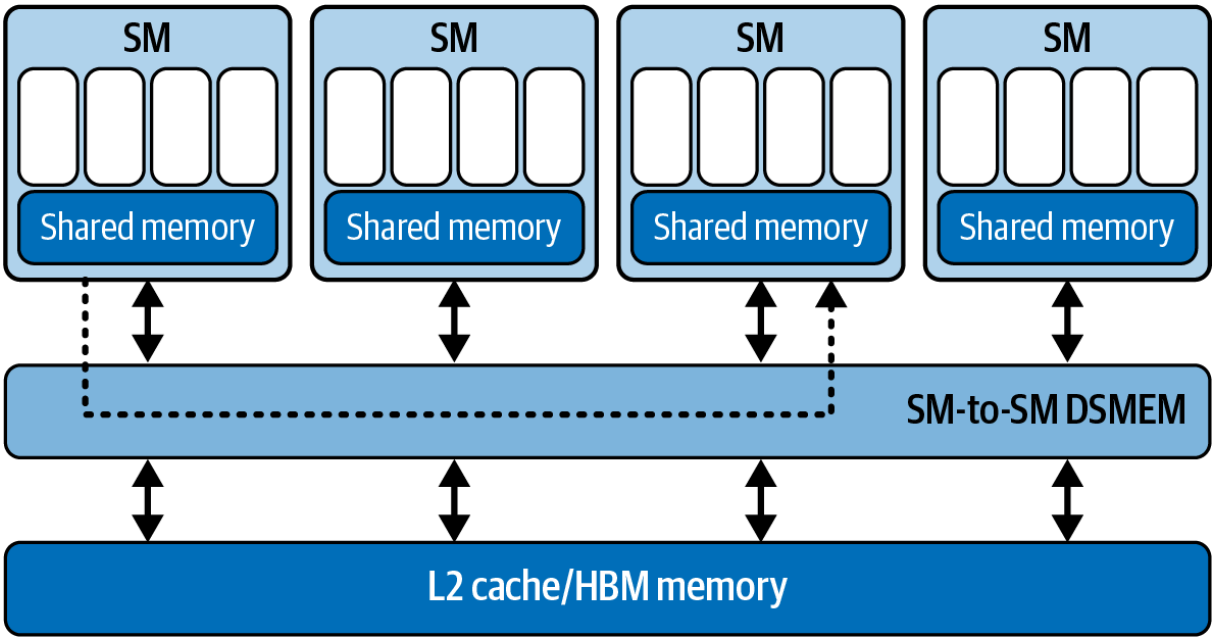


图 6-5. 多 block thread block cluster 中的硬件支持 DSMEM

DSMEM 通过片上互连让 cluster 内 block 访问彼此的 shared-memory segment，并支持相应 read、write 和 atomic operation。它不是把“所有 SM 的 bank”合并成均匀共享池：cluster 只覆盖被协同调度的 block，remote DSMEM access 通常比 local shared memory 延迟更高，目标 block 也必须保持存活直到访问完成。它可避免部分 global-memory traffic，但同步与远端访问成本仍需测量。

> 第 10 章将进一步讨论 thread block cluster 与 DSMEM。本章先聚焦 block 内 shared-memory 优化；使用 cluster 前应确认 compute capability、portable cluster size、launch API 和资源限制。

在一个 thread block 内，thread 可通过低延迟 on-chip shared memory 交换数据，并用 `__syncthreads()` 建立 block-wide barrier 与 shared/global memory ordering。该 barrier 必须由 block 中所有未退出的 thread 以一致控制流到达，否则可能 hang 或产生未定义行为。只应删除经依赖分析证明冗余的同步，不能单纯因为 barrier 有开销而最小化，如图 6-6。

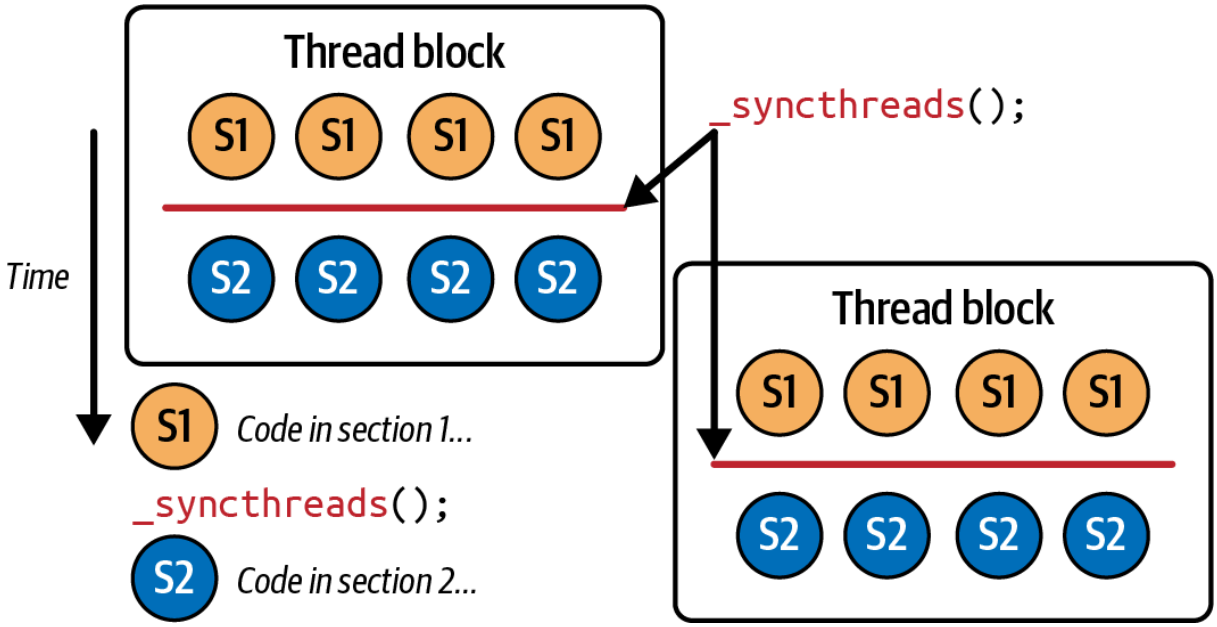


图 6-6. 在两段代码之间同步一个 block 内的所有 thread

目标是保留 correctness 所需的同步，并消除不必要的 barrier。warp scheduler 可在一个 warp 因 global-memory dependency、cache miss 或 pipeline dependency 无法发射时选择其他 ready warp；这是零开销硬件调度，而不是类似 CPU context switch 的“快速切换”。能否隐藏 latency 取决于 resident warp、独立工作、occupancy 和资源限制。

thread block 会进一步划分为由 32 个 thread 组成的 warp。在 SIMT model 下，warp scheduler 以 warp 为调度单位发射指令；各 active lane 执行同一条 warp instruction，但 independent thread scheduling、predication 和 divergence 会使“lockstep”只是一种简化描述，如图 6-7 所示。

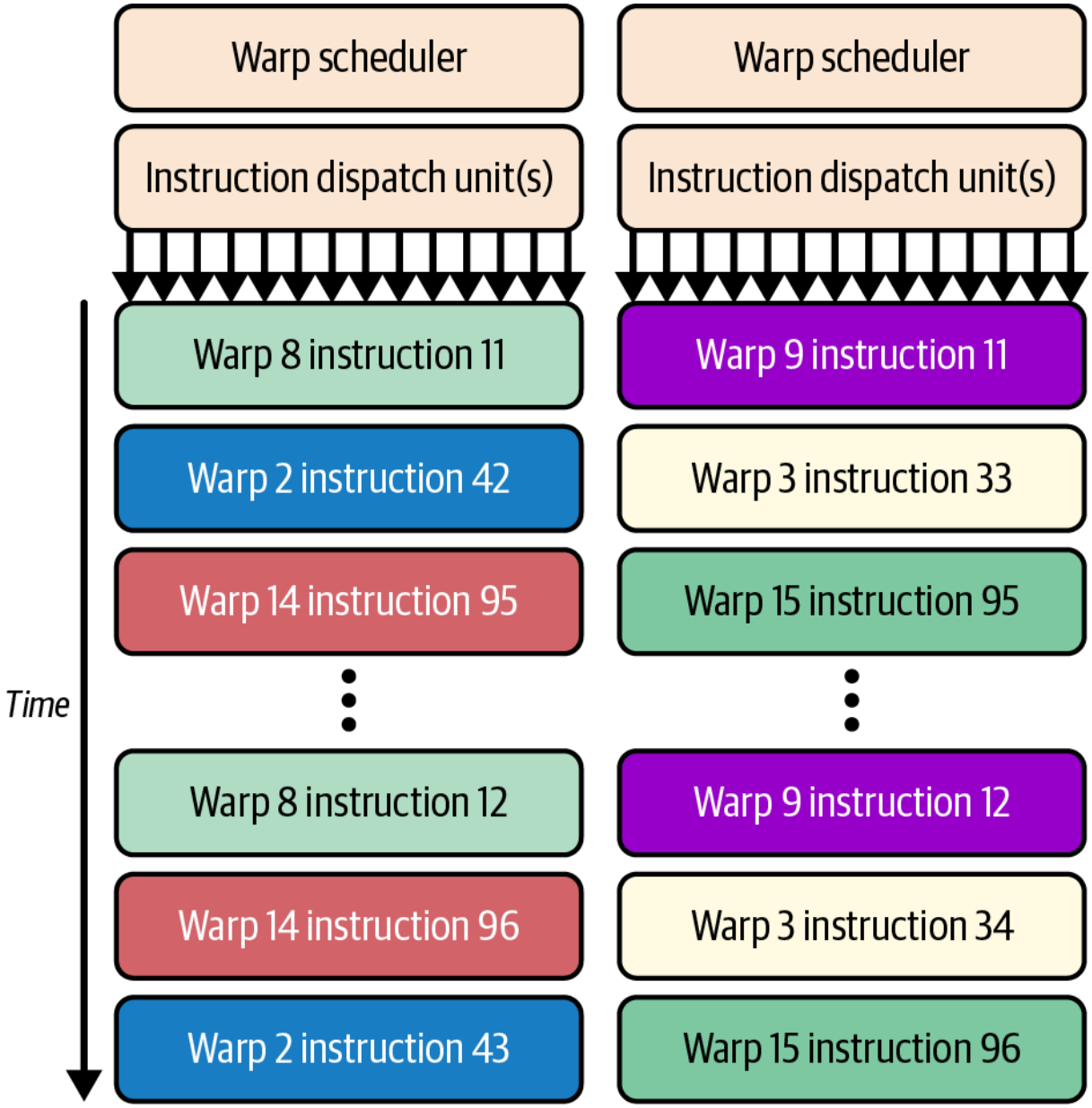


图 6-7. warp（32 个 thread）作为调度单位推进，其指令由 warp scheduler 管理

让较多 warp 同时驻留在 SM 上，通常称为 *high occupancy*（高占用率）。当一个 warp stall 时，较多 resident warp 会提高 scheduler 找到另一个 ready warp 的机会，从而帮助隐藏 latency。不过 occupancy 衡量的是 active warp 相对于硬件上限的比例，并不直接等同于 execution-unit utilization 或较高性能。

因此，occupancy 必须与每个 thread/block 的 register 和 shared-memory 需求一起权衡。register pressure 过高时，compiler 可能把部分值 spill 到 local memory；local memory 位于 device-memory address space，访问会经过 cache/global-memory hierarchy，因而可能引入额外 traffic 与 stall。应结合 achieved occupancy、register/shared-memory usage、stall reason 和实际 throughput 进行 profiling，再选择既不过度压缩每线程资源、也不造成资源争用的 block size。

第 8 章将详细介绍 occupancy tuning；在理解 SM、warp 与 thread 的关系时，应先把 occupancy 视为影响 latency hiding 的一个因素，而不是单独追求的性能目标。

thread block 必须能够彼此独立地执行，且执行顺序没有保证。这让 GPU scheduler 可以把 block 分派到可用 SM，并随 SM 数量扩展并行执行。grid–block–warp hierarchy 为可扩展性提供了 programming-model 基础，但并不保证所有 CUDA kernel 都能在未来 GPU 上原样获得正确性、兼容性或最佳性能；程序仍须遵守同步与内存模型，并受 compute capability、binary/PTX compatibility 和资源上限约束。

throughput 还取决于 warp execution efficiency。对同一 warp 而言，uniform control flow 通常能保持更多 lane 同时有效，而合并良好的 memory access 有助于减少 memory transaction。如果部分 thread 进入 `if` path、其余 thread 进入 `else` path，warp 会在不同 path 上分别执行，并屏蔽当前 path 中 inactive lane；这称为 *warp divergence*，如图 6-8 所示。具体实现还可能使用 predication 和 architecture-specific reconvergence mechanism。

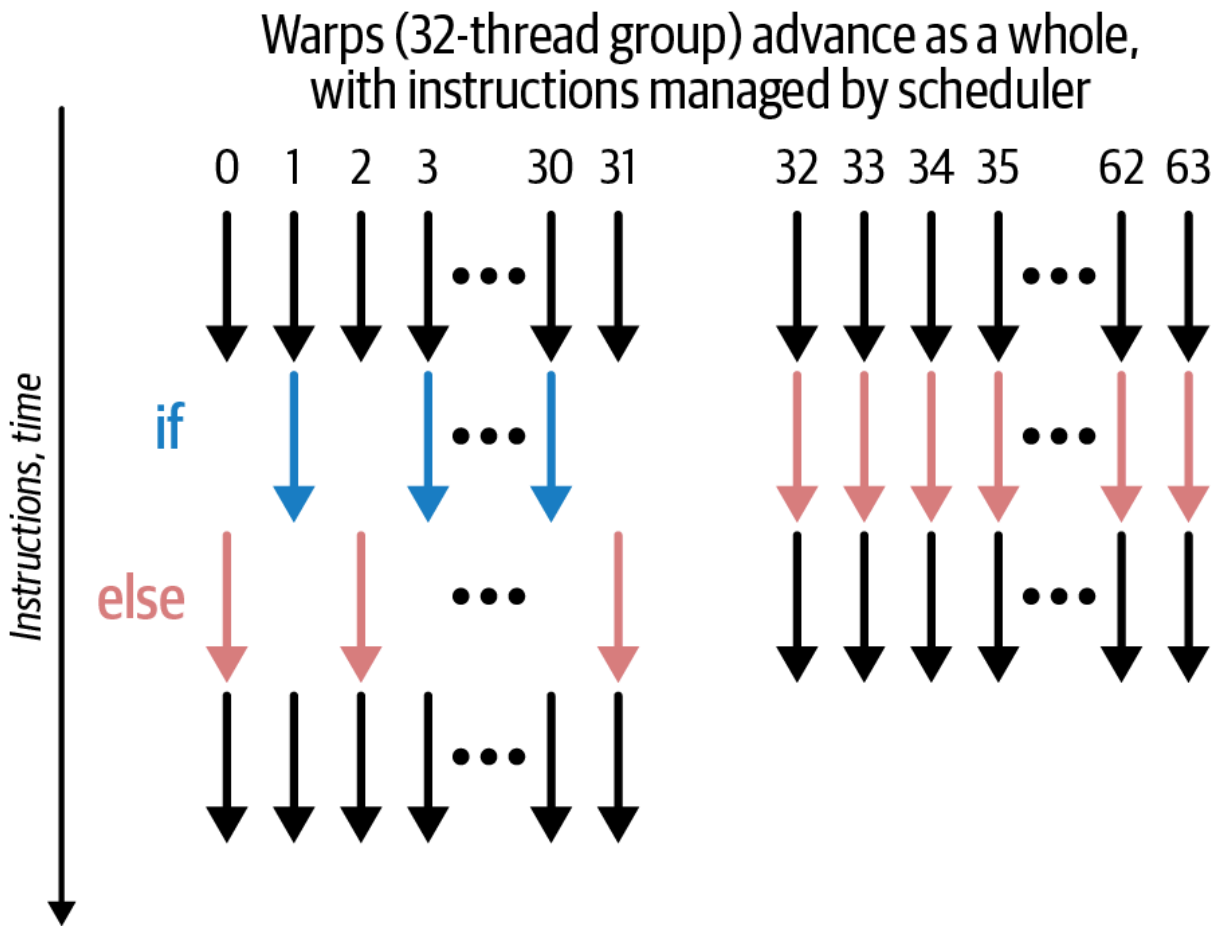


图 6-8. SIMT warp divergence（左）与 uniform execution（右）

warp divergence 会屏蔽 inactive lane，并为实际采用的 path 执行相应指令，因此会减少 useful lane utilization。其代价并不必然等于 branch 数：它取决于各 path 的指令长度、active mask、predication、memory behavior 与 reconvergence。第 8 章将进一步讨论如何检测、profile 和缓解 warp divergence。

严格意义上的 warp divergence 发生在同一 warp 的 lane 采取不同 control-flow path 时。不同 warp 采取不同 path 不会产生同一 warp 内的 divergence penalty，但仍可能因 path 长度不均、block tail effect、memory access 或资源竞争而降低整体效率。

选择每 block 的 thread 数和每 grid 的 block 数

影响 GPU 性能的一个重要因素，是选择与硬件 32-thread warp 对齐的 thread-block size。因此，通常选用 32 的整数倍。例如，包含 256 个 thread 的 block 正好形成 8 个 warp（256 ÷ 32）；包含 33 个 thread 的 block 则需要两个 warp，第二个 warp 仅有 1/32 的 lane active。后者会降低 lane utilization，因为不完整的 warp 仍占用相应 warp resource；不过最佳 block size 还取决于 register、shared memory、latency、instruction mix 和 workload shape，不能只凭 32 的倍数决定。

不同 GPU generation 还有不同的 hardware limit，包括每 SM 最大 thread 数、每 block 最大 thread 数和 register-file capacity。这些限制会约束可用 block size 和 resident-block 数。大 block 往往消耗更多 block-level resource；但是否发生 *register spilling* 主要由每 thread register demand、compiler allocation granularity 和 launch constraint 决定，并非 block 较大就必然 spill。

大 block 也可能提高每 block 的 shared-memory allocation，从而减少同一 SM 可同时驻留的 block 数。原文以 Blackwell 为例，给出每 SM 约 228 KB shared memory、单 block 最多约 227 KB usable 的数字；实际可用量取决于具体 GPU、compute capability、L1/shared-memory carveout、static allocation 和 opt-in dynamic-shared-memory setting，应查询目标设备属性。

这些 hardware limit 共同决定一个 SM 上能同时 active 的 block 与 warp 数。occupancy 正是对此类 resident warp 比例的度量。较小 block 有时能让更多 block 驻留并改善 latency hiding，但也可能形成不完整 warp、减少每 block 可协作的 thread，或受到 active-block 上限约束；因此并不保证 occupancy 或 throughput 更高。

调优时需要了解目标 GPU 上 thread、thread block、warp 和 SM 的数量级及限制。图 6-9 以 Blackwell GPU 为例展示这些资源的相对层级和原书列出的上限。

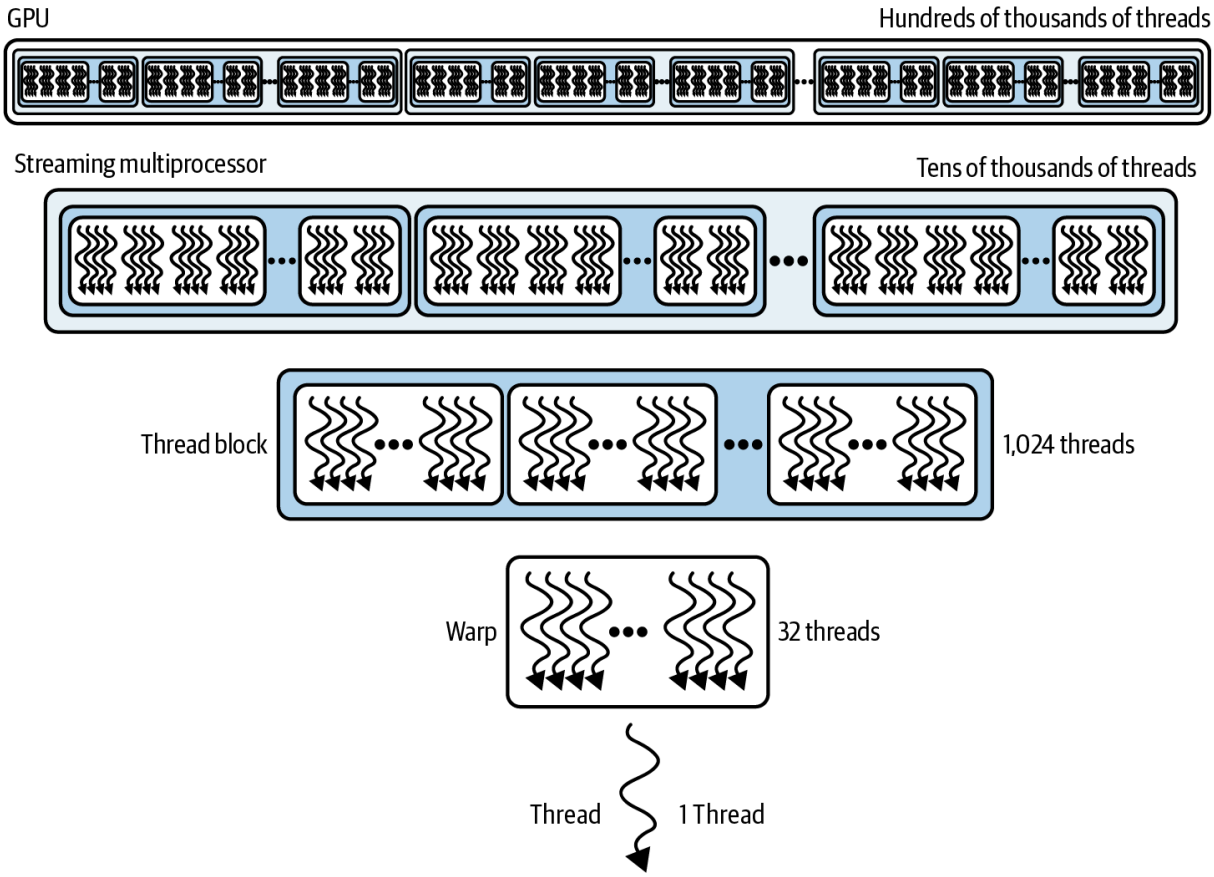


图 6-9. Blackwell GPU 上 thread 的相对规模与硬件限制

表 6-2 汇总了原书为 Blackwell B200 列出的部分 GPU limit。其他限制可参阅 NVIDIA 网站。不同 generation、SKU 和 compute capability 可能不同，应用还应在运行时查询目标系统的 device attribute。

表 6-2. thread-level 与 block-level limit（Blackwell B200）

资源	硬件上限	说明
warp size	32 个 thread	warp 包含 32 个 thread，是 SIMT execution 的基本调度粒度。 通常选择 32 的倍数以避免不完整 warp，但最终 block size 应由 profiling 决定。
每个 thread block 的最大 thread 数	1,024 个 thread	`blockDim.x * blockDim.y * blockDim.z ≤ 1024.`
每个 thread block 的最大 warp 数	32 个 warp	1,024 个 thread ÷ 每 warp 32 个 thread = 每 block 最多 32 个 warp。

32-thread warp size 使 32 的倍数通常成为 block dimension 的合理起点，从而形成完整 warp。表中每 block 最多 1,024 个 thread，因此最多对应 32 个 warp。block 一旦驻

留，SM 的 warp、thread、block、register 和 shared-memory 上限会共同决定还能容纳多少 block，进而限制 occupancy。

不同 GPU generation 还具有 per-SM limit，通常称为 *SM-resident limit*。表 6-3 汇总了原书给出的 Blackwell B200 数值；这些数值须按具体 device specification 验证。

表 6-3. SM-resident resource limit（Blackwell B200）

资源（每 SM）	硬件上限	说明
每 SM 最大 resident warp 数	64 个 warp	原书给出的硬件上限为 64 个 resident warp（64 × 32 = 2,048 个 thread）。 注意：多个近期架构采用这一上限，但它不是 CUDA 对所有 generation 的永久保证。
每 SM 最大 resident thread 数	2,048 个 thread	等于 64 个 warp × 每 warp 32 个 thread。若每 block 使用 1,024 个 thread，仅按 thread 上限计算，一个 SM 最多可驻留两个此类 block；register、shared memory 等约束可能使实际数量更低。 使用较小 block（例如 256 个 thread）可能允许更多 block 驻留；只按 thread 上限计算，最多为 8 × 256 = 2,048 个 thread。这可能提高 occupancy 并帮助隐藏 latency，但 active-block limit、资源分配和 workload 粒度仍会约束实际结果。
每 SM 最大 active block 数	32 个 block	原书列出的上限为一个 SM 同时驻留 32 个 thread block；只有足够小且其他资源约束允许时才可能达到。

按原书给出的 Blackwell 上限，每 SM 最多有 64 个 concurrent resident warp、32 个 active block 和 2,048 个 resident thread。这三项是彼此独立的上限，不能由其中一项简单“对应”推出另一项；实际 residency 取所有 thread/register/shared-memory/block constraint 中最严格者。CUDA grid 的各维也有上限，如表 6-4 所示。

表 6-4. CUDA grid limit

网格维度	上限	说明
X、Y 或 Z 维的最大 block 数	X：2,147,483,647 个 block Y：65,535 个 block Z：65,535 个 block	表中理论 3D dimension 上限为 2,147,483,647 × 65,535 × 65,535 个 block；实际 launch 还受参数范围、workload 和运行资源约束。
最大 concurrent grid（kernel）数	128 个 grid	原书列出单 device 最多并发执行 128 个 kernel。该上限依架构而定，且表示 kernel 可并发取得执行进展，并非 128 个完整 grid 的所有 block 同时 resident。

了解理论 grid limit 很有用，但 performance 通常更早受到每 block/per-SM resource、memory traffic 和可用 parallel work 的限制。原文称“单维超过 65,535 个 block”就应改用 2D/3D grid 或 multiple launch，这只适用于 Y/Z 上限或旧式限制的语境；对表中所述现代 CUDA device，X 维本身可达到 2,147,483,647。需要跨维映射或超出合法 dimension 时，可采用 2D/3D grid、grid-stride loop 或 multiple kernel launch，且必须使用足够宽的 index arithmetic 防止 overflow。

CUDA GPU 的后向与前向兼容模型

CUDA 的重要优势之一，是其跨 GPU generation 的 compatibility model。若 binary 中包含目标 GPU driver 能够 JIT 的 PTX，今天编译的 kernel 通常可以在未来受支持的 GPU 上重新生成 machine code。若只发布单一 architecture 的 SASS/cubin，例如 Hopper 的 `sm_90` 或 Blackwell 的 `sm_100`，而不包含合适的 PTX，则不能据此保证在更新 architecture 上运行。family-specific target（如 `sm_100f` 或 `compute_100f`）只承诺在相应 architecture family 范围内兼容；architecture-specific feature 的范围可能更窄。实践中通常发布 fat binary：为已知目标包含相应 cubin，并包含适当 virtual architecture 的 PTX 作为未来 GPU 的 JIT fallback；具体 target 组合应由 CUDA Toolkit 文档和部署矩阵确定。

可以设置 `CUDA_FORCE_PTX_JIT=1`，在 load 时忽略 embedded cubin 并强制 driver 从 embedded PTX 进行 JIT，以测试是否存在可用的 PTX fallback；生成结果通常会进入 compute cache。若应用所需 kernel 没有可 JIT 的 PTX，load 或 launch 可能失败，表明需要用合适 target 重新构建。不过测试通过只证明当前 driver/GPU 能 JIT 所含 PTX，并不自动证明所有未来 driver、feature 或 dependency 都兼容。cubin 与 PTX 的组合让同一 codebase 可以覆盖多个 legacy 和新 GPU，但仍需分别构建、测试并满足 driver/toolkit compatibility。

要同时覆盖现有旧 GPU 和未来 GPU，应为需要直接支持的现有 architecture 包含 cubin，并显式包含合适的 PTX 以提供 forward-JIT path；不存在一个可无条件覆盖所有 GPU 与 feature 的“generic target”。使用新硬件的 family-specific 或 architecture-specific optimization 时，还应为其他目标提供功能 fallback，并在每个受支持的 GPU、driver 和 Toolkit 组合上验证 correctness 与 performance。

CUDA 编程回顾

在 CUDA C++ 中，可通过编写 kernel 定义并行工作。kernel 是用 `__global__` 标注、由 host 调用并在 GPU device 上执行的特殊函数。CPU host code 使用 `<<< >>>`（chevron syntax）指定 launch configuration。本例使用 `blocksPerGrid` 表示 grid 中的 thread-block 数，使用 `threadsPerBlock` 表示每个 block 的 thread 数；完整 configuration 还可包含 dynamic shared-memory byte 数和 stream。

下面的简单示例展示 CUDA kernel 与 kernel launch 的主要组成部分。该 kernel 原地把 input array 的每个元素乘以 2，因此不另行分配 output array。CUDA toolchain 会把 `__global__` 函数编译为 GPU device code；launch 创建由许多 logical thread 组成的 grid，每个 thread 执行该 kernel 的一个实例：

```
//------------------------------------------------------
// Kernel: myKernel running on the device (GPU)
//   - input : device pointer to float array of length N
//   - N   : total number of elements in the input
//------------------------------------------------------

__global__ void myKernel(float* input, int N) {
    // Compute a unique global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Only process valid elements
    if (idx < N) {
        input[idx] *= 2.0f;
    }
}

// This code runs on the host (CPU)
int main() {
    // 1) Problem size: one million floats
    const int N = 1'000'000;
    float *h_input = nullptr;
    float *d_input = nullptr;

    // 1) Allocate input float array of size N on host
    cudaMallocHost(&h_input, N * sizeof(float));

    // 2) Initialize host data (for example, all ones)
    for (int i = 0; i < N; ++i) {
        h_input[i] = 1.0f;
    }

    // 3) Allocate device memory for input on the device
    cudaMalloc(&d_input, N * sizeof(float));

    // 4) Copy data from the host to the device
    cudaMemcpy(d_input, h_input, N * sizeof(float),
```

```
  cudaMemcpyHostToDevice);

// 5) Choose kernel launch parameters

// Number of threads per block (multiple of 32)
const int threadsPerBlock = 256;

// Number of blocks per grid (3,907 for N = 1000000)
const int blocksPerGrid = (N + threadsPerBlock - 1) /
  threadsPerBlock;

// 6) Launch myKernel across blocksPerGrid blocks
// Each block has threadsPerBlock number of threads
// Pass a reference to the d_input device array
myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input,
  N);
// 7) Wait for the kernel to finish running on device
cudaDeviceSynchronize();

// 8) When finished, copy the results
//    (stored in d_input) from the device back to
//     host (stored in h_input)
cudaMemcpy(h_input, d_input, N * sizeof(float),
           cudaMemcpyDeviceToHost);

// Cleanup: Free memory on the device and host
cudaFree(d_input);
cudaFreeHost(h_input);

// return 0 for success!
return 0;
```

这段代码并未充分优化，后续章节会逐步讨论性能优化；这里先把它作为一个简单而完整的 CUDA kernel 模板。生产代码还应检查 allocation、copy、launch 和 synchronization API 的返回状态。

launch 时向 kernel 传入 `d_input` 和 `N`。每个 logical thread 都能读取这些参数，但通过计算不同 index 处理不同元素，从而并行划分工作。

该示例的数据流如下：

1. 在 host 上分配 pinned memory（`h_input`）。
2. 使用 `cudaMemcpy` 和 `cudaMemcpyHostToDevice`，将数据从 host（`h_input`）复制到 device（`d_input`）。
3. 在 device 上启动 kernel，传入 `d_input`。
4. 执行同步，等待 kernel 在 device 上完成，并取得异步执行错误。
5. 使用 `cudaMemcpy` 和 `cudaMemcpyDeviceToHost`，把结果从 device（`d_input`）复制回 host（`h_input`）。
6. 分别使用 `cudaFree` 和 `cudaFreeHost` 释放 device memory 与 pinned host memory。

launch kernel 时还可指定其他 CUDA configuration field，使用 `<<< >>>` 的完整形式包括 grid dimension、block dimension、dynamic shared-memory size 和 stream。本例的 `blocksPerGrid` 与 `threadsPerBlock` 对应前两个字段，也是后续小节重点讨论的 launch parameter。

为什么还要传入 input array 的长度 `N`？裸 device pointer 不携带 array length metadata，kernel 无法从 d_input 自行推断 allocation 的 logical element count。所有 thread 接收同一个 `N`，并用它判断各自计算出的 index 是否位于整个 array 的合法范围；它不是某一个 kernel instance 所处理“partition”的大小。

kernel 使用 built-in variable `blockDim`、`blockIdx` 和 `threadIdx` 计算 input array 中的 `idx`。这些变量都是三维值；本例只使用其 x component，通过 `blockIdx.x * blockDim.x + threadIdx.x` 为每个 launched thread 生成 linear index。若 grid 足够大且 index mapping 正确，各有效 thread 可以处理一个不同元素；block 实际分布到多少 SM 以及是否同时执行由 scheduler 和资源决定。

请注意 bounds check `if (idx < N)`。由于 `N` 可能不是 block size 的整数倍，向上取整的 grid 会启动若干 index 超出 array 范围的 thread。例如 input array 含 63 个元素，即 `N = 63`，若本例按 64 个 thread 覆盖它，就会形成两个 warp。

第一个 warp 的 lane 对应 index 0–31；第二个 warp 对应 index 32–63。合法 array index 为 0–62，因此第二个 warp 的最后一个 lane 不执行乘法。warp 不会在“到达 `N = 63`”时整体停止，而是由每个 lane 对 predicate 求值，并在不满足条件时保持 `inactive`。

若没有 `if (idx < N)` bounds check，第二个 warp 会让最后一个 lane 尝试访问 `idx = 63`，而不是原文所写的 64。这是 one-past-the-end access，可能导致 memory-checking tool 报错或 kernel fault（例如后续 API 返回 `cudaErrorIllegalAddress`），也可能表现为未定义的数据破坏；越界访问不保证每次都立即产生可见错误。`bounds check` 使有效 index 执行操作，越界 lane 跳过该代码块。

CUDA kernel launch 通常相对于 host 异步执行，device code 也不提供常规的 per-thread C++ exception propagation。launch-configuration error 可在 launch 后较早取得，而 execution fault 常在 synchronization 或后续同步性 CUDA API 处才报告，例如 `cudaErrorIllegalAddress` 或 unspecified launch failure。具体 fault handling 依错误类型而定，不能统一概括为每次非法操作都设置一个应用可见的“global fault flag”。

调试时通常在 launch 后调用 `cudaGetLastError()` 检查 launch error，再用 `cudaDeviceSynchronize()` 暴露 execution error，从而把错误定位到相邻 launch。生产 hot path 不应无条件在每次 launch 后进行 device-wide synchronization，因为这会破坏异步执行与 overlap；应检查 API return status，并在既有 dependency、event 或必要 synchronization point 处处理错误。

许多 CUDA kernel 都包含 bounds check。若没有显式检查，应确认 launch dimension、padding、tile mapping、grid-stride loop 或调用方 invariant 如何保证每次 memory access 都有效，而不能仅假定越界会被硬件安全忽略。

最后是 kernel 的实际计算逻辑。thread 计算 input array 中的 `idx` 后，把 `idx` 位置的值乘以 2，并原地写回。该 kernel 不分配额外的 device buffer；局部变量 `idx` 的类型为 `int`，物理存放位置由 compiler 决定，通常可驻留 register，但不能由 C++ source 单独保证。示例启动大量 logical thread，实际并发程度和 SM 分布由硬件资源与 scheduler 决定。

配置 launch parameter：每 grid 的 block 数与每 block 的 thread 数

如前所述，选择 warp size（32）的整数倍通常可以避免不完整 warp。将 `threadsPerBlock` 设为 256（8 个 warp）是常见的初始测试点，可在许多 kernel 中兼顾 occupancy 和 resource usage；但它不会自动隐藏 latency 或平衡 SM，最终选择必须结合 kernel resource、problem size 和 profiler 数据：

32 个 thread 的整数倍

选择 32 的整数倍作为 block size，有助于避免最后一个 warp 只有部分 lane active。若 block size 不是整数倍，不完整 warp 的 inactive lane 不贡献 useful work，但 warp 仍消耗相应 execution/resource capacity。

隐藏 latency

足够多的 resident ready warp 可以帮助隐藏 DRAM、instruction 和 dependency latency，但所需数量依 kernel 而定。若某 SM 的 thread 上限为 2,048，仅按 thread 数计算，它最多可容纳 8 个各含 256 thread 的 block；实际 residency 还可能受 register、shared memory 和 active-block limit 限制，runtime 也不会让应用直接指定某个 SM 驻留哪 8 个 block。

占用率（occupancy）

例如，256 个 `threadsPerBlock` 对应每 block 8 个 warp。它经常能提供合理 occupancy，但是否耗尽 register 或 shared memory 取决于每 thread register 数和每 block shared-memory allocation，不能由 block size 单独判断。

> 在 Blackwell 等现代 GPU 上，可把每 block 256 个 thread 作为候选起点，并与 128、512 等值比较。512 不保证最大 occupancy；应使用 occupancy API/calculator 和 profiler，在 register、shared-memory、latency hiding 与实际 throughput 之间选择工作点。

资源平衡

256 明显低于许多现代 NVIDIA GPU 每 block 1,024 个 thread 的上限，同时提供 8 个 warp。ready warp 是否充足仍取决于每 SM resident block 数、dependency 和 instruction/memory behavior，而不是单个 block 较大就能保证。

可以从 `threadsPerBlock=256` 开始，根据 kernel 的 register/shared-memory requirement、achieved occupancy、stall reason 和 throughput，向下或向上测试 128、512 等值。

`blocksPerGrid` 可由 input element 数 `N` 和 `threadsPerBlock` 计算。常见 ceiling division 写法是令 `blocksPerGrid` 等于 `(N + threadsPerBlock - 1) / threadsPerBlock`，使 N 不是 block size 整数倍时仍有 thread 覆盖最后一批元素。该公式要求正的 `threadsPerBlock`、足够宽且不会在加法中溢出的 integer type，并且结果不超过目标 device 的 grid limit；若 `N` 为 0，通常应跳过 kernel launch。示例代码如下：

```
//-----------------------------------------------------
// Kernel: myKernel running on the device (GPU)
//   - input : device pointer to float array of length N
//   - N   : total number of elements in the input
//-----------------------------------------------------
__global__ void myKernel(float* input, int N) {
    // Compute a unique global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Only process valid elements
    if (idx < N) {
        input[idx] *= 2.0f;
    }
}

// This code runs on the host (CPU)
int main() {
    // 1) Problem size: one million floats
    const int N = 1'000'000;

    float* h_input = nullptr;
    cudaMallocHost(&h_input, N * sizeof(float));

    // Initialize host data (for example, all ones)
    for (int i = 0; i < N; ++i) {
        h_input[i] = 1.0f;
    }

    // Allocate device memory for input on the device (d_)
    float* d_input = nullptr;
    cudaMalloc(&d_input, N * sizeof(float));

    // Copy data from the host to the device using cudaMemcpyHostToDevice
    cudaMemcpy(d_input, h_input, N * sizeof(float),
      cudaMemcpyHostToDevice);

    // 2) Tune launch parameters
    const int threadsPerBlock = 256; // multiple of 32
    const int blocksPerGrid = (N + threadsPerBlock - 1) /
      threadsPerBlock; // 3,907, in this case

    // Launch myKernel across blocksPerGrid number of blocks
    // Each block has threadsPerBlock number of threads
    // Pass a reference to the d_input device array
    myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input, N);
    // Wait for the kernel to finish running on the device
    cudaDeviceSynchronize();

    // When finished, copy results (stored in d_input) from device to host
    // (stored in h_input) using cudaMemcpyDeviceToHost
    cudaMemcpy(h_input, d_input, N * sizeof(float), cudaMemcpyDeviceToHost);

    // Cleanup: Free memory on the device and host
    cudaFree(d_input);
    cudaFreeHost(h_input);
```

```
    return 0; // return 0 for success!
```

该 kernel 与前例相同，但会根据 `N` 的大小计算 `blocksPerGrid` 和 `threadsPerBlock`。熟悉的 `if (idx < N)` bounds check 让最后一个 block 中 index 超出 `N` 的 lane 跳过 memory access，从而避免本例中的越界访问。接下来讨论 image 和 volume 等 2D/3D input。

2D 与 3D kernel input

当 input data 具有自然的二维逻辑结构时，例如 image，可以启动由 2D block 组成的 2D grid。下面的 kernel 用 16 × 16、共 256 个 thread 的 block 处理一个逻辑尺寸为 1,024 × 1,024 的 matrix。底层 allocation 仍是线性 float buffer，kernel 显式将二维坐标转换为 row-major linear index：

```
// 2d_kernel.cu

#include <cuda_runtime.h>
#include <iostream>

//------------------------------------------------------
// Kernel: my2DKernel running on the device (GPU)
//   - input  : device pointer to float array of size width×height
//   - width  : number of columns
//   - height : number of rows
//------------------------------------------------------
__global__ void my2DKernel(float* input, int width, int height) {
    // Compute 2D thread coordinates
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    // Only process valid pixels
    if (x < width && y < height) {
        int idx = y * width + x;
        input[idx] *= 2.0f;
    }
}

int main() {
    // Image dimensions
    const int width  = 1024;
    const int height = 1024;
    const int N      = width * height;

    // 1) Allocate and initialize host image
    float* h_image = nullptr;
    cudaMallocHost(&h_image, N * sizeof(float));

    for (int i = 0; i < N; ++i) {
        h_image[i] = 1.0f;  // e.g., initialize all pixels to 1.0f
    }

    // 2) Allocate device image and copy data to device
    cudaStream_t s; cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);
    float* d_image = nullptr;
    cudaMallocAsync(&d_image, N * sizeof(float), s);
    cudaMemcpyAsync(d_image, h_image, N * sizeof(float),
                    cudaMemcpyHostToDevice, s);
```

```
// 3) Configure and launch the 2D kernel
dim3 threadsPerBlock2D(16, 16); // 256 threads per block
dim3 blocksPerGrid2D((width + threadsPerBlock2D.x - 1)
                      / threadsPerBlock2D.x,
                     (height + threadsPerBlock2D.y - 1)
                      / threadsPerBlock2D.y);

// 4) Launch the kernel
my2DKernel<<<blocksPerGrid2D, threadsPerBlock2D,
             0, s>>>(d_image, width, height);

cudaMemcpyAsync(h_image, d_image, N * sizeof(float),
                cudaMemcpyDeviceToHost, s);
cudaStreamSynchronize(s);

cudaFreeAsync(d_image, s);
cudaStreamDestroy(s);
```

以上仍给出了完整的 device kernel 和 host invocation code。可以为 `blocksPerGrid` 与 `threadsPerBlock` 使用 `dim3(x, y, z)`，把 logical thread coordinate 推广到 3D；但 volumetric data 的实际地址仍需根据 width、height、depth 及 layout/stride 显式计算。使用 pitched allocation 时还必须按 pitch/slice pitch 寻址，并确保 dimension product 和 linear index 不发生 integer overflow。

> 本书大多为 `blocksPerGrid` 和 `threadsPerBlock` 使用 1D 或 2D tiled configuration。一维情况下，`blocksPerGrid` 和 `threadsPerBlock` 可以传入 scalar-like integer，CUDA 会将其转换为 x 维相应、y/z 维为 1 的 `dim3`。示例清单应补充 CUDA error checking，并在结束前调用 cudaFreeHost(h_image) 释放 pinned host allocation。

异步内存分配与 memory pool

前例使用的传统 `cudaMalloc`/`cudaFree` 是 host-side allocation API，调用成本可能较高，并可能引入 device synchronization 或 driver/OS memory-management work。原文以 `mmap`/`ioctl` 概括内部实现只是示意；具体系统调用、缓存与同步行为依 CUDA、driver 和平台而定。

driver 与 OS interaction 可能产生 kernel-space transition 和管理开销，因此频繁细粒度 allocation 往往代价明显。对于生命周期可按 stream 排序、且平台支持的 workload，可考虑 `cudaMallocAsync` 和 `cudaFreeAsync`；它们通过 pool 与 stream-ordering 减少重复分配和全局同步，但并非所有一次性或低频 allocation 都一定更快。

CUDA runtime 为支持的 device 维护默认 per-device memory pool，也可创建和设置显式 pool。stream-ordered free 完成后，allocation 可回到相应 pool 供后续兼容 allocation 复用。`cudaMallocAsync` 与 `cudaFreeAsync` 使用这种 memory-pool mechanism。

memory pool 可复用已释放的 memory block，减少向底层 allocator 重复申请和归还 memory 的次数。在长期 training loop 等重复 allocation pattern 中，这通常降低 allocation latency，并可能缓解某些 external fragmentation；但 pool 自身会保留

reserved memory，也可能出现 size-class/internal fragmentation。许多高性能 library/runtime（包括 PyTorch）都采用 caching 或 pooling allocator。

PyTorch 使用自定义 CUDA caching allocator，可通过 `PYTORCH_ALLOC_CONF` 配置（旧别名为 `PYTORCH_CUDA_ALLOC_CONF`）。其思想与 CUDA memory pool 相似：复用 GPU allocation，避免每次创建 tensor 都直接调用 `cudaMalloc`。具体 backend 可随 PyTorch version/configuration 改变，选项也不能在不同 backend 之间一概而论。

对于频繁、细粒度 allocation，pool-based `cudaMallocAsync` 与 `cudaFreeAsync` 通常比反复使用传统 `cudaMalloc`/`cudaFree` 更适合，因为 allocation/free 可按 stream 排序并复用 pool。收益取决于 allocation pattern、pool state 和 synchronization，不应假定每次传统调用都必然执行相同的 full-device synchronization 或 OS call。下面创建一个 nonblocking stream：

```
cudaStream_t stream1;
cudaStreamCreateWithFlags(&stream1, cudaStreamNonBlocking);
```

> explicit CUDA stream 可表达 operation ordering，并在 hardware capability、dependency、memory 和 copy-engine 条件允许时重叠 transfer、kernel 与 allocation。stream 不是完全隔离的 channel：它们共享 device resource，并可通过 event/dependency、memory hazard 和 default-stream semantics 交互。使用 `cudaStreamCreateWithFlags(..., cudaStreamNonBlocking)` 可避免与 legacy default stream 的隐式同步关系，但不会自动产生 overlap。第 11 章将详细讨论 multistream overlap。

需要容纳 `N` 个 float 的 buffer 时，可在同一 stream 上用 `cudaMallocAsync` 分配，并在最后一次使用之后 enqueue `cudaFreeAsync`，如下所示：

```
float* d_buf = nullptr;
cudaMallocAsync(&d_buf, N * sizeof(float), stream1);

// ... launch kernels into stream1 that use d_buf ...
myKernel<<<blocksPerGrid, threadsPerBlock, 0, stream1>>>(d_buf, N);

// Free is deferred until all work in stream1 completes—
cudaFreeAsync(d_buf, stream1);
```

这些 API 从 per-device memory pool 分配，并遵循传入 stream 的 ordering；free operation 只有在该 stream 中此前使用完成后才生效。`cudaFreeAsync` 本身是 enqueue，不是在 host 上等待 `stream1` 完成，也不隐式执行全局 `cudaDeviceSynchronize`。如果 pointer 会在其他 stream 使用，应用必须建立正确的 cross-stream dependency；否则同一 stream 的顺序不足以保证 lifetime。对于大量 allocate/free cycle，pool reuse 可降低 allocation overhead 和 latency variance，并减少某些全局同步；相对传统 `cudaMalloc`/`cudaFree` 的实际收益和 fragmentation 仍需测量。

还可调整 device memory pool 的行为。例如，`cudaMemPoolAttrReleaseThreshold` 控制 pool 在 synchronization opportunity 后尝试向 OS 释放 reserved memory 时使用的阈值；它不是严格的 memory cap。也可调用 `cudaMemPoolTrimTo` 请求把 reserved

memory 缩减到给定目标附近。二者用于权衡 retained footprint、后续 allocation latency 和复用机会，而不是直接消除 fragmentation。

对于简单的一次性 buffer，传统 `cudaMalloc` 和 `cudaFree` 可能已经足够。对于长期循环中的重复 allocation/free，可评估在适当 stream 上使用 `cudaMallocAsync` 和 `cudaFreeAsync` 及其 pool；它通常降低 allocator overhead，但一致性和 throughput 改善取决于 workload 与 pool policy。

在合适的 stream/lifetime model 下，`cudaMallocAsync` 与 `cudaFreeAsync` 可以利用 pool 减少重复管理开销。可通过 `cudaMemPoolSetAttribute` 调整 pool behavior，例如设置 `cudaMemPoolAttrReleaseThreshold`，以便*在 retained memory footprint、allocation latency 与复用率之间选择经测量的工作点，而不是假定某一阈值同时最小化 footprint 和 fragmentation。*

理解 GPU memory hierarchy

到目前为止，我们主要从高层讨论 global-memory allocation。stream-ordered allocation 可来自目标 device 的 current/default memory pool；memory pool 属于 device/allocator 作用域，并不存在一个专属于“default stream 0”的独立 memory pool。

GPU 实际提供多级 memory hierarchy，以不同 capacity、latency、bandwidth 和作用域支持数据访问，包括 register、shared memory、cache、global memory，以及 Blackwell 上特定 Tensor Core path 使用的 TMEM。TMEM 是每 SM 约 256 KB 的 dedicated on-chip tensor memory，供第 5 代 Tensor Core instruction（`tcgen05.*`）保存 accumulator 等数据；普通 CUDA C++ 不能像 global/shared memory 那样用通用 pointer 直接寻址。TMA 主要负责 global memory 与 SMEM 之间的 multidimensional async transfer，而 TMEM 的分配、访问和 SMEM/TMEM movement 由相应 tcgen05 instruction 与 descriptor/control operand 管理；原文括号也缺少右括号。

global memory（通常为 HBM/DRAM）容量大、位于 chip 外，相对 on-chip storage 延迟更高；register 容量很小、位于 SM 上，通常具有最低访问延迟。L1、L2 与 shared memory 在 scope、capacity 和 access mechanism 上各不相同，不能简单视为一条统一的中间层级。cache 可利用 locality 减少下层访问，shared memory 则由程序显式管理以实现 reuse/cooperation；它们可降低 exposed global-memory latency 和 traffic，但不会自动“隐藏”所有延迟。图 6-10 给出包含 CPU 的高层示意。

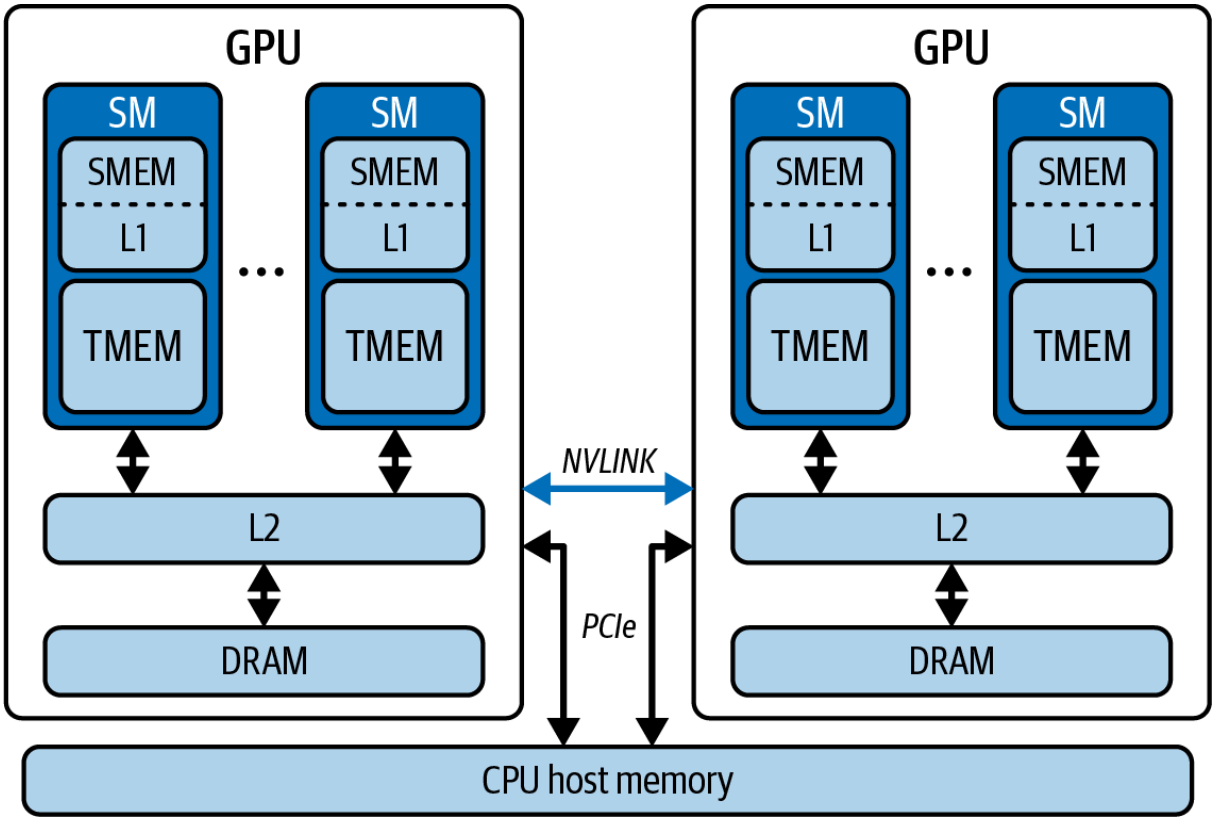


图 6-10. 包含 CPU 的 GPU memory hierarchy

TMEM 是 Blackwell 上每 SM 约 256 KB、与特定 Tensor Core instruction 紧密耦合的 storage。原书给出其与 Tensor Core 之间为每秒数十 TB 的内部带宽口径，但该数值依 instruction、数据布局和统计口径，不能等同于 application 可持续带宽。TMEM 可让 accumulator 等中间值留在片上，减少部分 SMEM/register/global-memory traffic；它不透明地替代整个 global-memory data path。图 6-11 示意 TMEM 与 SMEM 配合 Tensor Core 计算 $C = A \times B$。

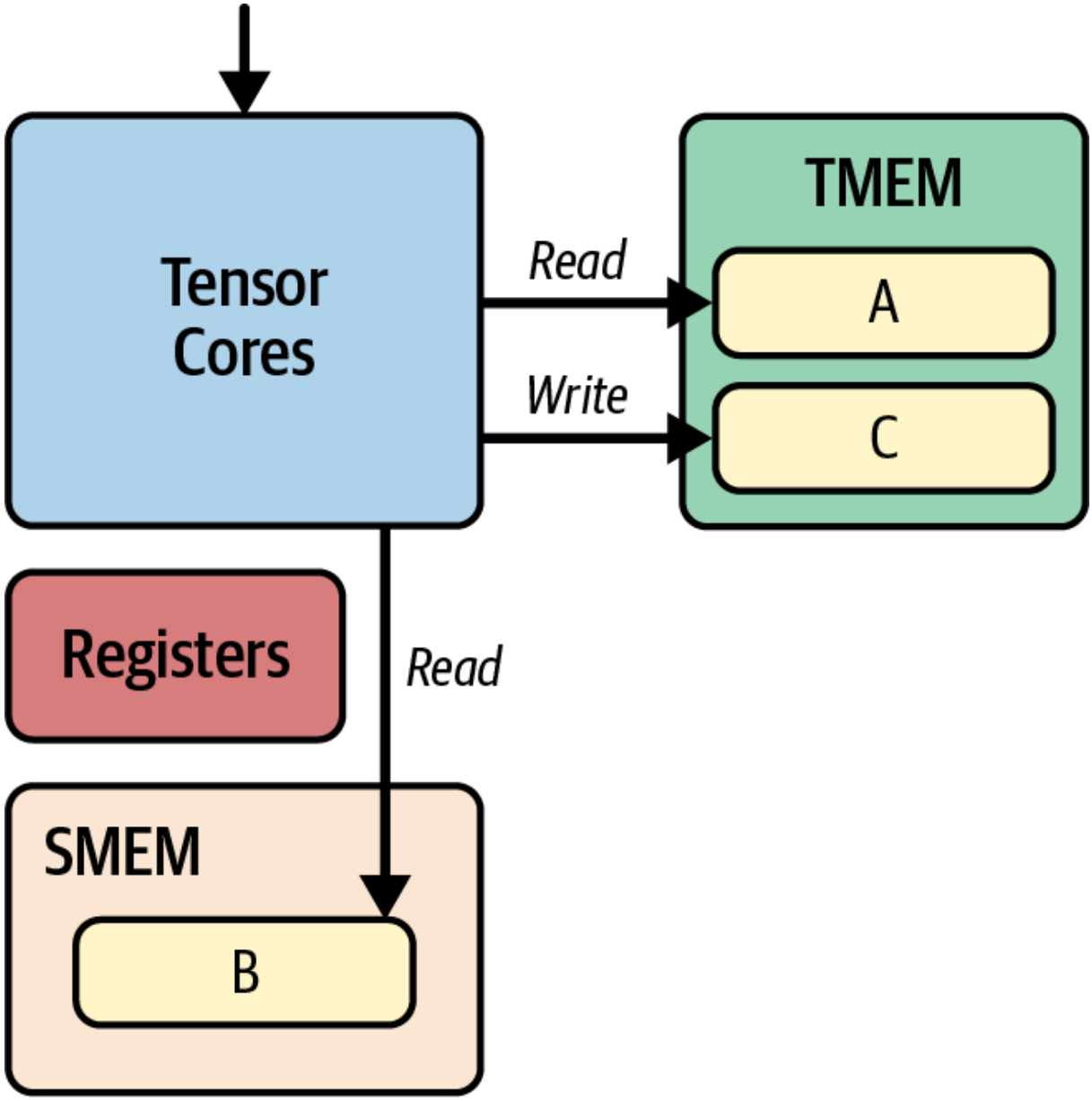


图 6-11. TMEM 与 SMEM 为 C = A × B matrix multiply 提供数据

图中把 operand B 画为来自 SMEM，并把 A/accumulator 画在 TMEM；这只是简化示意。对 Blackwell tcgen05 MMA 而言，operand source、descriptor 和 TMEM accumulator layout 取决于具体 instruction form，不能普遍断言 A 预先存于 TMEM。tile 可由 TMA 从 global memory 经 memory hierarchy 搬到 SMEM；高级 API 可使用 `cuda::memcpy_async` 等抽象，但它与底层 TMA 的映射取决于 architecture、shape、alignment 和 compiler。随后由 UMMA/`tcgen05.mma` 及相关 data-movement instruction 使用 SMEM/TMEM 数据。

表 6-5 汇总原书给出的 Blackwell memory level 与特征。capacity 中部分是 architecture/device limit，latency/bandwidth 多为近似值或 microbenchmark 口径，会随 SKU、clock、instruction、access pattern、cache state 和 contention 改变，不能视为 CUDA contract。后文再逐层说明。

表 6-5. Blackwell memory hierarchy 与特征

层级	作用域	容量	延迟	带宽（近似）
register	per thread，物理 register file 位于 SM	每 SM 64 K 个 32-bit register（每 thread architecture limit 为 255）	最低层级延迟（dependency、bank/port conflict 和 instruction scheduling 仍可能产生等待，不能视为“免费”）	每 SM 数十 TB/s 量级（register-file port 的理论/推导口径）
shared memory 与 L1 cache	per SM	原书列约 228 KB shared（单 block 约 227 KB usable），与 L1/data cache 共享/配置的 SRAM 口径依 device	约 20–30 cycle（特定 L1/shared microbenchmark）	每 SM TB/s 量级（bank-conflict-free 的特定访问）
TMEM	per SM	每 SM 约 256 KB，供特定 Tensor Core path 使用	原书列约 10 cycle（非通用 CUDA load/store latency）	与 Tensor Core 通信为 TB/s 量级（依 instruction/ 口径）
constant-memory cache	per SM cache；constant address space 为 device scope	原书列约 8 KB cache，对应 64 KB `__constant__` address space	同一 warp 访问同一 cached address 时支持 broadcast；“约 1 cycle/等同 register”不是通用保证 不同 address 通常按 distinct address 数序列化 cache miss 则访问下层 memory，并产生更高 latency	broadcast pattern 下可达高 throughput；依 cache/access pattern
L2 cache	device-wide，供所有 SM 共享	原书列总计 126 MB；依具体 SKU	原书列约 200 cycle；依 hit path、clock 与 contention	aggregate multi-TB/s 量级；依访问与 SKU

层级	作用域	容量	延迟	带宽（近似）
local memory	per-thread address space；物理存储位于 device memory，可由 cache 服务	受 address-space、allocation 与 device-memory capacity 限制，并非 near-unlimited	cache hit 到 DRAM miss 的跨度很大；原书列数百至约 1,000 cycle	受 cache/HBM traffic 与 coalescing 限制，不能直接等同 HBM peak
global memory（HBM/DRAM）	设备范围的片外内存	原书列 Blackwell B200 每 GPU 最多约 180 GB 可用口径（Blackwell B300 每 GPU 最多约 288 GB）	数百至约 1,000 cycle 的经验范围；依 cache miss、queueing 和 clock	原书列总计约 8 TB/s；具体 B200/B300 SKU 与口径不同

因此，高 throughput kernel 通常要利用 register/shared memory/cache 中的数据 reuse，减少不必要的 off-chip traffic，并保持访问合并和足够并发度。但 register/shared-memory 使用过多会降低 occupancy，cache 由硬件管理且 hit rate 依 workload，local memory 也可能命中 cache；目标不是无条件“最小化”某一层，而是优化 end-to-end data movement。下面逐层展开说明：

寄存器（register）

在 Blackwell 上，每个 SM 的 register file 包含 64 K 个 32-bit register（合计 256 KB），用于保存各 resident thread 的 scalar value、address 和 compiler temporary；architecture limit 为每 thread 最多 255 个 register。register 是 SM 上的低延迟资源，但实际 allocation 以 warp/CTA 等粒度取整，并由 compiler 与 launch constraint 共同决定。

register file 可提供很高 throughput，但 read/write 并非无条件单 cycle 或“几乎不争用”：dependency、bank/port conflict、operand delivery 和 instruction scheduling 都可能限制性能。当 compiler 无法把所有 live value 放入 register 时，部分值会 spill 到 local-memory address space；其物理 backing 位于 device memory，并可经过 L1/L2，因此 latency 从 cache hit 到 DRAM miss 差异很大，不必然每次都是数百至上千 cycle。图 6-12 示意 per-thread local memory。

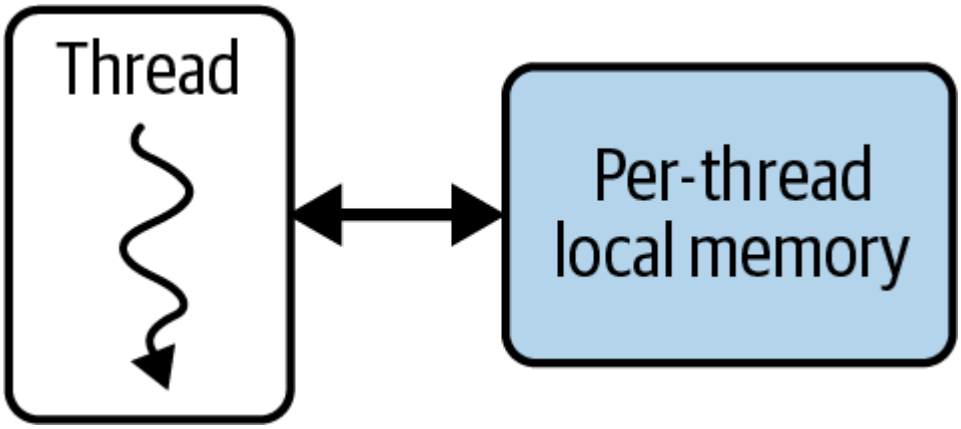


图 6-12. 每 thread 的 local-memory address space

shared memory 与 L1 data cache

Blackwell 等 architecture 的 L1/texture/shared-memory path 使用每 SM 的 unified on-chip SRAM，原书列出总计 256 KB。应用可用 `cudaFuncSetAttribute()` 与 `cudaFuncAttributePreferredSharedMemoryCarveout` 表达 L1/shared carveout 偏好；这通常是 hint，不保证精确 partition。若 kernel 需要超过默认上限的 dynamic shared memory，则应另用对应的 max-dynamic-shared-memory opt-in attribute。原书给出每 SM shared-memory 上限约 228 KB、每 block dynamic shared memory 约 227 KB；实际值、保留量、static+dynamic 合计及 opt-in requirement 都应查询目标 device/function attribute。

原书给出约 20–30 cycle 和每 SM TB/s 量级的 microbenchmark 结果。shared-memory bank conflict、address pattern、instruction width 和 occupancy 会影响 throughput；L1 与 shared memory 的访问机制也不同，不能共用单一 latency 结论。图 6-13 示意 block-scoped shared memory；其 allocation 随 block residency 占用 SM SRAM，并非永久专属于某个 block 的独立物理存储。

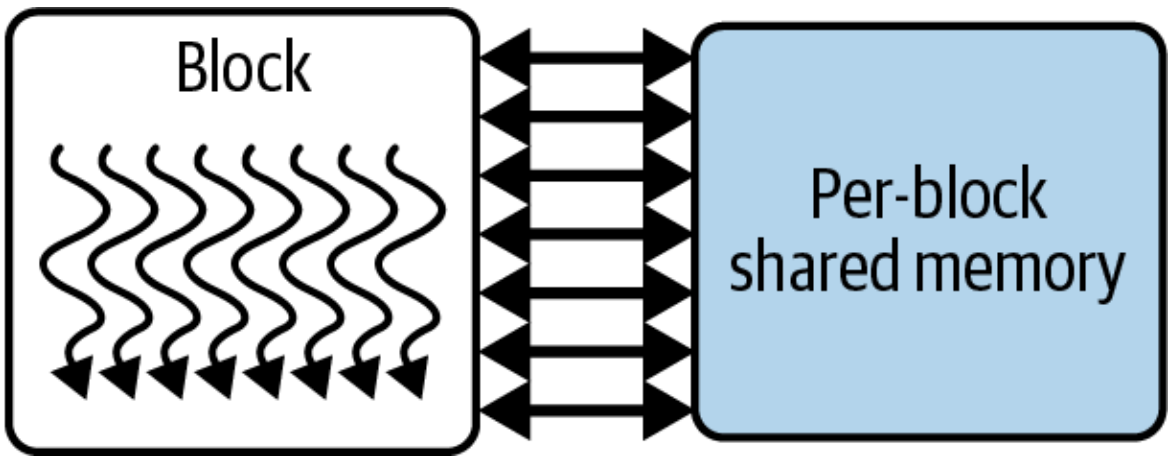


图 6-13. thread-block scoped shared memory

TMEM

TMEM 是 Blackwell 每 SM 约 256 KB 的 dedicated on-chip tensor memory，供 UMMA 与 `tcgen05` 等 Tensor Core-specific instruction 使用，第 10 章还会讨论。它不是普通 CUDA C++ pointer-addressable space，而是通过 tcgen05 allocation、load/store/copy 与 MMA instruction 管理。TMA descriptor 主要描述 global↔SMEM transfer，并不会自动管理 global、shared、TMEM 之间的全部数据流。典型 path 可让部分 operand 位于 shared memory、让 accumulator 位于 TMEM，但开发者/library/compiler 仍须明确安排 descriptor、barrier、lifetime 和相应 computation。

常量内存缓存

对于小型 read-only table，原书列出 Blackwell 每 SM 约 8 KB 的 constant cache，用于缓存 64 KB `__constant__` address space。当一个 warp 的 active lane 读取同一 address 时，constant path 可 broadcast 该值；具体 latency 依 cache hit 和 architecture，不应保证为 single cycle。

若同一 warp 读取多个不同 constant address，request 通常按 distinct address 数拆分/序列化。constant memory 可适合小型 lookup table，例如 rotary positional encoding parameter、Attention with Linear Biases（ALiBi）slope、部分 LayerNorm γ/β 或 quantization scale；但是否合适取决于容量和 access uniformity。cache hit 可减少下层 traffic，不能保证所有 thread 的所有访问都完全没有 global/L2 traffic。

L2 cache

L2 cache 位于 SM-local cache 与 off-chip HBM3e 之间，供 device 上各 SM 共享。原书为该 Blackwell configuration 列出 126 MB、约 200 cycle 和每秒数十 TB 的 aggregate bandwidth；这些均依 SKU、clock、partition、hit path 与 contention，不是固定值。

一个 block 取入 L2 的 data 有机会被其他 block 重用，从而避免再次访问 DRAM，但是否命中取决于 eviction、working set、access policy 和 scheduling。global access 应尽量让 warp lane 访问连续、对齐且可合并的数据；实际 memory transaction sector/cache-line 行为依 architecture 与 access width，不能把所有最佳访问固定为单个 128-byte transaction。

> 让 warp 的 global load/store 尽量连续、自然对齐并可 coalesce，可减少不必要的 sector/transaction；具体 alignment 和 transaction 数应按 element width、active lane 和目标 architecture 检查。128-byte aligned segment 是常见理想模式之一，但不保证完全避免 split，也不自动最大化 L2/DRAM bandwidth。

全局内存（HBM/DRAM）

global memory 的物理 backing 通常是 off-chip HBM/DRAM；local-memory object 和 register spill 也位于 device-memory backing，但访问可命中 L1/L2，不必每次承担完整 DRAM latency。原书给出的数百至上千 cycle 与约 8 TB/s

bandwidth 分别是 latency 范围和特定产品 peak 口径，不能据 peak bandwidth 推断单次访问延迟。

原书为 Blackwell B200 列出每 GPU 最多约 180 GB 可用 HBM 和总计约 8 TB/s。具体 capacity/bandwidth 随 B200/B300 SKU 与计量口径不同；应用可达 bandwidth 还受 coalescing、read/write mix、cache、ECC 和 contention 影响。图 6-14 示意 per-device global memory。

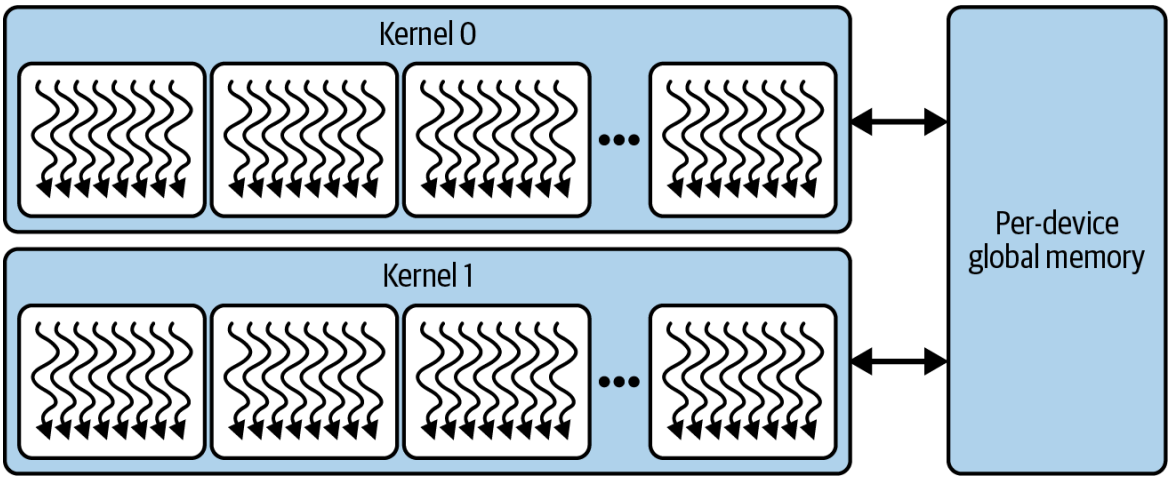


图 6-14. per-device global memory（HBM）

可以使用 Nsight Compute 查看 register spill、memory transaction、cache hit rate、throughput 和 stall metric，以判断 kernel 的实际 data path。profiler 能帮助定位 register、shared/L1、constant cache、L2 与 HBM 的瓶颈，但接近某个 on-chip peak 并非普遍目标，metric 也需要结合 workload phase 和 counter definition 解读。Blackwell 的 L2 与 unified L1/shared SRAM 可支持 reuse 和显式 staging，而 coalescing 主要由 warp address pattern 决定。

> Blackwell B200 对 CUDA 程序呈现为单个 GPU 和统一 global address space，但物理上由两个 die 组成，原书列出 die 间约 10 TB/s interconnect，每个 die 连接四个 HBM3e stack。统一寻址和单 GPU programming model 不代表所有跨 die/HBM access 在 latency、bandwidth、contention 上完全 uniform；实际 locality 和 fabric behavior 应用 profiler/microbenchmark 验证。

memory visibility 与 ordering 的有效 scope取决于 memory space、cache/coherence domain、参与者和所用 synchronization primitive。CUDA 提供 thread、thread block（*CTA*）、受支持的 cluster（*CTA cluster*）、device 和 system 等 scope，但仅选择 scope 并不能建立 data race-free communication；还需正确的 atomic/fence/barrier 与 happens-before relationship。图 6-15 给出这些 scope 的概念示意。

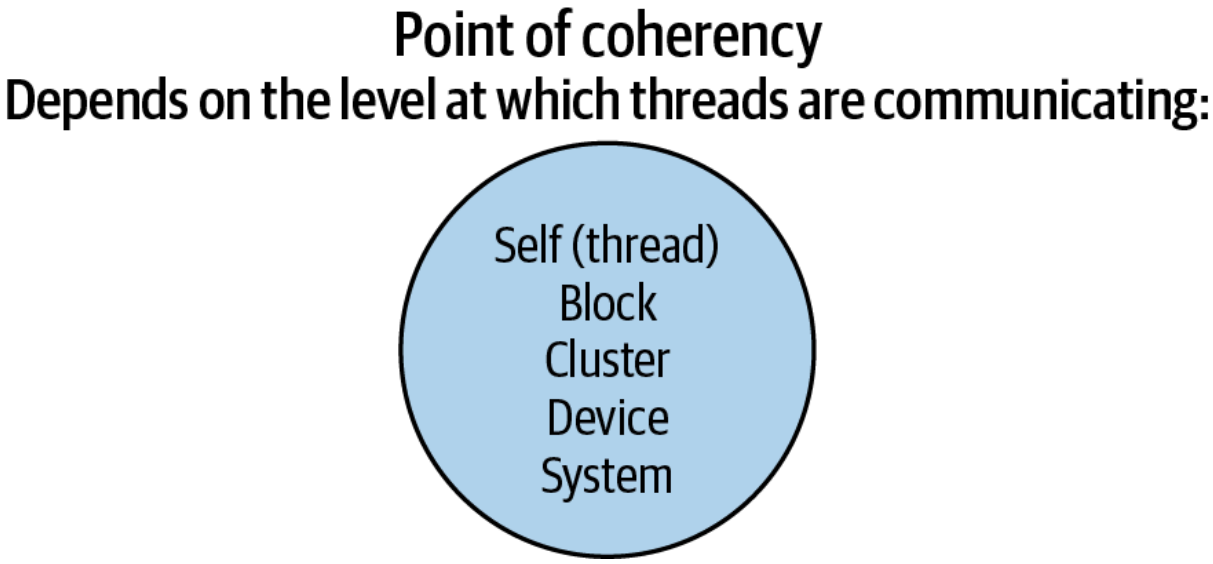


图 6-15. GPU thread 的 memory visibility/coherence scope

总之，应根据 access pattern、reuse、scope 和 synchronization 选择合适的 memory level，并用 profiler 验证。良好的 locality、合并访问、足够并发度和受控 resource pressure 可减少 exposed latency；但提高 occupancy 与最大化某层使用量并非独立目标，也不保证“fully leverage”硬件。下一节讨论 NVIDIA Unified Memory，并说明它与 Grace Hopper/Grace Blackwell 等 coherent CPU-GPU design 的关系。

Unified Memory（统一内存）

Unified Memory（也称 CUDA Managed Memory）为 managed allocation 提供 CPU 与 GPU 可使用的统一虚拟地址，从而减少手工维护 host/device buffer 和显式 `cudaMemcpy` 的需要。可访问性、page placement、migration 与 hardware coherence 能力依平台而异，不能把统一地址一概等同于所有 CPU/GPU access 的硬件 cache coherence。`cudaMallocManaged()` 创建的 page 可由 driver 按 access、advice 和 topology 迁移或远程访问，如图 6-16 所示。

Unified Memory 简化了编程，但 on-demand page fault、migration 或 remote access 可能引入难以从 source 直接看出的 latency 与 stall。例如 GPU 访问当前位于 CPU memory 的 managed page 时，driver 可 fault 并迁移 page，或在受支持平台按当前 policy 远程访问；传输互连可能是 PCIe、NVLink 或 NVLink-C2C，并非总是 NVLink-C2C。因此性能高度依赖 hardware topology、OS、driver、page size 和 access pattern。

在 PCIe 或较早的 discrete-GPU system 上，fault-driven migration 常因 fault handling、transfer granularity 和 interconnect bandwidth 而比预先安排的 bulk `cudaMemcpy` 更慢。Grace Hopper/Grace Blackwell 的 NVLink-C2C 提供最高约 900 GB/s 的双向聚合 coherency interconnect 口径，连接 Grace CPU memory subsystem 与 GPU HBM；CPU 侧不是原文所称 HBM。高互连带宽可降低 data movement 成本，但 page fault、placement 和 remote-access latency 仍与 device-native HBM access 不同。

kernel 执行期间的 managed-memory fault 会阻塞受影响的 access/warp，并可能通过 fault batching、dependency 和 migration 影响更大范围，但不必然让整个 GPU 完全

停顿。可在已知下一使用位置时调用 `cudaMemPrefetchAsync()` 提前请求 placement，如图 6-17 所示。

prefetch 向 driver 请求把指定 range 迁移到目标 location，并按给定 stream 排序；在 hardware/stream/dependency 允许时可以与其他工作 overlap。它并非保证在 kernel launch 前自动完成，调用方仍需建立正确 stream ordering。还可提供 memory advice，如下：

```
cudaMemAdvise(ptr, size, cudaMemAdviseSetPreferredLocation, gpuId);
cudaMemAdvise(ptr, size, cudaMemAdviseSetReadMostly, gpuId);
```

`PreferredLocation` 对应 preferred-location advice，提示 allocation 通常应驻留的位置；`ReadMostly` 表示 data 大多被读取，driver 可在适用平台创建 read-only duplicate。它不是 const correctness 或禁止 write 的保证，write 可能使 replica invalid。图 6-18 给出概念示意。

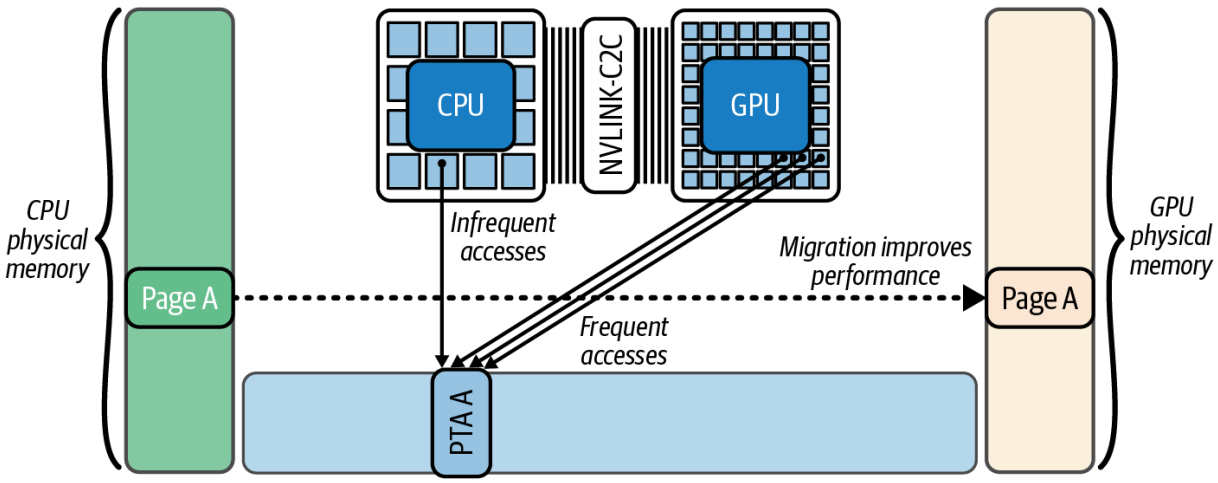


图 6-16. CPU-GPU Unified Memory 的自动 page migration

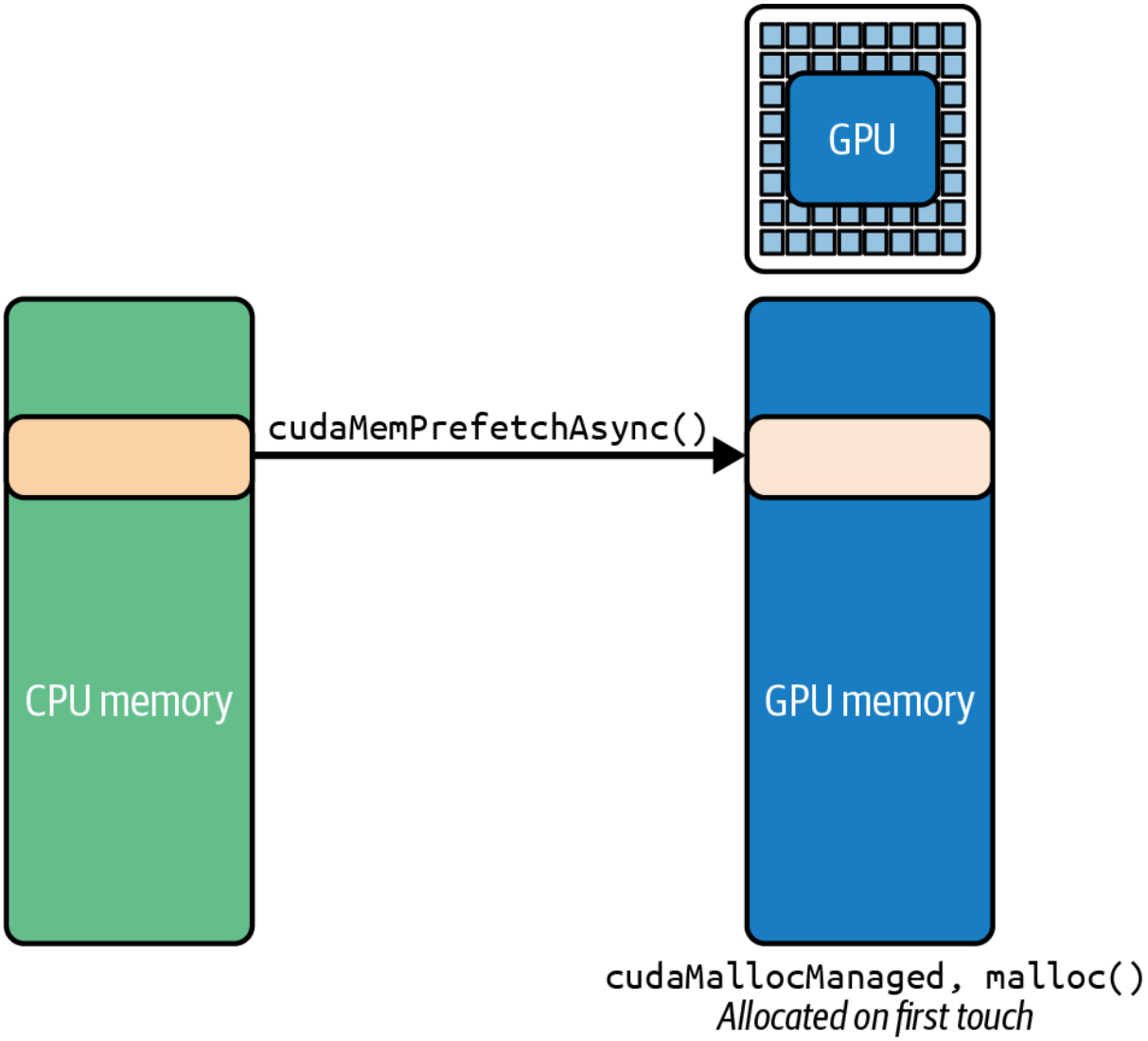


图 6-17. 使用 `cudaMemPrefetchAsync()` 请求经 NVLink-C2C 从 CPU 向 GPU prefetch data

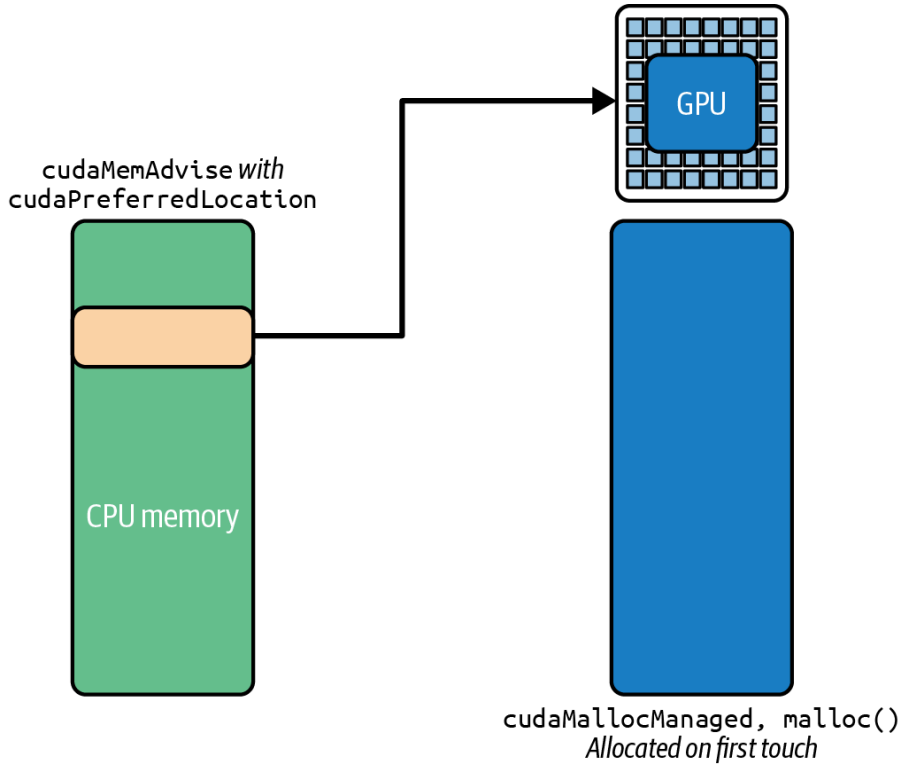


图 6-18. 指定 preferred location，并用 `ReadMostly` 等 advice 描述预期访问模式

还可用以下 advice 告知 driver：另一个 GPU 将访问该 range，以便预先建立必要 mapping、减少首次访问开销；这不保证 page 永不迁移：

```
cudaMemAdvise(ptr, size, cudaMemAdviseSetAccessedBy,
   otherGpuId);
```

managed allocation 的 access 可能触发 page fault、mapping 或 migration，并产生额外 ordering/latency。对于支持 stream attachment 的 managed memory，可把 association 改为 single-stream，以声明只有指定 stream 会访问该 allocation；这是访问约束和 coherency optimization hint，不是通用的 migration-overlap primitive。调用如下：

```
cudaStreamAttachMemAsync(stream, ptr, 0,
    cudaMemAttachSingle);
```

使用 cudaMemAttachSingle 后，应用承诺在 association 生效期间仅通过指定 stream 访问 memory；其他 stream/device 的并发访问可能违反 API contract，而不是被安全阻止。该调用按 stream 异步改变 association，但并不保证只有该 stream 能产生 fault、也不自动把 migration 与该 stream 工作 overlap，更不能笼统保证消除 cross-stream synchronization。

在 multi-GPU system 中，可按 topology 使用 `cudaMemcpyPeerAsync()` 显式复制，或把 managed range prefetch 到特定 device。prefetch 是 placement request，不等于永久 pin，也不存在适用于所有系统的“nearest NUMA-local GPU memory”自动选择；peer access、NVLink/PCIe route、memory capacity 和后续访问仍可能导致 remote access 或 migration。

总之，适当 prefetch 和 memory advice 可以减少 first-touch fault 与不期望的 migration，但不能保证消除大多数 stall。要取得效果，prefetch 必须覆盖正确 range/location、按 dependency 在使用前完成，并避免 CPU/GPU ping-pong；working set 超出 capacity、irregular access 或多 device 写共享仍可能产生 fault 和迁移。

主动 prefetch、合适 advice 与正确 stream association 有时能让 Unified Memory 接近手工 `cudaMemcpy` 的性能，同时保留统一地址的便利；这不是跨 workload/platform 的保证。应对比 end-to-end time、fault/migration counter、interconnect traffic、capacity pressure 和 correctness，再选择 managed memory、显式 copy 或混合方案。

保持足够的 occupancy 与 GPU utilization

GPU 可让多个 resident warp同时驻留；一个 warp 因 dependency 无法 issue 时，scheduler 可选择另一个 ready warp，从而隐藏部分 memory/instruction latency。*occupancy* 是 active warp 数相对于目标 SM architecture maximum 的比例，不是时间利用率、FLOP utilization 或 throughput 本身。

occupancy 很低时，ready warp 不足可能让 SM 在 memory/dependency stall 期间没有指令可发射；但低 occupancy 不必然表示低 SM utilization，例如具有足够 instruction-level parallelism 的 compute-heavy kernel 仍可能表现良好。Blackwell 每 SM 的 64 K register file 是 residency constraint 之一；能否支持更多 warp 且不 spill 取决于每 thread register demand、allocation granularity、block size 和其他资源。

每 thread 最多 255 个 register 是 architecture limit，不是性能目标。应同时查看 theoretical/achieved occupancy、register allocation、spill、eligible warp、stall reason 和 throughput，再调整 block size、launch bound 或 code；单纯压低 register 数可能增加 spill 并降低性能。

较高 occupancy 会增加存在 ready warp 的机会，但不会保证 compute unit busy；所有 warp 可能同时受 memory bandwidth、barrier 或相同 dependency 限制。warp 也不是从 SM 外“swap in”，而是 resident warp 之间进行硬件调度。用其他 ready work 覆盖某个 warp 的等待时间称为 *latency hiding*。

下面用一个极端示例说明：从单 thread 改为足够多的并行 thread，通常会同时改善 occupancy、device activity 和 throughput。CUDA 优化的基础要求是提供足够 parallel work 覆盖目标 GPU，但并非必须或总能“fully occupy”每项硬件资源。

若 achieved occupancy 很低、性能也差，且 profiler 显示 latency/eligible-warp shortage，可以增加 grid 中的 block、调整 block size 或降低不必要的 resource pressure。目标应是足以隐藏当前 latency 的并行度，而不是机械追求现代 GPU 的 80%–100%；许多高性能 kernel 在更低 occupancy 下达到最佳 throughput。

若 occupancy 已足够，而 kernel 受 memory throughput 限制，继续提高到 100% 通常无益，甚至会因限制 per-thread resource 而变慢。所需 resident warp 数取决于 latency、dependency 和 instruction mix；瓶颈也可能是 memory bandwidth、execution pipeline、serialization 或 synchronization。

为了展示 parallelism 的影响，考虑长度为 N 的 vector addition（C = A + B）。比较两个 kernel：`addSequential` 与 `addParallel`。`addSequential` 只让一个 thread 在 loop 中处理全部 N 个元素；`addParallel` 则让多个 thread 分担元素。这个对比同时改变了 parallelism、instruction count per thread、memory-level parallelism 和 launch geometry，不能把全部加速仅归因于 occupancy。

sequential version 由一个 GPU thread 串行处理整个 workload：

```
#include <cuda_runtime.h>

const int N = 1'000'000;

// Single thread does all N additions
__global__ void addSequential(const float* A,
                              const float* B,
                                    float* C,
                              int N)
{
    if (blockIdx.x == 0 && threadIdx.x == 0) {
        for (int i = 0; i < N; ++i) {
            C[i] = A[i] + B[i];
        }
    }
}

int main()
{
    // Allocate and initialize host
    float* h_A = nullptr;
    float* h_B = nullptr;
    float* h_C = nullptr;
    cudaMallocHost(&h_A, N * sizeof(float));
    cudaMallocHost(&h_B, N * sizeof(float));
    cudaMallocHost(&h_C, N * sizeof(float));

    for (int i = 0; i < N; ++i) {
        h_A[i] = float(i);
        h_B[i] = float(i * 2);
    }

    // Allocate device
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, N * sizeof(float));
    cudaMalloc(&d_B, N * sizeof(float));
    cudaMalloc(&d_C, N * sizeof(float));

    // Copy inputs to device
    cudaMemcpy(d_A, h_A, N * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, N * sizeof(float), cudaMemcpyHostToDevice);
```

```
    // Launch: one thread
    // Note: This kernel assumes <<<1,1>>>
    // (one block, one thread).
    // Do not change the launch config when running this example.
    addSequential<<<1,1>>>(d_A, d_B, d_C, N);

    // Ensure completion before exit
    cudaDeviceSynchronize();

    // Copy d_C => h_C (back to host)
    cudaMemcpy(h_C, d_C, N * sizeof(float), cudaMemcpyDeviceToHost);

    // Cleanup
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    cudaFreeHost(h_A);
    cudaFreeHost(h_B);
    cudaFreeHost(h_C);

    return 0;
}
```

该版本只启动一个有效 thread；硬件仍按 warp 执行，但其余 lane inactive，且只有一个 block/warp，绝大多数 SM 没有工作。因此 occupancy、lane utilization 和 device-level parallelism 都很低，性能也差。

在 PyTorch 等高层 framework 中也要避免从 host 反复发起 tiny GPU operation。下面的 naive Python loop 对每个元素执行 indexing/add/assignment，形成 N 次高层迭代以及大量 dispatch、kernel 和可能的 host-device synchronization 开销；其具体 kernel 数取决于 PyTorch implementation，不能简单断言恰好每次只有一个 add kernel：

```
import torch

N = 1_000_000
A = torch.arange(N, dtype=torch.float32, device='cuda')
B = 2 * A
C = torch.empty_like(A)

# Ensure all previous work is done
torch.cuda.synchronize()

# Naive, Sequential GPU operations - DO NOT DO THIS
with torch.inference_mode(): # avoids unnecessary autograd graph construction
    # This launches N tiny GPU operations serially
    for i in range(N):
        C[i] = A[i] + B[i]

torch.cuda.synchronize()
```

该代码把本可向量化的工作拆成大量 tiny operation，主要问题是 Python/dispatcher/launch 与 scalar indexing overhead、缺乏跨元素并行，以及可能的同步；其行为和单个 `addSequential` CUDA kernel 并不等价，也不能只用 occupancy 概括。

下面改为并行 CUDA kernel 和 PyTorch vectorized operation。图 6-19 示意各元素并行相加。

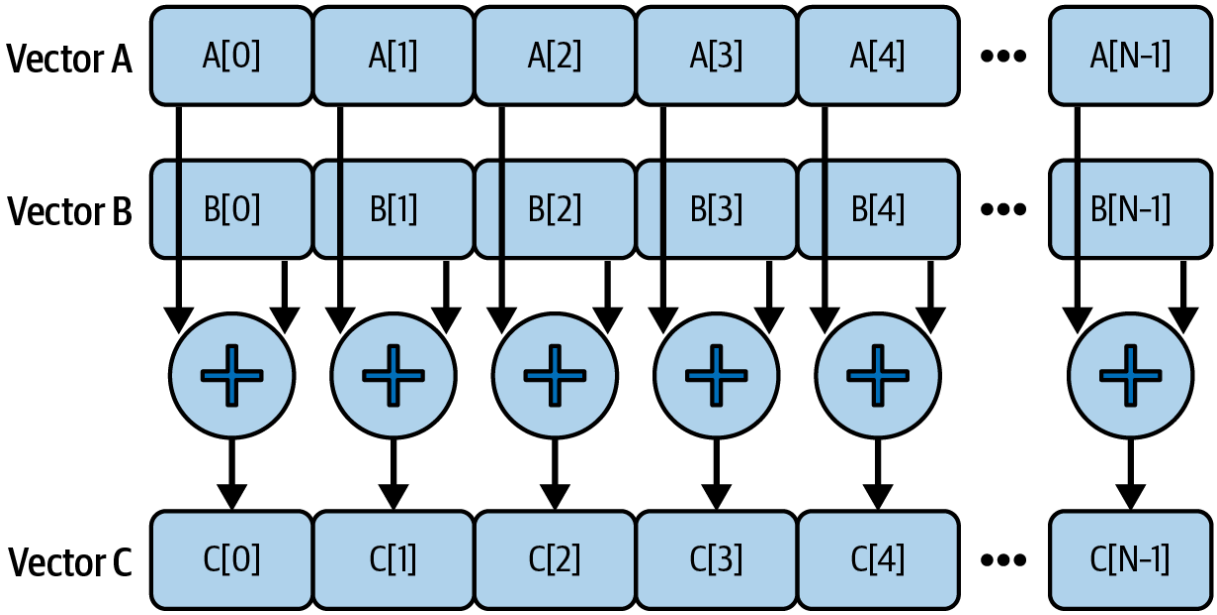


图 6-19. vector 中各元素的并行加法

下面的 CUDA C++ code 启动足够 thread 覆盖所有元素（`<<< (N+255)/256, 256 >>>`），每 block 256 个 thread，并以 ceiling division 计算 block 数，从而并行处理 N 个元素。bounds check 处理最后一个不完整 block；大规模输入仍需注意 integer overflow 与 grid limit：

```
#include <cuda_runtime.h>

const int N = 1'000'000;

// One thread per element
__global__ void addParallel(const float* __restrict__ A,
                            const float* __restrict__ B,
                                  float* __restrict__ C,
                            int N)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}

int main()
{
  // Allocate and initialize host (pinned for faster DMA)
  float* h_A = nullptr;
  float* h_B = nullptr;
  float* h_C = nullptr;
  cudaMallocHost(&h_A, N * sizeof(float));
  cudaMallocHost(&h_B, N * sizeof(float));
  cudaMallocHost(&h_C, N * sizeof(float));
  for (int i = 0; i *lt; N; ++i) { h_A[i] = float(i); h_B[i] = float(2*i); }

  // Create a non-blocking stream and allocate device buffers
  cudaStream_t s; cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);
```

```
    float *d_A = nullptr, *d_B = nullptr, *d_C = nullptr;
    cudaMallocAsync(&d_A, N * sizeof(float), s);
    cudaMallocAsync(&d_B, N * sizeof(float), s);
    cudaMallocAsync(&d_C, N * sizeof(float), s);

    // Async HtoD copies on the same stream
    cudaMemcpyAsync(d_A, h_A, N*sizeof(float), cudaMemcpyHostToDevice, s);
    cudaMemcpyAsync(d_B, h_B, N*sizeof(float), cudaMemcpyHostToDevice, s);

    // Launch (same stream)
    int threads = 256;
    int blocks  = (N + threads - 1) / threads;
    addParallel<<<blocks, threads, 0, s>>>(d_A, d_B, d_C, N);

    // Async DtoH copy and stream sync
    cudaMemcpyAsync(h_C, d_C, N*sizeof(float), cudaMemcpyDeviceToHost, s);
    cudaStreamSynchronize(s);

    // Cleanup (stream-ordered free)
    cudaFreeAsync(d_A, s); cudaFreeAsync(d_B, s); cudaFreeAsync(d_C, s);
    cudaStreamDestroy(s);
    cudaFreeHost(h_A); cudaFreeHost(h_B); cudaFreeHost(h_C);
    return 0;
}
```

当 N 足够大时，并行版本可利用更多 GPU resource。PyTorch 的 vectorized expression（`A + B`）通常会发起面向整个 tensor 的 GPU operation，让 backend 选择类似前述 `addParallel` 的并行 kernel；在 eager mode 下该 expression 通常对应一个 elementwise kernel，但 compilation/fusion/backend 可能改变实际 launch。下面是并行版本：

```
# add_parallel.py
import torch

N = 1_000_000
A = torch.arange(N, dtype=torch.float32, device='cuda')
B = 2 * A

torch.cuda.synchronize()

# Proper parallel approach using vectorized operation
# Launches a single GPU kernel that adds all elements in parallel
C = A + B

torch.cuda.synchronize()
```

> 在 PyTorch 中，优先使用 vectorized tensor operation、library primitive 或 compiler-generated fusion，通常能减少 Python/dispatch/launch overhead 并采用成熟 kernel。Python loop 不一定使 device operation 完全串行，因为 launch 可异步排队，但大量细粒度 operation 往往降低 efficiency。不存在“几乎总有”完全匹配的最优 native implementation；仍应检查 semantics、fusion、layout、temporary allocation 和 profiler timeline。

可使用 Nsight Systems 与 Nsight Compute比较 sequential 与 parallel implementation 的 timeline、kernel duration、active warp 和 execution

efficiency。下面给出原书的 Nsight Systems（nsys）与 Nsight Compute（ncu）命令；metric availability/name 依工具与 architecture version，命令本身也未显式收集表 6-6 的全部指标：

```
# Sequential add
nsys profile \
  --stats=true \
  -t cuda,nvtx \
  -o sequential_nsys_report \
  ./add_sequential.py

ncu \
 --section SpeedOfLight \
 --metrics
     sm__warps_active.avg.pct_of_peak_sustained_active,gpu__time_duration.avg \
 --target-processes all \
 --print-summary per-gpu \
 -o sequential_ncu_report \
 ./add_sequential.py

# Parallel add
nsys profile \
  --stats=true \
  -t cuda,nvtx \
  -o parallel_nsys_report \
  ./add_parallel.py

ncu \
 --section SpeedOfLight \
 --metrics sm__warps_active.avg.pct_of_peak_sustained_active \
 --target-processes all \
 --print-summary per-gpu \
 -o parallel_ncu_report \
 ./add_parallel.py
```

nsys 用于观察 CPU/CUDA timeline、launch、copy、dependency 与 idle interval；仅凭 timeline 不能断言 GPU 是“starved”还是某一具体瓶颈。ncu 收集 per-kernel hardware counter，用于分析 occupancy、warp state、memory/execution throughput 等原因。两者需结合 workload context 与 metric definition 解读。

只运行 nsys 可能看不到细粒度 per-kernel counter；只运行 ncu 则可能缺少端到端 launch/data/dependency context，而且 replay 会扰动 timing。表 6-6 保留原书示例结果，但缺少 GPU、软件版本、warm-up、重复次数和完整 metric command，不能视为可跨系统复现的 benchmark。

表 6-6. sequential 与 parallel CUDA kernel 的原书示例对比

指标	`add_sequential`	`add_parallel`
内核执行时间（ms）	48.21	2.17
GPU utilization（原书口径未定义）	1.5%	95%
达到的占用率	1.3%	38.7%
warp 执行效率	3.1%	100%

不同工具和版本的 metric 名称、分母与时间窗口不同。Nsight Systems timeline-derived GPU activity、Nsight Compute per-kernel SM Active、active-warps occupancy 和 warp execution efficiency 衡量的是不同对象，不能互换，也不能都解释成“SM 被 active warp 占满的程度”。报告结果时必须给出精确 metric 名称与采样范围。

按表中原书数据，从 single-thread implementation 改为 multiwarp implementation 后，achieved occupancy 由 1.3% 升至约 38.7%，duration 从 48.21 ms 降至 2.17 ms，算术比值约为 22.2×。加速来自大量 parallelism、lane utilization、memory-level parallelism 和每 thread 工作量变化，不能只归因于 occupancy。

sequential case 只有一个 block/warp，且仅一个 lane 执行 loop，因此 device coverage 和 active-lane ratio 都很低。parallel case 可让更多 SM/warp 处理不同元素，表中 warp execution efficiency 从 3.1% 变为 100% 与 1/32 lane 到完整 warp 的理想算术一致；但表中的 1.5%/95% “GPU utilization”缺少 metric definition，不能由 occupancy 或 active lane 直接推出。

该极端示例说明 throughput-oriented GPU 需要足够 parallel work；所需 thread/block 数依 GPU size、kernel duration、resource usage 和 workload，不能只用“越多越好”概括其 目标。

GPU 由 CPU/runtime 发起 kernel，并通过不同 memory space/cache 访问数据。host launch、instruction dependency 和 memory access 等 latency 只有在存在可独立推进的工作且资源允许时才能 overlap/hide；有些 bandwidth 或 serialization bottleneck 不能靠增加 warp 消除。

kernel 提供足够 independent warp 后，hardware scheduler 可在 ready warp 之间发射 memory/math instruction，从而覆盖部分等待时间。source code 并不直接“指示 GPU interleave”，实际行为由 dependency、scheduler 和 pipeline availability 决定。

multiwarp parallel kernel 可受益于 warp-level latency hiding：一个 warp 等待 load dependency 时，另一个 ready warp 可发射 add 或其他 load。是否并行发生以及能隐藏多少 latency 取决于 execution/memory pipeline、cache 和 bandwidth；后续章节将继续讨论。

sequential kernel 没有其他 warp 可在 dependency stall 时运行，因此大量 SM 完全空闲，唯一 active warp 的 pipeline 也可能出现等待。parallel version 提供更多可调度工作，但不保证 GPU 持续 busy；图 6-20 是概念性 timeline 对比，并非 profiler trace。

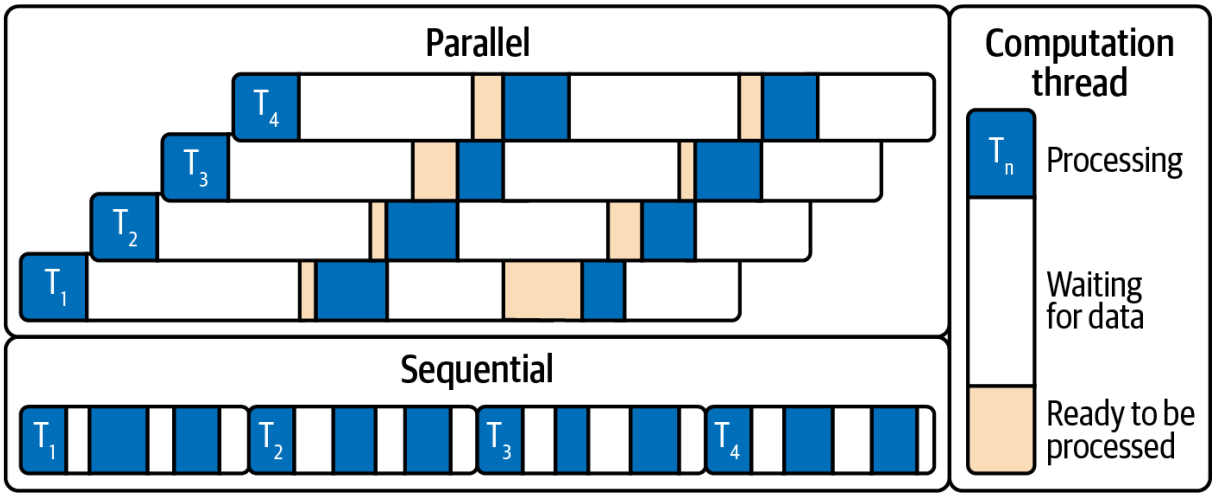


图 6-20. parallel 与 sequential timeline 的概念对比

图中 sequential timeline 在 memory wait 期间出现 idle gap；parallel timeline 用其他 warp 的工作覆盖部分 gap。真实 kernel 仍可能受 bandwidth、dependency、barrier、tail effect 或资源竞争限制，因此不会必然连续 busy。

关键是先提供足够 parallel work 覆盖目标 device，再确认是否存在 latency-hiding shortage。足够 occupancy 可以减少部分 idle stall，但不保证最大 throughput；本例原书报告约 95% 的未定义 GPU-utilization metric，应只视为该测试结果。

有足够 thread 后，还应优化 active-lane efficiency、instruction-level parallelism、memory access 和 per-thread work。即使 occupancy 为 100%，kernel 仍可能受 memory bandwidth/latency、execution pipeline、dependency 或 synchronization 限制；memory-bound 也不只是“memory 很慢”，而是当前 arithmetic intensity 下性能受 memory subsystem 上界约束。

LLM 的 token-by-token decode 常见于低 arithmetic-intensity、memory-bandwidth-sensitive 场景，尤其 batch 较小时。执行各 layer 时会从 HBM/cache 读取 weight、KV cache 和 activation，并通过 register/shared-memory 等片上路径参与计算；不是把整套 model weight 一次性搬进 register/shared memory。

若模型有数千亿 parameter，按每 parameter 8 bit 粗略计算，weight storage 可达数百 GB；实际 footprint 还包括 scale/metadata、KV cache、activation 和并行分片。每 token 读取多少 byte 取决于 quantization、batch、cache、parallelism 和 kernel fusion，并非每次都把整个模型“移入移出 GPU”，但 repeated HBM reads 足以成为 bandwidth bottleneck。

> 峰值 compute throughput 的增长通常快于 off-chip memory bandwidth。原书为特定 Blackwell configuration 列出约 8 TB/s HBM3e peak；实际 SKU、精度、operation 和可达 bandwidth 不同。对 arithmetic intensity 较低的 workload，应优化 reuse、layout、batching、fusion 和 data movement，并用 roofline/profile 判断是否 memory-bound，而不是预设所有现代 AI workload 都受同一瓶颈限制。

使用 launch bounds 调优 occupancy

仅增加 thread 数有时无效，尤其当每 thread/block 使用大量 register 或 shared memory 时。CUDA kernel annotation `__launch_bounds__` 可向 compiler 提供预期 launch constraint，使其据此权衡 register allocation 与 code generation；它并不直接“优化 occupancy”。

该 annotation 的前两个参数分别是：应用承诺的 maximum threads per block，以及希望 compiler 在资源预算中支持的 minimum blocks per multiprocessor。第一个是 launch contract，超过它启动 kernel 会失败；第二个用于推导 register-use upper bound，但不保证 runtime 实际驻留该数量，因为 shared memory、block limit 和 grid size 等仍可能限制。示例如下：

```
__global__ __launch_bounds__(256, 16)
void myKernel(...) { /* ... */ }
```

`__launch_bounds__(256, 16)` 承诺该 kernel 每 block 不超过 256 个 thread，并要求 compiler 尝试按每 SM 至少 16 个 block 的目标约束 register usage。16 × 256 = 4,096 个 thread，超过原书所述目标 GPU 每 SM 2,048-thread limit，因此第二个参数在该 target 上不可实现；它不是让 compiler“分配足够 register 和 inline function”来突破硬件上限。

> 原书以许多现代 NVIDIA GPU（包括所述 Blackwell target）的每 block 1,024 thread、每 SM 2,048 resident thread 为例；实际 limit 应由目标 compute capability/device attribute 确认。

对于每 SM 最多 2,048 thread 的 target，仅按 thread limit 计算最多驻留 8 个 256-thread block；其他资源可能进一步降低数量。compiler/ptxas 不会把 16 自动“缩减为 8”并继续保留等价 request，而可能警告该 minimum-CTA constraint 超出范围并忽略相应 directive。

> 原书给出的 warning 类似：“ptxas warning: Value of threads per SM…is out of range. `.minnctapersm` will be ignored.” 具体文字和行为依 Toolkit/target；build system 不应忽略 ptxas warning。

`__launch_bounds__` 会影响 compiler 推导的 register-use threshold。为满足 minimum-block target，compiler 可能降低 register use，从而提高潜在 residency，也可能增加 instruction count 或 spill；在某些情况下 compiler 还可能提高 register use 到不降低目标 residency 的上限，以减少指令。unrolling/inlining 变化是优化结果，不是 annotation 的固定语义，因此不保证更一致的 warp throughput。

*提高 occupancy 必须与 per-thread resource 权衡。*spill 是 compiler 因 register pressure/constraint 把 value 放入 local memory 的结果，不是运行时“thread 太多而

耗尽 register”。减少 register 可提高 residency，也可能增加 local-memory traffic 或 recomputation。

CUDA Occupancy API 可在 host 侧根据目标 device 和 kernel resource metadata 估算 launch configuration。例如，`cudaOccupancyMaxPotentialBlockSize()` 搜索潜在 occupancy 较高的 block size，并考虑 register、static/dynamic shared memory 与 block limit。`cudaOccupancyMaxPotentialBlockSize` 是静态 occupancy calculator，不执行 kernel、不测量时间，也不是 performance autotuner：

```
int minGridSize = 0, bestBlockSize = 0;

// If your kernel uses dynamic shared memory (extern __shared__),
// set this correctly:
size_t dynSmemBytes = /* bytes per block (e.g., tiles * sizeof(T)) */ 0;

cudaOccupancyMaxPotentialBlockSize(
  &minGridSize, &bestBlockSize,
  myKernel,
  dynSmemBytes,      // must match your kernel's dynamic shared memory use
  /* blockSizeLimit = */ 0);

// Compute a grid that covers N, but don't go below the min grid
// that saturates occupancy
int gridSize = std::max(minGridSize, (N + bestBlockSize - 1) / bestBlockSize);

myKernel<<<gridSize, bestBlockSize, dynSmemBytes>>>(...);
```

API 返回候选 `bestBlockSize` 和达到该潜在 occupancy 所需的 minimum grid size。`minGridSize` 不是覆盖长度 N 的正确性 grid，也不保证运行时“saturate”任何 utilization metric。示例使用 `gridSize = max(minGridSize, ceil_div(N, bestBlockSize))` 时会在小 N 下启动额外 thread，kernel 必须有 bounds check 或 grid-stride loop；还要处理 N=0、integer overflow 和 grid limit。若使用 extern `__shared__`，必须传入与 block size 对应的真实 dynamic-shared-memory function/value，而不是错误常量。

> 应对 Occupancy API 候选及相邻合法 block size 进行 warm-up 和重复计时，并检查 correctness。register pressure、instruction count、shared memory、cache、memory coalescing 和 problem shape 都可能让 sub-maximal occupancy 更快；“±1–2”应理解为相邻 warp-multiple candidate，而不是任意增减 1 个 thread。

compiler heuristic 通常是起点，`__launch_bounds__` 和 occupancy calculator 可表达 constraint/提供候选，但不是对 runtime occupancy 的显式控制。只有在 compiler report 与 profiler *证明* resource-limited residency 位于关键路径，并且 kernel 能安全权衡 per-thread resource 时才应使用；否则可能损害性能或限制可用 block size。

register 与 occupancy 的 trade-off 很重要。较少 register 可能允许更多 resident warp并改善 latency hiding，也可能造成 spill、更多 instruction 或更短 live range；较多 register 可能提高 per-thread efficiency 而降低 residency。最佳点需实验。可结合 Nsight Compute 的 register、theoretical/achieved occupancy、spill、eligible-warp、stall 与 duration metric 判断。

使用 NVIDIA Compute Sanitizer 调试功能正确性

CUDA kernel 可启动大量 thread，memory bug、race 和 synchronization error 往往具有 timing-dependent behavior，普通 debugger/test 不易覆盖。CUDA Toolkit 中的 NVIDIA Compute Sanitizer 通过 runtime instrumentation 和相应检查器报告一部分 correctness error。它能提高缺陷发现率，但会显著增加运行开销，也不能证明未报告的程序完全正确或可靠。

可通过 `compute-sanitizer` CLI 启动工具，并结合 NVIDIA Tools Extension（NVTX）annotation 改善报告上下文。是否应广泛插入 NVTX 取决于可观测性需求和 annotation overhead；它不会替代 sanitizer coverage。CLI 通常使用 `--option value`，可用 `--error-exitcode` 让发现 error 的进程返回指定非零状态。`--kernel-name`、`--kernel-name-exclude` 等 filter 可缩小 instrumented kernel 范围；`--nvtx yes` 启用 NVTX 相关支持，但具体 filtering/leak-report behavior 依 Compute Sanitizer version/tool，应查当前 CLI 文档：

```
compute-sanitizer [--tool toolname] [options] <application> [app_args]
```

可把 Compute Sanitizer 纳入 CI 的代表性小规模 GPU test，并用 `--error-exitcode` 使报告 error 时任务失败。kernel filter 和 NVTX 可控制成本/上下文，但过滤过窄会漏掉跨 kernel/lifecycle 问题；还需固定 CUDA/tool version、保存日志并设置合理 timeout。

Compute Sanitizer 包含四个主要 tool：memcheck、racecheck、initcheck 和 synccheck。它们分别覆盖不同类型的 memory access、shared-memory hazard、未初始化 global-memory read 和 synchronization misuse，覆盖范围不能由名称无限外推：

memcheck

: memcheck 可精确检测/归因许多 global、local 和 shared-memory 的 out-of-bounds 或 misaligned access，报告 GPU hardware exception，并检查部分 device-side allocation leak。支持项与默认值依版本；例如可用 `--check-device-heap` 控制 device heap checking。它不能检测所有逻辑越界、host race 或未覆盖 execution path。

racecheck

racecheck 主要报告 shared-memory access hazard，包括 Write-After-Write、Write-After-Read 与 Read-After-Write，并分析相应 warp/block synchronization pattern。它不是通用 global-memory data-race detector；hazard report 也要结合 programming model 判断 severity。Volta 及后续 independent thread scheduling 下，过去依赖 implicit warp lockstep 的代码尤其需要显式合适同步。

initcheck

initcheck 检测对未初始化 device global memory 的读取，例如遗漏 host-to-device initialization 或某些 device write path。它不检查所有 memory space/逻辑初始化条件，通常应先运行 memcheck，避免 invalid access 干扰 initialization analysis。

synccheck

synccheck 检测一部分 synchronization primitive misuse，例如 divergent/mismatched barrier、无效 barrier argument 或 warp-level synchronization mask 问题，具体能力依 architecture/tool version。它不是完整的 deadlock detector，也不会证明所有 thread ordering、memory fence 和 application state 都正确。

总之，NVIDIA Compute Sanitizer 能帮助发现 CUDA application 中一部分 memory、shared-memory race、initialization 和 synchronization bug。将其用于有代表性的 CI/test workload，可更早发现回归；仍应结合 unit/property test、error checking、compiler warning、debug build 和真实并发场景，因为 sanitizer clean run 既不证明完整 correctness，也不证明 production performance。

Roofline 模型：计算受限还是内存受限工作负载

Roofline 模型是一种实用的可视化方法，它刻画两类硬件性能上限：处理器峰值浮点运算率形成的水平线，以及由峰值内存带宽决定的斜线。两者共同构成“屋顶线”包络，可用来判断给定 kernel 在该模型及所选度量口径下更接近计算受限（compute bound）还是数据移动受限（memory bound）。

两条线的交点称为*山脊点（ridge point）*，它对应算术强度阈值：在基础 Roofline 模型中，山脊点左侧受内存带宽上限约束，右侧受计算上限约束。算术强度是执行的 FLOP 数与某一内存层级传输字节数之比。这里采用 GPU 与片外 HBM/global memory 之间的数据流量作为字节口径；若改用 L1、L2 或算法层面的流量，所得强度和对应 roof 也会不同。

来看一个简单示例。假设某个 kernel 读入两个 32 位浮点数（共 8 字节），执行一次加法（1 FLOP），再写回一个 32 位浮点结果（4 字节）。若这 12 字节都形成片外流量，则算法以 12 字节内存流量完成 1 FLOP，算术强度为 0.083 FLOP/byte（1 FLOP / 12 byte ≈ 0.083 FLOP/byte）。缓存命中、write allocation、合并访问和实际事务粒度会使测得流量偏离这个简化估算。

再将其与 10 FLOP/byte 的 GPU 山脊点比较（约 80 TFLOP/s ÷ 8 TB/s = 10 FLOP/byte）。示例 kernel 的算术强度为 0.083 FLOP/byte，位于 roofline 左侧的内存受限区域，比该阈值低 100 倍以上，因而无法仅靠增加计算单元就达到计算峰值。在这个简化模型中，其性能上限主要由内存流量决定。图 6-21给出了 Blackwell 级 GPU 的示意性 Roofline：水平线表示约 80 TFLOP/s 的 FP32 计算上限，斜线对应 8 TB/s 的 HBM 带宽上限。

山脊点应由同一测量口径下的计算吞吐上限除以内存带宽上限得到；采用理论峰值或持续实测值都可以，但不能混用。图中的两个示意上限相除得到 10 FLOP/byte。示例 kernel 位于斜向内存带宽 roof 上的 0.083 FLOP/byte 处，因此 Roofline 模型将其归入内存带宽受限区域。

要减轻这种内存受限程度，可以通过数据复用、算子融合或减少不必要的数据移动，让每传输一个字节完成更多有效运算，使工作点向右移动并接近计算 roof。不过，单纯增加无用 FLOP 并不会改善实际性能；优化目标应是提高有用计算与必要流量之比。

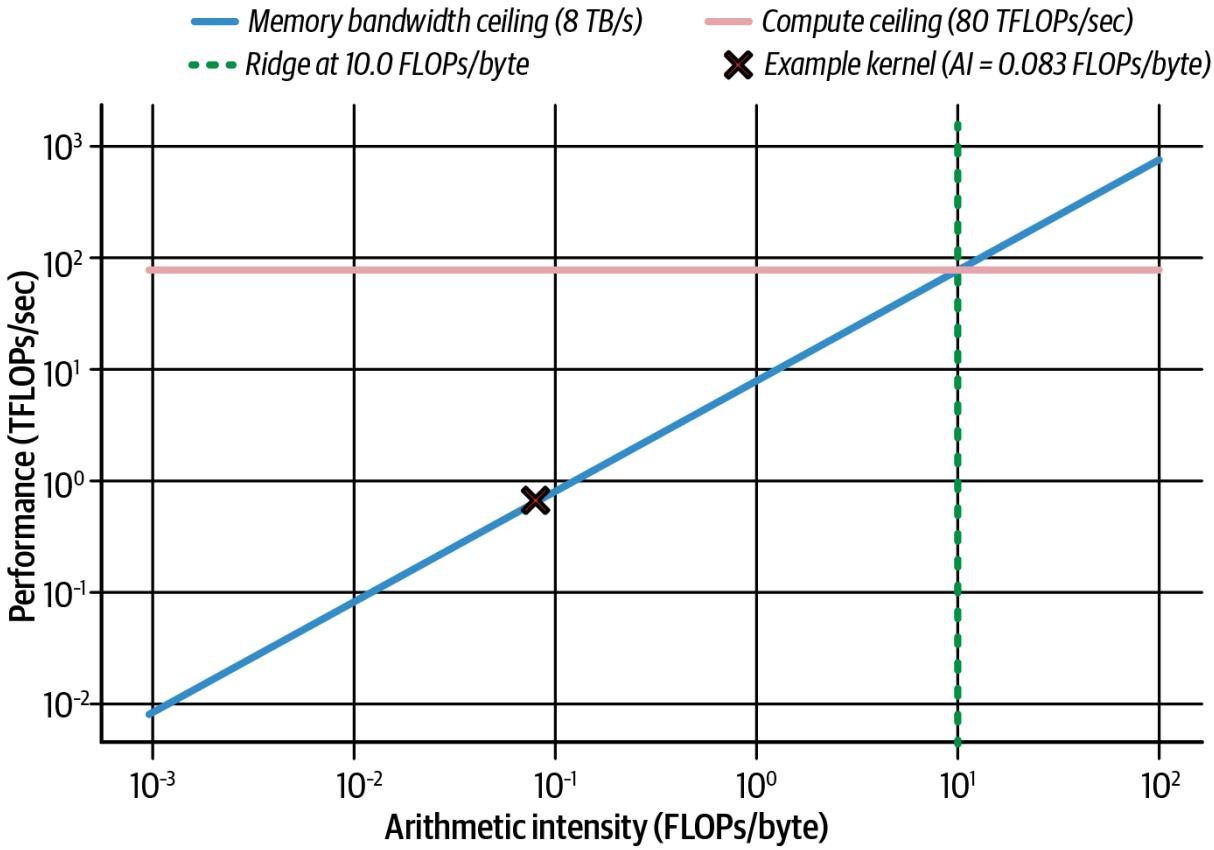


图 6-21。Blackwell 级 GPU 的 Roofline 模型（FP32 约 80 TFLOP/s，HBM3e 约 8 TB/s），其中标出了示例 kernel 的工作点以及算术强度约 10 FLOP/byte 的山脊点

降低数据精度是减少字节流量的一种方法。例如，在运算次数和访存次数不变、且没有额外转换或元数据开销时，以 16 位浮点数（FP16）替代 32 位浮点数（FP32），每个值的传输字节数减半，FLOP/byte 算术强度可相应翻倍。

现代 GPU 还提供面向 8 位浮点（FP8）的 Tensor Core。Blackwell 又为特定 AI 工作负载引入原生 4 位浮点（FP4）Tensor Core 支持。较窄的数据格式可以继续减少每个值的字节数，但实际收益取决于算子是否使用相应 Tensor Core 路径、量化尺度与元数据开销，以及精度要求。

例如，FP8 每个裸数据值占 1 字节，相比 FP16 可将值存储量减半；FP4 每个裸值占半字节，可用于部分推理工作负载。不能仅由位宽断言计算吞吐必然翻倍，因为不同精度的峰值、指令形态、累加精度、稀疏性口径和实际 kernel 效率各不相同。

若忽略对齐、打包和元数据开销，一次 128 字节内存传输可容纳 32 个 FP32、64 个 FP16、128 个 FP8 或 256 个 FP4 值。Blackwell 还提供针对压缩模型权重的数据解压机制，可在数据送入 Tensor Core 路径时进行硬件辅助解压。若权重在 HBM 中以受支持的压缩格式存储，读取的物理字节数可能减少，从而提高该权重流的有效带宽；具体格式和路径必须以相应 GPU、CUDA 与算子文档为准。

因此，Blackwell 对 transformer token generation 等经常受权重读取限制的工作负载具有潜在优势。权重可以采用低位宽格式存储，在加载与计算路径中解码，并以适当精度进行乘加或累加。不过，“4 位或 2 位权重自动由硬件解压并统一转换为 FP16/FP32”并不是所有 kernel 的通用行为。低精度只有在软件栈选中受支持路径且模型质量可接受时，才会减少数据移动并提高有效吞吐。

对于内存受限工作负载，优化方向通常是减少必要的数据传输或提高数据复用，使有效工作点向右、向更高性能区域移动。目标不一定是把 kernel 变成计算受限，而是在精度、延迟和资源约束下缩短实际执行时间。

> Transformer 模型（如 LLM）在不同阶段可能分别表现为计算受限或内存受限。prefill 通常具有较大的矩阵维度和较高并行度，许多 GEMM 更容易接近计算受限；逐 token decode 的小批量矩阵运算则常受权重或 KV cache 带宽、kernel launch 与延迟限制。但这不是按“attention 与矩阵乘法”划分的固定结论，实际边界随 batch size、sequence length、并行策略、量化和硬件而变化。第 15–18 章将在深入讨论推理时进一步说明。

内存带宽受限 kernel 常表现为较高的 DRAM 吞吐利用率和较低的计算管线利用率，但不能仅凭这一组合断言所有 warp 都在等待内存。低计算利用率还可能来自指令依赖、低 occupancy、分支、同步或 launch 配置；延迟受限 kernel 也可能尚未打满 DRAM 带宽。

要进一步定位原因，可使用 Nsight Compute 查看单个 kernel 的吞吐、cache hit rate、warp issue stall reason 和源代码级指标。不同版本支持的 replay、source correlation 和 launch 相关指标有所差异，应以当前版本文档和采集权限为准。这些信息有助于区分内存依赖、寄存器压力和 launch 配置等影响。

随后可用 Nsight Systems 从全局时间线观察 GPU 空闲区间、CPU 工作重叠以及 PCIe/NVLink 活动。两者结合，可以同时回答*为什么*某个 kernel 受限，以及这些事件在应用整体执行中*何时*发生。

关键是迭代进行性能分析，结合 Nsight Compute 与 Nsight Systems 的指标定位内存热点。可以在可疑代码周围添加 NVTX range，放大检查时间线，再依据测量反馈进行优化。

例如，可用 NVTX 将区域标记为“memory copy”或“kernel execution”，并在 Nsight Systems 时间线中观察它们。这有助于确认前文所述的主机与设备传输是否真正和计算重叠。

要验证重叠，可以为数据传输和 kernel 调用相关的主机代码添加 NVTX range。Nsight Systems 会显示这些 range 以及关联的 CUDA 活动，但 NVTX range 本身通常表示主机 API 区间，并不等同于设备操作的精确起止时间。使用异步内存复制（`cudaMemcpyAsync`）只是实现重叠的必要条件之一；是否能与 GPU kernel 执行重叠，还取决于 stream、依赖关系、页锁定主机内存、copy engine、传输方向和硬件并发能力。图 6-22对比了同步传输与可能发生重叠的异步内存传输。

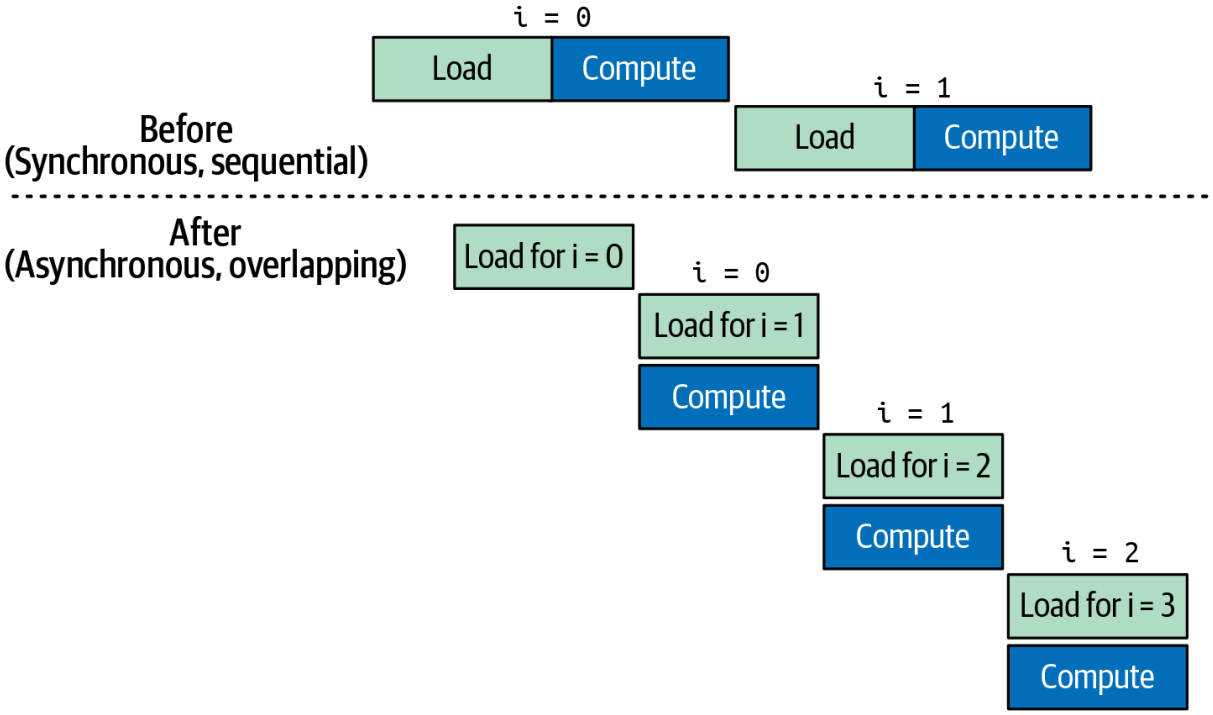


图 6-22。与 kernel 计算串行的同步数据传输，以及具备条件时可与计算重叠的异步数据传输

如果预期发生重叠，但时间线中复制与 kernel 仍然串行，应检查意外的 default-stream synchronization、事件依赖、API 的同步语义、是否使用不同的 non-default stream、主机缓冲区是否为 pinned memory，以及设备能否并发执行相应 copy 与 kernel。不能只凭串行现象锁定某一个原因。

> 对于涉及普通 pageable host memory 的主机与设备传输，`cudaMemcpyAsync` 可能需要内部 staging 或产生同步行为，不能依赖其与 kernel 稳定重叠。要获得可预期的异步 H2D/D2H 行为，通常应使用 pinned（page-locked）host memory；此外仍须满足 stream、依赖关系与硬件并发条件。

怀疑 kernel 供数不足时，可以先用 Nsight Compute 分析 kernel，再用 Nsight Systems 检查端到端时间线。global load efficiency 较低通常表示线程请求的有效字节与实际传输字节之比较低，即访问未充分合并或存在浪费；它并不直接表示 DRAM 响应速度不足。Nsight Systems 中 kernel 之间的空档也可能来自主机 launch、同步、I/O 或依赖，需结合 CUDA API 与传输轨迹判断。

应用本章的内存层次结构优化后，应重新测量相关指标。成功的优化可能减少时间线空档、提高有效带宽或缩短 kernel 时间，但 memory pipe 利用率并非越接近峰值越好：若

用更少流量完成同样工作，利用率可能下降而性能反而提高。最终应以正确性、端到端延迟和吞吐量为判断依据。

> 每次改动后都要测量。性能分析工具可以验证优化是否真正减少了内存相关停顿并改善端到端性能。

要点回顾

本章介绍了如何选择 launch parameter、权衡 occupancy，如何异步管理 GPU memory，以及如何运用 Roofline 分析判断 kernel 更接近计算受限还是内存受限。下面回顾几个关键要点：

SIMT 执行模型

GPU 采用单指令多线程（SIMT）模型，以 warp（32 个 thread）为调度单位发射指令。同一 warp 的活跃 thread 执行同一条指令，但分支可能使不同路径分阶段执行。保持足够多的 resident warp 有助于隐藏 memory 与 pipeline latency；所需 occupancy 取决于 kernel 的依赖关系和资源使用，并非越高越好。

Thread 层次结构：thread → block → grid

thread 组成 thread block（每个 block 最多 1,024 个 thread），thread block 再组成 grid，使同一 kernel 可扩展到大量 thread。同步机制（如 `__syncthreads()` 或 cooperative groups）可协调 shared memory 中的数据复用，但会引入等待和约束；应在正确性所需的范围内减少不必要的 barrier。

Occupancy 与资源限制

block size 通常选择 32 的倍数，以免最后一个 warp 未满，但最终值仍需结合访存、并行度和资源使用进行测量。还要注意每个 SM 的架构与型号限制。原文列出的 Blackwell 示例上限包括每个 thread 255 个 register、每个 SM 228 KB shared memory、64 个 resident warp 和 32 个 resident thread block；具体可用 shared memory 与驻留限制应以目标 GPU 的 compute capability、配置和 CUDA 文档为准。

CUDA 内核启动参数

对于简单的一维 element-wise kernel，可将 `threadsPerBlock = 256`（8 个 warp）作为起始测试点，并以 `blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock` 覆盖全部元素。这不是通用最优值，应依据性能分析结果调整，包括 register 使用与 spilling、shared-memory 占用、achieved occupancy、访存效率和执行时间。

异步内存管理

在适合 stream-ordered allocation 的场景中，可使用 `cudaMallocAsync`/`cudaFreeAsync` 和 CUDA memory pool，减少传统分配路径可能引

起的 device-wide synchronization 与重复底层分配开销；不要求把它们固定放在某个“专用 stream”，但必须正确表达对象生命周期与跨 stream 依赖。PyTorch caching allocator 采用类似的复用思路来提高 tensor 分配效率，减少昂贵的 `cudaMalloc()` 和 `cudaFree()` 调用，具体 backend 与行为随配置和版本而异。

GPU 内存层级

register → L1/shared memory → L2 → global memory（HBM3e）→ host memory：不同层级在容量、延迟、带宽、作用域和可编程性之间存在权衡。应在资源约束允许时提高 register、shared memory 与 cache 中的数据复用，但过多占用 register 或 shared memory 也可能降低并发度。

Unified Memory 注意事项

CUDA Managed Memory（Unified Memory）可以简化编程，但访问模式不佳时可能触发 page fault、migration 或远程访问。可依据目标架构和访问阶段测试 `cudaMemPrefetchAsync` 与 memory advice；它们是放置提示和迁移工具，并不能保证消除所有意外停顿。

Roofline 模型分析

算术强度（FLOP/byte）与所选计算、内存 bandwidth roof 共同决定 kernel 在 Roofline 模型中的上限区域。FP16、FP8、FP4 或受支持的硬件解压路径可能减少数据字节数，但收益取决于精度、数据复用、元数据和实际指令路径。应结合 Nsight Compute 的单 kernel 指标与 Nsight Systems 时间线判断真正瓶颈。Blackwell 的 TMEM 为 Tensor Core 累加结果提供专用片上存储，UMMA 配合低精度数据路径可能减少部分 register/shared-memory traffic 或改善 pipeline，但不能笼统保证把 memory-bound kernel 转为 compute-bound。第 10 章将进一步讨论 UMMA。

总结

本章梳理了 GPU 的 SIMT 模型、thread 层次结构与多级内存系统，为高性能 CUDA 开发奠定基础。Occupancy 通常指 active warp 数相对于该 kernel 在目标 SM 上可达到的最大 resident warp 数之比，是隐藏延迟的重要因素之一；工具中 theoretical 与 achieved occupancy 的具体定义还需结合指标说明。

不过，最大化 occupancy 并不保证在所有场景中获得最佳性能。如果 thread 具有足够的指令级并行性（ILP），或者瓶颈位于其他资源，中等乃至较低 occupancy 也可能实现很高吞吐。

较高 occupancy 有助于隐藏延迟，但降低 active thread/block 数有时能让每个 thread 使用更多 register，减少 spilling，或允许更大的 tile，从而提高吞吐。这里释放的是每个 SM 的资源预算，使 resident block/thread 在不同配置间重新权衡，并不是“为其他 thread 免费增加 register”。应针对目标工作负载和硬件基准测试不同配置，寻找实际最优点。

掌握这些基础知识和性能分析方法后，就可以进一步开展有针对性的优化，例如减少 warp divergence、利用 GPU memory hierarchy，以及异步预取数据。后续还会深入介绍

TMA：它可以把受支持的 bulk tensor transfer 从 thread 的逐元素地址生成与搬运工作中分离出来，并与计算流水化；但数据传输仍占用相应内存与 copy 资源，实际 computational goodput 提升需要通过测量确认。

第 7 章　分析与调优 GPU 内存访问模式

随着 AI 模型规模和复杂度增长，GPU memory system 往往成为理论计算能力与实际性能之间的瓶颈。正如第 6 章所述，现代 NVIDIA GPU 将大量面向吞吐优化的 core 与专用 Tensor Core 结合起来，并配有高带宽内存（HBM）、CPU-GPU 统一虚拟地址与一致性机制（具体能力随 Grace Blackwell Superchip 等平台而异）、片上 shared memory、cache，以及 Tensor Memory Accelerator（TMA）等专用数据搬运引擎。

本章将介绍多种 CUDA C++ 与 PyTorch 优化技术，包括合理组织和对齐数据以提高访存效率、消除重复加载，以及利用硬件能力让数据传输与计算重叠。

通过矩阵乘法、tensor operation 等优化前后的具体示例，你将看到 memory access pattern、tiling strategy 和异步数据传输方面的改动如何减少浪费的带宽并提高有效算术强度。它们可能改变 kernel 的主要瓶颈，但是否从 memory-bound 转为 compute-bound 仍需结合目标硬件和实测 Roofline 判断。

学完本章后，你将能够编写更有效利用 GPU memory hierarchy 与硬件数据传输引擎的 CUDA kernel。

合并与非合并的 global memory access

代码的 memory access pattern会显著影响性能。当 warp 中各 thread 访问相邻地址时，硬件通常能用最少的 memory transaction 覆盖所请求的数据。若地址分散或跨越额外边界，则会访问更多 32-byte sector，取回较多未使用字节并浪费带宽。现代 NVIDIA GPU 的 L2 cache line 与 sector 组织具有架构细节；进行 coalescing 分析时，应关注指令所请求的有效字节、实际访问的 sector 数及 cache 命中层级，而不能把所有访问简单等同为单个 128-byte transaction。

部分 Blackwell GPU 的单 GPU HBM3e 标称带宽可达约 8 TB/s；若把双 GPU 模块两侧的带宽相加，聚合值可写为约 16 TB/s，但这不是任一单 kernel 可访问的统一共享带宽。非合并访问会增加实际传输字节数，使有效带宽显著低于物理带宽上限。

在非合并情形中，warp 内各 thread 从分散地址加载数据，因而触及许多 sector。即使地址连续，若整个 warp 的访问范围没有按 sector 边界良好覆盖，也可能比理想情形多访问 sector；具体数量取决于每 thread 访问宽度、首地址偏移和 cache 层级。

以 32 个 thread 各读取一个 4-byte 值为例，连续且 128-byte 对齐的 128-byte 范围恰好覆盖 4 个 32-byte sector。若首地址只偏移 4 字节，这一连续范围通常会覆盖 5 个 sector，而不是原文所暗示的必然产生两个完整 128-byte 传输；更大的偏移或不同访问宽度会产生不同结果。判断浪费应比较 requested bytes 与实际 sector bytes。

在合并情形中，thread 访问相邻地址，硬件可用较少的 sector request 覆盖这些加载。图 7-1对比了合并与非合并 memory access。

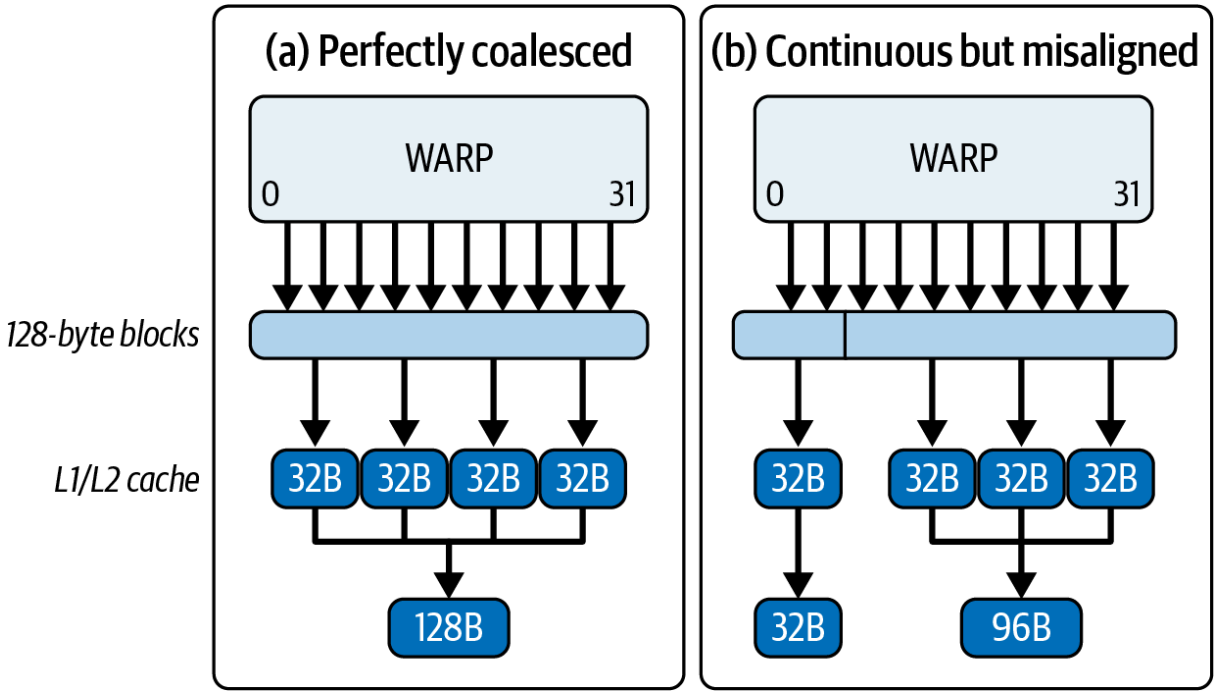


图 7-1。合并与非合并 memory access pattern 对比

在 kernel 代码中，这类问题通常表现为*跨步（strided）*或不规则索引，使 thread 触及不同 sector 乃至不同 cache line。此时一个 warp 的 global-memory instruction 需要访问更多 sector，而不是用最少数量覆盖连续有效数据。

在 Nsight Compute 中，Memory Workload Analysis 可显示 requested/actual throughput、sector 数和每 request 的 sector 数等指标；具体名称随版本变化。对每个 thread 加载 4-byte 的满 warp 指令，理想连续访问常对应 4 个 32-byte sector，但其他向量宽度、active mask 或 replay 情形不能套用固定阈值。低 load efficiency 或相对于 requested bytes 过多的 sector 表明存在带宽浪费；DRAM 吞吐低于峰值本身并不能证明 warp 正在等待 memory，也可能是 cache hit、低并行度、指令依赖或其他瓶颈。

若分析确认 kernel 因非合并访问而浪费带宽，可以重新组织数据，使 warp 的 32 个 thread 尽量访问连续元素。例如，以 `input[idx]` 访问数组，其中 `idx = blockIdx.x * blockDim.x + threadIdx.x`；也可以在按字段处理数据时，将 array of structures（AoS）改为 structure of arrays（SoA），使 thread `i` 访问字段数组中的元素 `i`。图 7-2展示两种布局：AoS 中相邻 thread 读取同一字段时可能形成跨步访问，而 SoA 可让 thread `i` 对应连续字段数组中的元素 `i`。AoS 并非天然低效；若 thread 读取整个紧凑结构，或结构宽度与访问模式合适，它也可能合并良好。

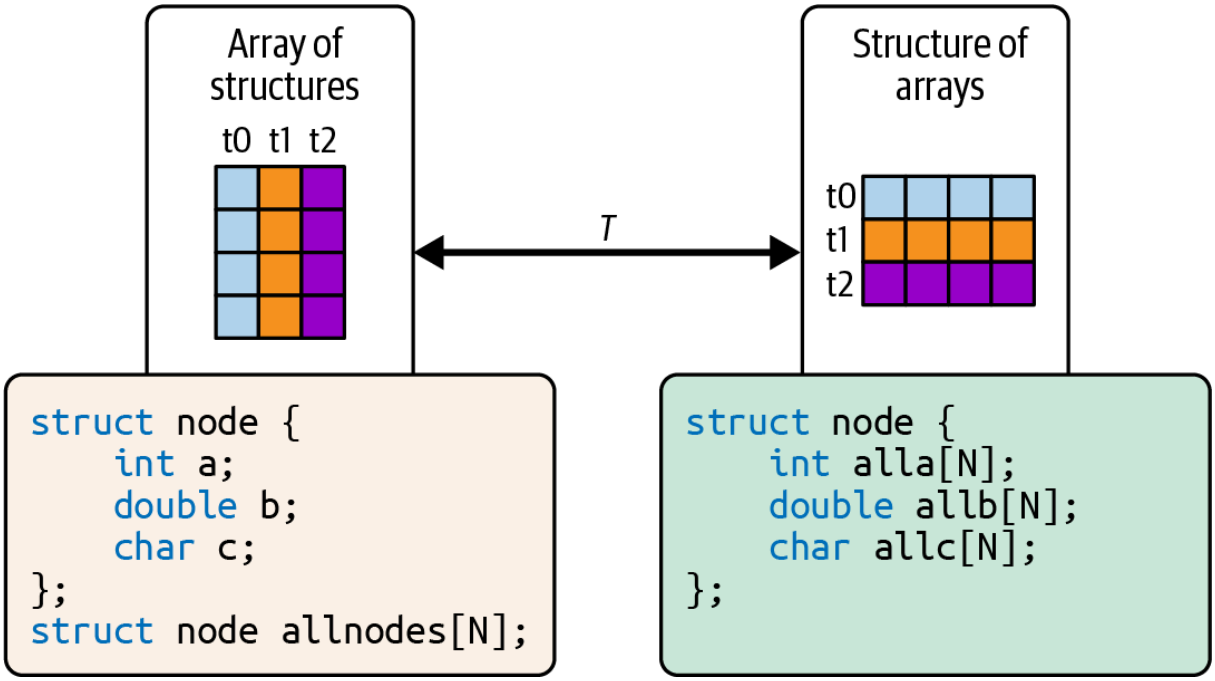


图 7-2。array of structures（AoS）的跨字段访问与 structure of arrays（SoA）的连续字段访问

完成布局调整后，硬件通常能以更少的 sector request 完成 warp 的 global-memory load，从而提高返回数据中的有效字节比例。是否改善以及幅度多大，仍应通过 Nsight Compute 指标和 kernel 时间验证。

下面通过优化前后的示例，说明如何把跨步 global-memory access 改为连续访问。性能增益取决于目标 GPU、cache 行为和测量方法。

*优化前示例（C++）：非合并的跨步访问。*在这个示例中，每个 thread 以 stride 2 从输入数组复制元素，形成连续 thread 之间相隔 8 字节的访问。它是跨步访问，但不是通常所说的“地址未对齐”：

```
#include <cuda_runtime.h>
#include <iostream>

__global__ void uncoalescedCopy(const float* __restrict__ in,
float* __restrict__ out,
int N, int stride) {
    // n = 1048576, stride = 2
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        // Loads from in[] with a stride, causing
        // multiple memory segments to be fetched
        out[idx] = in[idx * stride];
    }
}

int main() {
    const int N = 1 << 20;
    const int stride = 2;

    float* h_in = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_in, N * stride * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));
```

```
    for (int i = 0; i < N * stride; ++i) {
        h_in[i] = static_cast<float>(i);
    }

    float *d_in, *d_out;
    cudaMalloc(&d_in,  N * stride * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_in, h_in, N * stride * sizeof(float),
      cudaMemcpyHostToDevice);

    // Number of threads per block (multiple of 32)
    const int threadsPerBlock = 256;

    // Number of blocks per grid
    const int blocksPerGrid = (N + threadsPerBlock - 1) /
      threadsPerBlock;

    uncoalescedCopy<<<blocksPerGrid,
      threadsPerBlock>>>(d_in, d_out, N, stride);
    cudaDeviceSynchronize();

    cudaFree(d_in);
    cudaFree(d_out);
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);
    return 0;
}
```

该 CUDA C++ kernel 的相邻 thread 从间隔大于 1 个元素的地址执行 global-memory load，因此一个 warp 触及的 sector 多于连续访问。与此同时，输出侧的 out[idx] 写入仍是连续且可合并的；分析时应将读写两条路径分开。

*优化前示例（PyTorch）。*在 PyTorch 中，可以用跨步 index 的 gather 构造类似情形：

```
import torch

def uncoalesced_copy(input_tensor, stride):
    # Flatten to 1D so we know exactly
    #   which dimension we're indexing
    flat_tensor = input_tensor.contiguous().view(-1)

    # Generate indices with a fixed stride to gather
    assert flat_tensor.numel() % stride == 0,"stride must divide tensor length"
    idx = torch.arange(0, flat_tensor.numel(), stride,
        device=flat_tensor.device, dtype=torch.long)
    # index_select uses a gather kernel that issues uncoalesced loads
    return torch.index_select(flat_tensor, 0, idx)

# Usage
n, stride = 1 << 20, 2
inp = torch.arange(n * stride, device='cuda',
                   dtype=torch.float32)
out = uncoalesced_copy(inp, stride)
```

这段 PyTorch 代码使用带跨步 index pattern 的 `torch.index_select`，底层 gather kernel 会按 index 加载数据。对于 FP32 输入，相邻索引对应的地址间隔是 `stride * 4 bytes`；stride 为 2 时，一个满 warp 的请求覆盖约 252 字节范围并触及约 8 个 32-byte sector。

这里仍是一条 warp memory instruction 中的 32 个 lane request，不应描述成必然生成“32 个独立 memory transaction”。每个 thread 从 `inp` 加载一个值，硬件会按所触及的 sector 合并请求；stride 2 通常约为 8 个 sector，而更大 stride 或随机访问才可能逐渐接近每个 lane 一个 sector。图 7-3展示合并、跨步与随机访问模式。

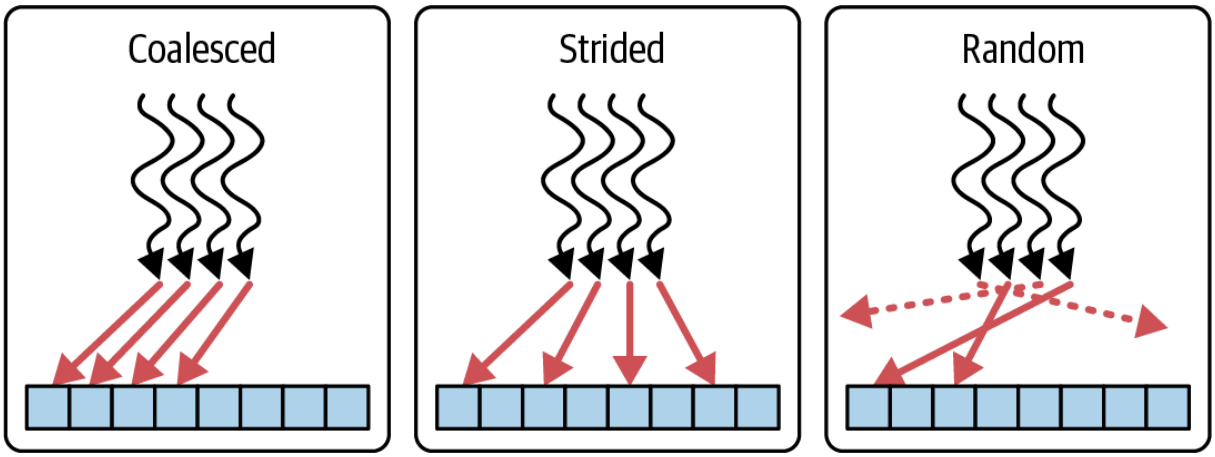


图 7-3。合并、跨步与随机 memory access pattern

原书给出的示意测量中，非合并版本的每个 warp request 平均触及 8 个 32-byte sector。对于 stride 2 的 FP32 满 warp load，这与其覆盖约 256 字节的范围相符；但实际 sector 数还受首地址对齐、active lane、cache 和生成指令影响，不能由“跨两条 128-byte cache line”这一说法单独推导。

相对于连续 FP32 满 warp load 通常触及的 4 个 sector，8 个 sector 意味着约一半加载字节未被该指令使用。表中的 Nsight Compute DRAM throughput 约为 sustained peak 的 25%，只是概念示意；低吞吐并不能单独证明 memory pipeline “饥饿”，需要结合 kernel duration、requested throughput 和 stall 指标解释。

现在把 thread 改为读取连续元素，使每个 warp 用更少的 sector 覆盖相同数量的有效数据。

*优化后示例（C++）：合并访问。*移除 stride（或设为 1）后，每个 thread 复制一个相邻元素，硬件可将 warp request 合并到最少的 32-byte sector：

```
#include <cuda_runtime.h>
#include <iostream>

__global__ void coalescedCopy(const float* __restrict__ in,
                              float* __restrict__ out,
                              int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        // Contiguous load
        // Threads copy neighboring elements
        out[idx] = in[idx];
    }
}

int main() {
    const int n = 1 << 20;
    const size_t bytes = n * sizeof(float);

    // 1) Allocate pinned host memory
```

```
    float *h_in = nullptr, *h_out = nullptr;
    cudaMallocHost(&h_in,  bytes);  // page-locked host alloc
    cudaMallocHost(&h_out, bytes);

    // 2) Initialize input
    for (int i = 0; i < n; ++i) {
        h_in[i] = static_cast<float>(i);
    }

    // 3) Allocate device memory
    float *d_in = nullptr, *d_out = nullptr;
    cudaMalloc(&d_in,  bytes);
    cudaMalloc(&d_out, bytes);

    // 4) Copy to device
    cudaMemcpy(d_in, h_in, bytes,
      cudaMemcpyHostToDevice);

    // 5) Launch kernel
    dim3 block(256);
    dim3 grid((n + block.x - 1) / block.x);
    coalescedCopy<<<grid, block>>>(d_in, d_out, n);
    cudaDeviceSynchronize();

    // 6) Copy back to host
    cudaMemcpy(h_out, d_out, bytes,
      cudaMemcpyDeviceToHost);

    // 7) Clean up
    cudaFree(d_in);
    cudaFree(d_out);
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);

    return 0;
}
```

这里省略合并访问的 PyTorch 实现。对于 contiguous tensor，可写成 `out = inp.clone()` 来获得内容相同的新 tensor。其底层 copy kernel 通常会选择适合输入布局、dtype 与对齐条件的实现，但不能把所有版本和形状下的 `clone()` 都等同于固定的 vectorized memory copy。

> 还可以使用采用默认 TorchInductor backend 的 `torch.compile`，在语义允许时消除冗余 copy 或融合相邻 operation。是否保留、消除或融合 `clone()` 取决于 aliasing、mutation、graph capture 和编译器版本，不能仅因它是 device-to-device copy 就断定其行为。对稳定 shape，可测试 autotuning（如 `mode="max-autotune"`），但它会增加编译成本，也不保证所有算子采用特定的合并或向量化 schedule。第 13 和第 14 章将深入介绍 PyTorch compiler。

采用连续访问后，warp 内各 thread 访问相邻地址。对于 32 个 active thread 各读取一个 FP32 值的特定示例，128-byte 对齐范围通常覆盖 4 个 32-byte sector；跨 sector 边界时可能多一个 sector。表 7-1给出原书用于说明优化方向的一组示意指标。

表 7-1。合并与非合并 memory access 的示意性能对比

指标	优化前（非合并）	优化后（合并）
DRAM throughput（峰值百分比）	25%	90% (3.6×)
全局内存加载效率	23%	99%
每个 request 的平均 sector 数	8.0	4.0（最优）
SM 活跃百分比	62%	99%
Kernel 执行时间（ms）	4.8 ms	1.3 ms (3.7×)

> 注意：所有指标表中的数值都只是用于解释概念的示意值，并非由本页代码可复现地推导出的通用 benchmark 结果。不同 GPU architecture 上的实际结果请参阅该 GitHub repository并核对相应硬件、软件版本与测量方法。

在这组示意数据中，调整 data layout 并合并 global-memory access 后，DRAM throughput 从峰值的 25% 上升到 90%，约为原来的 3.6 倍；kernel 执行时间从 4.8 ms 降至 1.3 ms，约改善 3.7 倍。这些数值只说明可能的趋势，不能作为该代码在任意 Blackwell GPU 上的预期结果。

示意的 Global Memory Load Efficiency 从 23% 上升到 99%，表示实际传输字节中更大比例是所请求的数据；同时 Average Sectors per Request 降至约 4.0。

> 只有在本例的访问宽度和 active mask 下，接近 4.0 才对应满 warp 各加载一个 FP32 值的理想连续访问：32 个 4-byte 值共 128 字节，覆盖 4 个 32-byte sector。对 FP64、vector load、部分 warp 或其他指令，理想 sector/request 值会不同。

对于满 warp 的 scalar load，极端分散访问可能接近 32 个 sector，因为每个 active lane 都可能落在不同的 32-byte sector。不能一概认为“高于 4.0 就是错误”，而应先根据每 lane 字节数和 active lane 数计算理想值，再比较 requested bytes 与 sector bytes。

本例约 4.0 sector/request，与连续 FP32 满 warp load 的理想覆盖一致。示意表中 SM Active 从 62% 升至 99%，但 SM Active 只表示至少有一个 warp active 的周期比例或相应版本定义，并不直接等同于计算单元利用率；应结合 duration 与其他 throughput/stall 指标判断。

总之，让相邻 thread 访问相邻地址，可以减少完成同一有效 load 所需的 sector 数，提高 Global Memory Load Efficiency。它可能改善 kernel 时间和 SM 活跃度，但结果取决于 cache、occupancy、指令组合及其他瓶颈。

至此可以看到，coalescing 优化的是一个 warp memory instruction 中 lane address 到 memory sector 的映射。32 个 thread 各自请求一个 4-byte 值，并不意味着硬件先执行 32 个独立 DRAM read 再把它们“拼接”为 128-byte transaction；coalescer 会依据地址直接生成所需 sector request。

下一节将讨论 vectorized memory access：让每个 thread 用一条更宽的 load/store instruction 获取对齐数据，例如 16-byte 的 `float4`。向量化主要减少动态指令数和地址计算开销；一个 warp 搬运的数据量也随每 thread 宽度增加，因此不能把它描述为把同样的数据从“32 个 transaction”固定变成“四个 128-byte transaction”。下面将介绍如何使用 CUDA 内置 vector type，并分析其对齐与边界条件。

向量化 memory access

Memory coalescing 是 NVIDIA GPU 在运行时根据 lane address 完成的硬件行为；vectorized memory access 则是在编译时让一条 load/store instruction 为每个 thread 显式搬运多个连续元素，例如使用 `float4` 搬运 16 字节。它主要减少完成同等数据搬运所需的动态指令数和地址计算，不存在需要额外消除的“拼接开销”。

高效的 global-memory access要求 warp 的 lane address 尽量覆盖连续、对齐的 sector。32 个 thread 各读取一个 4-byte float 时，coalescer 会直接生成覆盖这 128 字节所需的 sector request，并非先发出 32 个独立 DRAM request 再拼成一条 cache line。

在理想合并情形下，一个 warp 读取 32 个 4-byte word，并且访问范围按 128 字节对齐时，恰好覆盖 4 个 32-byte sector。Nsight Compute 的 sector 与 requested/actual byte 指标可揭示额外流量；对本访问宽度而言，未对齐、跨步或分散访问可能使 sector/request 高于 4.0，并降低 Global Memory Load Efficiency。其他 per-thread 宽度对应不同的理想 sector 数。

若 kernel 对同一连续数组逐元素执行多条 scalar load/store，可尝试让每个 thread 处理一个更宽且自然对齐的 vector。CUDA 内置 `float4` 将 4 个 4-byte float 组成一个大小与对齐要求均为 16 字节的类型。下面给出简化示意：

```
// Example of a custom float4 for illustration purposes.
// Note: it's recommended to use the built-in CUDA float4 type.
// Note: if N is not divisible by 4, handle the last N % 4 elements
// (e.g., a short scalar cleanup) or assert divisibility in host code. The
// input and output pointers must be 16-byte aligned for float4 loads/stores.
struct my_float4 {
    float x;  // 4 bytes
    float y;  // 4 bytes
    float z;  // 4 bytes
    float w;  // 4 bytes
};
```

当 warp 的 32 个 thread 各执行一次 `float4` load 时，共请求 $32 \times 16 = 512$ 字节。若 lane address 连续且整体对齐，该范围覆盖 16 个 32-byte sector，也可按 4 个连续的 128-byte 范围理解。这里应报告每条 warp instruction 约 16 sector，而不是沿用 scalar FP32 示例的 4 sector/request；“每个 128-byte 范围含 4 个 sector”与 profiler 的“每条 request 含多少 sector”不是同一个分母。

若原代码每个 thread 用 4 条 scalar instruction 搬运 4 个相邻 float，改用一次 `float4` load 可把这部分 load instruction 数降至四分之一。Coalescing efficiency 仍由所有 lane 的地址决定；向量化本身不会把理想 sector/request 固定为 4.0。

> PyTorch compiler 等高层 GPU compiler在能够证明 alignment、contiguity、边界与 aliasing 条件时，可能生成 vectorized memory operation。与 vector width 匹配的 per-thread tile 有助于编译器选择这种 schedule，但实际代码生成必须检查 IR/PTX/SASS 或 profiler，不能假定 framework 一定减少 memory transaction。

如果原访问既未合并又能通过重新分组而改善，vectorization 与 layout 调整可能同时提高 load efficiency；但对本来已经连续合并的 scalar copy，向量化通常不会把 sector efficiency 从低值自动提升到接近 100%。更宽的 warp request 会访问相应更多的 sector。

Vectorized load 可减少每字节对应的 memory instruction 数，并可能提高有效带宽。指针必须满足所用 vector type/instruction 的 alignment 要求。CUDA Programming Guide 说明 runtime/driver allocation（如 `cudaMalloc`）返回的基址至少按 256 字节对齐，但切片偏移和自定义 allocator 仍可能破坏后续地址的对齐。

该基址足以满足 allocation 起点处 16-byte 或 32-byte 的对齐，但加上 element offset 后，只有字节偏移是 vector alignment 的整数倍时仍保持对齐。下面的 cast 只是向 compiler 表达预期类型，并不会在运行时修复或验证指针：

```
auto ptr4 = reinterpret_cast<const float4*>(ptr);
```

cast 前应确认指针值按 16 字节（4 个 `float`）对齐。虽然 `cudaMalloc()` 返回至少 256-byte aligned 的基址，但若 element offset 不是 4 个 `float` 的倍数，地址便不再满足 float4 alignment。尾部不足一个 vector 的元素还需要单独处理；否则示例中的 N/4 thread 会漏掉余数。

若 base address 与每 thread stride 都按 32 字节对齐，每个 thread 每轮读取 32 字节，则满 warp 共请求 1,024 字节，对应 32 个 32-byte sector，也可分为 8 个连续的 128-byte 范围。源码最终编译为两条 16-byte instruction 还是一条 32-byte instruction，取决于目标 compute capability、PTX/SASS instruction、数据类型和 compiler 证明；不能只由“Blackwell”这一名称保证。正确对齐可避免单个 lane vector 跨越额外边界。

截至原书写作时，CUDA的传统内置 vector type 中没有由 8 个 `float32` 组成的通用 `float8`。原文件名 `<vector_types.h>` 中还混入了零宽字符，译文已清除。可以定义含 8 个 `float` 的 struct，并以两个 `float4` load 显式搬运；也可用 `alignas(32)` 表达 32-byte type alignment。`alignas` 只约束合法对象地址，是否生成单条 32-byte global load 必须查看目标 Blackwell 的实际 SASS，不能由 cast 或 alignment 单独保证。Hopper 与 Blackwell上的代码生成也会随 Toolkit 与编译选项变化。

每 thread 读取 32 字节（8 个 float × 4 byte）时，满 warp 搬运 1,024 字节，即 32 个 sector 或 8 个 128-byte 范围。每 thread 读取 16 字节（一个 `float4`）时，满 warp 搬运 512 字节，即 16 个 sector 或 4 个 128-byte 范围。base pointer 与 per-thread stride 的自然对齐有助于避免 split sector。若确实使用要求 32-byte alignment 的 256-bit

type/instruction，就必须满足该对齐；但“对齐”是正确且高效代码的必要条件之一，不等于必然达到 peak throughput。

下面用一个简单 vector copy kernel 对比 scalar 与 vectorized memory access。需要注意，两版代码都采用连续索引，因此 scalar 版已经具备良好的 coalescing。

*优化前示例（C++）：scalar copy。*每个 thread 复制一个 float：

```cpp
#include <cuda_runtime.h>

__global__ void copyScalar(
const float* __restrict__ in,
float* __restrict__ out, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        // Scalar load: 4-byte copy per thread
        out[idx] = in[idx];
    }
}

int main() {
    const int N = 1 << 20;

    float* h_in = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_in, N * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));

    for (int i = 0; i < N; ++i) h_in[i] = float(i);

    float *d_in, *d_out;
    cudaMalloc(&d_in, N * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice);

    dim3 block(256), grid((N + 255) / 256);
    copyScalar<<<grid, block>>>(d_in, d_out, N);
    cudaDeviceSynchronize();

    cudaFree(d_in); cudaFree(d_out);
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);
    return 0;
}
```

*优化前示例（PyTorch）。*为了说明逐元素操作，可以用 Python loop 编写 scalar copy；但这主要测到大量 Python dispatch 和小 kernel launch 开销，不能作为 CUDA scalar memory instruction 的公平基准：

```python
import torch

def copy_scalar(inp: torch.Tensor) -> torch.Tensor:
    out = torch.empty_like(inp)
    flat_in = inp.view(-1)
    flat_out = out.view(-1)
    for i in range(flat_in.numel()):
        # Each iteration issues a 4-byte load on the GPU
        # This is extremely slow. DO NOT DO THIS!
        # Use vectorized operations to avoid Python loops
```

```
        # on GPU tensors as shown in optimized version
        flat_out[i] = flat_in[i]
    return out

# Usage
N = 1 << 20
inp = torch.arange(N, device='cuda', dtype=torch.float32)
out = copy_scalar(inp)
```

对于连续、对齐的 4-byte scalar load，满 warp 的一条 instruction 请求 128 字节并覆盖 4 个 sector；轻微错位通常会多触及一个 sector。使用 `float4` 后，每条 warp instruction 搬运的数据增至 512 字节并覆盖 16 个 sector，但每个 thread 可用一条 vector instruction 代替 4 条 scalar instruction 来完成相同的总元素数。下面改用 vector load。

*优化后示例（C++）：vectorized copy。*每个 thread 复制一个 `float4`（16 字节）：

```
#include <cuda_runtime.h>

// 16-byte (128-bit) vector copy: one float4 per thread
static_assert(alignof(float4) == 16, "float4 alignment must be 16 bytes");
__global__ void copyVector16B(const float4* __restrict__ in,
                              float4* __restrict__ out,
                              int N4)  // number of float4 elements
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N4) {
        // Per-thread 16B load+store.
        // On sm_90, NVCC emits ld.global.v4.f32 / st.global.v4.f32.
        // for 16-byte vector loads and stores
        // On sm_90 (Hopper) (Blackwell), NVCC emits
        // ld.global.v8.f32 and st.global.v8.f32 for 32-byte aligned data
        out[idx] = in[idx];
    }
}

int main() {
    const int N  = 1 << 20;   // total number of floats
    const int N4 = N / 4;     // total number of float4s (16B chunks)

    float4 *d_in = nullptr, *d_out = nullptr;
    cudaMalloc(&d_in,  N4 * sizeof(float4));   // >=256B aligned
    cudaMalloc(&d_out, N4 * sizeof(float4));

    dim3 block(256);
    dim3 grid((N4 + block.x - 1) / block.x);
    copyVector16B<<<grid, block>>>(d_in, d_out, N4);
    cudaDeviceSynchronize();

    cudaFree(d_in);
    cudaFree(d_out);
    return 0;
}
```

这里使用 `<cuda_runtime.h>` 中的 CUDA 内置 `float4` type，并启动 N/4 个 thread，使每个 thread 复制一个 `float4`（16 字节），即 4 个 `float`；满 warp 共处理 128 个 float，覆盖 16 个 sector。`cudaMalloc` 返回至少 256-byte aligned 的基址，满足 `float4` 要求；但 N 必须能被 4 整除或另行处理 tail，指针偏移也必须保持对齐。misaligned reinterpret

cast 会产生未定义或不受支持的访问，而不仅仅是“失去 vectorization”。下面的代码尝试展示 32-byte aggregate 路径，但是否生成单条 Blackwell 32-byte instruction 仍需检查编译结果：

```cpp
// --- Blackwell-only variant: 32-byte per-thread vector copy
// (PTX: ld.global.v8.f32)
// Requires 32B alignment.
#include <cuda_runtime.h>

// Proof of 32-byte alignment
static_assert(alignof(float8) == 32, "float8 alignment must be 32 bytes");
struct alignas(32) float8 { float v[8]; };

__global__ void copyVector32B(const float8* __restrict__ in,
                              float8* __restrict__ out, int N8) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N8) {
        // Per-thread 32B load+store
        // NVCC emits ld.global.v8.f32/st.global.v8.f32 on sm_100
        out[idx] = in[idx];
    }
}

int main() {
    const int N = 1 << 20;
    const int N8 = N / 8;

    float8 *d_in, *d_out;
    // cudaMalloc returns pointers aligned to ≥256 bytes
    // This will allow the compiler to emit
    // optimized ld.global.v8.f32 and st.global.v8.f32 instructions
    cudaMalloc(&d_in,  N8 * sizeof(float8));
    cudaMalloc(&d_out, N8 * sizeof(float8));

    dim3 block(256), grid((N8 + block.x - 1) / block.x);
    copyVector32B<<<grid, block>>>(d_in, d_out, N8);
    cudaDeviceSynchronize();

    cudaFree(d_in); cudaFree(d_out);
    return 0;
}
```

优化后示例（PyTorch）。可以用 tensor view 与 clone 表达按 4 个元素分组后的连续 copy：

```python
import torch

def copy_vectorized(inp: torch.Tensor) -> torch.Tensor:
    # Reshape into groups of 4 floats for bulk copy
    vec = inp.view(-1, 4)
    # clone() on a contiguous CUDA tensor performs a device-to-device copy
    # using optimized runtime paths such as cudaMemcpyAsync().

    out_vec = vec.clone()
    return out_vec.view(-1)

# Usage
N = 1 << 20
inp = torch.arange(N, device='cuda', dtype=torch.float32)
out = copy_vectorized(inp)
```

对 reshaped tensor 调用 `clone()` 会执行连续 copy，而前面的 Python loop 会启动大量逐元素 operation。`view(-1, 4)` 本身只是改变 shape metadata，不会要求 PyTorch copy kernel 按 `float4` 发射，也不会改变底层 contiguous storage；是否向量化由实现和编译结果决定。因此该 PyTorch 对照主要体现批量 operation 与 Python loop 的差异，不能用于量化 CUDA scalar 与 vector instruction 的收益。表 7-2保留原书示意数据，但不应视为这些代码的可复现结果。

> 原文将 Blackwell/CUDA 13 概括为把每 thread vector load/store 从 Hopper 的 16 字节翻倍到 32 字节。更准确地说，特定目标架构、类型和指令路径可能支持 32-byte operation，但源码是否生成单条相应 SASS instruction 取决于 compiler 能否证明 alignment 与其他合法性条件；否则可能拆成较窄 instruction。指令数增加可能带来开销，但在 memory-bandwidth-bound copy 中未必按比例降低性能。

使用正确对齐的 `float4` vector load 时，每个 thread 每条 instruction 搬运 16 字节。它可能减少 instruction overhead，但原 scalar C++ kernel 已是连续合并访问，因此不能据此断言 memory bandwidth utilization 必然出现表 7-2所示幅度的改善。

表 7-2。Scalar 与 vectorized memory access 的 Nsight Compute 示意指标

指标	优化前（scalar）	优化后（vectorized）
全局内存加载效率	28%	97%
每个 request 的平均 sector 数	31.8	4.0
DRAM throughput（峰值百分比）	25%	90% (3.6×)
Kernel 执行时间	4.2 ms	1.2 ms（改善 3.5 倍）

按原书表中数值，Global Memory Load Efficiency 从 28% 升至 97%，DRAM throughput 从峰值的 25% 升至 90%，kernel runtime 约改善 3.5 倍。然而，所列 scalar C++ kernel 对连续数组执行合并 copy，正常不应产生 31.8 sector/request 或仅 28% load efficiency；因此这些指标不能“证实”该 scalar/vector code 对照，可能来自不同访问模式或仅为教学构造。

表中 runtime 从 4.2 ms 降至 1.2 ms，同样缺少 GPU 型号、clock、warmup、iteration、计时范围与编译产物，无法复现。Global Memory Load Efficiency 的含义是 requested bytes 相对于实际传输 bytes 的效率，不应表述为“几乎每个 128-byte transaction 都被使用”。

表中的 average sectors/request 从 31.8 降至 4.0 也与本例的 request 宽度不相容：满 warp 的 `float4` instruction 请求 512 字节，理想情况下覆盖 16 个 sector，而非 4 个。若 metric 按不同 instruction 或不同分组统计，必须给出准确 metric 名称与 source counter 后才能比较。

在合法且对齐的场景中，以一条 vector load 替代 4 条 scalar load 可以减少 instruction 数；若 scalar 访问原本分散，配套的 layout 改动还可提高 sector efficiency。但对已经合并的 copy，load efficiency 通常不会仅因向量化从 28% 跳到 97%，实际收益应以同等数据量、同一 kernel 语义和同一测量方法验证。

> Vectorization 主要减少 instruction count；coalescing 与 alignment 主要减少完成给定 requested bytes 所需的额外 sector。两类效果应分别测量。

下面对比上一节的 coalesced global-memory access 与本节的 vectorized memory access。Coalescing 关注 warp 内各 lane 的地址能否由尽量少的 sector 覆盖；vectorization 关注每个 thread 能否用较少的宽 instruction 搬运多个相邻元素。表 7-3 概括两者差异。

表 7-3。Coalesced memory access 与 vectorized memory access 对比

方面	Coalesced warp load（thread 间）	Vectorization（thread 内）
粒度	thread ↔ 连续 memory address	每个 thread ↔ vector load
典型方法	连续索引（stride = 1）	使用 `float4`、显式要求 32-byte alignment 的自定义 struct，或目标 toolchain 支持的 32-byte-aligned vector type
每 warp 数据覆盖	32 thread × 4 byte → 128 byte；对齐时覆盖 4 个 32-byte sector	32 thread × 16 byte → 512 byte；对齐时覆盖 16 个 32-byte sector，即 4 个连续 128-byte 范围
Instruction count 变化	只调整 lane address 时，load instruction 数通常不变	同等 per-thread 数据量下，以 1 条 vector load 替代 4 条 scalar load
收益	减少分散或浪费的 sector	减少 instruction 与地址计算；不一定减少同等数据量的 sector

结合这两种策略，可以让 warp 访问连续 sector，同时让每个 thread 用合适的宽 instruction 搬运多个元素。这有机会减少 sector waste 与 instruction overhead，但能否接近 memory bandwidth peak 还取决于 occupancy、并发 request 数、cache、ECC、clock 和 kernel 其他工作。

原文称 Blackwell 与 CUDA 13 之前每 thread global-memory vector load 一律限于 16 字节，并称两者新增通用 32-byte load/store instruction 与 type。实际可用宽度需按

compute capability、PTX ISA、SASS 与 CUDA header 中的具体 type 核对；CUDA Toolkit 版本本身不能让旧架构获得新硬件指令。

若目标架构与 toolchain 确实提供可用的 32-byte type/instruction，并且 profile 显示 instruction overhead 有意义，可以测试其用于 8-float aggregate；同时必须满足 alignment、tail 和 aliasing 条件。

普通自定义 8-float aggregate 可能被 compiler 拆为多条较窄 load。仅添加 32-byte alignment 也不保证生成单条 32-byte instruction，应检查 PTX/SASS，并以执行时间验证。

> 无论 per-thread instruction 宽度如何，coalescing 都应按 lane request 覆盖的 32-byte sector 分析。32 个 thread 各搬运 32 B 时总量为 1,024 B，对齐连续访问覆盖 32 个 sector；各搬运 16 B 时总量为 512 B，覆盖 16 个 sector。这里的算术与 GPU 型号无关，区别在于目标架构是否能用所期望的 instruction 数实现这些 per-thread access。正确对齐可达到 sector efficiency，但不自动保证整体性能最优。

使用 shared memory 进行 tiling 与数据复用

常见性能问题之一是反复从 global memory 读取相同数据。*Tiling* 将数据分块加载到更快的片上 shared memory，并让 block 内多个 thread 在同步与生命周期允许的范围内复用这些 tile。

例如，在朴素的 $N \times N$ matrix multiplication 中，每个 output element 都沿 K 维读取一行 A 与一列 B。若不考虑 cache，A[i,k] 会被同一输出行的 N 个列结果重复使用，B[k,j] 会被同一输出列的 N 个行结果重复使用，因此每个输入元素在整个计算中最多有 N 次逻辑读取，而不是原文所说“A 的每个元素对应 B 的每一行”。Blackwell 具有很高的计算吞吐，若复用无法被 cache 捕获，冗余内存流量会限制 SM 获得操作数的速度。

Tiling 让每个 thread block 将 A 与 B 的小型子矩阵（*tile*）协作加载到 shared memory，再由多个 thread 为多次 multiply-accumulate 复用。这里的“只加载一次”是指每个 block、每个 K tile 内的逻辑加载；同一输入 tile 仍可能被计算不同 output tile 的多个 block 分别加载。下面使用 32 × 32 tile 说明。

block 内的 thread 可以协作将 tile 载入 shared memory，再调用 `__syncthreads()`，确保所有参与 thread 完成写入后才开始读取。随后各 thread 使用 shared-memory 数据并行计算，使一次 global load 服务多次运算。示例按连续列安排 tile load，因此每个满 warp 各读取一行 32 个 FP32 值，覆盖连续 128 字节和 4 个 32-byte sector。

在每个 block 的 tile iteration 中，把所需元素从 DRAM载入 shared memory 一次并多次复用，可以减少相对于朴素实现的 global-memory traffic。下面以 $N \times N$ matrix multiplication 为例，先看朴素实现。

*优化前示例（CUDA C++）：朴素 matrix multiplication。*每个 thread 计算结果矩阵 C 的一个元素，并为该 output 沿 K 维读取 A 的一行和 B 的一列：

```
#include <cuda_runtime.h>
#include <iostream>
```

```cpp
__global__ void naiveMatMul(const float* A, const float* B, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; ++k) {
            // Each thread loads A[row, k] and B[k, col]
            // from global memory for every k.
            // This is very memory heavy.
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

int main() {
    const int N = 1024;
    size_t bytes = N * N * sizeof(float);

    float* h_A = nullptr;
    float* h_B = nullptr;
    float* h_C = nullptr;
    cudaMallocHost(&h_A, N*N * sizeof(float));
    cudaMallocHost(&h_B, N*N * sizeof(float));
    cudaMallocHost(&h_C, N*N * sizeof(float));

    for (int i = 0; i < N*N; ++i) { h_A[i] = 1.0f; h_B[i] = 1.0f; }

    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);
    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);

    dim3 block(32, 32);
    dim3 grid((N + 31) / 32, (N + 31) / 32);
    naiveMatMul<<<grid, block>>>(d_A, d_B, d_C, N);
    cudaDeviceSynchronize();

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    cudaFreeHost(h_A);
    cudaFreeHost(h_B);
    cudaFreeHost(h_C);
    return 0;
}
```

这个 CUDA C++ kernel 在内层循环的每次乘法中发出 global-memory load。每个 thread 读取 `A[row, k]` 与 `B[k, col]`，并遍历每个 k。相邻 thread 在同一 k 上读取相同的 A 值，硬件可通过 warp broadcast/cache 服务部分复用；它们读取的 B 值则沿 col 连续并可合并。尽管如此，显式 shared-memory tiling 通常能提供更可控的跨 thread 复用。是否为 memory-bound、SM utilization 多低，必须通过 profile 判定。下面是朴素 PyTorch 版本：

```python
import torch

def naive_matmul(A, B):
    N = A.size(0)
```

```
    C = torch.zeros((N, N), device='cuda')
    for i in range(N):
        for j in range(N):
            # Each dot product loads A[i,:] B[:,j] from global memory repeatedly
            C[i, j] = (A[i, :] * B[:, j]).sum()
    return C

# Usage
N = 1024
A = torch.ones((N, N), device='cuda', dtype=torch.float32)
B = torch.ones((N, N), device='cuda', dtype=torch.float32)
C = naive_matmul(A, B)
```

这个 PyTorch 实现使用嵌套 Python loop。每个 output element 都执行一次 tensor slice、elementwise multiply 与 reduction，产生大量 Python dispatch 和 GPU kernel launch；其慢速主要不只来自重复 global load，也来自极细粒度 operation 与同步/调度开销。因此它不能忠实模拟单个朴素 CUDA kernel 的 memory-bound 行为。

> 这段 PyTorch 代码故意采用极慢的逐 output Python loop，用于说明缺少批量化的反例，不适合作为 tiling benchmark。实际代码应使用 PyTorch 已优化的 matrix-multiplication operator。

接下来应用 tiling，将矩阵划分为 32 × 32 tile。该示例用 32 × 32 thread block，每个 warp 的 32 个 thread 协作加载 tile 的一行。两个 FP32 input tile 共占 2 × 32 × 32 × 4 = 8 KB shared memory；但 1,024-thread block 已达到每 block thread 上限，未必是所有架构或 GEMM 的最佳 tile/thread mapping。

每个 thread block 在一次 K-tile iteration 中将一个 32 × 32 A tile 和一个 32 × 32 B tile 载入 shared memory，并让每个 thread 为一个 output 累加 32 次乘加。同一 block 内，每个载入值最多被 32 个 output 复用；相比不利用 cache 的逻辑读取次数，这可将该层级的输入流量降低约 32 倍，但 HBM 实际流量还受 L1/L2 cache 与跨 block 重复加载影响。下面是使用 shared memory 缓存 tile 的版本：

```
#include <cuda_runtime.h>
#include <iostream>
#define TILE_SIZE 32

__global__ void tiledMatMul(const float* A, const float* B, float* C, int N) {
    __shared__ float sA[TILE_SIZE][TILE_SIZE];
    __shared__ float sB[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    float sum = 0.0f;

    // compute partial results using the tile
    // in shared memory
    for (int t = 0; t < N; t += TILE_SIZE) {
        // Cooperative load of a tile of A and B into shared memory
        // Load tile A with boundary check
        if (row < N && (t + threadIdx.x) < N) {
            sA[threadIdx.y][threadIdx.x] = A[row * N + t + threadIdx.x];
        } else {
            sA[threadIdx.y][threadIdx.x] = 0.0f;
```

```
        }

        // Load tile B with boundary check
        if ((t + threadIdx.y) < N && col < N) {
            sB[threadIdx.y][threadIdx.x] = B[(t + threadIdx.y) * N + col];
        } else {
            sB[threadIdx.y][threadIdx.x] = 0.0f;
        }

        // We will optimize this later to use a
        // thread-block-scoped cooperative-groups barrier
        __syncthreads();

        // Compute using the tile loaded in shared memory
        for (int k = 0; k < TILE_SIZE; ++k) {
            sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
        }
        // We will optimize this later to use a
        // thread-block-scoped cooperative-groups barrier
        __syncthreads();
    }

    if (row < N && col < N) {
        C[row * N + col] = sum;
    }
}

int main() {
    const int N = 1024;
    size_t bytes = N * N * sizeof(float);

    float* h_A = nullptr;
    float* h_B = nullptr;
    float* h_C = nullptr;
    cudaMallocHost(&h_A, N*N * sizeof(float));
    cudaMallocHost(&h_B, N*N * sizeof(float));
    cudaMallocHost(&h_C, N*N * sizeof(float));

    for (int i = 0; i < N*N; ++i) { h_A[i] = 1.0f; h_B[i] = 1.0f; }

    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);
    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);

    dim3 block(TILE_SIZE, TILE_SIZE);
    dim3 grid((N + TILE_SIZE - 1) / TILE_SIZE, (N + TILE_SIZE - 1) / TILE_SIZE);
    tiledMatMul<<<grid, block>>>(d_A, d_B, d_C, N);

    // synchronize the kernel with the device
    // for timing accuracy
    cudaDeviceSynchronize();

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    cudaFreeHost(h_A);
    cudaFreeHost(h_B);
    cudaFreeHost(h_C);

    return 0;
}
```

在这个 tiled kernel 中，每个 block 协作把 A 的 32 × 32 tile 载入 `sA`，把 B 的 tile 载入 `sB`。加载位于 K-tile loop 开头，随后以 `__syncthreads()` 确保 tile 对 block 内所有 thread 可见。该示例假设 N 是 32 的倍数；否则最后一个 tile 的全局读取会越界，必须增加边界检查与 zero fill。

随后 block 使用 shared-memory tile 计算 32 × 32 个 output，每个 thread 的内层循环遍历 tile 内 32 个 `k`。在 block 范围内，每个 A 值被 32 个不同列 output 使用，每个 B 值被 32 个不同行 output 使用，因此相对于无 cache 的逻辑访问可获得最高约 32 倍复用；这不等于所有元素的实测 DRAM read 必然精确减少 32 倍。

完成全部 K tile 后，每个 thread 将累加结果写入 `C`。原文随后声称“每 thread global-memory access 减少 8 倍”，但没有给出与 tile size、N 或 load 计数一致的推导；这里应直接按每 block input load 与每 output FLOP 计算，或使用 profiler 的 sector/byte counter 实测。

> 这些示例关注 memory access pattern，使用普通 FP32 CUDA core instruction，而非低精度 Tensor Core path。第 9 章将介绍 16-bit、8-bit、4-bit 等低精度计算；其性能收益还取决于数据类型、精度要求和是否调用相应 Tensor Core instruction。

下面的 PyTorch 版本在 Python 中手动遍历 output tile，并对每对 K tile 调用 `torch.mm`。CUDA 上的 `torch.mm` 通常通过 ATen 调度到 cuBLAS/cuBLASLt 或其他选定 backend；不能笼统说它同时“使用 cuBLAS 和 CUTLASS”。每次小 tile 调用本身已经执行内部 tiling，并且 Python loop 会引入大量 launch 与临时 tensor 累加开销，因此该示例用于表达 block GEMM 数学分解，不代表高性能实现：

```python
import torch

def tiled_matmul(A, B, tile_size=32):
    N = A.size(0)
    C = torch.zeros((N, N), device='cuda')
    for i in range(0, N, tile_size):
        for j in range(0, N, tile_size):
            C_block = torch.zeros((tile_size, tile_size), device='cuda')
            for k in range(0, N, tile_size):
                A_block = A[i:i+tile_size, k:k+tile_size]
                B_block = B[k:k+tile_size, j:j+tile_size]
                # torch.mm uses an optimized kernel (likely tiling internally)
                C_block += torch.mm(A_block, B_block)
            C[i:i+tile_size, j:j+tile_size] = C_block
    return C

# Usage
N = 1024
A = torch.ones((N, N), device='cuda', dtype=torch.float32)
B = torch.ones((N, N), device='cuda', dtype=torch.float32)
C = tiled_matmul(A, B)
```

原文称 32 × 32 tile 会因 warp 内 thread “访问同一列”而发生 shared-memory bank conflict，但本 CUDA kernel 的计算访问是 sA[ty][k] 与 sB[k][tx]：前者对同一 warp 是同地址 broadcast，后者随 tx 访问连续 bank，通常不会产生该断言所述的 32-way conflict。Padding 到 33 列常用于 naive transpose 等列访问模式，是否需要应按实际 lane address 与 bank width 分析。下一节将详细讨论 bank conflict。

Tiling 通过在 shared memory 中复用数据，可减少 DRAM traffic 并提高相对于片外流量的算术强度。不过 shared-memory allocation、register 使用和 block size 可能使 occupancy 上升、下降或不变，不能由“global transaction 更少”推出 achieved occupancy 更高。表 7-4将在下一批结合其示意指标继续分析。

表 7-4。使用 shared-memory tiling 后的示意性能变化

指标	优化前（naive kernel）	优化后（tiled kernel）	说明
DRAM throughput（峰值百分比）	90%	25%（为原来的约 1/3.6）	示意上 naive 版本传输了更多无效或重复数据；tiled 版本因复用而减少 DRAM traffic。吞吐百分比下降是否表示优化，必须同时确认工作量与执行时间。
达到的占用率（%）	42%	89%	原书将其解释为更多 resident warp 推进执行；但 achieved occupancy 与 stall/progress 不是同一指标，且该数值缺少复现配置。
浮点吞吐（GFLOP/s）	15 GFLOP/s	170 GFLOP/s	原书称矩阵较小且 memory-limited；复用可提高持续计算吞吐。（绝对值还取决于矩阵大小 N = 1,024、计时、clock、编译与 GPU 型号。）
Global-memory load sector 数	9800	1200	这是原书给出的 32-byte sector read 示意总数。Tiling 可减少 block 内重复逻辑加载，但 9,800 与 1,200 并非由所列 N = 1,024 GEMM 代码直接推导。
Shared-memory throughput（峰值百分比）	52%	99%	本 tiled kernel 的 `sA` broadcast 与 `sB` 连续访问通常无 bank conflict；高 shared-memory throughput 仍不是无 conflict 的单独证明。

把 32 × 32 tile 暂存到 shared memory，可使每个 block 在一次 K-tile iteration 中只协作加载一份 input tile 并供 block 内 thread 复用。表中 DRAM throughput 从峰值的 90% 降到 25%，比值约为 3.6；但 throughput 是单位时间流量，不等于总传输字节数。只有结合 kernel 时间或 byte/sector 总数，才能证明 redundant traffic 减少了多少。

如果 kernel 用更少的 DRAM byte 完成相同 FLOP，算术强度会提高；表中持续浮点吞吐从 15 GFLOP/s 增至 170 GFLOP/s，约为 11.3 倍。不过这些数字是原书示意值，缺少可复现环境，且不能仅由 DRAM throughput 百分比推出。

示意的 170 GFLOP/s远低于图 6-21 所采用的约 80 TFLOP/s FP32 roof。较小问题、朴素 CUDA-core GEMM、同步和访存都可能造成差距；但没有同一 GPU 上的完整 benchmark，不能把 11 倍数字作为已验证结果。增大问题规模或提高数据复用可能改善效率，却不保证接近 peak。

算术强度提高会使 Roofline 工作点向右移动，但是否跨过 ridge point、从 memory-bound 变为 compute-bound，需要同口径的 FLOP、HBM byte 和性能 roof。表中 global load sector 从 9,800 降到 1,200 是示意值，不能由“每个元素一次载入 shared memory”直接推得。该 kernel 的 `sA` broadcast 与 `sB` 连续访问通常避免 bank conflict，但 shared-memory throughput 接近 100% 也可能表示 shared-memory pipeline 接近饱和，不能单独作为无 conflict 证明。

因此，本例能合理说明 tiling 可缓解 memory pressure 并提高 compute throughput，但现有证据不足以断言 kernel 已从 memory-bound 转为 compute-bound。优化目标应是降低端到端时间，而不是刻意把瓶颈转换成某一类别。

表中的 achieved occupancy 从 42% 增到 89% 也不能解释为“stall 更少所以 resident warp 更多”。Occupancy 表示 active/resident warp 相对于架构上限的比例，主要受 block、register、shared memory 和调度状态影响；warp 是否因 memory stall 无法 issue 是另一类指标。需要核对具体 Nsight metric 才能解释变化。

引入 shared-memory tiling 会增加 block 的 shared-memory allocation，并可能改变每 thread register 使用；资源没有超过硬件上限只说明 kernel 可以 launch，不保证更高 occupancy。原文以每 SM 65,536 个 32-bit register、每 thread 最多 255 个 register 概括所有 Blackwell，但具体限制随 compute capability 与型号而变，并且 compiler allocation granularity 也会影响驻留。

即使架构允许每 thread 最多 255 个 register，也不能仅由源码确认该 tiled kernel 没有 spilling。应查看 compiler 的 `ptxas` register/spill 报告或 Nsight Compute；接近 per-thread 上限通常还会严重限制 resident warp。

原书用持续 FLOP rate 除以平均 DRAM bandwidth 估算算术强度：naive 版本 15 GFLOP/s ÷ 10 GB/s = 1.5 FLOP/byte；tiled 版本 170 GFLOP/s ÷ 21 GB/s ≈ 8.1 FLOP/byte，算术成立。但这里 tiled 版本的绝对 DRAM bandwidth 反而是 naive 的 2.1 倍，与表中“峰值百分比从 90% 降到 25%”在同一设备和定义下不相容。两组数字不能混作同一次 benchmark；可靠方法是用实际 FLOP 与 DRAM byte counter 按同一 kernel invocation 计算。

进一步把 output tile 增大到 64 × 64 或 128 × 128，可能提高复用并减少单位 FLOP 的片外流量，但也会增加 shared memory、register、同步和 instruction-level work。资源增加可能减少每个 SM 的 resident block 数。

例如，若某个目标 Blackwell 配置允许每 SM 约 228 KB shared memory，64 × 64 的单个 FP32 tile 占 16 KB（64 × 64 × 4 = 16,384 byte），A 与 B 两个 tile 合计 32 KB；

double buffering 则可能再翻倍。除了 shared memory，还必须预算每 thread register 和 block/thread 限制。

原文继续假设每 SM 有 64K 个 32-bit register、每 thread 最多 255 个 register，并用 32 × 32 thread block（1,024 thread）计算 64 × 64 output tile，让每个 thread 负责 2 × 2 subtile、产生 4 个 output。这只是一个 mapping 示例，具体架构限制应查询目标 device attribute。

原文手工估算每 thread 需要 4 个 accumulator register、2 个 A 值、2 个 B 值，以及约 4 个 loop/index/address register，共约 12 个。真实 compiler allocation 还包含 pointer、predicate、temporary、unrolling 和 instruction scheduling 所需 register，不能由源码变量数可靠推断。

按这一简化假设，12 register/thread × 1,024 thread/block = 12,288 register/block；65,536 ÷ 12,288 的整数部分为 5。但 1,024-thread block 每个包含 32 个 warp，若目标 SM 最多驻留 2,048 thread/64 warp，则 thread/warp limit 会先把 resident block 限为 2。实际还要考虑 register allocation granularity、shared memory、block 上限与架构差异。

> 若 kernel 因 double buffering、vectorized load 等使用更多 register，可结合 compiler 输出、`cudaOccupancyMaxPotentialBlockSize` 系列 API 与 Nsight Compute occupancy report 重新计算。该 API 给出基于资源约束的 launch 建议，不替代实际 throughput benchmark。

原文又无推导地把同一 64 × 64 tile 改为 22 register/thread，即 22 × 1,024 = 22,528 register/block。65,536 个 register 理论上仍能容纳 2 个这样的 block（共 45,056），而第 3 个会超限；同时两个 block 已达到假设的 2,048-thread 上限。因此这里不是“occupancy 只有 2,048 thread”，而是在该假设下达到 100% thread/warp occupancy，只是 resident block 数为 2。原文所称每 SM 最多 16 block 也并非所有 Blackwell 型号的统一值。

较大 tile 如果降低 resident warp，可能削弱 latency hiding，但 occupancy 降低不一定损害性能：更高数据复用、较少 instruction 或更好的 ILP 可能抵消它。Tile dimension 必须满足 shared-memory、register、thread/block 等限制，并以 kernel 时间和端到端吞吐选择。

对于假设有约 228 KB 可分配 shared memory 的 GPU，64 × 64 FP32 input tile 每个约 16 KB，通常能够容纳。但 tile 边长翻倍时，tile 存储量增为 4 倍，而经典方形 GEMM 的 FLOP/byte 与每个输入元素的 block 内复用因子大致随边长线性增长，并非“复用因子平方”。因此会出现资源权衡与收益递减。

应测试不同 tile dimension，在片上复用、并行度、register、shared memory、bank mapping 与 pipeline stage 之间取舍。32 × 32 可作为教学起点，但真实高性能 GEMM 往往采用不同的 thread-block/warp/instruction tile 组合，不能把它当作通用默认最优值。

CUTLASS 等库提供 profiler，可在给定 operation、dtype、layout、shape 和硬件上搜索候选 kernel。它能找出候选集合中的较优配置，而不是证明跨所有输入的唯一“最优 tile size”。

总之，我们把只显式访问 global memory 的朴素 matrix multiplication 改为使用 shared memory 的 tiled 实现，从而在 block 内协作复用数据，并提高相对于片外流量的算术强度。CUDA C++ 示例直接展示这一机制；PyTorch 手写分块示例则调用了现成的 torch.mm，其行为与性能不能归因于同一层的手写 tiling。

高性能 GPU library 确实广泛使用多层 tiling，但实现远比上述 32 × 32 教学 kernel 复杂。例如，NVIDIA CUTLASS 提供构建 GEMM 的模板化组件，在 thread block、warp、instruction 等层级组织 tile，并利用 register、shared memory、异步 copy 和 Tensor Core pipeline 计算 partial result。

cuBLAS 与 cuDNN 的许多 kernel 也使用 blocking/tiling，但算法选择会随 operation、shape、dtype、layout 与硬件而变化，不能保证所有调用都接近 peak。NVIDIA 在 2025 年发布了 Python 风格的 cuTile 编程模型，使程序员以 tile 为中心描述计算；相关编译栈使用 tile-level IR。具体 API、IR 名称与可用性仍应以当前 CUDA/cuTile 文档为准。

其他高性能工具也封装 tiling pattern。CUTLASS 的 C++/Python 接口和 profiler 提供 tile abstraction 与配置搜索；TorchInductor 可根据 graph、shape 和 backend 生成 tiled kernel，常见 GPU codegen 路径会使用 Triton，但也可能调用 external kernel/library。它们降低了手写 boilerplate 的需求，不过仍需 profile 生成代码。

核心思想是在返回 DRAM 之前，尽可能在 register/shared memory/cache 中有效复用数据，同时不超过片上资源预算。优先使用经过验证的 library kernel；编写自定义 kernel 时，也可参考这些实现的 tiling 与 pipeline pattern。

例如，PyTorch 的 `torch.mm` 或直接调用 `cublasSgemm` 通常会选择包含某种 blocking、coalescing 与架构专用优化的 kernel，但不一定“恰好”采用本节的 32 × 32 shared-memory 方案。前面的 PyTorch 示例也没有提供能证明相同收益的实测。

生产代码通常应先采用 cuBLAS、PyTorch `torch.matmul` 等高性能 operator，并针对实际 shape、dtype 和精度设置 benchmark。直接使用 `torch.mm` 或 `torch.matmul` 往往优于 Python 手写分块，但 view、batch、broadcast 与 backend 选择会影响最终 kernel。

即使主要复用现有 tiling library 与 framework，理解其基本机制仍有助于诊断性能问题，并在通用实现不能覆盖特殊数据布局、融合或数值需求时编写 custom kernel。若改进具有通用价值，也可以向相关开源项目反馈或贡献。

Shared-memory bank conflict 取决于同一 warp 的 lane 如何映射到 bank，而不是仅由 tile 为 32 × 32 决定。本节 GEMM 访问通常使用 broadcast 与连续 bank；下一节将通过确实存在列式访问冲突的模式说明问题及优化方法。

避免 shared-memory bank conflict

在本节讨论的现代 NVIDIA GPU（包括相应 Blackwell 配置）上，shared memory 通常组织为 32 个 bank，常见 32-bit word 访问可按 4-byte bank 单元理解，即 bank index 对 32 循环（`mod 32`）。若 warp lane 的地址相隔 128 B（32 float × 4 B），它们会映射到同一 bank。多个 lane 在同一 instruction 中访问该 bank 的不同 word 时发生 *bank conflict*，硬件需要把请求拆成多个 wavefront/replay，而非必然让整个 memory system 完全串行。

代码中的 bank conflict 常由 shared-memory stride 使多个 lane 映射到同一 bank 的不同地址而产生。图 7-4展示两种无 conflict 的 bank access。

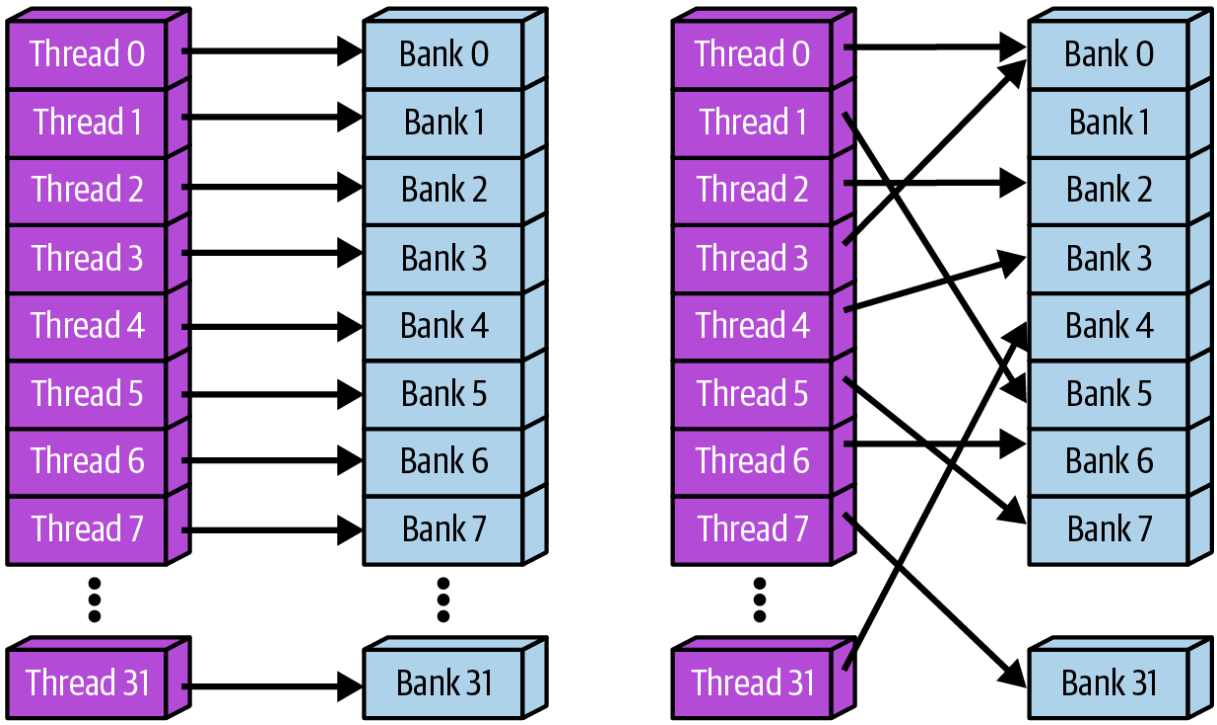


图 7-4。无 bank conflict

图中各 lane 在同一 instruction 中访问不同 bank，是理想情况。多个 lane 读取同一地址的 broadcast 是另一种无 conflict 情形。图 7-5展示 2-way 与 16-way bank conflict。

多个 lane 访问同一 bank 的不同 word 会产生 conflict，严重程度取决于 access pattern 与架构。经典示例是使用 32 × 32 shared-memory tile 的 matrix transpose。若 32 个 lane 读取 `tile[i][threadIdx.x]`，并且这里的 column index (`threadIdx.x`) 固定、row index 随 lane 改变（`i`），各地址跨 row 相隔 128 字节，可能形成 32-way conflict。原文变量描述含混；关键是不同 row、固定 column。

具体来说，在 row-major tile 中沿同一列读取时，warp lane 的 column index（`threadIdx.x`）固定、row index（`i`）变化。每行相隔 128 字节，因此 lane address 的差值为 128 字节的整数倍。

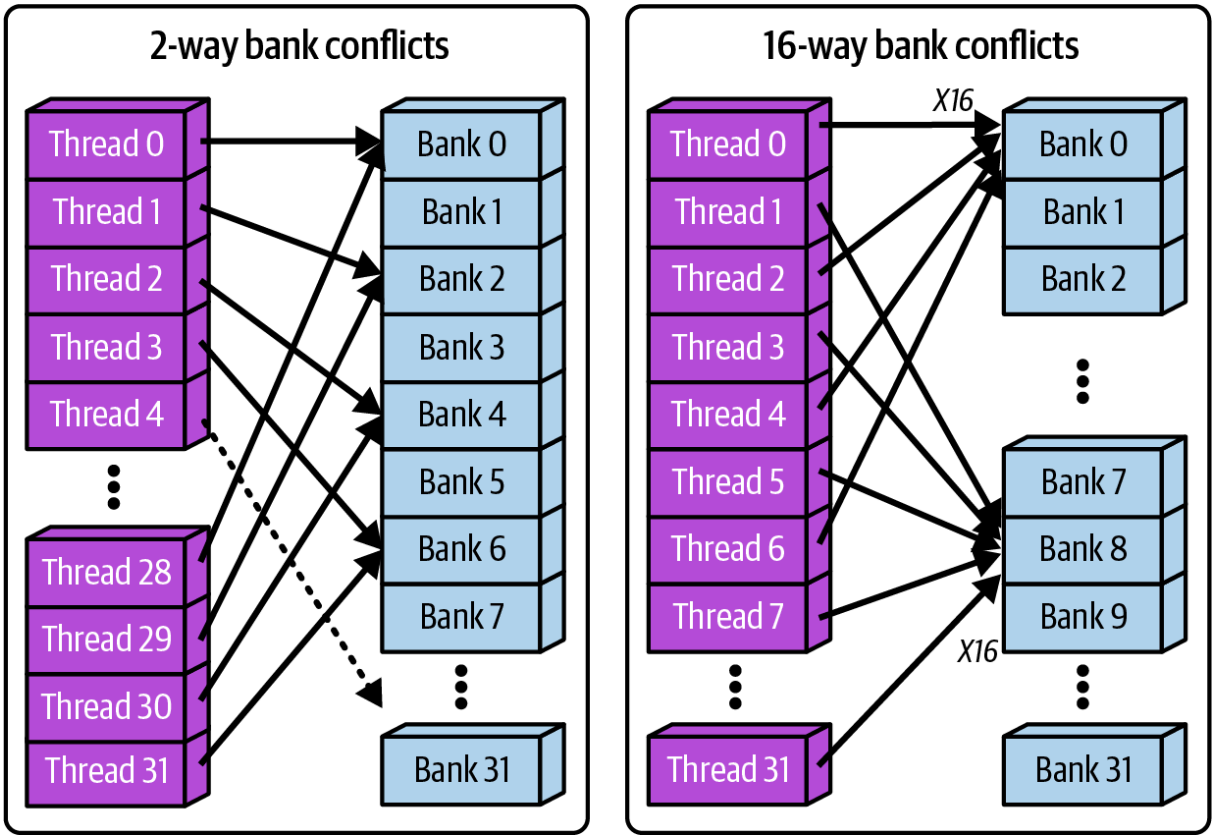


图 7-5。2-way 与 16-way bank conflict

在 32 bank × 4 byte 的简化 mapping 中，bank index 每 128 字节重复。相隔 128 字节的地址会落到同一个 bank，但不一定是 bank 0；具体 bank 由起始地址决定。若 32 个 lane 访问该 bank 中 32 个不同 word，就形成 32-way conflict。

> 一个重要例外是：若 warp 的多个 lane 读取完全相同的 shared-memory word，硬件可 broadcast 该值，不构成 bank conflict。某些架构与访问宽度还具有 multicast 或不同的 bank 服务能力；同 bank 不同地址通常会增加 wavefront/replay，但不应笼统断言总是逐 lane完全串行。

另一个常见问题是使用与 bank 数相同的 element stride，本例为 32。若索引每次跨 32 个 float，每个 4 字节，则相邻 lane address 相差 128 字节，并映射到同一 bank。以下代码在基址对应 bank 0 时全部落入 bank 0：

```
// Allocate a shared buffer big enough for several warps (warpCount)
__shared__ float arr[32 * warpCount];

// Each thread reads from arr[threadIdx.x * 32]
float x = arr[threadIdx.x * 32];
```

这里以 float 为单位的 `threadIdx.x * 32` 对应字节偏移 `(threadIdx.x * 32 * 4)`。由于 `32 * 4 = 128`，每个 lane 的 load address 相对基址偏移 `threadIdx.x * 128` 字节，而 `threadIdx.x * 128 mod 128 = 0`。若各 lane 地址不同，它们会命中同一 bank 并形成 32-way conflict。

发生 conflict 时，一条 warp shared-memory request 会被拆成多个 wavefront/replay。Nsight Compute 可提供 shared-memory bank-conflict、wavefront 与相关 stall 指标，具体名称随版本变化。Shared-memory throughput 可能下降，也可能因重复服务而看似较高；Nsight Systems 通常不提供足以单独归因 bank conflict 的细粒度 per-kernel counter，应以 Nsight Compute 为主。

严重 bank conflict 会增加 shared-memory instruction 的服务周期并降低性能，但它只是潜在瓶颈之一。若 kernel 加速不及预期，应同时检查 global memory、occupancy、barrier、dependency 和 instruction throughput。

> 应根据同一 warp 的实际 lane address 选择 stride 与 data layout，使不同地址尽量分布到不同 bank；相同地址的合法 broadcast 无需刻意拆开。

常见解决方法是调整 shared-memory layout。*Padding* 会改变 row stride，使原本同 bank 的列访问分散。例如，把 32 × 32 tile 声明为 `[32][33]`，每行增加一个 padding element。这样 thread `k` 访问 `tile[i][k]` 时，相邻 row 的起始 bank 逐行偏移。图 7-6 展示这一方法。

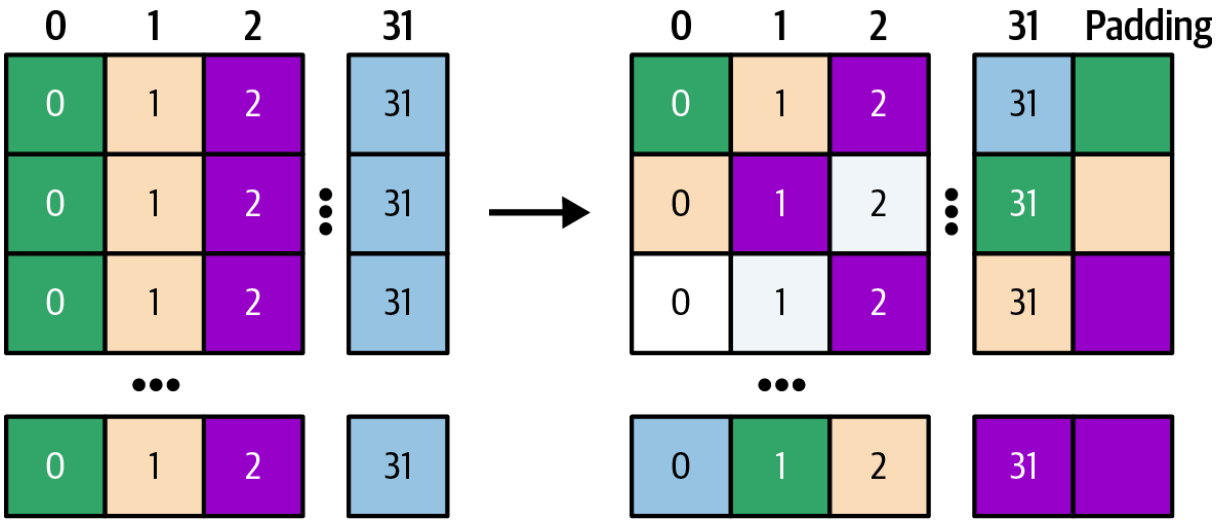


图 7-6。使用 padding 避免 bank conflict

把 row stride 改为 33 个 float 后，沿列访问的 32 个 lane 会轮转到不同 bank，从而消除该特定 32-way conflict。其他 vector width、element size 或 tile pattern 仍需重新分析。

Padding 让 32-wide tile 的元素数增加 1/32，约 3.125%。对于 FP32 32 × 32 tile，原大小为 4,096 字节，增加一列只增加 32 × 4 = 128 字节，而不是原文所称的 1 KB。是否值得取决于 conflict 成本、shared-memory occupancy 与整体 kernel 时间；Blackwell 每 SM 容量也应按具体型号查询。

下面以简单 transpose kernel 展示如何消除 shared-memory bank conflict。naive 版本的转置读取使 warp lane 访问同一 bank 的不同 row，增加 shared-memory wavefront 数。以下是包含 conflict 的实现。

优化前示例（C++）：存在 bank conflict 的 naive transpose：

```cpp
#include <cuda_runtime.h>
#define TILE_DIM 32

__global__ void transposeNaive(const float *idata, float *odata, int width) {
    __shared__ float tile[TILE_DIM][TILE_DIM];
    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    // threads in a warp write a row
    tile[threadIdx.y][threadIdx.x] = idata[y * width + x];
    __syncthreads();

    // Read from shared memory with transposed indices
    // This is a classic case of all threads in a warp
    // hitting the same bank causing a bank conflict
    // Read transposed from shared memory and write out
    odata[x * width + y] = tile[threadIdx.x][threadIdx.y];
}

int main() {
    const int N = 1024;
    size_t size = N * N * sizeof(float);
    float *h_idata = (float*)malloc(size);
    float *h_odata = (float*)malloc(size);
    // Initialize input h_idata...
    float *d_idata, *d_odata;
    cudaMalloc(&d_idata, size);
    cudaMalloc(&d_odata, size);
    cudaMemcpy(d_idata, h_idata, size, cudaMemcpyHostToDevice);

    dim3 block(TILE_DIM, TILE_DIM);
    dim3 grid(N / TILE_DIM, N / TILE_DIM);
    transposeNaive<<<grid, block>>>(d_idata, d_odata, N);
    cudaDeviceSynchronize();

    cudaFree(d_idata);
    cudaFree(d_odata);
    free(h_idata);
    free(h_odata);
    return 0;
}
```

该 kernel 写入 shared memory 的 `tile` 时使用 row-major `tile[ty][tx]`，同一 warp 的 tx 连续，因此访问不同 bank。（若写成 `column-major tile[tx][ty]`，同一 warp 中 tx 变化会使地址跨 row 相隔 128 字节，形成 32-way conflict。）随后以转置索引读取 `tile`：同一 warp 中 `threadIdx.y` 固定为 column index，而 `threadIdx.x` 变化并用作 row index；原文把二者说反了。

对给定 warp，thread 0–31 的 `threadIdx.y` 固定、threadIdx.x 变化，所以实际访问是 `tile[varying_row][constant_col]`。32 个地址位于不同 row 的同一列，每个相隔 128 字节并映射到同一 bank，形成 32-way conflict。原文把 `tile` 访问误写为同一 row 的不同 column；后者反而会连续映射到不同 bank。

本例假设 Blackwell shared memory有 32 个 bank，与 warp 的 32 个 lane 数相同。固定 column、改变 row 且 row stride 为 32 个 FP32 时，所有 lane 落入同一 bank 的不同 word，形成 32-way conflict。固定 row、改变 column 则无此问题。并非任何 misaligned warp access 都会全冲突；必须计算实际 bank index，并考虑 broadcast 与架构服务规则。

在该代码的典型对齐下，32 个 thread 读取 32 个不同地址，并映射到同一 bank，产生大量 replay。下面加入 padding 消除这一 conflict。

优化后示例（C++）：padded transpose（避免 bank conflict）。增加一个未使用的 padding column，使每个 shared-memory row 的起始 bank 相对前一行偏移 1:

```
#include <cuda_runtime.h>
#define TILE_DIM 32
#define PAD 1  // padding columns to avoid bank conflicts

__global__ void transposePadded(const float *idata, float *odata, int width) {
    // Each row is TILE_DIM+1 elements to shift bank mapping
    __shared__ float tile[TILE_DIM][TILE_DIM + PAD];
    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    tile[threadIdx.y][threadIdx.x] = idata[y * width + x];
    // We will optimize this later to use a thread-block-scoped
    // cooperative-groups barrier
    __syncthreads();

    odata[x * width + y] = tile[threadIdx.x][threadIdx.y];
}
```

（host code 与 naive 版本相同，为简洁起见省略。）

PyTorch 没有用于直接声明 custom-kernel shared-memory padding 的高层 tensor API；可用 CUDA extension（如 `torch.utils.cpp_extension`）或 Triton/CUDA 等 custom kernel 路径实现。原文括号多写了一个 `)`。PyTorch operator 可能调度到 cuBLAS、cuDNN 或生成 kernel，它们会按具体算法处理 bank mapping，但不能保证所有 operator 无 conflict 或达到最大吞吐。

每个 shared-memory row 增加一个元素后，row stride 变为 33 个 float。对 `tile[row][col]` 的沿列访问，row 每增加 1，bank index 也偏移 1，使 32 个 lane 分布到 32 个 bank。这里应依据 word-address modulo 32 解释，而不是笼统称“地址低 5 bit”不同，因为 byte address 的低位还包含 4-byte word 内偏移。

padded kernel 实际读取的是 `tile[threadIdx.x][threadIdx.y]`，而不是原文写的相反索引。对于同一 warp，变化的 row index 配合 33-float stride，使 lane 映射到不同 bank，从而消除该特定 transpose read 的 conflict。表 7-5给出原书示意指标。

表 7-5。消除 shared-memory bank conflict 的示意效果

指标	优化前（无 padding）	优化后（padding）
共享内存加载银行冲突	480 万	0
Shared-memory utilization（峰值百分比）	52%	100%
warp 停顿比例（内存）	38%	0.5%
Kernel 执行时间（ms）	4 ms	1.3 ms（改善约 3 倍）

表 7-5给出的 Nsight Compute 示意指标中，shared-memory load bank conflict 降为 0，throughput 从 52% 升至 100%，memory-related warp stall fraction 从约 38% 降到 0.5%。但原书未给出准确 metric 名称、采样方法和 GPU/clock/iteration，不能称这些数值“证实”所列代码的通用结果。

在该 transpose pattern 中，padding 可以消除 32-way conflict；表中 kernel time 从 4 ms 降至 1.3 ms，约为 3.1 倍加速。该数字仍是教学示意，且 global-memory transpose store、launch 与其他开销不会因 shared-memory padding 自动消失。

padding 恢复了该 shared-memory read 的 bank parallelism。每 32 个 column 增加 1 个 float，使 tile storage 增加约 3.125%；是否相对性能收益可忽略，需要结合 shared-memory resource limit 与实测判断。

Swizzling 是 padding 的一种替代方案：通过 XOR、位重排或其他可逆 index transformation 改变 logical element 到 shared-memory address 的映射，使目标 lane access 分散到 bank。具体变换必须同时保持生产者与消费者使用一致 mapping，并满足 vector/Tensor Core layout。

Swizzling 可以不增加 tile element 数，但会增加 index/layout 复杂度，也不保证对所有访问阶段都实现“perfect bank parallelism”。Padding 较简单；swizzle 更适合经证明与目标 instruction layout 匹配的模式。

> CUTLASS 等高性能 CUDA library 会在部分 tile layout/copy path 中使用 swizzle 或专用 layout，以满足 coalescing、bank conflict 与 Tensor Core instruction 的共同要求；不能概括为所有 iterator 都让每个 thread 永远映射到独立 bank。

总之，使用 shared memory 时，应按 warp instruction 分析 lane address、bank index、broadcast 与访问宽度。Padding 和 swizzling 可消除特定 conflict，但不保证“只要使用 shared memory 就一定提高性能”；还应测量额外资源、index 开销与整体 kernel 时间。

下一节讨论在 warp 内直接在线程之间交换 register value，从而在适用场景中省去 shared memory。

Warp shuffle intrinsic：避免 shared memory 与显式同步

上一节的 bank-conflict 优化假设 thread 通过 shared memory 通信。如果通信只发生在一个 warp 内，有些场景可以完全省去 shared memory，也就不存在相应 bank conflict。

NVIDIA GPU 提供 warp-level primitive，使同一 warp 的 lane 通过 shuffle 数据路径交换 register value，而不经过 shared memory。这些 primitive 的作用域限于一个 warp，并且只有 mask 中参与的 lane 才满足同步契约；用于 intrawarp communication 时不访问 memory bank，因此不会产生 shared-memory bank conflict。

最常见的是 `__shfl_sync` 系列 intrinsic。`__shfl_sync` 可以从指定 source lane 读取 value，从而实现 broadcast、permutation 或 warp-level reduction，而无需写 shared memory。`_sync` 后缀表示 mask 中所有非退出 lane 都必须以同一 mask 执行对应 intrinsic；它不是跨 warp synchronization。

> 现代 GPU 在多个 lane 读取 shared memory 中完全相同的地址时可 broadcast，因此这个单地址情形本就没有 bank conflict。Warp shuffle 则可按 lane id 交换多个不同 register value；它不是所有任意模式的无条件替代方案，仍受 mask、source lane 与数据类型规则约束。

假设需要对一个完整 warp 的 32 个 per-thread partial result 求和。shared-memory 方案可以让每个 thread 写入数组，再按 reduction tree 读取并同步；若布局正确，它不一定发生 bank conflict，但会增加 shared-memory instruction 与同步操作。

使用 `__shfl_sync` 系列可以在 register 中完成 tree reduction。下面使用 `__shfl_down_sync`，每轮从 lane id 加 `offset` 的 source lane 读取值。这是 down-shuffle reduction，并非通常由 XOR partner 构成的 butterfly all-reduce。示例假设 mask 对应完整的 32-lane warp；对任意部分 warp，必须按参与 lane 的连续性和数量调整算法，避免读取 inactive source lane：

```
unsigned mask = __activemask();
float val = threadVal;  // each thread's partial sum

// Perform butterfly reduction: exchange values with increasingly distant lanes
for (int offset = 16; offset > 0; offset >>= 1) {
    float other = __shfl_down_sync(mask, val, offset);
    val += other;
}

// After the loop, lane 0 holds the warp's total sum
```

每次 `__shfl_down_sync` 让 lane 读取指定 offset 处 source lane 的 register value。对完整 warp 的这个 reduction pattern，lane 0 最终得到总和；其他 lane 通常只得到部分后缀和。代码没有显式把“active thread 数减半”，所有参与 lane 仍执行循环，只是超出 width 的 source read 返回值语义需遵循 intrinsic 规则。通信不经过 shared memory，因此无 bank conflict，也不需要 block-wide barrier，但 `_sync` mask 本身承担 warp 参与约束。

总之，`__shfl_sync` 及其变体在 warp 内交换 register value，不使用 shared memory。它们常能减少 shared-memory load/store 与 barrier，但是否更快取决于 instruction dependency、mask、数据布局与目标架构。

许多高性能 warp-level reduction 使用 shuffle intrinsic，在少量 instruction 中交换 register value，绕过 shared memory。CUDA Cooperative Groups 提供 tiled partition 与 group collective；具体 shuffle API 属于支持相应操作的 thread-block tile/group type，并非任意 `thread_group.shuffle()` 都存在。`__reduce_sync()` 是独立的 warp reduce intrinsic，compiler 可映射为 target architecture 的 reduction instruction 或其他实现，不能断言最终总是展开为 shuffle pattern。

> Shuffle 只在单个 warp 内工作。跨 warp reduction 可使用 shared memory 配合 block synchronization、Cooperative Groups collective，或在支持时使用 cluster distributed shared memory/cluster synchronization；这些机制并非“multiwarp shuffle”。第 10 章将讨论相关概念。

Shuffle 的 source/destination lane 限于同一 warp，常见 warp size 为 32。跨 warp 传递普通数据通常需要 shared/global memory 或架构支持的 cluster memory，并配合正确同步；也可通过更高层 collective/library 封装这些步骤。

第 10 章将讨论 Cooperative Groups synchronization、multiwarp pattern 与 thread-block cluster（CTA cluster）。它们可能使用 barrier、shared/distributed shared memory、shuffle 或其他硬件机制，并非最终都使用同一种 intrinsic。理解 intrawarp 与 interwarp communication 有助于定位 memory、synchronization 和 dependency bottleneck。

Read-only data cache

当多个 thread 读取相同value，或某个 thread 重复读取不变数据时，cache locality 会显著影响性能。例如，推理期间只读的 embedding/lookup table 可能被大量 thread 并行访问；若 working set 和访问分布适合 cache，片上 cache 可减少 HBM traffic。若访问随机且表远大于 cache，声明只读也不能创造 locality。

这里讨论的 read-only load path 与 64 KB constant address space/constant cache 机制不同。Constant memory 适合容量较小、warp 内地址较一致的数据；大型 embedding 通常位于 global memory，并通过 L2 与架构相应的 L1/texture/read-only cache path 服务。现代架构中这些路径可能共享或统一物理 cache，不能把 read-only cache 简单描述成专用于 immutable data 的独立“大缓存”。

现代 GPU 的 global-memory load至少经过 L2，并可按 architecture/cache operator 使用 L1/texture path。`const __restrict__` 中，`const` 限制通过该 pointer 修改对象，`__restrict__` 是关于 aliasing 的 programmer promise；两者都不表示“non-coherent”或“read-mostly”。这些信息可能帮助 compiler 选择 read-only load 或进行其他优化，但 read-only traffic 仍会占用 cache capacity，也可能驱逐其他数据。

> 在支持的 CUDA target 上，compiler 常能为可证明只读的访问自动选择合适 load，因此通常无需显式调用 `__ldg()`。`const __restrict__` 可能提供所需的 alias/immutability 信息，但不是生成特定 instruction 的保证。`__ldg()` 的可用性和实际 cache 行为随 compute capability 与 Toolkit 变化，应检查当前文档和 SASS。

若 programmer 能保证 buffer 的只读与 non-aliasing 语义，却没有向 compiler 表达这些信息，可能错失 load、code motion 或 scheduling 优化。但普通 global load 仍然可以命中 L1/L2；缺少 `__restrict__` 不会自动使每次访问都落到 DRAM，也不会必然造成“虚假 cache miss”。错误使用 `__restrict__` 反而会违反 aliasing contract并导致未定义行为。

性能分析时，应检查实际 SASS/cache operator、L1/TEX 与 L2 hit rate、requested/sector byte、DRAM byte 和 stall reason。Instruction stream 中没有 `__ldg()` 源码名称并不证明未使用 cache；现代 machine instruction 与 profiler counter 不一定沿用该名称。L2 hit rate 低也可能只是 working set 过大或访问缺乏 locality。

在语义真实成立时，可用 `const __restrict__` 表达只读与 non-aliasing，或在支持且必要时测试 `__ldg()`。这是给 compiler 的语义/显式 load 选择，不是向 hardware 永久标记整个 allocation。现代 L1/texture/read-only 路径的物理组织和 latency 随架构变化。

对于 constant memory 的 warp-uniform address，constant cache 可 broadcast；某些 read-only/global load 路径也能合并相同地址请求。`__constant__` 与旧架构上的 uniform `__ldg()` 不是同一种 cache 语义。Broadcast 可减少 transaction，但不是“免费”或保证单周期：cache miss、dependency、pipeline occupancy 与 contention 仍会影响 latency。

下面以大小 T = 1,024 的 table 为例，每个 thread 查找一个 value 并写入 output。FP32 table 只有 4 KB，而且 `t = idx % T` 使 warp 内访问连续、跨 warp 周期重复，因此它是高度 cache-friendly 的 microexample，并不能代表大型随机 embedding lookup。先看 CUDA C++:

```
#include <cuda_runtime.h>
#define T 1024

__global__ void naiveLookup(const float* table, float* out, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        // __ldg not used here, each access goes to
        // global memory without using read-only cache
        int t = idx % T;
        out[idx] = table[t];
    }
```

```
}

int main() {
    const int N = 1 << 20;

    float* h_table = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_table, T * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));

    for (int i = 0; i < T; ++i) h_table[i] = float(i);

    float *d_table, *d_out;
    cudaMalloc(&d_table, T * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_table, h_table, T * sizeof(float),
        cudaMemcpyHostToDevice);

    dim3 block(256), grid((N + 255) / 256);
    naiveLookup<<<grid, block>>>(d_table, d_out, N);
    cudaDeviceSynchronize();
    cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost);
    cudaFree(d_table);
    cudaFree(d_out);
    cudaFreeHost(h_table);
    cudaFreeHost(h_out);

    return 0;
}
```

下面是粗略的 PyTorch 等价表达：

```
import torch

def vectorized_lookup(table, N):
    flat = table.view(-1)
    T = flat.size(0)

    # build indices [0,1,2,...,N-1] % T all on GPU
    idx = torch.arange(N, device=flat.device) % T

    # one gather kernel does all N loads in parallel
    return flat.index_select(0, idx)


# Usage
T = 1024
N = 1 << 20
table = torch.arange(T, dtype=torch.float32,
                     device='cuda')
out = vectorized_lookup(table, N)
```

这些版本在首次装入后很可能命中 L2，甚至命中相应 L1 path；不能从源码断言“没有使用 specialized read-only cache”。本例同一 warp 的 lane 读取不同但连续的 `table[t]`，并非 warp-uniform broadcast。PyTorch index_select 的 backend 与 cache instruction 也由当前实现决定。

接下来在 CUDA kernel 的 table pointer 上加入 `const __restrict__`。两版其实都已有 `const float*`；唯一新增的是 `__restrict__` aliasing promise。Compiler 可能据此改

善 code generation，但它不是“提示 hardware 必须使用 read-only cache”：

```
#include <cuda_runtime.h>
#define T 1024

__global__ void lookup(const float* __restrict__ table,
        float* out, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        int t = idx % T;
        // Compiler can turn this into a load from the
        // read-only cache for faster loads
        out[idx] = table[t];
    }
}


int main() {
    const int N = 1 << 20;

    float* h_table = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_table, T * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));

    for (int i = 0; i < T; ++i) h_table[i] = float(i);

    float *d_table, *d_out;
    cudaMalloc(&d_table, T * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_table, h_table, T * sizeof(float), cudaMemcpyHostToDevice);

    dim3 block(256), grid((N + 255) / 256);
    lookup<<<grid, block>>>(d_table, d_out, N);
    cudaDeviceSynchronize();
    cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost);
    cudaFree(d_table);
    cudaFree(d_out);
    cudaFreeHost(h_table);
    cudaFreeHost(h_out);

    return 0;
}
```

> 在 PyTorch 中，可以通过 `torch.utils.cpp_extension` 编写 CUDA extension，并在 aliasing contract 成立时把 table pointer 声明为 `const __restrict__`。PyTorch compiler 也可能基于 graph alias analysis 优化访问，但不会因用户意图自动给任意 tensor pointer 添加不成立的 restrict promise。第 13、14 章将深入介绍 PyTorch compiler。

简言之，table pointer 上的 `const __restrict__` 向 compiler 表达通过该 pointer 不修改对象且其访问不与其他相关 pointer alias；在安全时，这可能允许 read-only load 或其他优化，但不保证提高 cache hit rate或减少片外 traffic。表 7-6保留原书给出的示意指标。

表 7-6。Read-only cache 的示意收益

指标	**优化前**（无 `__restrict__`）	**优化后**（使用 `__restrict__`）
全局内存加载效率	52%	97%
DRAM 吞吐量（GB/s）	600 GB/s	200 GB/s
L2 读取吞吐量（GB/s）	1,500 GB/s	1,800 GB/s
SM 活跃百分比	45%	93%
Kernel 执行时间（ms）	2.5 ms	1.0 ms

原书表中加入 `const __restrict__` 后，kernel time 从 2.5 ms 降到 1.0 ms，约为 2.5 倍加速；SM Active 从 45% 升到 93%。但两份 CUDA 代码都已有 `const float*`，且 4 KB table 高度 cache-friendly，只有 `__restrict__` 不足以可信地解释如此巨大变化。没有 GPU、compiler/SASS、metric 和计时方法，这些数字只能视为示意。

表中 DRAM throughput 从 600 GB/s 降到 200 GB/s，L2 read throughput 从 1,500 GB/s 升到 1,800 GB/s，Global Memory Load Efficiency 从 52% 升到 97%。Throughput 是单位时间速率；kernel 变短时，不能仅凭速率推断总 byte。Load efficiency 主要反映 requested 与 transferred byte 比例，而 `__restrict__` 通常不改变本例连续索引的 coalescing，因此该组指标机制也不自洽。

> Nsight Systems 提供应用级 timeline；Nsight Compute 提供 per-kernel hardware metric。定量分析 cache path 时应使用 Nsight Compute，并记录 metric 全名、采集 section、版本与 replay 开销；SM Active 本身不等于“每个周期都有有用算术工作”。

若更多 request 由 cache 命中而无需访问 DRAM，相对于 HBM byte 计算的实测算术强度会提高，但算法层面 FLOP/byte 未变。是否由 memory-bound 移向 compute-bound 仍需 Roofline 实测。Texture object 可用于具有 1D/2D/3D spatial locality 的读取，并按 resource/descriptor 支持 address mode、filtering 与格式转换；将合适 resource 绑定为 `cudaTextureObject_t` 后可使用 `tex1Dfetch`、`tex2D` 等对应 intrinsic。Surface object 为 CUDA array 提供读写路径，但不提供 texture filtering 的同等语义。

> Legacy texture reference 与 surface reference API 已弃用，texture object 与 surface object API 仍受支持。是否适合取决于 resource type、addressing、locality 与所需读写语义。对于一般 1D AI tensor，普通 global load 与 compiler/library 生成的 cache policy 通常更简单；constant memory 仅适合较小且 warp-uniform 的数据，不能与 read-only global cache 合并称为统一首选。

总之，只在语义成立时为 read-only data使用 `const __restrict__`，让 compiler 获得正确的只读/aliasing 信息；随后检查实际 codegen 与 cache metric。Texture/surface object 适合特定 multidimensional resource 与 addressing 需求，不是 regular cache 的普遍替代品。任何性能收益都取决于 locality、working set、架构与整体 kernel 瓶颈。

异步内存预取与 Tensor Memory Accelerator

前几节已经说明，将数十个 4-byte load 合并为一次 128-byte transaction，可以显著提高 global load 效率，并减少每个 request 中浪费的 sector。然而，即使 load 已经完美合并，warp 仍可能因等待完整的 DRAM 往返而停顿。

以 Blackwell 为例，在具体访问路径和系统状态不同的情况下，一次完整 DRAM 往返的延迟可能达到数百个 cycle，之后计算才能使用返回的数据。要隐藏这部分延迟，就需要让数据传输与计算重叠；这种 overlap 可以遮蔽相当一部分 DRAM latency。

CUDA Pipeline API 与 Tensor Memory Accelerator（TMA）硬件引擎将这一思路扩展到 thread block 层级。每个 warp 不必都通过 SM 的 load/store（LD/ST）单元从 global memory 取数；在满足目标架构、传输方向、对齐、shape 和 stride 等要求时，可以使用 TMA engine 将整个 tile 从 global memory 异步搬运到 shared memory，如图 7-7所示。

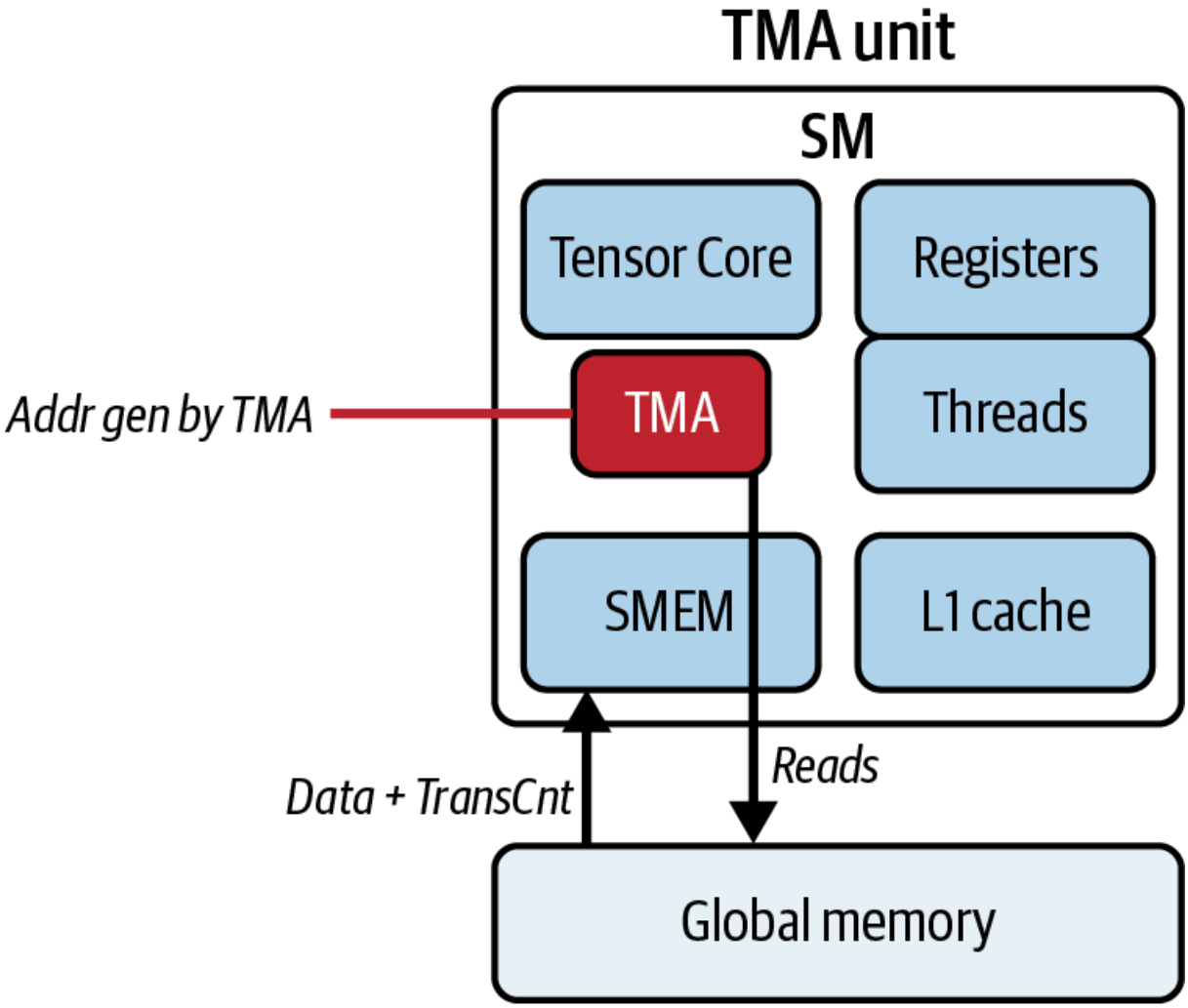


图 7-7. TMA 将数据从 global HBM 异步取入 shared memory

要发起异步传输，可以调用 `cuda::memcpy_async()`。在现代 GPU 架构上，`cuda::memcpy_async()` 与 `cuda::pipeline` 为 global-to-shared 异步传输提供了高级接口；当目标硬件以及方向、对齐、大小和布局等条件满足时，编译实现可以利用 TMA 或其他异步复制机制执行 bulk data transfer。因此，是否实际生成 TMA 指令，需要结合目标架构、CUDA toolchain 和编译结果确认。CUDA 写法如下：

```
cuda::memcpy_async(sharedBuf,
                   globalPtr + offset,
                   cuda::aligned_size_t<16>(bytes),
                   pipe);
pipe.producer_commit();
```

在异步复制机制处理 bulk copy（包括合并访问、strided transfer，乃至 multidimensional transfer）的同时，kernel 可以计算前一个 tile。这种方法称为*双缓冲（double buffering）*，也称*乒乓缓冲（ping-ponging）*。

使用 TMA 配合双缓冲，可以把一部分地址生成和数据搬运工作从常规 SM load/store 指令路径卸载出去。TMA 搬入下一个 tile 时，SM 中的 warp 处理前一个 tile；若 pipeline 深度、计算量、occupancy 和内存带宽等条件允许，这种重叠便能隐藏相当一部分

DRAM latency。这里的延迟不是固定的 800 cycle，而会随架构、cache 命中情况、访问模式和系统负载变化。

具体而言，TMA 支持 global memory 与 shared memory 之间的 1D–5D bulk copy，并能按 tensor descriptor 表达 multidimensional shape 和 stride。它可减少 kernel 显式发出的 LD/ST 指令，并使部分数据搬运与计算并行；但传输仍会占用内存与复制资源，也仍需要正确的 async barrier 或 pipeline phase 同步，不能理解为完全没有同步或调度开销。

下面的代码片段展示了如何通过 CUDA C++ Pipeline API 与相应的异步复制硬件，从 global memory 向 shared memory 发起异步复制；是否使用 TMA，取决于目标架构及相关约束：

```
#include <cuda/pipeline>
#include <cuda_runtime.h>
#include <cstdint>
#include <cassert>

#define TILE_SIZE 1024  // example tile size
// User-provided compute function operating on a shared-memory tile
__device__ void processTile(const float* tile);

__global__ void kernelWithTMA(const float* __restrict__ global_ptr,
                              int nTiles) {
    // Two ping-pong buffers in shared memory
    // On Blackwell (CUDA 13+), prefer 32B alignment for 256-bit vectors
    // (v8.f32 / double4).
    __shared__ __align__(VEC_BYTES) float tile0[TILE_SIZE];
    __shared__ __align__(VEC_BYTES) float tile1[TILE_SIZE];

float* tiles[2] = { tile0, tile1 };

    // Alignment / size guards for vectorized copies
    // --- choose vector width ------------------------------------------------
    #ifndef VEC_BYTES
      // Prefer 32B vectors on CUDA 13+/Blackwell.
      // Define -DUSE_256B_VEC to force.
      #if defined(USE_256B_VEC) || (defined(CUDA_VERSION)&&CUDA_VERSION >= 13000)
        #define VEC_BYTES 32      // 256-bit: v8.f32, double4
      #else
        #define VEC_BYTES 16      // 128-bit: v4.f32, float4
      #endif
    #endif
    constexpr int VEC_ELEMS = VEC_BYTES / sizeof(float); // 4 (128b) or 8 (256b)
    constexpr int WARP      = 32;

    // Alignment / size guards for vectorized copies/compute
    static_assert((TILE_SIZE % (WARP * VEC_ELEMS)) == 0,
                  "TILE_SIZE must be a multiple of WARP_SIZE * VEC_ELEMS");
    // Also guarantee the tile byte count
    //  is a multiple of the vector width
    static_assert(((TILE_SIZE * sizeof(float)) % VEC_BYTES) == 0,
                  "Tile byte size must be a multiple of VEC_BYTES");

    // On Blackwell (CUDA 13+), prefer 32B alignment
    // for 256-bit vectors (v8.f32 / double4).

    assert((reinterpret_cast<std::uintptr_t>(global_ptr) % VEC_BYTES) == 0);

    size_t bytes = TILE_SIZE * sizeof(float);
```

```
    // Block-scoped pipeline for TMA
    __shared__ cuda::pipeline_shared_state<
        cuda::thread_scope_block, 2> state;

    auto pipe =
        cuda::make_pipeline(cuda::this_thread_block(),
            &state);

    // Prime pipeline with the first async copy into tile0
    pipe.producer_acquire();
    cuda::memcpy_async(tiles[0],
                       global_ptr + 0 * TILE_SIZE,
                       cuda::aligned_size_t<32>{bytes},
                       pipe);
    pipe.producer_commit();

    // Loop over the remaining tiles
    for (int t = 1; t < nTiles; ++t) {
        // Wait for the previous copy to finish, then compute on it
        pipe.consumer_wait();
        processTile(tiles[(t - 1) & 1]);
        pipe.consumer_release();

        // Enqueue the next async copy into the alternate buffer
        pipe.producer_acquire();
        cuda::memcpy_async(tiles[t & 1],
                           global_ptr + t * TILE_SIZE,
                           cuda::aligned_size_t<32>{bytes},,
                           pipe);
        pipe.producer_commit();
    }

    // Final wait and compute on the last tile
    pipe.consumer_wait();
    processTile(tiles[(nTiles - 1) & 1]);
    pipe.consumer_release();
}
```

kernel 启动后，每个 block 会分配两个 shared-memory tile（`tile0` 和 `tile1`），并构造一个 block scope 的 pipeline，使 thread block 中的所有 thread 按同一 pipeline phase 协调异步复制，如下所示：

```
__shared__ cuda::pipeline_shared_state<
    cuda::thread_scope_block, 2> state;

auto pipe =
    cuda::make_pipeline(cuda::this_thread_block(),
        &state);
```

为了预热 pipeline，producer 先取得一个可写 stage，并提交从 global memory 到 `tiles[0]` 的异步复制，然后 commit 该 producer phase，如下所示。在满足硬件与 descriptor 等条件时，底层可以使用 TMA 完成 bulk、strided 或 multidimensional transfer；这里的 `bytes` 表示本次复制的数据量。

```
pipe.producer_acquire();
cuda::memcpy_async(tiles[0],
                   global_ptr + 0 * TILE_SIZE,
                   cuda::aligned_size_t<32>{bytes},
```

```
                    pipe);
    pipe.producer_commit();
```

使用该数据之前，consumer 必须等待对应 stage 的复制完成；处理完毕后再 release 该 stage，使其能够被后续 producer phase 复用：

```
    pipe.consumer_wait();
    processTile(tiles[0]);
    pipe.consumer_release();
```

在 main loop 中，两个 buffer 交替承担 consumer 与 producer 的角色：先通过 `pipe.consumer_wait();` 等待前一个 tile 就绪，再调用 `processTile()` 处理它；处理结束后调用 `pipe.consumer_release()` 释放相应 stage。另一侧则通过 `pipe.producer_acquire()` 取得可写 stage，调用新的 `cuda::memcpy_async()` 将数据送入另一个 tile，最后用 `pipe.producer_commit()` 提交该 phase。随后重复这一过程。实际代码必须让所有参与 thread 遵守一致的 collective 调用与 phase 生命周期，不能仅把这些调用任意拼接。

在 `tile0` 与 `tile1` 之间进行乒乓切换后，每次新的 `memcpy_async` 都有机会与前一个 buffer 上的 `processTile` 重叠。若底层采用 TMA，部分搬运工作不再通过常规 per-thread LD/ST 指令完成，因此 SM load/store 路径的压力可能下降，并为计算留下更多 instruction issue 机会。不过，这种方法不会自动消除算法本身的重复 global load，也不会取消必要的 pipeline synchronization；收益应通过实际 counter 与执行时间验证。

从 global memory 到 shared memory 的异步预取，可以尝试用计算遮蔽数据传输延迟：thread 处理此前已经载入的数据时，pipeline 同时预取后续数据。

这种模式尤其适合具有可预测 tile 流、且存在足够计算可与传输重叠的 memory-bound loop 和 tensor computation。例如在现代 GPU 上处理当前 matrix-multiply tile 时，可以并行搬入下一个 tile；若计算过短、带宽已经饱和或 occupancy 因额外 shared memory 而下降，实际收益则可能受限。

> 在支持 TMA 的架构上搬运 2D 或 N-dimensional tile 时，TMA 是 global memory 与 SMEM 之间 tiled bulk copy 的重要路径；TMEM 是另一种具有特定指令和用途的片上存储，不能与普通 SMEM 路径混为一谈。可以在 block scope 使用 `cuda::memcpy_async` 与 `cuda::pipeline` 表达异步复制，但 Hopper/Blackwell 是否生成 TMA（`cp.async.bulk.* family`）指令，仍取决于具体 API overload、方向、对齐、shape、stride、destination 与 toolchain。不要仅凭源码 API 名称推断实际指令路径。

这种方法把许多分散的 global read 重组为较规则的 bulk global copy，再从 shared memory 多次复用数据。考虑到 DRAM 与片上 SRAM 的带宽和延迟差距，在具有充分数据复用的算法中通常更有利。表 7-7汇总了原文给出的 TMA 双缓冲实现前后 Nsight Compute 指标；这些数值缺少完整复现条件，应视为示意对照。

表 7-7. 对比 naive kernel（无预取）与 TMA 加速的双缓冲

指标	之前（无预取）	之后（异步预取）
全局内存加载效率	23%	99%
每个 request 的平均 sector 数	6.4	4.0
DRAM throughput（peak 百分比）	25%	90%
SM 活跃百分比	62%	100%
Kernel 执行时间	18 ms	7 ms

在原文给出的这组示意数据中，SM Active % 接近 100%，表示采样期间 SM 在近乎所有 cycle 都有 active warp，但它并不等于这些 warp 每个 cycle 都在执行有效计算。Global Memory Load Efficiency 从 23% 提升到 99%，按该指标的通常定义，说明 requested byte 与实际传输 byte 的比例大幅改善；具体含义仍应以所用 Nsight Compute 版本中的 metric 定义为准。

每个 request 的平均 sector 数从 6.4 降至约 4.0。对于常见的 32-byte sector 口径，4 个 sector 合计为 128 byte，但这本身并不能证明每个 request 都“整齐映射”为单个 cache line，还需结合 request 类型、地址分布和相关 sector counter。表中 DRAM throughput 从 peak 的 25% 上升到 90%，执行时间从 18 ms 降至 7 ms，按算术约加速 2.57×。这些数字可以说明某个具体实现可能通过 bulk copy 与 shared-memory 乒乓缓冲改善 overlap，但由于原文未提供复现实验配置，不能据此推导 TMA 的通用加速比或断言已隐藏大部分 DRAM latency。

NVIDIA CUDA `pipeline` API 与 TMA 可以视为 hardware-software codesign 的一个例子：高级 pipeline abstraction 能够表达异步 producer/consumer 操作，而支持的底层实现可将适合的 bulk 或 multidimensional copy 映射到 TMA。不过，Pipeline API 并不只对应 TMA，任意 `cuda::memcpy_async` 也不必然生成 TMA 指令。

API 与硬件复制能力相互配合，使开发者能够用较高层的 pipeline operation 表达接近硬件传输模型的工作流，并在依赖关系、资源和数据量允许时，让 memory movement 与 compute 有效重叠，从而改善性能。

> 为 global-to-shared tiling 编写 CUDA kernel 时，通常应优先考虑目标 CUDA 版本中受支持、抽象层次合适的 API，其中包括 CUDA Pipeline API（`cuda::memcpy_async`）。较高层 API 可能在条件满足时利用 TMA 执行 bulk、strided 或 2D/3D transfer，也可能支持结合 thread block cluster 的 TMA multicast；但 multicast 需要相应架构、cluster launch、shared-memory/descriptor 配置和专用路径，并非所有调用都会透明启用。升级 toolchain 后仍应检查 generated code、correctness 与性能，不能假定这些能力无需任何代码或配置变化即可“免费”获得。

总之，当 memory access 限制 kernel 性能时，可以把合理 tiling、双缓冲和 TMA 异步预取结合起来，尝试卸载并重叠数据搬运。将 tile 暂存到 shared memory，并正确配合 `cuda::memcpy_async`、`pipe.producer_commit` 和 `pipe.consumer_wait`，可以表达 global-to-shared 的异步复制；满足 TMA 条件时，底层可将相应 bulk 或 multidimensional transfer 交给 TMA。

使用 TMA 卸载部分 memory transfer，可能缓解 SM load/store instruction path 的压力，并帮助 compute pipeline 保持繁忙。需要注意的是，TMA 主要负责 global memory 与 shared memory 之间的特定搬运；数据进入 shared memory 后，kernel 对 shared memory 的常规访问仍由 SM 指令及相关片上资源完成。在 Blackwell 的高带宽 HBM3e 系统上，这些技术有机会隐藏 DRAM latency 并提高 throughput，但 kernel 是否从 memory-bound 接近 compute-bound，仍取决于 arithmetic intensity、传输重叠、bandwidth、stage 数、occupancy 与实际 profiler 结果。

关键要点

通过 coalescing、数据复用和异步传输优化 GPU memory access pattern，有可能缓解 kernel 的 memory bottleneck，使其更接近硬件能力上限。合理组织 thread、使用 shared memory、避免 bank conflict 等看似局部的代码调整，有时会带来显著性能收益，但效果取决于 workload、架构和实际瓶颈。以下是本章的关键要点：

全局内存合并访问

> 当一个 warp 的访问覆盖尽可能少的 memory sector/cache line 时，就形成了较好的 coalescing。应根据元素宽度和访问模式组织数据与 thread index，使相邻 active lane 尽量访问连续且正确对齐的地址。对于 32 个 lane 各读取一个连续 FP32 word 的完整 warp，常见 32-byte sector 口径下最低通常是 4 个 sector，但不同访问宽度、active mask 和 request 类型的最优值并不固定为 4.0。使用相应版本的 Nsight Compute 时，可以结合 `sm__sass_data_bytes_mem_* counters` 和 `gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed` 等实际可用 metric，分析 requested/actual byte、sector 和 DRAM throughput，从而优化 memory coalescing。

向量化加载/存储

> 可以使用 `float4` 等内置 vector type 表达 16-byte vector access。对于 Blackwell 和相应 CUDA toolchain，只有在类型、指针和 offset 的 32-byte alignment 均可证明，且目标指令路径支持时，才考虑 `double4` 或自定义 `alignas(32) { float v[8]; }` 等 32-byte per-thread access。Vectorization 可能减少每 byte 的 instruction 数，但不会让任意 warp request 都保持 4.0 sectors/request；transaction 数会随总请求 byte、对齐和地址覆盖范围变化。`cudaMalloc` 返回的 base address 通常满足较高对齐要求，但后续 pointer offset 仍可能破坏 `float4` 所需的 16-byte alignment。应检查 PTX/SASS 和 profiler，而不是仅由源码类型推断实际 vector instruction。

避免 bank conflict

针对具体 lane address pattern 调整 shared-memory layout。例如，在典型 FP32 transpose 的列访问中，可把每行从 32 个 float pad 到 33 个，以打散同 bank 的 stride；这并不能保证所有访问模式下任意两个 thread 都不命中同一 bank。消除主要 bank conflict 可以减少 replay/wavefront，但 shared-memory throughput 仍受 instruction、带宽和其他资源限制。Swizzling 也可减少某些模式的冲突，不过必须与生产者、消费者及 Tensor Core layout 保持一致，且不一定总比 padding 更节省或更快。

Shared-memory tiling 与数据复用

把 working set 分块暂存到片上 shared memory，例如使用 32 × 32 matrix tile，使每个 block 在一个 tile iteration 中协作加载所需元素并多次复用。不同 block 仍可能重复读取同一输入数据，cache 也会影响实际 DRAM traffic。Tiling 可提高 arithmetic intensity，但是否使 kernel 转为 compute-bound，需要结合 shared-memory/register 占用、occupancy 和 Roofline/profile 数据判断。

只读数据与 cache

对于只通过某个 pointer 读取且确实不存在相关 alias 的 lookup table 或 coefficient，可以使用 `const __restrict__` 向 compiler 提供语义信息；错误的 restrict 承诺会导致未定义行为。Compiler 是否采用特定 read-only/cache load path 取决于架构与代码生成。Warp-uniform access 有机会通过 broadcast 和片上 cache 减少 transaction，但 cache miss 时仍可能访问更低层 memory，延迟也不是固定值。

使用 stream 重叠 host–GPU copy

需要稳定重叠 H2D/D2H transfer 与 kernel execution 时，通常应使用 page-locked（pinned）host buffer，并通过显式 stream 和 event 调用 `cudaMemcpyAsync`。实际 overlap 还要求设备 copy engine、传输方向、stream dependency 和资源并发条件允许。对于 pageable memory，CUDA runtime 可能使用内部 staging 或表现出同步行为，语义和 overlap 能力会随方向及实现变化，不能简单视为所有情况下都“禁用 DMA overlap”。应确认实际使用的 host-memory 类型，并用 timeline 测量 PCIe、NVLink 或其他互连上的传输与计算是否真正重叠；可再次通过 `cudaMemcpyAsync` 的实测行为确认，观察到的速率会随该配置显著变化。

使用 TMA 与 Pipeline API 异步预取

可以用 C++ libcu++ barrier 与 pipeline API（`cuda::barrier` 和 `cuda::pipeline`）配合 `cuda::memcpy_async` 表达 global memory 到 shared memory 的异步 bulk copy。在目标架构以及 API overload、alignment、shape、direction 和 scope 等要求满足时，底层可能使用 TMA（包括相应的 `cp.async.bulk.tensor` 路径）。结合双缓冲，可让受支持的 strided 或 multidimensional copy 与计算重叠，并可能降低 SM LD/ST instruction path 的压力，使 SM 有更多机会执行计算。

`libcu++` pipeline API 与 TMA 仍在演进。对于 two-stage ping-pong buffer，应正确使用 staged operation，例如 `producer_acquire()`、`producer_commit()`、`consumer_wait()` 和 `consumer_release()`，并维持一致的 phase 生命周期。除非确实

需要且 API/硬件支持 cluster 或更大 scope 的协作，否则通常使用 block-scoped pipeline（如 `cuda::thread_scope_block`）或 block-scoped barrier（如 `cuda::barrier<cuda::thread_scope_block>`）；具体可用 scope 需以目标 CUDA 文档为准。

以 profiling 指导优化

使用目标版本 Nsight Compute 中实际可用的 Global Memory Load Efficiency、每 request 平均 sector 数、shared-memory wavefront/conflict、SM Active % 和 warp stall reason 等 metric。还应查看 Nsight Systems timeline，验证 transfer 与 compute overlap、发现宏观空隙；但具体 bank conflict 等微架构原因通常需要 Nsight Compute counter 才能归因。每项优化都应以同一 workload 的正确性和执行时间为最终验证依据。

总结

通过合并 global-memory load、针对实际访问模式避免 bank conflict、进行 tiling 与 vectorized fetch、合理利用 cache，并在适用时采用 TMA 异步预取，可以逐项缓解重要的数据搬运瓶颈。它们并不保证 SM 自动达到 full speed；最终效果取决于 kernel 的真实瓶颈和各项资源之间的权衡。

本章使用 Nsight Compute 与 Nsight Systems 分析 warp 在何处等待数据，并验证优化是否减少 stall 和无效 transaction、提高 sustained bandwidth。应根据工具各自的观测层级解释结果：Systems 适合查看时间线和并发，Compute 适合分析 kernel 内部 counter。调优新 kernel 时，始终以可复现 benchmark、正确性检查和相应 profiler 证据闭环验证每一步。

下一章将介绍 CUDA 与 GPU 编程中一些基本的 latency-hiding 技术，包括调节 occupancy、提高 warp efficiency、避免 warp divergence，以及发掘 instruction-level parallelism。

第 8 章. Occupancy 调优、Warp 效率与指令级并行

现代 GPU 加速 workload 正不断逼近硬件极限。以特定 Blackwell data-center GPU 为例，其两个接近 reticle limit 的 compute die 通过总带宽约 10 TB/s 的 NV-HBI 互连，并配有 126 MB L2 cache；这些数字并不适用于所有 Blackwell 产品。此类硬件设计会显著改变 memory 与 compute 之间的权衡，也会改变合适的 occupancy 区间，因此 profiling 与优化比以往更加重要。在前述 memory optimization 基础上，本章将转向更高级的 latency-hiding 方法和 throughput 优化，以更充分地利用现代 GPU。

我们将首先识别性能瓶颈，再系统地选择并验证相应优化策略。核心主题包括调节 occupancy、提高 warp efficiency，以及增加 instruction-level parallelism（ILP，指令级并行）。

学完本章后，你将能够识别 GPU 利用不足的根因，并组合使用适合的优化方法。本章也会为后续更高级的技术打下基础，例如 kernel fusion，以及使用 CUDA Graph 和 CUDA stream 等 primitive 构建 pipeline；这些内容将在后续章节介绍。

虽然本章主要讨论 CUDA C++ 等较高层语言和 PyTorch 等 AI framework，但 profiling 与调优原则贯穿整个 software/hardware stack。理解底层硬件性能，对于诊断较高层 abstraction 难以彻底解决的瓶颈仍然至关重要。

分析并诊断 GPU 瓶颈

开始优化之前，必须先识别代码中的瓶颈，确定究竟是哪一种硬件或软件资源限制了性能。现代 NVIDIA GPU 十分复杂，性能下降可能源自 memory bandwidth、memory latency、instruction throughput、同步开销、并行度不足或 host-device transfer 延迟等多种因素。

NVIDIA profiling 工具生态包括 Nsight Systems（命令行工具 `nsys`）和 Nsight Compute（命令行工具 `ncu`）。Nsight Systems 采集 CPU thread、GPU kernel 和 memory transfer 的系统级 timeline；在相应配置和环境下，它还可以采集 Python backtrace 与 sampling 数据。

结合 PyTorch profiler 和各种 visualization 工具，Nsight Systems 与 Nsight Compute 可以帮助诊断 kernel 性能瓶颈、分析 Roofline 图，并衡量迭代优化的实际效果。两者观测层级不同：Systems 侧重系统 timeline，Compute 侧重单个 kernel 的硬件 counter。

Nsight Systems Timeline 视图

Nsight Systems timeline 视图有助于发现 concurrency 问题、transfer 开销和 idle period。例如，可以运行下面的命令，采集详细 timeline，用于观察 kernel launch 是否重叠、CPU preparation gap、data-transfer timing，以及由 NVTX 标记的 range：

```
nsys profile \
    --trace=... \
    --capture-range=... \
    --force-overwrite=true \
    <application>
```

此外，Nsight Systems GUI 支持交互式检查 timeline。它可以可视化 CPU thread、GPU kernel 和用户定义的 NVTX range，并通过缩放和平移进行细致分析（见图 8-1）。

对于复杂应用，NVTX annotation非常有用。可以在代码中用 NVTX range 标记关注区域，再让 Nsight Systems 按 range 控制或分析采集。CUDA profiler Start/Stop API 也可用于 capture control，但在 framework workflow 中，NVTX range 往往更便于标注业务阶段。例如，可以通过 PyTorch 的 `torch.cuda.nvtx` 使用 `NVTX` range push/pop，标记“forward pass”和“backprop”等阶段。这样能够清晰划分关键计算区段，使 timeline 更容易解释；是否能严格按 NVTX range 启停采集，则取决于具体 `nsys` capture 配置。

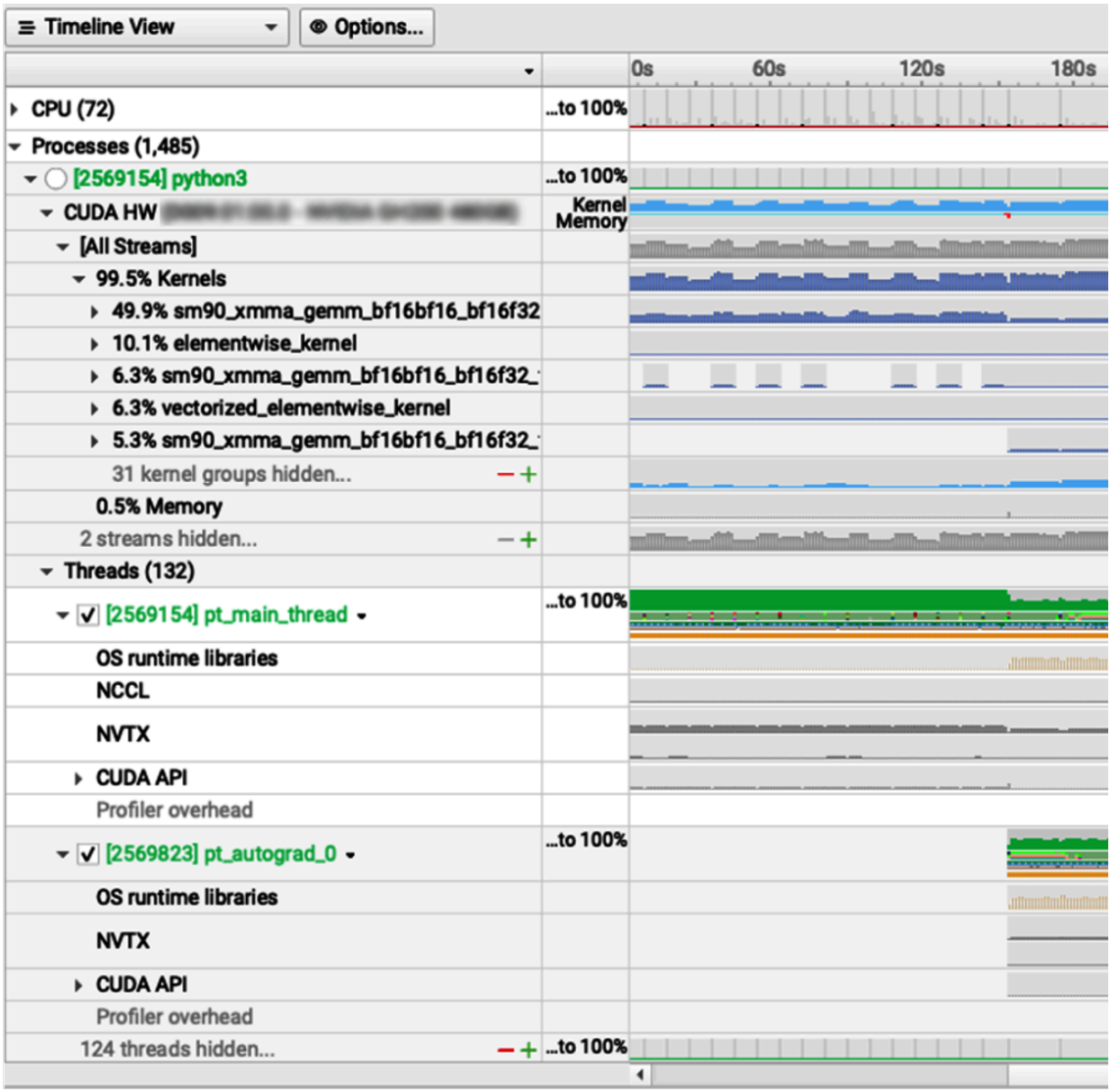


图 8-1. Nsight Systems 交互式 UI（来源：https://oreil.ly/YEiWS）

分析并调优 Data Pipeline

如前所述，Nsight Systems 可以从较高层观察完整 pipeline，包括数据加载和处理步骤。例如，在 training loop 中，如果 CPU 正忙于准备数据或执行 augmentation，而 GPU 处于 idle 状态，那么瓶颈可能并不在 GPU kernel，而在 data pipeline 本身。

在真实且具有代表性的数据集上确认 data pipeline 是瓶颈后，可以调节 data loader 的并发度。在 PyTorch 中，这通常涉及 DataLoader worker process 数、prefetch 和 persistent-worker 等配置，而不只是 thread 数。还可以通过 producer/consumer buffer 让 CPU preprocessing 与 GPU compute 重叠，或者在收益大于 transfer 和 GPU resource 开销时，把更多 preprocessing 移到 GPU。每项调整都应同时观察 throughput、GPU idle gap、CPU 利用率和 host-memory 占用。

> 务必确认看似“GPU 性能问题”的现象，是否实际上由 kernel execution 上游或下游的环节引起，例如数据加载、CPU preprocessing、同步、通信或输出处理。

通过 profiling 建立清晰的端到端视图后，就可以进一步分析具体瓶颈。接下来的几个小节将先介绍 Nsight Compute、Roofline、PyTorch Profiler 和 profiler-guided analysis，再进入 occupancy 调优等核心优化技术。足够的 eligible warp 有助于隐藏某些 latency，但更高 occupancy 并不必然带来更高 throughput，后文将进一步讨论这种权衡。

Nsight Compute 与 Roofline 分析

另一方面，Nsight Compute（`ncu`）用于采集单个 kernel 的深入 metric。例如，它可以报告 achieved occupancy、每 cycle 发出的 warp instruction、memory throughput、execution-pipeline utilization 等。把这些 counter 与 kernel 配置、输入和执行时间结合起来，可以建立较完整的 kernel-level 性能画像；但它不能取代 Systems 提供的端到端系统视图。

这些 metric 按 memory、compute、scheduler、launch 和 throughput 等 section 组织。Nsight Compute 的自动分析规则能够标记某些低利用率、branch divergence 或 stall 现象，并给出优化提示；规则和 metric 会随 GPU architecture 与工具版本更新。它们可以帮助形成 memory-bound、latency-bound 或 execution-pipeline-bound 等假设，但 FLOP/byte、stall reason 或单个 utilization 指标通常不足以独立证明根因，还需交叉验证。

Nsight Compute 还提供 Roofline 分析与自动指导，可以把 kernel 的 achieved FLOP/s 和 arithmetic intensity 投射到相应 hardware ceiling 上，帮助判断它与 memory-bandwidth roof 或 compute roof 的距离。

Roofline model 用 memory-bandwidth ceiling 与 compute-throughput ceiling 的最小值，描述给定 arithmetic intensity 下的性能上界。Nsight Compute 可以按其支持的 precision 和 memory-byte 口径计算 kernel 的 achieved FLOP/s 与 arithmetic intensity（FLOP/byte）。若 arithmetic intensity 位于 ridge point 左侧，且测得性能接近相应 memory roof，才有较强证据说明 bandwidth-bound；接近水平 compute

roof 则表明 compute throughput 可能是主要限制。仅仅“落在 memory roof 下方”不能证明 memory-bound，因为几乎所有未达到 ceiling 的点都在 roof 下方，latency、dependency、occupancy 或 instruction mix 也可能造成差距。应确认 FLOP 计数、byte 所属 memory level 和 precision，再与目标硬件的相应 roof 比较。示例 Roofline 图见图 8-2。

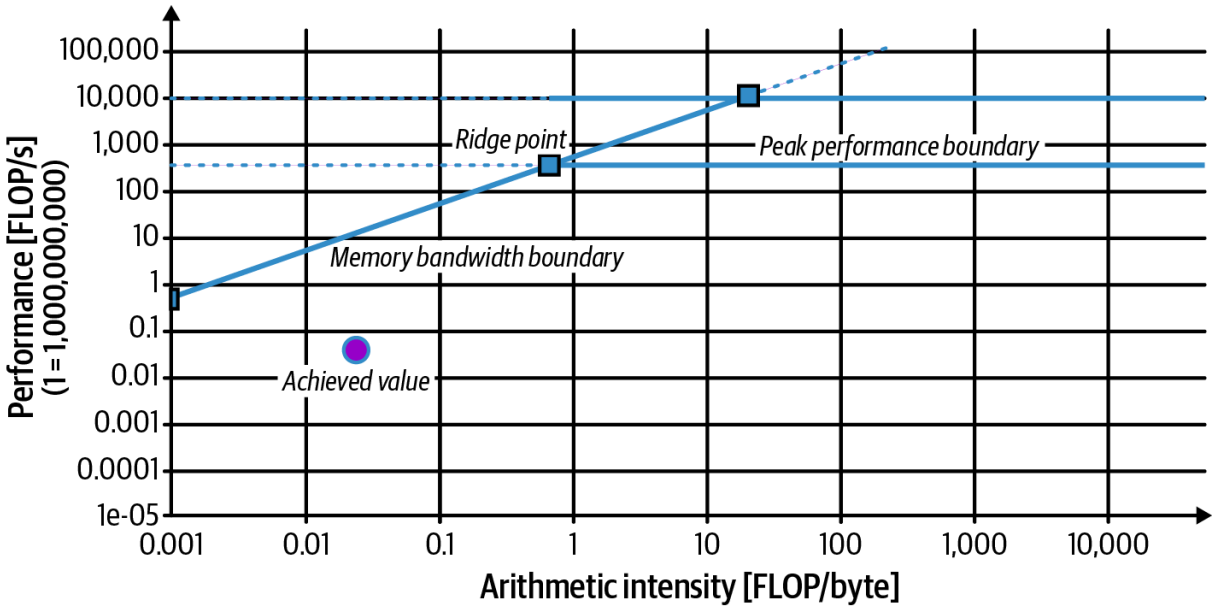


图 8-2. Nsight Compute UI 中显示的 Roofline 图（https://oreil.ly/wUbIz）

一种常用方法是先用 Nsight Systems 在 timeline 中找到耗时较多的 kernel、同步空隙或其他瓶颈操作，再用 Nsight Compute 深入分析选定 kernel。这个从系统级视图逐步下钻到 kernel-level counter 的两阶段 workflow，是处理复杂 GPU 性能调试与调优的常见方式。采集 Compute counter 可能需要 replay 并扰动执行，因此应使用代表性但受控的 workload，并把结果与未插桩基准时间对应起来。

PyTorch Profiler 与可视化工具

使用 PyTorch 等高级 framework 时，`torch.profiler` API 可以在 model training 或 inference 期间采集 operator、CPU 与 CUDA activity 等性能数据。PyTorch Profiler 底层使用 Kineto 组织相应的数据采集与 trace。

Kineto 可与 CUDA Profiling Tools Interface（CUPTI）后端集成，采集 operator execution time、GPU kernel launch 和 memory copy 等 activity；某些扩展配置或后端还可提供额外性能数据，但不能把默认 PyTorch Profiler trace 等同于 Nsight Compute 的完整 hardware-counter 集合。Kineto 也可在受支持的平台和配置下结合 Linux `perf` 等 CPU 数据源。它把事件关联为按时间排序的 trace，可通过 PyTorch Profiler/TensorBoard 相关界面，或将 Chrome trace 格式导入 Chrome tracing/Perfetto 等兼容 viewer 查看。不同 trace 格式不能假定可被 Nsight Systems GUI 直接互换导入。示例 Chrome trace 可视化见图 8-3。

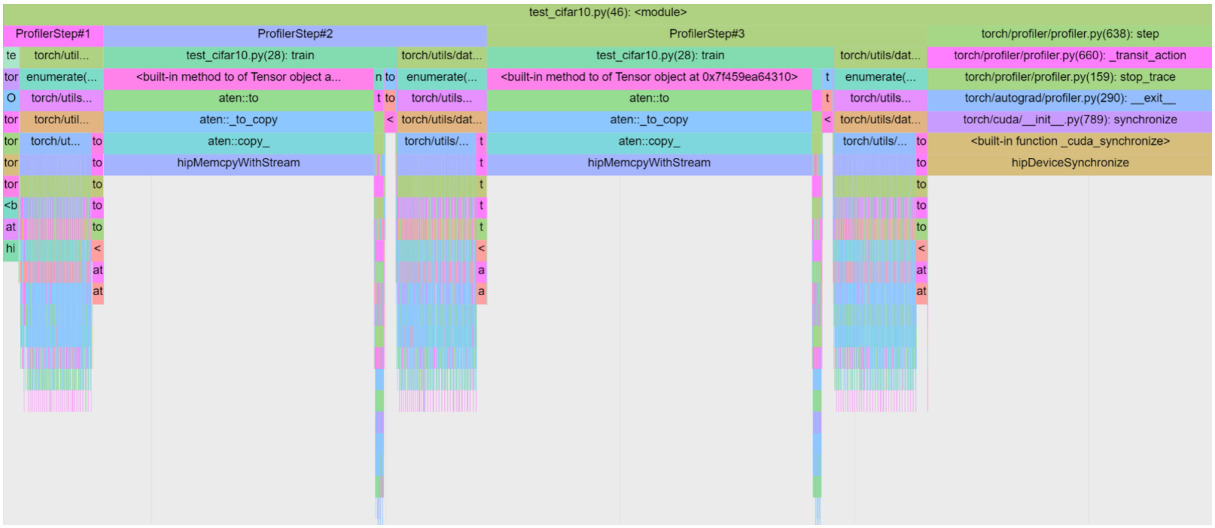


图 8-3. PyTorch Profiler 生成的 Chrome trace 可视化

PyTorch Profiler 可以帮助定位 model code 中耗时较高的 operation。例如，可以用 `torch.profiler.profile(..., with_flops=True, profile_memory=True)` 分析 training loop，记录 profiler 所跟踪的 memory allocation，并为 matrix multiplication 等受支持 operator 估算 FLOP。这里的 FLOP 是 operator-level 公式估算，而非每个 kernel 的 hardware counter；memory profile 也不等同于完整的 allocator lifetime、峰值 device memory 或 DRAM traffic。该集成有助于把 PyTorch model code 与底层 CUDA activity 对应起来。

现代版本的 Nsight Systems在受支持的平台与配置下可采集 Python call-stack sampling，并可能提供 PyTorch 相关 trace domain，从而把 framework activity 与系统 timeline 关联起来。具体功能名称和支持范围随版本变化。Nsight Compute 可将 CUDA C/C++ source 与 PTX/SASS 关联；编译 device code 时使用 `-lineinfo` 可提供 source-line mapping，但也必须保留相应 binary/source 信息。要关联 Python model phase 与 kernel，可以通过 `torch.cuda.nvtx` 添加 NVTX range。Nsight Compute 的 source view 还可显示 instruction mix 和 throughput/stall 相关数据，不过 source line 上的归因是采样或 counter 映射结果，不应理解为该行单独造成全部瓶颈。

> 过去，Python 信息关联能力有限，降低了部分 PyTorch 开发者使用 Nsight Systems/Compute 的意愿。当前版本在相应平台和配置下已经改进 Python、NVTX 与 framework activity 的关联，因此值得结合 PyTorch Profiler 使用，以获得更完整的系统级与 kernel-level 分析。

本节给出一套诊断 GPU 瓶颈的 workflow，说明如何解释关键 profiler metric，以及它们能为瓶颈类型提供哪些证据。后文将用 Nsight Compute 深入分析 kernel，包括 warp stall 与 memory throughput；同时用 Nsight Systems 分析 application-level concurrency 和 CPU-GPU overlap。

Profiler 引导的分析

关键在于用这些观测结果指导下一步实验，因为不同瓶颈需要不同优化。可以参考 Nsight Compute 的 guided analysis rule；根据工具版本和采集 section，它可能提示 memory workload 较高、L2 transaction 相对计算量较多，或 issue stall 与某条 execution pipeline 的 contention 相关。这些提示应视为待验证的假设，而不是固定名称或已经成立的根因结论。

Nsight Compute 会针对包括 Blackwell 在内的新架构更新 metric、section 和分析规则。它们可以把注意力引向 memory throughput、instruction dispatch、dependency latency 或 latency hiding 等方向，缩小需要检查的 counter 与代码范围，从而加快诊断；但最终仍需通过受控实验，例如改变 data size、访问模式、launch configuration 或 instruction mix，并观察执行时间是否按预期变化，才能确认瓶颈。

> Nsight Systems 与 Nsight Compute 会持续增加对新 GPU architecture、metric 和 workflow 的支持。迁移到新一代 GPU 时，应使用与目标 driver、CUDA Toolkit 和 GPU 相兼容的工具版本，同时记录版本与采集配置；“最新”版本也不意味着旧 metric 名称、阈值和报告可以直接横向比较。

使用 Nsight Compute 分析 Warp Stall Reason

如第 6 章所述，NVIDIA GPU 以 SIMT 方式执行 warp，每个 warp 通常包含 32 个 thread。当 warp scheduler 无法从某个 warp 发出下一条 instruction 时，profiler 会按采样时的 warp state 对原因分类。这里的 “stall” 是 scheduler-level 状态，不一定表示整个 SM 或 kernel 停止工作，因为 scheduler 可能从其他 warp 发射 instruction。

Nsight Compute 的 Warp State Statistics 可展示 kernel 中各类 warp state，通常也被称为 *warp stall reason*。这些数据有助于缩小调查范围，但单个类别往往对应多种上游原因，不能只凭最高百分比直接确定限制因素。

常见类别涉及 memory dependency、execution dependency、pipeline throttle、barrier，以及 eligible warp 未被选中等状态。具体 metric 名称和定义随 GPU architecture 与 Nsight Compute 版本变化。下面逐项讨论其中若干类别。

Memory-Related Stall 分析

当 warp 的下一条 instruction 依赖尚未完成的 L1TEX operation 时，Nsight Compute 可能报告较高比例的 “Stall: Long Scoreboard”。Scoreboard 跟踪尚未满足的 instruction dependency；*Long Scoreboard* 常与 global/local/texture 等经 L1TEX 路径的 load dependency 有关，但不能由此断定请求一定到达 global DRAM，因为它也可能命中 cache，且 local-memory spill 等访问也会出现该状态。示意见图 8-4。

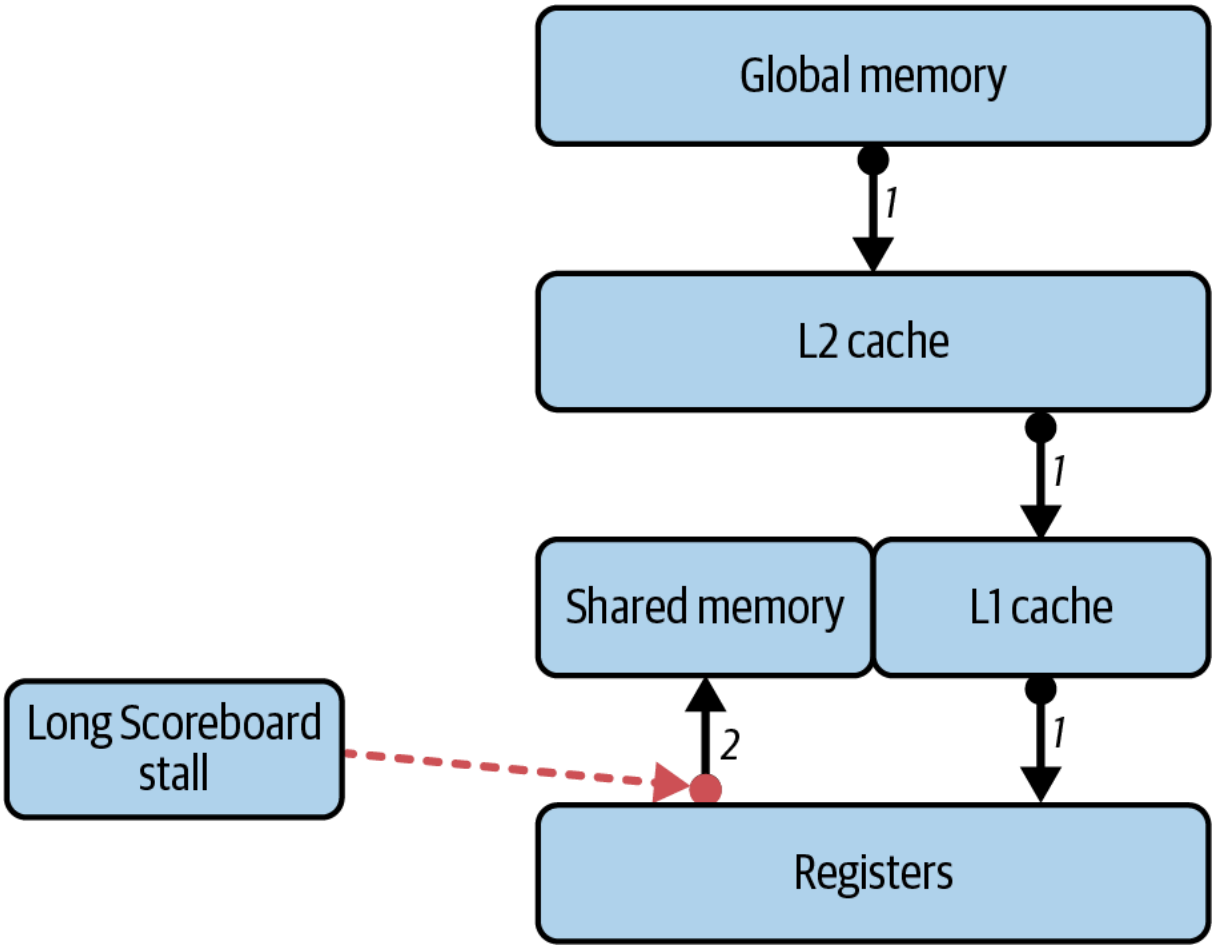


图 8-4. 等待经 L1TEX 路径的高延迟 memory dependency 时出现 Long Scoreboard stall（例如 global/local-memory load，包括可能由 register spill 产生的 local-memory access）

Short Scoreboard stall通常表示下一条 instruction 正在等待 MIO（memory input/output）相关 dependency，例如 shared-memory operation，也可能涉及某些特殊数学或其他经 MIO pipeline 的 instruction，并不只等同于 shared memory 与 register 之间的“传输”。示意见图 8-5。

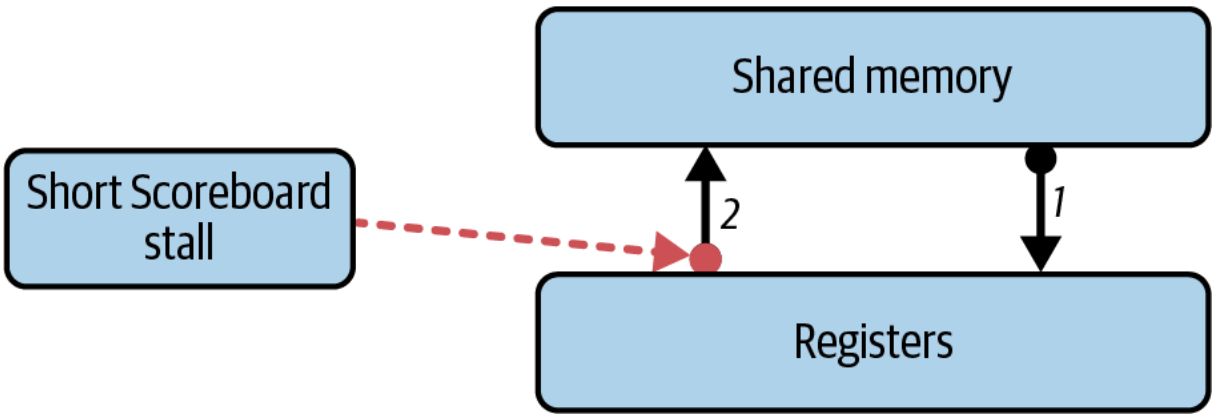


图 8-5. 等待较短延迟的 MIO dependency 时出现 Short Scoreboard stall，例如 shared-memory operation

带有 memory-pipeline throttle 含义的 metric（不同版本可能显示为 LG Throttle、MIO Throttle 等）表示相关 instruction pipeline 或 queue 无法接收更多请求，需要根据具体 metric 定义判断是 local/global、shared/MIO 还是其他资源。相比之下，“Stall:

Not Selected”通常表示该 warp 已 eligible，但 scheduler 当周期选择了另一个 eligible warp；它不是等待空闲 memory transaction slot 的专用状态，较高值甚至可能说明可供 scheduler 选择的 warp 充足。即使这些类别占比较高，也必须结合 DRAM/L2 throughput、sector、latency、instruction mix 和受控实验，才能判断 kernel 是否真正 memory-bound。

> 可以使用 Nsight Compute source view 定位与 scoreboard dependency、pipeline throttle 或 special-function-unit throughput 等现象相关的 instruction/source line，再结合 counter 与代码依赖关系验证根因。源码行上的标记是归因线索，不是单独的因果证明。

Execution-Dependency Stall 分析

如果 “Stall: Exec Dependency” 占比较高，通常表示warp 的下一条 instruction 正在等待前序 compute instruction 的结果。例如，后一条 instruction 依赖尚未完成的前一条 instruction。这可能表明 thread 内可用于覆盖 latency 的 instruction-level parallelism 不足，或存在较长的 arithmetic dependency chain；具体名称和涵盖 pipeline 仍应以目标 Nsight Compute 版本为准。

在一串相互依赖的 arithmetic operation 中，后续 instruction 必须等待前序结果，因此该 warp 暂时不能发射依赖 instruction。不过 scheduler 仍可能执行其他 warp。如果 execution dependency 占比较高，且 execution-pipeline throughput 未饱和，kernel 可能受 compute dependency latency 限制；还需结合 eligible warp、issue rate、instruction mix 和 occupancy 判断，不能仅由该比例推导 ALU latency bottleneck。

Execution Unit Contention 分析

如果某条 compute pipeline 的 throttle state 占比较高，且对应 FP32、FP64 或 Tensor Core pipeline 的 active/throughput utilization持续接近目标架构的相应 peak，kernel 才较可能受该 compute resource 限制。此时，eligible warp 有 instruction 可发射，但所需 execution pipeline 暂时不能接收更多 instruction。旧版或概括性文案可能称为 “Compute Unit Busy”“ALU pipe busy” 或 “Math Pipe Throttle”，现代报告常使用更具体的 pipeline metric。Throttle 表示采样时所需 pipeline 忙碌，不等于所有 math unit 在每个 cycle 都饱和；power 较高也不是 compute-bound 的充分证据。

其他 Stall Reason

此外还有 instruction fetch、barrier、dispatch、texture/L1TEX throttle 等类别。某些旧报告或概括性资料使用 “Memory Dependency” 一词，现代 Nsight Compute 往往通过 Long/Short Scoreboard 等更具体状态描述依赖；必须按目标版本定义解释，不能把多个类别简单视为同义词。

“Texture Throttle” 或相应 L1TEX throttle metric 表示相关 request pipeline 暂时无法接收更多 instruction，但不必然证明 cache capacity 或 bandwidth 是最终瓶颈。“No Eligible” 表示 scheduler 当周期没有可发射下一条 instruction 的 warp，可能源

自 dependency、barrier、低 residency 或 workload tail；“Idle” 的精确定义应按报告版本核对。两者都不能只归因于先前同步或 launch 的 warp 数不足。

实践中无需死记每个 stall reason。先查看占比较大的 scheduler state，再把它与 instruction source、throughput、eligible/active warp 和实验变化联系起来。Memory-related state 占比较高并不自动等于 bandwidth-bound，execution dependency 较高也不自动等于 ALU latency 是唯一瓶颈。

如果某条 pipeline 的 throughput 接近其适用 peak，且其他证据一致，kernel 可能受该 execution resource 限制。“Not Selected” 较高通常表示存在多个 eligible warp，而不是并行工作不足；No Eligible/Idle 较高才可能提示 latency hiding 不足，但仍需区分 dependency、barrier、tail effect 和 residency。Warp-state breakdown 的作用是缩小候选瓶颈，而不是单独完成诊断。

> Nsight Compute 可以按被分析的 kernel invocation 汇总这些 warp-state metric。应选择代表性的 input、shape 和 execution phase，并注意 replay 与采样开销。极小测试 kernel 的 cache、occupancy 和 tail behavior 可能与真实应用不同，因此在下结论前应从可复现的现实 workload 采集数据。

表 8-1列出若干常见 warp stall reason、典型解释和可供验证的优化方向。名称与建议均需结合目标 Nsight Compute 版本和实际 kernel。

表 8-1. 常见 warp stall reason 与优化提示

Warp 停顿原因	含义/可能原因	可验证的优化方向
Execution dependency	Warp 的下一条指令等待前序计算依赖。可能表示线程内 ILP 不足或依赖链较长。	在不显著增加 register pressure 的前提下增加独立工作，或重排 instruction。Loop unrolling 有时能暴露 ILP，也可能放大 code size/register usage。若 ILP 受限，可测试增加 eligible warp 是否能隐藏 latency，但不要只追求更高 theoretical occupancy。
Memory dependency（Long Scoreboard stall）	Warp 等待经 L1TEX 路径的 memory dependency。其他独立 instruction 是否可继续取决于实际 dependency graph；请求不一定到达 DRAM。	检查 coalescing、cache locality、spill 和实际 DRAM/L2 traffic。可测试更多 eligible warp、软件预取或 shared-memory staging 是否改善时间。在支持 TMA 且方向、shape、alignment、descriptor 与 scope 条件满足的 Hopper/Blackwell 路径上，TMA 可用于特定 bulk/multidimensional copy；它不是所有现代 GPU 或所有 memory copy 的默认方案。
Synchronization（barrier）	Warp 等待 `__syncthreads()` 或其他 barrier 的依赖条件满足。可能源自同步频繁、参与者工作不均或 barrier 前的长 latency。	在保持 memory-ordering 和 correctness 的前提下移除冗余 barrier、平衡工作，或采用语义匹配的更小 synchronization scope。Warp-level 或 cluster-level primitive 不能直接互换；必须保证参与集合、visibility 和 architecture support 正确。
Instruction fetch/issue	Warp 可能等待 instruction fetch，或因所需 dispatch/execution pipeline 暂时不可用而无法 issue；应按具体 stall metric 区分这两类原因。	若证据指向 instruction-cache/code-size 问题，可测试减少过度 unrolling 或拆分 kernel，但拆分会增加 launch 和 intermediate-memory 开销。若是 pipeline throttle，可调整 algorithm/instruction mix 或暴露跨 pipeline ILP，并检查 compiler 生成结果。
Not selected（scheduler）	Warp 已 eligible，但 scheduler 在该 cycle 选择了另一个 eligible warp。它不是	通常无需直接优化这一比例。它表明 scheduler 有其他工作，但并不证明 theoretical occupancy 很高或性能已最优。CUDA 一般不提供用于纠正

Warp 停顿原因	含义/可能原因	可验证的优化方向
	dependency stall，通常说明存在多个可选 warp。	所谓单 warp “独占 scheduler” 的通用 scheduler hint；应优化真实的 throughput、dependency 和 workload balance。

解释这些 stall reason 后，可以设计更有针对性的优化实验。例如，memory dependency 较高时，可分别测试增加 eligible warp、改善访问模式或异步预取，并用执行时间与 memory counter 判断哪一种假设成立。

Execution dependency 较高时，可以测试增加 ILP 或重排代码；barrier state 较高时，则检查同步必要性与工作不均。Profiler 的价值在于指导可证伪的实验，而不是根据单一比例机械套用优化。

检查 Achieved Occupancy 与 GPU Utilization

Profiler 报告的另一项重要 metric 是*achieved occupancy*，通常表示采样期间每个 SM 的平均 active warp 数，相对于该架构每 SM 最大 resident warp 数的比例。例如，若目标 GPU 每个 SM 最多 resident 64 个 warp，而 achieved occupancy 为 30%，则平均约有 19.2 个 active warp/SM。它与 theoretical occupancy、eligible warp 数和 issue efficiency 是不同概念。

较低 achieved occupancy 有时意味着可用于隐藏 latency 的 resident warp 较少，但并不自动等于硬件利用不足：具有较高 ILP 或受固定资源限制的 kernel 可能在较低 occupancy 下达到最佳性能。反之，接近最大 occupancy 也不保证高性能；memory bandwidth、instruction dependency、访问模式或其他 pipeline 仍可能成为瓶颈。

因此，简单增加 thread 数通常不足以解决问题，甚至可能改变 block residency 或增加资源竞争。应根据证据检查 memory coalescing、arithmetic intensity、warp-level efficiency、ILP 和 workload size。后续小节将讨论这些技术。

Nsight Compute 还会报告 theoretical occupancy limiter，说明在给定 block size、register allocation、static/dynamic shared memory 和架构 residency limit 下，哪项资源限制了每 SM 可驻留的 block/warp 数。例如，register limiter 表示编译后的每 thread register 用量及 allocation granularity 限制了更多 block/warp 同时驻留。

Shared-memory limiter 表示每 block 的 static/dynamic shared-memory allocation 限制了 residency。Thread/block limiter 则可能来自每 block thread 数、每 SM thread/warp/block 上限及其取整效应；grid 过小造成的 SM 利用不足属于 launch/workload coverage 问题，不应与单 SM theoretical occupancy limiter 混为一谈。

这些 limiter 为实验方向提供线索。例如，register 是限制因素时，可以检查是否能通过算法、lifetime 或 compiler option 降低 register pressure；`__launch_bounds__` 是对 compiler 的 launch constraint/hint，可能改变 register allocation 和 spill，并不保证

允许更多 block 或提高速度。Shared memory 是限制因素时，可以测试较小 tile 或较少 stage，但这也可能降低复用与 arithmetic intensity。

Occupancy 增加到一定程度后通常会出现收益递减；所需最低水平取决于 dependency latency、ILP、scheduler 和 workload。即使 10%–20% occupancy 也不必然性能较差，而追求 100% 可能因 register spill、较小 tile 或额外 instruction 反而减速。应以执行时间和 eligible-warp/throughput counter 判断最佳点。

因此，还应检查 occupancy 之外的 hardware utilization。Nsight Compute 可报告 achieved memory throughput、按支持口径估算/测量的 compute throughput、instruction/issue rate、issue-slot utilization 及各 execution pipeline 统计。只有将 metric 的分母、precision、clock 和 peak normalization 与目标架构对应起来，这些数字才能说明 kernel 距离某项 physical ceiling 有多远。

例如，如果 DRAM throughput 持续接近适用 peak，且 Roofline、sector 与时间变化也一致，那么 kernel 很可能受 memory bandwidth 限制；低 ALU utilization 本身并不足以证明这一点。反之，某条 compute pipeline 的 throughput 接近相应 peak而 memory throughput 较低，才支持 compute-resource bound 的判断。优化也不只限于降低 precision 或改用 Tensor Core：FP16、FP8、FP4 会改变数值精度、layout 和支持矩阵，Tensor Core 还要求适用 operation、shape、dtype 与 data movement。

如果 compute 与 memory throughput 都低，可能存在长 dependency latency、同步、launch/tail effect、低并行度、instruction issue 限制或其他 bottleneck。这可能与较低的*instruction-level parallelism*有关，也可能是 workload/grid 太小、resident/eligible warp 不足或 pipeline imbalance；需要进一步区分。

可以用 Roofline model 组织这项分析：它把 kernel performance（FLOP/s）画在 arithmetic intensity（FLOP/byte）横轴上，并用*compute roof*和由 bandwidth × arithmetic intensity 得到的*memory roof*定义上界。应明确采用的 precision 与 memory-byte level。

如果 FLOP/byte 低于 hardware ridge point，Roofline 上界由 memory roof 决定，但只有实际点接近该 roof 时，才能较有把握地判断 bandwidth-bound；远低于 roof 仍可能是 latency、dependency 或其他 ceiling。若 arithmetic intensity 较高但实际 FLOP/s 远低于 compute roof，则可能受 dependency latency、ILP、occupancy、instruction mix 或未被模型纳入的资源限制。

> 不少 GPU 世代中，特定低精度 compute peak 的增长速度高于 HBM bandwidth，使相应 compute-to-bandwidth ratio 上升，某些未提高 arithmetic intensity 的 kernel 因而更容易受 data movement 限制。但这一趋势依产品、precision、memory tier 和 workload 而异，不能概括为所有新 GPU 与所有 kernel 都会越来越 memory-bound。

实践中，应同时比较 kernel 的 memory throughput 与相应 memory-level peak，以及 compute throughput 与相应 precision/pipeline peak，并结合 arithmetic intensity、执行时间和 stall/issue 数据。这些比较可以帮助选择下一项关于 memory access、compute 或 parallelism 的实验，但不能仅凭两个百分比完成归因。下面分别讨论。

Kernel Memory Throughput 与 Peak HBM Bandwidth

Nsight Compute 可以报告 kernel 的 memory throughput（GB/s）及相对于相应 peak 的比例。若持续 achieved DRAM throughput 已接近目标设备在当前 clock/ECC/traffic mix 下可达到的 bandwidth，并且 Roofline 与实验结果一致，那么单纯增加相同数据上的 compute work 通常不会缩短时间；应减少每项有效工作对应的 DRAM byte，或增加能复用已加载数据的有效计算，从而提高 arithmetic intensity。

降低数据 precision 可以减少 operand/intermediate byte，但前提是数值精度与硬件路径允许；Tensor Core 主要提高适用 matrix/tensor operation 的 compute throughput，本身不保证减少 global traffic。Kernel fusion 可避免某些 intermediate tensor 写回和重新读取，从而提高 arithmetic intensity。增加 concurrency 或更多 warp 可能隐藏 latency，却不会改变算法的 FLOP/byte，因此应与减少 memory traffic 区分。

例如，DRAM throughput 达到某个经验阈值（如 peak 的约 80%）可以作为 bandwidth pressure 较高的线索，但不是跨 GPU、metric 和 workload 的固定判据。应确认读写合计方式、sustained peak normalization、采集时间，以及性能是否会随减少 DRAM byte 或改变 memory clock/bandwidth 而变化。

特定 data-center Blackwell GPU 配有 126 MB L2 cache，并用总带宽约 10 TB/s 的 NVIDIA High-Bandwidth Interface（NV-HBI）连接两个 compute die，对 software 呈现为一个逻辑 GPU。这些规格不适用于所有 Blackwell 产品，而跨 die/cache slice 的实际 locality、contention 和 latency 也不能只用总互连带宽概括。相较旧 GPU，更大的 L2 可能让部分 working set 获得更高 hit rate，但是否缓解瓶颈取决于 footprint、reuse、mapping 和并发 traffic。

可以使用 Nsight Compute memory chart 和相关 counter 区分 L1TEX、L2 与 DRAM traffic。较高 L2 hit rate 可能减少 DRAM byte，但不自动意味着 kernel 已 compute-limited：L2 bandwidth、latency、request efficiency 或其他 pipeline 仍可能限制性能。应同时检查各层 byte/throughput、hit/miss、sector 和 execution time。

> CUDA 的 L2 persisting-access 机制可通过 access-policy window 和相应 set-aside 配置，为选定访问提供 persistence hint；该能力并非 Blackwell 独有，也不保证 working set 永久驻留。可用容量、替换行为、并发 stream、MIG 模式和目标架构支持都会影响效果，应以 CUDA 文档与实测为准。

Kernel Compute Throughput 与 Peak GPU FLOP/s

Achieved FLOP/s 较低可能源自 occupancy/eligible warp 不足、memory dependency、instruction dependency、pipeline imbalance，或算法本身包含大量不计入 FLOP 的 instruction。例如，平均 active warp 较少或 compute pipeline 经常因 memory dependency 无法获得可发射工作时，kernel 可能远低于 compute roof；但 30% occupancy 本身不能证明这是原因。

可以结合 Nsight Compute 的 Occupancy section、Scheduler Statistics、Source Counters 和 pipeline metric 定位问题。Grid 应有足够 block 覆盖目标 SM，而每 SM resident warp 上限由具体架构决定；64 warp/SM 只是某些设备的示例。无需也通常无

法通过单个 block 填满该上限，实际目标是提供足够 resident/eligible warp，同时控制 register、shared memory 和 block-size 权衡。

> 同一 SM 内的 warp 共享该 SM 的 scheduler、register、shared memory 和 execution pipeline，并能在 scheduler 层面相互隐藏 latency，因此 occupancy 按 SM 定义和汇总。不同 SM 的 warp 不直接参与同一 warp scheduler，但仍可能通过 L2、HBM、interconnect、atomic 或其他 shared resource 相互影响，不能说它们“没有交互”。

还应检查 instruction issue efficiency 与各 execution pipeline throughput。特定 Blackwell device 集成许多 SM，因此每 SM 或 grid coverage 的利用不足会累积为较大的 device-level throughput 损失；双 die packaging 本身不会提高单个 SM 的 issue rate，还需考虑 workload 是否均匀覆盖所有 SM。

如果 achieved FLOP/s 中等或较高而 DRAM throughput 较低，kernel 可能以 compute 为主，也可能主要命中 cache，或受 compute dependency/pipeline throttle 限制。可以查看目标版本中对应的 execution-dependency state、eligible warp、issue rate 和 pipeline utilization，再通过重排独立工作等实验验证；单个 “Exec Dependency” 比例不能确认根因。

如果按相同 precision、dense/sparse 口径、instruction type 和实际 clock 计算的 compute throughput 接近相应可实现 peak，并且其他证据一致，kernel 才可视为受该 compute pipeline 限制。这既不证明所有 memory access 已经最优，也不意味着一定使用低精度 Tensor Core；不同 workload 可以在不同 execution pipeline 上接近各自 ceiling。

持续的高 compute utilization 可能使现代 GPU 达到 power、thermal、reliability 或 voltage/frequency policy 的限制，从而改变 clock。Profiling 与 benchmark 时应同时记录 power draw、clock 和 clock-event/throttle reason。下面的命令尝试实时查询这些字段，但具体字段名称与支持范围取决于 GPU 和 driver；应先用 nvidia-smi --help-query-gpu 核对：

```
nvidia-smi \
  --query-gpu=\
  power.draw,clocks.current.sm,clocks.current.memory,\
  clocks_event_reasons.active \
  --format=csv -l 1
```

在字段受支持时，该命令每秒输出一行当前 power draw、SM/memory clock 与 active clock-event reason。它有助于把降频与 power/thermal 等状态在时间上对应起来，但 1 秒采样可能漏掉短暂事件，`power.draw` 与 enforced power limit 也不能单独区分所有降频原因。

还可以使用 NVIDIA Management Library（NVML）API，通过独立的 NVML C 接口 `nvmlDeviceGetPowerUsage()` 和 `nvmlDeviceGetEnforcedPowerLimit()` 等函数进行程序化查询，也可使用相应的 NVML Python binding。它们适合自定义脚本和监控系统，但并不是 CUDA C++ runtime API，支持的 telemetry 仍依设备、权限和 driver。

总之，应结合 occupancy、eligible warp、warp-state reason、memory throughput、compute throughput、clock 和执行时间诊断瓶颈。目标不是让所有 metric 都“高”或强求 Tensor Core utilization，而是让 workload 所需的资源得到有效使用，并以正确性与端到端时间证明优化。

迭代 Profiling 并确定 Kernel 瓶颈

可以用四种常见状态组织 GPU 性能诊断：device underutilization、latency-bound、memory-bandwidth-bound 与 compute-resource-bound。它们不是互斥的物理定律，同一 kernel 可能同时受多个 ceiling 影响；但分别理解这些模式，有助于选择下一项实验。修复一个主要瓶颈后，另一个限制因素往往会变得可见。

Device underutilization 可能发生在 grid/problem 太小、kernel 之间存在 gap、并发度不足或 load imbalance 时。此时 compute 与 memory throughput 可能都低，timeline 也可能出现 idle 区间。增加可并行工作后，后续表现不一定必然转为等待 memory load，还可能暴露 compute、synchronization 或其他瓶颈。

Latency-bound 与 bandwidth-bound 都可能涉及 memory，但证据不同。Latency-bound kernel 的 achieved bandwidth 可能远低于 peak，同时 warp 因 dependency 缺少可发射工作。可以测试增加 eligible warp、ILP、prefetch 或 pipeline 是否改善 overlap；这些方法并非总能提高性能，还可能增加 resource pressure。

Memory-bandwidth-bound kernel 的 DRAM throughput 接近相应可实现 peak，且性能随 memory byte/bandwidth 变化。Compute pipeline 利用率可能较低，并常伴随 memory dependency state；原文所谓“不是因为 stall”并不成立。优化方向包括通过 tiling/fusion/cache reuse 提高 arithmetic intensity、改善 request efficiency，或在数值允许时降低 transferred byte。

Compute-resource-bound 表示某条 arithmetic/execution pipeline 的 throughput 接近相应 ceiling。Unrolling、software pipelining 或其他 ILP 技术只有在存在独立 instruction 且不过度增加 register/code-size 时才可能改善利用率。INT32 与 FP32 的并行 issue 能力取决于具体 GPU architecture、scheduler、execution pipeline 和 instruction mix，不能笼统断言所有现代 GPU 的“统一 core”都无法在同一 cycle 处理二者。

优化过程有时会经历 underutilized → latency-bound → bandwidth-bound → compute-bound 的变化，但这不是固定顺序，也不要求最终达到 compute-bound。每次改动后都应重新测量 correctness、时间和关键 counter，因为瓶颈可能移动、并存或重新出现。

结构化流程应从可复现的 baseline 开始：先用 Nsight Systems 查看 application-level concurrency、idle gap 和 transfer，再用 Nsight Compute 分析选定 kernel 的 warp state、occupancy、memory 与 compute pipeline，并用受控改动验证假设。

综合这些证据后，可以把主要问题暂时归入 memory bandwidth、compute resource、latency hiding 或 device underutilization 等类别。这里的 underutilization 指 workload 没有让足够多的 device resource 获得有效工作，而不只是“instruction issue 数不够”。表 8-2汇总了这四种诊断模式、常见信号和可验证的优化方向。

表 8-2. Memory-bound、latency-bound、compute-bound 与 GPU underutilization

主要限制因素	描述	Profiler 信号	可验证的优化方
Memory-bandwidth-bound	DRAM traffic 接近可实现 peak bandwidth，但每 byte 的有效工作不足以让 compute pipeline 达到高利用率。	DRAM throughput 接近匹配口径的 peak，performance 接近相应 memory roof；compute throughput 可能较低。	通过 tiling、f 或 cache reus 高 arithmetic intensity；改 coalescing/re efficiency；减 必要的 transfe byte，并验证执 间随 byte/bandwic 化。
Compute-resource-bound	某条 arithmetic/execution pipeline 成为主要限制；这不要求 memory latency 已完全隐藏，也不要求所有 ALU/Tensor Core 同时饱和。	按相同 precision、dense/sparse、clock 和 pipeline 口径计算的 throughput 接近相应 peak；memory throughput 可能较低。	减少 depende 与无效 instruction，ILP/unrolling 适合的 algorit 仅在数值、sha layout 和硬件 满足时使用低 precision、sparsity 或 specialized u fusion 也可能 register pressure。
Latency-bound	Dependency latency 未被其他 eligible warp 或独立 instruction 充分覆盖，导致 scheduler 缺少可发射工作；来源可为 memory 或 compute dependency。	Throughput 远低于相关 peak，并伴随 No Eligible、scoreboard/dependency state 或较低 issue rate。Not Selected 较高通常不是 latency-hiding 不足的证据。	在资源允许时减 多 eligible wa ILP、multipl accumulator、 pipeline 或 prefetch；同时 cache miss、 spill、barrier dependency chain，避免为 occupancy 引 多开销。
GPU underutilization	Problem/grid、并发或 workload distribution 没有为足够多 SM 提供有效工	Device/timeline coverage 低，compute 与 memory throughput 均低；可能出现 kernel 间	在语义和 mem capacity 允许 加 problem/b 或 block 数、

主要限制因素	描述	Profiler 信号	可验证的优化方
	作，device-level compute 与 memory resource 大量空闲。	gap、少量 active SM 或明显 tail。低 occupancy 本身不足以证明 device underutilization。	理小任务、改善 load balance，评估 fusion、persistent ker（第 10 章）和 stream concurrency（第 11 章）；每法都有额外资源度成本。

Memory-bandwidth-bound kernel 的性能受 memory throughput 限制；更准确地说，其 arithmetic intensity 较低，实际 performance 接近相应 memory roof，并且 DRAM throughput 接近可实现 ceiling。此时即使有足够 thread，也无法靠增加同类并发请求继续提高 bandwidth；应减少每项有效工作对应的 byte 或增加数据复用。

相对地，compute-resource-bound kernel 的某类 arithmetic unit，例如 FP32 pipeline 或特定 Tensor Core pipeline，throughput 接近匹配 precision/clock/dense-sparse 口径的 ceiling。Achieved occupancy、memory utilization 和 execution-dependency state 可以提供背景，但确认分类还需对应 pipeline throughput、Roofline 位置和受控实验。

Latency-bound kernel 的关键问题是 dependency latency 没有被其他工作充分覆盖。例如，warp 发出读取 `A[idx]` 的 global-memory load 后，依赖该值的后续 instruction 会被 scoreboard 阻塞；并非所有 32 个 thread 都以独立实体“停住”，也不是该 warp 在 load 完成前绝对不能发出任何 instruction。若存在由 compiler/scheduler 可见的独立 instruction，它们仍可能先执行。实际 latency 还取决于 cache/TLB 命中与 memory path，并非每次都是固定的数百个 DRAM cycle。

Warp scheduler 可以在某个 warp 等待 dependency 时选择其他 eligible warp。但如果 resident warp 较少，或所有 warp 都处于相似的 load→dependent-compute chain，scheduler 可能没有足够可发射工作。此时问题是独立 operation/eligible warp 不足以覆盖相关 dependency latency，而不一定是每个访问都属于 long-latency DRAM。

改善 latency hiding 的一种方法是在 resource 与 workload 允许时增加 resident/eligible warp，使一个 warp 等待时另一个 warp 可运行。增加 launch thread 数只有在 grid coverage 或 residency 确实不足、且不触发更高 resource pressure 时才有帮助；超过 block/SM limit 后不会继续提高 occupancy。

同样重要的是增加 warp/thread 内可见的独立工作。本章后文会详细讨论 ILP。简而言之，如果每个 thread 在使用结果前先发出两个相互独立的 load，例如分别读取 `A[idx]` 和 `B[idx]`，硬件有机会让多个 memory request 同时 outstanding。这体现了 instruction independence，并提高 memory-level parallelism；是否实际重叠仍取决于 compiler code generation、dependency、queue capacity 和地址路径。

在资源允许时，第二、第三个独立 load 可以在第一个 request 未完成时进入 flight；某个结果返回后，其依赖 compute 便可能与其他 pending request 重叠。不过，更多 outstanding load 会增加 register/lifetime 与 memory-system pressure，若 bandwidth 或 queue 已饱和则可能没有收益。

> 适度 unroll 小型 loop 有时可以暴露多个独立 load 或 accumulator，从而增加 ILP/MLP；但 compiler 可能已自动 unroll，进一步展开也可能增加 register pressure、spill 和 instruction-cache footprint。应检查 generated code 并实测。

提高 ILP/MLP 可能让每个 warp 在某项 dependency 未满足时仍有独立工作，从而改善 latency hiding 和 throughput；但不会保证“始终”有足够工作，也不一定把 kernel 转入理想状态。收益上限由 register、outstanding request、bandwidth、pipeline 和算法依赖决定，应以执行时间验证。

当 GPU 因 problem/grid 太小或任务间 gap 而 underutilized 时，增加 batch、合并小任务或提供更多 block 可能提高 device coverage。但更大 batch 会增加 memory 和 latency，更多 thread 也可能不改变 grid tail 或 per-SM residency；应先确认真正缺少的是 device-level work，而不是 dependency 或其他 bottleneck。

优化 Kernel

实践中的性能调优应遵循可证伪的 workflow：建立 baseline，识别主要 limiting regime，做一项针对性改动，再验证 correctness、时间和 counter。Memory-bound、latency-bound、compute-bound 与 underutilized 可能并存。

若 kernel 受 memory bandwidth 限制，应优先减少实际 memory traffic 或增加 data reuse，例如改善 coalescing/request efficiency、cache reuse 和 tiling。提高 occupancy 或 ILP 主要用于隐藏 latency，并不会降低每项工作的 byte，也无法突破已经饱和的 bandwidth。

若 kernel 受 dependency latency 限制，应提供足够的独立 work in flight，例如每 thread 的独立 operation、更多 eligible warp 或适用的 pipeline/prefetch。目标不是保证 scheduler “始终”有 ready warp，而是在 register、shared memory、queue 和 bandwidth 约束内改善 issue efficiency。

若某条 compute pipeline 已接近匹配口径的 ceiling，可以考虑更高效的 algorithm、减少 dependency/无效 instruction，或在数值与硬件支持允许时采用 FP16/FP8/FP4、Tensor Core 等路径。Fusion 可能减少 memory round trip，也可能增加 register pressure。若 device underutilized，则先确认是 grid coverage、task gap 还是 load imbalance，再决定是否增加 block、batch 或 concurrency；低 occupancy 本身不等于 device underutilization。

下面列出一组高级优化方向。把 GPU 调优作为科学过程时，每项工作都应包括：提出 limiting-factor 假设、应用范围明确的改动、保持 correctness，并测量结果。

缓解 memory-bandwidth bottleneck

若 arithmetic intensity 较低且 performance 接近 memory roof，可通过 tiling/shared-memory reuse 减少重复 global access，通过 fusion 避免不必要的 intermediate round trip，并改善 coalescing 和适用的 shared-memory layout（相关基础见第 6 章）。Compression 或 lower precision 只有在编码/解码成本与数值语义允许、且确实减少所测 memory level 的 byte 时才有益。目标是缩短时间，不必刻意把 workload “转为 compute-bound”。

进一步优化 compute-resource-bound workload

若某 compute pipeline 是主要限制，可在 register/code-size 可接受时使用 multiple accumulator、适度 unrolling 等 ILP 技术；检查 branch divergence（见第 6 章）及 instruction mix。Tensor Core 与 lower precision 只适用于支持的 operation、shape、layout 和 accuracy requirement；FP32 → FP16 → FP8 → FP4 不是普遍可行的线性升级路径，必须验证数值稳定性与实际 kernel selection。

为 latency-bound workload 增加可用并行性

若 dependency latency 未被覆盖，可测试更多 block/eligible warp、平衡 register/shared-memory residency、增加 ILP，或用 async copy/intra-kernel pipeline 重叠适用的数据搬运。多个 stream 只有在任务独立且硬件资源允许时才能实现 inter-kernel 或 copy/compute overlap（见第 11 章）。许多 tiny kernel 的 launch overhead 可通过 fusion、batching 或 CUDA Graph 等方式缓解；Cooperative Groups 是线程协作 abstraction，不会自动把多个 kernel 合并。

提高有效 GPU utilization

若 timeline 和 active-SM/grid-coverage 证据表明 device underutilized，应确保 grid/problem 能为足够多 SM 提供工作，并减少 task gap 与 tail imbalance。每个 element 一个 thread 并非错误，thread 工作量少也不意味着需要“更多 thread/element”；mapping 应由 operation、memory access、cooperation 和 launch overhead 决定。Multiple pass 还会增加 traffic，因此必须实测。

减少不必要的同步与 host-side stall

移除不必要的 cudaDeviceSynchronize() 或 host wait，但保留 correctness 和 timing 所需的 dependency。对于较小 workload，可评估 batching 或用 CUDA stream 并发执行独立实例。CUDA Graph（见第 12 章）可以降低重复 launch sequence 的 CPU overhead，但 capture、instantiate 和 update 也有约束与成本。

利用 specialized hardware 与 reduced/mixed precision

Blackwell 的部分 Tensor Core 路径在相应 PTX ISA 中使用第五代 instruction family，例如 `tcgen05.mma` 及配套的 `tcgen05.ld`、`tcgen05.st`。支持的 microscaling/block-scaled format（包括 MXFP8、MXFP4、NVFP4）、scale layout 与 MMA shape 取决于具体 architecture、PTX/Toolkit 和 library。Library 只有在 dtype、metadata、shape、layout 与实现条件满足时才会选择相应 kernel，并非出现 scale metadata 就自动使用。TMEM 参与特定 Tensor Core

operand/accumulator 数据路径，TMA 负责受支持的 tensor copy；两者角色不同。后文及第 9 章将继续讨论。

验证并迭代

每项优化后都重新 profiling，并首先确认 correctness 与未插桩总时间。再检查目标 metric 是否按假设变化，例如 tiling 是否减少实际 DRAM byte/scoreboard impact，block-size 调整是否改善 eligible warp 或时间。CUDA stream 主要改变 application-level concurrency，不能简单用单 kernel “SM Throughput %” 证明收益。一个瓶颈被缓解后可能暴露另一个，但只在业务目标仍未满足时继续优化。

保持 correctness 与可接受 accuracy

使用 lower precision 或新的并行策略时，应与可信 reference 比较，并覆盖代表性输入、边界条件、随机性和长时间累积误差。只有应用明确接受时，才能用 accuracy 换 speed。FP16/FP8 在部分 AI model 上可通过 mixed precision、scaling 和校准保持可接受质量，但 “negligible impact” 不是普遍保证，必须按 model/task 验证。

调优 Occupancy

回顾第 6 章：occupancy通常指 SM 上 active/resident warp 数相对于该架构每 SM 最大 resident warp 数的比例。Achieved occupancy 低于 50%只表示平均 active warp 不到最大值的一半，并不自动等于性能低；它可能由 register、shared memory、block/thread residency、grid coverage 或 kernel phase 等因素造成。

*Occupancy*以 warp residency 衡量并行资源，而不是把 active thread 与 warp 两种分母混用。更多 resident warp 有机会为 scheduler 提供更多 eligible warp：当一个 warp 等待 dependency 时，scheduler 可以从另一个 eligible warp 发射 instruction。但 latency hiding 还取决于 ILP、dependency pattern 和 execution resource，更高 occupancy 不保证更快。

在 latency hiding 不足时，增加每 SM warp 数可能让 pipeline 更繁忙；达到足够水平后，收益通常递减。*Occupancy tuning* 是调整 launch parameter 与 register/shared-memory 等资源使用，在 residency、per-thread work、data reuse 和 spill 之间寻找最佳实测点，如图 8-6所示。

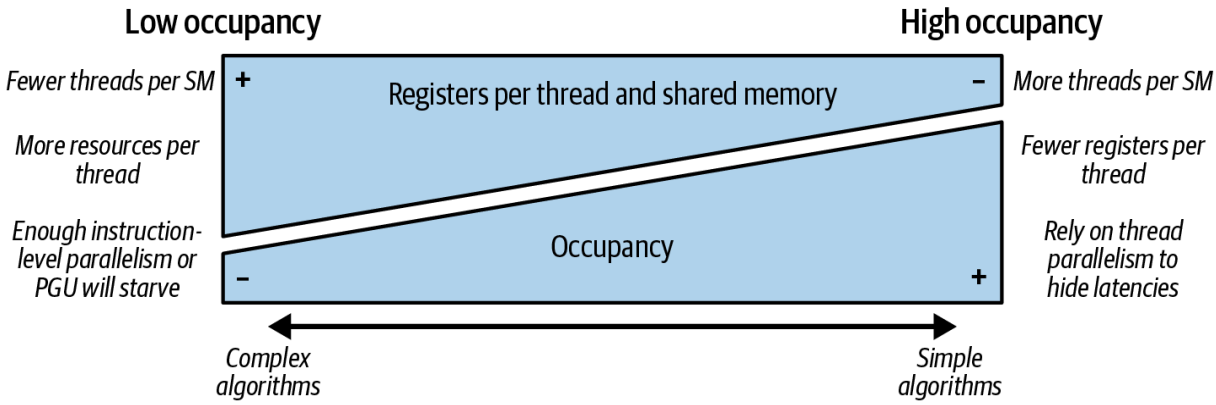


图 8-6. 在资源使用（如每 thread register 与 shared memory）和并行性（如 thread/warp 数）之间权衡 occupancy

Occupancy 调优的目标是提供足够 warp 来隐藏相关 latency 并支持有效 throughput，而不是以 100% 为理想终点。以 Blackwell B200 的 compute capability 10.0 为例，公开架构限制可包括每 SM 最多 64 个 resident warp，即 2,048 个 resident thread；实际可驻留数量还受 block、register 和 shared-memory allocation 等限制，并应通过目标设备属性确认。

> 部分 Ampere、Hopper 和 data-center Blackwell 型号的上限为 64 warp/SM，但不能据此概括所有现代 GPU；每代性能提升也不只来自更多 SM、更大 cache 或 multi-die。应使用 device attribute、CUDA Programming Guide 和 Nsight Compute Occupancy 分析确认目标设备。某些 compute capability 12.x 产品（如特定 NVIDIA RTX PRO Blackwell 型号和搭载 GB10 Superchip 的 DGX Spark）可能采用 48 warp、1,536 thread/SM 的上限；具体型号和版本仍应核对官方规格。

许多实际 kernel 在较低 occupancy 下仍能表现良好，例如其 dependency latency 较短、ILP 较高，或每个 warp 能为 Tensor Core 等高吞吐 pipeline 提供足够工作。反过来，过多 resident warp 也可能需要牺牲 register、tile 或 stage。

例如，compute-heavy kernel 可能在 50% occupancy 附近已达到最佳 throughput。Memory-latency-bound kernel 有时受益于更多 eligible warp；但真正 memory-bandwidth-bound 的 kernel 已接近 bandwidth ceiling，提高 occupancy 往往不能再增加 throughput。因此必须区分 latency 与 bandwidth。

为 Workload 找到合适的 Occupancy

实践中，occupancy 调优通常在达到足以隐藏主要 latency 后出现收益递减。例如，对 latency-hiding 不足的 kernel，从 10% 提高到 50% achieved occupancy 可能显著改善性能；但若它已经 bandwidth-bound，提高 occupancy 未必有类似收益。

从 50% 继续提高到 100% 可能收益很小，也可能因 register spill、较小 tile 或其他变化而减速。Profiling 有助于寻找最佳点，例如结合 *eligible warps per cycle* 与 *active warps per scheduler*，观察每个 scheduler 的 resident warp 中有多少在采样 cycle 已具备发射条件。

Eligible warps per cycle 表示每个 scheduler 在每 cycle 平均可发射下一条 instruction 的 warp 数。Active warps per scheduler 则是分配/驻留在每个 scheduler 上的平均 active warp 数，并不等于 SM 上 scheduler 的数量，也不是每 cycle 可 issue warp 的硬件上限。还应同时查看 issued warp、No Eligible 和具体 stall state。

如果 active warp 很多但 eligible warp 经常接近零，scheduler 可能缺少 ready warp，说明 dependency latency 未被充分覆盖。更高 occupancy 或 ILP 是候选方法，但也可能需要处理 cache miss、barrier、spill 或 dependency chain，不能只比较 eligible 与 active 两个平均值。

如果 eligible warp 供应充足但 issue rate 或性能仍低，可能是 dispatch/execution pipeline throttle、memory bandwidth、instruction mix 或其他 resource 限制，而不是“warp 有 dependency 所以无法 issue”这一自相矛盾的解释。此时应查看对应 pipeline 与 throughput counter。Coalescing、async copy 或 unrolling 是否适用，取决于已确认的瓶颈，而不是统一答案。应做受控实验。

提高 occupancy 后，若 eligible warp 和 issue efficiency 改善、No Eligible 减少且执行时间缩短，才说明 latency hiding 获益。“Not Selected” 可能因为可供选择的 eligible warp 增多而上升，并非应当下降；Idle 的含义也需按版本核对。High occupancy 下 Long Scoreboard 仍高时，应检查 actual memory path、cache/spill、访问模式和独立工作。

不存在跨 workload 固定的 60%–70% 收益递减点。最佳 occupancy 可能更低或更高，应扫描 block size、resource configuration，并同时观察 runtime、spill、eligible warp、memory locality 和 ILP。

即使 occupancy 很高，warp 仍可能等待 dependency、barrier 或受到 divergence/imbalance 影响。因此达到足够 latency hiding 后，应转向真实限制因素，而不是执着于 100% 或预设的 50%–70% 区间。

> Occupancy 是实现 latency hiding 的一种手段，而不是最终目标。具备足够 eligible work 后，继续增加 resident warp 未必有益；应把优化转向 profiler 和执行时间所证明的下一个限制因素。

Occupancy 调优技术

当 profiling 表明 residency/eligible warp 不足确实是限制因素时，可以测试增加 device-level work、调整 block size、改变 per-thread/per-block resource usage，以及使用 `__launch_bounds__` 或 Occupancy API。它们主要影响 theoretical residency，不保证实际性能提升。下面分别讨论。

增加可并行工作（launch 更多 thread）

如果当前 grid 太小、没有覆盖所有 SM，增加可处理的 work/block 是改善 device utilization 的直接方法。但“每 SM 只 launch 20 个 warp”并不是 host launch 的固定映射；block 由 scheduler 分配到 SM，resident warp 数由 block size、资源和架构限制共同决定。增加 grid size 可以改善 coverage，增加 threads/block 则可能提高或降低 residency。

通常需要至少与 SM 数量同量级、并常常更多的 block，以减少 wave/tail 效应；但 block 数等于 SM 数并不保证全部 SM 同时有效繁忙。若 timeline/device metric 显示部分 SM 缺少工作，可在 memory/latency 允许时扩大或 batch workload。每 thread 的 register 与每 block shared memory 仍可能限制 residency。

调整 block size

Threads/block 会通过 warp 取整、per-SM thread/warp/block limit 和资源 allocation granularity 影响 occupancy。1,024-thread block 常使 resident block 数较少，但其中本身已有 32 个 warp，occupancy 未必低；128–256 thread 只是常见候选范围，不是普遍最优值。

最佳 block size 随 kernel 与 architecture 变化。较小 block 的总 register/shared-memory footprint 可能较低并允许更多 block 驻留，但 fixed per-block shared

memory、allocation granularity、coalescing、cooperation 和 tail behavior 都会改变结果。

可以使用 Nsight Compute Occupancy Calculator 与 CUDA Occupancy API 生成 launch configuration 候选，再扫描实际执行时间、spill、memory access 和 eligible warp。

Occupancy Calculator 估算的是给定资源需求下的 theoretical occupancy/residency。最佳性能可能对应更低、相同或偶尔更高的配置目标，不能只按最大 theoretical occupancy 选择。

> 始终用实际 timing 验证 Occupancy API/Calculator 给出的 `bestBlockSize` 候选。较低 occupancy 的配置可能因更少 spill、更大的 tile、不同 instruction mix 或更好的 memory access 而更快。

调整 per-thread register 与 per-block shared-memory 用量

每个 thread 消耗 register；shared memory 通常按 block 静态或动态分配，虽可按 thread 协作使用，却不能笼统称为 per-thread allocation。Register file、shared-memory capacity 及其 allocation granularity 会共同限制每 SM resident block/warp 数。

可以通过缩短 live range、减少不必要 temporary 或改变 algorithm 来降低 register pressure，从而可能允许更多 warp 驻留。也可以向 compiler 传递 `-maxrregcount=<N>`，尝试限制每 thread register 数。

Register cap 只是 compiler allocation constraint；只有跨过 register allocation/residency 阈值时才可能增加 resident warp，而且 compiler 可能无法完全按期望分配，或通过 spill 满足限制。必须检查 ptxas/SASS、local-memory traffic 和 achieved occupancy。

Cap 过低会把 live value spill 到 local memory，增加 instruction 与 cache/DRAM traffic。不存在“能最大化 occupancy 的最小 register limit”这一普遍最佳目标；应寻找使总执行时间最低且 correctness 不变的配置。

若每 block 使用大型 shared-memory tile/stage，shared memory 可能限制 resident block 数。可以删除未使用数据、调整 tile/stage 或复用 buffer，但改用 cache 或缩小 tile 可能降低 locality/reuse。释放容量只是一项权衡，并非无成本增加 occupancy。

减少 register 或 shared memory 过度时，会降低 ILP、增加 spill/global traffic 或减少 data reuse。Occupancy limiter 只能指出 theoretical residency 的限制资源，不能证明降低该资源一定更快；应结合执行时间与相关 counter。

例如，aggressive unrolling 可能触及目标 Blackwell 设备的 per-thread register 编码/架构限制或更早跨过 per-SM allocation 阈值。255 register/thread 是多种 NVIDIA 架构常见的上限口径，但实际 usable allocation、granularity 和设备限制应查询目标 architecture。减少 unroll 或拆分工作也可能增加 loop/launch/memory 开销。

使用 `__launch_bounds__`

在某些情况下，可以通过 `__launch_bounds__` 向 compiler 声明 kernel 允许的最大 threads/block，以及可选的 minimum blocks/SM 等 launch assumption。它不直接“设定 occupancy”，而是影响 compiler 的 register allocation 与优化决策；违反最大 launch bound 的 launch 不合法。

若实测最佳配置约为 50% 而非 100% occupancy，这不必然表示 register/shared memory“过多”；也可能是更大的 tile、ILP 或其他配置更高效。只有在 compiler allocation 确实妨碍目标 launch/residency 时，`__launch_bounds__` 才是候选工具。

`__launch_bounds__` 的 maxThreadsPerBlock 限制合法 block size，可选 minBlocksPerMultiprocessor 会让 compiler 尝试控制 register 使用以支持该 residency。它不能直接减少 per-block shared memory，也不保证实际同时驻留指定 block 数，因为其他资源和架构限制仍然适用。

若 latency hiding 确实不足，改变 compiler allocation 后可能因更多 eligible warp 而提高 throughput，也可能因 spill 或较低 ILP 而减速。Compute-bound 与 latency-bound 都必须实测，不能由分类直接保证结果。

使用 CUDA Occupancy API

除 `__launch_bounds__` 外，CUDA Occupancy API 函数（如 `cudaOccupancyMaxPotentialBlockSize()` 和 `cudaOccupancyMaxActiveBlocksPerMultiprocessor()`）可依据 kernel function、block size、dynamic shared memory 和设备属性估算候选 block size 或每 SM active block 数。Register 使用来自已编译 kernel；API 计算的是 occupancy/residency，不是运行时性能预测。

这些函数能减少 launch-configuration 搜索空间，但不能告诉你哪种配置在 memory access、ILP、spill、tail 和 workload shape 上最高效。实践中应枚举候选并 timing；`__launch_bounds__` 会改变编译代码，不能在运行时用它“锁定”任意 occupancy level。

确定 launch configuration 后，还可以用 CUDA Graph降低重复 workload 的 CPU launch overhead。CUDA Graph 不直接改变 kernel occupancy，也不会因为 block 数增加而“改善 occupancy”；它主要减少 host submission gap，并受 capture/instantiate/update 约束。CUDA Graph 见第 12 章，PyTorch 相关内容见第 13、14 章。

总之，应分阶段 profiling：先确认 latency hiding/residency 是否限制性能，再用 calculator/API 生成候选，通过 compiler option、launch configuration 和资源设计进行实验。目标是以足够的 eligible warp 覆盖 latency，同时保留必要的 register、shared-memory reuse 和 ILP，而不是达到某个预设 occupancy。下面进一步讨论 `__launch_bounds__`、Occupancy API 和 PyTorch。

使用 Compiler Hint 调整 Occupancy

可以使用 CUDA kernel annotation `__launch_bounds__` 向 compiler 提供 launch assumption。常用的前两个参数分别是允许的最大 threads/block（例如 256），以及

compiler 在 register-allocation 计算中采用的 minimum blocks/SM 目标。下面的代码使用 8，而不是原文上一句举例所称的 4：

```
__global__ __launch_bounds__(256, 8)
void myKernel(...) { /* ... */ }
```

这里声明 `myKernel` 的 launch 不得超过 256 threads/block；若实际超过该上限，行为不受支持。第二参数 8 不是让 GPU runtime “尝试保持”至少 8 个 active block，而是让 compiler 计算 register 上限时以支持至少 8 blocks/SM 为目标。Inlining 等其他优化是否改变，还取决于 compiler。

`__launch_bounds__` 只有在目标设备的 thread、block、register 和 shared-memory 等全部限制都允许时，才可能使 8 × 256 = 2,048 threads/SM 理论驻留；annotation 本身不能保证运行时达到这一 residency。Compiler 根据 launch bound 推导 register threshold，可能降低 register 使用以满足 minimum-block 约束；在某些情形下，也可能利用可用 register 提高单 thread 性能。它不直接限制 shared memory，也不保证避免 spill、限制 unrolling/inlining 或提高 achieved occupancy。对于 B200 等具有 2,048-thread/SM 上限的设备，`__launch_bounds__` 中的这一组合只是一个理论候选，必须检查编译报告、其他 residency limiter 和实测时间。

> 提高 occupancy 必须与每 thread 所需的 register/ILP 及每 block shared-memory reuse 权衡。过强的 register constraint 可能造成 spill；spill slot 位于 local memory address space，通常由 device memory 支撑并经过相应 cache path，并非每次都直接访问 HBM。应结合 ptxas register/spill 报告、Nsight Compute 的 register、local-memory traffic、occupancy/eligible-warp metric 和实际执行时间寻找最佳点。

使用 Occupancy API 确定候选 Launch Configuration

可以在运行时使用 CUDA Occupancy API 估算 launch configuration，例如 `cudaOccupancyMaxActiveBlocksPerMultiprocessor()` 和 `cudaOccupancyMaxPotentialBlockSize()`。其中，`cudaOccupancyMaxPotentialBlockSize()` 根据指定 kernel 的 register 使用、每 block shared memory 和架构资源限制，计算有望最大化 theoretical occupancy 的 block size，如下所示。这里的“optimal”仅针对该 occupancy model，并不表示执行时间最短：

```
int minGridSize = 0, bestBlockSize = 0;

cudaOccupancyMaxPotentialBlockSize(
    &minGridSize, &bestBlockSize,
    myKernel,
    /* dynamicSmemBytes = */ 0,
    /* blockSizeLimit = */ 0 );

// bestBlockSize contains number of threads per block that maximizes occupancy
myKernel<<<minGridSize, bestBlockSize>>>(...);
```

该 API 返回在其资源模型中可能最大化 occupancy 的 threads/block 数。随后可将建议的 `bestBlockSize` 和 `minGridSize` 作为 kernel launch 的候选值；但后者是达到相应 occupancy 所需的建议最小 grid size，不一定是覆盖实际问题规模所需的 grid，也不能直接替代由数据规模决定的 grid 计算。

实践中，compiler 和 library 的默认 heuristic 往往已能给出合理配置；需要显式探索 resource/residency trade-off 时，可结合 `__launch_bounds__` 与 Occupancy API 生成候选。它们适用于评估每 thread register、每 block shared memory 与 active warp 的取舍，但不会预测 memory coalescing、cache locality、ILP、spill、tail effect 或执行时间。因此仍应扫描附近的合法 block size，并用 compiler report、Nsight Compute 和端到端计时验证。

在 PyTorch 中调优 Occupancy

使用 PyTorch 编写高层代码时，通常不直接管理 occupancy，因为 PyTorch 的 CUDA kernel 和底层 library 会选择 launch parameter。Matrix multiplication、convolution 等常用 operation 通常已有针对受支持设备和 shape 的优化实现，但实际效率仍取决于 dtype、layout、shape、backend 和 hardware。

理解 occupancy 仍然有价值。若以 PyTorch extension 编写自定义 CUDA kernel，同样需要保证 grid 足以覆盖设备、选择合适的 block size，并在 register、每 block shared memory、ILP 和 residency 之间权衡；目标不是一味压低资源使用或最大化 occupancy。

即使只在 Python 层工作，也应留意 workload granularity。若 profiling timeline 显示 GPU 利用不足，例如只有少量 SM 在工作，原因之一可能是 tensor operation 太小，grid 不足以覆盖设备；也可能是 host launch gap、synchronization、data input 或其他瓶颈，需要结合 timeline 判断。

极小 workload 往往无法在大型 GPU 上良好扩展。此时可尝试 batching 或合并 operation，使每次 launch 完成更多有效工作。PyTorch graph compiler 可以使用 `torch.compile` fusion 某些 operation，从而减少 launch 和 intermediate-memory overhead；但 fusion 也可能增加 register pressure 或降低 parallelism，并不必然提高 occupancy 或性能。

> `torch.compile` 可在底层生成 fused GPU kernel。它会在语义和 backend 条件允许时把多个较小 operation 合并为较大的 kernel，并可能在某些 workload 上接近手写 CUDA kernel 的效率。对于 shape 稳定且运行时间较长的 training/inference，可评估 `torch.compile(mode="max-autotune")`；对于 graph-compatible 的小 batch loop，可评估 `mode="reduce-overhead"`。这些 mode 是否以及如何使用 CUDA Graph，取决于 PyTorch 版本、backend、device 和 graph compatibility；不希望使用 graph 时，可评估 `mode="max-autotune-no-cudagraphs"`。第 13 章和第 14 章将深入讨论 PyTorch compiler 及其 mode。

PyTorch 内置 kernel 通常也包含经过调优的 launch heuristic。例如某些 reduction 和 elementwise implementation 会依据 device、tensor size、dtype 与 algorithm 选择 block size 和 block 数；这是候选配置 heuristic，而非对所有输入都“optimal”的保证。

查看 PyTorch 的 C++/CUDA 实现时，可以看到某些 algorithm 对 block size 设置上限，并选择足够的 block 以覆盖设备。这类逻辑综合考虑 occupancy、work distribution 和算法约束。为减少 optimizer 的 launch 与 memory-traffic overhead，可在目标 device、dtype 和版本支持时评估 fused implementation，例如 `torch.optim.AdamW(..., fused=True)`，并按数值精度要求配合 automatic mixed precision（AMP）；是否更快仍需测量。

对 PyTorch 用户而言，优先采用已针对常见路径优化的高层 library 和 operation 通常更稳妥。确需自定义 kernel 时，则应应用前述 occupancy 原则，并以实际 profiling 结果选择配置。

在了解如何提供足够的并行工作后，下一步是提高每个 warp 的执行效率。即使 occupancy 很高，若大量 lane 被屏蔽或 warp 频繁等待，性能仍可能受限。下面讨论 warp divergence 和 warp 内 thread cooperation 等 warp-level 优化。

提高 Warp 执行效率（Warp Divergence）

即使良好的 occupancy 提供了足够多的 resident warp，每个 warp 内部的 lane 利用率仍然重要。Warp execution efficiency 类指标描述执行 instruction 时平均有多少 warp lane 处于 active 状态。同一 warp 内的 thread 采用不同 control-flow path，称为 *warp divergence*。等待 data load 则会使整个 warp 无法发射后续 dependent instruction，它属于 latency/stall 问题，不能仅凭这一点称为 lane divergence。

较低的 active-thread-per-warp 或 thread-inst-per-inst 类指标可能伴随 divergent branch 和大量 predicated-off lane，但 memory-related stall 是另一维度。诊断时应区分：control-flow divergence 会在不同 path 上以不同 active mask 执行，而 data dependency 通常使相关 warp 等待；两者可能同时存在，却不能从单个汇总百分比互相推导。

确认瓶颈后，可重构 control flow、改善 data grouping、减少不必要的 thread divergence，并在适用时使用 warp-level intrinsic 进行 warp 内通信。Memory coalescing 也很重要，但它优化的是 memory transaction/access efficiency，并不直接消除 branch divergence。

这些技术带来的收益没有通用的 5%–30% 区间，取决于 divergent region 占比、各 path 成本、active mask、compiler predication 和其他瓶颈。在 occupancy 与 memory access 已经调优后，仍应通过 source-level metric 和执行时间验证 warp-level 改动，而不是假定一定还能获得额外性能。

Warp Divergence 的成因

如第 7 章所述，当同一 warp 内的 thread 采用不同 control-flow path 时，就会发生 warp divergence，也称 branch divergence。例如，kernel 中有一个 `if/else`，warp 内一部分 thread 执行 `if` 子句，另一部分执行 `else` 子句。

在 SIMT execution 中，如果 compiler 没有把这段代码转换成其他等价形式，warp 会用不同的 active mask 执行所需 path：执行 `if` path 时，不采用该 path 的 lane 被屏蔽；执行 `else` path 时则反过来，随后在 reconvergence point 汇合，如图 8-7所示。

在 divergent region 中，每条 path 只有采用该 path 的部分 lane active，因此会降低该区域的 lane utilization。若 `if` 与 `else` 各由 16 个 lane 执行，且两条 path 的 instruction 数相同，则这两段 path execution 的平均 active-lane 比例为 50%；这不等于整个 kernel 的 throughput 必然下降 50%。若分布为 1 个 thread 对 31 个 thread，则在单 lane path 执行期间，另外 31/32 的 lane 被屏蔽，而另一条 path 的 active 比例是 31/32。

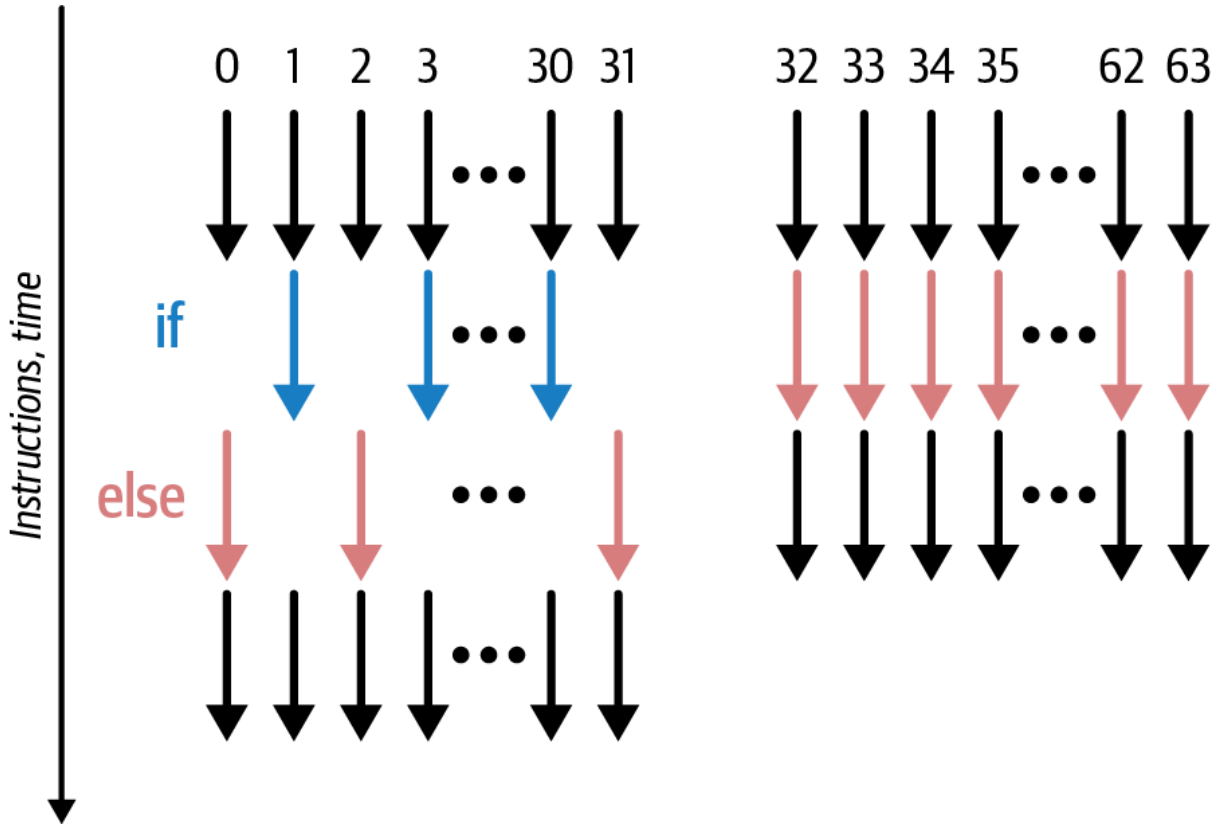


图 8-7. Divergent 与 nondivergent warp execution 对比

若 kernel 包含多个 divergence point，或 divergent path 承担大量工作，减少 divergence 可能带来显著收益。在一个严格的理想化情形中，两个等长 path 各激活一半 lane、该区域主导运行时间，且改写后只需执行等量的单一路径工作，理论上才可能接近 2×；实际收益还受额外 instruction、memory behavior、load balance 和其他瓶颈限制，多个 divergence point 的收益也不能简单相乘。

总体而言，warp divergence 会使部分 lane 在某些 path execution 期间被屏蔽，并可能使 warp 执行多条 control-flow path，增加 executed instruction 与有效工作的比值。因此，减少不必要的 intrawarp divergence 是提升 warp-level efficiency 的重要手段；但对本就需要执行不同工作的算法，还必须计入 regrouping、compaction 或额外 launch 的成本。

避免 Warp Divergence 的方法

减少 warp divergence 的方法包括重构 condition、按数据或工作类型分组，以及在收益足以抵消额外开销时将不同 path 拆分到多个 kernel。

Branch divergence 的 active-mask serialization 发生在同一 warp 内。不同 warp 不会因为彼此采用不同 branch path 而发生这种 divergence；但它们仍会竞争 scheduler、execution pipeline、cache 和 memory bandwidth 等共享资源，因此不能笼统地说不同 warp 绝不会使彼此等待。

还可以使用 warp-unanimous branch、warp-vote intrinsic 和 predication。下面逐一讨论。

重构 Condition

条件允许时，可组织 computation，使同一 warp 内的 thread 尽量采用一致的 execution path。例如，把 divergent condition 移出 inner loop、提升到更高层，或放入独立 kernel launch；也可对数据排序或分组，使每个 warp 处理更同质的 case。重排是否值得，取决于额外 memory traffic、latency、stability requirement 和后续 reuse。

例如，对数组中的负数和非负数采用不同处理时，直接在 kernel 中按 element 使用 `if(x<0)` 可能使混合符号的 warp divergence。另一种方案是先 partition 数据，再由不同 kernel 处理负数与非负数。这样能提高 warp coherence，但需要支付 partition/compaction、额外 storage、launch 和可能破坏原始顺序或 locality 的成本。

拆分为多个 Kernel

还可以把工作拆成多个 kernel launch：一个处理 `if` case，另一个处理 `else` case，并使用 prefix sum 或 compaction 把 item 分配给不同 kernel。它可消除处理阶段的相关 divergence，却会增加分类、memory traffic、temporary storage、synchronization 和 launch overhead。只有当 divergent path 足够昂贵、分类结果可复用或 batching 足够大时，收益才可能覆盖这些成本。

把 Condition 改写为 Warp-Unanimous

某些情况下，可把 condition 改写为 *warp-unanimous*，即一个 warp 的所有 active lane 对 condition 得到相同结果。常见方式是在 condition 中使用 warp index。例如先计算 `int warpId = threadIdx.x / 32;`，再让一半 warp 执行 task A、另一半执行 task B：`if (warpId % 2 == 0) { /* Task A */ } else { /* Task B */ }`。这里假设一维 block 且 CUDA warp size 为 32；更通用的代码可依据 linear thread index 与 warp size 构造 ID。

此时，同一 warp 内的 thread 一起进入 task A 或 task B，因此 branch 在 warp 内是 uniform 的；不同 warp 可以执行不同 path，而不会由此产生 intrawarp branch divergence。

每个 warp 仍需处理 condition，且这种方法约束了 work mapping。若 task A/B 的成本不同，还可能产生 warp/block 间 load imbalance。因此它是以调度灵活性换取 warp coherence，而不是无条件优化。

当工作能按完整 warp 的粒度划分、各组成本较均衡且不会显著损害 memory locality 时，可考虑 warp-unanimous branch。

使用 Warp-Vote 并行算法

Warp intrinsic（`__ballot_sync`、`__any_sync`、`__all_sync`）、Cooperative Groups 的 `warp.ballot/any/all` 类 operation，以及底层 PTX vote instruction（原文所写的 `%WarpVote` 并不是规范 CUDA 可编程接口）可让 warp 集体判断哪些 lane 需要“特殊”工作。Ballot 返回 lane mask，但不会自动 delegate、repartition 或 compact；程序必须根据 mask 使用 population count、prefix/rank、shuffle 等显式实现这些步骤。这样有时能缩短 divergent region，也可能引入额外 instruction、serialization 和 load imbalance，必须 profiling。后文将更详细介绍 warp intrinsic。

对短 Condition 使用 Predication

CUDA compiler 有时会对较短的 conditional code 使用 *predication*。Compiler 可把 `if` 转换为每 thread 的 predicate；predicate 为 false 的 lane 不写结果，且是否执行 address evaluation、memory access 或其他行为取决于具体 instruction 与生成代码。它不应简单理解为 compiler 总是让所有 thread 完整计算两条 high-level path，再只提交一边结果。

Predication 可避免 control-flow branch/reconvergence overhead，但仍可能发射更多 predicated instruction。短 path 且 warp 内 condition 混合时可能有利。程序员可以用 `?:` ternary operator 等表达 branch-free intent，但 compiler 最终可能生成 predicated select、branch 或其他指令，应检查 SASS 和性能。先看下面使用 if/else 的直接实现：

```
if (x > 0)
    y = f(x);
else
    y = g(x);
```

原文建议改写成下面的 arithmetic selection：所有 thread 都计算 `f(x)` 和 `g(x)`，再乘以 `cond` 选择结果。这并不等同于 compiler predication，而且只有在两函数对所有输入都定义良好、额外计算可接受且 floating-point 语义允许时才安全：

```
float cond = x > 0 ? 1.0f : 0.0f;
y = cond * f(x) + (1.0f - cond) * g(x);
```

这段源代码没有显式 branch，但每个 thread 都执行两个函数，包含本不需要的那一个。它还可能因 0 × NaN 仍为 NaN、overflow、side effect 或 domain error 而改变原程序语义，所以不能作为通用 predication 模板。只有确认数值语义等价，并且额外工作比 divergent execution 更便宜时，才应考虑类似改写。

实践中必须 profiling 这些优化。使用 predication 时，可查看 source/SASS、每 instruction active thread、predicated-off thread instruction 和总 executed instruction，并以 kernel 时间判断。Nsight Compute 能帮助确认改写是减少了 branch/active-mask 成本，还是仅增加了无效工作；memory 或 dependency stall 的变化还需用对应 metric 分析。

Predication 通常更适合只有少数 instruction 的简单 conditional path，尤其是许多 warp 的 lane condition 混合时。若 branch 内工作较多、path 成本悬殊，或实际 condition 在 warp 内大多 uniform，predication 可能更差。应让 compiler 生成候选代码并实测，而不是套用固定阈值。

> CUDA compiler 会依据 target architecture、优化级别和 control-flow 形态自动决定是否对简单 condition 使用 predication；对性能关键 kernel，仍应检查生成代码并 profiling。

总之，减少 warp divergence 需要同时考虑 algorithm、data organization 和重排成本。可行时，让同一 warp 处理相似 item、把 divergent condition 移出 hot inner loop，或在额外 launch/traffic 成本可接受时拆分 kernel，使 execution path 更一致。

若 divergence 难以避免，例如 tree 或 graph workload 的每 thread 工作天然不同，应尽量缩短高成本 divergent region，并使 control flow 在合适位置 reconverge。也可利用 `__any_sync`、`__ballot_sync` 等 warp-level vote primitive 获取集体状态，再显式 compact 或重新分配工作；所有参与 lane 必须遵守 `_sync` intrinsic 的 mask/participation contract。

Profiling 与检测 Warp Divergence

Nsight Compute 的 Source Counters 和 Warp State Statistics 可帮助把低 active-lane 行为定位到 source line 和 instruction。应结合 branch instruction、branch target/thread divergence、每 instruction active thread 与 predicated-off thread 等目标版本实际提供的 metric。Poorly coalesced load、replay 或 L2 cache miss 描述 memory access 行为，本身不能作为 branch divergence 的证据。

Nsight Compute 也能报告 shared-memory bank conflict、Tensor Core pipeline 利用与各种 warp stall reason。例如较高的 `memory_throttle` 通常意味着 memory pipeline、request queue 或相关资源受到压力，而不是 memory-level parallelism 不足的同义词；盲目增加 outstanding access 反而可能加剧 throttle。应先结合 Long Scoreboard、memory throughput、queue/pipe metric 和 source instruction 确定是 latency 还是吞吐/队列限制，再考虑增加独立 memory access，或在架构与数据路径适用时使用 `cp.async`、CUDA Pipeline API 等 asynchronous copy（见第 9 章和第 10 章）。

Profiler 中较低的 *warp execution efficiency* 或等价 active-thread 指标可能是 divergence 的症状，它表示相关 executed warp instruction 平均有多少 lane active。若值为 30%，通常表示所统计 instruction 的平均 active lane 约占 30%，不等于任意时刻只有 30% thread 在做“有用工作”，也不能断言其余全部因 branch divergence

inactive；boundary condition、partial warp、early exit 和 predication 都可能造成低值。

含 divergent branch 的 kernel 可能呈现较多 *predicated-off* thread instruction，并且相对于有效 element 增加 executed instruction。Predicated-off lane 不产生该 instruction 的有效结果，但 profiler 的具体计数口径取决于 metric 与 architecture。只有把这些信号与 source correlation、branch target、active mask 和 SASS control flow 结合，才能判断 warp 是否依次执行多个 conditional path；两个汇总数字本身不足以证明每个 warp 都遍历了所有 path。

例如，一部分 *lane* 执行 path A 时其余 lane 被屏蔽，随后另一部分执行 path B。若两条 path 都必须执行，warp 会为不同 active mask 发射相应 instruction，降低 lane utilization；但 instruction traffic 是否恰好翻倍取决于两条 path 的长度、共同代码、compiler predication 和 reconvergence，inactive lane 也不应被表述为独立地再次 fetch 相同 instruction。

Source code 中的 per-thread conditional 是检查起点，例如 threshold check、sparse-data loop 或 data-dependent filter，但它不一定实际产生昂贵 divergence：condition 可能在 warp 内 uniform、由 compiler predication，或其 path 很短。应先证明该 control flow 对时间有实质影响，再决定保留、flatten、分组或拆分，而不是必须消除所有 divergent branch。

可行的改写包括让 condition 在 warp 内 uniform、移出 hot loop，或在语义等价且成本合适时使用 arithmetic、select 或 lookup table。目标是减少昂贵的多 path execution，同时避免引入额外计算、memory access 或数值语义变化。下面通过示例进一步说明。

若 active-thread 指标偏低，应检查 kernel branch 及对应 source/SASS，并排除 partial warp、boundary mask 与 early termination。证实 divergence 是瓶颈后，可评估将 conditional logic 拆为独立 kernel，或使用 warp-level primitive（例如 ballot sync）收集 lane condition 并显式组织工作；下一节将继续讨论。

使用 Predication 减少 Divergence

下面用一个 predication 示例说明如何 profiling 并减少可能的 warp divergence。考虑用 `if (x[i] > 0) y[i] = x[i]; else y[i] = 0;` 对数组做 threshold。是否存在实际 branch 和 divergent path，必须以目标 architecture 上生成的 PTX/SASS 为准。

若一半值满足 condition、一半不满足，同一 warp 内可能混合两种结果。若 compiler 保留 `if` 与 `else` 两条 control-flow path，warp 会用不同 active mask 执行 `if` 与 `else` 所需 instruction；但这种极短赋值通常也可能被转换成 select 或 predicated instruction。因此不能仅凭输入比例断言执行两套 path，更不能预先认定效率恰好下降 50%。

改写前示例（CUDA C++）。 若目标生成代码确实保留 branch，并且同一 warp 中部分 thread 满足 `X[i] > threshold`、其余不满足，才会发生相应 branch divergence：

```
// threshold_naive.cu
__global__ void threshold_naive(const float* X, float* Y,
                                float threshold, int N) {
```

```
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        if (X[i] > threshold) {
            Y[i] = X[i];    // branch 1
        } else {
            Y[i] = 0.0f;    // branch 2
        }
    }
}
```

在保留 divergent branch 的情形下，一部分 thread 对 `Y[i]=X[i]` path active，另一部分对 `Y[i]=0` path active。若两条 path 的 executed instruction 成本相同且各有 16 个 active lane，该区域的平均 active-lane 比例可接近 50%；但 compiler 优化、共同 instruction 和 boundary mask 都会改变实际结果。

改写后示例（CUDA C++）。 下面使用 ternary operator 表达 conditional selection。Compiler 可能把它转换为 select、predicated move，或仍使用 branch；原文所称的 `SEL/MOV` 不是跨 NVIDIA architecture 与 compiler version 都固定成立的 SASS 形式：

```
// threshold_predicated.cu
__global__ void threshold_predicated(const float* X, float* Y,
                                     float threshold, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        float x = X[i];
        // Use a conditional move or multiplication by boolean
        float val = (x > threshold) ? x : 0.0f;
        Y[i] = val;
    }
}
```

从 CUDA C++ source 看，所有 in-range thread 都计算一个 `val` 并存储；condition 为 false 时得到 `val=0`。若生成无分支 select，threshold condition 不会形成 divergent control-flow path。不过外层 boundary check 仍可能使最后一个 partial warp 的 lane inactive，而 active-thread metric 是否提高取决于最终机器代码。

> 对简单 condition，NVCC 可能生成 select 或 predicated instruction。PTX 中 `@p` 表示 predicate guard，但 ternary 不保证一定呈现为被 predicate 保护的 write；PTX 还会继续经由 ptxas 转成目标 SASS。应使用对应 Toolkit 的 PTX、`cuobjdump`/`nvdisasm` 或 Nsight Compute source view 检查实际代码。

PyTorch 改写示例。 在 PyTorch 中，可以用 elementwise operation 表达 threshold：

```
# threshold_op.py
import torch

X = torch.randn(N, device='cuda')
Y = torch.maximum(X, torch.zeros_like(X))  # equivalent to Y = X > 0 ? X : 0
torch.cuda.synchronize()
```

`torch.maximum` 与 0 的 elementwise maximum 通常可由 GPU max/select 类数据路径实现，而不需要 source-level per-element branch；但具体 kernel 与 machine instruction 取决于 dtype、backend、PyTorch/Toolkit 版本及 fusion。所谓 vectorized elementwise API 并不构成“必然无 divergence”的实现保证。还可尝试 compiler fusion：

```
# jit_threshold_op.py
import torch

# In PyTorch, we can compile and fuse this operation
# for even higher throughput
@torch.compile()
def threshold_op(X):
    return torch.maximum(X, torch.zeros_like(X))

X = torch.randn(N, device='cuda')
Y = threshold_op(X)
torch.cuda.synchronize()
```

> 如第 13 章和第 14 章将介绍的，`torch.compile` 可通过 TorchInductor 等 backend fusion 多个 pointwise operation，减少 intermediate traffic 和 launch；是否接近手写 CUDA C++ 取决于 workload。此外，`torch.compile` 不保证最佳 occupancy、ILP 或 runtime，仍需 profiling。

这类 elementwise function 最终可能生成 max、select、predicate 或其他 target-specific instruction。无分支实现可避免该 condition 的 divergent control flow，但不能保证每个 warp 的所有 lane 始终 active，因为 grid boundary、broadcast/layout、其他 condition 和 kernel 实现仍会影响 active mask。

当 divergence 确为主要瓶颈时，减少它可能提高 warp efficiency；收益必须实测。表 8-3 是原书用于说明“用 predicated operation 替换 branch”概念的假设性数值，不是由本节代码与指定 GPU/命令产生的可复现实验结果。

表 8-3. 移除 warp divergence 的说明性 profiling 数值

指标	改写前	改写后
Kernel 执行时间	30 ms	15 ms (–50%)
Dynamic instruction 数量	6 亿条指令	3 亿条指令（–50%）
平均 warp branch-resolving stall latency	200 个周期	100 个周期（–50%）
Warp execution efficiency	50%	99%
Predicated-off thread（%）	50%	0%

注意：所有 metric 表中的数值均为解释概念的示意值。不同 GPU architecture 上的实际 benchmark 结果见 GitHub repository；具体结果仍需结合相应代码版本、环境和复现说明核验。

表中假设把 two-path branch 换成单一 predicated `max()` operation 后，多项 Nsight Compute metric 同时改善：以 `gpu__time_elapsed.avg` 表示的 kernel 时间从 30 ms 降至 15 ms，对应示意吞吐提高 2×。这些数字不是本节可验证的实测结论，简单 threshold 的 branch 与 ternary 还可能被 compiler 优化成相同机器代码。

表中 warp execution efficiency 接近 100%，predicated-off ratio 为 0%，仅用于表示假设所有有效 lane 都执行同一路径。即使在 predicated `max()` 方案中观察到这些值，也不能推出“没有 lane 曾被屏蔽”：最后一个 partial warp、boundary predicate 和 metric 采样/定义均需考虑；也不能只凭 0% 汇总值证明全部 reconvergence overhead 已消除。

同样，表中 dynamic instruction count（`smsp__inst_executed.sum`）从 600 M 降至 300 M，branch-resolving stall latency（原文写作 `smsp__average_warp_latency_issue_stalled_branch_resolving`）从 200 cycle 降至 100 cycle，均是说明性假设。Metric 名称和可用性随 Nsight Compute/architecture 变化，应查询目标版本；`smsp__inst_executed.sum` 统计 warp-level executed instruction，并不能直接等同于每 lane 的 scalar instruction 数。

多个 divergence point 或较重 path 可能提供更大优化空间，但每移除一个 branch 并不存在“超过 2×”的独立收益保证，多个改动的 speedup 也不能相乘。最终上限受该区域原始时间占比和其他瓶颈约束。

Predication 并非免费。一些 lane 可能执行其结果最终不用的 instruction（例如 `val = x`）；若两条 branch 都包含大量工作，无条件计算两边可能比适度 divergence 更昂贵。不过真正的 predicated instruction 与 source-level 同时计算两边也应区分。

对于 threshold 这类简单 case，branch 和 ternary 可能生成相同代码；应 benchmark 而不是预设 predication 获胜。可比较生成的 SASS、active-thread/branch metric、executed instruction 和时间，并使用多种数据分布测试。低 efficiency 且 path 很短只是候选信号，不是收益保证。

若 branch 很重，保留 divergence、按数据分组或使用独立 kernel 可能更合适。SIMT 优化的目标是减少昂贵且无必要的多 path execution，同时保留正确性、load balance 和 locality；不需要把所有剩余 conditional 都隔离到独立 kernel 或 warp-sized region。

CUDA compiler 常会自动对简单 condition 选择 predication、select 或 branch；性能关键 kernel 值得检查生成代码并 profiling，但手工改写只有在实测更快且语义等价时才应保留。

使用 Warp Intrinsic 高效进行 Warp 内通信

当同一 warp 内的 thread 需要共享数据以完成 reduction 或 aggregation 时，可以使用 `__shfl_sync`、`__shfl_down_sync` 等 warp shuffle intrinsic（见第 6 章），让参与 lane 直接交换 register value。另一种方案是把数据暂存到 shared memory，并在需要 block-wide synchronization 时调用 `__syncthreads()`；后者会增加 shared-memory instruction、storage 和 barrier 成本，但并非必然降低整体性能。

Shared-memory 方案产生 shared-memory/L1 路径上的访问，而不是原文所称的额外 L1/L2 traffic；是否需要 full-block barrier 也取决于 producer/consumer 范围和所用同步 primitive。Shuffle 不使用 shared-memory staging，但 `_sync` intrinsic 只等待给定 mask 中、实际执行同一 intrinsic 的参与 lane，并不隐式同步固定的 32 个 lane。其 latency 和 throughput 随 architecture、instruction 与 dependency chain 变化，不能统一为“几个 cycle”。

有关性能比较、代码示例以及 warp-level reduction 或 prefix sum 的完整讨论，参见第 6 章。若协作跨越多个 warp，通常需要适用作用域的 storage 与 synchronization，例如 block 范围的 `__syncthreads()`，或 Cooperative Groups 提供的相应 group primitive（见第 10 章）。Grid-wide synchronization 则要求 cooperative launch 等特定机制，不能由 block barrier 实现。对于纯 warp 内通信，shuffle 常是有力候选，但 shared memory 可能因 data layout、broadcast/reuse、类型支持或后续跨 warp 消费而更合适。

总之，若算法需要同一 warp 的 lane 交换中间结果或协同计算，可优先评估 `__shfl_sync` 及相关 warp-level primitive，并正确构造 participation mask。即使协作没有跨 warp，若数据需要多次复用、按不同索引访问或与其他 shared-memory tile 组合，使用 shared memory 和适当同步仍可能是更好的设计；`__syncthreads()` 只应在确实需要 block-wide barrier 时使用。

PyTorch 中的 Warp-Level Efficiency 注意事项

在纯 Python 层，PyTorch 不直接暴露 CUDA warp intrinsic，也不允许用户直接控制 warp-level execution。不过，许多 PyTorch 内部 CUDA kernel 会使用 warp-level primitive，因此高层用户可能间接受益。

例如，`torch.sum(x, dim=0)` 对 tensor 某一维做 reduction。某些 shape、dtype、layout、device 和 backend 路径会用 warp-level reduction；reduction dimension 较小或不超过 32 并不保证一定选择该实现。适用时，warp 内 thread 可通过 shuffle 协作生成 partial sum，也可能结合 shared memory 或其他 reduction strategy。具体逻辑由 library 实现，用户通常无需自行编写。

若以 PyTorch extension 编写自定义 CUDA kernel，可以在适用时使用 warp-level 优化。例如需要在 warp 内求和或交换 thread 数据时，可比较 `__shfl_sync` 与 shared-memory 方案的正确性和性能，而不是无条件偏好前者；同时应检查自定义 control flow 的 warp divergence。

若高层代码包含逐 element 的 `if`，可以用 `torch.where` 等 tensor operation 表达 condition，使 backend 有机会采用 elementwise/fused kernel。不过这不保证 warp-

coherent execution；并且作为函数调用传入的两个 tensor expression 通常会在调用前求值，不能用它跳过某一侧的昂贵或无效计算。

PyTorch compiler 与 graph execution 机制（相关内容见第 10 章）可以 fusion 某些 elementwise operation。Fusion 常通过减少 launch 和 intermediate-memory traffic 获益，只有在减少 moved byte 或增加每 byte 的有效 operation 时才会提高 arithmetic intensity；它也可能增加 register pressure 或 code size。

多个 operation fusion 到一个 kernel 后，warp 可在一次读取/写回路径中完成更多工作。但 fusion 不会自动消除算法固有的 divergence，也不保证 divergence overhead 会被摊薄：若 fused region 增加 divergent work，成本甚至可能上升。

例如 ReLU 后接 `abs()` 时，fused kernel 可能一次完成两者并省去 intermediate tensor traffic。最终是否存在 branch 取决于 codegen；把两个 operation 放进同一 warp execution 并不能单独证明 divergence 更低。

CUDA programmer 应减少经 profiling 证明昂贵且不必要的 intrawarp divergence，并在适合的 warp collective 中使用正确的 warp intrinsic。这有助于提高 active-lane utilization，却不能保证所有 32 个 thread 始终同时做有用工作，因为 boundary、algorithm 和 participation mask 都可能使部分 lane inactive。

PyTorch library 已在许多路径中实现这些优化，但并非每个 operator、shape 和 backend 都一样。高层用户通常应优先使用内置 tensor operation，而不是 Python 逐 element loop；前者把工作交给 optimized backend 并减少 interpreter/launch overhead。使用自定义 CUDA extension 时，则应应用前述原则并对目标输入 实测。

让 warp lane 对当前 instruction 保持较高的有效参与度是有用目标，但并非任何时刻 32 lane 全部一致执行才最优。减少 divergence 和选择合适的 communication primitive 的 speedup 没有通用 1.1×–1.3× 区间，可能接近零，也可能更大，取决于原始瓶颈和改写成本。

这些优化也为 instruction-level parallelism 的讨论提供背景：ILP 关注单个 thread/warp 中 independent instruction 的重叠，而不是 active-lane 比例本身。下面继续介绍。

暴露 Instruction-Level Parallelism

如 occupancy 讨论所述，并发 resident warp 为 SM scheduler 提供切换候选：某个 warp 因 global-memory load 等 long-latency operation 暂时不能发射 dependent instruction 时，scheduler 可选择其他 eligible warp。除了提供足够的 thread-level parallelism，还可在单个 thread/warp 内暴露 ILP，使其在一条 instruction 的结果尚未就绪时，有其他 independent instruction 可供调度。

可以重排或展开代码，让每个 thread 在消费结果前形成多个彼此 independent 的 operation，例如多条 load 或 arithmetic chain。Compiler 与 hardware 才能在 dependency、pipeline、scoreboard 和资源允许时把它们安排为重叠执行；source 中并列写出并不保证实际 issue pattern。

ILP 使 warp 有机会连续发射某些 independent instruction，从而帮助 latency hiding。例如 thread 发起 load 后，可在不依赖该 load 的 arithmetic 已就绪时继续执

行。图 8-8 示意多个 instruction 在 pipeline 中重叠；它不是逐 cycle 的通用硬件时序保证。

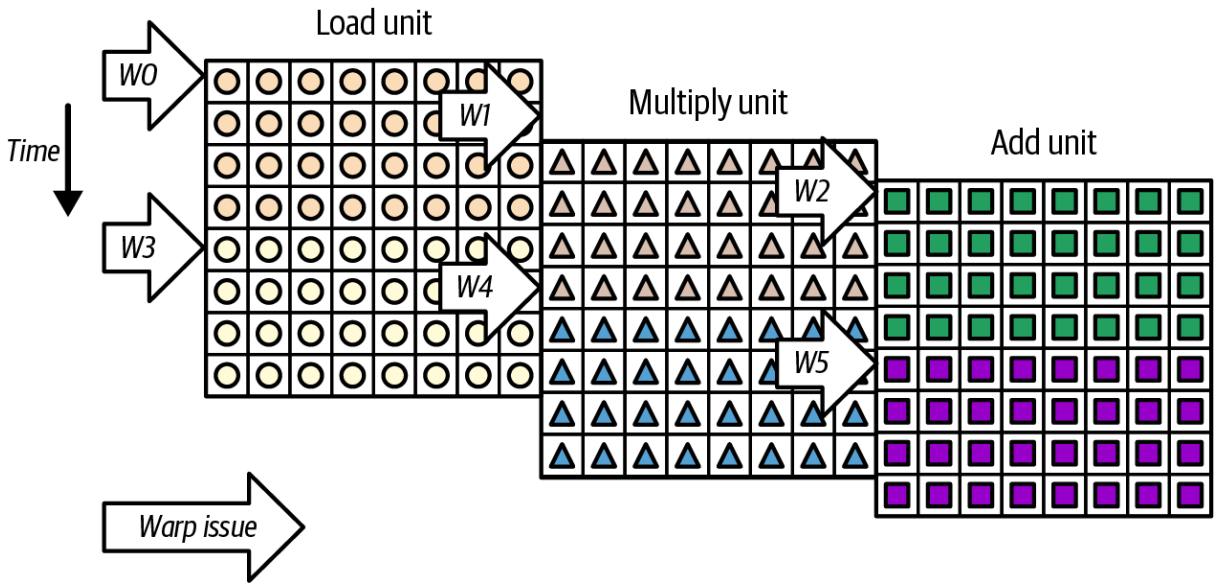


图 8-8. 使用 ILP 重叠 instruction

展开 loop body 可以把每轮依次处理一个 element 的 load、multiply 和 accumulation，改写为在回到 loop header 前处理多个 element，从而暴露更多候选 instruction。例如，先看未展开版本：

```
for (int i = 0; i < N; ++i) {

    float ai = a[i];       // load
    float bi = b[i];       // load
    sum += ai * bi;        // multiply after both loads complete
}
```

下面把每轮改为两个 element，使两组 load 和 multiply 在数学上彼此独立，为 hardware 提供 overlap 机会。第二组 operation 可能在第一组仍处于 pipeline 中或等待 operand 时推进；是否实际如此由 compiler schedule、memory dependency 和 target pipeline 决定。

这种改写可能提高 IPC 或 latency hiding，但不能保证。下面计算两项乘积 `ai0*bi0` 和 `ai1*bi1`；最终仍写成单个 `sum +=` expression，compiler 如何形成 multiply/add dependency chain 需看生成代码：

```
for (int i = 0; i + 1 < N; i += 2) {
    float ai0 = a[i];        // load a[i]
    float bi0 = b[i];        // load b[i]
    float ai1 = a[i + 1];    // load a[i+1]
    float bi1 = b[i + 1];    // load b[i+1]
    sum += ai0 * bi0 + ai1 * bi1;
}
```

这里先加载两组值（`a[i]`、`b[i]` 与 `a[i+1]`、`b[i+1]`），再形成两项独立乘积，给 compiler 暴露更多 scheduling freedom。不过代码的 loop condition 是 `i + 1 < N`，没有处理奇

数长度的最后一个 element；作为完整实现必须增加 tail handling。单一 sum accumulator 也保留跨 iteration dependency，若它成为瓶颈，可在保证数值语义可接受时评估多个 partial accumulator。

仅重排相同 operation 通常不直接改变 algorithmic arithmetic intensity（FLOP/byte），但可能通过重叠 independent work 提高 achieved throughput。Unrolling 还可能改变 instruction overhead、memory transaction、register pressure、spill 和 occupancy，因此既不保证 dual issue，也不保证“没有额外工作”；应以生成代码和执行时间判断。

> 需要区分“暴露更多 independent operation”与“增加算法所需 operation”。理想重排不改变有效工作量，但 unrolling 可能增加 code、register 和 control overhead。ILP 主要在 dependency/latency 造成 eligible issue opportunity 不足时有帮助；若相关 pipeline 已饱和，更多 ILP 通常不会提高该 pipeline 的吞吐上限，还可能因资源压力降低性能。

为向 compiler 暴露 ILP，可以对合适的短 loop 评估 `#pragma unroll`，并检查 register report。原文所写的 `-maxrregcount`（规范 nvcc 拼写为 `--maxrregcount`）设置每 thread register 使用上限，并不是“允许 compiler 分配更多 register”的开关；过低会限制可用 register 并可能 spill。若要容纳更多 intermediate value，应避免不必要的 cap，同时权衡 occupancy。

因此，ILP 与 occupancy 提供不同的 latency-hiding 来源：更多 eligible warp 让 scheduler 可在 warp 之间选择，单 warp 内更多 ready independent instruction 则增加该 warp 的发射机会。高 occupancy 不保证 warp eligible，ILP 也不保证填满 pipeline。与 CPU superscalar/out-of-order execution 的类比有限，因为 GPU warp scheduling、scoreboard 和 compiler scheduling 的机制不同，必须依据目标 architecture 与测量结果判断。

Warp Scheduling 与 Dual-Issue Instruction

每个 SM 包含一个或多个 warp scheduler。某些 Blackwell 产品的 SM 可按四个 scheduler/processing block 描述，但 scheduler 数、dispatch port 和可发射 instruction 组合必须依据具体 GPU、compute capability 与官方 tuning guide。原文笼统声称每个 scheduler 每 cycle 最多发射两条 independent instruction；图 8-9 只能作为概念示意，不能替代目标 architecture 的 issue rule。

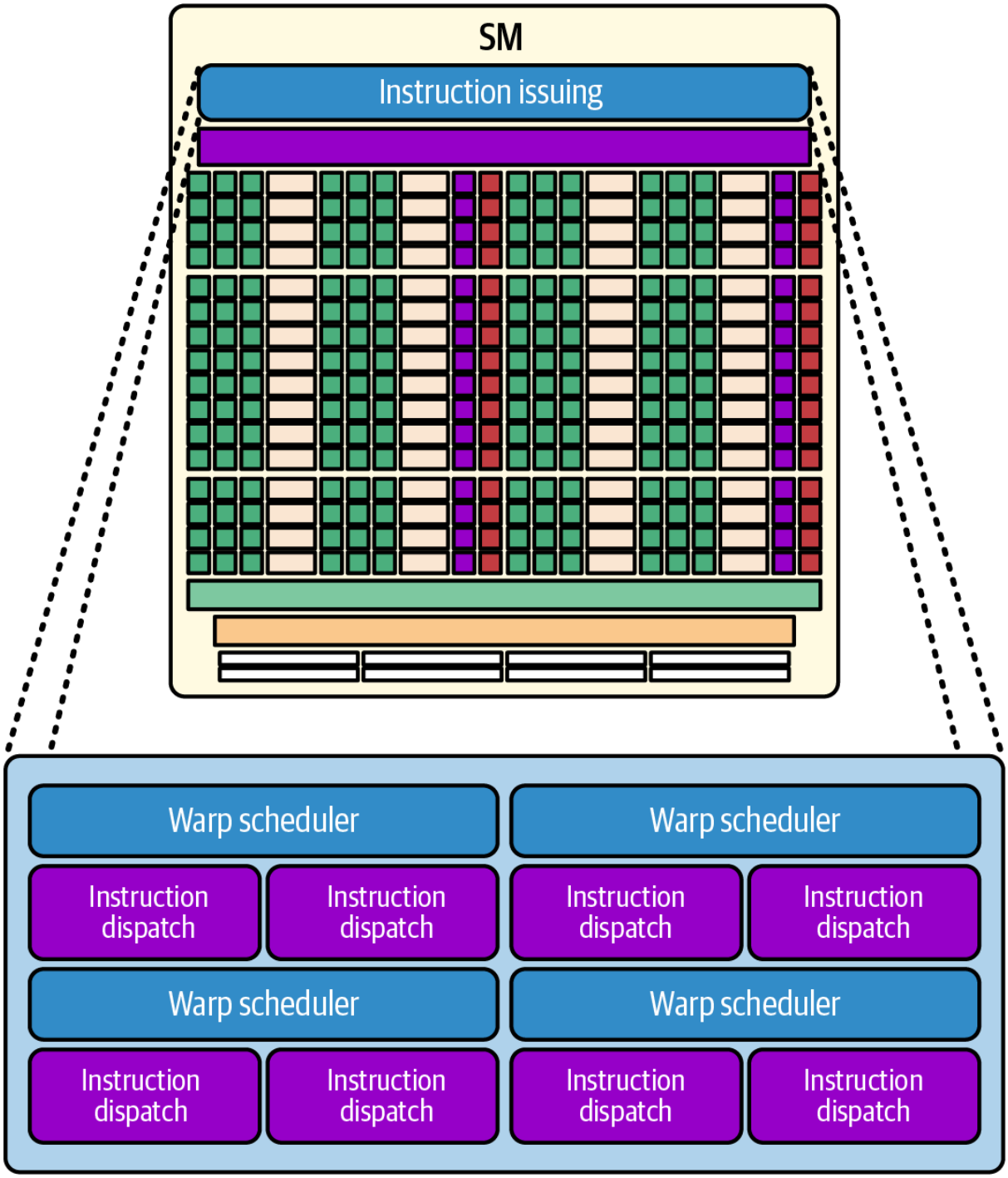


图 8-9. 原书所示的每 warp scheduler “dual issue” 概念图

实践中，单个 warp 的多条 independent operation 可以在连续 cycle 中进入不同 pipeline，并在硬件允许的 instruction pairing/dispatch 条件下可能于同一 cycle 被发射。Math、memory 和 SFU pipeline 是否可由同一 scheduler/warp 配对发射具有 architecture-specific 约束。多条 instruction 同时处于 pipeline 中是 ILP 的一种体现，但 in-flight 不等于同 cycle issue。

原文称所有 Blackwell 的 FP32 与 INT32 pipeline 已完全合并为 unified CUDA core，因此每 core 每 cycle 只能二选一，并进一步推导 FP32/INT32 mixed stream 不再能 dual issue。这种“per core”叙述不足以准确描述 warp instruction 的 scheduler/partition/pipeline issue 规则，也不能无产品限定地外推全部 Blackwell。应查询目标 compute capability 的 instruction throughput 与 tuning documentation，并用 SASS/metric 验证。独立 math 与 memory instruction 常能提供有用 ILP，但两条 math（包括 FP32/FP16）是否可配对及是否提高 issue efficiency，同样取决于实际 pipeline、dependency 和资源压力。

ILP 不要求两条 instruction 在同一 cycle 发射。若一条 long-latency load 尚未完成，而该 warp 另有 operand 已就绪且不依赖该 load 的 instruction，scheduler 可能在后续 cycle 发射它；“immediately”并无保证，还受 warp eligibility、dispatch port 和 pipeline availability 限制。

若 arithmetic 与 pending memory operation 独立，它可以在数据返回前取得进展，从而覆盖部分 observed latency。Blackwell 及其他现代 GPU 可让单 warp 的多条 instruction 因 pipelining 同时 in flight，但数量与顺序并非固定。实际 ILP 应从 dependency chain、issue activity、stall reason 与生成代码判断。

考虑每个 thread 从两个数组加载 element，分别计算后再组合结果的场景。若第一项 arithmetic 不依赖第二项 load，它可能与后者的 memory latency 重叠。原文称乘以不同 constant，但下面代码实际计算的是 `x*x` 与 `y*y`。

这种 independent chain 为 overlap 提供机会，却不保证 execution unit 始终忙或 throughput 必然提高。下面是暴露 ILP 的示例：

```
// Launch configuration: e.g., 256 threads per block, enough blocks to cover N
__global__ void independentOps(const float *a, const float *b,
                               float *out, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        float x = a[i];
        float y = b[i];
        // Two independent operations (no dependency between u and v):
        float u = x * x;
        float v = y * y;
        // Dependent operation that uses both results:
        float sum = u + v;
        out[i] = sqrtf(sum);
    }
}
```

该 kernel 中，每个 thread 从不同数组加载 `x` 与 `y`，并计算 `u` 和 `v`。`u = x*x` 与 `v = y*y` 在数据依赖上彼此独立，因为任一项都不使用另一项结果。

相比直接写成 `sum = x*x + y*y`，显式命名 temporary 可能让 dependency 更易读，但 optimizing compiler 通常可从两种表达式识别同样的 independent multiply。它可能连续发射 `u = x*x` 与 `v = y*y`，并在 `u` 尚未 ready 时推进另一条 chain，但不保证恰好相隔一个 cycle 或性能更高。

第一条 multiply 尚在 pipeline 中时，第二条 independent multiply 可能推进；是否使用“另一个 arithmetic unit”取决于目标 pipeline。执行 `sum = u + v` 前，两项结果都

必须 ready。由此产生的多 instruction in flight 可覆盖部分 latency，但 multiply operation 本身是否构成瓶颈仍需测量。

下面把 v 的 source statement 放在使用 u 之后。原文称这会创建强制 dependency chain；实际上 v 与前述语句没有数据依赖，compiler 在语义允许时仍可把它提前调度：

```
float u = x * x;
float temp = u + 1.0f;    // some dependent use of u
float v = y * y;
...
```

Source 先计算 `u = x * x`，再形成 `temp = u + 1.0f`；只有*在完成* `temp` 后才写 `v = y * y`。只有前两项之间存在 true dependency；v 不依赖 u 或 temp。因此把它放在 temp 之后不必然强迫 GPU 等待 `u + 1.0f` 完成，compiler 可在没有 volatile、memory alias、side effect 或其他 constraint 时提前调度 v。

真正的 serial dependency chain 是 `x * x → u` 再到 `u + 1.0f → temp`，source 随后才写出 `y * y → v`。执行 `u + 1.0f` 时，hardware 并非因 source 顺序就不能开始 independent 的 `y * y`，因为 v 在数学上独立。显式提前可能帮助复杂代码中的编译分析或可读性，但不能以 source 顺序推断最终 schedule。

下面给出手工重排后的代码：

```
// Optimized ordering with better ILP:

float u = x * x;    // Independent multiply on x
float v = y * y;    // Independent multiply on y (issues immediately after u)
float temp = u + 1.0f;  // Only now use u
// ... use v or temp as needed ...
```

这里在 source 中把 `x * x` 与 `y * y` 相邻放置，即 `float u = x * x; float v = y * y; float temp = u + 1.0f;`。两条 multiply 独立，因此 compiler 有机会在 `u + 1.0f` 依赖结果前安排它们；是否在连续 cycle 发射取决于 target schedule。

当 `x * x` 的结果尚未 ready 时，hardware 可能开始处理 `y * y`。但 `temp = u + 1.0f` 只依赖 u，并不需要等待 `v`，所以原文称必须等两条 multiply 都完成后才执行 add 也不准确。

理想情况下，两条 multiply 可同时 in flight，尽早发射 `v = y * y` 可减少 dependency-induced idle cycle；这不保证对 y 的 multiply 一定同时覆盖对 x 的 multiply 与后续 add，也不保证填满 execution unit 或直接提高 throughput。Compiler schedule、instruction latency、pipeline throughput 和其他 resident warp 都会影响结果。

GPU code 的 independent instruction scheduling 主要由 compiler 与 hardware scheduler 完成，programmer 可通过清晰的 dependency structure、unrolling 或 pragma 提供机会。现代 CUDA compiler 常能识别此类简单模式，但“optimal”必须以目标 architecture 和 workload 衡量。

该示例的核心是避免引入真实的、非必要的数据依赖，并让 compiler 看见 independent work。若 thread 需要加载两个数组并分别计算，可评估交错多个 chain；手工 source 重排是否优于原写法，必须比较生成代码与性能。

ILP 与 Occupancy

GPU 的每 SM 最大 resident warp 数因 architecture 与产品而异；64 warp、2,048 thread（32 threads/warp × 64 warps）不是所有现代 GPU 或所有 Blackwell 产品的统一上限。Warp scheduler 在 dependency 与 dispatch/pipeline 条件允许时可发射相应 instruction。FP32/INT32 pipeline 的组织以及每 SM register file 容量也必须查询目标 device attribute/tuning guide；原文给出的 256 KB，即 64K 个 32-bit register，只能作为特定配置口径。Register capacity 与 allocation granularity 会影响 ILP、occupancy 和 spill。

原文进一步估计：几乎没有 ILP、每 thread 一次只推进一个 operation 的 kernel，通常需要约 1,536 active thread（48 warp，在 64-warp 分母下为 75% occupancy）才能 saturate SM execution unit。这个数字没有给出 instruction mix、latency、scheduler/pipeline throughput 或 benchmark，不能作为通用近似；所需 warp 数随 bottleneck 与 eligible-warp behavior 变化。

适量 ILP 有时能降低隐藏特定 latency 所需的 resident warp 数，但不存在只由“每 thread independent operation 数”决定的固定换算。表 8-4 保留原书的启发式示意值，用于说明 ILP/occupancy trade-off；这些数值不是 Blackwell SM saturation 的硬件规范或可复现实测结果。

表 8-4. 原书关于 ILP 降低所需 thread/occupancy 的启发式示意

ILP（每线程独立操作）	原书假设达到约 100% utilization 所需的 threads/SM	以 2,048 thread 上限换算的 occupancy
1（无 ILP）	约 1,536 thread（48 warp）	75%
2	约 1,024 thread（32 warp）	50%
3	约 768 thread（24 warp）	37.5%
4	约 512 thread（16 warp）	25%

表中把 two-way ILP 与约 1,024 active thread（32 warp，按 64-warp 上限为 50% occupancy）对应，并假设每个 warp 可在某项 pending 时保持两条 operation in flight。这只是概念模型；两条 independent operation 不保证始终 ready、连续 issue 或覆盖目标 latency，也不能据此断言 full throughput。

同理，表中把 three-way ILP 对应约 768 thread（24 warp、37.5%），把 four-way ILP 对应约 512 thread（16 warp、25%）。一个 warp 暴露四条 independent operation 并不等于替代四倍 warp：它们共享同一 warp 的 issue opportunity、register 和 instruction stream，而跨 warp concurrency 还能覆盖不同类型的 stall。

> 增加每 thread 的 independent in-flight candidate 可能在较低 resident warp 数下改善 math-pipeline utilization，但只有相关 instruction ready 且 pipeline 未饱和时成立。

因此，增加 ILP 有时能以较少 warp 接近某个 workload 的 throughput 上限，尤其在 occupancy 受 register/shared memory 或问题规模限制时。但它不保证达到 peak compute throughput，也不会自动“释放”on-chip resource；更多 per-thread state 往往反而占用更多 register。

可暴露的有效 ILP 存在实际上限。单 warp 可同时跟踪的 operation 受 instruction window/scoreboard、dependency、pipeline queue、memory subsystem、register lifetime 和 compiler schedule 等约束；不能只用一个未限定的“pending instruction 数”描述全部 Blackwell。

Compiler 还要在 register pressure、code size、instruction fetch/decode、dispatch 和 execution throughput 等约束下安排 independent instruction。Decode bandwidth 是其中一环，但许多 kernel 会先受 dependency、issue port、pipeline、memory bandwidth 或 occupancy 限制。越过任一相关上限后，继续增加 ILP 便可能收益递减。

当已有足够 independent work 使目标 issue slot 或 memory system 接近瓶颈上限时，更多 ILP 通常帮助有限，并可能因 register pressure、spill、instruction/cache footprint 和 occupancy 下降而伤害性能。不存在 five-way 或 six-way ILP 普遍无益的固定分界，应扫描实际 unroll/interleave depth。

> 原文把 Blackwell 的理想 ILP 设为每 thread 2–4 条 independent operation，并称此时 scheduler 与 cache 通常 saturated；这不是通用硬件阈值。应在目标 kernel、input 与 device 上结合 Nsight Compute 的 issue/stall、register/spill、occupancy、cache/memory throughput 和执行时间决定停止点。

Loop Unrolling、Interleaving 与 Compiler Hint

在自定义 kernel 中利用 ILP，需要寻找能在消费任一结果前暴露多条 independent arithmetic 或 memory instruction 的位置。常见模式如下，但最终 issue 顺序由 dependency analysis、compiler schedule 和 target hardware 决定。

展开短 Loop

Loop unrolling 可让每个 thread 同时维护 N 个 independent chain，例如 2 路或 4 路 accumulator；但仅展开 syntax 不会自动创建独立 accumulator。先看下面使用单一 `sum` 的 loop：

```
float sum = 0.0f;
for (int k = 0; k < 4; ++k) {
    float a = A[idx * 4 + k];
    sum += a * w[k];  // dependent on load a
}
```

可改写为下面的形式，为 ILP 提供机会。它创建四组独立的 load/multiply pair，分别对应 `a0..a3`；compiler 可在合法且有利时交错调度，而不保证“快速连续发射”：

```
float a0 = A[idx * 4 + 0];
float a1 = A[idx * 4 + 1];
float a2 = A[idx * 4 + 2];
float a3 = A[idx * 4 + 3];
float sum0 = a0 * w[0];
float sum1 = a1 * w[1];
float sum2 = a2 * w[2];
float sum3 = a3 * w[3];
float sum = sum0 + sum1 + sum2 + sum3;
```

Source 中四条 load 写在前面，但 compiler/SASS 不一定严格先发射全部 load。每条 load 的结果 ready 后，对应 multiply 才可执行；若另一些 independent load/multiply 已就绪，hardware 可能交错推进以覆盖部分 memory latency。最后的 `sum0 + sum1 + sum2 + sum3` 仍有 reduction dependency；与原 loop 的逐项 floating-point accumulation 相比，association/order 可能不同，因此结果可能有舍入差异。该改写增加可见 ILP，却不保证隐藏全部 latency 或更快。

交错 Independent Operation

也可交错使用不同 data element 的 operation，如下所示：

```
float x = A[idx];
float y = B[idx];
// If you wrote float u = x * c; then float v = y * d;,
// the multiplies are independent.
float u = x * c;  // can start as soon as x is ready
float v = y * d;  // can start as soon as y is ready, overlapping with u
```

这里，`u = x*c` 与 `v = y*d` 没有彼此的数据依赖，为 concurrent/in-flight execution 提供机会。Compiler 可能把它们相邻调度，使一条 multiply 尚在 pipeline 中时另一条推进；但 source 结构不保证 back-to-back issue，也不能保证 execution unit 每 cycle 都忙。这是暴露 ILP，而不是实际 schedule 的承诺。

使用 Compiler Hint

在短 loop 上使用 `#pragma unroll` 可帮助 compiler 展开并发现多条 arithmetic chain，但 compiler 仍可依据 trip count、optimization 和资源成本决定具体 codegen。原文建议调整 `-maxrregcount` 来“允许更多 register”，语义相反：规范 `--

maxrregcount` 设置 register/thread 上限。若 unrolled variable 需要更多 register，应避免过低 cap，并以 ptxas report、spill 与 occupancy 权衡 ILP。

总之，为隐藏 latency 并提高 throughput，应在目标 Blackwell 或其他 GPU 上寻找足够的 eligible warp 与适量 ILP，而不是同时最大化二者。当一条 instruction 等待 memory 或 long-latency result 时，只有其他 independent instruction ready 且 issue resource 可用，warp 才能继续发射。该方法可能改善 achieved FLOPS，但没有显著提升保证；register/spill、occupancy、instruction footprint 和实际时间必须一并验证。

使用较大 unroll factor 时，应在 Nsight Compute 中检查 instruction-fetch/issue stall、register、spill 与 occupancy。过度展开可能增大 code size 或给 fetch/decode/dispatch 带来压力，也可能在这些资源尚未饱和前就因 register 与 cache footprint 降低性能。

Profiling 与缓解 Register Pressure

暴露 ILP 时需要关注 register pressure。多条 independent chain 通常需要 register 保存同时 live 的 value 与结果。过度 unroll 或增加 parallel computation 可能提高 register/thread，跨过 allocation threshold 后降低 resident block/warp，或引发 spill；但 occupancy 下降本身不等于性能下降，关键是总体执行时间。

CUDA compiler 可自动展开某些 loop。手工使用 `#pragma unroll` 增加 ILP 后，可能观察到 occupancy 降低；应同时检查 register、spill、eligible warp、issue/stall 和时间，不能只凭 occupancy 判断是否“过头”。

Nsight Compute 的 Occupancy 分析可能显示 theoretical occupancy “Limited by Registers” 或对应版本中的 register limiter。这只说明 register allocation 是当前 residency 上限之一，并非自动表明配置错误。若时间或 spill 恶化，可减少 unroll factor、缩短 live range、减少 accumulator，或重新评估 launch bound；若较低 occupancy 换来足够 ILP 且更快，则无需回退。

对于 `__launch_bounds__`，降低可选的 `MinBlocksPerSM` 会放宽 compiler 为该 bound 推导的 register threshold，可能允许每 thread 使用更多 register；提高 `MaxThreadsPerBlock` 通常会收紧或改变约束，并不普遍让 block 使用更多 register而避免 spill。调整 launch bound 也不会“恢复 occupancy”与“降低 register pressure”同时发生；应检查 ptxas 输出和实际 residency。

从 profiling 角度看，若 ILP 缓解了相关 dependency stall，目标版本中对应的 execution-dependency stall 可能下降，issued instruction/cycle 可能上升。但 stall percentage 是归一化分布，其他 stall 的变化会影响比例；ILP 也可能转而暴露 memory、throttle 或 resource bottleneck，因此这些不是必然方向。

Nsight Compute 提供若干 *Issue Efficiency* 或 *IPC* 相关 metric，具体名称、分母和聚合层级随版本与 architecture 变化。ILP 有效时某些 issue metric可能提高，但从 1.0 到 1.8 的 aggregate IPC 不能直接解释为“单 warp 在每个 scheduler 上平均从一条变成接

近两条 instruction/cycle”。必须确认 metric 定义、是否按 SM/subpartition/scheduler 归一化，以及 architecture scheduling constraint。

也可能在较低 occupancy 下获得更好性能。合理状态是有足够 eligible warp 和 independent ready instruction 覆盖关键 latency，同时不过度牺牲 register、locality 与 occupancy；“每个 warp 都忙”和“SM 都忙”需要用适用 metric 定义，而非作为绝对理想值。

至此，我们讨论了 warp-level parallelism 与 instruction-level parallelism，它们都能增加可调度的有效工作，但机制不同。接下来进入本章总结；后续章节还将继续讨论 arithmetic intensity，即相对于 memory traffic 完成多少有用 computation。

要点总结

本章介绍了如何发现 GPU kernel bottleneck，并在适用时通过减少 global-memory traffic、利用 on-chip resource、提高 pipeline utilization 等方式缓解它们。遵循 profiling、诊断、优化、再 profiling 的循环，可以逐步改善 underutilized、latency-limited、bandwidth-limited 或 compute-limited workload；但目标不是把所有 kernel 都变成 compute-saturated，而是在正确性和资源约束下缩短端到端时间。主要要点如下。

使用 Nsight 与 `torch.profiler` 进行 Profiling

> 先使用 Nsight Systems 查看端到端 timeline，定位 CPU-GPU gap、kernel overlap、communication 和 NVTX range；再用 Nsight Compute 的适用 stall metric、roofline 与 occupancy 分析深入目标 kernel。PyTorch 代码可通过基于 Kineto 的 `torch.profiler` instrumentation 关联 Python-level operator 与 GPU activity，但映射会受 asynchronous execution、fusion 和 runtime 影响。PyTorch profiler 的 `with_flops` 仅对有限 operator 按公式估算 FLOP，不读取 GPU hardware counter；需要硬件 counter 时，应在可接受 profiling overhead 的条件下分析目标 kernel。

调优 Occupancy 以覆盖 Latency

> 足够的 eligible warp 可帮助覆盖 memory 与 instruction latency，但所需数量依 ILP、pipeline、memory behavior 和 architecture 而异。可以通过重构、launch configuration 或 compiler option 调整 register/thread 与 shared memory/block，并扫描符合资源限制的 block size；128–256 thread 只是常见候选，不是固定最佳值。不存在跨 kernel 通用的 50%–75% optimal occupancy 或 Blackwell 32–48 warp 目标，应以 runtime、spill、eligible/issued warp 与 locality 决定。

减少 Warp Divergence 以改善 SIMT Efficiency

> 同一 warp 内采用不同 control-flow path 时，hardware 会以相应 active mask 执行 path，部分 lane 在某些 instruction 上被屏蔽。应减少经 profiling 证明昂贵且不必要的 divergence，而非要求所有 thread 始终走同一路径。Ternary、`torch.maximum` 等表达可能生成 select/predication，warp-vote intrinsic 则提供 mask；

compaction 仍需显式算法实现。这些方法可能改善 active-lane efficiency，但要计入额外计算、重排、memory traffic 和数值语义。

识别 Performance Regime

Underutilization、latency、memory throughput 与 compute throughput 是有用的诊断维度，但不是每个 kernel 必须且只能落入其一的四个互斥 bucket；瓶颈还会随 input、phase 和配置改变。应结合 achieved FLOPS、bandwidth、roofline、timeline、source correlation 与 stall reason 建立证据链。例如 Long Scoreboard 较高表示 warp 常在等待相应 long-latency dependency，可能来自 global/local/texture 等数据路径；它本身不足以判定整个 workload 为 latency-bound。

暴露 ILP

通过适量 unroll、消除不必要的 dependency chain 和 prefetch，可以向 compiler/hardware 暴露 independent instruction；这不表示每个 thread 每 cycle 能发射多条 operation。Two-way 或 four-way ILP 不会固定把填充 SM 所需 warp 减半，也不保证 execution-dependency stall 下降或 IPC 上升。应联合扫描 ILP depth、thread/block、register/spill、occupancy 与 input，比较生成代码和执行时间。

迭代优化与验证

每次修改 occupancy、ILP structure、tiling 或 precision 后，都应重新 profiling 并验证目标 bottleneck 与端到端时间。成功不要求 memory stall、dependency stall、achieved occupancy 和 FLOPS 全部同时改善，因为优化可能改变工作量和瓶颈分布。降低 precision 时，应与经过认可的高精度/reference baseline 比较，使用适合任务的 absolute/relative tolerance、domain invariant 和 end-to-end quality metric；baseline 不一定必须是 FP32，简单 assert 也未必足够。

结论

有效调优 GPU 性能需要迭代流程：先用 profiler 测量端到端行为与目标 kernel，再识别主要限制，例如 compute、memory、launch/synchronization 或 interconnect，随后应用与证据相符的优化并重新测量。瓶颈可能在修改后迁移，因此每轮都要验证正确性与实际时间。

新一代 GPU architecture 往往提高 compute、memory 和 communication 能力，同时引入新的数据路径、resource constraint 与 programming model。要接近目标 workload 的硬件上限，需要依据具体产品、software version 和官方资料持续更新测量与调优方法，而不是套用跨架构固定规则。

本章讨论了如何调优 occupancy、减少昂贵的 warp divergence 以及暴露适量 ILP，也介绍了如何使用 profiler 关联 CPU activity、kernel execution 与多 GPU NVLink traffic，并通过 Nsight Compute 深入分析单个 kernel。所有 metric 都应结合其版本定义、source correlation 和端到端结果解释。

后续章节将在此基础上讨论 kernel-level efficiency 与 arithmetic intensity，并介绍 Tensor Core 等 compute resource、Blackwell 特定的 TMEM 路径，以及受支持 GPU 上的 TMA data transfer。对于支持 TMA 的 architecture，TMA 适合某些 global/shared multidimensional bulk copy 与 asynchronous pipeline；它并非所有 global-to-shared transfer 的统一首选，是否采用仍取决于 copy shape、alignment、reuse、latency、software support 和实测性能。

第 9 章. 提高 CUDA Kernel Efficiency 与 Arithmetic Intensity

即使用大量 parallelism 与较高 ILP 覆盖了部分 latency，kernel 性能仍可能受每次 memory traffic 完成多少有效工作的限制。*Arithmetic intensity*，也常称 *operational intensity*，通常表示相对于某一明确 memory level 传输的每 byte 数据所执行的 operation 数；对 floating-point workload 常写作 FLOP/byte，而不是 FLOP/s per byte。

许多新 GPU 的 peak compute throughput 增长快于 off-chip memory bandwidth，使 compute-to-bandwidth ratio 上升。提高 data reuse、减少不必要 traffic 因而更重要；但是否需要提高 arithmetic intensity，以及提高后能否利用更多 compute capability，仍取决于算法、所选 precision、实际 bandwidth 与其他 bottleneck。

Arithmetic intensity 是 Roofline performance model 的关键坐标。Roofline 把 achieved performance（FLOP/s）相对于 arithmetic intensity（FLOP/byte）作图，并用选定 memory bandwidth 与 compute throughput 构造 ceiling。它帮助判断测得点更接近 bandwidth roof、compute roof，还是远低于两者；后者还可能受 latency、occupancy、instruction mix 或实现效率限制。

实践中可使用 Nsight Compute 的 Roofline analysis view 等工具生成图表。解释结果时必须确认 FLOP 计数、byte 所属 memory level、dtype 对应的 compute roof 和实际 clock/configuration。随后可在每次优化后重新测量点的位置与端到端时间，而不是仅根据标签判定 memory-bound 或 compute-bound。

优化目标不是一律把 kernel 推入 compute-bound regime，而是接近适用于该算法和数据路径的 attainable roof，并缩短目标 workload 的时间。Roofline 可指导优先调查哪类 ceiling，但不能单独决定具体优化。

如前章所述，经典 Roofline 用水平线表示选定 operation/dtype 的 peak compute throughput，用从原点出发的斜线表示由某层 memory bandwidth 与 arithmetic intensity 决定的上限。Kernel 的 arithmetic intensity 决定横坐标，measured performance 可与这些 ceiling 比较，如图 9-1 所示。

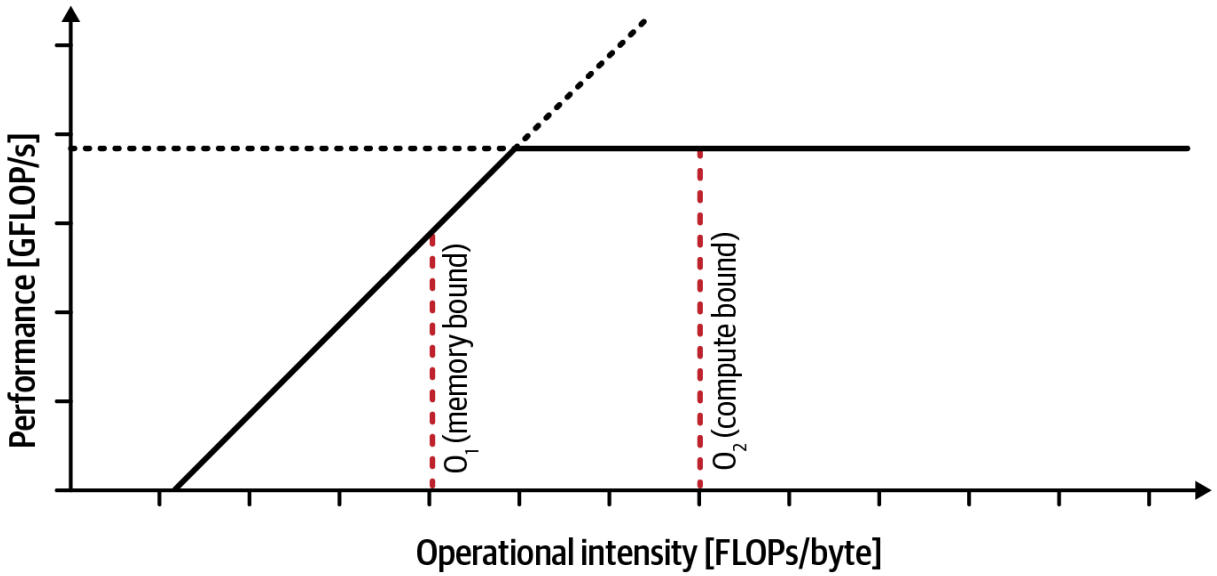


图 9-1. Roofline model 示例（GFLOP/s 与以 FLOP/byte 表示的 arithmetic intensity）

Arithmetic intensity 较低的 kernel 在经典 Roofline 中更容易落在 bandwidth sloped roof 下，但“低”本身不证明 achieved performance 已受 memory bandwidth 限制；若带宽远未饱和，还可能是 latency、访问不合并、并行度不足或其他原因。

反之，较高 arithmetic intensity 只会提高 bandwidth roof 与 compute roof 的交点机会，不保证 kernel 已 compute-bound，更不保证 ALU 或 Tensor Core 接近 peak。Instruction mix、dependency、data layout、Tensor Core eligibility 和 occupancy 都会影响实际 utilization。

可行时应通过减少不必要的 global-memory byte、提高 data reuse 或融合 intermediate traffic 来提高 arithmetic intensity，而不是为了增大比值添加无用 computation。常见方法包括 loop tiling、利用 register/L1/shared memory 复用数据，以及 fusion 多个 kernel 以避免 intermediate result 往返 global memory。

> PyTorch TorchInductor 等 compiler framework 可在支持的 graph、shape 和 backend 上执行某些 fusion、layout 与 code-generation 优化，从而减少 off-chip traffic 或提高 effective intensity；它不保证保持数据“最优复用”或始终优于 library/custom kernel。Developer 仍需 profiling 目标 workload，并只在必要时组合技术或编写自定义 CUDA kernel。

还可在数值要求和硬件支持允许时使用 FP16、FP8、FP4 等较低 precision，以减少每 element byte，并使用适用 Tensor Core 路径提高 compute throughput。若 operation count 的定义保持一致而 byte 减少，measured FLOP/byte 可能上升；但 lower precision 同时改变 compute roof、conversion/scaling/metadata traffic 与数值行为，不能笼统说会自动提高 arithmetic intensity 和性能。下面讨论这些技术。

并非每个 workload 都能轻易提高 arithmetic intensity，它受 algorithmic data movement lower bound、reuse opportunity、working-set capacity 和 correctness constraint 限制。可以评估 algorithm 改进、data reuse、operation fusion 与

batch/shape 调整，但更大 batch 不一定提高单 kernel intensity，也可能增加 memory footprint 和 latency；任何改写都必须保持可接受的数值与任务结果。

多级 Microtiling 与 Software Prefetching

如第 7 章所述，tiling（也称 *chunking* 或 *blocking*）与 data reuse 是提高 arithmetic intensity 的常用方法。把 A/B 的 submatrix tile 从 global memory 加载到 shared memory，可让一次 off-chip fetch 服务多次 multiply-accumulate；收益取决于 tile shape、reuse、bank/layout、occupancy 与实际 cache behavior，不能只由“SRAM 更快”概括。

若一个 element 从 DRAM 加载一次后在片上复用 R 次，针对该 memory level 的 byte traffic 理想情况下可按复用程度 amortize，但 FLOP/byte 不一定严格乘 R，因为还存在 output、metadata、边界和其他 traffic。以典型 32×32×32 GEMM tile 为例，整个 output tile 包含 1,024 个 output element 和 32,768 个 multiply（或相应 FMA）；每个 A 或 B tile element 通常参与约 32 个 multiply，而不是原文所称的每个 shared-memory element 产生 1,024 个 independent multiply。

在 shared-memory tiling 之外，multilevel tiling 可进一步组织 register/shared-memory reuse并暴露 ILP。把 tile stage 到 shared memory 后，每个 thread/warp 可按 microtile 将所需 value 加载到 register；`float4` 与 `<half2>` 是原书示例，其中 CUDA 类型通常写作 half2 而不是带尖括号的模板形式。Vector type 只在 alignment、layout 与 codegen 合适时产生有利 vector access。图 9-2 展示了概念层级。

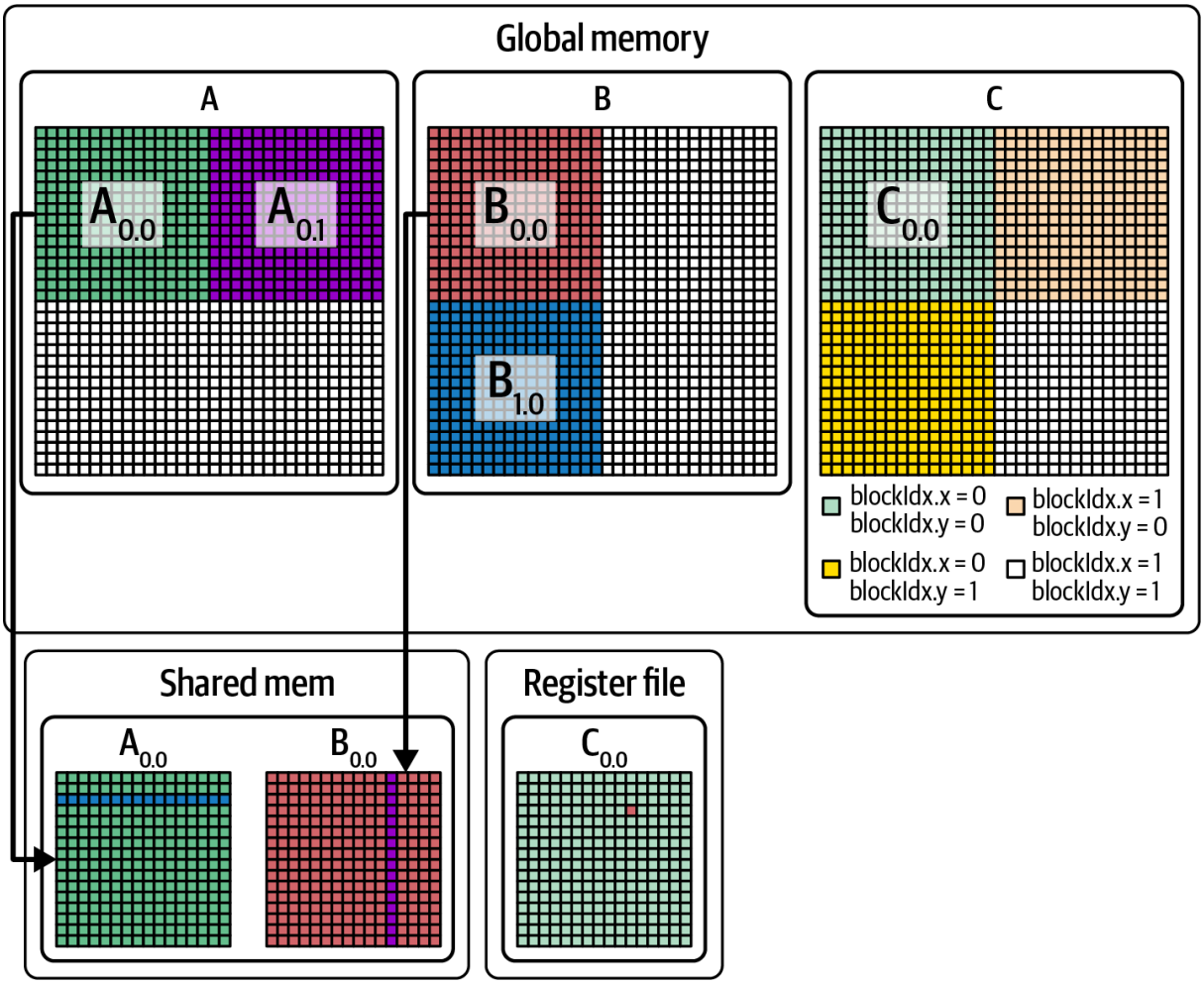


图 9-2. Global memory（DRAM）、shared memory（SMEM）与 register 之间的 multilevel tiling

这种 intra-SM reuse 的数据供给方向通常是 DRAM → SMEM → register，而 value 也可能在层级间写回或重新读取；原文的 register → SMEM → DRAM 顺序不能概括 tiling load path。填充 shared memory 时应尽量 coalesce global access，并在确有 conflict 时使用合适 padding/swizzle，相关内容见第 7 章；这些变换也有 capacity 与 instruction 成本。

Tensor Core inner loop 可通过 WMMA、MMA、CUTLASS 或其他 library/API 表达，实际 operand staging 依 architecture。TMEM 是特定 Blackwell compute capability 上的独立 address space，tcgen05 API 所对应的 `tcgen05` instruction family 包含 MMA、copy、load/store 等不同 operation；不能笼统说 Tensor Core instruction 会把 shared memory 隐式 stage 到 TMEM，也不能外推全部现代 GPU。Compiler/library 可能管理部分资源，低层实现仍需遵守显式 allocation、operand layout 与 synchronization contract。Developer 也可在支持的 path 上使用 `cp.async` 或 TMA 把 tile 移入 shared memory。

相关技术是 software prefetching，常与 double buffering 结合：当前 tile 计算尚在进行时，提前发起下一 tile 到 shared memory 的 asynchronous copy。只有 copy 与 compute 能并行、buffer 生命周期和 barrier 正确、且资源压力可接受时，才能覆盖部分 DRAM → SMEM latency；它不保证 ALU 永不缺数据，也可能因增加 shared memory/register 使用降低 occupancy。

在 Grace Blackwell 等 CPU-GPU platform 上使用 Unified Memory 时，可用 `cudaMemPrefetchAsync()` 请求把 managed-memory region 预取到指定 location。具体 placement、migration path 与是否经 NVLink-C2C 依 topology、driver/runtime、allocation state 和 API 语义而定，不应保证一定迁页；prefetch 也不能替代正确 stream ordering/synchronization。有效 overlap 可能减少 on-demand fault stall，但不保证 ALU 始终有数据或 achieved FLOPS 提高。

> Unified Memory 可简化某些数据管理，但性能取决于 access pattern、placement、oversubscription 和 platform coherence。Explicit `cudaMemcpy` 与 pinned host allocation 可提供更明确的 transfer schedule，却不“完全避免”所有 migration/transfer overhead，也不一定总优于 managed memory；应按目标 platform 实测。

总之，在有效 operation 不变的前提下，DRAM byte 在 register/shared memory 中复用越充分，针对 DRAM traffic 计算的 arithmetic intensity 通常越高。它会提高 bandwidth roof，但 measured kernel 是否更接近 compute-bound 仍取决于实际 traffic、compute utilization 与其他 bottleneck。

使用 Thread Block Cluster 进行 Tiling

在支持 cluster 的 GPU 上，可使用 CUDA thread block cluster 扩展 tiling 与 reuse；相关 cluster group API 属于 Cooperative Groups，详见第 10 章。同一 cluster 中的 thread block 可通过 distributed shared memory（DSMEM）访问彼此的 shared-memory segment，但各 block 仍拥有各自的 shared-memory allocation，并非共享一块无边界的单一 SMEM，如图 9-3 所示。

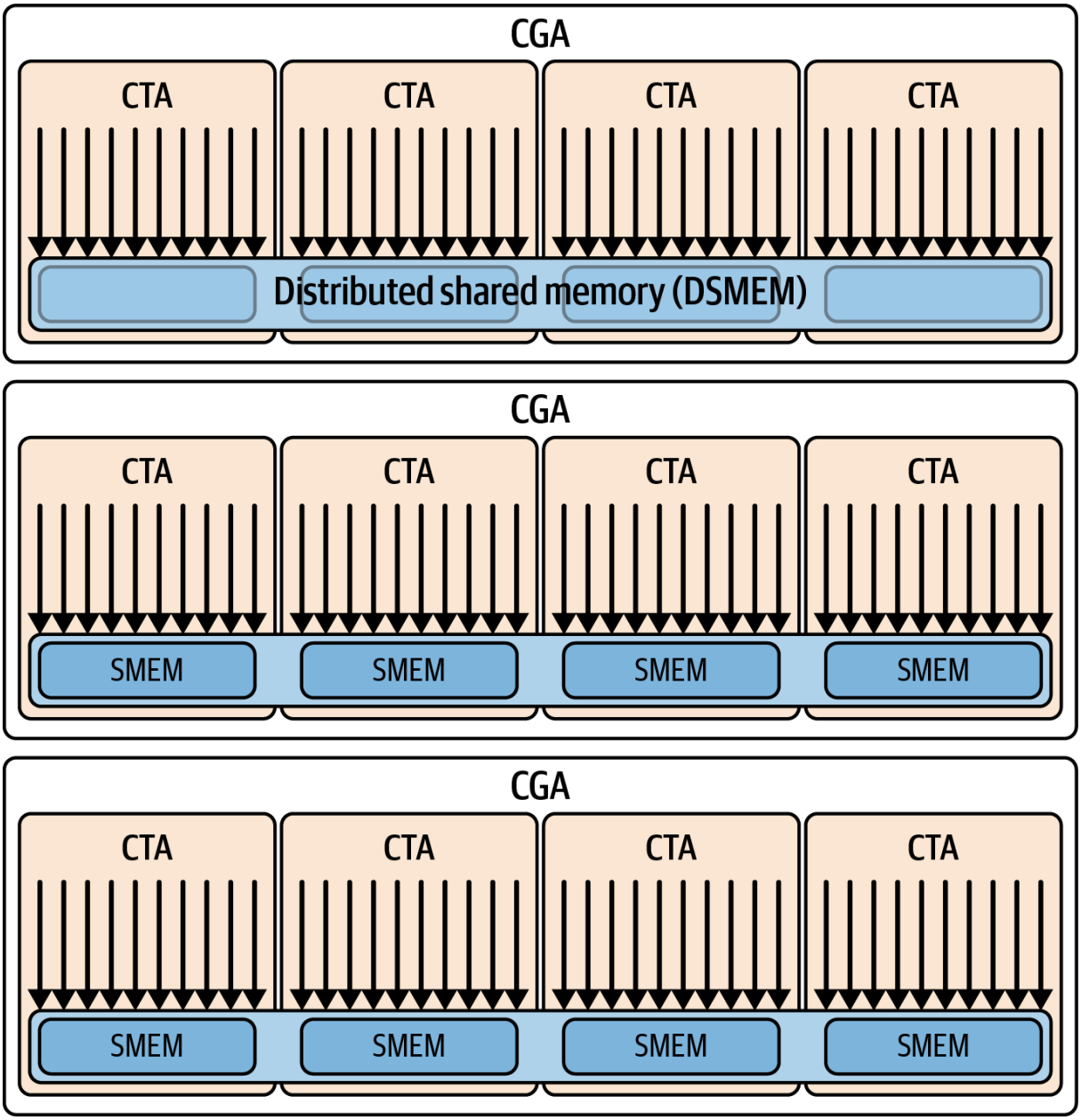


图 9-3. CGA 内各 CTA（thread block）可通过 DSMEM 访问 cluster 中的 shared-memory segment

下一章将详细介绍 CGA 与 thread block cluster。这里先说明它们如何在存在跨 CTA reuse 时减少 global-memory traffic，从而提高相对于 DRAM byte 的 arithmetic intensity。例如，TMA multicast 可把一次 bulk tensor transfer 的数据投递到 cluster 内多个 CTA 的目标 shared memory。图 9-4以四个 CTA 演示该机制；它要求支持的 architecture、合法的 cluster launch、tensor-map/layout、multicast mask 和同步协议。

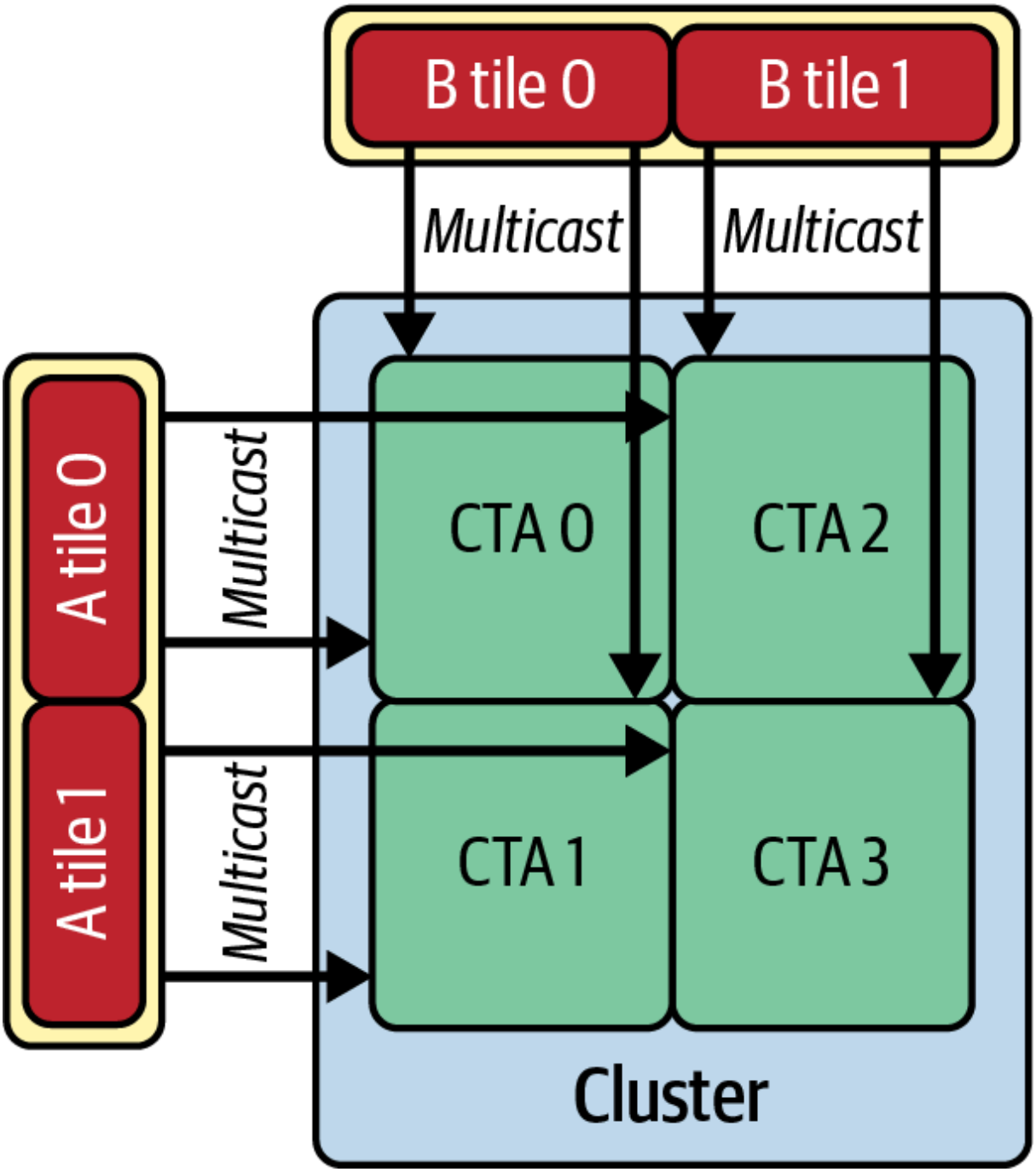


图 9-4. 在 2 × 2 thread block cluster 中，通过 multicast 将 A、B tile 投递到四个 CTA（thread block）的目标 shared memory（来源：https://oreil.ly/EEO_O）

图中的 logical tile 工作由四个 thread block 协作完成；被多个 CTA 共同需要的 A/B region 可借助 multicast 避免每个 CTA 分别从 global memory 发起一份等价读取。数据仍会写入所选 CTA 各自的 shared-memory destination，因此“只取一次”应理解为理想化地减少源端 global-memory transaction，而不是 cluster 中仅保存一份 tile。

若 2 × 2 cluster 的四个 CTA 确实复用完全相同的数据，且基线原本会产生四份独立 DRAM traffic，multicast 对该部分 traffic 的理想降幅可接近 4×。实际 kernel 的总 global-memory byte 还包含不共享的 operand、output、metadata、边界与 cache effect，因此 arithmetic intensity 不会必然按 4× 提高，整个 GPU 的性能也不会按比例增长。后文将在 Tensor Core tiling 的语境中介绍一种专门形式 *thread block pair*。

在支持相应 nonportable cluster size 的 Blackwell 配置上，thread block cluster 可选择超过 portable 上限 8、最多 16 个 CTA；可用性还取决于具体 device、MIG/configuration、launch API 与 resource constraint。Kernel 必须通过 `cudaFuncAttributeNonPortableClusterSizeAllowed` 显式允许 nonportable cluster size，并应查询 runtime/device attribute，而不能仅按架构名称假定可用。更大的 cluster 可能扩大 reuse domain，也可能降低可同时驻留的 cluster 数；只有共享数据比例和传输模式理想时，局部 reuse 才可能接近 CTA 数，不能保证 16× arithmetic-intensity 或性能提升。应对目标 shape、occupancy、DSM/TMA traffic 与运行时间进行 profiling。

内核融合

提高 arithmetic intensity 的另一种方法是把多个 operation 或 loop iteration 融合到一个 kernel 中。若 intermediate value 能留在 register、shared memory 或 cache，而不必在 kernel 间写回并重新读取 global memory，同一批输入数据便可服务更多 computation。

类似地，上一章讨论的 loop unrolling可能让单个 thread 对已加载数据执行更多 calculation 并暴露 ILP，但并不自动产生 reuse，而且常增加 register lifetime、code size 与 register usage。Fusion 也可能提高 per-thread register pressure、shared-memory footprint 或 synchronization cost，从而降低 occupancy，因此需要权衡。

应对 fused kernel 做正确性与性能 profiling。若 register allocation 导致 spill 到 local memory，新增 memory traffic 可能抵消 fusion 收益；即使没有 spill，过大的 kernel 也可能受 occupancy、instruction-cache、launch geometry 或并行度限制。只有减少目标 memory level 的 byte 且有效 work 保持不变时，FLOP/byte 才会提高；这在 bandwidth 是真实限制因素时尤其可能有益。

现代 deep-learning framework可通过 JIT compiler 与 graph optimizer 对受支持的 graph pattern 执行 fusion、unrolling 和 code generation。例如，PyTorch 的 `torch.compile`，尤其是其 TorchInductor backend，可融合某些 elementwise operation sequence；是否融合取决于 graph break、shape、aliasing、device/backend 与 cost model。PyTorch compiler 将在第 13 章和第 14 章介绍。

Elementwise operation 也称 *pointwise operation*，对 tensor 的每个 element 独立应用同一种 computation。

若这些 elementwise operation 的 intermediate value 能保留在 register 或其他片上 storage，fusion 可消除相应 global-memory round trip，从而增加每个 DRAM byte 对应的有效 work。不过，是否真的“留在片上”由 generated code、live range 和 resource pressure 决定。

例如，朴素实现启动两个 kernel：第一个读取 `x` 并把 `y` 写入 global memory，第二个读取 `y` 并写入 `z`：

```
y = sin(x);
z = sqrt(y);
```

这里的 intermediate value 需要一次 global write 和下一 kernel 的 global read，例如计算 `sin(x)` 后写出、计算 `sqrt(y)` 前读入。每个 kernel 只对每个 element 执行一个 transcendental operation，但不能仅凭 operation 个数断言其 arithmetic intensity “很低”：Roofline FLOP counting 对 special function 的处理依工具与约定，这类 throughput/latency 也不能等同于单条普通 ALU FLOP。对照之下，fused kernel 可在一次 logical pass 中完成两步：

```
z[i] = sqrt(sin(x[i]));
```

理想 code generation 下，每个 `x[i]` 只需加载一次，`sin` 的结果直接供 `sqrt` 使用，最后仅将 `z[i]` 写回。这样消除了 intermediate `y` 的 global write/read；DRAM traffic 会下降，但 point 是否靠近 compute roof 仍取决于 FLOP-count convention、special-function pipeline、memory traffic 与 achieved throughput。

> 若同一 thread block 内多个 thread 会复用数据，可把 shared memory 作为候选 staging location；但 hardware cache 可能已消除部分 redundant load，而显式 staging 会增加 copy、barrier、bank-conflict 和 capacity 成本。应比较 direct/cache path 与 shared-memory 版本的实际时间，不能假定它必然把 kernel 从 memory-bound 变为 compute-bound。

因此，这个 fusion 的明确收益是移除 intermediate materialization。不能说有效 FLOP 数因 fusion 而翻倍：fused 与 unfused 版本完成相同的 `sin` 和 `sqrt`；变化的是 byte traffic 和 launch overhead。Arithmetic intensity 应写作 FLOP/byte，并使用明确的 operation-count convention。

下面用一个更具体的例子说明。设二维 tensor `x (shape [batch, hidden])` 的每一行需要做 L2 normalization。对每行 `b`，先计算 `norm_b = sqrt(Σ_i x[b,i]*x[b,i] + ε)`，再写入 `y[b,i] = x[b,i] / norm_b`，其中该式适用于所有 `i`。

一种 unfused pipeline 可分别用 kernel 计算 square、对每行 reduction 成 scalar，再做 normalization/divide。这需要多个 launch，并可能把 elementwise intermediate 写入 HBM 后重新读取；实际 byte 数取决于 reduction 实现、temporary layout 与 cache。

原文随后称有 4 个 kernel、每个仅 1 FLOP，并按每个 kernel 都有两次 float read 和一次 float write 得出 1/12 = 0.083 FLOP/byte。这与前述 three-kernel pipeline 不一致，也不符合 square、reduction、sqrt/rsqrt 和 normalization 各自不同的 operation 与 traffic；不能用该数值代表朴素实现。

可把这些阶段融合成单个 kernel，以减少 intermediate traffic 和 launch。下面保留原书给出的手写 fused kernel；它是说明性代码，后文将指出其 correctness 与计数限制：

```
__global__ void fusedL2Norm(const float* __restrict__ x,
                            float* __restrict__ y,
                            int hidden) {
  extern __shared__ float sdata[];        // reduction buffer
  const int batch   = blockIdx.x;         // one block per batch row
```

```
    const int tid     = threadIdx.x;
    const float* batch_ptr = x + size_t(batch) * hidden;

    // 1) Accumulate sum of squares into shared memory
    float local = 0.f;
    for (int i = tid; i < hidden; i += blockDim.x) {
      float v = batch_ptr[i];
      local   = fmaf(v, v, local);          // v*v + local
    }
    sdata[tid] = local;
    __syncthreads();

    // 2) Parallel reduction to sdata[0]
    for (int offset = blockDim.x >> 1; offset > 0; offset >>= 1) {
      if (tid < offset) sdata[tid] += sdata[tid + offset];
      __syncthreads();
    }

    // 3) Normalize (guard tiny norms)
    float norm = sqrtf(sdata[0]);
    float inv = rsqrtf(sdata[0]); // prefer inverse

    float* out_batch = y + size_t(batch) * hidden;
    for (int i = tid; i < hidden; i += blockDim.x) {
      // multiply by inverse (rsqrt) vs. divide by sqrt
      out_batch[i] = batch_ptr[i] * inv;
    }
  }
```

在这个 fused kernel 中，每个 thread 两次遍历自己负责的 `x[b,*]`：第一次累积 local sum of squares，第二次写出 normalized output。若第二遍输入没有由 cache 命中且忽略 reduction traffic，则 global traffic 约为每 element 12 byte（两次 float read 加一次 float write）；若输入仍在 cache，DRAM byte 可能更少，因此必须说明计数的 memory level。按常见 FLOP convention，FMA 计 2 FLOP，normalization multiply 计 1 FLOP，即每 element 约 3 FLOP，另有 reduction add。

按上述简化的 global-byte model，elementwise 主体约为 3 FLOP / 12 byte ≈ 0.25 FLOP/byte，但原代码并非每行只执行一次 `sqrt`/`rsqrt`；其 Roofline contribution 也不能写成简单的 1/(hidden * 12)。Block 中每个 thread 都计算 `sqrt` 和 inverse square root，且 unused `sqrt` result 随后未使用。Compiler 可能删除 unused work，但 remaining `rsqrt` 仍可能由每个 thread 执行。让每个 thread 处理多个 element 可提高部分 loop ILP 或 amortize overhead，但不会自动隐藏 duplicated special-function latency。

> 代码以 inverse `sqrt`（`rsqrtf`）后乘法代替逐 element division，这是常见 micro-optimization，但 throughput、precision、rounding 与 exceptional-value behavior 可能不同，必须按数值要求验证。代码还同时计算了未使用的 `sqrtf`；若 compiler 未完全消除相关 work，会造成浪费。更合理的方案可由一个 thread 计算一次 `rsqrtf`，而不是让所有 thread 重复 `rsqrtf`，再经 shared memory 或 warp/block collective 广播，并配合正确 synchronization。是否更快仍取决于 row length、block shape，以及 generated `sqrtf`/inverse-square-root instruction；“整体必然 memory-bound”也应由 profiling 证明。更多实现细节见本书的 GitHub repository。

与包含 HBM intermediate 的 three-kernel pipeline（square → reduce → divide）相比，fused version 可移除至少一组 global write/read round trip 和 kernel boundary。它通常减少 traffic 与 launch overhead，但不能据此保证 runtime “好很多”；收益取决于 shape、cache、reduction efficiency、occupancy 和 framework overhead。约 0.25 FLOP/byte 也只是特定简化计数，不是所有 memory level 上的固定值。

在合适 workload 上，单个 fused kernel 可能因较少 global traffic、较好的 locality 和较低 launch overhead 而快于多个独立 kernel；仍应针对真实输入与 reference result 测量，不能把可能收益写成普遍事实。

Fusion 的核心并非必然把 kernel 推向 compute-bound，而是避免朴素 multikernel version 在 stage 之间 materialize intermediate。示例的 per-thread local partial sum 保留在 register，随后只把每个 thread 的 partial 写入 shared memory 以完成 block reduction；这与“`ai*ai` 全程不离开 register”并不完全相同。

Reduction 使用 shared-memory partial sum，并在每个 offset 后同步。最终的 row sum 位于 shared memory，`sqrt` 或 inverse-square-root 可使用该 sum，normalization pass 再次读取 input 并写出 result。代码没有把定义中的 ε 加到 sum 中，因此 zero row 会使 inverse square root 为 infinity，随后 0 × infinity 可产生 NaN；它也隐含要求 reduction launch configuration 适合该 power-of-two halving algorithm。

因此，不能从这段代码严格得出 fused version 固定为 4 FLOP/12 byte、unfused version 固定为 4 FLOP/36 byte。FMA、tree-reduction add、special function、input reread、shared-memory traffic 和 intermediate buffer 都需按明确范围计数。可以可靠得出的结论是：fusion 能移除某些 HBM intermediate traffic 和 launch boundary，具体 DRAM byte、latency 与 speedup 应实测。

这个例子展示了 fusion 可能如何提高相对于 DRAM traffic 的 arithmetic intensity 并改善性能。下面转向利用 GPU Tensor Core hardware unit 的另一类方法。

> 高性能 GPU kernel 常采用 vertical fusion，把同一数据上的 sequential operation 合并；也可采用 horizontal fusion，把可共同调度或共享输入的 parallel operation 合并。NVIDIA CUTLASS 与 OpenAI Triton（PyTorch compiler backend TorchInductor 使用的 code-generation technology 之一）可帮助实现部分 fused pattern。是否使用 Tensor Core、TMEM 或 TMA 取决于 operation、dtype、layout、target architecture 与 generated kernel，并非 fusion library 自动具备全部硬件路径。

结构化稀疏性

在支持相应 sparse MMA path 的 NVIDIA GPU 上，2:4 structured sparsity可由 Sparse Tensor Core hardware 加速，cuSPARSELt 等 library 提供 compression 与 sparse matrix-multiplication API。2:4 通常表示沿规定维度的每组四个 value 中保留两个；具体 grouping、layout、dtype 和合法 shape 由 operation 与 software stack 规定，不能仅理解成任意 storage order 中“四个连续 weight”。把 dense model 约束或转换成这种 pattern 通常涉及 *pruning*。

把一半 weight 剪枝并压缩成受支持的 2:4 representation 后，kernel 可只存取和计算 retained value，并携带用于描述其位置的 metadata。相较于读取 dense zero 并执行 dense multiply，这可减少 weight byte 与无效 multiply；但 metadata、activation、output 和其他 traffic 仍存在，所以每次 load 的有效 nonzero 比例与总 memory traffic 不会简单地恰好改善 2×，如图 9-5 所示。

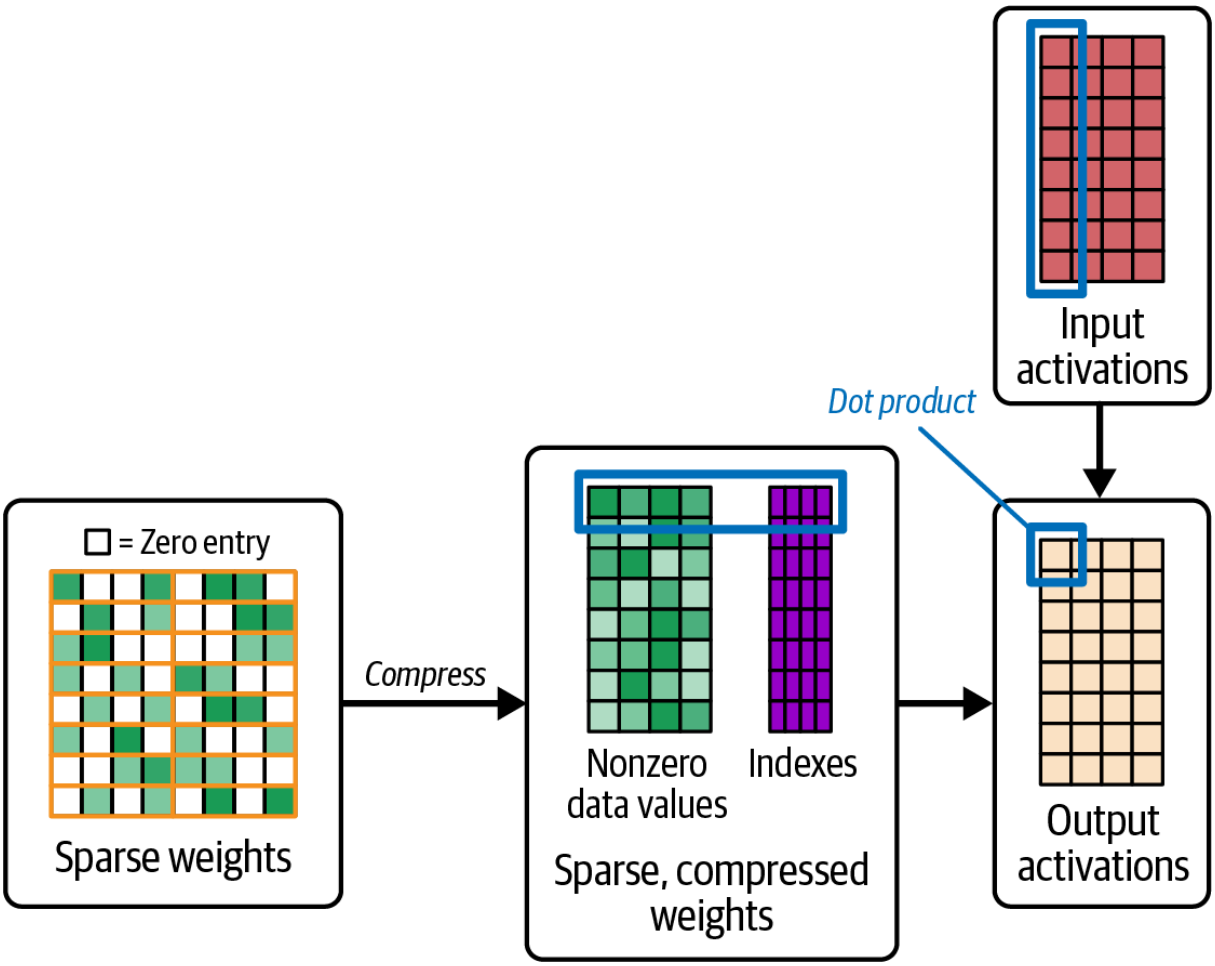


图 9-5. 2:4 structured sparsity 的 value 与 index/metadata compression

Structured sparsity 可通过 post-training pruning 与 fine-tuning 用于 inference，也可在训练或 retraining 过程中施加 mask/constraint；并非只能在模型训练结束后应用。Pruning、合法性检查、compression 和 format conversion 由 cuSPARSELt、framework tooling 或其他 software 完成。Hardware execution path 消费已经符合要求的 sparse representation，不会自动把任意 dense weight 变成保持精度的 sparse model。

Pruning 与 format conversion 通常由 cuSPARSELt 和 framework tooling 等 software 处理。在 PyTorch 中，可对满足其 dtype、shape、layout、device 和 backend 限制的 tensor 使用 `to_sparse_semi_structured()`，将已经符合 semi-structured pattern 的 dense tensor convert 为 compressed representation；该 API 本身不负责选择应剪除哪些 weight，也不保证任意 trained dense module 可直接转换或获得 sparse GEMM acceleration。

完成 conversion 并不意味着整个 model必然改用 sparse kernel；只有 supported operator、shape、dtype、layout 和 backend dispatch 匹配时，相关 matrix

multiply 才可能调用 optimized sparse GEMM。Sparse Tensor Core 对特定 sparse MMA 的 dense-equivalent peak throughput 可达到对应 dense path 的约 2×，但 application speedup 通常更低，并可能为零；较大 problem/batch 有时可 amortize launch、packing 或 dispatch overhead，但也受 latency、workspace 和 memory capacity 约束。实际收益必须逐 operator 与端到端测量。

> Batching 是提高某些 inference workload weight reuse、parallelism 与 launch amortization 的常见方法：一次处理多个 item 时，同一组 weight 可服务更多 activation computation。不过 batch 增大不保证单个 kernel 的 arithmetic intensity 上升，也可能增加 queueing latency、activation traffic 和 memory footprint。

足够大的 sparse matrix multiply 通常更容易提供并行 work，并 amortize metadata、compressed representation、dispatch 或 conversion overhead；small batch/problem 上这些成本可能占较大比例。但是否隐藏 overhead 取决于 kernel shape 与 implementation，而非 batching 自动保证。

2:4 sparsity 常在 transformer/LLM 中的大型 matrix multiply 上更有机会受益，因为这些 operation 可能满足 Sparse Tensor Core 的 tile、alignment 和 datatype requirement。Hardware 根据 compressed value 与 metadata 执行 supported sparse MMA，避免对应的 pruned multiply；所谓“同一 cycle budget 做两倍工作”是 dense-equivalent peak-throughput 口径，不代表所有 instruction、kernel 或 model 的实际 throughput 都翻倍。

Blackwell 的 compute roof 相对 HBM bandwidth 很高，使减少 weight traffic 具有潜在价值，但 structured sparsity 不会让 workload 必然“保持 compute-bound”。若压缩降低 compute work 和 byte 的比例不同，Roofline position 可能向任一方向变化；Grace Blackwell 上从 CPU memory 访问数据的实际路径、bandwidth 和 latency 也不能由 NVLink-C2C 的 peak link rate替代测量。目标仍是缩短 workload 时间，而非最大化一个 regime 标签。

把 50% weight 剪成 2:4 pattern 后，compressed weight value 的 payload 大致减半，但还要传输 metadata，activation、output 和其他 byte 也未消失。因此不能保证总 global-memory read 减半或 effective arithmetic intensity 接近 2×；还必须明确 numerator 是原 dense-equivalent work、实际 executed operation，还是 algorithmic useful work。

NVIDIA sparse MMA 对 operand 的 sparsity grouping、metadata encoding 与 supported shape 有精确定义；把它概括为“每 16 element 任意清零 8 个”会遗漏每组四个 value 的合法 selection constraint。其他 structured pattern 或 datatype 是否有 hardware/library acceleration 随 GPU architecture 与 software release 演进，也不能断言除 2:4 外任意 sparsity 永远没有特殊加速。

> Sparsity speedup 只有在 task quality 仍满足要求时才有意义。Pruning 后通常需要 fine-tuning、retraining 或 calibration，并用目标 dataset 和 metric 验证 accuracy、robustness 与数值行为；不能只验证 kernel speed。

应用 sparsity 前，应先确认真实 bottleneck、operator support 和 dense baseline。Coalescing、tiling 与 pointwise fusion 是常见候选，但并非必须按固定顺序全部手工实施；optimized framework/library kernel 可能已包含这些策略。Structured sparsity 可作为 inference 优化候选，并须与强 dense baseline 比较。

> Structured sparsity 常用于部署 inference，但“training gradient 不受益”不是普遍规则。Sparse training 是否加速取决于 forward、backward 与 optimizer 中哪些 matrix operation 有受支持 sparse operand，以及 mask/metadata 是否可维护；许多 stack 的 training support 确实比 inference 更有限，且 model-dependent、framework-dependent。应分别验证 training correctness、convergence、kernel dispatch 与端到端时间，不能从 inference support 推断 training support。

Recomputation 与 Memory 的权衡

如果目标瓶颈是 memory bandwidth 或 capacity，可考虑不存储或加载预计算值（例如 `x²`），而是在需要时 recompute。例如，在 register 中重新计算 `x*x` 可能比从 global memory 加载预先保存的 `x²` 更快，但前提是 x 已可用、expression 足够便宜、额外 dependency/compute 不成为新瓶颈。Recomputation 会增加 executed operation，并可能减少 memory traffic；它改变 measured arithmetic intensity，但不应为了提高比值而做无益计算。

某些 LLM inference implementation 会用 rematerialization 等策略降低 peak memory：不长期保存部分 large intermediate，而在稍后按需重算特定 activation 或 transformation。它在“用 compute 换 storage”这一点上类似训练中的 *activation checkpointing*，但训练 checkpointing 通常在 backward 时重算 forward activation，inference 没有相同的 backward 生命周期，二者不能视为同一机制。

Recomputation 可能降低 peak memory 或目标 memory level 的 traffic，从而允许不同 batch/sequence configuration；也可能增加 latency、energy、register pressure 或 compute contention。是否改善 FLOP/byte 与端到端时间取决于保存值的大小、重算成本、reuse count、cache behavior 和原始 bottleneck，应以 memory peak、traffic、correctness 和 runtime 联合判断。

PyTorch 与 Arithmetic Intensity

PyTorch 可通过 compiler 与 optimized backend 应用本章部分策略，但不是自动覆盖所有 operation。如第 13 章和第 14 章所述，PyTorch compiler 可对受支持 graph 中的某些 elementwise chain 和 reduction 做 fusion。Graph-level optimization 可能减少 launch 与 intermediate traffic；是否保留 locality/reuse 仍受 graph break、aliasing、dynamic shape、backend 与 cost model 约束。

PyTorch 会为许多 operation dispatch 到 cuBLAS、cuDNN 或生成的 kernel。例如，`torch.matmul` 可能使用内部含 tiling、register/SMEM staging 等优化的 library implementation，但具体 algorithm 可能使用 cache、register、shared memory 或其他路径，不能保证每次都以同一方式 tiling。PyTorch 的

`scaled_dot_product_attention`（SDPA）可依据 device、shape、mask、dropout 和 `dtypes` 等条件选择 FlashAttention、memory-efficient、math 或 cuDNN backend。可用 `torch.nn.attention.sdpa_kernel(SDPBackend.FLASH_ATTENTION)` 等 context manager 限制候选 backend，但若约束不满足，可能报错或无法使用所选 backend；应通过 profiler/log 验证实际 dispatch。

PyTorch compiler 能 capture 和 lower 的 operation coverage 取决于版本与 backend；data-dependent control flow、graph break、unsupported/custom operator 或 mutation/aliasing 都可能阻止 fusion。此时可先调整 graph 表达、注册 decomposition/custom operator，或在 profiling 证明必要时使用 custom kernel；manual fusion/tiling 不是唯一选项，也不保证更快。

> 编写 PyTorch workload 时，可优先评估 fused operator、optimized library call 与 compiler-friendly graph，避免不必要的大量 tiny kernel launch。例如可使用 `torch.nn.functional` 中合适的 activation 或 `torch.matmul`，而不是在 Python 中手工拆成许多小 operation。但 high-level API 不保证总调用单一高效 kernel，compiler 也不保证能与 surrounding operation 融合；应检查 graph、generated code 和实测时间。

PyTorch 的 nested tensor（也称 ragged representation）可表示 variable-length batch，减少显式 padding。具体 storage/layout 依 nested-tensor implementation 而定，并非所有 nested tensor 都只是一个无额外 metadata 的 packed buffer。`NestedTensor` 也不是完整等同于普通 dense Tensor 的全部 operator interface；它可避免部分 zero-padding compute/traffic，但 metadata、offset、alignment 与 irregular access 仍有成本，不能说 fetch 的每个 byte 都必然用于 arithmetic。

对 sequence length 差异较大的 LLM workload，nested tensor 可能减少 padding，并让受支持的 attention 或其他 operator 只处理有效 token。然而 operator coverage、backend、layout conversion、load balance 和 shape distribution 会决定实际效果；batch matrix multiply 也不会普遍自动支持或只读取“必要数据”。它可能降低 memory traffic，也可能引入 irregularity，不能保证 Roofline point 靠近 compute-bound 或 sustained throughput 上升。

> 使用 nested tensor 时应验证 operator coverage、autograd/compile/backend compatibility 与 performance。用具有代表性的 sequence-length distribution、attention pattern 和 batch size 检查端到端 correctness、conversion overhead、memory traffic、kernel time 与 latency，而不只测试单一理想 shape。

总之，PyTorch 提供 compiler fusion、optimized library、specialized attention backend 与 nested representation 等机制，它们可能减少 traffic、提高 reuse 或降低 launch overhead。应根据 workload 验证实际 dispatch、正确性与端到端时间，而不是假定它们都提高 arithmetic intensity。下面讨论 reduced-precision Tensor Core；lower precision 会同时改变 byte、compute roof 与数值行为，也需单独分析。

Mixed Precision 与 Tensor Core 利用

现代 NVIDIA GPU 的 Tensor Core 支持 TF32、FP16、FP8、FP4、INT8 等特定 reduced-precision matrix operation，但支持的 input、accumulator、shape 与 throughput 依 architecture 和 instruction 而异。Blackwell TMEM 是 Tensor Core data path 使用的独立 on-chip memory，容量和组织应按具体 compute capability/PTX 文档确认，不能把“每个 SM 固定 256 KB”无条件外推。TMA 可异步执行受支持的 global ↔ shared-memory bulk tensor transfer；`tcgen05.mma` 等 instruction 则按明确 operand/accumulator contract 使用 SMEM、register 与 TMEM，并非笼统地在 SMEM 与 TMEM 间隐式移动所有数据。TMEM accumulator 可在适用 kernel 中降低部分 register pressure，但效果取决于 tile、lifetime 和 code generation。

Lower precision 会改变 element byte、operation count convention 和可用 compute roof；Tensor Core path 也可能提高 matrix throughput。但这些特性不会保证把 memory-bound kernel 变成 fully compute-bound，也不存在普遍固定的 2×、4× 或 8× arithmetic-intensity 增幅。应先验证数值正确性，再在 Nsight Compute 中核对实际 instruction path、各 memory level byte、Roofline definition、throughput、stall 与端到端时间。

Nsight Compute 的 *Speed of Light* analysis汇总 compute/memory utilization；warp stall metric 与 cache metric 则来自不同 section，不能把 “Memory Throttle” 或 cache miss 简单称为 Speed-of-Light 的 memory-bound stall reason。使用 lower-precision Tensor Core 后，memory-related stall 不保证显著下降，因为 byte、instruction mix、occupancy、dependency 和新 bottleneck 都可能变化。Roofline chart 也只有在选对 dtype roof、FLOP count 和 memory level 时才可比较。

从 FP32 改为 TF32、FP16、FP8 或 FP4 Tensor Core kernel 后，warp-stall 百分比分布可能变化，但“memory stall 下降、dependency/pipeline stall 相对上升”既不是 arithmetic intensity 提高的充分证据，也不是从 memory-bound 转为 compute-bound 的必要模式。Stall reason 是归一化 sampling/issue-state metric；应结合 achieved bandwidth、Tensor Core utilization、executed instruction、byte count、Roofline position 与 runtime 判断。

用 TMEM 与 TMA 为 Tensor Core 供给数据

TMEM 是 Blackwell Tensor Core data path 的重要组成部分，属于独立 address space 的片上 memory；其容量与组织依具体 architecture，原文统一称每 SM 256 KB SRAM 过于宽泛。使用 high-level library 时，programmer 往往不直接管理 TMEM，但低层 PTX 实现需要显式 allocation/deallocation、layout、load/store 和 synchronization instruction，不能说一切都由 hardware 自动处理。TMEM 概念结构见图 9-6。

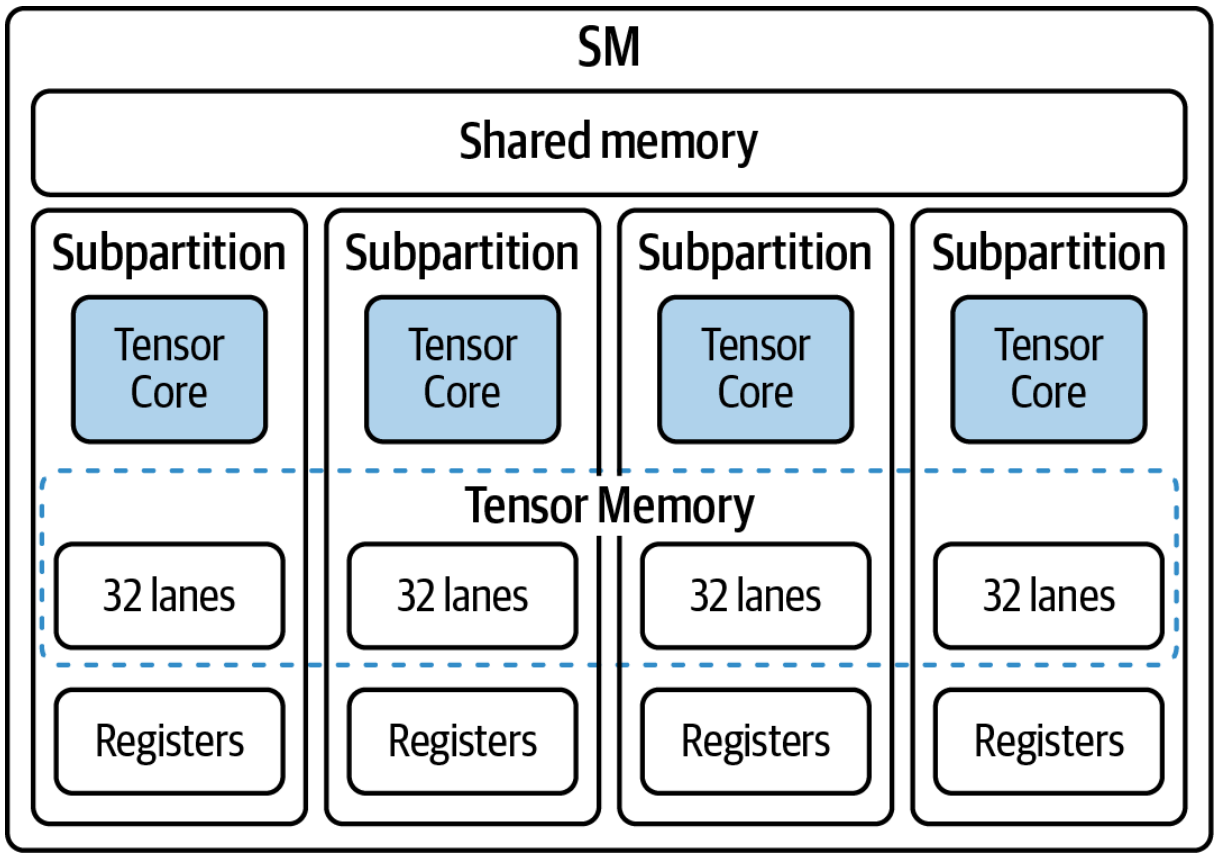


图 9-6. TMEM 可为 Blackwell Tensor Core accumulator 与相关数据路径提供存储，不应概括为完全取代 register

Blackwell 的 `tcgen05.mma` instruction 按具体 variant 从规定的 operand source 读取并把 accumulator 写入 TMEM。CUTLASS 等 library kernel 可生成所需 PTX，并管理 allocation、layout 与 synchronization。Transformer Engine 是支持低精度 format/scaling 的 software/hardware stack 概念，不能据此断言其所有 operation 都使用 TMEM。适用的 TMEM accumulator 可缩短部分 register live range，但 kernel 仍需 register 保存 address、control 和其他 value。

CUTLASS 等 high-level library 可封装大量 architecture-specific detail，应在适合时优先评估，但并非每个 CUTLASS kernel 都使用 `tcgen05.*`；选择取决于 target architecture、kernel schedule、dtype、shape 与 toolkit version。CUDA MMA intrinsic 或 CUTLASS GEMM 也不保证统一经过相同的 SMEM/TMEM movement path。TMA 执行受支持的 bulk tensor copy，而 tcgen05 family 中 MMA、copy、load/store 等 instruction 有各自显式 contract；不能概括成 Tensor Core instruction 自动隐式完成全部 shared-memory 与 TMEM transfer。

> Nsight Compute 提供 Roofline 与 Speed-of-Light 等 analysis，可帮助比较 lower-precision Tensor Core path 前后的 ceiling、utilization 和 bottleneck；它们不能单独“确认” regime 转换，还需核对 metric definition、实际 instruction、byte、正确性和 runtime。

Tensor Core operand 与 accumulator movement 由所选 MMA/PTX contract 和 library schedule 决定，低层实现并非都“在幕后、无需显式代码”。Global-to-shared

transfer 可在适用条件下使用 TMA 或 CUDA Pipeline API 中的 `cuda::memcpy_async`，并包含 `<cuda/pipeline>`；两者不是同义 API，实际 lowering 取决于 overload、alignment、shape 与 architecture。下面原书代码展示一个名义上的 two-stage `cuda::memcpy_async` pipeline，但不包含 Tensor Core compute，只保留 placeholder comment，因此不能证明 TMEM/Tensor Core overlap：

```
# two_stage_pipeline.cu

#include <cuda/pipeline>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

extern "C" __global__
void stage_ab_tiles(const float* __restrict__ globalA,
                    const float* __restrict__ globalB,
                    float* __restrict__ outC,
                    int tile_elems,
                    int num_tiles) {
     // Alignment / size guards for vectorized copies (runtime parameter)
    assert((tile_elems % (32 * 4)) == 0 &&
           "tile_elems must be multiple of 128 for float4 vectorization");
    // If you cannot guarantee 16B alignment or sizes, handle
    // the tail/ragged edges with a fallback 4B loop.
  extern __shared__ float smem[];
  auto block = cg::this_thread_block();

  // Shared buffers for double buffering of A and B
  float* A0 = smem + 0 * tile_elems;
  float* A1 = smem + 1 * tile_elems;
  float* B0 = smem + 2 * tile_elems;
  float* B1 = smem + 3 * tile_elems;

  constexpr auto scope = cuda::thread_scope_block;
  constexpr int stages = 2;
  __shared__ cuda::pipeline_shared_state<scope, stages> pstate;
  auto pipe = cuda::make_pipeline(block, &pstate);

  // Prime the pipeline with tile 0
  pipe.producer_acquire();
  cuda::memcpy_async(block, A0, globalA + 0 * tile_elems,
                     cuda::aligned_size_t<32>{tile_elems * sizeof(float)}, pipe);
  cuda::memcpy_async(block, B0, globalB + 0 * tile_elems,
                     cuda::aligned_size_t<32>{tile_elems * sizeof(float)}, pipe);
  pipe.producer_commit();

  for (int t = 1; t < num_tiles; ++t) {
    // Stage the next A and B tiles
    pipe.producer_acquire();
    cuda::memcpy_async(block, (t & 1) ? A1 : A0,
                       globalA + t * tile_elems,
                       cuda::aligned_size_t<32>{tile_elems*sizeof(float)}, pipe);
    cuda::memcpy_async(block, (t & 1) ? B1 : B0,
                       globalB + t * tile_elems,
                       cuda::aligned_size_t<32>{tile_elems*sizeof(float)}, pipe);
    pipe.producer_commit();

    // Consume the previously staged tiles
    pipe.consumer_wait();
    float* prevA = (t & 1) ? A0 : A1;
    float* prevB = (t & 1) ? B0 : B1;
    // Perform compute using prevA and prevB
```

```
    pipe.consumer_release();
  }

  // Consume the final staged tiles
  pipe.consumer_wait();
  int last = (num_tiles - 1) & 1;
  float* lastA = last ? A1 : A0;
  float* lastB = last ? B1 : B0;
  // Perform compute using lastA and lastB
  pipe.consumer_release();
}
```

启动该 kernel 时，dynamic shared-memory allocation 至少需容纳 `4 x tile_elems x sizeof(float)`，对应 `A0`、`A1`、`B0` 和 `B1`；还须满足 device limit、alignment 和 pipeline state resource。代码在进入 loop 前无条件 prime tile 0，所以要求 num_tiles 大于 0；它没有写入 outC，也没有实际消费 prevA/prevB 做 compute，不能称为完整 kernel。`cuda::memcpy_async` 是否与 compute 并行取决于合法 async path、producer/consumer sequencing、copy size、occupancy 与真实 compute duration。TMEM、SMEM 和 lower-precision tile 也不会自动保证“完全片上”或减少 stall。只有 pipeline 调优且 profiling 证明确有 overlap 时，`cuda::memcpy_async` 才可能隐藏部分 global-to-shared latency。

TF32 与 Automatic Mixed Precision（PyTorch）

支持 TF32 的 NVIDIA Tensor Core可用 FP32 的 8-bit exponent range 与 10 个显式 fraction bit（计入 implicit leading bit 时有效 precision 为 11 bit）执行特定 matrix operation。TF32 input 通常由 FP32 round 到较低 significand precision，它不是介于 FP32 与 FP16 之间的通用 storage dtype。相对 FP32 CUDA-core path，其受支持 matrix throughput 可能更高，但倍率依 GPU 与 operation。PyTorch 可设置如下 float32 matmul precision policy：

```
import torch
torch.set_float32_matmul_precision('high') # {'highest'|'high'|'medium'}
```

设置后，`torch.matmul`、`torch.nn.Linear` 等符合条件的 float32 matrix multiply 可选择较低 internal precision 的高性能 algorithm，output dtype 仍可为 float32。high policy 不保证 dispatch 到 TF32，也可能依据 backend/hardware 使用其他允许的 internal algorithm；small shape、unsupported layout 或不同 device 也可能采用其他 path。应结合版本、backend policy、数值误差和 profiler 验证。

除 TF32 外，PyTorch 的 automatic mixed precision（AMP）中的 autocast 会依据 operator eligibility/policy，在指定 autocast dtype 与 float32 等类型间选择执行 dtype；它不是为每个 operation 搜索“最优”FP16/BF16，也不保证所有 result 都以 FP32 accumulate。BF16 与 FP32 具有相同 exponent bit 数，通常比 FP16 更不易 overflow，但 significand precision 更低，仍可能出现数值问题。CUDA `autocast` 的默认 dtype 通常为 float16；在支持的 GPU/backend 上可传入 `dtype=torch.bfloat16` 使用 BF16 policy。例如可用如下 context manager：

```
with torch.amp.autocast("cuda", dtype=torch.bfloat16):
    output = model(input)
```

Autocast 是 PyTorch AMP/runtime dispatch 机制，并非由 TorchInductor 独自实施；TorchInductor（见第 13 章和第 14 章）可能在 compiled graph 中融合 conversion 或生成 mixed-dtype kernel，但不保证 large GEMM 全部使用 FP16/TF32 Tensor Core、所有 accumulation 均为 FP32，或 layer normalization/softmax 一律完整运行于 FP32。`GradScaler` 通过动态 scale loss/gradient 降低 FP16 gradient underflow 风险，但不能防止全部 underflow/overflow，也不修复 forward precision error。BF16 因 exponent range 较大，训练时通常不需要 `GradScaler`，仍应按 model 与 convergence 验证。

PyTorch AMP、operator implementation 与 compiler 可协同选择/传播 `dtype`，但 FP8 通常还需要专门 API、scaling recipe 或 library support，不能从普通 autocast 推断自动采用 FP8；FP16/FP8 compute 与 FP32 accumulator 也只是部分 operation 的 contract。Developer 仍需选择 autocast dtype、使用适当 scaler，并验证 unsupported/custom operation 和数值质量。图 9-7给出 mixed-precision MMA 概念示例。

Automatic mixed precision 可能以少量 code change 提升 Tensor Core eligibility、减少 operand byte 或 conversion overhead，但不保证最大化 arithmetic intensity。Fused Tensor Core kernel 是否减少 HBM round trip，以及是否把 operand stage 到 shared memory、把accumulator放入 TMEM，取决于 architecture、schedule、layout 与 generated instruction；TF32/FP16 等较低 internal precision 也不会自动改变所有 storage traffic。

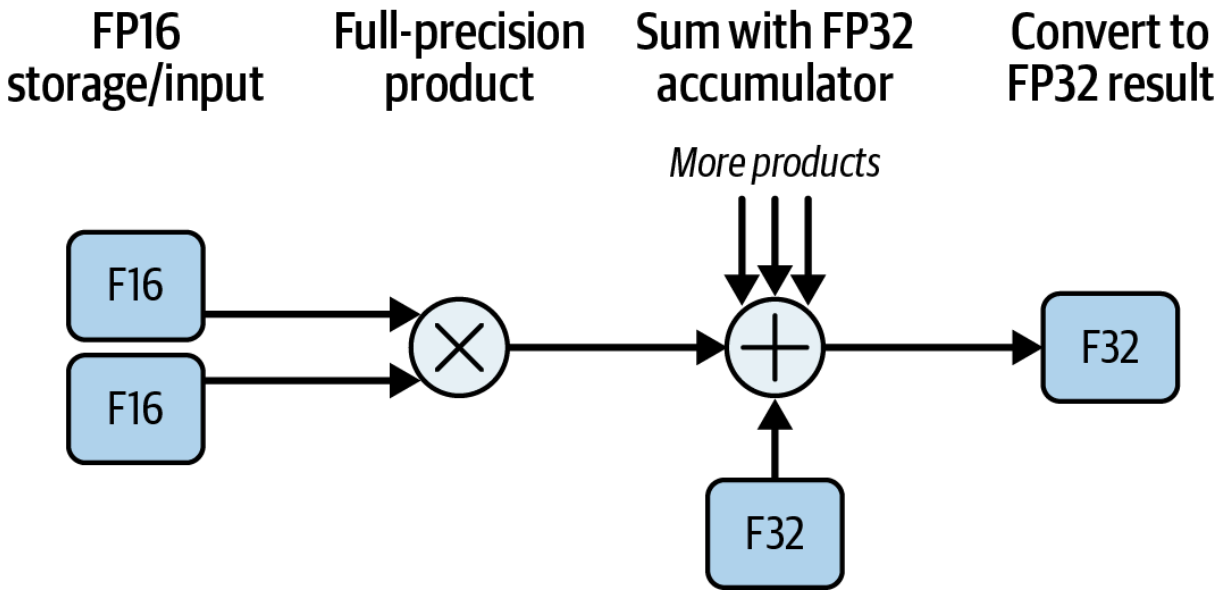


图 9-7. Mixed-precision matrix multiply-accumulate（MMA）概念图

使用 structured sparsity、FP8 或 FP4 时，需要满足 operator 的 shape、alignment、tile、scale/metadata 与 backend requirement，并提供足够 parallel work。Small batch 可能使 conversion、launch、sparse metadata 或 irregular-access overhead 更显著，但 batch 并非唯一因素；M/N/K shape、sequence length、grouping、

workspace 和 latency target 同样重要，TMEM/Tensor Core utilization 不能只由 batch size 推断。

例如，FP8 或 2:4 sparsity 在 batch size 1 的某些 GEMV/小 GEMM 上可能收益有限，也可能因 weight compression、specialized kernel 或其他维度足够大而受益。Batch size 128 或 256 同样不保证 fully utilize TMEM pipeline 或达到 near-peak throughput；应扫描实际 shape/configuration，并核对 dispatch、occupancy、Tensor Core utilization、memory traffic、accuracy 和 latency。

BF16/FP16、FP8 与 FP4 Reduced Precision

BF16 和 FP16已被多代 GPU 支持，但二者并非同一种 “half-precision” format：它们都是 16 bit，却采用不同 exponent/fraction 分配。现代 Tensor Core 对支持的 BF16/FP16 MMA 可提供高峰值 throughput，但 achieved performance 超过 peak 的 90% 或约为 FP32 peak 的 4×都不是通用规律；必须比较同一 device、dense/sparse 口径、operation、accumulator、clock 与 shape。较高 throughput 来自专用 matrix pipeline 并行执行规定 tile 的 MMA，而不是 source level 每 cycle 简单发射任意数量 FMA。

FP16 只有 5-bit exponent，较小 gradient 更容易 underflow，较大 value 也更容易 overflow。Static 或 dynamic loss scaling 可把 loss/gradient 暂时移到更可表示的范围，降低部分 underflow 风险；它不能保证 backpropagation 数值稳定，也不能补偿 forward quantization、overflow 或所有 reduction error。

BF16 与 FP32 都有 8-bit exponent，因此 normal exponent range 接近，通常比 FP16 少需要 loss scaling；但 BF16 只有 7 个显式 fraction bit，仍会发生 underflow、overflow、rounding 与 accumulation error。“天然避免 underflow”不成立，training accuracy 也不保证高于 FP16 或等同 FP32。

> 在原生支持 BF16 的现代 GPU 上，它常是 training 的候选，因为 exponent range 较大且通常无需 FP16 式 loss scaling。但最佳 dtype 取决于 model、optimizer、kernel support、throughput、memory 和 quality target；必须用 FP32/reference baseline 验证 convergence 与最终 metric。

FP8 可进一步减少 value payload，并在支持的 Tensor Core path 上提高 peak throughput。把 16-bit weight 压成 8 bit 会使未压缩 value payload 减半，但 scale/amax/metadata、activation、output、alignment 与其他 traffic 仍存在，因此总 HBM traffic 不保证减半。FP8 `matmul` 的 accumulator/output format 由具体 instruction/library recipe 决定，不能笼统写作 “FP32 或 TF32 accumulation”；TF32 通常描述 input compute precision而非 accumulator dtype。相对 BF16/FP16 的 2–3× TFLOP/s 也不是普遍 achieved ratio，应按 device、format、shape 和 scaling overhead 实测，并验证 task quality。

Transformer Engine 等 stack 支持 FP8，并可在相应 Blackwell/software configuration 上支持 NVIDIA NVFP4 microscaling format。NVFP4 使用 microblock scale 与更高层 scale 等信息改善有限 4-bit value 的 dynamic-range utilization，但不能保证 model 保持 accuracy。原书给出的 B200 10 PFLOP/s、FP32

约 80 TFLOP/s，以及 B300 15 PFLOP/s，只有在明确 SKU、boost/base clock、dense 或 structured-sparse、operation 与 marketing FLOP-count convention 后才能比较；这些 peak 数字的比值不是 application speedup，也不是“每个 weight 的 throughput”。应以目标产品的当前 datasheet 为准。

若 hardware、library、shape 与 quantization recipe 均支持，且 calibration/fine-tuning 后 model quality 达标，NVFP4 kernel 可能比 FP32 path 提供更高 matrix throughput和更低 value traffic；仍需逐 model、operator 和端到端 workload 验证 accuracy、latency 与 conversion/scaling overhead。

一个 256 × 256 的 packed 4-bit value tile 本身约为 32 KiB，但实际 kernel 还需另一 operand、scale/metadata、accumulator、pipeline stage 与 alignment space；tile 是否适配 TMEM/SMEM 由 instruction layout 和 architecture resource limit 决定，不能仅以“256 KB per SM”推导 reuse 或 performance。Low-precision input 的 unpack、scale、convert 与 accumulation 也不一定全部自动完成或采用 BF16/FP16/FP32 任一 accumulator；必须遵守所选 MMA API 和 library recipe。

降低 storage precision 可减少每 element payload，并可能提高支持 instruction 的 peak operation/byte，但 algorithmic arithmetic intensity、executed FLOP 定义、scale/metadata traffic 与 output byte 会同时变化，不能说每降一级 precision 就必然翻倍或四倍。TMEM/TMA 也只可能在合法 pipeline 中重叠部分 movement 与 compute，不保证把 memory-bound kernel 变成 entirely compute-bound或完全利用 multi-PFLOP Tensor Core。正确目标仍是满足数值要求并缩短目标 workload 时间。

用于 Inference 的 INT8 Reduced Precision 与 DP4A Instruction

部分 LLM inference workload可在 calibration、quantization-aware training 或合适 scaling 后使用 INT8，但可接受性取决于 model、layer、activation outlier、quantization granularity 与 quality target，不能假定 LLM 通常都能无损容忍。GPU 可通过 DP4A packed dot-product instruction 或支持的 integer Tensor Core MMA 加速相应 operation。DP4A 从两个 packed 32-bit operand 取四对 signed/unsigned 8-bit value，执行四次 integer multiply 并累加到 32-bit accumulator；把“一条 DP4A”与“一条 FP32 FMA”比较不能直接得到 throughput 倍率，因为 issue rate、pipeline、lane semantics 和 instruction count 不同。

相对于 dense FP32 weight，raw INT8 weight payload 从每 element 4 byte 降为 1 byte，即 payload 减少 75%；但 scale、zero point、group metadata、activation、output、alignment 和其他 traffic 仍存在，total memory traffic 不一定下降 75%。INT8 inference 可能受益于更高 integer Tensor Core peak 和较低 weight bandwidth，但也可能受 quantize/dequantize、unsupported operator、小 shape 或 memory/launch overhead 限制。“GPU 每秒从 memory 处理约 4×更多数据”混淆了相同 byte/s 下的 element count 与 achieved bandwidth，不能作为 speedup 结论。TMEM/TMA 只适用于特定 architecture 和 data path，也不能保证 data movement 与 compute 完美 overlap；应验证实际 kernel、byte、Tensor Core utilization、accuracy 和端到端 latency。

深入理解 Transformer Engine 与 TMEM

NVIDIA Transformer Engine把支持低精度 format 的 Tensor Core capability 与 scaling、casting、recipe 和 high-level software library 结合起来；它不是所有现代 NVIDIA GPU 上同一种独立 hardware block。cuBLASLt、cuDNN、CUTLASS 或 Triton kernel 可能使用 `cp.async`、TMA 或其他 load/copy path，取决于 architecture 与 schedule；Tensor Core instruction 也不会统一隐式完成全部 SMEM ↔ TMEM movement。

TMEM 是特定 Blackwell compute capability 的独立 on-chip memory/address space，不应再次概括为所有 Blackwell 每 SM 固定 256 KB、完全替代 register 的 result buffer。High-level application 通常不直接操作 TMEM，但 low-level PTX/library kernel 必须生成 allocation/deallocation、layout、MMA、copy/load/store 与 synchronization instruction；hardware 不会仅因调用任意 MMA 就自动完成全部 allocation 和 transfer。

使用 MMA instruction 时，参与 thread 的粒度可能是 warp、warpgroup、CTA pair 或其他 architecture-defined collective，不能统一称“每个 warp 直接驱动 Tensor Core”。WMMA、MMA 和 high-level API 对 fragment、operand layout、register mapping 与 synchronization 的抽象程度也不同；programmer 或 library 仍需满足对应 contract。

> PyTorch ecosystem 的 INT8 quantization可通过 TorchAO、framework integration 与 vendor backend 等实现，具体 API/support 随版本变化。Quantized module 可能 dispatch 到 dedicated INT8 kernel，也可能 fallback、dequantize 或采用其他 representation。cuBLASLt/CUTLASS 只提供 Tensor Core eligible implementation 的可能性，不能确保实际 Tensor Core utilization；shape、dtype/layout、alignment 和 selected algorithm 都需验证。

启动 Tensor-Core-based kernel 或 CUTLASS 等 GEMM 并不保证 operand 一律经过 shared memory 与 TMEM，也不保证 Tensor Core 始终有 ready tile。Library implementation 会按 architecture、schedule 和 resource constraint 生成 movement/pipeline；application 层可能不直接分配 TMEM，但 library/kernel code 仍需管理它。

Transformer Engine workflow 不是由 firmware 在任意 MMA call 后自动安排。High-level framework/library 先依据 recipe 选择 format、scale 与 kernel，selected kernel 再按自身 schedule 使用 TMA 或 `cuda::memcpy_async` 等 mechanism。`tcgen05.mma` 具有明确 operand/accumulator location contract，而不是笼统隐式搬运 SMEM/TMEM。Weight 与 activation 能否使用 FP8/FP4 取决于 hardware、operator、scaling scheme 与 quality；FP16/FP32 也不是无代价的简单 fallback。

Tensor Core MMA 支持的 input/accumulator combination 依 instruction，例如某些 FP8 或 FP16 operation 使用更高 precision accumulator。Blackwell 某些 tcgen05 MMA 的 accumulator state 位于 TMEM，但不能外推为全部 Tensor Core operation 或 BF16/FP16/FP32 partial sum 都在 TMEM。State 通过 `tcgen05` family 的显式

instruction 与 synchronization protocol 访问，并非 hardware 对 low-level code 完全透明。

Custom tile loop 可尝试以 `cuda::memcpy_async` 和 CUDA Pipeline API overlap movement 与 compute。下面原书清单仅为不完整 sketch：它没有实际 Tensor Core compute，并包含 compile/bounds 问题，因此不能直接使用：

```
#include <cuda/pipeline>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

extern "C" __global__
void double_buffer_a(const float* __restrict__ globalA,
                     int tile_elems,
                     int numTiles) {
  __shared__ float tileA0[TILE][TILE];
  __shared__ float tileA1[TILE][TILE];

  auto block = cg::this_thread_block();

  constexpr auto scope = cuda::thread_scope_block;
  constexpr int stages = 2;
  __shared__ cuda::pipeline_shared_state<scope, stages> pstate;
  auto pipe = cuda::make_pipeline(block, &pstate);

  // Prime pipeline with tile 0
  pipe.producer_acquire();
  cuda::memcpy_async(block,
                     &tileA0[0][0],
                     globalA + 0 * tile_elems,
                     cuda::aligned_size_t<32>{tile_elems * sizeof(float)}
                     pipe);
  pipe.producer_commit();

  for (int t = 1; t < numTiles; ++t) {
    // Stage next tile into the alternate buffer
    pipe.producer_acquire();
    float* nxtA = (t & 1) ? &tileA1[0][0] : &tileA0[0][0];
    cuda::memcpy_async(block,
                       nxtA,
                       globalA + t * tile_elems,
                       cuda::aligned_size_t<32>{tile_elems * sizeof(float)}
                       pipe);
    pipe.producer_commit();

    // Consume the previously staged tile
    pipe.consumer_wait();
    float* curA = ((t - 1) & 1) ? &tileA1[0][0] : &tileA0[0][0];
    pipe.consumer_release();
  }

  // Consume the final staged tile
  pipe.consumer_wait();
  float* lastA = ((numTiles - 1) & 1) ? &tileA1[0][0] : &tileA0[0][0];
  // Use lastA with your compute
  pipe.consumer_release();
}
```

该清单只把 A tile copy 到 double-buffered shared memory；没有 TMEM instruction、Tensor Core MMA 或 output。`cuda::memcpy_async` 只有在 supported

async path、alignment/size、正确 pipeline sequencing 且存在足够 compute 时，才可能让 next-tile transfer 与 current-tile compute 重叠。代码还缺少 TILE 定义、未验证 tile_elems 是否适配静态 buffer、无条件读取 tile 0，并在 aligned-size argument 后缺少逗号；程序清单按原书保留。

Overlap 能覆盖部分 latency，但不会自行改变 algorithmic FLOP/byte；只有减少 transaction、改善 reuse 或改变 precision/byte 时，measured intensity 才可能变化。Transformer Engine、TMEM 与 TMA 也不构成自动协作的单一 engine，更不能保证接近 speed-of-light efficiency。应以正确性、实际 overlap、throughput 和 runtime 为证据。

> Load/store 的 completion、visibility 和 calling-thread semantics 依具体 instruction/API；compute-copy overlap 可由 CUDA Pipeline、barrier 或 TMA async mechanism 等实现。`cuda::memcpy_async` 不会普遍映射到 TMA，可能使用 `cp.async` 或其他 path；TMA 适合支持 architecture 上符合 tensor-map、shape、alignment 和 bulk-transfer 条件的 workload，也不是所有 bulk copy 的绝对首选。无论选择何种 path，都必须在消费数据前使用匹配的 wait/barrier 并遵守 buffer lifetime。

使用 CUTLASS 优化 Arithmetic Intensity 与 Tensor Core Performance

NVIDIA CUTLASS 是实现高性能 GEMM 与相关 kernel 的常用 template/library framework。它可封装 architecture-specific tile、pipeline 和 epilogue，但通常仍需选择 operation、dtype/layout、architecture、schedule、alignment 与 problem shape；不能概括为“一个 template call 自动应用所有优化”。

CUTLASS kernel 可能采用 shared-memory tiling、asynchronous copy、multistage/double buffering 与 TMEM accumulator path，但具体组合取决于 CUTLASS 版本与 kernel schedule。TMEM 不是 double buffering 的通用 256 KB SMEM extension；near-peak Tensor Core throughput 也不由使用 CUTLASS 自动保证，仍需 kernel selection、autotuning 与 profiling。

> 部分 CUTLASS schedule 也实现 warp specialization，把 producer/consumer 或不同 pipeline role 分配给不同 warp/warpgroup。它不是所有 CUTLASS kernel 的固定特性，下一章将进一步讨论。

例如，设要计算 half-precision input/output 的 GEMM `C = A * B`，并使用合适的 accumulator type。可以从 CUTLASS abstraction 入手，而不必直接手写 MMA loop；但下面原书清单只是版本相关且含语法/API 问题的示意，不能直接编译：

```
#include <cutlass/numeric_types.h>
#include <cutlass/gemm/device/gemm.h>

using Gemm = cutlass::gemm::device::Gemm<
  cutlass::half_t,  // A (FP16)
```

```
  cutlass::layout::RowMajor,
  cutlass::half_t,  // B (FP16)
  cutlass::layout::ColumnMajor,
  cutlass::half_t,  // C / output (FP16)
  cutlass::layout::RowMajor,
  float, // accumulator (FP32 accumulate)
  cutlass::arch::OpClassTensorOp,
  cutlass::arch::Sm100 // e.g., Blackwell B200
>;

// ... (allocate device pointers A_d, B_d, C_d,
// set up dimensions M,N,K, and strides lda, ldb, ldc) ...

Gemm gemm_op;
cutlass::Status status = gemm_op(
    { M, N, K },              // GEMM shape
    float(1.0f)               // alpha
    A_d, lda,                 // A pointer + leading dimension
    B_d, ldb,                 // B pointer + leading dimension
    float(0.0f),              // beta
    C_d, ldc                  // C pointer + leading dimension
);
```

原清单按展示文本不能直接编译：float(1.0f) 后缺逗号，argument grouping 也不符合常见 CUTLASS device GEMM invocation；此外 legacy-style device::Gemm 与 Sm100 的合法 template signature/support 依 CUTLASS release。若改为该版本支持的 API，kernel/configuration 会依据 template 与 dispatch policy 选择 tile，以权衡 register、SMEM、occupancy 和 Tensor Core utilization。Tile shape 可能类似 128 × 128 或 256 × 128，但不是 runtime 自动经验选择的统一结果；Blackwell schedule 可能使用 TMEM accumulator，不能因此断言整个 tile 全程驻留于固定 256 KB TMEM。

原文用 256 × 512 × 2 byte = 256 KiB 和 256 × 256 × 4 byte = 256 KiB 估计 “TMEM tile budget”，算术本身成立（后者应写 KiB），但概念不成立：operand tile、accumulator tile、TMEM column/row allocation、scale metadata 与 pipeline stage 不能仅用一个 dense rectangle 的 byte 数替代。Larger tile 可能提高 reuse，也可能增加 register/SMEM/TMEM resource 并降低 residency；small tile 可能增加 parallelism，却不必然降低 arithmetic intensity。应按 actual schedule/resource 与 runtime 评估。

所选 CUTLASS schedule 可能使用 `cp.async` 或 TMA 把 tile 从 global memory stage 到 shared memory，也可能采用其他 path。`cp.async` 支持 global-to-shared async copy，并可根据 modifier 控制 cache behavior；它避免传统的显式 per-thread register intermediate，但“是否经过 L1”须按 .ca/.cg 等语义和 architecture 解释。图 9-8展示 `cp.async` 的概念路径。

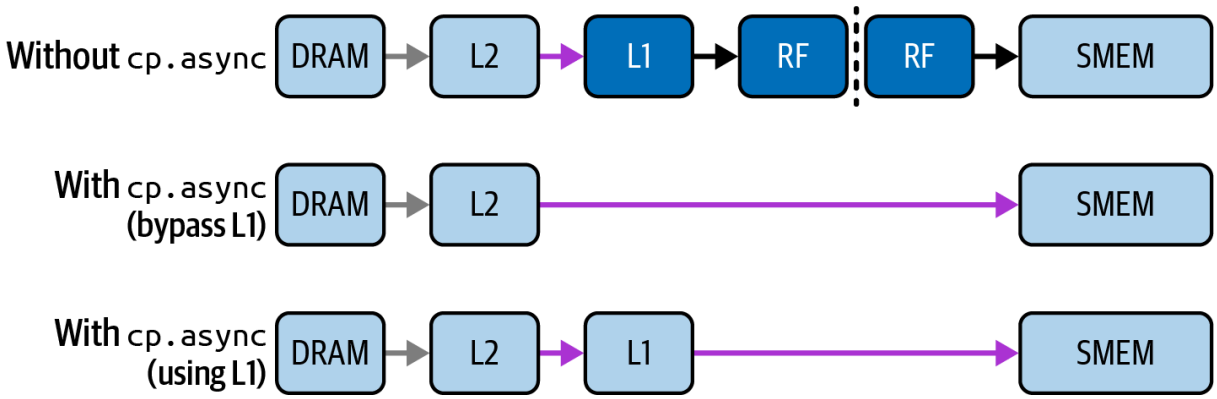


图 9-8. 使用 asynchronous copy instruction（`cp.async`）从 global memory 加载到 shared memory，避免显式 register intermediate；L1 behavior 依 modifier 与 architecture

部分 CUTLASS schedule 使用 `cp.async` 或 TMA（对应 PTX 中如 `cp.async.bulk.tensor` 的 path）把 global-memory tile stage 到 SMEM。特定 Blackwell `tcgen05.mma` variant 可从规定的 SMEM/TMEM operand location 读取并累加到 TMEM。SMEM double buffer 与 TMEM accumulator 是不同 resource；只有 copy/compute stage、barrier 和 tile lifetime 正确安排时，TMA 才可能在 current tile 计算期间预取 next tile。

CUTLASS 可使用 CUDA Pipeline-compatible primitive 与 warp-specialized stage（下一章讨论）提高 overlap，但不保证所有 Tensor Core pipeline 始终繁忙。Accumulator precision 由 selected MMA/kernel contract 决定；FP16/FP8 input 不必然都使用 FP32 accumulator，也不能仅凭 accumulator precision 保证 numerical fidelity。Epilogue 如何从 TMEM/SMEM 写出、是否 coalesced，同样取决于 schedule、layout 和 output shape。

> 支持相应 architecture 与 schedule 时，部分 CUTLASS kernel 可使用 thread block cluster 跨多个 SM 协作，以扩大 reuse domain；是否有益取决于 shape、cluster residency、DSMEM traffic 和 synchronization。下一章将进一步介绍。

CUTLASS 封装这些复杂性后，可提供高性能 GEMM 候选，但并非对所有 shape、layout、fusion 和 GPU 都是 drop-in 或与手调 kernel 相差几个百分点。表 9-1只是原书的概念性示例，不是可复现 benchmark。

表 9-1. 手调 MMA 与 CUTLASS kernel 的示意性 performance/resource 对比

指标	手调 MMA kernel	CUTLASS GEMM
Tensor Core utilization（示意）	98%	98%
每 thread register（示意）	~52	~60（略高，示意）
每 thread block（CTA）的 shared memory（示意）	~2 KB	~4 KB
Development effort	高	低（简单 template configuration，示意）

> 注：所有 metric table 中的数值仅用于说明概念，不构成 benchmark evidence。不同 GPU architecture 上的实际结果应查看可复现实验配置与本书 GitHub repository 中的当前资料。

表中假设两者使用 FP16 input 与 FP32 accumulation，并以提高 Tensor Core utilization 为目标。由于数值明确是 illustrative，不能据此声称 CUTLASS 与手调 MMA 相差约 2%、register/SMEM 增量在 hardware limit 内或 occupancy 不受影响。必须给出 device、problem shape、kernel config、metric definition、compiler/toolkit 和 measured runtime 才能得出这些结论。

> CUTLASS 的 generic abstraction 有时增加 resource usage，也可能通过 specialized schedule 与 autotuning优于朴素手写实现；差异不能统一归因于“为 flexibility 做 generalization”。手调是否值得取决于 performance gap、maintenance、portability 和 workload lifetime，CUTLASS 与 hand-tuned option 也不保证 virtually identical。

部分 CUTLASS use case 可用较少 template/configuration code 代替大量 low-level implementation，但 integration、layout、build、dispatch 和 tuning 仍可能复杂。CUTLASS 对 FP4、FP8、FP16、TF32 等 support 依 release、architecture 和 operation；部分 collective/epilogue 可融合 bias-add、activation 等 postprocessing，并非任意组合都受支持。

> 特定 Blackwell CUTLASS schedule 可能使用 thread block pair、multi-SM tiling、TMA multicast 与 DSMEM，但 template 不会对所有 kernel 透明启用这些机制，也不保证最大化 reuse。应检查 instantiated schedule/generated code 和 profiler；下一章会更详细讨论。

Custom MMA kernel 可能需要选择 tile、实现 async-copy/double-buffer、warp specialization 与 cluster coordination。CUTLASS 能封装其中许多模式，但用户仍需选择或搜索 kernel configuration、处理 unsupported layout/fusion、设置 alignment/workspace 并验证 dispatch；不能说所有工作都自动完成。

cuBLAS 与 CUTLASS 都由 NVIDIA 提供高性能 linear-algebra implementation，但不能笼统说 cuBLAS 整体“built on CUTLASS”；其内部实现是 proprietary 且可采用多种 kernel technology。PyTorch 会调用 cuBLAS、cuDNN、Triton/CUTLASS-derived 或其他 backend。某次 TorchInductor attention dispatch 不能外推为固定 CUTLASS fused-attention kernel、更不能无 benchmark evidence 声称使用完全相同 TMEM pipeline、达到 98% Tensor Core utilization 和 near-zero memory stall。

> 当 PyTorch 或其他 high-level library 的某个 backend/operator采用 CUTLASS-derived kernel 时，用户可能无需编写 CUDA C++ 即可受益；但 backend selection、coverage 和 performance 随版本与 shape 变化，仍需验证。

如果需要 CUTLASS 当前不支持的 specialized layout、communication、synchronization 或 fusion pattern，manual MMA/custom DSL kernel 仍可能必要；也可先评估新增 CUTLASS collective/epilogue 或 library extension。

Custom implementation 需按 selected MMA contract 同时设计 operand tile、accumulator layout 和 SMEM/TMEM/register resource，而不是只选“能放进 TMEM”的 128 × 128 FP16 tile。可使用 `<cuda/pipeline>` 与 `cp.async`，也可按 architecture/shape 选择 TMA、barrier 或其他 copy mechanism。

随后可能实现 warp-/warpgroup-specialized MMA loop 与 SMEM double buffer；TMEM 是否、如何 double-buffer 取决于 tcgen05 allocation/layout 和 schedule，不能照搬 SMEM 概念。Softmax 或 nonlinear epilogue 能否融合到同一 loop 还受 dependency、numerical stability、resource 和 synchronization 约束，需在可行时验证。

对 standard GEMM，可优先评估 cuBLAS/cuBLASLt 或合适 CUTLASS kernel；对 fused attention，通常还应评估 cuDNN、framework SDPA、FlashAttention、Triton 等成熟 backend。不存在“几乎所有”场景都由 CUTLASS 单一路径最佳的保证。

CUTLASS 的 template design、GPU-specific schedule 与 TMEM/TMA support 可在受支持 shape 上实现高 Tensor Core utilization，但 96%–98% 不是普遍目标或保证；metric 定义、problem size、clock、precision 和 bottleneck 都会影响数值，最重要证据仍是正确性与 runtime。

依赖经过验证的 CUTLASS/library kernel 可减少 low-level engineering，使团队更专注于 model、precision 与端到端 performance；但 library 不保证 near-peak arithmetic intensity，也不能替代版本兼容、kernel selection、numerical validation 和 profiling。

NVIDIA 会更新 CUTLASS、cuBLAS 等 library以支持新的 FP8/FP4、cluster、TMEM 等 feature。采用 library 可减少跨代重写，但新 architecture 仍可能需要升级 toolkit/CUTLASS、修改 template/API、重新编译并 retune；切换 GPU 时应检查 compatibility matrix、release note 和 regression benchmark，而不只选择最新版本。

用于 Microoptimization 的 Inline PTX 与 SASS Tuning

需要进行 low-level microoptimization 时，CUDA C++ 可嵌入 inline PTX。SASS 则是特定 target architecture 的 native instruction representation，通常由 assembler/compiler 生成；CUDA C++ 并没有同等稳定、受支持的通用 inline-SASS interface。二者都不保证比 compiler-generated code 更快。

这是高级且高维护成本的优化方式。Compiler 已能完成许多 scheduling、instruction selection 与 register allocation；只有在反汇编和 profiler 证明特定 hot path 存在可修正 gap 时，手工 PTX 或专用 instruction 才可能带来收益，幅度没有固定范围，并可能随 toolkit/GPU 改变。

PTX 有时先于 high-level intrinsic 暴露 architecture feature，也可指定 cache operator、memory-order/scope、special register 或精确 instruction form。SASS 观察有助于确认最终 code generation，但直接依赖 machine encoding 更脆弱。Cache hint 也只是 hint/semantic choice；CPU-GPU coherent/unified-memory coordination 必须遵守 documented memory model，不能靠随意调整 cache strategy 保证正确性。

PTX（常读作 “pee-tex”）是 NVIDIA 的 virtual ISA/programming model，而不是直接执行的通用 GPU machine code。CUDA compiler 可生成 PTX，driver/toolchain 再针对目标 GPU 翻译为 native SASS。PTX version、target feature 和 driver compatibility 仍有限制；SASS 则与具体 architecture 更紧密。

适用场景可能包括：high-level CUDA 尚无对应 intrinsic；某个 access 需要 documented cache operator；memory fence/barrier 必须采用特定 scope/order；或 profiler/SASS 显示 compiler schedule 在目标 kernel 上不理想。不过 memory ordering 与 barrier 关系 correctness，不能仅为性能随意插入或重排；inline PTX 还会约束 compiler optimization。

Inline PTX 也可读取 SM ID、lane ID 等 special register。CUDA C++ 使用 `asm()` statement 嵌入 PTX，并通过 operand constraint、output、clobber/volatile 与 surrounding code 交互。Compiler 会把 PTX lowering 到 target SASS，但 inline block 不保证原样保留 scheduling，也不自动告知 compiler 隐含 memory side effect。

下面清单试图用 inline PTX 把 global-memory address prefetch 到 L2，并把 instruction 概括为 `cp.async.bulk.prefetch.global`。实际展示字符串是带 .L2 qualifier 的另一拼写；其 availability、operand grammar 和 target requirement 应按所用 PTX ISA version 验证。该示例还有 bounds/alignment 与说明错误，不能直接使用：

```
__global__ void PrefetchExample(const float *in, float *out) {
    // ... assume idx is our thread's data index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // Manually prefetch the next cache line (128B) of in[] into L2:
    // Prefetch 128B from global to L2.
    // Address must be 16B-aligned
    // and size is a 16B multiple.
    asm volatile("cp.async.bulk.prefetch.L2.global [%0], %1;"
                 :: "l"(in + idx + 32), "n"(128));
    float x = in[idx];
    // (do some work here before using in[idx+32] to give time for prefetch)
    out[idx] = x;
}
```

清单中的 inline PTX `cp.async.bulk.prefetch.L2.global [%0]` 使用 address operand `in + idx + 32`。由于 in 是 float pointer，C++ pointer arithmetic 表示向前 32 个 float，即 128 byte，不是原文所写“32 bytes, i.e. 32 floats”。而且 idx 可为任意 thread index，base alignment 并不保证 resulting address 满足 instruction requirement；代码也没有 length/bounds guard。`volatile` 告诉 compiler 不应删除或随意移动该 asm statement，但不等于 memory clobber、completion fence 或数据一定进入 cache。

Inline PTX 会参与生成 target machine code，可提供细粒度 instruction selection。原文称把 `.L1` 换入即可 prefetch L1，但 qualifier 是否存在、位置和语义取决于具体 instruction grammar，不能从 L2 form 类推。Prefetch distance 在这里是 32 个 float；实际最佳值依 latency、iteration work、cache capacity 和 access pattern。

不能无依据断言它就是 `__prefetch_async` “可能编译成”的形式；built-in/intrinsic 的 lowering 随 compiler、type、scope 与 architecture 变化。Inline PTX 也可为 normal load 指定 cache operator，例如 `asm("ld.global.cg.f32 %0, [%1];" : "=f"(val) : "l"(ptr))` 使用 `.cg`，但实际 inline asm 还应正确表达 address space、alignment、memory dependency 与 operand constraint。

在支持的 architecture/semantic 下，.cg global load 通常在 L2 cache，并不在 L1 分配；`.ca` 则表示在 all coherent levels cache。它们不是性能保证，default load policy 也依 target/codegen。只有 profiler 证明 L1 allocation 对特定 streaming access 有害时，override 才可能有益。

> 在支持的 PTX ISA/architecture 上，可评估 `cp.async.bulk.prefetch.tensor.L2` 等 documented tensor-map prefetch form；它不是任意 pointer prefetch 的通用替代品，并有 tensor map、coordinate、shape 与 target requirement。应优先使用 documented API/instruction，而不是 undocumented built-in，并验证 generated SASS 与实际收益。

Inline PTX 的另一候选用途是影响 instruction selection/scheduling。Compiler 会依据 dependency、latency model、register pressure 和 target architecture 调度；source/PTX order 不等于最终 issue order。只有在检查 generated SASS 后发现具体问题时，才应尝试约束 sequence。

例如有两个 independent load 及其 consumer，若 final schedule 是 load1-use1-load2-use2，而 load-to-use latency 未被覆盖，把两个 load 提前可能增加 in-flight memory-level parallelism。但 back-to-back load 也可能提高 register lifetime、scoreboard pressure 或 memory-pipeline contention，并非必然更好。

Inline PTX 可使 load 在 source block 中更早出现，但 compiler 仍可能在合法范围内调度 surrounding instruction；要固定 native schedule 需要更低层控制，而且代价更高。它不会给出“确定性”，也不自动增加真实 ILP。应比较 SASS、register/spill、stall 与 runtime。

在 CPU-GPU shared/coherent memory 上实现 GPU polling，需要完整 producer-consumer protocol：atomicity、memory order、scope、volatile/atomic load、cache/coherence semantics 和 progress 都必须满足。`membar.sys` 或 `__threadfence_system()` 主要约束调用方先前 memory operation 的 ordering/visibility，本身不会让 CPU write 自动可见、不会 invalidate arbitrary stale load，也不构成 synchronization handshake。High-level CUDA/C++ 已提供部分 system-scope atomic/fence mechanism，应优先采用 documented memory model。

> PTX 相对 SASS 更具 forward-compatibility，但仍受 PTX ISA version、target feature、driver/toolchain 和 removed/deprecated behavior 限制。SASS encoding、instruction 与 scheduling model 会随 GPU architecture 改变。

Inline PTX 也可读取 special register。例如可用 `asm("mov.u32 %0, %smid;" : "=r"(smid))` 读取当前 SM ID，用于 diagnostics 或经过验证的 work distribution；SM ID 的范围/稳定性与 scheduling semantics 不应作超出文档的假设。

Persistent kernel 有时读取 `%smid` 辅助每-SM work assignment，但多个 block 仍可能驻留或先后运行在同一 SM，block 也不可固定绑到指定 SM。仅检查 SM ID 不会自动实现“一 block per SM”或可靠 manual partitioning，还需 launch/resource/protocol 设计。

Inline PTX/SASS 的收益没有“通常几 percent”或双 load stream 固定 5%–10% 的普遍范围。Memory-bound kernel 若已饱和 bandwidth，减少 load-to-use bubble 可能不改善时间；若问题是 latency/MLP 不足，则 source rewrite、intrinsic 或 compiler option 也可能达到同样结果。任何百分比都需可复现 benchmark。

Compute-bound path 可在数值容许时选择 faster approximate math；CUDA 已提供包括 `__sinf()` 在内的 intrinsic。Inline assembly 仍需验证 accuracy、range、special value 和 instruction throughput，不能只以“更快”替代 precise operation。

早期开发者有时在 high-level intrinsic/library 尚未支持时用 PTX 访问 Tensor Core 等 feature。如今优先使用 documented intrinsic、CUTLASS/library 或 compiler support通常更可维护；PTX 能否提前使用新 feature 仍取决于 PTX ISA、assembler、driver 与 hardware support。

Nsight Compute 可辅助 assembly tuning，但 metric 名称和 section 随版本/architecture 变化，不一定存在统一的 “SASS throughput” 或 “Stall Memory Dependency”。应从 source/SASS correlation、instruction/pipe metric、warp scheduler state、memory throughput 和 dependency analysis定位问题，而不是仅凭一个 stall percentage重排 load。

优化后可能看到相关 dependency stall 降低或 issue rate 上升，但 stall reason 是归一化分布，instruction count/work 变化会影响比例；这些都不是成功的必要条件。首要证据是 correctness 与目标 workload runtime，同时检查 register/spill、traffic 和 portability。

这种 tuning 通常只应作用于已确认的 hot path，并隔离 architecture-specific implementation。GPU/toolkit 变化后必须重新验证 latency、cache、instruction availability、register allocation 与 numerical behavior。

下面用一个 hypothetical scenario 说明 microoptimization：假设已优化 kernel 的 remaining bottleneck 是包含 integer address calculation 与 memory load 的 tight loop。因为没有实际 source、device、compiler 或 measurement，后续数字只能解释概念。

PTX 的 `mad.wide.u32 (base + index * stride)` 表述并不是可直接替换的完整 instruction syntax；mad.wide 通常计算 widened integer multiply-add，而 byte address 还涉及 base type、operand width 与 constraint。随后可对特定 streaming load 评估 `ld.global.cg`，但是否减少 instruction、避开 L1 eviction 或提速均需看 generated SASS/cache behavior。原文称从 1.07 ms 降至 1.0 ms、约 7%，而表 9-2明确是 hypothetical before/after，不能当作 benchmark。

表 9-2. Optimized CUDA C++ 与手写 PTX 的 hypothetical scheduling/cache-hint 对比

版本	Warp stall（memory，示意）	Issue IPC（示意）	Kernel time（示意）	Speedup（示意）
优化的 C++（编译器调度）	35% 的周期	1.5	1.07 ms	1.0× 基线
手调 PTX（manual scheduling 与 hint）	20% 的周期	1.6	1.00 ms	1.07×

表中设定 tuning 后 memory-stall percentage、IPC 与 time 同时改善，但这是构造数据，不能证明 overlap/cache hint 是原因，也不能推出 7% net improvement。真实实验需保持 workload/correctness/clock 一致并多次测量，使用 source/SASS 与 counter 验证 causal change。

这些数字也不能代表 already-optimized kernel 对 manual assembly 的“合理预期”。收益可能为正、零或负；compiler 版本变化也可能自动采用相同 transform。错误 ordering、constraint、memory clobber 或 undefined behavior 甚至会破坏 correctness，因此每项修改都需测试和 profiling。

Inline PTX 可视为 architecture-specific escape hatch，而 SASS 更接近 target-specific diagnostic/implementation detail，并不提供“ultimate control”：compiler、assembler、scheduler、driver 和 hardware仍参与。适用情况包括 documented feature 尚无合适 high-level interface，或已精确定位 compiler codegen gap。Cache hint、prefetch、fence 等必须先满足 semantics，不能把 synchronization 当作纯性能 tweak。

应用 inline assembly 时，应验证 result、instruction sequence、resource usage 与 runtime，而不是要求 stall reason 或 instruction count 必然下降。把实现隔离、记录 PTX/SM requirement、提供 high-level fallback，并在新 architecture/toolkit 上回归；直接依赖 SASS 尤其脆弱。

> PTX 通常比 SASS 更具 portability，但 hardware、PTX ISA、compiler 与 driver 变化仍可能要求为 correctness、compatibility 或 performance 更新 inline PTX，而不只是“性能原因”。

Inline PTX tuning 适合具备 ISA、compiler、memory model 和 profiling 专业知识的开发者，用于已确认的 critical loop 或缺失 feature。它可能降低 latency或提供 custom behavior，但 source order 仍不等于 exact native schedule。反汇编与 microbenchmark 也能帮助理解 hardware，却不能单凭一次运行说明真实执行机制。

总之，应谨慎使用 inline PTX，SASS dependency 更应严格隔离。收益不保证存在，maintenance 与 portability cost 较高。多数 use case 应先采用 CUDA intrinsic、compiler optimization 或 CUB、Thrust 等成熟 library；确需下沉时，也只能获得对部分 instruction semantics 的更细控制，而非“完全控制 GPU”。

DeepSeek 使用 Inline PTX 优化Cache Allocation Policy

DeepSeek DeepEP expert-parallel library 是 custom PTX 的一个实例。它使用 `ld.global.nc.l1::no_allocate.l2::256b` load form 为 global-memory access 指定 L1 eviction-priority 与 L2 prefetch-size hint。这里优化的是 cache allocation/eviction behavior，不是 general memory allocation；它可能减少 streaming access 对 L1 working set 的干扰，但“ideal”与否取决于 reuse、cache、alignment 和 architecture。

原文称该 instruction 不属于 NVIDIA official PTX ISA、由 DeepSeek engineer 在 H800 上 “out-of-doc” 发现，这不准确。相关 load form 与 qualifier 在相应 PTX ISA 版本中有 documented syntax、semantic 和 target requirement；DeepEP 的具体组合/使用可能是 aggressive architecture-specific optimization，但不能称 instruction 本身为 undocumented bespoke discovery。

分解 `ld.global.nc.l1::no_allocate.l2::256b`：`ld.global.nc` 通过 non-coherent read-only cache path 执行 global load；其中 `nc` 有明确 consistency restriction，基础 operation 是 `ld.global`，不能用于可能被同一 kernel/并发 producer 修改而又缺少相应可见性保证的数据。`l1::no_allocate` 是 L1 eviction-priority hint，`l2::256b` 是 L2 prefetch-size hint。具体说，`l1::no_allocate` 不保证完全 bypass L1，而 `l2::256b` 也不表示 load 精确“每次取 256 byte”或 operand 必须按 256 byte 对齐；这些 performance hint 可被 implementation 忽略。

降低 streaming load 在 L1 的 allocation priority，可能减少对具有 reuse 的 L1-resident working set 的 eviction；但 nc path、L1 behavior 与 cache hierarchy 依 architecture，不能简单描述成绕过 L1并直接送入 L2。若 streaming data 后续有 reuse，no-allocate 也可能损害 performance。

Expert-parallel all-to-all kernel 常访问 large communication buffer，因此这种 hint 可能有益；但 buffer 是否 contiguous、是否只读一次、cache line 是否仍活跃以及 bottleneck 在 network、HBM、L2 还是 synchronization，均需针对 DeepEP version、message size 与 GPU topology 测量。

L2::256B hint 可请求在 accessed address 周围预取额外 data，不能保证 aligned 256-byte transaction，也不能单独证明 L1 traffic 减少或 communication/compute throughput 同时提高。应比较 relevant cache/sector metric 与端到端 time，并检查多余 prefetch 是否浪费 bandwidth/cache capacity。

这种 PTX 仍有 portability risk：qualifier 需要最低 PTX ISA/SM target，未来 architecture 的 cache behavior 与 profitability 也可能变化。只要使用 documented syntax 并正确限定 target，风险主要是 compile compatibility、performance regression 或错误使用 nc consistency semantic，而不是 qualifier 必然“不稳定”并随机产生错误。`ld.global.nc.l1::no_allocate.l2::256b` 的 correctness 必须依据数据是否允许 non-coherent read。

DeepEP build 提供 `DISABLE_AGGRESSIVE_PTX_INSTRS=1` 等开关，可在 compatibility/performance 问题时禁用 aggressive path。该开关本身不证明指令 undocumented 或 speedup 显著。Architecture-specific PTX 应具备 target guard、fallback、correctness test 和 representative benchmark，并在 GPU、driver、toolkit 或 DeepEP version 更新后重新验证。

要点回顾

本章介绍了如何发现 GPU kernel 的 bottleneck，并通过减少不必要的 global-memory traffic、提高 on-chip data reuse 以及更有效地利用 compute unit 来缓解这些瓶颈。反复执行 profiling、diagnosis、optimization 和 reprofiling，可以改善 underutilized 或 memory-bound kernel 的 throughput；但优化是否会改变其 performance regime，取决于 workload、hardware 和最终暴露出的新瓶颈。以下方法有助于提高 GPU 的实际利用率：

通过 tiling 与 fusion 提高 arithmetic intensity

要提高每 byte data movement 对应的 FLOP，可使用 multilevel tiling 将 data staged 到 shared memory 和 register。例如，将一个 32 × 32 submatrix 载入 SMEM，使每个 element 可供多次 FMA 复用。还可以 fusion 连续 kernel 中的 elementwise operation，让 intermediate result 尽量保留在 on-chip storage。对于满足 API contract 且存在 independent computation 的 copy，可通过 CUDA Pipeline API 的 `cuda::memcpy_async` 执行 asynchronous memory loading，使部分 data movement 与 computation overlap，从而可能覆盖一部分 DRAM latency；实际效果仍取决于 schedule、copy path、resource pressure 和 workload。

利用 mixed precision、Tensor Core 与 Transformer Engine

相对于 FP32，BF16/FP16、FP8 和 FP4 可将相应 raw value payload 分别缩小到约 1/2、1/4 和 1/8；但 TF32 是 compute format，不会把 stored FP32 weight traffic 减半，而 scale、metadata、activation、output、alignment 与 conversion traffic 也不会按相同比例消失。因此，lower precision 可能提高 arithmetic intensity 和 Tensor Core throughput，但增益必须结合完整 data path 与 numerical-quality target 测量。在 PyTorch 中，`torch.set_float32_matmul_precision('high')` 设置 float32 matmul 的 internal-compute precision policy，`torch.cuda.amp` 则提供 autocast 与 gradient scaling 等 mixed-precision 机制；普通 AMP 并不自动启用 FP8/FP4。TMA 可为符合条件的 kernel 执行 global memory 与 shared memory 之间的 bulk tensor transfer，特定 Blackwell Tensor Core data path 则使用 TMEM；它们不是一个会自动把 tile stream 到 Tensor Core 的统一 engine。Supported hardware 与 software stack 还可通过 Transformer Engine 的 scaling、casting 和 low-precision recipe 提高 AI workload 的 throughput。

使用 CUTLASS 构建 high-performance GEMM 与 fused kernel

与从头手写 MMA loop 相比，可以先评估 CUTLASS GEMM template。CUTLASS 提供 architecture-specific tile、pipeline 与 epilogue abstraction，但 double buffering、TMA、TMEM 和 Tensor Core path 的实际使用取决于 target architecture、CUTLASS version、schedule、dtype、layout、alignment 与 problem shape，性能也不保证始终落在 hand-tuned kernel 的几个百分点之内。cuBLAS 是独立的 proprietary library，不能笼统视为构建于 CUTLASS 之上；TorchInductor 也会根据 graph、device 和 backend availability 选择不同 code-generation path。只有在 representative benchmark 证明现有 library/backend 不满足 layout、fusion、latency 或 throughput 要求时，才值得承担 custom kernel 的开发与维护成本。

PyTorch 特定最佳实践

优先评估 `torch.matmul`、fused attention 和 nested tensor 等 built-in tensor operation，因为 PyTorch backend 会持续采用合适的 CUDA library 与 hardware optimization。若要为 PyTorch 编写 custom kernel extension，应沿用相同的基本原则：为 coalesced memory access 正确安排 data layout 与 alignment，在适用时使用 Tensor Core，并在 register use、spill、occupancy 与 ILP 之间取得平衡，而

不是一律最小化 register。Custom kernel 是否达到或超过 native kernel，必须通过同一 workload 下的 correctness test 与 benchmark 验证。

通过 profiling、diagnosis 和 targeted optimization 的系统化流程，可以依次评估 occupancy tuning、warp-level refinement、ILP，以及借助 tiling、fusion 和 Tensor Core 提高 arithmetic intensity 等手段。它们可能显著缩短 memory-bound kernel 的执行时间，有时也会使瓶颈转移到 compute；但 regime 转换或数量级 speedup 都不是普遍保证，必须相对于明确 baseline，在 representative workload 上测量端到端收益。

总结

本章介绍了与 TMA、TMEM、Transformer Engine 和 Tensor Core 等 advanced memory/compute feature 相关的优化技术，其中不少分析原则也适用于从 single GPU 扩展到 multi-GPU cluster 的场景。可以先用 system-level profiler（例如 NVIDIA Nsight Systems）关联 CPU activity、GPU kernel 以及多个 GPU 之间的 NVLink/NVSwitch traffic，再用 Nsight Compute 深入分析单个 kernel 的细节。

系统化 profiling 有助于识别 dominant stall，并判断 advanced hardware feature 是否适合目标 path。成功的优化可能带来较大收益，强 memory-bound path 相对较差 baseline 有时甚至能获得数量级改善；但工具与 feature 本身不保证把 memory-bound workload 转为 compute-bound，也不保证固定幅度的 speedup。

即使在 NVIDIA GB200/GB300 NVL72 这类具有高 NVLink/NVSwitch bandwidth 的 ultrascale multi-GPU system 中，per-GPU arithmetic intensity 仍是重要考量，但它不是消除大多数 bottleneck 的唯一关键。每 byte 完成的有效工作较少，通常会增加 memory movement 压力；是否最终饱和 HBM 或 interconnect，则取决于 communication pattern、topology、message size、compute-to-communication ratio、reuse 和 overlap。

Interconnect 或 memory bandwidth 也不一定先于 compute 饱和。Multi-GPU scaling 还可能受 collective algorithm、topology mapping、load imbalance、latency、serialization、synchronization、host/input pipeline 以及 communication-computation overlap 限制。因此，提高 kernel arithmetic intensity 应与减少 communication volume、改进 collective/scheduling、保持 load balance 和端到端 profiling 一并考虑。

下一章将继续深入 persistent kernel、megakernel、warp specialization、cooperative group 和 thread block cluster 等技术。这些思想构成许多 modern LLM runtime optimization 的基础，因此理解其实现方式以及如何将其用于 low-level system optimization 十分重要。

第 10 章　Intra-Kernel Pipelining、Warp Specialization 与 Cooperative Thread Block Cluster

前几章介绍了多项基础优化，包括调整 memory access、提高 parallelism、让 computation 与 data transfer overlap、改善 occupancy，以及减少 warp stall。这些方法有助于覆盖 latency 并缓解 bottleneck。现代 GPU 还提供 advanced hardware feature 与 execution model，可在这些基础上进一步优化。

本章将介绍一些更高级的 CUDA 技术，例如 warp-specialized pipeline、支持 grid-level 和 cluster-level synchronization 的 cooperative group、循环处理 dynamic work queue 的 persistent kernel，以及使用 distributed shared memory（DSMEM 或 DSM）和 Tensor Memory Accelerator（TMA）multicast 的 thread block cluster（原文称 *cooperative thread array cluster* [CTA]，术语订正见下文）。概括来说，thread block cluster 是一组具有 cluster-level co-scheduling guarantee 的 thread block；在支持该特性的 architecture 上，它们可通过 DSMEM 访问彼此的 shared-memory allocation，包括 read、write 和 atomic operation。原文的别名并不准确：CTA 通常指 cooperative thread array，也就是 CUDA thread block；这里讨论的是由多个 CTA 组成的 CTA cluster。

这些方法可以在不需要 host 逐阶段介入的情况下，使部分 memory access 与 compute operation overlap，也能让同一 cluster 内的 thread block 共享 on-chip data。它们可能提高 SM utilization，但不能保证每个 SM 始终 fully utilized；实际结果仍受 resource footprint、cluster size、grid shape 和 workload 限制。

理解这些 modern GPU execution model 后，下一章将进一步讨论使用 CUDA stream 构建的 inter-kernel pipeline。下一章的跨 kernel pipeline 建立在本章所述 intra-kernel optimization 的基础之上。

Intra-Kernel Pipelining 技术

Intra-kernel pipelining 是指在一次 kernel execution 内让 memory operation 与 computation overlap 的一组技术。（下一章将讨论 inter-kernel pipelining，即让不同 stream 中多个 kernel 的工作彼此 overlap。）

其核心思想是把 kernel 组织成可重叠推进的 stage：在载入或存储一块 data 时，处理此前已经载入的数据。不同 stage 可同时作用于不同 tile 或 data chunk，从而可能提高 throughput 并覆盖部分 latency；能否真正并发取决于 instruction dependency、hardware pipeline 和 resource availability。

传统上，GPU 依靠 warp-level multithreading 隐藏 latency。当一个 warp 因 memory load 而 stall 时，scheduler 可以选择其他 ready warp 执行 computation。

这是 single instruction, multiple threads（SIMT）execution model 中 latency hiding 的基础。

Intra-kernel pipelining 在此基础上进一步安排 memory 与 compute，使它们在同一 kernel 内、某些情况下甚至在同一 warp 的 instruction stream 中交错推进。它通过 fine-grained coordination 错开 memory load 与 computation。

CUDA Pipeline API 可协调 asynchronous memory transfer 与 computation，但原文所称“无需任何 `__syncthreads()`”并非通则：是否需要额外 block-level synchronization，取决于 pipeline scope、参与线程、shared-memory access pattern 以及 API contract。常见的 intra-kernel pipelining 组织方式包括 double buffering 与 warp specialization。

在 double-buffered（two-stage）pipeline 中，通常由一组线程协作推进 producer/consumer stage；在 warp-specialized pipeline 中，不同 warp 被分配为 memory loader、compute 或 memory storer 等角色。具体选择取决于 workload、architecture 和 performance target。表 10-1概括了这两种使用 `<cuda/pipeline>` 的组织方式；它们是设计模式，不是彼此排斥的独立 API variant。

表 10-1　现代 GPU 上使用 CUDA Pipeline API 实现 intra-kernel pipelining 的两种组织方式

组织方式	适用场景	主要用途
双缓冲流水线	基于 loop 的 tiling 与 double buffering	在同一 warp 或 block 内安排 load 与 compute overlap
Warp-specialized pipeline（例如 memory loader、compute、memory storer 三个 stage）	包含多个不同 warp role 的 persistent kernel（本例为三个）	把 warp 分配给 memory load、compute 和 memory store 等不同 role/stage

使用 CUDA Pipeline API 实现 Cooperative Tiling 与 Double Buffering

可以使用 C++ Pipeline API 实现传统的 double-buffered tiling pattern：实例化一个 two-stage pipeline，尝试让 memory load 与 computation overlap。具体来说，可以声明一个 two-stage `cuda::pipeline_shared_state<cuda::thread_scope_block, 2>` object，并用 cooperative groups（稍后讨论）取得的 thread-block group 创建 block-scoped pipeline。这本质上是 producer-consumer pattern，如图 10-1所示。

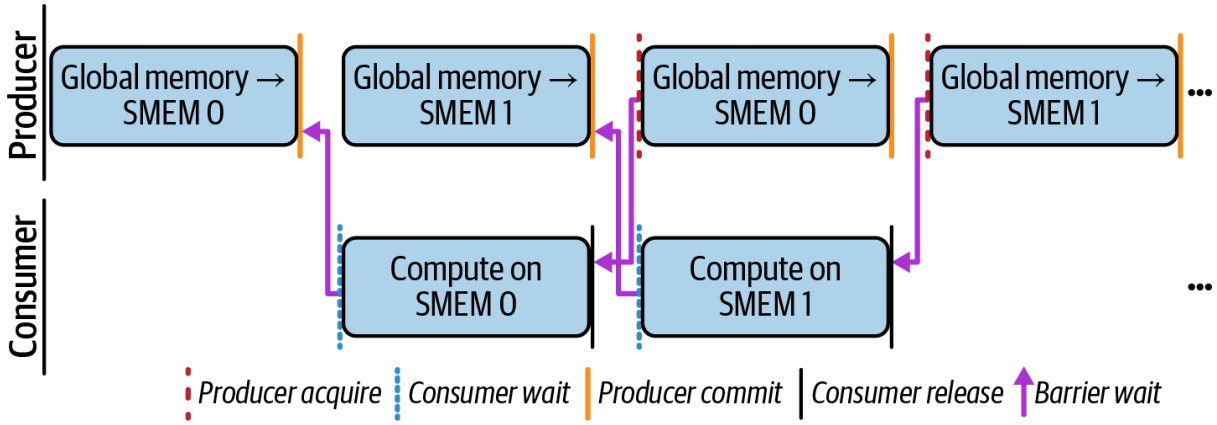


图 10-1　使用 CUDA Pipeline API 的 two-stage producer-consumer pattern

下面列出关键 CUDA Pipeline API call，随后给出一个使用该 API 的 double-buffered cooperative-tiling kernel 示例：

`pipe.producer_acquire()`

等待并取得下一个可供 producer 写入的 pipeline stage

`pipe.producer_commit()`

提交当前 producer stage，使此前关联到该 stage 的 asynchronous operation 可由 consumer 等待

`pipe.consumer_wait()`

等待下一个已提交 consumer stage 的 operation 完成，避免读取尚未就绪的数据

`pipe.consumer_release()`

释放当前 consumer stage，使其可再次供 producer 使用

Two-stage pipeline 的目标是让 global-memory load 与 computation overlap。概念图可让一个 producer role 把下一 tile asynchronous prefetch 到 shared memory，同时 consumer role 处理已就绪的 tile。对于满足 alignment、size、collective participation 等契约的 global-to-shared copy，`cuda::memcpy_async` 可能使用 hardware-accelerated asynchronous copy path；但不能笼统保证每个 cooperative call 都按“每线程一条 `cp.async`”lowering。

原文把 Stage 0 描述为一个 warp loading、其余 warp 在 Stage 1 computing，但下方清单并未实现这种 warp specialization：所有线程沿同一控制流调用 producer 与 consumer API。Double buffering 仍可能让先前发出的 copy 与后续 compute overlap，从而覆盖部分 DRAM latency；是否实现及覆盖程度需要检查 generated code 与 profiler，如图 10-2的概念时序所示。

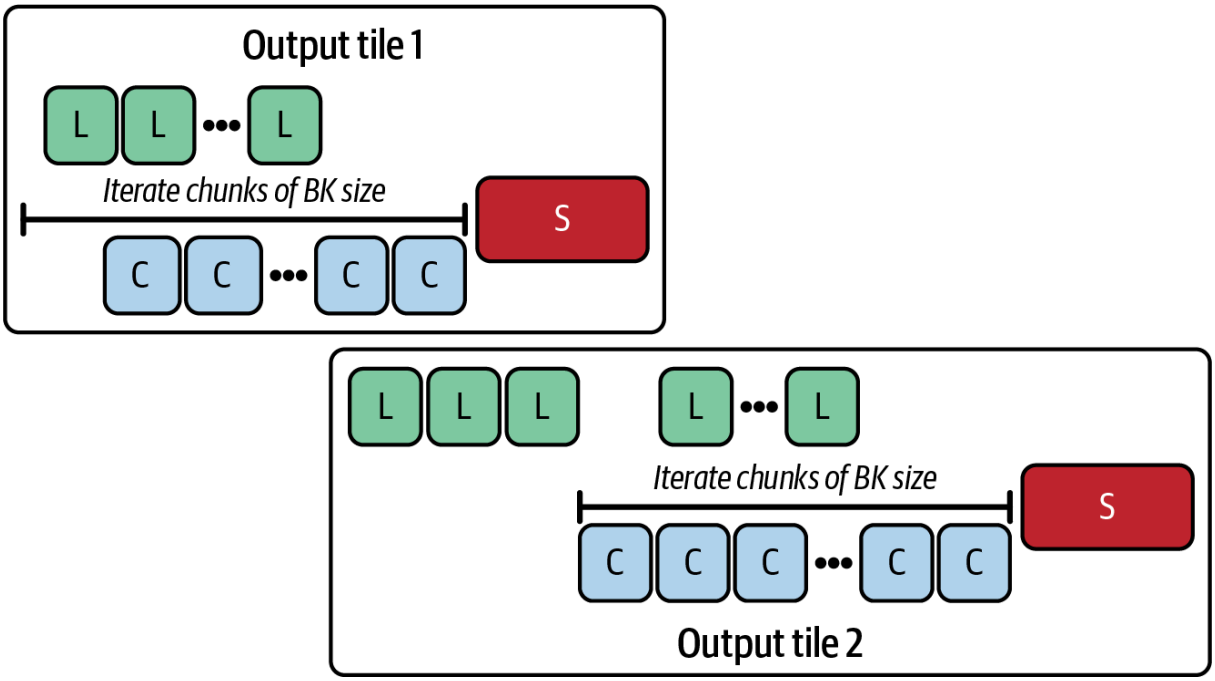


图 10-2　使用 producer-consumer pipeline 以 compute（C）覆盖部分 global DRAM load（L）latency

这种 pattern 可能提高 SM utilization 并缩短 kernel time，但 kernel 属于 compute-bound 还是 memory-bound，取决于 operation、tile size、resource pressure 与 overlap efficiency。原文以 FlashAttention-3 约达到 peak FP16 FLOP/s 的 75% 为例；该数字来自特定 Hopper workload、precision、shape 与计数口径，不能当作 attention kernel 或 pipelining 的通用上限。

下面是使用 CUDA C++ Pipeline API 的 two-stage double-buffering 示例。它演示 fine-grained producer-consumer synchronization 的基本形式，但按展示文本存在 collective-copy contract 与 benchmark provenance 问题，不能直接作为正确、可复现的 GEMM implementation：

```
#include <cuda/pipeline>
#include <cooperative_groups.h>
#include <algorithm>
namespace cg = cooperative_groups;

#ifndef TILE_SIZE
#define TILE_SIZE 32
#endif
#ifndef STAGES
#define STAGES 2          // 2 = double buffer, 3 = triple buffer, etc.
#endif

__device__  float computeTile(const float* __restrict__ A_sub,
                              const float* __restrict__ B_sub,
                              int tx, int ty) {
    float s = 0.0f;
    #pragma unroll
    for (int k = 0; k < TILE_SIZE; ++k) {
        s += A_sub[ty * TILE_SIZE + k] * B_sub[k * TILE_SIZE + tx];
    }
    return s;
}
```

```
extern "C" __global__
void gemm_tiled_pipeline(const float* __restrict__ A_global, // [M x K]
                         const float* __restrict__ B_global, // [K x N]
                         float* __restrict__ C_global,       // [M x N]
                         int M, int N, int K) {
    cg::thread_block cta = cg::this_thread_block();

    // Shared memory layout: A[STAGES] then B[STAGES]
    extern __shared__ float shared_mem[];
    float* A_buf[STAGES];
    float* B_buf[STAGES];
    {
        float* p = shared_mem;
        for (int s = 0; s < STAGES; ++s) {
            A_buf[s] = p;
            p += TILE_SIZE * TILE_SIZE;
       }

        for (int s = 0; s < STAGES; ++s) {
            B_buf[s] = p;
            p += TILE_SIZE * TILE_SIZE;
       }
    }

    __shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, STAGES> state;
    auto pipe = cuda::make_pipeline(cta, &state);

    int tx = threadIdx.x, ty = threadIdx.y;
    int block_row = blockIdx.y * TILE_SIZE;
    int block_col = blockIdx.x * TILE_SIZE;

    float accum = 0.0f;
    int numTiles = (K + TILE_SIZE - 1) / TILE_SIZE;

    // Prologue: load first STAGES tiles (or fewer if short)
    for (int s = 0; s < std::min(STAGES, numTiles); ++s) {
        int aRow = block_row + ty;
        int aCol = s * TILE_SIZE + tx;
        int bRow = s * TILE_SIZE + ty;
        int bCol = block_col + tx;

        pipe.producer_acquire();
        if (aRow < M && aCol < K) {
            cuda::memcpy_async(cta, A_buf[s] + ty*TILE_SIZE + tx,
                               &A_global[aRow*K + aCol],
                               cuda::aligned_size_t<32>(sizeof(float)), pipe);
        } else {
            A_buf[s][ty*TILE_SIZE + tx] = 0.0f;
        }
        if (bRow < K && bCol < N) {
            cuda::memcpy_async(cta, B_buf[s] + ty*TILE_SIZE + tx,
                               &B_global[bRow*N + bCol],
                               cuda::aligned_size_t<32>(sizeof(float)), pipe);
        } else {
            B_buf[s][ty*TILE_SIZE + tx] = 0.0f;
        }
        pipe.producer_commit();
    }

    // Steady state
    for (int tile = 0; tile < numTiles; ++tile) {
        int s = tile % STAGES;

        // Block-scope wait
```

```
            pipe.consumer_wait();

            accum += computeTile(A_buf[s], B_buf[s], tx, ty);
            pipe.consumer_release();

            // Prefetch next tile into the same slot s (ring buffer)
            int nextTile = tile + STAGES;
            if (nextTile < numTiles) {
                int aRow = block_row + ty;
                int aCol = nextTile * TILE_SIZE + tx;
                int bRow = nextTile * TILE_SIZE + ty;
                int bCol = block_col + tx;

                pipe.producer_acquire();
                if (aRow < M && aCol < K) {
                    cuda::memcpy_async(cta, A_buf[s] + ty*TILE_SIZE + tx,
                                       &A_global[aRow*K + aCol],
                                       cuda::aligned_size_t<32>{sizeof(float)}, pipe);
                } else {
                    A_buf[s][ty*TILE_SIZE + tx] = 0.0f;
                }
                if (bRow < K && bCol < N) {
                    cuda::memcpy_async(cta, B_buf[s] + ty*TILE_SIZE + tx,
                                       &B_global[bRow*N + bCol],
                                       cuda::aligned_size_t<32>{sizeof(float)}, pipe);
                } else {
                    B_buf[s][ty*TILE_SIZE + tx] = 0.0f;
                }
                pipe.producer_commit();
            }
        }

        // Epilogue: final store (guard tails)
        int cRow = block_row + ty;
        int cCol = block_col + tx;
        if (cRow < M && cCol < N) {
            C_global[cRow * N + cCol] = accum;
        }
    }
```

代码先使用 cooperative groups（CG，稍后讨论）取得当前 thread block 的 handle，然后创建绑定到该 block 的 two-stage `cuda::pipeline` object。Shared state 必须在相关 asynchronous operation 之前正确初始化，而且参与 collective operation 的线程必须遵守相应 convergence 与 argument contract。

Kernel 通过 `extern __shared__ float shared_mem[]` 为 A、B tile 分配一块 contiguous dynamic shared-memory region，再用 pointer arithmetic 将其分成四个 subregion：两个 A buffer（`float* A_buf[2]`）和两个 B buffer（`float* B_buf[2]`）。这提供 double-buffer storage，不涉及额外 runtime device allocation；host launch 时仍须配置足够的 dynamic shared-memory byte。

进入 main loop 前，代码尝试预取前 `STAGES` 个 tile，并按 `producer_acquire()` → `memcpy_async()` → `producer_commit()` 的顺序操作 pipeline。随后用 `consumer_wait()` 等待当前 consumer stage 的 asynchronous operation 完成，并在使用后调用 `consumer_release()`。

这套 pipeline synchronization 只在 API contract 得到满足时才建立 stage reuse 所需的 ordering，不能概括为一个“initial barrier”永久取代后续所有 `__syncthreads()`。

是否还需要其他 synchronization，取决于 participating thread 与 shared-memory access pattern。

原文称每轮先调用 `pipe.producer_acquire()` 取得下一 buffer，再为 A、B tile 各发出一次 `cuda::memcpy_async`，并通过 `cuda::memcpy_async(..., pipe)` 把 copy 关联到 pipeline。实际清单的 steady-state 顺序却是先 compute/release 当前 stage，之后才 enqueue next tile；潜在 overlap 发生在后续 iteration，并非同一 iteration 内由独立 producer warp 与 consumer warp 并发。

这些 asynchronous copy 只有在 access、tiling 与 API contract 均正确时才可能与 compute overlap。清单中的 `memcpy_async` 却在 per-thread 边界分支中被不同线程调用或跳过，并接收不同 pointer；这不符合 block-group collective copy 的通常用法。随后调用 `pipe.producer_commit()` 也不能修复 convergence、argument 或 alignment contract，因此不能据此断言 pipeline 始终 filled，或 copy 不会 block 整个 thread block。

原文声称“其他 warp”调用 `pipe.consumer_wait()`，且只有依赖由 `s = tile % STAGES` 索引的 stage 的线程 stall；清单实际上让整个 CTA 中所有线程操作同一个 block-scoped pipeline。Wait 完成后，各线程调用 `computeTile(...)`，计算自己 output element 的 K-tile partial dot product，而不是每线程计算一个完整 `TILE_SIZE x TILE_SIZE` tile。变量 s 也未传给 wait；stage progression 由 pipeline sequence 决定。

完成当前 buffer 的 computation 后，参与线程调用 `pipe.consumer_release()`，使相应 stage 可供后续 producer reuse。Fine-grained pipeline 可以避免不必要的 synchronization，但是否减少 block-wide stall 或最大化 overlap 仍需由合法实现和 profiler 证明。

每轮结束时，ring buffer 以 stage index 循环。下一次 prefetch 使用 `nextTile = tile + STAGES`，并写入 slot `s`；该 slot index 按 `STAGES` 取模循环。

Producer/consumer sequence 遍历 K dimension 的 tile。每个 thread 的 accumulator 对应一个 output element 的完整 K-dimension dot product，而不是原文所称“完整 `TILE_SIZE x TILE_SIZE` dot-product result”；整个 CTA 的线程集合才共同覆盖 output tile，并将各自结果写入 `C_global`。

Double buffering 中，每个 asynchronous copy（`memcpy_async`）必须满足 copy contract，并与适当的 `producer_commit()` 和 `consumer_wait()` 配对，才能保证 consumer 在 data ready 后读取。Compiler 可能优化某些简单 pattern，但不会自动修复语义错误，也不保证自动构造完整 multistage pipeline。

> 应使用与 GPU architecture、tool version 和 copy path 对应的 Nsight Compute metric，验证 pipeline 是否按预期执行。除 achieved occupancy 与 shared-memory bank conflict 外，还应检查 asynchronous-copy request、barrier wait、memory throughput、eligible warp 和 kernel duration。优化目标是缩短目标 workload 的时间，而不是要求 SM Active percentage 必然上升或 stall metric 必然下降；DRAM latency 能覆盖多少还取决于 dependency、occupancy、access pattern、resource pressure 和 shared-memory behavior。

原文称该 double-buffered scheme 比 naive tiling 快约 2×，并在表 10-2中给出对比。但表中数值明确是 illustrative，且当前代码存在上述 correctness/contract 问题，因此不能将其视为该清单的实测结果。

表 10-2　Naive tiling 与使用 CUDA Pipeline API 的 double-buffered kernel 的示意性性能对比

指标	朴素分块	两阶段双缓冲流水线（`double_buffered_pipeline`）
内核执行时间	41.3 ms	20.5 ms（相对于 naive 约快 2×）
SM 活跃百分比	68%	92%（相对于 naive 增加 24 percentage point）

> 注意：所有 metric table 中的数值都只是用于解释概念的示意值。不同 GPU architecture 上的实际 benchmark 结果请参阅 GitHub repository。

若把这些示意值当作假设场景，`gemm_tiled_pipeline` 的时间由 41.3 ms 降至 20.5 ms，约为 2× speedup，SM Active 则由 68% 增至 92%，即增加 24 percentage point；原文写“增加 24%”容易与 relative increase 混淆。由于没有给出 device、shape、compiler、launch configuration、naive baseline 和可复现实测，本表不能证明 `pipe.producer_commit()`/`pipe.consumer_wait()` 使 pipeline 始终 filled，也不能证明这些数字来自上方清单。

Warp Specialization 与 Producer-Consumer Model

Warp specialization 在 double buffering 的基础上，把不同 operation role 分配给不同 warp，例如由 producer warp 协调 data movement（如 TMA），由 consumer warp 执行 compute（如 Tensor Core MMA）。这不同于让同一组 warp 同时承担 load 与 compute，如图 10-3所示。

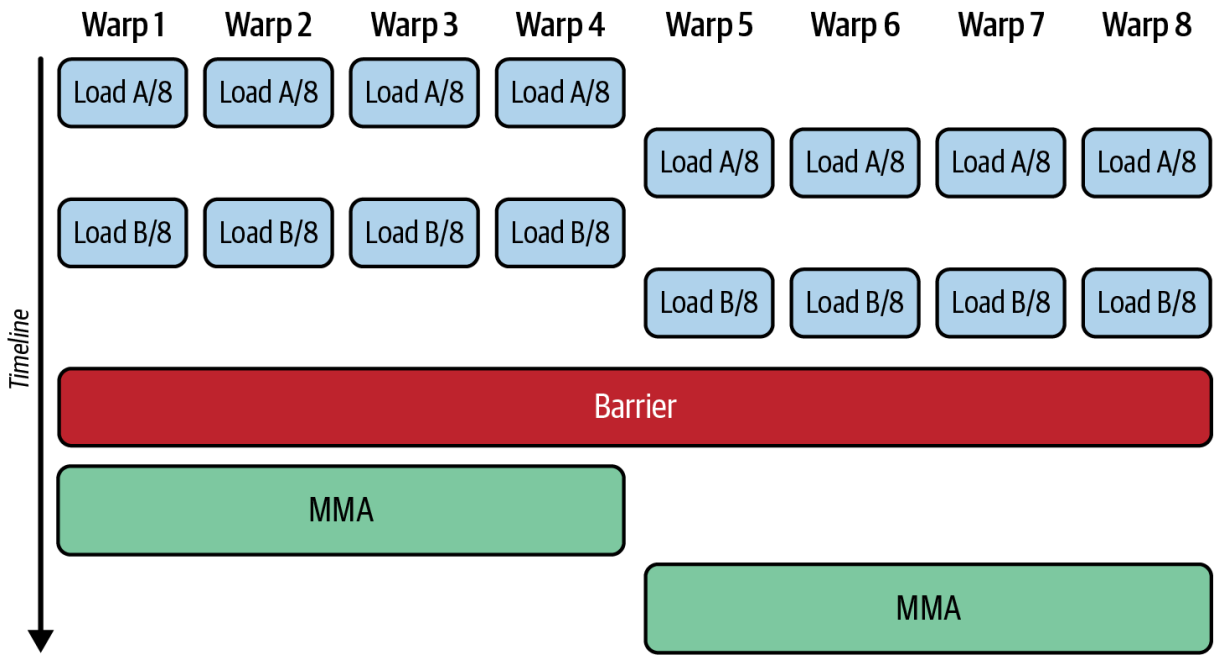


图 10-3　未采用 warp specialization 的 kernel：每个 warp 混合执行 data loading 与 compute（来源：https://oreil.ly/WZDbM）

这种 specialization 让不同 warp set 执行各自的 instruction sequence，从而减少单个 warp 内不同 role 的交错；但 instruction 能否连续 issue 仍受 dependency、scheduler、execution-pipeline availability 和 barrier 限制。可以让一组“producer”或“memory”warp 使用 `cuda::memcpy_async` 等机制 asynchronous prefetch tile，再由“consumer”或“compute”warp 执行 computation，如图 10-4 所示。

图中把 4 个 warp 分配为 producer，其余 8 个 warp 分配为 consumer。Producer/consumer 的数量可以不同，但合法且高效的比例取决于 threads per CTA、warpgroup requirement、pipeline rate、resource usage 和 target architecture，并非任意配置都可行。

原文称“每个 warp 都有自己的 scheduler”，这不准确。一个 SM 通常包含多个 warp scheduler，每个 scheduler 管理一组 resident warp，并从 ready warp 中选择 instruction。若 architecture 的 issue partition 与 execution pipeline 允许，不同 scheduler 可能在同一 cycle 从不同 warp 发出 memory、math 或其他 instruction；但 load、math、store 能否同时 issue 并无普遍保证。

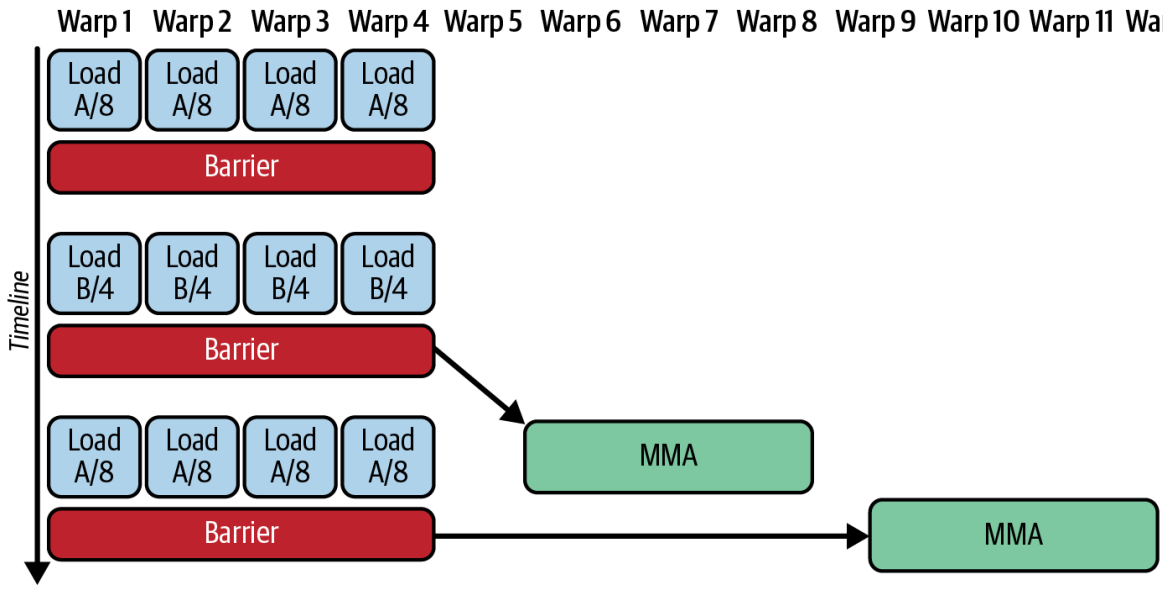


图 10-4　Warp-specialized kernel：一组 warp 负责 loading，其余 warp 负责 computation（来源：https://oreil.ly/WZDbM）

这种组织方式可增加同一 thread block 内不同 ready warp 向不同 execution pipeline 提供 instruction 的机会。原文将其称为 thread-block-level multi-issue，并断言 single-warp double buffering 不可能做到；更准确地说，单个 warp 的 issue capability 受 architecture 限制，而多个 warp role 可以提供更多 independent ready instruction，但实际 issue width 与 concurrency 仍由硬件决定。

一种 warp-specialization pattern 使用三类 warp：“loader”“compute”和“storer”。Loader warp 把 tile 推入 pipeline queue，compute warp 处理各 tile，storer warp 写出 result，如图 10-5所示。具体 implementation 还需要 matching barrier、buffer ownership 与 memory-order protocol。

Warp-specialized pipeline 可能减少 sequential load-and-compute loop 中仍然存在的 idle cycle。对于 long-running loop 和 persistent kernel，role coordination 与 data handoff 的开销可在多次 iteration 中摊销，因此 data transfer 与 computation 的 overlap 可能提高 GPU utilization；但 barrier cost、role imbalance 或资源占用也可能抵消收益。

一篇关于 warp scheduling 的论文在其特定 kernel 与硬件上展示了接近完整的 memory/compute overlap。该 workload 原先具有分离的 memory phase 与 compute phase，两者交替成为 bottleneck；应用 warp specialization 后，profiling 显示 SM memory subsystem 与 compute unit 在大部分测量区间内同时活跃。这个结果依赖论文中的 workload、schedule 和 metric，不能外推为通用保证。

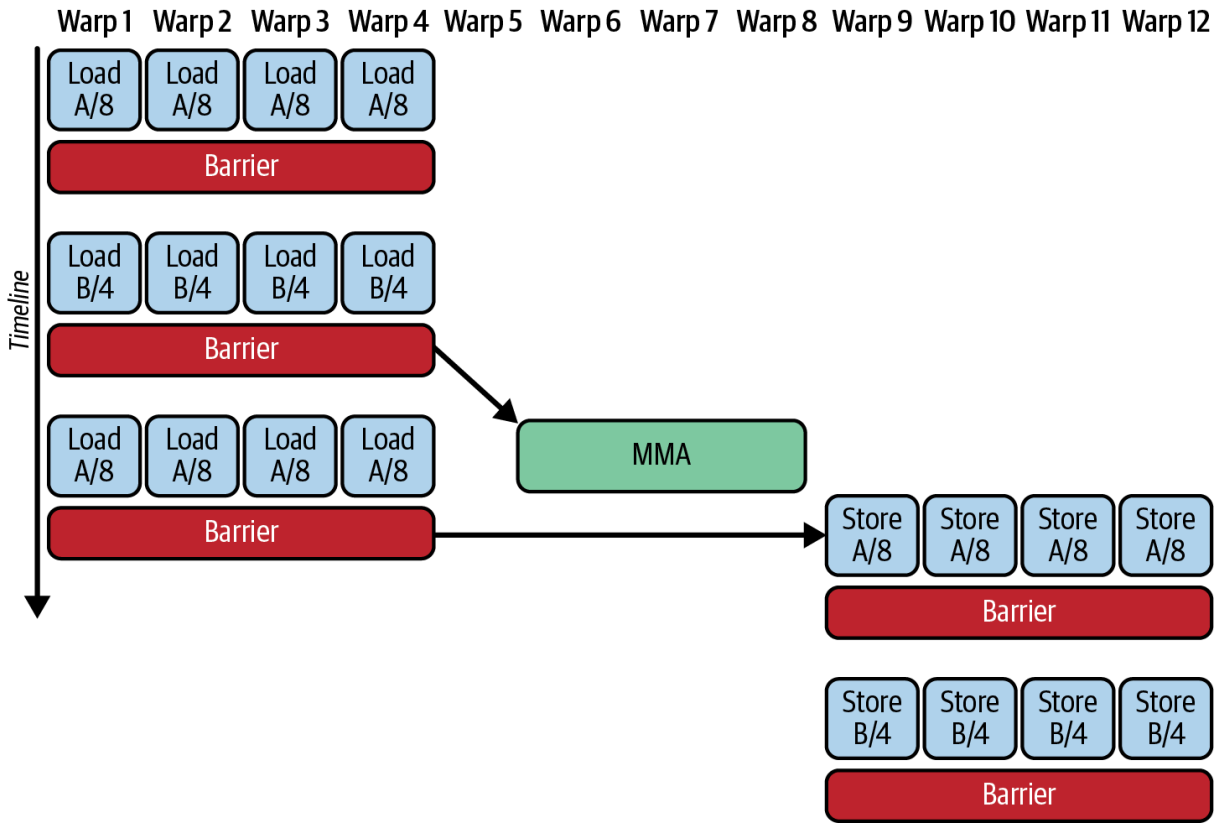


图 10-5　Three-role warp-specialized pipeline：三组 warp 分别负责 loading、compute 和 storing（来源：https://oreil.ly/xs7YN）

在该研究的 profiling 中，使用 warp specialization 前，L2 bandwidth utilization 与 Tensor Core utilization 的活跃区间错开；使用后，两者的活跃区间更多重叠，并提高了 effective throughput。这说明 warp specialization 能在某些已使用 asynchronous pipeline 的 workload 中进一步减少 idle time，但收益大小必须由具体 timeline 与 runtime 验证。

另一种 pattern 修改了 three-role warp-specialized pipeline：仍用一组 warp 充当 memory loader，同时让两组 consumer warp 在 compute 与 memory-storer role 之间“ping-pong”。CUTLASS 的特定 Hopper GEMM schedule/示例中使用名称 `gemm_tma_warpspecialized_pingpong` 表示这类组织方式，如图 10-6所示；具体 API 名称与 availability 依 CUTLASS version。

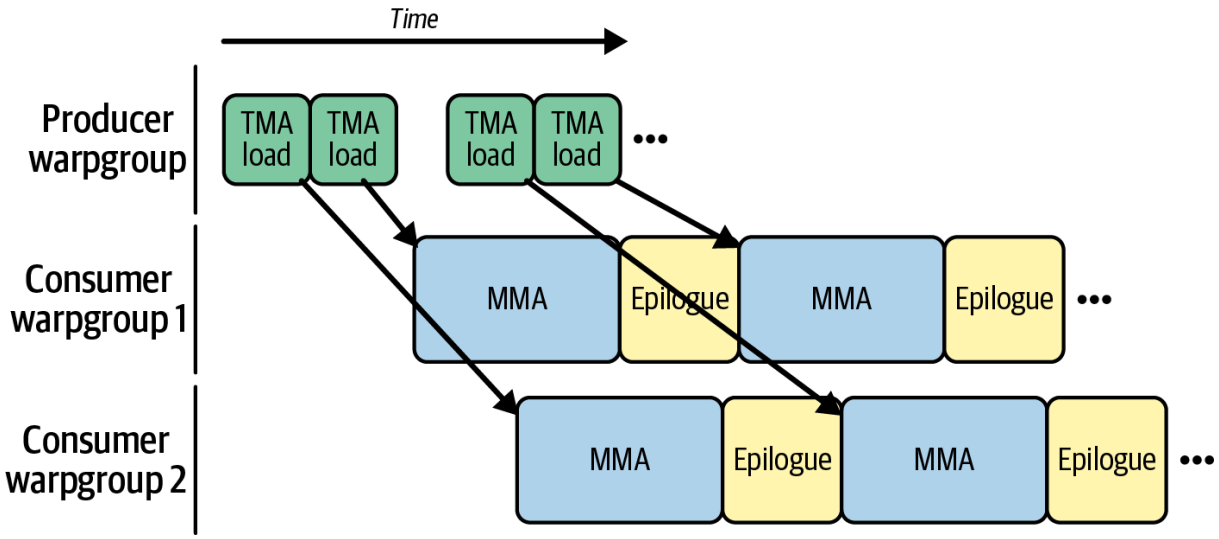


图 10-6　使用 three-role warp-specialized kernel 的 ping-pong architecture（来源：https://oreil.ly/xs7YN）

图中两组 consumer 的 MMA/epilogue phase 交错推进。MMA 后的 *epilogue* 可包含 accumulation、scaling、写回 global memory，或在线程/warp 间重新排列 result；control flow 还可推进 tile pointer、更新 loop counter，并通过 synchronization signal 允许后续 TMA load 或 MMA 继续。并非所有 per-MMA epilogue 工作都必须在“下一条 MMA”启动前串行完成；依赖关系取决于 buffer、accumulator 与 schedule。

> 图中没有显示 MMA 前的 prologue。Prologue 可先发出若干 TMA request，把 global-memory tile 搬入 shared memory 并填充 pipeline，随后 consumer 再按 MMA operand contract 从 shared memory 取数或使用相应 tensor-core data path。原文称 TMA 直接把 data 搬入 register 不准确。

在适合的 workload 中，warp specialization 可以进一步提高 performance。FlashAttention-3 把部分 speedup 归因于 warp-specialized pipeline：它让 GEMM、softmax 与 data movement 更充分地 overlap。但其 achieved FLOP/s、利用率与收益依 Hopper、head dimension、sequence length、dtype 和 benchmark configuration，不能概括为让所有 hardware unit 始终 busy 或普遍达到 near-peak FLOP/s。

此外，第 13 章和第 14 章将讨论 PyTorch compiler。某些 PyTorch/TorchInductor backend 或 CUTLASS integration 可为 supported operation 选择 warp-specialized schedule，并使用 architecture-appropriate barrier 协调 loader 与 compute role。`torch.compile` 和 Triton 是否生成此类 kernel，取决于 version、backend、architecture、operator、shape 和 heuristic；它们不会为每个 operator 启用 warp specialization。“Low-cost”也不是固定属性，barrier overhead 应纳入 profiling。

当 workload 存在可分离的 producer/consumer role、需要隐藏 latency，且额外 role coordination 有望摊销时，可以评估 warp specialization，尤其是在 simpler schedule 无法覆盖 memory-load latency 时。若 kernel 很小、缺少 independent compute，或 role/barrier/resource overhead 较高，simpler double buffering 可能以较低代码复杂度取得相近甚至更好的效果。应以 correctness、kernel time 和端到端 benchmark 决策。

使用 CUDA Pipeline API 实现 Warp Specialization

Warp specialization 可建立在 CUDA Pipeline API 之上，使用 fine-grained producer/consumer primitive 协调 specialized warp。这些 call 可与 `cuda::memcpy_async` 等 asynchronous copy 配合；是否避开 full-block barrier，取决于采用 unified 还是 partitioned pipeline 以及具体 synchronization design。

CUDA Pipeline API 的 producer/consumer call 包括 `pipe.producer_acquire()`、`pipe.producer_commit()`、`pipe.consumer_wait()` 和 `pipe.consumer_release()`。Partitioned pipeline 可让明确的 producer/consumer role 只参与所需 handoff；但本节清单使用 unified CTA pipeline，其 primitive 是 participating CTA threads 的 collective，不能描述为只同步特定 warp。

Block-wide barrier 要求 participating threads 到达 barrier 后才能继续，因此也会等待未参与当前 producer-consumer work 的 warp，如图 10-7所示。对于图中 unified block-scoped pipeline（`cuda::make_pipeline(cta, &pipe_state)`），producer/consumer call（`pipe.producer_acquire()`、`pipe.producer_commit()`、`pipe.consumer_wait()` 和 `pipe.consumer_release()`）是 participating CTA threads 的 collective，必须以一致顺序执行。

Pipeline API 在内部维护 per-stage state，但这个 unified 示例仍要求所有 participating thread 在每个 acquire/commit/wait/release point 有序参与 collective。Collective 之间可以由 role-specific warp 执行 loader-staged copy、compute 或 store，并在明确 handoff point 同步。若要让不同 warp 直接调用不同 pipeline primitive，应使用具有 explicit producer/consumer role 的 partitioned pipeline，而不是 unified CTA pipeline。

CUDA Pipeline API 可用于组织 three-role pattern：loader warp 为 compute warp 准备 input，compute warp 消费 input 并产生 result，storer warp 再消费 result 并写出。下方 pipeline object 是 block-scoped，并在内部跟踪 stage sequence；role 之间仍需正确的 visibility 与 ordering。

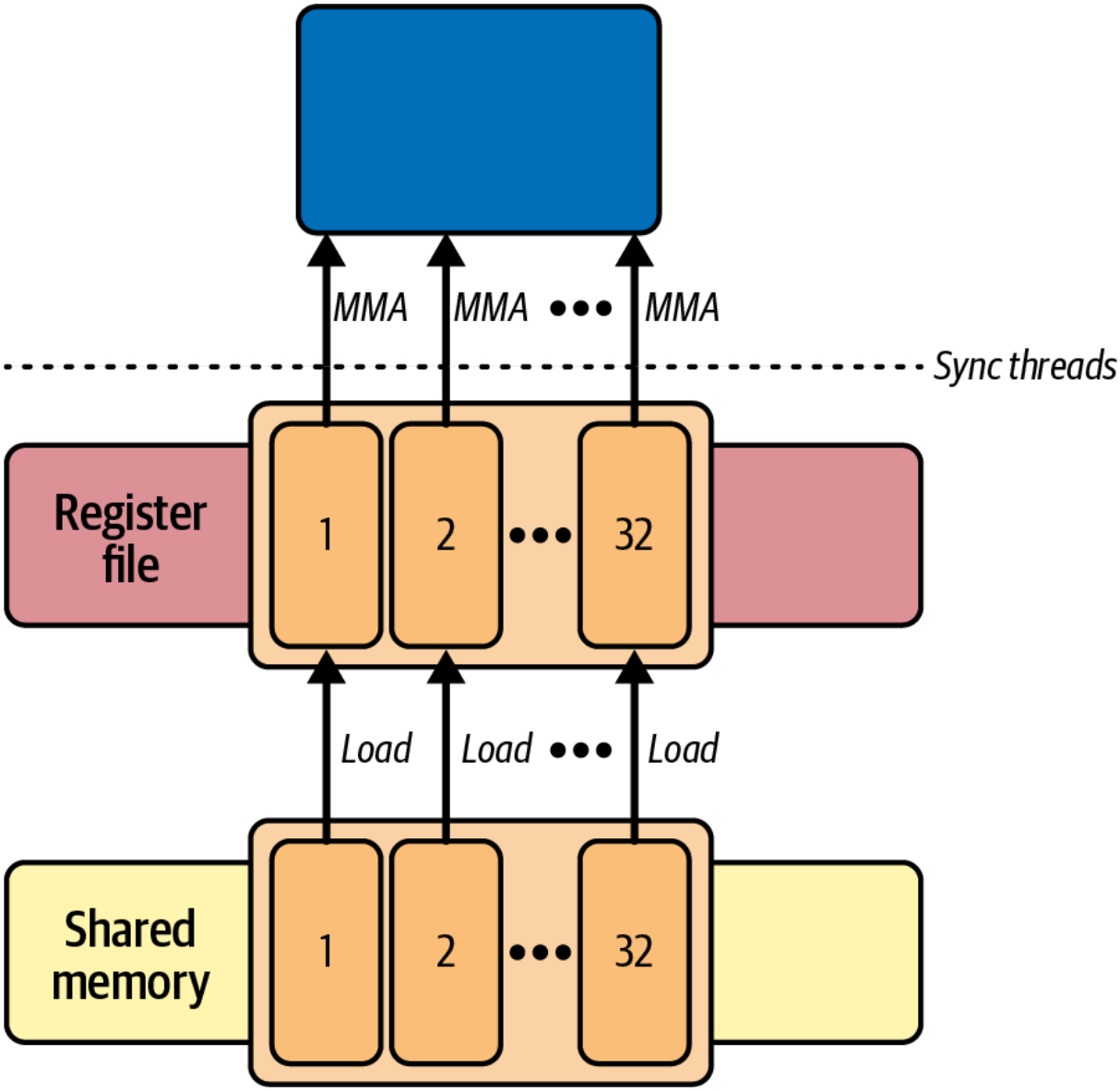


图 10-7　Block-wide barrier 要求 threads 完成同步并载入新数据后才能继续

下面的 three-role kernel 示例使用 loader warp（`warp_id 0`）、compute warp（`warp_id 1`）和 storer warp（`warp_id 2`）处理 tile。它展示 role 分支，但不构成高性能、可复现实测的完整 GEMM：

```
// warp specialized roles using the CUDA Pipeline API

#include <cuda/pipeline>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

// smem_bytes = 3×TILE_SIZE^2×sizeof(float) (three roles, single-pipeline)
// 3 tiles * 112 * 112 * 4 byte per float =
// 150,528 bytes < 227,328 bytes (~227 KB)
// per-block dynamic SMEM limit on Blackwell
// Choose TILE_* so total shared memory per block is ≤ the device's
// reported per-block shared memory limit.
// Query cudaDevAttrMaxSharedMemoryPerBlockOption and set
// cudaFuncAttributeMaxDynamicSharedMemorySize accordingly.
#define TILE_SIZE 112
```

```
// Example tile compute: C = A x B for one TILE_SIZE by TILE_SIZE tile
__device__ void compute_full_tile(const float* __restrict__ A_tile,
                                  const float* __restrict__ B_tile,
                                  float* __restrict__ C_tile,
                                  int lane_id) {
    for (int idx = lane_id; idx < TILE_SIZE * TILE_SIZE; idx += warpSize) {
        int row = idx / TILE_SIZE;
        int col = idx % TILE_SIZE;
        float acc = 0.0f;
        #pragma unroll
        for (int k = 0; k < TILE_SIZE; ++k) {
            acc += A_tile[row * TILE_SIZE + k] * B_tile[k * TILE_SIZE + col];
        }
        C_tile[idx] = acc;
    }
}

extern "C"
__global__ void warp_specialized_pipeline_kernel(
        const float* __restrict__ A_global,
        const float* __restrict__ B_global,
        float* __restrict__ C_global,
        int numTiles) {
    thread_block cta = this_thread_block();

    // three square tiles in dynamic shared memory: A, B, and C
    extern __shared__ float shared_mem[];
    float* A_tile = shared_mem;
    float* B_tile = A_tile + TILE_SIZE * TILE_SIZE;
    float* C_tile = B_tile + TILE_SIZE * TILE_SIZE;

    // three stage pipeline shared by the block
    __shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, 3>
        pipe_state;
    auto pipe = cuda::make_pipeline(cta, &pipe_state);

    int warp_id  = threadIdx.x >> 5;
    int lane_id  = threadIdx.x & 31;
    int warps_per_block = blockDim.x >> 5;

    // grid wide warp indexing for persistent tiling
    int totalWarps = gridDim.x * warps_per_block;
    int global_warp = warp_id + blockIdx.x * warps_per_block;

    for (int tile = global_warp; tile < numTiles; tile += totalWarps) {
        size_t offset = static_cast<size_t>(tile) * TILE_SIZE * TILE_SIZE;

        if (warp_id == 0) {
            // loader produces A_tile and B_tile
            pipe.producer_acquire();

            auto bytes = TILE_SIZE * TILE_SIZE * sizeof(float);
            cuda::memcpy_async(cta, A_tile, A_global + offset,
                               cuda::aligned_size_t<32>{bytes}, pipe);
            cuda::memcpy_async(cta, B_tile, B_global + offset,
                               cuda::aligned_size_t<32>{bytes}, pipe);
            pipe.producer_commit();
        }

        extern "C"
        __global__ void warp_specialized_pipeline_kernel(
                    const float* __restrict__ A_global,
            const float* __restrict__ B_global,
```

```
                float* __restrict__ C_global,
                int numTiles) {
                thread_block cta = this_thread_block();

                // three square tiles in dynamic shared memory: A, B, and C
                extern __shared__ float shared_mem[];
                float* A_tile = shared_mem;
                float* B_tile = A_tile + TILE_SIZE * TILE_SIZE;
                float* C_tile = B_tile + TILE_SIZE * TILE_SIZE;

                int warp_id = threadIdx.x >> 5;
                int lane_id = threadIdx.x & 31;
                const int bytes = TILE_SIZE * TILE_SIZE * sizeof(float);

                // three stage pipeline shared by the block
                __shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, 3>
                pipe_state;
                auto pipe = cuda::make_pipeline(cta, &pipe_state);

                // Block-strided tile ownership: one tile per block iteration.
                for (int tile = blockIdx.x; tile < numTiles; tile += gridDim.x) {
                    size_t offset = static_cast<size_t>(tile) * TILE_SIZE * TILE_SIZE;

                    // Block-scoped pipeline collectives execute
                    // before role-specific branches.
                    pipe.producer_acquire();
                    cuda::memcpy_async(cta, A_tile, A_global + offset,
                                       cuda::aligned_size_t<16>{bytes}, pipe);
                    cuda::memcpy_async(cta, B_tile, B_global + offset,
                                       cuda::aligned_size_t<16>{bytes}, pipe);
                    pipe.producer_commit();

                    pipe.consumer_wait();
                    cta.sync();

                    if (warp_id == 1) {
                        compute_full_tile(A_tile, B_tile, C_tile, lane_id);
                    }
                    cta.sync();

                    if (warp_id == 2) {
                        for (int idx = lane_id; idx < TILE_SIZE * TILE_SIZE;
                          idx += warpSize) {
                            C_global[offset + idx] = C_tile[idx];
                        }
                    }
                    cta.sync();
                    pipe.consumer_release();
                    cta.sync();
                }
    }
```

各 warp 在 role-specific branch 中分别执行 compute 或 store，而 unified block-scoped pipeline collective 在 branch 外统一执行。每轮调用 `producer_acquire()`，在 pipeline collective 下执行 cooperative staged copy（该 path 选用时例如 `cuda::memcpy_async`），再调用 `producer_commit()`。在 `consumer_wait()` 后，`cta.sync()` 强制 load -> compute -> store 的 phase order；另一次 `cta.sync()` 与 `consumer_release()` 用于 recycle stage。由于这些 CTA-wide barrier 把 role phase 分隔开，清单本身并未让三个 role 持续并发 overlap。

这个 unified 示例的重要 correctness rule 是：`producer_acquire()`、`producer_commit()`、`consumer_wait()` 和 `consumer_release()` 必须由 participating CTA threads 按一致 collective sequence 执行。不要把这些 collective 放进 divergent warp-role branch，除非有意切换到明确指定 producer/consumer role 的 partitioned pipeline API。若 warp 0 load、warp 1 compute、warp 2 store，应把 unified pipeline collective 放在 branch 外，并在 role phase 之间使用 `cta.sync()`；但这也意味着当前示例以 CTA-wide phase barrier 换取简单 correctness，而不是 fine-grained overlap。

> 原文称该 kernel 作为 persistent kernel 跨多个 tile 运行以摊销 launch overhead；是否属于严格意义上的 persistent kernel，还取决于 launch grid、resident work loop 和 global work assignment。稍后将进一步讨论 persistent kernel。

总之，CUDA Pipeline API 与 cooperative groups 可表达 producer-consumer handoff，但 synchronization 粒度取决于所选 pipeline form；“SM-wide”也不准确，本例 scope 是 CTA。表 10-3对比了三种实现：naive tiled kernel、使用 `double_buffered_pipeline` 的 two-stage double-buffered GEMM，以及 warp-specialized pipeline kernel `warp_specialized_pipeline`。数值均应视为示意，不能归因于上方清单。

表 10-3　Naive tiled kernel、two-stage double-buffered GEMM 与 warp-specialized pipeline kernel 的示意性对比

指标	朴素分块	两阶段双缓冲流水线（`double_buffered_pipeline`）	warp 专用流水线（`warp_specialized_pipeline`）
内核执行时间	41.3 ms	20.5 ms（相对于 naive 快 2.01×）	18.4 ms（相对于 two-stage speedup 10.2%）
warp 执行效率	68%	92%（相对于 naive 增加 24 percentage point）	96%（相对于 two-stage 增加 4 percentage point）
共享内存停顿延迟 / warp 同步停顿	高	低	极低
L2 吞吐量	80 GB/s	155 GB/s（相对于 naive 增加 94%）	165 GB/s（相对于 two-stage 增加 6.45%）
吞吐量可扩展性	仅扩展到每 SM 2–3 个 warp	可良好扩展到每 SM 约 6 个 warp	接近随 SM warp 数线性扩展（例如 64 个 warp）
DRAM read byte 与 SM cycle 的 overlap	较差	良好	优秀
指令数量	1.7 B	1.05 B（相对于 naive 减少 38%）	约 1.00 B（相对于 two-stage 减少 5%）

表中的示意场景把 double-buffered GEMM 写为 20.5 ms，把 warp-specialized version 写为 18.4 ms。清单在 branch divergence 之前统一发出 pipeline collective，以满足 collective ordering，并在 loader、compute 与 storer work 之间使用 explicit handoff point。前一个 two-stage double-buffered kernel 中所有 thread 都参与 consumer phase，因此 `consumer_wait()` 是 CTA collective；但“stall 整个 block”的实际时间仍由 stage readiness 与 warp scheduling 决定。

原文随后称 finer-grained per-warp synchronization 消除了 full-block wait，并让 loader、compute、storer 三个 warp 持续 overlap；这与清单中的 unified collective 和多次 cta.sync() 相矛盾。清单展示的是 CTA-wide phased handoff，不能证明 SM utilization 从 92% 升至 96%，也不能证明 warp-stall cycle 降至接近零。

表中称 Nsight Compute 显示 naive tiling kernel 在每 SM 2–3 个 active warp 后饱和，并把原因归为 tile load 必须先于 computation 完成。没有 device、shape、

launch、metric 或 report，无法验证该 saturation point；load/compute dependency 也不能单独推出 active-warp scaling。

Two-stage double-buffered kernel 可能通过 overlap load 与 compute 改善 throughput，但“扩展到每 SM 6 个 warp 后才受 shared-memory/register limit 约束”同样是配置相关结果，不是该 pattern 的固有界限。

原文称 `warp_specialized_pipeline` 只要增加 load/compute/store warp 就近似线性扩展，这忽略了 fixed role ratio、CTA granularity、scheduler/pipe throughput、barrier、register 与 shared-memory pressure。Blackwell 的 maximum resident warp 数也应按具体 compute capability/device attribute 查询，不能把 64 warps/SM 当作所有 Blackwell GPU 的统一上限；实际 residency 还受 block size 与资源限制。

表 10-3的示意数值假设 double-buffered 与 warp-specialized approach 都优于 naive tiled kernel：`double_buffer_pipeline` 把 runtime 减半，`warp_specialized_pipeline` 再增加 10.2% speedup。但当前 unified pipeline 仍有 block-wide collective 和 CTA synchronization point，不能把假设增益归因于“避免 block-wide producer/consumer participation”。

表中 instruction count 从 naive version 的 1.7 billion 降至 two-stage pipeline 的 1.05 billion（减少约 38%），再降至 warp-specialized kernel 的约 1.00 billion（相对减少约 4.76%）。这些数字没有说明 instruction metric、compiler 或 workload，不能视为实测证据。

表中 L2 load throughput 从 naive tiling 的 80 GB/s 增至 two-stage approach 的 155 GB/s（+94%），再到 warp-specialized kernel 的 165 GB/s（相对 +6.45%）。更高 throughput 不自动代表更少 byte 或更好 performance。清单由 loader warp 把 tile 写入 shared memory，consumer warp 再读取，并没有使用 TMA multicast；“把 single copy multicast 给所有 compute lane并消除全部剩余 redundant L2 read”的说法不准确。是否存在 redundant transaction 应以 requested/actual byte 与 cache metric 验证。

实践中，uniformly tiled GEMM 可先评估 simpler two-stage double-buffered pipeline；warp-specialized approach 则适合具有可分离 role、足够工作量与 deeper pipeline 的场景，例如某些 fused-attention kernel。两者都不保证隐藏大部分 DRAM latency，也不能保证 specialized warp 永不迫使其他 warp 等待。应根据 architecture、shape、resource、correctness 与 measured runtime 选择。

PyTorch、CUDA Pipeline API 与 Warp Specialization

PyTorch public API 不直接暴露 `cuda::pipeline`。调用 `torch.compile` 后，TorchDynamo/TorchInductor 可根据 graph 与 target 生成或选择 kernel；某些 GPU path 使用 Triton，也可能调用 cuBLAS、cuDNN、ATen/external kernel 或其他 backend。生成代码可能包含 asynchronous copy、barrier 或 warp-specialized schedule，但不能统一视为实现了与 `<cuda/pipeline>` primitive 等价的 producer/consumer。

因此，即使 generated source 中看不到显式 `cuda::pipeline` call，也可能在最终 PTX/SASS 或 backend kernel 中观察到相关 instruction 与 barrier；是否出现必须实际

检查 generated artifact。这些优化可能改善 data-transfer/computation overlap，但 occupancy 不一定上升，也不保证获得与 hand-written `<cuda/pipeline>` implementation 相同的 latency 或 throughput。

PyTorch 可为 fused attention选择多个 backend；其中某些 kernel 使用 warp specialization，另一些则使用不同 pipeline。先看下面由 matmul、softmax 和另一项 matmul 写成的 PyTorch 表达式。原文称它必然对应三个独立 GPU kernel，但 eager/compiled execution 的实际 kernel count 取决于 dispatch、fusion 与 graph：

```
scores = torch.matmul(queries, keys.transpose(-2, -1))
probabilities = F.softmax(scores, dim=-1)
context = torch.matmul(probabilities, values)
```

可以改用 PyTorch 的 scaled-dot-product-attention API：

```
context = torch.nn.functional.scaled_dot_product_attention(queries,
   keys, values)
```

该 API 是 backend dispatcher，而不是固定的三类 warp implementation。根据 device、dtype、shape、mask、dropout、causal setting 和 PyTorch/CUDA version，它可能选择 FlashAttention、memory-efficient attention、cuDNN attention 或 math fallback。Fused backend 可避免显式 materialize 某些 intermediate，从而减少 `matmul` 与 `softmax` stage 之间的 global-memory round trip；fallback 或不支持的配置则不一定如此。

`torch.compile` 值得用于可捕获、具有可优化 graph 的 workload，但并非所有 performance-sensitive 或 irregular workload 都“ideal”；graph break、dynamic shape、compile time、unsupported operation 和 backend choice 都会影响收益。Blackwell 等 modern GPU 提供较多资源与高 bandwidth，warp specialization 可能减少 idle cycle，但不会保证 across-warp near-linear scaling；瓶颈还可能来自 dependency、barrier、register/shared memory、instruction throughput 与 latency。

> FlashAttention-3 报告中的约 75% peak FP16 FLOP/s 对应特定 Hopper、precision、shape 与 FLOP-count convention，并不是 fused attention 的通用 ceiling。更合理的结论是：优化目标应为 workload latency/throughput 与 efficiency，而不是强求 100% compute utilization；但是否达到“显著里程碑”仍需相对于明确 baseline 评价。

把 load、compute 与 store 组织成不同 stage，可能提高 transformer attention、fused-operator pipeline 和 custom task scheduler 等 long-running kernel 的 overlap 与 resource utilization；但 pipeline 并不自动 maximize throughput，也不是所有场景的首选方案。Fine-grained inter-warp communication 与 deep pipeline 只有在 role rate 匹配、synchronization 正确且 overhead 可摊销时才有益，linear warp scaling 不是保证。

Persistent Kernel 与 Megakernel

Persistent kernel（也称 *persistent thread*）改变了常见的一次 launch 处理一批固定 work 的方式。与连续 launch 许多 small kernel 相比，可以启动一个 long-running kernel，由其 thread 持续从 work queue 领取任务。跨 thread block 共享的 queue 通常位于 global memory；shared-memory queue 只能由相应 CTA 或 cluster 在其可见范围内使用，不能笼统称为全 grid shared queue。

Persistent thread 可在 kernel 内循环处理不断到达的 data chunk，而不在每批 work 后退出，从而摊销部分 repeated launch overhead；它不会消除首次 launch、queue management 或 synchronization cost。Host signal 与 CPU-visible memory 还需要合法的 mapped/coherent memory、atomic scope/order 和 progress protocol。Persistent kernel 可以让 CTA 中的 Warp 0 协调 global-to-shared copy，同时 Warp 1 处理上一批 data，形成 GPU software pipeline；普通 device code 不能未经支持的 memory path 就把任意 CPU host memory 直接复制到 shared memory。

例如，假设有 1,000 个很小且彼此 independent 的 task。一种朴素实现可能为每个 task 单独 launch kernel；若每次 grid 都很小，kernel 可能只在少量 SM 上短暂运行。

这些 sequential tiny launch 可能产生显著 launch gap，并因 grid parallelism 不足而让部分 SM idle。不过 GPU 并不是每次 kernel 都物理“ramp up/ramp down”；多个 stream、CUDA Graph、batching 或合并 grid 也可能减少 launch overhead，不只有 persistent kernel 一种方案。

Persistent-kernel 方案可以启动一个 worker grid，在整个 workload 期间循环领取 work。以 132 个 SM 的 GPU 为例，可以尝试 launch 132 个 block、每 block 256 个 thread，共 33,792 个 thread。但“block 数等于 SM 数”不保证每个 SM 恰好固定驻留一个 block；实际 placement、concurrent residency 与 forward progress 由 scheduler 和 resource usage 决定。下面是简化伪代码：

```
__device__ int g_index;  // global counter for next task; initialize to 0 on host

__global__ void persistentKernel(Task* tasks, int totalTasks) {
    // Every thread loops, atomically grabbing next task index until none remain
    while (true) {
        int idx = atomicAdd(&g_index, 1);
        if (idx >= totalTasks) break;
        processTask(tasks[idx]);
    }
}
```

Launch 前，host 需要把 device memory 中的 `g_index = 0`。不过上方把它声明为 device symbol，下面直接对 host 侧的 &g_index 调用 cudaMemset 的写法通常不成立；应使用 cudaMemsetToSymbol，或把显式分配的 device pointer 作为 kernel argument 传入。清单按原书保留：

```
cudaMemset(&g_index, 0, sizeof(int));

// one block per SM on a 132 SM GPU
int blocks         = 132;
int threadsPerBlock = 256;
```

```
    persistentKernel<<<blocks, threadsPerBlock>>>(d_tasks,
        totalTasks);

    cudaDeviceSynchronize();
```

使用 `persistentKernel` 后，不必为每个 task 单独支付 launch overhead，但仍需一次 launch 以及 queue/atomic/synchronization cost。原文假设每次 launch 约 0.02 ms、每个 task 运行 0.1 ms，并把 1,000 个 task 的 work 相加为约 100 ms；这只是串行 work-time sum，不能直接作为 GPU wall-clock time，因为 independent task 可能并行运行。

在同一示意假设下，1,000 次 launch × 0.02 ms 等于 20 ms 的 host/launch overhead sum。但 launch 可被 asynchronous submission、driver batching、CUDA Graph 与 GPU execution overlap，最终 wall-clock 增量不一定正好为 20 ms。把 small task 合入 persistent loop 可能显著减少 launch overhead，具体收益必须与 batching、CUDA Graph 和普通 batched kernel 比较。

Persistent kernel 可以在 work supply 充足且 load balance 良好时提高 utilization，但“数万 thread 处理约 1,000 个 task”本身不能证明 nearly all SM 同时忙碌：大量 thread 可能只执行一次 contended atomic，然后立即发现 queue 为空。Sequential tiny kernel 也不必固定只使用约 35% GPU capacity；原文没有给出该数字的 device、task shape、metric 或测量依据。

原文进一步假设每个 SM 恰好运行一个含 256 thread 的 block，并以每 SM 最多 2,048 thread 计算 occupancy 为 12.5%（原文写 12%）。Maximum threads/SM 依 compute capability，occupancy 也受 warp、block、register、shared memory 和 architecture limit 共同约束；即使 launch block 数等于 SM 数，也不能据此保证 nearly 100% SM active。

这种 persistent-kernel design 可能维持较高 measured SM Active percentage，但还取决于 work availability、atomic contention、load balance、register/shared-memory usage 与 memory/compute bottleneck。应使用 Nsight Systems 检查 launch/timeline 和端到端 gap，再用 Nsight Compute 分析 kernel metric；SM Active 高也不等于 useful throughput 高。

Modern GPU 的 shared-memory capacity、register file 与 asynchronous data-movement feature 可支持更深的 persistent pipeline，但更大的 per-CTA state 也可能降低 residency。支持 TMA 的 architecture/kernel 可由某些 thread 发起 global-to-shared tensor transfer，同时其他 warp compute；TMA 可减少用于地址生成和逐元素 copy 的 instruction overhead，但仍占用 memory path、barrier 与 issue/control resource，不能称为完全不增加 SM pipeline 负担。

Persistent Kernel 的常见 Workload

当 workload 包含大量 small task 或 size 不均的 task，若逐项处理会产生较高 launch overhead，persistent kernel 可以作为候选方案。Device-side queue 支持 dynamic load balancing，使较早完成的 worker 能继续领取 work；但 queue starvation、tail imbalance、atomic contention、memory latency 或过少的 ready task 仍会使部分 SM idle，不能保证“没有 SM 提前空闲”。

这种 pattern 可用于 graph traversal、custom batched transformation，以及部分 LLM inference per-token operation 等 irregular workload，因为各 task 的执行时间可能差异较大。不过 graph frontier、batch size、token scheduling 和 operator granularity 会决定 persistent queue 是否优于 batched kernel、CUDA Graph 或 framework scheduler。

Persistent kernel 也有代价。开发者通常需要显式管理 task queue、termination、memory ordering 和 synchronization；若许多 thread 同时更新同一 counter，atomic contention 会很严重。常见缓解方式包括按 warp/CTA 成批领取 work、hierarchical queue、work stealing 或减少 polling frequency，但这些设计又增加 correctness complexity。

调试一个 large persistent loop 往往比调试多个 small kernel 更复杂。Divergence 本身通常只导致 path serialization，不会自动使整个 kernel hang；真正的 hang/deadlock 更常来自部分 thread 未到达 collective barrier、非法 grid-wide synchronization、queue termination protocol、memory-order bug 或 circular resource dependency。Long-running kernel 还可能占用大量 GPU resource，影响 concurrent workload、latency/QoS 和 watchdog；需要通过 grid sizing、stream priority、MIG/resource partition 或 scheduler design 控制。

总之，persistent kernel 把 GPU 组织成持续领取 task 的 dynamic worker pool，相对于高开销的 naive baseline 可能显著提高 throughput；原文给出的 2–3× 没有统一 benchmark 条件，不能作为预期值。它可以摊销 repeated launch overhead，但不会消除首次 launch 与 queue overhead，也不保证 maximize occupancy。结合 cooperative groups 或 thread block cluster（稍后介绍）时，某些 intermediate 可以在支持的 scope 和 lifetime 内保留 on-chip，但 multistage pipeline 仍可能需要 global-memory spill、communication 或 synchronization。

一些 framework 与 library 使用 persistent kernel 或 megakernel 改善 inference 等 latency-sensitive workload。关键是减少不必要的 repeated launch、选择合适的 device-side scheduling granularity，并让 worker 有足够 useful work；追求 SM fully occupied 不是目标本身，过度 occupancy、polling 或 queue traffic 反而可能降低性能。

> PyTorch/TorchInductor 不会普遍把任意完整 multistage workload 自动 fusion 成一个 kernel，但某些 operator/backend 已使用 persistent 或 fused implementation。获得更广泛的 persistent/megakernel benefit 往往需要 custom CUDA、Triton 或 specialized compiler/runtime。对 multiphase algorithm 进行这种重构可能带来收益，也可能因 register/SMEM pressure、code size、barrier、load imbalance、numerical behavior 或 reduced concurrency 而退化；必须处理 synchronization/deadlock，并与 library kernel、CUDA Graph 和较小 fusion 方案比较。

用于 Inference 的 Megakernel

源自 large-scale inference 的一种 modern persistent-kernel approach称为 *megakernel*。它把较长的 operation sequence，甚至多个 layer 的 scheduling 与 execution，合入一个 large/persistent kernel。原文进一步称其“跨 GPU fusion 成单一 kernel”，这不应按字面理解：普通 CUDA kernel 在单个 device context 上执行；

multi-GPU megakernel system 通常是在每个 GPU 上运行 cooperating persistent kernel，并通过通信与 distributed scheduling 协同。如图 10-8所示，所引 benchmark 在 Llama-1B、batch size 1 和若干特定 GPU 上报告 megakernel 相对 vLLM/SGLang 的 decode throughput 提升。原文把图中的约 1.2×–6.7× throughput ratio 写成“latency 降低 1.2×–6.7×”，指标不一致；该范围也不能归因于仅仅消除 repeated launch，收益还可能来自 scheduling、fusion、memory traffic 与 baseline 差异。

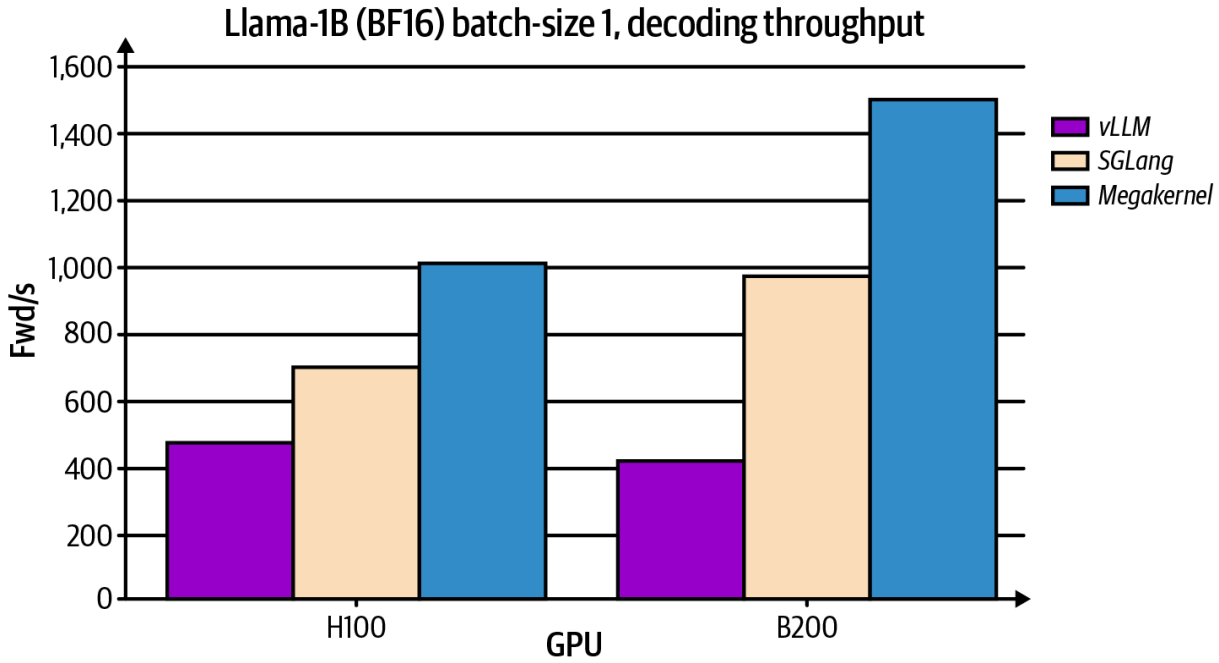


图 10-8　Megakernel 相对于 vLLM 与 SGLang 的 decode throughput 改善（来源：https://oreil.ly/2aZiF）

Persistent Kernel 与 Warp Specialization

Warp specialization 常与 persistent kernel 组合，因为 long-running worker 会执行许多 iteration，使 deeper pipeline 的启动/排空和 role-coordination overhead 更容易摊销，并可能改善 overlap 与 long-lived state reuse。但二者互不依赖：warp-specialized kernel 不必 persistent，persistent kernel 也不必采用 warp specialization。对于 short-running kernel，额外 code、barrier 与 resource complexity 可能得不偿失。

Persistent-kernel scheduling 的一个限制是，它通常希望保留足够 resident worker 来覆盖目标 GPU。若其他 kernel 已占用大量 SM resource，普通 launch 的 block 可能排队，或只能获得较低 concurrent residency；不应笼统说整个 kernel“没有足够 SM 就无法 launch”。Cooperative launch、thread block cluster 或依赖 all-worker concurrency 的自定义 synchronization 则有更严格的 occupancy/residency requirement，若配置超过设备能力可能 launch failure 或 deadlock。因此必须结合 concurrent workload、stream priority、resource footprint 与 launch API 做 scheduling/load-balancing。

Modern GPU 还支持 *thread block cluster*，也称 *cooperative thread array（CTA）cluster*，因为 thread block 也称 cooperative thread array。Cluster 可与 persistent kernel 或 warp specialization 组合，但并不是二者的前提。稍后将详细讨论 thread block（CTA）cluster；简言之，它把多个 block 组织成具有 cluster-level co-

scheduling、synchronization 与 DSMEM capability 的 group。原文称这些 block 会占据多个“邻近 SM”，这不是通用 placement guarantee：block 可能位于一个或多个 SM，具体 mapping 不由程序固定控制。

协作组

Cooperative Groups API 可用于定义和同步具有明确成员关系的 thread group。支持的 group 可对应 thread block、partitioned tile、grid 或 thread block cluster 等 scope，如图 10-9所示；并非任意 thread 集合或任意粒度都能自由组成可同步 group，具体能力取决于 group type、launch mode 和 architecture。

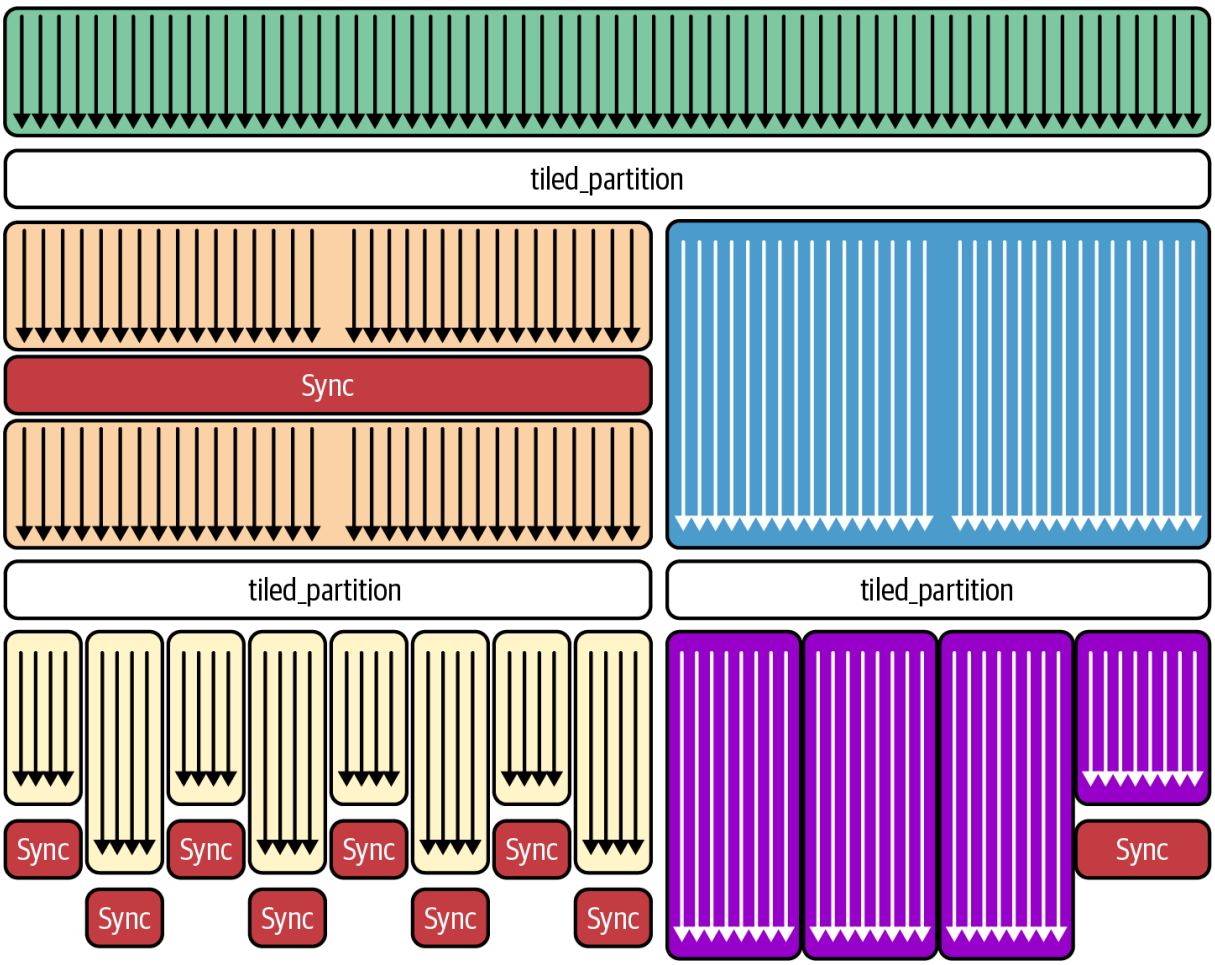


图 10-9　使用 cooperative groups 在不同 thread granularity 上同步

Cooperative groups 为相应 group type 提供 sync、broadcast、reduce 等可复用 collective，减少 ad hoc barrier 的使用。传统 `__syncthreads()` 只同步同一 thread block；普通 kernel launch 中没有可供任意 grid 直接使用的全 grid barrier。Cooperative grid launch 则可提供受容量与参与规则约束的 grid-level synchronization。

Cooperative groups 支持 kernel 内不同 scope 的 synchronization，可用于协调某些 multistage pipeline；是否适合 warp、block、grid 或 cluster 间协作，取决于数据可见范围、collective contract 与 launch capability。使用时需包含

`<cooperative_groups.h>`，取得合法 group object，并让规定的成员一致参与相应 collective。

API 包括 `cg::this_thread_block()`、`cg::tiled_partition()` 和 `cg::this_cluster()` 等 call。随后可调用 `group.sync()` 或其他 supported collective。这里的 `cg::this_cluster()` 需要支持 thread block cluster 的 architecture 与 cluster launch；不同 group 的 sync 语义和 memory visibility 也不能互换。

要以 cooperative mode launch kernel，可使用 `cudaLaunchCooperativeKernel()`。设备必须支持 cooperative launch，而且 grid 必须满足同时驻留所需的 capacity constraint，否则 launch 返回错误。可用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 结合实际 block size、dynamic shared memory 和 device property 估算上限，并检查所有 API error。Kernel 内可用下面的 call 实现 all-thread-block barrier：

```
cooperative_groups::this_grid().sync(); // or grid.sync()
```

在有效 cooperative grid 中，参与 grid group 的所有 thread 到达该点后，任何 thread 才能越过 barrier。这样可以在不结束 launch、也不把控制权交回 host 的情况下，把 kernel 分成 sequential phase；所有成员必须以一致控制流参与。

以 LLM 中常见的 softmax 为例，它通常包含 reduction（实际稳定实现还先求 maximum，再求 exponential sum）以及使用 reduction result 的 per-element normalization。传统 multi-kernel 实现可由第一个 kernel 把 partial/final result 写到 device global memory，后续 kernel 再读取；通常不必把结果复制回 host。额外 kernel boundary 与 global-memory intermediate 会产生开销，但“需要大量 host memory movement”的说法不准确。

使用 cooperative grid 时，可在一个 kernel 中让各 block 计算 partial result，经 barrier 后由指定 block 聚合，再次调用 `grid.sync()` 后让全部 thread 进入下一 stage。不过跨 block 的 aggregate 必须存放在所有 block 都可见的 memory（通常是 global memory，或在适当 architecture/scope 下采用其他明确机制）。某个 block 的 register 或普通 shared memory 不能被整个 grid 直接读取，因此原文声称全部 thread 从 register/shared memory 读取 aggregate 是错误的。

Cooperative launch 要求整个 grid 满足 concurrent-residency capacity；若请求的 block 多于该 kernel 配置可同时支持的数量，launch 应返回错误。仍需检查 device support、launch return status 和后续 asynchronous error。

Cooperative kernel 的 grid size必须可同时驻留，因此合法 launch 不会因尚未调度的“nonexistent block”而永久等待 `grid.sync()`。但 runtime 只保证 launch/residency 条件，不保证每个 block/thread 在程序逻辑上一定到达 `grid.sync()`；divergence、early return、fault 或无限循环仍会 deadlock。必须检查 `cudaLaunchCooperativeKernel()` 的 return status。

例如，若 GPU 有 132 个 SM，且给定 kernel/block/dynamic-SMEM 配置下每 SM 最多 concurrent resident 4 个 block，则单设备 cooperative grid 上限可估为 528 个 block；超过容量应导致 launch failure。应使用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 对实际配置求值，并检查

`cudaLaunchCooperativeKernel` return status。该算术不代表所有 528 block 在整个执行期间固定映射或同步前都有相同进度。

在不使用 cluster DSMEM 的跨 block 协作中，可以把 `cudaLaunchCooperativeKernel` 与 global-memory atomic、flag 或 grid barrier 配合，代价可能包括 global-memory traffic。并非所有 intermediate 都必然搬到 HBM：cache、register/shared-local state 和 algorithm design 会影响实际 traffic。

Thread block cluster 与 DSMEM 从 Hopper（compute capability 9.0）开始提供，Blackwell 继续支持并扩展相关能力，并非 Blackwell 才出现。Cluster 内 block 可访问彼此的 distributed shared memory 并执行 cluster synchronization，从而在 cluster scope 内避免部分 global-memory round trip；它不是全 grid coordination 的等价替代，性能也需实测。后文将详细介绍。

当单次 kernel launch 确实需要 all-thread-block barrier 时，可以评估 cooperative kernel。它能避免某些 repeated launch，但不会让 register 或普通 shared memory 跨 block 可见，也不自动把所有 intermediate 保留在 fast memory。Grid 必须限制在 concurrent capacity 内；简单增大 block 并不总能减少风险，因为更大的 block 可能提高 register/SMEM pressure、降低 blocks/SM 或超过其他 limit。

Cooperative kernel 的主要限制之一是 grid size 受 concurrent-residency capacity 约束，可能降低可用 parallelism 或要求 grid-stride loop。Thread block cluster 解决的是 cluster-scope cooperation，不会消除 cooperative grid 的全局容量约束，也不是简单的替代方案。

Cooperative grid barrier 要求 group 中每个 thread 都调用 `grid.sync()`。若任何 thread 跳过或永远无法到达 `grid.sync()`（例如 divergent `if` 的一条 path 未参与、early return 或 fault），其他参与者可能永久等待并形成 deadlock。因此 barrier 必须位于全体成员一致可达的控制流中。

概括来说，cooperative grid 让单个 device 上的整个 launched grid 参与 `grid.sync()` global barrier，适合确需 multi-stage global synchronization 的 algorithm；“整个 GPU”不应理解为跨 device。Grid barrier 通常比 block-或 cluster-scoped barrier 协调范围更大、成本更高，但具体 overhead 依 grid、architecture 和 workload。应减少不必要的调用，并保证 barrier 间有足够 useful work，同时与 multi-kernel/CUDA Graph 方案实测比较。

> 若只需有限 cross-block coordination，global-memory atomic 或 per-block flag 可能比 full-grid barrier 更简单，但 correctness 仍需明确 atomic scope/order 与 progress。它们不一定比 thread block cluster/DSMEM 更慢：cluster 只适用于 cluster scope，并受 architecture、placement、resource 和 data-sharing pattern 约束，应按目标 workload 选择。

Cooperative Grid Synchronization 与 Persistent Kernel

如果 workload 在 persistent loop 内偶尔需要 cross-thread-block barrier 来聚合 partial result，可以把 persistent kernel 与 cooperative groups 组合起来，调用

`grid.sync()`。它提供 grid-level barrier，使 kernel 可在 device 上推进多 stage reduction 或其他 global step，而不必结束当前 launch 再重新 launch。

例如，某 workload 在 1,000 个 iteration 中反复执行两种 computation，且每次都因 cross-block data dependency 需要 global barrier。朴素实现可能每轮 launch 两个 kernel，共 2,000 次 launch；即使在同一 stream 中第二个 kernel 会按 stream order 等待第一个，也仍需承担 launch/dispatch overhead。可以尝试把两阶段合并为 cooperative persistent kernel，如下所示，但前提是整个 grid 能同时 resident 且所有 thread 一致到达每个 barrier：

```
#include <cuda_runtime.h>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

__device__ inline float someComputationA(float x) { return x * 2.0f; }
__device__ inline float someComputationB(float a, float b) { return a + b; }

__global__ void combinedKernel(float* __restrict__ dataA,
                               float* __restrict__ dataB,
                               int N,
                               int iterations) {
    cg::grid_group grid = cg::this_grid();                 // whole-grid group
    const int tid = blockIdx.x * blockDim.x + threadIdx.x; // thread's linear id
    const int stride = gridDim.x * blockDim.x;             // total grid threads

    for (int it = 0; it < iterations; ++it) {
        // Stage 1: update all elements of A using a grid-stride loop
        for (int i = tid; i < N; i += stride) {
            dataA[i] = someComputationA(dataA[i]);
        }

        grid.sync();  // global barrier; also provides memory visibility

        // Stage 2: use finished A to update B (again grid-stride)
        for (int i = tid; i < N; i += stride) {
            const float mid = dataA[i];
            dataB[i] = someComputationB(mid, dataB[i]);
        }

        grid.sync();  // barrier before next iteration
    }
}
```

在主机端，只需按如下所示调用这个结合协作与持久化特性的内核：

```
// ... allocate/copy dA, dB, set N, iterations, choose blockSize
cudaDeviceProp prop{};
cudaGetDeviceProperties(&prop, 0);
if (!prop.cooperativeLaunch) {
    // Fallback: run two kernels per iteration instead of grid.sync()
    // (see alternative below)
}

// Compute the maximum grid size allowed for a cooperative kernel
int maxBlocksPerSm = 0;
const int blockSize = 256;
cudaOccupancyMaxActiveBlocksPerMultiprocessor(&maxBlocksPerSm,
                                              combinedKernel,
                                              blockSize, 0);
```

```
const int maxCoopGrid = maxBlocksPerSm * prop.multiProcessorCount;

// Choose grid size, but clamp to cooperative limit
int gridSize = (N + blockSize - 1) / blockSize;
if (gridSize > maxCoopGrid) gridSize = maxCoopGrid;

// Launch cooperatively
void* args[] = { &dA, &dB, &N, &iterations };
cudaLaunchCooperativeKernel((void*)combinedKernel, gridSize, blockSize, args);
cudaDeviceSynchronize();
```

在理想结构比较中，该 single kernel 取代原本 2 × 1,000 = 2,000 次 kernel launch，并避免每轮重复 dispatch。循环内的 `grid.sync()` 保证 phase ordering：所有 block 完成 Stage 1 后，才能进入 Stage 2 以及下一 iteration；但它不自动消除 computation、queue、memory traffic 或 synchronization cost。

每个 thread 的 register state 和每个 CTA 的 shared-memory state 可以跨 `grid.sync()` 保持；跨 block data exchange 仍需使用 global memory，或使用适用的 thread block cluster/DSMEM。GPU 内循环只意味着首次 launch 后不再重复 launch，不代表没有 barrier、polling 或 global-memory traffic。

Cooperative kernel 要求 device 支持 `cooperativeLaunch`，并使用 `cudaLaunchCooperativeKernel` launch；grid 中所有 block 必须满足 concurrent residency。应使用 CUDA Occupancy API 按实际 kernel resource size grid，并处理 launch error。若 `((N + threads - 1) ÷ threads)` 超过 cooperative capacity，不能简单截断后假设所有数据都被处理；需要 grid-stride loop、分批 launch 或 fallback kernel。

> 使用 CUDA Occupancy API 为 cooperative grid sizing；同时检查 launch return status、dynamic shared memory 和实际 device property。

不能用“modern GPU 通常可同时运行几千个 block”作为 capacity 假设。Concurrent block 数由 SM 数、block size、register、shared memory、warp limit、cooperative launch mode 与 kernel attributes 共同决定，可能远低于或不同于该数量级。必须在目标 device 上查询并验证 block size 与 resource consumption。

该示例在 iteration 之间不把控制权交回 host，因此外层 loop 具有 persistent-kernel 的形态；内层 stage 通过 grid-wide synchronization point 排序。它是否能长期安全运行，仍取决于 cooperative residency、progress、barrier convergence 和 work duration。

这种 combined pattern 可用于 LLM inference 中包含 reduction/normalization（Stage 1）和 per-element transformation（Stage 2）的 multistep algorithm，但不一定适合每一 layer。Cooperative persistent kernel 可以减少 inter-kernel launch boundary；它不会消除跨 block global-memory exchange，也不保证 maximize on-chip locality。应比较 fused kernel、CUDA Graph、分阶段 kernel 和 framework backend 的端到端结果。

何时组合 Persistent Kernel 与 Cooperative Groups

一种常见做法是在资源允许时尝试让 persistent kernel 使用接近 one-block-per-SM 的 worker grid。这样可以在每个 SM 上提供 resident worker，并从 work queue 迭代领取 task；但 block-to-SM mapping、occupancy 和可用 residency 必须由目标 device/resource 配置验证，不能把它称为最大化 occupancy 或保证 SM 不 idle。若各 phase 需要 global synchronization，worker 可使用 `grid.sync()`；决定是否组合 cooperative 与 persistent strategy 时，可询问以下问题：

是否存在需要 global synchronization 的多个 sequential phase?

> 如果是，可以评估 cooperative kernel（或 cooperative persistent kernel），在 phase 之间调用 `grid.sync()`；前提是 grid capacity 与一致参与条件满足。

是否有许多因 launch overhead 而受损的 small/irregular task?

> 如果是，可以评估 persistent kernel，让 worker 在 device-side queue 上循环而不为每个 task 返回 host；还需比较 batching、CUDA Graph 与普通 fused kernel。

是否能为该 workload 独占整个 cooperative grid（或一组 thread block cluster）的资源?

> 如果是，cooperative persistent kernel 可能值得 benchmark；“best performance”仍需与其他实现实测，且 cluster 不等于整个 GPU 资源隔离。

是否需要与其他 workload 共用 SM 或保持 QoS?

> 如果是，不要仅依赖 thread block cluster 实现资源隔离。应评估较小 grid、stream/MPS/MIG 或其他 partition/scheduling 机制；cluster 主要定义协作与数据可见 scope。

概括来说，单一 kernel 同时执行 persistent task loop，并在 phase 之间调用 `grid.sync()`，可以减少 separate kernel launch boundary 和一部分 intermediate transfer。但 barrier、global-memory exchange、queue traffic 与 launch capacity 仍存在。

这不意味着 data 会在整个 computation 中始终留在 shared memory 或 register；跨 CTA data 通常仍需 global memory，资源压力也可能迫使 spill。GPU 是否持续做 useful work、是否接近 peak hardware limit，必须由端到端 benchmark 与 profiler 证明。

一个重要 caveat 是：cooperative kernel 的 concurrent-residency 要求可能限制其他 kernel 在同一设备上的共置。若还需调度 asynchronous prefetch 或低优先级 inference kernel，应评估 grid sizing、stream priority、MPS/MIG 或其他资源 partition。Thread block cluster（下一节）提供的是 cluster-scope cooperation，不是通用 GPU partition。

Thread Block Cluster 与 Distributed Shared Memory

Cooperative group 是 software-level abstraction，它提供 API，将 kernel threads 组织成用于 synchronization 和 data movement 的 collectives。Group 可以是 warp、tile、thread block，或在支持的 API/launch scope 下的 grid；“任意”不代表任意线程集合都自动具备同样的 memory visibility 或 synchronization guarantee。

相比之下，thread block cluster（也称 cooperative thread array [CTA] cluster）是 hardware-level hierarchy。它为一组 CTA 提供 cluster-level co-scheduling、synchronization 和 DSMEM capability；这不等于程序可任意“保留一部分 SM 并把剩余 SM 留给其他 kernel”，实际 residency 与共置取决于 cluster size、grid、资源和 scheduler。在支持的 cluster launch 中，GPU 会把 cluster blocks 安排在同一 GPU processing cluster（GPC）或规定的 GPC partition 范围内，如图 10-10所示。

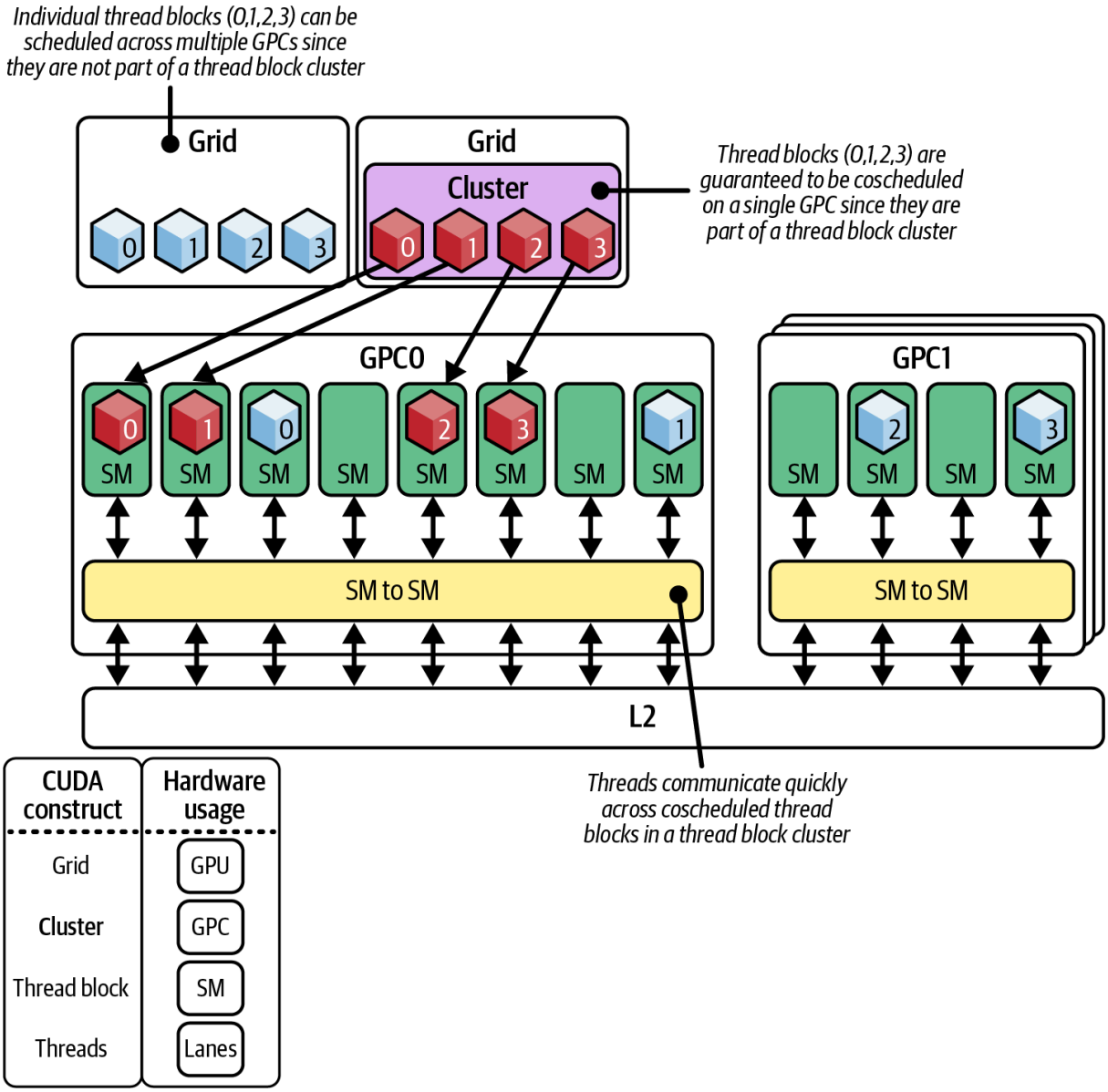


图 10-10　多个 thread block cluster 保证在同一 GPC 或 GPC partition 范围内协同调度

GPC（GPU processing cluster）是由若干 SM 及相关互连/固定功能单元组成的硬件分组。GPU scheduler 会依据 kernel、cluster 和 resource constraint 把 blocks 安排到可用 GPC/SM；这不是像“一个 block 内线程固定在同一 SM”那样的简单一一映射。

> 在 NVIDIA Blackwell B200/300、GB200/300 等 multidie module 上，可能存在多个 GPC partition；具体 partition 数依 SKU、封装和架构实现。多 die GPU 可能通过 die-to-die interconnect 对 CUDA 呈现为单一 device，但 cache coherence、GPC partition 和 cluster placement 不能仅凭“几块 die”推断，应以目标产品文档和 device attribute 为准。

GPU 为 thread block cluster 提供 distributed shared memory（DSMEM，下一节详述），让 cluster 内 blocks 在规定 scope 中访问 distributed shared-memory allocation；同时可通过 Cluster Group API 的 `cluster.sync()` 执行 cluster-level barrier。

Cluster-level barrier 只协调 cluster 成员，不要求整个 grid 参加；但 cluster blocks 仍可能占用资源并影响其他 kernel。Thread block cluster 可将 cooperative launch 的 grid 划分成较小 block group，如图 10-11所示。

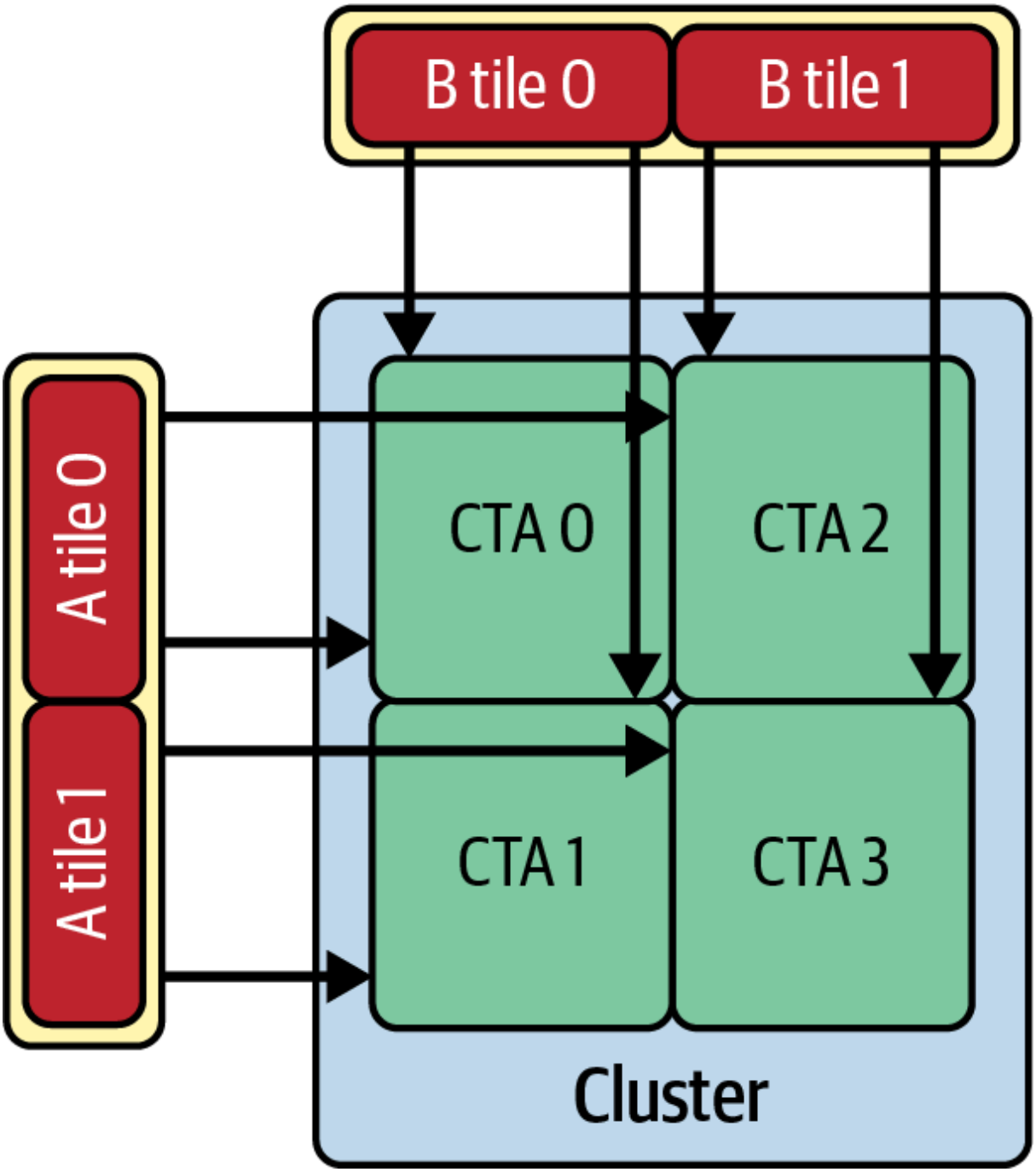


图 10-11　对于这四个（2 × 2）thread cluster，A 与 B 的每个 tile 同时载入两个 thread block（来源：https://oreil.ly/kEZsv）

在每个 cluster group 内调用 `cluster.sync()` 可提供 local barrier。它让 cluster blocks 在明确的 cluster scope 内通过 on-chip resource 共享 data，但不保证不会占用或影响其他 SM。支持 DSMEM 的 GPU 可让 cluster 访问逻辑上由各 block local shared-memory allocation 组成的 distributed on-chip SRAM；实际 latency、capacity 与 access path 依 architecture 和 address pattern。

Thread block cluster 将一部分 block 组成 group，在 cluster 内调用 `cluster.sync()` 执行局部同步。传统上 shared memory 只允许同一 block 内 threads 协作；在支持 cluster/DSMEM 的 modern GPU 上，cluster 内不同 block 可按规定 address/synchronization protocol 协作。

换言之，没有 thread block cluster/DSMEM 时，同一 block 之外的 block 通常需通过 global memory、atomic/flag 或 full-grid mechanism（例如 `grid.sync()`）协调；full-grid barrier 的范围更大，可能增加 stall 与 capacity constraint，但不是所有实现都必须使用它。

不同 block 之间若要共享 state，还可使用经过设计的 global atomics、flags、CUDA Graph/多 kernel protocol 或其他 communication path；`grid.sync()` 只是 coarse-grained grid-wide synchronization 选项。Global-memory traffic 与 full-grid barrier 可能成为 bottleneck，但其成本依 access、reuse、contention、grid size 和 workload，不应一概称为“较慢”。

支持 thread block cluster 的 GPU 提供相应 hardware scheduling/launch mechanism；其中某些架构还提供 *cluster launch control*，用于在 persistent work queue 场景中调度 cluster。它可能帮助 worker 在资源受限或部分 SM 被占用时维持 load balance，但不是所有 GPU、CUDA version 或 launch path 都提供同一机制，也不保证自动实现 balanced work 或高效 warp specialization。

Thread block cluster 可使用 hardware-supported communication/synchronization、相应 PTX instruction 和 CUDA intrinsic 执行 cluster-wide barrier。Cluster barrier 的参与范围通常小于 full-grid `grid.sync()`，可能降低 coordination cost；但“much faster”不是通用保证，实际 latency 依 cluster size、memory traffic、architecture、contention 与 workload 测量。

线程块重排

在直接按网格启动时，thread block 会严格按行主序或列主序处理 tile。这可能导致较早执行的 block 驱逐后续 block 仍需要的数据，从而造成复用率低下并增加内存流量。更理想的做法是让同一 wave 中需要的 A、B tile 能够留在 L2 cache 中读取。

为避免这种低效，可以使用 thread block swizzling。类似于用 swizzling 优化内存访问、避免 shared-memory bank conflict，thread block swizzling 可避免按低效的行主序或列主序分配 tile，如图 10-12所示。

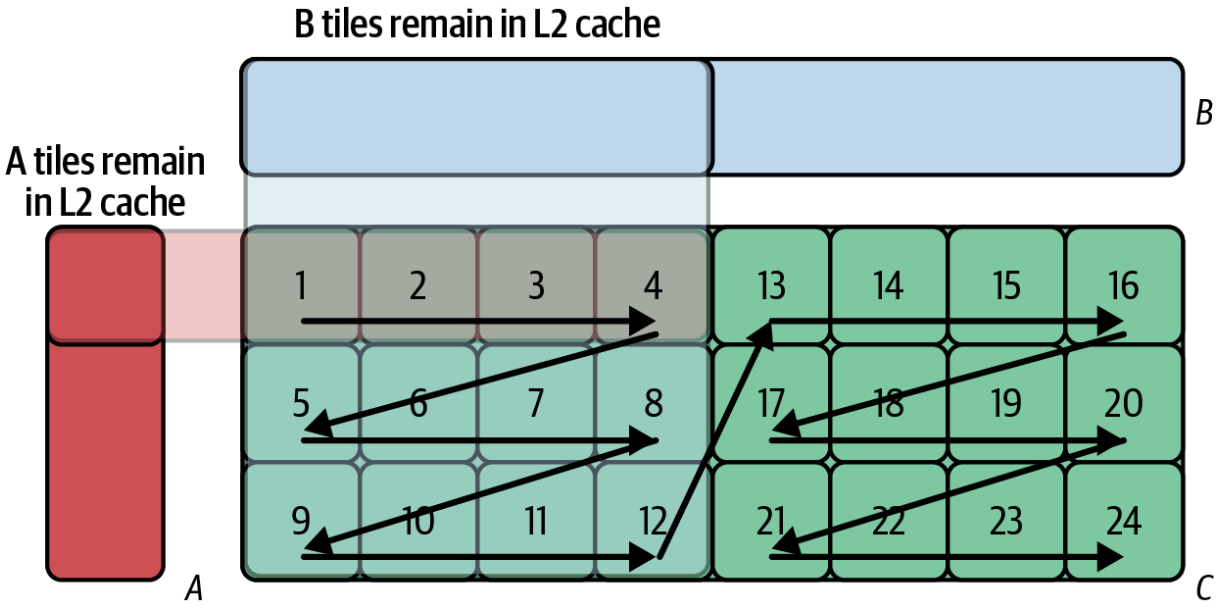


图 10-12　使用 thread block swizzling，在同一 wave 中从 L2 cache 读取 A、B tile

Thread block swizzling 让同一 wave 所需的 A、B 两个矩阵 tile 留在 L2 中，以实现最大复用。在 persistent kernel 和 tiled GEMM workload 中应用这种 swizzling，可以通过减少 memory miss 与带宽压力带来两位数百分比的性能提升；但具体收益仍应通过目标 workload 的 benchmark 验证。Thread-block swizzling 是一种简单而有力的模式重排技术，使 kernel 的启动顺序与 cache locality 对齐。

借助 thread block cluster 硬件支持、较多的 SM 以及改进的 pipeline 调度，现代 GPU 可以运行带有复杂 warp-specialization pipeline 的大型 persistent kernel，以优化 kernel 性能。

简而言之，thread block cluster 在 cooperative groups 模型之上扩展，使部分 block 能够同步并共享 state，而不必锁住整个 GPU。这样可以在一次 launch 内构建多阶段、细粒度的 pipeline；不过其实际共置和资源影响仍取决于 cluster 配置与 scheduler，不能把剩余 SM 一概视为完全空闲。

在 thread block cluster 中，block 之间共享 state 的两个关键机制是 DSMEM 和 scratch memory。下面分别介绍它们。

分布式共享内存

Distributed shared memory（DSMEM）扩展了单个 thread block 的 `__shared__` memory 概念，使其覆盖整个 thread block cluster。传统 kernel 中，每个 thread block 都有自己的私有 `__shared__` 区域，其他 block 无法访问。

使用 DSMEM 时，同一 cluster 中的多个 block 可以读写一个逻辑上合并了各自 local `__shared__` 区域的共享空间。也就是说，cluster 的 shared memory 被拼接成一个分布式片上 buffer。

将 block 间数据保留在片上 memory 中，DSMEM 可提高有效 arithmetic intensity，因为 block 能交换中间结果或执行 reduction，而无需往返 global memory；当数据集超出单个 block 的 shared memory 容量时尤其有价值。

不必退回到 global load/store，可以启动 thread block cluster，让各 block 分别处理数据的一部分，然后通过 DSMEM 同步和共享。通信在 SM 之间进行，可避免代价较高的 HBM 流量；实际收益取决于架构和访问模式。

简而言之，DSMEM 是 block 间通信的一种更结构化的片上方案，在某些 workload 中可能比 global memory 更快，但作用域仅限于同一 cluster。需要频繁、低延迟 block 间协调的 persistent kernel、attention mechanism、LLM 多阶段算法，以及适合把中间结果留在片上的 workload，都可以考虑 DSMEM。

> portable cluster size 的最大值是 8 个 block。部分 GPU 在显式启用 `cudaFuncAttributeNonPortableClusterSizeAllowed` attribute 后支持更大的 non-portable cluster size（例如最多 16 个 CTA）；这会扩大 DSMEM footprint，同时占用 cluster 中更多 SM。

暂存内存

Scratch memory 是支撑 DSMEM 和 thread block cluster synchronization 的底层硬件基础设施。DSMEM 是 CUDA 代码可隐式使用的共享数据 buffer，而 scratch memory 是 GPU 用来跟踪协调元数据的独立片上 SRAM 区域，例如 barrier 状态、group 进度和 DSMEM 访问标志。

kernel 不能直接访问 scratch memory；它由 GPU 自动管理。启动 thread block cluster 时，硬件会分配一部分 scratch memory，用于维护 cluster-wide barrier counter（`cluster.sync()` state）、跟踪哪些 block 已到达 DSMEM operation 或 synchronization point，并协调跨 SM 对分布式 `shared` 区域的安全访问。

由于 scratch memory 针对这些元数据操作进行了优化，它有助于 cluster-level barrier 和 DSMEM access 快速完成。对于很大的 cluster 或复杂 synchronization pattern，如果元数据超过可用 scratch SRAM，具体架构可能把部分状态转移到由 L1/L2 支持的其他存储路径；是否发生以及代价取决于实现，不能把这种 spill 视为 CUDA 的通用保证。

简而言之，DSMEM 是用于在 block 之间共享数据的 abstraction，而 scratch memory 是幕后设施，使 DSMEM operation 能够高效且可扩展。二者让 thread block cluster 在逻辑上表现为一个整体，突破传统 block 间的边界，从而有利于需要多 block 紧密协调并行的 workload。

启动线程块集群

下面说明如何实际使用 thread block cluster。首先，需要使用此前未用过的 `cluster` 维度启动 kernel。

CUDA 通过可指定 cluster size 的扩展 launch API 实现这一点。例如，如果希望每个 cluster 包含两个 block（也称 *thread block pair* 或 *CTA pair*），可以配置如下 launch attribute：

```
// Host code: launch a kernel with thread block clusters of size 2
cudaLaunchConfig_t config{};
config.gridDim = dim3(128, 1, 1);
```

```
config.blockDim = dim3(256, 1, 1);
config.dynamicSmemBytes = 0;

cudaLaunchAttribute attr{};
attr.id = cudaLaunchAttributeClusterDimension;
attr.val.clusterDim.x = 2;
attr.val.clusterDim.y = 1;
attr.val.clusterDim.z = 1;

config.attrs = &attr;
config.numAttrs = 1;

// Allow non-portable cluster sizes if you intend to use > 8 later
cudaFuncSetAttribute(MyClusterKernel,
                     cudaFuncAttributeNonPortableClusterSizeAllowed,
                     1);

cudaLaunchKernelEx(&config, MyClusterKernel, args, nullptr);
```

这段主机端代码使用 `cudaLaunchKernelEx` 启动 `MyClusterKernel`，每个 cluster 包含 2 个 thread block（128 个 block ÷ 每 cluster 2 个，因此共 64 个 cluster）。`clusterDim` 设置为 2，表示每个 cluster 含 2 个 thread block。在需要 cluster-cooperative kernel 时，它替代传统的 `<<<gridDim, blockDim>>>` 语法。CUDA runtime 会依据硬件和资源约束调度这些成对的 block，使其能够使用 DSMEM 通信。

> 请记住，每个 thread block cluster 必须在目标 GPU 的 SM 和资源约束下驻留。portable maximum cluster dimension 是 8 个 thread block；部分受支持的 GPU 可能允许更大的 non-portable size，但必须启用相应 attribute，并确保 block 的资源占用使整个 cluster 能够同时驻留。具体上限取决于 GPU、CUDA 版本和 launch 配置，不能仅凭“Blackwell”这一名称推断。

使用 Cooperative Groups API 协调线程块集群

要协调集群中的多个线程块，首先使用 `cooperative_groups::this_cluster()` 获取这组线程块的句柄。该集群句柄可执行硬件加速的集群范围屏障，并直接访问另一线程块的共享内存。

这一切无需离开内核，也无需借助全局内存标志。以下示例内核将每个线程块的局部值累加到线程块 0 的共享内存中：

```
#include <cuda_runtime.h>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

__global__ void MyClusterKernel(/* args */) {
    // 1. Form a cluster group for all thread blocks in this thread block cluster
    cg::cluster_group cluster = cg::this_cluster();

    // 2. Allocate the same extern shared buffer in each block
    extern __shared__ int shared_buffer[];

    // 3. Each block learns its rank and the total cluster size
    int clusterRank = cluster.block_rank();
    int clusterSize = cluster.num_blocks();
```

```
    // 4. Initialize this block's portion of shared memory (a simple local sum)
    int localSum = threadIdx.x;
    shared_buffer[threadIdx.x] = localSum;

    // 5. Barrier across all blocks in the cluster; no block proceeds until
    //    every block reaches this point and has written its shared_buffer.
    cluster.sync();

    // 6. Map a pointer to block 0's shared_buffer so block can write into it
    // Pointers returned by cluster.map_shared_rank
    // refer to the remote CTA's shared memory
    // and support remote atomics
    // and memory operations within
    // the cluster
    // Pair updates with cluster.sync at well defined point
    int* remote_buffer = cluster.map_shared_rank(shared_buffer, 0);

    if (clusterRank != 0) {
        // 7. Nonzero blocks atomically add their local shared_buffer[0] into
        //    rank 0's buffer. This atomicAdd is routed on-chip over DSMEM
        //    (not through DRAM.)
        atomicAdd(&remote_buffer[0], shared_buffer[0]);
    }

    // 8. Another cluster-wide barrier ensures all atomic adds have completed
    cluster.sync();

    // 9. Finally, block 0 (rank 0, thread 0) can read the combined result
    if (clusterRank == 0 && threadIdx.x == 0) {
        printf("Combined sum in cluster[0]: %d\n", shared_buffer[0]);
    }
}
```

这里通过调用 `cg::this_cluster()` 获取 cluster handle。它返回一个 `cluster_group` object，恰好表示作为一个 thread block cluster 一起启动的那些 block。对于要求 cluster 成员协同驻留的 launch，必须满足目标 GPU 的并发驻留与资源约束；否则启动可能失败。

这里假定每个 block 都使用一致的 shared-memory allocation；cluster 中各 block 必须为 `extern __shared__ int shared_buffer[]` 声明相同大小。支持 DSMEM 的硬件随后会把各 block 的 shared memory 逻辑上组合成一个虚拟地址空间。由于所有 block 为 `shared_buffer` 预留相同大小，指向 thread block 0（rank 0）buffer 的指针才能按 API 规定在 SM 之间正确使用。

> 使用 thread block cluster 时，应像考虑 global-memory coalescing 一样组织各 block 的 DSMEM access。让每个 warp 访问连续且对齐的区域通常有助于吞吐；但具体对齐粒度、bank 行为和 serialization 取决于架构与访问路径，不能把 32-byte 对齐视为所有设备的硬性充分条件。

前面的示例还展示了使用 `cluster.sync()` 作为 cluster 内各 thread block 之间的 cluster-wide barrier。`__syncthreads()` 只同步单个 block 内的线程，而 `cluster.sync()` 会同步该 cluster 中每个 block 的参与线程。

该 barrier 由硬件支持，通常比 grid-level synchronization 的协调范围更小，因此在某些架构和 workload 上可能具有更低延迟；实际成本仍需测量。

不能因为 cluster launch 就认为绝不会 deadlock。虽然 launch 机制会约束 cluster 的驻留条件，但如果参与线程发生 divergence、提前返回、fault，或并非所有成员以一致控制流调用 barrier，仍可能挂起；代码必须保证所有参与者都到达每个 `cluster.sync()`。

在前面的代码块中可见，跨块共享内存访问通过 `map_shared_rank()` 进行。第一个屏障之后，每个块的 `shared_buffer[]` 均已初始化。为了获得指向块 0 共享内存的指针，其他块调用 `int* remote_buffer = cluster.map_shared_rank(shared_buffer, 0)`；其中指定了集群内线程块 ID `(0)`。

在支持 DSMEM 的架构上，runtime/hardware 会将该指针映射到 cluster 内目标 CTA 的 shared-memory 地址；访问路径和一致性语义由 CUDA API 与架构规定，并不应笼统理解为所有操作都直接绕过 DRAM/L2。远程 DSMEM access 的性能也不同于 local shared memory，应通过合并访问和适当对齐来减少额外开销。

例如，在前面的代码块中，当块 1…n 需要将局部值加到块 ID 0 的共享缓冲区时，它们调用 `atomicAdd(&remote_buffer[0], shared_buffer[0])`。由于 `remote_buffer` 是用 `cluster.map_shared_rank()` 获取的，`atomicAdd` 会通过片上 DSMEM 网络直接写入块 0 的 SMEM。换言之，无需往返 DRAM 或 L2 缓存，每次写入都以片上速度发生。

在前面的代码示例中，所有块完成 `atomicAdd` 后，第二个 `cluster.sync()` 会确保块 0 在继续前能从自己的 `shared_buffer[0]` 看到完整求和结果。

总之，`cooperative_groups::this_cluster()`、`cluster.sync()` 与 `cluster.map_shared_rank()` 提供了在 thread block cluster 内跨多个 block 同步和共享数据的接口。它们能在适用架构上实现 cluster-scope cooperation；是否比 global-memory fallback 或手工 atomic/flag 方案更快，取决于访问模式、同步频率、资源占用和实测结果。

线程块对

在支持该功能的现代 NVIDIA GPU 上，可以将恰好两个 thread block 组成 cluster，也就是 thread block pair（又称 *CTA pair*），并在同一 GPC 的规定调度范围内协同驻留。将 block 分组后（例如 2-block cluster，如图 10-13所示），共享数据的 kernel 可以使用 TMA 将 tile 搬入各 block 的 shared memory。

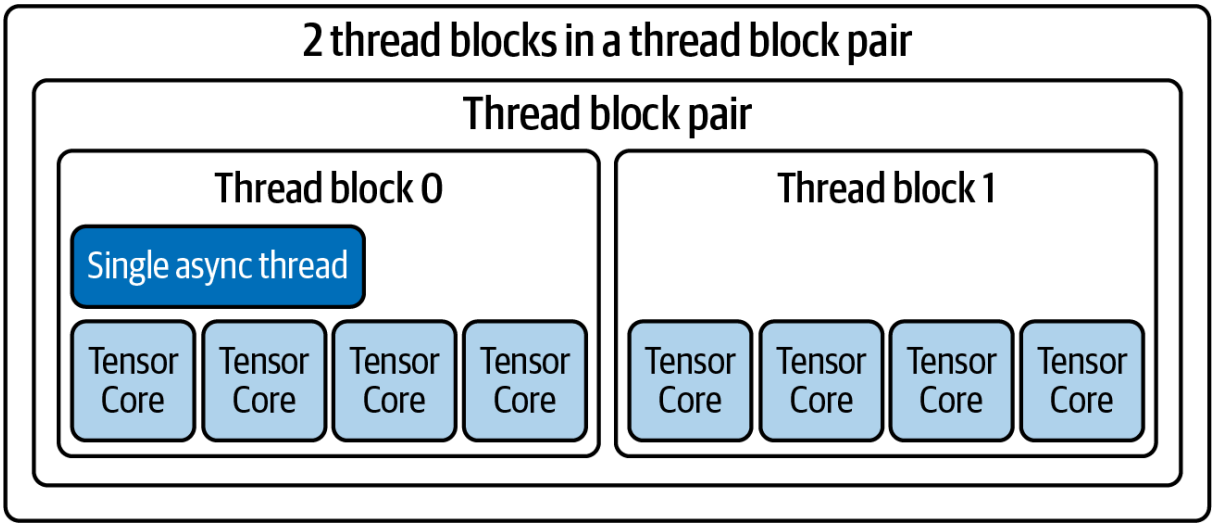


图 10-13　Thread block pair 将两个 thread block 组合起来

单个 thread block 可能缺少处理超大 tile 所需的 register 或 shared-memory 容量（例如 256 × 256 的矩阵子 tile）。通过在 GPC 规定范围内将两个 thread block 配对并使用 DSMEM，这两个 block 可以拆分一个大 tile 的工作，同时通过统一的 shared-memory 区域共享数据，如图 10-14所示；实际 block-to-SM 映射由硬件调度决定。

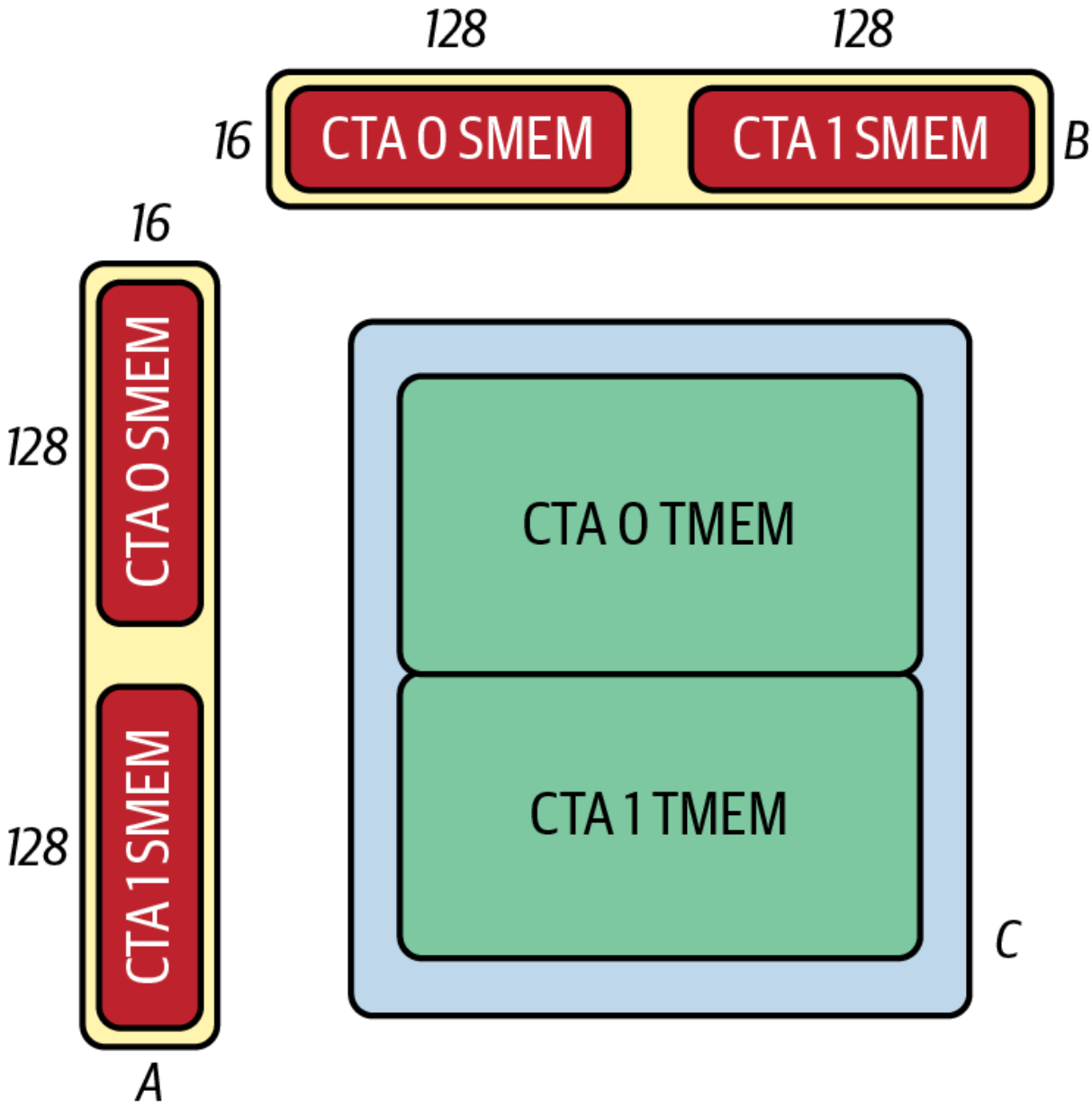


图 10-14　Thread block pair（又称 CTA pair）载入 A × B 矩阵乘法的 operand tile（来源：https://oreil.ly/kEZsv）

在这个配对中，每个 thread block 可以将 operand tile 的一部分（例如 128 × 16）载入片上 SMEM，用于矩阵乘法；此外，每个 block 在 Tensor Memory（TMEM）中保存部分 accumulator（例如 128 × 256）。这样 CTA pair 中的两个 block 就能协作处理单个 tile，如图 10-15所示。

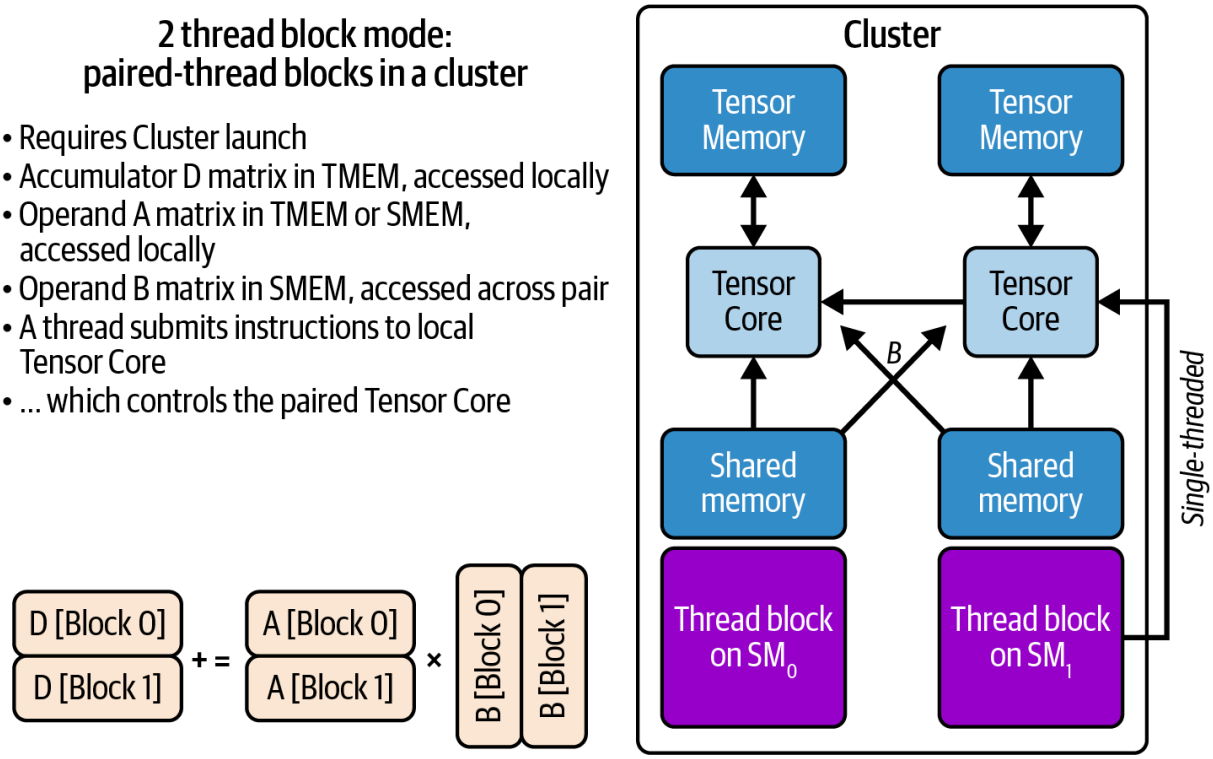


图 10-15　带有 Tensor Core 和 TMEM 的 thread block pair

Thread block pair 可以支持跨两个 SM 的更大矩阵乘法。由于每个 SM 处理 tile 数据的一部分，使用两个 block 可能扩大单个 tile 可处理的规模，但并非无条件地等同于将 tile size 加倍。

SM 硬件通过 DSMEM 在 SM 之间共享 operand data。DSMEM 可减少重复 load、改善数据复用并提高有效 arithmetic intensity；收益取决于访问模式和资源占用。图 10-16展示了使用 DSMEM 的 thread block cluster 如何在 SM 之间共享数据。

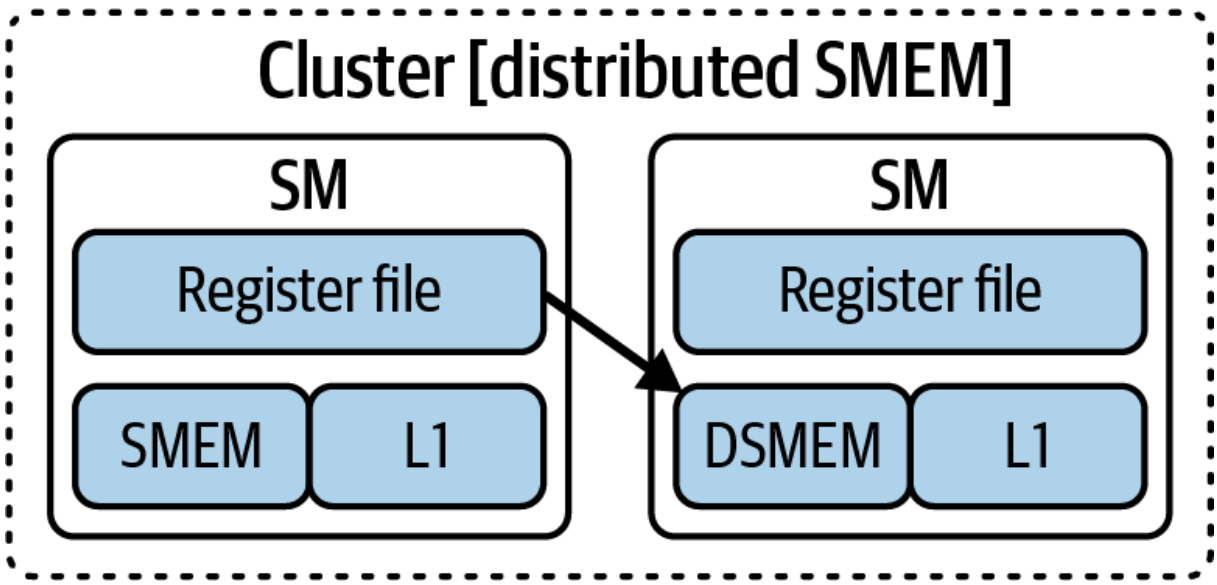


图 10-16　使用 DSMEM 在 thread block cluster 内的 SM 之间共享数据

CUDA 为这类多 SM 操作提供相应的 multicast、barrier 与 synchronization primitive。配对的 block 使用轻量的 `cluster.sync()` 调用彼此同步；因此，当一个 SM

完成 tile 的使用后，CTA pair 中的另一个 block 才会在满足同步条件时继续消费它。

每个 tile 可以被送入这两个 block，而不必重复执行 global-memory transaction。这可能改善 register、shared-memory bank 和 Tensor Core 的资源利用；但是否提高 Tensor Core utilization，仍取决于 tile shape、occupancy、调度和实际 workload。

从程序员角度，只需启动一个包含两个 block 的 cluster，并将其视为拆分到两个 block 上的大型协作单元。例如在 CUTLASS 中，这可表现为使用 DSMEM 与 cluster barrier 进行 inter-CTA 数据共享的 2-SM UMMA（Unified MMA）GEMM operation；具体 cluster launch、DSMEM 配置和同步由库实现处理，仍需确认目标架构与版本支持。

如果请求的 tile size 超过单个 thread block 的处理能力（例如 FP16 Tensor Core operation 中的 128 × 128），某些 CUTLASS 配置可能使用两个 block 组成 pair；具体是否自动采用以及所需资源取决于 kernel 配置。SM 会拆分工作，并在底层通过 DSMEM 与 `cluster.sync()` 共享 tile，从而支持 UMMA operation 的配对协作。

总之，thread block pair 与 DSMEM 以多 SM 协作为形式提供并行性，可在适用架构上实现 block 间的片上数据共享和同步，减少某些原本需要 global-memory handoff 或额外 kernel launch 的场景。实际收益与实现复杂度仍需通过目标算法和设备实测。

使用 Thread Block Cluster 减少 Global Memory 流量

从性能角度看，DSMEM 可以减少重复的 global-memory 流量，并可能提高有效带宽。例如在 tiled GEMM 中，如果 cluster 内多个 thread block 共享 A 或 B 矩阵的片段，一个 block 可以从 global memory 载入 tile，再通过 TMA 将其 multicast 给 cluster 中的其他 block。

当 thread block 属于同一 cluster 时，TMA engine 支持将数据直接送入 DSMEM 的 multicast copy mode。一次 TMA transfer 可以同时把数据放入每个参与 block 的 shared memory，从而避免重复的 DRAM fetch。

使用 TMA multicast 时，支持该路径的 GPU 可以在一次操作中将数据分发到 cluster 成员（thread block）的 shared memory，减少同一 tile 的重复 global-memory load。这可能改善 bandwidth utilization、减少 DRAM 流量，尤其适用于许多 block 需要相同输入的场景，如图 10-17所示。

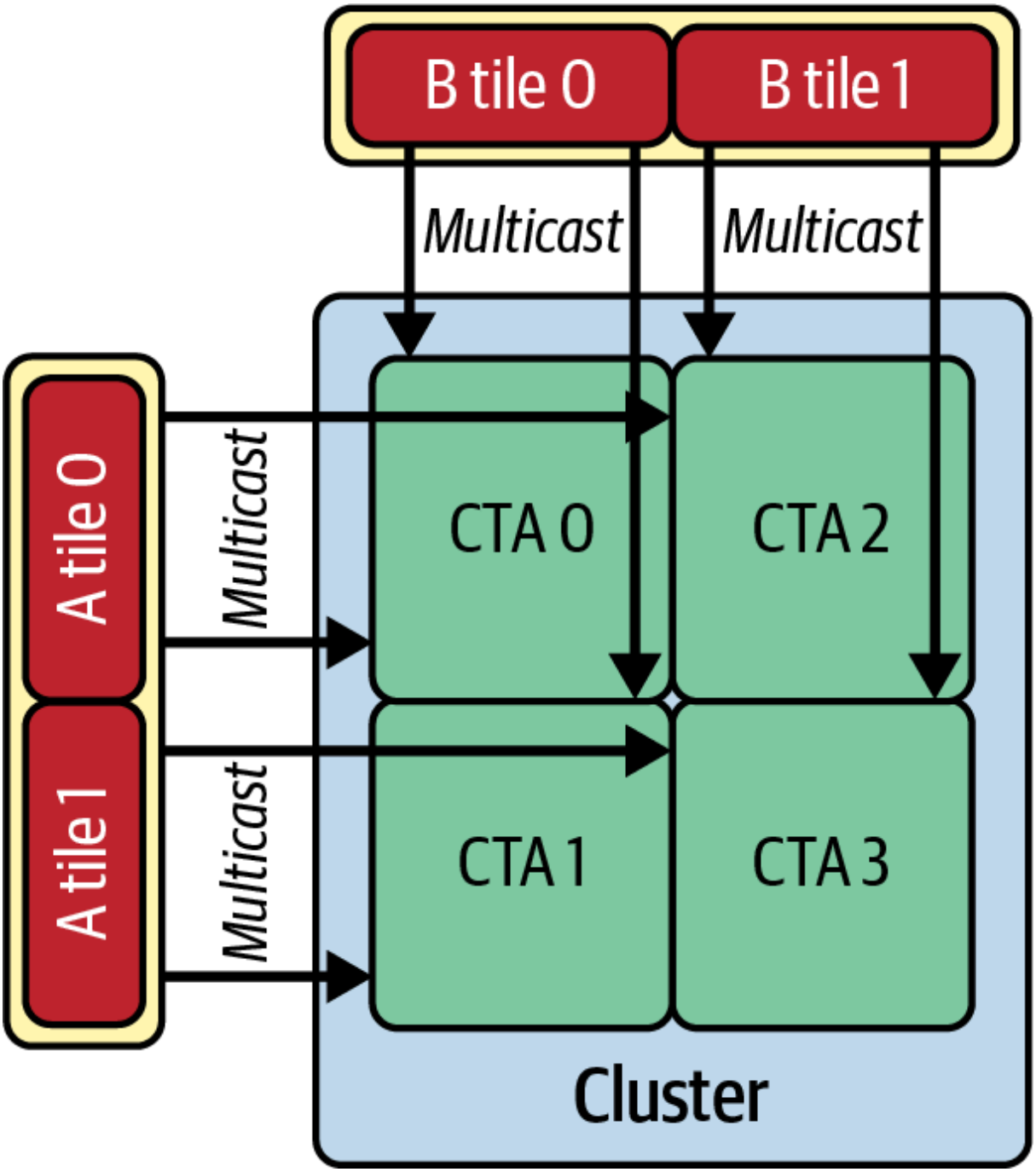


图 10-17　对于这四个（2 × 2）thread cluster，每个 tile 只载入一次，然后 multicast 到每个 cluster 中所有 CTA 的 shared memory（来源：https://oreil.ly/kEZsv）

这里 TMA engine 执行从 global memory 到 DSMEM 的一次 multicast，将 tile 分发到 cluster 中每个 thread block 的 SMEM，从而减少重复 DRAM read。TMA multicast 通过 tensor-map descriptor 配置，并以目标为 `shared::cluster` 的 `cp.async.bulk.tensor` 发出。CUTLASS/cuTe、Triton 等高层库可以代为生成这些 multicast operation、descriptor 和 bulk tensor copy；相关 PTX 指令（包括带 tensor-map operand 的 `cp.async.bulk.tensor`）见PTX 文档。是否可用仍取决于目标架构、CUDA 版本和库实现。

另一个常见用途是 multiblock reduction 或 scan。各 block 不必先把 partial sum 或 scan result 写入 global memory、再启动单独 kernel 合并；thread block cluster 中

的 block 可以将 partial result 写入 DSMEM。

在同一 cluster 内，可以利用 shared memory 和 cluster-level barrier `cluster.sync()` 对这些 partial result 执行片上 reduction 或 prefix-sum。通常只需将最终结果写入 global memory，这可能减少 global-memory read/write。

组合 thread block cluster、DSMEM 与 TMA multicast，可以构建将大部分数据交换留在片上的多阶段、细粒度 pipeline。无论是为 GEMM 共享 tile、为 reduction 累加 partial sum，还是执行 multiblock scan，这些机制都可能减少 HBM 往返并提高有效 arithmetic intensity；收益需由具体 workload 验证。

> block-scope 的 `cuda::pipeline` 不会同步其他 block；cluster-wide distribution 需要使用 TMA multicast，或配合 `cluster.sync` 的 DSMEM。

考虑 CTA 0 和 CTA 1 两个都需要矩阵 A 同一 tile 的 thread block。没有 DSMEM 时，每个 CTA 都会发起自己的 global-memory load，导致相同数据被取两次并浪费 DRAM 带宽。

使用 DSMEM 时，CTA 0 可将 tile 载入自己的 shared memory 一次，再通过片上 DSMEM network multicast，使 CTA 1 直接从 shared memory 读取。如果 cluster 包含超过两个 CTA，同一 tile 可以在所有 block 之间共享；不过所需的 global HBM load 数量仍取决于具体 TMA/访问协议。表 10-4比较了两个 thread block 组成的 cluster 与两个独立 thread block。

表 10-4　DSMEM 对双 block workload 的性能影响（示意数据）

指标	两个独立 CTA（无 DSMEM）	带 DSMEM 的 CTA pair（2-block cluster）
全局内存加载事务	2×（每个线程块加载一次 tile）	1×（tile 仅加载一次）
L2 cache 命中率	50%	85%
CTA 间数据复用	不适用（无复用）	显著（CTA 1 复用该 tile）
每个 CTA 的有效 DRAM 带宽	300 GB/s	减少 50% 150 GB/s
Kernel 时间（相对值）	1.0×	0.6×（加速 40%）

表中给出的 40% kernel speedup、300→150 GB/s 和 50%→85% 命中率应视为该示例 workload 的示意 benchmark，而非 DSMEM 的通用保证。实际结果会受到 tile shape、cache 状态、cluster residency、TMA 配置和竞争影响。

简而言之，multicast 共享数据可以让多个 block 以片上路径复用数据；对于 memory-bound workload，这可能带来明显加速，但应通过端到端测量确认。

需要注意，DSMEM 与 L2 是不同的数据路径；这为 block 间共享提供了两条可能的路径。DSMEM 面向 cluster-local address 的访问使用专用的 SM-to-SM 互连，而 L2 仍服务于 global-memory/cache 层级。具体是否绕过某一级 cache、以及两条路径如何协同，取决于架构和指令语义，如图 10-18所示。

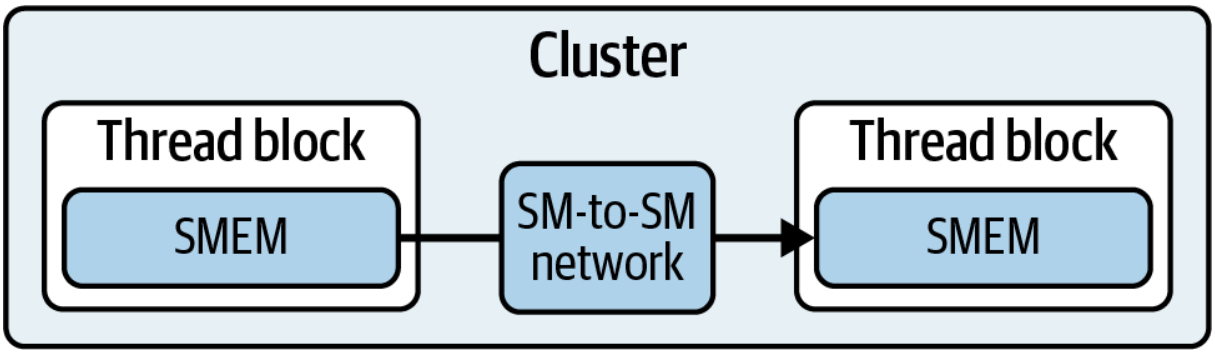


图 10-18　DSMEM 使用 SM-to-SM 的 cluster-local network，在两个 thread block cluster 之间交换数据

远程 DSMEM access 通过 thread block cluster interconnect 路由，与 global-memory traffic 不同。thread block 使用 DSMEM 获取 tile 时，会走 cluster 规定的 shared-memory 传输路径；其延迟和吞吐通常不同于 local shared memory，不能笼统保证二者完全相同。

如果 tile 不在可访问的 DSMEM 区域中，就必须通过正常的 global-memory/cache 层级获取，可能命中 L2。例如 tile 已不再驻留于相关 shared-memory allocation 时，thread block 仍可能从 L2 取得数据，而不必每次都回到 DRAM；具体行为取决于 cache 状态。

因此，合理设计的 DSMEM/TMA 路径可能减少 memory stall 和重复流量，但 occupancy 是否保持较高、整体是否显著快于两个独立且未优化的 block，必须由 profiler 与端到端 benchmark 证明。

使用 Thread Block Cluster 设计高效算法

Thread block cluster 为过去需要 global-memory communication 或多次 kernel launch 的 workload 提供了新的并行化策略。例如，大型矩阵乘法的 output tile 可能因 shared-memory 限制而无法由单个 thread block 处理。

过去可以把工作拆到两个 thread block，但每个 block 都要通过 global memory 交换 partial result，可能带来额外流量和延迟；另一种做法是再启动 reduction kernel 合并两个 block 的输出。

借助 thread block cluster 和 DSMEM，这两个 block 可以组成 cluster，直接共享片上 shared-memory allocation，并使用硬件支持的 primitive 合并结果。在支持的架构上，DSMEM 允许一个 SM 通过 cluster interconnect 对另一个 SM 的 shared memory 执行 load/store/atomic。

要有效使用 thread block cluster，必须仔细设计算法。同步开销可能较低，但并非为零；过于细粒度的数据共享未必值得。

Thread block cluster barrier 类似第 7 章介绍的 warp-level intrinsic（例如 `__shfl_sync()`）：所有参与者都必须到达同一个同步点。调用 `cluster.sync()` 时，cluster 中所有 thread block 都必须到达该行，之后 block 才能继续。

如果某个 block 提前完成，它会等待；如果某个 block 因线程走了不同分支等原因永远不到达 barrier，整个 cluster 可能 deadlock。

换言之，warp intrinsic 要求 warp 内线程遵循一致的 instruction path 以避免 divergence；thread block cluster 也要求所有 block 在每次 `cluster.sync()` 前遵循一致的控制流。

由于这种 lockstep 要求，block 间细粒度数据共享必须与同步开销和 deadlock 风险权衡。正确使用时同步可能很便宜；但只要一个 block 绕过或延迟到达 `cluster.sync()`，性能就可能受影响，甚至使 kernel 挂起。

应组织代码，使每个 block 一致到达每个 barrier，就像使用 warp-level intrinsic 时 warp 内所有线程都必须到达 barrier 一样。这是利用 thread block cluster 低延迟片上通信、避免 deadlock 的必要条件。

通常，当每个 block 在同步或数据交换点之前都有足够多、可独立执行的工作时，thread block cluster 更可能取得较好结果。同步并完成片上数据传输后，各 block 可以继续处理。

Thread block cluster 对 block-sparse matrix operation 可能特别有用，这类操作常见于 sparse attention、LLM model pruning 和 compression。此时各 block 处理不同的非零区域，并共享边界数据。

Thread block cluster 也可用于多阶段 reduction，例如 transformer layer 中的 softmax 和 normalization；这些操作通常需要使用 cluster 内各 block 的 partial result 执行最终合并。

更一般地说，当 large GEMM 超出单个 block 的资源时，可能受益于 thread block cluster。GEMM 是现代 LLM 中 transformer attention、multilayer perceptron（MLP）层和 embedding lookup 的核心计算。

不过，更大的 cluster size 可能降低整体 occupancy。例如，16-block cluster 可能在某段时间占用较多 SM，减少同一设备上其他工作可用的资源。除非特殊场景确实需要更大 cluster，否则可以先从 2 或 4 个 block 的 cluster 开始；最终应使用具体 workload profiling，确认片上资源共享带来的收益超过潜在并行度损失。

使用 cluster launch 时，应使用 Nsight Compute launch statistics 和标准 occupancy API（例如 `cudaOccupancyMaxActiveBlocksPerMultiprocessor`）验证 active block 与 cluster residency。使用 non-portable cluster size 时，设置 `cudaFuncAttributeNonPortableClusterSizeAllowed`，或通过 `cudaLaunchKernelEx` attribute 传入 `cudaLaunchAttributeNonPortableClusterSizeAllowed`；否则 launch 可能失败，或实测 occupancy 较低。

使用 Thread Block Cluster 进行 Warp Specialization

现在重新讨论 warp specialization，并将其与 CUDA Pipeline API 和 thread block cluster 结合。以 cluster leader 所在 block 的 block-scope pipeline 分阶段执行 copy，再通过 DSMEM 执行 cluster-wide barrier。这样 cluster 中的 block 可以消费同一组 input tile，而不必各自从 global memory 重复加载。

每个 block 内按 warp 分配角色。leader block 的 loader warp 为每个 tile 协作执行一次 copy 到自身 shared memory；cluster-wide barrier 发布这些 tile 后，每个 block 使用 compute warp 通过 DSMEM 读取 leader 的 tile，并计算互不重叠的行区间；各 block 的 storer warp 再把该区间写回 global memory。异步 copy 只由 leader 使用 block-scope pipeline，其他 block 不等待这条 pipeline。代码如下：

```
// Warp specialization across a thread-block cluster
// using DSMEM and a block-scoped pipeline

#include <cuda/pipeline>
#include <cooperative_groups.h>
#include <algorithm>
namespace cg = cooperative_groups;

#define TILE_SIZE 128
#define TILE_ELEMS (TILE_SIZE * TILE_SIZE)

// Compute a band of rows of the TILE_SIZE×TILE_SIZE product from DSMEM sources.
// Each lane processes rows [row_begin, row_end) in a 32-way striped loop.
__device__ void compute_rows_from_ds(const float* __restrict__ A_src,
                                     const float* __restrict__ B_src,
                                     float* __restrict__ C_dst,
                                     int row_begin, int row_end,
                                     int lane_id) {
    for (int row = row_begin + lane_id; row < row_end; row += warpSize) {
        for (int col = 0; col < TILE_SIZE; ++col) {
            float acc = 0.0f;
            #pragma unroll
            for (int k = 0; k < TILE_SIZE; ++k) {
                acc += A_src[row * TILE_SIZE + k] * B_src[k * TILE_SIZE + col];
            }
            C_dst[row * TILE_SIZE + col] = acc;
        }
    }
}

extern "C"
__global__ void warp_specialized_cluster_pipeline(
    const float* __restrict__ A_global,
    const float* __restrict__ B_global,
    float* __restrict__ C_global,
    int numTiles) {
    thread_block cta = this_thread_block();
    cluster_group  cluster = this_cluster();

    extern __shared__ float shared_mem[];
    float* A_tile_local = shared_mem;
    float* B_tile_local = A_tile_local + TILE_ELEMS;
    float* C_tile_local = B_tile_local + TILE_ELEMS;

    // Block-scoped pipeline used only by the cluster leader
    // to stage asynchronous copies
    __shared__
```

```
cuda::pipeline_shared_state<cuda::thread_scope_block, 2> pipe_state;
auto pipe = cuda::make_pipeline(cta, &pipe_state);

const int lane_id = threadIdx.x & 31;
const int warp_id = threadIdx.x >> 5;

auto warp = tiled_partition<32>(cta);

const int cluster_rank      = cluster.block_rank();
const dim3 cluster_dims     = cluster.dim_blocks();
const int  blocks_in_cluster = cluster_dims.x * cluster_dims.y *
                               cluster_dims.z;

// 1D cluster arrangement along x;
// each iteration processes one tile per cluster
auto loader = cooperative_groups::tiled_partition<32>(cta);
for (int tile = blockIdx.x / cluster_dims.x; tile < numTiles;
     tile += gridDim.x / cluster_dims.x) {
    const size_t offset = static_cast<size_t>(tile) * TILE_ELEMS;

    // Leader block stages A and B once for the entire cluster.
    // For block-scoped pipeline collectives, entire leader CTA participates.
    if (cluster_rank == 0) {
        pipe.producer_acquire();
        if (warp_id == 0) {
            cuda::memcpy_async(warp, A_tile_local, A_global + offset,
                               TILE_ELEMS * sizeof(float), pipe);
            cuda::memcpy_async(warp, B_tile_local, B_global + offset,
                               TILE_ELEMS * sizeof(float), pipe);
        }
        pipe.producer_commit();
        // Make loads visible to leader CTA before publishing cluster-wide.
        cuda::pipeline_consumer_wait_prior<1>(pipe);
    }

    // Publish the leader's tiles to every block via DSMEM
    cluster.sync();

    const float* A_src = cluster.map_shared_rank(A_tile_local, 0);
    const float* B_src = cluster.map_shared_rank(B_tile_local, 0);

    // Divide rows among blocks in the cluster
    const int rows_per_block = (TILE_SIZE + blocks_in_cluster - 1)
                               / blocks_in_cluster;
    const int row_begin = std::min(cluster_rank * rows_per_block, TILE_SIZE);
    const int row_end   = std::min(row_begin + rows_per_block, TILE_SIZE);

    // Compute warp produces block's band of rows into local shared memory
    if (warp_id == 1) {
        pipe.producer_acquire();
        compute_rows_from_ds(A_src, B_src, C_tile_local, row_begin, row_end,
                             lane_id);
        pipe.producer_commit(); // publish C_tile_local
    }

    // Storer warp writes this block's rows back to global memory
    if (warp_id == 2) {
        pipe.consumer_wait(); // observe C_tile_local
        for (int row = row_begin + lane_id; row < row_end; row += warpSize) {
            for (int col = 0; col < TILE_SIZE; ++col) {
                C_global[offset + row * TILE_SIZE + col] =
                    C_tile_local[row * TILE_SIZE + col];
            }
        }
```

```
            pipe.consumer_release();
        }

        // All blocks finish this tile before the leader reuses its buffers
        cluster.sync();
    }
    // dynamic shared memory size: 3 * TILE_ELEMS * sizeof(float)
}
```

这里 leader 使用 block-scope pipeline，将每个 tile 的两块 operand 协作 copy 到自己的 shared memory；cluster-wide barrier 确保这些数据按 cluster 语义对成员可见，并可通过 DSMEM 访问。

这种“copy once、通过 DSMEM share”的模式，在 cluster barrier 之后使用 DSMEM 和 `map_shared_rank` 共享 leader 的 tile。它不同于前面介绍的 tensor-map descriptor、`cp.async.bulk.tensor` 与 `shared::cluster` TMA multicast。必须明确这一点：在 DSMEM 模式中，如果没有先调用 `cluster.sync()`，follower 可能读取到过期的 leader SMEM 数据。因此，应在 `map_shared_rank()` 之前执行满足协议要求的 `cluster.sync()`。

> 下面的经验可帮助选择 TMA multicast 或 DSMEM sharing：如果每个 block 的 tile reuse 很高，或 cluster size（C）较大，TMA multicast 可能更合适，因为可以写入一次、再由各 block 多次 local read。如果 SMEM 紧张，或 C 较小且每个 follower 只访问 tile 一次，DSMEM sharing 可能 footprint 更小。最终选择仍应依据架构、访问模式和 benchmark。

每个 block 使用 `map_shared_rank` 从 leader 的 shared memory 直接读取，并计算不同的行区间，再将结果写入 global memory。这可以减少 cluster 内重复 global load，同时保留 pipeline 在关键阶段的 overlap 优势。

leader block 的 loader warp 调用 `producer_commit()`，将本地 stage 提交给该 block。其他 block 不等待 leader 的 pipeline，而是在 cluster-wide barrier 后观察 leader 的 tile，再从 distributed shared memory 读取。

这样，tile loading、compute 和 store 可以跨多个 block 交错执行，可能提高 SM 利用率。表 10-5比较了单 block warp-specialized kernel 与本节介绍的 thread-block-cluster（multiblock）warp-specialized kernel。

表 10-5　Naive tiling、两阶段 double buffering、warp-specialized 与 thread block cluster pipeline kernel 的对比（示意数据）

指标	朴素分块	两阶段双缓冲（`double_buffer_pipeline`）	warp 专用（`warp_specialized_pipeline`）	线程块集群流水线（`warp_specialized_cluster_pipeline`）
内核执行时间	41.3 ms	20.5 ms（比朴素版本快 2.01 倍）	18.4 ms（比两阶段快 10.2%）	17.2 ms（比 warp 专用快 6.5%）
warp 执行效率	68%	92%（比朴素版本高 24%）	96%（比两阶段高 4%）	97%（比 warp 专用高 1%）
Warp 状态停顿百分比（共享内存与屏障等待）	高	低	极低	极低（进一步降低）
L2 吞吐量	80 GB/s	155 GB/s（比朴素版本高 94%）	165 GB/s（比两阶段版本高 6.45%）	170 GB/s（比 warp 专门化版本高 3%）
吞吐量可扩展性	可扩展至每个 SM 2–3 个 warp	可扩展至每个 SM 约 6 个 warp	几乎线性扩展至 SM 的 warp 上限（Blackwell 上每个 SM 可驻留 64 个 warp）	在线程块间充分扩展，直至受 Blackwell 上每个 SM 可驻留 64 个 warp 的上限约束
DRAM 读取吞吐量与内核时长的重叠情况	重叠较差	重叠良好	重叠极佳	重叠极佳（即使在线程块交接时）
指令数量	17 亿	10.5 亿（比朴素版本少 38%）	约 10.0 亿（比两阶段版本少 4.76%）	约 9.8 亿（比 warp 专门化版本少 2%）

该表意在说明：thread block cluster 实现可在一次 cooperative launch 中将 loader、compute 和 storer 角色分布到多个 block，从而可能改善 SM 利用率并减少执行时间和重复内存流量。表中数值是示例 benchmark，不代表通用结果。

在示例中，thread block cluster pipeline kernel（`warp_specialized_cluster_pipeline`）比单 block warp-specialized kernel 约快 1.2 ms（6.5%）；原因是 cluster 版本在多个 block 间交错执行 tile load、compute 和 store。该提升依赖具体设备与 workload。

示例中 SM utilization 约为 97%，L2 load throughput 约为 170 GB/s；这些数字取决于实现和测量方法。DSMEM 模式下，leader 只加载一次 tile，其他 block 在调用 `cluster.sync()` 后通过 `map_shared_rank()` 读取；TMA multicast 则可将一次 global tensor copy 分发到各 block 的 shared memory。

示例中 instruction count 约降至 0.98 B，但真实计数取决于编译器和 kernel。只要有足够 block 与 warp，pipeline 可能扩展到目标 GPU 的 resident-warp 上限；具体上限不能仅凭 Blackwell 名称推断。

单 block 与 thread block cluster 的 warp specialization 最终都受同一类“每 SM resident warp”资源上限约束，但二者在数据供给和同步方式上不同。是否形成性能优势，应通过类似表 10-5的实测确认。

在单 block warp-specialized kernel 中，每个 block 可分配三个 warp：一个将 A、B 的 tile-sized chunk 加载到 shared memory，一个计算该 chunk，一个将结果写回 global memory。三个 warp 完成各自角色后，block 继续处理下一个 tile，循环往复。

即使每个 SM 都有三个分别负责 load、compute、store 的 active warp，也不意味着达到了 SM 的 warp 上限。现代 GPU 的并发 warp 上限因架构而异，实际 residency 还受 register、shared memory 和每 SM block 数限制。

在总 live warp 数相近时，单 block warp-specialized design 与 cooperative grid 可能显示相近 occupancy，但内存行为不同。单 block design 中，每个 block 都获取自己需要的 input tile 副本；多个 block 大致同时复用同一 tile 时，设备可能执行重复 read，消耗 L2 容量和 HBM 带宽。

Thread block cluster 可以改变这种模式。cluster 内 block 可通过 DSMEM 共享数据，也可接收一次将 global tile multicast 到所有 block shared memory 的传输。可以在 leader block 中使用 block-scope pipeline，配合 cluster synchronization 和 `map_shared_rank`；若更适合从 global memory 广播，则配置 TMA multicast。这样可能减少 cluster 内重复 load，降低 L2 和 HBM 压力。

`cuda::pipeline` 的作用域是创建它的 block。一个 block 中的 `producer_commit()` 不会释放其他 block 的 `consumer_wait()`。要在 cluster 内跨 block 协调，应先发布数据（例如写入 leader 的 shared memory），再调用 `cluster.sync()`；follower 随后使用 `map_shared_rank()` 访问 leader 的 shared memory。也可以选择 TMA multicast，将 global memory 数据直接广播到各 block 的 shared memory。

此时，每个 block 都可以通过 DSMEM（或 TMA multicast）观察 leader 的 tile 并继续执行。block-scope pipeline 本身不会显式同步其他 block；cluster-wide visibility 由 `cluster.sync()` 和 DSMEM 语义提供。

如果正确设计 TMA multicast 和 DSMEM，thread block cluster pipeline 可以让每个 tile 在 cluster 内只获取一次，并分发到各 block 的 shared memory，而不是由每个 block 重复加载。SM 仍受自身 resident-warp、register 和 shared-memory 限制；cluster coordination 可能减少重复 load，使 warp 将更多时间用于计算。

在单 block 版本中，每个 block 完成当前 tile 的计算后会加载下一个 tile。如果一个 block 比另一个稍早完成，它可能立即为下一个 tile 发起自己的 load，即使另一个 block 已在加载同一 tile，从而产生重复内存流量。

而在 thread block cluster pipeline 版本中，如果 SM 0 上的 compute warp 提前完成，它不会立即获取下一个 tile，而是在 `consumer_wait()` 处等待 global loader（可能运行在另一个 SM）将该 tile 放入 cluster 可访问的 shared memory。compute warp 等待 cluster 的一次共享 load，而不是发起重复 copy；具体等待协议取决于实现。

等待 tile 按协议可用后再继续，cluster 中的 SM 可能减少重复 load；但等待本身也会产生 stall。合理的阶段对齐有助于平滑 load time 差异、提高计算 warp 的有效工作比例，实际 throughput 需测量。

如果问题足够大、grid 工作足够多，使单 block 方案下每个 SM 已有 loader、compute 和 storer warp，那么两种 kernel 的 raw warp count 和 per-SM occupancy 可能相近。cluster 版本的优势主要来自多个 block 复用同一 input tile 时减少 cluster 内重复 global read，并用 cluster-wide synchronization 对齐阶段。

在 tile reuse 较高的 workload 上，这通常可能带来适度 throughput 改善；收益取决于 tile reuse、alignment 与 resource limit。两种设计都受 resident warp、register、shared memory 等相同类型的架构上限约束。

本节 kernel 使用 CUDA pipeline 完成 producer/consumer handoff，没有调用 block-wide `__syncthreads`；这不表示任何 pipeline 都不需要额外同步，是否需要取决于 API contract 和访问模式。naive tiled kernel 不重叠 memory 与 compute；两阶段 double-buffered pipeline 可在 memory-bound 场景中将 copy 与 compute 重叠，并可能缩短完成时间。

Warp-specialized pipeline 将不同 warp 分别用于 load、compute 和 store，以减少隐式的 full-block wait。Thread block cluster 版本通过 DSMEM 或 multicast 在 cluster 内共享 tile，使每个 tile 可能按 cluster 而不是按 block 获取一次；这些模式有助于提高利用率、减少重复内存流量，但必须满足相应的同步和可见性协议。

要点总结

本章聚焦于从现代 GPU 中提取性能，以下要点有助于避免 kernel 因 DRAM、warp scheduler 或 global barrier 而空转：

用 pipeline 深度隐藏延迟

> 使用两阶段（`cuda::pipeline_shared_state<cuda::thread_scope_block, 2>`）tiling，使异步 load 与 compute overlap；当 compute 明显多于 memory operation 时，可考虑增加 stage（例如 `cuda::pipeline_shared_state<cuda::thread_scope_block, 3>`）。具体深度应依据资源和实测选择，以减少 idle warp。

用 warp specialization 平衡 workload

> 当 compute phase 占主导时，可将不同 warp 分别分配给 load、compute 和 store；是否接近峰值效率仍取决于硬件和 workload。

用 persistent kernel 减少 launch overhead

让一个长生命周期 kernel 循环处理 device-side work queue，并在多阶段算法中使用 `grid.sync()`；这可能减少 host-device 往返和 launch 次数，但必须满足 cooperative launch 与同步条件。

用 thread block cluster 和 DSMEM 启用片上共享

将 thread block（CTA）组成 cluster，使其按支持的语义共享片上 buffer。需要 cluster-wide broadcast 时，可使用带 TMA multicast descriptor 的 `cp.async.bulk.tensor`；tile 是否只需传输一次、以及对 L2/DRAM 的影响，取决于架构和访问模式。

特别注意 barrier 语义

`cluster.sync()` 和 `grid.sync()` 都要求所有参与 thread block 到达同一同步点。控制流不一致可能导致 deadlock；过大的 cluster size 还可能无法满足驻留条件并导致 launch failure。

先 profile，再调优

使用 Nsight Compute 判断 kernel 是 memory-bound 还是 compute-bound。memory-bound 可先尝试两阶段 pipeline；compute-bound 可评估 warp specialization 或 thread block cluster，但应以 profile 和 benchmark 为准。

手工调优前先验证编译器生成的 pipeline

profile 并检查 PyTorch compiler、NVCC 等生成的代码。如果编译结果已经使用 `cuda::memcpy_async`、`producer_commit()` 和 `consumer_wait()` 组成异步 pipeline，手工调整未必还能带来明显加速。

结语

本章讨论的技术用于系统性隐藏延迟并减少重复 load，可能改善 kernel 执行期间的 GPU 利用率；是否接近峰值必须由 profiler 和 benchmark 证明。

Warp-specialized pipeline 可以重叠 load、compute 和 store。Cooperative-group barrier（`grid.sync()` 与 `cluster.sync()`）可在满足参与条件时协调多阶段工作，减少 host 往返；persistent kernel 则循环处理 device-side queue，以减少 launch 次数。

应始终从 profile 开始。如果 global-memory stall 占主导，可以先评估两阶段 asynchronous-copy/double-buffer pipeline；如果 compute warp 仍 stall，再考虑 multistage warp-specialized pipeline，使 loader、compute 和 storer warp 减少不必要的争用。实际瓶颈可能还包括 memory subsystem、barrier、register 和 shared-memory 资源。

> 请记住，并发只有在硬件资源（例如 memory bandwidth）尚未饱和时才可能有益；超过饱和点后，并行任务会争用吞吐资源。

对于多阶段 reduction 或 irregular task，可以评估用单个 persistent kernel 加 `grid.sync()` 替代多次 launch；但要检查 cooperative residency、同步安全和共置影响。当多个 thread block 需要相同数据（例如 multiheaded attention）时，可以组成 thread block cluster，使用 DSMEM 或 TMA multicast 减少重复访问 global memory。最终性能是否接近 GPU 理论峰值仍取决于完整 workload。

后续章节中的许多优化都会用到这些原则：在 warp 和 thread block 层级重叠工作，在 device 上直接同步而不是依赖 host，并尽量在片上共享数据。这些是调优高性能 GPU workload 的重要机制。

下一章将保留这些 intra-kernel building block——`cuda::pipeline` double buffering、warp-specialized role 和 thread block cluster——并说明如何通过 CUDA stream 驱动它们。目标是隐藏 kernel 之间以及 host ↔ device 通信之间的延迟，而不只是在单个 kernel 内部。具体会复用本章 kernel，在 multistream pipeline 中结合 `cudaMemcpyAsync`、`cudaMallocAsync`/`cudaFreeAsync` 和基于 event 的 synchronization，使整个 AI system 在多 GPU 上更接近其实际可达到的性能上限。

第 11 章　Kernel 间 Pipeline、Synchronization 与 CUDA Stream-Ordered Memory Allocation

到目前为止，我们一直聚焦于内核内工具：`cuda::pipeline` 双缓冲、warp 专用化（加载/计算/存储 warp）、持久化内核，以及配合 DSMEM/TMA 的线程块集群——它们让单个内核中的 SM 保持忙碌。本章保留这些内核，说明如何借助 CUDA 流、事件和流有序内存分配器跨内核与批次建立流水线。简言之，第 10 章关注隐藏内核内部延迟。本章说明如何隐藏内核之间以及 GPU 与主机之间的延迟。

这种 inter-kernel concurrency 对真实 workload 中持续利用 GPU 各类 engine 至关重要。要在现代 GPU 上获得较高利用率，需要让 GPU 的 compute engine 与 direct memory access（DMA）engine 尽可能并行工作。

CUDA stream 是这种 inter-kernel concurrency 的基础。组合 asynchronous memory operation、细粒度 synchronization 与 CUDA Graph（本章简要介绍、下一章详细讨论），可以构建减少 host-side stall 的高效 pipeline。

使用 CUDA Stream 重叠 Kernel Execution

CUDA stream 是一系列操作（kernel launch、memory copy 和 memory allocation），这些操作按发出顺序在该 stream 中执行。如图 11-1所示，可以从 CPU 使用两个 stream 向 GPU 发出 5 个 kernel。

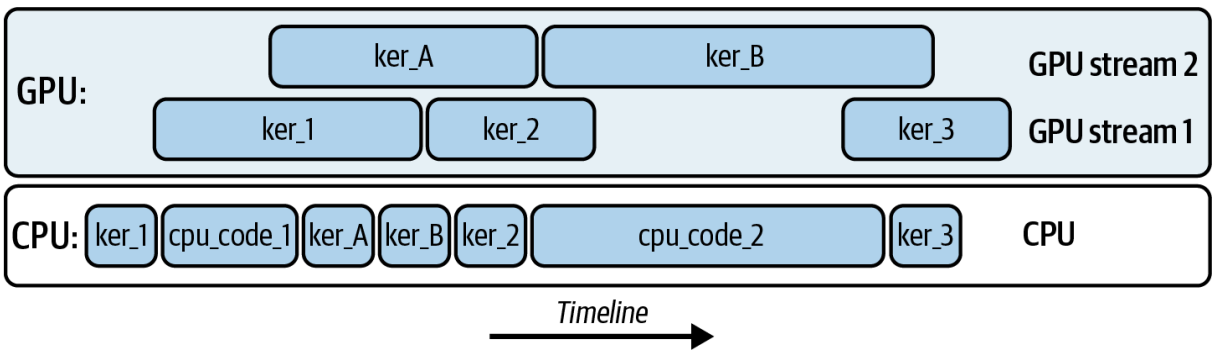


图 11-1　从 CPU 向 GPU 上运行的两个 stream 发出五个 kernel

这里，`ker_A` 和 `ker_B` 在 stream 2 上运行，而 `ker_1`、`ker_2` 和 `ker_3` 在 stream 1 上运行。只要硬件资源允许，kernel 之间以及不同 CUDA stream 之间都可能发生 overlap。

当 stream 异步执行 kernel operation 时，CPU 可以继续执行工作（`cpu_code_1` 和 `cpu_code_2`）。下面是使用两个 CUDA stream 发出这五个 kernel 的代码：

```
#include <cstdio>
#include <cuda_runtime.h>
```

```cpp
__global__ void ker_A()  { /* ... do some work ... */ }
__global__ void ker_B()  { /* ... do some work ... */ }

__global__ void ker_1()  { /* ... do some work ... */ }
__global__ void ker_2()  { /* ... do some work ... */ }
__global__ void ker_3()  { /* ... do some work ... */ }

int main() {
    // 1) Create two CUDA streams
    cudaStream_t stream1, stream2;
    cudaStreamCreateWithFlags(&stream1, cudaStreamNonBlocking);
    cudaStreamCreateWithFlags(&stream2, cudaStreamNonBlocking);

    // 2) Define your grid/block sizes
    dim3 grid(128);
    dim3 block(256);

    // 3) Launch ker_1 on stream1
    ker_1<<<grid, block, 0, stream1>>>();

    // 4) CPU code 1 runs immediately (asynchronously wrt GPU)
    printf("CPU code 1 executing\n");

    // ... do some host-side work here ...
    cpu_code_1();

    // 5) Launch ker_A on stream2
    ker_A<<<grid, block, 0, stream2>>>();

    // 6) Launch ker_B on stream1
    ker_B<<<grid, block, 0, stream1>>>();

    // 7) Launch ker_2 on stream2
    ker_2<<<grid, block, 0, stream2>>>();

    // 8) CPU code 2 runs immediately
    printf("CPU code 2 executing\n");

    // ... do some other host-side work here ...
    cpu_code_2();

    // 9) Launch ker_3 on stream1
    ker_3<<<grid, block, 0, stream1>>>();

    // 10) Wait for work on each stream to finish
    cudaStreamSynchronize(stream1);
    cudaStreamSynchronize(stream2);

    // 11) Clean up
    cudaStreamDestroy(stream1);
    cudaStreamDestroy(stream2);

    return 0;
}
```

`ker_1` 被加入 `stream1` 后，控制权立即返回 CPU。GPU 执行 `ker_1` 时，host 可以运行 `cpu_code_1()`。与此同时，将 `ker_A` 加入 `stream2`、将 `ker_B` 加入 `stream1`；随后把 `ker_2` 加入 `stream2`，穿插执行 `cpu_code_2`，再把 `ker_3` 加入 `stream1`。最后对每个 stream 执行 synchronization，等待其中的工作完成，再销毁 stream 并清理资源。

这个示例展示了五个 kernel execution 如何在两个不同 stream 之间 overlap。稍微增加复杂度，并基于第 10 章的内容，下面展示同一个 warp-specialization pipeline 如何使用 CUDA stream。

```
// Run the warp-specialized kernel in multiple CUDA streams.

#include <cuda_runtime.h>
#include <cuda/pipeline>
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

#define TILE_SIZE 128
#define TILE_ELEMS (TILE_SIZE * TILE_SIZE)

// re-using from Chapter 10
__global__ void warp_specialized_pipeline(const float* __restrict__ A_global,
                                          const float* __restrict__ B_global,
                                          float*       __restrict__ C_global,
                                          int numTiles);

int main() {
    const int NUM_STREAMS = 2;                  // keep it small; tune as needed
    const int batches     = 8;                  // in-flight batches
    const size_t elems    = TILE_ELEMS;         // elements per batch
    const size_t bytes    = elems * sizeof(float);

    // Create streams that do NOT synchronize with the legacy default stream
    cudaStream_t s[NUM_STREAMS];
    for (int i = 0; i < NUM_STREAMS; ++i)
        cudaStreamCreateWithFlags(&s[i], cudaStreamNonBlocking);

    // Allocate pinned host buffers so H2D/D2H can truly overlap
    float *hA = nullptr, *hB = nullptr, *hC = nullptr;
    cudaMallocHost(&hA, batches * bytes);
    cudaMallocHost(&hB, batches * bytes);
    cudaMallocHost(&hC, batches * bytes);

    // ... initialize hA/hB ...

    for (int b = 0; b < batches; ++b) {
        const int sid = b % NUM_STREAMS;

        float *dA = nullptr, *dB = nullptr, *dC = nullptr;
        cudaMallocAsync(&dA, bytes, s[sid]);
        cudaMallocAsync(&dB, bytes, s[sid]);
        cudaMallocAsync(&dC, bytes, s[sid]);

        cudaMemcpyAsync(dA, hA + b * elems, bytes,
                        cudaMemcpyHostToDevice, s[sid]);
        cudaMemcpyAsync(dB, hB + b * elems, bytes,
                        cudaMemcpyHostToDevice, s[sid]);

        const dim3 block(96);    // three warps: loader(0), compute(1), storer(2)
        const dim3 grid(1);
        const size_t shmem = 3 * elems * sizeof(float); // [A|B|C] per tile

        // Reuse the Chapter 10 kernel exactly as-is
        warp_specialized_pipeline<<<grid, block, shmem, s[sid]>>>(dA, dB, dC,
                                                                  /*numTiles=*/1);

        cudaMemcpyAsync(hC+b*elems, dC, bytes, cudaMemcpyDeviceToHost, s[sid]);
```

```
            cudaFreeAsync(dA, s[sid]);
            cudaFreeAsync(dB, s[sid]);
            cudaFreeAsync(dC, s[sid]);
        }

        for (int i = 0; i < NUM_STREAMS; ++i) {
            cudaStreamSynchronize(s[i]);
            cudaStreamDestroy(s[i]);
        }

        cudaFreeHost(hA); cudaFreeHost(hB); cudaFreeHost(hC);
        return 0;
    }
```

这里复用了 warp-specialized pipeline，并展示 stream 如何增加第二层 overlap：当 stream 1 计算 batch n 时，stream 2 可以对 batch n+1 执行 DMA load，同时把 batch n−1 的结果 copy 回 host。kernel 内部的 `cuda::pipeline` overlap 不变。

本章后面会继续加入 thread block cluster 以增加复杂度；但先讨论 stream 如何将 compute 与 data transfer overlap，以巩固 CUDA stream 基础及其在 GPU performance engineering 中的作用。

使用 Stream 重叠 Compute 与 Data Transfer

例如，可以将每个kernel launch 和 memory copy 加入自己的 stream。这样，SM 可以执行 kernel，同时两个专用 DMA engine（一个负责 Host → Device、一个负责 Device → Host transfer）并行搬运数据；具体并发能力取决于设备。

由于 SM compute pipeline 与 DMA engine 相对独立，CUDA stream 可以让 kernel compute 与 data transfer overlap。但如果 kernel 已耗尽 SM 吞吐，或 copy pipeline 已使 memory bandwidth 饱和，继续增加 overlap 不会改善性能。

当 compute 和 memory throughput 饱和时，两个并发 operation 可能各自只达到约 50%，因为它们争用同一资源。应通过 GPU profiling 确认饱和阈值。

例如，AI model training 或 inference workload 可以将工作拆成 batch：在 stream 0 中执行 batch 0 的 kernel，同时让 stream 1 调用 `cudaMemcpyAsync()` 将 batch 1 从 host copy 到 device，如图 11-2所示。

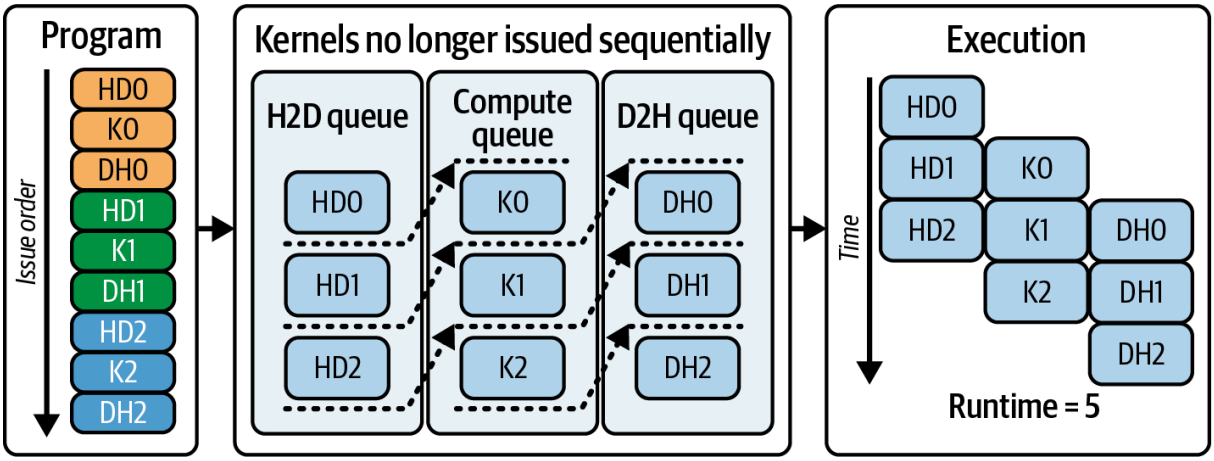


图 11-2　三路 overlap 的时间线

在至少有两个 copy engine 的现代 GPU 上（可查询 `deviceProp.asyncEngineCount()`），可以扩展为三路 overlap：stream 0 运行 batch 0 kernel，stream 1 将 batch 1 从 host copy 到 device，stream 2 将上一 batch 的结果写回 host。还可以使用更多 stream；不过实际并发受 engine、带宽、资源和依赖限制。该模式用 compute 隐藏 data-transfer latency，反之亦然，减少 idle time。

要实现这种并发行为，kernel 和传输必须满足若干条件。首先，异步 transfer 使用的 host pointer 必须是 page-locked（pinned）。如果在 pageable memory 上调用 `cudaMemcpyAsync()`，runtime 会先执行 host-side staging copy 到 pinned memory；在 staging 完成前，调用线程和发起该 operation 的 stream 可能被阻塞。

因此该 transfer 不再具备预期的异步特性。调用线程被阻塞时，GPU 仍可能在其他 stream 中 overlap compute 和 copy，但这一次 transfer 自身不会正确 overlap。若要实现真正异步的 host transfer，应使用 pinned host memory。

> 在 PyTorch DataLoader 中设置 `pin_memory=True`，会将 host buffer page-lock，使 DMA 能直接将数据传入 GPU memory。这样 copy 可能与 compute overlap，并立即把控制权返回 host。Pageable memory 会触发隐藏 staging copy，破坏该 transfer 的 overlap。

其次，应优先评估异步 allocation/deallocation routine：`cudaMallocAsync()` 和 `cudaFreeAsync()`，而不是同步且可能阻塞的 `cudaMalloc()` 或 `cudaFree()`。PyTorch 等 framework 提供使用 CUDA asynchronous、stream-ordered memory allocator 的选项。目标是在 allocation 时不阻塞所有 active stream。

异步 stream-ordered allocator 允许每个 stream 请求或归还 device memory，而不必等待其他 stream。这样，一个 stream 中的 memory operation 通常不会阻塞其他 stream，并可减少不必要的 global synchronization。下一节将详细讨论该 allocator。

PyTorch 默认 CUDA caching allocator 具有 stream awareness；正常从缓存满足 allocation 时，通常不会执行 device-wide synchronization。只有需要通过 `cudaMalloc` 向系统申请更多 memory 时，才可能出现额外同步。因此大多数 tensor allocation/free

不会阻塞其他 stream；启用 `cudaMallocAsync` backend 在许多 workload 中还可能减少 fragmentation、改善复用。

流有序内存分配器

在启动 PyTorch script 前设置环境变量 `PYTORCH_ALLOC_CONF=backend:cudaMallocAsync`，即可请求使用 CUDA 的 stream-orderedallocator。设置后，PyTorch tensor memory 的 allocation（`cudaMallocAsync()`）和 free（`cudaFreeAsync()`）会按调用顺序加入相应 CUDA stream；未设置时，PyTorch 使用自身的 caching allocator。

如果使用 legacy `cudaMalloc(...)`，要注意它可能是返回前执行 device-wide synchronization 的 blocking operation，从而阻塞其他 stream 的工作。具体同步行为取决于 CUDA runtime 和 allocator 状态，但不应假设 allocation 一定与其他 stream 完全并行。

相比之下，使用 stream-ordered allocator 时，`cudaMallocAsync(...)` 会将 allocation request 记录到将使用该 memory 的同一 CUDA stream 中，无论该 stream 正在执行 kernel 还是 memory operation。它不会按 API 设计主动对其他 stream 做全局串行化，但资源池、依赖和回收仍可能影响实际并发。

> PyTorch 使用的 CUDA stream-ordered allocator 可减少不必要的 device-wide lock/synchronization 和 allocation overhead；实际效果取决于 backend 与 memory pool。

例如，stream 0 执行 batch N 的 attention kernel，stream 1 将 batch N+1 从 host copy 到 device，stream 2 为 batch N+2 加入 `cudaMallocAsync(...)`。由于 allocation request 按 stream order 进入 stream 2 的队列，stream 0 和 stream 1 通常可以继续执行；仍需考虑 memory pool 和硬件资源。

使用 stream-aware allocator 时，每个 mini-batch 的 GPU memory 可以按 stream order allocation，而不必主动阻塞其他 stream。这对按 batch 分配 scratch space 的 LLM pipeline 很重要；在 heavy memory churn 下，异步 allocator 可能减少 stall，但不能保证完全没有阻塞。

> 如果 pipeline 为每个 mini-batch 分配 scratch buffer（LLM training 和 inference 中很常见），stream-ordered memory allocator 尤其有用。例如，每个 mini-batch 可能需要独立 temporary workspace 来保存 attention key/value 或 intermediate activation buffer；此时可按 stream order 请求 GPU 上的 scratch buffer。

Allocation 通常从 per-device memory pool 满足。可以使用 `cudaMemPoolSetAttribute()` 调整 pool 的 release threshold，在归还 memory 给 OS 与保留 memory 以便复用之间权衡。较高 threshold 会让 pool 更久地保留已分配

memory，减少归还 OS 的次数和重复 allocation/deallocation；代价是可能占用更多显存。

下面的示例使用 `cudaMallocAsync`、`cudaFreeAsync` 和 `cudaMemPoolSetAttribute()` 实现基于 stream 的 overlap，展示如何通过 CUDA stream 将 memory allocation、data transfer 和 kernel execution 组织成 pipeline：

```
// initialize the async memory allocator
cudaMemPool_t pool;
int device = -1;
cudaGetDevice(&device); // Current device
cudaDeviceGetDefaultMemPool(&pool, device);

// Desired number of bytes to keep in pool before
// releasing back to the OS (tune as needed)
uint64_t threshold =/* e.g., prop.totalGlobalMem / 2 */; // bytes

cudaMemPoolSetAttribute(pool,
  cudaMemPoolAttrReleaseThreshold, &threshold);

cudaStream_t stream1, stream2;
cudaStreamCreateWithFlags(&stream1, cudaStreamNonBlocking);
cudaStreamCreateWithFlags(&stream2, cudaStreamNonBlocking);

// Allocate memory using stream-ordered async allocation
void *d_data1, *d_result1;
void *d_data2, *d_result2;

size_t dataSizeBytes = N * sizeof(float);

// Use cudaMallocAsync as a best practice in modern multi-stream apps
cudaMallocAsync(&d_data1, dataSizeBytes, stream1);
cudaMallocAsync(&d_result1, dataSizeBytes, stream1);
cudaMallocAsync(&d_data2, dataSizeBytes, stream2);
cudaMallocAsync(&d_result2, dataSizeBytes, stream2);

// Asynchronously copy first chunk and launch its kernel in stream1
cudaMemcpyAsync(d_data1, h_data1, dataSizeBytes,
  cudaMemcpyHostToDevice, stream1);
computeKernel<<<gridDim, blockDim, 0,
  stream1>>>((float*)d_data1, (float*)d_result1);
cudaMemcpyAsync(h_result1, d_result1, dataSizeBytes,
  cudaMemcpyDeviceToHost, stream1);

// In parallel, do the same on stream2
cudaMemcpyAsync(d_data2, h_data2, dataSizeBytes,
                cudaMemcpyHostToDevice, stream2);
computeKernel<<<gridDim, blockDim, 0,
  stream2>>>((float*)d_data2, (float*)d_result2);
cudaMemcpyAsync(h_result2, d_result2, dataSizeBytes,
   cudaMemcpyDeviceToHost, stream2);

// Wait for both streams to finish
cudaStreamSynchronize(stream1);
cudaStreamSynchronize(stream2);

// Cleanup
cudaFreeAsync(d_data1, stream1);
cudaFreeAsync(d_result1, stream1);
cudaFreeAsync(d_data2, stream2);
cudaFreeAsync(d_result2, stream2);
```

```
cudaStreamDestroy(stream1);
cudaStreamDestroy(stream2);
```

这里创建两个 CUDA stream（`stream1` 和 `stream2`），并使用 `cudaMallocAsync` 分配 device memory；allocation request 按各自 stream 排序。随后为两个独立数据 chunk 发出工作。

在 `stream1` 上，先执行 host 到 device 的异步 copy（`H2D`），再启动 compute kernel，最后把结果从 device 异步 copy 回 host（`D2H`）。同时，在 `stream2` 上对第二个数据 chunk 执行同样流程。

由于这些 operation 被加入不同 stream，GPU 可能在它们之间执行 overlap：`stream1` 的 kernel 可与 `stream2` 的 H2D copy 并发；`stream1` kernel 完成后，D2H copy 还可能与 `stream2` kernel overlap，前提是 copy engine、带宽和资源允许。

示例利用 CUDA stream 和 stream-ordered allocation，使 memory allocation 与 kernel computation 交错进行。错峰调度可能减少 idle time；如果不使用 stream-ordered allocation，可能需要预先分配所有 memory（增加 footprint），或承担额外 synchronization 开销。

使用 `cudaMallocAsync`，memory management 可以集成到 CUDA stream 中，使 per-stream allocation/deallocation 不必主动触发 global device synchronization；实际依赖和 pool 行为仍需检查。

此外，stream-ordered allocator 支持为 variable-length buffer（例如 token cache 或 intermediate activation）发出细粒度 memory request。随后可以在同一 stream 中启动依赖这些 buffer 的 kernel，利用 stream order 表达先后关系。

> 要获得较高 throughput，需要调节 data-chunk size，并遵守 GPU 的 concurrency limit。现代 GPU 可能提供多个 copy engine，可重叠 H2D 与 D2H transfer；应查询 `deviceProp.asyncEngineCount`，再据此规划 overlap。

GPU 对整个 device 上可并发执行的 kernel 数量有硬件与资源限制。具体上限取决于架构、driver、kernel 资源和调度；超过 active-kernel capacity 后，额外 launch 会排队等待资源槽位，如第 5 章所述。

共享 SM 的 kernel 只有在合计 register、shared memory 和 thread block 需求符合 SM 资源限制时，才可能同时执行。因此必须平衡 chunk（tile）size、launch order 和每个 kernel 的资源使用。

chunk 太小会使 copy engine 和 SM 资源利用不足；chunk 太大或同时加入太多 kernel，则可能超过 kernel slot 或耗尽 per-SM resource，导致 stall。

总之，经过正确调优后，CUDA stream 与 stream-ordered memory allocator（`cudaMallocAsync`）可以帮助 data transfer、kernel execution 和 memory management 交错执行，减少不必要排队；是否能让多个 DMA engine 和 SM 持续繁忙，仍需由目标 workload 的 profile 验证。

在 LLM 中使用 CUDA Stream 与 Stream-Ordered Memory Allocator

CUDA stream 的 nonblocking behavior与 stream-ordered memory allocator 结合，对 LLM training 和 inference workload 很重要。这些 workload 可以在多个 stream 间重叠 computation 与 data movement，以提高 GPU 利用率并减少端到端延迟。

LLM 还经常动态分配 scratch memory，这正是上一节介绍的 stream-ordered memory allocator 所支持的场景。例如运行 transformer layer 时，可能需要额外的 shared memory 或 device memory（称为 *scratch memory*），暂存矩阵乘法结果，再将其交给 softmax。

由于 LLM workload 的不同 mini-batch 可能有不同长度（token count），可以使用 stream-ordered memory allocator 为每个 input batch 在 GPU 上分配适合其大小的 scratch buffer，从而减少过度分配；实际复用策略还需考虑 memory pool 和生命周期。

如果使用旧式 blocking allocation API（`cudaMalloc(...)` 与 `cudaFree(...)`）动态分配 scratch buffer，每次 allocation/deallocation 都可能与整个 GPU 同步；如果调用路径触发 global device synchronization，pending kernel 和 copy 完成前就无法实现预期 overlap。

> Global device synchronization 可能严重破坏性能。尽量避免在 CUDA stream 中使用 blocking call，例如 `cudaMalloc()` 和 `cudaFree()`；优先使用 event 和 stream wait。也不要在不必要时对 default stream 0 调用 `cudaStreamSynchronize(0)`。

在一个 stream 为 batch *N* 执行 attention kernel、另一个 stream 准备 batch N+1 的 pipeline 中，如果第二个 stream 调用 blocking `cudaMalloc(...)` 并触发设备级同步，就可能阻塞其他 stream；这会破坏 data transfer、computation 与 memory management 之间的 overlap。

可使用 stream-ordered allocator 的 `cudaMallocAsync()` 与 `cudaFreeAsync()`。这些 API 将 device-memory allocation/free work 加入指定 stream，按 stream order 管理；它们不应被理解为自动消除所有依赖，实际同步仍由 stream、memory pool 和生命周期决定。

例如，某个 stream 需要为一批输入的 attention 分配 16 MB scratch buffer，可以调用 `cudaMallocAsync(&scratchPtr, scratchBytes, stream1)`。该请求被记录到 stream1 的 operation queue，通常不会主动要求其他 stream 等待，如图 11-3所示。

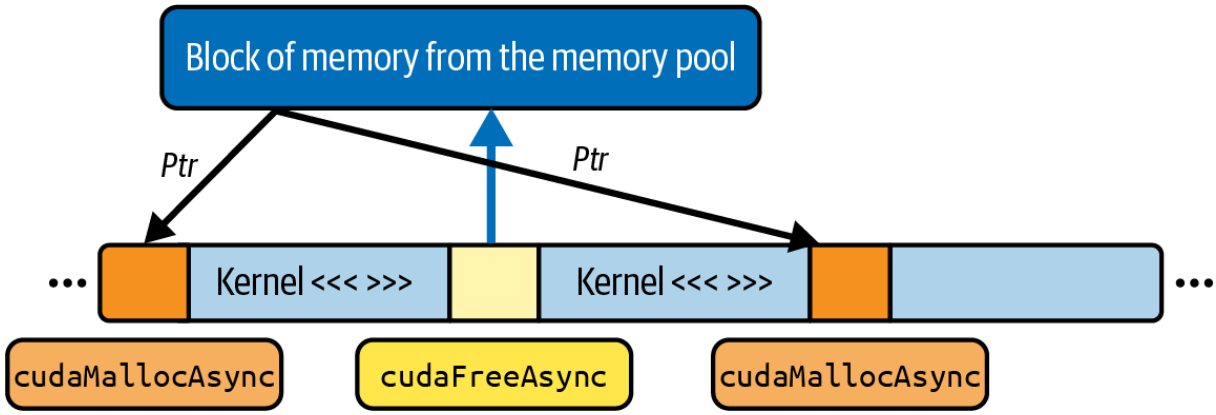


图 11-3　Stream-ordered memory allocation

即使 stream1 的 allocation 正在处理，其他 stream 仍可继续 launch kernel、copy data 或执行自身工作。当 CUDA runtime 完成 memory reservation 后，stream1 可继续推进并在新区域启动 attention kernel；实际行为还受 pool、资源和依赖影响。

> 与 legacy `cudaMalloc` 不同，`cudaMallocAsync` 按 stream order 记录 allocation；它通常不会主动 stall 其他 stream，但 allocation 的依赖和内存池压力仍可能产生等待。

在 LLM training 和 inference 中，variable-length sequence 常导致 scratch-buffer size 波动。如果 batch *N* 每条 sequence 有 512 个 token，而 batch N+1 有 1,024 个 token，attention module 对后者需要更多空间，复用前一个 batch 的 allocation 可能不够。使用 `cudaMallocAsync()` 可以按 stream order 请求更大的 buffer，而不必主动停止其他 stream。

典型 LLM 的自回归逐 token 生成阶段（即 *decoding*）会使用不断增长的 key/value cache。每生成一个 token，就要向 per-sequence buffer 追加新的 KV pair；buffer 增长后，可能需要重新分配或扩展 scratch region。`cudaMallocAsync(...)` 允许在运行 attention kernel 的同一 stream 中安排该请求；上游 data loading 和下游 result copying 则可在各自 stream 中继续推进，前提是依赖关系正确。

CUDA stream 在 LLM 中的另一个用途是大型模型的 layerwise pipeline。假设把 transformer-based LLM 分成两半，分别在不同 CUDA stream 运行：stream 0 执行 layer 0–5，stream 1 执行 layer 6–11；两半之间需要用于 activation 的 intermediate buffer。

每当 stream 0 完成一个 mini-batch 后，它可以为下一 batch 的 activation 安排 `cudaMallocAsync(...)`。只要没有额外依赖阻塞，stream 1 可以继续对前一 batch 的结果计算 layer 6–11，同时 stream 0 为下一 batch 的 input 准备 memory。

相反，如果在该 pipeline 中使用 legacy `cudaMalloc(...)`，每次为下一 mini-batch 或扩展 KV cache 分配 scratch region 时，都可能触发 device-wide synchronization，破坏跨 stream 的 computation/data movement overlap。

总结来说，LLM 经常需要用于 attention、layer normalization、softmax、KV cache 或 intermediate activation 的临时 buffer，这些统称为 *scratch buffer*。

在不同 stream 中使用 `cudaMallocAsync(...)` 与 `cudaFreeAsync(...)` 管理 scratch buffer，可以把 allocation/free 纳入对应 stream order，减少因 memory management 导致的 global cross-stream stall；仍应显式处理 buffer 的使用与释放依赖。

这样，其他 stream 可以继续运行，attention kernel、data transfer 和 host-side work 得以尽可能 overlap；大型实时 LLM workload 的 GPU 利用率是否提高，需要通过端到端 profile 验证。

旧式默认流

如果没有显式创建或指定stream，operation 会进入 legacy default stream，通常称为 *stream 0*。在 legacy default-stream 语义下，它有两个重要行为：

与自身隐式同步

> 加入 stream 0 的 operation 严格按序执行。在同一 stream 0 中，两个 kernel 或一个 copy 与一个 kernel 不能彼此 overlap，因为 stream 会序列化自身 command。

与其他 stream 隐式同步

> 在 legacy default-stream 模型中，加入 stream 0 的 operation 会等待其他 blocking stream 中此前已加入的工作完成；反过来，nondefault blocking stream 中的 operation 也会受此前 stream 0 工作约束。stream 0 因而形成隐式的 device-wide ordering point。使用 `cudaStreamNonBlocking` 创建的 stream 不参与这类 legacy default-stream 隐式同步。

由于同一 stream 内的严格顺序，把所有工作都放入 stream 0 会阻止这些 operation 彼此并发；例如，同一 stream 中的 kernel 与 copy 不会 overlap。隐式跨 stream 依赖还可能使 GPU 等待 default-stream ordering point。

要获得可控并行性，除非确实需要与 legacy blocking stream 建立隐式顺序，否则应避免在 performance-critical 路径依赖 stream 0，优先使用显式 stream 和 event dependency。

现代每线程默认流

为缓解 legacy default stream 的“global barrier”行为，CUDA 引入了 per-thread default stream，简称 PTDS。

在 per-thread default-stream 语义下，每个 CPU thread 的 default stream 相互独立。启用 PTDS 后，每个 host thread 都有自己的隐式 default stream。

加入 thread A default stream 的 operation 不会因 PTDS 语义自动等待 thread B default stream 的工作；硬件资源允许时可以并发。反向关系也相同。

PTDS 广泛用于 multithreaded CUDA application，以避免 host thread 之间的隐式 default-stream barrier。可以使用 `nvcc --default-stream per-thread` 编译，或在包含

CUDA header 之前定义 `CUDA_API_PER_THREAD_DEFAULT_STREAM=1`；后者是编译期宏语义，不应笼统当作运行时环境变量开关。

> 启用 PTDS 后，每个 host CPU thread 的 default stream 类似普通 stream，不会与其他 thread 的 default stream 隐式同步。不同编译单元或库混用 legacy 与 per-thread 语义容易产生难以发现的 ordering 问题，应统一配置并显式表达依赖。

使用 PTDS 时，没有显式 stream 参数的 kernel launch、copy 或 allocation 会进入 thread-local default queue。只有同一 host thread default stream 内的 command 按序执行，不会仅因 default-stream 语义对其他 thread 的 stream 施加隐式 global barrier。

总之，启用 PTDS 后，legacy default-stream 的跨 thread 隐式 barrier 被移除。不同 CPU thread 在各自 default stream 发出的 kernel launch 或 memory copy，在依赖与硬件资源允许时可能 overlap，如图 11-4所示；PTDS 本身不保证实际并发。

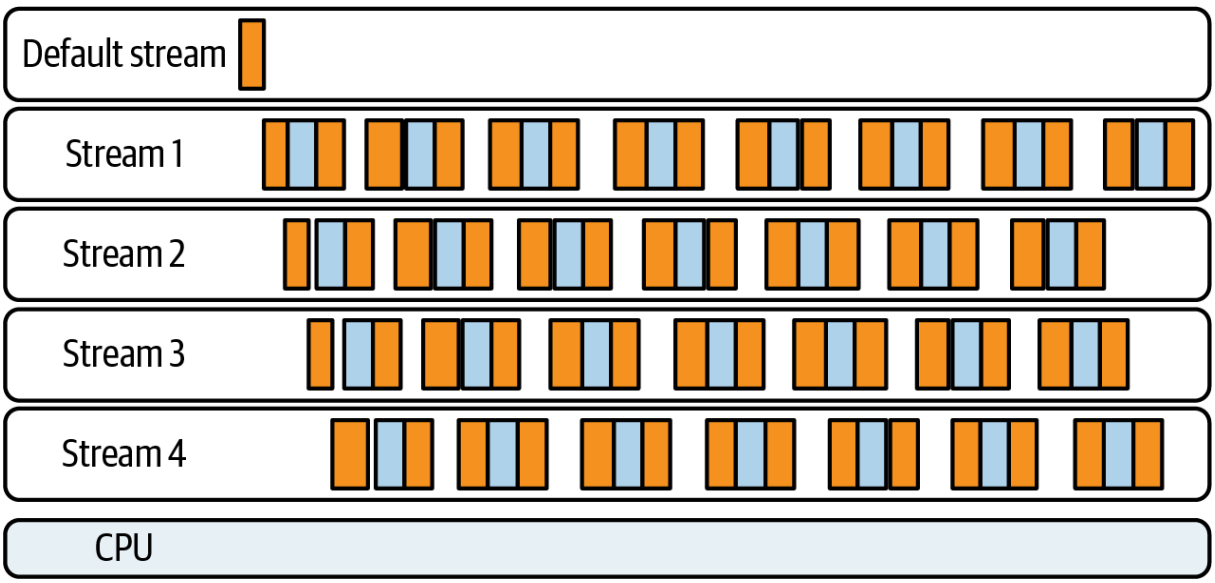


图 11-4　启用 PTDS 后，不同 CPU thread 从各自 default stream 发出的多个 GPU kernel 在独立 CUDA stream 上并发执行的时间线

Default Stream 与显式（Nondefault）Stream

在 performance-critical code 中依赖 default-stream behavior容易造成隐藏依赖。在 legacy 语义下，加入 default stream（stream 0）的工作会与其他 blocking stream 建立隐式顺序；`cudaStreamNonBlocking` stream 是重要例外。

在 performance-critical code 中，最好创建并使用明确命名的 nondefault explicit stream，避免 operation 意外进入 stream 0。在 legacy blocking-stream 模型下，一个误发到 stream 0 的 kernel 或 copy 可能阻塞其他相关 stream。cuBLAS、Thrust 等许多 library 支持显式指定 stream；应按照各 library API 将 handle 或 execution policy 绑定到预期 stream。

PyTorch 的 operation 通常遵循当前 CUDA stream，并不意味着 cuDNN/cuBLAS 调用总是自动使用各自独立 stream。Distributed backend 常在专用 communication stream 上调度 NCCL operation，以便在依赖允许时 overlap gradient communication 与 compute；stream priority 和具体调度由 framework/backend 配置决定。

自行管理 stream，或使用 per-thread default，可以更明确地控制 concurrency。下面是在 CUDA C++ 中创建 explicit nondefault stream 的示例（第 13 章和第 14 章将介绍 PyTorch stream）：

```
cudaStream_t streamA, streamB;
cudaStreamCreateWithFlags(&streamA, cudaStreamNonBlocking);
cudaStreamCreateWithFlags(&streamB, cudaStreamNonBlocking);

myKernel<<<grid, block, 0, streamA>>>(...);                // streamA
cudaMemcpyAsync(dest, src, size, cudaMemcpyHostToDevice, streamB); // streamB
```

这里，`streamA` 和 `streamB` 都以 `cudaStreamNonBlocking` 创建，依赖和资源允许时可 overlap；它们不会因 legacy default-stream 语义而与 stream 0 隐式同步。原文称后续 stream 0 调用会迫使二者等待，并不适用于此处的 nonblocking flag。

如果 explicit stream 使用默认 blocking flag，则 legacy stream 0 的 pending task 可能形成隐式顺序；本示例的 nonblocking stream 不受该规则约束。即便如此，仍建议在 performance-critical 路径使用显式 event 表达依赖，而不是依赖 default-stream 副作用。

总之，多 host-thread application 可考虑启用 PTDS，使各 CPU thread 的 default stream 不再彼此隐式同步；对于 performance-critical kernel 和 copy，则使用 `streamA`、`streamB` 这样的 explicit stream。

这样可以减少与另一个 explicit stream、其他 thread default stream 或 legacy stream 0 的意外隐式同步；但数据 hazard 仍必须用 event、wait 或其他 dependency 明确处理。使用 `cudaStreamNonBlocking` 创建 stream，可使其不与 legacy default stream 隐式同步，从而避免这一类隐藏 barrier。

Default Stream 使用最佳实践

Default stream 可能带来性能问题，下面概括 legacy default、per-thread default 和 explicit（nondefault）stream 的同步特征：

Legacy default stream (`cudaStreamLegacy`)

> 它与 legacy blocking stream 之间存在双向隐式顺序，但不与 `cudaStreamNonBlocking` stream 隐式同步。若需要可控 concurrency，不要依赖该隐式行为。

Per-thread default stream (`cudaStreamPerThread`)

每个 host thread 的 default stream 是独立的；它仍会序列化自身 command，但不会仅因 PTDS 语义等待或阻塞其他 thread 的 default stream。

Explicit streams (created with `cudaStreamCreateWithFlags()`*)*

Explicit stream 是独立 queue，可以通过 `cudaStreamWaitEvent()` 等方式插入显式依赖；blocking explicit stream 仍可能参与 legacy default-stream 隐式同步。

使用 default 与 explicit（nondefault）stream 时，可遵循以下建议：

除非确实需要 legacy ordering，否则不要把 performance-critical kernel 发到 stream 0

误发到 stream 0 的 kernel 或 copy 可能使 legacy blocking stream 等待。某些没有显式 stream 参数的 API 会使用 default stream，因此应查阅具体 API，并在可用时明确指定 stream。

按需启用 per-thread default stream

如果 application 有多个 CPU thread 分别加入 GPU work，可以使用 PTDS，避免这些 thread default stream 之间的隐藏 barrier。也可以用 `cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking)` 创建不与 legacy stream 0 隐式同步的 explicit stream。

创建并管理 explicit nondefault stream

Explicit stream 便于表达 kernel 与 memory copy 的 overlap。可向 `<<<...>>>()`、`cudaMemcpyAsync()` 或 `cudaMallocAsync()` 传入 nondefault `cudaStream_t`；如果还需避免 legacy stream 0 的隐式关系，应使用适当的 nonblocking flag。

Use `cudaStreamWaitEvent()` *to coordinate fine‑grained dependencies instead of* `cudaStreamSynchronize()`

细粒度 dependency 应优先使用 stream event，而不是在 default stream 上调用 `cudaStreamSynchronize()`。只在确实需要 host 等待某个 stream 完成的明确边界调用 `cudaStreamSynchronize()`；它只等待指定 stream，但会阻塞调用 host thread。

明确配置 stream mode 与 flag

PTDS 通过编译选项或在包含 CUDA header 前定义宏配置，不是通过 device flag 在运行时切换。创建 explicit stream 时还应明确选择 blocking 或 `cudaStreamNonBlocking`；“任何 stream 0 调用都会形成 global barrier”并不适用于 nonblocking stream。

简而言之，legacy default stream（stream 0）会与其他 blocking stream 形成隐式 ordering point；这并不等同于对所有 `cudaStreamNonBlocking` stream 都建立 barrier。为避免不可见 stall，performance-critical kernel 和 copy 应使用显式命名 stream，例如通过 `cudaStreamCreateWithFlags` 创建，并依据数据依赖插入 event/wait。

如果程序有多个 CPU thread 分别发出 GPU work，可以启用 PTDS，使各 thread 的 default stream 相互独立；同时应统一编译配置，避免 legacy 与 PTDS 语义混用。

需要 overlap 的两个 operation（例如 copy 和 kernel）最好分别加入明确的 nondefault stream，并确保 host memory、copy engine、SM 资源和 event dependency 都满足条件。这样可避免依赖 stream 0 的隐藏 ordering，并让 GPU 在资源允许时获得可控并行性。

使用 Event 与 Callback 进行细粒度 Synchronization

即使多个 stream 和DMA engine 正在 overlap，某个 stream 的 operation 仍可能必须等待另一个 stream。考虑 producer-consumer stream pair：producer stream 负责加载和准备数据，consumer stream 负责处理。

此时容易想到在 host 侧调用 `cudaDeviceSynchronize()`。但它会让调用 CPU thread 等待 device 上所有 stream 和 operation 完成，通常会扩大等待范围。也可以调用 `cudaStreamSynchronize()`，如图 11-5所示；它会阻塞调用 host thread，直到指定 stream queue 中的 operation 完成。

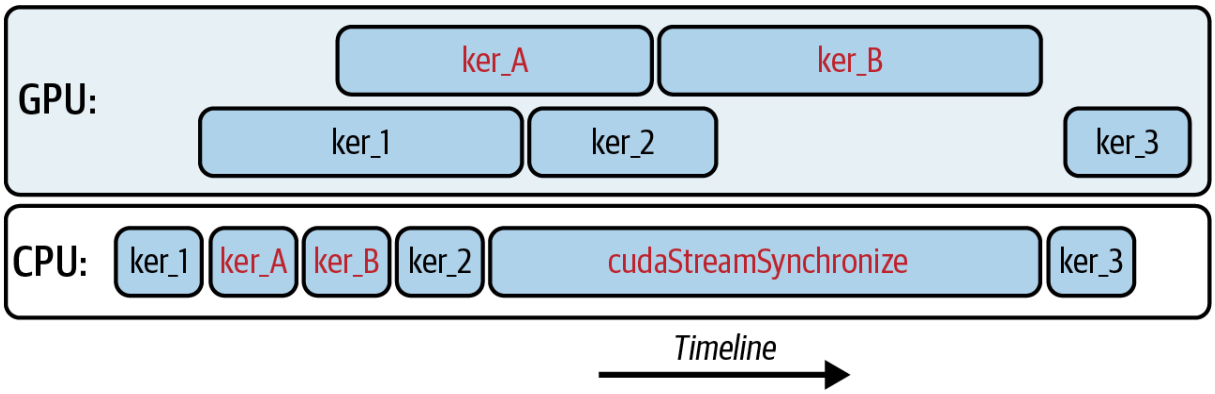


图 11-5　使用 `cudaStreamSynchronize()` 会让 CPU 等待指定 stream 中的 operation 完成

更合适的方式是使用 CUDA event 为 stream 提供细粒度 synchronization：在产生数据的 stream 中记录 `cudaEvent_t`，再让 consumer stream 等待该 event。

例如，在 producer stream 中启动 producer kernel；数据准备好后记录 event（代码中对应 `cudaEventRecord(doneEvent, stream0)`）。consumer stream 在启动 consumer kernel 前调用 `cudaStreamWaitEvent(stream1, doneEvent, 0)`。这样只有 consumer stream 在 event 完成前等待，host thread 和其他无关 stream 可以继续执行，如图 11-6 所示。

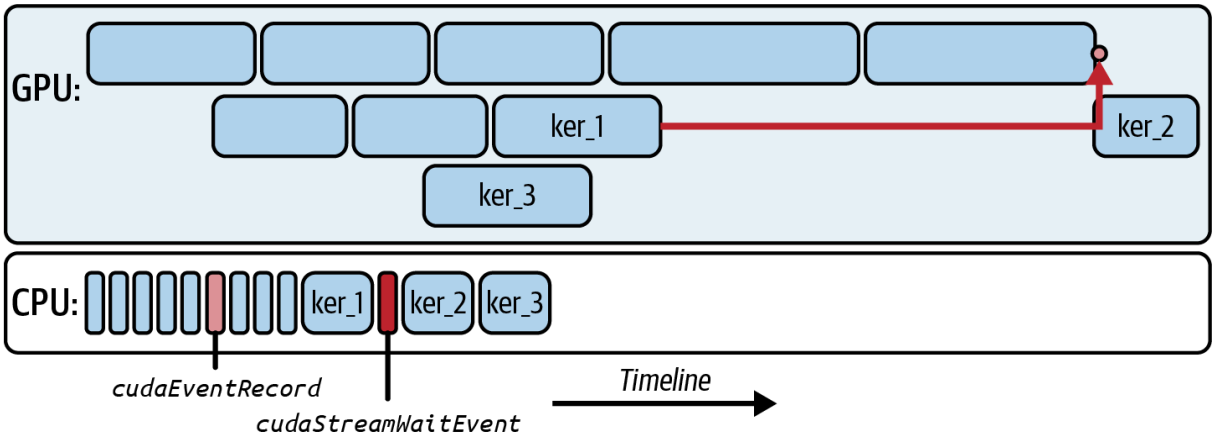


图 11-6　使用 CUDA event 进行细粒度 synchronization

除了用于 interstream coordination，CUDA event 也可用于 GPU device 与 CPU host 的通信。可以从 host 使用 `cudaLaunchHostFunc()` 注册 host callback。

假设 GPU kernel 在某 stream 完成后，需要立即在 host 回收 custom memory pool。可以从 host 使用 `cudaLaunchHostFunc()` 注册 callback，并将其加入该 stream；随后启动 kernel。

kernel 完成并到达 callback 前置位置后，CUDA runtime 会在 host thread 执行 callback，由 CPU 更新 memory pool；不需要 polling loop，也不必调用 full `cudaDeviceSynchronize()`。

GPU work 完成并满足 stream order 后，CUDA runtime 会在 host thread 执行 callback。这样 CPU 不必持续轮询，也不会为无关 GPU work 添加全局等待。

不要在 `cudaLaunchHostFunc()` 启动的 host callback 内调用需要 device progress 的 CUDA API。例如在 callback 内调用 `cudaMemcpy()` 可能造成 deadlock：callback 运行于 CUDA runtime 管理的 host thread，而 device 或 stream 可能正等待 callback 返回。

> callback 最好限制为纯 CPU-side task，例如释放或回收 host memory。这样可避免 callback 试图启动新的 GPU work，而 GPU 同时等待 callback 完成所造成的循环依赖。

使用 CUDA Event 进行 Cross-Stream Synchronization

当多个 stream 并行运行时，经常需要在它们之间协调和同步。CUDA event 是一种主要的 cross-stream synchronization 机制，能将依赖范围限制在相关 stream，而无需让 host thread 或整个 device 等待。

Event 类似 stream 在 command queue 特定位置记录的 marker。其他 stream 或 host 可以等待该 marker，以判断某个 event（例如 kernel 完成）是否发生。与等待所有 stream 的 `cudaDeviceSynchronize()` 不同，event 允许在 stream 之间表达细粒度 ordering。

可以用 stream/event 确保 transformer layer 所需数据在 compute 前已准备好；也可以在 pipeline parallelism 中确保 GPU 0 生产 tensor 后 GPU 1 再消费它，同时不影响无关工作。

例如，四个 stream（`Streams A-D`）组成 pipeline，部分 operation 之间存在依赖。可以使用 CUDA event 强制这些依赖，如图 11-7所示。

为确保 stream B 只有在 stream A 生产数据后才开始处理，可以在 stream A 中记录 event，让 stream B 在继续前等待它；类似地，stream D 可以等待 stream B。通过串联 event，可以保持跨 stream 的正确 ordering，同时并发处理不同 batch。

深度学习 framework 经常使用这种 event-based synchronization，将 gradient computation 与 all-reduce communication overlap。compute stream 在 gradient ready 时记录 event，communication stream 等待该 event 后启动 NCCL operation，从而与剩余 computation overlap。

总之，CUDA event 是面向 device signaling 的轻量机制。stream 到达 command queue 中的特定位置时记录 event，其他 stream 可以高效等待它。这样可以在不强制 global wait 的情况下编排复杂依赖图，并支持 multistream LLM workload 的 pipeline execution。

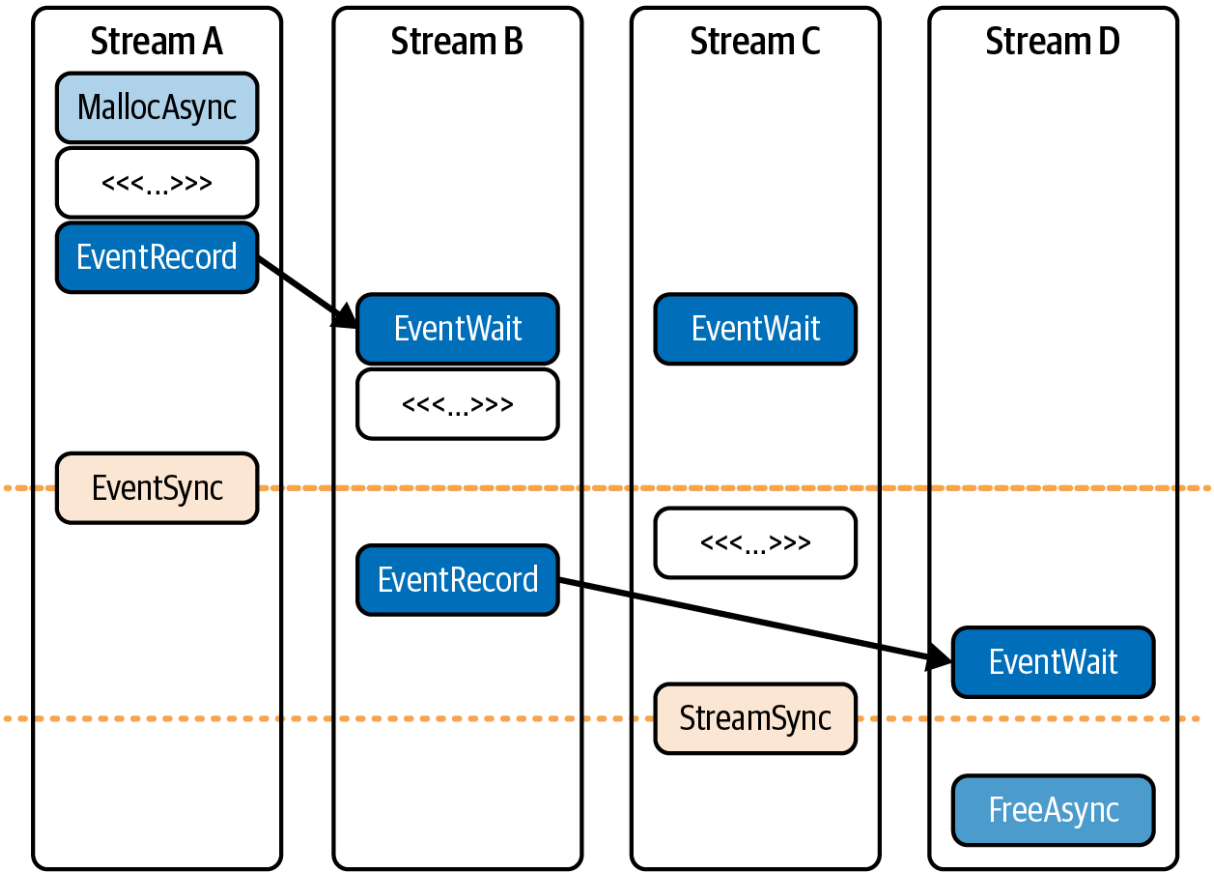


图 11-7　使用 event 同步 CUDA stream（改编自 NVIDIA 的stream-ordered memory allocator 概述）

NVIDIA 持续改进 event timestamp 的时间粒度并降低记录开销。因此，CUDA event 可用于粗粒度 kernel timing 和 profiling：它们在 GPU timeline 中记录 timestamp，可测量经过时间；精确分析仍应结合 Nsight 等 profiler。

使用 Warp Specialization（Intra-Kernel）与 CUDA Stream（Inter-Kernel）构建 Pipeline

第 10 章演示了如何使用 warp 专门化与 CUDA Pipeline API，在单个内核内隐藏内存延迟。该节启动了一个持久化且采用 warp 专门化的内核：每个线程块被拆分为三类 warp，分别负责加载、计算和存储。这些 warp 共享一块连续共享内存及一个两阶段（双缓冲）CUDA Pipeline（`<cuda::pipeline>`）对象来协调工作。

现在保留同一个 warp-specialized device kernel，通过不同 CUDA stream 驱动多个 batch，形成两层 pipeline：

Intra-kernel overlap（每个 block 内）

loader ↔ compute ↔ storer 使用各自 pipeline，在依赖和资源允许时 overlap。

Inter-kernel overlap（跨 batch）

当 stream 0 计算 batch *t* 时，stream 1 可以 DMA-load batch t+1，stream 2 回传 batch t−1 的结果。实际 overlap 需要适当的 stream、pinned host memory、copy engine 和依赖管理；stream-ordered allocator 可减少 allocation/copy 对其他 work 的串行化。

如果多个 block 需要同一 tile，可以使用第 10 章介绍的 thread block cluster + DSMEM 路径减少重复 global load。但 cooperative/cluster launch 受并发驻留约束，可能影响其他 cooperative kernel；这里的 stream pattern 主要用于 host ↔ device overlap，也适用于 noncooperative kernel。

如果瓶颈是 device 内 tile reuse，可评估 thread block cluster；如果瓶颈是 host ↔ device communication 和 batching，可评估 CUDA stream。二者可以组合，但最终应由 profile 确认瓶颈和收益。

下一个 kernel 每个 block 使用三个 warp（0 = loader，1 = compute，2 = storer）和两个 pipeline 进行 stage ping-pong，因此需要 `6 * TILE_SIZE * sizeof(float)` dynamic shared memory（`[A0|B0|C0|A1|B1|C1]`）。这里使用两个独立的 block-scope pipeline，每个深度为 2：一个用于 loader → compute（`pipe_lc`）handoff，另一个用于 compute → storer（`pipe_cs`）handoff。

此外，使用 double-buffered shared memory，使 kernel 能在计算 tile *i*、存储 tile i−1 的同时加载 tile i+1。因此后续代码中会看到两个 `cuda::pipeline` object，以及 shared

memory 中的 “[A|B|C] × 2 stages”。这两个 pipeline 是 kernel 内部加深 overlap 的设计选择；两种 kernel style 都可以与 CUDA stream 结合。代码如下：

```
#include <cooperative_groups.h>     // thread_block, etc.
#include <cuda/pipeline>            // CUDA Pipeline API
namespace cg = cooperative_groups;

// shmem bytes = 2(stages)×3(buffers)×TILE_SIZE×sizeof(float)=6×TILE_SIZE×4
//             = 6 * 1024 * 4 = 24,576 B (24 KB) << 227 KB SMEM
// We keep this at 1024 (versus going higher) as a safe starting point
// (good occupancy balance)
#define TILE_SIZE 1024  // one tile = 1,024 floats per buffer (1-Dimension)

    // Alignment / size guards for vectorized copies
    static_assert((TILE_SIZE % (32 * 4)) == 0,
                  "TILE_SIZE must be multiple of 128 for float4 vectorization");
    // If you cannot guarantee 16B alignment or sizes, handle
    //  the tail/ragged edges with a fallback 4B loop.

// Three warps per block: 0 loads, 1 computes, 2 stores.
// Two block-scoped pipelines (each depth=2) implement ping-pong across tiles.
__global__ void warp_specialized_two_pipelines(
    const float* __restrict__ A_global,
    const float* __restrict__ B_global,
    float*       __restrict__ C_global,
    int          numTiles)
{
    thread_block cta = this_thread_block();

    // Stage s∈{0,1}: [A_s | B_s | C_s], each length TILE_SIZE
    extern __shared__ float shared_mem[];

    // loader -> compute pipeline (2 in-flight stages)
   using pipe_state = cuda::pipeline_shared_state<cuda::thread_scope_block, 2>;
   __shared__
   alignas(pipe_state) unsigned char state_lc_storage[sizeof(pipe_state)];
   __shared__
   alignas(pipe_state) unsigned char state_cs_storage[sizeof(pipe_state)];
   auto* state_lc = reinterpret_cast<pipe_state*>(state_lc_storage);
   auto* state_cs = reinterpret_cast<pipe_state*>(state_cs_storage);
   if (threadIdx.x == 0) {
       new (state_lc) pipe_state();
       new (state_cs) pipe_state();
   }
   cta.sync();
   auto pipe_lc = cuda::make_pipeline(cta, state_lc);

   // compute -> storer pipeline (2 in-flight stages)
   auto pipe_cs = cuda::make_pipeline(cta, state_cs);

   const int warp_id = threadIdx.x >> 5;               // 0,1,2
   const int lane_id = threadIdx.x & 31;
   auto warp = tiled_partition<32>(cta);

   // Prime first tile handled by this block
   const int first = blockIdx.x;
   if (first > numTiles) {
       const int stage0 = first & 1;
       float* A0 = shared_mem + stage0 * 3 * TILE_SIZE;
       float* B0 = A0 + TILE_SIZE;

       pipe_lc.producer_acquire();
       #pragma unroll
```

```
    for (int chunk = 0; chunk < TILE_SIZE; chunk += 32 * 4) {
        if (warp_id == 0) {
            cuda::memcpy_async(
                warp,
                reinterpret_cast<float4*>(A0 + chunk) + lane_id,
                reinterpret_cast<const float4*>(A_global + size_t(first) *
                  TILE_SIZE + chunk) + lane_id,
                sizeof(float4),
                pipe_lc);
            cuda::memcpy_async(
                warp,
                reinterpret_cast<float4*>(B0 + chunk) + lane_id,
                reinterpret_cast<const float4*>(B_global + size_t(first) *
                  TILE_SIZE + chunk) + lane_id,
                sizeof(float4),
                pipe_lc);
        }
    }
    pipe_lc.producer_commit();
}

// Block-scoped collectives remain CTA-uniform
 pipe_lc.consumer_wait();
 cta.sync();

 // Compute consumes loader output and signals storer.
 if (warp_id == 1) {
     #pragma unroll
     for (int chunk = 0; chunk < TILE_SIZE; chunk += 32) {
         C_buf[chunk+lane_id] = A_buf[chunk+lane_id]
                              + B_buf[chunk+lane_id];
     }
 }
 cta.sync();
 pipe_lc.consumer_release();

 pipe_cs.producer_acquire();
 pipe_cs.producer_commit();
 pipe_cs.consumer_wait();
 cta.sync();

 // Storer waits on compute and writes back.
 if (warp_id == 2) {
     #pragma unroll
     for (int chunk = 0; chunk < TILE_SIZE; chunk += 32) {
         C_global[offset + chunk + lane_id] = C_buf[chunk + lane_id];
     }
 }
 cta.sync();
 pipe_cs.consumer_release();

 // Loader prefetches next tile after the current stage is released.
 const int next = tile + gridDim.x;
 if (next < numTiles) {
     const int next_stage = next & 1;
     float* A_next = shared_mem + next_stage * 3 * TILE_SIZE;
     float* B_next = A_next + TILE_SIZE;
     pipe_lc.producer_acquire();
     if (warp_id == 0) {
         #pragma unroll
         for (int chunk = 0; chunk < TILE_SIZE; chunk += 32 * 4) {
             cuda::memcpy_async(
                 warp,
                 reinterpret_cast<float4*>(A_next + chunk) + lane_id,
```

```
                    reinterpret_cast<const float4*>(A_global +
                        size_t(next) * TILE_SIZE + chunk) + lane_id,
                        sizeof(float4), pipe_lc);
                cuda::memcpy_async(
                    warp,
                    reinterpret_cast<float4*>(B_next + chunk) + lane_id,
                    reinterpret_cast<const float4*>(B_global +
                        size_t(next) * TILE_SIZE + chunk) + lane_id,
                        sizeof(float4), pipe_lc);
            }
        }
        pipe_lc.producer_commit();
    }

    // Launch requirements: blockDim.x = 96 (3 warps), gridDim as needed,
    // dynamic shared memory = 6 * TILE_SIZE * sizeof(float).
}
```

这里有几个值得注意的性能要点。首先，要求 `A0`、`B0`、`A_global + base` 和 `B_global + base` 按 16 字节对齐。如果某条路径未按 16 字节对齐，就在未对齐的前导段和尾段回退到 4 字节循环。

> 每个 lane 传输 16 字节（`float4/int4`）符合现代 GPU 的常见最佳实践，可使一个 warp 达到 128 字节的合并访问，从而有助于降低流水线开销。

接下来假定 `TILE_SIZE` 是 128 的倍数（32 个 lane × 4 个 float）。如果不是，则用标量循环处理尾部。此外，在适用时，可在流水线集体操作中使用分阶段协作拷贝，例如 `cuda::memcpy_async` 配合 `float4`，或使用等价的合并式协作拷贝循环。这样可以保持异步流水线语义，并降低每个 tile 的拷贝开销。

下面是主机端驱动程序：它在非阻塞 stream 间轮转 batch，并使用页锁定主机内存和按 stream 排序的分配器（`cudaMallocAsync` ÷ `cudaFreeAsync`）。它启动前述的 `warp_specialized_two_pipelines` 内核：

```
#include <cstdio>
#include <cuda_runtime.h>

#define TILE_SIZE   1024
#define NUM_STREAMS 2
#define BATCHES     8

// Device kernel
__global__ void warp_specialized_two_pipelines(
    const float* __restrict__ A_global,
    const float* __restrict__ B_global,
    float*       __restrict__ C_global,
    int          numTiles);

int main() {
    // Create nonblocking streams (do NOT use legacy default stream)
    cudaStream_t s[NUM_STREAMS];
    for (int i = 0; i < NUM_STREAMS; ++i)
        cudaStreamCreateWithFlags(&s[i], cudaStreamNonBlocking);
```

```
    // Pinned host buffers so cudaMemcpyAsync truly overlaps
    float *hA = nullptr, *hB = nullptr, *hC = nullptr;
    const size_t bytesPerBatch = TILE_SIZE * sizeof(float);
    cudaMallocHost(&hA, BATCHES * bytesPerBatch);
    cudaMallocHost(&hB, BATCHES * bytesPerBatch);
    cudaMallocHost(&hC, BATCHES * bytesPerBatch);

    // Initialize inputs
    for (int b = 0; b < BATCHES; ++b) {
        for (int i = 0; i < TILE_SIZE; ++i) {
            hA[b * TILE_SIZE + i] = float(i);
            hB[b * TILE_SIZE + i] = 1.0f;
        }
    }

    // Enqueue batches in a round-robin across streams
    for (int b = 0; b < BATCHES; ++b) {
        cudaStream_t st = s[b % NUM_STREAMS];

        float *dA = nullptr, *dB = nullptr, *dC = nullptr;
        cudaMallocAsync(&dA, bytesPerBatch, st);   // stream-ordered allocator
        cudaMallocAsync(&dB, bytesPerBatch, st);
        cudaMallocAsync(&dC, bytesPerBatch, st);

        cudaMemcpyAsync(dA, hA + size_t(b) * TILE_SIZE, bytesPerBatch,
                        cudaMemcpyHostToDevice, st);
        cudaMemcpyAsync(dB, hB + size_t(b) * TILE_SIZE, bytesPerBatch,
                        cudaMemcpyHostToDevice, st);

        const dim3 block(96);         // 3 warps: loader/compute/storer
        const dim3 grid(1);
        const size_t shmem = 6 * TILE_SIZE * sizeof(float); //[A0|B0|C0|A1|B1|C1]

        // Each batch is one tile in this 1-D example
        warp_specialized_two_pipelines<<<grid, block, shmem, st>>>(
            dA, dB, dC, /*numTiles=*/1);

        cudaMemcpyAsync(hC + size_t(b) * TILE_SIZE, dC, bytesPerBatch,
                        cudaMemcpyDeviceToHost, st);

        cudaFreeAsync(dA, st);        // stream-ordered free
        cudaFreeAsync(dB, st);
        cudaFreeAsync(dC, st);
    }

    // Clean up
    for (int i = 0; i < NUM_STREAMS; ++i) {
        cudaStreamSynchronize(s[i]);
        cudaStreamDestroy(s[i]);
    }
    cudaFreeHost(hA); cudaFreeHost(hB); cudaFreeHost(hC);
    return 0;
}
```

这里每个 stream 都承载自己的操作序列：分配 → H2D → 内核 → D2H → 释放。由于分配和拷贝按 stream 顺序入队，且主机缓冲区是页锁定的，GPU 的拷贝引擎就能与另一个 stream 中 SM 的计算重叠。这就是内核间层，它补充了 warp 专门化产生的内核内重叠；后者在第 10 章中介绍过。

具体而言，该示例让共享内存保存两组缓冲区，每组含三个长度为 `TILE_SIZE` 的缓冲区，以便使用一组时准备下一 tile。带有两个阶段的 `<cuda::pipeline>` 为各 tile 提供双缓冲，并强制正确顺序，使加载、计算和存储 warp 能在不同 tile 上重叠。内核在循环前预置第一个 tile，随后在计算 tile `i` 时预取 tile `i + gridDim.x`。

单独来看，这个 warp 专门化内核通过在每个 tile 内重叠三种角色来隐藏内存延迟。不过，单个 warp 专门化内核一次只能处理一批 tile。

为了让 GPU 持续处理多个 batch，并让这些 batch 的 H2D 传输、内核计算和 D2H 传输相互重叠，我们在不同 CUDA stream 中启动这个 warp 专门化内核的多个实例；每个 batch 都使用按 stream 排序的分配器。

这第二层流水线——内核间并发——让连续的 mini-batch 以波次方式流过 GPU。这样，当一个 batch 的内核正在 stream 0 中计算时，另一个 batch 的数据仍可在 stream 1 中传入；与此同时，更早一个 batch 的结果可能正在 stream 2 中传回主机。

实践中，我们选择少量 stream（通常为两个或三个）并在它们之间轮转。每个 stream 使用 `cudaMallocAsync` 分配自己的设备缓冲区，用 `cudaMemcpyAsync` 异步拷贝输入数据，启动 warp 专门化内核处理这些缓冲区，将结果异步拷回主机，最后用 `cudaFreeAsync` 释放缓冲区。

由于这些操作分别在特定 stream 中入队，它们可以与其他 stream 中的同类操作重叠。在配备多个拷贝引擎和按 stream 排序分配器的现代 GPU 上，这种模式有望同时利用芯片的多个部分，从而显著提高利用率；但实际重叠程度受拷贝引擎数量（查询 `cudaDeviceProp::asyncEngineCount`）、带宽和内核占用率限制。

当一个内核中的加载 warp 等待全局内存读取时，下一个 batch 的 H2D 拷贝可以已在拷贝引擎上进行。当上一个 batch 中的存储 warp 正把数据写回全局内存时，另一个 stream 中的分配器可以为下一个 batch 获取内存，而不必强制执行全局同步。

本质上，第 10 章的 warp 专门化示例展示了如何在线程块内重叠加载、计算和存储，让一个内核隐藏内存延迟。本章的多 stream 示例在此基础上进一步展示：让许多分别处理不同 batch 的内核同时进入 GPU，并在整个流水线中重叠主机 ↔ 设备传输、计算以及设备 ↔ 主机传输。

现代 GPU 具备多个拷贝引擎，并支持异步内存分配。结合 Tensor Core，它们可以并行工作，形成由内核内 warp 专门化和内核间 stream 组成的两级流水线策略。对于 LLM 工作负载，这些机制有助于提高硬件利用率，但能否接近峰值仍需根据具体工作负载进行测量和验证。

结合线程块集群与 CUDA Stream 的 Warp 专门化

现在把本章及前面章节的内容组合起来：通过多个 stream 推送多个在途 mini-batch，每个 stream 都启动一个协作式线程块集群 warp 专门化内核。这代表了在最新 GPU 硬件上进行 CUDA 性能优化的一种高级组合。

尽管为了完整性我们介绍这一主题，但需要注意的是，由于组合方式非常复杂，除高度专业化的研究项目和极度关注延迟的推理引擎外，很少会采用全部这些技术。不过，它仍值得学习，因为这个代码示例把目前为止学到的许多概念组合到了一起。

第 10 章展示了单个线程块内的 warp 专门化，以及如何使用 DSMEM 将 warp 专门化与线程块集群结合。这样，一个 leader 线程块只需加载一次 tile，线程块集群中的每个线程块都从这份共享的片上副本进行计算，从而避免重复的全局内存加载。

本示例复用了这一模式：leader 使用 block 作用域流水线分阶段准备拷贝；所有线程块都通过 `cluster.map_shared_rank` 读取数据，因此能够继承数据复用带来的收益。

在前面的 CUDA stream warp 专门化示例中，每个线程块负责自己共享内存区域中的三个流水线阶段——加载、计算和存储。现在将此前的实现扩展为线程块集群流水线，使加载、计算和存储 warp 分布在线程块集群中。这不同于之前局限在单个线程块内的实现。

下面给出线程块集群、warp 专门化与 CUDA stream 结合的示例代码。这里选择 `NUM_STREAMS = 2`，使主机能够在不同 CUDA stream 中排入两个独立的启动操作。我们复用第 10 章“线程块集群中的 warp 专门化”小节里的 `warp_specialized_cluster_pipeline` 实现，并为其加入 CUDA stream：

```
// Warp specialization across a thread block cluster using DSMEM
// and a block scoped pipeline, launched from multiple CUDA streams

#include <cuda_runtime.h>
#include <cuda/pipeline>
#include <cooperative_groups.h>
#include <algorithm>
namespace cg = cooperative_groups;

#define TILE_SIZE   128   // 3×TILE_ELEMS×4 bytes=196,608 bytes < 227 KB SMEM
#define TILE_ELEMS  (TILE_SIZE * TILE_SIZE)
#define NUM_STREAMS 2
#define CLUSTER_BLOCKS 4  // blocks per cluster along x (tune to device)

// ---- Device helpers ----

__device__ void compute_rows_from_ds(const float* __restrict__ A_src,
                                     const float* __restrict__ B_src,
                                     float*       __restrict__ C_dst,
                                     int row_begin, int row_end, int lane_id)
{
    for (int row = row_begin + lane_id; row < row_end; row += warpSize) {
        for (int col = 0; col < TILE_SIZE; ++col) {
            float acc = 0.0f;
            #pragma unroll
            for (int k = 0; k < TILE_SIZE; ++k) {
                acc += A_src[row * TILE_SIZE + k] * B_src[k * TILE_SIZE + col];
            }
            C_dst[row * TILE_SIZE + col] = acc;
        }
    }
}

// Clustered, warp-specialized kernel (leader loads once, others consume DSMEM)
extern "C"
```

```
__global__ void warp_specialized_cluster_pipeline(
    const float* __restrict__ A_global,
    const float* __restrict__ B_global,
    float*       __restrict__ C_global,
    int numTiles)
{
    thread_block cta      = this_thread_block();
    cluster_group cluster = this_cluster();

    extern __shared__ float smem[];
    float* A_tile_local = smem;
    float* B_tile_local = A_tile_local + TILE_ELEMS;
    float* C_tile_local = B_tile_local + TILE_ELEMS;

    // Leader uses a block-scoped pipeline to stage A/B exactly once per tile
    using pipe_state_t = cuda::pipeline_shared_state<cuda::thread_scope_block,1>;
    __shared__
    alignas(pipe_state_t) unsigned char pipe_storage[sizeof(pipe_state_t)];
    auto* pipe_state = reinterpret_cast<pipe_state_t*>(pipe_storage);
    if (threadIdx.x == 0) { new (pipe_state) pipe_state_t(); }
    cta.sync();
    auto pipe = cuda::make_pipeline(cta, pipe_state);

    const int lane_id = threadIdx.x & 31;
    const int warp_id = threadIdx.x >> 5;
    auto warp = tiled_partition<32>(cta);

    const int cluster_rank      = cluster.block_rank();
    const dim3 cluster_dims     = cluster.dim_blocks();
    const int blocks_in_cluster = cluster_dims.x*cluster_dims.y*cluster_dims.z;

    // Each iteration handles one tile per cluster (1-D cluster along x)
    for (int tile = blockIdx.x / cluster_dims.x; tile < numTiles;
         tile += gridDim.x / cluster_dims.x) {
        const size_t offset = static_cast<size_t>(tile) * TILE_ELEMS;

        // Leader block stages A and B once for the entire cluster.
        // Block-scoped pipeline collectives stay CTA-uniform.
        if (cluster_rank == 0) {
            pipe.producer_acquire();
            if (warp_id == 0) {
                cuda::memcpy_async(warp, A_tile_local, A_global + offset,
                                   TILE_ELEMS * sizeof(float), pipe);
                cuda::memcpy_async(warp, B_tile_local, B_global + offset,
                                   TILE_ELEMS * sizeof(float), pipe);
            }
            pipe.producer_commit();
            // Make visible inside leader before publishing
            pipe.consumer_wait();
            pipe.consumer_release();
        }

        // Publish to all blocks in the cluster
        cluster.sync();

        const float* A_src = cluster.map_shared_rank(A_tile_local, 0);
        const float* B_src = cluster.map_shared_rank(B_tile_local, 0);

        // Divide rows among blocks in the cluster
        const int rows_per_block = (TILE_SIZE + blocks_in_cluster - 1)
                                   / blocks_in_cluster;
        const int row_begin = std::min(cluster_rank * rows_per_block, TILE_SIZE);
        const int row_end   = std::min(row_begin + rows_per_block, TILE_SIZE);
```

```
        // Compute warp produces this block's band into local SMEM
        if (warp_id == 1) {
            compute_rows_from_ds(A_src, B_src, C_tile_local,
                                 row_begin, row_end, lane_id);
        }

        // Ensure the storer sees computed rows
        cta.sync();

        // Storer warp writes band back to global
        if (warp_id == 2) {
            for (int row = row_begin + lane_id; row < row_end; row += warpSize){
                for (int col = 0; col < TILE_SIZE; ++col) {
                    C_global[offset + row * TILE_SIZE + col] =
                        C_tile_local[row * TILE_SIZE + col];
                }
            }
        }

        // Done with this tile; allow leader to reuse buffers
        cluster.sync();
    }
    // Dynamic shared memory size: 3 * TILE_ELEMS * sizeof(float)
}

// ---- Host driver: stream-staged batches + clustered kernel launch ----

void launch_warp_specialized_cluster_pipeline_multistream(
    const float* h_A,        // Host input A: length = numBatches * batchLength
    const float* h_B,        // Host input B: length = numBatches * batchLength
    float*       h_C,        // Host output C: length = numBatches * batchLength
    int          batchLength,// elems per batch; must be multiple of TILE_ELEMS
    int          numBatches)
{
    // Basic validation
    if (batchLength % TILE_ELEMS != 0) {
        fprintf(stderr, "batchLength must be a multiple of TILE_ELEMS (%d)\n",
                TILE_ELEMS);
        return;
    }
    const int numTiles = batchLength / TILE_ELEMS;

    // Nonblocking streams avoid legacy default-stream barriers
    cudaStream_t streams[NUM_STREAMS];
    for (int i = 0; i < NUM_STREAMS; ++i)
        cudaStreamCreateWithFlags(&streams[i], cudaStreamNonBlocking);

    // Size and launch geometry
    int device = 0;
    cudaGetDevice(&device);
    cudaDeviceProp prop{};
    cudaGetDeviceProperties(&prop, device);

    // grid.x must be a multiple of CLUSTER_BLOCKS
    const int blocksPerGrid = prop.multiProcessorCount * CLUSTER_BLOCKS;
    const dim3 blockDim(96);  // 3 warps: loader/compute/storer
    const size_t shmemBytes = 3ull * TILE_ELEMS * sizeof(float);

    // Cluster attributes
    cudaLaunchAttribute attr[2]{};
    attr[0].id = cudaLaunchAttributeClusterDimension;
    attr[0].val.clusterDim = dim3(CLUSTER_BLOCKS, 1, 1);
    attr[1].id = cudaLaunchAttributeNonPortableClusterSizeAllowed;
    attr[1].val.nonPortableClusterSizeAllowed = 1;
```

```
    // Enqueue batches round-robin across streams
    for (int b = 0; b < numBatches; ++b) {
        cudaStream_t st = streams[b % NUM_STREAMS];

        float *dA = nullptr, *dB = nullptr, *dC = nullptr;
        const size_t bytes = static_cast<size_t>(batchLength) * sizeof(float);

        // Stream-ordered allocator avoids global sync
        cudaMallocAsync(&dA, bytes, st);
        cudaMallocAsync(&dB, bytes, st);
        cudaMallocAsync(&dC, bytes, st);

        // H2D (host pointers should be pinned for true overlap)
        cudaMemcpyAsync(dA, h_A + static_cast<size_t>(b) * batchLength, bytes,
                        cudaMemcpyHostToDevice, st);
        cudaMemcpyAsync(dB, h_B + static_cast<size_t>(b) * batchLength, bytes,
                        cudaMemcpyHostToDevice, st);

        // Clustered launch config
        void* args[] = { &dA, &dB, &dC, (void*)&numTiles };
        cudaLaunchConfig_t cfg{};
        cfg.gridDim  = dim3(blocksPerGrid);
        cfg.blockDim = blockDim;
        cfg.dynamicSmemBytes = shmemBytes;
        cfg.stream = st;
        cfg.attrs = attr;
        cfg.numAttrs = 2;

        // Launch the clustered kernel (cooperative/cluster launch)
        cudaKernel_t k;
        cudaGetKernel(&k, warp_specialized_cluster_pipeline);
        void* fptr = reinterpret_cast<void*>(k);
        cudaLaunchKernelExC(&cfg, fptr, args);

        // D2H and cleanup
        cudaMemcpyAsync(h_C + static_cast<size_t>(b) * batchLength, dC, bytes,
                        cudaMemcpyDeviceToHost, st);
        cudaFreeAsync(dA, st);
        cudaFreeAsync(dB, st);
        cudaFreeAsync(dC, st);
    }

    for (int i = 0; i < NUM_STREAMS; ++i) {
        cudaStreamSynchronize(streams[i]);
        cudaStreamDestroy(streams[i]);
    }
}
```

本示例使用 `cudaLaunchKernelExC` 配置集群启动。协作式启动（包括集群启动）必须按照启动约束提供足够资源，使其线程块保持驻留。这通常会限制并发度；不过，在资源允许时，协作式内核仍可能与其他工作交错执行。并发并无保证，因为它取决于拓扑和启动配置。（注意：其他 stream 中的数据传输和内核仍可能重叠，但受资源限制和启动约束影响。）

与第 10 章一样，我们以集群维度启动内核，使线程块加入 `cluster_group` 并访问彼此的分布式共享内存。leader 线程块通过 block 作用域的 `cuda::thread_scope_block` 分阶段准备拷贝，所有线程块则使用 `cluster.map_shared_rank` 读取 leader 准备的 tile。

借助集群作用域和分布式共享内存，加载阶段由 leader 在每个集群中执行一次，而计算和存储则在集群的各线程块之间并发运行。和前面一样，每个 warp 的 `warp_id` 决定其角色；各 warp 持续遍历所有 tile，在同步轮转中完成获取、计算和存储。

通过加入 CUDA stream，并在不同 CUDA stream 中启动该内核的独立副本（`NUM_STREAMS = 2`），我们可以让 GPU 同时处理多个输入 batch。每个 stream 执行以下步骤：

1. 使用 `cudaMallocAsync` 为每个 batch 分配设备缓冲区。
2. 使用 `cudaMemcpyAsync` 将输入从主机分阶段传到设备。
3. 使用 `cudaLaunchKernelExC` 启动集群式 warp 专门化内核。
4. 使用 `cudaMemcpyAsync` 将输出拷回主机。
5. 使用 `cudaFreeAsync` 释放设备缓冲区。

由于每个 stream 都将自己的协作式启动以及异步拷贝和释放操作入队，即使同一时间只能运行一个协作式内核，主机仍可让多个 mini-batch 保持在途。请记住，GPU 会串行化协作式内核启动，因为每个协作式内核都会同时固定其所有线程块（代码示例后还会再次强调这一限制）。

使用多个 stream（`NUM_STREAMS = 2`）仍可让主机端分配和拷贝与前一个内核的执行重叠。例如，当 stream 0 的线程块集群处理 tile *n* 时，stream 1 可以异步分配缓冲区（`cudaMallocAsync`）并将 tile n+1 拷入设备内存，而 stream 2 则可能把 tile n–1 的结果写回主机。

> 协作式内核必须占用 GPU 上它所需的全部线程块槽位（CTA）。因此其他协作式启动无法同时运行，因为这些 CTA 资源已经全部被占用。

实践中，stream 0 发出协作式启动并开始执行时，stream 1 可以立即将自己的启动操作入队；但第二个启动会一直等待，直到 stream 0 的内核完成。

不过，stream 0 一完成，stream 1 的启动就能立即开始，因为它的输入已经由 `cudaMemcpyAsync` 预先准备好，所需缓冲区也已由 `cudaMallocAsync` 分配。

通过结合线程块集群，我们创建了跨越多个线程块的两层流水线。具体来说，将内核间 CUDA stream 与内核内 warp 专门化结合起来：在 leader CTA 内实现 block 作用域流水线，并配合集群范围同步和 DSMEM 共享。这个两层流水线可充分利用硬件资源，但实际效果仍取决于工作负载和资源约束。

第一层中，线程块集群流水线让加载、计算和存储 warp 跨所有线程块协作。线程块集群可以使用 TMA multicast，将一次全局内存 tile 传输复制到集群内每个线程块的共享内存中；multicast 的作用域局限于该线程块集群。也就是说，在适用的访问模式下，集群可以避免为同一 tile 重复发起全局加载。第二层中，多个 stream 让主机端的分配、拷贝和内核启动彼此隐藏。

简而言之，这些组合技术试图在 grid 层面和主机到设备层面同时隐藏内存延迟。在现代 GPU 上，它们可能使硬件利用率接近峰值，并提高 LLM 工作负载的吞吐量；具体收益需要通过实际测量确认。

> 在许多场景中，DRAM 带宽等内存层次瓶颈，以及 NCCL all-reduce 等模型并行通信，才是主要限制。因此，当内核已经接近饱和 HBM 带宽、stream 也已隐藏 CPU → GPU 延迟时，再叠加 warp 专门化、进而叠加线程块集群所带来的少量 SM 利用率提升，通常很难抵消其高昂的工程成本。实践中，大多数真实 LLM 训练和推理工作负载从前面介绍的简单设计中就能获得足够收益，例如双缓冲内核或使用 CUDA stream 的两阶段流水线。

使用 CUDA Stream 重叠多 GPU 计算与数据传输

在多张 GPU 上训练或服务 LLM 时，CUDA stream 可让本地计算、点对点传输、集合通信、数据准备和内存管理重叠，以避免任一设备空闲。例如，假设将 Transformer 模型拆分到两张 GPU：GPU 0 处理第 0–3 层，GPU B 处理第 4–7 层。在 GPU A 上可这样编写：

```
// Stream 0 on GPU A: compute layers 0-3
myTransformerLayers0to3<<<gridA, blockA, 0, stream0_A>>>(
    inputActivationsA, outputActivationsA);
cudaEvent_t eventA, eventFromA;
cudaEventCreateWithFlags(eventA,
    cudaEventDisableTiming); // lowers overhead for sync-only events
cudaEventCreateWithFlags(eventFromA,
    cudaEventDisableTiming);  // lowers overhead for sync-only events
cudaEventRecord(eventA, stream0_A);
```

与此同时，CPU 已经解码或准备好下一个 microbatch（N+1），并将其通过 `cudaMemcpyAsync` 发往页锁定的主机缓冲区。这样，当 GPU A 的 stream 0 完成 batch *N* 的前向传播时，batch N+1 的 CPU 到 GPU 拷贝可以立即开始，无需等待。

同时，调用 `cudaStreamWaitEvent(stream1_A, eventA, 0)`，确保 `stream1_A` 在 `outputActivationsA` 就绪后，才启动写入 GPU B 内存的 NVLink/PCIe 直接拷贝，如下所示：

```
// Stream 1 on GPU A: wait for layer computation, then copy to GPU B
cudaStreamWaitEvent(stream1_A, eventA, 0);
cudaMemcpyPeerAsync(
    destActivationsB, /*dstGPU=*/1,
    outputActivationsA, /*srcGPU=*/0,
    bytes, stream1_A);
```

一旦记录 `eventA`，GPU A 的 `stream0` 就可以立即启动 batch N+1 的下一个前向内核，并使用 `cudaMemcpyAsync` 将输入从页锁定主机缓冲区拷入设备内存，而不会阻塞。在 GPU B 上，则在拷贝开始或结束时记录对应事件：

```
// Stream 1 on GPU A: after peer copy starts,
// record eventFromA
cudaEventRecord(eventFromA, stream1_A);
```

随后，GPU B 等待数据到达并运行第 4–7 层：

```
// Stream 1 on GPU B: wait for data arrival, then run layers 4-7
cudaStreamWaitEvent(stream1_B, eventFromA, 0);
myTransformerLayers4to7<<<gridB, blockB, 0, stream1_B>>>(
    destActivationsB, nextActivationsB);
```

这种显式的事件—等待模式确保 GPU B 的第 4–7 层只会在点对点拷贝完成后开始计算。同时，GPU B 的 `stream0_B` 可以预取权重或执行其他准备工作。

这里的 P2P 传输运行在 GPU A 的 DMA 拷贝引擎上，不占用 GPU A 的 SM。因此，GPU A 的计算 stream 可以立即继续处理 batch N+1，无需负责管理数据传输。

结果形成三路重叠：GPU A 的 SM 可在 `stream0_A` 中立即启动 batch N+1；GPU A 的对等 DMA 引擎可在 `stream1_A` 中将 batch *N* 的激活传送至 GPU B；GPU B 的 SM 则可在 `stream0_B` 中运行 batch *N*，或在其 `stream1_B` 中开始 batch *N*。通过将工作划分到不同 stream，并在主机上使用页锁定内存，可将 P2P 与 H2D 延迟隐藏在持续的计算和数据准备之后。

需要在多个 GPU 之间同步梯度或广播参数时，NCCL 会通过低占用率设备内核大规模处理通信，并在 NVLink、PCIe 或 InfiniBand 上驱动 GPUDirect P2P 或 RDMA。NCCL 会在内部将张量拆分为多个连续 chunk。

这种设计表明，每张 GPU 都可拥有多个 stream：包括承担主要工作的计算 stream、接收输入数据的接收 stream，以及进行归约的通信 stream。为每种角色使用专用 stream，并为通信 stream 赋予更高优先级，可以实现最佳重叠。事实上，PyTorch 等框架通常会在这些专用高优先级 stream 上运行 NCCL 集合通信以进行网络传输。

> PyTorch 和 NCCL 都使用专用高优先级 stream，使通信与计算密集型操作交错执行，从而不会被大型计算内核拖延。

NCCL 会根据 NVLink 或 PCIe 拓扑选择环形或树形算法。以图 11-8所示的四 GPU 环为例，其中有四个 chunk（1–4）。

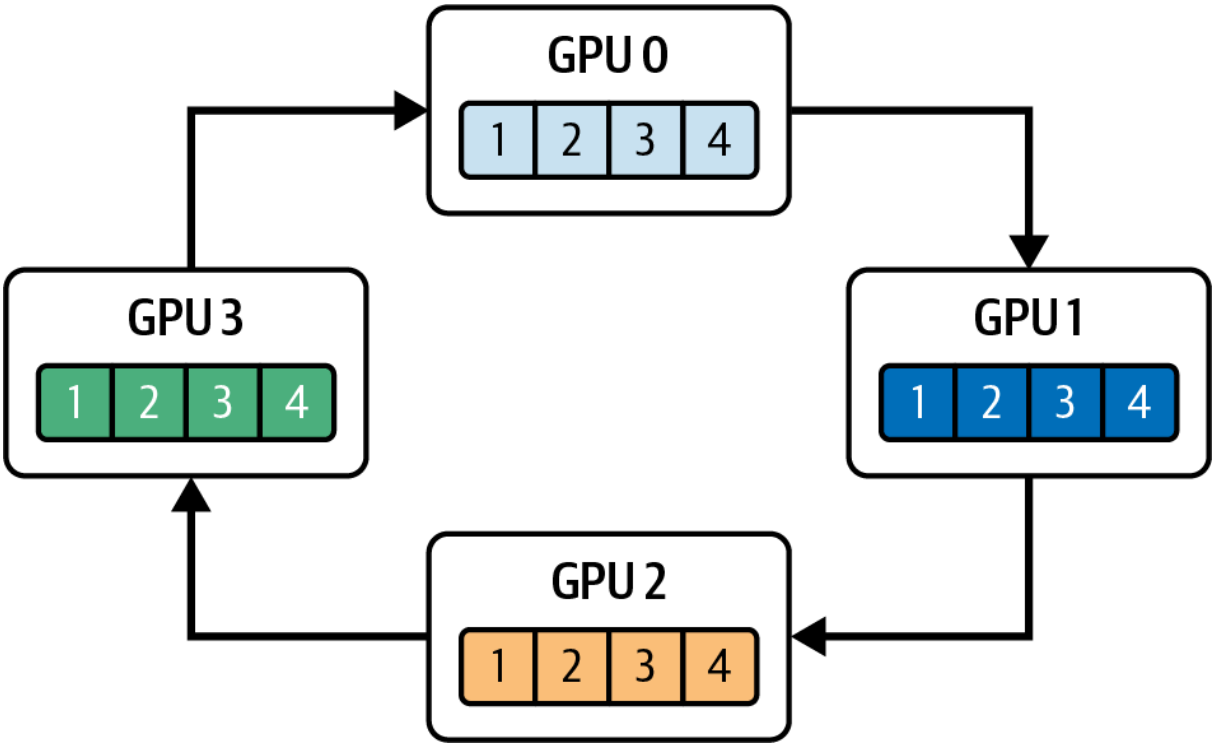


图 11-8. 四块 GPU 在环形拓扑中进行分块 all-reduce

在这个环形 all-reduce 中，每张 GPU 将 chunk *i* 发送给相邻 GPU（k → k+1），同时从另一相邻 GPU（k−1 → k）接收 chunk i−1，并通过设备内核经 NVLink 或 PCIe 移动和归约这些 chunk。

通过将这些分块发送和接收流水化，NCCL 可以充分利用 NVLink。当 chunk *i* 从 GPU 0 → GPU 1 传输时，chunk i−1 同时从 GPU 1 → GPU 2 传输，依此类推，从而减少空闲间隙。代码大致如下：

```
// In a high-priority communication stream on each GPU:
cudaEventRecord(eventK, computeStream);
cudaStreamWaitEvent(commStream, eventK, 0);
ncclAllReduce( // this is asynchronous
    gradBuffer, gradBuffer, numElements,
    ncclFloat, ncclSum, comm, commStream);
```

NCCL 仅使用少量 SM 线程块，因此可在 SM 上以低占用率方式编排跨 NVLink 或 NVSwitch 的分块发送/接收 + 归约。与此同时，例如正在对第 k+1 层执行反向传播的主计算 stream 可继续运行。all-reduce 完成后，便如下面所示记录事件：

```
cudaEventRecord(eventAllReduceDone, commStream); // signals collective completion
cudaStreamWaitEvent(computeStream, eventAllReduceDone, 0);
// Now apply optimizer updates on computeStream...
```

请记住，all-reduce 等 NCCL 集合操作是异步的，会立即返回控制权。在调用后记录事件，并让计算 stream 等待该事件，可以确保归约真正完成后计算 stream 才继续执行。

NCCL 内核专门针对低占用率下的最大带宽进行了优化，每张 GPU 只使用少量 SM 资源。实践中，得益于针对互连调优的低占用率内核，NCCL 可用每张 GPU 上少量线程块饱和 NVLink 或 PCIe 带宽。此外，若网络硬件支持，还可将部分集合聚合操作卸载至 NVIDIA SHARP，进一步释放 SM 资源以进行更有价值的计算。

NIXL 提供统一的高吞吐 API，用于跨 NVLink、RDMA 和存储后端执行点对点及解耦式传输，因此通常直接使用 NIXL API，而不是自行调用 `cudaMemcpyPeerAsync`。NIXL 可以跨不同内存层级和互连快速、异步地移动数据。当使用 `nixlCommStream` 调用 NIXL 操作时，数据会被分块，并通过 GPUDirect RDMA 或 NVLink 等可用的最快传输机制在网络上流水化传输。

与 NCCL 一样，NIXL 可以为分块点对点和集合数据传输使用专用高优先级 stream。这有助于减少排队延迟，使拷贝命令在发出后不久就到达 GPU 的 DMA 引擎，并可能抢先执行低优先级工作。

将这些传输 stream 标记为高优先级，并不意味着它们会预先占用 SM 资源；它只保证拷贝命令就绪时，会优先于其他低优先级操作发出。

实践中，拷贝引擎会利用计算内核尚未占用的 DRAM 到 DRAM 带宽。因此，它们实际上主要使用内存互连上空闲的带宽来移动数据。

这种设计可以最大化重叠：一个 stream 在拷贝引擎上驱动 `cudaMemcpyPeerAsync` 传输或 NCCL 归约内核时，另一个 stream 的 SM 内核仍可继续执行前向/反向工作，第三个 stream 则可以执行异步分配或事件等待。这样有助于让各类硬件单元保持忙碌，同时减少资源争用。

通过 `cudaMemcpyPeerAsync` 启动的点对点传输完全在 GPU 拷贝引擎上运行，因此只消耗 DRAM 到 DRAM 带宽，不消耗 SM 周期。相反，all-reduce 等集合操作被实现为设备内核：它们使用少量 SM，同时将互连带宽推向峰值。

同时，会异步分配和释放激活或混合精度梯度的临时缓冲区，以避免全局停顿。例如，在每张 GPU 的计算 stream 内可调用以下代码：

```
cudaMallocAsync(&tempBuffer, tempBytes, computeStream);
// ...use tempBuffer in kernels...
cudaFreeAsync(tempBuffer, computeStream);
```

由于按 stream 排序的内存分配器记录这些操作时不会强制设备范围同步，CUDA 在分配或释放内存时不会暂停 P2P、NCCL 和 NIXL 等其他 stream。这有助于避免缓冲区管理打断计算与通信的重叠流水线。

这正是前面讨论过的按 stream 排序内存分配器，只是现在应用到了多个 GPU。使用它有助于避免内存管理成为分布式工作负载的瓶颈。

将一次迭代的前向、反向、P2P 拷贝、NCCL all-reduce、内存释放以及事件等待操作入队后，可以把这条执行链（例如一次训练或推理迭代）捕获为一个 CUDA Graph。

具体来说，调用 `cudaStreamBeginCapture` 后，向该 stream 入队的每个操作（例如内核启动、计算、通信、数据传输和事件）都会作为节点插入图中。这些操作可以包括 `ncclAllReduce()`、`cudaMemcpyAsync`、`cudaMemcpyPeerAsync`、`cudaMallocAsync`、`cudaFreeAsync`、`cudaEventRecord`、`cudaStreamWaitEvent` 及其依赖。使用 `cudaStreamEndCapture(stream, &graph)` 结束捕获。代码大致如下：

```
cudaStreamBeginCapture(streamA, cudaStreamCaptureModeGlobal);

computeGradientsLayer1<<<... , 0, streamA>>>(...);
ncclAllReduce(..., comm, streamB);
computeGradientsLayer2<<<... , 0, streamA>>>(...);
ncclAllReduce(..., comm, streamB);
cudaStreamEndCapture(streamA, &graph);
```

捕获提交到多个 stream 的工作时，使用 `cudaStreamCaptureModeGlobal`，这样同一线程在所有参与 stream 上入队的操作都会记录到同一个捕获会话中，并保留跨 stream 依赖；否则，只会捕获传给 `cudaStreamBeginCapture` 的那个 stream 上入队的操作。

随后可以使用 `cudaGraphLaunch()` 实例化并启动该图，以接近零的 CPU 开销重放整个 DAG。只需捕获一次完整的多 GPU 迭代，就能消除每个操作单独入队和启动的开销。

此外，CUDA Graph 支持条件节点（例如梯度裁剪），因此低频分支也可以保留在图的逻辑中。下一章将更详细地介绍 CUDA Graph；现在需要理解它与 CUDA stream 的关系，包括 `cudaStreamBeginCapture` 和 `cudaStreamEndCapture`。

> PyTorch 等现代框架隐藏了许多复杂性。例如，PyTorch 的 `DistributedDataParallel` 会自动在不同 stream 中调度数据传输、计算和通信，也可以使用 CUDA Graph 减少每次迭代的开销。但 CUDA Graph 并不会自动启用，必须使用支持捕获的代码和 API 显式开启。

构建 CUDA Graph 时不一定要显式使用这些 CUDA stream 开始与结束 API；但如果希望在向 stream 入队操作的同时捕获 graph，它们会很方便（下一章将更详细介绍 CUDA Graph）。

CUDA stream 通过重叠 CPU、DMA 引擎和 SM 上的工作，为多 GPU 系统提供精细调节的流水线。在 CPU 侧，使用 `cudaMemcpyAsync` 将数据送入页锁定主机缓冲区，为下一个 batch 做准备；GPU 同时执行当前工作负载，从而确保 batch N+1 的输入在 batch N 完成前就绪。与此同时，GPU 之间的点对点传输（使用 `cudaMemcpyPeerAsync`）可以安排在专用 stream 上，并通过 `cudaStreamWaitEvent` 同步，在不阻塞正在进行的计算的情况下交接激活值或梯度。

如前所述，NCCL 或 NIXL 等集合通信库使用单独的 stream 运行低占用率通信内核。这样，当一个 GPU 正在归约梯度或广播参数时，同一 GPU 上的其他 stream 仍可继续执行本地计算内核（例如后续层的前向或反向传播）。此外，按 stream 顺序使用 `cudaMallocAsync` 和 `cudaFreeAsync`，可以避免为临时缓冲区进行全局同步，使分配和释放与计算、通信并发进行。

将完整迭代（例如 CPU 分阶段准备、P2P 传输、计算内核、NCCL 调用、分配、释放以及事件/等待）捕获为 CUDA Graph，可以进一步降低 CPU 开销。图实例化后，调用 `cudaGraphLaunch()` 即可一次性重放记录的 DAG，消除每次调用的入队开销，并自动保留所有依赖。

这些技术结合起来，可以让每个 GPU 的 SM 流水线、拷贝引擎和互连链路尽量保持忙碌：一个 stream 运行矩阵乘法内核，另一个执行点对点拷贝，第三个执行集合通信，而 CPU 为下一个 microbatch 准备数据。

简而言之，可使用 CUDA stream 编排跨多个层的工作，并让计算、内存操作和数据传输重叠。这种方法将点对点和集合通信延迟隐藏在活跃计算之后，使每个硬件单元尽可能发挥能力。总体而言，CUDA stream 可为许多 AI 工作负载（包括 LLM 训练和推理）最大化吞吐量、利用率和效率。

程序化依赖启动（Programmatic Dependent Launch）

另一种内核间流水线与通信方式称为*程序化依赖启动*（Programmatic Dependent Launch，PDL）。PDL 允许一个内核在设备端使用同一 CUDA stream 直接触发另一个内核执行，无需 CPU 参与。例如，主内核 A 可以触发正在等待其信号的次内核 B。

这种触发甚至可发生在内核 A 尚未完成执行之前。为此，它使用 `cudaTriggerProgrammaticLaunchCompletion()`，如图 11-9所示。这里内核 B 通过调用 `cudaGridDependencySynchronize()` 等待内核 A。

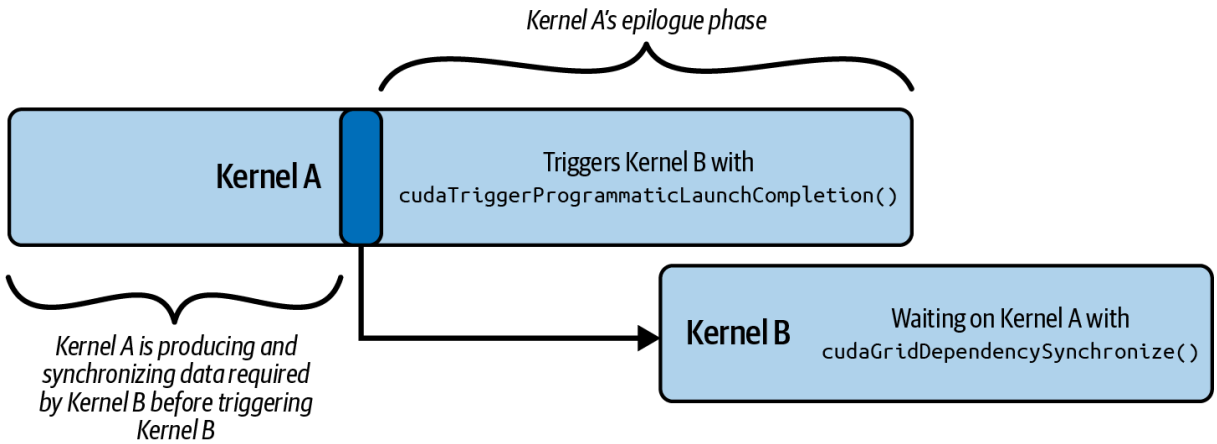


图 11-9. 使用 PDL 从内核 A 启动内核 B——使内核 B 的执行与内核 A 的尾声阶段部分重叠（在同一 CUDA 流中）

利用 PDL 提供的构造，内核 A 可在其*尾声（epilogue）*阶段（如收尾阶段）向内核 B 发信号使其执行。这样，内核 A 可与内核 B 短暂并行。需要注意，只有在内核 A 已生成并同步内核 B 所需的所有数据（如 L2/共享内存/全局内存）后，才应触发内核 B 执行。

> 依赖内核继续处理前，其数据依赖应已在 L2、共享内存、全局内存等位置可见。

这里的代码展示内核 A 使用 `cudaTriggerProgrammaticLaunchCompletion()` 向内核 B 发信号，表示主要工作已完成。这也通知内核 B：所有必要的全局内存刷新均已发生，可以安全继续：

```
#include <cuda_runtime.h>
#include <cuda_runtime_api.h>

// Kernel A must trigger the PDL flag when it's
//    safe to launch Kernel B
__global__ void kernel_A(int *d_ptr) {
    // Perform work that produces data used by
    //   Kernel B

    // Signals that Kernel A's global-memory
    //   flushes are complete
    // This enables dependent Kernel B's launch

    cudaTriggerProgrammaticLaunchCompletion();

    // ... any further work that can overlap with  ...
    ...
}

// Kernel B must wait for Kernel A to write its memory
//    to global memory and become visible to Kernel B
__global__ void kernel_B(int *d_ptr) {

    // Wait on Kernel A to complete.
    // This ensures the Kernel B waits for the
    //   memory flush before accessing shared data
    cudaGridDependencySynchronize();

    // ... dependent work on d_ptr ...
    ...
}

int main() {
    // 1) Allocate device buffer
    int *d_ptr = nullptr;
    // Allocate an int (example)
    cudaMalloc((void**)&d_ptr, sizeof(int));

    // 2) Create a nonblocking stream for maximum overlap
    cudaStream_t stream;
    cudaStreamCreateWithFlags(&stream,
        cudaStreamNonBlocking);  // Nonblocking

    // 3) Define grid/block sizes
    dim3 gridDim(128), blockDim(256);

    // 4) Launch Kernel A asynchronously
    kernel_A<<<gridDim, blockDim, 0,
        stream>>>(d_ptr);    // Async launch

    // 5) Configure PDL for Kernel B
    cudaLaunchConfig_t launch_cfg{};
    launch_cfg.gridDim          = gridDim;
    launch_cfg.blockDim         = blockDim;
    launch_cfg.dynamicSmemBytes  = 0;
    launch_cfg.stream           = stream;

    // Sets the PDL flag so cudaLaunchKernelExC overlaps
```

```
    //   with Kernel A's epilogue
    static cudaLaunchAttribute attrs[1];
    attrs[0].id  = cudaLaunchAttributeProgrammaticStreamSerialization;
    attrs[0].val.programmaticStreamSerializationAllowed =
        1;
    launch_cfg.attrs    = attrs;
    launch_cfg.numAttrs = 1;

    // 6) Pack the pointer argument
    void* kernelArgs[] = { &d_ptr };

    // 7) Launch Kernel B kernel early using PDL
    // Lookup device pointer for secondary_kernel
    cudaKernel_t kB;
    cudaGetKernel(&kB, kernel_B);
    void* funcPtr_kernel_B = reinterpret_cast<void*>(kB);
    cudaLaunchKernelExC(&launch_cfg, funcPtr_kernel_B, kernelArgs);

    // 8) Wait until all work in the stream completes
    cudaStreamSynchronize(stream);

    // 9) Cleanup
    cudaStreamDestroy(stream);
    cudaFree(d_ptr);

    return 0;
}
```

这里，Kernel B 通过 `cudaGridDependencySynchronize()` 等待 Kernel A 发出的程序化启动完成信号。交接发生后，两个内核就可以重叠执行。将 Kernel A 中的触发、Kernel B 中的同步以及主机端的启动属性配合起来，可以尽可能重叠 Kernel A 和 Kernel B。

如本例所示，使用 PDL 时，需要创建 `cudaLaunchConfig_t`，并在 `cudaLaunchKernelExC()` CUDA 调用中使用专用属性。具体来说，在主机端，为 Kernel B 的启动配置 `cudaLaunchConfig_t`，并启用 `cudaLaunchAttributeProgrammaticStreamSerialization` 属性，以允许提前且重叠地分发。

使用 `cudaLaunchAttributeProgrammaticStreamSerialization` 调用 `cudaLaunchKernelExC()`，会告知 CUDA runtime 即使内核 A 尚未完全结束，也可安全地将内核 B 入队。随后 `cudaLaunchKernelExC()` 以这些扩展属性执行实际启动，并依赖触发机制完成交接。

结合 PDL、线程块集群与 Warp 专门化

现在将三种正交技术——PDL、warp 专门化流水线和线程块集群——组合为一个执行模型。

PDL 提供了一种机制，使一个内核能够发出其前导阶段完成的信号，并触发依赖内核执行；随后，前一个内核逐步收尾，而依赖内核逐步启动并执行。

Warp 专门化将每个线程块细分为 producer warp 和 consumer warp。具体而言，producer warp 使用 TMA 异步传输 tile，而计算 warp 执行矩阵乘加（MMA）操作。

线程块集群则使多个线程块运行在彼此邻近的 SM 组上，从而为大规模工作负载提供多 SM 协作和共享内存 multicast。

这些内核间与内核内技术结合起来，可以隐藏 DRAM 延迟、降低内核边界开销并提高 Tensor Core 利用率，形成具有以下特征的流水线：

内核内流水线

Warp 专门化通过多阶段流水线，确保每个线程块内的数据传输（producer warp）与计算（consumer warp）充分重叠。

内核间重叠

PDL 允许依赖 GEMM 内核的前导阶段（可能处理神经网络的下一层）在主内核完成数据（权重）预取后立即开始，而不必等待整个线程块完全退出。

线程块间协作

线程块集群使线程块组能够协同预取（例如 multicast），并在 SM 之间进行动态负载均衡，从而让 producer 和 consumer 任务均匀分布到整个集群。

例如，可以在同一个内核内让模型权重移动（数据传输）与 GEMM（计算）重叠：一个 tile 正在计算时，下一个 tile 已在传输途中。如图 11-10所示，采用 warp 专门化的多阶段软件流水线（阶段 0⋯N）可通过 PDL 和 *mbarrier* 原语协调这些角色。

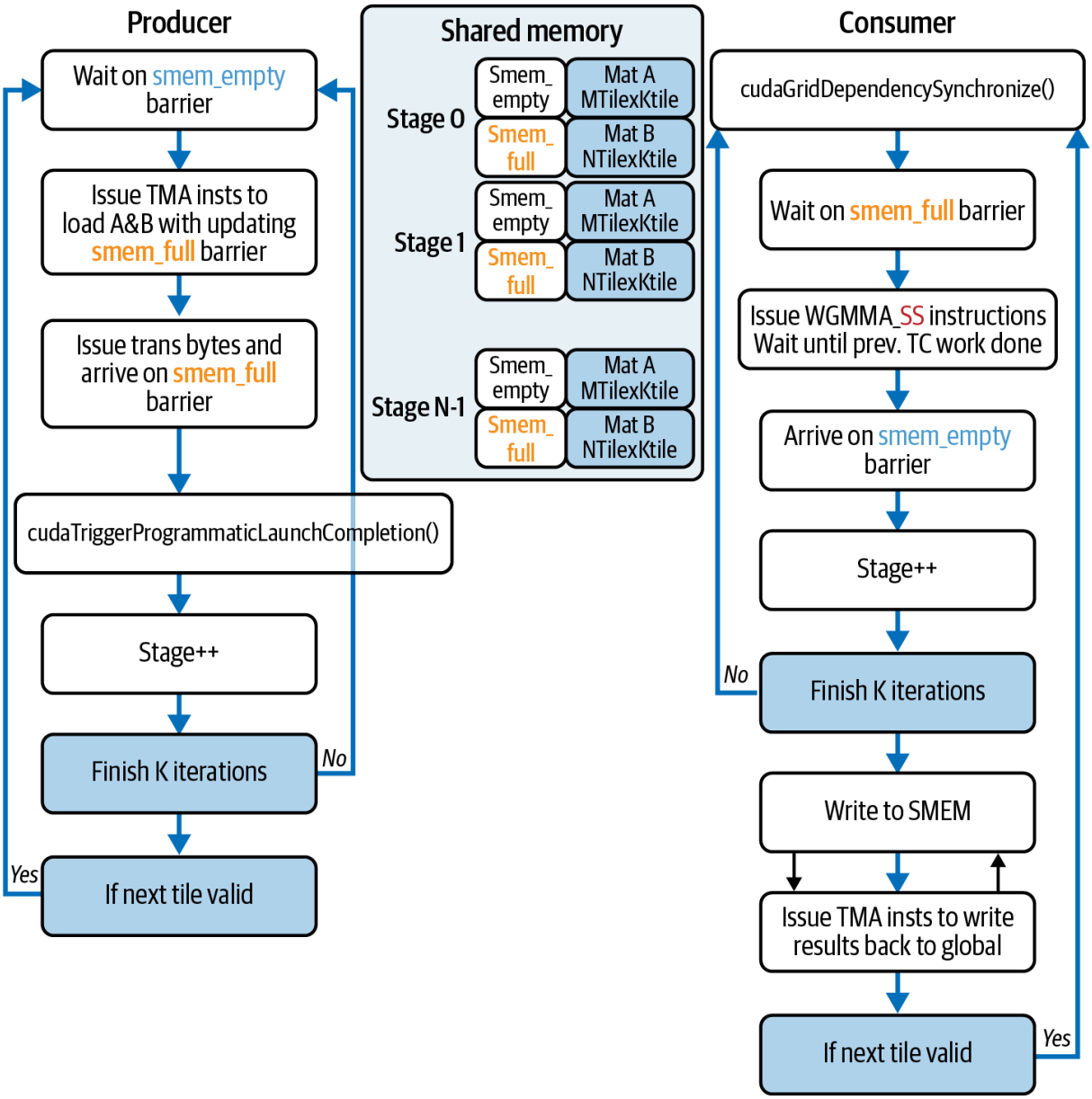


图 11-10　结合 PDL、线程块集群与 TMA 多播的 warp 专门化多阶段流水线，用于高性能内核间和内核内 GEMM 实现

下面的 CUDA C++ 示例展示如何将 PDL、线程块集群和 warp 专门化组合成简单的 TMA 风格异步拷贝 + 计算流水线。具体来说，主内核 `primary_gemm` 使用 `cudaTriggerProgrammaticLaunchCompletion()` 向次内核发出信号，表示所有内存刷新已经完成，数据可以读取。因此，次内核 `secondary_gemm` 可以安全地从 `cudaGridDependencySynchronize()` 返回并继续执行：

```
#include <cstdio>
#include <cuda_runtime.h>
// Cooperative Groups for clusters/barriers
#include <cooperative_groups.h>
// C++ barrier for TMA-like sync
#include <cuda/barrier>
// Async copy API
#include <cuda/pipeline>

namespace cg = cooperative_groups;
```

```cpp
// Tile size for our toy GEMM
constexpr int TILE_M = 128;
constexpr int TILE_K = 128;
constexpr int TILE_N = 128;

// A very simple “producer/consumer” pipeline within each CTA
__global__ __cluster_dims__(2,1,1)    // Compile-time cluster of 2 CTAs
void primary_gemm(const float* __restrict__ A,
                  const float* __restrict__ B,
                        float* __restrict__ C,
                  int M, int N, int K)
{
    // Identify thread-block cluster & within-block group
    cg::thread_block_cluster cluster = cg::this_thread_block_cluster();
    cg::thread_block       cta     = cg::this_thread_block();
    int tid = threadIdx.x + threadIdx.y * blockDim.x;
    int warpId = tid / warpSize;
    const int numProducerWarps = 1;

    // Shared-memory tile buffers
    __shared__ float tileA[TILE_M * TILE_K];
    __shared__ float tileB[TILE_K * TILE_N];

    __shared__
    cuda::pipeline_shared_state<cuda::thread_scope_block, 1> pipe_state;
    auto pipe = cuda::make_pipeline(cta, &pipe_state);
    auto warp = tiled_partition<32>(cta);
    pipe.producer_acquire();
    if (warpId < numProducerWarps) {
        // Producer warps issue copies,
        // but pipeline collectives are CTA-uniform.
        cuda::memcpy_async(warp, tileA, A, TILE_M * TILE_K * sizeof(float), pipe);
        cuda::memcpy_async(warp, tileB, B, TILE_K * TILE_N * sizeof(float), pipe);
    }
    pipe.producer_commit();
    cta.sync();
    pipe.consumer_wait();

    // ... perform “compute” on the tile ...
    // (e.g., a few fused multiply-adds)
    do_compute();

    // Inter-CTA cluster-scope sync for load balancing
    cluster.sync();

    // Signal to dependent kernel that prologue is done
    cudaTriggerProgrammaticLaunchCompletion();

    // ... perform remaining epilogue work ...
    // ...
}

__global__ __cluster_dims__(2,1,1)
void secondary_gemm(const float* __restrict__ A,
                    const float* __restrict__ B,
                          float* __restrict__ C,
                    int M, int N, int K)
{
    // Wait for primary’s PDL signal before starting
    cudaGridDependencySynchronize();

    // Similar warp-specialized pipeline as above...
    //  (Omitted for brevity. Duplicate of primary logic,
    //  but reading from different offsets to compute
```

```
    //  next GEMM tile.)
}

// cudaLaunchKernelExC, cudaGetKernel
#include <cuda_runtime_api.h>
// cudaLaunchConfig_t
#include <cuda/launch_config.h>

int main()
{
    // Problem dimensions (must be multiples of TILE_)
    int M = TILE_M, N = TILE_N, K = TILE_K;

    // Allocate and initialize matrices A, B, C on device
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, M*K*sizeof(float));
    cudaMalloc(&d_B, K*N*sizeof(float));
    cudaMalloc(&d_C, M*N*sizeof(float));
    // (Initialize d_A, d_B via cudaMemcpy or kernels...)

    // Create a nonblocking stream for overlap
    cudaStream_t stream;
    cudaStreamCreateWithFlags(&stream,
        cudaStreamNonBlocking);

    // Launch primary GEMM
    dim3 gridDim(M/TILE_M, N/TILE_N), blockDim(256);
    primary_gemm<<<gridDim, blockDim, 0, stream>>>(d_A,
d_B, d_C, M, N, K);

    // Configure PDL attributes for secondary launch
    cudaLaunchConfig_t launch_cfg = {};
    launch_cfg.gridDim          = gridDim;
    launch_cfg.blockDim         = blockDim;
    launch_cfg.dynamicSmemBytes = 0;
    launch_cfg.stream           = stream;
    static cudaLaunchAttribute attrs[1];
    attrs[0].id  = cudaLaunchAttributeProgrammaticStreamSerialization;
    attrs[0].val.programmaticStreamSerializationAllowed = 1;
    launch_cfg.attrs    = attrs;
    launch_cfg.numAttrs = 1;

    // Prepare arguments and get function pointer
    //    for secondary_gemm
    void* kernelArgs[] = {&d_A, &d_B, &d_C, &M, &N, &K};

    cudaKernel_t k;
    cudaGetKernel(&k, secondary_gemm);
    void* funcPtr = reinterpret_cast<void*>(k);

    // Early enqueue of secondary GEMM via PDL
    cudaLaunchKernelExC(&launch_cfg, funcPtr, kernelArgs);

    // Wait for everything to finish
    cudaStreamSynchronize(stream);

    // Cleanup
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    cudaStreamDestroy(stream);
    return 0;
}
```

一个细微但重要的合法性规则是：在本章统一的 block 作用域流水线示例中，参与的 CTA 线程必须以一致顺序执行 `producer_acquire`/commit 和 `consumer_wait`/release。仍然可以保留 warp 专门化，只让选定的 warp 在这些集体操作之间发出分阶段拷贝工作。

为保证内存可见性和角色间交接，应在所有权变化处将统一流水线序列与显式的 `cta.sync()` 同步点结合。如果希望不同 warp 直接调用不同流水线原语，应使用明确 producer/consumer 角色的分区流水线 API，而不要对统一流水线序列进行分支。

在主机端，将主内核 `primary_gemm` 启动到非阻塞 stream 中，并创建带有 `ProgrammaticStreamSerialization` 属性的 `cudaLaunchConfig_t`，再用它通过 `cudaLaunchKernelExC` 启动次内核 `secondary_gemm`。

在 `primary_gemm` 内，调用 `cudaTriggerProgrammaticLaunchCompletion()` 标记内存刷新完成，使依赖内核可以开始处理，即使主内核尚未完全退出。

随后，依赖内核调用 `cudaGridDependencySynchronize()`，等待主内核发出的信号和必要的内存刷新完成，从而可以与主内核的收尾阶段并行执行。

在每个线程块内，使用 warp 专门化重叠数据移动与计算。将每个线程块拆分为一个“producer” warp 和多个“consumer” warp 后，producer 在流水线集体操作下发出协作式分阶段拷贝。这可以利用支持 TMA multicast 的路径，将一次 DMA 传输广播到集群中的所有线程块，如图 11-11所示。

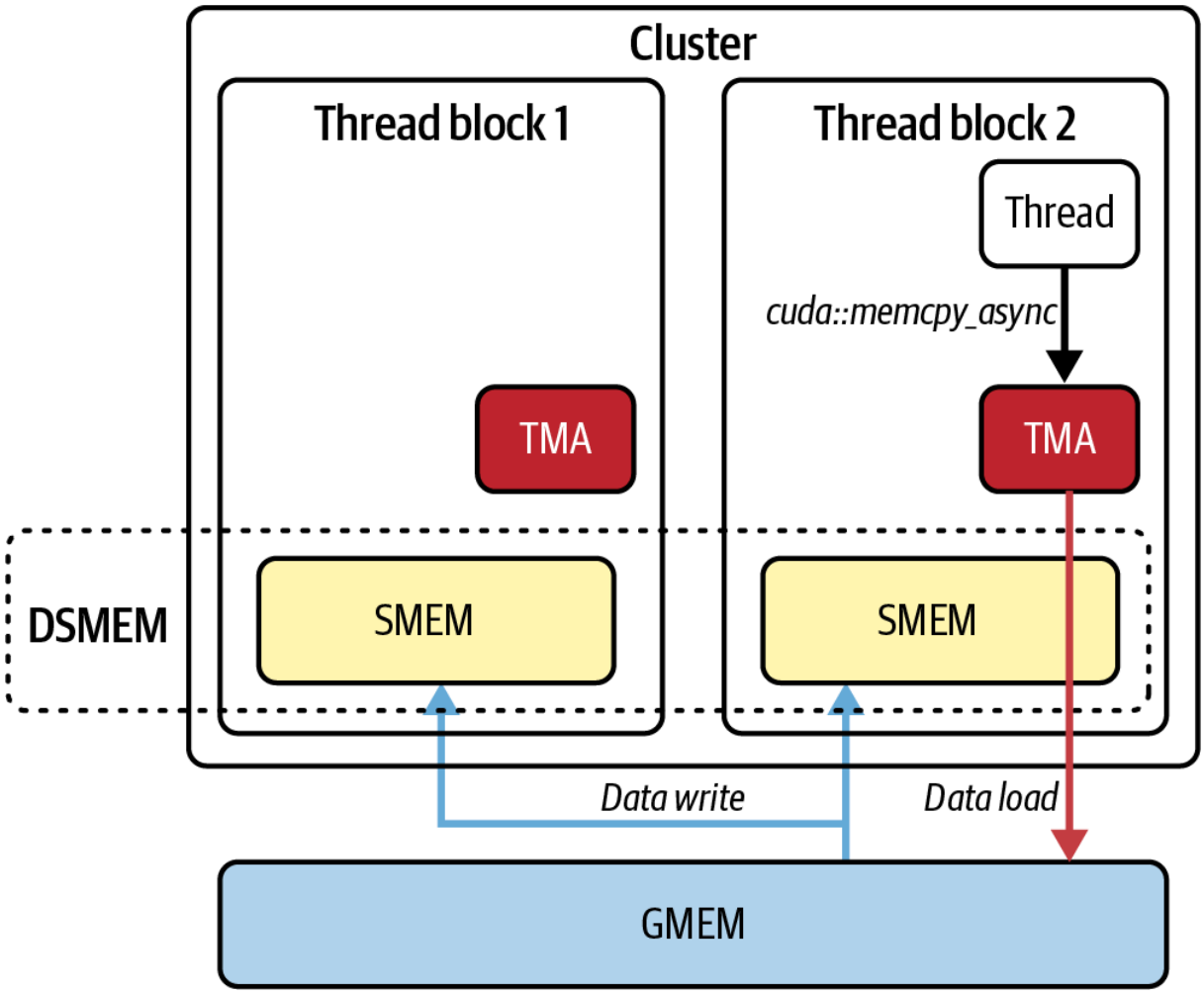


图 11-11　TMA 多播：主 CTA 向 DSMEM（集群共享内存）发出 `cp.async.bulk.tensor`；从属 CTA 使用 `cluster.map_shared_rank` 消费 tile；流水线集体操作下的协作式分阶段拷贝可驱动此路径，包括基于 `cuda::memcpy_async` 的实现

producer warp 使用 TMA multicast 加载数据时，consumer 会在 C++ block 作用域屏障（`cuda::barrier<cuda::thread_scope_block>`）上等待，然后执行矩阵乘法步骤（`do_compute()`）。这样，每个 tile 的加载与融合乘加（FMA）操作就能在细粒度多阶段软件流水线中交错，从而在内核内部隐藏全局内存延迟。

为协调跨 SM 的工作，我们给两个内核都标注 `__cluster_dims__(2,1,1)`，将两个线程块分组到相邻 SM 上。调用 `cluster.sync()`（cooperative groups 对 PTX `mbarrier` 指令的封装）可充当集群范围屏障和共享内存栅栏。这样，集群中的所有线程块都能看到相同的 TMA 加载数据，并动态重新平衡剩余 tile。这种块间协作可以减少 SM 空闲，并提升 warp 专门化流水线的收益。

> 生产内核通常对二维 tile 使用 Tensor Memory Accelerator（TMA）`cp.async.bulk.tensor`；当多个线程块需要同一 tile 时，会在线程块集群间进行多播。
>
> 可考虑在 Blackwell 上的分块复制期间使用基于描述符的 TMA 归约操作（如 `cp.reduce.async.bulk(.tensor)`）进行即时归约。将小型归约融合到数据移动流水线时，优先采用基于描述符的加载/存储加上 TMA 归约操作，这可降低寄存器压力并改善重叠。

最后回顾这一高性能 GEMM 流水线的三个特征：

内核内流水线

在每个线程块内通过 warp 专门化细分工作：producer warp 在流水线集体操作下发出协作式分阶段 tile 拷贝（例如 `cuda::memcpy_async`），consumer warp 则在 block 作用域屏障上等待，使数据传输与计算充分重叠。

内核间流水线

使用 PDL，通过 `cudaTriggerProgrammaticLaunchCompletion()` → `cudaGridDependencySynchronize()` 机制，让下一个 GEMM 内核在主内核完全退出前就开始处理，从而隐藏内核启动开销。

线程块间协作

通过使用 `__cluster_dims__` 标注内核，代码创建了在相邻 SM 上共同调度的线程块对。再配合调用 `cluster.sync()`（基于 mbarrier 的集群屏障），这些线程块可以共享 TMA multicast 数据，并在整个 grid 中动态均衡工作负载。

简言之，将带 TMA 的 warp 级流水线、集群级同步与多播，以及使用 PDL 的内核间重叠叠加起来，便形成高度重叠的流水线，能够隐藏 DRAM 延迟、掩盖内核启动开销，并最大化 Tensor Core 利用率。结果是在每个阶段实现高性能 GEMM：warp 内复制、跨线程块屏障和内核交接期间均如此。这些机制协同使硬件保持忙碌并避免停顿。

要点回顾

本章介绍了 CUDA stream、按 stream 排序的内存分配器、基于事件的同步、内核间流水线、线程块集群和 PDL 等高级主题。这些机制有助于为推理和训练工作负载创建高效流水线。主要要点如下：

显式 stream 与默认 stream

避免使用 legacy default stream（stream 0），因为它可能使工作串行化并引入隐式同步。应创建显式非阻塞 stream（`cudaStreamCreateWithFlags(..., cudaStreamNonBlocking)`），让内核和拷贝在没有隐藏同步的情况下并发运行。

按 stream 排序的内存分配器

使用 `cudaMallocAsync` 和 `cudaFreeAsync` 在特定 stream 中分配和释放设备内存。这个非阻塞分配器将请求记录到 stream 队列中，避免设备范围同步，并允许分配与在途内核和拷贝重叠。

重叠 H2D、计算与 D2H

将异步主机到设备拷贝（`cudaMemcpyAsync`）、内核启动和设备到主机拷贝分别安排到不同 stream，可以实现三路重叠。当一个 stream 运行内核时，另一个可以将下一个 batch H2D 拷入，第三个可以将结果 D2H 拷回，从而隐藏延迟并减少空闲时间。

用于细粒度同步的 CUDA event

使用 `cudaEventRecord` 和 `cudaStreamWaitEvent` 协调跨 stream 的 producer-consumer 依赖，而不阻塞整个 GPU 或 CPU。事件让 consumer stream 精确等待 producer stream 完成拷贝或内核，从而保持较高并发度。

内核间流水线

将内核内的 warp 专门化（多角色）、两阶段（双缓冲）流水线与多 stream 启动结合起来。在不同 stream 中启动多个 warp 专门化内核实例（加载 → 计算 → 存储），即可将连续 mini-batch 送入 GPU，把内核内的内存/计算重叠与内核间并发结合起来。

结合 stream 的线程块集群

将内核内 warp 专门化流水线扩展到覆盖整个 grid 的线程块集群（协作式启动），可让加载/计算/存储 warp 跨线程块协作。在多个 stream 中启动这些协作式内核时，后续 batch 的主机端分配和拷贝可以与协作式内核执行重叠。

内核内信号与 PDL 重叠

Kernel A 的数据写入刷新后调用 `cudaTriggerProgrammaticLaunchCompletion()`，Kernel B 使用 `cudaGridDependencySynchronize()` 等待该信号。这允许 Kernel B 的前导阶段开始，并与 A 的收尾阶段重叠，无需 CPU 介入。

Host-side PDL setup

主机通过 `cudaLaunchKernelExC()` 和 `cudaLaunchConfig_t` 配置 Kernel B 的启动，并设置 `cudaLaunchAttributeProgrammaticStreamSerialization`。这允许驱动提前将 B 入队，以最大化内核间重叠。PDL 在主内核中使用 `cudaTriggerProgrammaticLaunchCompletion`，在依赖内核中使用 `cudaGridDependencySynchronize`，并在主机端设置 `cudaLaunchAttributeProgrammaticStreamSerialization` 启动属性。

结论

总之，借助 CUDA stream 的内核间并发，已经从手动优化逐渐发展为现代 AI 框架和 GPU runtime 使用的自动化能力。理解 stream、资源占用率和同步点等核心原则后，开发者就能更充分地利用 GPU。

在现代工作负载中，内核间并发对于最大化 GPU 利用率至关重要：它可以在单个 GPU 内部以及多个 GPU 之间重叠内核执行和数据传输。随着硬件增加更多并行能力、软件抽象掉更多调度复杂性，理解如何最大化并发，已成为充分发挥高性能 AI 系统硬件能力的关键。

本章演示了如何跨多个 CUDA stream 编排内核、内存操作和分配，使 GPU 的各类硬件单元保持活跃，包括计算流水线、DMA 引擎和互连。CUDA stream 是将内核、内存操作和分配安排到独立队列中的基础机制，使 GPU 的计算引擎与 DMA 引擎能够同时运行。

通过避免 default stream 的隐藏屏障、利用按 stream 排序的分配器、使用事件进行精确同步，并将内核内 warp 专门化与内核间多 stream 流水线结合起来（扩展第 10 章介绍的线程块集群），即使面对复杂 LLM 工作负载，也有机会实现接近峰值的利用率。

在多 GPU 场景中，在不同 stream 上重叠点对点传输、集合通信和计算，可以进一步减少空闲时间。我们还回顾了第 10 章的相关内容，展示如何使用 CUDA Graph 捕获这些 stream 工作流，以降低重复迭代的 CPU 开销。

下一章将在这些原则之上进一步介绍使用动态并行和 CUDA Graph 的动态内核编排与元调度。我们将在运行时协调完整的内核与数据移动流水线，以适应不断变化的工作负载。这种动态资源均衡和设备端编排将把大规模 AI 系统的性能优化推进到更高层次。

第 12 章　动态调度、CUDA Graph 与设备发起的内核编排

到目前为止，我们已经在单个内核层面释放了计算和内存吞吐。现在需要编排这些内核，让 GPU 尽量不再空闲。

本章将从主机端调度转向设备自身的调度。我们会探讨由快速 L2 缓存原子操作驱动的动态工作队列，合并重复的内核启动，并使用 CUDA Graph 批处理固定流水线、减少 CPU 握手。

随后，我们将通过设备端图启动和动态并行进一步推进编排能力，使 GPU 无需回调 CPU 就能决定接下来运行什么。

最后，我们将进入多 GPU 环境，通过重叠点对点拷贝、NCCL 集合通信、支持 CUDA 的 MPI 以及 NVSHMEM 单边 put/get 操作开展工作。这样，GPU 集群就能表现得像一个巨大的共享内存协处理器。例如，NVIDIA DGX GB200 NVL72 系统将 36 个 Grace CPU 和 72 个 Blackwell GPU 连接到单一 NVLink 域中，在该域内提供统一寻址以及最多 30 TB 的 CPU 与 GPU 合并统一内存，并支持在 72-GPU 域内通过 NVLink fabric 访问远端 HBM。更大的 NVLink 网络拓扑还可以扩展到单个机架之外。

在此过程中，我们会把每项技术联系到 roofline 分析，帮助你选择合适的工具——stream、graph、原子操作或动态内核——以提高内核的运算强度，从而改善工作负载的整体性能。

读完本章后，你将理解动态、设备端和基于 graph 的内核编排技术，掌握如何在多 GPU 集群中持续为每个 SM 提供工作。

使用原子工作队列进行动态调度

线程之间不均匀的工作分配会导致部分 SM 空闲，而其他 SM 仍在忙碌。这会浪费计算资源并降低整体吞吐量。

当循环次数取决于输入，或工作负载包含条件分支时，不同线程或线程块可能处理数量不同的工作，从而经常出现不均衡。一些线程块很快完成，使所在 SM 空闲，而其他 SM 仍在执行耗时更长的线程块。在拥有数百个 SM 的现代 GPU 上，如果工作分配不均，空闲阶段可能让大量 SM 闲置，显著损害性能。

最长的工作完成时，GPU 的一部分已经空闲了一段时间。这会降低实际达到的 occupancy，因为许多周期没有活跃 warp。可以使用 Nsight Systems 进行分析，在 GPU 时间线上清楚地显示这些空闲间隙。

还可以比较活跃 SM 周期与 SM 总耗时周期，以衡量利用不足。Nsight Compute 将其作为单一指标提供，表示至少有一个 warp 活跃的时间比例。活跃周期与总周期的比值较低，说明许多周期没有活跃 warp；换言之，GPU 经常处于空闲状态。

除了 Nsight Systems，还可以使用 Nsight Compute 检查实际 occupancy（每个 SM 的活跃 warp 平均比例，相对于硬件最大值）或 SM Active cycles 百分比（至少一个 warp 活跃的时间比例），以量化这种利用不足。

> 若要将时间线上的间隙与具体代码段对应起来，可以在重要 GPU 工作周围插入 NVTX 范围标记。

接下来讨论如何在内核内部实现原子队列，以动态分配工作。它们对于在所有 SM 之间平衡任意工作负载、避免线程空闲非常重要。在此之前，先介绍原子计数器。

原子计数器

原子计数器是原子队列的基础，后者允许动态分配工作。

在现代 GPU 上，全局原子操作会在设备端 L2 缓存中处理并串行化。当目标缓存行驻留在 L2 中时，相比往返 DRAM，这可以降低延迟。原子计数器仍有延迟，并会在发生竞争时串行化；但没有竞争时，`atomicAdd` 可以留在片上极快地完成。两个线程递增原子的示例见图 12-1。

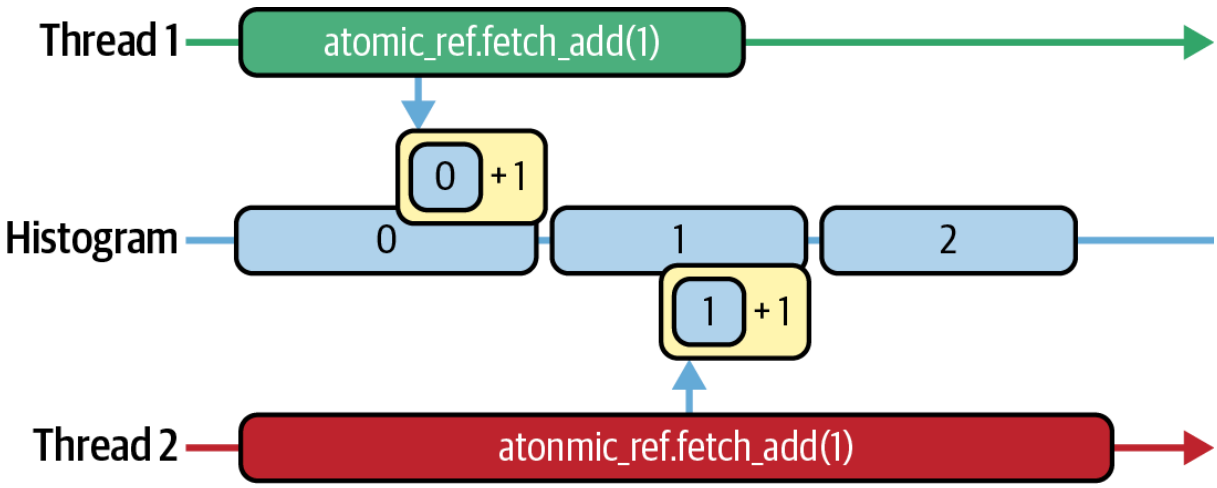


图 12-1. 直方图计算场景中跨多个线程的超高速片上原子内存加操作

但 `atomicAdd` 并非没有代价。它仍有延迟，在竞争下还可能使等待同一内存地址的线程串行化，因此 L2 也需串行处理更新。这会形成热点，需要优化。Nsight Compute 可帮助量化这一成本。

在 Nsight Compute 的 Memory Workload Analysis 部分，可找到 `atomic_transactions` 和 `atomic_transactions_per_request`。原子事务计数器表示 L2 缓存原子事务总数，包括由竞争导致的任何重放；每请求原子事务指标（即竞争比）表示每条 `atomicAdd` 指令平均生成的 L2 事务数。

当每次 `atomicAdd` 恰好触发一次 L2 事务时，`atomic_transactions_per_request` 会接近 1.0，表示开销已接近最低。如果该比值升到 1.0 以上，说明线程正在停顿并重试原子更新，而不是执行有用工作；每次重试都表示存在竞争。

这里的优化方法是按批次获取工作，摊薄原子操作成本。不要让每个线程或 warp 都执行一次 `atomicAdd`，而是让一次原子操作获取一组任务。下面展示批大小为 32 时的前后对比：

```
// Before batching
int idx = atomicAdd(&queue_head, 1);
if (idx < N) process(data[idx]);

// After batching
const int batchSize = 32;
int start = atomicAdd(&queue_head, batchSize);
for (int i = start; i < start + batchSize && i < N; ++i) {
    process(data[i]);
}
```

现在，一次原子更新就能为一个 warp（或线程块）分配一整段工作——本例中为 32 个项目——之后才再次访问计数器。每个批次仍需支付一次 L2 事务，但两次原子操作之间可以完成 32 倍的有用工作。

实践中，每个 warp 只让一个线程执行此 `atomicAdd`，获取下一个批次的起始索引。该线程随后将索引广播给 warp 中其他线程（例如使用 `__shfl_sync`），整个 warp 再并行处理这 32 个项目。这样每个 warp 只需一次原子操作，而不是每个线程一次，大幅降低竞争。

在 Nsight Compute 中，你会看到 `atomic_transactions` 大幅下降，transactions-per-request 重新接近 1.0。这表明你已经用持续计算替代了代价高昂的竞争。

在 L2 缓存原子操作速度极快的现代 GPU 上，得益于较高的 L2 带宽，即使批大小仅为 8 或 16，也可能消除大部分竞争。不过，仍应确认瓶颈没有被转移到其他位置。

要验证这一优化没有负面影响其他性能指标，请使用 Nsight Compute 的 Warp Stall Reasons 和 Register Pressure 报告，确保融合循环没有受寄存器溢出或共享内存银行冲突限制。

> 如果完成这些优化后原子操作仍是热点，可以考虑其他设计，例如每块计数器或分层归约式工作分配。

简而言之，按原子操作批量获取工作，可以让 GPU 的大量 warp 持续执行实际计算，而不是在一个跟不上处理速度的计数器前排队。

原子队列

现在使用全局原子计数器协调动态工作队列。目标是使用原子计数器和 `atomicAdd` 在所有 SM 之间平衡任意工作负载，让线程或 warp 都尽量不空闲。动态工作队列示例见图 12-2。

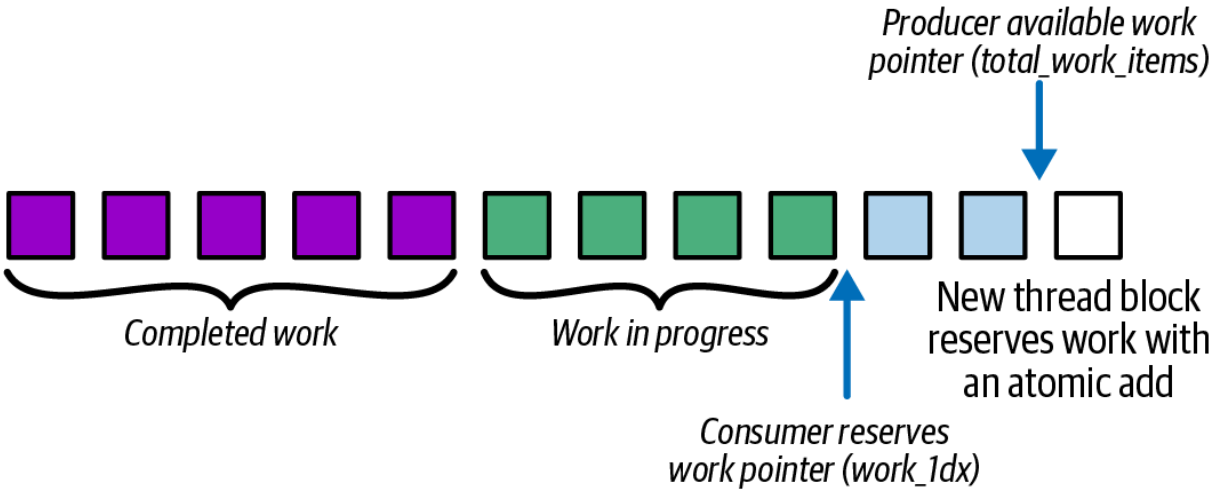


图 12-2. 使用原子计数器和 atomicAdd 作为动态工作队列，以平衡 SM 与 warp 间的工作负载

在下面的代码示例（computeKernel）中，每个线程根据 idx % 256 计算不同的迭代次数。idx % 256 较小的线程只做很少工作，而该值较大的线程要做很多工作。由于这种不均衡，线程会在不同时间完成，一些 SM 只能空等最长的线程结束。下面是每个线程采用静态、不均匀工作负载的代码：

```
// uneven_static.cu
#include <cuda_runtime.h>
#include <cmath>

__global__ void computeKernel(const float* input, float* output, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        // Each thread does a variable amount of work based on idx
        int work = idx % 256;
        float result = 0.0f;
        for (int i = 0; i < work; ++i) {
            result += sinf(input[idx]) * cosf(input[idx]);
        }
        output[idx] = result;
    }
}

int main() {
    const int N = 1<<20;

    float* h_in = nullptr;
    float* h_out = nullptr;
    cudaMallocHost(&h_in, N * sizeof(float));
    cudaMallocHost(&h_out, N * sizeof(float));

    for (int i = 0; i < N; ++i) h_in[i] = float(i) / N;

    float *d_in, *d_out;
    cudaMalloc(&d_in, N * sizeof(float));
    cudaMalloc(&d_out, N * sizeof(float));
    cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice);

    dim3 block(256), grid((N + 255) / 256);
    computeKernel<<<grid, block>>>(d_in, d_out, N);
    cudaDeviceSynchronize();

    cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_in);
    cudaFree(d_out);
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);

    return 0;
}
```

> PyTorch 没有用于 GPU 端动态工作分配的高级 API，因此需要通过自定义 CUDA 内核实现。为简洁起见，这里省略具体实现。

下面的优化版动态任务分派使用设备内存中的单个全局计数器作为 warp 级工作队列。通过批量原子操作，我们把它变成持久化的 warp 级工作队列。这样，提前完成的 warp 会立即获取下一批工作，而不是进入空闲状态：

```
// uneven_dynamic.cu

#include <cuda_runtime.h>

__device__ unsigned int globalIndex = 0;

// Warp-batched dynamic queue: 1 atomic per active warp
__global__ void computeKernelDynamicBatch(const float* input,
                                          float* output,
                                          int N) {
  // lane id in [0,31]
  int lane = threadIdx.x & (warpSize - 1);

  while (true) {
    // Elect an active leader each iteration (handles divergence safely)
    unsigned mask = __activemask();
    int leader = __ffs(mask) - 1;

    // Warp leader atomically grabs a contiguous batch for the whole warp
    unsigned int base = 0;
    if (lane == leader) {
      base = atomicAdd(&globalIndex, warpSize);
    }

    // Broadcast starting index to all active lanes in the warp
    base = __shfl_sync(mask, base, leader);

    unsigned int idx = base + lane;
    if (idx >= (unsigned)N) break;  // dynamic termination

    // Hoist invariants out of the variable trip-count loop
    // Note: You can also use __sincosf on Blackwell
    float s = sinf(input[idx]);
    float c = cosf(input[idx]);
    int   work = idx % 256;

    float result = 0.0f;
    #pragma unroll 1
    for (int i = 0; i < work; ++i) {
      result += s * c;
    }
```

```
        output[idx] = result;
        // loop continues until counter >= N
      }
    }

    int main() {
        const int N = 1 << 20;
        float *d_in, *d_out;
        cudaMalloc(&d_in,  N * sizeof(float));
        cudaMalloc(&d_out, N * sizeof(float));
        // Host buffers (pinned) for a realistic data path
        float *h_in = nullptr, *h_out = nullptr;
        cudaMallocHost(&h_in,  N * sizeof(float));
        cudaMallocHost(&h_out, N * sizeof(float));
        for (int i = 0; i < N; ++i) {
            h_in[i] = static_cast<float>(i % 1000);
        }

        // Copy inputs to device
        cudaMemcpy(d_in, h_in, N * sizeof(float),
            cudaMemcpyHostToDevice);

        // Reset global counter
        unsigned int zero = 0;
        // If you call this kernel repeatedly (e.g., in a loop),
        // reset 'globalIndex' to 0 before each launch.
        cudaMemcpyToSymbol(globalIndex, &zero,
            sizeof(unsigned int));

        // Launch with 256 threads per block
        dim3 block(256), grid((N + 255) / 256);
        cudaStream_t stream;

        cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);

        computeKernelDynamicBatch<<<grid, block, 0, stream>>>(d_in, d_out, N);

        cudaStreamSynchronize(stream);
        cudaStreamDestroy(stream);
        cudaDeviceSynchronize();

        // Copy results back and clean up
        cudaMemcpy(h_out, d_out, N * sizeof(float),
                   cudaMemcpyDeviceToHost);

        cudaFree(d_in);
        cudaFree(d_out);
        cudaFreeHost(h_in);
        cudaFreeHost(h_out);

        return 0;
    }
```

每个 warp 都从全局队列以原子方式领取大小为 `warpSize`（32 个线程）的下一批任务，并在循环中处理。这可以让 SM 尽量保持工作。该代码使用单个全局原子工作队列实现动态工作分配。

这里，每个 warp 会反复从全局计数器获取下一批的基准索引。每个 warp 的第一个线程（`if (lane==0)`，称为 warp leader）使用 `base=atomicAdd(&globalIndex, warpSize)` 执行原子加法，获得这个连续块的起始索引。随后，它使用 `__shfl_sync(__activemask(), base, 0)` 将基准索引广播给 warp 中其他线程。

换言之，不再让每个线程绑定固定元素索引，而是让每个 warp 从共享计数器获取连续的一段任务，然后使用 `idx = base + lane` 执行计算。

该 warp 中的所有线程都对动态获取的索引执行相同的 `sin`/`cos` 循环。因此，工作不再预先分配给线程，而是通过全局原子队列在运行时拉取并平衡。

请注意，当没有更多工作时，`if (idx >= N)` 边界检查会使 warp 退出，从而避免越界内存访问。除此之外，warp 中每个线程执行的 `sin`/`cos` 循环与静态版本完全相同。

在一个简单微基准中，取 `N = 1 << 20`、`work = idx % 256` 时，静态分配内核耗时约 200 ms，而动态队列版本约 100 ms。这 2 倍加速来自消除 SM 空闲时间和减少原子竞争。Nsight Compute 将 active SM cycles 定义为至少有一个活跃 warp 的耗时周期占比。

加速幅度会随工作不均衡程度变化；但只要性能分析显示 warp 空闲停顿、实际 occupancy 较低，或每项任务运行时间不均导致时间线出现明显间隙，就值得探索动态工作分配。在这些场景中，尤其是存在中等程度不均衡时，通常可能获得 10%–20% 的加速。

> 在极度不均衡的情况下，只需将静态索引替换为原子驱动的工作队列，就可能获得这类 2 倍加速。对于轻微不均衡，原子操作和 shuffle 的开销可能抵消收益。

简言之，动态工作分配能确保接近均匀的 SM 利用率，因为每个 warp 都会持续获取和处理新任务，直至计数器超过 `N`。这与许多 warp 远早于最慢 warp 完成、从而让硬件资源闲置的情形形成对比。

CUDA Graph

当流水线包含多个内核、拷贝、stream 事件记录和回调时，每次迭代都在主机上逐个启动它们，仍会产生 CPU 开销。CUDA Graph 允许你一次捕获整个工作流，并以近乎零的 CPU 开销反复重放。图 12-3比较了不使用（上）和使用（下）CUDA Graph 时的内核启动。

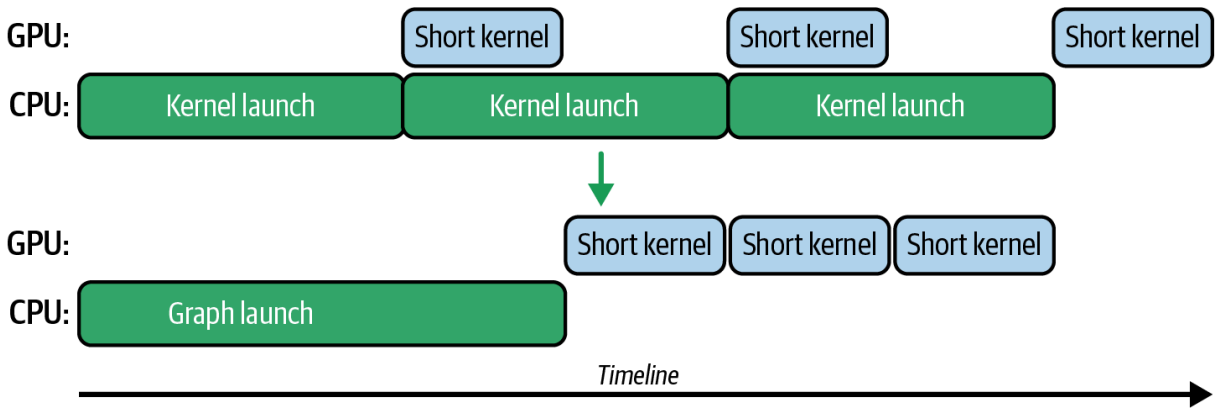


图 12-3. 不使用（上）和使用（下）CUDA Graphs 的内核启动时间线

为什么使用 CUDA Graph？首先，它能降低启动开销，多个小内核或拷贝基本可以通过一次 CPU 调用启动。其次，它能改善 GPU 上的调度：工作以批次提交，CUDA 驱动有机会减少操作之间的一些内部延迟。

此外，CUDA Graph 的依赖关系预先已知，因此不需要 CPU 在操作之间频繁同步。在内存传输场景中，CUDA Graph 可以正确地将异步拷贝与内核执行关联为依赖，而无需手动同步。它并不会天然比普通 stream 更好地重叠拷贝和内核，但能简化执行流程。

PyTorch、推理引擎与 CUDA Graph

PyTorch 等 AI 框架会在底层为深度学习模型中形状固定的部分利用CUDA Graph。具体来说，PyTorch 支持使用 `torch.cuda.Graph` 上下文捕获一系列操作，并持续优化内部实现，以便对可预测的代码段使用 CUDA Graph。

高性能推理引擎（如 vLLM 和 NVIDIA TensorRT-LLM）也可以利用 CUDA Graph，将模型执行捕获为针对不同序列长度范围和输入 batch 大小的一组预定义 graph。启用 graph 捕获后，这些系统通常会对输入分桶或填充，使其匹配受支持的 graph batch 大小，从而以固定形状重放捕获的 graph。这可以显著降低大规模生产推理工作负载的延迟。

例如，可以在启动或加载模型时为每个 batch 大小捕获一个 CUDA Graph；运行时，再启动与传入请求 batch 大小匹配的预捕获 graph。

> PyTorch 编译器 `mode='reduce-overhead'` 可能将符合条件的代码段包装进 CUDA Graph，以降低启动开销，但必须满足静态张量地址、仅 CUDA 区域等捕获要求。它不保证所有代码路径都会被 graph 化，而且由于使用缓冲区池，可能增加内存占用。应始终在自己的模型上进行 profile，以确认收益。

CUDA Graph 的内存池

使用 CUDA Graph 时，一个重要问题是内存管理。CUDA Graph 内的内存操作遵循与 CUDA stream 相同的规则。如果在捕获期间分配 GPU 内存，该分配就会成为 graph 执行的一部分。

通常应避免在图内部进行 GPU 内存分配，而是在图外预先分配内存。如图 12-4所示，PyTorch 等许多框架会将静态内存池与 CUDA Graphs 结合使用。静态内存池可防止内存分配成为被捕获图序列的一部分。

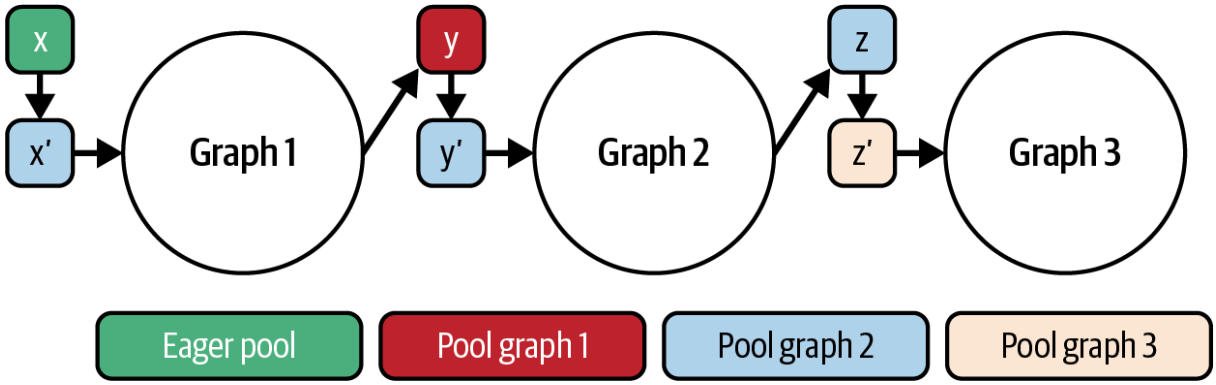


图 12-4. PyTorch 为 CUDA Graphs 使用静态内存池

CUDA Graphs 不会让单次内存复制或内核执行本身更快，但它们可像 CUDA 流一样，自动重叠图中相互独立的数据传输和计算。由于依赖图预先已知，这消除了每次迭代中的 CPU 调度。

使用 CUDA 流捕获 CUDA Graph

要捕获 graph，需要在 stream 上调用 `cudaStreamBeginCapture()`，将所有内存传输（`cudaMemcpyAsync()`）、内核启动、事件（`cudaEventRecord()`）和回调（`cudaLaunchHostFunc()`）入队，然后调用 `cudaStreamEndCapture()` 创建 CUDA Graph 定义（`cudaGraph_t`）。

随后，CUDA 驱动可以在每次迭代中使用 `cudaGraphLaunch()` 启动 CUDA Graph。由于驱动预先知道完整依赖图，因此可以直接在 GPU 上重放预构建的 stream 序列，启动开销很小。

> 下一节将讨论的 `cudaGraphExecUpdate` 允许在迭代之间大小、维度或指针发生变化时，对已捕获 graph 做有限修改。如果输入大小变化，只需更新 graph 节点参数，而不必为每种新输入大小重新捕获完整 graph。

即使流水线只有部分操作是重复的，CUDA Graph 也可以只捕获这一部分。例如，如果总是执行一次主机 → 设备拷贝、两个内核、再执行一次设备 → 主机拷贝，就可以只捕获该子图，并通过一次函数调用重放。

要重放 CUDA Graph，只需提供 stream 句柄，GPU 就能执行操作序列，无需额外 CPU 指令。这与内核间并发直接相关：CUDA Graph 可以混合异步拷贝、细粒度事件屏障和内核，保持复杂的重叠行为，同时完全消除 CPU 瓶颈。

通常应先进行一次重放试运行以确保正确性。创建可执行 graph `cudaGraphExec_t` 来实例化 graph，然后通过一次 graph 重放调用启动它。启动捕获的 graph 后，runtime 会按正确顺序在 GPU 上执行所有操作。

为了展示 CUDA Graph 的用法，考虑一个简单的内核序列。下面分别给出 C++ 和 PyTorch 中捕获并启动 CUDA Graph 的代码片段：

```
cudaStream_t stream;
cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
cudaGraph_t graph;
cudaGraphExec_t instance;

cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);

// Enqueue operations on 'stream' as usual
kernelA<<<grid, block, 0, stream>>>(d_X);
kernelB<<<grid, block, 0, stream>>>(d_Y);
kernelC<<<grid, block, 0, stream>>>(d_Z);

cudaStreamEndCapture(stream, &graph);
cudaGraphInstantiate(&instance, graph, nullptr, nullptr, 0);

// Now 'instance' can be launched in a loop
for (int iter = 0; iter < 100; ++iter) {
    cudaGraphLaunch(instance, stream);
    // No per-kernel sync needed; graph ensures dependencies
}
cudaStreamSynchronize(stream);

// (Destroy graph and instance when done)
cudaGraphExecDestroy(instance);
cudaGraphDestroy(graph);
```

在这段伪代码中，我们先在 CUDA stream 上开始捕获，按顺序在该 stream 中启动三个内核（A、B 和 C），结束捕获后得到一个 graph，然后实例化该 graph。

现在可以根据需要多次调用 `cudaGraphLaunch(instance, stream)`，重放完整的 A → B → C 序列。每次迭代只需支付一次启动成本，而不是三次独立启动。GPU 会按照记录顺序执行 A、B 和 C，在本例中即连续执行。循环结束后再同步，以确保所有迭代完成。

如前所述，PyTorch 等高级 AI 框架支持 CUDA Graph。PyTorch 让 Python 开发者无需深入了解 CUDA C++，也能获得接近原生 CUDA 的性能。下面使用 PyTorch 的 `torch.cuda.Graph` 上下文管理器捕获并重放操作：

```
import torch, time

X = torch.randn(1<<20, device='cuda')

# Define operations for reference
def opA(x): return x * 1.1
def opB(x): return x + 2.0
def opC(x): return x.sqrt()

# Persistent buffers for pointer stability
static_x = torch.empty_like(X)
static_y = torch.empty_like(X)
static_z = torch.empty_like(X)
static_w = torch.empty_like(X)

# Warm up once to initialize CUDA kernels and caches
_ = opC(opB(opA(X)))
torch.cuda.synchronize()

# Seed the static input before capture
static_x.copy_(X)

# Capture the graph
```

```
g = torch.cuda.CUDAGraph()
stream = torch.cuda.Stream()
torch.cuda.synchronize()
with torch.cuda.graph(g, stream=stream):
    # Record A then B then C using out parameters to avoid allocations
    torch.mul(static_x, 1.1, out=static_y)
    torch.add(static_y, 2.0, out=static_z)
    torch.sqrt(static_z, out=static_w)

# Replay the captured graph 100 times
for i in range(100):
    # If inputs change, copy new values into static_x before replay
    # static_x.copy_(new_X)
    g.replay()
```

在生产环境中，应分配持久的输入和输出缓冲区，使捕获的内核看到固定内存地址。例如，在捕获前创建 `static_y = torch.empty_like(X)`，然后在 graph 内使用 `static_y.copy_(opA(X))`。这样可以避免捕获期间分配内存，并满足 CUDA Graph 对指针稳定性的要求。PyTorch CUDA Graph 要求重放时捕获张量使用相同的内存地址。

在这个 PyTorch 示例中，我们定义了 `opA`、`opB` 和 `opC` 操作；实践中，它们可以是神经网络层或任意 GPU 操作。先运行一次预热 `opC(opB(opA(X)))`，确保所有内核、内存分配和库上下文（例如 cuBLAS/cuDNN）预先初始化。这是必要的，因为 CUDA Graph 捕获不会记录这些延迟初始化步骤。

> 跳过预热可能导致 graph 捕获失败，或在发生延迟初始化时引入意外停顿。

我们先运行一次序列预热 GPU，以初始化所有内核和库。然后在新的 CUDA stream 上，将前向（`A`）、变换（`B`）和反向（`C`）操作放入 `with torch.cuda.graph(g, stream=stream)`: Python 上下文管理器块，从而捕获到单个 `torch.cuda.CUDAGraph` 中。捕获后，调用 `g.replay()` 100 次，每次迭代只需一次主机调用即可启动完整的 A → B → C 流水线。结果汇总于表 12-1。

表 12-1　CUDA Graph 对迭代开销的影响

指标	使用 CUDA Graph 前	使用 CUDA Graph 后
每 100 次迭代的 CPU 启动调用	300 次独立内核启动	100 次图重放（每次迭代 1 次）
主机同步调用	300 次 `cudaDeviceSynchronize` 调用	0
内核之间的平均 GPU 空闲时间	每次迭代约 3 µs 间隙	0 µs（连续背靠背执行）
端到端迭代延迟	~1.00 ms	约 0.75 ms（快 25%）

> 注意：所有指标表中的数值仅用于说明概念。不同 GPU 架构的实际基准测试结果请参阅 GitHub 仓库。

这里可以看到，CUDA Graph 消除了每次迭代的 CPU 调度和主机—设备握手。这是因为每次迭代的 GPU 工作被批量放入一次 `g.replay()` 调用，而不是三次独立的内核启动。因此，迭代速度提高了 25%；CPU 只需发出轻量级重放命令，并与 GPU 保持完全异步。

使用 CUDA Graph 时存在一些常见陷阱，需要妥善处理。例如，如果工作负载大小发生变化，已捕获的 graph 可能不再有效，需要重新捕获或调用下一节介绍的 `cudaGraphExecUpdate`。

某些 CUDA API 调用（例如内存分配以及主机—设备同步原语）通常不应包含在 graph 捕获中。尽管现代 CUDA Graph 支持在已捕获 graph 内执行有限的内存管理操作，但仍建议在捕获前完成所有内存分配，并确保 graph 使用的数据始终位于相同内存地址。

> graph 执行期间必须保持内存地址不变，这是 PyTorch 等框架将静态内存池与 CUDA Graph 一起使用的主要原因。例如，PyTorch 提供 `torch.cuda.graph_pool_handle()` API，为指针稳定的 CUDA Graph 捕获创建专用内存池。使用独立分配器池可以确保张量地址在捕获和重放期间保持固定，满足指针稳定性要求。迭代之间应将输入拷贝到静态张量中更新，不要每次迭代重新分配张量。

还应避免在 CUDA Graph 捕获中包含主机端回调或不受支持的操作，包括 `print()`、随机数生成器（RNG）调用、嵌套捕获和新的内存分配。这是因为 graph 必须记录纯粹且确定性的 GPU 工作序列。

此外，捕获中使用的所有张量必须已在固定地址上分配且形状固定。在捕获期间调整大小或调用 `cudaMalloc` 都会破坏图。

动态更新 Graph

记录 CUDA Graph 后，并不需要仅因某些启动参数改变就丢弃它。无需重新捕获，而是调用图更新 API，直接在现有图中更新网格/块维度、指针地址或内核参数。图更新 API 包括 `cudaGraphExecUpdate` 和更底层的 `cudaGraphExecKernelNodeSetParams`。

`cudaGraphExecUpdate` 允许将一个内核节点替换为形状相同的新节点。例如，可以换入不同的融合内核实现，只要形状相同即可。CUDA runtime 会验证这些修改，并允许立即重放修改后的 graph，从而避免完整捕获的成本。

> 截至本书撰写时，不能任意添加或删除节点。如果更新超出既有 graph 所能处理的范围，运行时会返回错误；此时必须重新捕获一个 graph。

例如，考虑前文使用最大 batch 大小的三内核 A → B → C graph。在每次推理循环中，只需更新内核 B 的启动维度以匹配当前 batch，然后重放同一个 graph。对于整体流水线固定但少数参数可能变化的半静态工作负载，这能降低开销。

实践中，典型工作流分三步。首先使用预期最大尺寸（例如最大 batch）捕获模板 graph，比如最大 batch 大小为 128。之后如果收到 batch 大小为 64 的请求，就调用 `cudaGraphExecUpdate` 将启动参数调整为 64，并可将内存指针更新为更小的缓冲区。

使用 `cudaGraphExecUpdate` 后，即使内核参数、grid/block 维度或内存地址发生变化，也无需重建 graph。更新只需几微秒，因此仍可保持 CUDA Graph 重放低于 100 μs 的快速启动开销，同时保留运行时调整关键参数的灵活性。注意，不兼容的修改会返回错误状态，并要求重新捕获。

例如，若确实需要改变图结构来指定不同数量的内核，可退回到“重新捕获再更新”的工作流。此时，用 `cudaStreamBeginCapture` 和 `cudaStreamEndCapture` 包裹代码的一次迭代以构建新图；随后运行中的微小调整再使用较轻量的 `cudaGraphExecUpdate`。

实际上，动态 graph 更新可以让你完全在 GPU 上创建“参数化”或条件执行路径。当 GPU 工作循环频率很高、但只有少数参数（例如 batch 大小）变化时，可以一次捕获、快速更新，同时获得较低 CPU 开销和工作负载所需的适应性。

设备发起的 CUDA Graph 启动

现在你已经了解如何低开销地从 CPU 启动并调整已捕获流水线，下一步就是完全移除 CPU 的启动决策。借助设备发起的 CUDA Graph 启动，正在运行的 GPU 内核可以直接在设备上触发预先记录的 graph，完全绕过主机。

要启用设备端启动，先像往常一样在主机上捕获 graph。然后使用 `cudaGraphInstantiate` 实例化，并传入 `cudaGraphInstantiateFlagDeviceLaunch`。实例化后，在任何设备端启动之前，先在主机 stream 上使用 `cudaGraphUpload` 上传可执行 graph。

随后使用 `cudaGraphUpload` 将 graph 上传到 GPU 内存；必须在 GPU 启动 graph 之前完成。（未上传 graph 就尝试设备端启动会导致错误。）

实践中，可以在持久内核或动态内核中嵌入“graph launch”节点，或调用设备端 graph API。条件满足时，GPU 会在自己拥有的 stream 上启动整个 graph，如图 12-5所示。

设备发起的 graph 启动让数据驱动工作流完全留在 GPU 上。由内核而不是 CPU 计算决策条件，因此可以直接启动下一个 graph，消除 CPU 往返并进一步降低延迟。

由于 graph 已驻留在 GPU 上且不需要 CPU-GPU 握手，设备发起启动可以将主机调度移出关键路径，降低受主机限制的循环的端到端延迟。实践中，设备发起的 CUDA Graph 启动相较等价的主机端 graph 启动，启动延迟曾显示出约 2 倍的降低；即使 graph 规模或复杂度增加，开销也基本保持不变。

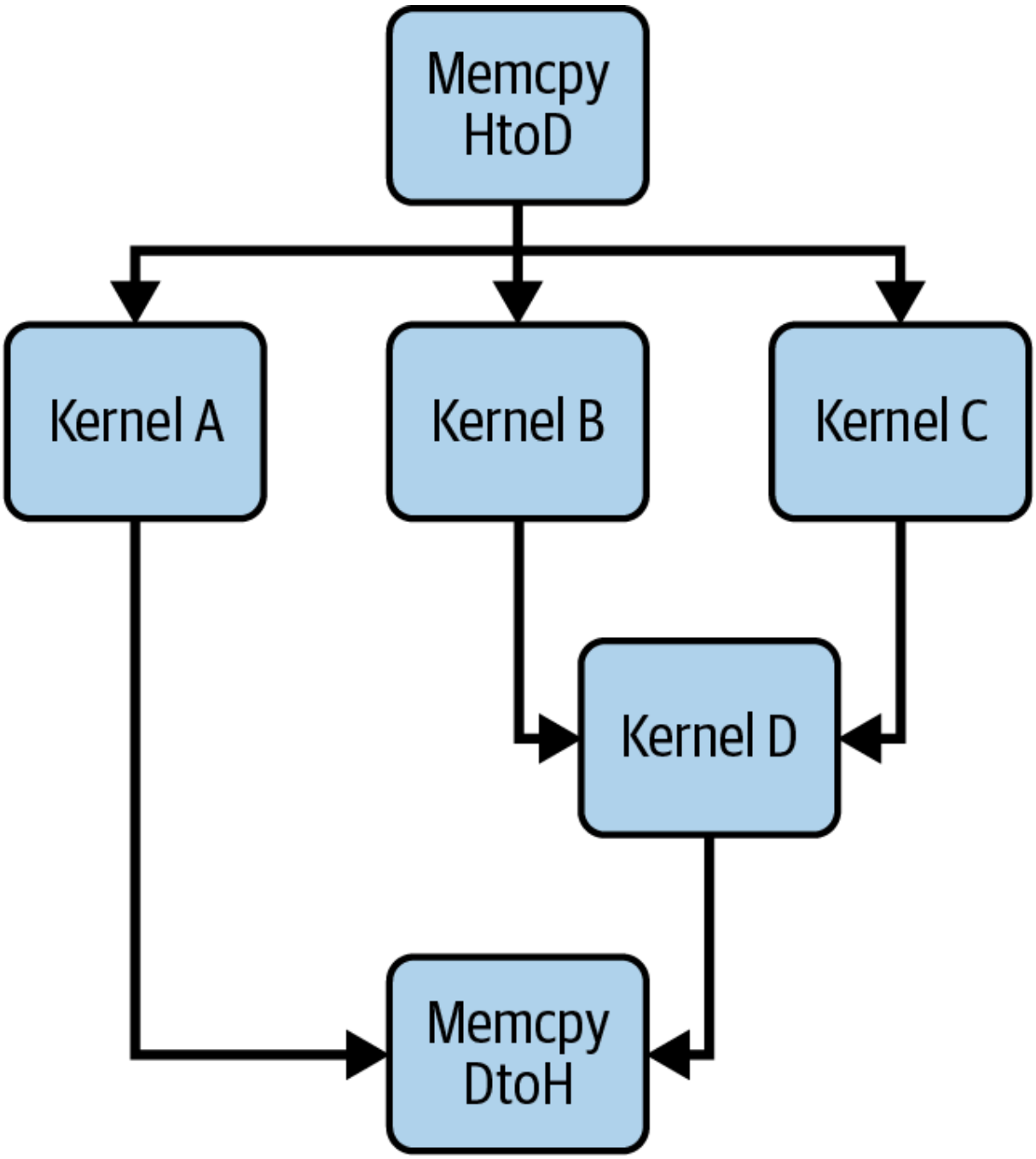


图 12-5. 由 CUDA Graph 启动的操作序列（内核和数据传输节点）及其依赖关系（边）

设备启动 graph 的延迟不受 graph 中节点数或并行分支数影响；相比之下，主机启动 graph 的延迟会因 graph 变大而受到 CPU 调度开销影响并增加。

此外，设备启动可以很好地随 graph 宽度扩展。增加更多并行节点时，主机端启动会产生额外同步成本，而设备启动延迟仍近似保持平坦。

调试设备启动的 graph 可能比较棘手，但 Nsight Systems 等工具会在 GPU 时间线上将子 graph 显示为独立的工作 stream。建议在父内核中 `cudaGraphLaunch` 调用前后使用 NVTX 标记，标出设备启动发生的位置，以便验证 graph 是否按照相对于父线程的预期运行。

在设备代码中，可以使用简单 API `cudaGraphLaunch(graphExec, stream)` 启动 graph。runtime 使用特殊的保留 `cudaStream_t` 值区分以下启动模式：“fire-and-forget”（`cudaStreamGraphFireAndForget`）、“tail”（`cudaStreamGraphTailLaunch`）和“sibling”（`cudaStreamGraphFireAndForgetAsSibling`）。这些模式会自动在 CUDA stream 中强制执行正确顺序，无需主机介入。

在 fire-and-forget 启动中，子 graph 会立即开始执行，并与发起它的父内核并发运行。父内核不会等待子 graph 完成，类似于启动一条独立工作线程。fire-and-forget 适合从内核内部生成异步任务。

> 一个 graph 在执行期间最多可以包含 120 个 fire-and-forget graph。

相比之下，设备发起的 graph tail 启动会推迟 graph 执行，直到发起内核到达同步点或完成。它实际上将 graph 作为延续排队到当前内核之后运行，如图 12-6所示。

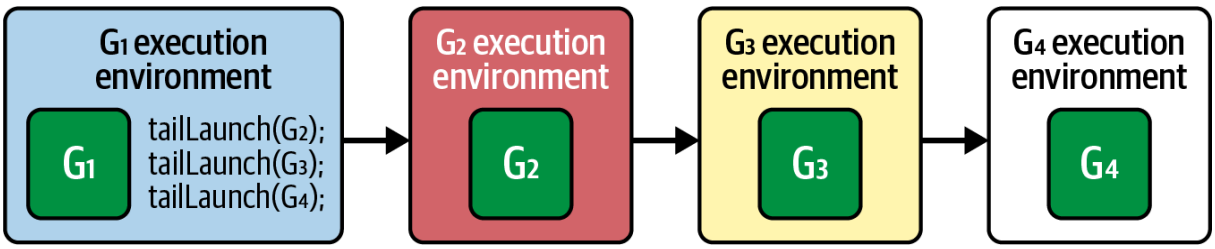


图 12-6. 给定图排入队列的尾部启动将按入队顺序逐一执行

tail 启动对于实现驻留 GPU 的工作调度器尤其有用。持久“scheduler”内核可以 tail 启动一个 graph，待该 graph 完成后再重新启动自身。这种技术无需主机再次调用，就能在 GPU 上形成循环。内核调用 `cudaGetCurrentGraphExec()` 获取自身正在执行的 graph 句柄，然后使用 `cudaGraphLaunch(..., cudaStreamGraphTailLaunch)` 再次将自身入队。

此外，tail graph 还可以执行额外的 tail 启动。在这种情况下，新的 tail 启动会先于之前 graph 的 tail 启动执行，如图 12-7所示。

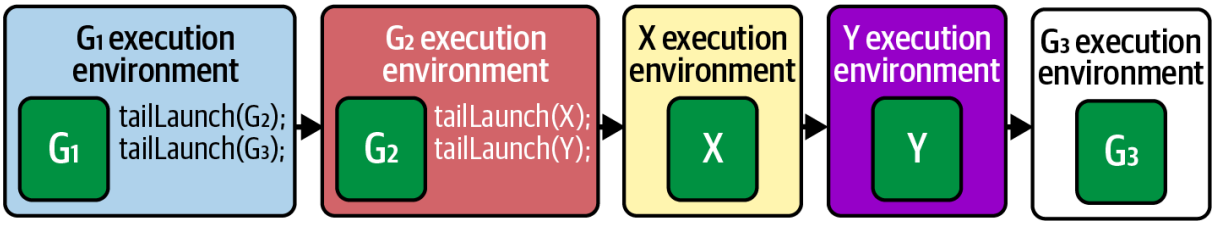


图 12-7. 从多个图排入队列的尾部启动

> 一个 CUDA Graph 中最多可以排队 255 个待处理 tail 启动。不过，对于 self-tail-launch（例如 graph 将自身重新入队），同一时间最多只能有一个待处理的 self-tail-launch。

sibling 启动是 fire-and-forget 的一种变体：被启动的 graph 作为父 graph 的同级执行，而不是子 graph；同时，sibling 运行在父 graph 的 stream 环境中。这意味着它会立即独立运行，但不会延迟父 graph 的任何 tail 启动，如图 12-8所示。

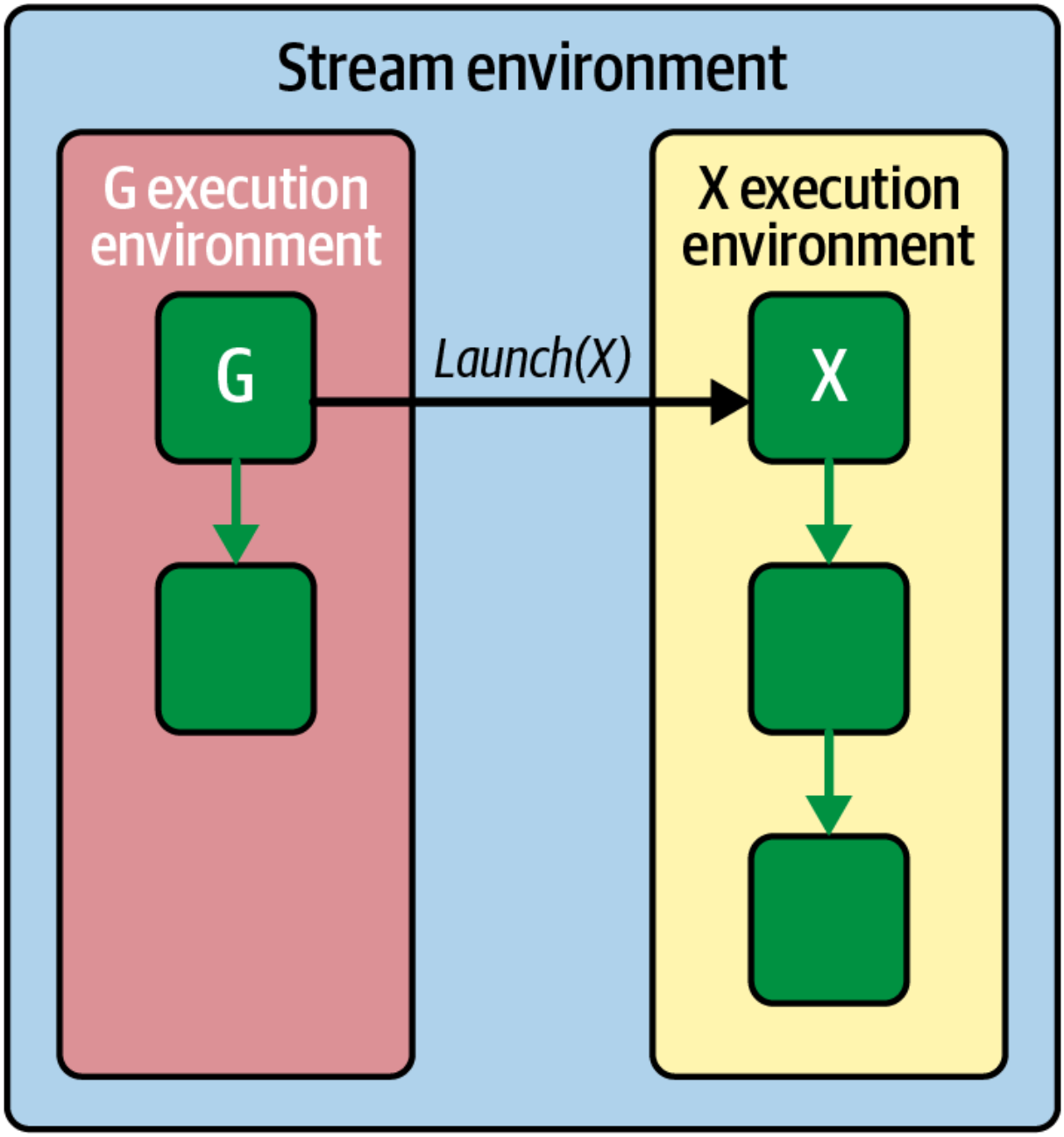


图 12-8. 父图流环境中的同级图启动

对于此模式，可以使用 `cudaGraphLaunch(graphExec, cudaStreamGraphFireAndForgetAsSibling)` 以“sibling 模式”启动，将 graph 作为当前 graph 执行环境的同级提交。

使用设备发起 CUDA Graph 时，需要仔细管理依赖。例如，如果被启动内核必须使用 graph 的结果，适合采用 tail 启动，因为父内核会暂停，直到 graph 工作完成。相反，如果被启动 graph 只是辅助任务，fire-and-forget 模式可以让父内核无需等待即可继续。

实践中，设备发起的 CUDA Graph 开启了新的模式。例如，设想一个 GPU 压缩流水线：内核必须根据数据内容在不同压缩算法之间做选择。GPU 内核无需结束并通知 CPU 启动选定的压缩内核，而是可以直接启动与“LZ 压缩”或“Huffman 压缩”对应的预记录 graph。

在此压缩示例中，GPU 不会空闲等待 CPU 作出决定。接下来看看另一种有用模式：它在持久化内核内部，将原子队列/计数器与设备发起、尾部启动的 CUDA Graphs 结合，用于 LLM 推理调度。

原子队列与设备发起 CUDA Graph 的内核内持久调度

我们可将前面的原子计数器工作队列与设备发起的图尾部启动结合。设想一个 LLM 推理循环用例：它通过捕获包含 Transformer 块前向传播（注意力 + 前馈）的图，使用 CUDA Graph 执行解码。

轻量级持久 scheduler 内核可以使用 `atomicAdd(&queueHead,1)` 获取下一个工作项。随后，它 tail 启动预捕获的 decode CUDA Graph 来计算输出，并立即循环回队列获取下一项。

每个 CUDA Graph 完成后，内核内 scheduler 循环使用 `atomicAdd(&queueHead,1)` 获取下一个索引，并 tail 启动另一个 decode graph。这实际上创建了完全驻留 GPU 的调度器，无需接触 CPU 就能决定并执行任务。

通过串联这些 tail 启动，每个 token 都能从头到尾在设备上处理，CPU 开销几乎为零。由于 CPU 不再进入关键路径，SM 可以保持较高利用率，单 token 延迟降低，并能动态适应不同序列长度和 batch 大小；只需更新 graph 参数或切换预记录 graph 即可。

条件 Graph 节点

在传统 CUDA Graph 中，每个节点及其依赖都在捕获时固定，因此任何决策逻辑都必须回到主机。条件 graph 节点打破了这种刚性：它根据与节点关联的小型“条件句柄”，将分支决策推迟到 GPU 自身执行。

graph 执行时，GPU 会计算该句柄，并选择性地运行某个主体子图（或循环执行它），全程无需将控制权交回 CPU。具体而言，条件 graph 节点允许把 IF、IF/ELSE、WHILE、SWITCH 控制流直接嵌入 CUDA Graph，在 GPU 设备上运行。条件节点可以消除主机往返，并可能在现代 GPU 上带来显著性能收益。

本质上，条件 graph 节点允许根据设备内核计算的值控制 graph 执行，而无需 CPU 介入。这使复杂分支工作流可以实现为一次可重复的 graph 启动。CUDA Graph 支持多种条件节点，如图 12-9所示：

IF

条件非零时，将单个主体 graph 执行一次。

IF/ELSE

指定两个主体 graph；条件为真时选择一个，为假时选择另一个。

WHILE

只要条件保持非零，就反复执行主体 graph，并在每次迭代后重新检查。

切换（SWITCH）

保存 N 个主体 graph；当条件等于 i 时执行第 i 个；若条件 $\geqslant N$，则完全跳过执行。

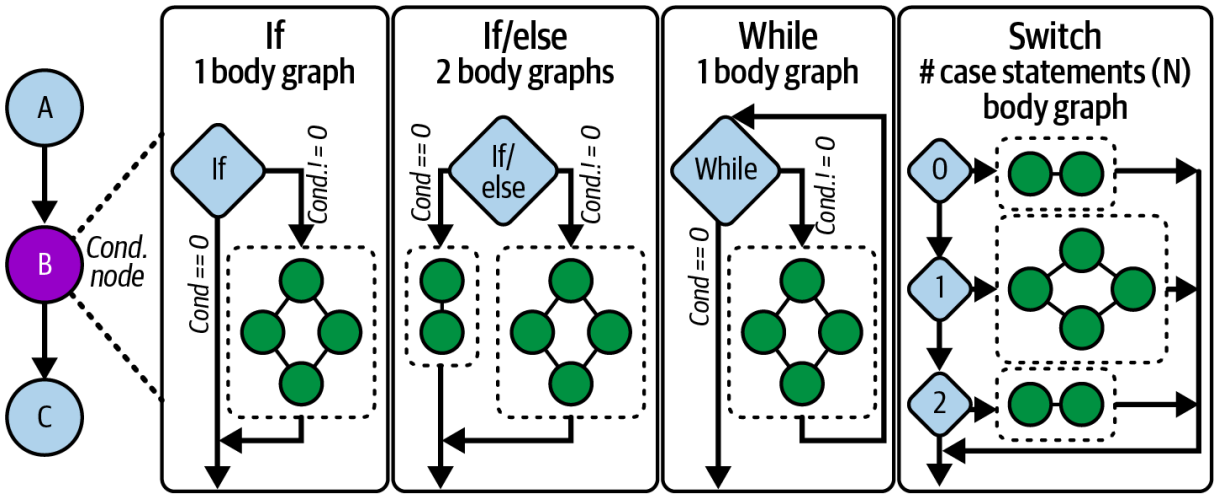


图 12-9. 条件图节点的类型

下面的示例展示如何创建并填充 IF 条件节点。注意使用 `cudaGraphSetConditional` 写入控制 IF 节点的标志。本例中的条件检查总和是否大于给定阈值；如果数据满足条件（`flag = 1u`），就运行下一个子图，否则条件节点不运行该子图：

```
#include <cuda_runtime.h>
#include <cstdio>

// Device kernel that computes and sets the condition handle
__global__ void setHandle(cudaGraphConditionalHandle handle,
                          int *data, size_t N) {
    // Example threshold
    constexpr int threshold   = 123456;

    // Test whether the sum of data exceeds a threshold
    //     using a custom reduce_sum() function
    //     (recommended to implement this with
    //     CUB's DeviceReduce::Sum routine.)
    unsigned int flag =
        (reduce_sum(data, N) > threshold) ? 1u : 0u;

    cudaGraphSetConditional(handle, flag);
}

// A simple body kernel that runs only if flag != 0
__global__ void bodyKernel() {
    printf("Conditional body executed on GPU!\n");
}

int main() {
    cudaStream_t stream;
    cudaStreamCreateWithFlags(&stream,
        cudaStreamNonBlocking);

    // 1) Create the graph
    cudaGraph_t graph;
    cudaGraphCreate(&graph, 0);

    // 2) Create a condition handle associated with graph
    cudaGraphConditionalHandle condHandle;
    cudaGraphConditionalHandleCreate(&condHandle, graph);
```

```
    // 3) Add the upstream kernel node to set the handle
    cudaGraphNode_t setNode;
    cudaKernelNodeParams setParams = {};
    setParams.func          = (void*)setHandle;
    setParams.gridDim       = dim3(1);
    setParams.blockDim      = dim3(32);

    // 4) Allocate input data
    constexpr size_t N              = 1 << 20;
    int*             d_data         = nullptr;
    cudaMalloc(&d_data, N * sizeof(int));

    void* setArgs[] = { &condHandle, &d_data, &N };
    setParams.kernelParams  = setArgs;
    cudaGraphAddKernelNode(&setNode, graph, nullptr, 0,
        &setParams);

    // 5) Add the IF conditional node
    cudaGraphNode_t condNode;
    cudaConditionalNodeParams ifParams = {};
    ifParams.handle = condHandle;
    ifParams.type   = cudaGraphCondTypeIf;
    // One-node body graph, in this case
    ifParams.size   = 1;
    cudaGraphAddConditionalNode(&condNode,
                                graph,
                                &setNode,
                                1,
                                &ifParams);

    // 6) Populate the body graph: one kernel that prints a message
    cudaGraph_t bodyGraph = ifParams.phGraphOut[0];
    cudaGraphNode_t bodyNode;
    cudaKernelNodeParams bodyParams = {};
    bodyParams.func         = (void*)bodyKernel;
    bodyParams.gridDim      = dim3(1);
    bodyParams.blockDim     = dim3(32);
    cudaGraphAddKernelNode(&bodyNode, bodyGraph, nullptr,
        0, &bodyParams);

    // 7) Instantiate, upload, and launch the graph on the device
    cudaGraphExec_t graphExec;
    cudaGraphInstantiate(&graphExec, graph, nullptr, nullptr,
    cudaGraphInstantiateFlagDeviceLaunch);
    cudaGraphUpload(graphExec, stream);
    cudaGraphLaunch(graphExec, stream);


    // 8) Wait for completion and clean up
    cudaStreamSynchronize(stream);
    cudaGraphExecDestroy(graphExec);
    cudaGraphDestroy(graph);
    cudaStreamDestroy(stream);

    cudaFree(d_data);

    return 0;
}
```

这里使用 `cudaGraphCreate` 创建新的 CUDA Graph，用来容纳之后的所有节点。随后使用 `cudaGraphConditionalHandleCreate` 创建条件句柄。（它将一个可在设备上设置的小整数值

与 graph 关联起来。）

随后添加上游内核 `setHandle`。该内核只在一个线程上运行，以避免竞态条件，然后调用 `cudaGraphSetConditional` 写入控制 IF 节点的标志。

使用 `cudaGraphAddConditionalNode` 添加 IF 条件节点，并指定 `cudaGraphCondTypeIf` 和 `size=1`。这定义了计划支持的条件分支或迭代数量。

这里分配一个空子图（`body`），当条件标志返回非零时执行。从 `ifParams.phGraphOut[0]` 取得主体 graph，并添加 `bodyKernel` 来填充它；该内核执行时只打印一条消息。

构建 graph 后，调用 `cudaGraphInstantiate` 生成可执行 graph 对象。若要从设备代码启动 graph，必须使用 `cudaGraphInstantiateFlagDeviceLaunch` 标志实例化，并在任何设备端启动之前通过 `cudaGraphUpload` 上传。

在 CUDA stream 上使用 `cudaGraphLaunch` 启动后，会依次触发上游 set-handle 内核、条件检查，并在标志已设置时执行主体内核。所有这些都直接发生在 GPU 上，如图 12-10所示。

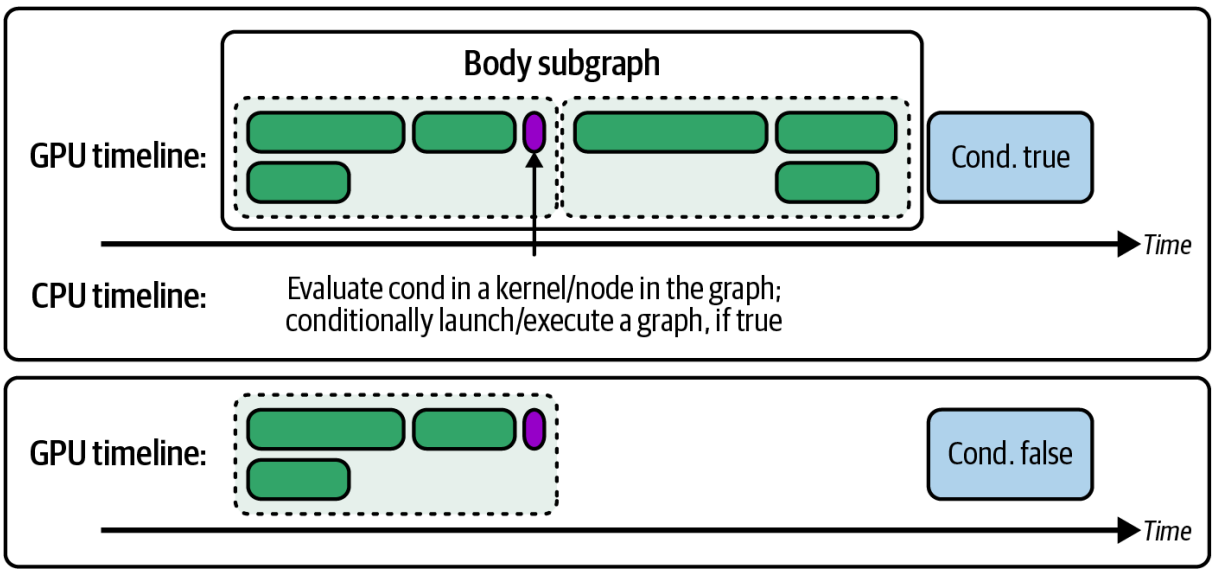


图 12-10. 满足条件时进行额外处理（body 子图）

随后使用 `cudaStreamSynchronize` 同步 stream 并等待完成。最后销毁实例化 graph、graph 本身和 stream，完成清理。

为尽量避免竞态条件，应始终只在一个线程中设置条件（例如 `if (threadIdx.x == 0)`），并确保前序内核刷新内存，使该值在条件节点执行前可见。

条件节点也可以嵌套。例如，WHILE 节点的主体可以包含 IF 节点，如图 12-11所示。这样可以实现多级决策逻辑，而无需往返 CPU。

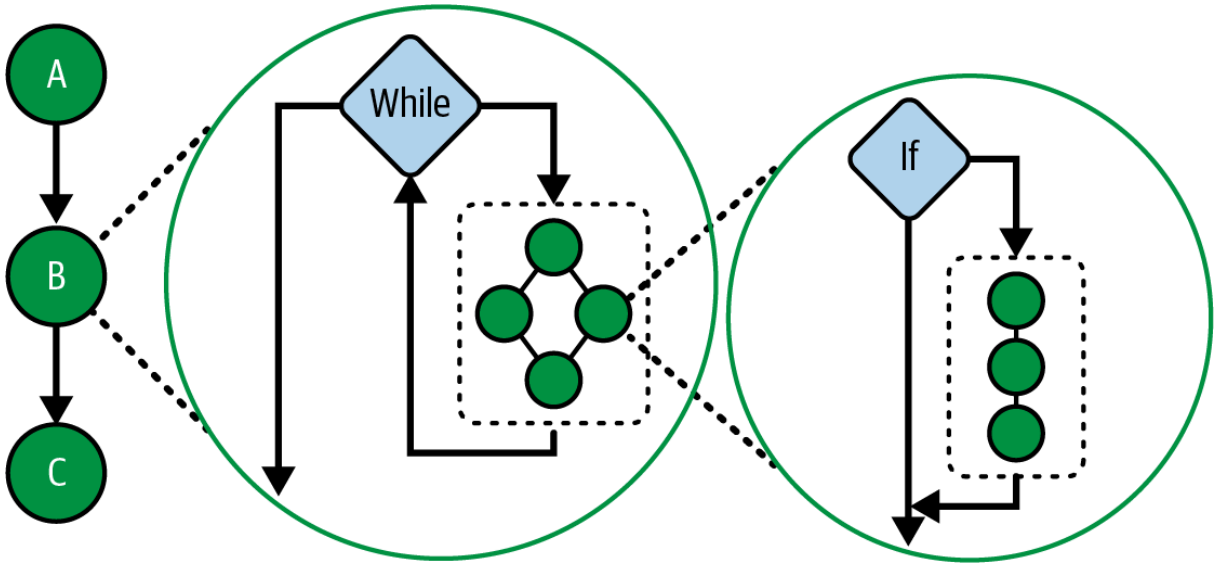


图 12-11. 嵌套条件图节点

简而言之，应使用条件 graph 节点将决策留在 GPU 上，降低 CPU 开销，并直接在 CUDA Graph 中表达复杂控制流。由于 graph 创建成本可以摊薄到多次迭代中，将动态工作流完全放在设备端可能带来显著性能改善。

> 截至本书撰写时，PyTorch 的 CUDA Graph API 尚不能直接通过 Python 创建条件 CUDA graph 节点。框架和工具对条件 graph 执行的支持仍在演进；若要在 PyTorch 中实现 if/while/switch 节点，目前需要自定义 C++ 集成。

动态并行

前面介绍了设备发起的 CUDA Graph 启动如何在极少 CPU 参与的情况下捕获并重放固定操作序列。但该模型要求预先知道完整执行流，而这并不总是可能。许多工作负载会根据输入数据、中间结果或问题复杂度在运行时改变形状，这正是动态并行（DP）的用武之地。

DP 让 GPU 内核能够自行生成新工作，而不必等待 CPU。CUDA Graph 要求预先知道完整流水线，而 DP 允许正在运行的“父”内核检查自己的输出，并即时决定接下来启动多少个“子”内核。对于分层归约、自适应网格细化和图遍历等真正不规则的问题，这一点尤为重要，因为只有在处理数据的过程中，后续任务数量才会逐渐明确。

设想一个推理流水线：某些输入偶尔需要特殊的“插件”模型评估。在 CPU 驱动流程中，需要运行 Kernel A，将结果拷回主机，决定是否启动 Kernel B，再发出启动；GPU 在这次往返期间会空闲。使用 DP 后，Kernel A 检查自己的输出，并在条件成立时直接在设备上启动 Kernel B。整个决策与分发都发生在一个驻留 GPU 的工作负载内部，从而缩短空闲间隙，让 SM 保持忙碌。

在 LLM 中，大多数 token 遵循标准 Transformer 路径，但少数 token 需要辅助 attention block。启用 DP 的 Transformer 内核可以在运行时识别这些特殊 token，只为相应位置启动额外 attention 内核，无需主机干预。NVIDIA 库已在自适应算法的类似模式中利用 DP：数据流经计算过程时动态产生新子任务。

如果性能分析时间线出现 `Kernel A → GPU 空闲间隙 → Kernel B` 的连续模式，而空闲时间对应 CPU 准备下一次启动，那么 DP 可能适用。用设备端启动替代该间隙，有助于让 SM 保持工作并显著降低依赖阶段之间的延迟。

当然，DP 的性能收益并非没有代价。每次子内核启动都会使用 GPU 调度资源，并需要额外栈空间。为避免“stack overflow”错误，可能需要使用 `cudaDeviceSetLimit(cudaLimitStackSize, newSize)` 增大 runtime 栈大小。达到默认限制时，CUDA 会发出警告。

与此相关，CUDA 对待处理的子内核启动数量设有最大限制。默认情况下，同一时间允许 2,048 个未完成的设备启动，但该值可以配置。

如果父内核通过大型循环启动数千个微小内核，需要生成超过 2,048 个子内核，可以使用 `cudaDeviceSetLimit(cudaLimitDevRuntimePendingLaunchCount, newLimit)` 提高限制。否则，超过默认的 2,048 限制会导致 runtime 错误。实践中，默认值通常足以满足大多数用途，但极端场景必须考虑这一限制。

> 在 GPU 上生成大量子内核时，务必监控设备内存使用量，因为每个待处理的子内核启动都会预留资源，并可能超过 GPU 硬件的硬性限制。

由于 DP 会增加一些指令级开销，最好只在持久内核、stream 或 CUDA Graph 等静态编排会让 GPU 在 CPU 决定下一步时空闲的场景中使用。换言之，如果工作负载确实是固定操作序列，通常更适合将其捕获为 CUDA Graph，并在设备端重放。

当工作由数据本身动态产生时，DP 可以让调度和执行全部留在 GPU 上，为嵌套、数据依赖或不可预测的并行任务带来更好的扩展性和更低的端到端延迟。

> 由于 DP 每次启动都有开销，并会占用待处理启动槽位，因此应在新并行性于运行时从数据中产生、且无法表达为预记录 CUDA Graph 时使用 DP。相反，如果控制流可表达且会重复执行，应优先采用设备启动的 CUDA Graph，因为它能摊薄成本。

下面比较一个简单父子工作流的两种实现。主机驱动版本先启动父内核并等待其完成，然后由 CPU 发出两个子内核启动；GPU 会在这些决策间隙中空闲：

```
// dp_host_launched.cu
#include <cuda_runtime.h>

__global__ void childKernel(float* data, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        data[idx] = data[idx] * data[idx];
    }
}

__global__ void parentKernel(float* data, int N) {
    // Parent does setup work. Here, CPU decides on child launches.
```

```
    if (threadIdx.x == 0 && blockIdx.x == 0) {
        // maybe mark regions or compute flags here
    }
}

int main() {
    const int N = 1 << 20;
    float* d_data;
    cudaMalloc(&d_data, N * sizeof(float));
    // ... initialize d_data ...

    // 1) Launch parent and wait
    cudaStream_t s; cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);

    parentKernel<<<1,1,0,s>>>(d_data, N);

    cudaStreamSynchronize(s);

    // 2) CPU splits work in half and launches children
    int half = N / 2;
    childKernel<<<(half+255)/256,256>>>(d_data,      half);
    childKernel<<<(half+255)/256,256>>>(d_data+half, half);
    cudaStreamSynchronize(s);
    cudaStreamDestroy(s);

    cudaFree(d_data);
    return 0;
}
```

在主机驱动版本中，GPU 运行 `parentKernel` 后进入空闲，而 CPU 依次准备并启动各个 `childKernel`。注意父内核之后和子内核之间显式调用的 `cudaDeviceSynchronize()`；这些调用会造成应当消除的空闲间隙，如图 12-12所示。

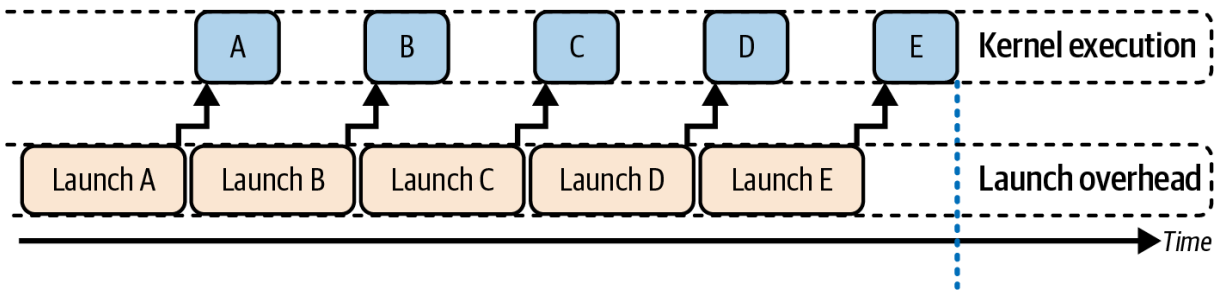


图 12-12. 子内核启动造成的空闲间隙

相比之下，设备启动的 DP 版本允许父内核在设备上生成子内核。父子内核启动之间不需要主机同步；父内核发出的子内核启动会隐式地将子内核排队，只在最后同步，如以下代码所示：

```
// dp_device_launched.cu
// Dynamic parallelism requires relocatable device code enabled with -rdc=true.

#include <cuda_runtime.h>

__global__ void childKernel(float* data, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        data[idx] = data[idx] * data[idx];
    }
}
```

```
__global__ void parentKernel(float* data, int n) {
    // Launch children from a single thread to avoid duplicate launches.
    if (blockIdx.x == 0 && threadIdx.x == 0) {
        const int threadsPerBlock = 256;
        const int firstHalfCount  = n / 2;
        const int secondHalfCount = n - firstHalfCount;

        const int blocksFirst  = (firstHalfCount  + threadsPerBlock - 1)
                                 / threadsPerBlock;
        const int blocksSecond = (secondHalfCount + threadsPerBlock - 1)
                                 / threadsPerBlock;

        // Device launched child kernels.
        // No device side cudaDeviceSynchronize is needed.
        childKernel<<<blocksFirst,  threadsPerBlock>>>(data,
                                                       firstHalfCount);
        childKernel<<<blocksSecond, threadsPerBlock>>>(data + firstHalfCount,
                                                       secondHalfCount);
        // Parent kernel will not finish until both children finish.
    }
}

int main() {
    const int N = 1024 * 1024;    // 1M elements, avoids bit shifting for clarity
    float* d_data = nullptr;

    cudaMalloc(&d_data, N * sizeof(float));
    // Initialize to zero as a concrete, valid initialization.
    cudaMemset(d_data, 0, N * sizeof(float));

    // Launch parent on the default stream.
    parentKernel<<<1, 1>>>(d_data, N);

    // Wait for completion without cudaDeviceSynchronize. Sync stream instead.
    cudaStreamSynchronize(0);

    cudaFree(d_data);
    return 0;
}
```

这里，`parentKernel` 直接在 GPU 上发出两次子内核启动。主机只提交一个内核并等待一次。父内核完成时，设备 runtime 会确保所有已启动子内核都已完成，然后再继续。

> 注意，这个动态并行版本完全不使用 `cudaDeviceSynchronize()`。它依赖一条隐式规则：父内核要等到所有由设备启动的子内核完成后才算完成，而主机只需等待 stream。

采用设备端 DP 后，不再有用于 CPU 决策的空闲间隙，因此可以缩短依赖阶段之间的延迟，并让 SM 从头到尾保持工作。代价是设备上每次启动会产生少量开销，但 GPU 利用率可能提高，如图 12-13的时间线所示。

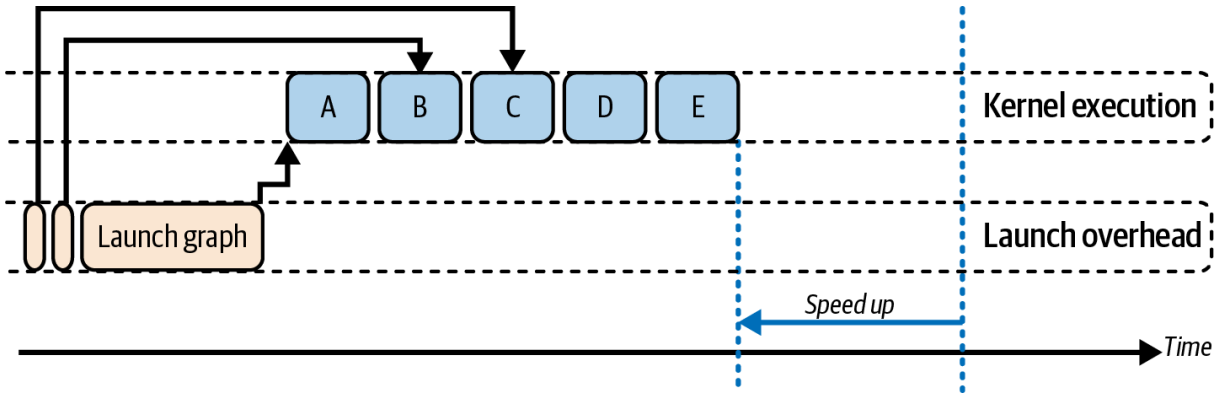


图 12-13. 使用设备端启动和动态并行时没有间隙

在这个包含两个子内核的简单示例中，主机驱动版本发出三次独立启动（1 个父内核 + 2 个子内核）。CPU 决定何时启动每个子内核时，GPU 会处于空闲状态。

相比之下，设备驱动 DP 版本只在主机端启动一次父内核，随后父内核在 GPU 上生成两个子内核，无需主机进一步干预。表 12-2比较了主机驱动和设备驱动 DP 子内核启动的性能。

表 12-2.
主机驱动与 GPU 驱动的嵌套子内核启动性能比较（两个子内核）

指标	之前（主机启动）	之后（设备端启动）
主机启动总数	3	1
每次调用的平均启动开销	~20 µs	~25 µs
GPU 空闲周期（序列期间）	~40%	~5%
总体执行时间	1.00 ms	0.75 ms

在这个说明性示例中，将子内核分发移入 GPU 后，DP 消除了约 35% 的空闲时间，并将总运行时间缩短约 25%。每次启动成本略微上升（20 µs → 25 µs），反映了 GPU 设备端调度开销；相对于消除多次 CPU-GPU 握手带来的节省，该开销较小。实际结果取决于硬件和工作负载。

GPU 驱动启动的另一项收益是改善数据局部性，因为阶段之间无需将中间结果拷回 CPU。在本例中，父内核计算的数据无需离开 GPU 内存，就能立即供子内核使用。这避免了额外内存传输，并有助于保留缓存数据。没有 CPU 干预，也意味着本可在 GPU 上复用的数据较少遭遇缓存驱逐或从 DRAM 重新获取。

简而言之，DP 将时停时启的主机驱动工作流转变为连续的 GPU 驻留流水线，有助于保持较高 SM 利用率并减少主机—GPU 协调。请记住，与其他高级技术一样，动态并行的实际影响也必须通过测试确认。

虽然 DP 消除了 CPU 交互，但设备发起的内核启动开销与主机启动仍处于大致相同的数量级。因此，并非所有算法都能获益；有些算法使用 DP 后甚至可能更慢，尤其是被启动内

核的工作量太小，无法摊薄启动开销时。换言之，对于某些小内核，设备端启动开销可能抵消 DP 的收益。

> 每次更改前后都应进行性能分析。Nsight Compute 等工具可分析用 DP 启动的子内核，帮助量化其成本，并确认 GPU 驻留流水线的收益确实超过额外开销且改善吞吐量。

介绍完单 GPU 编排后，接下来转向多 GPU 与多节点场景；在这里，互连带宽和集合操作将屋顶线考量扩展到集群层面。

跨多个 GPU 和集群节点进行编排（NVSHMEM）

从单个 GPU 扩展到多个 GPU 时，核心目标仍然相同：将数据移动隐藏在有用工作之后，让每台设备保持忙碌。主机通过独立 CPU 线程、异步启动或多 GPU graph 将任务分发到各 GPU 后，GPU 接管执行。每台设备的一个 stream 驱动计算时，第二个 stream 可以通过 NVLink 或 PCIe 点对点传输数据，全程无需经过主机内存。

这意味着在大规模环境中，必须将点对点传输与计算重叠。需要注意，即使使用 NVLink，其带宽和延迟也不同于设备内 HBM，因此应通过重叠隐藏通信。

实践中，随着集群规模增长，将工作与数据传输重叠对于接近线性扩展非常重要。对于直接的数据交接，可以使用 GPUDirect Peer Access 在后台移动大块内存：

```
cudaMemcpyPeerAsync(dest_ptr, dest_gpu, src_ptr, src_gpu, size, comm_stream);
```

需要集合通信时，例如 PyTorch Distributed Data Parallel（DDP）中的梯度 all-reduce，可以使用 NCCL 异步集合调用，在单独 stream 上启动 NCCL 的非阻塞例程。NCCL 随后将张量组织成 ring 或 tree，以利用 NVLink 和 NVSwitch 路径；与此同时，计算内核继续在自己的 stream 上运行。

如果 MPI 库支持 CUDA-aware 并能识别 GPU 设备指针，它会通过类似下面的调用，自动使用 GPUDirect RDMA 在 InfiniBand 上传输数据：

```
MPI_Send(device_buf, count, MPI_FLOAT, peer_rank, ...);
```

MPI（以及 NCCL）的 CUDA-aware 能力意味着 GPU 数据可以通过 InfiniBand 上的 GPUDirect RDMA 直接跨网络移动，无需在主机内存中暂存。（节点内部的点对点拷贝则使用 GPUDirect Peer-to-Peer，经 NVLink 或 PCIe 传输。）

使用这些原语并绕过 CPU，有助于降低数据传输延迟，使 GPU 到 GPU 传输接近链路速率。这样，节点间数据传输与通信就能与 GPU 计算适当地重叠。

使用 NVSHMEM 进行细粒度 GPU 间内存共享

对于需要高度紧密、事件驱动协调的工作负载（例如动态任务队列和细粒度事件通知），NVIDIA SHMEM（NVSHMEM）是很好的选择。它把每个 GPU 视为分区全局地址空间（PGAS）中的一个处理单元（PE）。

使用 PGAS 后，GPU 可以从设备代码直接写入另一个 GPU 的内存，绕过 CPU。延迟取决于互连；NVLink 通常低于 PCIe 或网络传输。下面是 NVSHMEM 的经典 send-and-signal 模式：

```
#include <cstdio>
#include <cuda_runtime.h>
#include <nvshmem.h>
#include <nvshmemx.h>

// Device symbols for the symmetric buffers
__device__ int   *remote_flag;
__device__ float *remote_data;

//---------------------------------------------------------------------------
// GPU 0: send data then signal GPU 1
//---------------------------------------------------------------------------
__global__ void sender_kernel(float *local_data, int dest_pe) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float value = local_data[idx];

    // 1) Put the payload into remote_data[1] on dest_pe
    nvshmem_float_p(remote_data + 1, value, dest_pe);
    // 2) Wait for the RMA to complete before setting the flag
    nvshmem_quiet();
    // 3) Signal completion by setting remote_flag[0] = 1 on dest_pe
    nvshmem_int_p(remote_flag + 0, 1, dest_pe);
}

//---------------------------------------------------------------------------
// GPU 1: wait for flag then consume payload
//---------------------------------------------------------------------------
__global__ void receiver_kernel(float *recv_buffer) {
    // 1) Spin until remote_flag[0] == 1
    nvshmem_int_wait_until(remote_flag + 0,
        NVSHMEM_CMP_EQ, 1);
    // 2) Once flag is set, the payload at remote_data[1] is valid
    float val = remote_data[1];
    recv_buffer[0] = val * 2.0f;
}

//---------------------------------------------------------------------------
// Host-side setup and teardown
//---------------------------------------------------------------------------
int main(int argc, char **argv) {
    // 1) Initialize the NVSHMEM runtime
    nvshmem_init();

    // 2) Determine this PE's rank and bind to the matching GPU
    int mype = nvshmem_my_pe();
    cudaSetDevice(mype);

    // 3) Allocate symmetric buffers on each PE
    //    - Two ints for the flag
    //    - Two floats for the data payload
    int   *flag_buf = (int*)   nvshmem_malloc(2 * sizeof(int));
```

```
    float *data_buf = (float*) nvshmem_malloc(2 * sizeof(float));

    // 4) Zero out flags on PE 0 and synchronize
    nvshmem_barrier_all();
    if (mype == 0) {
        int zeros[2] = {0, 0};
        cudaMemcpy(flag_buf, zeros, 2 * sizeof(int),
                   cudaMemcpyHostToDevice);
    }
    nvshmem_barrier_all();

    // 5) Register the device pointers for use in kernels
    cudaMemcpyToSymbol(remote_flag, &flag_buf, sizeof(int*));
    cudaMemcpyToSymbol(remote_data, &data_buf, sizeof(float*));

    // 6) Launch either the sender or receiver kernel
    dim3 grid(1), block(128);
    if (mype == 0) {
        // Example input buffer for the sender
        float *local_data;
        cudaMalloc(&local_data, 128 * sizeof(float));
        // ... initialize local_data as needed ...
        sender_kernel<<<grid, block>>>(local_data, 1);
        cudaFree(local_data);
    } else {
        float *recv_buffer;
        cudaMalloc(&recv_buffer, sizeof(float));
        receiver_kernel<<<grid, block>>>(recv_buffer);
        cudaFree(recv_buffer);
    }

    // 7) Wait for all GPU work to finish
    cudaDeviceSynchronize();

    // 8) Clean up NVSHMEM resources
    nvshmem_free(flag_buf);
    nvshmem_free(data_buf);
    nvshmem_finalize();

    return 0;
}
```

这里，单边远程内存操作完全发生在设备上。GPU/PE 0 将结果直接写入 GPU/PE 1 的内存，并在该处翻转标志。具体而言，GPU/PE 0 发出 `nvshmem_float_p`，把 payload 数据直接写入 GPU/PE 1 的内存；调用 `nvshmem_quiet()` 确保完成，然后使用 `nvshmem_int_p` 翻转标志。

与此同时，GPU/PE 1 的内核在 `nvshmem_int_wait_until()` 上轮询；标志一设置就读取 payload。此过程不需要 CPU 干预或额外拷贝，只需通过 NVLink 进行硬件加速的 GPU 到 GPU 传输。

NVSHMEM 通信通过 NVLink 或 PCIe 使用 GPU 发起的单边操作，从而消除主机暂存。因此，NVSHMEM 通信可能接近链路峰值速率。原因是 NVSHMEM 单边操作绕过了 CPU，也避免了额外内核启动的软件开销；本质上，它把原先的多步通信缩减为一次硬件事务。

当然，能力越强，责任也越大。NVSHMEM 本质上是 GPU 级共享内存编程，因此必须谨慎管理同步并避免竞态。此外，过度使用全局屏障会让所有 GPU 等待最慢的 peer。

实践中应避免过度同步；尽可能使用 NVSHMEM 的细粒度 signal 或点对点同步，而不是总调用 `nvshmem_barrier_all()`。

现代 NVSHMEM 实现提高了这些同步例程的效率，但它们仍是同步点，误用时可能成为瓶颈。当只有部分设备需要协调时，NVSHMEM 提供细粒度原语，例如等待设备变量的 `nvshmem_wait_until`，以及用于点对点同步的 signal 操作：`nvshmem_signal_fetch`、`nvshmem_signal_wait_until` 或 `nvshmemx_signal_op` 变体。发送 GPU 与接收 GPU 之间使用 NVSHMEM 共享数据并通过 signal 同步的底层细节见图 12-14。

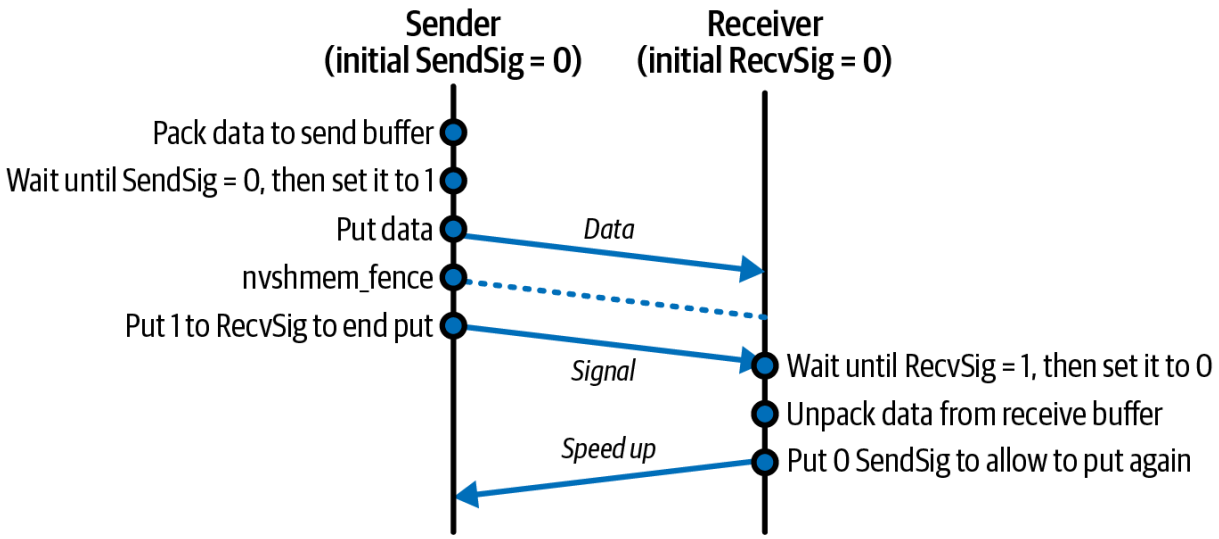


图 12-14. NVSHMEM 单边通信示例

当工作负载不规则或依赖数据时，例如图算法、动态负载均衡和离散事件模拟，NVSHMEM 尤具优势。在这些情况下，静态 graph 和集合操作并不足够。

使用 NVSHMEM 的内核可以像其他内核一样被捕获进 CUDA Graph。NVSHMEM 设备操作发生在这些内核内部，不是独立 graph 节点。不过，NVSHMEM 的真正优势在于让内核即时适应并协调，而无需 graph 使用固定通信脚本。

简而言之，NVSHMEM 将 GPU 集群转变为共享内存域，使内核启动、数据传输和同步能够完全在设备端完成；相较 CPU 介导的方式，它通常能获得更低延迟和更高吞吐。

设想一个两阶段 Transformer 推理流水线（attention + 多层感知机）：GPU 0 计算 attention，然后通过 NVSHMEM 将激活值 put 到 GPU 1 并发送 signal；GPU 1 上运行的持久内核看到标志后，开始 MLP 阶段。

GPU 1 仍处理 batch 1 时，GPU 0 可以立即继续 batch 2。经过几次迭代后，两台设备形成流水化协作，每次交接都隐藏在活跃计算 warp 之后。这有望获得较高利用率且不引入主机停顿，但实际效果取决于负载平衡、互连和同步开销。

需要灵活负载均衡时，NVSHMEM 的原子操作允许每个 PE 动态获取工作。PE 是参与并行 NVSHMEM 应用程序的一个操作系统进程。

以下代码展示每个 GPU 从全局计数器获取下一个索引、处理相应 chunk，然后循环。这实现了完全位于设备端的真正 work stealing，无需主机协调：

```
__global__ void work_steal_kernel(/*...*/, int *queue_head, Task *tasks) {
    while (true) {
```

```
        // Atomically claim the next task index
        int idx = nvshmem_int_atomic_inc(queue_head);
        if (idx >= N_tasks) break;
        // Process tasks[idx]...
    }
}
```

对于要求尽可能低抖动的场景，例如紧密的多 GPU 集合操作或同步模型并行步骤，可以使用 NVSHMEM 启动跨所有 GPU 的协作式内核：通过 `nvshmemx_collective_launch()` 同时在 GPU 0 和 GPU 1 上启动发送端与接收端内核。这样，它们就能借助 NVSHMEM 协调，无需主机干预。随后可以使用 NVSHMEM 的设备端屏障：

```
__global__ void synchronized_step_kernel(/*...*/) {
    nvshmem_barrier_all();
    // All GPUs proceed in lockstep here
    // ...
}
```

这里，每个 PE 一同进入 `nvshmem_barrier_all()`，然后同时继续执行。这保证集群范围内对齐的执行。

> 所有使用 NVSHMEM 设备级同步或集合操作的内核，都必须通过 `nvshmemx_collective_launch()` 启动，以确保内核在作业的所有 PE（GPU）上并发运行。

使用 NCCL 与 CUDA Graph 捕获多 GPU 集合操作

需要 broadcast、reduce 和 all-to-all 等批量集合操作时，NVIDIA NCCL 是多 GPU 系统的常用库。传统方式由每个 GPU 从主机启动 `ncclAllReduce` 或类似集合操作，然后等待（同步），再进入下一计算阶段。这种顺序式主机编排会在前向与反向传播之间增加开销和空闲时间。

不过，NCCL 调用也可以像内核一样记录到 CUDA Graph 中。这样就能把前向内核、all-reduce 和反向内核“固化”进单个 graph，并在每次迭代中重放：

```
cudaStreamBeginCapture(captureStream,
    cudaStreamCaptureModeGlobal);

forwardKernel<<<...>>>(...);

ncclAllReduce(sendBuf, recvBuf, count, ncclFloat,
    ncclSum, comm, captureStream);

backwardKernel<<<...>>>(...);

cudaStreamEndCapture(captureStream, &graph);

// Instantiate and upload before launching
cudaGraphExec_t graphExec;
cudaGraphInstantiate(&graphExec, graph, ...);
cudaGraphUpload(graphExec, captureStream);
```

```
// Each training step:
cudaGraphLaunch(graphExec, captureStream);
```

注意，所有操作（包括 NCCL）都使用同一个捕获 stream。使用 graph 后，NCCL 调用会像内核一样成为 graph 节点。由于每个进程都重放相同 graph，NCCL 内部逻辑可以找到 peer 并执行 all-reduce，无需额外主机协调。在大型集群上，捕获进 graph 的 all-reduce 可以消除每次迭代的启动抖动，并有助于让所有 GPU 重叠计算与网络操作。

由于每个 GPU 都启动包含集合节点的相同 graph，NCCL 可以在内部跨 rank 会合，无需额外主机干预。主机每次迭代的工作缩减为一次 `cudaGraphLaunch`，从而降低 CPU 开销和启动抖动。

除了降低 CPU 负载，将 all-reduce 捕获进 graph 还能在多个 GPU 和计算节点之间重叠通信与计算。假设把梯度计算分为两遍：第 1 层到 $L/2$，以及第（$L/2 + 1$）层到第 L 层，并将它们映射到不同 stream：

```
// Pseudocode in capture:
computeGradientsLayer1<<<...>>>(..., streamA);
ncclAllReduce(..., comm, streamB);       // in streamB, overlaps with streamA
computeGradientsLayer2<<<...>>>(..., streamA);
ncclAllReduce(..., comm, streamB);
```

由于这些节点连同各自的 stream 分配和依赖一起被捕获，CUDA 驱动可以将 `streamB` 上的 NCCL 网络传输与 `streamA` 上的独立工作重叠。这种分桶 all-reduce 模式能够把通信延迟隐藏在计算之后，从而改善多 GPU 扩展。

> 当所有 rank 使用同一 communicator 捕获并重放相同序列时，NCCL 集合操作与 graph 捕获兼容。所有 rank 必须用同一 communicator 捕获和重放相同 NCCL 序列；graph 重放之间 communicator 不匹配会带来死锁或错误结果的风险，最坏情况下可能静默失败且难以调试。建议预先运行集合操作进行 warm-up、初始化 communicator，并复用实例化 graph，以降低稳态延迟。

实践中，分桶 all-reduce 是大模型训练中的常见方法。将梯度归约 chunk 与后续层计算重叠，可以隐藏大部分网络时间。DDP 在不同进程（进程 1 和进程 2）中运行分桶 all-reduce 的示例见图 12-15。

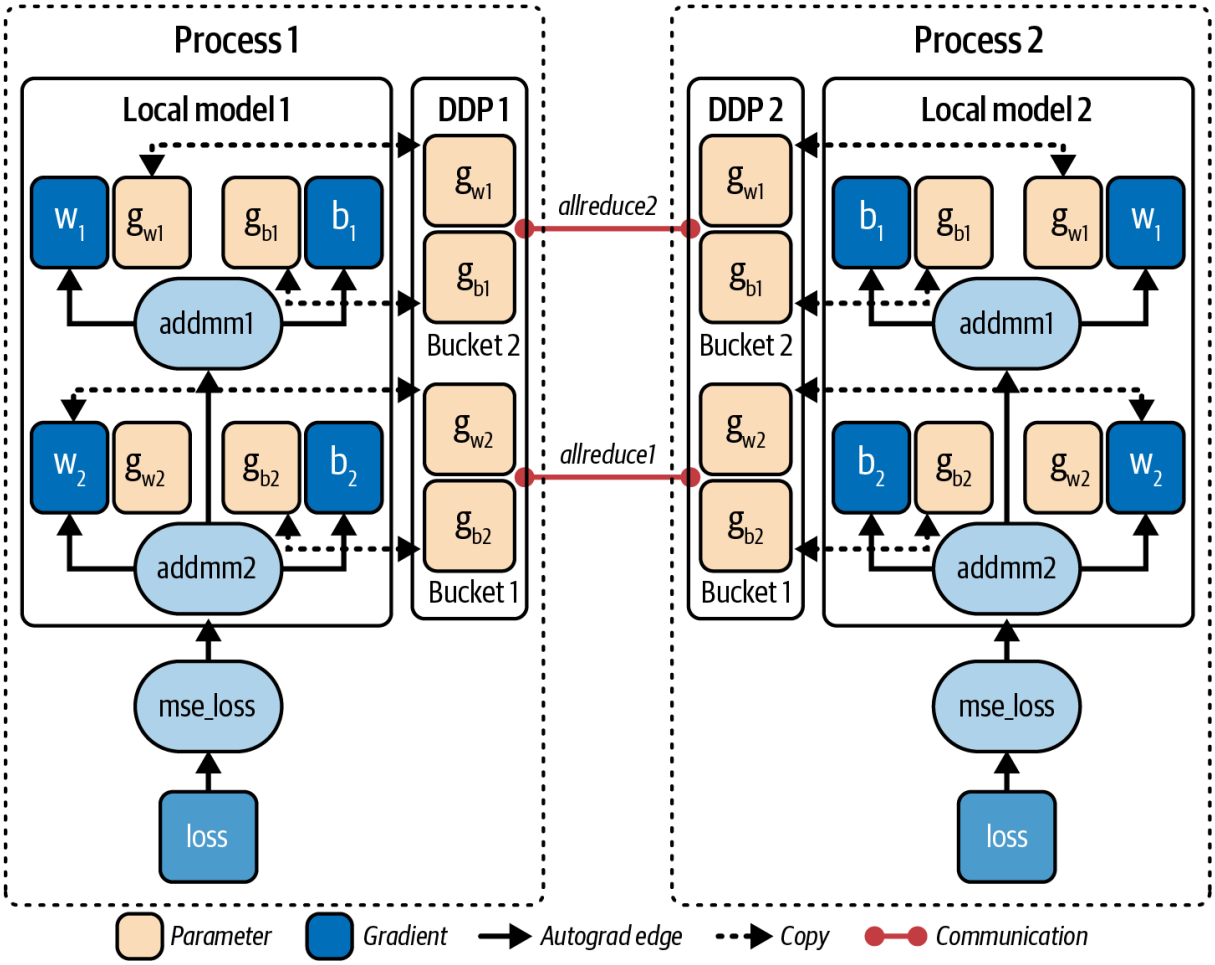


图 12-15. all-reduce 与计算重叠

PyTorch DDP 等现代库会自动实现这种方法的变体，而捕获 CUDA Graph 还能进一步降低 CPU 开销，并提供更确定的性能表现。

在多 GPU 环境中使用 CUDA Graph 时需要注意：所有参与 GPU 必须按完全相同的顺序记录并重放集合操作，以避免死锁；这类似 MPI 的集合规则，所有 rank 都必须按相同顺序进入集合操作。

此外，CUDA Graph 会固定并复用 GPU 缓冲区，因此应在捕获前完成梯度和通信缓冲区的分配。如前所述，如果需要修改 batch 大小等 graph 参数，可以使用 `cudaGraphExecUpdate` 修补参数，无需完整重新捕获。

实践中，将 NCCL 和计算一起捕获进 graph，可以减少每一步的 CPU 时间，并加速多 GPU 大模型训练。在超大规模环境中，这些节省会不断累积，有助于改善整个集群的同步和利用率；实际收益取决于拓扑、通信计算比和软件配置。

> NCCL 和 CUDA Graphs 为我们提供了让集合通信与计算并行调度的高效方式。但并非所有多 GPU 通信都是集合通信；有时需要在 GPU 之间更细粒度或异步地共享数据。这正是前文所述 NVSHMEM 能发挥作用的场景。

N-GPU 扩展模式

无论使用简单点对点拷贝、NCCL ring，还是 NVSHMEM 单边原子操作，扩展到许多 GPU 的模式都相同：系统应尽量一次分发内核，将数据传输流水化并与计算重叠，同时把主机 CPU 移出关键路径，以争取接近线性扩展。

理想情况下，4 个 GPU 应表现得像速度提高 4 倍的单个 GPU，前提是能并行向所有 GPU 持续供给数据（见图 12-16），并让主机退出循环。充分重叠通信和计算后，可能获得接近线性的加速；如果没有重叠，当通信时间达到计算时间时，扩展效果就会趋于平台期。

随着 GPU 数量增加，需要更积极地将数据传输和计算流水化，并减少 CPU 端编排和同步。因此，应使用异步拷贝、graph 中的 NCCL 集合操作和 NVSHMEM PGAS 原语，将更多编排卸载到设备端；这也会把更多责任转移给软件。

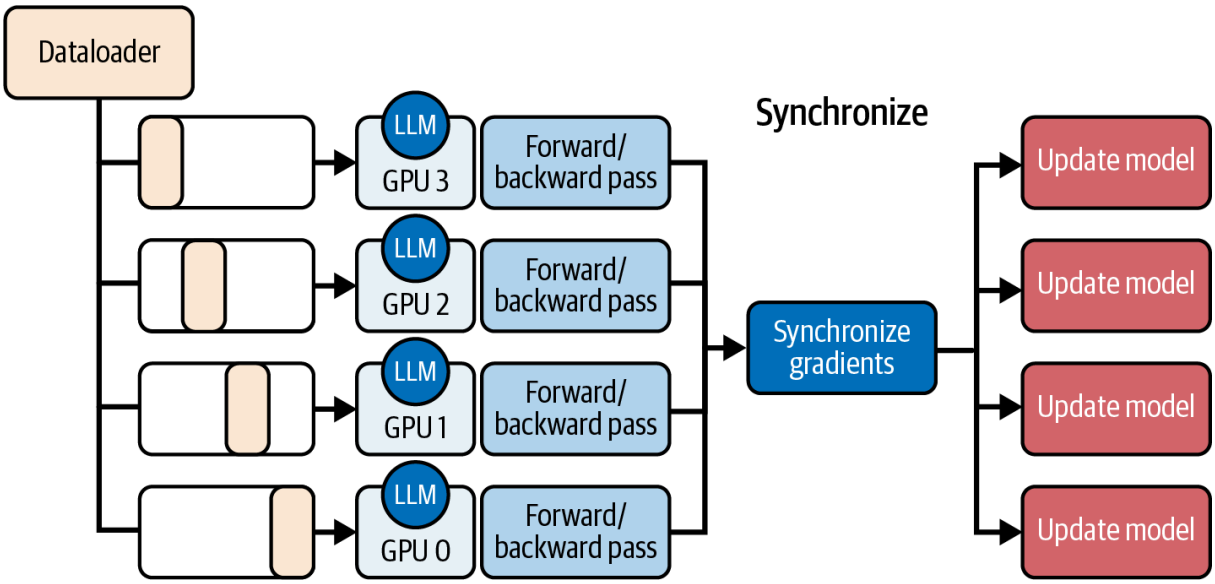


图 12-16. 每块 GPU 并行计算并并发交换数据——没有空闲 GPU，也没有停滞的数据传输

应用这些技术，可消除 CPU 瓶颈、饱和高速互连、充分利用计算 FLOPS，并构建真正低延迟的多 GPU 流水线。接下来，在动态和设备端调度与编排的语境下重新审视屋顶线模型。

由 Roofline 指导调度与编排决策

过去几章已经积累了一组可靠的编排技术，包括 CUDA stream、内核融合、持久内核、CUDA Graph 和动态并行等。roofline 模型有助于判断哪种工具最可能在当前场景中带来最大收益。

roofline 的核心是运算强度，即执行的 FLOPS 与移动字节数之比。它包含两个硬件“上限”：倾斜的内存 roof 表示受带宽限制时的峰值吞吐；平坦的计算 roof 表示受 ALU 限制时的峰值运算速率，如图 12-17所示。

如果内核接近内存 roof（例如 FLOPS/byte 较低），因而受内存限制，最佳优化通常是隐藏内存传输或将其与计算重叠。这意味着应结合 CUDA stream 使用异步拷贝，甚至并发运行多个内存受限内核，以更充分利用内存系统的不同部分。

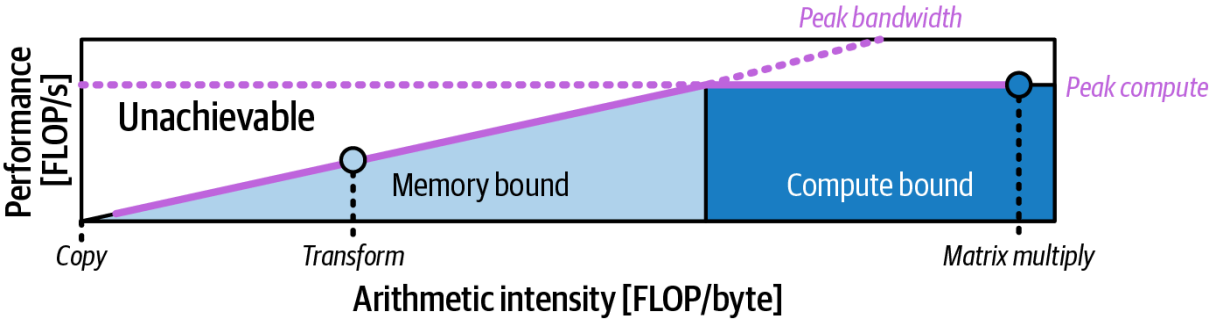


图 12-17. 具有两个硬件上限的算术强度：内存受限（如数据变换操作）和计算受限（如矩阵乘法）

对于内存受限工作负载，内核融合的帮助通常有限，它可以减少若干中间全局内存往返；更大的收益往往来自隐藏延迟并增加在途加载/存储。

相比之下，位于计算屋檐下的高强度内核需要让 ALU 始终保持繁忙。此时内核融合大有用武之地：把分开的加法和缩放操作（前文的融合示例）合并为一次遍历，提高每字节 FLOPS，使 roofline 图上的点向右移动，并让内核性能更接近峰值 FLOP/s 的更高比例。

同样，持久化内核、线程块集群和设备发起的 CUDA Graph 不会改变运算强度数值，但能减少重复启动造成的空闲间隙，从而让内核性能更接近平坦的计算上限。

许多实际工作负载处于两者之间，既不是明显的内存受限，也不是计算受限。此时并发是关键。可以通过 stream、并发 graph 或多个持久化内核并行启动若干中等强度内核，将它们组合起来，使聚合吞吐量的点在两个坐标轴上都更高，从而更充分地利用设备资源。

完整的 roofline 分析需要有纪律的测量。使用 Nsight Compute 统计 FLOPS 和传输字节数，标出内核的数据点，并观察它距离两条 roofline 各有多远。

如果工作负载受内存限制，应考虑使用 stream 和重叠，并在适当时降低精度（FP16、FP8、FP4），以减小算术强度公式的分母（例如传输的字节数）。

如果内核受计算限制，却因启动开销或空闲时段未达到峰值 FLOPS，应重点降低启动开销。正如前文所述，可以将操作融合到一个内核中、使用持久化内核、捕获 CUDA Graph，或执行设备端启动，从而持续为 ALU 提供工作。

如果内核明显低于两条 roofline，既不完全受内存限制也不完全受计算限制，可以尝试提高并发度，让多个内核和 stream 并行运行，以更充分地利用系统资源。

借助这种定量指导，你可以为内核选择合适的编排策略，而不必一次尝试所有技巧。仍要验证每项优化是否让性能更接近硬件的实际潜力；应用优化后务必重新测量。

roofline 模型可以指导预期，但只有真实的性能测量（包括计算吞吐量、实际 occupancy、内存吞吐量等）才能说明完整情况。将 roofline 分析与迭代式、持续性的分析器采样结合起来，才能确认所选优化策略确实有效。

关键要点

要实现 GPU 的高性能，关键是以尽可能低的开销协调计算和数据移动。高效的编排能简化跨 CPU 与 GPU 的复杂工作负载，避免任何一方等待另一方。本章要点如下：

使用 L2 缓存原子队列进行动态调度

现代 GPU 上的 L2 缓存原子操作速度极快。结合批量递增，用快速的 L2 缓存原子操作在 GPU 上平衡不规则工作负载。批量分配工作可以减少竞争，消除 warp 空闲间隙并保持 warp 忙碌。在极端不均衡情况下，吞吐量最高可能提升约 2 倍，但通常提升约 10%–30%；由于 L2 带宽很高，即使批量大小只有 8 或 16，也能消除大部分竞争。

使用 CUDA Graph 处理固定流水线

将一系列 GPU 操作记录一次，然后每次迭代只需一次主机调用即可重放。这可以降低每次迭代的 CPU 调度开销，延迟常可降低 20%–30%（更大规模下可能更多）。同时要确认 GPU 上存在依赖关系的操作实现了尽可能大的重叠。

使用 CUDA Graph 实现低开销启动

将异步拷贝、内核启动、事件记录和内存分配组成的序列捕获到 CUDA Graph 中（`cudaStreamBeginCapture/cudaStreamEndCapture`）。使用 `cudaGraphLaunch` 重放 graph，可以消除每次调用的 CPU 入队开销，同时保留所有 stream 间依赖，进一步减少运行时瓶颈。

设备端编排

通过尾部启动预先记录的 CUDA Graph，或使用动态并行生成子内核，从 GPU 自身启动工作。这可以消除 CPU 调度间隙，并让 GPU 在无需主机干预的情况下从头到尾保持忙碌。

多 GPU 重叠

始终尝试将通信与计算重叠。使用独立 stream，对 GPU 点对点传输（`cudaMemcpyPeerAsync`）、NCCL 集合通信、支持 CUDA 的 MPI（RDMA）或 NVSHMEM 单边操作进行流水线化。在计算与通信比合适且重叠充分时，这可以用有用工作隐藏通信延迟，使多 GPU 甚至跨集群节点的扩展接近线性；实际效果仍取决于拓扑、同步和负载平衡。

基于 roofline 的选择

让 roofline 图指导策略。如果内核受内存限制，应通过异步 memcpy 和 FP8/FP4 等混合精度来重叠操作并减少数据移动。如果内核受计算限制但因开销而未达到理想水平，应使用内核融合、持久化内核和 CUDA Graph 等降低启动开销的技术，逐步接近计算上限。对于处于中间区域的内核，应并行运行多个操作以提高并发度、利用所有硬件单元。务必通过分析器验证所选优化确实带来改善。

将这些技术结合起来——动态分派、协作式内核、graph 捕获/重放以及 GPU 原生内存共享——就能构建充分利用 GPU 集群各个部分的超大规模 AI 工作负载流水线。

结语

本章超越了单个内核的优化，探讨了端到端的编排技术。我们介绍了如何使用动态并行完全在设备上启动工作、在 CUDA Graph 中捕获复杂工作流，以及使用 NCCL 和 NVSHMEM 协调多个 GPU。每项技术的目标都相同：持续为每个执行引擎提供工作、隐藏延迟并缩小主机与设备之间的间隙，让硬件尽可能持续运行。

NVIDIA 的现代 GPU 平台比以往任何时候都更模糊 CPU 与 GPU 的界线。例如，Grace Blackwell 和 Vera Rubin Superchip 通过具备一致性的高带宽 NVLink，将 CPU 与多个 GPU 连接起来。

即使硬件正在减少 CPU 与 GPU 之间的障碍，仍需要软件充分发挥这些高性能硬件的能力。本章介绍的方法——无论是 C++ 中的 CUDA，还是更高层的库 API——都是利用这些进步的途径。

下一章将介绍 PyTorch 如何整合其中许多思想，包括 stream、graph、异步操作和优化内核，使你只用几行 Python 就能获得这种性能。接下来我们将深入 PyTorch 生态，理解它为何如此适合实现高性能 AI 工作负载。

第 13 章　PyTorch 的性能分析、调优与扩展

AI 训练和推理流水线的每一层都可能存在性能瓶颈，包括 Python 解释器开销、CPU 主机端数据加载停顿、CUDA 内核利用不足，以及 GPU 设备内存竞争。要有效优化，需要在整个软件栈的多个层级进行分析，并使用能够覆盖完整系统的多种工具。

本章聚焦于现代 NVIDIA GPU 上运行的 PyTorch 工作负载的性能分析、调试和系统级调优。我们将使用 PyTorch 内置分析器、NVIDIA Nsight 工具以及 Linux 的 CPU 分析工具 `perf` 来识别和修复瓶颈，同时讨论 PyTorch 内存分析和内存分配器调优；还会介绍 PyTorch 如何使用 CUDA stream 实现并发，以及使用 CUDA Graph 降低内核启动开销。

接下来，我们将展示如何优化数据流水线，并使用 PyTorch Distributed Data Parallel（DDP）、Fully Sharded Data Parallel（FSDP）及其他模型并行策略扩展到多个 GPU。随后还会演示如何分析多 GPU 和多节点环境，包括 Holistic Trace Analysis（HTA）和 Perfetto。

全章都将强调性能权衡和定量示例，关注内核执行时间、硬件利用率指标、内存占用、数据加载效率以及扩展的总体成本效益。读完本章后，你应能理解如何在整个软件栈上建立有效而全面的 PyTorch 工作负载分析与调优方法。

NVTX 标记与性能分析工具

要全面了解性能，必须在多个层级进行分析，并使用覆盖整个系统的工具。业界实践者和性能工程师已经形成了一套常用工具与最佳实践，用于覆盖系统软件栈的所有层级开展整体性能分析。

在介绍工具前，需要先强调 NVIDIA Tools Extension（NVTX）和 NVTX 标记。这些标记表示性能分析器时间线视图中的时间范围，并允许不同分析器关联同一阶段的事件。

例如，`"forward"` 的 NVTX 区间会同时出现在 PyTorch profiler 追踪和 Nsight Systems 时间线中。这使软件栈不同层级的跨工具分析容易得多。包括 PyTorch 及 CUDA 生态相关库在内的大多数现代 AI 框架和库都支持 NVTX 标记。

可以通过 CUDA C++、PyTorch，或任何支持 NVIDIA GPU 的 C++/Python 库（如 OpenAI Triton、PyCUDA、CuPy、cuTile、cuTe、CUTLASS 等）将 NVTX 标记注入代码。大多数库已为 `"train_step"`、`"forward"`、`"backward"`、`"optimizer_step"` 等关键代码区域自动注入标记。也可以自行在 PyTorch 中使用 `torch.profiler.record_function()` 和 `torch.cuda.nvtx.range_push()` 注入它们。

既然已介绍如何使用 NVTX 标记注释代码中值得关注的区域，下面讨论可摄取、对齐并可视化这些标记的工具。常见性能分析工具及其适用范围、关键功能和典型用途汇总于表 13-1。该表可帮助你在优化历程的各个阶段选择合适工具。

表 13-1　性能分析与可视化工具汇总

工具	范围	功能	典型用途
PyTorch 分析器（Kineto）	PyTorch 内部算子级分析（CPU/GPU）	支持 NVTX 标记、记录张量形状、内存统计、导出轨迹，以及识别编译 graph 断点	细粒度拆解模型代码；识别慢算子、GPU 内核启动开销，或前向与反向时间的不均衡。
Nsight Systems (`nsys`)	全系统时间线（CPU、GPU、操作系统、I/O）	统一显示 CPU 线程和 GPU stream 的时间线，集成 NVTX，支持多进程	端到端查看训练/推理流水线；检测数据加载器停顿、CPU-GPU 重叠问题或 GPU 间同步延迟。
Nsight Compute (`ncu`)	GPU 内核分析（逐个内核）	逐内核硬件指标、源代码关联、roofline 分析、occupancy 和吞吐量报告	找到热点内核后深入分析其效率；判断内核为何受内存或计算限制。
PyTorch 内存分析器	按操作统计 GPU 内存使用	内存快照时间线、逐操作峰值内存，以及集成 `torch.cuda.memory_stats()` 和 `torch.cuda.mem_get_info()`	诊断内存碎片或异常高的内存使用；找出哪些操作在何时分配了最多内存，以优化内存占用。
Linux `perf`	CPU 分析与系统事件	采样 CPU 周期/指令/缓存，火焰图，CPU 外（休眠）分析	识别 Python 开销（解释器时间、GIL 竞争）、CPU 端数据加载瓶颈或主机操作系统调度问题。
Holistic Trace Analysis（整体轨迹分析）	分布式训练轨迹可视化	基于浏览器的 Kineto 轨迹浏览器、多工作进程轨迹聚合；集成 Perfetto 后端	整体分析多 GPU/多节点执行；查找不均衡或空闲时间，并验证通信与计算是否重叠。
Chrome 轨迹/Perfetto	离线轨迹查看（基于 Web）	标准轨迹格式的 Web 界面、高级筛选（Perfetto SQL 引擎）、便于共享轨迹	无需专用软件即可检查轨迹；适合性能仪表板或围绕分析数据进行远程协作。
TorchEval（指标）	模型指标与性能日志	标准化指标 API（吞吐量、准确率等），易于集成	在训练和评估中记录并监控模型吞吐量/延迟

工具	范围	功能	典型用途
			及准确率，以关联性能与模型质量。
ExecuTorch	用于移动端、嵌入式和边缘设备的部署运行时，提供轻量性能分析钩子和导出工具	模型导出、性能分析、调试、内存分析、可视化	用于运行 PyTorch 模型，并在受限平台（如移动端、嵌入式、边缘设备及 Meta 智能眼镜等）收集运行时指标。

下面详细介绍表 13-1中的各项性能分析工具：

PyTorch 分析器（Kineto）

在 PyTorch 中，`torch.profiler` 基于 Kineto 开源项目，提供 CPU 和 CUDA/GPU 运行时的算子级拆解。此外，它可以记录输入形状，并通过简单的 Python 上下文管理器获取内存快照。PyTorch 分析器利用 NVTX range 对齐事件，可在训练和推理工作负载中捕获详细的时间线轨迹和硬件计数器。它提供从 Python 代码一直到 CUDA 内核的端到端可观测性，还能针对数据加载停顿、CUDA 代码效率低等常见问题给出性能提示。

Nsight Systems (`nsys`)

为了进行全系统关联分析（包括 CPU 线程、GPU 内核、操作系统事件、I/O 和互连流量），NVIDIA Nsight Systems 会生成统一的时间线视图。它的 GUI 和 CLI 报告可以在多进程、多节点运行中合并 NVTX 区域、Python 调用栈和 CUDA stream，从而很容易发现 I/O 或同步停顿可能影响计算性能的位置。

Nsight Compute (`ncu`)

在 Nsight Systems 之外，NVIDIA Nsight Compute 用于逐内核分析。它会收集 occupancy、内存带宽和 SM 利用率等详细硬件指标，还能生成与源代码对应的 roofline 图，帮助分析热点内核为何缓慢。

PyTorch 内存分析器

PyTorch 还包含内存分析器，可以在 `torch.profiler` 中通过 `profile_memory=True` 启用。它会按操作拆解 GPU 内存的峰值和累计分配量，揭示可能被忽略的内存使用热点。

Linux `perf`

在主机端，Linux 的 `perf` 工具可以采样 CPU 硬件计数器（包括周期、指令和缓存未命中），并展开完整的 C/C++ 与 Python 调用图。使用 `perf sched` 可以查看 CPU 线程何时因 I/O、线程调度或同步而空闲，从而发现数据预处理循环、Python GIL 或同步导致 GPU 无法获得工作的瓶颈。

Holistic Trace Analysis

Meta 的开源 Holistic Trace Analysis（HTA）工具可以读取 PyTorch 分析器轨迹，帮助诊断多 GPU 瓶颈。借助 HTA，可以将带有 NVTX range 的分布式训练时间线与 CUDA 内核轨迹结合显示，并深入查看随时间变化的内存分配模式，识别 GPU 空闲时段，包括 GPU 彼此等待的情况。

> TensorBoard 的 PyTorch 轨迹可视化插件已弃用。应改用 Perfetto 查看时间线，并使用 Meta 的 HTA 分析分布式轨迹。

Chrome 轨迹与 Perfetto 查看器

要通过 Web 探索大型 PyTorch 分析器轨迹文件，可以使用 Chrome tracing（例如浏览器中的 `chrome://tracing`）和 Perfetto 界面。它们会加载 JSON 轨迹，让你交互式查看时间线和火焰图，还能对轨迹数据执行细粒度过滤和 SQL 查询，事件追踪与关联的粒度可达到亚毫秒级。Chrome 轨迹和 Perfetto 界面适合在组织成员之间共享分析结果，开展跨团队分析。（注意：Chrome 的旧版轨迹查看器已弃用，因此查看和分析轨迹时应优先使用 Perfetto Web 界面和 SQL 引擎。）

TorchEval（PyTorch 指标库）

另一个项目 TorchEval 可以在统一界面中记录并监控模型吞吐量、延迟和质量指标，以及训练和评估指标。TorchEval 是 PyTorch 的官方指标库，提供用于端到端性能和质量指标的简单 API，便于接入训练循环并集成到分布式环境中。

ExecuTorch

对于嵌入式、移动和边缘设备，ExecuTorch 项目可以在 Meta 眼镜等轻量级运行环境中对 PyTorch 模型进行性能分析、可视化和调试。ExecuTorch 的动态内存占用较小，支持 Linux、iOS、Android 和嵌入式系统。Hugging Face 通过 Optimum ExecuTorch 项目支持 ExecuTorch；如果已经使用 Hugging Face 生态（例如 Hugging Face Transformers），就很容易集成这一环境。

接下来，我们将通过一个示例使用这些分析器识别性能瓶颈，然后应用针对性优化并验证性能改善。

分析 PyTorch 以识别瓶颈

下面通过一个混合专家（MoE）Transformer 模型示例，观察这些工具的实际用法。MoE 是包含多个专家层的 LLM，每个专家都是一个前馈网络；专家门控系统负责将 token 路由到不同专家。我们将运行一次训练迭代，捕获详细的性能轨迹，并分析输出结果以指导优化。

使用 PyTorch 分析器

首先设置模型和输入。如下面所示，使用 Hugging Face Transformers 加载模型和 tokenizer，将模型移到 GPU，并准备一小批输入：

```
# train.py

# Set up model and data
model_name = "..."

tokenizer = AutoTokenizer.from_pretrained(model_name)
device = torch.device("cuda")
model = AutoModelForCausalLM.from_pretrained(model_name).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, fused=True)

batch_size = 4
input_texts = ["MoEs are great."] * batch_size
enc = tokenizer(input_texts, return_tensors="pt", padding=True, truncation=True)
input_ids = enc.input_ids.to(device)
attention_mask = enc.attention_mask.to(device)
labels = input_ids.clone()  # For LM training, labels are the inputs
                            # (next-token prediction)
```

为了避免把一次性的初始化开销计入结果，我们在性能分析前运行几次预热迭代。这样可以通过编译 JIT 内核、填充缓存等操作让模型准备就绪，使测量的迭代更能代表稳态性能。下面是准备模型的代码：

```
# Warm-up (not profiled)
for _ in range(5):
    with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
    loss = outputs.loss
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

现在使用 PyTorch 分析器和 NVTX 分析一次训练迭代。我们将迭代包裹在 `torch.profiler.profile()` 中，并用 `record_function` 和 NVTX range 标记高层区域，包括 `"forward"`、`"backward"` 和 `"optimizer_step"`，如下所示：

```
from torch import profiler

with profiler.profile(
    activities=[profiler.ProfilerActivity.CPU,
                profiler.ProfilerActivity.CUDA],
    record_shapes=True,  # record tensor shapes
    profile_memory=True, # track GPU memory usage per op
    with_stack=True,     # enable stack tracing
    with_flops=True      # capture FLOPs counters
) as prof:
    with profiler.record_function("train_step"):
        # Forward pass
        torch.cuda.nvtx.range_push("forward")
        with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
            outputs = model(input_ids, attention_mask=attention_mask,
                            labels=labels)
```

```
        loss = outputs.loss

        # end of forward
        torch.cuda.nvtx.range_pop()

        # Backward pass and optimization
        torch.cuda.nvtx.range_push("backward")
        loss.backward()
        torch.cuda.nvtx.range_push("optimizer_step")
        optimizer.step()

        # end of optimizer_step
        torch.cuda.nvtx.range_pop()
        optimizer.zero_grad()

        # end of backward
        torch.cuda.nvtx.range_pop()
```

在这段代码中，PyTorch 分析器会记录 `train_step` 期间的所有 CPU 和 GPU 活动。我们使用 `record_function("train_step")` 定义顶层区域，并为子阶段（`"forward"`、`"backward"`、`"optimizer_step"`）插入 NVTX 标记。这些标记会出现在分析器时间线中，用来划分迭代的不同阶段。

分析器还可以区分模型中已编译和未编译的区域。本章后面以及下一章将介绍 PyTorch Compiler、graph break 以及减轻 graph break 的机制。

例如，使用 `torch.compile` 时，轨迹会显示 `CompiledFunction` 等事件，并指出发生的 graph break（见图 13-1）。这有助于定位模型退回 eager 执行的位置，为后续优化提供依据。

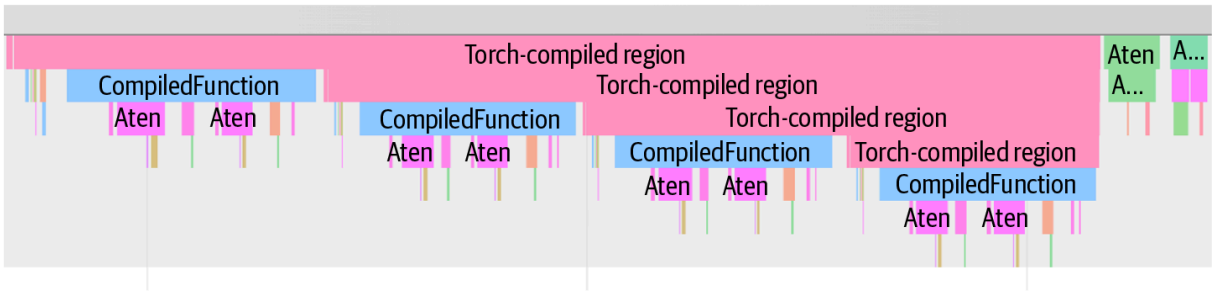


图 13-1　已编译区域（左侧和中间，粉色）与未编译区域（右侧，绿色）（来源：https://oreil.ly/Z_fJG）

执行后，可以调用 `prof.key_averages().table()`，按运行时间打印排名靠前算子的简明表格。在下面的代码块中，我们按 self CUDA time 对前 10 个操作排序；该指标表示操作自身 CUDA 内核所花的时间，不包括该内核生成的子操作。按 CUDA 执行时间排名前 10 的操作汇总于表 13-2：

```
with profiler.profile(
    activities=[ProfilerActivity.CPU,
                ProfilerActivity.CUDA],
    record_shapes=True,
    profile_memory=True,
) as prof:
    train_step(...)

...
```

```
print(
    prof.key_averages()
        .table(
            sort_by="self_cuda_time_total",
            row_limit=10,
            fields=["self_cuda_time_total",
                    "calls"]
        )
)
```

表 13-2　一次训练迭代中按 CUDA 执行时间排名前 10 的操作

操作	Self CUDA 总时间	调用次数
aten::matmul	43.00 ms	128
aten::linear	35.50 ms	64
dispatch	18.70 ms	2
combine	12.10 ms	2
aten::layer_norm	10.20 ms	16
aten::softmax	5.70 ms	4
aten::scatter	4.10 ms	16
aten::gather	3.60 ms	16
aten::to	2.90 ms	8
aten::add_	2.20 ms	64

可以看到，矩阵乘法操作（aten::matmul 及其在 aten::linear 中的使用）占据了大部分 CUDA 时间，消耗了迭代的大多数时间。这些操作对应专家前馈网络（FFN）的 GEMM。具体来说，每次迭代会调用 matmul 128 次；这是合理的，因为共有 64 个专家，每个专家在前向和反向传播中各执行一次 matmul。

从表 13-2还可以看到，接下来最大的开销来自 dispatch 和 combine 操作。这些是自定义 C++/CUDA 内核，用于将 token 重新分发给专家，再收集专家的输出。dispatch 执行两次（前向一次、反向一次），总计 18.7 ms；combine 也执行两次，总计 12.1 ms。两者合计又占用 30.8 ms 的 GPU 时间，其余时间分散在 layer norm、激活等较小操作中。

这个性能分析示例的关键结论是：专家 FFN 的 matmul 是首要瓶颈，其次是 dispatch 和 combine 内核。它们共同主导了一次训练迭代的运行时间。要进一步提升性能，应直接优化这些操作，或减少它们的调用次数。

使用 Nsight Systems 和 NVTX 时间线进行系统级分析

插入的 NVTX 标记使得使用 Nsight Systems 分析时间线变得直接。为了按阶段聚合指标，可以使用带 NVTX 汇总的 `nsys` 分析代码，如下面的 CLI 命令所示：

```
nsys profile \
   --output=profile \
   --stats=true \
   -t cuda,nvtx \
   python train.py
```

这里，`-t cuda,nvtx` 选项会指示 Nsight Systems 同时追踪 CUDA API 调用和 NVTX range。分析完成后，可以在 Nsight Systems GUI 中打开 `profile.nsys-rep` 文件（`--output=profile`），也可以使用 CLI 将 NVTX 汇总打印到终端。随后可对该 `profile.nsys-rep` 文件使用以下命令之一生成 NVTX GPU Projection Summary，以便通过 Nsight Systems 将 range 与预计的 GPU 工作进行核验：

```
nsys stats --report=nvtx_gpu_proj_sum \
  profile.nsys-rep

# or

nsys recipe nvtx_gpu_proj_sum \
  profile.nsys-rep
```

可以在持续构建和集成流水线中使用其中一个命令，监控并检测性能回归。该 CLI 命令的结果汇总于表 13-3。

表 13-3　使用 Nsight Systems 分析一次 `train_step` 迭代得到的 NVTX GPU 投影汇总

NVTX range	GPU 时间 (ms)	Self GPU 时间 (ms)	子 GPU 时间 (ms)	实例（调用次数）
`train_step`	138.0	0.0	138.0	1
`forward`	60.5	60.5	0.0	130
`backward`	58.3	58.3	0.0	130
`optimizer_step`	19.2	19.2	0.0	1

可以看到，`train_step` range 包含 forward、backward 和 optimizer 子 range。NVTX GPU 投影汇总确认 `train_step` 下的 GPU 总时间为 138 ms，与 PyTorch 分析器在表 13-2中给出的 `forward`、`backward` 和 `optimizer_step` 时间之和一致，说明不同工具之间的结果相互吻合。

虽然表 13-3显示只有一次 `optimizer_step` 调用，但它的 NVTX range 实际上包围了全部 64 次 `aten::add_` CUDA 内核启动（如表 13-2所示，每个专家执行一次 `add_`），这些启动

都归入同一个 optimizer_step 标记。

请注意，Nsight Systems 会将 64 次 aten::add_ 调用（例如 64 路专家并行策略）归入单个 optimizer_step 标记，因为它通过 CUDA Profiling Tools Interface（CUPTI）捕获主机端的 NVTX push/pop 事件，然后将异步 GPU 内核执行时间“投影”到这些由 CPU 定义的时间区间上。具体来说，它会累加 GPU 起止时间戳落在相应 push 和 pop 调用之间的每个内核的持续时间，得到一个累计 GPU 时间，恰好等于各个 aten::add_ 内核时间之和。

> 未连接分析器时，NVTX 标记的开销非常低。因此，这种投影机制很理想：几乎不增加开销，却能将 GPU 工作与高层代码区域进行端到端关联。

由于没有在 forward 和 backward 内进一步嵌套更深层的 range，它们各自的 self GPU 时间都等于总时间，因此子 GPU 时间为 0 ms。相比之下，train_step 只是嵌套阶段的外层包装器，因此几乎全部时间都计为子 GPU 时间。

NVTX GPU 投影汇总还显示，每次迭代在 train_step 内观察到 130 个 GPU 活动。这些活动包括内核启动以及内存拷贝等其他设备操作，因此并不严格与内核一一对应。

如表 13-3所示，前向和反向传播也都包含这 130 个 GPU 内核调用。操作与 NVTX 实例之间的一一对应关系表明，当前插桩捕获了所有重要操作。

> 表 13-3中的 NVTX 汇总提供了便捷的文本概览。进行可视化分析时，时间线 GUI 可以显示重叠的内核执行、CPU 线程状态，甚至 CUDA API 开销事件。实践中，应在可视化时间线中确认主机端数据加载和预处理与 GPU 计算发生重叠。任何较大的间隙或“气泡”都表示存在问题；用于同步的小间隙则是预期且合理的。

在多 GPU 运行中，Nsight Systems 或 HTA 的时间线视图可以显示 NVLink 或 InfiniBand/Ethernet 是否得到有效利用，也能发现某个节点是否因等待通信或网络延迟而缺少工作。这通常提示同步策略不理想或负载不均衡。

使用 Nsight Systems 和 HTA 生成轨迹时，务必追踪 GPU 通信事件，包括 NCCL all-reduce 调用以及 NVLink/NVSwitch 活动。这有助于确认在基于 NVL72 的大型 GPU 域等环境中，GPU 能持续获得工作。

细致的性能分析可以确保大型 NVLink 集群使用正确的 GPU 间同步方式并实现负载均衡。下面将聚焦系统中开销最大的内核之一：矩阵乘法，即 matmul。

通用矩阵乘法（GEMM）的内核 Roofline 分析

为了深入分析专家 matmul 内核，使用 Nsight Compute（ncu）命令行分析器按名称定位特定 GEMM 内核。如下所示，我们将收集 roofline 相关指标，以判断其受计算还是内存限制：

```
ncu \
  --target-processes all \
  --kernel-name-regex "matmul" \
  --metrics \
    gpu__time_duration.avg, \
    gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed, \
    lts__throughput.avg.pct_of_peak_sustained_elapsed, \
    sm__sass_thread_inst_executed_op_fp32_pred_on.sum, \
    sm__warps_active.avg.pct_of_peak_sustained_active \
  --csv full \
  -o matmul_roofline_report \
  python train.py
```

这里收集名称匹配“matmul”的所有内核的硬件计数器。具体包括几个关键指标：GPU DRAM 带宽和 L2 带宽占峰值的百分比（`gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed`、`lts__throughput.avg.pct_of_peak_sustained_elapsed`）、作为计算代理的 FP32 指令数（`sm__sass_thread_inst_executed_op_fp32_pred_on.sum`）、内核时间（`gpu__time_duration.avg`）以及实际达到的 occupancy（`sm__warps_active.avg.pct_of_peak_sustained_active`）。

> 不同版本的 Nsight Compute 中，指标标识符可能略有差异。如果找不到某个指标，请使用 UI 定位当前名称并据此替换。这里分析的是 SM 和 DRAM 相对持续峰值的百分比，以及实际达到的 occupancy。

应用前文讨论的优化（例如降低精度、融合内核和提高算术强度）后，可以重新运行 `ncu` 命令，验证优化是否带来正面影响。表 13-4展示了提高算术强度前后的对比。

表 13-4　专家 `matmul` 内核在算术强度优化前后的 Roofline 分析

内核配置	峰值 FLOPS 占比（SM 计算吞吐量）	峰值内存带宽占比（内存吞吐量）	实际达到的 SM occupancy	特征
Baseline	50%	70%	60%	内存受限（内存传输导致停顿）
Optimized	85%	40%	80%	计算受限（接近硬件 roofline）

可以看到，在基线运行中，主要 GEMM 内核只达到峰值计算 FLOPS 的约 50%、峰值内存带宽的 70%，以及中等水平的 60% SM occupancy（每个 SM 的平均活跃 warp 数）。这一 occupancy 不足以完全隐藏内存延迟。

不存在适用于所有内核的统一 occupancy 目标。许多高性能内核在实际 occupancy 为 25%–50% 时就能充分隐藏延迟。应结合 Nsight Compute 的 occupancy 指标、停顿原因拆解以及每个活跃周期的可调度 warp 数，判断提高 occupancy 是否能减少该内核的停顿。如果内核无法调度足够多的 warp 覆盖停顿时段，就会出现空闲周期，因为内存请求未能及时得到处理。

基线指标表明该内核受内存限制，因为执行会因内存传输而停顿。这造成大量计算能力未被利用，进一步说明该工作负载目前不是计算受限。目标是让内核更多地转为计算受限，以利用 GPU 提供的大量 FLOPS。

在优化版本中（例如融合内核、提高算术强度并减少内存移动），峰值 FLOPS 占比提升到 85%，峰值内存带宽占比降至 40%，occupancy 提升到 80%。这实际上将内核从内存受限转为计算受限，使其更接近硬件 roofline 上限，如图 13-2所示。

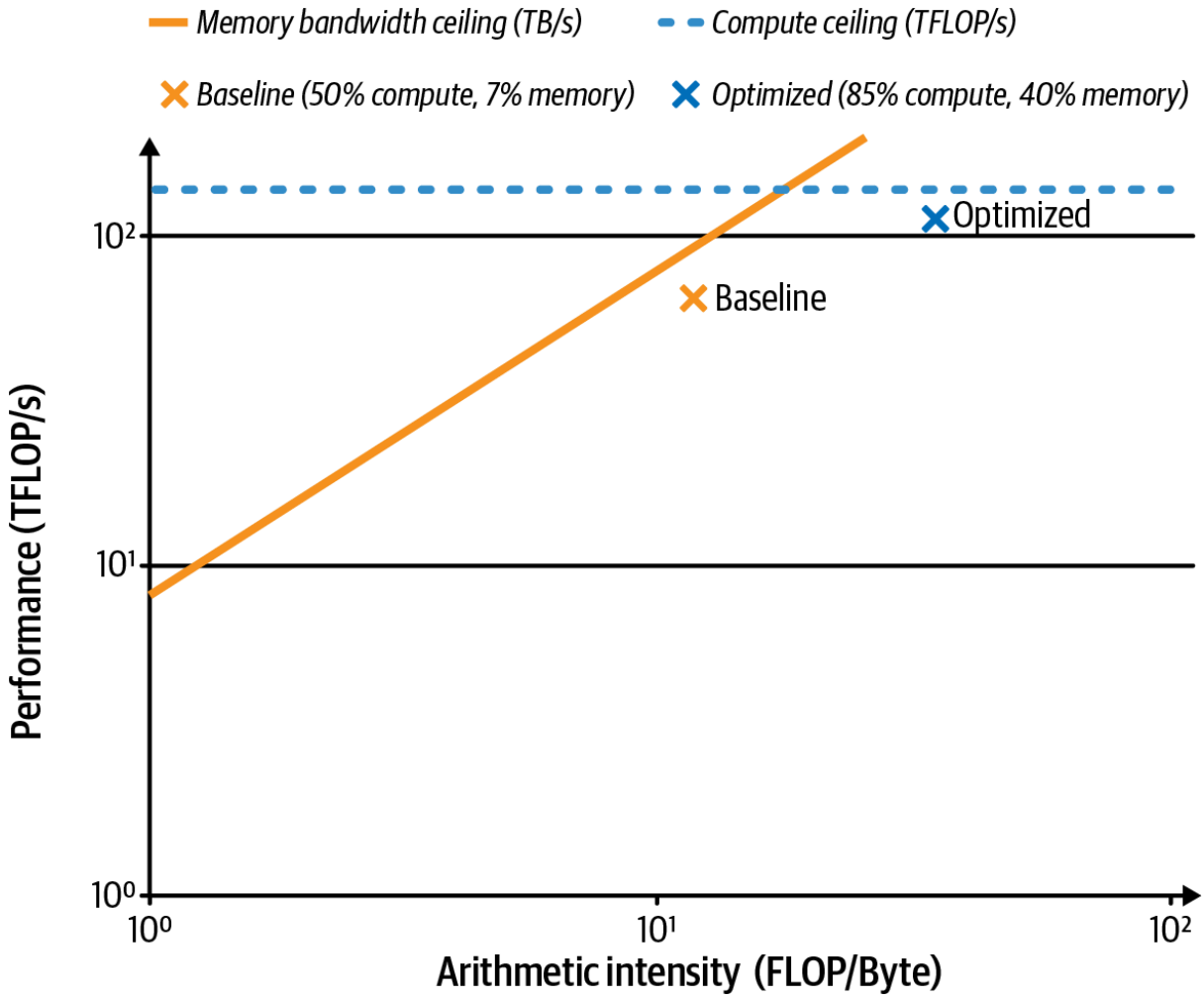


图 13-2　提高该内核算术强度前后的 Roofline 图

到目前为止，性能分析主要聚焦于 GPU 性能。同样重要的是不要让 CPU 或 I/O 浪费时间。下一节将转向主机端继续分析。

使用 Linux perf 分析 CPU 和 GPU

为了全面了解主机和设备分别消耗了多少时间，可以使用 Linux `perf` 分析 CPU 周期、缓存未命中、分支未命中等指标。然后根据这些信息制定一系列优化，逐项应用并测量改善效果。

首先运行轻量级的 `perf stat`，收集 MoE 训练期间的 CPU 端统计信息。示例节点配有基于 ARM 的 Grace CPU 和 Blackwell GPU。下面给出 CLI 命令及示例输出：

```
perf stat -e \
  cycles,instructions,cache-misses,branch-misses \
  python train.py
  Performance counter stats for 'python train.py':

# 0.600 CPUs utilized
1,200.345 msec task-clock
# Approximately 2.0 GHz
2,400,567,890      cycles
# 1.58 insn per cycle
3,800,123,456      instructions
# 0.32% of all cache refs
12,345,678      cache-misses
# 0.12% of all branches
4,567,890      branch-misses

 1.234567890 seconds time elapsed
```

`perf stat` 报告显示 CPU 利用率、周期、每周期指令数（IPC）以及缓存/分支未命中情况。在本次运行中，task clock 约为 1.2 秒，而测量期间只使用了单个 CPU 核心约 60%（0.600）。这是预期现象，因为主要计算工作由 GPU 完成。较低的缓存未命中率和分支未命中率表明，对于该工作负载，CPU 端的内存访问模式和分支预测相对高效。

不过，IPC 仅为 1.58，说明 CPU 的发射率远低于单个 Grace CPU 核心（ARM Neoverse V2）每周期 8 条指令的理论最大 IPC。这表明该工作负载中可能存在内存延迟、I/O 停顿或主机计算方面的低效。

还可以使用 `perf record` 和 `perf report` 进一步定位训练期间占据 CPU 执行时间最多的 Python 和 C++ 函数。下面给出相应的 CLI 命令：

```
perf record -F 2000 -g --call-graph dwarf -o perf.data \
    python train.py
```

这里使用 `perf record` 以 2000 Hz（`-F 2000`）采样，并通过指定 `-g --call-graph dwarf` 捕获完整的 C/C++ 与 Python 调用栈。DWARF 是 Debugging With Attributed Record Formats 的缩写，是嵌入编译二进制文件（例如 ELF 文件）中的标准调试数据格式。DWARF 输出轨迹保存到 `perf.data`（`-o perf.data`），然后使用 `perf report` 生成最热点调用栈及其采样百分比的汇总报告：

```
perf report --stdio -n -g -i perf.data

# Samples  Command   Shared Object        Symbol
# ........ ........  ..................  ................................
     45.0%  python    python               py::forward <...> /src/train.py
```

```
20.5%  python    python              aten::matmul
10.2%  python    python              dataloader_iter_next
 8.7%  python    libnccl.so          ncclAllReduce
 5.3%  python    libc.so.6           read
 ...   ...       ...                 ...
```

可以看到，Python 解释器的 `forward` 函数（训练循环的 Python 部分）占 CPU 采样的 45.0%；PyTorch 的 C++ `aten::matmul` 操作占 20.5%，DataLoader 迭代器的 `next` 函数占 10.2%，NCCL all-reduce 调用占 8.7%，I/O 读取占 5.3%。

这些百分比指出了优化投入的方向。根据这份分析结果，可以针对每个瓶颈制定具体的缓解方案，以改善系统性能：

过高的 Python 开销（`py::forward` 中占 45%）

使用 PyTorch 的 JIT 编译器 `torch.compile`（下一节讨论）消除解释器开销，并将 Python 端操作融合为优化后的 CUDA 代码。

大型 `matmul` 热点（`aten::matmul` 中占 20.5%）

可以使用 PyTorch Compiler 优化这段代码，或将关键矩阵乘法移入自定义 CUDA C++ 内核（例如融合内核），绕过 Python，直接使用优化后的 CUDA 代码。

数据加载停顿（`dataloader_iter_next` 中占 10.2%）

增加 PyTorch 的 `DataLoader num_workers`。常见建议是每个 CPU 配一个 worker，但可以通过实验找到合适的 I/O 并行度，同时避免 CPU 核心过度订阅。还应启用 `persistent_workers=True`，让 worker 进程跨 epoch 保持运行，避免每个 epoch 的启动开销。可以融合或并行化多个 `torch.utils.data.DataPipe`，以减少复杂数据流水线中的 Python 开销。

梯度同步开销（`ncclAllReduce` 中占 8.7%）

优化多 GPU 通信。例如，通过实验调节 DDP 梯度 bucket 大小（`bucket_cap_mb`）。较大的 bucket 可以降低每次集合通信的开销，但在反向计算中也会更晚准备就绪；根据模型和层的时序，这可能改善或损害与反向计算的重叠，应针对具体工作负载调优。还可以考虑梯度压缩技术，或使用 NVIDIA 的 NCCL 8 位梯度压缩来减少带宽占用，但这些方法可能会对准确率造成轻微影响。

主机 I/O 瓶颈（`read` syscalls 中占 5.3%）

在 DataLoader 中使用锁页内存（`pin_memory=True`）和非阻塞 GPU 拷贝（`.to(device, non_blocking=True)`），以重叠 CPU 到 GPU 的数据传输。还可以批量读取文件，或将许多小文件打包成 Arrow、WebDataset（tar 分片）、TFRecord 或 Parquet 分块等适合大规模顺序读取的数据集格式。相比逐样本文件，应优先使用连续分片格式。预取深度较大时（例如 `prefetch_factor=4` 或 `8`），优先使用锁页主机缓冲区。结合 `persistent_workers=True`，计算与通信高效重叠，加载器线程可以持续工作，从大型语料库读取大量小文件时也能消除逐文件开销。

这些方法结合较大的操作系统预读和 NVMe SSD，可以提高 I/O 吞吐量。较新的文件系统和存储库（例如 NVIDIA Magnum IO）也有助于将数据高效地流水线传输到 GPU。

制定方案后，应系统地逐项应用优化并测量效果。务必逐一实现和测试，确认每项优化确实改善性能。这样可以避免多项变更相互作用而产生意外结果；隔离每项变更后，就能知道哪些调整有效、哪些无效。

如果系统安装了 NVIDIA Performance Monitoring Unit（PMU）驱动，就可以使用 `perf` 将 NVIDIA 芯片互连和 fabric 计数器与 CPU 计数器一起采样。例如，NVLink-C2C 设备会在 `/sys/bus/event_source/devices` 下以 `nvidia_nvlink_c2c*` 的形式暴露。可以使用 `perf list` 并检查 `sysfs` 下的 `nvidia_pmu` 条目来确认是否可用。

> Linux `perf` 对 NVIDIA PMU 的支持仅限于设备级链路和 fabric 事件，例如 Grace-Blackwell 上的 NVLink-C2C。SM 流水线、warp 停顿和内存吞吐量计数器仍只能通过 CUPTI 和 Nsight 工具获得；这些 PMU 不会暴露 SM 级内核指标。要分析 SM 利用率、occupancy 和内存吞吐量，应使用 Nsight Compute 或基于 CUPTI 的分析器。还要设置 `NVreg_RestrictProfilingToAdminUsers=0`，以允许非 root 用户分析 SM 级硬件计数器。

PMU 设备可用后，就可以同时收集 CPU 和 NVIDIA 事件。使用 `perf list` 报告的符号事件名称：

```
perf list | grep -i nvidia

perf stat -a \
  -e nvidia_nvlink_c2c0_pmu_0/cycles/ \
  -e cycles,cache-misses \
  python train.py
```

这里，NVLink-C2C PMU 上的 cycles 事件可以将 GPU 互连活动与主机 CPU 行为关联起来。下面是前述 `perf stat` 命令的示例输出，显示运行期间 NVLink C2C PMU 记录到了活动，同时 CPU 产生了周期和缓存未命中：

```
Performance counter stats for 'python train_deepseek_v3.py':

       3,567,890,123  nvidia_nvlink_c2c0_pmu_0/cycles
          45,678,901  cycles
           7,890,123  cache-misses

       2.345678901 seconds time elapsed
```

根据 PyTorch 分析器和 Nsight 工具，初步分析显示 GPU 计算（例如专家矩阵乘法）和 GPU 通信（例如 dispatch 与 combine 操作）是主要瓶颈。不过，CPU、数据加载和 GPU 集合通信同样会影响性能；`perf` 可以显示慢速区域中哪些 CPU 线程和互连 PMU 处于活动状态，从而证实这一点。

要进一步查看链路和 fabric 请求计数器，可以从系统的 `perf list` 中选择其他 NVIDIA PMU 事件，并像前面所示那样将其添加到 `perf stat` 命令中。

总之，将 `perf` 的高层 CPU 吞吐量指标和调用图热点，与 Nsight Systems 和 Nsight Compute 的设备指标及时间线结合起来，就能建立覆盖主机和设备的完整性能图景。首先处理最大的 CPU 端瓶颈和数据停顿，然后优化 GPU 通信并调优 GPU 内核。

PyTorch Compiler（torch.compile）

在 PyTorch 中，最快速的性能收益之一就是使用带有 `torch.compile()` 的 PyTorch 编译器。编译器栈包括 TorchDynamo、AOT Autograd 和 TorchInductor，它们可以捕获 graph、融合操作，并为目标后端（例如 NVIDIA GPU）生成高性能代码。

PyTorch Compiler 可以将许多小操作融合成更大的内核，从而消除大量 Python 解释器开销和 GPU 内核启动延迟。完成基线性能分析后，我们在模型上启用 `torch.compile`，观察能否轻松获得加速。下面介绍这一过程及其结果。

使用 PyTorch Compiler

使用默认设置的 PyTorch compiler 非常直接，除了按下例将模型包装起来（`model = torch.compile(model)`）之外，无需修改代码。在底层，TorchDynamo 会追踪 Python 代码，AOT Autograd 捕获反向传播，而 TorchInductor 借助 OpenAI 的 Triton 生成 GPU 内核代码（下一章将讨论），自动产生高效的融合内核。

编译器会观察模型的前向传播，识别连续操作（例如逐元素激活、layer norm 等）的融合机会，并为这些操作以及部分反向传播生成融合内核。结果是内核启动次数显著减少，每次迭代的 CPU 开销也更低。

编译步骤确实会带来数秒的开销，大型模型甚至可能需要数分钟；但对于长时间训练或反复推理，这一成本可以摊销。幸运的是，TorchInductor 会缓存编译后的内核，后续运行无需再次承担编译成本。PyTorch 社区也在持续改进编译和启动性能，允许跨运行保存并复用编译产物。可以使用 `torch.compiler.save_cache_artifacts()` 和 `torch.compiler.load_cache_artifacts()`，在不同运行或节点之间持久化 TorchInductor 输出，从而减少长时间训练或服务启动时的开销。

一个例子是 PyTorch Mega-Cache 功能。这是一种端到端编译缓存，可以将编译后的内核保存到磁盘，并在后续运行中重新加载。借助 PyTorch Mega-Cache，可以只编译一次（例如离线编译），然后在多次训练会话之间复用优化后的内核，从而缩短启动时间。你仍能获得 TorchInductor 的 warp specialization 等内核优化收益，同时避免每次重新编译 graph。

还可以在其他计算节点上使用该编译缓存；此时必须确保各节点的 CUDA、PyTorch 和 Triton 版本兼容。

需要注意，PyTorch Compiler 在内部应用了复杂的优化技术。例如，第 10 章曾介绍 warp specialization。TorchInductor 的自动调优器会针对不同 tile 大小、内存访问模式等生成多个内核变体，并在后台应用 memory warp 与 compute warp specialization 等技术，然后自动为硬件选择最快的变体。

TorchInductor 支持在 GEMM 内核周围进行 prologue和 epilogue 融合。例如，bias-add 位于 `matmul` 之前；而 `matmul` 之后的 epilogue 包含激活、dropout 和 residual 等逐元素操作。

将这些内核 prologue 和 epilogue 操作合并到单个优化内核中，TorchInductor 可以减少内存流量、降低内核启动开销并提高 occupancy。可以通过分析器验证这一点：SM 利用率应会更高。

由于 PyTorch 提供简洁、以张量为中心的接口，而不暴露 CUDA 层面的 warp 细节，这些复杂优化对开发者完全透明。因此，虽然 PyTorch API 中看不到“memory warp”或“compute warp”标志，但这些技术确实在底层运行。代码编译后，可以从分析器指标中看到 warp specialization 的收益，包括更高的 occupancy、更少的内存延迟停顿和更高的 SM 利用率。

为了说明编译模式的收益，下面在 MoE 模型上比较 PyTorch eager 模式和编译执行。我们先测量模型在普通 eager 模式下的一次训练迭代，再使用 `"max-autotune"` 编译模式进行测量。代码如下，后面给出示例输出：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

# ---- Setup Model ----
device = 'cuda'
model_name = "..."
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, fused=True)

# ---- Create a dummy batch of token IDs ----
batch_size = 4
input_texts = ["MoE's are awesome!"] * batch_size
enc = tokenizer(input_texts, return_tensors="pt", padding=True, truncation=True)
input_ids = enc.input_ids.to(device)
attention_mask = enc.attention_mask.to(device)
labels = input_ids.clone()  # for causal LM, labels = input_ids

# ---- Make runs deterministic ----
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True

# --- Eager timing ---
torch.cuda.synchronize()
start = torch.cuda.Event(enable_timing=True)
end = torch.cuda.Event(enable_timing=True)

for _ in range(iters_warmup):
    out = model(input_ids, attention_mask=attention_mask, labels=labels)
    loss = out.loss if hasattr(out, "loss") else out
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

```
torch.cuda.synchronize()
start.record()
for _ in range(iters_meas):
    out = model(input_ids, attention_mask=attention_mask, labels=labels)
    loss = out.loss if hasattr(out, "loss") else out
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
end.record()
torch.cuda.synchronize()
print(f"Eager mode step time: {start.elapsed_time(end)/iters_meas:.3f} ms")

# --- Compile the model (choose one mode)
# enables graph trees
compiled_model = torch.compile(model, mode="reduce-overhead")
# Alternatives:
# more tuning, longer compile
# compiled_model = torch.compile(model, mode="max-autotune")
# balanced
# compiled_model = torch.compile(model, mode="default")

# Warm-up compiled
for _ in range(iters_warmup):
    out = compiled_model(input_ids, attention_mask=attention_mask, labels=labels)
    loss = out.loss if hasattr(out, "loss") else out
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)

torch.cuda.synchronize()
start.record()
for _ in range(iters_meas):
    out = compiled_model(input_ids, attention_mask=attention_mask, labels=labels)
    loss = out.loss if hasattr(out, "loss") else out
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
end.record()
torch.cuda.synchronize()
print(f"Compiled mode step time: {start.elapsed_time(end)/iters_meas:.3f} ms")
```

在这个示例中，PyTorch eager 模式每次迭代约需 248 ms。预热并让编译器完成优化后，编译模式约需 173 ms。使用 `"max-autotune"` 的编译版本比 eager 执行快约 30%。实际加速幅度会随模型结构、batch 大小和动态形状而变化；由单个大型 GEMM 主导的稠密模型，提升可能只有 <10%。

这些收益主要来自合并大量小型 GPU 内核。MoE 架构中的 token dispatch/combine 和逐 token 激活模式经常包含这类小内核。将小操作融合成更少、更大的内核后，中间数据可以保留在寄存器、共享内存等更快的片上内存中，而不必反复与全局设备内存（HBM）交换。

> 对于 MoE 架构使用的高度动态 token 路由，优先选择 default 或 `max-autotune-no-cudagraphs`。当输入形状趋于稳定，或使用具有有限离散形状的 CUDA Graph Trees 时，再切换到 `max-autotune`。

在本例中，dispatch/combine、激活函数等许多小操作都被 TorchInductor 融合掉了。检查编译运行的轨迹时，会看到时间线上的 GPU 内核大幅减少；剩下的内核数量更少但运行时间略长，因为每个内核对应多个合并后、一次完成多个步骤的操作。

在大量迭代中，一次性编译开销被摊销，收益会更加明显。对编译模型执行进行分析会发现 GPU 内核启动次数大幅减少，因为 eager 模式中分开的许多操作现在已经融合在一起。

需要注意，如果稠密模型由于大型线性层而主要由一个巨大 GEMM 主导，那么使用 `torch.compile` 的收益可能很有限（例如 <10%）。这是因为模型很可能已经高效利用 Tensor Core，同时几乎没有可供内核融合和消除 Python 开销的空间。

但 MoE 等稀疏架构包含数百个中等规模的 `matmul` 操作，通常能获得显著收益，因为编译可以减少 Python 开销、降低内核启动延迟，并将多个步骤融合为优化内核。因此，与稠密模型相比，PyTorch Compiler 对 MoE 的性能提升往往更大。

除了自动算子融合，还可以将用户定义的自定义内核直接集成到 `torch.compile` 工作流中。这样可以兼得两者优势：一般模式交由编译器执行 graph 级优化，需要时仍保留完全控制权。

例如，可以为性能关键操作编写专用 Triton 或 CUDA 内核，并将其注册为自定义算子。模型被追踪和编译时，TorchInductor 会将其作为更大执行 graph 中的单个融合操作处理。最终得到的是：手工调优的自定义代码嵌入完全优化、由编译器管理的执行 graph。

TorchInductor 的灵活性使自定义内核也能受益于编译器对周围操作的优化（例如与相邻层融合）。实践中，这意味着可以使用自己的高性能内核，同时不牺牲 PyTorch Compiler 优化模型其余部分的能力。

总之，在训练循环中使用 PyTorch Compiler（包括 `"max-autotune"` 模式），通常可以用相对较少的工作在现代 GPU 上获得可观加速。可以使用 `torch.profiler`、Linux `perf`、Nsight Systems、Nsight Compute 等分析器进行全面验证。

编译与编写自定义内核的比较

使用 TorchInductor 等编译器后端，许多操作会自动融合成高效内核。如前所见，只需使用 `torch.compile` 就能以很少的工作获得可观提升。但有时自动生成的代码不如手工编写的自定义内核高效，或者某个操作根本没有被编译器捕获。这就引出了一个问题：何时应依赖编译器融合，何时应自己编写自定义 CUDA 内核?

大多数情况下，首选使用带 graph 捕获的高层 `torch.compile`（底层由 TorchInductor 执行）。这比编写自定义 CUDA 内核省力得多，而且通常无需专门编码就能获得足够好的性能提升。

TorchInductor 已在内部应用许多高级优化，例如逐元素操作融合、层操作合并和布局优化等。手工编写融合内核既耗时又脆弱，而编译器在大多数情况下可以自动完成这些工作。

如果模型使用了编译器处理不佳的新颖操作或模式，可能需要编写自定义内核，并将自定义算子集成到 PyTorch 中。下一章将详细介绍这一过程。

总之，性能调优时应首先尝试 `torch.compile`，因为它简单、通常足够且成本相对“免费”。当内置自动化不足时，再进入下一层优化：创建自定义内核。之后才考虑为剩余热点编写自定义内核，以融合某些不受支持的优化器操作或专用 attention 模式。不过，即使对于专用 attention 模式，PyTorch 也提供了 FlexAttention API（`prefill`）和 FlexDecoding API（`decode`）；在 PyTorch 中实现训练和推理用的自定义 attention 内核时，优先使用这些 API，后文将会介绍。

编译模式及速度、内存和编译时间之间的权衡

PyTorch 为 `torch.compile` 提供了多种模式，可针对不同场景调节编译器的激进程度和能力。可以使用 `torch.compile(model, mode="...")` 显式选择模式，包括 `"default"`、`"reduce-overhead"`、`"max-autotune"` 和 `"max-autotune-no-cudagraphs"`。每种模式都组合了不同的 CUDA Graph、自动调优和优化级别选项，汇总见表 13-5。

表 13-5　编译模式及其主要特征汇总

模式	说明	编译时间	额外内存	主要特征
`default`	均衡优化（速度较好，编译时间和额外内存较少）；包含轻量自动调优；稳定片段可能使用 CUDA Graph	低–中	否	通用融合、基础自动调优
`reduce-overhead`	降低每次迭代开销（适合小 batch）；适合推理或小 batch；检测到动态形状时会自动跳过 CUDA Graph，以保证正确性	中	是（工作区缓存）	尽可能使用 CUDA Graph 消除启动开销
`max-autotune`	最大化运行时性能（适合长时间运行）；编译时间较长；适合拥有大量 SM 和 GPU 内存时进行激进调优	高（编译较慢）	可能（若使用 graph）	激进的 Triton 自动调优；在 GPU 上启用 CUDA Graph
`max-autotune-no-cudagraphs`	执行 `max-autotune` 的全部工作，但不捕获 CUDA Graph 适合动态形状，或调试被 CUDA Graph 掩盖的问题	高	否	与 `max-autotune` 相同，但禁用 graph 以获得灵活性

下面详细介绍表 13-5中的各个模式：

`default`

未显式指定模式时，这就是默认模式。它在编译代码速度、编译时间和内存使用之间取得平衡，执行标准优化并使用默认 TorchInductor 后端。对于需要适中编译时间的大模型，或内存紧张的场景，该模式通常较合适。它包含轻量自动调优，稳定片段可能使用 CUDA Graph，整体目标是平衡速度和编译成本。

`reduce-overhead`

该模式重点降低 Python 和运行时开销，尤其适合小模型，或每次迭代执行次数较少的模型；在这些情况下，即使少量开销也会损害性能。它会激进地利用 CUDA Graph 消除每次迭代的启动开销，也可能为持久使用分配额外内存，例如重复使用的工作区，从而避免频繁调用 CUDA malloc/free。比如，它可能缓存大型工作张量，而不是每次迭代都重新分配。如果检测到动态形状，该模式会自动跳过 CUDA Graph 并回退到 eager，以保证正确性。

该模式可以在低延迟场景中加速推理和训练，但会消耗一些额外内存。CUDA Graph 要求 graph 的内存地址保持不变，因此只有在输入大小不变且不存在动态形状操作等特定操作时才能使用。否则 graph 会失效或需要重新编译；如果某个片段无法应用 CUDA Graph，编译器会自动回退。

`max-autotune`

该模式不考虑编译时间，目标是生成尽可能快的代码。它会对内核进行广泛自动调优，例如尝试矩阵乘法的多种 tiling 配置，并利用 TorchInductor 已知的各种性能优化。在现代 GPU 上，`max-autotune` 还会默认对稳定执行启用 CUDA Graph。

该模式的编译过程可能明显更长，大型模型可能需要数分钟。它适合只编译一次、随后多次运行模型的场景，例如长时间训练，或部署长期处理大量请求的模型。用较长的前置编译时间换取的通常是最佳运行时性能；例如自动调优后，`matmul` 可能针对特定 GPU 和张量形状采用最优 block 大小，优于默认启发式选择。

`max-autotune-no-cudagraphs`

顾名思义，该模式与 `max-autotune` 相同，但禁用 CUDA Graph 捕获。当 CUDA Graph 会干扰所需的运行时行为，或运行时行为与 CUDA Graph 不兼容时，这一点很重要。例如，CUDA Graph 要求静态形状和内存地址，因此不能使用变化的输入形状，也不能依赖每次迭代分配新内存。

此外，在性能测量中，CUDA Graph 可能掩盖内核启动开销，而某些基准测试并不希望这样。因此，该模式可以在不使用 CUDA Graph 的情况下实现最大程度的内核优化，保持灵活性，并便于调试 CUDA Graph 可能引入或掩盖的问题，例如依赖形状的控制流，或 graph 捕获期间分配器偶尔重新分配地址。输入大小每次迭代都变化，或需要调试可能被 CUDA Graph 掩盖的问题时，应使用该模式。

对大多数用例而言，默认模式是很好的起点，能够以最少的麻烦带来可观加速。如果模型仍不够快且可以接受更长编译时间，可以尝试 `reduce-overhead` 和 `max-autotune`，以获得

可能更好的融合内核，尤其是模型由可自动调优的大型 `matmul` 操作主导时。`max-autotune` 在某些模型上也可能使延迟变差，因此务必针对具体工作负载和硬件分析不同模式。

另一方面，如果优化的是非常小的模型，或 Python 开销是瓶颈的代码片段（例如包含大量小张量操作的紧凑训练循环），使用 `reduce-overhead` 通过 CUDA Graph 捕获几乎消除所有内核启动运行时开销，可能获得最大收益。但要注意其约束：每次迭代的工作负载应保持一致，并符合 graph 捕获要求，包括不能改变动态形状、不能进行新的内存分配等。

`max-autotune-no-cudagraphs` 更像是专用选项：当输入大小变化而无法使用 graph，或希望在不考虑 graph 摊销的情况下测量原始内核性能，同时又需要最大程度内核优化时，它很有用。

无论如何，改变 PyTorch Compiler 模式后都应重新分析和测量。性能工程不存在适用于所有场景的单一模式。此外，选择模式时应监控内存使用；使用 CUDA Graph 的模式会分配较大的静态缓冲区，增加内存占用。

在内存极度受限的场景中，可以优先使用这一无 graph 模式，避免 CUDA Graph 的额外内存开销。下面将讨论如何检查编译器的行为，包括它创建了一个 graph 还是多个 graph，以及是否使用了 CUDA Graph 等。

区域编译

对于包含大量相同 block 的模型（例如 Transformer 和 MoE），可以使用区域编译减少冷启动执行时间，同时降低反复重新编译的开销，而不牺牲内核融合能力。

具体来说，区域编译只编译一次重复 block（例如 Transformer block），然后在所有出现位置复用代码，从而减少编译时间。PyTorch 通过 `torch.compiler.nested_compile_region()` 支持区域编译；该 API 将某个 block 标记为嵌套编译区域，该区域首次运行时编译，后续运行复用。

区域编译还会保持正确性。如果编译器检测到新的输入条件（`shape`、`dtype`、`device`、`stride`、`globals`），就会透明地重新编译该区域。

区域编译适合重视启动延迟的推理引擎和短任务，也适合 graph 经常失效的场景。区域编译代码的性能与完整编译代码的吞吐量相近。

分析和调试编译器性能问题

使用 `torch.compile` 时，需要知道如何调试编译器无法优化模型某部分的情况，例如某些操作未被融合，怀疑 graph break 导致回退到 eager 执行。PyTorch 提供了检查这些情况的工具。

> 现代 PyTorch 版本通过 shape guard 部分支持动态形状，可以消除一些不必要的 graph break。但真正动态的工作负载仍可能需要回退到 eager，或使用 `max-autotune-no-cudagraphs`，以确保正确性。

`torch._dynamo.explain(model)` 会打印 graph break 报告，包括发生 graph break 的原因，以及模型中未被 TorchDynamo 捕获的部分。它还会列出未被捕获、必须在速度更慢的 Python eager 模式中执行的操作或数据相关控制流。

graph break 的常见原因是模型包含不支持的操作。Dynamo 的 `explain()` 输出会建议如何获取更多细节并诊断问题。利用这些提示可以定位导致 break 的确切操作或控制流。

另一种有用方法是在运行脚本前设置环境变量 `TORCH_LOGS="+dynamo"` 或 `TORCH_LOGS="+dynamo,+inductor"`。前缀 `+` 会为 `torch.compile` 流程中的 TorchDynamo、TorchInductor 等组件启用详细（`DEBUG` 级别）日志。详细日志包括 graph break、回退到 eager 模式以及编译阶段等信息。如果模型使用 `torch.compile` 后异常缓慢，这些日志可以帮助定位执行何时、何处退出了编译 graph。

如果模型包含无法在编译时解析的真正动态形状或动态控制流，可能需要引导编译器。例如，将模型拆成可编译的片段，让真正动态的部分留在 Python 中执行。

要分析和基准测试 TorchInductor 生成的内核，可以指定环境变量 `TORCHINDUCTOR_UNIQUE_KERNEL_NAMES=1` 和 `TORCHINDUCTOR_BENCHMARK_KERNEL=1`。设置后，Inductor 会为生成的内核模块生成基准测试 harness 代码；该 harness 产生的日志有助于定位意外的 graph break 和性能问题。

还可以将部分代码标记为 `torch._dynamo.mark_dynamic(tensor, dim)`，让编译器知道应预期动态形状。这可以消除因形状不匹配导致的不必要 graph break。下一章深入介绍 PyTorch Compiler 时会详细讨论这些技术。

总之，当 `torch.compile` 没有带来预期加速时，可以结合编译器日志使用 `torch._dynamo.explain()`，识别导致回退的操作或代码区域。然后可以采用替代操作、以不同方式重塑张量、接受较低的动态性，或只对模型的特定部分禁用编译等变通方案。这样既保留模型大部分区域的性能收益，也能处理边缘情况。

PyTorch 优化的 Attention 机制

Transformer 模型会在 attention 机制上花费大量时间。可以应用多种 PyTorch attention 优化技术，避免 attention 成为瓶颈。下面快速总结几种技术及其适用时机：

缩放点积 attention（SDPA）

> PyTorch 的高层 API `torch.nn.functional.scaled_dot_product_attention`（即 SDPA）会自动为指定硬件选择最快的可用 attention 内核（例如 FlashAttention）。当模型的 attention 模式和 dtype 得到所选后端（Flash、memory-efficient 或 math）支持时，使用它即可轻松获得加速；如果不支持，则回退到标准 attention 实现。

FlexAttention

> 这是一种基于编译器的 attention 自定义稀疏模式方法。FlexAttention 可以针对特定稀疏 attention 模式（例如块稀疏或滑动窗口 attention）生成优化内核，从而显著加速，如图 13-3所示。对于 `scaled_dot_product_attention` 不支持的特殊情况，应使用 FlexAttention。

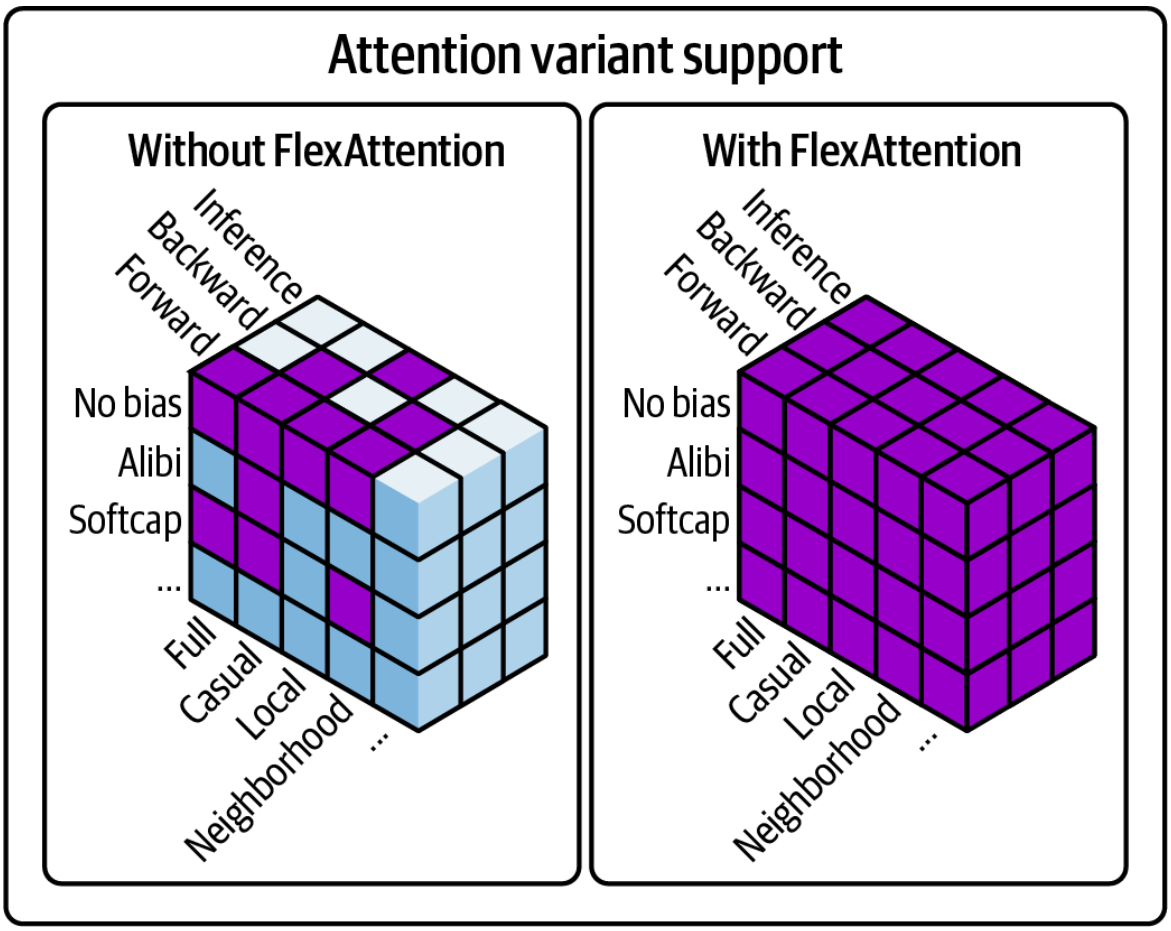


图 13-3　FlexAttention 支持自定义 attention 变体

FlexDecoding

FlexDecoding 是 FlexAttention 的对应方案，用于优化解码或文本生成阶段。FlexDecoding 可与 `torch.compile` 和动态缓存布局集成，在序列生成的 decoder 侧进行编译时优化，包括跨时间步高效使用 KV cache。它通过减少解码期间的冗余计算来加速自回归生成，面向 LLM 推理工作负载（包括长序列生成），不会改变训练时的 attention 语义。

Context parallel（上下文并行）

Context Parallel 沿序列长度维度在参与的设备或 rank 之间切分 attention，以扩展上下文长度。使用 `context_parallel()` API，可以在指定作用域内将 `scaled_dot_product_attention` 替换为支持上下文并行的内核。该机制按序列在各 rank 之间切分 query-key-value（QKV），并在 attention 期间同步，而不是在单个 GPU 的线程之间并行化 attention。

PyTorch 架构优化（torchao）、量化、稀疏与剪枝

PyTorch Architecture Optimization（torchao）将量化、稀疏、剪枝及相关数值调试工具集中到同一个命名空间中。其量化子包（`torchao.quantization`）提供常用的 FX graph

模式工作流，包括训练后量化（PTQ）、量化感知训练（QAT）以及 `QConfigMapping` API，用于将模型转换并优化到 INT8、FP8 和新兴格式。

除量化外，`torchao` 还支持剪枝（`torchao.pruning`）以及 2:4 和块稀疏等稀疏技术（`torchao.sparsity`），可以在准确率损失很小的情况下显著加速。

`torch.compile()` 可与 `torchao` 量化框架集成。底层的 TorchDynamo 会将每个子模块的计算捕获为优化 graph，随后 TorchInductor 生成利用 `torchao` 的硬件感知内核。这能为模型训练和推理带来一致的端到端性能改善，同时保留对数值格式和内存布局的精确控制，使其适合量化等生产级性能优化。

使用 CUDA Stream 实现并发

正如前章所述，CUDA stream 支持 GPU 上操作的并发和重叠。默认情况下，PyTorch 会在设备默认 stream（stream 0）上顺序调度所有操作。不过，许多任务彼此独立，如果资源允许，GPU 可以使用多个 stream 并行执行。例如，使用独立的非默认 stream，可以将数据传输与计算重叠，或并行运行神经网络的不同分支。

> 请记住，现代 GPU 具有多个 DMA 拷贝引擎。为 H2D 拷贝使用独立 stream，可以在不阻塞计算的情况下实现真正并行的数据传输。这种硬件支持让 stream 并发更加有效。

在 PyTorch 中，可以使用 `torch.cuda.Stream()` 创建 stream。随后可以使用 Python 上下文管理器 `torch.cuda.stream(stream)` 在该 stream 上启动工作，或显式将操作分配给该 stream。PyTorch 会按 FIFO 顺序将操作提交到指定 stream，就像提交到默认 stream 一样。

重叠通信与计算

CUDA 流的常见用途是让主机到设备（host-to-device，H2D）数据加载与 GPU 计算重叠。这有助于掩盖使用 GPU 等外部设备时、相对于主机 CPU 所产生的数据传输延迟。

例如，一个 stream 可以在默认 stream 训练当前 batch 时，将下一批输入数据从 CPU 拷贝到 GPU 内存。默认 stream 准备好处理下一批时，数据传输已经完成，GPU 可以立即处理，从而有效隐藏 I/O 延迟。下面示例使用两个 stream：默认的 `compute_stream` 和非默认的 `transfer_stream`，在 PyTorch 中重叠数据传输与计算：

```
# Set up streams
device = 'cuda'

# for H2D data transfers
transfer_stream = torch.cuda.Stream(device=device)

# for compute
compute_stream  = torch.cuda.default_stream(device=device)

# Create an iterator so we can preload "next" batches
```

```
dataloader_iter = iter(dataloader)

# Preload the very first batch onto GPU
first_batch = next(dataloader_iter, None)
if first_batch:
    with torch.cuda.stream(transfer_stream):
        next_inputs, next_labels = (
            first_batch[0].to(device, non_blocking=True),
            first_batch[1].to(device, non_blocking=True),
        )

for _ in range(len(dataloader)):
    # 1) Wait for transfer of `next` batch to finish, then swap into compute var
    # Multiple copy engines allow H2D/peer copy concurrency.
    # Verify parallelism in Nsight Systems (Copy engines lanes)
    # And tracing/profiling tools (HTA, etc.)
    compute_stream.wait_stream(transfer_stream)
    inputs, labels = next_inputs, next_labels

    # 2) Kick off transfer of the *following* batch on the transfer_stream
    batch = next(dataloader_iter, None)
    if batch:
        with torch.cuda.stream(transfer_stream):
            next_inputs, next_labels = (
                batch[0].to(device, non_blocking=True),
                batch[1].to(device, non_blocking=True),
            )

    # 3) Run forward/backward on compute_stream
    with torch.cuda.stream(compute_stream):
        outputs = model(inputs)
        loss    = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

该示例使用两个 CUDA stream：专用传输 stream 进行异步主机到设备拷贝，默认计算 stream 执行模型工作。这可以隐藏 H2D 延迟，并符合 PyTorch 推荐的通信与计算重叠模式。

具体来说，模型在默认 stream 处理一个 batch 时，下一批数据已经在 `transfer_stream` 上传输。消费预加载 batch 前调用 `compute_stream.wait_stream(transfer_stream)`，可以在不设置全设备屏障的情况下保证正确顺序；`.to(device, non_blocking=True)` 调用则确保拷贝使用异步 DMA，不阻塞调用它的 CPU 线程。

使用 `next(dataloader_iter, None)` 可以显式控制传输入队与内核操作执行的时机，确保一个 batch 在传输 stream 上移动时，另一个 batch 正在计算 stream 上执行，如图 13-4 所示。

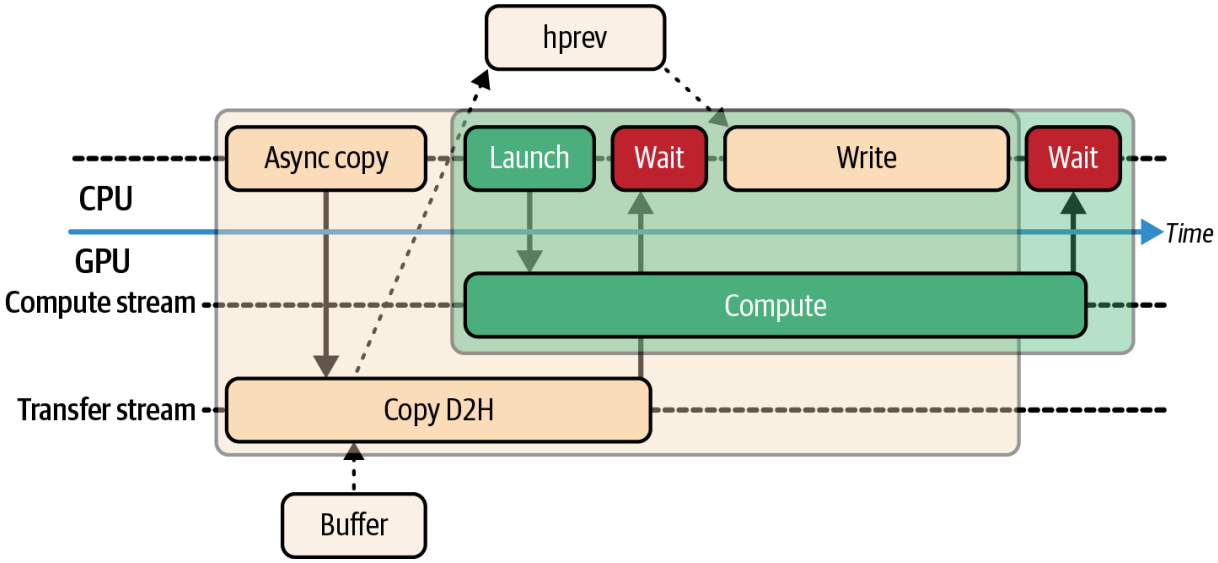


图 13-4　使用专用计算和传输 stream重叠计算与数据传输

此外，代码提前从 `dataloader_iter` 取出数据并存入 `next_inputs next_labels`，将 batch 加载（CPU 上的 `transfer_stream`）与 batch 处理（GPU 上的 `compute_stream`）分离。这样每个 stream 始终都有一个正在处理的 batch，使数据加载与计算解耦并最大化重叠。

> 添加 stream 时，务必使用 Nsight Systems或 PyTorch profiler 进行分析，观察 GPU 利用率。正确时，传输与计算会重叠，利用率可能接近 100%；如果利用率下降，或数据传输与计算呈顺序执行，应检查 CUDA tensor 的 `print()` 等隐式同步，以及代码中额外的 CUDA同步。

使用 Event 进行 Stream 同步

使用多个 stream 时，有时需要在它们之间协调，确保一个 stream 的工作完成后，另一个 stream 才使用其结果。使用 CUDA Event 可以轻量地同步特定位置，详见第 11 章。

借助 CUDA Event，可以在一个 stream 上记录事件，让另一个 stream 等待该事件。这避免了 `torch.cuda.synchronize()` 的重量级全局同步，只同步处理特定事件所需的 stream。

实际上，即使在多 GPU 环境中，也可以使用 Event 同步设备间的工作：一个 GPU 在自己的设备 stream 上记录事件，另一个 GPU 在自己的设备 stream 上等待事件。这正是 NCCL 在底层处理依赖的方式。

合理使用 Event，可以尽可能让多个 GPU 保持并行工作。下面是在 PyTorch 中使用 CUDA Event 同步 `stream1` 和 `stream2` 的示例：

```
# Disable timing on the event since we're using it purely for synchronization.
event = torch.cuda.Event(enable_timing=False)

# In first stream:
with torch.cuda.stream(stream1):
    kernel_launch(...)
```

```
    event.record()           # record event at end of work in stream1

# In another stream or on host:
stream2.wait_event(event)    # make stream2 wait until event is signaled

with torch.cuda.stream(stream2):
    other_kernel_launch(...)
```

在这段代码中，我们在 `stream1` 的一段工作结束时记录 Event。之后，在 `stream2` 启动工作前调用 `stream2.wait_event(event)`，插入依赖关系，使 `stream2` 在 `stream1` 到达该位置并发出事件信号前不会执行下一个内核。Event 适合调度 stream 之间的轻量依赖，因为它避免了会阻塞所有 stream 执行的重量级全局同步。

下面回顾上一节的 PyTorch 数据加载器/重叠示例，并改用 CUDA Event 进行同步。仍使用前面的两个 stream（`transfer_stream` 和 `compute_stream`），但添加 `transfer_done` CUDA Event，在细粒度、事件特定的层级上同步：

```
import torch

device = 'cuda'

# for H2D copies
transfer_stream = torch.cuda.Stream(device=device)

# for compute
compute_stream  = torch.cuda.default_stream(device=device)
# sync-only event (low overhead)
transfer_done = torch.cuda.Event(enable_timing=False)

# Iterator so we can preload ahead
dataloader_iter = iter(dataloader)

# ---- Preload first batch ----
first_batch = next(dataloader_iter, None)
if first_batch:
    with torch.cuda.stream(transfer_stream):
        next_inputs, next_labels = (
            first_batch[0].to(device, non_blocking=True),
            first_batch[1].to(device, non_blocking=True),
        )
        # mark when H2D is done
        transfer_done.record(stream=transfer_stream)

for _ in range(len(dataloader)):
    # ---- Sync: wait for the transfer to complete ----
    compute_stream.wait_event(transfer_done)
    inputs, labels = next_inputs, next_labels

    # ---- Kick off next transfer ----
    batch = next(dataloader_iter, None)
    if batch:
        with torch.cuda.stream(transfer_stream):
            next_inputs, next_labels = (
                batch[0].to(device, non_blocking=True),
                batch[1].to(device, non_blocking=True),
            )
            transfer_done.record(stream=transfer_stream)

    # ---- Compute on the compute stream ----
    with torch.cuda.stream(compute_stream):
```

```
        outputs = model(inputs)
        loss    = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

该示例与上一节使用相同的重叠模式，但采用 CUDA Event 实现传输 → 计算同步，而不是另一个示例中的 `wait_stream()` 机制。

代码仍使用 `next(dataloader_iter)` 提前预加载 batch，使数据传输与计算始终重叠。

不过，这里在异步拷贝入队后，立即通过 `transfer_done.record(stream=transfer_stream)` 在 `transfer_stream` 上记录 `transfer_done` Event，为事件记录时间点。

`compute_stream.wait_event(transfer_done)` 会让 `compute_stream` 等待拷贝完成并触发 `transfer_done` Event，然后消费预取 batch，并在 `compute_stream` 上执行计算。

除了数据加载，CUDA stream 还适用于许多场景。下面讨论它们在 MoE 模型中的用法。

在 MoE 模型中使用 CUDA Stream

实际上，Transformer 模型层之间通常存在顺序依赖，因此不能任意并行运行各层。但在 MoE 架构中，不同专家的计算彼此独立，可以在不同 CUDA stream 上并发运行。

每个专家处理输入中的不同片段，在汇合点聚合专家输出。必须确保每个专家只写入输出张量分配给自己的切片。如果两个专家意外写入重叠内存区域，就会产生竞态条件，导致结果损坏，或因专家写入顺序不确定而触发同步问题。

为避免这些问题，应使用正确的 stream 级同步（例如 stream Event），并确认专家内核之间的内存已清晰分区。通过强制分离专家执行与输出聚合，可以在不牺牲并行性的情况下保持正确性。

> NVIDIA 的 Compute Sanitizer 可以检测 CUDA 代码中的并发和同步问题，包括竞态条件和死锁。还可以设置 `CUDA_LAUNCH_BLOCKING=1` 强制内核同步执行，通过使执行确定化来暴露顺序和依赖错误，并发现输出是否在完全生成前就被消费。

在本例中，理论上每个专家都可以运行在自己的 stream 上；如果框架扩展到多 GPU，甚至可以运行在自己的 GPU 上。此时聚合结果时需要同步，最好使用 stream Event。

流水线并行（在不同设备上的不同模型阶段之间传递 microbatch）以及服务多个推理请求，是天然受益于多 stream 的两个场景。例如在流水线并行工作流中，模型每个阶段都有自己的 stream，可并发处理不同 microbatch，同时与相邻阶段通信。

在多请求推理服务中，可以将每个请求的模型执行启动在自己的 stream 上。如果硬件资源充足，通过重叠推理计算可以提高吞吐量，但资源共享开销会增加一些单请求延迟。

总之，CUDA stream 通过重叠多个内核、阶段或请求的工作，帮助硬件释放额外性能。它们需要谨慎同步以避免竞态条件；使用得当时，可以隐藏延迟并提高 GPU 利用率。

引入并发时，建议持续分析代码。还要注意，100% 利用率的顺序执行可能比引入资源竞争的并行执行更快。不过，stream 往往能利用原本会空闲的 GPU 部分，找到合适的平衡很重要。

> 务必确认工作启动在预期的 stream 上。例如，意外使用默认 stream 会重新引入不必要的串行化。这很容易出错，因此再次强调。

使用 CUDA Graph 降低内核启动开销

前面章节已经介绍，CUDA Graph 可以消除每次迭代的启动开销、降低 CPU 启动开销，并消除内核之间细小的空闲间隙。即使移除最小的空闲间隙，也能提高有效利用率并使迭代时间更稳定。下面介绍如何在 PyTorch 中使用它们。

捕获 CUDA Graph 并预分配内存

PyTorch 提供 `torch.cuda.CUDAGraph` API，用于捕获和重放 CUDA Graph。一般流程是先正常运行几次迭代预热模型，初始化所需数据和分配；然后创建 `CUDAGraph` 对象及专用的非默认 CUDA stream，以隔离捕获过程。

> 使用 `"reduce-overhead"` 或 `"max-autotune"` 时，如果模型稳定，编译器会自动捕获 CUDA Graph。此时使用 PyTorch Compiler 的这些模式无需手写样板代码。如果模型每次迭代的形状不同，应考虑使用 `"max-autotune-no-cudagraphs"` 避免 graph 捕获，因为当前 CUDA Graph 要求静态形状。

随后在 `torch.cuda.graph()` 上下文中使用指定的捕获 stream 完整执行一次模型，以记录操作序列。操作捕获到 CUDA Graph 后，就可以根据需要用新输入调用 `replay()` 重放 graph（例如进行模型训练或推理）。

捕获 CUDA Graph 前，捕获期间使用的所有静态内存都必须预分配，最好按最大大小分配。这些缓冲区包括输入、输出和中间张量。如果捕获期间发生新的内存分配，graph 会失败，并报出类似“operation not permitted when stream is capturing”的错误。

为了减少碎片并为固定缓冲区最大化连续内存空间，可以在进入捕获块前立即调用 `torch.cuda.empty_cache()`。这会清除未使用的缓存内存，让分配器更有机会不受干扰地布置预留缓冲区。

> 频繁使用 `torch.cuda.empty_cache()` 会破坏分配器效率并带来长期性能成本。应将其视为捕获 graph 时的一次性安全措施，而不是日常维护工具。

请记住，PyTorch 缓存分配器支持 CUDA 异步分配器（`cudaMallocAsync`），可以复用固定内存地址。但这并不能绕过 CUDA Graph 不允许在 graph 内创建新分配的要求。

仍需预先分配固定大小的缓冲区，避免在 graph 内申请新内存时产生运行时错误。确保所有张量在捕获前的预热期间达到所需最大大小。后面的 graph 重放小节会进一步介绍。

必须使用专用的非默认 stream 进行 graph 捕获，以避免不应包含在 CUDA Graph 中的操作产生干扰。下面的代码展示如何使用专用 `capture_stream` 在 PyTorch 中捕获 CUDA Graph：

```
g = torch.cuda.CUDAGraph()
capture_stream = torch.cuda.Stream()

# Prepare static inputs and outputs
static_input = torch.randn(batch_shape, device='cuda')
static_output = torch.empty(output_shape, device='cuda')

# Warm-up step on capture_stream to allocate buffers without recording
with torch.cuda.stream(capture_stream):
    tmp = model(static_input)
    static_output.copy_(tmp)

# ensure warm-up is complete
capture_stream.synchronize()

# Begin graph capture
with torch.cuda.graph(g, stream=capture_stream):
    tmp = model(static_input)
    static_output.copy_(tmp)

# ensure capture is complete before using the graph
capture_stream.synchronize()
```

在这段代码中，首先在 GPU 上以固定形状分配 `static_input` 和 `static_output`。然后在 `capture_stream` 上运行一次预热迭代，确保 `model(static_input)` 内部所需内存已分配，并完成一次性初始化。

> 通过预分配输出缓冲区，并在预热和捕获阶段都使用 `static_output.copy_(tmp)`，代码会将结果写入固定内存区域。这样捕获的 CUDA Graph 正确、可重放且可复现，不会发生意外张量分配。

随后同步 `capture_stream`，确保预热步骤完全结束，再开始实际 graph 捕获。接着在同一 stream 上进入 `torch.cuda.graph(...)` 上下文，重新运行模型前向传播。

捕获阶段不会真正启动内核，而只是将操作记录到 CUDA Graph 对象 `g` 中。退出捕获块后还需再次同步，确保记录已经完成。

捕获 CUDA Graph 时，严格隔离至关重要。捕获 stream 上的操作不能受到其他 stream 活动的影响；即使另一个线程中看似无关的内核启动，也可能使捕获失效，并导致“operation not permitted when stream is capturing”等运行时错误。

如果捕获进行期间意外在其他 stream 上执行 CUDA 操作，就会出现该错误，graph 上下文会失效并触发报错。如果在捕获 stream 上启动会执行动态内存分配的内核，也可能发生同样问题，例如创建张量或在捕获期间调用 `torch.empty`。

> 务必在开始 graph 捕获前和退出捕获后都调用 `capture_stream.synchronize()`，确保所有操作正确记录，并使 graph 可以安全重放。前面的代码示例遵循了这一最佳实践。

捕获图后，它可在任何流（包括默认流）上重放，与用于捕获的流无关。若要触发任何依赖图完全执行结束的 CUDA 操作，必须先同步，再执行这些操作，如下所示。否则，由于图重放时异步运行，依赖图结果的 CUDA 操作可能在图完成前执行：

```
# replay the graph which writes to static_output
g.replay()

# synchronize on the stream that is replaying the graph
torch.cuda.current_stream().synchronize()

# Now it's safe to read or post-process the output
print(static_output)
...
```

没有显式同步时，程序可能继续执行，错误地假设 graph 已完成并将结果写入 `static_output`。如果没有同步，graph 可能尚未写完，代码就会读取过期或部分写入的数据，导致非确定性行为、读取损坏、隐蔽竞态条件和死锁。

重放 Graph

要使用新数据重放图，将新输入复制到预分配的输入张量 `static_input`，然后调用 `g.replay()`。GPU 会以这些张量的当前内容为输入，执行整个已捕获操作序列。图会将结果置于预分配输出张量（`static_output`）中，如下所示：

```
# load new data into pre-allocated input tensor
static_input.copy_(new_batch)

# execute the captured graph
g.replay()

# retrieve the output (clone if you plan to modify it)
result = static_output.clone()
```

这里将 `new_batch` 数据加载到 `static_input` 的内存空间，然后调用 `g.replay()`。graph 使用当前 `static_input` 数据运行完全相同的捕获操作，并将输出写入 `static_output`，之后可按需使用或克隆 `static_output`。

建议用几个测试输入验证 graph 输出是否与普通执行一致，确认捕获成功。还要记住，不同步就不能直接打印 `static_output` 或调用其 `.item()`。如需调试，应在重放后且位于所有异步区域之外执行 `result = static_output.cpu().numpy()`。

由于 graph 每次都复用相同的输入、输出和内部内存分配，它会在每次迭代覆盖同一个输出张量。如果需要将输出保留到下一次迭代之后，就必须像前面的代码那样 `clone()` 缓冲区。因此，预热/捕获阶段的分配必须覆盖所需最大大小；graph 无法在运行中处理额外内存分配。

需要注意，使用 CUDA Graph 后，某些 PyTorch 操作要等重新捕获 graph 才会反映出来。例如，如果在 graph 外动态修改模型权重，这些更新不会生效，因为 graph 保存了自己的操作副本。因此，graph 最适合每次迭代重复相同操作的稳态循环。

此外，使用 `cudaMallocAsync` 为 CUDA Graph 预分配内存时，重放会复用图捕获期间预分配的相同内存地址；例如图从磁盘加载时，会复用预热期间的地址。这样，后续图重放不需要额外内存。

默认情况下，每个 CUDA Graph 实例使用自己的私有内存池。这样可以保证每个 graph 预分配自己的内存缓冲区，不与其他 graph 实例竞争；即使两个相同 graph 并发重放，也不会争用内存。

也可以通过传入 `torch.cuda.graph(pool=...)` 让 graph 实例共享内存池。不过，这只适用于非常特殊的情况：为了性能而有意编排相关 graph，使其复用预分配缓冲区。例如，同时运行多个不同 batch 大小（如 1、2、4、8）的推理 graph 变体时，可以让形状特化变体为 PagedAttention 复用一块大型内存分配，从而降低总体内存使用；PagedAttention 使用名为 `block_tables` 的变长张量。

这种方法见于 FireworksAI 的一篇博客文章。其中针对每种 batch 大小编译不同的 CUDA Graph 变体；它们不为每个 graph 创建独立内存池，而是在所有 graph 变体之间共享一个内存池。按 batch 大小递减顺序编译 graph 时，共享池中的内存可从最大变体（本例为 8）复用。较小 batch 的 graph 变体使用前一次迭代已分配的更大缓冲区，因此可支持多个 graph 变体而不使用过多 GPU 内存。

> 这是一种较少见但巧妙的实现选择，需要仔细协调各内存段；不过，在同时运行多个形状特化 graph 时，它确实能降低总体 GPU 内存用量。对于必须尽可能减少总内存使用的部署场景，这很有价值。

CUDA Graph 最佳实践

CUDA Graph 是训练和推理达到稳态峰值吞吐量的强大工具，尤其适用于 CPU 开销和内核启动波动损害性能的大规模部署。在能极快执行数千项操作的现代 GPU 上，graph 对保持设备繁忙和降低逐内核启动开销非常重要。

需要注意，除非显式禁用，PyTorch 的 `torch.compile` 在许多情况下会在底层使用 CUDA Graph，以降低编译模型的内核启动开销。使用 CUDA Graph 时应牢记以下要点：

避免在 graph 捕获期间分配内存

> 记住，不能在 graph 捕获内部动态分配 GPU 内存。所需张量应像前面所示，在预热步骤中提前分配。如果 graph 需要临时缓冲区，应使用基于 `cudaMallocAsync` 后端、感知 graph 的 PyTorch 缓存分配器，使每次重放复用相同缓冲区地址。确保这些临时张量在捕获前的预热阶段按最大可能大小创建，提前分配所需全部内存。

保持 graph 结构固定

捕获的 graph 不能改变操作序列、张量形状或内存大小。如果工作负载偶尔改变形状，可以捕获多个 graph（例如每种预期输入大小一个），运行时选择合适的 graph。也可以在这些迭代中禁用 graph（PyTorch Compiler 提供 `max-autotune-no-cudagraphs` 模式）。

> CUDA 支持低层 Graph Management API，包括前章介绍的 `cudaGraphExecUpdate()`，允许对捕获 graph 做小幅修改。但截至本书撰写时，PyTorch 尚未暴露此 API；在 PyTorch 中，最好将 graph 视为不可变对象。

尽可能多地捕获

尽可能将训练循环纳入 graph，理想情况是完整的一次迭代（前向、反向、优化器步骤及 all-reduce 通信）。捕获越多，消除的 CPU 开销和启动延迟越多。

如果内存允许，还可以在一个 graph 中捕获多次迭代（包括循环展开等）。虽然 graph 会变大，但跨迭代启用更多优化，通常能提高吞吐量；代价是灵活性降低，仍值得探索和分析。

在多 GPU 环境捕获大型 graph 时，如得到支持，应使用 CUDA stream 优先级。例如，希望计算内核以更高优先级运行且不被延迟时，可以将 NCCL 调用放入低优先级 stream。Graph 捕获也会记录这些优先级。

> NVIDIA 的 MLPerf 提交结果（以及内部基准）经常将整个训练步骤捕获为每次迭代一个 graph，包括前向、反向、优化器和 all-reduce 通信步骤。这样几乎消除了所有运行时开销（启动抖动等），代价是内存占用增加、灵活性降低。

规划内存复用

Graph 捕获后，不能释放或重新分配 graph 使用的张量，也不能改变其大小。最好按预期最大 batch 大小捕获，预留略多于需求的容量，这样稍大的输入不会在后续重放时破坏 graph。

例如，最大 batch 大小为 64，但偶尔运行 96 时，可以按 batch 大小 96 捕获以确保安全。通常最好捕获最坏情况、运行更小 batch 并浪费少量内存，而不是冒 graph 失败的风险。

> 记得将优化器状态的大小计算在内（例如 Adam 优化器的中间缓冲区），因为它们属于模型训练 graph 的一部分。这些内容很容易被遗漏！

合理使用 CUDA Graph，可以显著加速大模型训练和推理工作负载。完成计算优化后，下面转向 PyTorch 内存优化。

CUDA Graph Trees（PyTorch Compiler 内部机制）

前面介绍过 PyTorch Compiler；在 CUDA Graph 语境下，还应了解 PyTorch 的 CUDA Graph Trees。它由 `torch.compile`（尤其是 `mode="reduce-overhead"`）使用，为每种输入形状编译并缓存独立的静态 graph。

静态 graph 记录后，其张量维度必须保持固定。任何新的输入形状都会触发重新记录并产生新的缓存条目。为最大化缓存命中并减少 graph 捕获，建议让各次迭代的输入形状尽可能一致；形状种类越少，记录的 graph 复用越多，开销越低。

需要注意，通常不需要直接调用 CUDA Graph Trees API。指定 `mode="reduce-overhead"` 时，`torch.compile` 会为你处理这些工作。

由于 CUDA Graph 要求静态地址和稳定控制流，支持可变输入大小、batch 大小和步数的 LLM 推理很难进行完整 graph 捕获（例如采样、KV cache 增长和主机端决策等）。

对于模型训练，CUDA Graph Trees 允许多个捕获的子 graph 在前向和反向捕获之间共享一个内存池。它对推理也很有用，因为可以根据输入形状和 batch 大小动态选择子 graph，这通常称为“分段捕获（piecewise capture）”。PyTorch 利用 CUDA Graph Trees 维护各形状的分段捕获并管理共享池。

> vLLM 推理引擎使用 CUDA Graph Trees 提供的分段捕获模式，为不同输入形状和 batch 大小组合支持不同 graph。第 15 章将更详细地介绍 vLLM 和推理引擎优化。

PyTorch 内存分析与调优

大模型可能受到 GPU 内存容量和内存带宽限制。此外，即使 HBM 容量充足，内存碎片等低效使用方式也会损害性能。可以从多个方面处理内存问题，包括内存分配器调优、激活检查点、内存卸载和输入流水线优化。

PyTorch 还在 `torch.profiler` 中内置了内存分析器，只需启用 `profile_memory=True`，如前文所示。可以用它找出分配大量内存的操作并优先处理，这一点在 PyTorch内存可视化工具生成的图 13-5中有所展示。

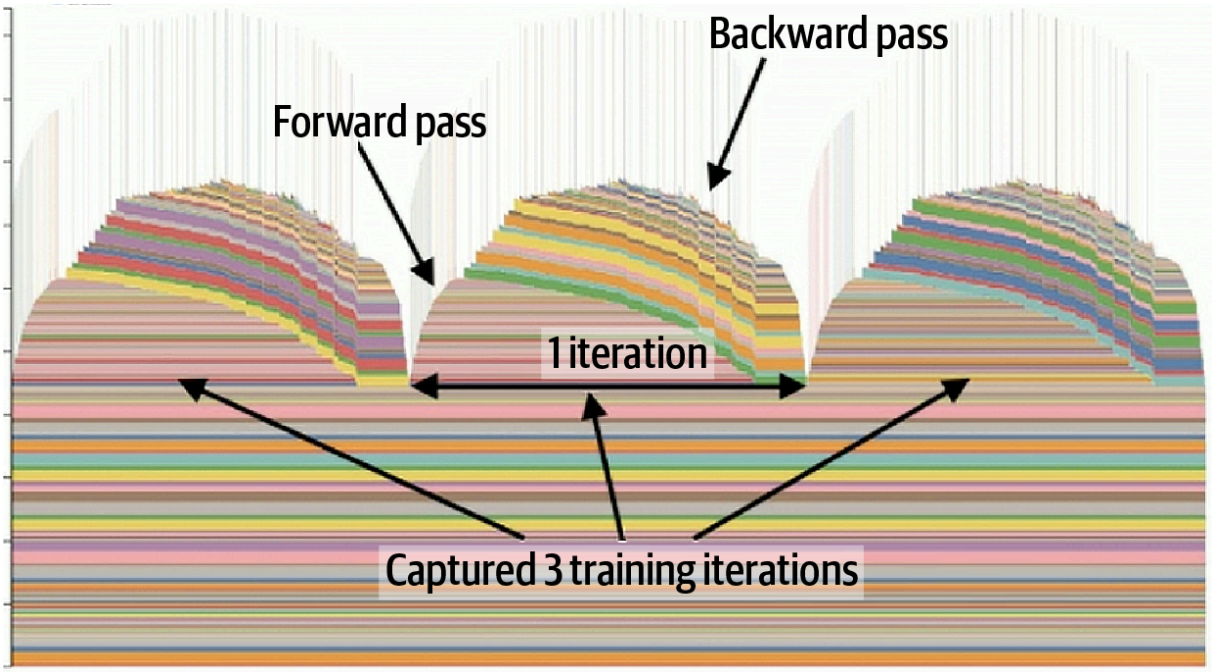


图 13-5　三次前向和反向迭代的 PyTorch 内存分析可视化

NVIDIA Nsight Systems 的 CUDA Memory Inspector也能帮助可视化内存碎片随时间产生的过程。利用这些工具可以指导内存分配器调优，下面将进行探讨。

调优 CUDA 内存分配器

PyTorch 使用 CUDA 缓存内存分配器。默认情况下，它会根据分配模式按需拆分和回收 GPU 内存块。但使用可变大小内存分配的某些工作负载模式可能导致内存碎片。

随着时间推移，GPU 内存被拆成许多不连续的空闲块时，就会发生内存碎片。即使剩余总空闲内存足够，也可能难以分配大型张量。MoE 模型尤其容易遇到此问题，因为每个 batch 路由给各专家的 token 数量可能变化，导致每个专家的输出激活张量在每次迭代中大小不同。

可变大小的内存分配会留下不均匀、碎片化的内存块，这些碎片会在训练或推理运行中累积。

为避免这种情况，应提前分配固定大小的专家输出缓冲区，并按 batch 中任一专家可能处理的最大 token 数量确定大小，然后在每次迭代复用该缓冲区。

保持缓冲区维度不变，可以避免 GPU 内存分配器随时间产生碎片。每个专家写入预分配缓冲区中的专属切片，而不是触发新分配。该方法能稳定内存使用、提高复用效率，并避免碎片导致的失败或变慢。

可以使用环境变量调整 PyTorch 分配器配置，从而调节其行为。示例如下：

```
export PYTORCH_ALLOC_CONF=\
max_split_size_mb:256,\
roundup_power2_divisions:[256:1,512:2,1024:4,>:8],\
backend:cudaMallocAsync
```

下面说明代码中指定的各个配置参数：

`max_split_size_mb:256`

> 该参数指示分配器保持大型空闲块完整（最大 256 MB），而不是不断将其拆成小块，有助于减少碎片。默认情况下，PyTorch 不会激进地拆分大型分配；显式设置 `max_split_size_mb` 可以为现代 LLM 的大型神经网络层保留大型连续空闲块。

`roundup_power2_divisions:[N:M,...]`

> 该参数控制 PyTorch CUDA 缓存分配器如何将张量大小请求分组到固定 bucket 中。它将每个“2 的幂次”范围划分为 *N* 个等大的子 bucket，例如请求落在 512 MB 到 1,024 MB 之间时，代码中的 `512:2` 会将该范围划分为 `:2` 个 bucket，并向上取整到 `[512MB, 768 MB, 1 GB]` 之一。比如在 512 到 1,024 MB 范围内，`'512:2'` 会将 600 MB 请求向上取整为 768 MB。应检查分配器日志，确认环境中的实际 bucket 划分。该策略可以减少内存碎片、统一分配大小并提高缓存复用率，因为相似请求会命中同一 bucket。

`backend:cudaMallocAsync`

> 指定此项会启用 NVIDIA CUDA 异步分配器作为底层内存分配机制，有助于避免内存释放事件上的同步，并改善多 worker 数据加载等多线程场景的性能。

自定义内存分配器配置可以保持更稳定、更可预测的内存使用模式。长时间运行时，可以使用 `torch.cuda.memory_stats()` 监控内存碎片，确保内存占用保持稳定而不会不断膨胀。

还可以在运行时使用 `torch.cuda.mem_get_info()` 获取空闲内存与总内存。如果分配张量数量保持不变而空闲内存下降，就说明碎片正在增加；因此该方法可以间接跟踪碎片。

使用激活检查点节省内存

对于极大模型，激活检查点（一些实践者也称为梯度检查点）是管理内存的必要技术。对于大型 LLM，如果保存反向传播所需的全部中间激活，可能会耗尽内存。

使用激活检查点时，不在前向传播期间保存供反向传播使用的中间激活，而是在反向传播需要时动态重新计算。PyTorch 提供 `torch.utils.checkpoint` 自动完成这一过程，只需包装模型层或层序列，其 `forward` 激活就不会被保存。

可以在模型中以每个 Transformer block 和每个专家前馈神经网络（FFN）层为粒度应用检查点。这样计算完每个 block 的前向输出后，无需将中间激活留在内存中；反向传播时，PyTorch 会重新运行该 block 的前向传播，为梯度计算再生激活。

不必为所有内容设置检查点，可以只关注最大层。常见策略是仅对保存大量激活的 Transformer block 设置检查点，而不处理 layer norm 和 embedding 等较小层，以最小的重计算开销获得最大的内存节省。

使用 FSDP 时，还可以启用自动检查点，它会递归地为多个层应用检查点。

这种方法以增加计算换取减少内存使用。幸运的是，相对于 HBM 容量，现代 GPU 通常拥有充足 FLOPS，因此该技术很适合最新一代硬件；GPU 的额外计算余量可以承担这些重计算。

这种权衡通常是值得的。没有激活检查点时，可能需要减小输入 batch 大小或专家数量，才能适应有限的 GPU 内存；使用检查点后，则可以更轻松地将模型装入内存并保留较大的 batch。

激活检查点会使训练略微变慢，但能训练更大的模型变体并使用更大的输入 batch，否则它们可能无法装入内存。本质上，这是用 GPU 充足的计算 FLOPS 换取克服 HBM 容量限制。

将参数卸载到 CPU 和 NVMe

除了检查点，还可以将无需持续存储在 GPU 上的部分模型参数卸载到 CPU 和 NVMe。例如，MoE 模型中访问频率较低的专家层可以卸载到 CPU 内存，仅在需要时传输到 GPU。

应将传输与计算重叠：一个层在 GPU 上运行时，异步从 CPU 或 SSD 预取下一层权重。实践中，DeepSpeed 的 ZeRO-Infinity（训练）和 ZeRO-Inference（推理）等框架可以自动完成预取。它们逐层将模型权重从 CPU 或 NVMe 流式传输到 GPU，在重叠数据传输与计算的同时最小化 GPU 峰值内存使用。

可以将这些组件固定在主机内存中，并使用 `.to(device)`、`cudaMemcpyAsync` 等异步非阻塞 DMA 调用，在其他计算运行时将其传输到 GPU，从而隐藏 CPU 拷贝带来的额外传输延迟。

NVIDIA Unified Memory 也是一种选择，尤其适用于 Grace Blackwell GB200/GB300 等 CPU 与 GPU 之间通过 NVLink-C2C 高速互连的 Superchip 系统。在这些系统中，Unified Memory 允许将很少使用的 GPU 内存页驱逐到 CPU 系统内存。

如果容量需要，操作系统甚至可能将其交换到 NVMe/SSD。NVMe 应通过普通操作系统交换作为最后手段，而不是 Unified Memory 的主要目标。

不过，Unified Memory 可能因内存分页等机制引入不可预测性。因此，为保持完全控制和可预测性能，通常更适合显式管理卸载。

GPUDirect Storage 等近期技术允许 GPU 直接读取 NVMe 驱动器。这意味着在某些情况下，模型参数可以在运行时直接从 NVMe 分页到 GPU，无需 CPU 参与。使用超大模型进行训练和推理服务时，这两者都很有用。

对于参数量达到万亿规模的更大模型，可以将组件卸载到 NVMe 存储，并按需及时交换到 GPU 内存。关键是将数据传输与计算重叠，避免阻塞训练循环。

SuperOffload：面向 CPU-GPU 超级芯片优化的卸载

SuperOffload 是一种专为发挥 CPU-GPU 超级芯片硬件效率而设计的卸载系统。超级芯片（如 Grace Hopper、Grace Blackwell、Vera Rubin 等）在 CPU 与 GPU 之间提供高带宽 NVLink-C2C 互连。结合共享的一致性内存地址空间，针对超级芯片优化的卸载策略与传统卸载技术相比，可显著提升速度和硬件利用率。

SuperOffload 展示了几项关键创新，包括先推测后验证（speculation-then-validation，STV）、异构优化器计算、感知超级芯片特性的数据类型转换和内存复制。下面逐一讨论。

传统卸载会等待梯度归约和全局检查完成后再更新参数；相比之下，STV 会在 GPU 执行反向传播的同时，在 CPU 上推测性地执行优化器更新，随后再验证结果，从而让这些步骤相互重叠。这可以有效减少同步停顿并提高 GPU 的总体利用率。

SuperOffload 通过异构优化器计算，在 CPU 与 GPU 之间划分优化器工作。例如，它把计算密集的张量更新交给 GPU，而由 CPU 核心处理较轻量的状态更新，如 Adam 优化器使用的动量缓冲区。这能让两类设备都保持忙碌，减少空闲周期，提高整个芯片的利用率。

由于 NVLink-C2C 在 CPU 与 GPU 之间提供高带宽，SuperOffload 可以调整张量类型转换与数据传输所采用的精度和执行位置。它将类型转换和复制更多地移到 GPU 上，借助高速的一致性互连来尽量降低 CPU-GPU 传输延迟。

此外，SuperOffload 使用了针对 CPU 优化的 Adam 变体 GraceAdam；它专为 Grace 的 ARM 可伸缩向量扩展（Scalable Vector Extension，SVE）架构设计。SVE 是 ARM A64 指令集采用的长向量架构，具体提供 32 个向量寄存器和 16 个谓词寄存器。与 SuperOffload 结合后，它们有助于提高卸载参数更新的吞吐量和能效。

FSDP 自动检查点与卸载

PyTorch 的 FSDP 是一种分布式并行策略；在模型训练期间，它把模型参数、梯度和激活分片到多个 GPU 上。这可降低训练时的内存开销，使你能够训练大于不分片时容量上限的模型。从技术上说，FSDP 实现了 ZeRO Stage-3 策略，将模型状态分片到多个 GPU 上，如图 13-6所示。

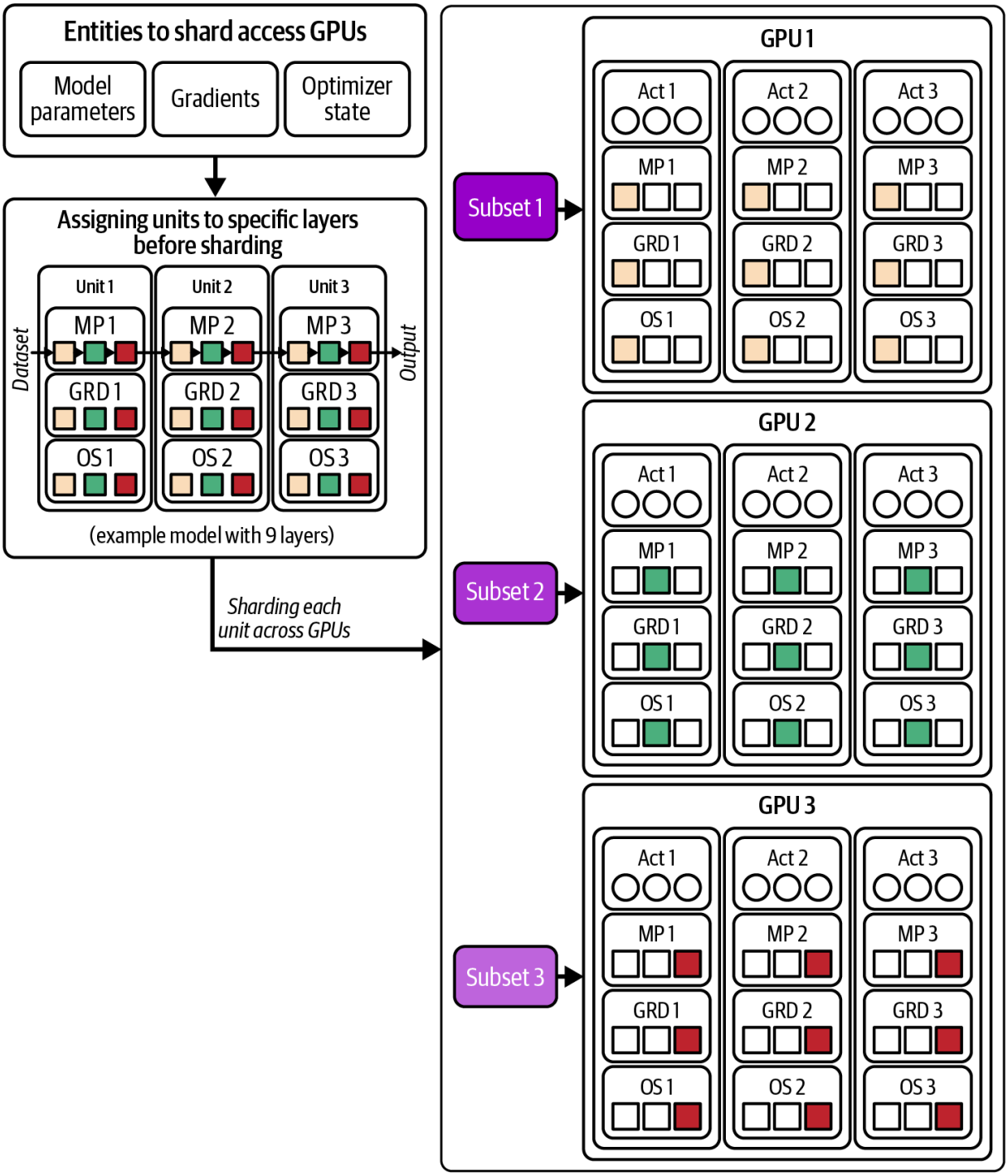


图 13-6　FSDP 将模型参数、梯度和优化器状态分片到多个 GPU 上（ZeRO Stage-3）

FSDP 可以在内部自动应用激活检查点并卸载参数与梯度。只需使用 `FSDP()` 封装模型，再指定 `activation_checkpointing_policy` 和 `CPUOffload` 参数，如下所示：

```
import torch
import torch.nn as nn
import torch.distributed as dist
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP,
    CPUOffload, ShardingStrategy, BackwardPrefetch, MixedPrecision
)
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
```

```
# Initialize distributed
dist.init_process_group("nccl")
torch.cuda.set_device(dist.get_rank() % torch.cuda.device_count())

# Build your model
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(4096, 4096),
            nn.ReLU(),
            nn.Linear(4096, 4096),
        )

    def forward(self, x):
        return self.layers(x)

model = MyModel().cuda()

# Auto-wrap transformer blocks if needed
auto_wrap_policy = transformer_auto_wrap_policy(
    model,
    min_num_params=1e8,
)

# Wrap with FSDP + checkpointing + CPU offload
fsdp_model = FSDP(
    model,
    auto_wrap_policy=auto_wrap_policy,
    sharding_strategy=ShardingStrategy.FULL_SHARD,
    use_orig_params=True,
    cpu_offload=CPUOffload(offload_params=True, pin_memory=True),
    mixed_precision=MixedPrecision(
        param_dtype=torch.bfloat16,
        reduce_dtype=torch.bfloat16,
        buffer_dtype=torch.bfloat16
    ),
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
    activation_checkpointing_policy={
        nn.TransformerEncoderLayer,
        nn.TransformerDecoderLayer,
        nn.MultiheadAttention
    }
)

# Setup optimizer
optimizer = torch.optim.AdamW(fsdp_model.parameters(), weight_decay=0.01)

...
```

这里使用 `transformer_auto_wrap_policy`，使 FSDP 能根据 Transformer 块的结构自动对参数、梯度和优化器状态进行分片。同时通过 `CPUOffload(offload_params=True)` 启用 CPU 卸载。当 GPU 不需要参数和梯度时，FSDP 会透明地将二者移到 CPU，从而降低 GPU 峰值内存用量。

> 设置 `use_orig_params=True` 后，每个 FSDP 单元无需将参数扁平化便可对其进行处理。这有利于实现更好的重叠，并简化状态字典处理，从而改善内存管理和优化器兼容性。

设置 `activation_checkpointing_policy` 会指示 FSDP 在指定的核心 Transformer 子模块中重新计算（即重新物化）激活，以额外计算换取显著降低的峰值内存。它可以取得与手动添加 `checkpoint()` 封装器及自定义卸载脚本相近的大幅内存节省效果，同时无需额外的样板代码。FSDP 甚至可以处理各 GPU 批量大小不一致的情况，这对 MoE 工作负载很有用。

FSDP 还支持通过 `ShardingStrategy.HYBRID_SHARD` 使用混合分片策略。该策略将每个节点的参数、梯度和优化器状态分片到节点内的多个 GPU 上，同时在其他节点复制同样的分片。实质上，混合分片通常能提供高于 `FULL_SHARD` 的吞吐量，但代价是每个节点占用更多内存。

当互连性能尚可、但并非极快时，可以使用 `HYBRID_SHARD`。它在每个节点内部对参数、梯度和优化器状态进行分片，并跨节点复制这些分片。与完全分片相比，这会略微增加每节点内存用量，但可减少跨节点流量。

混合分片使你能够权衡内存与通信。例如，由于每个节点都持有一套完整的本地分片，它会比 `FULL_SHARD`（ZeRO Stage-3）使用更多的每节点内存，但能减少节点间通信，往往可获得更高的端到端吞吐量。如果多节点网络非常快，并且希望把每个 GPU 的内存占用降至最低，则更适合使用 `FULL_SHARD`。

当网络较慢或只有少量 GPU 时，可以使用 `ShardingStrategy.SHARD_GRAD_OP`（ZeRO Stage-2），仅在所有 GPU 间对梯度和优化器状态进行分片。此策略会在每个 GPU 上保留一份完整的参数副本。

结合使用 FSDP、张量并行与流水线并行

如果模型大到单个层无法装入一块 GPU 的内存，就需要将 FSDP 与张量并行（TP）等其他并行策略结合，把这个大型模型层分布到多个 GPU 上。

也可以跨 GPU 和计算节点使用 FSDP：在节点内通过 TP 将巨型层拆分到多个 GPU 上，再用流水线并行（PP）跨节点串联这些经 TP 拆分的层。FSDP 支持这类灵活组合。

简而言之，FSDP 能以少得多的编码工作降低内存用量。应用检查点和卸载后，可以让模型装入内存并采用更大的批量，从而有助于提高 GPU 利用率。

可插拔内存分配器与跨 GPU 数据传输

可以配置 PyTorch，为 NCCL 梯度全归约等关键 GPU 通信操作使用专用内存分配器。通过 `torch.cuda.MemPool` 接入自定义分配器，可以让 NCCL 以能够利用 NVLink 复制引擎、InfiniBand 卸载或 NVIDIA SHARP 等专用硬件机制的方式分配缓冲区，从而改善通信与计算的重叠。例如，PyTorch 支持在 NVSwitch 配置上使用 NCCL 的内存分配器执行快速的零复制归约，如下所示：

```
import torch
import torch.distributed as dist
from torch.cuda.memory import MemPool
from torch.distributed.distributed_c10d import _get_default_group

# Initialize NCCL distributed backend
dist.init_process_group(backend="nccl")
```

```
torch.cuda.set_device(
  dist.get_rank() % torch.cuda.device_count()
)

# Get the NCCL backend object for this device
default_pg = _get_default_group()
backend = default_pg._get_backend(
  torch.device(f"cuda:{torch.cuda.current_device()}")
)

# The backend exposes ncclMemAlloc using mem_allocator
nccl_allocator = backend.mem_allocator

# Create a dedicated memory pool using this allocator
nccl_pool = MemPool(nccl_allocator)

# Register the pool so NCCL uses it for collective gradient buffers
backend.register_mem_pool(nccl_pool)

# Use the pool explicitly for NCCL operations
with torch.cuda.use_mem_pool(nccl_pool):
    tensor = torch.randn(10_000_000, device="cuda")
    dist.all_reduce(tensor)
```

将 NCCL 的原生分配器（`backend.mem_allocator`）绑定到 `torch.cuda.memory.MemPool`，并使用 `backend.register_mem_pool(...)` 后，PyTorch 会把梯度归约缓冲区直接放在针对 NVLink、InfiniBand 和支持 NVIDIA SHARP 的硬件路由而优化的内存区域中。这样，全归约操作便可利用硬件加速，在大规模数据归约期间减少对 SM 的争用，进而提高大型多 GPU 工作负载的吞吐量。

> 使用第 4 章介绍的 SHARP，需要兼容的网络结构（例如支持 SHARP 的 HDR InfiniBand）。如果条件具备，为 NCCL 启用基于 SHARP 的网内聚合，可以大幅降低大型集群的全归约延迟；扩展到大量节点时值得考虑。

通过 `backend.mem_allocator` 使用 NCCL 原生分配器，并将其封装进 PyTorch 的 `MemPool` 后，梯度全归约缓冲区会分配在针对 GPUDirect RDMA 和网内 SHARP 卸载优化的内存区域中。这会使缓冲区按大页边界对齐，并可能帮助把数据传输交给专用 DMA 引擎，从而减少 SM 的参与，释放计算能力来完成更有用的计算工作。

因此，张量并行归约等 NCCL 集合操作既能受益于硬件加速，也能减少 SM 争用。这会显著提高多 GPU 同步吞吐量，并降低重叠执行的计算内核和通信内核对 SM 与复制引擎的争用，对张量并行工作负载尤其有益。

当工作负载逼近带宽上限时，这类优化会变得更加重要。例如，Blackwell 的 NVLink 5 理论上可为每块 GPU 提供最高 1.8 TB/s 的双向总带宽（每个方向约 900 GB/s）。使用专用复制引擎、SHARP 等网络硬件以及优化的内存分配器，有助于接近网络峰值吞吐量，同时释放 SM 以争取更高的计算吞吐量。

借助 `torch.cuda.MemPool`，还可以通过注册自定义库（共享对象，即 `.so` 文件）来创建自己的内存分配器。此外，同一个 PyTorch 应用中可以混用不同的 CUDA 内存分配器，如下所示：

```python
import torch  # PyTorch main namespace
import os     # for path operations (used here for .so extensions)
from torch.cuda.memory import CUDAPluggableAllocator

# 1. Create a CUDAPluggableAllocator and MemPool

# Build a pluggable allocator that calls into your NCCL library:
#   - allocator_path: path to your .so
#   - "ncclMemAlloc": symbol for allocation
#   - "ncclMemFree":  symbol for deallocation
# .allocator() returns a callable that matches the CUB/CUDA allocator API
allocator = CUDAPluggableAllocator(
    "./nccl_allocator.so",
    "ncclMemAlloc",
    "ncclMemFree"
).allocator()

# Wrap that allocator in a MemPool for efficient sub-allocations
pool = torch.cuda.memory.MemPool(allocator)

# 2. Start recording events (set a high cap for long runs)
torch.cuda.memory._record_memory_history(max_entries=100000)

# 3. Allocate tensors with different allocators

# tensor0 uses the *default* cudaMalloc allocator
# - Shape: (1024, 1024), you can change to your desired size
tensor0 = torch.randn(1024, 1024, device="cuda")

# tensor1 uses *your* NCCL-backed allocator using MemPool
with torch.cuda.use_mem_pool(pool):
    # Inside this context, all cuda allocations go through `pool`
    tensor1 = torch.randn(1024, 1024, device="cuda")

# Exiting the context restores the default allocator
# tensor2 again uses the *default* cudaMalloc allocator
tensor2 = torch.randn(1024, 1024, device="cuda")

# 4. Inspect memory pool stats

# Pool-specific snapshot with list of segments/blocks in use
pool_state = pool.snapshot()
print(f"Pool segments count: {len(pool_state)}")

# 5. Dump the snapshot and optionally load in the PyTorch
# memory viewer tool (https://oreil.ly/tX6gA)
torch.cuda.memory._dump_snapshot('memory_snapshot.pkl')

# Global allocator stats (allocated/reserved, peak, counts)
global_stats = torch.cuda.memory_stats()
print("Peak allocated bytes:", global_stats["allocated_bytes.all.peak"])

# 6. Stop recording
torch.cuda.memory._record_memory_history(enabled=None)

# 7. Reset peak counters for a fresh measurement
torch.cuda.reset_peak_memory_stats()
```

这里，`CUDAPluggableAllocator` 会加载自定义 `.so` 文件，并绑定到指定的两个符号。`MemPool` 在底层 CUDA 内存分配器之上提供缓存，以提高内存分配和释放的性能。

`torch.cuda.use_mem_pool(pool)` 会在 Python 上下文管理器（如 `with`）代码块内，将此内存池用于随后发生的所有内存分配；退出上下文管理器代码块后，则恢复先前的分配器。

启用点对点 DMA 与 UCX

在多 GPU 系统中使用流水线并行时，通常希望通过尽可能快的点对点（P2P）连接，把激活直接从一块 GPU 移到下一块 GPU，避免数据绕经主机及占用 CPU 资源。当同一进程中的张量跨设备移动时，PyTorch 会自动探测并启用对等访问。若要手动确认，可以使用 `torch.cuda.can_device_access_peer(i, j)` 检查 `GPU i` 与 `GPU j` 之间是否支持 P2P 访问。对于自定义 C++ 操作，可以通过 CUDA 驱动 API 显式启用对等访问。

启用 P2P DMA 后，可以进行高效的直接传输，无需经由 CPU 内存；数据搬运通常由复制引擎完成，也不要求 SM 执行常规复制内核。启用后，跨 GPU 的 `copy_()` 和 `.to()` 可在拓扑与运行时支持时使用更快的对等内存路径，而无需额外的应用代码，如下所示：

```
dst.copy_(src, non_blocking=True)
# or
src.to(device="cuda:1", non_blocking=True)
```

当系统拓扑启用了 P2P 访问时，这些方法可使用相应的 CUDA 对等复制路径（例如 `cudaMemcpyPeerAsync`）。要让 P2P 传输正常工作，硬件拓扑必须提供支持。GB200/GB300 NVL72 机架内部使用支持 P2P 的 NVLink 和 NVSwitch，因此其机架内硬件已具备 P2P DMA 条件。

到目前为止，我们在 P2P 数据传输的语境中只讨论了多 GPU 配置。而在多节点 GPU 拓扑中，通常需要使用通过网络通信的 NCCL，底层有时会采用 GPUDirect RDMA。根据网络、插件和软件栈配置，将 UCX（Unified Communication X，统一通信 X）与 NCCL 配合使用可能改善性能。在 GB200 和 GB300 NVL72 拓扑中，NVLink 与 NVSwitch 提供机架内 P2P 路径；跨机架通信则需要通过网络结构和相应的 NCCL 网络传输插件。

若要通过 UCX 路由跨节点传输，需要安装相应的 NCCL-UCX 插件，并按照该插件及部署环境的要求设置 `NCCL_PLUGIN_P2P=ucx`。许多环境还需要设置 `NCCL_NET=UCX`，并通过 `UCX_TLS` 为实际网络结构选择合适的传输层。配置正确时，这可以利用硬件卸载，并改善 NVL72 机架之间 InfiniBand 网络上的流水化传输。

下面给出一份 NCCL-UCX 插件的示例配置：

```
export NCCL_NET=UCX
export NCCL_PLUGIN_P2P=ucx
# UCX transports vary by fabric.
# These are safe defaults to start with:
export UCX_TLS=rc,self,gdr_copy,cuda_copy
```

如果应用需要扩展到多个节点或机架，例如训练 TB 级模型、为多个租户运行大规模 LLM 推理，或管理数据密集型流水线，那么 UCX 是一种重要的通信方案。NCCL 与合适的 UCX 网络插件和传输配置结合后，可提供支持 RDMA 硬件卸载与拓扑感知的高吞吐、低延迟跨节点通信。它是许多生产级 AI 基础设施的重要组成部分，但是否采用仍取决于网络硬件、NCCL 插件支持及现有软件栈。

简而言之，在拓扑、传输后端和实现允许的情况下，直接 P2P DMA 或采用 UCX 的网络传输可以减少中转开销，并改善通信与计算的重叠；但相对于 NCCL `send/recv` 的实际收益并非固定，仍需通过基准测试验证。这属于较底层的优化；如果系统具有合适的拓扑和 NVLink/NVSwitch 等专用高速互连，可能显著改善系统性能。

PyTorch 对称内存

对称内存是一种跨 GPU 暴露分区全局地址空间的编程模型，使内核能够执行单边 put 和 get 操作。借助它，可以调用低延迟、跨 GPU 的直接访问集合操作，而无需 CPU 参与逐次握手。实质上，对称内存会分配一组可由通信组内各 GPU 直接寻址的缓冲区，应用无需显式发起逐项 P2P 复制。

使用对称内存时，可以完全在设备端执行 MoE token 重排等全互连操作。如果所用内核和通信后端均支持 CUDA Graph 捕获，整个全互连就可纳入 CUDA Graph。若采用需要主机协调或同步的全互连实现，时间线上可能出现设备到主机（D2H）间隙，并中断或阻碍 CUDA Graph 捕获；这并不是所有非对称内存集合通信实现都必然具有的行为。

可以分配对称张量，通过进程组执行会合以取得远程访问句柄，然后调用直接访问集合操作（如 `all_to_all_v` 和 `one_shot_all_reduce`）。此外，还可以启动利用远程访问句柄执行单边读写的内核。与 OpenAI Triton 和 NVSHMEM（`triton.nvshmem.put()/get()`）结合后，对称内存可成为自定义内核内数据传输的有力通信机制。

> 对于 MoE 全互连 token 重排这类细粒度、延迟敏感且希望避免主机 CPU 介入的设备端交换，可以考虑使用 PyTorch 对称内存。在后端与操作均兼容的情况下，它有助于消除由主机协调造成的设备到主机（D2H）时间线间隙，并改善 CUDA Graph 捕获。

优化数据输入流水线

讨论完模型的内存与计算后，下面转向数据输入流水线。GPU 空闲等待数据是效率低下的常见原因。PyTorch DataLoader 支持启动多个工作进程（以及由数据处理库使用的线程），在模型训练的同时并行加载和预处理数据，例如文本分词。对于经由离散 CPU-GPU 路径的主机到设备传输，通常应设置 `pin_memory=True`，让 DataLoader 返回位于页锁定（pinned）主机内存中的张量，以支持更高效的传输及异步复制。

如果不使用锁页内存，某些工作负载可能出现数据加载进程 CPU 利用率较高、而训练进程 GPU 利用率较低的现象。不过，CPU 忙碌也可能来自解码、分词或变换，不能仅凭这一现象断定锁页内存是根因。可以使用 `htop` 和 Nsight Systems 等工具观察：CPU 线程忙于加载或预处理数据时，GPU 是否存在相应的空闲间隙。

如果每次迭代前都有 GPU 等待数据的间隙，说明数据加载可能跟不上训练速度。可以逐步提高 DataLoader 的 `num_workers`，直到数据队列能及时准备足够的批次。对于受 CPU 限制的数据流水线，每块 GPU 配置 4 个 worker 是一种常见起点，但最佳数量取决于工作负载、CPU 核数、存储、NUMA 拓扑及每节点 GPU 数量。

要找到最佳数量，可以快速扫描一组候选值（如 4、8、16、32），找出继续增加 worker 已无收益的拐点。同时观察 CPU、存储带宽和内存压力；如果所有核心都已满载，增加 worker 通常不会有帮助，反而可能增加上下文切换和内存开销。

请记住，Grace Hopper、Grace Blackwell 和 Vera Rubin 等采用 NVLink-C2C 的超级芯片系统提供大容量、高带宽的 CPU-GPU 一致性内存地址空间。在这些环境中，数据访问路径及迁移机制不同于传统 PCIe 离散系统，因此 `pin_memory=True` 的收益可能较小，但不能仅依据互连类型预先断定。

在非一致性路径上进行可分页主机内存到设备的复制时，为了获得预期的异步传输与计算重叠，仍可能需要锁页内存。统一内存和一致性映射可能降低 NVLink-C2C 系统对锁页内存的依赖，但必须针对实际工作负载测量。

即使采用 CPU-GPU 超级芯片架构，在数据加载受 CPU 限制时，也可评估组合使用 `pin_memory=True`、`non_blocking=True` 和 `persistent_workers=True`。其中，`persistent_workers=True` 可避免每个 epoch 结束后关闭并在下一 epoch 重新创建 worker 进程，对分词密集的训练工作负载尤其有用。注意，大页不能普遍替代 CUDA 锁页内存，而 `non_blocking=True` 也不单独保证复制与计算重叠；应使用 Nsight Systems 验证实际执行时间线。

在具备 CPU-GPU 硬件一致性内存的系统中，处理器可通过高速一致性路径访问共享地址空间，不一定需要传统离散 GPU 路径中的显式固定与暂存。相比之下，在缺少这类硬件一致性的标准 CPU+GPU 系统上，启用 `pin_memory=True` 会让 DataLoader 使用页锁定主机内存，通常能改善主机到设备的传输效率，但具体收益仍取决于批次大小和流水化程度。

> 如果在 Grace Blackwell、Vera Rubin 等超级芯片系统上不使用锁页内存，而依赖 CPU-GPU 一致性或统一内存路径，应通过 Nsight Systems 验证数据访问或迁移是否确实与计算重叠。大页可能改善地址转换开销，但并非锁页内存的等价替代品；NVLink-C2C 改变了两者的权衡，需要实测。

提高数据加载响应能力的另一种方法，是增大 DataLoader 的 `prefetch_factor`。它控制每个 worker 预先加载的批次数，因此待预取的批次总数约为 `prefetch_factor * num_workers`。这样，当模型处理当前批次时，worker 已在加载后续批次。

预取有助于保持数据流水线充足并减少 GPU 空闲时间。应根据数据集和硬件能力联合调整 `prefetch_factor` 与 `num_workers`，并通过性能分析避免过度预取造成不必要的主机内存占用。

还可以预先完成数据集分词并将结果存储到磁盘，避免在数据加载循环中反复执行繁重处理。Hugging Face Datasets 的缓存机制和 WebDataset 等工具可让文件只预处理一次并重复使用。也可根据数据语义和精度要求，考虑使用较低精度的存储格式或压缩来降低 I/O 带宽需求。

PyTorch 生态中还有 TorchData 与 DataPipes，可组合构建数据处理流水线。其能否与训练计算重叠，取决于 DataLoader、worker、预取与执行方式，并不是由调度器自动保证。

另一个实用工具是 NVIDIA 数据加载库（DALI）。DALI 可以让 CPU 或 GPU 预处理与训练流水化执行，尤其适合图像和视频数据的解码、增强等操作，并能通过 CUDA 流水线把数据送入训练代码。对于适合卸载到 GPU 的即时数据变换，它很有用。

所有这些方法的目标，都是尽量让数据加载与 GPU 计算重叠。可以通过 Nsight Systems 确认数据加载器是否与 GPU 并行工作：时间线应显示一个或多个 CPU worker 在准备后续批次，同时 GPU stream 执行训练步骤；还应检查 GPU 是否存在等待数据的间隙。理想状态是数据输入不再成为瓶颈，但 SM 是否能接近满负荷执行有用工作，还取决于模型内核、同步、通信与批量大小等因素。

如果已经通过激活检查点和卸载缓解了内存约束，可以尝试增大每块 GPU 的输入批量。较大的批量可能提高部分算子的并行度和 GPU 利用率，并因每个 epoch 的训练步数减少而降低某些固定通信开销的相对占比；不过，算术强度是否提高取决于算子实现和数据复用，并非批量增大后必然发生。

不过，批量过大可能改变优化动态，并在超参数未相应调整时影响收敛或泛化；不能把原因简单归结为优化器必然进入尖锐极小值。显著增大批量时，还要监控 GPU 内存用量。请记住，梯度累积会增大有效批量；若各并行维度每步样本数相同，其通常还应计入数据并行规模，而不仅是 `batch_size * accumulation_steps`。PyTorch 的 TorchEval 指标可用于评估更大批量是否损害验证集表现。

可以相应调整超参数，尽量缓解大批量带来的训练不稳定。例如调整学习率、结合预热使用线性学习率缩放，或评估 LAMB 等大批量优化器。这些方法并非普适规则，仍需根据模型和训练配方验证。

简而言之，如果内存允许，并且前述内存优化已经实施和验证，可以尝试增大批量以改善 GPU 利用率；最终收益和收敛质量必须通过性能与训练指标共同验证。

> 优化系统的过程中，应定期重新检查并调优批量大小、学习率等超参数。改变批量或系统执行方式后，最初选择的取值可能不再合适。

使用 PyTorch Distributed 扩展

在多个 GPU 和计算节点上扩展 PyTorch 并进行性能分析，通常会使用 PyTorch DDP、FSDP 等分布式库。PyTorch 编译器可以与这些并行方法配合使用，但其中有一些细节，下面逐一说明。

结合 DDP 与 torch.compile

`DistributedDataParallel` 使用全归约集合操作在 GPU 间同步梯度。实际运行中，DDP 将梯度划分为多个 bucket，并在反向计算就绪时调度相应通信，以争取通信与计算重叠。`torch.compile` 对 DDP 的处理会依据 PyTorch 版本、优化后端和图捕获支持，在合适的边界组织反向计算与归约，而不应假定每个同步点都必然表现为用户可见的 graph break。

在某些版本的 `torch._dynamo.explain(model, ...)` 输出中，可能会看到与全归约或 `torch.distributed` 操作有关的 graph break；另一些受支持的集合操作则可能被跟踪进图中。关键是确认编译没有破坏 DDP 的 bucket 调度和通信重叠，应以实际 profiler 轨迹为准。

将 DDP 与编译器结合使用时，应合理设置 DDP bucket 大小。`bucket_cap_mb` 的常见默认值为 25 MiB，但具体默认值和首个 bucket 行为应以所用版本为准。如果覆盖默认值，把所有梯度放进一个巨型 bucket，通常要等更多反向计算完成后才能启动大规模全归约，从而减少交错通信与计算的机会；它并不必然对应“一个巨型编译图”。

为了减少 bucket 数量或尝试扩大编译区域，使用单一 bucket 可能很有吸引力，但这往往会牺牲通信重叠机会。应对系统进行性能分析，为具体工作负载和硬件环境寻找编译、通信粒度与开销之间的平衡。

> 将 DDP 与 `torch.compile` 结合时，某些通信点出现 graph break 可能是正常现象，但网络通信并非必须依赖 graph break 才能执行。应结合所用 PyTorch 版本检查 TorchDynamo 的 `explain()` 输出，并用 profiler 验证 bucket 归约是否按预期与反向计算重叠。

结合 FSDP 与 torch.compile

PyTorch 的 FSDP 并行策略会把模型参数、梯度和优化器状态分片到多个 GPU 上。以完全分片策略为例，它在前向传播期间按需执行全收集，在反向传播期间执行归约散播；与复制完整参数的 DDP 相比，这能让更大的模型装入 GPU 内存。

FSDP 可以受益于 `torch.compile`，但要获得良好的计算—通信重叠，需要谨慎选择 FSDP 单元边界和编译顺序。对于 FSDP1 与特定编译支持组合，通常采用 `use_orig_params=True`，并以 Transformer 块等有意义的模块粒度进行封装；具体要求应以所用 PyTorch 版本为准。通信边界如何表示为图内操作或 graph break，同样依版本和实现而定。

在合适的封装与编译配置下，各块的前向和反向计算可形成编译区域，而全收集与归约散播围绕这些区域按依赖关系调度。这在粒度上类似 DDP 的 bucket 化思路，可以为计算与通信重叠创造机会，但并不保证每个块恰好对应一个图，也不保证通信完全隐藏。

相比之下，仅用一个 FSDP 实例封装整个模型，可能形成过粗的 FSDP 单元，使参数物化、通信调度和内存峰值不够理想。不过，TorchDynamo 产生多少图以及融合范围如何，并不能只由 FSDP 实例数量推断，还取决于模型控制流、编译支持和其他 graph break。

以 Transformer 块粒度应用 FSDP，通常能在内存占用、通信粒度、预取和编译区域之间取得较好的平衡。块内计算可能被编译和融合，块间通信也可能与相邻计算重叠；实际效果需要通过 profiler 验证。

PyTorch 在 `torch.distributed.fsdp.wrap` 中提供 `transformer_auto_wrap_policy`，便于按指定的 Transformer 块类型应用 FSDP，而无需手动逐层嵌套。采用块级完全分片时，可以按需物化各 FSDP 单元的完整参数，并调度全收集与归约散播；峰值时是否只有一个块的完整权重存在，以及通信能隐藏多少，取决于前向/反向预取、保留策略和执行时序。

下面的原书示例展示自动封装策略、激活检查点、FSDP 与编译的组合思路；其中若干 API 与调用顺序依赖版本，且代码存在需按实际版本修正之处，详见勘误记录：

```
import torch
import torch.nn as nn
import torch.distributed as dist
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP,
    CPUOffload, ShardingStrategy, BackwardPrefetch, MixedPrecision
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from torch.distributed.algorithms._checkpoint.checkpoint_wrapper
    import apply_activation_checkpointing, checkpoint_wrapper

# Prefer BF16 for Blackwell/Hopper-class GPUs; allow TF32 matmul for speed
# where numerically safe.
# enables TF32 use for FP32 matmuls when allowed
torch.set_float32_matmul_precision('high') # {'highest'|'high'|'medium'}

# Compile the model with a mode that reduces Python/launch overhead and
# enables CUDA Graphs
compiled_model = torch.compile(model, mode="reduce-overhead")

# Auto-wrap transformer blocks; adjust set below to match model's block classes
auto_wrap_policy = torch.partial(
    transformer_auto_wrap_policy,
    transformer_layer_cls={
        nn.TransformerEncoderLayer,
        nn.TransformerDecoderLayer,
        nn.MultiheadAttention,
    },
    min_num_params=int(1e8),
)

# Optional activation checkpointing for target modules
def _ckpt_check_fn(m: nn.Module) -> bool:
    return isinstance(m, (nn.TransformerEncoderLayer,
                          nn.TransformerDecoderLayer,
                          nn.MultiheadAttention))

apply_activation_checkpointing(
    compiled_model,
    checkpoint_wrapper_fn=checkpoint_wrapper,
    check_fn=_ckpt_check_fn,
)

# FSDP wrapper with correct argument types; MixedPrecision explicitly specified
fsdp_model = FSDP(
    compiled_model,
    auto_wrap_policy=auto_wrap_policy,
    sharding_strategy=ShardingStrategy.HYBRID_SHARD,
    cpu_offload=CPUOffload(offload_params=True),
    use_orig_params=True,
    mixed_precision=MixedPrecision(param_dtype=torch.bfloat16,
                                   reduce_dtype=torch.bfloat16,
                                   buffer_dtype=torch.bfloat16),
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
)

# Example input
batch = torch.randn(8, 128, dtype=torch.float32, device='cuda')
labels = torch.empty(8, 128, dtype=torch.long, device='cuda').random_(0, 100)

optimizer = torch.optim.AdamW(fsdp_model.parameters(), lr=1e-4, fused=True)
```

```
    # Forward + backward
    outputs = fsdp_model(batch)
    loss    = nn.CrossEntropyLoss()(outputs.view(-1, outputs.size(-1)),
                                    labels.view(-1))
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

这里将 `transformer_layer_cls` 设置为模型的块类型，使自动封装策略能够为相应块建立 FSDP 单元。采用块粒度完全分片时，前向传播会按执行顺序为所需单元全收集参数，并在适当时机释放或重新分片。峰值内存节省取决于单元大小、预取、重分片策略、激活和优化器状态，不能简单等同于某个块占总参数的比例。

此外，在每个块执行计算时，可以根据 FSDP 配置预取下一块的参数，从而尝试用块计算隐藏部分通信延迟。能隐藏多少取决于计算—通信比、互连、stream 依赖和资源竞争。

从调用接口看，训练循环通常无需因自动封装而改变。`auto_wrap_policy` 抽象了手动嵌套过程，只需指定目标块类型和策略参数，FSDP 便会管理相应单元的分片与通信。

简而言之，以 Transformer 块粒度封装 FSDP 往往是值得评估的起点，有机会改善内存效率和通信重叠。编译融合主要减少计算开销，并不自动降低参数物化峰值；FSDP 分片、按需全收集、重分片和预取共同决定内存峰值与通信时序。应针对具体模型、FSDP 版本和并行拓扑进行测量。

> 检查 TorchDynamo 的 `explain()` 输出时，可能会在某些 FSDP 边界看到 graph break，但这不是判断重叠正确性的充分条件。应使用分布式 profiler 轨迹核实全收集、归约散播与计算的实际时序。

结合张量并行、流水线并行与 torch.compile

张量并行和流水线并行是与 `torch.compile` 可组合但相互独立的机制。对于跨 GPU 通信操作，TorchDynamo 会依据操作、版本和后端支持情况，将其跟踪进图、作为高阶操作处理或在相应位置产生 graph break。

PyTorch 编译器主要优化各编译区域内的计算。不过，不能笼统断言它绝不改变通信：现代编译路径可能跟踪、重排或融合某些受支持的计算—通信模式，而 NCCL 集合操作的语义约束和高层顺序仍由分布式策略及依赖关系决定。TorchInductor 可为计算选择 cuBLASLt、cuDNN、Triton 等后端。采用任何分布式训练策略时，都应比较启用和禁用 `torch.compile` 的正确性与性能，并用 profiler 轨迹验证编译后的实际重叠。

可以使用 `torch.compile` 优化层、块或其他合适区域内的计算，同时由 DDP、FSDP、TP、PP 等分布式策略管理跨 GPU 通信。应关注 TorchDynamo `explain()` 输出中与全归约有关的警告，但更重要的是检查数值结果、重编译、graph break 和 profiler 时间线。分布式编译支持仍在快速演进，不能假定所有模型和并行组合都无需调整。

TorchTitan、AsyncTP、AutoParallel 与 SimpleFSDP

TorchTitan 是一套基于 PyTorch 的大规模模型训练参考实现。它提供可扩展的训练配方，可组合 FSDP、张量并行和异步张量并行（AsyncTP）等多种分布式训练策略。

具体而言，AsyncTP 使用双 stream 和感知 SM 执行 wave 的调度方式，错开 TP 集合操作，并尝试让较晚到达的 TP 全收集与下一 wave 的矩阵乘法重叠。这有助于隐藏传统 TP 中的部分通信和调度空隙。相关团队在特定模型、硬件和配置的实验中观察到前向传播及端到端加速，但这些结果不能直接外推；只有在能够验证数值正确性和扩展效率的场景中才应采用异步并行。

AsyncTP 可通过相应的 `torch.compile` 集成路径启用。编译器可识别适用的矩阵乘法和集合通信模式，将其降低为分块或融合的全收集/归约散播与计算调度。具体可用 API、模型约束和后端支持依 PyTorch/TorchTitan 版本而定。

AutoParallel 是 PyTorch 的另一项实验性计划，旨在为模型图自动规划并应用 FSDP、张量并行和流水线并行的不同组合。

AutoParallel 构建在 DTensor 的算子策略系统之上。DTensor 是 PyTorch 的原生分片张量抽象，为可组合的张量并行等方案提供基础；TorchTitan 参考实现大量使用了它。流水线并行还涉及独立的阶段划分与运行时机制，不能仅归结为 DTensor 分片。

AutoParallel 使用基于启发式规则的选择机制，评估给定工作负载的内存与通信成本，并尝试从不同并行方案中选择分片和划分计划。

对于大型模型和复杂集群，可以评估 AutoParallel 以减少部分手工并行调优工作，并与 TorchTitan 配合试验。但其成熟度、支持模型和 API 需要以实际版本为准，不应默认用于所有生产工作负载。

SimpleFSDP 使用 DTensor，并可结合选择性激活检查点等技术，以更便于 `torch.compile` 处理的方式实现 FSDP 语义。编译器能够跟踪计算与通信，并由相应的调度逻辑对中间表示（IR）节点进行分组或重排，以尝试改善二者重叠。

在特定 TorchTitan 实验中，SimpleFSDP 将内存用量最多降低约 28%；与其他分布式技术组合时，相对于该实验中的传统 FSDP2 eager 基线，训练吞吐量最多提高约 69%。这些是特定基准结果，会随模型、硬件、并行配置、软件版本和基线而变化。

> TorchTitan 及其相关的 AsyncTP、AutoParallel、SimpleFSDP 工作值得关注。它们展示了 PyTorch 大规模训练的参考实现和前沿优化，但各组件的维护状态、稳定性与公开 API 成熟度并不完全相同，应根据目标版本核实后再用于生产环境。

使用 HTA 分析多 GPU 性能

随着系统扩展到大量 GPU，管理各 rank 分别产生的轨迹和分析结果会变得困难。Meta 的 HTA（Holistic Trace Analysis，整体轨迹分析）可帮助汇总和分析多 worker 轨迹。这个开源工具读取各 GPU/rank 由 `torch.profiler` 生成的 JSON 轨迹，并提供跨 rank 的分析视图。原文所谓“数百万 GPU”是夸张表述；实际如此规模不可能逐 rank 持续采集并集中载入完整轨迹。

例如，使用 HTA 可以比较 8 块 GPU 的时间对齐轨迹，并查看各 rank 的 NVTX 标记。可能会发现 rank 0 在时刻 T 进入“backward”，而 rank 1 到 T+1 才进入；这提示存在不同步或上游延迟，但仅凭时间顺序不能断定 rank 1 正在等待 rank 0 的某次全归约。也可能

看到 rank 0 出现空隙，而 rank 1–7 正在计算；如果该模式反复出现，可能提示负载不均衡，需要结合通信、数据输入和依赖事件进一步判断。

HTA 还可报告 GPU 空闲时间，并提供与重叠和效率有关的分析指标。对于分布式训练，它有助于定位慢 rank 和同步问题，但工具建议仍需结合原始轨迹与系统指标验证。

> 过去有人会手工合并轨迹，再使用 TensorBoard 轨迹查看器分析。截至 2025 年，PyTorch 的 TensorBoard profiler 插件已弃用。可以使用 Perfetto 查看兼容的时间线轨迹，并使用 Meta HTA 分析多 rank 聚合数据；具体导入能力应按工具版本确认。

使用 HTA 时，通常通过 `torch.profiler.schedule` 等机制，让各 rank 仅记录少量代表性迭代，再将结果保存到共享位置。将这些轨迹载入 HTA 后，可以分析各 rank 和线程的操作时间、重叠、空闲区间以及轨迹中已记录的内存事件。为控制开销与文件规模，应避免无选择地长时间采集所有 rank。

例如，可以使用 HTA 检查全归约优化是否有效。优化前的轨迹可能呈现明显的串行模式：一段反向计算结束后出现等待全归约的空隙，之后才继续计算。调整 bucket 大小或归约调度后，如果更多通信与剩余反向计算并发，轨迹中的等待空隙应缩小。注意，增大 bucket 不一定改善重叠；过大的 bucket 可能推迟归约启动，因此应扫描参数并实测。

简而言之，HTA 面向 PyTorch 多 GPU 轨迹分析，适合深入研究跨多个 GPU 和计算节点的分布式训练循环行为。使用前应确认当前项目和版本对所生成轨迹格式的支持情况。

持续集成与性能基准测试

应用所有优化后，关键是持续维护这些收益并不断检查性能回归。随着代码和配置演进（例如 PyTorch 新版本、新模型特性和代码重构），如果没有持续跟踪，性能很容易悄然下降。

可以使用 TorchBench 和 GitHub Actions 建立简单的性能回归持续集成（CI），自动捕获变慢情况。TorchBench 是一套开源的 PyTorch 模型基准测试集合，也包含使用 `torch.compile` 运行的常见模型和基准，可用于跟踪编译器性能。

还可以将自己的模型或较小的代理模型加入 TorchBench。例如，在本地派生 TorchBench、添加模型，并在构建和 CI 流程中运行它。这样就能建立专门的性能回归任务，按计划或随每次代码提交持续运行。

性能回归任务会加载模型（也可以使用规模更小但具有代表性的变体），运行若干次迭代，并测量 tokens/s 或 samples/s 等吞吐量。CI 任务将结果与保存的基线比较；如果吞吐量低于阈值，就让构建失败。下面是一段定义 TorchBench 性能回归任务的 GitHub Actions 示例：

```
- name: Run MoE benchmark
  run: |
    torchbench run --model moe --iters 10 --batch-size 4 --json results.json

- name: Compare throughput
```

```
    run: |
      python scripts/compare_perf.py baseline.json results.json
```

第一步运行 MoE 基准 10 次迭代，批量大小为 4。这样的短运行适合在 CI 中快速估计性能；最终数据则应使用更长的预热和测量阶段。这里保持短时运行是为了符合 CI 的时间限制。

这里通过 `--json results.json` 输出 JSON 文件。第二步运行小脚本，将新结果与例如保存在 main 分支中的基线 JSON 比较。如果新提交变慢达到约 5%，就标记为回归并使 CI 构建失败；实际阈值应结合噪声和重复测量设定。

> 应确保 CI runner 使用一致的硬件；也可以使用云厂商预留的专用实例，以提高可复现性。不同 GPU 类型和云平台之间的性能数字可能差异很大。

还应编写正确性单元测试，确保优化后的自定义内核与 PyTorch 等价操作产生符合要求的结果，尤其要测试边界情况和多个随机种子。

自定义内核可能通过基本测试，却在极大输入下因溢出而失败。可以使用 PyTorch 的 `torch.allclose`，并根据 dtype、误差传播和算法稳定性设置严格但合理的容差，尽早发现正确性问题。

在 CI 流程中记录内存用量和数据加载时间也很有用。例如，记录运行期间的 `torch.cuda.max_memory_allocated()`，并实际测量数据加载耗时。性能优化是多维的：计算加速 20% 但内存用量增加 200% 的改动，在实践中可能反而得不偿失。

还要记住，软件并不完美。即使 PyTorch 更新也可能意外改变工作负载的性能特征，例如改变内核融合模式或调度启发式规则。

如果这类更新让模型变慢 5%，应尽早发现，调整代码，并作为 PyTorch issue 向上游报告。未纳入 PyTorch 常规性能测试的自定义或小众 LLM 更容易遇到这种情况，需要持续关注。

PyTorch 团队通常会积极处理性能回归；收到报告后，可能在 nightly 构建中推送修复。及早发现回归后，你也可以贡献修复，或至少提交包含环境、配置和复现数据的详细报告。

简而言之，只优化一次是不够的。随着系统演进、应用新的性能优化和加入新功能，需要持续保护已有收益。把性能测试集成到 CI 后，损害性能的代码变化就能及时暴露和处理，避免性能收益在下一次重构中悄然消失。

> 建议在使构建失败前运行多次迭代，并为外部噪声留出一定方差容忍度。例如，可要求连续 3 次运行都出现至少 5% 的回归后再失败。PyTorch 自身的持续性能回归系统（下一节介绍）也会使用统计平滑来减少误报。

除了自己的 CI，还可以关注 PyTorch 的性能*抬头显示面板*（HUD）。这是利用 PyTorch CI 系统跟踪常见模型性能的公共仪表板，下面将介绍它。

PyTorch HUD 性能仪表板

优化过程中，了解整个框架随时间发生的性能变化很有价值。PyTorch 开源性能 HUD 根据 nightly 基准运行提供反馈。这个 Web 界面显示 PyTorch 仓库在多种硬件后端（包括 NVIDIA GPU、AMD GPU 和 CPU）及多种常见模型上的构建状态、测试结果和性能指标。

PyTorch 工程师和社区成员可以使用 HUD 检测回归。如果新提交导致某个模型的 tokens/s 显著下降，HUD 会将其标为红色预警，促使进一步调查引发回归的改动。

HUD 的典型告警阈值约为 5% 回归，但实际规则取决于指标、硬件和统计策略。轻微噪声波动很常见；超过阈值的结果可能触发调查。HUD 还显示趋势线，因此多个小提交造成数周内逐渐变慢时，也可能被标记。

在 HUD 的 Benchmarks → vLLM 部分，可以查看各种语言模型的基准结果。仪表板会按模型和硬件类型跟踪编译时间、内存用量、吞吐量（每秒 token 数）及 FLOPS 利用率等指标，如图 13-7所示。

例如，可能会看到某个日期后吞吐量下降、内存用量上升。这提示内存碎片增加或选择了效率较低的内核等可能原因；HUD 可以帮助把这些变化与 GitHub 提交关联起来，但关联不等于因果证明。

监控 HUD 有助于判断 PyTorch 上游变化是否可能影响模型。如果 HUD 没有直接覆盖你的模型，可以用架构相近的模型作为代理。HUD 也是开源的，因此可以为自己的模型、硬件和环境构建类似系统。

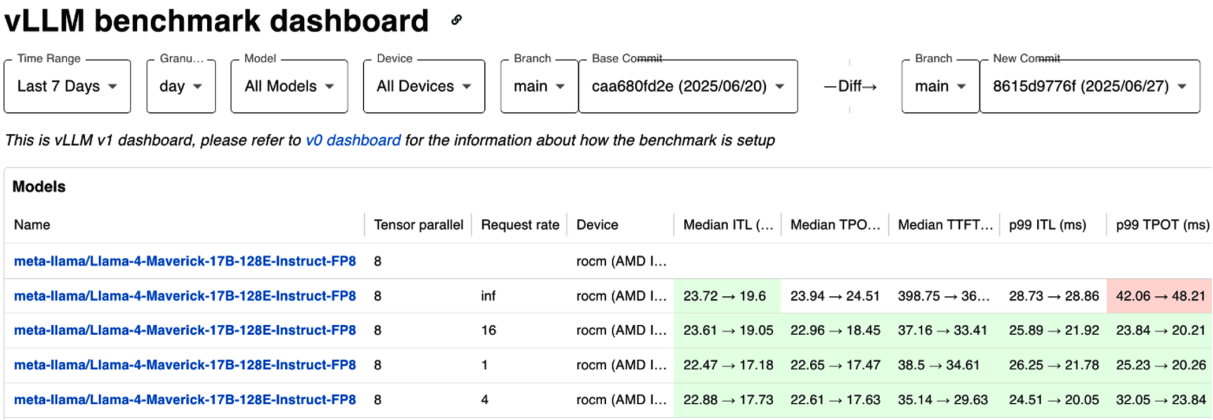


图 13-7　PyTorch HUD 仪表板（来源：https://oreil.ly/JxLKJ）

图表中的每个数据点，都是比较 PyTorch `main` 分支某个基准提交与最新提交的性能后生成的。仪表板支持选择时间范围（如最近 7 天、30 天）和粒度（小时、日、周），以缩放查看长期趋势。

HUD 的实现位于开源的 `pytorch/test-infra` 仓库中，结合使用 Python 脚本和 Grafana 仪表板。可以持续收集模型性能数据并可视化，在内部运行一个简化版 HUD；电子表格或 Grafana 实例也能实现类似效果。关键是始终一致地跟踪相同指标。

添加或修改仪表板、贡献新的 YAML 配置以及更新 CI 中的基准脚本相对直接。原则上，如果希望在 HUD 中跟踪自定义模型，可以为它添加基准并让 PyTorch CI 执行；这样，影响该模型性能的 PyTorch 改动就会显示在图表中。

> 强烈建议为自己的模型建立类似 `pytorch/test-infra` 的持续基准和性能回归系统，以便尽早发现特定工作负载的回归。

性能基准与 MLPerf 日志

除了临时基准和 CI，MLPerf 等行业标准基准也能提供有价值的反馈并促进良好实践。MLPerf Training 和 Inference 基准通过严格日志记录推动最先进性能，同时确保结果可信并可进行公平的同条件比较。即使不参加竞赛，MLPerf 的日志方法也适合用于分析自己的系统性能。

MLPerf 日志规范化训练过程中关键事件和指标的输出。每条日志都是带有类似 `:::MLL` 前缀的 JSON 对象，包含键值对及元数据。在 PyTorch 中，可以使用 GitHub 上的开源 MLPerf Logging 库，将输出格式化为 MLPerf 标准。

在 MLPerf 训练运行中，日志会记录初始化开始时间、训练开始时间、每个 epoch 的耗时、训练结束时间、最终准确率等。事件应带有毫秒级时间戳和规范标签。

MLPerf 合规脚本会解析这些日志，验证运行是否遵守规则。例如，开始后不应更改超参数，并且必须使用规定的 epoch 数量等。

日志还会包含计算出的吞吐量，以及是否在允许时间内达到目标准确率。这保证了公平性：不能只报告吞吐量而不满足准确率要求。

> 在自己的训练中，应始终将性能测量与准确率检查配对，避免把模型优化到不稳定状态。

一些 MLPerf 日志（尤其是研究提交）还会细分每个 epoch 和迭代的耗时。竞赛未必要求如此详细，但对内部分析很有用。例如，可以记录每次迭代花在前向、反向以及全归约等通信上的时间，也可以记录多节点集群中各节点的平均耗时，以定位分布式瓶颈。

下面是一段 MLPerf JSON 日志示例。随后给出表 13-6，它根据这些日志计算各组件占总步耗时的百分比：

```
{
  "step_time_ms": 24.0,
  "forward_ms": 10.5,
  "backward_ms": 9.0,
  "allreduce_ms": 4.0,
  "other_ms": 0.5
}
```

表 13-6　MLPerf 日志示例：每次训练迭代的耗时分解

步骤组件	每次迭代耗时（ms）	占步骤百分比
前向传播	10.5 ms	43.8%
反向传播	9.0 ms	37.5%
全归约（梯度同步）	4.0 ms	16.7%
其他开销	0.5 ms	2.1%
步骤总耗时	**24.0 ms**	**100%**

在这个 24 ms 的训练步骤示例中，计算（前向和反向）耗时 19.5 ms（10.5 ms + 9.0 ms），占步骤时间的 81.3%（43.8% + 37.5%）。通信（梯度全归约）耗时 4.0 ms（16.7%），其他开销（如数据加载）耗时 0.5 ms（2.1%）。需要注意，逐项耗时之和可能因重叠而不等于端到端步骤耗时；这里按示例表格进行加和。

这种分解很有用，它显示约六分之一的示例步骤时间用于梯度同步。若要进一步加速训练，可以尝试减少全归约时间，或让它与计算更好地重叠。

例如，可以尝试激活压缩，或使用异步全归约、流水线并行等方法改善通信与计算重叠，以减少全归约占用的 4 ms。如果“其他开销”足够大，则应转而优化数据加载等环节。

MLPerf 中的全归约优化包括延迟全归约（称为 *slack*）以及让多个较小的全归约与计算重叠等技术。这些高级技巧超出本节范围，但这种耗时分解能准确指向需要优化的位置。

虽然 MLPerf Logging 针对竞赛规则设计，但结构化日志、性能指标和耗时分解的一般实践同样适用于自己的训练和推理实验。例如，可以在训练循环中记录每个 epoch 的 JSON 行，并加入吞吐量、延迟、GPU 利用率（来自 `nvidia-smi`）等指标。

在长时间训练中，这些日志会成为训练后分析的宝贵资料。可以绘制第 1 天到第 7 天的性能变化，判断任务是否因内存碎片而变慢，也可以观察数据加载、计算等不同阶段如何随规模变化。

记录指标而不只是最终准确率，可以让结果更容易复现和调试。如果之后有人重新训练模型但速度更慢，日志有助于定位原因：可能是数据输入层变慢，也可能使用了不同的硬件配置。这种实践与 CI 相辅相成，形成“记录、监控、比较”的方法。

通过在流水线中插入监测逻辑，输出各耗时组件的 JSON 日志，可以在实施优化时持续跟踪改进，也便于以标准化方式与团队沟通瓶颈和调优结果。

> 由于 MLPerf 包含跨多个 GPU 和计算节点的大模型基准，可以研究 MLPerf 提交结果，了解常见 LLM 与集群配置的实践。不过，提交结果受特定模型、软件版本、硬件、功耗和竞赛规则约束，不能直接视为所有工作负载的最佳方案。它们可作为持续学习性能技巧和大规模拓扑设计的资料来源。

要点回顾

PyTorch 相对简单且抽象层次较高，有时会让人产生“性能天然安全”的错觉。因此，开发过程中很容易引入隐蔽的性能问题。下面总结常见的 PyTorch 性能陷阱及其应对方法：

坚持先分析性能

在超大规模系统中，瓶颈可能隐藏在任何层级：Python 开销、PyTorch 框架调度、CPU 数据加载停顿、GPU 内核效率、内存问题等。仅凭直觉往往找不到真正的热点。应像本章一样使用多工具的整体性能分析策略，覆盖各层级。现代分析器提供低开销模式，可在生产环境中捕获回归。结合 `nvidia-smi` 提供的 GPU SM 利用率等硬件指标，才能可靠识别瓶颈并正确安排优化优先级。

优先使用编译模式而非 eager 模式

eager 模式下，每个细小操作都可能作为独立内核启动，反复产生 Python 调度和 GPU 启动开销。可以使用 PyTorch 的 `torch.compile` 编译路径；只需近似一行改动（`model = torch.compile(model)`），PyTorch 就可能捕获模型图并生成融合、优化后的代码。但编译效果取决于模型、输入形状和后端。

使用工作负载允许的最高编译优化模式

对于长时间运行的任务，`max-autotune` 往往有利于稳态速度；但小批量或动态形状可能更适合 `reduce-overhead`。必须在实际工作负载上验证。`max-autotune` 中的 CUDA Graph 可以减少启动开销，但要求满足捕获条件，频繁变化的形状可能不兼容。

保存编译产物以便复用

如果启动时间很重要，最好缓存编译产物供以后复用。可以使用 `torch.compiler.save_cache_artifacts()` 和 `load_cache_artifacts()`。对于多节点长期运行的任务，可将编译产物作为“mega-cache”持久化到共享路径（例如 `TORCHINDUCTOR_CACHE_DIR`），并在各节点挂载到相同位置，以减少新节点启动时的冷启动。

避免同步陷阱

PyTorch 注重易用性，因此很容易无意中写出强制 CPU-GPU 同步的代码。例如，对 CUDA 张量调用 `tensor.item()` 获取 Python 值会同步 GPU。在 stream 之间协调时，结合事件使用 `torch.cuda.Stream.wait_stream()`，不要主动强制同步。同样，将数据从 GPU 传到 CPU 时不使用 `non_blocking=True` 可能导致同步；应使用异步传输，并通过性能分析器查找隐藏同步。

避免使用 `time.time()` 在 Python 侧计时，因为它可能隐式同步

使用 `time.time()` 测量 GPU 代码块时，测量结果可能包含同步开销。更适合使用 `torch.cuda.Event(enable_timing=True)`，在 GPU 时间线上计时并减少额外同步。

利用 Tensor Core

模型可能在不知情的情况下退回完整 FP32，从而没有使用 Tensor Core。为了让 GEMM 使用 Tensor Core，可以用 `torch.autocast` 包裹前向和损失计算，并选择较低精度的 `dtype`。（`autocast` 不会改变模型权重的存储 `dtype`，除非显式转换模型；它会为适用操作选择计算 `dtype`，并让数值敏感操作保持 `float32`。）

使用 TF32 代替 FP32 以启用 Tensor Core

对于能够接受 TF32 精度语义的 `FP32` 工作负载，可以使用 `torch.set_float32_matmul_precision()` 设置 `"high"` 或 `"medium"`，并根据 CUDA/PyTorch 版本启用相应 TF32 矩阵乘法路径。在现代 GPU 上，TF32 矩阵乘法通常使用 Tensor Core。应使用 Nsight Compute 的 SpeedOfLight 视图或 `sm__inst_executed_pipe_tensor` 指标验证。注意，`"highest"` 要求更严格的 FP32 计算并可能禁用 TF32，适合调试但通常不利于追求性能。

验证确实使用了 Tensor Core

要验证内核是否使用 Tensor Core 以及是否采用预期的 `dtype`，应使用性能分析器。在 PyTorch 中先用 `torch.profiler` 检查算子耗时，再用 Nsight Compute 确认 Tensor 管线活动。Nsight Compute 的 SpeedOfLight 和 Compute Workload Analysis 部分会报告 Tensor 管线利用率，并通过性能监控器（PM）采样展示内核执行时间线。

可能时优先使用 BF16 而非 FP16

在现代 GPU 上优先考虑 `dtype=torch.bfloat16`，因为 `BF16` 保留 `FP32` 的指数范围，通常不需要梯度缩放。如果必须使用 FP16 训练，应启用 `torch.cuda.amp.GradScaler`，并在日志中检查溢出或下溢。现代 GPU Tensor Core 通常可直接使用 BF16，而无需缩放。

融合小操作

许多模型计算由长串小型逐元素或矩阵操作组成，每个操作都会产生额外的内存读写和启动开销。可以用 `torch.profiler` 找出连续出现的许多小操作（例如小于 1 ms）。如果看到 linear → gelu → dropout 这样的模式，可以使用融合模块或让 `torch.compile` 尝试融合；对于编译器未覆盖的操作，可考虑编写自定义融合内核。单次融合可能只节省几个百分点，但多个“最后一公里”改进可以累积。

减少内存碎片

如果训练任务或推理流水线运行数天、数周甚至数月，内存碎片可能成为问题，尤其是多数 LLM 应用的张量大小随迭代变化时。可以使用 PyTorch 缓存分配器等内存池机制，并尽量预先分配大型张量。不同分配大小越少越容易稳定，因此应尽可能复用缓冲区并保持形状固定。还可以通过环境变量（如 `max_split_size_mb`）调节 CUDA 分配器；避免把大块拆成许多小块，为常用缓冲区按固定最大尺寸预分配并重复使用，以保持分配尺寸稳定，减少碎片导致的变慢或崩溃。

使用激活检查点

默认情况下，PyTorch 会保存反向传播所需的激活，大模型因此会消耗大量内存。可以使用激活检查点，以额外计算换取内存；`torch.utils.checkpoint` 可封装前向的一些片段，使其激活不被保存，而在反向时重新计算。这会降低内存用量但增加计算，是否值得取决于工作负载；不能笼统说对所有数百亿参数模型都“必需”。还可以混合精度，例如保留少数关键激活的较高精度，同时以较低精度重新计算其他激活，以兼顾准确率、内存和时间。

将内存卸载到 CPU 或 NVMe 存储

模型参数、梯度、优化器状态和激活会共同争用有限的 GPU 显存。即使 HBM 很大，数千亿参数模型在低精度下也可能超出容量。可以利用系统内存层次，将不常使用的数据卸载到 CPU 内存或 NVMe，并在需要时取回。数据传输可以通过异步复制进行流式化并与计算重叠，但要监控互连吞吐量，避免链路饱和。

减少输入流水线停顿

即使训练循环已经优化，如果输入流水线供数不够快，仍会成为瓶颈，通常表现为 GPU 等待下一批数据的空闲气泡。性能分析往往能直接看到迭代前的空隙，或看到 CPU 线程忙于加载而 GPU 空闲。应通过足够的 DataLoader worker 和合适的 `prefetch_factor` 提高数据吞吐量。

应合理利用 CPU 核心进行数据加载和准备。在离散 CPU-GPU 路径上，可使用 `pin_memory=True` 与非阻塞传输改善 H2D 复制，但仍需实测。将分词等预处理离线完成，让数据加载器只做必要工作；还可以增大 DataLoader 的 `prefetch_factor`，持续提前准备后续批次。

分析并酌情卸载 CPU 侧数据变换

Python 数据变换（如分词）可能出人意料地慢，因此应让主机侧变换尽量向量化，并在可用时使用优化的 C++ 库。对于复杂变换，也可以评估使用 NVIDIA DALI 等库卸载到 GPU。

优化多 GPU 与多节点通信

如果管理不当，分布式训练常会因通信开销触及性能上限。可以使用 DDP 等 PyTorch 分布式实现；它通常会尝试让通信与计算重叠。应通过实测调节梯度 `bucket_cap_mb`：较大 bucket 可减少每条消息的开销，但通常会推迟第一次归约的启动；较小 bucket 能更早开始通信，却会增加集合操作数量。

监控网络带宽

如果网络带宽接近饱和，可以评估激活压缩以减少传输量。应在依赖允许时让全归约与反向计算重叠，以隐藏部分通信时间。多节点场景还要考虑拓扑，把频繁通信的进程尽可能放在同一节点或交换机域内，以降低延迟。

避免随时间产生“性能腐化”

持续的代码变化、框架更新和新功能很容易造成性能回归。例如，小型重构可能引入意外同步，PyTorch 升级也可能改变某个操作的实现。应把性能作为一等指标，设置 CI

测试、仪表板和告警持续监控。每次提交（或每日/每周运行）都应包含快速性能基准，以便捕获回归。

硬件变化时更新基线

例如从 B200 换到 GB300 时，应重新建立性能基线和阈值。新硬件对不同模式的处理可能有好有坏，应重新校准并相应调整告警。

在自动化工作流中使用 TorchBench 或自定义计时脚本

关注 PyTorch nightly 性能基准的外部信号（公开 HUD 回归仪表板会报告），尤其是与自己模型相似的模型。这样在升级前就可能发现 PyTorch 的一般性回归。检测到变慢后，应立即调查并找出根因（例如自身代码或上游改动），采取回滚或适配等缓解措施，并向 PyTorch、Triton、NVIDIA 等上游报告，让社区修复问题。还要保留优化的正确性测试，避免准确率悄然下降。将性能回归测试纳入工作流，才能让加速收益经受时间考验。

结论

应将微观性能分析（如逐算子、逐内核分析）与宏观基准测试（如端到端吞吐量和延迟）结合起来，形成对系统性能的完整视图。优化单个内核与调优完整的端到端训练和推理流水线并持续维护同样重要。随着硬件演进，这种整体方法尤其关键。

本章介绍的基础知识能帮助你适应新平台。应始终进行性能分析、识别瓶颈，并在软件栈各层应用合适的优化。系统地应用编译器和内存优化，可以让工作负载更接近硬件极限，同时保持可接受的准确率。

正如本章所示，仔细分析能找到真正的瓶颈，有针对性的优化可以显著提升训练和推理性能。同时必须建立自动化机制，防止这些收益随时间发生性能回归。

这段端到端优化过程需要在软件栈各层投入工作。最终目标是构建优化、高效且能抵抗回归的系统，充分发挥现代 GPU 硬件性能，扩展到多节点集群和机架级拓扑，并适应未来硬件、软件和算法的发展。

第 14 章　PyTorch 编译器、OpenAI Triton 与 XLA 后端

在第 13 章中，我们讨论了优化和调优 PyTorch 训练与推理工作负载的多种方法，介绍了 PyTorch 编译器如何自动执行内核融合及其他内核级优化，并以很少的代码改动提升性能。

本章深入介绍动态 PyTorch 编译栈，包括 TorchDynamo、Ahead-of-Time Autograd（AOT Autograd）和 PrimTorch 中间表示（IR）（也称 *Prims* 或 *Prims IR*），以及 TorchInductor、Accelerated Linear Algebra（XLA）和 OpenAI Triton 生态等编译器后端。PyTorch 编译栈如图 14-1所示。

我们还将介绍调试编译流水线的工具，以及在多 GPU、多节点集群上扩展 PyTorch 的库；随后探讨 `torch.compile` 的内部工作方式，以及高效处理动态形状和可变序列长度的方法。

此外，我们会考察 PyTorch 编译器与 OpenAI Triton 生态的集成。目标是在不牺牲 PyTorch 灵活 eager 执行开发体验的前提下，加速并扩展 PyTorch 模型和应用。

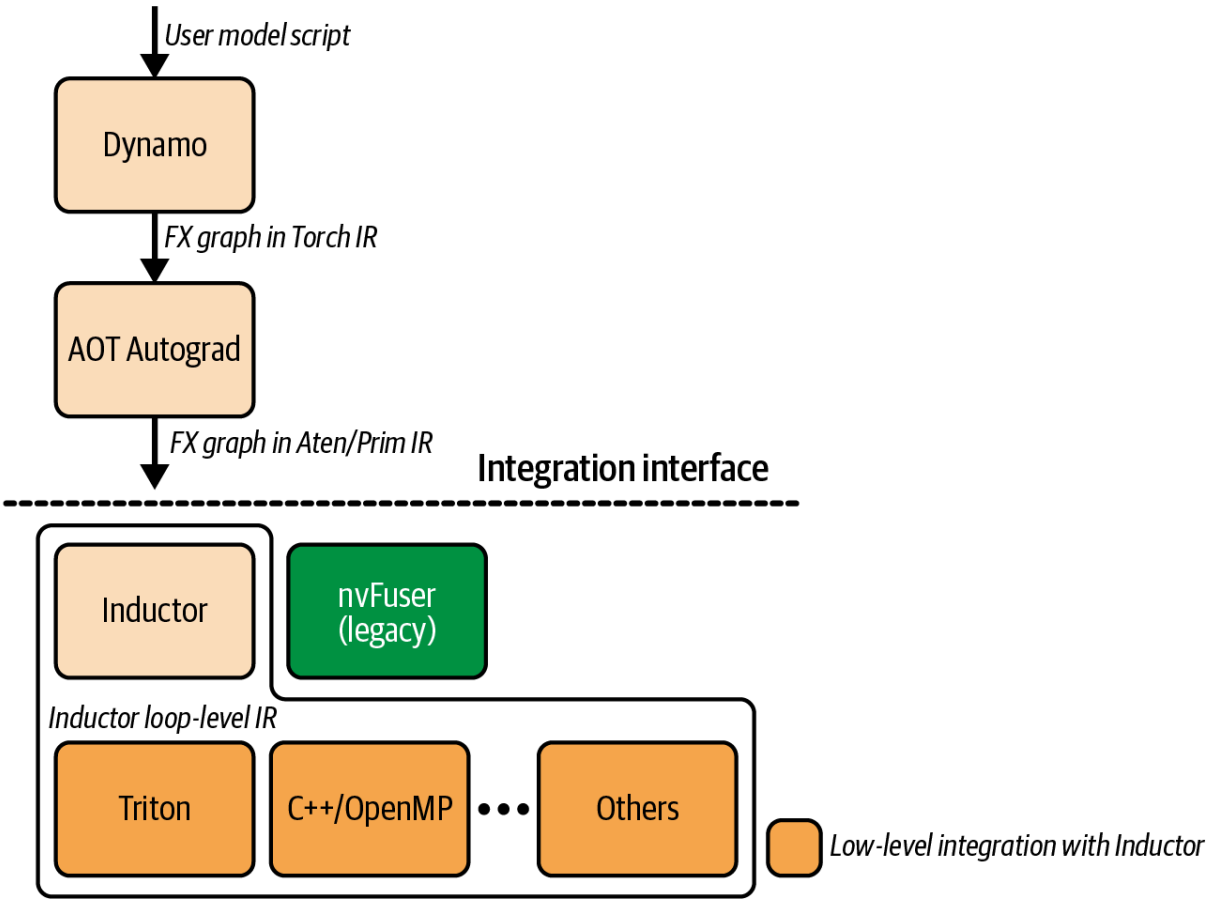


图 14-1　PyTorch 编译栈概览

深入理解 PyTorch 编译器

如第 13 章所述，PyTorch 的 `torch.compile` 可以编译 PyTorch 代码和模型，从而显著提速。多数情况下只需一行代码，如下所示；后文会逐步介绍不同选项：

```
compiled_model = torch.compile(model,
  mode="max-autotune",
#  ...
)
```

本节拆解PyTorch 编译流水线，包括 TorchDynamo 图捕获、AOT Autograd 前向/反向联合图优化、PrimTorch IR 和 TorchInductor 代码生成。该流水线负责为目标 GPU 硬件生成优化内核，如图 14-2所示。

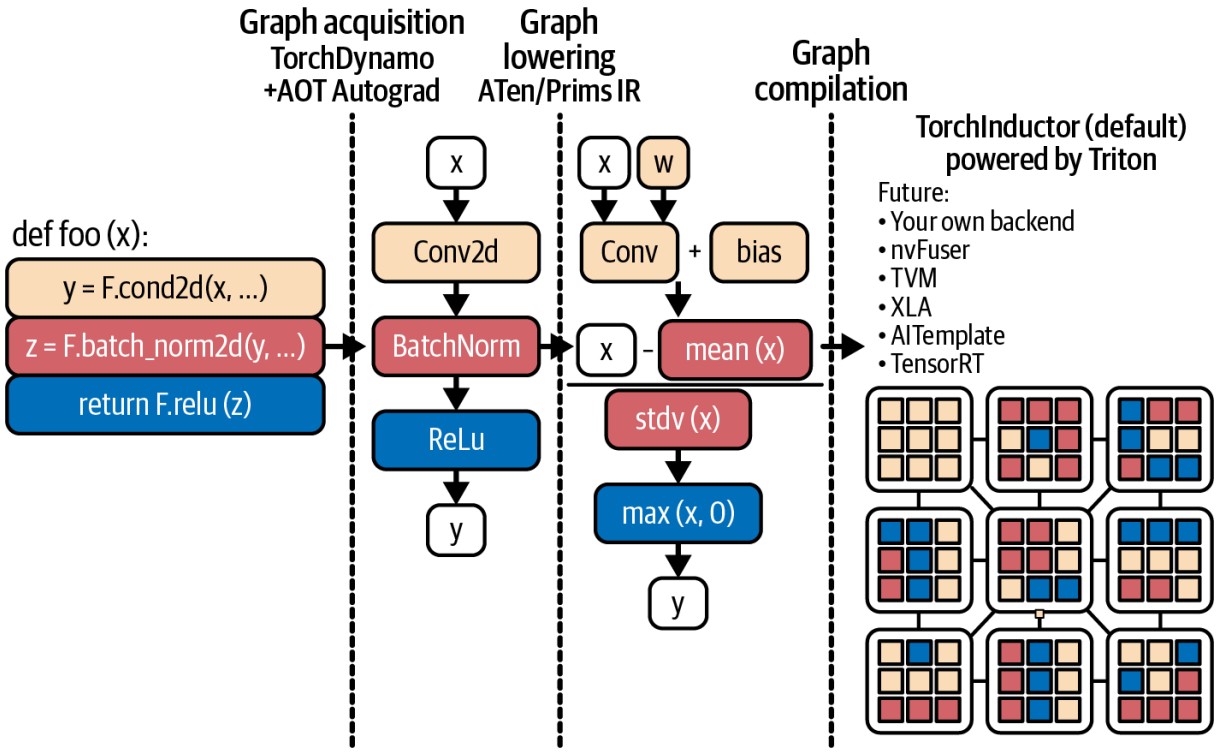


图 14-2　PyTorch 编译流水线（改编自 PyTorch 2.x 概览）

使用 TorchDynamo 捕获字节码并提取图

TorchDynamo（简称 Dynamo）是 `torch.compile` 的第一阶段。它接入 Python 的帧求值机制，在字节码层拦截模型执行。

Dynamo 接入 CPython 帧求值流程，识别产生张量的字节码区域，并为这些区域构造执行图。随后使用选定后端执行编译图；不支持的代码则保留为 eager 执行。

正是这种拦截和重写机制，使 TorchDynamo 能把连续的 PyTorch 操作捕获为图表示，再交给后续的 AOT Autograd 和 PrimTorch IR 优化，后文将介绍这些阶段。

TorchDynamo 利用 CPython 帧求值 API（PEP 523）安全地捕获操作，同时尽量降低开销。通常 Python 解释器逐个执行操作；启用 Dynamo 后，解释器会把执行转交给

Dynamo，由它先将张量操作聚合成图再执行。这支持内核融合等整图优化，减少逐操作的 Python 和主机侧开销。

编译图可以把多个操作融合到一个或少数几个内核中，而不必为每个操作启动新的 GPU 内核并承担逐操作 Python 开销。这能减少调度开销并改善内存访问模式。

在第 13 章中，我们看到 Python 开销和大量小操作如何成为模型瓶颈。只要操作链适合构图且不会产生过多 graph break，TorchDynamo 就能在可能时把小操作批量组织为更大单元，从而缓解这一问题。

通过 `torch.compile` 启用 TorchDynamo 后，它会检查每个 Python 操作码。遇到算术或神经网络层等 PyTorch 张量操作时，不会立即执行，而是将其追加为 FX 图中的节点，并把捕获区域的执行交给编译后端。TorchDynamo 会持续这一过程，直到遇到无法处理的代码；此时发生 graph break（稍后详述），不支持的操作在普通未编译 eager 模式中运行。

TorchDynamo 会尽可能把程序的大部分编译成单个图，但遇到复杂控制流或非 PyTorch 库调用等不支持结构时，会退回 Python eager 执行，以确保正确性。

不支持的操作块执行完后，Dynamo 会从后续可跟踪代码开始捕获新图。这种编译与未编译执行的组合兼顾了两者优点：需要灵活性时保留 PyTorch 的 eager 能力，其余部分则编译以获得速度。

> 应尽可能避免 graph break，因为它会中断整图优化并限制编译器的性能收益；但某些边界是为了正确性或通信语义而有意保留的。

可以使用 `torch.compiler.set_stance("fail_on_recompile")`，让 Dynamo 在发生不安全重编译时抛出错误。它会记录重编译原因，帮助调试 graph break，并让你清楚了解图为何被拆分。代码如下：

```
// fail on graph breaks, recompiles
torch.compiler.set_stance("fail_on_recompile")
compiled_model = torch.compile(model,
  mode="max-autotune",
  ...
)
```

这样可以重构导致 break 的代码路径，或使用 `torch._dynamo.allow_in_graph()` 将其标记为预期的图边界（稍后介绍）。图清理完成后，可切换回 `torch.compiler.set_stance("eager_on_recompile")`。注意，恢复该设置后，如果再次发生 graph break，TorchDynamo 可能静默退回 eager 模式。

TorchDynamo 捕获的输出称为代码的 *FX Graph*。FX 是一种中间表示（IR），其中每个节点都是对 PyTorch `aten` 操作符或内置 Python 函数的调用。下面以一个简单 Python 函数为例：

```
def f(x, y):
    z = x.sin() + y
```

```
    return z.sum()
```

TorchDynamo 会生成大致等价于以下伪代码的 FX Graph：

```
graph():  # pseudo-code for FX IR
    %x : Tensor = Placeholder[target=x]
    %y : Tensor = Placeholder[target=y]
    %sin : Tensor = torch.sin(%x)
    %add : Tensor = torch.add(%sin, %y)
    %sum : Tensor = torch.sum(%add)
    return %sum
```

FX Graph 节点对应 `Placeholder` 输入，以及 `aten::sin`、`aten::add`、`aten::sum` 等原始 ATen 操作调用，清晰表示计算图结构。构建完成后，该 FX Graph 会交给 AOT Autograd，以联合跟踪前向和反向传播。

AOT Autograd 生成的前向/反向联合轨迹随后会交给 TorchInductor 或 XLA 等后端编译器，执行内核融合并生成优化设备代码。TorchDynamo 本身与框架后端相对解耦，专注于准确、低开销的图捕获，把繁重优化交给下游编译阶段。

TorchDynamo 会针对影响图轨迹的 Python 值插入 guard，例如张量形状、dtype，甚至全局变量。如果原本假定恒定的值发生变化（如张量形状或 dtype），guard 会使编译图失效并在需要时重新编译，从而稳健处理动态形状和模型修改；代价是违反假设时会增加重编译。

> 使用 PyTorch 编译器时，建议监控图重编译次数。

默认 `dynamic=None` 时，编译器会针对观察到的形状进行特化。遇到动态形状后，它可能重新编译更具动态性的内核；若之后发现其他动态来源，也可能再次编译。

实际重编译次数取决于形状变化和 guard；不能保证第一次变化只多一次编译。如果已知会变化的输入维度，可以在运行模型前使用 `torch._dynamo.mark_dynamic(tensor, dim)`，帮助编译器预先处理该动态性。

还可以使用 `torch.compiler.set_stance()` 调整编译器的“stance”，改变 TorchDynamo 何时退回 eager 或重编译。stance 控制编译器面对错误和 fallback 时的宽严程度，在开发反馈与不中断执行之间提供权衡。以下名称和语义以书稿所用版本为准：

`default`

: 编译器会尽量编译支持的部分，遇到不支持代码时静默退回 eager 执行。这是标准默认设置，会产生通常的编译行为和 fallback。

`fail_on_recompile`

: 遇到 graph break 或不支持操作时，编译器抛出错误。开发阶段希望捕获意外 break 或未编译路径时，这很有用。

`eager_on_recompile`

需要重编译时，以 eager 模式运行代码。如果已缓存的编译代码对当前输入仍有效，则继续使用缓存代码。

`force_eager`

使用 eager 模式，并忽略所有 `torch.compile` 指令。

通过捕获连续操作序列，TorchDynamo 可以执行内核融合等整图优化，减少启动开销和内存移动，也消除融合操作的 Python 层开销。图 14-3比较 eager 与编译模式，并展示编译器缓存。

编译器可以把多个操作融合进单个内核，而不必分别执行每个小操作并单独启动内核。这能减少 CPU-GPU 同步点并改善内存局部性；否则 GPU 可能被大量细粒度操作、频繁同步和过多全局内存访问拖慢。

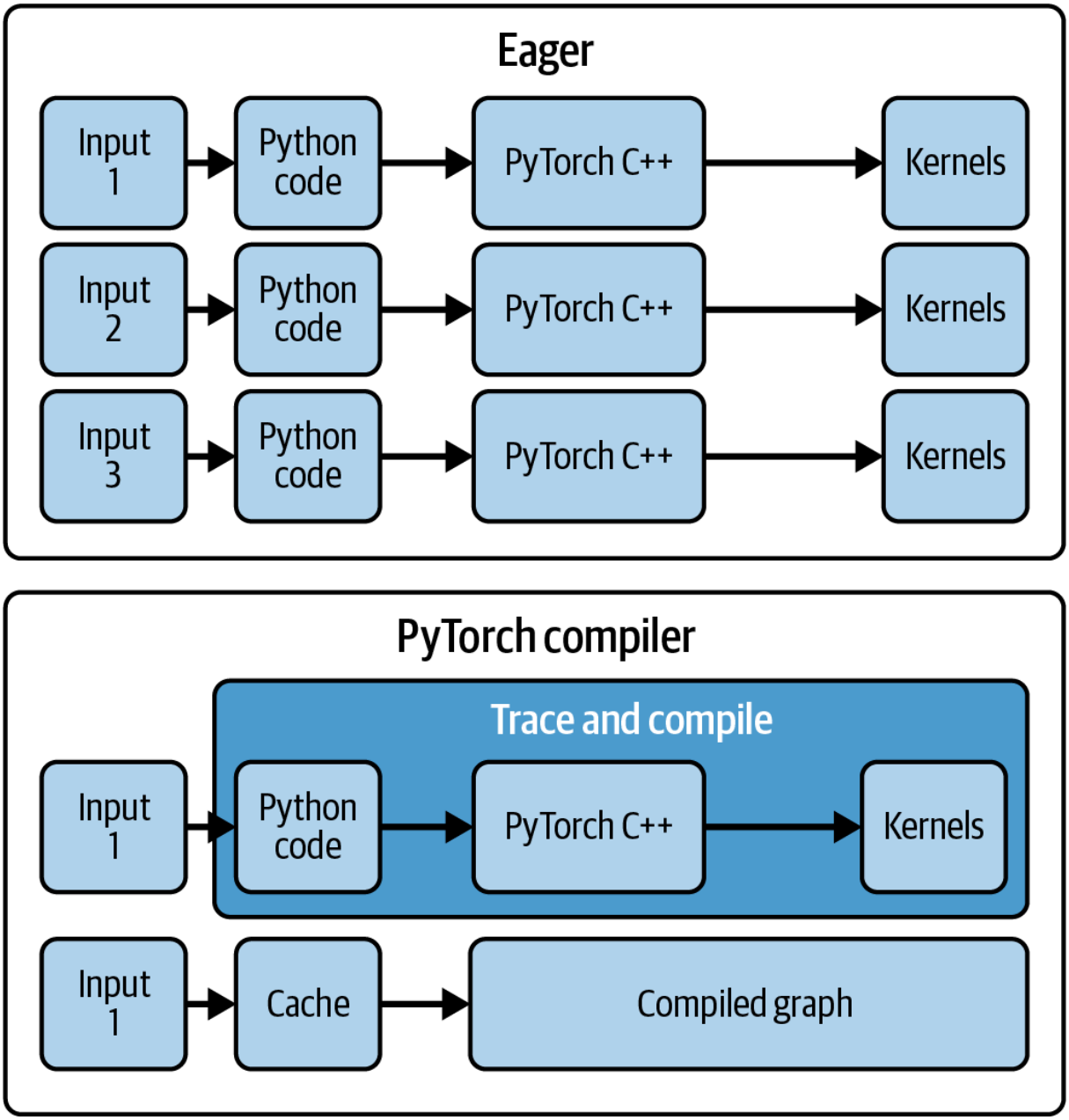


图 14-3　PyTorch eager 与编译模式

TorchDynamo 会持续捕获，直到需要 graph break。触发原因可能是不支持的 Python 结构（例如使用 Python bool 而非张量操作的某些控制流），也可能是不支持的操作。发生 graph break 后，当前图片段结束，无法捕获的代码退回 eager；回到可跟踪代码后，TorchDynamo 再开始新的图轨迹。

可以将条件逻辑表达为纯 PyTorch 张量操作（包括带掩码的 `torch.where()`），从而实现适合编译图的条件。下面把不适合构图的 Python `if` 替换为纯张量掩码操作：

```
# Compute a boolean mask from a data-dependent tensor condition
# mask shape (batch_size,1) broadcastable to x's
mask = x.sum(dim=1, keepdim=True) > 0

# Use torch.where to select element-wise between two tensor expressions
# picks f(x) where mask is True, otherwise g(x)
out = torch.where(mask, f(x), g(x))
```

这样可以避免会导致 graph break 的 Python `if x.sum() > 0:`，全程使用适合构图的 PyTorch 张量操作。在此例中，TorchDynamo 能在不中断图的情况下捕获包括掩码和 `torch.where()` 在内的整个序列。

随着版本更新，PyTorch 支持无 graph break 捕获的操作范围会扩大。例如，`torch.cond` 等高级条件原语可以在图中捕获某些 if/else 逻辑，两个分支都会被跟踪和编译。`torch.cond` 要求布尔标量谓词，两个分支必须返回相同结构和 `dtypes`，并且运行时形状要一致。依赖数据值的分支仍常会导致 graph break。

> 建议通过 `torch.where()` 等张量操作重构代码，尽量减少 graph break，扩大 TorchDynamo 能连续捕获的区域。

AOT Autograd 融合前向与反向传播

TorchDynamo 尽可能捕获前向传播的 FX Graph 后，下一个编译阶段是 AOT Autograd。AOT Autograd 以“functional”模式让 Dynamo 捕获的前向图通过 PyTorch autograd 引擎，记录反向操作，从而生成静态反向图；这不同于依靠默认 autograd 引擎逐个执行反向操作。

本质上，AOT Autograd 生成联合前向—反向图，然后整体优化和融合。对于支持的操作和保持相同数值语义的编译路径，它应与 eager 模式产生等价结果，但仍需测试验证。

AOT Autograd 通过 autograd 引擎跟踪前向图，捕获梯度计算。它会以 `torch.autograd.forward_ad`（或类似机制）记录反向计算所需的操作，生成联合前向和反向图；随后可进行公共子表达式消除等优化。之后由 TorchInductor 或 XLA 等后端编译。

通过提前规划前向和反向，PyTorch 编译器可以跨越二者边界进行整体融合，实现跨两个阶段的提前融合。例如，在条件允许时，把前向中的逐元素操作与反向中对应的逐元素梯度计算融合到一个内核。

在安全的情况下，编译器还可复用前向与反向之间的中间结果以消除开销。对于反向操作占总运行时间较大的训练负载，这可能显著改善性能。

没有 AOT Autograd 时，PyTorch 需要使用默认 autograd 引擎独立于前向逐个执行反向操作；使用 AOT Autograd 后，整个执行图可以整体优化。

AOT Autograd 生成的联合图在编译器能够保持正确性和数值语义的前提下，应计算等价结果；无法保证时就需要 graph break。由于前向和反向的完整操作序列已知，联合图还可以进一步优化。

跨前向和反向融合操作及内存规划，可以减少内存访问和内核启动开销。当对涉及梯度的操作或训练工作负载调用 `torch.compile` 时，PyTorch 编译模式通常会在内部使用 AOT Autograd；具体路径取决于后端和版本。

简而言之，`torch.compile` 可在训练期间通过 AOT Autograd 提前组织梯度计算。它能处理许多 autograd 操作，并在语义保持一致时取得与 eager 等价的结果；还可复用前向和反向之间的缓冲区，以降低峰值内存。虽然这一阶段对用户大多不可见，却是现代 AI 训练获得加速的重要组成部分。

PrimTorch IR（Prims）简化操作集

将图交给低层代码生成器之前，PyTorch 会执行一种中间表示（IR）转换，在部分文档和源码中称为 PrimTorch IR（Prims）。它把图中多样的操作归约为较小的核心“原语”操作集，因此得名 *PrimTorch*。

作为背景，PyTorch 完整 API 包含数千个（超过 2,000 个）操作。PrimTorch IR 将其缩减为编译器可以重点支持的更小原语集合；在书稿所对应的版本中，这一集合约含 250 个原语，如基本算术、归约、复制和 reshape 等。

许多复杂或高级的 PyTorch `aten` 操作都可以用 PrimTorch IR 分解为这些原语。例如，`x.add_(y)` 这样的“原地”操作会降低为函数式 `add`，再显式 `copy` 回 `x` 的存储，如下所示：

```
%z    = aten::add(x, y)
%copy = aten::copy_(x, %z)    # writes z's data into x
```

这里的 IR 包含独立的 `aten::copy_` 节点，而不是特殊的原地变更。这样所有张量更新都显式表示，下游编译器可以把变更视为普通复制操作，从而简化内核处理。

将原地变更转换为函数式操作加独立复制节点后，编译器无需再推理别名关系或隐藏副作用。融合过程和内存规划算法可以在纯函数式数据流上工作，从而进行更积极的内核融合，并生成更可预测的高性能代码。

PrimTorch IR 还有助于消除图中的别名和变更。只要可能，它会把操作转换为不执行原地更新的形式，因为原地更新会使优化复杂化。PrimTorch IR pass 的输出是只包含 `aten` IR 和 PrimTorch IR 操作的 FX Graph，随后即可由后端降低。

通过这种 IR 标准化，PrimTorch IR 为编译器后端提供稳定、简化的目标接口。TorchInductor 等后端无需实现数千个 PyTorch 操作，只需支持这约 250 个原语；其他高级操作由原语派生，复杂度因此大幅降低。

由于大多数高级操作（如 2,000 多个 PyTorch 操作）都能分解为现有原语，即使 PyTorch 为新算法、模型和技术增加操作，PrimTorch IR 原语集合的演进速度也相对较慢。

> 例如，要支持新的加速器，稳定的 PrimTorch IR 接口意味着开发者只需实现约 250 个原语，而不是数千个 ATen 操作；具体数量和接口仍以目标版本为准。

总结目前的流水线：TorchDynamo 捕获图，AOT Autograd 添加反向传播并形成联合前向—反向图，PrimTorch IR 再把操作规范化为更精简的原语集合。此时，整个计算（包括前向和反向）已经用核心操作表示为较标准、与设备无关的形式。下面转向编译器后端实际执行的代码生成阶段。

TorchInductor 后端代码生成

`torch.compile` 编译栈的最后阶段是编译器后端。TorchInductor（简称 Inductor）是 PyTorch 的默认编译后端。它接收由 `aten` + PrimTorch IR 操作组成的优化联合前向/反向

FX Graph，并为 NVIDIA GPU、AMD GPU、CPU 等目标硬件生成高性能代码。

> XLA 是面向非 CUDA 硬件的另一种后端，主要通过 OpenXLA 项目用于 Google Cloud TPU，也可支持采用 XLA IR 的其他加速器。例如，Meta 的内部推理 ASIC——Meta Training & Inference Accelerator（MTIA）——使用 XLA；AWS 的 Inferentia 和 Trainium 定制加速芯片也可通过开源 AWS Neuron SDK 使用 XLA 编译器运行 PyTorch。不过，NVIDIA GPU 通常使用 TorchInductor。

TorchInductor 会将图 *lowering* 为循环级、define-by-run 风格的 IR，再编译成高效代码。内部表示把操作视为对多维数据的循环；在可能时，它会将 FX Graph 节点自动分组为融合循环块，代码生成时每个分组通常对应一个内核。

Inductor 还支持符号形状，从而支持动态维度。该 IR 比原始 CUDA 更高层，把逐元素计算等表示为遍历张量索引的循环；用户可以检查 IR，这有助于调试和扩展。

TorchInductor IR 还可用于 `torch.export()` 与 AOTInductor 的提前编译流程。AOTInductor 会编译 `torch.export` 生成的产物，用于打包和部署等 AOT 场景，从而可以保存并跨运行复用编译代码。图 14-4在 `torch.compile()` 语境下突出展示了 export。

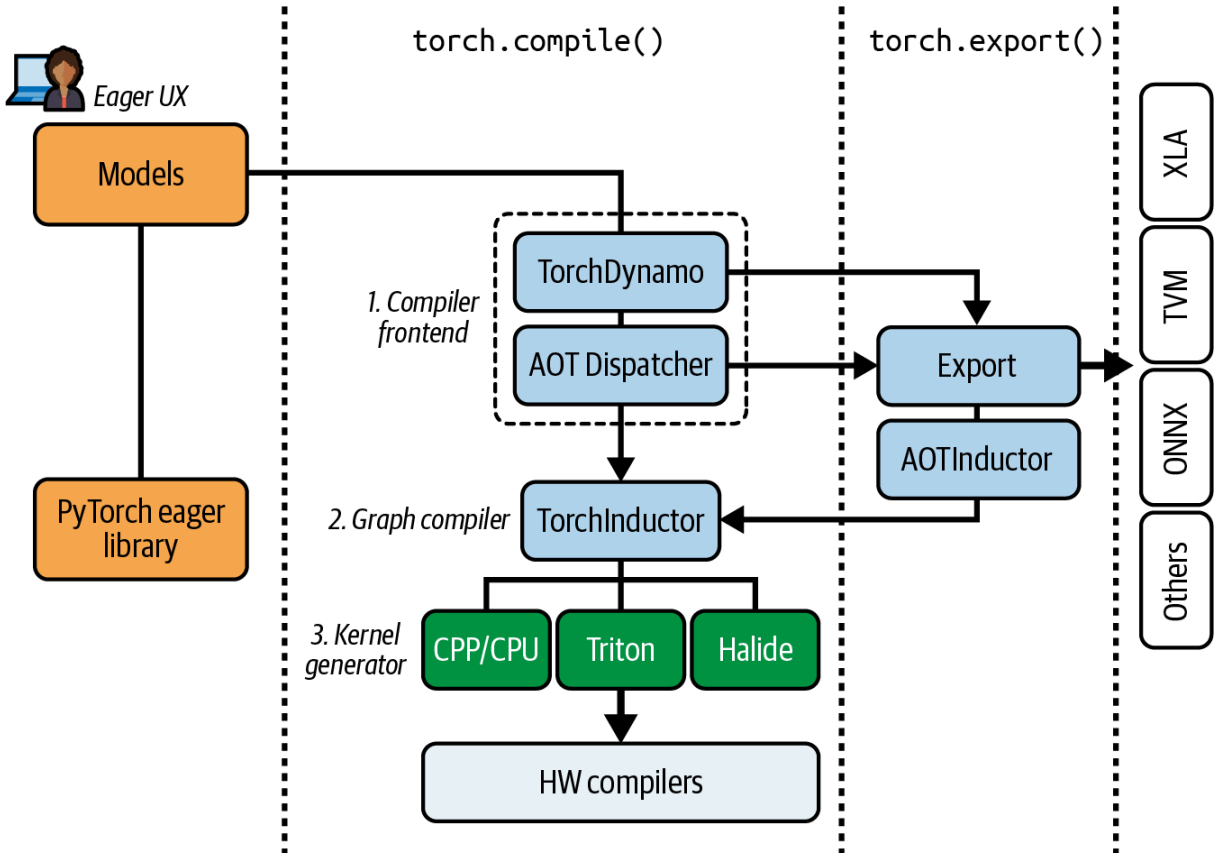


图 14-4　PyTorch 编译与导出（TorchDynamo → AOT Autograd → PrimTorch IR → TorchInductor → Triton/LLVM NVPTX）；通过 `torch.export`/AOTInductor 导出；形状静态时可使用 CUDA Graph

对于 NVIDIA GPU 后端，TorchInductor 使用 OpenAI Triton JIT 编译器生成实际 GPU 内核。Triton 是用 Python 编写、类似 CUDA 的领域专用语言（DSL），并包含该 DSL 的编译器，稍后会详细介绍。

TorchInductor 将循环级 IR 转换为 Triton 代码，再使用 Triton 编译器和 LLVM 将其直接转换为 NVIDIA PTX。请记住，PTX 是 NVIDIA GPU 的低层指令集架构（ISA）。

> 重要的是，Triton 使用 LLVM NVPTX 降低到 NVIDIA PTX，并不调用 NVCC 编译内核。这样 TorchInductor 可以动态生成针对特定模型或算法定制的内核；具体编译链仍可能随版本和后端配置变化。

循环级 IR 用 Python 实现，便于检查和扩展。例如，假设图中有操作 `z = x.permute(1,0) + x[2,:]`，Inductor 可能用以下 IR 表示：

```
def inner_fn(index: List[sympy.Expr]):
    i1, i0 = index  # index variables for dims
    tmp0 = ops.load("x", i1 + i0*size1)         # x[i1, i0]
    tmp1 = ops.load("x", 2*size1 + i0)          # x[2, i0]
    return ops.add(tmp0, tmp1)                 # elementwise add

torchinductor.ir.Pointwise(
    device=torch.device("cuda"), dtype=torch.float32,
    inner_fn=inner_fn, ranges=[size0, size1]
)
```

这里，`size0` 和 `size1` 是输入 `x` 的维度，`inner_fn` 描述如何计算一个输出元素。`Pointwise` 节点表示遍历这些范围的循环嵌套，对每个元素应用 `inner_fn` 生成输出。

这是 define-by-run 风格的 IR。执行该 IR 时，实际上会运行遍历并调用 `ops.load` + `ops.add` 的 Python；随后 Inductor 使用 Triton JIT 编译器和 LLVM 生成对应 NVIDIA PTX 代码。

> 可以将 `torch.library.wrap_triton` 与 `triton_op` 结合，用于注册带 autograd 和 fake tensor 支持的一等 PyTorch 操作。这样编写的 Triton 内核就能作为模型图的一部分由 TorchInductor 优化；可用性依赖目标版本的 API 支持。

使用 TorchInductor 自动调优

TorchInductor 内置基于 Triton 自动调优能力的自动调优器，后文会介绍 Triton 的相关能力。调优器为每个生成的 GPU 内核寻找合适的启动配置，并按内核缓存结果，后续运行无需重复调优。

首次使用 TorchInductor 编译代码时，它会额外基准测试不同 block 大小、tile 大小等配置的内核变体，并选择较快者供后续使用。内核自动调优会增加初始编译延迟，但可能改善稳态运行性能；收益依模型和硬件而异。

> 上一章提到的激进自动调优，对应 `max-autotune` 编译模式。它通常也是最耗时的编译模式，其内部正进行上述调优。

除了内核融合和自动调优，TorchInductor 还应用多种低层优化，包括简化索引以减少循环中的复杂索引算术、对生成代码执行公共子表达式消除，以及通过高效内存规划复用缓冲区并减少分配。

TorchInductor 还可使用 CUDA Graph 在运行时捕获内核序列，以极低 CPU 开销加速图重放。满足捕获条件时，Inductor 可能尝试把生成内核包装进 CUDA Graph，减少每次迭代的启动开销；这对推理或包含大量内核的模型尤其可能有益。

> 第 13 章介绍的 `reduce-overhead` 和 `max-autotune` 模式可能启用 CUDA Graph，但实际行为依版本和捕获条件而定。CUDA Graph 通常要求地址和执行形状稳定；启用动态形状时可能无法使用。需要自动调优但不捕获 CUDA Graph 时，可以考虑 `max-autotune-no-cudagraphs`（若目标版本支持）。
>
> 一般可从默认模式开始，再对大型或关键工作负载评估 `max-autotune`；它可能显著增加编译时间，小模型未必有明显收益。

TorchInductor 最终会为工作负载生成高度优化、面向设备的代码。在某些情况下，Inductor 的性能可接近手工调优库，但是否超过库实现必须通过基准验证。例如，它可以把包含激活和逐元素变换的一系列操作融合到单个内核，也可能把矩阵乘法后接 bias-add + activation 等模式融合到一次启动中；手工实现这类融合较困难且需要持续维护。

PyTorch 编译器使用启发式规则和多个后端，可能把大 GEMM 等操作路由到 cuBLAS/cuBLASLt 或 CUTLASS 等高性能库，也能为可融合模式生成 Triton 内核。实际选择和缓存结果取决于形状、后端可用性与调优测量。

对于 Transformer 模型，TorchInductor 可能融合大 GEMM 周围的 layernorm 和残差连接逐元素操作，同时仍使用 cuBLAS 执行实际 GEMM。对于内存访问不规则的模型，Inductor 的定制 Triton 内核有时也能通过只执行所需工作而超过通用库内核；这些结果都需要按目标模型实测。

在现代 GPU 上，PyTorch 编译器可以与 NVIDIA Transformer Engine（TE）配合处理某些 Transformer 块和层。不过，调用 `torch.compile` 并不会自动把普通 PyTorch 操作替换为 NVIDIA TE 内核。TE 是独立库，必须显式调用其模块或融合操作；调用 TE API 后，`torch.compile` 可以编译并融合其周围代码，与生成的 Triton 内核互补。

> 只有计划在模型中直接调用 TE 模块时，才需要安装 NVIDIA Transformer Engine，例如 `transformer_engine.pytorch.layers`。`torch.compile` 不会自动把 TE 内核替换进普通 PyTorch 模型。

本质上，TorchInductor 负责把代码和模型转换为高性能 GPU 内核。随着 PyTorch 更新，硬件覆盖面扩大，新的优化技术不断出现。例如，PyTorch 提供 FlexAttention，这是一种可由 TorchInductor 编译为融合内核的新注意力操作。在特定基准中，FlexAttention 融合内核的前向和反向性能最高可达到现代 FlashAttention 的约 85%–90%，同时支持块稀疏和自定义掩码等更灵活功能。启用快速路径可按目标版本设置相应 CUDA SDPA 后端选项：

```
# Ensure SDPA fast paths are enabled
torch.backends.cuda.enable_flash_sdp(True)
torch.backends.cuda.enable_math_sdp(True)
torch.backends.cuda.enable_mem_efficient_sdp(True)
```

使用 `torch.nn.attention.flex_attention` 时，应确认输入满足快速路径约束，这样 TorchInductor 才可能生成融合 Triton 内核。

TorchInductor 和 Triton 支持在现代 GPU 上自动进行 warp specialization；编译器会在判断有益时选择性启用。可通过 `num_consumer_groups`、`num_buffers_warp_spec` 等 Triton 元参数调节。它们可能改善 GEMM 吞吐量。Triton 的自动 warp specialization 在包括 Blackwell（如 `tcgen05`）在内的现代 GPU 目标上支持 TMA 和张量描述符 API，具体能力依版本和架构而定。

> 建议使用基于描述符的分块 load/store 映射到 TMA，以降低寄存器压力；在适用时，这通常优于手写 `tl.load` 循环。

简而言之，许多模型使用 `torch.compile` 后会更快，但具体收益取决于模型特征。首次运行需要承担编译和自动调优成本；后续运行可复用缓存图和内核，减少启动开销。

动态形状与可变序列长度

LLM 训练和推理的一大挑战是序列输入长度不同。传统编译器和加速器中，变化形状常会触发重编译，或要求把输入填充到固定的公共长度。本节介绍 `torch.compile` 如何跟踪动态形状，以处理可变长度序列。

PyTorch 编译栈支持动态形状。它可以使用 SymPy 库以符号方式表示未知维度，使模型在一定范围内接受不同输入，而不必每次都重编译；具体效果取决于操作和版本。

观察到维度大小变化时，PyTorch 编译器可能自动将其标为动态。TorchInductor 通常先基于静态假设编译，检测到形状变化后再通过重编译泛化。预先设置 `dynamic=True` 会让编译器从一开始就按动态维度处理，但通常会限制 CUDA Graph；更建议使用 `torch._dynamo.mark_dynamic()` 只标记已知会变化的维度。

TorchDynamo 和 TorchInductor 在跟踪动态维度时会插入类似 `sequence_length <= 256`（或你指定的范围）的守卫（guard），以生成适用于多种尺寸范围的代码。例如，若输出大小为 `x.size(0) + y.size(0)`，Inductor 可以将其表示为符号表达式，并确保生成的代码适用于满足守卫条件的任意值。

Dynamo 遇到新的 `sequence_length` 形状时，可能设置如 `sequence_length <= 1024` 的新 guard，并在该假设下编译内核；若之后更长序列违反 guard，编译器会为更大范围重新编译图。随着运行推进，可能为不同形状范围建立编译内核缓存。

还可以使用 `torch._dynamo.mark_dynamic(tensor, dim)` 在张量上手动标记预期的动态维度，以提前避免重编译。也可以使用 `torch.compiler.set_stance()` 调整处理重编译的方

式；例如，采用 eager-on-recompile 策略，在发生一定次数重编译后回退到即时模式。稍后将讨论避免重编译的最佳实践。

TorchInductor 会在首次重编译后尝试泛化形状，而非针对每个新形状反复专门化。例如，它会在生成的内核中使用 `if` 语句发出条件代码，使一个内核能够适用于一系列 `sequence_length` 而不报错，从而减少为每个尺寸单独编译的需要。

> 具有数据依赖输出秩的某些操作，或极其复杂的索引，仍可能触发形状专门化。在这些情况下，编译器会插入更多守卫；若它们经常被违反，即使使用 `mark_dynamic()` 或 `set_stance()`，也可能看到频繁重编译。

如果不支持动态形状，一个简单但低效的办法是将批次内所有序列填充到最大长度，以便所有输入使用同一静态计算。实现虽简单，但长度差异大时会在无意义的 padding token 上浪费大量计算。

若最大长度远大于所有输入的平均长度，填充会损害 GPU 利用率。然而采用动态形状编译后，可让编译器生成只迭代至每个输入实际序列长度的代码。动态形状能够避免可变长度的过度填充。

以一个典型的文本生成式 AI 场景为例：序列长度会随生成过程不断增长。使用动态形状编译在序列长度增加时仍可能持续优于即时执行，但实际结果应以模型、硬件和版本上的测量为准。

相比之下，若为使用静态形状而将所有内容填充到 2 的幂长度，会引入大量无效计算，并因张量尺寸变大而增加编译时间。换言之，使用动态形状可能带来更好的编译期与运行期性能，也更易使用，因为无需手动填充输入；这仍取决于具体模型和形状分布。

> 建议按大小将输入分桶，以限制不同形状的数量，再用动态形状处理其余变化。这种混合方法既可避免过多重编译，也能减少填充浪费。

借助动态形状，可以一次编译，并在不同形状的输入上使用同一个已编译模型。只要变化落在受支持范围内，一个已编译模型即可处理多个配置。

在内部，TorchInductor 使用SymPy 库以符号方式表示动态维度。它会在 IR 中传播这些符号，使 `z.size(0) = x.size(0) + y.size(0)` 这样的表达式可以被符号化处理；Inductor 会将条件归约为守卫表达式。

如果维度超出预期范围，或数据依赖条件改变导致守卫失败，Inductor 就会触发重编译。实质上，TorchInductor 会尝试为一段尺寸范围编译通用内核，而不是只针对单一固定尺寸编译。

动态形状在近期版本中已有显著改进。但若编译器无法以符号方式处理某些操作，它们仍可能强制形状专门化。此时编译器可能插入更多守卫，若经常违反，就会导致频繁重编译并抵消编译收益。

数据依赖的控制流仍会触发专门化。动态形状适用于变化的序列长度，却不适合真正由数据决定的分支。

值得注意的是，截至本书写作时，CUDA Graph 重放要求静态形状和固定内存地址；实例化图只支持有限的参数更新，内存地址和内核启动拓扑必须与捕获兼容。因此，启用动态形状通常会禁用这些区域的图捕获，使编译器无法获得 CUDA Graph 的性能收益（包括降低内核启动开销）。行为会随 PyTorch/CUDA 版本变化。

> 若指定 `reduce-overhead` 编译模式又设置 `dynamic=True`，由于形状可变，`reduce-overhead` 的 CUDA Graph 优化不会适用。启用动态形状会改变守卫和内存规划，从而禁用图捕获。实践中，应只在形状稳定时使用 `mode="reduce-overhead"` 获取 CUDA Graph；对于可变序列长度，可优先考虑 `mode="default"` 或 `mode="max-autotune-no-cudagraphs"`，并在 ±10–20% 范围内分桶/填充，以限制重编译。

应对系统进行性能分析，判断动态形状是否适合实际场景。有时固定填充长度、使用静态形状和 CUDA Graph，反而能避免每种长度重编译并获得更高吞吐；另一些场景则动态形状更好。

应对不同方法进行性能分析，找出最适合自身工作负载的方案，同时务必监控内存使用量。支持动态形状的代码因最大范围所需的额外守卫和通用代码，可能具有略高的内存占用。

> 经验法则是：如果序列长度只变化 10%–20%，固定长度填充可能更有利；仍应以实际基准结果为准。

简而言之，动态形状支持意味着面对 LLM 常见的可变长度输入时，不必直接禁用 `torch.compile`。在支持的操作和形状范围内，PyTorch 编译器可以跨不同输入大小执行内核融合和其他优化。

禁用 PyTorch 编译器并回退到 eager 模式

如果希望在不改动代码结构的情况下完全禁用 `torch.compile`（这对 A/B 性能测试和隔离问题很有用），可以使用 `@torch.compiler.disable` 装饰器，禁用某个函数的编译。需要区域级控制时，可将 `torch.compiler.set_stance()` 用作上下文管理器，强制代码以 eager 模式运行。例如，可以禁用复杂数据加载或一次性初始化逻辑的编译，让编译图聚焦于计算；对于不适合跟踪的代码也很有用。

也可以改用 eager 后端：`torch.compile(model, backend="eager")`。这样代码仍经过编译器前端但使用 eager 后端执行，便于调试并比较编译与 eager 模式的正确性和性能；若要完全移除编译包装，应直接调用原模型。

当某些操作无法与 PyTorch 编译配合，或出于性能原因不希望它们进入图时，`torch.compiler.disable()` 和 `torch.compiler.set_stance()` 都是有价值的退出机制。下面利用 PyTorch 编译器日志探讨改善编译图和生成代码性能的方法。

性能提示与调试生成代码

另一个很有用的日志选项是 `TORCH_LOGS="perf_hints"`。日志会显示错过的性能优化机会，例如某模式无法融合，或因输入被修改而无法使用 CUDA Graph；可能出现“*PerfHint: CUDA Graph not used because input is mutated*”“*PerfHint: fell back to eager for random op*”等提示，用于定位限制代码或模型性能的因素。

要进行更深入的性能调试和调优，通常需要查看 TorchInductor 生成的确切代码。可以设置 `TORCH_LOGS="output_code"`，打印每个编译图的生成代码，包括内核原始源码；必要时可据此分析并进一步优化，但不应直接修改临时生成文件作为稳定方案。

还可以设置 `TORCH_COMPILE_DEBUG=1` 启用 TorchInductor 调试模式。运行程序时，Inductor 会创建调试目录（如 `/tmp/torchinductor_<pid>/...`），其中包含 FX Graph（`.fx`）、`outputcode.py`、*fx_graph_runnable.py*、IR dump 和生成的 Triton 源码等产物。

查看生成的 `.triton` 代码时，可能会看到 Triton 特有结构，甚至在高级场景中看到原始 PTX。检查调试产物中的已编译 PTX 时，可能看到 `mma.sync` 指令；这表示 `tl.dot` 被降低为 Tensor Core 操作。日志、工具和产物有助于确认编译器是否应用了内核融合、warp specialization 或双缓冲等优化。发现低效之处后，可以针对具体场景手写 Triton 内核，但需通过基准测试验证。

> 如果自定义内核具有普适价值，也可以将其贡献回 PyTorch 或 Triton 生态，让其他人受益。

调试数值正确性与准确率

虽然并不常见，但 `torch.compile` 可能产生与 eager 模式数值不同的结果。怀疑编译器问题时，应先验证并收集数据，再向社区反馈或创建 GitHub issue。

首先，可以使用 PyTorch 的 minifier 工具生成可复现脚本。PyTorch 提供 TorchDynamo minifier 和 TorchInductor minifier，会尝试把程序缩减到仍能复现错误的最小版本。这样可以为 PyTorch 团队提供小型可复现脚本，必要时将其附加到 GitHub issue。

还可以配置 TorchDynamo，在编译栈各层调试数值准确性。为确定差异从哪一阶段引入，可在编译时设置以下环境变量，将 eager 模式与不同编译阶段比较，判断问题来自 TorchDynamo、AOT Autograd 还是 TorchInductor：

```
# Dump the outputs after each compilation stage
TORCHDYNAMO_REPRO_AFTER="aot"
TORCHDYNAMO_REPRO_LEVEL=4
```

这些设置会让 TorchDynamo 在各阶段后 dump 图，并以 eager 模式运行各图进行比较，有助于定位引入错误的阶段。

设置 `TORCHDYNAMO_REPRO_AFTER="aot"` 会让 TorchDynamo 在 AOT Autograd 阶段后 dump FX Graph，并触发生成错误复现脚本的逻辑；这不同于在最初 Dynamo 捕获后生

成脚本。

使用 `TORCHDYNAMO_REPRO_LEVEL=4` 时，TorchDynamo 会在 eager 模式运行每个 dump 图，并将输出与编译版本比较；发现数值不匹配时，会停止并保存最小复现脚本。

> 如果确认存在真正的正确性错误，欢迎在 GitHub 报告。请通过设置 `TORCHDYNAMO_REPRO_AFTER="aot"` 和 `TORCHDYNAMO_REPRO_LEVEL=4`，附上精简后的可复现产物。

使用随机数（种子）或随机序列时，应确保生成方式一致。默认情况下，TorchInductor 可能不会产生与 eager 模式完全相同的随机种子或序列，因为融合或重排内核的随机数生成顺序可能不同。

必要时可设置 `torch._inductor.config.fallback_random=True`，强制 TorchInductor 采用更接近 eager 的随机数生成路径。这可能略微降低性能，但在需要数值一致性时有帮助；该配置属于内部接口，使用前应核对版本。

数值差异也可能来自浮点精度。例如使用 PyTorch 自动混合精度（AMP）或 BF16 时，融合内核的操作顺序可能与 eager 的未融合序列略有不同。

这类差异通常不影响收敛，但某些情况下会影响。如果怀疑精度导致不稳定，可禁用 `torch.compile`，以完整 FP32 运行模型来隔离问题。还可以使用 `torch.set_float32_matmul_precision('highest')` 控制 TF32 使用，在 FP32 `matmuls` 的准确率与性能之间取舍。

还要认识到，混合精度（如 FP16/BF16）可能产生小幅差异。设置 `torch.use_deterministic_algorithms(True)` 可要求确定性行为；使用非确定性操作时 PyTorch 会抛出错误。虽然 `torch.compile` 的某些路径会减少不确定性来源，调试期间仍可启用该标志。

但并非所有操作都有确定性实现。例如依赖 cuBLAS 的默认 `torch.matmul()` 可能没有确定性实现。

具体而言，cuBLAS 可能使用 split-K 等并行优化，以不同顺序执行归约，因此不同运行之间的浮点结果可能无法逐位复现。

因此，启用确定性要求可能使代码在没有 fallback 时失败。对于 `torch.matmul()` 等依赖 cuBLAS 的操作，应调用 `torch.use_deterministic_algorithms(True)`，并按文档设置固定的 `CUBLAS_WORKSPACE_CONFIG`：

```
# Set this before starting the Python/PyTorch process
export CUBLAS_WORKSPACE_CONFIG=:4096:8   # or :16:8

# Use this with the PyTorch process
torch.use_deterministic_algorithms(True)
```

这里，第一个值（如 `4096` 或 `16`）选择 cuBLAS workspace 缓冲区大小；第二个值（如 `8`）选择预留的缓冲区数量。应按目标 CUDA/cuBLAS 版本文档设置支持的值，如 `:4096:8` 或 `:16:8`，不能假定所有版本都接受相同配置。

若要在 `torch.use_deterministic_algorithms(True)` 下要求 cuBLAS 使用确定性算法，应按文档将 `CUBLAS_WORKSPACE_CONFIG` 设置为支持的值，如 `:4096:8` 或 `:16:8`。未设置时，对于本会选择非确定性 cuBLAS 算法的操作，PyTorch 可能在运行时抛错。

> 始终应在实际硬件和模型配置上测试确定性，以确认可复现性。

对于关键工作负载，还可以临时禁用部分编译器优化，例如设置 `torch._inductor.config.triton.cudagraphs=False`，以更好地隔离差异原因；这会禁用 TorchInductor 生成 Triton 内核的 CUDA Graph 捕获。该配置是内部接口，需按版本确认。

调试 PyTorch 编译器优化需要稍微改变思路：除了低层生成代码，还要通过日志和图可视化观察元层执行步骤。`torch._dynamo.explain()` 等工具提供代码如何转换为图、graph break 和子图的高层视图；各种 `TORCH_LOGS` 选项则能查看编译器决策及生成的确切代码。

简而言之，结合这些工具和调试机制，可以逐步减少不必要的 graph break，并确认模型和代码得到充分捕获与优化。付出调试成本通常是值得的，但编译模型能否显著超过 eager，仍取决于模型和配置；大型 LLM 中的小幅收益可能累积。

解释并减少 Graph Break

使用 `torch.compile` 时，诊断性能和正确性需要专门工具。本节介绍如何使用工具和最佳实践调试并定位过多的 graph break，包括 `torch._dynamo.explain()`、记录编译器决策的环境变量，以及调试捕获图和生成内核的方法。

Graph Break 与 TorchDynamo explain()

当 TorchDynamo无法继续把连续操作捕获到单个图时，就会发生 graph break。此时，这部分代码退回 eager 执行。

Graph break 通常会损害性能，因为每次 break 都会截断优化图并增加 Python 开销。模型编译后加速有限，可能是频繁 graph break 阻碍了大型融合图。应尽量减少不必要的 break，但通信、动态控制流等边界可能是有意且必要的，不能盲目追求整个模型只有一张图。

包含集合通信（如 all-gather、reduce-scatter）的复杂图常需要在通信边界分段。图 14-5展示了 PyTorch FSDP 因集合通信产生的 graph break。

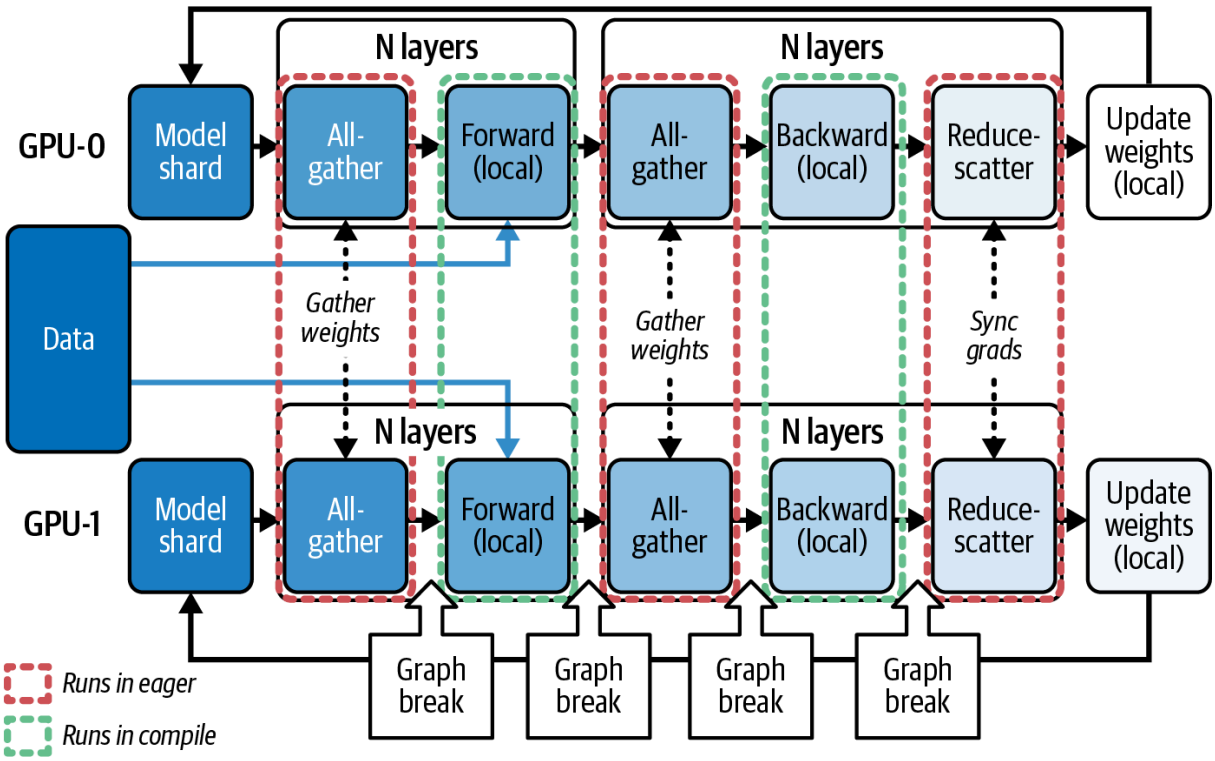


图 14-5　PyTorch FSDP 因通信层产生的 Graph Break（来源：https://oreil.ly/TJW42）

PyTorch 提供 `torch._dynamo.explain()`来分析和调试 graph break。使用模型和示例输入调用它时，会在 TorchDynamo 中运行模型，并报告生成了多少张图、break 发生位置及原因；示例如下，后面是详细分析：

```
import torch._dynamo as dynamo

def toy_example(a, b):
    x = a / (torch.abs(a) + 1)
    print("woo")          # a print statement in the model
    if b.sum() < 0:       # dynamic control flow depending on data
        b = -b
    return x * b

explanation = dynamo.explain(toy_example)(torch.randn(10), torch.randn(10))
print(explanation)
Graph Count: 3
Graph Break Count: 2
Op Count: 5

Break Reasons:

  Break Reason 1:
    Reason: builtin: print [...ConstantVariable] False
    User Stack:
      <frame at toy_example: line 3, in toy_example>

  Break Reason 2:
    Reason: generic_jump TensorVariable()
    User Stack:
      <frame at toy_example: line 5, in toy_example>

Ops per Graph:
  ...
```

这里的报告显示 TorchDynamo 在两个 graph break 处分成三个图片段。“User Stack”部分指出发生问题的具体代码行，有助于定位导致 graph break 的代码。

第一个 break 由第 3 行附近的 `print("woo")` 引起。`print()` 会向 `stdio` 写文本，具有副作用，不能直接捕获。因此 Dynamo 将 `print()` 前后的代码分成两张图。

第二个 graph break 由第 5 行附近的动态控制流 `if b.sum() < 0:` 引起。该条件依赖数据值，Dynamo 在此场景下不能将其放入单个图；前文已说明这是当前限制。

如果 PyTorch 编译器性能不及预期，应尽早对模型使用代表性输入运行 `dynamo.explain()`。它能快速显示生成了多少张图，以及为何无法形成一张大图。

理解原因后，可以重构代码，逐个处理 graph break。在前面的示例中，可移除 `print()`，或如下面所示将其包装在 `if not torch._dynamo.is_compiling()` 等守卫中，以避免跟踪期间执行它：

```
import torch

def model(a, b):
    x = a / (torch.abs(a) + 1)

    # avoid during compiling/tracing
    if not torch._dynamo.is_compiling():
        print("do not print during tracing/compiling")

    if b.sum() < 0:
        b = -b
    return x * b

explanation = dynamo.explain(model)(torch.randn(10),
  torch.randn(10))
print(explanation)
```

如前所述，若模型确实需要数据依赖分支，可将其包装在 `torch.cond()` 中。这样会将“真”和“假”两个分支都捕获为图子例程，如下所示：

```
import torch

def model_cond(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    # Compute x as before
    x = a / (torch.abs(a) + 1)

    # Retain the compile-time check as a
    #   Python-level guard
    # Avoid side-effects during tracing/compilation
    if not torch._dynamo.is_compiling():
        print("do not print during tracing/compiling")

    # Handle the data-dependent sign flip on b
    b = torch.cond(
        b.sum() < 0,  # predicate (0-dim bool tensor)
        lambda b: -b, # true_fn: flip sign
        lambda b: b.clone(), # false_fn: leave unchanged (clone() avoids aliasing)
        (b,)          # operands tuple
    )

    return x * b
```

```
# Generate and print the Dynamo explanation just like before
explanation = dynamo.explain(model_cond)(torch.randn(10), torch.randn(10))
print(explanation)
```

这里，谓词 `b.sum() < 0` 必须是 Python `bool` 或单元素 `torch.bool` 张量。`true_fn` 和 `false_fn` 是接收相同操作数（这里仅为 `(b,)`）并返回相同形状、相同 dtype 张量的可调用对象。

这段代码将 Dynamo 编译期检查（`dynamo.is_compiling()`）保留为 Python `if`，因为它在运行时不依赖数据，而且我们希望避免跟踪期间出现 `print` 等副作用。

请注意，`torch.cond()` 目前只接受张量谓词；两个分支必须具有相同的输入，并返回形状和 dtype 均相同的单个张量，且不允许原地变更或任意副作用。

相对地，可以如前所述采用 `torch.where()` 的纯张量掩码方法。它没有这些限制，并能避免 graph break；当不需要 `torch.cond()` 的完整表达能力时，这是更简单、可靠的选择。代码如下：

```
import torch
import torch._dynamo as dynamo

def model_where(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    # Compute x as before
    x = a / (torch.abs(a) + 1)

    # Preserve compile-time guard to avoid side-effects during tracing
    if not torch._dynamo.is_compiling():
    print("do not print during tracing/compiling")

    # Data-dependent branch expressed using torch.where
    b = torch.where(
        b.sum() < 0,  # predicate: a 0-dim bool tensor
        -b,           # true branch: flip sign
        b             # false branch: unchanged
    )

    return x * b

# Display the Dynamo explanation just as before
explanation = dynamo.explain(model_where)(torch.randn(10), torch.randn(10))
print(explanation)
```

这里，`torch.where(condition, input, other)` 会在 `condition` 为 `True` 的位置选择 `input` 中的元素，在其为 `False` 的位置选择 `other` 中的元素并返回张量。由于 `b.sum() < 0` 生成零维布尔张量，它可广播到 `b` 的所有元素；因此可通过一次向量化符号翻转，替代逐元素的 Python `if`。

> 在编译与跟踪流水线中使用 `torch.where()` 可避免 graph break，使 TorchDynamo 能内联优化这些操作。

还可使用 `torch.compiler.set_stance("fail_on_recompile")`：若代码无法干净地捕获为完整图，它会强制报错并拒绝运行。这在开发期间很有用，因为可以在编译时提前发现

graph break，而非静默回退到较慢的 PyTorch eager 执行。

> 也适合在 CI 构建中加入 `torch.compiler.set_stance("fail_on_recompile")`，以捕获后续开发过程中新引入的 graph break。在项目全生命周期中，健壮且持续的性能回归测试都极为重要。

减少图重编译

除 graph break 外，还应监控重编译次数。如果 guard 持续判定输入张量形状等条件失效，TorchDynamo 可能会反复编译图。张量形状在运行时改变时，guard 会失败并触发重编译。若重编译次数超出预期，应找出导致问题的 guard（形状、dtype 等）并处理它。

如果初始预热/编译后迭代仍持续缓慢，通常说明发生了重编译。可以设置 `TORCH_LOGS="graph_breaks,recompiles,guards"`，让 PyTorch 记录 guard 求值及触发重编译的原因。

频繁 guard 失败常表示 Python 侧常量（如随机种子、时间戳或随循环变化的值）每次迭代都在改变，持续使 guard 失效并触发重编译。应让这些值保持静态，或使用前述动态形状 API（如 `torch._dynamo.mark_dynamic`）处理，以避免不必要的重编译。

可根据情况采用几种常见机制减少图重编译。首先，对于刚才所说的常量情形，可将常量作为张量传入代码块，避免编译器对该值设置 guard 并反复失败。

其次，如前所述，可以用 `torch._dynamo.mark_dynamic(tensor, dim)` 标记已知会变化的动态维度，预先避免重编译。还可使用 `torch.compiler.set_stance("eager_on_recompile")`，在重编译 N 次后回退到 eager 模式，以避免反复重编译；这实际上为重编译次数设定了上限。

另一个选择是用 `torch._dynamo.allow_in_graph` 显式将图的该部分标记为安全。下一节将进一步介绍这一技巧。

使用 allow_in_graph 将函数和代码块标记为安全

例如，当函数或代码块使用了不受支持的操作，TorchDynamo 不知道如何处理时，可将 `torch._dynamo.allow_in_graph` 作为 Python 装饰器或上下文管理器来装饰函数或包裹代码，以告知 Dynamo 它没有副作用。随后 Dynamo 会以更宽松的分析和接受策略将其纳入跟踪。`allow_in_graph` 会绕过部分 Dynamo 安全检查，因此应优先修复 graph break 的根本原因。

这是高级功能，应谨慎使用。你实际上是在承诺函数是纯函数：相同输入总是返回相同输出，只依赖张量输入且没有副作用。错误使用可能静默产生错误结果；正确使用时，如果某个实际安全可跟踪的函数导致 graph break，它可能带来明显收益。

总之应少量使用 `allow_in_graph`。它适合高级用户覆盖 Dynamo 的保守策略，但前提是确定函数没有会影响正确性的副作用或隐藏状态。

处理 Graph Break 的技巧

Graph break 会限制编译器执行大规模优化，例如把许多内核融合为少数高效内核，并使部分图退回较慢的 eager 执行。

理解 graph break 的触发原因及预防方法很重要。下面列出常见原因和减少方法：

避免原地操作和意外变更

TorchDynamo 可通过称为*函数化（functionalization）*的机制处理某些变更：它会将原地操作转换为用于跟踪的非原地操作。不过，某些原地操作仍可能引发 graph break。若看到与变更有关的中断原因，例如 “mutation on data” 或 “modifying a global”，应重写该部分以避免原地操作。若问题由原地操作导致，通常只要把 `x.relu_()` 改写为非原地的 `x = x.relu()`，即可避免 graph break。

优先使用 PyTorch 数据结构、集合和张量操作，而不是等价 Python 实现

在函数内部向 Python 张量列表追加元素会让 TorchDynamo 难以处理，因为它不能很好地跟踪不断增长的列表。应尝试预分配张量，或使用 `torch.stack()` 等张量操作，而不是动态构造 Python（非 PyTorch）列表。调用许多 Python 库，包括 I/O 操作、`print`、`logging` 和 `math.*` 函数，很可能会导致 graph break；建议将它们从性能关键路径中移除。

> 在可行时，始终建议使用 Python 数据结构、集合及张量操作的 PyTorch 等价实现。它们针对 PyTorch 编译、GPU 处理和分布式数据传输进行了大量优化，这些能力在基于 PyTorch 的 AI 应用和模型中很常见。

尽可能避免依赖数据的控制流

若存在 `if tensor.sum() > 0:` 形式的逻辑，TorchDynamo 不易穿透跟踪，因为条件在编译时未知。它必须根据首次运行选择一个分支、为该条件设置 guard，并在后续调用中强制该 guard；这显然不正确，因此 Dynamo 会创建 graph break。

PyTorch 提供名为 `torch.cond()` 的高级操作，可在图中捕获某些动态流程。它可封装 `if/else` 语句，使两个分支都得到编译；但条件必须是张量，通常更适合依赖参数的切换，而非任意 Python 逻辑。

除此之外，大多数依赖数据的控制流仍会打断图。可行时仍应优先使用张量操作（`torch.where()`、掩码等）。若既不能使用 `torch.cond()`，也无法重构，则可能必须接受 graph break 及其性能影响。

理解 PyTorch DDP 子图重叠与同步的性能特征

PyTorch 的 DDP 会与 TorchDynamo 协作，在同步点（包括 all-reduce bucket）显式打断图。在 explain 输出中可能看到与 `allreduce` 或 `torch.distributed` 操作有关的中断。这是预期行为：PyTorch 可分别编译各梯度 bucket 的归约，以使通信能与反向计算重叠。

若要保留计算与通信重叠，无法避免 DDP 通信边界处的 graph break。PyTorch 编译器和 DDP 会有意在每个 all-reduce bucket 处插入中断，使梯度同步发生在子图之间；这样一个 bucket 的通信便能与下一 bucket 的反向计算重叠。

尽管这会阻止形成单一的大型图，却能保留性能。TorchDynamo + DDP 的性能与 eager 模式 DDP 相近，并且在大规模场景下甚至可能更快。因此，这些通信 graph break 虽无法消除，却是以适当重叠实现正确、高效分布式训练所必需的。

使用 PyTorch FSDP 封装图子模块

PyTorch 通过 `use_original_params=True` 支持编译模式下的 FSDP。最佳实践是将每个 Transformer block 等子模块分别封装为自己的 FSDP 子模块。Dynamo 会在每个 FSDP 子模块边界创建显式 graph break，使每个分片的通信可与计算重叠，类似 DDP 所述的分桶策略。

编译后的 FSDP 使用 AOT Autograd 和 Inductor 内存规划器融合前向与反向传播，并在模型分片之间复用缓冲区。因此，每张 GPU 上仅驻留活跃参数切片及最少的中间内存缓冲区，与 DDP 或 eager 模式相比可降低峰值内存用量。

内存节省来自避免冗余梯度存储、复用中间分配，以及在分片之间让通信与计算重叠；这也使更大的模型能够装入每张 GPU。若不单独封装子模块，FSDP 会退回为将所有参数视作一个大 bucket；虽然仍可运行，但会限制内存收益和重叠潜力。因此，建议将 `torch.compile` 与按模块封装的 FSDP 结合使用，以获得最高速度和内存效率，特别是在大规模训练作业中。

> 如果出现问题，调试可能非常复杂；在更大的集群/配置中则更加复杂。使用 FSDP 时，始终先在更小的配置中测试，并使用 `torch.compile`。

监控自定义及第三方 CUDA C++/Triton 操作的性能权衡

若依赖 PyTorch 不了解的自定义或第三方 CUDA 扩展，Dynamo 会创建 graph break，因为它无法推断该操作的行为或安全性。若该操作处于性能关键路径，可考虑用 Triton 在 Python 中重写该自定义操作。

PyTorch 支持 `torch.library.triton_op()` API，可将 Triton 内核无缝集成为 PyTorch 自定义操作，使编译器能够查看 Triton 代码内部以进行优化。在深入 Triton 前，先概述如何调试不同编译器阶段、graph break 与编译器性能。

> 许多流行第三方库现在都为其操作提供 Triton 实现或 Dynamo/FX 封装器。编写自己的实现前，先检查这些实现是否已存在。

调试编译器阶段、Graph Break 与性能

可以通过设置环境变量，在运行时记录并调试不同类型的编译器事件，例如 TORCH_LOGS、TORCH_COMPILE_DEBUG 和 TORCHDYNAMO_REPRO_*。这些变量可记录 graph break、重编译、guard 及其他编译器决策。下面展示 TORCH_LOGS 示例，常见值见表 14-1：

```
# "graph_breaks", "dynamo", "aot_graphs", "inductor",
# "graph_outputs", "graph_code", "dynamic", "perf_hints",
# "output_code", "recompiles", "guards", etc.
TORCH_LOGS="graph_breaks" python train.py
```

该设置会让 PyTorch 在发生 graph break 时打印信息。可以用这些选项调试 torch.compile 的各阶段、图、graph break、生成代码、性能、重编译和 guard，汇总见表 14-1。

表 14-1　torch.compile 日志选项

TORCH_LOGS value	说明
graph_breaks	记录 graph break 事件
dynamo	TorchDynamo 详细日志
aot_graphs	AOT Autograd 详细日志
inductor	TorchInductor 详细日志
graph_outputs	显示编译后的 FX 图
graph_code	dump TorchDynamo 生成的每个 FX 图的 Python 代码
dynamic	跟踪动态形状决策及维度何时被标记为动态
perf_hints	显示错过的性能优化机会
output_code	打印每个编译图的生成代码
recompiles	记录重编译触发原因
guards	记录 guard 及其求值

如果怀疑子图的切分方式或实际编译的形状存在问题，这些设置会很有用。启用后，无需修改代码即可获得大量内部调试信息。

> 请做好输出非常冗长的准备。例如，只调试 graph break 时，建议先仅启用 "graph_breaks"。

从底层看，设置 TORCH_LOGS 类似于调用 torch._logging.set_logs() API。不过，将 TORCH_LOGS 设置为环境变量，有时更便于从外部配置。

还可以设置 TORCH_COMPILE_DEBUG=1 来启用 TorchInductor 调试模式。它会记录 FX Graph、TorchInductor IR、生成的 Triton 代码；如果安装了 Graphviz，还会生成包含可视化内容的 HTML 报告。

也可以设置 TORCHDYNAMO_REPRO_AFTER 和 TORCHDYNAMO_REPRO_LEVEL，让 TorchDynamo 在每个阶段之后转储其图，并将运行结果与未编译的 eager 模式代码进行比较。

还可以使用 tlparse 工具追踪编译日志。追踪日志有助于调试编译事件（例如重编译），也可用于生成错误报告。

要启用追踪日志，请通过 TORCH_TRACE 环境变量指定 *trace-log* 目录。然后对该目录运行 tlparse，即可生成如下所示的栈帧树：

```
- /workspace/networks/layers/transformer.py:634 in forward
  .../torch/nn/modules/module.py in _wrapped_call_impl
 .../torch/nn/modules/module.py in _call_impl
  - [2/2] [2/3] ../torch/_dynamo/convert_frame.py in __call__
  - /workspace/networks/layers/transformer.py:753 in forward
    - [8/2] [8/3] .../torch/_dynamo/convert_frame.py in __call__
...
```

此外，还可以使用 Perfetto UI 显示追踪时间线。由于追踪带来的开销很小，甚至可以在生产环境中启用 TORCH_TRACE。

下面深入介绍 TorchInductor 使用的 OpenAI Triton 语言和编译器。我们将编写一些基础及高级 Triton 内核，然后把它们注册到 PyTorch 中。

使用 OpenAI Triton 编写自定义内核

到目前为止，我们只是简要提到 OpenAI 开源的 Triton 语言和编译器。现在需要深入学习，因为 TorchInductor 使用 Triton 作为后端代码生成实现，而且在 OpenAI 等大型公司的支持下，Triton 越来越流行。

如前所述，Inductor 在底层使用 Triton 生成优化的 GPU 内核。通过检查、理解并定制这些内核，可以在 TorchInductor 生成结果的基础上进一步提升性能。在 PyTorch 和 NVIDIA GPU 环境中，掌握 Triton 对性能优化至关重要。

概括来说，OpenAI Triton 是一种开源、原生基于 Python 的领域特定语言（DSL），让你可以用熟悉的 Python 编写 GPU 内核。Triton 还包含 JIT 编译器，可直接把 Triton 代码转换为 NVIDIA PTX。换言之，Triton 让你无需手写 CUDA C++，就能用 Python 创建高性能自定义 GPU 操作。Triton 与 PyTorch 紧密集成，因此成为该生态中编写自定义 GPU 内核的首选。

用 Triton 编写 GPU 内核比 CUDA C++ 更熟悉、更简单。对于希望一直使用 Python、快速迭代且不想处理复杂 C++ 模板或细粒度内存管理的研究人员，这一点尤其明显。在 PyTorch、Triton 这类专注 GPU 性能的编译器已经存在的时代，他们确实不必再使用 C++。

NVIDIA 已经注意到这一趋势。2025 年，他们宣布推出以 Python 为中心的 CUDA 库（例如 cuTile、CuTe Python DSL、CUTLASS Python DSL，以及可替代 `numpy` 的 cuPyNumeric）。这些基本上都是 Triton 的竞争性库。它们与 `torch.compile` 的集成仍在发展；截至本书撰写时，TorchInductor 仍以 Triton 作为主要 GPU 代码生成路径。

尽管 PyTorch 的 `torch.compile` 已自动完成大量内核生成工作，自定义 Triton 内核仍可榨取最后一点性能，尤其适用于复杂稀疏模式、新型层等超出 TorchInductor 当前范围的操作。如果具备领域知识，有时可以超过 TorchInductor 生成代码的性能。不过，这属于高级用法，需要持续维护，并可能为支持新硬件而重写。

下面先快速介绍 Triton 编程基础，然后讨论一些有趣的主题，包括访问共享内存、向 PyTorch 注册 Triton 内核、自动调优内核启动参数和性能分析。之后还会介绍 warp specialization、软件流水线（例如双缓冲）等高级 Triton 主题。

Triton 编程模型

Triton 使用单程序多数据（SPMD）模型，而不是 CUDA 的 SIMT 模型。这一点很重要，因为 Triton 有意隐藏了 CUDA 指令和线程的底层细节。

Triton 内核（也称为*程序*）在更高层次运行：以独立线程块（也称为*协作线程数组*，CTA）中的程序实例作为基本计算单元。CUDA 内核则运行在线程块内的单个线程上。

社区通常交替使用 Triton*内核*和 Triton*程序*这两个术语，但一般更偏好 Triton*内核*；因此本书大多数地方采用 Triton*内核*。

使用 Triton Python DSL 编写内核后，Triton JIT 编译器会将其编译为 GPU 代码，并并行运行该内核的多个实例。每个程序实例映射到一个 CUDA 线程块。

Triton 内核（也称为*程序*）通过使用 `@triton.jit` 装饰 Python 函数来定义。在内核中，使用通常别名为 `tl` 的 `triton.language` 模块中的特殊原语，处理内存指针、执行向量化加载/存储，并通过 `tl.program_id` 和块偏移运算计算每个程序的索引。

Triton 的 SPMD 模型意味着通常使用向量化操作，例如将两个 `tl.arange` 向量相加。Triton 编译器会把向量化 SPMD 代码映射到 CUDA 块中的线程上，但不保证一个元素对应一个线程。

使用 Triton 时无需显式管理单个线程或 warp，因为编译器会替你完成这些工作。下面是一个简单的 Triton 内核，用于将两个长度相同（此处为 `n_elements`）的向量相加：

```
import triton
import triton.language as tl

BLOCK_SIZE = 1024

@triton.jit
def vector_add_kernel(x_ptr,y_ptr,out_ptr,n_elements,BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)             # unique program ID for each block
```

```
    block_start = pid * BLOCK_SIZE
    # each program handles BLOCK_SIZE elements
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    # Create a mask to guard against out-of-bounds
    # (if n is not divisible by BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)        # masked load
    y = tl.load(y_ptr + offsets, mask=mask)
    result = x + y
    tl.store(out_ptr + offsets, result, mask=mask) # masked store
```

可以看到，Triton 隐藏了线程和 warp 的细节。请注意，`BLOCK_SIZE` 是编译期常量，定义每个程序实例处理的元素数量。每个 CUDA 块的线程数由内核配置中的 `num_warps` 控制，并不等于 `BLOCK_SIZE`。

具体而言，上述代码中的 `tl.arange(0, BLOCK_SIZE)` 返回大小为 `BLOCK_SIZE` 的索引向量（`[0, 1, ..., BLOCK_SIZE-1]`）。将 `pid * BLOCK_SIZE`（即 `block_start`）加到索引向量上，就能得到该线程块中此内核实例访问两个向量的实际索引：`x_ptr + offsets` 和 `y_ptr + offsets`。

只要启动足够多的 Triton 内核实例，覆盖每个向量中的全部 `n_elements` 个元素，该内核就会将 `x_ptr` 和 `y_ptr` 的对应元素相加，并把结果存入 `out_ptr`。本质上，Triton 让你可以用张量化方式编写内核逻辑。

例如，这里一次处理整个索引块（`offsets`）。Triton 编译器负责将工作分配给实际 GPU 线程，并在可能时确保内存访问（`tl.load` 和 `tl.store`）合并。

要启动该 Triton 内核的实例，请传入一个根据 `meta['BLOCK_SIZE']` 计算程序实例数量的 grid 函数：

```
import triton

def grid(meta):
    return (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)

vector_add_kernel[grid](x_ptr, y_ptr, out_ptr, n_elements, BLOCK_SIZE=1024)
```

当 `n_elements` 不是 `BLOCK_SIZE` 的倍数时，代码使用掩码避免越界内存访问。这类似于前面 CUDA 章节在内核中使用 `if (idx < N)` 来避免索引越界。

> 在加载/存储操作中使用掩码，是处理边界条件的一种简洁巧妙方式，无需显式检查或 if/else 分支。

在底层，Triton 会将该程序转换为 NVIDIA PTX，使每个程序使用单个 CUDA 线程块；每个程序都映射到一个 CUDA 线程块。`tl.arange` 在程序内生成每个 lane 的索引，编译器再将这个向量化索引空间映射到块内线程。对于矩阵操作，也可以直接管理多维索引。Triton 会自动向量化算术和内存操作。

简而言之，Triton 兼具 Python 的开发效率和优化 CUDA C++ 内核的性能。它还允许深入底层进行优化，操作并充分利用完整的内存层次结构（例如共享内存分块），下一节将对

此进行演示。

在 Triton 中访问共享内存

高效的 Triton 内核会利用 L2 缓存以及每个 SM 上由软件管理的共享内存。使用共享内存时，每个线程块会将矩阵 A 和 B 的一个分块加载到共享内存中，而不是让每个线程反复从全局内存加载相同数据。

随后，内核会在多次计算中复用这些分块。这样可以更好地利用片上内存缓存，并减少全局 HBM 与寄存器之间传输的数据量。

Triton 不提供显式的共享内存分配器。相反，它使用张量描述符（`tl.make_tensor_descriptor(...)`）和基于目标形状与步幅的异步流水线，将分块暂存到片上共享内存中。这样就可以在流水化的 `tl.range(..., num_stages=...)` 循环中通过这些描述符发起加载和存储。该循环会降低为 `cp.async`、TMA 和屏障操作。

向 PyTorch 注册自定义内核

编写 Triton 内核后，可以使用 `torch.library.triton_op` 将其注册为 PyTorch 中的自定义操作。这样，`torch.compile` 就能识别该 Triton 内核，而不会把它当作可能回退到 eager 执行模式的不透明黑盒操作。编译器能够在捕获图时纳入 Triton 内核，并与图的其余部分一起优化，从而实现融合等额外优化。

注册 Triton 内核有助于在 PyTorch 编译器中使用自定义 Triton 内核/程序时避免 graph break。下面示例展示如何从 PyTorch 注册并调用 `vector_add_kernel`：

```
import torch
import triton
import triton.language as tl
from torch.library import triton_op, wrap_triton
from torch import Tensor

# Triton compute kernel
@triton.jit
def vector_add_kernel(
    x_ptr, y_ptr, out_ptr, n_elements,
    BLOCK_SIZE: tl.constexpr
):
    pid = tl.program_id(0)
    start = pid * BLOCK_SIZE
    offsets = start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(out_ptr + offsets, x + y, mask=mask)

# Register as a Triton-backed PyTorch op
@triton_op("my_triton_lib::vector_add", mutates_args=())
def vector_add(x: Tensor, y: Tensor) -> Tensor:
    assert x.device.type == "cuda" and y.device.type == "cuda"
    n = x.numel()
    out = torch.empty_like(x)
    # Compute grid size
    def grid_fn(meta):
        return (triton.cdiv(n, meta["BLOCK_SIZE"]),)
```

```
    # Wrap and launch the Triton kernel
    wrap_triton(vector_add_kernel)[grid_fn](x, y, out, n, BLOCK_SIZE=1024)
    return out

# Usage
a = torch.randn(10_000, device="cuda")
b = torch.randn(10_000, device="cuda")
c = torch.ops.my_triton_lib.vector_add(a, b)
```

这里，`triton_op("my_triton_lib::vector_add", mutates_args=())` 会向 PyTorch 注册操作名称及变更元数据（此处为空）。随后，`wrap_triton(vector_add_kernel)` 将原始 Triton 内核包装成可调用对象，编译器可以在 `torch.compile` 图中将其内联并优化，然后与图的其他部分进行融合、重排和内联。

注册仅支持前向的操作很直接。不过，要利用 PyTorch 自动微分实现完整训练支持，通常需要实现并注册自定义反向计算；否则，就必须使用已有的可微原语组合实现。

要支持训练，可使用 `vector_add.register_autograd(backward, setup_context=setup_context)` 注册 autograd 公式。也可以将逻辑封装到 `torch.autograd.Function` 中，同时注册 `forward` 和 `backward` 参数。但对于与 `torch.compile` 的组合使用，推荐采用 `register_autograd`。

> 如果 OpenAI Triton 不支持所需功能，或无法达到预期性能，可以使用 CUTLASS 等库以 CUDA C++ 重写内核以提高效率。随后可用类似方式向 PyTorch 注册 CUDA C++ 扩展，并为反向传播注册 autograd 梯度计算。

调优内核启动参数

许多 Triton 内核每个块通常使用 4 个 warp（即 128 个线程）。不过，现代 GPU 每个 SM 具有更大的共享内存和寄存器文件，通常可以将 `num_warps` 提高到每块 8 或 16 个 warp。例如，`BLOCK_SIZE >= 2048` 时可增加到 8，`BLOCK_SIZE >= 4096` 时可增加到 16。

warp 数量取决于内核能否利用并行性而不会造成过度争用。最佳设置取决于内核的算术强度和内存访问模式。

例如这样启动内核：`my_kernel[grid](..., num_warps=8)`。这表示每个 Triton 内核使用 8 个 warp（256 个线程）。这种配置通常适合计算密集型内核；但受内存吞吐限制，内存受限型内核可能在约 4 个 warp 时就达到上限。

对于内存受限型内核，每个线程块使用更多 warp，可以通过增加并行工作来隐藏内存延迟。但 warp 过多也可能造成争用或缓存抖动。

新一代 GPU 具有更多 SM 和更宽的内存总线，使我们可以把每块 warp 数从默认的 4 个提高到 8 或 16 个，从而提升占用率、覆盖更多内存访问延迟，并使可用内存和计算资源达到饱和。

手动探索每个内核的 `BLOCK_SIZE` 与 `num_warps` 组合可能很繁琐。因此通常最好使用 Triton 内置的自动调优器。它会进行基准测试，并自动为你选择最佳的 `BLOCK_SIZE`、`num_warps`、分块大小及其他参数。下一节将介绍自动调优器。

自动调优 Triton 内核

GPU 内核性能对分块维度、warp 数量、循环展开阶段，以及寄存器和共享内存等片上资源的使用方式等编译期参数非常敏感。Triton 内置自动调优器允许用 `@triton.autotune` 装饰 `triton.jit` 内核，自动搜索最佳设置。可以传入一组 `triton.Config` 对象，描述 `BLOCK_SIZE`、`num_warps`、`num_stages`、分块大小及其他内核元参数的不同候选组合。

第一次调用内核时，Triton 会对每种配置组合进行 JIT 编译和基准测试。初次调用应使用具有代表性的输入负载，因为 Triton 会根据输入大小/形状等特征生成键，并为该输入缓存最快配置。

后续调用只要输入特征相同，就会自动复用缓存的（最快）配置。这样，每种输入大小/形状只需支付一次自动调优开销，之后的内核调用即可立即受益于最佳配置。

如果检测到新的输入形状，Triton 会根据新的输入特征遍历 `triton.Config` 对象，重新执行自动调优，再为该输入选择最佳配置并缓存。

为避免调优结果不理想，建议使用与生产负载接近的真实、有代表性的输入预热自动调优器，使 Triton 的缓存配置能够反映生产输入。

> 可以向 `@triton.autotune(key_fn=...)` 提供自定义 `key_fn`，将输入元数据（例如张量形状）映射到自定义缓存键，从而覆盖特定输入形状和负载的最佳设置。这是一种高级技术，可更精细地控制不同输入负载类型的缓存配置。

选择候选内核配置时，应记住：较大的分块和更多 warp 会提高算术强度，但每个线程块也会消耗更多寄存器和共享内存。换言之，提高计算/内存比会因资源需求增加而限制占用率，因为每个 SM 能同时执行的线程块更少。

相反，使用较小分块和较少 warp 会减少每线程工作量与数据复用，但允许每个 SM 同时活跃更多块和 warp。这会以较低算术强度为代价提高占用率。

总之，最佳折中取决于输入矩阵维度和 GPU 的具体资源限制。手动调优耗时且容易出错。Triton 自动调优器会在真实负载上采用数据驱动方法自动处理复杂性，找出手动搜索可能遗漏的最佳配置。较高的 `num_warps`（例如 8–16）和多阶段流水线在 Blackwell 上可能使 `tcgen05.*` 路径达到饱和，但这取决于具体内核和硬件；应尽可能使用自动调优。

高级 Triton 内核实现

为了巩固这些概念，下面给出一些自包含的 Triton 内核示例，演示 warp specialization 以及数据传输/计算的异步双缓冲。这些示例展示如何使用 Triton 将高级 Python 代码转换为高度优化的 GPU 内核。

使用 Triton 进行 Warp Specialization

TorchInductor 可为其许多生成的 GPU 内核利用 Triton 的 warp specialization 支持。它会通过生成带有 `warp_specialize=True` 的 `tl.range()` 循环，尝试将每个线程块的 warp

划分为“生产者”（内存）和“消费者”（计算）角色，如下例所示：

```
// warp_specialize=True is supported on modern GPUs
// Use it together with num_stages > 1
// to enable producer/consumer warp partitioning
// and overlap
for k in tl.range(0, K_tiles, _warn_unused=False, warp_specialize=True):
    # loop body
    ...
```

负责内存的 warp 会预取下一个分块，同时另一个 warp 计算当前分块，从而将内存延迟与计算重叠，提高吞吐量。Warp specialization 与基于描述符的 TMA 拷贝协同工作。也可以像代码示例那样，在自己的 Triton 自定义内核中向 `tl.range()` 传入 `warp_specialize=True`。

还可以通过 Triton 自动调优配置驱动 warp specialization：在 `triton.Config` 中设置 `num_consumer_groups>0`（例如 2）和 `num_buffers_warp_spec`（例如 3），如下方代码所示。这样可以让生产者和消费者持续有工作可做；如果提供这些值，TorchInductor 会在底层使用它们：

```
triton.Config(
    { 'BLOCK_M': 128, 'BLOCK_N': 128, 'BLOCK_K': 64,
      'num_warps': 8, 'num_stages': 2,
      'num_consumer_groups': 2, '
      num_buffers_warp_spec': 3 }
)
```

这种 specialization 方法特别适合 GEMM 中沿较大 K 维度迭代的长循环。该方法可以持续让内存子系统和 ALU 忙碌，最大化硬件利用率。

分块与持久化 GEMM 内核（Triton）

该 Triton 内核高效计算矩阵乘法（C = A * B）：每次启动内核时都在内部遍历 K 维度完成全部工作，而不是为每个 K 分块启动多个内核。这样只需承担一次启动开销，warp 也能持续工作直到所有分块完成。下面示例在一次启动中沿 K 维度分块，但不会在多个输出分块之间复用同一线程块：

```
@triton.jit
def tiled_gemm_kernel(
    A_ptr, B_ptr, C_ptr,
    M, N, K,
    stride_am, stride_ak,
    stride_bk, stride_bn,
    stride_cm, stride_cn,
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
):
    """
    Tiled GEMM with Triton tensor descriptors + autotuning.

    This is the BASIC PRODUCTION example showing:
    1. Tensor descriptors (maps to TMA on Blackwell)
    2. Autotuning across block sizes
    3. Standard 2D grid decomposition
```

```
    """
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    m0 = pid_m * BLOCK_M
    n0 = pid_n * BLOCK_N

    offs_m = m0 + tl.arange(0, BLOCK_M)
    offs_n = n0 + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)

    # On Blackwell, descriptor .load/.store map to TMA
    # tl.dot lowers to UMMA (tcgen05) with accumulators in TMEM.
    A_desc = tl.make_tensor_descriptor(
        A_ptr,
        shape=[M, K],
        strides=[stride_am, stride_ak],
        block_shape=[BLOCK_M, BLOCK_K],
    )
    B_desc = tl.make_tensor_descriptor(
        B_ptr,
        shape=[K, N],
        strides=[stride_bk, stride_bn],
        block_shape=[BLOCK_K, BLOCK_N],
    )

    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

    K_tiles = (K + BLOCK_K - 1) // BLOCK_K
    if K_tiles == 0:
        c_ptrs = C_ptr + (offs_m[:, None] * stride_cm
                          + offs_n[None, :] * stride_cn)
        c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
        tl.store(c_ptrs, acc, mask=c_mask)
        return

    k0 = 0
    if (m0 + BLOCK_M <= M) and (k0 + BLOCK_K <= K):
        a_cur = A_desc.load([m0, k0])
    else:
        col_ids = k0 + offs_k
        row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                                 dtype=offs_m.dtype)
        col_offsets = col_ids[None, :] + tl.zeros((BLOCK_M, BLOCK_K),
                                                  dtype=col_ids.dtype)
        a_cur = tl.load(
            A_desc,
            offsets=(row_offsets, col_offsets),
            boundary_check=(0, 1),
            padding_option="zero",
        )

    if (n0 + BLOCK_N <= N) and (k0 + BLOCK_K <= K):
        b_cur = B_desc.load([k0, n0])
    else:
        row_ids = k0 + offs_k
        row_offsets = row_ids[:, None] + tl.zeros((BLOCK_K, BLOCK_N),
                                                  dtype=row_ids.dtype)
        col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                                 dtype=offs_n.dtype)
        b_cur = tl.load(
            B_desc,
            offsets=(row_offsets, col_offsets),
            boundary_check=(0, 1),
```

```
            padding_option="zero",
        )

    for kt in tl.range(0, K_tiles, num_stages=2):
        k0 = kt * BLOCK_K
        acc += tl.dot(a_cur, b_cur)

        next_k = k0 + BLOCK_K
        if next_k < K:
            if (m0 + BLOCK_M <= M) and (next_k + BLOCK_K <= K):
                a_cur = A_desc.load([m0, next_k])
            else:
                col_ids = next_k + offs_k
                row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                                         dtype=offs_m.dtype)
                col_offsets = col_ids[None, :] + tl.zeros((BLOCK_M, BLOCK_K),
                                                          dtype=col_ids.dtype)
                a_cur = tl.load(
                    A_desc,
                    offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1),
                    padding_option="zero",
                )

            if (n0 + BLOCK_N <= N) and (next_k + BLOCK_K <= K):
                b_cur = B_desc.load([next_k, n0])
            else:
                row_ids = next_k + offs_k
                row_offsets = row_ids[:, None] + tl.zeros((BLOCK_K, BLOCK_N),
                                                          dtype=row_ids.dtype)
                col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                                         dtype=offs_n.dtype)
                b_cur = tl.load(
                    B_desc,
                    offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1),
                    padding_option="zero",
                )

    # Store results with masking
    c_ptrs = C_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn)
    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=c_mask)

def persistent_matmul(A: torch.Tensor, B: torch.Tensor) -> torch.Tensor:
    M, K = A.shape
    K2, N = B.shape
    assert K == K2
    C = torch.empty((M, N), device=A.device, dtype=torch.float32)

    MT = triton.cdiv(M, 128)
    NT = triton.cdiv(N, 128)
    grid = lambda META: (min(65536, MT * NT),)  # bound launch overhead

    matmul_kernel_persistent[grid](
        A, B, C, M, N, K,
        A.stride(0), A.stride(1),
        B.stride(0), B.stride(1),
        C.stride(0), C.stride(1),
    )
    return C
```

这里，内核在 M×N 分块上启动二维网格，并在一次内核启动中完成完整的 K 循环。这会减少启动开销；当 K 较大时还可能提高利用率，但代价是单个内核会更长时间占用资源。每个程序（线程块）将 A、B 的分块加载到共享内存，并使用 `tl.dot` 计算分块的部分点积。Triton 使用 FP32 累加结果。在 Blackwell 上，Triton 会将 `tl.dot` 降低为 `tcgen05` 和 UMMA 指令以调用 Tensor Core；Tensor Core 随后将结果累加到专用 TMEM，而不是通用寄存器。

> 在 Triton 中，最好使用张量描述符表达由共享内存支持的分块移动，例如 `desc=tl.make_tensor_descriptor(...)`。在现代 GPU 上，这些张量描述符调用会映射到基于 TMA 的硬件操作，并使用异步、合并的传输。

Triton 会展开并向量化这些循环和计算以提高效率。此外，当数据类型（`dtypes`）和分块形状受支持时（例如 FP16/BF16，或 FP32 对应的 TF32），Triton 会将 `tl.dot` 降低为 Tensor Core 指令。下面是启动该 Triton 内核的简单 Python 包装器：

```
def tiled_matmul(A: torch.Tensor, B: torch.Tensor) -> torch.Tensor:
    """
    Tiled matrix multiplication using autotuned Triton kernel.

    The kernel will automatically select the best block size configuration
    for the given matrix dimensions (M, N, K).
    """
    M, K = A.shape
    K2, N = B.shape
    assert K == K2, f"Inner dimensions must match: {K} != {K2}"
    C = torch.empty((M, N), device=A.device, dtype=torch.float32)

    # Grid is computed based on max block size from autotuning configs
    # Triton's autotuner will pick the optimal block size at runtime
    MAX_BLOCK_M = 128  # From largest config
    MAX_BLOCK_N = 128
    grid = (triton.cdiv(M, MAX_BLOCK_M), triton.cdiv(N, MAX_BLOCK_N))

    # Launch with autotuning - Triton will select best config
    tiled_gemm_kernel[grid](
        A, B, C, M, N, K,
        A.stride(0), A.stride(1),
        B.stride(0), B.stride(1),
        C.stride(0), C.stride(1),
    )
    return C
```

可以看到，在一次内核启动中完成整个 K 循环，就不必针对每个输出分块启动多个内核。在现代 GPU 上，当 K 较大时，这种方式可能提高利用率，但代价是单个内核会更长时间持有资源。

可以将这种分块方式与持久化内核结合，使同一线程块在多个输出分块之间复用。持久化内核使用一维网格，并按 `tl.num_programs(0)` 的步长递增分块索引，如下面代码所示：

```
@triton.jit
def matmul_kernel_persistent(
    A_ptr, B_ptr, C_ptr,
```

```
    M, N, K,
    stride_am, stride_ak,
    stride_bk, stride_bn,
    stride_cm, stride_cn,
    # compile-time constants
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
    NUM_STAGES: tl.constexpr,
):
    """
    Persistent thread GEMM with Triton tensor descriptors + autotuning.

    Key Blackwell Optimizations:
      1) Tensor descriptors -> TMA hardware acceleration
      2) Autotuning across multiple block-size configurations
      3) Persistent threads to amortize launch overhead
      4) TMEM accumulation (accumulators live in ~256 KB/SM TMEM on Blackwell)

    This is the PRODUCTION-READY version combining these best practices.
    """

    # 1-D persistent launch: each program processes multiple tiles
    pid   = tl.program_id(0)
    nprog = tl.num_programs(0)

    MT = tl.cdiv(M, BLOCK_M)
    NT = tl.cdiv(N, BLOCK_N)
    TILE_COUNT = MT * NT

    # --- Tensor descriptors (map to TMA on NVIDIA). Descriptor rules:
    #     leading strides must be multiples of 16 BYTES;
    #     last dimension contiguous.
    A_desc = tl.make_tensor_descriptor(
        A_ptr, shape=[M, K], strides=[stride_am, stride_ak],
        block_shape=[BLOCK_M, BLOCK_K],
    )
    B_desc = tl.make_tensor_descriptor(
        B_ptr, shape=[K, N], strides=[stride_bk, stride_bn],
        block_shape=[BLOCK_K, BLOCK_N],
    )


    # Persistent stride over all output tiles handled by this program.
    tile_idx = pid
    while tile_idx < TILE_COUNT:
        pid_m = tile_idx // NT
        pid_n = tile_idx %  NT

        m0 = pid_m * BLOCK_M
        n0 = pid_n * BLOCK_N

        offs_m = m0 + tl.arange(0, BLOCK_M)
        offs_n = n0 + tl.arange(0, BLOCK_N)
        offs_k = tl.arange(0, BLOCK_K)

        # FP32 accumulator (on Blackwell, accumulators live in TMEM,
        # not registers)
        acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

        # ---- K loop (double-buffered ring-of-two using descriptors -> TMA) ----
        K_tiles = (K + BLOCK_K - 1) // BLOCK_K
        if K_tiles == 0:
            # Masked store of zeros when K == 0
            c_ptrs = C_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :]
                              * stride_cn)
```

```
    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=c_mask)
    tile_idx += nprog
    continue

# Prefetch first CURRENT tiles
k0 = 0
if (m0 + BLOCK_M <= M) and (k0 + BLOCK_K <= K):
    a_cur = A_desc.load([m0, k0])
else:
    row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                             dtype=offs_m.dtype)
    col_offsets = (k0 + offs_k)[None, :] + tl.zeros((BLOCK_M, BLOCK_K),
                                                    dtype=offs_k.dtype)
    a_cur = tl.load(A_desc, offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1), padding_option="zero")

if (n0 + BLOCK_N <= N) and (k0 + BLOCK_K <= K):
    b_cur = B_desc.load([k0, n0])
else:
    row_offsets = (k0 + offs_k)[:, None] + tl.zeros((BLOCK_K, BLOCK_N),
                                                    dtype=offs_k.dtype)
    col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                             dtype=offs_n.dtype)
    b_cur = tl.load(B_desc, offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1), padding_option="zero")

# Pipeline: prefetch-NEXT -> compute-CURRENT -> swap
for kt in tl.range(0, K_tiles, num_stages=NUM_STAGES,
                   warp_specialize=True):
    next_k = (kt + 1) * BLOCK_K

    if kt + 1 < K_tiles:
        if (m0 + BLOCK_M <= M) and (next_k + BLOCK_K <= K):
            a_next = A_desc.load([m0, next_k])
        else:
            row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                                     dtype=offs_m.dtype)
            col_offsets = (next_k + offs_k)[None, :]
                          + tl.zeros((BLOCK_M, BLOCK_K),
                          dtype=offs_k.dtype)
            a_next = tl.load(A_desc, offsets=(row_offsets, col_offsets),
                             boundary_check=(0, 1),
                             padding_option="zero")

        if (n0 + BLOCK_N <= N) and (next_k + BLOCK_K <= K):
            b_next = B_desc.load([next_k, n0])
        else:
            row_offsets = (next_k + offs_k)[:, None]
                          + tl.zeros((BLOCK_K, BLOCK_N),
                          dtype=offs_k.dtype)
            col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                                     dtype=offs_n.dtype)
            b_next = tl.load(B_desc, offsets=(row_offsets, col_offsets),
                             boundary_check=(0, 1),
                             padding_option="zero")

    # Compute on CURRENT tiles (UMMA on Blackwell; accumulators in TMEM)
    acc += tl.dot(a_cur, b_cur)

    # Swap in the prefetched NEXT tiles
    if kt + 1 < K_tiles:
        a_cur = a_next
        b_cur = b_next
```

```
        # ---- Store C with masking ----------------------------------------
        c_ptrs = C_ptr + (offs_m[:, None] * stride_cm
                          + offs_n[None, :] * stride_cn)
        c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
        tl.store(c_ptrs, acc, mask=c_mask)

        # Advance to the next tile handled by this program (persistent stepping)
        tile_idx += nprog
```

这种持久化方法很适合现代 GPU：后者拥有相对较大的寄存器文件、共享内存和 L2 缓存，可容纳更大的分块尺寸（`BLOCK_K`），也意味着每次迭代可展开更多 K 循环。

这种持久化内核通常会优于一系列较小的 `matmul`内核，尤其是在 K 很大时（例如 > 1,024）。代价是一个 SM 会被占用更久；但如果内核能充分利用该 SM，这通常是理想的。

在前述代码中，可在现代 GPU 上尝试增大 `BLOCK_K`，因为其片上内存更多，每个分块可处理更多数据。不过，超过某个阈值后，寄存器压力可能增加并导致寄存器溢出。因此务必进行性能分析，为具体负载和硬件找到合适平衡。

使用 Triton 进行软件流水线和双缓冲

本例展示如何在 Triton 中实现双缓冲。双缓冲是两阶段的软件流水线形式，可在单个循环中让内存加载与计算重叠。

在现代 NVIDIA GPU 上，异步全局内存到共享内存拷贝可让多个在途预取阶段将内存传输与计算重叠。因此，双缓冲（以及三重缓冲等）是很有价值且重要的性能优化技术。

Triton 通过 Triton 循环迭代器使用的 `num_stages` 元参数实现流水线。该参数传给 `tl.range()` 循环迭代器；当传入 `num_stages>1` 时，迭代器会自动对循环进行流水化，为最多 `num_stages` 个在途迭代发出异步拷贝操作。

这会让这些分阶段迭代中的内存加载与计算重叠，并持续直到处理完所有分块。下面是 Triton 中基于分块的双缓冲（`num_stages=2`）实现：

```
@triton.jit
def pipelined_matmul(
    A_ptr, B_ptr, C_ptr,
    M, N, K,
    stride_am, stride_ak,
    stride_bk, stride_bn,
    stride_cm, stride_cn,
    # compile-time constants:
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
    NUM_STAGES: tl.constexpr,
):
    # Program (CTA) ids for the MxN tiling
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    m0 = pid_m * BLOCK_M
    n0 = pid_n * BLOCK_N

    offs_m = m0 + tl.arange(0, BLOCK_M)
    offs_n = n0 + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)
```

```
# ---------------- Descriptor creation (maps to TMA on Blackwell) ----------
# Requirements for descriptor/TMA on NVIDIA GPUs:
#  - leading strides are multiples of 16 BYTES
#  - last dimension contiguous
#  - block_shape matches the tile you intend to move
A_desc = tl.make_tensor_descriptor(
    A_ptr,
    shape=[M, K],
    strides=[stride_am, stride_ak],
    block_shape=[BLOCK_M, BLOCK_K],
)
B_desc = tl.make_tensor_descriptor(
    B_ptr,
    shape=[K, N],
    strides=[stride_bk, stride_bn],
    block_shape=[BLOCK_K, BLOCK_N],
)

# Accumulator in FP32; on Blackwell this resides in TMEM, not registers.
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

# Number of K tiles
K_tiles = (K + BLOCK_K - 1) // BLOCK_K
if K_tiles == 0:
    # Nothing to do, store zeros (masked)
    c_ptrs = C_ptr + (offs_m[:, None]
                      * stride_cm + offs_n[None, :] * stride_cn)
    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=c_mask)
    return

# --------- Prefetch the first "current" tiles (fast path if fully in-bounds)
k0 = 0
if (m0 + BLOCK_M <= M) and (k0 + BLOCK_K <= K):
    a_cur = A_desc.load([m0, k0])
else:
    # Boundary: compute row/col offsets and use boundary-checked load
    row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                             dtype=offs_m.dtype)
    col_offsets = (k0 + offs_k)[None, :] + tl.zeros((BLOCK_M, BLOCK_K),
                                                    dtype=offs_k.dtype)
    a_cur = tl.load(A_desc, offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1), padding_option="zero")

if (n0 + BLOCK_N <= N) and (k0 + BLOCK_K <= K):
    b_cur = B_desc.load([k0, n0])
else:
    row_offsets = (k0 + offs_k)[:, None] + tl.zeros((BLOCK_K, BLOCK_N),
                                                    dtype=offs_k.dtype)
    col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                             dtype=offs_n.dtype)
    b_cur = tl.load(B_desc, offsets=(row_offsets, col_offsets),
                    boundary_check=(0, 1), padding_option="zero")

# ------------- K loop with software pipelining and TMA prefetch -----------
# Put prefetch as early as possible in loop body; keep separate "current"
# tile so loads for the next iteration can overlap with the dot-product of
# current tile. Use warp_specialize to partition producer/consumer warps.
for kt in tl.range(0, K_tiles, num_stages=NUM_STAGES, warp_specialize=True):
    next_k = (kt + 1) * BLOCK_K

    # Prefetch NEXT tiles early (TMA), if there is a next tile
    if kt + 1 < K_tiles:
```

```
            if (m0 + BLOCK_M <= M) and (next_k + BLOCK_K <= K):
                a_next = A_desc.load([m0, next_k])
            else:
                row_offsets = offs_m[:, None] + tl.zeros((BLOCK_M, BLOCK_K),
                                                         dtype=offs_m.dtype)
                col_offsets = (next_k + offs_k)[None, :]
                              + tl.zeros((BLOCK_M, BLOCK_K), dtype=offs_k.dtype)
                a_next = tl.load(A_desc, offsets=(row_offsets, col_offsets),
                                 boundary_check=(0, 1), padding_option="zero")

            if (n0 + BLOCK_N <= N) and (next_k + BLOCK_K <= K):
                b_next = B_desc.load([next_k, n0])
            else:
                row_offsets = (next_k + offs_k)[:, None]
                               + tl.zeros((BLOCK_K, BLOCK_N), dtype=offs_k.dtype)
                col_offsets = offs_n[None, :] + tl.zeros((BLOCK_K, BLOCK_N),
                                                         dtype=offs_n.dtype)
                b_next = tl.load(B_desc, offsets=(row_offsets, col_offsets),
                                 boundary_check=(0, 1), padding_option="zero")

        # Compute on the CURRENT tiles (UMMA; accumulators in TMEM on Blackwell)
        acc += tl.dot(a_cur, b_cur)

        # Swap in prefetched NEXT tiles
        if kt + 1 < K_tiles:
            a_cur = a_next
            b_cur = b_next

    # ------------------------------ Store C ----------------------------------
    c_ptrs = C_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn)
    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=c_mask)
```

Triton 编译器会识别 `num_stages`，使用 Triton 张量描述符通过 TMA 和 TMEM 生成异步加载/存储，并自动管理同步。TMA 张量描述符和 TMEM 可以减轻寄存器压力，也更容易表达多维传输。上面的循环遵循“预取下一个分块 → 计算当前分块 → 交换”的顺序，在 `num_stages>1` 时保持重叠，并避免过早覆盖当前分块导致重叠悄然失效。具体而言，一个分块加载到共享内存的同时，前一个分块正在计算。

还可以使用 warp specialization，让 Triton 将 warp 划分为生产者和消费者，使全局内存到共享内存的拷贝与计算重叠。当启发式策略选择它时，该功能会自动启用。

必须保持流水线的重叠。当设置 `num_stages > 1` 时，应确保当前正在计算的分块在多次循环迭代中保持有效；在消费它的数学运算完成之前，不要覆盖它或丢弃其引用。

可以考虑一个两阶段环形缓冲区：异步获取 `next` 缓冲区的同时计算 `current` 缓冲区，然后交换二者。在这种情况下，必须先等待并消费“当前”缓冲区，再释放它；同时要用 commit/wait 在交换前保护“下一个”缓冲区（分块）。

如果当前分块无法跨迭代保持有效（例如覆盖了其缓冲区或让引用失效），编译器就可以将“下一次”迭代的拷贝重排、下沉或上提到“当前”使用之前，从而使加载/计算重叠悄然消失。

如果手动管理共享内存块指针/TMA，请使用显式握手流程：入队异步拷贝 → `tl.commit()` →（稍后）`tl.wait()` → 消费分块 → 推进指针。在读取分块前调用 `tl.wait()`，并仅在消费完成后释放/交换，就能保持真正的重叠。

在现代 GPU 上，Triton 使用张量内存加速器（TMA）支持描述符加载和存储。流水化循环（`num_stages > 1`）时，应尽早预取下一个分块，并为当前分块保留独立变量。在消费当前分块的操作完成之前，不要覆盖或丢弃“当前”引用；否则编译器可能合法地将下一次迭代的拷贝重排到当前使用之前，使重叠失效。最简单的模式是 `prefetch-next → compute-current → swap`。

简而言之，软件流水线可提高内存带宽利用率并隐藏延迟。如果内存带宽不是瓶颈，可以将 `num_stages` 增加到 3（例如三级缓冲），预取两个在途分块，进一步提高性能，但会因额外缓冲区而消耗更多共享内存。在共享内存预算更紧张的硬件上，或对于计算密集型内核，增加阶段可能收益递减、降低占用率并损害总体性能。

使用 Triton Proton Profiler 进行性能分析

要深入分析 Triton 内核性能，可以使用 Triton 的 Proton 分析器。这是一个独立的性能分析包，与 Triton 集成，并在 Nsight Systems 时间线中输出可见的 NVTX 区间。这些区间会为代码区域插桩、收集计时并跟踪关键指标；NVTX 标记随后会出现在 Nsight Systems 时间线中。可以利用这些 NVTX 区间从 Proton 摘要跳转到 Nsight Compute，进行更聚焦的内核级分析。

实际使用时，可用 `with proton.scope("name", metadata)` Python 上下文管理器包装 Triton 代码中的关键区域。运行负载前，使用 `proton.start("matmul", hook="triton")` 激活分析器。

Proton 中的 `metadata` 是用户提供的注释或指标字典，例如总 FLOPS、内存字节数、线程块索引、warp 级索引或记录槽数量。这些信息会与计时数据一起记录，以便进行更丰富的性能分析。

执行完成后，结束分析并获取性能数据。输出可以打印为分层计时表。下面是 Proton 分析结果的节选，用于比较不同 `matmul` 内核计算 8,192 × 8,192 矩阵时的表现：

```
168.314 ms    16331.291  ROOT
├─ 174.920 ms   3928.623  cublas [M=8192,N=8192,K=512]
├─ 165.349 ms   4156.033  matmul_kernel [M=8192,N=8192,K=512]
├─ 159.352 ms   4312.421  matmul_kernel_persistent [M=8192,N=8192,K=512]
└─ 174.671 ms   3934.214  torch [M=8192,N=8192,K=512]
```

在上面的分析输出中，标签 `K` 表示归约维度，即矩阵乘法共享的内维度（例如 M × K 矩阵乘以 K × N 矩阵）。它表示点积长度，同时决定计算成本和内存流量。

从这份分析报告可以看到，Triton 持久化内核优于 cuBLAS：159.352 ms 对 174.920 ms。Proton 可轻松量化这类差异，还能计算有效 TFLOPS、内存带宽等派生指标。如果提供总 FLOPS 数量等额外 `metadata`，Proton 会显示当前精度（例如 BF16/FP16、FP8、FP4）是否接近硬件理论 TFLOPS 上限。对于许多矩阵形状和精度，PyTorch 中的 cuBLASLt 或 CUTLASS 路径也可能达到或超过自定义 Triton 内核。

NVIDIA 的 Nsight Systems 和 Nsight Compute 工具也支持 Triton 内核。例如，Nsight Systems 会显示内核启动名称以及 PyTorch 或 Proton 输出的 NVTX 区间，可用于将 Proton scope 与 Nsight 时间线关联起来。这样可以用 Proton 输出在 Nsight Systems 中定位关键内核，再进入 Nsight Compute 做低层分析，例如寄存器使用量和实际占用率。联合使用这些工具可以获得更完整的系统性能分析。

PyTorch XLA 后端

TorchInductor 是 PyTorch 在 GPU 和 CPU 上的默认后端，而 PyTorch XLA 是面向 Google Cloud TPU 及其他加速器的独立后端编译选项。PyTorch XLA 将 PyTorch 操作映射到 XLA 图 IR，并通过目标硬件运行时执行，使 PyTorch 模型能够运行在这些加速器上，如图 14-6所示。

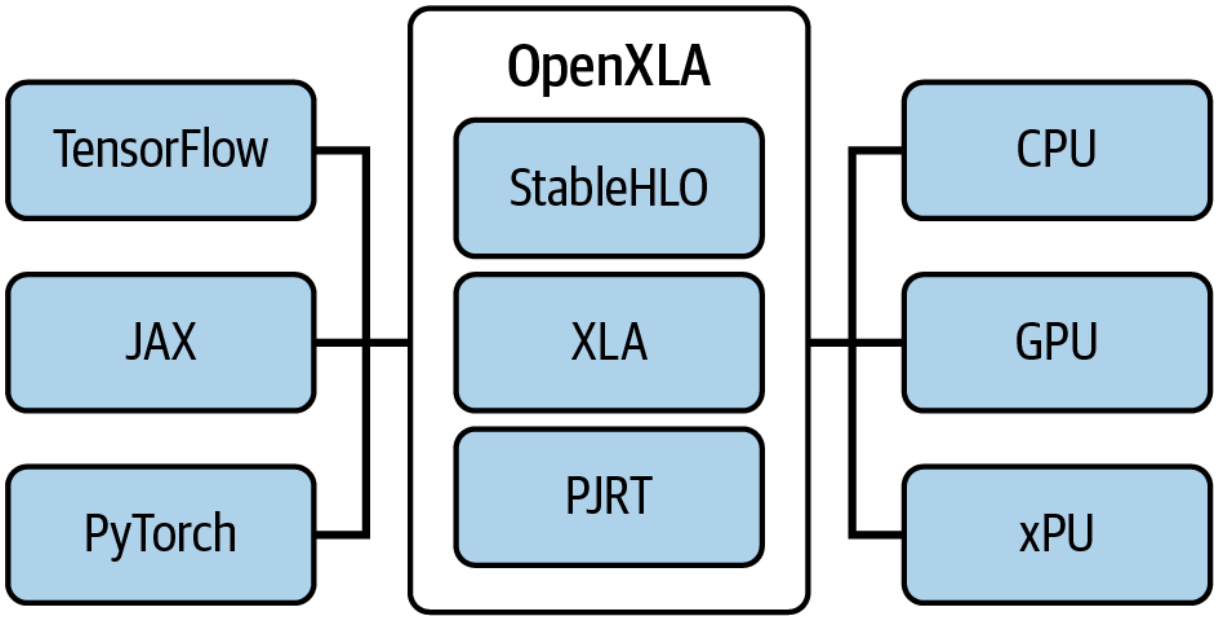


图 14-6　OpenXLA：PyTorch XLA 编译器后端的基础

要启用 XLA 后端，可以使用 `torch.compile(..., backend="openxla")`，基于 OpenXLA 激活 PyTorch XLA。该后端字符串由 PyTorch XLA 项目支持，并启用基于 OpenXLA 的编译。与 TorchDynamo 和 TorchInductor 类似，XLA 会捕获计算图；但由于 XLA 旨在生成静态图，它会提前编译整个程序。

XLA 针对静态形状或有界动态形状进行了优化。因此，使用 XLA 时对动态形状的支持相对有限。新形状会触发整个程序重新编译，这对延迟敏感的推理开销很大。不过，OpenXLA 会按形状签名缓存可执行文件，可能改善性能。

> 可能需要对输入进行填充或使用固定大小的 bucket，因为 XLA 遇到新形状时会重新编译，而不是以符号方式处理。

XLA 编译器会按每种唯一输入形状和签名缓存已编译图。因此，与 TorchInductor 类似，经过几次预热后性能会提升。主要差异是 XLA 不会在运行中渐进式编译：图会预先静

态构建；若遇到新形状，则会触发一次新的全图编译，代价很高，并会影响推理等延迟敏感型负载。

总之，如果运行设备目前不受 TorchInductor 后端支持，但支持 XLA，就可以考虑使用 XLA 后端。许多原则仍然适用，例如尽量减少 graph break；还可以使用数据并行、模型并行等分布式策略。XLA 虽不常用于 NVIDIA GPU，却是非 NVIDIA 硬件的强大后端（例如使用 OpenXLA 的 Google TPU，或支持 XLA IR 的其他加速器）。XLA 同样受益于本章讨论的许多编译技术。

要点回顾

本章内容丰富，我们深入探讨了 PyTorch 编译器栈以及 OpenAI 的 Triton 语言和编译器。下面是一些关键要点：

利用 `torch.compile` 轻松提速

根据需求选择编译 `mode`（例如，`"default"` 启动更快，`"max-autotune"` 追求最高性能）。始终先进行几次预热迭代，以度过初始编译阶段。对于运行时间短的任务或小模型，编译开销可能超过提速收益，因此应确保有足够工作量来摊薄一次性成本。

尽早设置性能标志

下面的标志可用于加速 FP32 矩阵乘法和包括 Flash Attention 在内的 SDPA 变体。始终验证模型精度，并在结果符合要求时保持启用，以获得最佳性能：

```
import torch

 # maps to TF32/BF16 fast paths

torch.set_float32_matmul_precision("high")

torch.backends.cuda.matmul.allow_tf32 = True

# affects convs

torch.backends.cudnn.allow_tf32 = True

torch.backends.cuda.enable_flash_sdp(True)

torch.backends.cuda.enable_mem_efficient_sdp(True)
```

尽量减少 graph break

使用 `torch._dynamo.explain` 或 `TORCH_LOGS="graph_breaks"` 检查 graph break。删除或重构导致中断的代码（例如 print、依赖数据的 Python 控制流、不支持的操作），尽可能扩大可编译的连续区域。通常图越少、越大，性能越好。必要时可使用新的 `torch.cond` API 表达条件逻辑，或将非关键的 Python 侧处理移出模型的 forward 函数。目标是向编译器提供一条足够长、纯粹“张量输入—张量输出”的代码路径。

谨慎使用动态形状

虽然并非必需，但预先设置 `torch.compile(dynamic=True)` 会迫使编译器将所有维度视为动态。这样一个已编译模型可以处理更广泛的输入形状，减少填充需求。也可以使用 `mark_dynamic()` 仅标记特定维度，从而把灵活性限制在真正需要的地方，这对可变序列长度很有用，但必须测量折中。启用动态形状支持可能禁用 CUDA Graph 并增加额外 guard。如果形状变化范围不大，混合方案通常更好：按大小将输入分 bucket，限制不同形状的数量，再对剩余变化启用动态形状。这样既避免过度重新编译，也减少填充浪费。

分析触发重新编译的 guard

如果出现多次编译，使用 `TORCH_LOGS="graph_breaks,guards,recompiles"` 找出触发的 guard。常见原因包括 Python 随机值、张量秩变化以及 device/dtype 变化。可以将这些方面固定，或在适当时使用 `allow_in_graph` 将其标记为安全。

尽可能避免重新编译

调优良好的训练循环在最初几次迭代后应不再重新编译。如果仍持续编译，应立即调查，因为这通常意味着每次迭代都有某些内容发生变化，例如带递增计数器的调试 print。使用 `set_stance()` API 和 guard 日志捕获这些问题；还要确保没有无意中混用设备，例如一次迭代发送 CPU 张量、下一次发送 GPU 张量，这会触发重新编译。

调优并监控内存使用

对于更大的融合内核和 guard 缓冲区，编译模式可能使用更多内存。请监控 GPU 内存；遇到内存问题时，可在 Triton 内核中使用较小的 `BLOCK_SIZE` 或禁用部分融合。还应及时释放大型中间结果，因为它们可能在编译图生命周期内保留更久。若编译期间出现内存不足，可能需要拆分模型或降低 `max_autotune` 设置。PyTorch 支持对大型图自动 checkpoint 以降低内存压力，但并非万无一失；也可以分别编译子模块（例如每一层或每个块），而非一次编译整个模型，以限制内核生成期间的内存使用。

明智地将 PyTorch 编译器与分布式训练结合

使用 DDP 或 FSDP 时，要注意通信点处有意设置的 graph break。这些中断是预期行为，系统会通过通信/计算重叠进行优化。将子模块包裹在 FSDP 中，可实现按分片编译并节省内存。留意 `dynamo.explain` 中与 all-reduce 相关的警告。PyTorch 的设计会尽量降低 graph break 的性能影响。使用 FSDP 和 `torch.compile` 时，梯度预归约和后归约步骤可能出现 graph break；这是预期且已处理的行为。应关注每个分片中被编译的主要前向和反向过程。

使用 `TORCH_LOGS="perf_hints"` 捕获遗漏的优化

例如，它会告诉你 CUDA Graph 未使用是否因为输入发生变更，或某个操作是否回退到 eager 模式。这些提示会指出应避免的模式或等待未来支持，从而引导改进；有时还会直接给出变通方案。例如，“input is mutated” 提示意味着编译前应避免对输入执行原地操作。

先用小输入调试

开发自定义内核或测试编译模式时，先使用较小的张量尺寸，以便快速发现正确性问题；可正常工作后再放大规模。使用 PyTorch 的 `torch._dynamo.config.verbose`，甚至 `TORCH_LOGS="output_code"` 检查小规模情况下生成的代码。请注意，小输入测得的性能可能无法代表真实规模的行为，但这种方法非常适合调试正确性和查看生成了哪些内核。

只为真正的瓶颈编写自定义内核

开始开发 Triton 内核前，应分别分析模型在编译和未编译情况下的性能，找出热点。TorchInductor 通常已经融合了许多操作，因此应把精力集中在 Inductor 不擅长的部分，例如自定义操作或不常见的融合。不要过早优化所有内容：大部分交给编译器，其余再手动调优。例如，如果 TorchInductor 无法融合某个性能关键的操作序列（如多步骤自定义激活），才可能值得编写独立 Triton 内核。还要权衡维护成本，因为每个自定义内核都需要维护，硬件变化或 PyTorch 最终提供原生支持时可能需要更新。若某种模式对许多用户都有益，向 TorchInductor 提交 issue 或功能请求通常更值得。

遵循 Triton 最佳实践

编写 Triton 代码时，确保内存访问合并，避免共享内存 bank 冲突（必要时进行填充），在边界处对 `tl.load()` 和 `tl.store()` 使用掩码；选择与 warp 大小对齐的块和分块大小（32 线程的倍数），选择适合 L1/共享内存 carve-out 的分块大小，并使用 `@triton.autotune([... triton.Config(num_warps=..., num_stages=...) ...], key=[...])` 调优 `num_warps` 与 `num_stages`。如果性能至关重要，应使用 Triton 自动调优器寻找最佳配置。可以从 `num_warps` $\in$ `{4,8}` 和 `num_stages` $\in$ `{2,3,4}` 开始。还应查看 Triton 的文档和示例，因为社区已经实现并优化了许多常见模式（如 FlashAttention）；至少可以将这些示例作为自己实现的起点。

谨慎使用缓存提示

某些 PTX 加载形式（例如 DeepSeek 风格的非一致性 + L1 不分配 + 256B L2 预取）没有文档说明，在某些硬件上可能未定义。它们可能在不同驱动版本和硬件代际之间失效。为提高可移植性，优先使用 Triton 的 `eviction_policy`（例如 `evict_last`）和 `cache_modifier` 选项。迁移到不同 CUDA 驱动和 GPU 代际时，记得重新分析性能。避免引入相互冲突的缓存提示，因为这类问题极难调试。

关注新的 PyTorch 发布版本

PyTorch 编译器正在快速演进。每个版本都可能带来更广泛的算子支持、更好的性能和更少的 graph break。升级通常可以免费获得提速或修复问题，Triton 也是如此。保持更新可以受益于最新成果，这对于快速发展的 GPU 硬件尤其重要。

结语

随着复杂性增加、PyTorch 生态持续扩展，应积极采用性能分析和迭代式优化。可以联合使用 Nsight Systems、Nsight Compute、PyTorch profiler 或 Triton profiler，验证 GPU 利用率和性能是否达到预期；如果没有，就调整性能策略。

迭代优化时还要保持全局视角。一个瓶颈消除后，下一个瓶颈就会出现。性能调优可能依次涉及 GPU 内核占用率、CPU 开销和输入流水线。编译器能解决其中很大一部分问题，但要达到最佳性能，通常还需要优化数据加载、I/O 和算法选择。

第 15 章　多节点推理、并行、解码与路由优化

LLM 的参数规模仍在持续增长。尤其是混合专家（MoE）LLM 的出现——这类模型将许多专门的子网络（“专家”）与内置的专家门控机制结合起来——已将模型参数规模推高至数千亿甚至数万亿。虽然对某个输入而言只有一部分参数会被激活，但对如此庞大的模型进行推理，仍需要将工作负载分布到多个 GPU 上。

本章聚焦于使用现代 NVIDIA GPU 对这些超大规模 LLM 进行高效、高性能多节点推理的高级优化技术。我们将讨论如何利用硬件和算法创新设计分布式推理系统，以最大限度降低延迟并提高吞吐量。

首先讨论解耦式 prefill 与 decode（PD、disagg PD）架构。该架构将推理负载拆分成可独立调优的不同阶段。随后介绍面向推理的核心并行策略，包括数据并行、张量并行、流水线并行、专家并行和上下文并行，以及如何组合使用这些策略，在众多 GPU 上部署大型模型。

之后介绍推测解码方法，包括 Medusa、EAGLE 和 draft-and-verify 等方案。它们允许推理时生成并评估多个 token，而不是像传统自回归 LLM 那样一次只生成一个 token，从而缓解顺序解码瓶颈。我们还会讨论用于强制输出格式（例如自定义 JSON schema）的约束解码，以及面向 MoE 模型的动态路由策略，以提高专家门控和负载均衡效率。

解耦式 Prefill 与 Decode 架构

如前所述，现代 LLM 的推理工作流包含两个不同阶段：prefill 和 decode。可以采用*解耦式 prefill 与 decode* 将二者分离，使 prefill 和 decode 集群能够独立扩展，甚至运行在不同硬件平台上，从而显著提高大规模 LLM 服务性能，本章后面会详细说明。

> 跨厂商或跨架构部署要求两端的 KV cache 布局和数据类型一致。实际生产系统应让 prefill 与 decode 使用兼容的 GPU 系列，从而采用相同数值格式，便于传输和复用 KV cache。

在 prefill 阶段，模型通过一次前向传播处理整个输入 prompt（通常包含数千、数万甚至数百万个 token），生成 LLM 计算出的初始隐藏状态，然后为输入 prompt 中的所有 token 填充注意力键值（KV）缓存。图 15-1展示了解耦式 prefill 与 decode 如何共享 KV cache，并将 KV 传输与计算重叠。

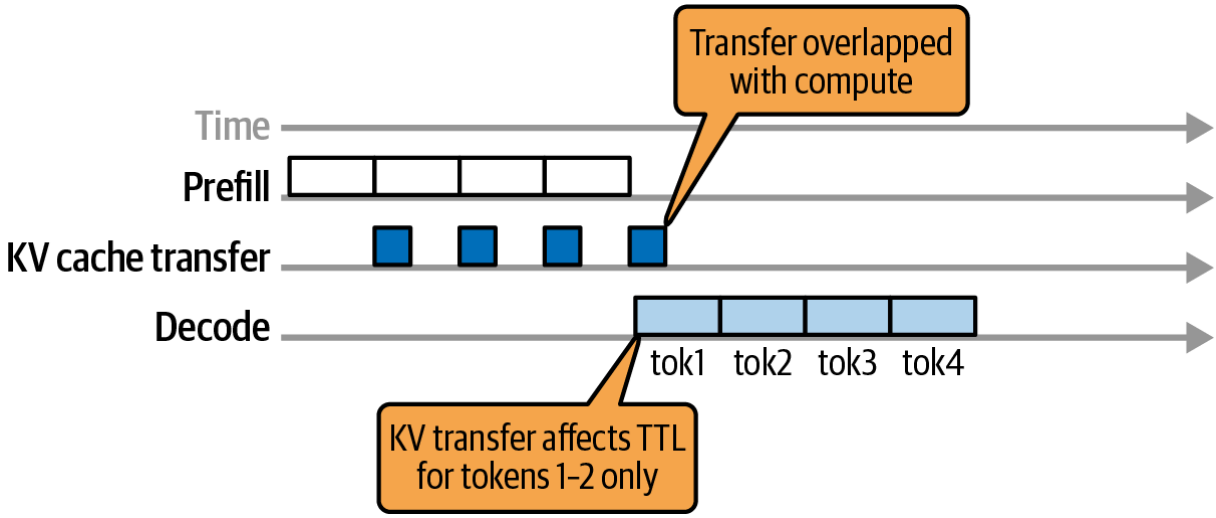


图 15-1　解耦式 prefill 与 decode 共享 KV cache，并将 KV 传输与计算重叠

在 decode 阶段，模型通过自回归生成预测序列中的每个新 token，并使用所有先前已生成 token 的缓存注意力 KV 表示。

> 推测解码通过在一个批次中预先生成多个 token，并行验证这些 token 是否正确，从而加速 decode 过程。这降低了标准逐 token 自回归解码的顺序性。稍后会详细介绍推测解码。

Prefill-Decode 干扰

传统 LLM 推理系统会将这两个阶段部署在同一节点，并简单地将所有计算一起批处理。然而，这种朴素方法会产生通常所说的 *prefill-decode 干扰*。例如，长 prompt 的 prefill 会占用 GPU，延迟其他请求中对时间敏感的 decode 工作，反之亦然。

将 prefill 和 decode 部署在同一节点，迫使特征迥异的两个阶段共用一种调度和资源分配策略。Prefill 包含大型并行计算，而 decode 需要大量小型顺序计算。因此，系统要么优先保证其中一个阶段的性能，要么过度配置硬件以同时满足二者需求。

采用*解耦式 prefill 与 decode* 架构时，prefill 与 decode 阶段被分配到不同 GPU 池，从而消除两类负载之间的直接干扰。DistServe 系统报告称，在同时满足 TTFT 和 TPOT 约束时，解耦 prefill 与 decode 可使 goodput 最高提高 7.4 倍（在最高严格 12.6 倍的延迟 SLO 下）。这些数字来自特定实验配置，并非所有部署都能达到。

独立扩展 Prefill 与 Decode 工作节点

消除跨阶段干扰后，可以减少资源“空转时间”，避免 decode 任务被长时间 prefill 计算阻塞，反之亦然。这样 GPU 会花更多时间完成有用工作，减少空闲，从而提高利用率和给定延迟目标下的有效吞吐量（即 *goodput*）。

可以用一组专用节点处理 prefill、另一组处理 decode，从而分别扩展二者。两个集群仅在将编码后的 prompt 状态（即注意力 KV cache）从 prefill worker 传输到 decode worker 时通信，如图 15-2所示。

图中，独立的 GPU worker 分别处理 prefill 阶段的输入 prompt 和 decode 阶段的迭代式输出 token 生成。Prefill 阶段输出包含 prompt 的 KV cache，随后传输给 decode worker 以生成后续 token。

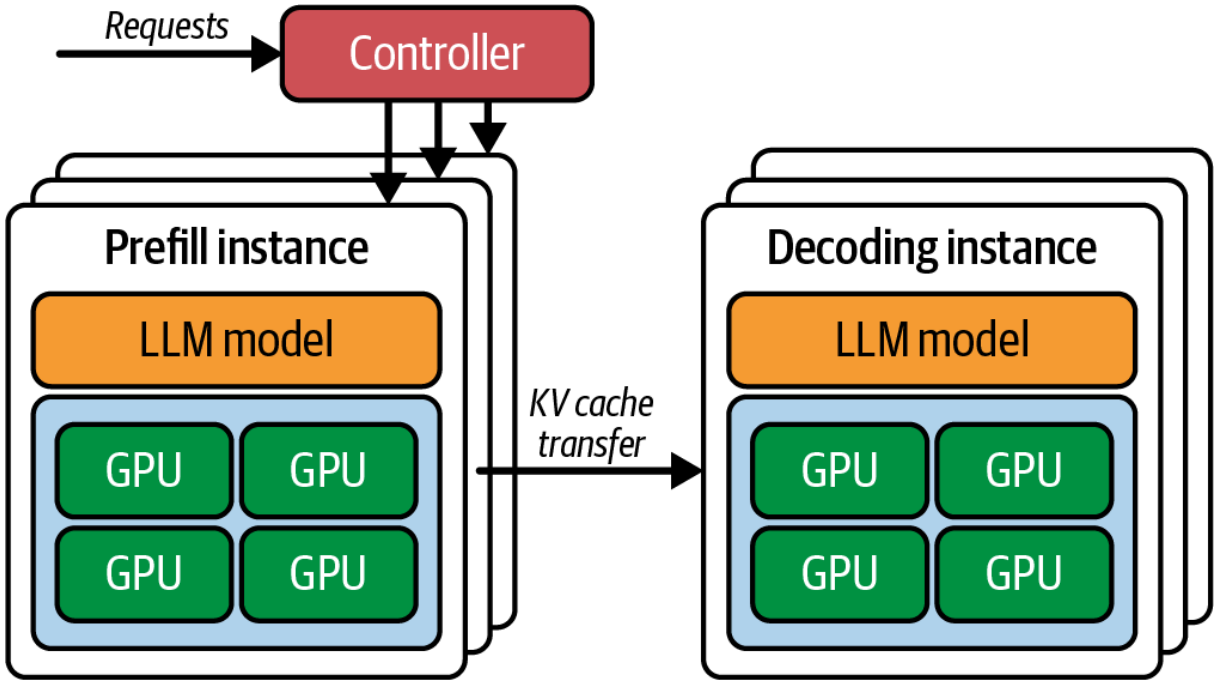


图 15-2　解耦式推理：Prefill 池（隐藏状态 + KV）→ 通过 NVLink/NVSwitch（节点内）或 GPUDirect RDMA（节点间）移交 KV → Decode 池

通过设置独立 GPU 池，系统可以让 prefill 和 decode 流水线并行保持忙碌。实践表明，在严格延迟约束下，解耦能够显著提高吞吐量。部分研究显示可能获得较大收益，但在拆分两个阶段后，实际结果从适度提升到 goodput 提高数倍不等，高度依赖工作负载和网络互连。

这种拆分允许针对吞吐量或延迟独立优化和扩展各阶段。可以在 prefill GPU 上激进地批处理，以最大化吞吐量，而不会拖累 decode 性能（例如增加 decode 延迟）。使用独立集群后，还可分别调节各阶段的并行设置、实例数量和调度策略。

对延迟（TTFT）和吞吐量（TPOT）的影响

Decode GPU 可以使用较小的批大小或专用调度，以尽量降低流式生成的*每个输出 token 时间*（TPOT）。例如，可以使用优先处理紧急 decode 任务的调度器，避免排队延迟。

这种拆分之所以有效，是因为两个阶段的性能目标不同。Prefill 阶段的*首 token 时间*（TTFT）针对低延迟优化，而 decode 阶段优先考虑较低的每输出 token 时间（TPOT）和稳定的流式延迟。端到端吞吐量主要由并发度和调度决定。传统部署必须在 TTFT 与 TPOT 之间折中；解耦则允许同时满足两个 SLO 目标。

> 在 KV 传输和 all-to-all 阶段监控 NVLink 与 NIC 利用率。目标是使用独立 stream 和 event 将拷贝与计算重叠。

KV 移交只增加较少延迟，因为多 GPU 和多节点传输使用高带宽互连，包括 NVLink、NVSwitch、InfiniBand，以及通过 GPUDirect RDMA 使用的以太网 RoCE。例如，配备 ConnectX-8 SuperNIC（800 GbE 级）的多节点集群，每个端口可提供最高 800 Gb/s 的 GPUDirect RDMA 带宽。与经由主机的通信路径相比，这能显著缩短 KV 传输时间。还建议每个 GPU 配置一个 NIC，以优化 prefill-decode 解耦并提高 MoE all-to-all 性能；实际收益仍取决于拓扑、消息大小和软件栈。

KV Cache 数据传输与 NIXL

Prompt 处理完成后，解耦系统使用 connector或调度器，将 prompt 的中间结果（最终隐藏状态和 KV cache）从 prefill worker 传输到 decode worker。这种移交会产生一定通信开销，但如果集群采用 NVLink、InfiniBand 等高带宽互连，与消除资源争用带来的收益相比，开销通常较小。

实际使用中，NVIDIA NIXL 库会根据拓扑和策略，在 NVLink/NVSwitch、RDMA 或经主机暂存的路径之间自动选择，以尽量降低传输开销。例如，第 4 章介绍的 NIXL 会自动选择最快的可用路径传输 KV cache。NIXL 可与 Dynamo、vLLM 等框架和推理引擎集成，NIXL-vLLM 集成如图 15-3所示。

具体来说，LMCache 和 NIXL 已集成到 vLLM 的解耦式 prefilling 中，作为受支持的数据路径。NVIDIA Dynamo 和 TensorRT-LLM 也使用 NIXL，通过 GPU 点对点互连和 RDMA 传输 KV cache 数据。

在节点内部，NIXL 通过 NVLink 和 NVSwitch 执行设备到设备传输，无需经主机暂存。跨节点时，NIXL 通过 InfiniBand 或 RoCEv2 使用 GPUDirect RDMA，避免主机拷贝。即使负载达到数 GB，这些路径也能将 KV cache 移交延迟保持在较低水平，具体效果取决于拓扑和并发流量。

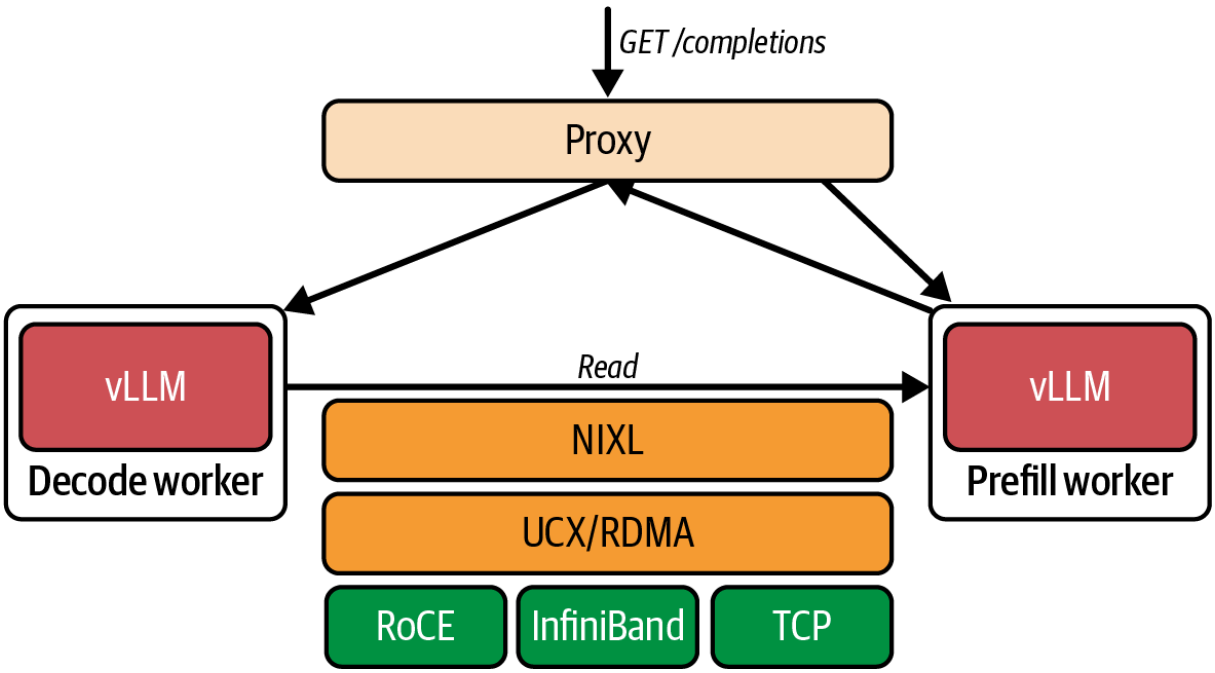


图 15-3　vLLM 推理引擎中使用 NIXL 传输 KV cache；节点内：NVLink/NVSwitch（设备到设备）；节点间：使用 ConnectX 级 NIC 的 GPUDirect RDMA（InfiniBand/RoCEv2）

Prefill 与 decode worker 的放置应遵循互连拓扑。同节点放置时，可通过 CUDA peer access 让 KV 传输走 NVLink 和 NVSwitch；跨节点放置则应通过 InfiniBand 或 RoCEv2 使用 GPUDirect RDMA。NVIDIA Dynamo 与 NIXL 集成，可在跨节点的 GPU、CPU 内存和存储之间移动 KV cache；vLLM 则通过 LMCache 和 NIXL 实现解耦式 prefilling。

> 如果互连或虚拟化层阻止直接 peer access，NIXL 可能回退到经主机暂存的非最优路径。务必在实际部署中验证端到端 KV 传输时间。

使用 Kubernetes 部署解耦式 Prefill 与 Decode

在高级部署中，Kubernetes 等集群编排系统可以根据负载和输入特征动态调整 GPU 池分配，或分别横向扩展各个池。例如，大量用户提交超长 prompt、但输出较短时（如大型文档摘要），Kubernetes 可以临时为 prefill 池分配更多 GPU，同时减少 decode 阶段的 GPU 数量。

相反，如果大量用户请求超长输出（例如长推理链、“逐步思考”等），则可将更多 GPU 转移到 decode 池。两种情况下，都可以分别为各类 worker 横向扩展新实例。

图 15-4展示了一个基于 Kubernetes 的分布式 vLLM 集群，它使用开源 llm-d 项目运行独立的 prefill 和 decode worker。vLLM 通过运行两个实例，并使用 LMCache 和 NIXL 移交 KV，实现解耦式 prefilling；llm-d 则进一步提供 Kubernetes 原生编排，用于解耦式服务和 KV 感知路由。图中名为*变体自动扩缩器*的组件负责更新池中 prefill 与 decode worker 的副本数量。

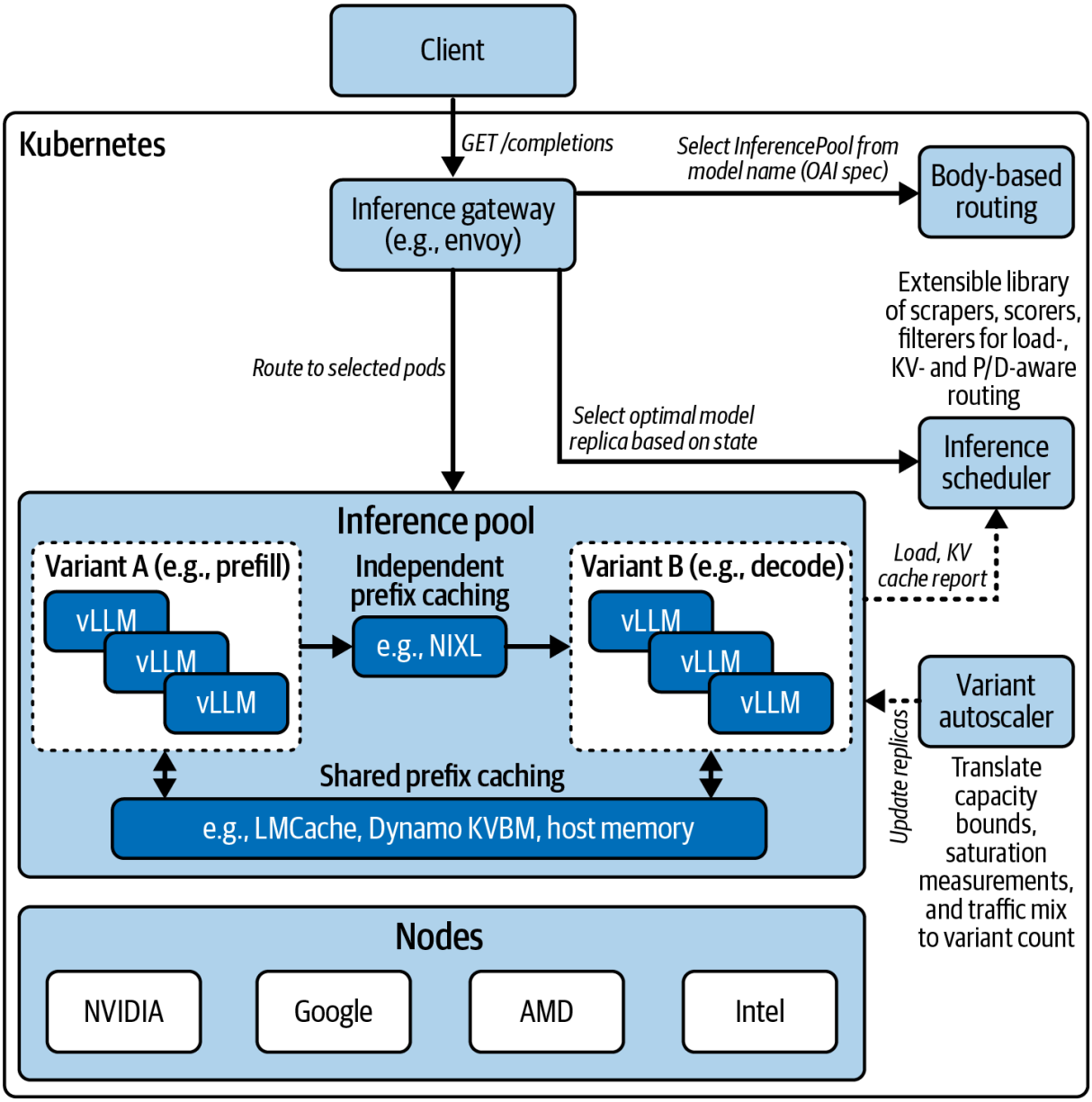


图 15-4　使用开源 llm-d 项目的 Kubernetes vLLM 集群，包含独立 prefill 与 decode worker；变体自动扩缩器根据 prompt/响应组合调整副本数；使用 LMCache 和 NIXL 移动 KV

> 在现代推理部署中，所有节点通常共享同一运行时和代码库（例如 vLLM、SGLang、NVIDIA Dynamo），因此均可执行 prefill 和 decode。集群编排器可在启动时静态分配具体角色，也可在 worker 生命周期中动态调整角色。

总体而言，解耦式 prefill/decode 架构为高吞吐、低延迟 LLM 服务奠定了基础。不过，它也会增加复杂性：必须传输和管理中间数据，长 prompt 的 KV cache 可能达到数 GB，调度也更加复杂。但在超大规模场景中，提高硬件利用效率带来的收益很可观。

> 第 17 章和第 18 章将更深入介绍解耦式 PD 的其他技术，包括高级调度、路由和部署优化。

服务超大规模 MoE 模型的并行策略

由于 GPU 内存有限，高效服务超大规模 MoE 模型需要多种并行形式。下面分解张量、流水线、专家、数据和上下文并行等关键策略，并讨论如何组合使用，在众多 GPU 上分布 LLM。表 15-1从高层汇总这些策略、典型用途及其在推理场景中的详细含义。

表 15-1　LLM 推理并行策略

并行策略	划分依据	使用场景	优点	缺点
张量并行	层内划分（将神经网络权重矩阵拆分到多个 GPU）	单个模型过大，或需要通过多个 GPU 加速层内重计算	计算受限层可能接近线性加速 将 all-reduce 通信与计算重叠可减少开销	每层都需要频繁通信（all-reduce） 需要高带宽互连（NVLink/NVSwitch） 跨节点或慢速网络因高延迟而效率较低 （建议将张量并行组限制在单个节点内）
流水线并行	不同 GPU 放置不同层（按层序列切分模型）	无法装入单个 GPU 的超深模型 跨层扩展内存容量	允许分布模型状态 使用微批处理，并发处理来自不同用户/请求的多个 token 对于长序列可并行处理多个层（提高大批次或长输入的吞吐量）	增加流水线填充/排空延迟（bubble），对逐 token 解码帮助不大 实现更复杂，激活内存占用更高（必须在流水线阶段之间保存中间激活）
专家并行	不同专家放置在不同 GPU（每个 token 稀疏激活）	包含大量专家的超大规模 MoE 模型 需要在 GPU 之间分片模型参数	模型规模可随 GPU 数量扩展，总参数量几乎不受单卡容量限制 每个 GPU 只计算部分 token（稀疏计算） 较高参数量可提升模型容量/质量	每个 MoE 层都有较高运行时通信开销（all-to-all） 门控不均匀时可能出现负载失衡 每个 GPU 上每位专家必须有足够 token，才能摊薄通信成本

并行策略	划分依据	使用场景	优点	缺点
数据并行	在多个 GPU 上复制完整模型，每个副本处理不同请求	模型部署固定后横向扩展吞吐量（增加并发请求） 面向大量用户的多实例服务	吞吐量可能接近线性扩展 实现简单（无需切分模型）	无法改善单个查询的延迟（即*每查询延迟*） 成倍增加内存用量（每个副本占用完整模型内存） 有状态服务（例如缓存）必须处理一致性，或采用无状态模型调用
上下文并行	在每一层将输入序列 token 划分到多个 GPU	超长序列（例如 10 万以上 token），用于降低 prompt 延迟和每 GPU 内存占用	长上下文 prefill 可能接近线性加速 通过拆分 KV cache，处理超过单个 GPU 内存容量的上下文	需要自定义注意力算法处理跨分区注意力 每层都为边界 token 增加通信 由于存在额外通信开销，通常只对超长上下文（10 万以上 token）有益

在 Blackwell NVL72 上进行节点内 TP 时，优先将 TP 组限制在单个 NVSwitch 域内；仅在拓扑允许且不会引入过多额外跳数时扩展到机架间。

这些并行策略定义了如何在 GPU 之间拆分模型权重和数据。图 15-5展示了不同并行策略以及常见策略组合的拆分方式。

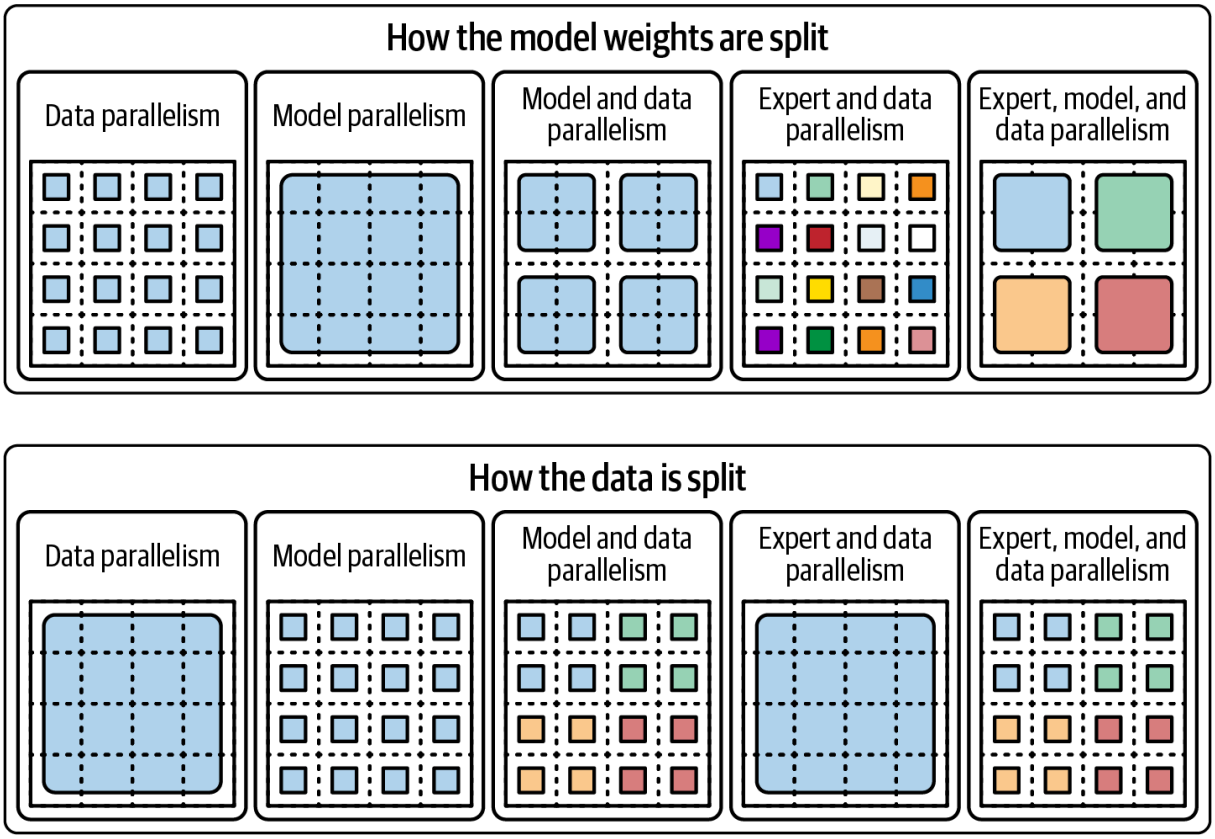


图 15-5　模型权重与输入数据在 GPU 之间的拆分

张量并行

张量并行（TP）将神经网络每一层内部的计算拆分到多个 GPU。例如，transformer 层中的大型矩阵乘法可以按列或按行划分，在两个或更多 GPU 上并行计算；随后这些 GPU 通过 all-reduce 交换并聚合各自的部分输出。

当模型层（即*隐藏层*）过大、无法装入单个 GPU 内存时，通常会使用 TP；也可以让多个 GPU 并行计算同一层，以加速单个模型实例。每层计算期间所有 GPU 必须保持同步，因此高效运行需要极高的 GPU 间带宽。

理想情况下，TP 运行在通过高带宽 NVLink 和 NVSwitch 互连的 GPU 上。在使用第五代 NVLink 的 Blackwell 多 GPU 系统中，GPU 间聚合双向带宽最高可达 1.8 TB/s；TP 可在 8–16 GPU 节点内扩展，在 NVL72 机架中甚至可覆盖 72 个 GPU。GB200/GB300 NVL72 的 NVLink Switch 在 72-GPU 域内提供约 130 TB/s 聚合 GPU 带宽。上述均为平台级峰值或聚合规格，实际应用带宽取决于通信模式与拓扑。

只要激活 all-reduce 等集合通信相对于计算足够快，TP 就可能使模型中计算受限部分接近线性加速。在推理中，张量并行主要应用于注意力投影和前馈多层感知机（MLP）网络中的大型矩阵乘法。

单个 token 的激活相对较小，因此在 NVLink 上执行这些 all-reduce 的成本通常不高。由此，TP 成为在不近似模型权重的情况下，将超大稠密模型拆分到多个 GPU 的常见策略。

TP 主要用于单个 MoE 专家或 transformer 层过大、无法装入单个 GPU 的模型；也可通过在多个 GPU 上并行执行每层计算来降低延迟。

不过，超过一定规模后 TP 效率会下降，尤其是通过 InfiniBand 或以太网跨节点、跨机架使用时。实践中，最好将 TP 用于节点内通信，或限制在同一 NVLink 域内的节点之间。

NVIDIA GB200/GB300 NVL72 等现代多 GPU 机架允许在更大范围内使用 TP。NVLink Switch 将单个 NVL72 的 NVLink 域扩展到 72 个 GPU；相关系统架构还可跨最多 8 个机架、共 576 个 GPU 构建互连。理论上可以形成极大的 TP 组，但“576 路 TP”并不意味着高效：all-reduce 延迟、拓扑层级和模型粒度通常会使收益提前递减，必须按实际系统验证。

> 虽然 TP 可以跨机架扩展，但选择 TP 组大小时应考虑拓扑，并尽可能将其限制在单个 NVLink/NVSwitch 岛内，以避免不必要的机架间交换延迟并提高系统整体效率。

流水线并行

流水线并行（PP）按*层*将模型划分到不同 GPU。例如，在一个 60 层 transformer 模型中，GPU 0 可保存第 1–20 层，GPU 1 保存第 21–40 层，GPU 2 保存第 41–60 层。

处理 token 序列时，数据按顺序流经各 GPU：GPU 0 计算第 1–20 层，然后将中间激活传给 GPU 1 计算第 21–40 层，依此类推。这样便可分布无法装入单个加速器的超深模型。

推理中，PP 可以像装配线一样，通过微批次在各流水线阶段间交错执行来提高吞吐量。在处理长输入序列的 *prefill* 阶段，可将不同微批次错峰送入各层，提高 GPU 利用率。

相比之下，在一次生成一个 token 的 *decode* 阶段，纯 PP 收益较小，因为每个新 token 仍必须依次通过所有流水线阶段。这会产生流水线 bubble（空闲期）：对于每个 token，前面的阶段都要等待后面的阶段完成。

为了减少流水线 bubble，实现通常采用微批处理，让流水线并发处理来自不同请求的多个 token。尽管如此，流水线并行的主要价值仍是内存扩展，即允许超大模型将不同层分布到多个 GPU；它也有助于提高吞吐量，尤其适合大批次或长输入。

由于流水线阶段之间存在传输开销，PP 往往会增加单个请求的端到端延迟。因此，选择 PP 通常是出于模型容量而非延迟考虑：当单个 GPU 无法容纳整个模型时，以一定延迟代价换取模型可装入内存。

PP 还经常与 TP 等并行策略组合，以平衡内存和速度。服务大型 MoE 模型时，可以在 2–4 个 GPU 上使用流水线并行拆分深层堆栈，同时依靠 TP 处理宽大的层内计算；具体规模应由拓扑和性能分析决定。

> PP 在层与层之间拆分模型，TP 则在层内部拆分模型。TP 主要适用于层或专家过宽、单 GPU 无法容纳的模型；PP 主要适用于过深、无法装入单 GPU 的模型。二者可以组合使用。

专家并行

专家并行（EP）专用于 MoE 架构。MoE 层包含许多专家网络，即前馈子层；每个输入 token 只激活一个或少数几个专家，因此天然适合把不同专家分布到不同 GPU。

例如，某个 MoE 层有 16 位专家、系统有 4 个 GPU，则每个 GPU 可承载 4 位专家。推理时，token 到达该 MoE 层后，内部门控网络可以为其选择排名最高的两位专家。随后 token 数据被发送到承载这两位专家的 GPU 处理，再合并结果以生成下一个 token，如图 15-6所示。

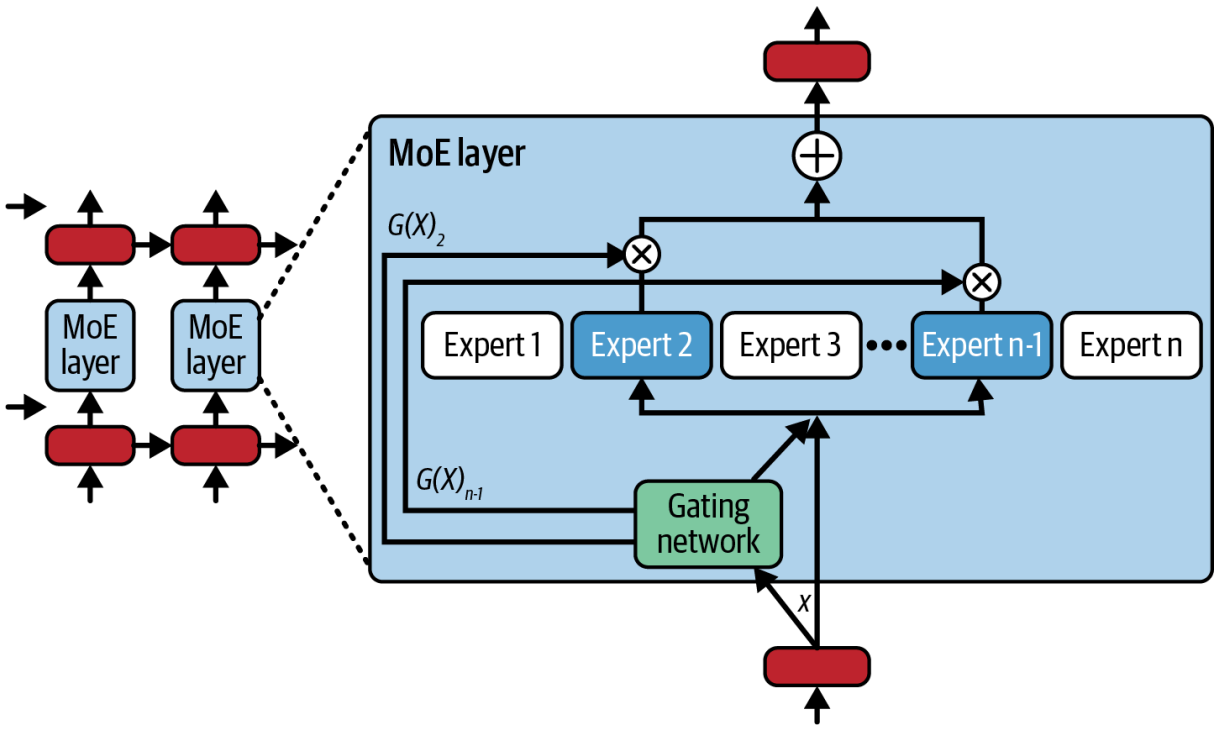


图 15-6　混合专家（MoE）通信（来源：https://oreil.ly/pzn5t）

实现这一过程需要采用 all-to-all 通信模式，在 GPU 之间动态重排 token（严格来说是它们的激活向量），使每个 token 到达其指定专家所在的 GPU。每位专家为分配给自己的 token 计算输出后，再执行一次 all-to-all，将 token 输出恢复到原始顺序。

这种动态路由会在每个 MoE 层引入通信密集步骤。如果不仔细优化，它可能成为性能瓶颈，尤其是在专家/GPU 数量增加时。好处是专家并行可以使模型总容量随 GPU 数量近似扩展。

假设一个 MoE 有 100 位专家，分布在 100 个 GPU 上。如果每个 token 只激活两位专家，则每 token 的计算量与较小的稠密模型相近。专家并行使某些超大 MoE 模型得以实际部署，因为模型权重被分片到众多 GPU，而每个 GPU 在任一层只处理部分 token。

> 在大规模且专家负载均衡良好的 MoE 模型中，所有专家很可能同时处于活动状态。虽然单个 token 只激活少数专家，但汇总所有终端用户的全部 token 后，所有专家都会工作，并发消耗和争用 GPU 资源。

高效的专家并行推理需要精细的负载均衡和高带宽互连，例如机架内使用 NVLink/NVSwitch，节点间使用 InfiniBand/RDMA。现代 MoE 推理框架采用 NCCL 等高速集合通信库。将多位专家组合放在每个 GPU 上，通常有助于减少通信步骤。

现代 MoE 推理经常使用 top-2 门控，即为每个 token 分配两位专家。为了减少通信，可以将经常配对的专家放在同一 GPU 或计算节点。例如，把频繁共同被选中的两位专家放在同一 GPU 或节点上，可使许多 token 分配保持在本地，从而局部化通信流量并降低开销。

另一种方法是使用 top-1 门控，每个 token 只路由到一位专家。与会产生两份专家输出的 top-2 门控相比，这能减少通信量。Top-1 门控速度更快，但可能降低模型质量并导致负载不均。

> Google GLaM 引入了负载均衡损失（以及门控噪声），以便在 MoE 模型训练期间均衡使用专家。在此基础上，研究人员正在探索真正自适应的推理时门控：当某位专家过载时，利用实时负载指标重新路由 token。这类方法旨在不降低质量的情况下提高利用率，但效果需按模型验证。
>
> 不过，在大多数生产服务环境中，采用适度 capacity factor 的 top-2 门控，并偶尔根据负载交换或复制专家，仍是兼顾质量与性能的常见折中方案。

许多 MoE LLM 至少使用 top-2 门控，以兼顾质量、速度和负载均衡。此外，还应合理放置专家，甚至使用备用专家副本，以避免专家过载。

当专家过载或成为热门专家时，如果门控网络不成比例地将 token 路由给它们，就会形成吞吐量瓶颈。所谓*拖尾效应*，是指系统必须等待所有专家计算完成后才能继续。因此，接收工作（token）多于其他专家的过载专家会阻塞推理流水线；等待期间，一些专家及其 GPU 会处于空闲状态。

为了防止这种情况，高性能 MoE 服务系统会提供 capacity factor 参数，常见设置为平均 token 负载的 1.2–1.5 倍，用于限制每位专家在每个批次中可处理的 token 数量。超出部分会被路由到第二选择的溢出专家，或排队等待第二轮处理。这与训练期间使用的负载均衡损失或门控噪声相互补充，后者用于鼓励 MoE 将 token 均匀分配给专家。具体取值应按工作负载调优。

一些功能完整的推理服务器还可在必要时将热门专家复制到多个 GPU，以分摊负载，但代价是消耗额外 GPU 内存。稍后会看到一个负载失衡示例。

结合推理时溢出（capacity factor）、训练时惩罚（负载均衡损失和门控噪声）以及热门专家副本，可以在容量达到上限时平滑 token—专家负载分布，从而提高整体 MoE 推理吞吐量。

数据并行

在推理中，*数据并行*（DP）是指在多个 GPU 上复制完整模型，并将不同传入请求或请求批次的不同分片分配给各 GPU。训练数据并行通过梯度平均保持模型同步，而推理数据并行让每个副本独立执行前向传播。这是横向扩展吞吐量最简单的方法。

例如，如果一个 GPU 每秒可处理 10 个请求，那么在请求相互独立且没有其他瓶颈的理想情况下，8 个 GPU 上的 8 个副本可达到每秒 80 个请求。实际使用推理数据并行时，需要启动多个模型服务引擎，并在它们之间对请求进行负载均衡。

使用 DP 时，每个 GPU 或 GPU 组从头到尾处理一部分查询。由于各副本彼此独立，推理期间通常不需要为模型计算进行跨副本 GPU 通信，吞吐量可能接近线性扩展；但它也有显著缺点。

DP 会成倍增加内存需求，因为每个副本都需要完整的模型权重和缓存。因此，与单 GPU 承载模型相比，使用 8 个副本会占用约 8 倍 GPU 内存，硬件成本也大致增加到 8 倍。

实践中，数据并行通常与请求批处理和每 GPU 多 stream 执行结合，以最大化利用率。每个副本最好保持无状态，或使用适当的一致性/亲和路由机制管理缓存，避免一致性问题。

需要注意，DP 不会降低单个请求的延迟；但在内存和计算资源充足且争用较少时，更多副本可同时处理请求，从而提高吞吐量。

> GPU 内存相对稀缺且昂贵，因此推理 DP 通常只在吞吐量需求很高，或能与 TP、PP 等其他并行方式结合时才值得采用。

由于 DP 会增加内存和成本，推理中常将它与 TP、PP 等方式组合。例如，若模型因内存限制必须分布到 8 个 GPU，可将 DP 与 TP、PP、EP 中的一种或多种结合，形成所谓四维并行；再加入上下文并行（CP），则可称为五维并行。

具体而言，可以结合 DP 和 TP，使用两个各含 8 个 GPU 的 DP 组部署两个大型模型副本，并在每个副本内通过 TP 将层内计算分片到 8 个 GPU。在没有其他瓶颈的理想情况下，两个副本可使吞吐量接近翻倍。

在大规模推理集群中，可以投入大量 GPU 和节点，运行大型模型的多个副本以处理高请求量。当吞吐量需要超过单个模型实例的能力时，DP 尤其有用。生产系统常运行多个大型模型 DP 副本来满足流量需求。

> 现代推理服务器通常把每个副本视为负载均衡器后的独立模型实例。虽然需要仔细进行请求路由，但与复杂的模型分片方法相比，这种方式相对直接。

上下文（序列）并行

上下文并行（CP）是一种更专门的策略，将单个 token 序列拆分到多个 GPU。它适用于包含数万乃至数百万 token 的超长上下文；否则，这类输入可能速度过慢、内存开销过大，甚至无法装入单个 GPU。

CP 的思路是将序列切分成多个块，让不同 GPU 在每一层并行处理序列的不同部分；GPU 只在序列块边界交换必要信息。

CP 通过在多个 GPU 上拆分 KV cache，处理超过单 GPU 内存容量的上下文。它还可能利用多个 GPU 并行执行长上下文 prefill，降低 prompt 处理延迟。在计算与通信条件有利时可接近线性加速，但并非简单与 GPU 数量成正比。

CP 的难点在于 transformer 具有全局自注意力，每个 token 都会关注此前所有 token。若简单拆分序列，为计算跨分区边界的注意力，GPU 之间需要交换大量信息。

CP 方法采用环并行、分块注意力等方案，降低自注意力通信成本，并使每个 GPU 主要处理自己上下文分区内的注意力，仅交换少量块边界数据。具体复杂度取决于所用算法，不能笼统保证消除二次复杂度。

实际上，每个 GPU 在每层处理输入序列中的一部分位置，并以流水化方式传递中间结果，常组织成环。CP 类似流水线并行，但它沿序列长度维度而不是层维度拆分。

上下文并行适合超长输入的 prefill：将文档切成片段，分配到多个 GPU 并发处理。GPU 数量翻倍时，prompt 处理时间在理想情况下可能接近减半，但跨片段注意力、通信和负载不均都会带来额外开销。

简而言之，CP 无法加速逐 token 的顺序 decode 阶段，但可以缩短超长 prompt 的 TTFT。它把注意力 KV cache 分布到多个设备，让每个 GPU 只存储自己片段的缓存，也使系统能够处理超过单 GPU 内存容量的上下文。

> 上下文并行需要能够跨分区通信的注意力实现，因而每层都会增加通信。对于短 prompt，CP 往往得不偿失；但面对超长输入和严格 TTFT 目标时，它可以降低 prefill 延迟及每 GPU 内存占用。应在最大上下文长度下同时测量 prefill 速度和准确性。

混合并行

实际服务超大规模 MoE LLM 时，会组合使用前述多种并行策略。现代 LLM 规模庞大且结构复杂，单一并行方法往往不足。图 15-7展示了使用 4 个 GPU 的混合并行配置。

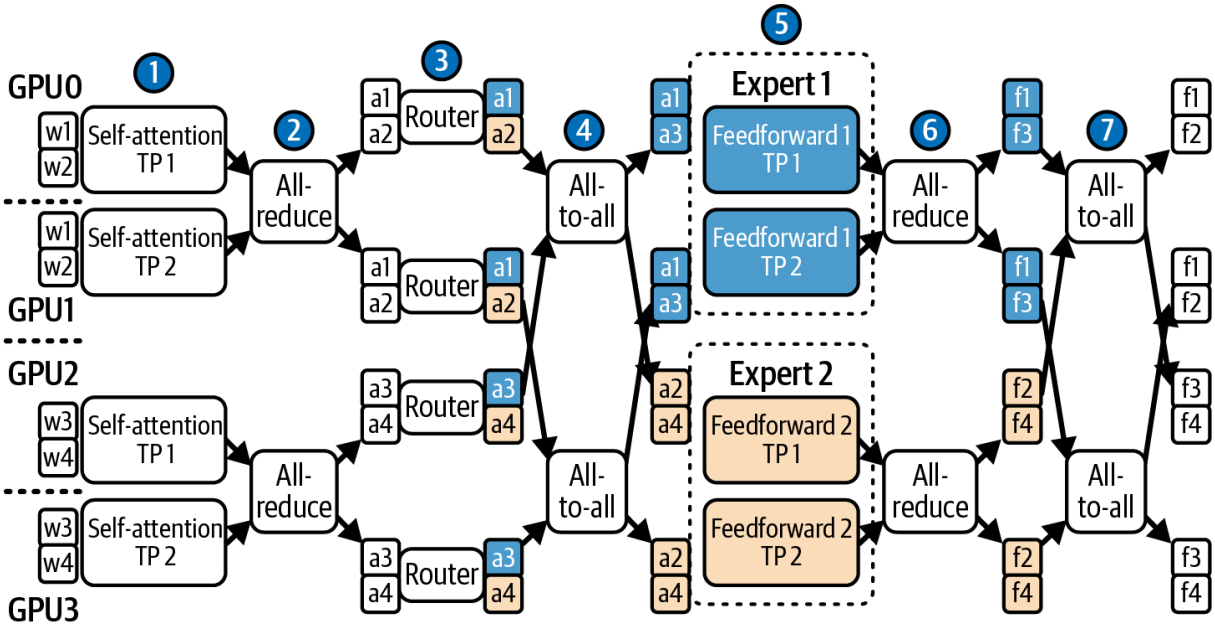


图 15-7　4 × 2 × EP 混合并行组合的高层示意图（来源：https://oreil.ly/q1AEf）

这里使用 4 个流水线阶段（每个 GPU 一个）和 2 路张量并行，并通过专家并行将 token 路由到两位专家。这称为 *4 × 2 EP 混合并行策略*。

还可以进一步扩大规模并创建 GPU 逻辑组。例如，对于包含 64 个 GPU 的集群，可将其分成 16 组，每组 4 个 GPU。假设 MoE 有 60 层和 64 位专家，可以使用 4 路流水线并行，每个阶段 15 层，从深度方向拆分模型；随后在每个阶段（15 层）内部采用 2 路 TP，拆分每层中的重型矩阵乘法，从宽度方向拆分模型。

对于 MoE 层，可以使用 EP，在每个组内分布 16 位专家，并采用 top-2 门控，使 token 最多发送到组内两个 GPU。最后，数据并行可以部署两个这样的 64-GPU 副本，以提高系统吞吐量；是否能够翻倍仍取决于负载均衡和共享基础设施瓶颈。

这只是一种配置。实际部署中，需要试验不同的并行策略组合，并借助性能分析工具寻找适当平衡。可以用 Nsight Systems 获取端到端追踪，用 Nsight Compute 查看特定内核的 GPU 指标。

> 每次更改前后，都要验证互连流量、Tensor Core 利用率及其他性能机制。

指导原则是使用 TP，直到收益开始递减，通常限制在单个节点或 NVL72 机箱等紧耦合单元内。随后只使用足以让模型装入内存的最少流水线并行；再尽量利用 EP，将 MoE 参数分布到专家和 GPU。最后，随着终端用户和并发推理请求增加，添加数据并行副本提高吞吐量。如果预计会有超长输入，还可以叠加 CP。

并行方式还应与硬件拓扑对齐。例如，在 NVL72 系统中，72 个 GPU 通过 NVLink 和 NVSwitch 组成统一高速互连域，可以在该域内构建 TP 和 EP 组。但当 TP 组接近整个域规模时，all-reduce 延迟往往导致收益递减，因此许多生产系统即使在 NVL72 内也会采用更小且拓扑感知的 TP 组。

相比之下，由 InfiniBand 连接的两个 8-GPU 节点，通常只有节点内部具备完整 NVLink 连接，跨节点流量需要经过 InfiniBand。在这种环境中，最好将 TP 限制在每个 8-GPU 节点内，并在可能时避免让高频层内集合通信跨节点，以规避更高延迟和较低有效带宽。

下面几节将讨论可与这些并行策略结合的更高层优化，以进一步加速多节点集群推理。设计良好的服务系统会组合多种高层和低层技术。

推测解码与并行 Token 生成技术

LLM 推理的基本性能挑战之一是解码的顺序性。Prefill 处理完初始 prompt 后，模型通常一次生成一个 token；每个 token 的计算依赖上一个 token 的结果，因此本质上是难以并行的串行过程，即使使用最新、最快的 GPU 也会形成延迟瓶颈。对于超大模型，在现代 GPU 上顺序生成数百个 token 仍可能耗时数秒，具体取决于模型、批次和硬件。

本节讨论几种加速 decode 阶段的技术。有些方法在每个推理步骤生成多个 token，另一些则直接减少顺序推理步骤的总数。

传统推测解码将一个小而快的“draft”模型与更大的“target”模型配对：draft 模型在一个批次中提出多个 token，target 模型再验证各候选项。它用额外一次推理过程换取并行生成，以提高多 token 吞吐量。不过，运行两个独立模型会增加部署复杂性；当验证过程成为瓶颈时，推理仍可能停滞。

Medusa 通过直接在单个 LLM 上附加多个轻量级解码头来简化这一流程。每一步中，这些解码头使用基于树的注意力机制，在一次前向传播内并发生成并验证多个 token 候选。

Medusa 的统一设计避免跨模型 token 传输，并将 draft-and-verify 合并到单模型过程。它可能在保持生成质量的同时获得可观提速，但实际收益取决于接受率、批大小和硬件。下面仔细介绍这些技术。

双模型、基于 Draft 的推测解码与 EAGLE

推测解码通过让小模型执行额外工作，节省昂贵大模型的时间。其思路是在主 LLM 旁运行一个轻量级“draft”LLM；draft 模型速度更快，会基于当前上下文推测生成一批 k 个 token。

随后，大模型（称为 *target* 模型）在发送给 GPU 的单个批次中，为整个 k-token 序列预测下一 token 概率，从而验证 draft token。一次处理全部 k 个 token，可以让 target 模型在每字节传输数据上执行更多计算，提高算术强度并更高效地验证 draft 序列。

如果 target 模型输出与 draft 模型提出的 k 个 token 一致，就相当于在大模型通常生成一个 token 的时间内生成了 k 个 token。如果大模型验证在序列中某个位置开始与 draft 预测不一致，则丢弃该位置之后的推测 token。标准验证过程每一步至少会保留一个由 draft 或 target 模型确定的 token。

使用 target 模型修正后的 token 并继续解码后，即可从该位置开始新的推测解码周期。图 15-8展示了小型 draft 模型提前预测多个 token，再由大型 target 模型验证这些 token。

总体来看，推测解码减少了大模型逐个 token 顺序调用的次数。理论上，上限与 draft 模型每次生成的 k 个 token 有关；实际由于额外开销和推测 token 被拒绝，提速通常远低于 k 倍，常见实验结果约为 2 倍，但并非普遍保证。

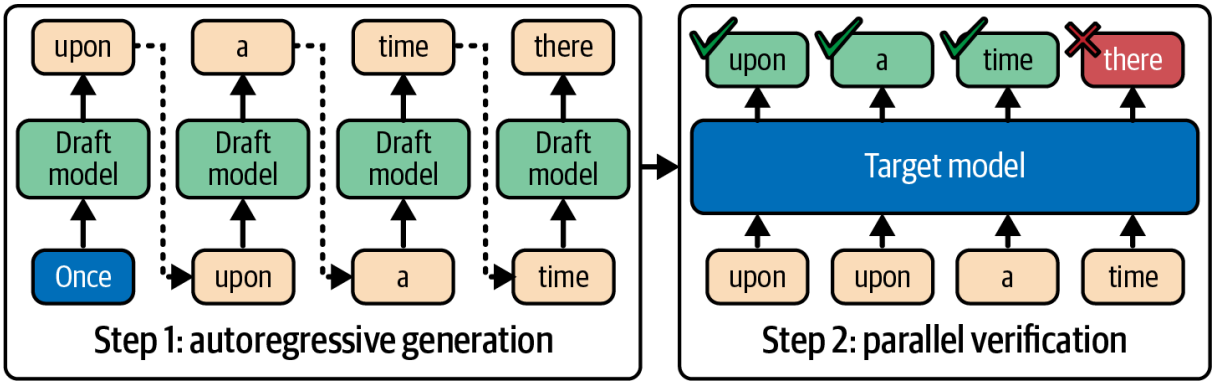


图 15-8　使用 draft（小）模型解码，并由 target（大）模型进行多 token 验证的推测解码

所选 draft 模型必须与大模型输出分布具有较高一致性，即其预测应与大模型的高概率输出高度重叠。如果 draft 经常猜错，推测解码收益很小且会浪费计算；如果它频繁预测 target 模型不会选择的 token，大量推测 token 会被拒绝，同样浪费计算时间。

Draft 模型还必须显著快于大模型；原文给出的常见经验是至少约 4 倍，推测解码才更可能获得实际性能提升。常见策略是使用主模型的蒸馏版本，或在相同数据上微调、输出相关性较高的小模型。具体阈值取决于接受率和部署开销。

> Draft 模型必须与大模型使用相同的 tokenizer 和词表。这一点有时会被忽略，从而导致较差结果。

Draft 模型使用相同 prompt 生成 token，也可能采用不同温度设置来改变候选多样性；但温度必须与接受/校正算法一致，不能简单假定更高温度必然提高匹配率。与此同时，target 模型向前推进，在一个批次中处理全部 draft token。

在标准推测解码接受程序下，只要采样设置和校正过程一致，就会保持 target 模型的输出分布，因此最终样本与大模型分布相符。如果 draft 偏离，target 模型验证会拒绝相应 token，从而保持正确性。区别在于，小模型正确猜中推测 token 时，可以减少 target 模型的顺序调用次数。

推测解码有多种实现方式。vLLM、SGLang、TensorRT-LLM 等现代 LLM 推理引擎通常内置了协调 draft 与 target 模型生成的支持。部分实验中常见约 1.5–2.5 倍提速；通过谨慎设计批处理和 draft 模型可能获得更高收益，但结果依模型、负载和硬件而异。

> 在 PyTorch 中，`torch.nn.functional.scaled_dot_product_attention` 会根据设备代际、形状、mask 和 dtype 自动选择可用后端（例如 FlashAttention 或 cuDNN 内核）。可以使用 `torch.nn.attention.sdpa_kernel()` 并指定 `SDPABackend` 类型来固定后端。对 LLM 解码进行基准测试时，应验证预期的融合后端确实已启用。

例如，用于提高 LLM 效率的外推算法（EAGLE）不在 token 层，而是在特征层重新设计推测解码。EAGLE 对大模型自身的中间表示进行一步外推，以预测下一个 token 的特征，旨在降低不确定性并提高接受率。

EAGLE 论文报告称，在使用 4-token draft 时，相比普通解码最高获得约 3.5 倍提速，同时保持大模型输出分布。在有利实验环境下，它接近相应 draft 深度的理论上限。这表明推测解码可以进一步缩短解码时间并保持输出质量，但该数字是特定评测结果。

EAGLE-2 通过引入上下文感知的动态候选*草稿树*扩展 EAGLE。EAGLE 在部分评测中相对普通解码最高提速 3.5 倍；EAGLE-2 根据任务和模型，报告称比 EAGLE 约快 20%–40%。其 draft 模型会生成具有分支的 token 序列集合，进一步提高验证步骤的并行度，如图 15-9所示。这些均为特定论文评测结果。

EAGLE-3 偏向直接预测 token 并融合多层特征，在 EAGLE-1 和 EAGLE-2 基础上继续改进。EAGLE-3 报告称，在某些任务中相对 EAGLE-2 最高提升 1.4 倍，相对未优化基线变体最高提速 6.5 倍。EAGLE-1 和 EAGLE-2 的 draft 模型先预测内部特征向量，再将其解码为 token；本质上是猜测内部特征后映射到 token。上述性能数字同样来自特定评测。

EAGLE-3 跳过显式的特征级预测步骤，更直接地预测 draft token。不过，它仍会使用内部特征，并融合低层、中层和高层的多层表示来引导 draft 预测，而不是只使用顶层。这使流程更精简、约束更少，并有利于扩展。

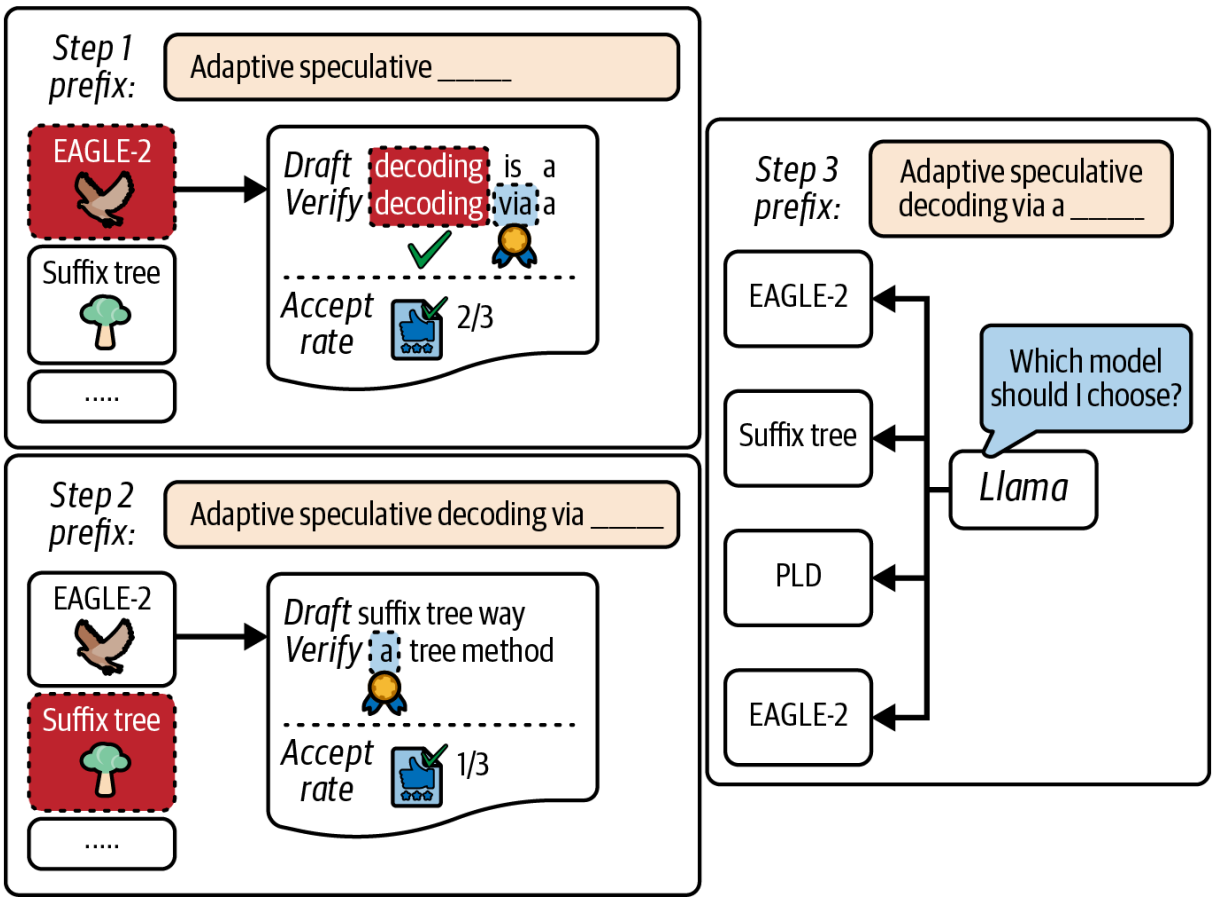


图 15-9　使用 EAGLE-2 的推测解码（来源：https://oreil.ly/uG07b）

另一种技术是*动态深度解码*，它会自适应跳过对输出质量影响较小的层。其他减少计算的方法包括跳过每第 N 个 transformer 层、让 draft 模型使用较低精度（例如 FP8 和 NVFP4；原文的“FFP8”疑为笔误），以及在 draft 阶段临时使用较小的隐藏维度。

一些研究原型提供跳过部分网络或在解码时降低精度的模式，以准确率换取速度。截至本书撰写时，这些模式尚未普遍用于生产模型，因此启用前务必在目标任务上验证质量和速度。

未来的 LLM 可能内置优化的“快速生成”模式，例如提供替代性的轻量层或低精度解码配置，让模型能够跳过部分计算。

单模型自推测解码

推测解码的另一种方法完全不使用外部 draft 模型，而是让大型 target 模型通过选择性跳过部分计算，既生成自己的草稿，又验证输出。这种方法称为自推测解码，也称 draft-and-verify 方案。

在自推测解码中，大型 target 模型先通过快速近似过程生成 k 个 token。例如，它可以针对每个新 token 只运行一半层（隔层跳过），并可能使用较低精度，从而像小型 draft 模型一样产生草稿输出。当接受率和草稿深度有利时，可能获得相近提速；原文提到约 2 倍，但这取决于具体实现和负载。

在自推测解码的第二阶段，target 模型执行完整前向传播，一次验证这 k 个 token。如果全部匹配，就节省了 draft 阶段未执行层的计算；如果不匹配，则接受不一致位置之前的 token，并按标准接受/校正流程继续。

由于同一模型同时负责生成草稿和验证，自推测解码无需训练、维护或在运行时加载独立模型。难点在于如何通过丢弃层、降低精度等方式减少 draft 阶段的计算量，同时又不过度损害准确率。

相关技术还有*一致性解码*，即训练一个 LLM 同时生成并验证多个 token。该单模型方法在特定实验中无需独立 draft 模型便获得约 3 倍提速，体现了将推测解码能力融入模型自身权重的趋势。

这些方法仍是非常活跃的研究领域。它们尝试利用模型内部冗余让模型自行加速；由于优化局限在模型内部，还可能简化推理引擎的外部协调逻辑。

使用 Medusa 多头进行多 Token 解码

传统推测解码最终仍由 draft 模型逐个生成 token，只是因为 draft 模型较小而更快。Medusa 则采用更彻底的方法：修改模型架构，使其在每个解码步骤并行预测多个新 token。

双模型推测解码仍是逐个生成 token，只是速度更快；Medusa 架构则真正由单个模型在每次迭代生成多个 token。已发表实验中，Medusa-1 和 Medusa-2 的多头方法报告约 2.2–3.6 倍提速，实际收益依模型和服务配置而异。

不过，Medusa 需要自定义模型训练，因为它会修改 transformer LLM，在某些层分出额外解码头，因此得名 *Medusa*。这些解码头让模型能够同时提出多个后续 token。

不同 Medusa 头生成的多个 token 候选组成树形结构。例如，Medusa 可以在一次处理中生成深度为 2 的二叉树，产生最多 4 个候选 token，然后并行验证多 token 序列，如图 15-10所示。

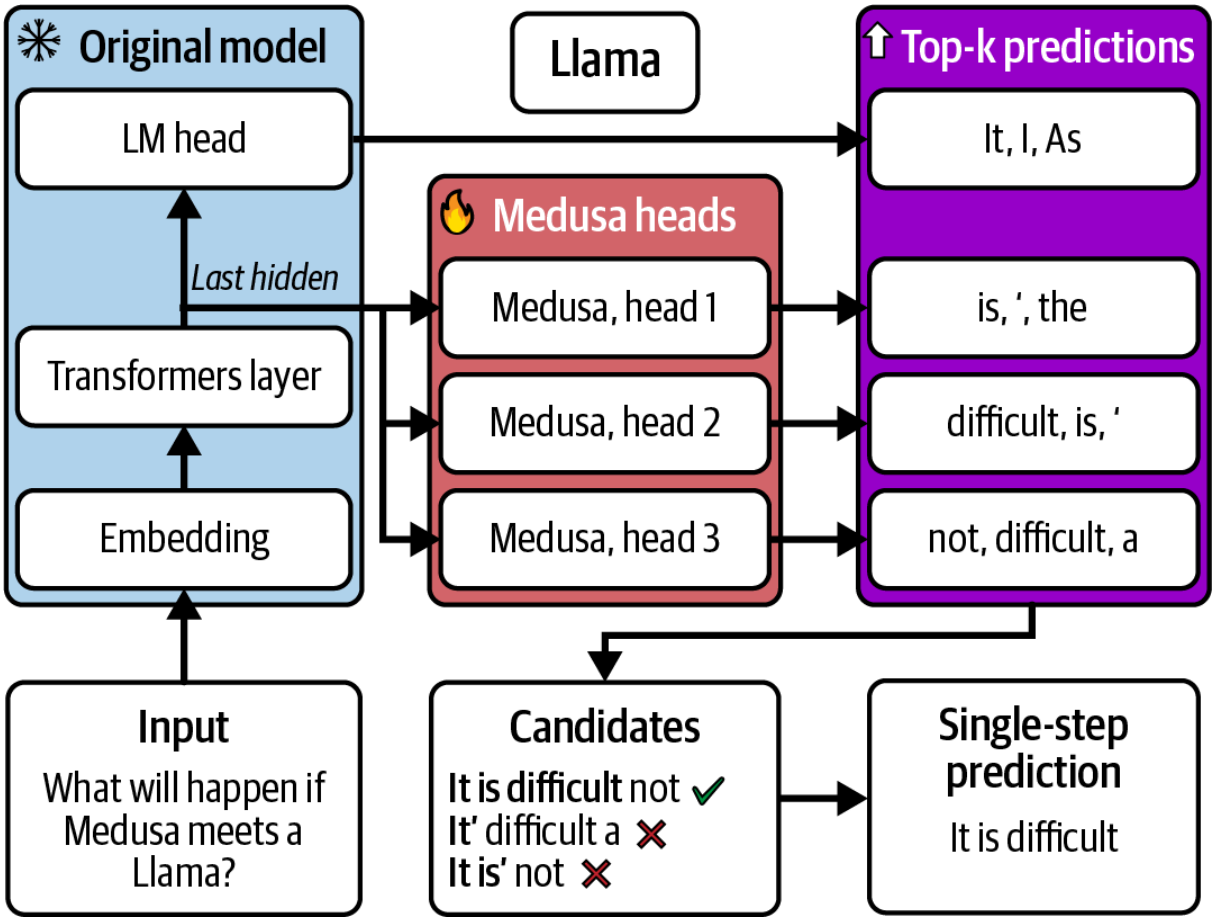


图 15-10　使用 Medusa 进行多 token 解码（来源：https://oreil.ly/MJMOQ）

Medusa 内部使用专门的树形注意力模式，提高并行 token 预测之间的一致性。模型学习并发扩展并验证多 token 序列；本质上，LLM 学会提前预测数个 token，并一次性输出。

推理时，Medusa 可以通过并行预测减少顺序解码迭代次数。例如，如果一次预测 4 个 token，对于深度为 2 的二叉树输出，理论上可将所需前向传播次数最多减少到四分之一。

但在实践中，由于预测分支验证失败时偶尔需要回溯并部分重做，Medusa 在相关实验中通常约为 2–3 倍提速。这类似于推测解码的验证和拒绝开销。只有在每次迭代生成的 4 个 token 都被接受的理想情况下，100-token 回复才只需 25 次而非 100 次模型迭代。

Medusa 需要修改并重新训练模型以添加额外解码头。额外头参数也会略微增大模型，增加开发复杂性和训练成本。训练完成后，它可能显著缩短推理时间，但收益取决于接受率和目标负载。

如果针对具体用例完成训练和验证，支持 Medusa 的 LLM 可以提供较好的低延迟性能。不过，该技术要求修改并微调模型架构以添加额外解码头。

交错执行多个请求的 Decode 步骤

另一种并行解码技术是在多个终端用户的并发请求之间交错执行 decode 步骤。这更多是一种推理引擎能力，而非算法技巧或模型架构修改；它通过跨请求进行 token 级批处理，使 GPU 保持忙碌。

vLLM 等框架通过请求路由、连续批处理和 token 调度实现这一机制。其思路是填补顺序步骤之间的空隙，让 GPU 保持忙碌。具体来说，如果某个序列等待 I/O 或数据依赖而停滞，调度器可让同一 GPU 暂时执行另一序列的下一 token 预测。

> 可将推理引擎的高级路由、批处理和调度能力与 CUDA stream 结合，再用 Nsight Systems 验证 token 步骤内核是否与 NIC/NVLink 传输重叠，而不是在单个 stream 上串行执行。这样可以在应用、系统和硬件层实现大规模并行。

需要注意，交错 decode 步骤不会降低单个序列的延迟；额外上下文切换甚至可能稍微增加每 token 开销。不过，在并发服务大量用户时，它可以显著提高总体吞吐量和 GPU 利用率，并可能降低高负载下的平均请求延迟。

组合解码技术并评估复杂性

这些解码优化可以组合。例如，可以让支持 Medusa 的 target 模型参与推测解码，由大模型一次验证小模型预测的多个 token。

这些技术也会增加复杂性：需要维护额外模型、修改主模型，或管理更复杂的控制逻辑。生产环境应权衡这种复杂性与性能收益。对于交互式应用，数毫秒的改善是否值得额外复杂性，应结合规模、SLO 和维护成本判断。

总之，无论是否使用独立 draft 模型，推测解码和 Medusa 式多 token 预测等高级技术都可能缩短传统逐 token 自回归 LLM 的总体推理时间。一次生成更多 token 可以提高总体算术强度，使解码步骤更接近计算受限区间，从而更充分利用 GPU 计算能力。

> 解码优化仍在持续演进，复杂性也不断增加，但方向很明确：让 LLM 解码更加并行、高效。

约束解码对性能的影响

LLM 解码过程另一个独立但重要的方面是在特定约束下生成文本。例如，可以强制输出匹配预定义格式（JSON）、遵循特定语法，或出于安全考虑禁止某些序列。因此，约束解码（通常称为*结构化输出*）经常是生产用例的必要能力。

例如，OpenAI 的 function-calling API 依赖结构良好的输出格式来确定要调用的函数及其输入和输出。这些约束必须匹配函数 API 签名，在应用层面不可协商。

约束解码会改变模型的 token 选择过程，使每一步只允许有效 token。实现可以很简单，例如提供允许 token 列表；也可以很复杂，例如嵌入形式化语法，并使用状态机过滤无效 token。

约束解码的缺点是强制执行约束需要额外延迟。朴素实现可能每 token 增加数毫秒，而优化的语法掩码引擎可以显著降低成本；确切开销取决于运行时、语法结构和实现路径。回溯发生在 LLM 生成循环内部，因此若缺少细粒度遥测，调试、分析和调优会比较困难。

另一个性能因素是缓存效率。每个 decode 步骤都裁剪完整概率分布，可能冲刷缓存并进一步降低吞吐量；允许 token 集合受到严格限制时尤为明显。

为缓解这些成本，许多框架会编译 JSON 语法，并为每个状态预计算有效 token，使推理引擎能够在运行时屏蔽无效 softmax 输出。这种方法可减少回溯、改善缓存并提高接受率。

对于简单 JSON schema 等中等约束，使用编译语法并将掩码计算与 GPU 执行重叠的现代引擎，在大规模场景中可能将开销压低到个位数百分比；但复杂语法或小批次仍可能产生两位数百分比开销。务必在实际 schema 下测量 TTFT 和每秒 token 数。

这些技术已在 Hugging Face Transformers、NVIDIA NeMo 和 vLLM 等常用库及推理引擎中实现。例如，TensorRT-LLM 和 Hugging Face Transformers 支持用户定义词表 mask 来实施约束。TensorRT-LLM 通过 XGrammar 后端提供支持 JSON schema 和上下文无关语法（CFG）的 guided decoding；vLLM 和 Transformers 也提供结构化输出 API。性能开销取决于后端和约束，不能一概视为可忽略。

> 使用 TensorRT-LLM guided decoding 时，XGrammar 提供 JSON schema 和 CFG 支持，并可将部分约束处理放在 GPU 路径中。XGrammar 会编译约束，避免较大的 Python 侧 token-mask 开销。但某些配置可能回退到较慢后端，导致首次请求明显停顿。因此，应尽量保持语法紧凑，以维护缓存局部性和 token-mask 带宽。

另一种常见策略是将约束解码下沉到内核中，在最终 softmax 步骤注入 token mask，以获得类似的性能收益。

高效实现时，约束解码通常可以接近无约束解码的速度，尤其当约束规则集不太大或不太严格时。如果约束过于严格，解码实际上会变成在小型 token 空间中的搜索，导致更多回溯和更慢的 token 生成。

> 可能时应避免过大或过于严格的约束，包括复杂语法、格式和词表。一种选择是让 LLM 正常解码，再对输出执行后处理或过滤；但这并不总是高性能或可行。应分析不同方案，选择最适合具体用例的方案。

MoE 推理的动态路由策略

要高效服务 MoE 模型，需要使用专家并行，将专家仔细划分到多个 GPU。此外，还需要智能、动态的机制（即门控网络），在运行时将 token 动态路由到这些专家。

MoE 推理期间，每个 token 的前向传播都必须由门控网络路由到一个或多个专家 GPU。系统需要以均衡方式处理通信，使 GPU 保持忙碌并避免 GPU 过载或出现热点。下面讨论在大规模多节点推理环境中处理这些问题的方法。

专家通信优化

在专家之间执行 token 激活的 all-to-all 交换时，MoE 会在 GPU 之间重排一批 token。每个 GPU 只接收发往其承载专家的 token，如图 15-11所示。该操作发生在 MoE 的每一层，成本很高；如果处理不当，可能占据大部分推理时间。

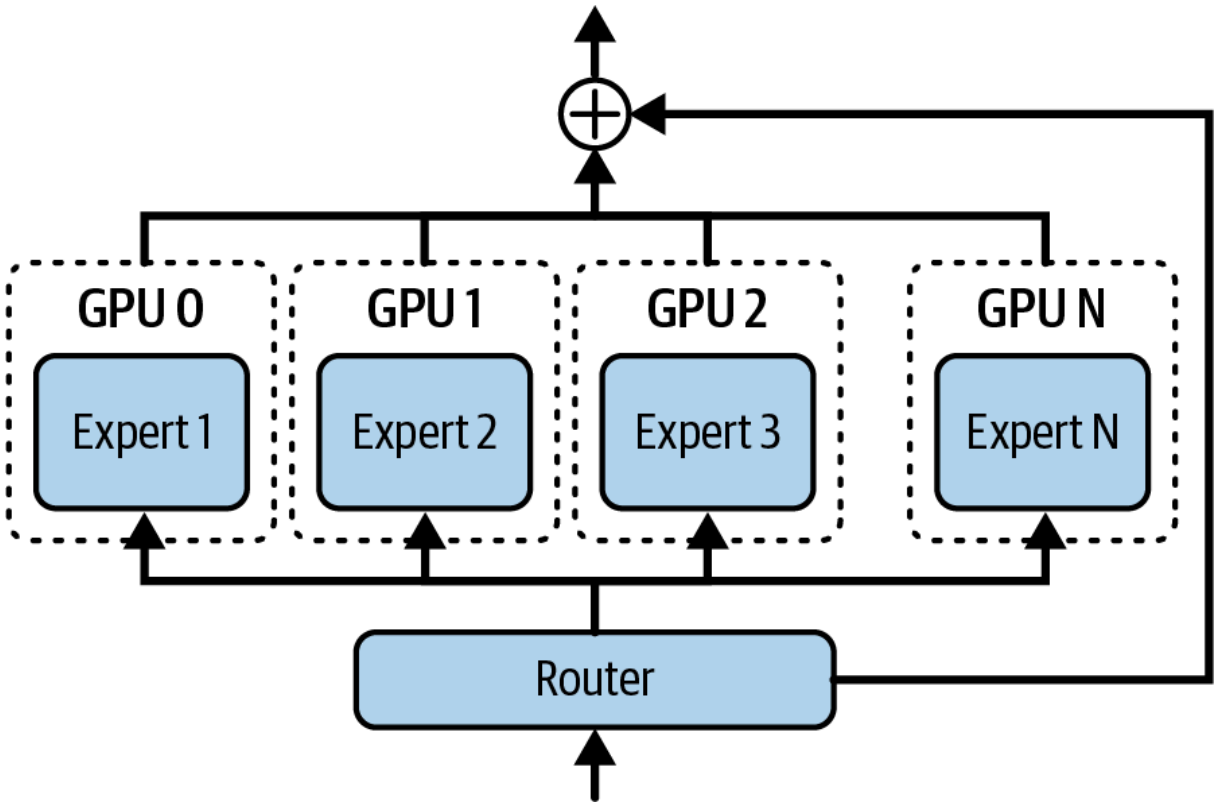


图 15-11　每个 GPU 只接收发往其承载专家的 token

降低通信开销的一种方法是对 GPU 集群采用分层路由：先通过高速 NVSwitch/NVLink 在同一节点内的 GPU 之间路由 token，只有需要非本地专家的剩余 token 才跨节点路由。这种两阶段 all-to-all 可以减少节点间流量。

还可以采用异步通信，将通信与计算重叠。高性能 MoE 推理服务器使用双缓冲通信：发送当前一批 token 的同时，并行执行上一批 token 的专家计算。该流水线可以隐藏大量通信延迟。

> 如果节点间网络保持充分利用，同时节点内重排在后台运行，all-to-all 完成时间可能接近相应拓扑的最佳水平。务必同时测量节点间 NIC 和节点内 NVLink 路径的链路利用率。

使用单个全局 all-to-all 屏障的朴素专家路由实现，可能让 GPU 等待同步；如果并非所有链路都得到充分利用，还会浪费带宽。

蝶形调度（也称*错位 all-to-all 调度*）等技术可以将通信拆成多个阶段轮次，使各 NVLink/NIC 执行部分交换，而不是等待一次大型同步。这种错峰方法可提高链路利用率并减少空闲时间。

All-to-all 交换可以使用内置 `ncclAllToAll` 集合操作，或分组 send/receive 调用。将交换分块并流水化，或跨节点使用蝶形等分层调度，通常可以提高吞吐量。应通过验证选择与拓扑匹配的算法。

> 第一阶段 all-to-all 尽量保留在机架内（NVLink），仅让剩余 token 溢出到机架间。分析链路利用率，确认 NVLink 在后台重排时 NIC 得到充分利用。当节点间链路成为瓶颈时，建议对 MoE all-to-all 通信和专家计算采用双缓冲。分块/流水化交换以及蝶形/错位调度可避免全局屏障减速；是否优于全局集合操作必须按拓扑实测。

减少通信流量的另一种方案称为*专家共置*：将某些专家放在同一 GPU 或节点上，避免不必要的通信。假设 token 路由器经常为同一 token 激活专家 5 和专家 7，将二者放在同一 GPU 上即可消除一次额外 all-to-all 跳转。性能分析工具和门控频率分析可帮助识别适合具体负载的专家配对。

还可以压缩专家间通信，包括专家交换的激活。例如，在执行 all-to-all 前，使用 NVIDIA Transformer Engine 将激活转换为 FP8 或 NVFP4。这样可降低 NIC 负载，但需要承担转换、打包/解包开销，并可能损失数值精度。只有当节省的网络和内存传输成本超过压缩计算成本时才有收益，应验证精度与性能。

总之，优化 MoE 通信需要分析硬件和网络拓扑，并据此优化专家在 GPU 上的放置。部署时应配置集群互连，以高效执行 all-to-all。例如，可利用 NVL72 机架的 NVLink Switch mesh，在单个域内连接最多 72 个 GPU。朴素的 all-to-all 选择可能占据大部分层执行时间，并造成很低的 SM 效率。应优先处理专家流量，尽可能实现重叠，并在选择不同互连和通信算法时进行分析与验证。

> 对于 MoE，应针对 all-to-all 通信配置集群：为拓扑选择合适的 NCCL all-to-all 算法或分组 send/receive 实现，并确认节点间路径已启用 GPUDirect RDMA。还应根据网络设计正确配置 InfiniBand 多链路；是否采用链路聚合、如何组合带宽和实现故障转移，取决于交换网络与软件栈，不能简单视为单一逻辑通道。换言之，需要同时调优硬件和软件层的 MoE 网络拓扑。

负载均衡、Capacity Factor 与专家复制

应尽量让每位专家和每个 GPU 获得较均衡的工作量，避免“热点”。如果 MoE 门控网络将不成比例的 token 路由到某位专家，承载它的 GPU 就会过载，增加延迟并降低系统总体吞吐量。

例如，某个专家 GPU 成为热点，利用率达到 99%，而其他专家 GPU 平均约为 60%。如果处理不当，这个热点就可能成为整个训练或推理集群的瓶颈。

训练期间，可以添加负载均衡损失项来缓解热点：如果门控过度使用某些专家而很少使用其他专家，就施加惩罚，使训练后的 MoE 倾向于在专家之间更均匀地分配 token。

但在推理时，特定输入 prompt 或主题仍可能集中激活一小部分“热门”专家，造成失衡。避免推理热点的一种策略是使用 *capacity factor* 触发溢出机制。

通过指定 capacity factor，可以限制每位专家在给定时刻最多处理的 token 数量（例如 32 个）。如果收到的 token 超过容量，额外 token 可以转发给路由分数次高的后备专家，或串行排队，在第二轮处理。图 15-12比较了 1.0 与 1.5 两种 capacity factor。

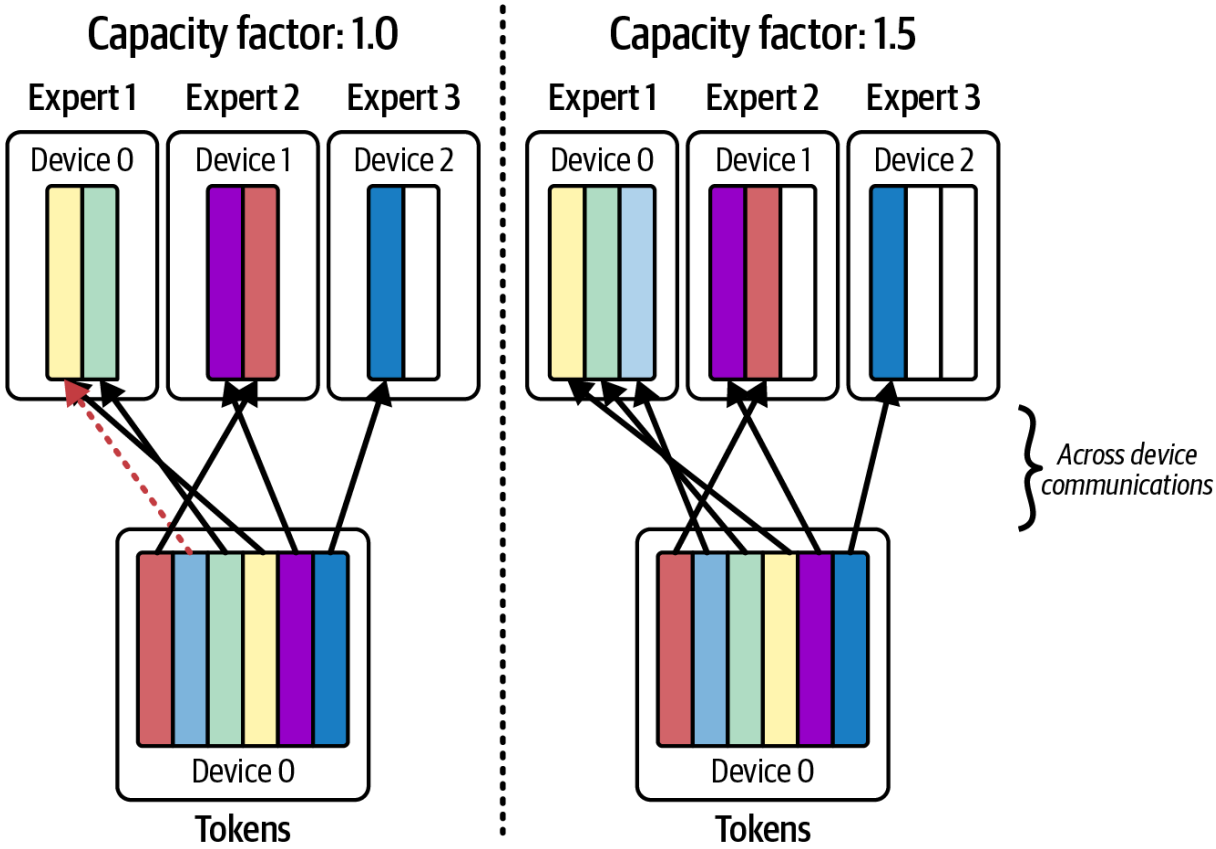


图 15-12　比较 1.0 与 1.5 的专家 capacity factor

实践中，top-2 门控常使用约 1.2 的 capacity factor（允许比平均负载高 20%）。这表示每位专家最多接收其平均负载的 120%，超出部分发送给下一位专家，从而平滑系统内的专家负载。该值只是常见起点，应按模型和负载调优。

避免热点的另一种策略是专家复制。如果某位专家持续成为热点，系统可将其复制到另一个 GPU，让门控函数把一部分 token 发送给运行在另一 GPU 上的专家副本。

副本是由推理引擎实现的应用层优化，模型本身并不知道它们。具体引擎可能维护原专家到多个物理副本的映射并据负载分流；原文称将副本注册为具有独立索引的专家，但这不是所有实现的通用方式。

> 复制专家会增加相应专家的内存占用和成本。不过，通常只有少数专家会过载，因此只复制少量热门专家是一种针对性修复，无需复制整个模型和所有专家。

如果模型会更新，复制还需要谨慎同步，确保所有专家副本保持一致（例如权重相同）；否则，相同逻辑专家的不同物理副本可能产生不一致输出。

通常，副本与原模型从同一 checkpoint 加载，且不会独立更新，从而防止原专家与副本发生分歧。

自适应专家路由与实时监控

传统 MoE 专家门控在训练后固定；自适应路由则可以在推理期间实时调整门控决策，以响应当前系统状况和专家负载。例如，如果检测到某个专家 GPU 落后，系统可指示门控函数将部分 token 转移到另一位有可用容量、但路由分数略低的专家。

应持续监控每位专家的利用率和响应延迟。现代 MoE 系统可与遥测框架集成，让每位专家向 Prometheus/Grafana 输出利用率指标，使系统能够动态调整 capacity factor 或门控算法。

大多数 LLM 专家门控函数只考虑训练时确定的路由分数；真正的自适应系统则需要由推理引擎处理，并在推理时动态执行。

要实现自适应路由，推理引擎需要用自定义逻辑包装模型前向传播。例如，它可以截获门控 softmax，并依据当前负载指标将部分 token 重新分配给其他专家。

推理引擎依赖每 GPU 利用率、每专家 token 数量等实时指标持续测量专家负载。如果某位专家的 GPU 利用率达到 99%，而其他专家 GPU 约为 60%，系统可以暂时将部分 token 路由到该专家的副本，或偏好分数略低的其他专家，以降低其负载。

图 15-13展示了一种采用偏置门控分数的自适应 MoE 路由策略。该方法最初用于训练场景，但其简化形式也可用于推理：当主要专家负载很高时，利用修改后的专家偏置算法将 token 转移给备选专家。

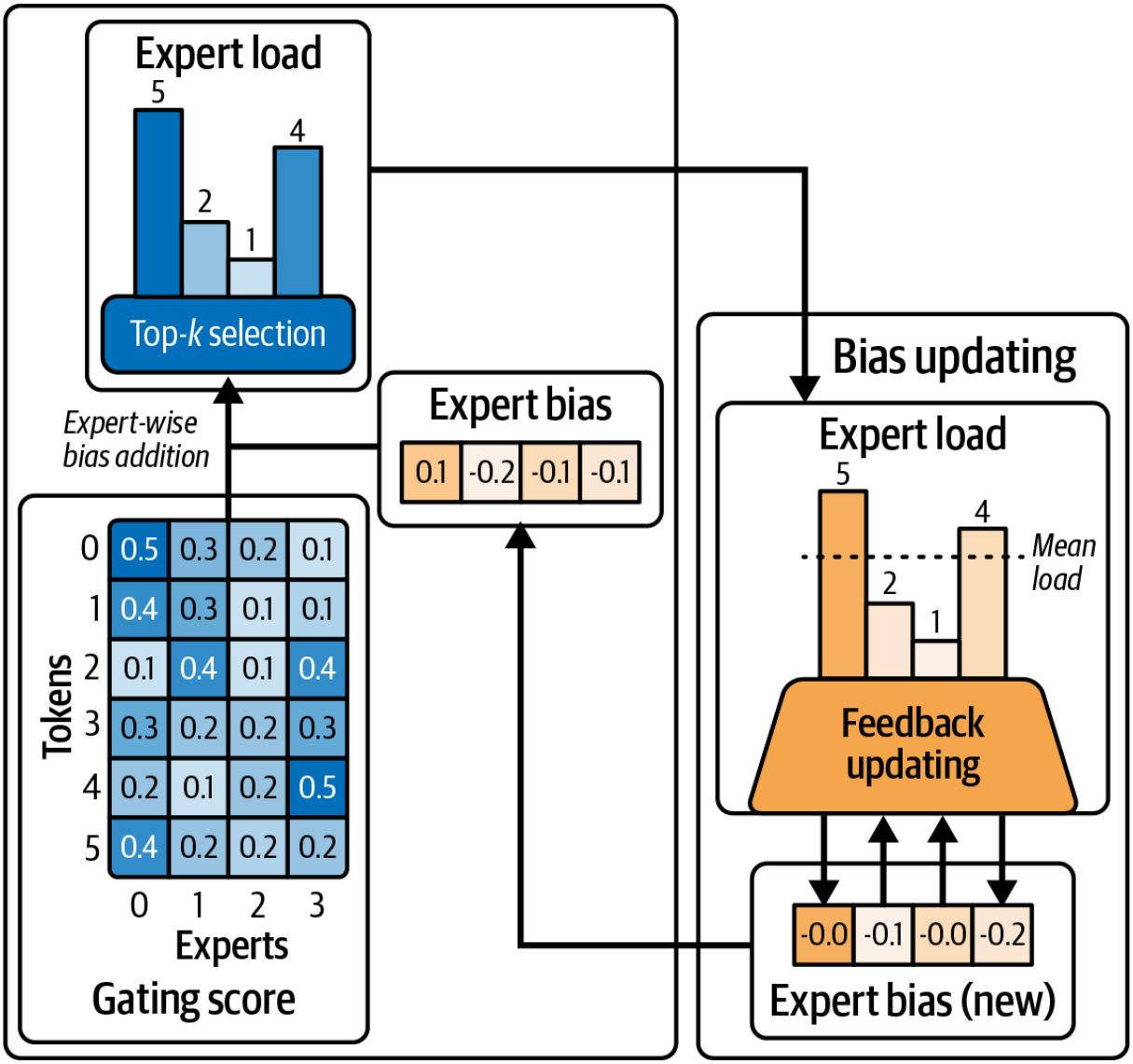


图 15-13　自适应 MoE 路由的运行过程

这种方法可以减少不必要的通信，并更均匀地平衡负载；但额外监控、决策、实现复杂性、配置管理和日志记录也会产生成本。收益是否超过成本取决于具体场景和负载，需要实验评估。

使用 Nsight Systems 等工具分析时，应监控专家 GPU 的 all-to-all 通信时间线。如果某个 GPU 的时间线片段明显更长，它可能正在处理更多 token，也可能受到链路或同步拖尾影响，需要结合计数器确认。

推理系统可利用这些信息调整专家门控概率，并将专家动态重新分配到不同 GPU；也可以创建额外专家副本。通过调整门控算法、修改专家放置或创建/移除副本，可以重新平衡负载。

> 在推理时动态创建新专家副本并不简单，需要预留容量，或具备快速加载专家权重的能力。这是一项高级优化技术。

调整专家在 GPU 上的分组方式，可能使 token 路由更均匀，并因并行性改善而提高总体吞吐量，因为各 GPU 可以更同步地完成每层工作。如果检测到持续失衡，现代 MoE 调度器可调整 capacity factor，并在具备预留资源和相应运行时支持时调度额外专家副本，以缓解门控不均造成的热点。

> 应记录系统做出的所有动态更改，并为特定专家利用率超过阈值设置告警；例如可从 80% 开始，但阈值应根据硬件、队列和 SLO 调整。

动态路由策略有两个核心目标：降低路由开销，并在专家 GPU 之间均匀分配工作。低开销依赖高带宽互连、数据传输与计算重叠，以及通过共置和智能调度减少冗余数据移动。

负载均衡可以通过简单的 top-1/top-2 门控、更高级的容量感知门控、动态复制和专家重新分配实现。通常会组合使用这些技术，让 GPU 尽可能用于计算并减少通信等待。

当路由开销足够低且负载均衡良好时，自适应 MoE 推理系统可能随着专家数量增加而接近线性扩展吞吐量。例如，GPU 数量翻倍时，推理吞吐量在有利条件下可能接近翻倍，但会受到通信、内存带宽和负载偏斜限制。

总之，这些自适应专家路由和负载均衡优化可以与前面介绍的并行及解码技术集成，从而调优超大规模 MoE 推理系统。持续性能分析和自适应算法有助于让 GPU 保持计算繁忙，减少等待通信时的空闲。

> 一些高级或研究型推理引擎可以动态绕过部分专家计算、停用利用率低的专家，或利用缓存为某些 token 跳过专家，从而进一步降低延迟。这些技术可能改变模型行为，必须验证输出质量与正确性。

要点回顾

要为数十亿终端用户服务超大规模 LLM，需要优化推理流水线的每个阶段。下面是本章的关键要点：

通过解耦同时优化延迟与吞吐量

> 将 prompt prefill 和 decode 阶段拆分到独立 GPU 池可以消除直接干扰，使系统有机会同时获得较低首 token 时间和较高每秒 token 数，而不必在两者之间简单取舍。这是大规模 LLM 服务的基础技术之一。

为超大模型使用混合并行

> 对于数万亿参数模型，单一并行策略通常不足。应按需组合张量、流水线、专家和数据并行。例如，通过 TP/PP 在 GPU 之间分片层，使模型装入内存；为 MoE 层使用专家并行以扩展模型容量；再添加数据并行副本满足吞吐量需求。最佳组合取决于硬件，必须针对实际负载分析并调优配置。

缓解顺序解码瓶颈

高级解码方法可以显著加速生成。在针对任务调优接受率后，快速 draft 模型的双模型推测解码在部分场景中约有 2–3 倍提速。EAGLE 系列和 Medusa 也在特定论文评测中报告了更高结果。这些技术提高 token 级并行性和算术强度；在采用标准接受/校正算法且采样设置一致时，可保持 target 模型输出分布。性能和质量必须在目标负载上验证。

通过约束保持输出质量与格式

生产环境常要求 LLM 遵循严格格式或避开某些 token。约束解码可在生成期间强制执行 JSON schema、禁用词等规则。它会增加每 token 开销，但编译语法和优化掩码路径可降低成本；复杂语法或小批次可能开销更高。务必测试实际约束对性能的影响，并尽可能避免造成大量回溯的过度严格规则。

平衡 MoE 工作负载以有效扩展

混合专家模型的容量可以随 GPU/专家数量扩展，但前提是高效处理路由。使用高带宽互连和分层 all-to-all 通信减少网络瓶颈；通过容量限制和适当的门控策略让各专家获得较均衡的工作量，避免拖尾专家；对持续热门的专家进行复制以分摊负载。调优良好的 MoE 推理系统可能随着 GPU 增加而接近线性扩展吞吐量，但仍受通信与负载偏斜限制。

利用软硬件协同设计

现代 GPU 硬件为这些并行和分布式推理方法提供支持。应使用能够充分利用硬件与拓扑的软件，例如 vLLM、SGLang 和 NVIDIA Dynamo 等推理引擎。根据硬件优势设计策略：节点内通信尽量走 NVSwitch，跨节点时使用 InfiniBand，并将通信与计算重叠。实际开销取决于配置，需通过分析验证延迟和成本效率。

理解复杂性与投资回报率（ROI）的权衡

每项优化都会增加系统复杂性。推测解码和自适应 MoE 路由可能显著提高性能，但需要额外模型或复杂逻辑，因此必须权衡成本。应从最大的瓶颈入手，例如消除 prefill/decode 干扰，再按需逐步增加复杂性。通过监控和性能分析判断哪些优化能够带来最佳投资回报，而不要预设某个提速倍数必然值得实施。

结语

将解耦式 prefill/decode 流水线、多 token 推测解码、动态专家路由和自适应编排结合起来，可以减少资源争用，并为实时 LLM 服务降低延迟。

vLLM、SGLang 和 NVIDIA Dynamo 等现代推理服务平台采用了其中许多优化，包括分配集群资源、跨节点协调 KV cache、实现推测解码与约束解码，以及调度 prefill/decode 任务等。

关键在于构建端到端、可适应变化的架构，将算法创新与高性能硬件能力相匹配。接下来的几章将进一步讨论模型推理性能优化，包括动态、自适应和多节点服务策略。

内容涵盖应用层的前缀缓存、延迟感知请求路由、基于强化学习（RL）的集群调优，以及系统层的自适应内存分配、精度切换和拥塞感知资源调度。

第 16 章　大规模推理的性能分析、调试与调优

运营大型 LLM 推理集群需要借助监控和调试工具，确保一切按预期运行。当性能偏离目标时，这些工具还能帮助你快速找出瓶颈。

本章将演示如何使用 NVIDIA Nsight Systems 等性能分析工具以及 Prometheus/Grafana 等集群级遥测工具，监控和调试这些复杂系统。我们还会说明如何收集并解读 GPU 利用率、内存压力、尾延迟百分位数、缓存命中率、逐 token 耗时等关键指标。这些指标能够指导推理引擎的性能优化。

接下来，我们将讨论运行层面的性能调优，包括在生产环境中得到验证的多种方法，用于优化 GPU 利用率、缩短推理延迟并提高大型集群的吞吐量。其中包括重叠计算与通信、调度请求和组成批次，以及有效利用 NVLink、NVSwitch 和 InfiniBand 等高速互连。

我们还会比较用于推理的实时量化技术，包括使用广义训练后量化（Generalized Post-Training Quantization，GPTQ）和激活感知权重量化（Activation-Aware Weight Quantization，AWQ）等实现，将模型压缩到 8 位和 4 位精度的方法。在此过程中，我们会讨论仅量化权重与同时量化权重和激活值之间的权衡，并给出在服务流水线中应用量化的实践指导，以便在保持模型准确率的同时减少内存用量并提高吞吐量。

最后，我们将考察能够配合底层性能调优的应用层优化，包括提示词压缩、前缀缓存、去重、查询路由（例如使用后备模型）和部分输出流式传输等策略。

分析、调试与调优推理性能

现代 LLM 推理引擎中有许多相互配合的组件，采用解耦式预填充和解码时尤其如此。一个典型请求的生命周期涉及许多组件，如图 16-1所示。

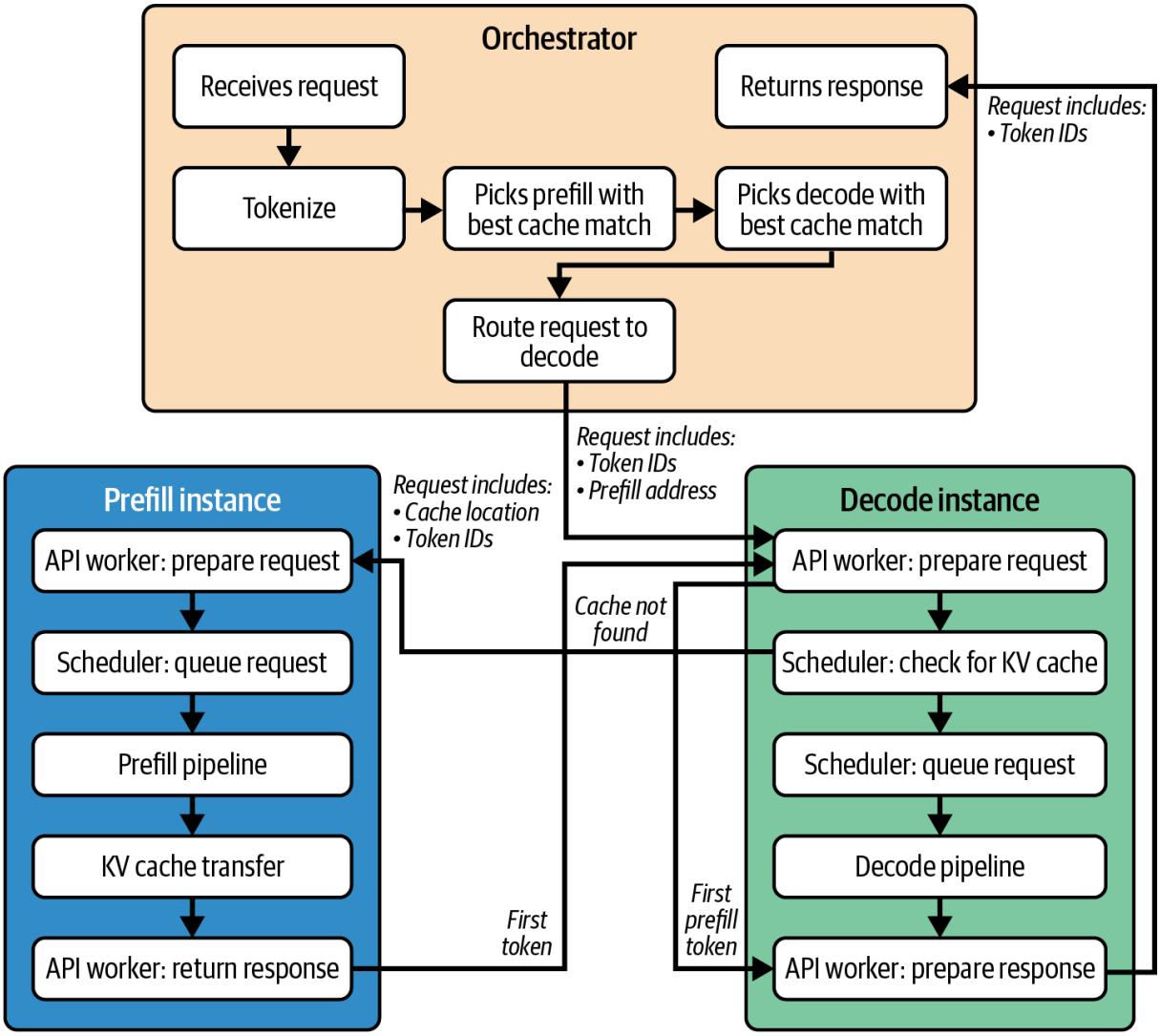


图 16-1　解耦式预填充和解码 LLM 推理系统中典型请求的生命周期

面对这样的复杂性，推理性能调优是一项高度迭代的工作，需要细致调节并持续验证。

首先，应观察指标并找出当前瓶颈，例如 GPU 未得到充分利用或延迟高于预期。接着提出改进假设，如“增大批大小”或“提高操作 X 的通信—计算重叠程度”。然后实施修复并检验这一假设。

之后，最好在预发布环境中使用具有代表性的工作负载测试修复方案，并借助性能分析工具验证更改是否符合预期。例如，可以验证某项操作是否正确实现了内存操作与计算的重叠。

最后，应将修复部署到生产环境，并监控 Grafana 和日志，以验证它是否改善了真实工作负载下的吞吐量和延迟。每当出现新瓶颈，就重复这一流程。

这种“观察—假设—调优”循环应持续进行。现代部署通常会自动执行这些步骤。例如，可以利用定期负载测试以及随后针对关键指标的异常检测来触发调优流程。

> 将优化部署到生产环境时，建议采用金丝雀发布，包括更新推理运行时和模型变体。先把优化应用于少量服务器承载的一小部分流量，便可在面向所有最终用户全面部署之前验证其效果。这种渐进式方法能够及早发现意外副作用，同时避免影响全部用户，从而缩小故障的“爆炸半径”。

设想这样一种场景：由于分词或推理数据预处理的工作量过大，主机端 CPU 利用率骤升至 100%。这会限制推理引擎能够处理的并发流数量。一种修复办法是使用 GPU 加速的分词器库，或使用 CUDA 或 OpenAI Triton 语言编写自定义 GPU 内核，把预处理迁移到 GPU。

部署新库或新内核后，应对比监控更改前后的 CPU 利用率。如果 CPU 利用率下降而总体吞吐量上升，就说明系统不再受基于 CPU 的输入预处理所限。

还应关注各种缓存的命中率，包括前缀缓存、提示词嵌入缓存和 KV 缓存。系统应提供“缓存命中”和“缓存未命中”指标。缓存命中率高，说明系统能够有效复用数据；反之，如果未命中率很高，通常就需要调整缓存大小、淘汰策略或缓存策略，以尽可能提高命中率。

vLLM 的 LMCache 组件支持调整 GPU 缓存与 CPU 缓存的比例。如果由于 GPU 内存限制而频繁未命中，可以启用其分页缓存卸载功能，让 CPU 分担缓存工作。务必确保缓存淘汰策略（最近最少使用 [LRU]、最不经常使用 [LFU] 等）符合实际访问模式。

另一种场景是使用 KV 缓存复用同一批请求中相同输入序列前缀的数据，从而避免重新计算该前缀的 KV。此时需要测量请求共享前缀的频率。共享前缀会产生一次*前缀合并*事件，并增加 vLLM 中的前缀缓存指标，包括 `vllm:gpu_prefix_cache_queries` 和 `vllm:gpu_prefix_cache_hits`。例如，可以用命中次数除以查询次数来计算命中率。

测量前缀合并率，有助于将其与实际缓存命中率关联起来，评估缓存层带来的真实收益。由此可以调整批处理和调度策略，尽可能增加共享前缀，并预测不同工作负载下端到端吞吐量和延迟的改善幅度。

可以让推理引擎运行合成数据，用包含大量重复前缀的提示词进行测试。如果前缀合并生效，预填充计算量应当有所下降。

vLLM 和 SGLang 等现代 LLM 推理引擎原生提供前缀合并相关指标。不过，如果推理引擎没有把前缀合并作为一等指标导出，就应插桩添加“前缀去重 token 数”的自定义计数器，以监控其效果。

> 如果前缀合并没有达到预期效果，应检查前缀匹配逻辑是否失效。调试时先检查分词器是否存在差异，因为这通常是前缀匹配问题的主要原因。

除了分析性能，监控还有助于容量规划。通过跟踪利用率和延迟随负载增长而发生的变化，可以推测系统何时会触及某项限制，例如 p95（第 95 百分位）延迟何时开始急剧上升。在这种情况下，动态批大小可能已经增大到边际收益递减的阶段。

如果采用分层缓存策略，包括基于 NVMe 的 KV 缓存扩展，务必监控设备的 I/O 延迟。I/O 延迟过高会显著降低缓存性能。

当单 GPU 并发度达到上限，继续增大批大小也无法提高吞吐量时，可以考虑横向扩展：增加 GPU、部署更多模型副本，或增加专家数量，把工作分散到更多计算单元上。

在横向扩展之前，还应考虑压缩模型或切换到较低精度（FP8/FP4），以提高每块 GPU 的有效吞吐量。不过，一旦硬件已经饱和（例如 SM 利用率达到 100%，内存带宽利用率接近峰值），要进一步提高吞吐量，通常只能增加 GPU，或采用张量并行/流水线并行。

同时务必权衡新硬件的成本与效率收益。有时，升级到内存容量更大、FLOPS 更高的新一代 GPU，会比横向扩展一批旧 GPU 更具成本效益。

增加专家数量可以提高吞吐量上限，但前提是同时改进专家路由和调度，以应对额外的全对全通信。否则，简单扩展可能只是把瓶颈转移到网络。下面讨论监控，以及如何验证优化工作是否真正取得了成效。

监控系统指标和计数器

传统微服务调用的执行时间相对一致且可预测，而 LLM 请求并不均匀，延迟可能相差悬殊。图 16-2展示了这种差异。

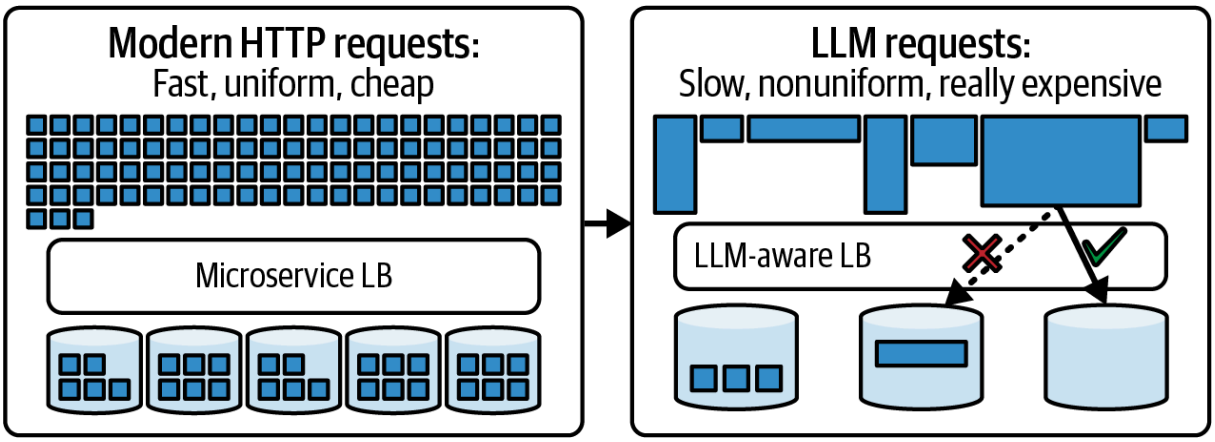


图 16-2　传统微服务调用与 LLM 调用的差异

在生产环境中进行持续监控时，通常使用 Prometheus 从各个 GPU 计算节点收集指标，再通过 Grafana 仪表板进行可视化。需要跟踪的关键 GPU 指标包括 GPU 利用率（SM 处于忙碌状态的时间百分比）、GPU 内存用量、复制引擎利用率、PCIe 和 NVLink 吞吐量，以及 GPU 温度和功耗（例如是否发生降频）。注意：L1/L2 活动、占用率和指令吞吐量等底层计数器，应使用 Nsight Compute 或 CUPTI 收集，而不是依赖 DCGM 和 Prometheus。

`cudaMemPool` 指标和异步分配器统计信息有助于监控内存碎片。应把它们纳入监控体系，这会显著方便生产环境中的系统性能问题调试。

监控互连利用率同样重要，包括 NVLink、NVSwitch 带宽和 NIC 吞吐量。这样才能发现多 GPU、多节点集群配置中的通信瓶颈。

NVIDIA 数据中心 GPU 管理器（DCGM）公开了许多可供 Prometheus 抓取和收集的 GPU 指标。例如，DCGM 提供表示 SM 利用率百分比的 `DCGM_FI_DEV_GPU_UTIL`、表示内存复制引擎利用率的 `DCGM_FI_DEV_MEM_COPY_UTIL`，以及表示已用帧缓冲区内存的 `DCGM_FI_DEV_FB_USED` 等指标。

DCGM 提供 NVLink 错误计数器，并能在某些平台和驱动程序版本上公开吞吐量计数器。若要观察持续的链路利用率，还应使用 `nvidia-smi nvlink` 和 Nsight 工具。应把这些指标集成到仪表板中并设置告警，以便识别网络何时因跨 GPU、跨节点的通信流量而饱和。DCGM 还会跟踪 Xid 计数器和严重 GPU 错误。

> 虽然 DCGM 提供 NVLink 计数器，但截至本书写作时，`dcgm-exporter` 并不会默认在所有平台上公开逐链路带宽。因此，如果需要链路级吞吐量，可能需要直接查询 DCGM 或扩展 exporter。

还建议收集高层应用指标，例如每秒查询数/请求数、平均延迟和 p95/p99 延迟、活动上下文数量，以及以 token/s 表示的吞吐量。KV 缓存利用率和容量（总体及逐节点）也是极为重要的监控指标。

可以部署 Prometheus node exporter，从每个节点采集所有这些指标，把数据汇集到一处，甚至还能针对关键阈值设置告警。随后，Grafana 可以把这些指标绘制到实时仪表板中，供团队共享。例如，图 16-3展示了如何从 Kubernetes 集群的各个 GPU 收集指标、将其导出到 Prometheus，再使用 Grafana 进行可视化。

这样一来，例如部署旨在扩大批处理的新优化后，Grafana 会立即显示各 GPU 的利用率是否上升。还可以通过监控确认 p95/p99 延迟仍处于目标范围内。

计数器同样非常有用，尤其适合动态自适应系统。例如，如果推理引擎会根据当前条件动态调整批大小，就可以在每次调整时递增“批大小变更”计数器。

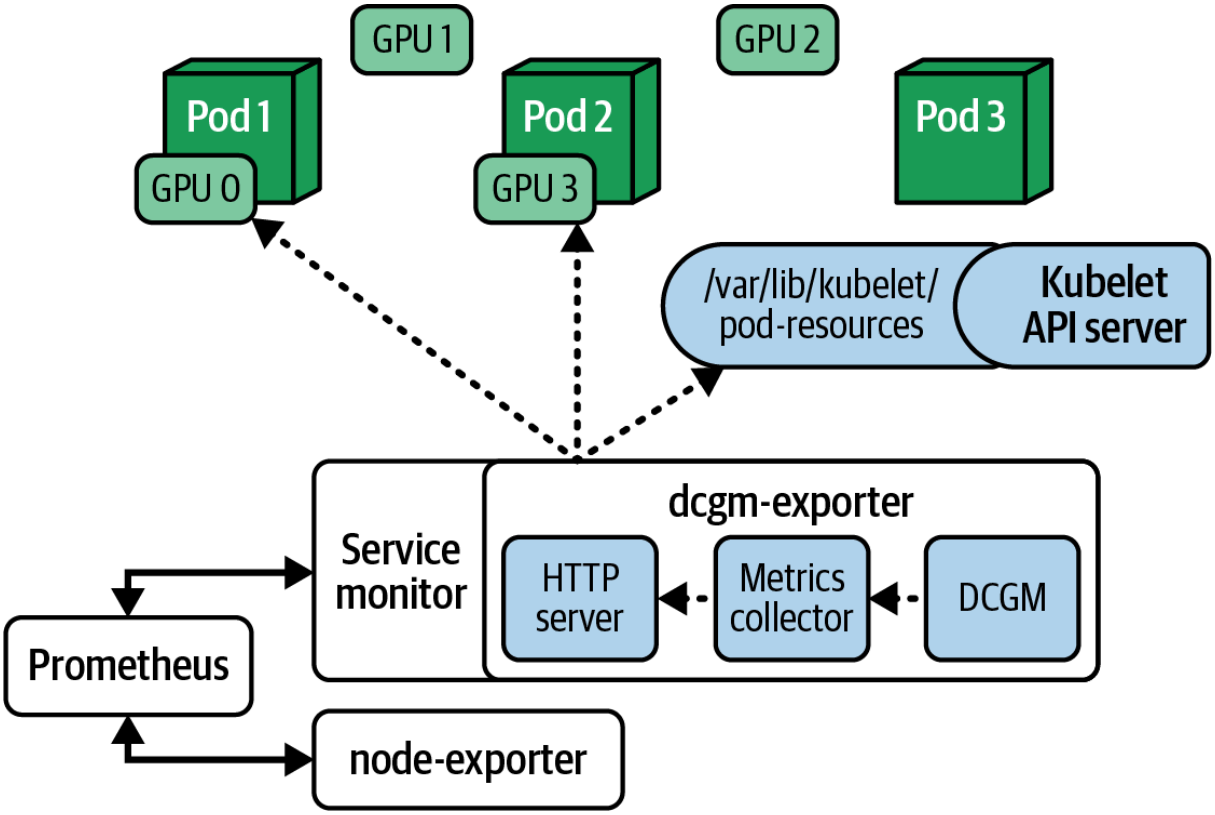


图 16-3　DCGM 从 Kubernetes GPU 节点收集指标并将其发送到 Prometheus

另一种选择是把变更写入日志文件，但之后需要进行缓慢的文本搜索和聚合，例如使用 Apache Spark 离线分析日志；随后还必须手动把日志分析结果与 Prometheus 指标关联起来。

如果为值得关注的应用层事件（包括错误）设置简单计数器并递增计数，数据就会推送到 Prometheus，并能与其他所有指标一起实时显示在 Grafana 仪表板中。此外，还应考虑对关键事件采用结构化日志和分布式追踪。

现代应用性能管理（APM）工具以及 OpenTelemetry 可以摄取这些日志和追踪，并将其与指标关联起来，从而提供覆盖整个系统、时间线一致的事件视图。掌握这条时间线有助于缩短性能问题的调试时间。

持续监控这些指标，就能判断下一步应在哪里调优。例如，如果 GPU 利用率低于预期，可以检查 GPU 内存是否已经用满。若尚未充分利用，则可尝试增大批大小或最大并发请求数。不过，务必留意延迟服务级别目标（SLO），不能超过这些目标。

> 现代推理服务器为动态批处理提供“最大延迟”设置。应调节该设置以满足 SLO。提高此值可以增大批大小（从而提高吞吐量），但提高过多会损害 p99 延迟。应根据延迟目标持续调整。

相反，如果 GPU 内存接近上限，推理引擎可能开始把不活跃的 KV 缓存数据换出到主机 CPU 内存或 NVMe 存储。由于 GPU 必须等待来自较慢 CPU 内存或磁盘的额外数据传输，其利用率会随之下降。

如果发现 GPU 内存复制引擎利用率骤升，或在 SM 利用率较低的同时出现异常 NVLink 利用率，那么推理引擎很可能正在 GPU 内存中换入、换出 KV 缓存数据。过高的数据传输延迟会因此成为系统瓶颈。

如果确实发生了换页，可以调整推理引擎的分页参数以减少抖动，应用 FP8 或 FP4 量化，增加分配给缓存的 GPU 内存，并视情况改变换页策略。这样应能降低复制引擎利用率并提高计算利用率，这正是期望看到的结果。

Grafana 也可用于跟踪延迟。可以绘制端到端请求延迟的分布，通常还会分别测量预填充延迟和逐 token 延迟。如果 p99 延迟在某些时段骤升，应将其与 GPU 指标和其他日志进行关联分析。

例如，p99 延迟骤升可能与 GPU 利用率下降的时段相吻合。延迟峰值也可能对应某次流量激增：流量激增触发了更大的动态批大小，导致该时段延迟升高。为验证这一点，可以在 Grafana 仪表板的延迟图上叠加 RPS（每秒请求数），观察两张曲线图是否相关。

如果按照假设，峰值确实源于批大小的动态增加，而且属于预期现象，仍需确认它没有超出服务级别协议（SLA）。如果超出，可以尝试缩短请求批次的最大排队延迟，或减小最大批大小，以限制延迟。

日志对于诊断问题也极具价值。应在代码中插桩，记录批次形成、通信开始/结束等关键事件。最好使用 `DEBUG` 级别，以便按需启用或禁用，避免影响请求—响应延迟。

启用调试日志后，可以看到文本形式的逐步时间线。在一次调试过程中，通常会结合使用日志时间线和 Prometheus/Grafana 指标。例如，可以查看全对全通信耗时超过 5 ms 的频率。

结合日志时间线和指标，可以发现网络问题等异常值；这类问题可能拖慢某一轮全对全通信交换。如果问题持续发生，可以提高专家容量因子，让超额 token 自动溢出到第二专家副本；理想情况下，该副本应位于网络路径更稳定的 GPU 上。这样可以平衡负载并尽量降低延迟。

实践中常把容量因子设为 1.2～1.5；这样主专家过载时，便可重新分配额外 20%～50% 的 token。在具体实现确实支持溢出路由且备用专家有可用容量的前提下，这有助于平滑 MoE 推理的尾延迟。与其让请求排在互连性能下降、略有停顿的 GPU 专家之后，不如将其溢出到第二专家。不过，这些容量因子并非通用最优值，额外副本、路由策略及通信开销会决定实际收益。

使用 Nsight Systems 和 Nsight Compute 进行性能分析

开发和调优推理代码时，可以使用 Nsight Systems 捕获横跨 CPU 和 GPU 的工作负载追踪。Nsight Systems 提供微秒级时间线视图，其中显示 CPU 线程、GPU 内核、CUDA 事件、NCCL 通信等活动。

通过在代码中加入 NVTX 标注，可以在时间线上清晰标出“预填充阶段”“解码步骤”或“全对全通信”等区间。下面的代码使用 NVTX v3 C API 的显式区间压栈和出栈操作，在示例预填充和解码步骤周围设置 NVTX 区间标记：

```cpp
// Example C++ snippet with NVTX annotations using the C API.

#include <nvtx3/nvToolsExt.h>   // or <nvToolsExt.h>
#include "my_model.hpp"         // Your model's C++ interface
#include <vector>

// Small helpers to keep callsites tidy.
#define NVTX_PUSH(name, argb)                                       \
  do {                                                              \
    nvtxEventAttributes_t a{};                                      \
    a.version = NVTX_VERSION;                                       \
    a.size = NVTX_EVENT_ATTRIB_STRUCT_SIZE;                         \
    a.colorType = NVTX_COLOR_ARGB;                                  \
    a.color = (unsigned int)(argb);                                 \
    a.messageType = NVTX_MESSAGE_TYPE_ASCII;                        \
    a.message.ascii = (name);                                       \
    nvtxRangePushEx(&a);                                            \
  } while (0)

#define NVTX_POP() do { nvtxRangePop(); } while (0)

struct Token { int id; };

void run_inference(
    const std::vector<Token>& prompt_tokens,
    Model& model,
    int num_generate_steps) {
  // Prefill
  NVTX_PUSH("Prefill", 0xFF4F86F7);
  model.encode(prompt_tokens);
  NVTX_POP();

  // Decode one token at a time
  for (int t = 0; t < num_generate_steps; ++t) {
    NVTX_PUSH("Decode", 0xFFFF8C00);
    Token next_token = model.decode_next();
    // ... (sampling / streaming to client)
    NVTX_POP();
  }
}
```

这里使用 `nvtxRangePushEx`/`nvtxRangePop` 显式标记区间。在 `model.encode(...)` 之前立即压入 `"Prefill"` 区间，并在其后弹出；在 `decode` 循环中，每次迭代开始时压入 `"Decode"`，在 `model.decode_next()` 之后弹出。小型 `NVTX_PUSH`/`NVTX_POP` 辅助宏还会附加颜色（十六进制值）和文本。这既能减少时间线可视化中的标记错配，又能保持调用点简洁。代码中清晰可见的压入/弹出配对也便于审计。

这些彩色块标注会出现在 Nsight Systems 的 GPU 活动时间线上，并标记为 `"Prefill"` 和 `"Decode"`。这样很容易看出每个阶段持续多久，以及各阶段如何与通信操作重叠，从而发现 GPU 空闲间隙和意外同步等问题。

> 注意，这里直接使用 NVTX C API（`nvToolsExt`），而不是 PyTorch 的 `record_function()`。这样可以在纯 C++ 运行时中标注热点路径，并确保从 Python 或其他语言启动工作时标记保持一致。

将范围收紧到 `model.encode(prompt_tokens)` 周围所需的最小区间后，性能分析标记就能准确覆盖预填充工作，而不会包含其他代码。这会提高追踪的清晰度并改善性能诊断。

在多个 CUDA 流上排队工作时，应使用逐流区间（例如为 H2D/D2H 复制设置专用的“transfer”流，为内核设置“compute”流）。具体而言，可以用不同的 NVTX 区间包围每个流对应的主机端代码。

例如，可以使用 `nvtxNameCudaStreamA(transfer_stream, "transfer_stream")` 和 `nvtxNameCudaStreamA(compute_stream, "compute_stream")` 为流命名。然后，在内存复制/传输周围使用 `nvtxRangePushA("transfer_stream")` 和 `nvtxRangePop()`，在内核启动周围使用 `nvtxRangePushA("compute_stream")` 和 `nvtxRangePop()`。

使用带 NVTX 名称的流后，Nsight Systems 时间线会清楚显示操作是否发生重叠。下面的代码展示了如何把这些内容组合起来：

```
// One-time after creating the streams
nvtxNameCudaStreamA(transfer_stream, "transfer_stream");
nvtxNameCudaStreamA(compute_stream,  "compute_stream");

// Around H2D/D2H copies (transfer stream)
nvtxRangePushA("transfer_stream");
cudaMemcpyAsync(h_logits, d_logits, bytes, cudaMemcpyDeviceToHost,
                transfer_stream);
nvtxRangePop();

// Around kernel enqueues (compute stream)
nvtxRangePushA("compute_stream");
my_kernel<<<grid, block, 0, compute_stream>>>(...);
nvtxRangePop();
```

这里为各个流命名，并用逐流区间包围排队点，使 Nsight 时间线保持易读。需要注意，NVTX 区间标注的是主机线程时间线；GPU 轨道则按流显示内核和 `memcpy` 操作。为流命名有助于在分析时把主机端区间对应到正确的 GPU 轨道。

Nsight Compute 可以分析单个内核，从而定位效率低下之处。可以利用 Nsight Compute 基于 section 的性能分析功能，专注于内核的特定部分，例如内存事务。

另一个非常有用但并不广为人知的工具，是 Nsight Compute 的 CUDA 程序计数器（PC）采样功能。如图 16-4所示，它可以采样程序计数器并识别热点，而不需要完整且开销很大的插桩。

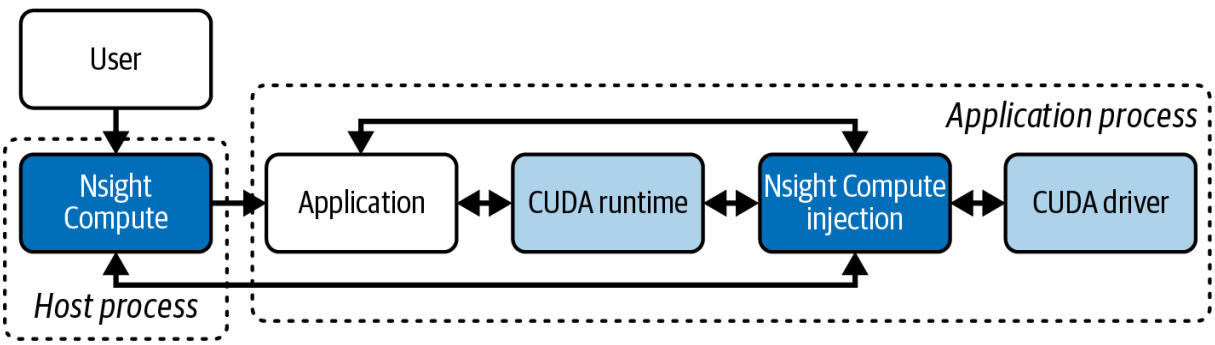


图 16-4　Nsight Compute 的 CUDA 程序计数器（PC）采样功能能够以低开销方式识别热点（来源：https://oreil.ly/DyKWR）

具体而言，可以使用它分析正在运行的推理服务器，准确定位哪些内核指令耗时最多，而且开销很低。既然已经介绍了使用 Nsight Systems 和 Nsight Compute 进行性能分析，下面讨论一些常见的推理故障排查方法。

> 在对线上生产服务进行调查时，建议先使用程序计数器采样，以极低开销定位热点。只有当采样结果指向某个特定内核或阶段时，再切换到完整追踪。

推理故障排查方法

在生产环境中，持续运行高开销性能分析器并不现实。因此，需要依靠轻量级、基于指标的监控来检测异常并指导针对性修复，例如 GPU SM 利用率、KV 缓存警告、尾延迟百分位数、缓存命中率和 OOM 告警。当某个指标越过特定阈值时，可以针对其根因形成假设，例如批大小过小、KV 缓存不足、路由热点、分片不均衡或内存超额分配等。

然后实施修复，例如调节批大小、提高内存利用率上限（如果可行）、调整路由器阈值或启用 CPU 卸载。推送修复后，应验证其影响，并确认指标已经回落到阈值以下。表 16-1列出了常见生产问题的一些关键指标、症状、可能原因和建议修复措施。

表 16-1　常见故障症状、原因和建议措施

指标/症状	可能原因	建议措施
SM utilization < 50%	批次过小或缺少融合内核	增大批大小，启用融合内核（FlashAttention 或 PyTorch 中由 cuDNN 融合的 `scaled_dot_product_attention`（SDPA）后端），或添加自定义融合内核（例如使用 Triton）；然后用 `nsys --trace=cuda` 进行性能分析。
KV 缓存抢占警告	KV 缓存空间不足（vLLM）	提高 GPU 内存利用率阈值，减少批处理中 token 的最大数量，并考虑使用 PagedAttention 动态分配 KV。
尾延迟高（p95 > 200 ms）	解码节点热点或队头阻塞	检查路由器日志中的路由模式；调节预取阈值；启用推测解码路径。
负载下缓存命中率 < 60%	分片放置不均衡或缺少前缀缓存	验证前缀缓存连接器配置（例如 vLLM 的 LMCache NIXL 和 NVIDIA Dynamo 的 NIXL 连接器），必要时增加前缀缓存 TTL 或副本数量。
多租户 GPU 上意外 OOM	GPU 内存超额分配	降低每个实例的 GPU 内存利用率，启用 CPU/NVMe 卸载，并将进程绑定到 CPU 套接字以减少跨套接字流量。
不规则的性能异常值	时钟不一致或温度降频	确保所有时钟同步，并监控温度和功耗降频。

> 注意：所有指标表中的数值仅用于说明概念。不同 GPU 架构上的实际基准测试结果，请参阅 GitHub 仓库。

部分信息也可能隐藏在日志文件中。AWS 等云服务商支持对日志文件使用正则表达式（RegEx）过滤器，从日志行中提取数值并直接导出为指标。例如，AWS CloudWatch 就支持这一实用功能。下一个代码块包含一些值得监控的日志示例。

下面是一段 vLLM 日志示例，它表示由于 KV 缓存空间不足而发生了 KV 缓存抢占。因此触发了 KV 重计算，这会占用更多 GPU 计算资源并增加延迟：

```
WARNING 2025-05-03 14:22:07 scheduler.py:1057 Sequence group 0 is preempted by
PreemptionMode.RECOMPUTE because not enough KV cache space.
total_cumulative_preemption_cnt=1
```

下面是 NVIDIA Dynamo 路由日志示例。第一行表示本地解码工作进程的前缀缓存命中率为 90%，因此预填充继续在本地运行；下一行表示本地缓存未命中。随后，路由器把预填充调度到远程的 `GPU-node-03` 工作节点：

```
[Router] 2025-05-03T14:23:11Z INFO KVRouter: prefix-cache hit (90%) for
model=DeepSeek-R1; routing to local vLLM worker
[Router] 2025-05-03T14:23:12Z INFO KVRouter: cache miss; dispatching remote
prefill to GPU-node-03
```

全栈推理优化

高性能 LLM 推理要求对技术栈的每一层进行协同优化，范围从模型架构和内核实现，一直到运行时引擎、系统编排和部署基础设施。

剪枝、蒸馏、稀疏性、MoE 路由、高效注意力（例如 FlashAttention）和量化感知训练等模型级技术，可以降低计算和内存需求。在内核层面，融合操作、自定义注意力引擎（例如 FlashInfer）、Tensor Core 利用、分块以及异步内存传输，有助于最大化 GPU 吞吐量。

动态批处理、分页 KV 缓存、CUDA Graph 以及计算与通信重叠等运行时策略，可以让 GPU 在负载变化时仍保持较高利用率。系统编排层可以采用预填充/解码解耦、智能路由、多租户隔离和自动扩缩容（配合热备用实例），以平衡延迟并提高成本效率。

> 许多生产系统使用基于 Kubernetes 的系统编排，分别运行预填充和解码部署。它们使用入口控制器，根据负载或用户优先级路由请求，并准备好热备用 GPU Pod，以便在流量激增时立即启动。

最后，还应探索地理分布式边缘服务、智能 API 网关批处理、模型变体的 CI/CD 以及实时性能分析等部署模式。这些方法能够提高生产环境的可靠性和适应性。表 16-2介绍了技术栈各层的一些常见优化方法。

表 16-2　技术栈各层的常见优化方法

技术栈层	关键技术
模型	通过剪枝和知识蒸馏在尽量减少准确率损失的同时缩小模型规模 使用稀疏性（MoE）跳过计算 使用高效注意力（FlashAttention）减少内存占用和中间缓冲区 在 FP16/BF16 或 INT4/FP8 中进行量化感知训练，以增强低精度下的鲁棒性
内核	融合算子内核（例如 Linear + GELU + LayerNorm），减少启动开销和内存流量 用于块稀疏 KV 的自定义注意力内核（FlashInfer）和 JIT 编译内核 在矩阵运算中利用 Tensor Core 和专用指令（cp.async、TMA）
运行时	采用 vLLM、SGLang 和 NVIDIA Dynamo 中实现的带延迟控制的动态批处理（例如连续批处理）来合并请求 使用分页 KV 缓存管理灵活分配内存并合并批次（vLLM 的 PagedAttention） 使用 CUDA Graph 和缓冲区池减少每次推理的开销 使用多个 CUDA 流（一个用于数据传输，另一个用于计算）来重叠计算与通信。采用基于事件的同步，并且只在需要时同步。
系统编排	预填充—解码解耦，以消除队头阻塞 智能路由和缓存亲和性，以平衡负载并提高缓存命中率 多租户隔离和逐用户配额，以防止嘈杂邻居影响 使用热备用实例自动扩缩容，隐藏模型加载时间；在流量激增时以略微增加成本换取显著更低的延迟
部署和基础设施	地理分布式和边缘部署，以减少网络 RTT 跨服务器池进行请求级批处理的智能 API 网关 用于以金丝雀模式推出新的量化或内核优化模型变体的 CI/CD 流水线 高带宽互连（NVLink/NVSwitch 和 InfiniBand），以及 GPU 与 CPU 之间的 NUMA 亲和性，以实现最佳内存访问
QoS 和扩缩容	感知 SLA 的动态批处理和尾延迟控制 使用 MIG 或流优先级隔离 GPU，以实施 QoS 用于 TTFT、TPOT、利用率和内存带宽利用率的实时性能分析仪表板 根据工作负载特征动态切换并行策略（TP、PP、DP）

进行优化时，必须考虑跨层协同效应。例如，量化（模型层）会减少内存占用，使运行时能够在不发生 OOM 的情况下使用更大的批大小，进而让编排组件在每个 GPU 周期合并更多请求。

还应以性能分析结果为依据。持续性能分析应指导下一步优化哪一层。例如，完成融合和量化后，如果 CPU 成为预处理和后处理的瓶颈，就应采用更快的分词器，或把部分任务卸载到 GPU。

应用优化总要权衡取舍。分层 CPU 卸载和高级解码方法等技术会增加复杂性。例如，推测解码需要增加草稿模型，Medusa 则增加多头并行解码。这些技术通常保留给超长上下文或延迟波动异常等极端场景；在生产环境中，稀疏性、批处理和解耦等较轻量的方法往往贡献大部分收益。

建议采用全栈优化方法，使模型架构、内核设计、运行时行为、系统编排和部署策略彼此协调。这意味着要及时更新软件栈，包括 CUDA、cuDNN、NCCL 等，因为新版本通常包含最新优化和错误修复。

> 为了在固定主机镜像上复现基准测试，应为原生扩展栈维护经过测试的兼容性矩阵，不要混用任意版本。这一点对于 PyTorch、服务引擎和自定义/原生扩展的组合尤其重要。在进行长时间的批量测试前，先运行一个小型导入冒烟测试。

全栈方法可以降低各层成为瓶颈的可能性。这样，团队就能系统地消除瓶颈，在大规模 LLM 推理中实现稳定的低延迟，并最大化硬件利用率。

调试正确性问题

监控也有助于发现由 bug 引起的异常。例如，如果内存用量持续增长，可能是 CUDA 内核发生了内存泄漏。建议在测试期间使用 Compute Sanitizer（`compute-sanitizer`），捕获设备内存错误、竞态条件和越界访问。示例如下：

```
compute-sanitizer --tool memcheck your_binary
```

如果某块 GPU 的利用率明显低于其他 GPU，它可能因 NCCL 静默失败或未捕获错误而退出 NCCL 通信组。可以在日志中搜索 `WARN NCCL_COMM_FAILURE` 来检查 NCCL 错误代码；NCCL 会提供非常详细的错误日志。

> 将环境变量设置为 `NCCL_DEBUG=WARN`，即可启用 NCCL 调试。这有助于暴露原本可能静默发生的错误。不过要注意，NCCL 日志非常详细！

使用 NCCL 测试套件调试 all-reduce 和 all-to-all 的性能与正确性问题。还可以将 `ncclCommGetAsyncError` 与 `ncclCommAbort` 配合使用，以检测和处理异步通信错误。此外，可以启用 NCCL 的 `IB GID` 追踪，并利用 NVSwitch 系统遥测从互连层面发现问题。

应在 Prometheus Alertmanager 中设置告警，以检测诸如“GPU 利用率至少 60 秒低于 10%”“内存用量超过 W% 阈值”“NVLink 错误率高于 X”“PCIe 重放次数超过 Y 阈值”“温度高于 Z 度”等异常模式。

实践中，可以配置如表 16-3所示的 Prometheus Alertmanager 规则，以便主动调查问题。

表 16-3　基于 GPU 的系统中常见 Prometheus 告警示例

指标	条件	严重性	备注
GPU 利用率	< 10% for > 60 s	空闲	利用率不足
GPU 利用率	> 90%	瓶颈	可能已饱和
内存用量	> 80%	警告	接近 OOM
内存用量	> 95%	严重	内存不足错误风险高
温度	> 85 °C	警告	接近温度降频
温度	> 95 °C	严重	存在关机或硬件损坏风险
NVLink 重放/恢复错误	⩾ 1	严重	表示链路正在重试或恢复
NVLink CRC 错误	> 每秒 100 个错误	严重	链路 CRC 失败率高
PCIe 重放错误	⩾ 1	严重	PCIe 总线上的数据包重试
不可纠正的 ECC 错误	⩾ 1	严重	数据损坏，需要重置

还应设置硬件错误计数器和告警。例如，一旦报告 ECC 错误或 NVLink 重试，就应立即告警，因为这些问题可能迅速降低性能或导致掉线。掉线是指 GPU 或其互连静默断开；例如，NVLink 可能断开，或者 GPU 在发生致命错误后“从总线掉出”。

> 可以使用 DCGM 获取逐链路 NVLink 吞吐量和总量，包括 `DCGM_FI_DEV_NVLINK_TX_BANDWIDTH_L*`、`DCGM_FI_DEV_NVLINK_RX_BANDWIDTH_L*` 和 `*_TOTAL`。如有需要，再使用 `nvidia-smi nvlink`、Nsight Systems/Compute 或 NVSwitch 计数器作为补充。

考虑一种 NCCL 故障：你可能收到告警，显示某个节点的 GPU 利用率接近 0%，而其他节点达到 90%。可以先检查节点日志，找出哪个节点正在产生 NCCL 错误。

这种主动监控和告警能更快捕获问题、找到故障节点并开始恢复。在这种情况下，可以重新初始化 NCCL 通信器，或完整重启节点（务必确认节点重启后重新加入 NCCL 组）。

还可以为 NCCL 错误递增“NCCL Error”计数器，从而更快捕获问题。此外，推理服务器可以记录 NCCL 错误，Fluentd 或 AWS CloudWatch 会自动抓取这些日志并将其转换为计数器。

随后，可以在 Grafana 中把错误计数器图表叠加到逐节点 GPU 利用率图表上。这样就能将 NCCL 错误与 GPU 利用率下降关联起来，更快识别并修复故障节点。

> 应用层计数器在生产环境中非常有用，尤其是与系统指标结合使用时。

在通过实际指标验证吞吐量提高、延迟降低、利用率改善等结果之前，不能认为优化已经成功。现代 AI 推理系统十分复杂，因此必须采用严格、以测量为驱动的方法调优系统性能。

简而言之，应结合应用层计数器、由 Fluentd 或 AWS CloudWatch 自动抓取的日志计数器，以及底层系统指标。这种全栈遥测为在生产工作负载上运行和优化高性能多节点 LLM 推理系统提供必要的可见性。应把指标、计数器和日志视为系统行为的事实依据。

> 直觉和主观感觉经常会把我们引向错误的调试路径，但指标不会说谎。应在一开始就正确地为代码插桩；出现问题时，相信指标。

动态批处理、调度与路由

即使已经在集群中对模型进行了最佳划分和并行化，多节点推理集群部署仍有许多应用层优化机会。本节聚焦于动态组成请求批次，并使用优化的调度和路由策略，以最大化 GPU 利用率、最小化延迟并提高吞吐量。

动态批处理

推理服务系统中最强大的性能技术之一是批处理。将多个输入合并为一个批次，让模型一起处理，可以把内核启动、内存加载等固定成本分摊到多个输入上，从而提高吞吐量。但代价是单个请求的延迟增加：某些请求可能需要等待一段时间（例如 2 ms）才能加入批次并得到处理。

动态批处理是请求批处理的一种特化形式，会即时将到达的推理请求组成大小动态变化的批次。可以配置为在一段时间内缓冲请求，或缓冲到达到指定批大小为止。所有现代 LLM 推理引擎都支持动态批处理，包括 vLLM、SGLang 和 NVIDIA Dynamo。

动态批处理与*静态批处理*相反：静态批处理固定批大小（或将所有序列填充到最长序列的长度），并等待该批次中的每个请求完成后才返回结果。这会让先到达的请求遭遇无界排队延迟，还会因填充而浪费 GPU 周期。

使用动态批处理时，系统会累积到达的请求，在达到目标批大小或短暂超时（例如 2 ms）后，立即调度“已经到达的所有请求”。这样可以把最大延迟限制在指定的超时值以内。

动态批处理能够即时调整批大小，把内核启动开销分摊到多个序列，同时避免静态批处理的最坏情况延迟。这既提高 GPU 利用率，也让可变负载下的延迟更可预测。图 16-5展示了静态批处理和动态批处理的差异。

动态批处理可以根据实际请求到达模式和延迟目标（例如最大延迟）自动增大或缩小批大小。关键是智能组批，在提高总体吞吐量的同时把延迟维持在可接受范围内。现代推理引

擎会实现微批处理，只累积几毫秒请求就将批次调度到 GPU。通常使用 2～10 ms 的延迟，但仍应根据延迟 SLO 调整。

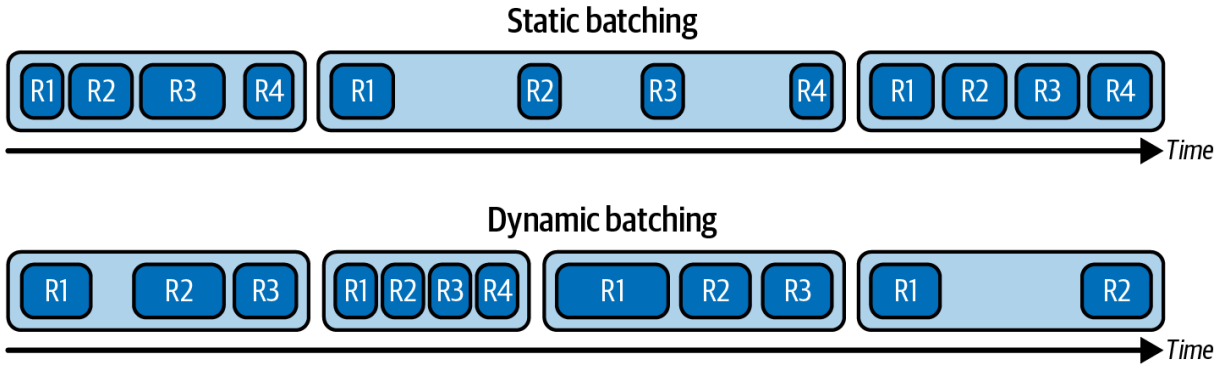


图 16-5　静态批处理和动态批处理的差异

例如，可以将批延迟配置为 2 ms，用来决定服务器在调度批次前等待额外请求的时间。如果这段时间内到达三个请求，批处理器会立即将它们组成一批并发送到 GPU。2 ms 计时器到期后到达的请求，则会被收集到下一批。这个由超时触发的机制将每个请求的排队延迟限制在不超过 2 ms，同时通过合并多个序列提高吞吐量。

这些微批次减少了额外延迟，并让 GPU 一次处理多个请求，而不是一次处理一个。大型 LLM 在批大小较小时通常受内存带宽限制，因此增大批大小可以提高算术强度和总体硬件利用率。

实践中，LLM 服务系统会选择平衡的批大小，在不引入过多延迟的情况下获得良好吞吐量。在高负载下，动态批处理可以避免 GPU 在请求之间空闲，因而可能同时改善吞吐量和排队延迟。当请求到达率较高且系统尚未超过饱和点时，批处理能够在相同时间内处理更多工作，减少总体排队等待，有利于端到端延迟和尾延迟；超过膝点后，延迟反而会快速上升。

在低 RPS（每秒请求数）下，动态批处理会等待其他请求加入批次，因此会增加少量延迟。在中高 RPS 且容量充足时，这段等待可由更高处理效率分摊；相对于逐个运行请求造成的排队延迟，它可能较小。因此，适当批处理可以降低总体延迟，包括尾延迟，但效果必须在实际负载曲线上验证。

> 应使用 Grafana 等工具绘制延迟百分位数随负载变化的曲线，验证改进效果。采用批处理后，随着吞吐量提高，总体 p50 延迟可能保持平稳甚至下降，但通常只持续到某个拐点。应记录这个拐点，并把运行状态保持在其下方。

配置批处理时必须考虑延迟 SLO。例如，如果承诺某种长度请求的 p99 延迟为 2 秒，就不能让请求在批队列中等待 500 ms。动态批延迟的初始值应远低于 p99 延迟预算；1～2 ms 只是可供基准测试的起点，需按服务目标和到达模式调节。

采用自适应批延迟后，在低 RPS 下，批延迟可以动态降至接近 0 ms；在峰值负载下有需要时，可以提高到例如 5～10 ms。具体引擎的调度语义和参数并不相同，应根据流量模式闭环调整，以满足 SLO。

批处理主要适用于高流量场景。流量低时，系统会直接运行单个请求，延迟很低；但在高负载下，系统可以采用积极的批处理来获得高吞吐量，并通过避免队列堆积、在多个请求之间分摊延迟，改善总体延迟。

连续批处理

连续批处理也称为*在途批处理*或*迭代级调度*，它不等待完整序列结束，而是在每次 token 生成迭代时重新填充批次，以保持较高 GPU 利用率。它会驱逐已完成的请求，并根据 GPU 是否就绪立即加入新请求。这项技术对于聊天助手等低延迟场景尤其重要。

与动态批处理等由超时驱动的方法不同，事件驱动的连续批处理策略消除了空闲计算槽位以及这些方法带来的填充开销。连续批处理不依赖固定的“最大延迟”计时器，因此允许新请求在生成中途加入正在运行的批次，也不会被最长序列阻塞，如图 16-6所示。

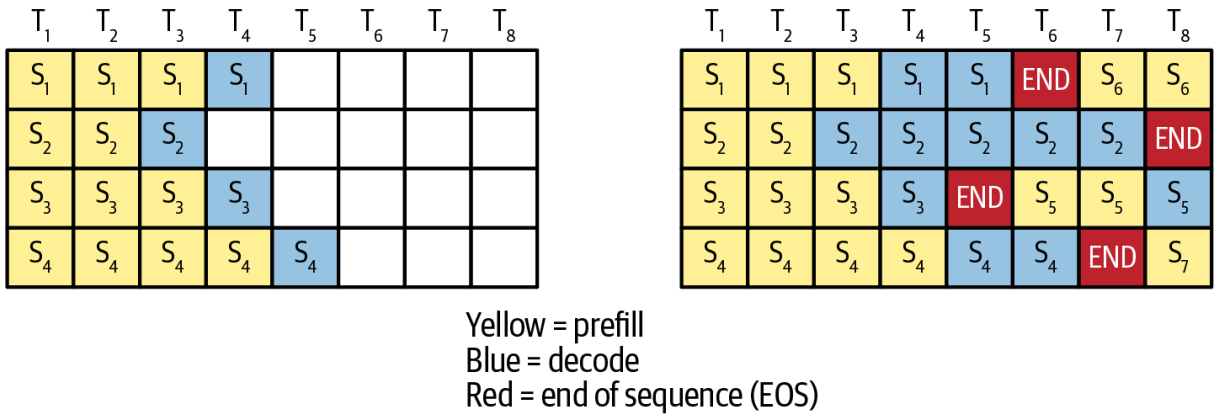


图 16-6　连续批处理允许新序列（请求）在生成中途加入批次

这里，连续批处理最大限度减少了推理计算流水线中的空闲槽位。它消除了每个批次等待最长响应完成所造成的空闲时间。连续批处理不等待批次中的所有序列结束（由于输出长度不同，这样做效率低且会导致 GPU 利用率不足），而是在每次迭代用新序列替换已完成的序列。这种方法允许新请求立即填充 GPU 槽位，从而提高吞吐量、降低延迟并提升 GPU 利用率。

在大型模型上，将 4～8 个请求组成批次，吞吐量常常可以达到 1～2 个请求批次的两到三倍，但实际增益取决于模型、硬件、序列长度和调度实现。这是因为 GPU 的数学单元和内存流水线得到了更充分利用。每个请求增加的延迟通常只有几毫秒，因此这种策略适合低延迟场景。表 16-4总结了前面讨论的不同批处理策略，包括朴素的静态批处理策略。

表 16-4　静态、动态和连续批处理对比

方面	静态批处理	动态批处理	连续批处理
触发条件	固定批大小或序列长度	批大小目标或超时	token 生成完成事件
延迟上限	无界；等待完整批次	受 `max_batch_delay_ms` 限制	极小；在批次中途驱逐并重新填充
填充开销	高：填充到最长序列	中等：在每个批次内填充	低：无需等待填充即可重新填充槽位
GPU 空闲缓解	差：尤其是在大小可变的负载下	较好：但计时器在小批次上触发时可能让 GPU 空闲	优秀：每次迭代都让 GPU 保持饱和
实现	简单	中等：需要计时器和队列管理	复杂：需要逐 token 协调和状态跟踪

连续调度

请求调度的另一个维度是并发执行模型。vLLM 社区将其称为*连续调度*。其基本思想是：如果模型较小，或批大小受到延迟目标限制，单个模型实例可能无法让 GPU 完全饱和。

连续调度对待 GPU 的方式类似于操作系统调度器对待 CPU：在不同 CUDA 流上启动相互独立的小内核。这样，硬件 warp 调度器便能在任务之间交错调度 warp，而无须显式上下文切换。

例如，如果推理引擎未能让 GPU 完全饱和，就可以在同一 GPU 上并发运行多个模型推理流。这样，当一个实例等待数据传输或非 GPU 操作时，另一个实例便可执行。

具体来说，在解码阶段一次生成一个 token，往往会使 GPU 利用率不足，尤其是只处理单个文本序列时。这是因为工作负载相对较小，并且相对于计算量而言需要移动大量内存数据。

为解决这一低效问题，推理引擎可以交错执行来自不同用户请求的多个解码任务。例如，如果 10 个用户同时解码不同序列，不能总是直接把它们组成一次大型矩阵乘法，因为各序列长度可能不同；若不进行大量填充，GPU 就难以高效执行这种统一矩阵运算。

连续调度器可以改为在不同 CUDA 流上快速连续启动 10 个小型解码内核，分别为每个序列生成下一个 token。借助 GPU 的细粒度 warp 调度器，不同序列可以真正并发，内核的 warp 会在各个 SM 上交错执行。这种近似轮询的处理模式有助于减少空闲周期。

把每个解码任务排入各自的流后，GPU 可以在不同序列之间重叠内存传输和计算，从而缩短逐 token 延迟并提高总体吞吐量。

GPU 在 warp 级别交错调度这些内核。当某个解码内核因 CPU 端控制流低效或等待网络 I/O 而无法继续提交工作时，其他已就绪内核仍可推进。这有助于提高 GPU 利用率，但实际并发程度仍受资源占用、依赖关系和硬件调度限制。

连续调度维护一个已就绪 token 任务队列，即使在 token 级粒度上也能获得较高利用率。最终效果类似于批处理：GPU 持续执行 token 生成，但不必把所有工作合并成一次巨型矩阵乘法；关键是避免 GPU 在相邻 token 计算之间空闲。这种 token 级细粒度调度有助于提升大规模用户服务的吞吐量。

在固定时间间隔，或每当 GPU 空闲时，连续调度器会把所有待处理的下一 token 请求组成大小可配置的批次，再以一次 GPU 调用调度该批次。这有助于平衡吞吐量与延迟。

> 设计自定义调度器时，可以考虑混合方法：为 token 调度同时设置短计时器和最大批大小。

vLLM 和 SGLang 等先进系统将动态批处理与连续调度结合起来。其自定义连续调度器的核心思想，是尽量消除 GPU 时间线中的“气泡”。

vLLM 的 PagedAttention 是这种混合方法的一个具体示例。PagedAttention 将注意力键值（KV）缓存划分为片段或页，并在活跃序列之间动态组织与页面相关的计算。通过交错执行大型预填充 GEMM 和快速的小型解码内核，它可以在合适的高负载工作负载下获得很高的 GPU 利用率；原文所称“接近 100%”并非跨硬件与负载的普遍保证。

vLLM 使用块级 KV 缓存结构分别跟踪每个请求的状态。这种细粒度记账方式通过实时、持续地装填和多路复用工作负载，支持快速流式响应并改善硬件利用率。

另一个示例是 SGLang 的 RadixAttention，它使用基于树的 KV 缓存组织方式，并对未使用的缓存页采用惰性淘汰。在合适的高负载条件下，它同样可以获得很高的 GPU 利用率，但不能把“接近 100%”视为通用结果。稍后将更详细地介绍 vLLM 的 PagedAttention 和 SGLang 的 RadixAttention。两者均为开源实现，因此可以直接查看其调度代码。

无停顿调度（分块预填充）

当提示词极长时，可以把它拆成多个区块，交错执行预填充和解码工作。这称为*无停顿调度*或*分块预填充*。

考虑一个包含 20K token 的提示词。将其拆分为 5K token 的区块，可以降低每次迭代的最大停顿，并把每次迭代的工作量限制在固定 token 数，从而获得可预测的迭代延迟。不同区块大小下的分块预填充成本见表 16-5。

表 16-5　20K token 提示词采用不同区块大小分块预填充时的成本

区块大小	区块数量	每个区块的注意力操作数	MLP token 总数
20K	1	20K × (20K + 1) ÷ 2 = 200M	1 × 20K = 20K
10K	2	50M + 150M = 200M	2 × 10K = 20K
5K	4	12.5M + 37.5M + 62.5M + 87.5M = 200M	4 × 5K = 20K

由于 KV 缓存，每个区块的注意力成本会随完整上下文增长。每个新区块都要为自身 token 计算注意力，并与前面区块中的全部 token 进行比较。

在自注意力中，长度为 N 的序列所需 QK 点积数量呈三角形增长，约为 $N(N + 1) \div 2$。这表示每层每个头的 Q × K 点积操作数，成本关于 N 呈二次增长（$O(N^2)$）。因此，当 N = 20,000 时，约有 2 亿次操作。分块不会减少总量，只会改变工作何时可供解码器使用。分块预填充的收益在于平滑延迟和改善重叠，而不是减少注意力点积操作总数。

> 预填充自注意力在每层每个头上执行 $N(N + 1) \div 2$ 次 Q × K 点积，成本关于 N 呈二次增长，即 $O(N^2)$。请注意，分块和流水线预填充只会改变重叠关系与延迟，不会改变注意力工作的总量。要降低注意力总成本，只有减少有效上下文，或使用局部注意力/稀疏注意力。例如，使用固定注意力窗口 W 可将成本降至 $O(NW)$。

简而言之，分块预填充以紧密交错的方式调度提示词预填充和 token 生成，将大批量预填充计算的高效率与流式解码的响应性结合起来。在许多工作负载下，它可以降低尾延迟，同时保持较高吞吐量。

延迟感知调度与动态路由

延迟感知调度可以分析到达的请求，创建动态批次并进行路由，以尽量降低延迟。假设六个提示词同时到达一个双 GPU 推理系统，其长度依次为 6K、2K、6K、2K、2K 和 2K。下面比较朴素的先进先出（FIFO）调度器与延迟感知调度器。

首先，FIFO 调度器根据到达顺序创建两个批次。批次 1 为 [6K, 2K, 6K]，发送到 GPU 1；批次 2 为 [2K, 2K, 2K]，发送到 GPU 2。结果汇总于表 16-6。

表 16-6　长度依次为 [6K, 2K, 6K, 2K, 2K, 2K] 的提示词采用 FIFO 与延迟感知调度的对比

GPU	提示词批次	自注意力 QK 操作数 T(N) = N(N + 1) ÷ 2	token 数 *N*
GPU 1	6K, 2K, 6K	T(6K) + T(2K) + T(6K) = 18,003,000 + 2,001,000 + 18,003,000 = 38,007,000	14K
GPU 2	2K, 2K, 2K	3 × T(2K) = 3 × 2,001,000 = 6,003,000	6K

可以看到，FIFO 策略把前三个提示词 [6K, 2K, 6K] 发送到 GPU 1，约需执行 38,007,000 次自注意力 Q × K 点积，并处理 14,000 个 MLP token；GPU 2 则约需执行 6,003,000 次自注意力 Q × K 点积，并处理 6,000 个 MLP token。关键路径由 GPU 1 决定，约为 38,007,000 次自注意力 Q × K 点积。

相比之下，延迟感知调度器分析六个请求的预期延迟，将其重新排列为两个 [2K, 2K, 6K] 批次。在这种策略下，每块 GPU 的自注意力成本均为 22,005,000 次点积，并处理 10,000 个 MLP token，如表 16-7所示。

表 16-7　对长度依次为 [6K, 2K, 6K, 2K, 2K, 2K] 的提示词进行延迟感知调度

GPU	提示词批次	自注意力 QK 操作数 T(N) = N(N + 1) ÷ 2	token 数 *N*
GPU 1	2K, 2K, 6K	T(2K) + T(2K) + T(6K) = 2,001,000 + 2,001,000 + 18,003,000 = 22,005,000	10K
GPU 2	2K, 2K, 6K	T(2K) + T(2K) + T(6K) = 2,001,000 + 2,001,000 + 18,003,000 = 22,005,000	10K

在这个简化成本模型中，关键路径上的自注意力工作量从 38,007,000 降至 22,005,000，减少约 42%。因此，延迟感知调度器有可能显著缩短 TTFT。由于它会考虑提示词长度、自注意力中呈三角形增长的 N(N + 1) ÷ 2 成本，以及 MLP 的 O(N) 成本，因此能够平衡计算权重并尽量降低总体延迟。例如在本例中，延迟感知调度器把 2K token 请求提前，使这些较轻的请求更早完成预填充并开始解码，而无须等待较重的 6K token 预填充完成。

这些计数假设采用只计算有效 token 的变长融合注意力内核，例如 FlashAttention，或使用 cuDNN 后端的 PyTorch scaled dot product attention（SDPA）。如果内核把序列填充到批次中的最大长度，那么注意力算术成本将接近批大小乘以该批次最长序列的 T(N)。在这种情况下，FIFO 与延迟感知策略之间的差异会缩小。

简而言之，如果批次中的到达顺序从全局角度看并不理想，朴素 FIFO 调度器可能使某块 GPU 负载过重。延迟感知调度器可以分析到达的请求，把它们重新排列成更加均衡的批次，以尽量降低延迟。

> 一些推理服务框架已经引入强化学习来在线调整调度策略。这是在此处所述延迟感知策略的基础上，针对不断变化的流量模式进行自动调优。

系统级优化

系统级优化技术包括重叠通信与计算、提高 GPU 利用率、管理功耗与热约束、处理错误，以及优化内存访问。总体目标是尽可能提高 GPU 执行有用工作的比例，即 goodput，而不是机械追求 100% 的繁忙率。本节还讨论吞吐量与延迟之间的权衡，以及如何找到适合生产 SLO 的平衡点。

重叠通信与计算

正如本书多次所述，对于分布在多块 GPU 上的大型模型，重叠通信与计算是推理性能的关键。GB200/GB300 NVL72 等系统具有极高带宽（机架内 GPU 到 GPU 的 NVLink 聚合带宽最高约 130 TB/s），数据传输足够快，计算不易因等待数据而断粮，因此重叠更加有效。不过，即使使用 NVL72，也必须采用多个 CUDA 流和非阻塞 collective，以隐藏延迟，让全部 72 块 GPU 保持忙碌——即使在这种带宽下也一样。

> 实践中，可以启用 NCCL 的 GPUDirect RDMA 支持——并使用 NCCL group 调用重叠多个小型 all-reduce 操作。此外，可以考虑使用 SHARP（在配备合适交换机的 InfiniBand 上可用），把归约操作卸载到网络结构中。这种优化在某些网络结构和拓扑上能够提高吞吐量；一些测试显示，大型 all-to-all通信的吞吐量约提升 10%～20%，但结果取决于具体配置。

考虑一个需要在 GPU 之间传输数据的推理步骤（例如 MoE 的 all-to-all），或者仅仅是把提示词数据从 CPU 发送到 GPU，再把响应从 GPU 返回 CPU 以交付最终用户。在所有这些场景中，只要可能，就应在 GPU 执行其他工作时异步进行传输。

一个基本示例是使用独立 CUDA 流重叠计算与数据传输。例如，在专用传输流（配合页锁定主机内存）上启动 `cudaMemcpyAsync`，同时让计算内核在默认流上运行。

> 通过 PCIe 或 NIC 传输时，务必使用页锁定主机缓冲区。这样 CUDA 驱动就能直接执行 DMA，并与计算重叠。

随后只在真正需要数据时使用 CUDA event 同步，避免 GPU 因 I/O 空闲。这样，数据传输使用非阻塞 CUDA 调用（例如配合页锁定内存）和 CUDA 流，GPU 不会停下来等待这

些操作。在具备全双工 NVLink 带宽的现代 GPU 上，这种重叠可以大幅隐藏通信延迟；不过，仍应针对具体工作负载进行性能分析验证。

此处的数据传输可以包括把新的提示词嵌入从 CPU 发送到 GPU，或把最后一个 token 的 logits 从 GPU 移到 CPU。例如，GPU 计算出一批新 token 的 logits 后，可以立即异步把 logits 复制到 CPU 进行后处理（例如采样），或作为流式响应返回客户端；与此同时，立即启动下一个计算内核，处理仍在 GPU 上的另一批 token。

在多节点场景中，可以把工作拆成区块，使 all-to-all 等通信 collective 与计算重叠。一些 MoE 运行时会把 token 批次拆成两半：专家处理前一半 token 时，同时开始发送后一半 token。

专家处理完前一半时，后一半数据已经到达目标 GPU，因而无需等待即可继续。这要求相对于计算内核谨慎安排 NCCL 调用。

例如，可以使用 CUDA event 指示批次的一半何时完成；此时启动该部分数据的 NCCL all-to-all，同时让下一部分继续计算。还可以使用 CUDA Graph 捕获这些异步模式，降低启动开销。由于 GPU 等待通信的空闲时间减少，利用率会得到改善。

另一个可以重叠的领域是 CPU 与 GPU 之间。例如，GPU 忙于生成下一个 token 时，CPU 可以并发准备下一个输入，或处理之前生成的 token 的结果。高度优化的推理引擎会让 CPU 端预处理（例如输入文本分词）与其他请求的 GPU 计算并行进行。例如，引擎可以准备下一批，同时与当前批的 GPU 计算重叠。

这听起来显而易见，但需要多线程架构：一个线程处理网络和排队，另一个线程启动 GPU 操作，还可以有另一个线程负责响应后处理等。Python 或 C++ 推理循环不应因等待本可并发执行的操作而阻止 GPU 启动新工作。

在流水线并行场景中，系统应让通信（流水线阶段之间）与每个阶段内部的计算重叠。例如，GPU 0 完成 token t 的部分工作后，可以向 GPU 1 发送激活值，同时开始为另一个序列处理 token t+1。

只要使用的是不同资源，现代互连和框架就允许计算内核与数据传输重叠。应在推理流水线中使用多个 CUDA 流，让每个流水线阶段拥有独立流。这样，激活值的发送/接收就可以与其他流中针对不同微批次的计算并行进行，从而减少流水线气泡。

在网络方面，NVIDIA GPUDirect RDMA 等技术允许 InfiniBand NIC 直接读写 GPU 内存，无须 CPU 参与，也不需要经过主机内存中转。利用 GPUDirect 跨节点传输 KV 缓存或专家激活值，可以降低延迟和 CPU 开销。

GPUDirect RDMA 去除了经由主机内存中转的步骤，使 NIC 的 DMA 引擎能够直接访问 GPU 内存。CPU 负责提交工作请求，但数据路径绕过 CPU，从而释放 CPU 核心执行其他任务。

考虑一个实际示例：使用 InfiniBand 在两个 NVL72 机架之间实现 MoE 层的 all-to-all 通信。如果先同步执行 all-to-all，再进行计算，在没有重叠时 GPU 利用率可能很低；但如果把 all-to-all 通信批次与计算重叠，就能显著提高 GPU 利用率。

在 MoE 层重叠 all-to-all 通信，需要把 token 批次拆成两半，并在第一半 token 的专家仍在计算时开始交换第二半。这样便可把通信延迟隐藏在交错的计算中。

基本做法是：每块 GPU 先为已经拥有的 token 计算本地专家输出，同时接收另一节点发来的剩余 token。完成第一批时，第二批已经到达，可以立即开始计算。

这种优化相当底层，涉及 NCCL group 与 CUDA 流之间的 CUDA event 同步，但能够带来可观的吞吐量改善和更平滑的延迟。实践中，先把 NCCL all-to-all 作为 NCCL group 的一部分启动，不等待其完成；然后立即启动下一个计算内核，并使用 CUDA event 检查完成状态。

对于流式输出，还可以让 I/O 与计算重叠。例如，一旦生成了几个 token，就通过套接字发送给最终用户，同时模型已经开始处理后续 token。这样就能把网络延迟（例如把响应发回用户）隐藏在计算（例如生成后续 token）之后，使用户看到没有停顿的稳定流。

所有现代 LLM 推理引擎都采用这种向客户端非阻塞流式传输的模式，使用独立线程通过 SSE/WebSockets 发送 token 更新。

> 如果自行实现流式输出，务必使用线程安全队列或锁，把生成的 token 交给网络线程。不要在交接过程中引入同步问题，这类问题往往很难定位和调试。

如果不进行重叠，系统会先生成少量 token，然后在把 token 发回最终用户时让 GPU 空闲。更好的做法是由异步 I/O 线程从响应缓冲区取出输出并流式返回用户，使推理调度线程能够继续提交更多 token 生成工作。这样就把 token 生成（计算）与输出传输（I/O）组成流水线。需要注意，主机线程与 CUDA 流是不同概念。

简而言之，重叠通信与计算需要用流水线思维，并尽可能使用异步操作。现代 GPU 和网络硬件提供了相应原语，包括 CUDA 流、非阻塞 collective 和 RDMA。在内核内部，CUDA 设备端原语（例如与 `cuda::barrier` 配合使用的 `cuda::memcpy_async`，后者来自 CUDA pipeline 头文件）有助于让全局内存到共享内存的移动与计算重叠。（注意：主机到设备以及设备到主机的传输，仍需显式 CUDA 流和页锁定主机内存才能与计算重叠。）高效的 LLM 推理系统会充分利用这些特性。

简而言之，只要可能，就应始终让通信与计算重叠。否则，即使采用 NVLink/NVSwitch，扩展到许多 GPU 时仍会低于硬件峰值性能。这些优化通常对最终用户不可见，因为模型输出不会改变；但它们对于榨取推理集群的每一点性能、保持用户对应用的体验至关重要。

最大化 GPU 利用率：吞吐量与延迟的权衡

性能调优的最终目标是让 GPU 尽可能忙于执行矩阵乘法等有用工作，同时尽量减少空闲和未充分利用的 GPU 资源。前面介绍的批处理、并行和重叠等技术，都是为了实现接近 100% 的 GPU 利用率。

GPU 利用率百分比，尤其是有用利用率（goodput），是需要持续监控的重要性能指标。虽然希望有用利用率达到 100%，但必须确保不违反延迟等 SLO。利用率提高到某个程度后，可能会出现收益递减。

假设一个采用朴素实现的推理服务器当前 GPU 利用率为 60%。调查发现，解码阶段受单线程顺序处理所有序列所限，形成瓶颈。引入使用多个流的并发解码后，通过交错解码（如前文所述），GPU 利用率提高到 95%，吞吐量翻倍。这表明并发解码发挥了作用，但具体增益需要实测。

可以把更多请求放入大型批次，尝试达到 100% 利用率，但这会拖慢单个查询。通常可以测试不同批大小，绘制吞吐量—延迟曲线。曲线上往往有一个明显的“膝点”，从这里开始，吞吐量收益会付出过高的延迟代价。

要在吞吐量—延迟曲线上找到最佳点，首先应开启完整并发并尽可能重叠操作，确认硬件能够保持忙碌。然后逐步增大批大小，直到资源利用率达到最大。此时测量单查询延迟，再适度缩小批大小（例如从 16 降到 8），以满足响应时间目标。

建议同时监控 p50 中位延迟以及 p95、p99 尾延迟。细小抖动和批次填充不均可能产生长尾异常值，在 p95/p99 中尤为明显。在承载大量请求的大型集群中，即使 0.1% 的异常值也可能频繁出现。因此，衡量用户体验时，p99 甚至 p99.9 可能比 p50 更重要。此外，为满足严格 SLO，长尾延迟常常迫使系统过度配置资源；降低尾延迟因而具有直接的成本收益。

> 一个实用经验是，把批大小限制在峰值吞吐量对应值略低的位置。团队通常以最大峰值吞吐量的 90% 为目标，这称为*余量缓冲*。这是因为全速运行可能造成不可预测的延迟峰值。例如，如果将批大小提高到 16 个请求后，少数请求开始出现轻微延迟增加，就应把批上限降到 8。这样可以获得更一致的延迟，而吞吐量只会略微下降。

一些推理系统可以根据这些指标即时动态调整批大小。第 17～19 章将介绍这一技术及更多自适应推理策略。

需要记住，GPU 持续运行在 100% 可能触及功耗和温度上限而发生降频。有时，使用高效内核并以 90% 利用率运行，反而会超过发生降频时的 100% 利用率。下面讨论 GPU 的功耗和热约束——CPU 应用开发者和系统工程师常常不会考虑这些特性。

功耗和热约束

还需考虑功耗和热约束。如果冷却不足，让 GPU 持续运行在 100% 会导致温度降频。现代 GPU 系统通常采用液冷，以减少温度降频并维持性能；如果使用较旧的风冷系统，就要留意降时钟。

GPU 利用率较高时，可以使用 `nvidia-smi` 或 DCGM 监控降时钟。具体来说，DCGM 通过 `DCGM_FI_DEV_CLOCK_THROTTLE_REASONS` 提供 XID 降频原因；当 GPU 因功耗而降频时，`nvidia-smi` 会显示“Pwr Throttle”标志。

GPU 还可能触及板卡功耗上限，尤其是在使用加速时钟时。现代 GPU 在加速状态下功耗显著增加，因此应监控 GPU 是否因触及功耗上限而降时钟。发生这种情况时，GPU 性能会低于理想水平。务必监控 DCGM 的 `DCGM_FI_DEV_POWER_USAGE` 指标，并在超出正常范围时告警。

> 如果 GPU 持续触及功耗上限，可以考虑启用 Dynamic Boost 模式，或略微降低时钟，以避免可能导致延迟骤升的温度降频。

在软件层面，可以通过增加极短的批间延迟稍微降低利用率，也可以限制并发度。现代 GPU 还支持时钟上限设置。因此，与其增加延迟，不如使用 `nvidia-smi -lgc` 等命令略微限制 GPU 时钟，避免触及温度上限。这样只会小幅降低吞吐量，却能带来更一致的性能。

从硬件角度，可以改进冷却；如果冷却已经足够，也可以提高功耗上限。要提高功耗上限，可使用 `nvidia-smi -pl` 或调整 GPU 加速设置。

这些都说明，把真实系统推到极限可能产生严重影响性能的意外副作用。建议在推理引擎中加入“boost-off”标志，使系统检测到功耗或温度约束导致降频、性能下降时，能够即时应用软件层面的规避措施。在完全恢复稳定性和性能之前，让系统以略低利用率、更低温度运行。

错误处理

错误处理通常不被视为性能问题，但高效处理错误十分重要。利用率已经很高的推理系统，处理大量错误峰值的余量更小，尤其是因为错误处理通常不是系统中最优的代码路径。

在故障场景中，关键是快速失败：如果请求注定会出错，最好立即返回错误，不要因缓慢失败浪费 GPU 时间。务必在推理调用周围实现适当的异常处理和超时，以捕获挂起或崩溃。

此外，建议实现背压。这意味着错误激增时，可以开始拒绝新请求，或减小批大小，为系统留出恢复余量。

在大规模环境中，建议预留余量，增加一些大部分时间处于空闲状态的额外副本。这些副本可以充当错误峰值或其他意外情况的缓冲。虽然自动扩缩容可能是降低成本的首选，但要记住，扩缩容需要时间来配置新资源。

空闲容量需要成本，但失去客户或违反 SLA 的代价通常更高。稳态期间建议至少预留 10%～15% 的缓冲容量；对于不能停机的关键服务，建议预留 100% 缓冲容量，即完整的一套集群副本。

预热且空闲、能够按需立即承载额外负载的节点没有替代品。失去最终用户的成本，很可能高于保留少量空闲副本、随时应对额外意外负载的成本。

内存

资源利用优化同样适用于内存。GPU 内存容量和带宽的增长相对渐进，而模型规模和上下文长度却快速增长。因此，内存仍是需要优化的宝贵资源。

因此，应有效利用 GPU 内存和内存带宽。内存通常由模型权重和 KV 缓存占用。推理期间，最好始终将模型权重保留在 GPU HBM 中。如果在 CPU DRAM 或 NVMe 存储之间换入换出内存，就会产生额外缺页和传输延迟。

即使是 Grace Blackwell 和 Vera Rubin 等现代 CPU-GPU 超级芯片，也存在这种额外传输延迟。因此，高性能 LLM 推理引擎会显式管理内存，而不是依赖按需换页。

压缩是降低推理系统内存用量的有效技术，尤其适用于预填充阶段为每个进入系统的查询生成的 KV 缓存。

由于 KV 缓存大小随查询数量和输入规模增长，应考虑 KV 缓存压缩和量化。KV 缓存压缩/量化意味着在模型能够承受精度损失时，以较低精度存储键和值。虽然并不理想，但对于很少使用的 KV 缓存数据，卸载也是一种选择。

KV 缓存卸载和内存池分配

将很少使用的 KV 缓存条目换出（分页换出）到 CPU 内存或磁盘后，推理引擎就能为更活跃的数据腾出空间。这类似于虚拟内存的处理方式。

例如，vLLM 使用受管理的 KV 缓存内存池和分页式块管理；CPU swap/offload 可由相应配置或扩展实现，但 PagedAttention 本身不等同于默认把 KV 卸载到 NVMe。类似地，SGLang RadixAttention 使用树状缓存并可惰性淘汰低优先级前缀；NVIDIA Dynamo 生态也提供 KV 缓存卸载和内存管理机制，具体能力取决于所用后端和版本。

如果没有良好的 KV 缓存分配器，当不同长度的请求流经系统时，内存碎片可能过多（例如约 20%～30%）。在发生 OOM 之前，这会限制系统能够处理的请求数。应使用大内存池分配器，正如前面章节所述。

采用适当的内存分页策略，可以把碎片降低到可容忍的几个百分点。这样就能在 GPU 中装入更多上下文，同时保持高利用率而不使系统崩溃。

借助适当的 KV 缓存内存管理，现代推理引擎可以服务更多并发用户，而不会耗尽 GPU 内存。它们管理自己的 KV 缓存内存池，并把数据卸载到 CPU 和 NVMe 存储，从而实现接近满载的内存利用率和极少的碎片。

代价是：上下文再次活跃并需要换入时，会增加一些数据传输延迟。不过，与请求总延迟相比，这项开销通常较小。

内存利用率和计算利用率相辅相成。内存管理不善会浪费内存，使 GPU 无法通过更多并发任务得到充分利用。尽可能把相关数据保留在 GPU 上，系统就能服务大量并发请求。

内存管理不善会在峰值负载下造成反复 OOM 崩溃，使 GPU 从资源池中退出，并引发级联延迟问题。通过适当的内存管理避免 OOM，才能最大化利用率并保持集群稳定。

简而言之，高效内存管理可以提高有效吞吐量、减少内存碎片、避免意外 OOM，并允许更多并发在途请求，尤其适用于高吞吐场景。

实时推理的量化方法

提高推理性能最有效的方法之一是降低精度。这会立即减少内存用量和内存带宽利用，并提高计算速度。

量化使用更少的位数表示模型权重，有时也表示激活值。现代 NVIDIA GPU 通过低精度 Tensor Core 原生支持 FP16、FP8 和 FP4 等格式的低精度算术。

本节专门讨论推理量化技术，包括GPTQ、AWQ、SpQR 及其他结构化稀疏感知方法等仅权重量化方法，以及同时降低权重和激活值精度的方法。具体而言，GPTQ 和 AWQ 已在实践中证明非常有效。

对于许多大型模型，使用 GPTQ 进行 4 位仅权重量化，在特定校准数据和评测任务上可以保留 FP16 模型 99% 以上的准确率，并可能带来约 2 倍推理加速和约 4 倍的模型体积缩

小。AWQ 还能改善 3～4 位量化的准确率。这些技术已集成到 Hugging Face Transformers、PyTorch、vLLM 等许多 AI 框架中，并支持直接加载 GPTQ 和 AWQ 量化模型。

> 量化收益取决于工作负载。不要假定每条低精度路径都更快；实际收益取决于硬件、内核覆盖范围、模型架构、序列长度分布以及批处理和并发组合。在投入生产前，务必在准确的服务配置上验证每条量化路径。

下面讨论使用低精度格式时准确率与性能之间的权衡，以及如何将量化有效、安全地集成到服务工作流中。

将精度从 FP16 降至 FP8 和 FP4

最初，LLM 推理从 FP32 转向 TF32、FP16 或 BF16 等低精度格式后获得了显著收益。例如，NVIDIA Tensor Core 使用 FP16 时的吞吐量可达到 FP32 的约 2 倍，原因是将半精度乘加操作融合到专用 Tensor Core 硬件流水线中，从而提高数学运算吞吐量；实际准确率损失仍取决于模型和工作负载。

FP8 将精度进一步降至 8 位浮点。与 FP16/BF16 相比，它可将存储占用减半；在支持相应 FP8 Tensor Core 路径的 GPU 和内核上，每周期可执行更多低精度乘加，数学吞吐量可能进一步提高。

在 PyTorch 中，仅通过 `torch.set_float32_matmul_precision("high")` 启用 TF32 数学运算就可能获得中等加速；若要充分利用 8 位和 4 位精度，则可使用 NVIDIA Transformer Engine（TE）提供的支持。TE 以库的形式提供 FP8 和 FP4 内核，使现有代码只需少量修改即可使用这些低精度。

NVIDIA TE 会在这些低精度下自动管理逐张量缩放因子。推理时，服务端可以以 FP16 加载模型，却使用 FP8 矩阵乘法。

TE 会对每个张量应用缩放，以维持数值稳定性。缩放因子通常有两种确定方式：使用训练期间的代表性数据进行固定的预先校准，称为*静态校准*；或者动态计算、跟踪张量最大绝对值的方式，称为*基于 amax 的动态缩放*。图 16-7展示了 TE 如何使用范围分析、缩放因子和目标格式进行精度转换。

如果需要更高压缩率，FP4 在减少模型权重存储和流量方面优于 FP8。计入缩放元数据和打包后，有效压缩通常约为 FP8 的 1.8 倍、FP16 的 3.5 倍；但实际比率取决于格式和实现。由于 FP4 动态范围非常有限，要实现可靠的 FP4 推理，需要逐通道缩放，或使用 GPU TE 支持的逐块*微缩放*等校准方法。这些技术有助于在大型网络中使用 FP4，同时尽量减少准确率损失。

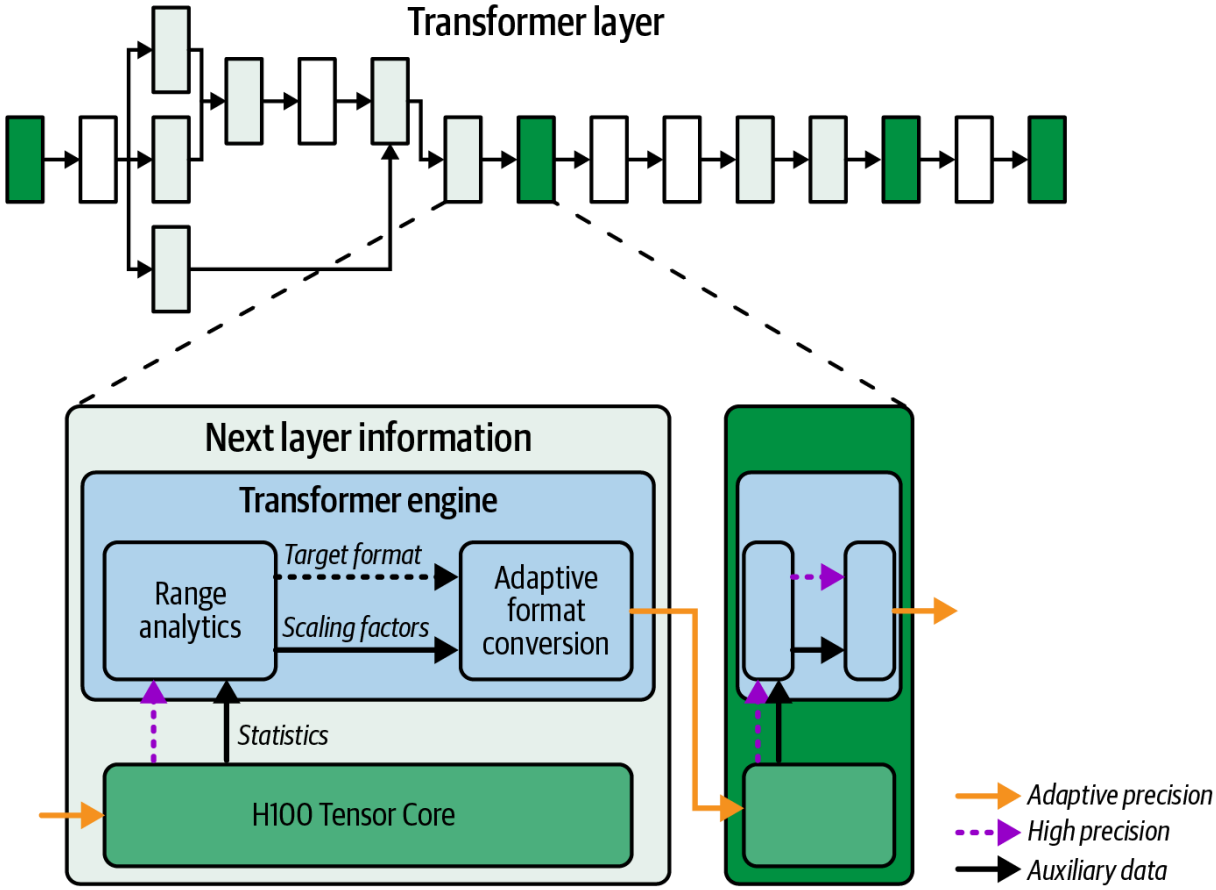


图 16-7　NVIDIA Transformer Engine（TE）在 Transformer 层中使用范围分析、缩放因子和目标格式进行精度转换

仅权重量化（GPTQ、AWQ）

现代 LLM 服务栈中的仅权重量化通常使用 GPTQ 或 AWQ 等方法把权重压缩为 4 位整数，同时让激活值保持 FP8、FP16 或 INT8 等较高精度。与 FP16 相比，这可将权重内存占用降至约四分之一，将权重带宽需求至少减半；在校准得当的情况下，准确率损失通常较小。

NVIDIA 的 FP4 实现正式名称为 *NVFP4*，本书简称 *FP4*，它在硬件中采用逐块微缩放。NVIDIA TE 提供块粒度 NVFP4 微缩放的硬件支持。

> 应根据内核使用逐张量或逐通道缩放。建议为激活值探索逐张量缩放，为权重探索逐通道缩放。例如，使用 FP8 E4M3 量化 KV 缓存时，常采用逐张量缩放。

逐块微缩放不为整个张量使用单一缩放因子，而是为张量内每个固定大小的块（通常为 32 个元素）维护独立 scale。这些 scale 会适应局部数值分布；与单一 scale 量化相比，可以更好地保留数值范围并减少量化误差。

始终使用能够反映推理工作负载的校准数据进行量化。仅在训练数据子集上进行一次性校准，可能无法捕获运行时使用模式。

实践中，GPTQ（训练后量化）和 AWQ 常用于 LLM 的 4 位权重；使用代表性校准数据时，特定任务上的准确率损失可能很小。Hugging Face 等提供的开源工具可以自动应用这些技术。

MoE 专家结构使权重量化更具吸引力，因为使用低精度权重后，内存可以容纳更多专家，从而减少专家与 CPU 内存之间的换入换出。如有需要，还可以使用规模更大、活跃专家更多、专家副本更多的模型。

GPTQ 等训练后量化（PTQ）工具使用近似二阶算法逐层把权重量化到 3～4 位。在部分模型和硬件上，这可以在数个 GPU 小时内完成，并保持较小的准确率退化。较新的 GPTQ 变体通过非对称校准和并行计算进一步改进算法，以减少量化误差，并将高效低位支持扩展到更大的模型。

AWQ 会识别一小部分“显著”权重通道。显著通道会产生较大的激活幅值，对模型输出产生不成比例的影响。

AWQ 在把全部权重转换为 4 位精度（例如 INT4）之前，使用通道专用缩放来保护这些通道。支持相应量化格式和内核的运行时会使用这些缩放因子，以尽量保持模型保真度。

PyTorch 等 AI 框架以及 vLLM、TensorRT-LLM 等推理引擎通常支持加载 GPTQ 和 AWQ 量化模型检查点，但具体格式和内核兼容性取决于版本。

激活量化

将激活值与权重一同量化，可以让 GEMM 输入乃至累加器使用较低精度，从而改善性能，并减少注意力 KV 缓存和 MLP 层中间激活值的内存流量。

不过，激活值分布会随输入显著变化，因此若没有适当微调或校准，激活量化可能很困难。一种折中方案是使用经过校准的 INT8 激活值。NVIDIA INT8 模式采用逐张量校准；TensorRT 会从代表性校准数据生成的激活直方图中选择缩放因子。

SmoothQuant 是一种面向 8 位激活量化的训练后量化方法。它通过简单的行/列缩放算法，将部分量化难度从激活值迁移到权重，使权重和激活值都能使用 INT8，而无须完整量化感知训练。在特定模型和评测中，这可实现准确率损失较低（例如低于 1%）的完整 INT8 推理；仍需要代表性校准数据确定迁移强度。

一些实验表明，先使用 SmoothQuant 量化激活值，再对权重应用 GPTQ/AWQ，在低精度下可能更有利于保持准确率；具体组合需要按模型和内核验证。

训练后量化工作流

前面介绍的量化技术是在训练后应用的，与模型训练期间进行的量化感知训练相对。典型工作流是先以 FP16/FP32 训练或微调模型，再在代表性数据集上运行 GPTQ 或 AWQ 等训练后校准脚本，确定量化参数，最后把量化后的模型权重加载到推理引擎中提供服务。

如有需要，还可以进行小规模微调以恢复损失的准确率；GPTQ 和 AWQ 是否需要微调，取决于模型、位宽和任务。如果需要完整 QAT，可以在模型图中加入“伪量化”操作并训练若干轮次，以模拟低精度数学运算。这有助于评估该精度下的预期准确率。

> 实践中，LLM 的量化感知微调计算成本很高，并不总是可行。不过，可以使用较小的校准数据集（例如 1,000 个提示词），结合百分位裁剪或 LMS（损失感知量化）等技术调整量化 scale，而无须完整重新训练。数据量只是示例，应按工作负载覆盖度验证。

需要特别谨慎地权衡压缩率与模型鲁棒性。量化可能放大某些错误。例如，如果模型对某项事实知识的掌握本来就处于临界状态，量化可能使其给出错误答案。

始终应在下游任务上验证，因为 PTQ 所作假设可能忽略细微的分布偏移。使用低精度量化模型时，应对响应进行充分 A/B 测试，确保评测没有显示质量或安全性退化。如果出现退化，就应让模型保持较高精度，或尝试在量化形式下进行轻量微调以恢复性能。

联合使用权重与激活量化

把激活值量化到 4 位仍然很有挑战。将低精度权重量化与较高精度激活量化结合，通常能在节省内存、计算效率和准确率之间取得较好平衡。因此，许多生产系统会把 4 位仅权重量化（例如 GPTQ/AWQ）与 8 位激活量化结合使用。

具体来说，在一种 W4A8（8 位激活）变体中，运行时解包 INT4 权重，使用学习或校准得到的 scale 将其转换为 FP8 输入，再在 FP8 Tensor Core 上执行矩阵乘法。TensorRT-LLM 等推理引擎提供这种混合路径；校准得当时，特定评测中的准确率损失可能很小。该方法保留 FP8 激活值的动态范围，同时使权重原始位宽相对 FP16 减少约 4 倍，但实际存储比率还需计入元数据和打包。

相比之下，传统 INT4/INT8 W4A8 方案将 4 位整数（INT4）权重与 8 位整数（INT8）激活值配对，并在 INT8 Tensor Core 上运行计算。这种方法依赖基于直方图的校准，把激活值范围映射到 INT8，并尽量减少质量损失。

现代整数 Tensor Core 流水线上的 INT4/INT8 内核可能提供略高的原始吞吐量，但需要谨慎校准激活值，而且动态范围不如 FP8。

INT4 与 FP8 混合方法试图结合两者优点。该方法会解包 4 位整数（INT4）权重，将其反量化为 FP8 输入，再在 FP8 Tensor Core 上执行计算。这种混合 W4A8 变体在受支持的现代 GPU、融合内核和适当校准下，可能获得较小准确率损失、较低权重带宽需求和良好吞吐量。

现代 GPU 的生产环境常见两条不同低精度路径。第一，NVFP4 工作流使用 FP4 块，并由 Transformer Engine 或 TensorRT 管理微缩放。第二，W4A8 工作流使用 INT4 权重和

FP8 或 INT8 激活值，并在 TensorRT-LLM 中采用融合反量化执行。应根据模型校准结果和准确率目标选择路径。

总之，量化是降低推理成本的有效方法之一。模型体积缩小 2 倍或更多并不自动意味着每块 GPU 的吞吐量翻倍；实际收益取决于工作负载是否受权重带宽限制，以及是否有匹配的融合内核。GPTQ、AWQ 和 SmoothQuant 等技术有助于在较小准确率损失下获得这些收益，但“低于 1%”等结果只适用于特定模型和评测。

> 建议先从 8 位权重开始，再评估 4 位仅权重量化以获得额外收益。只有在需要进一步优化并愿意投入校准时间时，再转向 W4A8。

下一步是消除转换开销。把量化—反量化操作直接融合进计算内核，有助于保留量化带来的收益。

将量化—反量化步骤融合进执行图

现代推理引擎使用 TE 执行权重打包，并提供高效混合精度数学运算；具体格式仍可能需要校准或 scale 管理。这些推理引擎通常实现 CUDA/Triton 内核，在需要时把“量化—反量化”步骤融合进执行图。

如果独立 Quantize/Dequantize 内核会引入过多额外启动，抵消低精度数学运算的延迟和带宽收益，就应把这些量化—反量化步骤融合进执行图。这对于缺少原生融合 INT8 支持的推理后端尤其有用。

对于高吞吐推理流水线，把量化—反量化融合进主要计算内核尤其有价值，可以消除转换造成的瓶颈、恢复性能。下面探索现代 AI 推理服务引擎支持的各种应用层优化。

应用层优化

除了核心模型和系统优化，还有一些更高层技术可以显著改善 LLM 服务的性能和用户体验。这些方法作用于应用层或推理服务层，用于改进提示词的构造和缓存、对话历史的保留方式、请求到不同模型的路由方式等。

这些优化不涉及修改模型权重或部署新硬件，而是应用层的算法和系统改进。它们通常只需较低成本，就可能显著提高效率和易用性。

本节讨论其中几种优化，包括提示词压缩、前缀缓存与去重、后备模型路由和流式输出。这些策略通过减小输入规模、避免冗余计算、妥善处理不同请求类型，以及改善最终用户感知延迟来提高性能。

提示词压缩

用户经常向 LLM 发送很长的提示词或对话历史，但生成合适响应并不一定需要全部上下文。提示词压缩是指在不丢失相关信息的情况下，缩短或简化输入提示词的一组技术。

有些系统提示词或系统注入指令非常冗长（“你是 ChatGPT，一位旨在帮助用户的友好助手……”）。大型系统提示词会在每个请求的输入上下文中占据大量空间。

提示词压缩会减少模型需要完成的工作量。输入更短意味着 GPU 计算更少，因此可直接节省成本。

> 请记住，基于 Transformer 的 LLM 中，稠密全注意力的理论计算复杂度为 $O(n^2)$，其中 n 是以 token 数衡量的输入长度；实际运行时间还取决于内核、硬件和内存行为。

一种简单的提示词压缩方式是删除冗余或无关文本。假设用户提示词中包含大段与回答问题无关的文本，例如复制粘贴了一篇文章，但问题只涉及其中一段。此时，上游组件可以先总结或提取相关部分，再送入 LLM。

另一种提示词压缩方式是对话摘要或截断。例如，在很长的聊天历史中，早期对话可能已经不再相关。系统可以把较早内容总结为精简形式，智能裁剪对话。

> 许多生产聊天机器人会自动进行提示词压缩，以保持在上下文长度上限以内，并加快整体处理。对于长期对话，这是一项重要能力。

为此，可以运行一个小型 LLM 或传统规则系统，把最早的对话内容总结成简短摘要。常见策略示例是：当对话超过最大上下文长度的 75% 时，总结最早的 25% 消息。然后把摘要放在较新对话内容之前，作为后续的新提示词。具体阈值应根据任务和质量评测调整。

实现提示词压缩时，必须确保关键事实没有丢失。一种方法是让模型压缩提示词（例如生成摘要），再通过针对压缩内容提问来验证。这样，算法就能检查压缩后的提示词是否保留了关键信息。

这可以避免超长聊天产生过高延迟，并让模型聚焦于最近的对话。截断早期轮次会减小有效提示词长度 N，使稠密因果预填充自注意力工作量从 $N(N+1) \div 2$（约为 $O(N^2)$）降至较短窗口对应的更小三角形成本。虽然成本仍随窗口大小近似二次增长，但更小的 N 会显著影响超长对话。

良好摘要可以过滤无关细节、改善响应；糟糕摘要却可能遗漏用户真正关心的重要输入。只有在系统对摘要质量有信心，或对话明显偏题时，摘要才更合适。

提示词清洗

另一种技术是提示词清洗，用于改善输入格式和分词，减少发送给模型的不必要空白或标记。分词器会处理空格和换行等字符，但字符与 token 并非一一对应；关键是减少不影响语义的 token。

OpenAI 的 tiktoken 等分词器对空白的处理较高效，但包含大量 Markdown 和 HTML 的大型提示词仍可能增加 token 数。删除无用 HTML 标签、把花式引号转换为普通文本

等简单预处理，可以避免异常分词并减少 token 数；但不得删除承载结构或安全语义的标记。

例如，不发送不影响输入含义的空行和重复标点，可能减少推理引擎的计算量。每次或许只能节省少量 token，但在数千个请求中会不断累积，长输入提示词尤其如此。

在某些情况下，可以用引用代替完整内容来压缩提示词。例如，如果用户提示词包含系统以前见过的一大段文本，也许是在引用之前存储的文档，就可以在应用层用“file0”之类的引用表示。

随后可以微调模型以解析该引用并获取实际内容，或由检索系统处理。这已经进入检索增强生成（RAG）领域，本书不再展开；重点是，如果有可靠替代方法，就不必始终把原始内容全部送入模型。

> 一些推理引擎支持每个会话只设置一次系统提示词，而不必每次发送。与为每个请求重新压缩系统提示词相比，这通常是更好的方案。

下一节将讨论如何通过前缀缓存提高大型系统提示词的效率。现在先使用提示词压缩创建更短、功能等价的指令集。例如，可以训练模型使用特殊 token 或元数据表示很长的系统提示词，从而无须每次处理冗长的自然语言版本。

假设一个包含 200 个 token、充满文本规则的系统提示词，可以替换成 10 个特殊 token，由它们触发原来以 200 个自然语言 token 表达的相同规则。这要求模型经过训练，能够解析元数据、推导规则并遵循规则。

“配置”token 仍是一个研究方向：某个配置 token 指示模型使用特定的预配置指令集。可以把它理解为给定提示词前缀（例如系统提示词）的唯一 ID。例如，可以微调模型，让它识别 `<POLICY_A>` 之类的 token，用来代替一份长达 500 个单词的策略。

> Hugging Face Transformers 库实现了著名的 CTRL 方法，可作为研究控制码/特殊 token 的起点；它并不等同于任意服务系统中的通用提示词压缩机制。

使用特殊 token 和元数据替代长系统提示词，更多属于训练设计。如果它确实缩短提示词并降低预填充开销，就可能提高推理速度。大规模服务中，这会减少每个查询所需计算并节省成本；实际收益取决于缩短比例和模型是否正确学习了等价语义。

前缀缓存

面向 LLM 的多个请求经常在输入中共享相同*前缀*，例如许多查询可能都以同一个系统提示词开头。无需每次为相同前缀重新计算模型状态，而是可以计算一次，并在后续请求中复用相同 KV 缓存。

这项技术称为*前缀缓存*，有时也称前缀记忆化。使用前缀缓存时，系统会存储 Transformer 的状态，包括注意力层中的键和值；再次出现相同前缀的请求时，就复用这

些状态。

> 前缀缓存通过复用已缓存的前缀计算，可以显著减少重复预填充工作。共享前缀只计算一次，随后每个请求只需承担自身未缓存后缀的计算成本。

vLLM 实现了前缀缓存（`enable_prefix_caching=True`）以避免重复计算。它先判断到达提示词的前 N 个 token 是否与先前请求或同一会话早些时候写入缓存的前缀匹配。如果匹配，vLLM 就复用缓存的 KV 块，避免为这 N 个 token 重新执行昂贵的注意力计算。应确保已启用前缀缓存，并分配足够内存。

vLLM 等推理引擎可以自动把到达的查询组成微批次，并使用连续批处理等方法提高 GPU 利用率、在多个请求之间分摊开销并控制延迟。如果没有直接使用此类引擎，也应在自己的技术栈中寻找利用共享前缀的方法。

前缀缓存可以加速具有重复前缀的工作负载。假设要针对一份长文档提出 10 个独立问题。每个提示词都包含该文档，后面跟一个问题，例如“[长文档文本] 问题 1：……”“[长文档文本] 问题 2：……”等。

10 个提示词中的文档文本完全相同。通常，模型需要为每个问题重新处理整份文档的 KV 条目，因而把长文档编码 10 次。使用前缀缓存后，只需编码一次；后续每个问题只承担较短问题后缀的计算成本。

具体而言，使用前缀缓存时，第一个查询会计算文档部分的 Transformer 状态；后续查询中的文档内容与 KV 缓存中的前缀匹配，因此模型可以直接处理“问题 1：……”“问题 2：……”等部分。

如果共享前缀远长于各自后缀、缓存全部命中且缓存访问开销较低，前缀缓存的预填充工作量收益可以接近请求数量的线性比例。例如，同一文档上的 10 个独立问题不再分别计算 10 次长文档前缀；但端到端速度通常不会严格达到 10 倍，因为仍需计算各问题后缀、解码输出并承担调度和缓存开销。

前缀缓存的另一个应用是聊天会话，因为在“多轮”会话中，对话历史是每个新轮次的共同前缀。最终用户发送新消息时，早先消息构成一个不断增长的前缀。

经过优化的推理系统会保留上一轮的 KV 缓存，只为新用户消息计算 KV。具体产品是否跨轮复用 KV、复用多久以及采用何种缓存机制属于实现细节，不能仅由界面上的会话状态推断。

> 前缀缓存的收益在交互式场景中最明显，尤其是用户在同一上下文/会话中很快追问时。如果缓存仍有效，第二次回答的 TTFT 可能更短，因为前缀（对话历史）已经缓存。

部分 API 支持有状态会话；服务端可能在许多优化中采用某种前缀缓存，但会话状态与底层 KV 缓存并不等价。对于无状态 API 调用，如果客户端传递完整对话历史，服务端仍可

能识别并复用匹配的 token 前缀，然后只为新的用户消息继续计算；是否命中取决于服务实现、模型、提示词字节/token 一致性和缓存保留策略。

自行实现时，可以对 token 化后的对话历史以及影响 KV 语义的模型/适配器配置生成哈希，得到一致的缓存查找键。不过，必须谨慎管理内存，因为为大量对话存储完整 KV 缓存会迅速消耗 GPU 内存。分页机制可以把不活跃缓存换出到 CPU 内存或磁盘，并在缓存命中时换回 GPU 内存。

还可以对多个请求中的部分提示词去重，某些实现也可能识别单个请求中的复用机会。这里的去重是指合并相同且位置、模型配置等语义兼容的子提示词，使其 Transformer 状态只计算一次。例如，如果两个用户几乎同时发送完全相同的提示词前缀，系统可以共用该前缀的预填充结果。

单个请求包含重复序列时也可能存在复用机会，但标准因果 Transformer 中，相同 token 子串处于不同位置时的状态通常受位置编码和之前上下文影响，不能仅因文本相同就直接共享 KV。在推理输入中出现很长的重复序列并不常见。

> 如果工作负载包含许多相同前缀的重复查询，应为缓存分配更多内存以提高命中率。反之，如果前缀很少复用，就应使用更小的缓存，甚至完全禁用前缀缓存。务必测量生产环境中的前缀命中率，并据此调优。

前缀缓存可以实现为 token 序列 *trie*（发音同“try”）。trie 常称为*前缀树*，是一种树形数据结构：每条边表示一个 token，每个节点编码从根到该节点的 token 序列。实际系统也常使用块哈希或压缩基数树等实现。

在 token 序列 trie 中，每个已观察到的提示词前缀都存储为一条 token 路径，从而支持快速查找共享前缀。新请求到达时，推理引擎从根开始逐 token 遍历，直到下一个 token 无法匹配。匹配 token 序列最终到达完成最长已缓存前缀的节点。图 16-8以系统提示词“You are ChatGPT, a friendly assistant designed to help users…”为例展示这一过程。

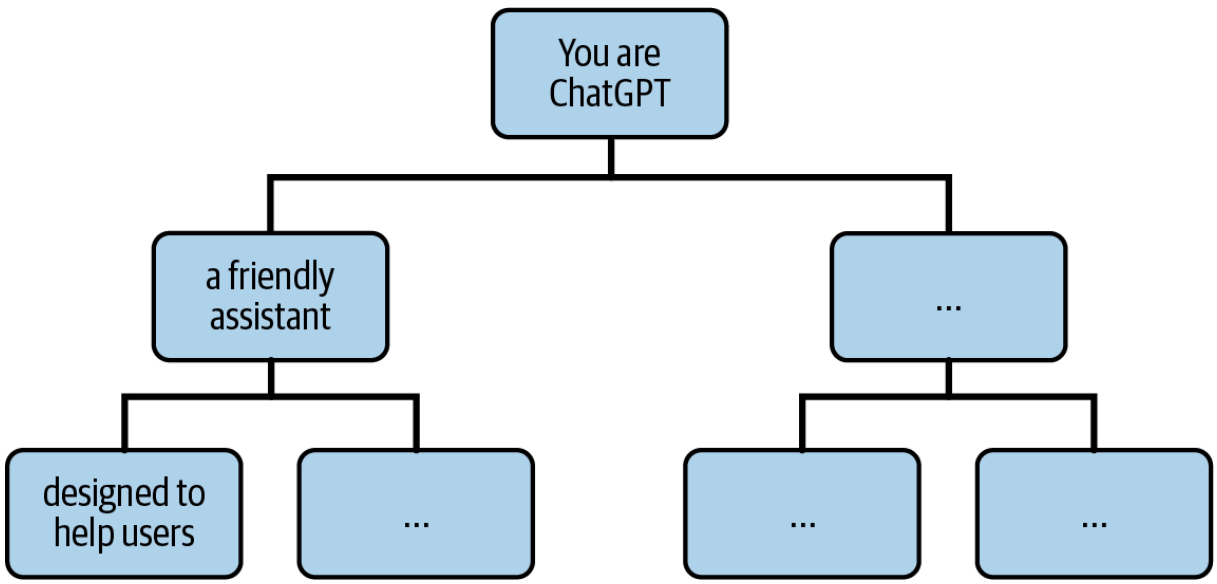


图 16-8　以 trie 数据结构实现的前缀缓存

此时，系统复用共享 KV 缓存数据（例如增加块引用或复制指针），并只为序列中剩余的提示词 token 继续预填充，之后再解码。这样就避免为已缓存前缀重复计算自注意力；缓存的是可供后续 token 使用的 K/V 状态，而非通常意义上完整保存所有注意力分数。

SGLang 的 RadixAttention 使用覆盖完整 token 序列的压缩 trie，即基数树。这种前缀缓存会把 token 序列压缩为树中的单条边，以节省空间。树节点关联相应 KV 缓存块；具体物理布局由实现的分页分配器决定，不应假定每个节点必然对应单个连续 GPU 页。

RadixTree 数据结构节省空间，并支持快速前缀搜索、高效插入和 LRU 风格淘汰。下面的伪代码展示 token 生成期间如何使用基数树查找 KV 缓存：

```
# Simplified RadixAttention KV cache example
# radix_attention_example.py

radix: RadixTree = RadixTree()  # holds edge labels + node.cache pointers

def generate_with_radix(prompt_tokens: List[int]):
    # 1) Find longest cached prefix
    node, prefix_len = radix.longest_prefix(prompt_tokens)
    # shallow-clone the KV cache for that prefix
    model_state = ModelState.from_cache(node.cache)  # refcount bump

    # 2) Process remaining prompt suffix
    for token in prompt_tokens[prefix_len:]:
        model_state = model.forward(token, state=model_state)

    # 3) As we go, insert or split edges in the radix tree
        matched = prompt_tokens[:prefix_len + 1]
        # insert returns the node for this full prefix
        node = radix.insert(matched, cache=model_state.kv_cache)

        prefix_len += 1

    # 4) Now generate new tokens autoregressively
    output_tokens = []
    while not model_state.is_finished():
        token, model_state = model.generate_next(model_state)
        output_tokens.append(token)

        # cache each generated prefix as well
        matched = prompt_tokens + output_tokens
        node = radix.insert(matched, cache=model_state.kv_cache)

    return output_tokens
```

生成开始时，引擎调用 `radix.longest_prefix(prompt_tokens)`，沿基数树中的多 token 边向下遍历，直到到达与提示词最长缓存前缀匹配的最深节点。然后使用 `ModelState.from_cache(node.cache)` 轻量复用该节点的 KV 缓存引用，以初始化 `model_state`，无须为已缓存前缀重新计算自注意力。

接下来，它只逐 token 处理提示词中尚未见过的后缀，并即时更新基数树。对每个新 token，它调用 `radix.insert(...)`，按需拆分已有边或创建新边，再把中间 `model_state.kv_cache` 与新节点关联。

处理完整个提示词后，循环切换到自回归解码阶段，使用 `model.generate_next(model_state)` 生成新 token。类似地，它会把每个生成后的前缀插

入基数树。这种方法在支持增量缓存更新的同时，减少冗余计算、节省 KV 缓存索引空间并快速查找前缀。此处代码是概念性伪代码，不代表 SGLang 的实际 API 或准确物理布局。

SGLang 的 KV 缓存设计能够捕获多轮聊天、少样本示例和分支逻辑等常见复用模式，并尝试以适合 GPU 访问的块组织共享前缀。实际内存访问是否完全合并，仍取决于块布局、调度和内核实现。

决定何时使缓存失效始终是一项挑战。如果其他用途需要缓存内存，就可能必须淘汰部分前缀。缓存系统支持不同策略，例如“最近最少使用”（LRU），优先淘汰最久未使用的前缀。例如，GPU 内存紧张时，SGLang RadixAttention 会惰性淘汰最近最少使用且当前未被引用的基数树叶节点，如图 16-9所示。

这里展示了 SGLang 面向多个到达请求所用的前缀缓存树结构和 LRU 淘汰策略。该示例改编自 LMSys 的一篇 SGLang 博客文章。

图中包含两个聊天会话以及这些会话中的多个查询。每个标签都是一段 token 序列（例如子串）。绿色节点表示树中的新节点，蓝色节点表示当前正在访问的缓存节点，红色节点表示已淘汰节点。各步骤如下：

1. 初始基数树为空。
2. 服务器处理到达的提示词“Hello!”，并返回 LLM 生成的“Hi!”。在这个简单响应中，多个 token 被作为一条压缩边加入树，并连接到一个新的绿色节点。具体来说，系统提示词“You are a helpful assistant”、用户消息“Hello!”和 LLM 响应“Hi!”被串接成相应序列。
3. 服务器收到新提示词，这是多轮对话的下一轮。服务器成功在树中找到提示词前缀并复用其 KV 缓存数据，再把新一轮作为新的绿色节点加入树中。
4. 新的聊天会话开始，步骤 3 中的节点 b 被拆分成两个独立节点，使两个聊天会话可以共享系统提示词。
5. 步骤 4 中的第二个聊天会话继续进行。不过，由于内存有限，步骤 4 的节点 c 必须被淘汰，图中以红色表示；新一轮被追加到步骤 4 的节点 d 之后。
6. 服务器收到一个新的提示词（查询）。处理后，服务器将其插入树中。因为该新提示词与现有提示词/节点不共享前缀，所以需要从根处分出新的分支。
7. 服务器收到包含更多提示词（查询）的批次。这些提示词与步骤 6 的提示词共享前缀，因此系统拆分步骤 6 的节点 e 并共享该前缀。
8. 服务器收到步骤 3 对话（第一个聊天会话）的新消息。此时，它淘汰步骤 5 第二个聊天会话中的节点（例如 g 和 h），因为它们在当时是最近最少使用且可淘汰的节点。
9. 服务器收到一条消息，请求为步骤 8 的节点 j 中的查询生成更多答案。由于内存限制，系统必须淘汰步骤 8 的节点 i、k 和 l。

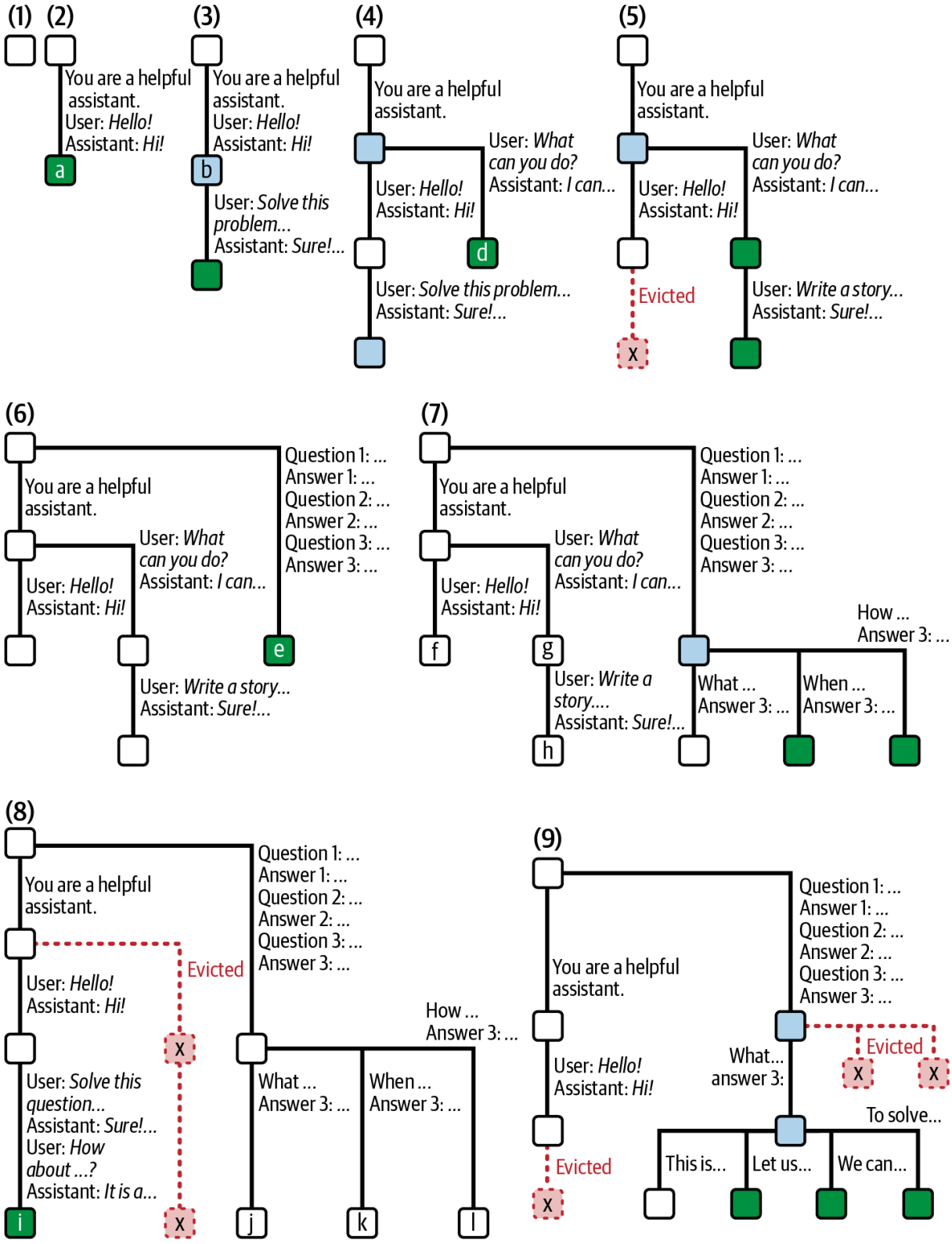


图 16-9　SGLang RadixAttention 前缀缓存树对多个到达请求采用 LRU 淘汰（改编自 LMSys 的 SGLang 博客文章）

该示例展示了多个请求如何共享前缀，也说明了 LRU 缓存淘汰策略在前缀缓存中的工作方式。下面转向级联模型部署模式，以更好地利用 GPU 资源。

模型级联与分层模型部署

并非所有查询都需要最大型先进模型的能力（以及成本）。有些用户请求足够简单，可以由规模小得多、速度更快且成本更低的模型回答。选择合适模型的做法称为*模型级联*或*后备模型路由*。

可以通过分层模型部署实现模型级联，即维护多个规模和能力不同的模型。对每个到达请求，根据查询复杂度、所需质量或当前系统负载动态选择模型。

假设一个问答（QA）推理部署同时包含 7000 亿参数的大型先进模型和 700 亿参数的小型模型。后者需要的 GPU 资源更少，因而托管更快、成本更低。如果用户提出非常直接的事实性问题（由分类器或启发式规则判断），700 亿参数模型可能足以处理。

对于直接的问题，可以把查询路由到较小模型。在本节的假设性示例中，小模型用 50 ms 响应，而 7000 亿参数模型需要 500 ms。如果小模型答案因置信度低而被判定为不合格，系统就把问题路由到较大模型。需要注意，生成式答案的 logits 并不会自动构成校准良好的答案正确性置信度，通常还需专门的验证器、分类器或校准方法。

这种两阶段方法可以降低总体平均延迟和计算用量；不过，如果请求因小模型初次响应置信度不足而先后路由到两个模型，其延迟可能更长。商业 AI 服务的内部路由策略通常未完全公开，不能把对具体产品的推测视为事实。

例如，把 60% 的查询路由到参数规模小 10 倍的模型，可能显著降低推理计算成本；原文所称最高约 5 倍总体成本降低只是条件性示例。真实收益取决于两种模型的硬件效率、输出长度、回退率、质量目标以及路由器本身的开销。

> 维护多个模型和路由系统会带来工程开销，因为不仅要监控一个模型，还要监控其他模型、模型之间的相互作用以及路由系统。只有在流量规模和成本结构值得这样做时，才应采用该方案。

实现模型级联需要查询分类器或启发式机制辅助模型路由决策。要正确解决这个问题相当困难，因为机制往往需要持续适应、全面的启发式规则和不断调优路由器。因此，许多组织仍依赖启发式或基于规则的路由器，并通过离线分析更新决策标准。

一种方法是把它看作类似前文推测解码的推测问题。可以先让较小模型以草稿模式生成答案，并由校准后的评分器或验证器给出置信评分。评分足够高时直接返回响应，否则调用较大模型。

如果最终需要大模型，应确保小模型增加的延迟不会让用户等待过久。实践中，如果怀疑问题很难，可以并行运行小模型和大模型，使两者延迟重叠。这看似浪费且违反直觉，但对于某些延迟敏感场景可能有益，代价是额外计算。

另一种方法是预先为已知类型的用户查询训练独立分类器。例如，可以对复杂度分类，判断“这个问题是否需要大模型的广泛知识或推理能力？”如果不需要，就使用小模型。

需要注意，随着用户查询随时间演化，该分类器本身需要定期重新训练。它还引入了另一个可能退化和失败的组件，因此必须受到监控。应确保分类器轻量、快速且稳健，否则它会成为推理系统的新瓶颈。

初期通常使用简单启发式规则。例如，若用户查询简短且偏事实性，如“今天天气怎样？”或“谁是 X 国总统？”，可以尝试使用较小模型。对于较长提示词（例如超过 50 个 token）或包含*解释*、*分析*、*详述*等词的创意查询，则可以直接发送给大型 LLM，绕过小模型。不过，长度和关键词只是脆弱的示例特征，必须用真实流量评估误路由和质量风险。

应记录并标记每个请求由哪个模型处理（例如“小”或“大”），以及是否发生回退。这样就能按模型细分延迟、质量指标和回退次数。

这些标记数据还能用于监控调度器的决策。统计小模型答案被拒绝的次数，可以识别相关模式，进而重新训练小模型或改进路由器的启发式规则。

反之，如果小模型能很好地处理许多查询，可以据此调整路由阈值，使更多合适请求使用小模型，从而降低响应延迟。阈值方向取决于评分定义：若分数越高表示越可信，提高接受门槛通常会减少而不是增加小模型使用率。

也可以根据容量路由。如果大模型集群达到最大负载，与其让请求排队并增加延迟，不如临时把低关键性请求路由到较小模型。小模型质量可能较低，但仍能快速给出响应。这是高负载下的一种优雅降级。用户可能察觉质量下降，因此应明确满足产品承诺和透明度要求。

在资源受限时，可以按服务等级设计容量优先级；但是否允许把用户路由到不同模型、是否需要披露，应遵守产品契约、隐私和合规要求，不能把未经证实的商业服务行为当作事实或默认最佳实践。

> 第 17～19 章将介绍更高级的动态和自适应推理服务器能力。这些功能高度依赖当前系统负载，包括可用 GPU 内存、内存带宽利用率、KV 缓存利用率等。

另一种模型级联依据内容本身。例如，当请求触发安全策略时，一些系统会把它交给专用安全分类器、策略模型或受控检索系统，以生成符合策略的处理结果。后备组件不应被描述为“专门回答不安全提示词”的模型，因为它仍需执行相同或更严格的安全策略。

这更偏向内容策略而非性能，但仍是模型路由逻辑的重要考量。专用安全组件可能更小、更快，从而减少主模型负担；不过，首要目标应是策略正确性与安全性，而不是性能收益。

从部署角度看，多模型配置需要谨慎扩缩容，因为每个模型都需要足够实例处理分配给自己的流量。例如，大量简单查询到达时，小型 LLM 可能饱和，而大型模型却处于空闲，导致前者成为瓶颈。

可以自动扩缩容，在一天中的特定时段增减小模型实例，甚至使用容器编排按需在大模型池和小模型池之间动态重新分配 GPU；但重新加载模型和预热会产生切换成本。

流式响应

为了改善推理系统的“感知”性能，应在生成输出 token 时流式传输，而不是强迫最终用户等到完整生成结束后才能看到响应。这样，用户可以在信息到达时立即开始阅读和处

理。即使总耗时相同，交互感觉也会快得多。

英文阅读速度常以每分钟约 200～300 个单词作为经验范围，但换算为 token/s 会随语言、分词器和文本类型变化，不能固定为 4～13 token/s。流式速率至少应避免明显落后于目标用户的消费速度；在作出性能优化和容量规划决策时，应以实际语言和客户端体验指标为准。

> 应将系统 token 吞吐量与目标用户的实际阅读/消费速度对照监控。例如，如果模型延迟使系统只能以 2 token/s 输出，交互体验往往会明显受影响，说明还需进一步优化。

vLLM、SGLang 和 NVIDIA Dynamo 等现代推理引擎通常支持流式传输。启用后，服务器通过 WebSocket、服务器发送事件（SSE）、HTTP 流式响应等方式把生成增量刷新给客户端。刷新策略应依据 TTFT、token 间延迟、网络开销和用户消费速度，而不是机械追求固定 token/s。

标准自回归模型在逻辑上逐 token 生成；EAGLE、Medusa 等推测或多 token 解码技术则可能一次接受多个 token。传输层可以把增量组成小批次后刷新给用户，但并不强制必须使用 2～5 token；合适大小取决于延迟目标、协议和网络开销。

刷新前不能在批次中累积过多 token，否则会失去流式传输的意义；另一方面，每个 token 单独形成网络包也可能因开销而低效。框架可以按时间、缓冲字节数、语义边界或组合条件刷新，而不必只依赖换行或句末 token。

例如，假设一个 100 token 的答案需要 5 秒生成，稳定解码速率为 20 token/s，且每 5 个 token 刷新一批；忽略独立 TTFT 和网络开销时，首批 token 约在 0.25 秒后到达（5 token ÷ 20 token/s = 0.25 秒），随后稳定发送各批次直到响应完成。真实首批到达时间还必须加上预填充、排队和网络延迟。

没有流式传输时，用户可能面对空白屏幕等待 5 秒，随后一次性看到整个答案，体验并不理想，还可能引发重复点击等挫败行为。

从性能角度看，流式传输会发送更多小型增量，而不是一个大消息，因此会增加协议、序列化和系统调用开销。相对于模型计算时间，该开销在很多部署中较小，但在高速小模型或大规模扇出场景中不能默认忽略。HTTP/2 和持久连接可避免反复建连，并帮助降低开销。

> 要留意 Nagle 算法与延迟确认的交互，在某些 TCP 栈和配置中可能增加数十毫秒，最坏配置下甚至更高。可以评估 `TCP_NODELAY`，并在系统支持时调整快速确认或延迟确认计时器，以降低增量刷新延迟。代价是小包更多、带宽效率下降；对 HTTP/2、HTTP/3 或代理链路，还需针对实际协议栈测量，而不能仅靠这一 TCP 选项。

必须正确管理流控。例如，客户端因网络问题而缓慢消费流时，不应让网络写入直接阻塞全局模型调度。不过，让引擎无条件生成完整响应也可能浪费计算并造成缓冲膨胀；应采用有界缓冲、背压、暂停/取消或公平调度策略。

系统应把网络发送与主 token 生成循环解耦，例如使用异步 I/O、独立线程或事件循环。CUDA 流管理 GPU 工作，并不直接负责向最终用户发送网络数据；两者应通过线程安全的有界队列衔接。

推理引擎维护有界缓冲区以管理流控，防止客户端停滞或断开时内存无界增长。这些边缘情况在生产环境中相当常见，必须处理。可以从允许累积 50～100 个 token 之类的值开始测试，但限制更适合按字节、时间和租户预算共同定义，因为 token 长度及编码后大小并不固定。

超过一定限制后，推理引擎可以暂停生成、取消请求或关闭连接。这样，如果客户端在生成途中断开，或因连接缓慢而无法跟上，引擎就能停止生成 token，把资源释放给其他请求。

选择缓冲上限需要权衡内存用量与用户体验。短响应很少出现问题，但超长响应和较慢的消费端连接更容易触发。

改善呈现的一种方法是 token 缓冲/节流。如果模型生成 token 的速度快于 UI 希望展示的速度，客户端展示层可以有意平滑输出。例如，模型在 0.5 秒内突发产生 20 个 token 时，一次性显示可能先出现大段文本、随后停顿。

应用 UI 可能希望呈现更稳定的打字机效果。此时可以在客户端渲染层引入例如 50 ms 的显示间隔，避免 token 突发造成卡顿感。服务器不应为此拖慢实际生成或资源释放；否则会增加完成时间和缓冲压力。

可以把显示节奏配置为适应不同 UI 体验。但若按用户等级限制实际模型生成或网络传输速度，应明确遵守服务契约和公平性要求；单纯延迟展示并不会降低已经完成的 GPU 计算负载。

流式响应还允许最终用户在响应完成前评估中间结果并采取行动。例如，如果用户看到答案开头后发现方向不符合预期，可以使用“停止”或“中断”按钮终止生成。

这能节省不必要的计算并改善体验，因为用户无须等到不满意的回答全部生成后才能反馈。应监控或至少测量提前停止事件。

显式停止过多可能表示模型相关性有问题。如果许多用户都在某个位置停止，可能说明模型经常偏题，或超过该长度后过于冗长。这可以提示进一步微调模型使其更简洁，或调整生成并发送给用户的最大 token 数。不过，停止事件也可能源于用户已获得答案、网络切换或 UI 行为，需要结合其他信号分析。

提前停止也可能来自频繁改变主意的用户。无论原因如何，最好允许用户中断 token 生成，使推理集群能够回收资源处理其他请求。

简而言之，流式传输是响应迅速的 LLM 服务的一项重要能力。它不会提高原始模型吞吐量，反而会增加少量开销，但能改善用户感知速度。实现时应谨慎避免干扰生成：网络发送应通过异步 I/O 或独立线程与推理调度解耦，并用有界队列和取消机制避免不稳定的终端网络连接拖住主要 token 生成循环。

建议在测试环境中持续比较启用与禁用流式传输时的端到端延迟，并分别测量 TTFT、token 间延迟和完成时间。确认 token 发出节奏符合目标，且没有引入锁竞争或 I/O 等待等显著瓶颈。对 Python 友好的 Locust 可以模拟客户端，以大规模测试低延迟流式工作负载。

防抖与请求合并

许多生产系统还会实现称为*防抖*和*请求合并*的 UX 功能。系统在执行动作前短暂停顿，通过防抖识别用户是否因误操作或急躁而快速连续发送多个请求，如图 16-10所示。

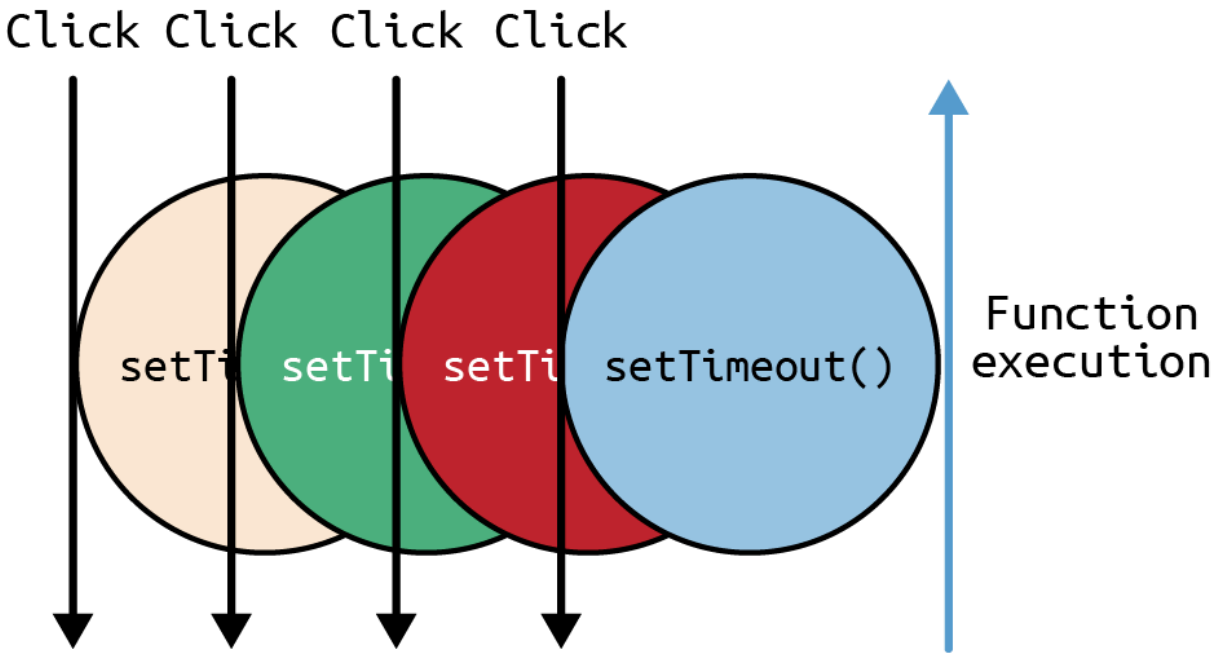


图 16-10　防抖会在执行动作前短暂停顿

此时，系统可以把多个查询合并成一个，或丢弃除最新查询以外的请求。这类应用层防护措施有助于减少后端过量、重复且浪费的负载。

例如，如果用户双击“提交”，或在一秒内发送两个非常相似的查询，系统可以丢弃重复请求。用户感到挫败时可能因应用响应过慢而不耐烦地多次重新提交。

但防抖和请求合并也会增加延迟，可能进一步加剧用户挫败感，导致更多点击。请求在途时，UI 可以禁用重复提交并显示明确的进行中状态，从界面层阻止双击；服务器端仍应使用幂等键、去重窗口或请求取消提供保护。

> 部分网关、批处理器或应用层组件支持防抖/合并窗口。可以从较短窗口（例如 2～5 ms）开始测试，在批处理效率和额外延迟之间寻找平衡。不能仅凭界面感受断言某个具体商业产品采用了这种防抖设置。

输出 token 限制与超时

响应 token 数会直接影响解码时间和成本，因此可以设置输出长度上限或超时。这类应用层约束能够防止失控生成，即模型持续生成，超过合理 token 数。

输出 token 上限和超时有助于保持延迟一致，也能防止恶意或意外提示词造成极长生成、长期占用 GPU 资源。许多公共 API 因此设置严格输出上限；这既是滥用防护，也是性能保护措施。

还应始终设置服务器端超时。例如，可以按产品 SLO 在生成超过一定时间后返回明确标记的部分结果或超时错误。原文的 30 秒只是示例；超时后必须真正取消后端生成并释放资源，不能只关闭客户端响应。

> 还要监控模型是否倾向于冗长离题。适度的 token 上限可能使模型更聚焦并避免超长回答，但 4,096 token 等数值只是示例；过低上限会截断代码、结构化输出或复杂答案，必须按任务评测质量。

必须根据用户需求选择上限和超时。这些限制既能使延迟更可预测，也能限制最坏情况下的计算量，从而帮助容量规划。

总之，输入优化（例如压缩和清洗）、缓存、智能路由和 UX 优化（例如流式传输）相结合，可以减少工作量、降低集群容量需求和成本，并改善用户满意度。这些应用层策略与前面的小节中的底层优化互为补充，形成改善 LLM 服务性能的整体方法。

要点回顾

本章介绍的技术表明，高效 LLM 服务是一项整体工程，需要结合现代 GPU 硬件、新型软件优化和全面监控。性能分析工具用于识别瓶颈，系统化调试技术用于解决问题。以下是本章的一些关键要点：

全面性能分析

在整个推理技术栈中进行端到端性能分析。工程师使用性能分析器测量各阶段的延迟和资源用量，从而准确定位减速和低效环节。这种数据驱动方法能够指导针对性优化，消除瓶颈。

监控与可观测性

为已部署的推理服务实现稳健监控。实时跟踪延迟百分位数、吞吐量、GPU 利用率和内存用量等关键指标，及早发现性能退化或资源饱和。使用日志和追踪观察逐请求处理，在大规模工作负载中识别热点或异常。

调试与迭代调优

采用系统化调试工作流，可以更快解决性能和正确性问题；即使跨越大量节点，也应通过分层遥测和抽样逐步定位。当出现意外变化（例如吞吐量下降或延迟升高）时，便可从高层症状逐层深入到底层问题，而不能假定数万节点环境会天然“容易”调试。

使用指标验证优化

GPU 调试器、内存泄漏检测器和性能相关单元测试等工具，有助于验证量化和内核融合等优化没有引入静默的性能或正确性错误。每次向生产环境发布新优化时，这种迭代式调优—测试循环对于保持高性能和可靠性至关重要。

效率与成本优化

应关注提高推理成本效益的改进。提高有效吞吐量和利用率的优化通常有助于降低单查询成本，但还需计入质量、尾延迟、功耗和工程复杂度。通过分析并改进系统，团队可能使用更少 GPU 服务更多请求，从而节省基础设施和能源成本。

结论

性能分析、调试和全栈系统调优，是维持大规模 LLM 推理高效、可靠且具成本效益的关键。随着部分模型和系统探索万亿级参数，大型生产与研究集群也在持续扩展；原文所称单次部署达到数十万乃至数百万 GPU 并非当前普遍生产现实，应视为远期规模设想。在这种规模下，软件与硬件持续协同设计依然重要。仅靠硬件提高性能已经不够，尤其是在功耗逐渐成为限制因素的情况下。软件与硬件需要在技术栈每一层协同设计并持续共同调优。

经过优化的推理基础设施可为最终用户提供快速响应、可预测且稳定的系统行为以及较低运营成本。具备这些条件，才能可靠地交付服务大型先进模型的推理系统。

接下来的章节将深入理解并优化现代 LLM 推理系统中计算、内存和网络最密集的部分：计算（预填充）KV 缓存，在集群中的相关工作进程之间共享缓存，以及使用缓存生成（解码）新 token。

第 17 章　扩展用于推理的预填充—解码解耦架构

如前文所述，LLM 推理可分为两个不同阶段：预填充阶段和解码阶段。预填充阶段处理输入提示词，为该提示词生成模型内部的键值（KV）缓存；解码阶段则利用这些缓存值逐个生成输出词元——采用推测解码时，也可能一次生成少量词元。

这两个阶段的性能特征有本质差异。预填充阶段受计算能力制约，需要对可能多达数千个词元并行执行大量矩阵乘法，消耗大量浮点运算资源。相比之下，解码阶段通常受内存 I/O 制约：每生成一个词元都要读取庞大的 KV 缓存并写入新值，因而对内存带宽形成压力。简而言之，预填充是高吞吐的并行工作负载，而解码是顺序执行且对延迟敏感的工作负载。

早期 LLM 服务系统把这两个阶段视为在同一套硬件上运行的单体流水线，因此通常通过请求批处理优先追求吞吐量，实际上更偏向预填充阶段。然而，随着交互式应用日益普及，首词元时间（TTFT，即整个提示词的预填充延迟）和每输出词元时间（TPOT，即每个词元的解码延迟）等延迟指标，变得与原始吞吐量同等重要。单个基于 GPU 的推理引擎同时承载两个阶段时，很难同时优化 TTFT 和 TPOT。

把大量请求组成批次可以提高吞吐量，却会恶化 TTFT，因为同一批次中的请求会受到最慢预填充任务的影响；它也会影响 TPOT，因为解码步骤可能排在新提示词的预填充任务之后等待执行。

单体推理系统不得不在两者之间取舍：要么缩短首词元时间，却让后续词元生成变慢；要么提高逐词元吞吐量，却让新请求承受较高的初始延迟。在极端情况下，一个很长的提示词可能长时间占用 GPU，阻塞其他用户的所有提示词预填充任务；而一旦进入解码阶段，逐词元处理又可能无法充分占用 GPU 的计算核心。

为解决这些问题，研究人员和工程师开始探索将两个阶段解耦的方法。关键洞见在于：预填充和解码其实不必运行在同一硬件上，甚至不必使用同一种硬件。

将预填充与解码阶段解耦，是指把它们分配给不同资源，并针对各阶段的需求分别优化。DistServe 论文系统化展示了这种思路：通过消除两个阶段之间的干扰，可以同时满足严格的 TTFT 和 TPOT 延迟要求。

在 DistServe 的特定评测配置中，与未进行预填充—解码解耦的先进基线相比，该系统在严格的延迟服务级别目标（SLO）约束下最多可服务 7.4 倍的请求。此后，一些业界框架开始试验彼此分离的预填充服务器和解码服务器。

开源 vLLM 可结合 LMCache 等组件实现解耦运行。NVIDIA Dynamo 则实现了带动态路由和自动扩缩容的预填充—解码解耦，并公开了部分运行细节。许多服务提供商和开源框架都在实现或评估这种架构。据公开报道，一些大规模服务系统也采用了类似思路来满足严格的延迟 SLO；不过，OpenAI、Meta 和 xAI 等公司的完整生产架构并未公开，具体实现不宜据此断言。总体而言，预填充—解码解耦已成为大规模 LLM 推理的重要架构选择。

在超大规模场景中，大型推理部署可能涉及数十万乃至更多 GPU，并服务数十亿次请求。在这类环境里，解耦架构潜在的成本和性能收益十分可观。

拆分工作负载后，可以分别优化每个阶段，避免一个阶段成为另一个阶段的瓶颈。本章余下内容将探讨如何设计和运行极大规模的预填充—解码解耦推理系统。

本章将介绍在预填充与解码工作节点之间路由请求的调度算法、重负载下维持服务质量（QoS）的技术，以及提高这种分离架构效率的机制。讨论范围涵盖高速互连、专用解码内核，以及为两个阶段采用不同 GPU 类型的异构硬件策略。

为什么要将预填充与解码解耦？

现代交互式 LLM 服务通常将 p99（99% 的请求）的 TTFT 目标设为低于 200–300 毫秒。如果不分离预填充工作，这一目标几乎无法保证，因为“一刀切”的 LLM 服务方式会浪费大量潜在性能。

作为参考，MLPerf v5.0（2025）的推理基准测试针对 Llama2 70B（700 亿参数）设定的 p99（第 99 百分位）SLO 目标约为 450 毫秒 TTFT 和 40 毫秒 TPOT；针对 Llama 3.1 405B（4050 亿参数），目标约为 6 秒 TTFT 和 175 毫秒 TPOT。具体而言，这些 SLO 分别是 Llama 2 Chat 70B 和 Llama 3.1 405B Instruct 的 p99 TTFT、p99 TPOT 目标。

设想这样一种情况：一位用户的请求包含数千个词元的超长提示词，而另一位用户的请求只有很短的提示词。如果没有预填充—解码解耦，这两个请求几乎同时到达时，长提示词的预填充计算会长时间占用 GPU。

没有解耦时，第二个请求必须等待很久才能开始解码。这被称为*干扰*：一个请求的预填充工作拖延了另一个请求的解码工作。图 17-1在连续批处理的背景下展示了预填充与解码之间的这种干扰。

在简单的 FIFO 调度策略下，长提示词会放大所有请求的尾延迟。通常，排在队列前端的长提示词或计算密集型预填充任务，会阻塞后面更短、更轻量的请求。这称为*队头阻塞*，会导致资源利用率低、延迟异常值增多，并损害用户体验。

在灵活的解耦架构中，可以把大型提示词的预填充任务发送到专用的计算优化型预填充工作节点池；轻量提示词则可以直接发送给解码工作节点，从而绕过预填充节点。这种灵活性使短请求不必遭受队头阻塞，既能最大化总体吞吐量，也能减小延迟尾部效应。

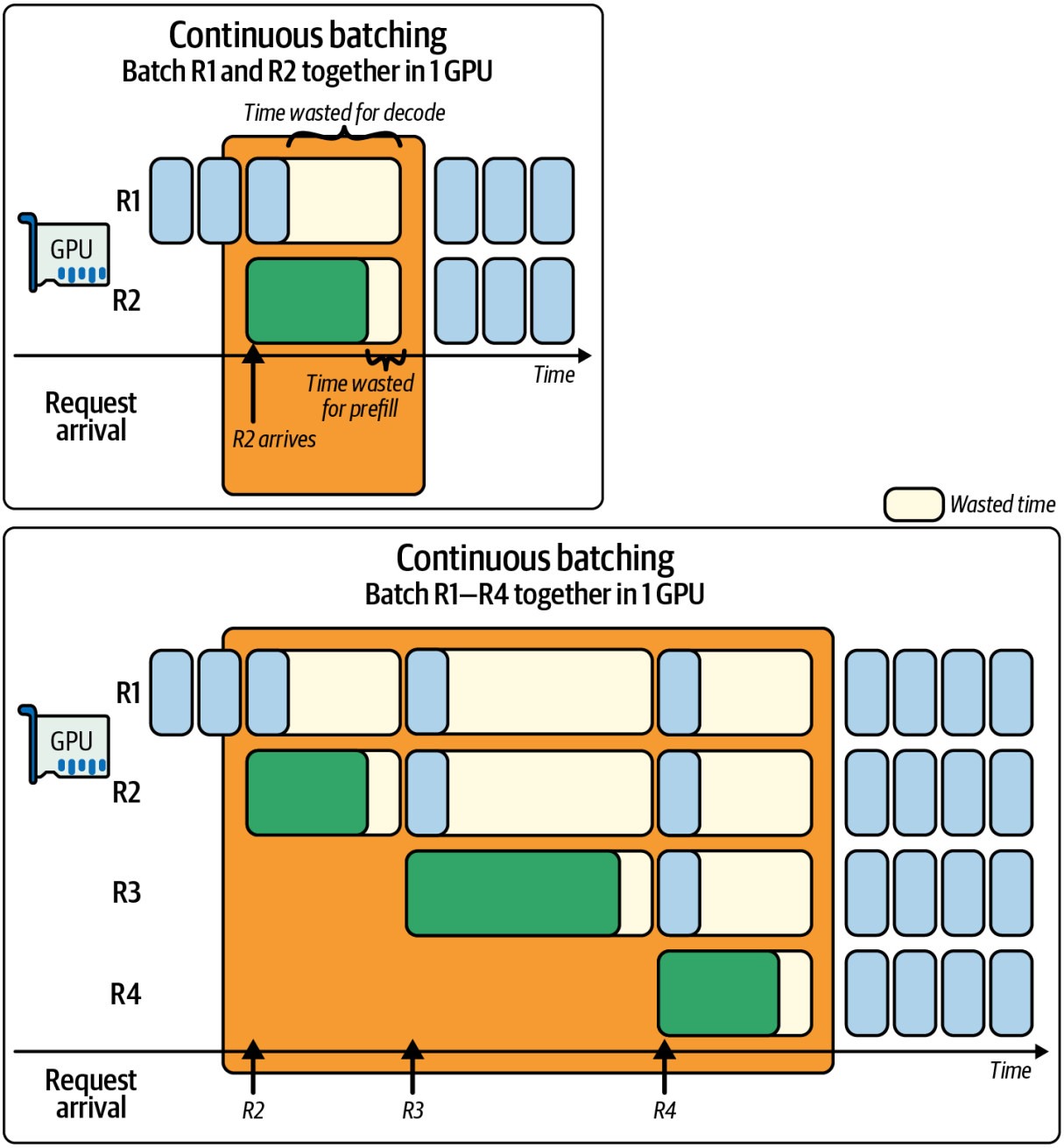


图 17-1. 在同一 GPU 上运行的共置预填充与解码产生的干扰（来源：https://oreil.ly/GRkHs）

解耦的优势

解耦有两个主要优势：减少相互干扰，以及针对阶段分别进行优化。下面分别讨论。

减少干扰

采用解耦后，预填充任务不再与解码任务争用同一设备。忙于生成大量词元的解码工作节点，不会阻止另一位用户的提示词被处理，反之亦然。

为每个阶段分配专用资源，意味着长提示词的计算不会阻塞另一位用户的词元生成，实际效果是延迟更加可预测。图 17-2比较了预填充和解码共置与解耦两种方式。DistServe 论文及其作者后续发布的博客文章对该实验有更详细的说明。

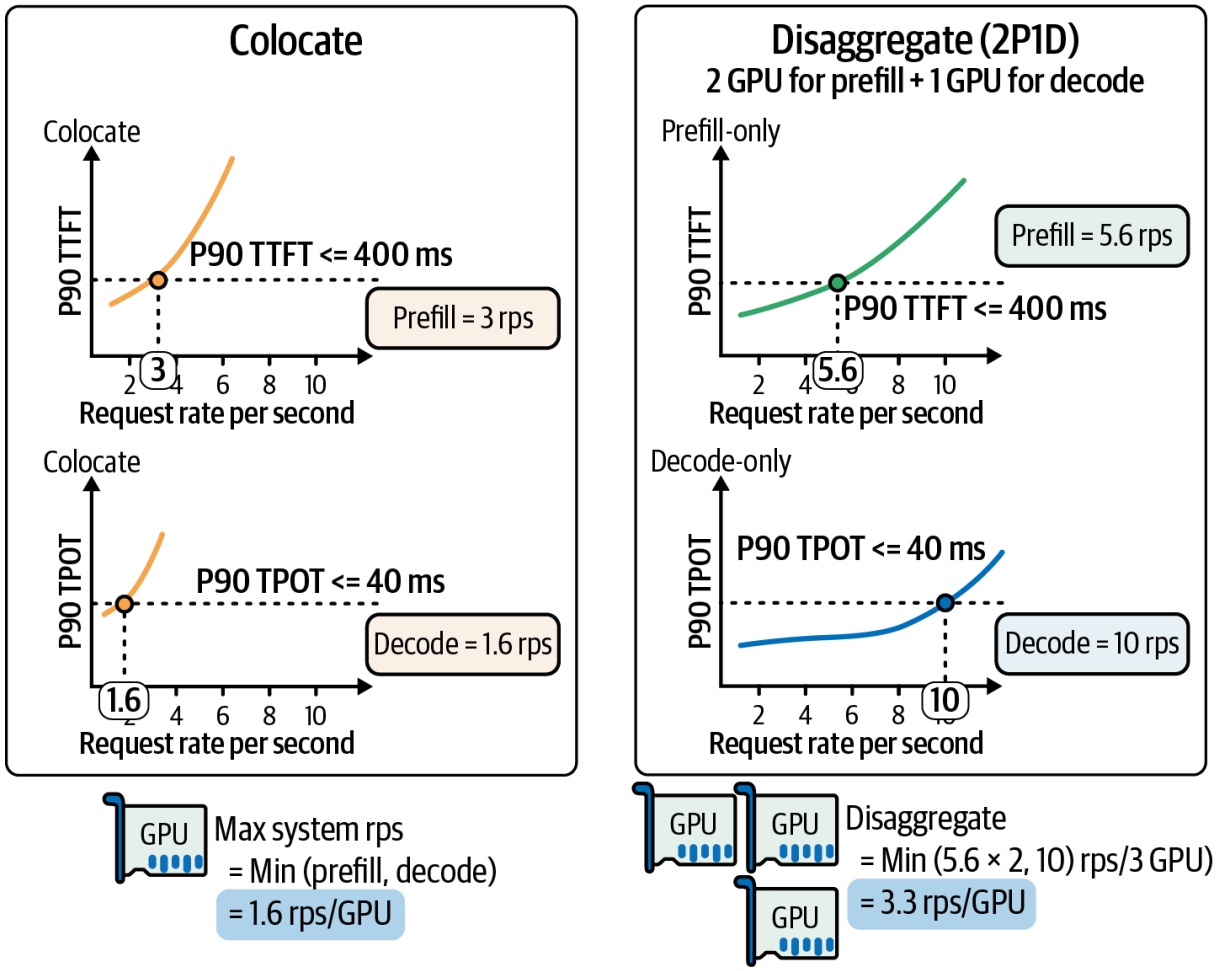


图 17-2. 共置与解耦的预填充和解码比较（来源：https://oreil.ly/GRkHs）

在该实验中，SLO 设为 P90 TTFT 0.4 秒、P90 TPOT 0.04 秒（例如图 17-2中的水平线）。在给定的 TTFT 延迟边界内，共置系统只能达到约每秒 3 个请求（RPS）的有效吞吐量；在给定的 TPOT 延迟边界内，则只能维持 1.6 RPS。因此，共置配置的有效吞吐量只有 1.6 RPS，因为必须同时满足 TTFT 和 TPOT 两项延迟 SLO。

将两个阶段解耦，并分配两个预填充工作节点（两块 GPU）和一个解码工作节点（一块 GPU）后，形成所谓的*2P1D 配置*。与单 GPU 共置配置相比，预填充和解码工作节点的总体 RPS 都更高：预填充工作节点约达到 5.6 RPS，解码工作节点约达到 10 RPS，分布在三块 GPU 上。因此，2P1D 配置的有效吞吐量为每 GPU 3.3 RPS。

每 GPU 3.3 RPS 的计算方式，是取预填充工作节点总 RPS（5.6 RPS × 2 = 11.2 RPS）与解码工作节点 RPS（10 RPS）中的较小值，即三块 GPU 合计 10 RPS。再除以 GPU 数量 3，得到在所配置 SLO 下的有效吞吐量：10 RPS ÷ 3 GPU = 每 GPU 3.3 RPS。

> 在这组比较中，解码侧的改进主要影响逐词元延迟，而预填充隔离主要改善首词元时间。只有同时满足两项 SLO，才能计入有效吞吐量。

这种隔离还可以改善尾延迟。经验上，解耦系统的延迟分布更集中，能够避免单体系统中常见的长尾。消除跨阶段干扰后，每个阶段都能更可靠地满足自身 SLO，延迟也更加稳定

可预测。

此时还应该问一句：“3 倍成本是否值得换取 2 倍改进？”提出这个问题是合理的。要让方案更具成本效益，还需要进一步调优；不过，这组结果说明了解耦有助于收紧延迟分布并提高有效吞吐量。最终仍需根据具体工作负载作出决定。对于需要达到特定有效吞吐量 RPS 的场景，解耦是常见选项。

阶段特定的优化

阶段特定的优化让每个阶段都能使用最适合自身的硬件和并行策略。例如，预填充阶段受计算能力制约，通常会在高 FLOPS GPU 上提高张量并行度，以发挥峰值 FLOPS。此外，现代 GPU 提供 FP8 和 FP4 等低精度模式，可提升计算密集型预填充阶段的吞吐量。

> 对于权重，以及经过验证的激活值，也应优先考虑 FP4。许多软件栈采用 FP4 权重和 FP8 激活值。这些低精度格式可以在精度损失很小的情况下最大化吞吐量、减少 HBM 占用；NVIDIA Tensor Core 和 Transformer Engine 等现代软硬件栈均支持这些精度。

相比之下，解码阶段受内存带宽制约，并且会承受跨 GPU 同步开销。因此，解码通常在很少甚至不使用张量并行（常见为 `TP=1`）时效率最高，同时依赖融合内核提高算术强度，并采用内存吞吐量高的 GPU。

在单体架构中，两个阶段必须共用一种 GPU 类型和一种并行策略，至少会使其中一个阶段处于次优状态。而解耦允许分别调优两个阶段，以获得最大效率。

拆分阶段还为异构集群打开了空间：可以为预填充和解码分配不同 GPU 类型，以优化成本与性能。例如，为提示词预填充使用计算优化型 GPU、为词元生成使用内存优化型 GPU，单位成本吞吐量可能优于同构部署。

> 实践中，最新一代 GPU 往往同时拥有更高 FLOPS 和更大的显存，因此直接在预填充和解码两侧都使用最新 GPU 很有吸引力，也更为常见。但要注意，异构配置同样是降低成本的可行选择。

本章稍后将进一步讨论异构集群，并展示在大规模场景中为提示词使用高端 GPU、为生成使用更便宜的 GPU 如何节省大量成本。

总之，解耦可以消除跨阶段干扰，并支持分别处理每个阶段。预期结果包括：由于提示词长度不匹配不再造成长尾，延迟分布更加集中；延迟约束下的吞吐量（有效吞吐量）提高；总体资源利用率改善。

> 应使用性能分析工具（例如 NVIDIA Nsight Systems）识别预填充和解码阶段的瓶颈。这些工具可以跟踪不同工作节点上的 GPU 内核和 RDMA 传输，有助于验证解码内核是否与通信充分重叠。

下面讨论如何实际实现解耦服务系统，包括系统架构、通信机制和调度策略，以充分利用解耦集群。

解耦的预填充与解码集群池

在解耦部署中，我们维护两个（或更多）工作节点池：一组 GPU 专门处理提示词预填充，另一组专门生成词元。这些工作节点可以位于数据中心内不同的节点或机架上；如果互连速度足够快，也可以位于不同数据中心。（注：为了达到现实可行的 SLO，将预填充和解码保留在同一数据中心内通常更实际。）

工作节点池通过网络通信，把预填充生成的模型 KV 缓存交给负责解码的 GPU。调度器或路由器负责协调这一通信过程。

考虑这样一种配置：模型权重分别加载在两组 GPU 服务器上。一组是预填充工作节点，负责处理提示词并计算 KV 缓存；另一组是解码工作节点，利用预填充工作节点生成的 KV 缓存完成词元生成。

两组工作节点通常通过高速互连（例如 NVLink/NVSwitch 和 InfiniBand）以及基于 RDMA 的 GPU 到 GPU 零拷贝传输进行通信。实际部署可能使用 GPUDirect RDMA 或 UCX；验证端到端行为时，可以在 Nsight Systems 中将这些传输与 CUDA 内核、NVLink 活动、存储指标及 InfiniBand 交换机指标关联起来。

> 对于基于超级芯片的 NVL 互连网络（例如 Grace-Blackwell、Vera-Rubin 等），可使用 NVIDIA Multi-Node NVLink（MNNVL）；为张量并行解码启用优先使用 NVLink 的集合通信，并在条件允许时为 AllGather 和 ReduceScatter 集合通信启用 SHARP。

系统收到新请求时，通常先由解码工作节点接收。这称为*以解码为中心的设计*。之所以偏好这种方式，是因为预填充工作节点已经受 KV 计算的算力限制。

让解码工作节点负责客户端 I/O、路由和会话状态管理，可以避免预填充工作节点过载。同时，将请求入口集中到解码节点还能简化网络管理、自动扩缩容和策略执行。

> 这只是预填充—解码系统架构的一种形式：由解码工作节点作为所有请求的集中入口。NVIDIA Dynamo 推理系统采用了这种架构。另一种常见架构是使用专用的集中式 API 路由器，把请求路由到预填充或解码工作节点。但这会在系统中增加一个组件，还需要路由器与预填充、解码工作节点之间进行额外协调，并带来扩缩容和延迟方面的额外考量。

解码工作节点收到请求后，会决定由自己执行预填充，还是将其“卸载”到预填充工作节点池。如果选择卸载，解码工作节点稍后会接收返回的 KV 结果，然后继续解码并生成下一个词元。

下面是一段简化的 NVIDIA Dynamo 集群配置片段。它定义了两个角色，并使用 NVIDIA Inference Xfer Library（NIXL）执行基于 GPUDirect RDMA 的 KV 缓存传输，使一块 GPU 能通过网络直接写入另一块 GPU 的内存：

```
roles:
  - name: prefill_worker     # Prefill worker role
    model_path: models/llm-70b
    instance_count: 4        # 4 prefill workers
    gpu_type: B200           # B200 Blackwell compute-bound prefill

  - name: decode_worker      # Decode worker role
    model_path: models/llm-70b
    instance_count: 8        # 8 decode workers
    gpu_type: B300           # B300 Blackwell Ultra high-memory decode
```

NVIDIA 的 Rubin CPX 加速器也是预填充工作节点的一种选择。Rubin CPX 中的 CP 代表“上下文处理”（context processing），专为预填充这类受计算能力制约的工作负载设计。Rubin CPX 表明 NVIDIA 正从“通用加速计算” GPU 走向专用芯片：针对推理等更广泛 AI 工作负载中的特定阶段（例如预填充）进行优化。

在此配置中，四个预填充工作节点使用 B200 GPU（为计算密集型预填充提供足够算力），八个解码工作节点使用 B300 GPU（为受内存限制的解码提供较大 HBM 容量）。混用 B200 和 B300，有助于匹配两者的 FLOPS 与 HBM 容量特征，同时控制成本。两个角色都使用 NIXL 和 GPUDirect RDMA 传输 KV 缓存块。NIXL 抽象了通过 NVLink 和 RDMA 网卡执行 GPU 到 GPU 数据移动时的传输层；它还提供 GPUDirect Storage 连接器，使 KV 缓存页能够从不同存储层读取或写入。

在底层运行时，每个解码工作节点都会注册一段 GPU 内存，使预填充工作节点可以通过 RDMA 直接写入其中。通常，NIXL 描述符等内存注册元数据会在启动时或首次建立联系时交换。这样，对于每个远程预填充任务，只需发送一个较小的标识符，而不必发送完整的内存地址结构。

例如，Dynamo 使用 etcd 完成工作节点发现和租约管理。工作节点把必要的内存句柄注册到路由器或控制平面，使对等节点能够在需要时获取描述符；预填充工作节点会在首次使用时检索这些描述符。这样，预填充请求只需包含目标 KV 缓冲区的 ID，从而使控制消息保持轻量。

此外，NVIDIA Dynamo 的 NIXL 实现为推理数据移动提供高吞吐 RDMA 和存储抽象，并包含 NVLink、基于 UCX 的互连网络以及 GPUDirect Storage 插件。因此，预填充工作节点可以直接把 KV 块写入解码 GPU 的内存。

在预填充和解码采用不同张量并行布局的混合并行部署中，需要在 NIXL 读取完成后立即在解码侧执行布局转换，使 KV 页符合解码内核预期的布局。与网络传输相比，这一转换的延迟通常可以忽略，并且能够避免重新预填充。

这种架构将两个阶段的扩缩容解耦。例如，如果大量并发长提示词使预填充成为吞吐量瓶颈，就可以增加预填充工作节点，提高提示词处理能力。

如果许多用户生成很长的输出导致解码成为瓶颈，则可以扩展解码工作节点。由于解码和预填充彼此分离，扩展其中一侧不会直接干扰另一侧。

NVIDIA Dynamo 等系统支持运行时动态配置解耦，因此无需停止集群即可随时增删预填充工作节点。新加入的预填充工作节点只需注册，然后开始从队列拉取任务。如果某个预填充工作节点因崩溃、重启、自动扩缩容事件或网络分区等原因离开集群，解码工作节点会暂时执行更多本地预填充来补偿。

NVIDIA Dynamo 的分布式运行时使用 `etcd` 进行工作节点发现和租约管理。其 Planner 组件可以通过撤销租约或启动能够自动发现的新工作节点来扩缩容。在负载经常波动的超大规模环境中，这种动态灵活性至关重要；发生变化时，可以按需在两种角色之间调换工作节点。

预填充工作节点的设计

预填充工作节点（也称为*提示词服务器*）是专门执行请求初始提示词预填充阶段的计算节点。本节讨论如何设计预填充节点以高效处理繁重计算，以及如何在负载下为 KV 缓存填充平衡延迟与吞吐量。

由于预填充工作负载计算密集，预填充节点应使用 FLOPS 较高、适合大型矩阵乘法的 GPU。每个预填充任务都会让 n 个输入词元通过模型的所有层。

预填充工作节点会并行使用数千个 GPU 线程；如果条件允许，还会跨多个 GPU 节点执行。它们使用张量并行、流水线并行等常见并行技术来缩短 TTFT。

内存管理

从内存角度看，预填充节点需要加载完整模型权重，并为提示词分配 KV 缓存。随后，这些 KV 缓存会被传输到解码工作节点，稍后将看到具体过程。

预填充会把模型参数以及模型对提示词输入执行前向传播时产生的工作激活值装入 GPU 内存。KV 缓存创建后会立即发送给解码工作节点，不会长时间驻留在预填充节点的内存中。

如果模型极大或提示词极长，受内存限制，预填充可能需要在多个 GPU 之间进行张量或流水线切分。为满足延迟目标，预填充服务器应灵活采用数据并行、张量并行、流水线并行、专家并行（MoE）和上下文并行等策略。

一些推理框架会预先分配一大块 GPU 内存，作为预填充的工作空间，从而减少整体内存碎片和缓冲区分配时间。

延迟与吞吐量的权衡优化

调优解耦的预填充集群时，必须面对一个基本取舍：在重负载下，是尽量缩短每个提示词的 TTFT，还是最大化总体每秒请求数（RPS），也就是降低 TPOT。

解耦系统通过支持不同调度策略来处理这一取舍，分别对应“延迟优先”和“吞吐量优先”两种方式。下面分别介绍。

延迟优先方法

> 为了降低 TTFT，预填充节点应在提示词到达后立即处理，尽量少批处理或不批处理。这样无需等待其他请求凑满一个批次；在集群有可用 GPU 的前提下，每个提示词都能立即开始执行并尽快完成。
>
> 延迟优先的缺点是 GPU 利用率较低，因为批次很小甚至没有批次，GPU 可能经常空闲；在集群规模固定时，系统能同时服务的请求也更少。此时可以超额配置预填充集群

容量，或使用为 1 的极小批次，以保证请求满足严格的延迟 SLO。

吞吐量优先方法

如果首要目标是峰值吞吐量（RPS）和最低 TPOT，就应将提示词组成更大的批次，以充分利用每个 GPU。把 8–32 个提示词累积为一个批次，可以提高算术强度并让 GPU 计算单元保持忙碌，从而提高总体吞吐量。

吞吐量优先的缺点是，每个请求都会产生一段批处理等待时间，至少等于收集完整批次所需的时间；批次越大，等待越久。

在追求极端吞吐量的推理系统配置中，还可以通过数据并行或流水线并行，为每个请求分配多个 GPU。

采用数据并行时，每块 GPU 上都会复制完整模型，批次则被拆成多个微批次分配给各 GPU。每块 GPU 使用自己的完整模型副本，对分得的数据子集执行前向传播，最后汇总所有 GPU 的结果得到最终输出。

数据并行汇聚所有 GPU 的内存带宽和算力，以提高每批次性能。但最大并发请求数会降为“GPU 总数 ÷ 每请求所用 GPU 数”，从而减少系统的总体并发容量。如果单个请求使用过多 GPU，还可能导致部分资源闲置，使吞吐量与并发性失衡。

流水线并行把模型层划分为位于不同 GPU 上的顺序阶段，例如 GPU 0 和 GPU 1。GPU 0 完成微批次 0 对应的阶段后，会立即把激活值转发给 GPU 1，同时开始处理微批次 1 的第一个阶段。这种装配线式模式让各 GPU 同时忙于不同工作片段。

流水线并行可以提高每批次吞吐量，但会增加 GPU 间通信开销；如果微批次大小或阶段划分没有精心平衡，还会产生流水线“气泡”。

归根结底，在集群规模固定时，每增加一块专用于单个请求的 GPU，都会提高该请求的吞吐量，却减少系统同时能够处理的请求数量。虽然可以横向扩展 GPU 集群，但在规模固定的前提下，应根据用例更看重延迟 SLO 还是吞吐量 SLO 来选择配置。

延迟感知调度与批处理

解耦系统会采用前文所述的延迟感知调度策略来平衡这些因素。例如，在负载足够高、合并少量请求又不会违反 TTFT 目标之前，系统可能始终保证单请求执行而不进行批处理。

许多集群设计会在调度器中加入 SLO 约束。例如，如果 p90 TTFT 必须 $\leqslant X$ 毫秒，系统就会选择在典型提示词长度下仍能满足该 SLO 的最大批次或并行策略。

另一种策略是自适应批处理窗口。例如，低负载时使用批次大小 1 立即运行请求；负载较高时，允许把 2–10 毫秒等短窗口内到达的请求组成微批次。这样，仅在确有必要且可以容忍时付出少量延迟，换取显著的 GPU 利用率提升。

许多推理引擎在预填充工作节点上偏向延迟优化。系统往往尽快执行提示词任务，甚至容忍一定程度的 GPU 利用不足，因为更快返回首个词元能显著改善用户体验。

常见做法是按照高于平均负载需求的水平配置预填充容量，使预填充集群能够吸收突发提示词流量而不产生延迟尖峰。下一章将讨论动态重新平衡资源的自适应机制，使预填充和解码工作节点都不会逐渐成为瓶颈。

Kubernetes 等现代编排器可以自动扩缩每个层级。例如，如果预填充 GPU 利用率持续偏高而解码侧较低，编排器可以触发自动扩缩容，增加预填充 Pod（或节点），甚至可能移除部分解码 Pod/节点。

> 这类自适应扩缩容通常借助预填充队列长度等指标来驱动决策。

另一种做法是实现优先级队列：短提示词进入批处理较少的独立快速通道，较长且适合批处理的提示词则进入吞吐量优化队列。NVIDIA Dynamo 在调度中支持延迟类别；也可以通过为请求打标签，并为不同类别设置不同批处理窗口来实现类似机制。

关键在于，预填充工作节点优先追求快速周转。由于解码运行在另一组工作节点上，解耦使我们能够这样做而不损害解码性能。低流量期间可能会“浪费”一些预填充 GPU 周期，却能在流量高峰保持较低 TTFT；对于交互式服务而言，这通常是值得的取舍。

解码工作节点的设计

解码工作节点（也称为*生成服务器*）专门负责自回归解码阶段。提示词的 KV 缓存准备好后，会被发送到解码工作节点；该节点利用 KV 缓存尽快生成剩余输出词元，以维持较低的 TPOT 延迟。

如果请求最初被路由到解码工作节点（如图 17-3所示），它必须先通过解耦路由器决定在本地还是远程执行预填充。如果选择远程预填充，就把预填充请求放入队列，等待预填充工作节点取走。

预填充工作节点持续从预填充队列拉取任务，读取前缀缓存中已有的 KV 块，并执行预填充计算。随后，它把 KV 块写回解码工作节点，由解码工作节点完成解码。

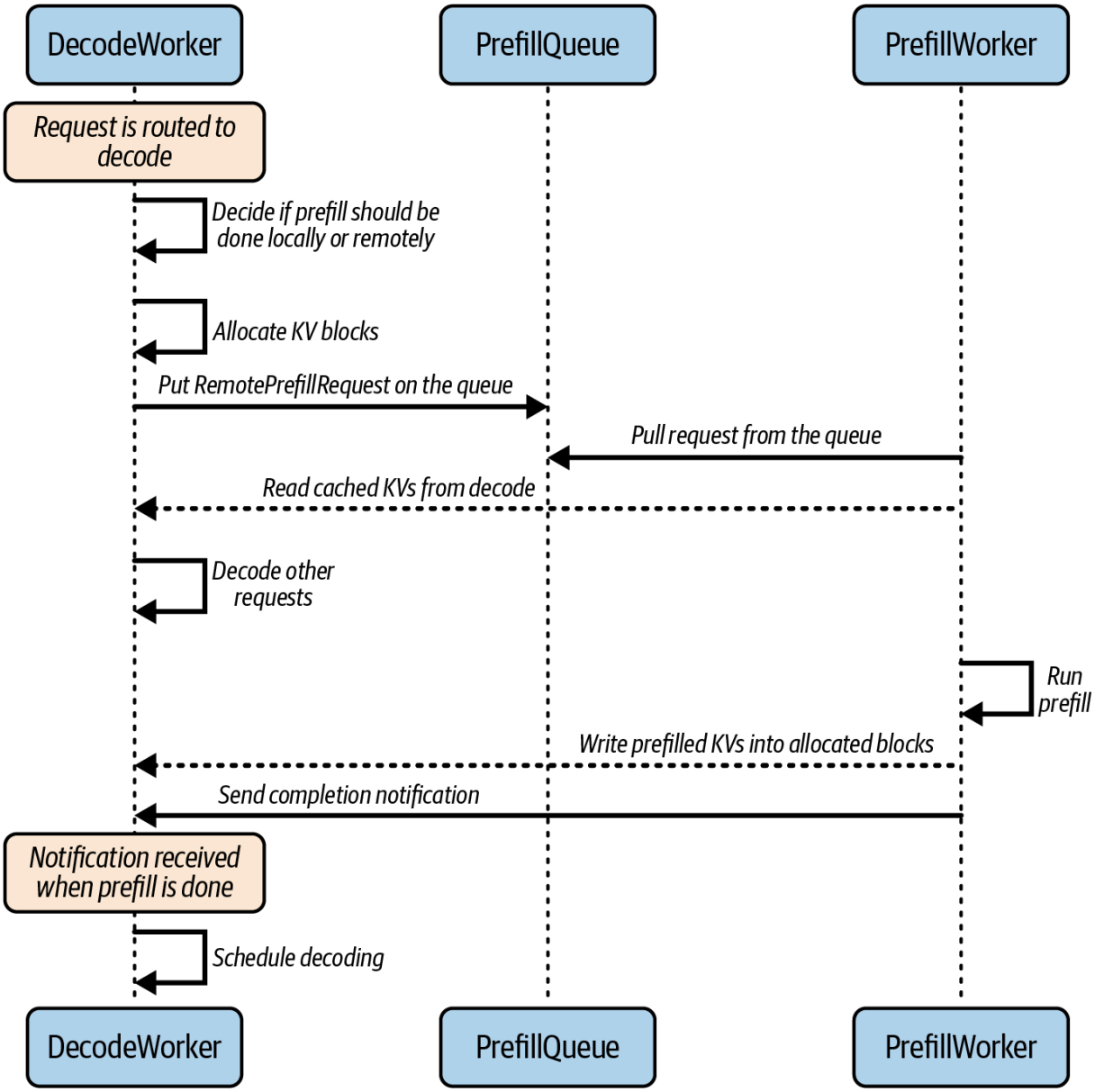


图 17-3. 预填充工作节点设计：读取前缀缓存 → 计算预填充 → 为解码写入 KV 缓存

解码工作节点的设计重点，是高效处理大量并发序列生成，同时管理 KV 缓存的内存占用。本节介绍解码服务器如何利用连续批处理和精细的内存管理技术实现高吞吐。这些技术有助于降低 TPOT 延迟并提高可扩展性，尤其适用于长序列。先从两个阶段之间的 KV 缓存传输开始。

预填充与解码之间的 KV 缓存传输

高性能解耦要求在预填充与解码工作节点之间尽可能高效地移动 KV 缓存数据。使用 NIXL 等库（见第 4 章）进行直接 GPU 到 GPU 传输，可以避免 CPU 参与并使用非阻塞操作。这样，一块 GPU 传输 KV 数据时，无需等待传输完成，也能服务其他前向传播请求。

考虑一个到达解码工作节点的用户请求。此时，解码工作节点的调度器会分配所需的 KV 块，并把远程预填充请求加入预填充队列。该请求包含这些 KV 块的标识符。图 17-4展示了这一交互过程。

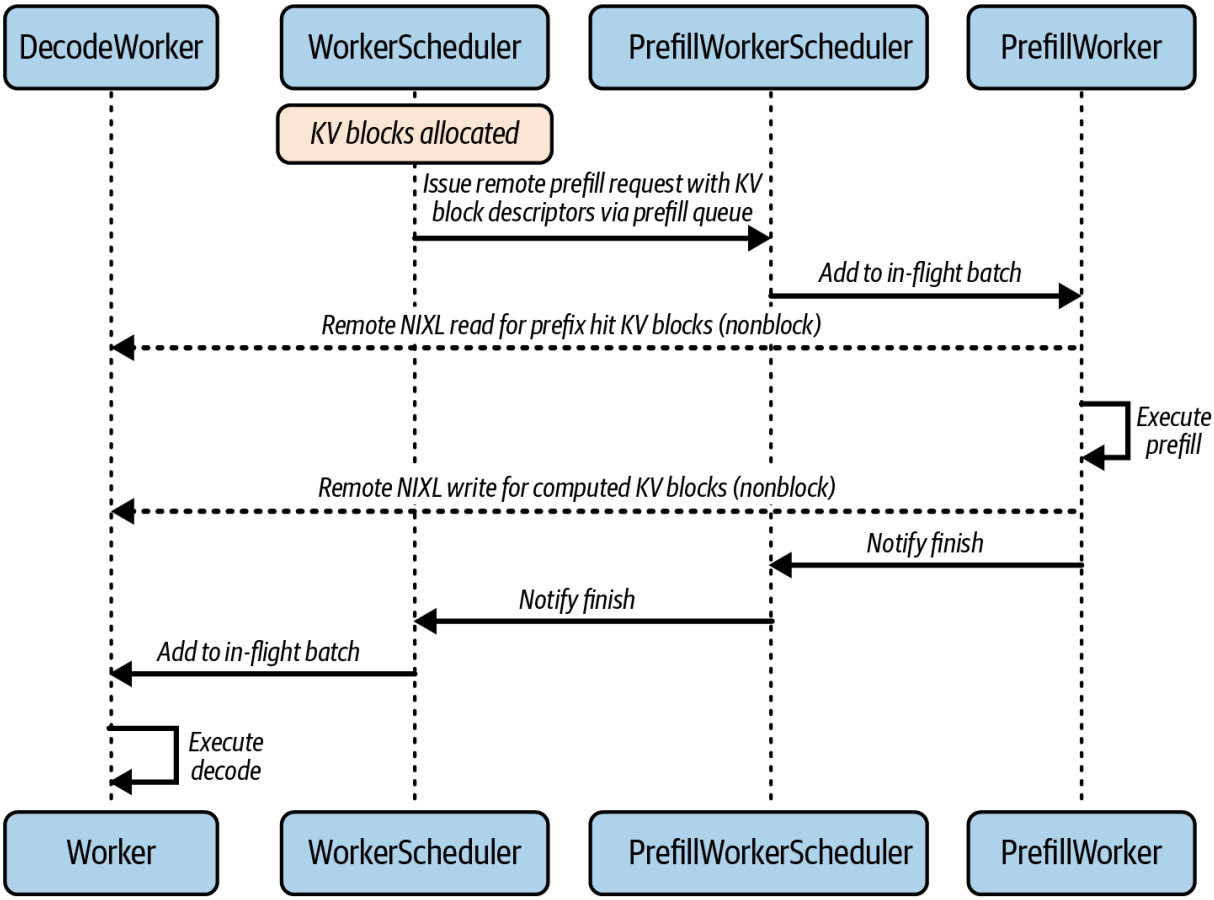


图 17-4. 使用 NIXL 在预填充与解码工作节点之间传输 KV 缓存数据；在 RDMA 前将多个 PagedAttention 块合并为约 128 词元的数据载荷（注：vLLM 在 CUDA 上默认每块为 16 个词元）

预填充工作节点使用 NIXL，通过选定的传输通道直接读写远程 GPU 内存，从而避免 CPU 拷贝并支持非阻塞推进。预填充工作节点完成请求后，解码工作节点的调度器会立即把对应的解码请求加入自己的解码流水线，使计算与数据移动能够顺畅重叠。应使用预先注册的对等内存和较大的固定窗口，以减少反复注册的开销；可以通过 Nsight Systems 时间线验证零拷贝传输是否与计算重叠。

验证端到端数据移动和重叠时，建议使用带跟踪选项的 Nsight Systems 进行性能分析。对于 InfiniBand 链路遥测，可使用 `--nic-metrics=true` 收集 HCA/NIC 计数器，并使用 `--ib-switch-metrics-device=<GUIDs>` 收集交换机计数器。这些选项会采集交换机指标并抽样主机/设备活动，从而得到相互关联的 CUDA 内核、UCX 活动、存储指标和网络行为。下面是一条统一命令，用于启用 CUDA/UCX 跟踪并收集 CPU 活动、GPU 指标、存储指标和 InfiniBand 交换机遥测数据：

```
nsys profile --trace=cuda-hw,osrt,nvtx,ucx,gds \
  --trace-fork-before-exec=true \
  --cuda-event-trace=true \
  --cuda-graph-trace=node \
  --cuda-memory-usage=true \
  --sample=cpu \
  --gpu-metrics-device=all \
  --nic-metrics=true \
  --ib-switch-metrics-device=<GUIDs> \
  --storage-metrics --storage-devices=all \
  --gds-metrics=driver \
```

```
-o nsys_reports/prefill_decode \
<your_launch_here>
```

预填充和解码引擎可能采用不同的并行布局（例如张量并行布局）。此时，系统可以在接收端插入布局转换内核：在 NIXL 读取之后、使用数据之前，将每个 KV 块重新排列为解码工作节点所需的布局。

连续批处理

解码服务器高度依赖连续批处理，也称为*迭代级批处理*。预填充阶段执行大型矩阵—矩阵计算，而解码阶段的每次新词元生成通常只是较小的向量—矩阵计算，因为单个词元表示为一个向量，所以解码会产生大量小型计算。

为避免逐词元小型工作负载导致 GPU 利用率过低，解码工作节点可以在每次迭代中将多个输入序列组成批次，形成更大的矩阵—矩阵计算（例如并行生成多个词元），从而提高每个解码任务的算术强度。

例如，假设 32 个不同的文本生成请求都处于生成过程中，并准备生成下一个词元。连续批处理调度器不会执行 32 次独立的单词元前向传播，而是将这些请求合并，在一次前向传播中并行生成 32 个词元（每个序列一个）。

这样，矩阵乘法看到的有效批次大小就是 32，GPU 计算单元可以保持繁忙。挑战在于，不是所有序列都会在完全相同的时刻请求词元：有些序列可能较早结束，另一些则可能较晚开始。

请记住，批处理可以跨越不同请求和不同用户。大多数现代推理服务器会在不同用户的序列当时具有相同上下文长度时，自动把它们的解码步骤分组。这相当于在线执行批量解码，可用于最大化解码吞吐量。

在 vLLM 中，CUDA Graph 的捕获范围由 `--max-seq-len-to-capture` 标志控制。捕获尺寸通常会与最大序列数对齐；当序列超过该长度时，vLLM 会回退到 eager 模式。需要注意，这个标志不控制运行时批处理的序列数量；并发解码槽位数由 `--max-num-seqs` 控制（即一次迭代中的序列数上限）。应结合 `--max-num-batched-tokens` 显式设置这些参数，以使内存使用更可预测。

连续批处理通过在每一步动态更新批次大小，解决不同序列在不同时间请求词元的问题。具体而言，每一步都会收集当时已经准备好生成下一个词元的所有序列，并将它们组成一个解码步骤。

如果新请求在解码进行期间完成预填充并准备就绪，就会加入下一个批次，无需等待一段任意长的空闲时间。这与静态批处理不同，后者会先等待收集满一个批次再解码。

如果序列遇到文本结束词元或达到序列长度上限而完成，就会立即从后续批次中移除。如果序列尚未准备好——例如长提示词仍在预填充——则要等它准备好后才会加入。

因此，连续批处理允许每次迭代的批次大小波动。不过，服务器会在上限以内尽量纳入当时可用的序列，以最大化批次大小。这样既能获得较高利用率，又能缩短逐词元等待时间。

连续批处理可以让解码 GPU 工作节点尽量不空闲，始终处理可用请求。即使某些序列正在等待新词元，也能让 GPU 保持忙碌，从而在延迟约束下最大化吞吐量。

类似地，Microsoft DeepSpeed 和 NVIDIA TensorRT-LLM 推理引擎会结合分页式 KV 缓存实现连续批处理或飞行中批处理，以保持解码期间较高的 GPU 利用率。具体来说，DeepSpeed 会合并多个生成请求，TensorRT-LLM 则使用调度器跨不同流分组解码任务。

在解耦的解码集群中，连续批处理的优势会更加明显。由于解码 GPU 只处理生成任务，可以把计算周期专用于这一连续循环，不会被大型定制提示词任务打断，因此吞吐量指标更加平滑，尤其是在负载较高时。

高负载下，一个解码节点可能同时运行几十或几百个序列，并可在每次迭代中将大量序列组成批次，从而最大化硬件利用率。

低负载下，即使只有一个活动序列，解码工作节点也能立即生成词元，无需等待凑满批次。此时 GPU 可能暂时利用不足，但单个序列的延迟仍然较低。因此，连续批处理能够兼顾两种极端：高并发时效率高，低并发时响应快，在高吞吐与低延迟之间取得良好平衡。

对可变长度序列分组

在 LLM 推理中处理变长序列，需要谨慎调度和批处理，以免浪费计算与内存。把长短提示词混入同一批次会产生填充开销；在某些工作负载中，填充词元甚至可能占到总词元数的 50%，从而浪费宝贵的 GPU 和网络资源。

将不同长度的提示词组成批次时，每个较短序列都必须填充至最长序列的长度。这会引入不执行实际工作的填充词元，但它们仍会消耗 GPU 周期、内存带宽以及 GPU 间或网络传输资源。在某些生成式 AI 工作负载中，填充可能占全部词元的一半，因此会显著降低推理效率。

一种直接的解决方法是按序列长度将请求分桶，使每个批次包含长度相近的序列。使用 0–512、513–1,024 个词元等静态长度桶，可以固定批次边界并减少填充开销。

vLLM 的解码调度器维护一个轮转的 `SequenceGroup` 实例池（每个提示词对应一个 `SequenceGroup`）。每次解码迭代使用固定词元预算推进各组。某个 `SequenceGroup` 处理完自己的工作块后便离开池，新组随即加入，使流水线持续有工作可做，而不必依赖静态填充桶。

这些批处理和调度技术很适合预填充与解码集群彼此分离的解耦部署。在这种配置下，独立解码节点可以使用连续批处理等技术，在严格 SLO 下减小 TPOT 波动；专用预填充节点则可独立调优，以最大化输入处理吞吐量并缩短 TTFT。

NVIDIA 的程序化依赖启动（PDL）和设备发起的 CUDA Graph 启动（参见第 12 章）可用于减少逐词元启动开销、重叠工作，并减少解码迭代之间的气泡。这些功能通常由框架启用，而不是在应用代码中手动控制。

使用设备启动的图时，应通过 `cudaGraphInstantiateFlagDeviceLaunch` 实例化，并将节点保留在同一设备上。可以用 PDL 重叠步骤末尾（例如解码迭代末尾）的依赖内核，进一步减少逐词元启动气泡。

通过结合长度分桶、连续批处理、解耦、PDL 和设备发起的 CUDA Graph，vLLM、SGLang 和 NVIDIA Dynamo 等现代推理系统即使面对差异很大的提示词长度，也能兼顾

高吞吐与低延迟。实际收益及资源效率仍取决于工作负载、硬件和具体配置。

> 在 vLLM 中，`--max-seq-len-to-capture` 控制 CUDA Graph 覆盖的最大序列长度，其默认值在本书所述版本中为 8192。连续批处理时，vLLM 可能填充到最近的已捕获尺寸，因此应协调设置 `--max-num-seqs` 和 `--max-num-batched-tokens`，以减少填充浪费。CUDA Graph 有助于避免为常见序列长度反复构建图，但不直接决定运行时批处理行为；后者由解码调度器的动态 `SequenceGroup` 池管理。生产环境中，建议将上述三个参数结合调优，以限制 HBM 中 KV 缓存的用量，并减少连续批处理的填充开销。

KV 缓存的内存管理

由于解码需要关注到目前为止看到的整个序列，包括此前已经解码的词元，因此 KV 缓存内存是解码工作节点的关键资源。每个序列都要为每个 Transformer 层和每个历史词元存储键、值张量。图 17-5展示了不同请求共享 KV 缓存的一个示例。

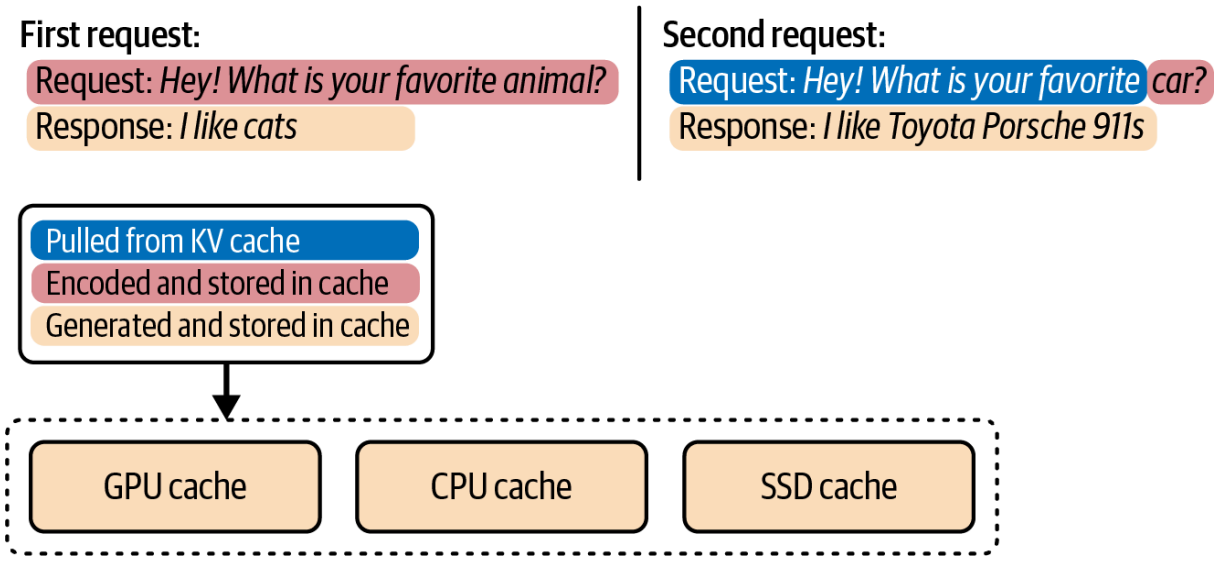


图 17-5. 在请求之间管理和复用 KV 缓存数据

对于大型模型和长序列，KV 内存随词元数线性增长，其大小还取决于注意力布局和 `dtype`。一个实用估算公式是 `bytes_per_token = 2 x n_layers x n_kv_heads x head_dim x bytes_per_element`。（注：`2 x` 表示每层、每词元都包含键和值。）

以一个 Llama 类 13B 模型为例：40 层、40 个注意力头（头维度为 128），采用 FP16。对于使用标准多头注意力（MHA）的 4,096 词元上下文，KV 大小约为每词元 0.819 MB，总计约 3.36 GB；采用 FP8 KV 时约为 1.68 GB。

采用分组查询注意力（GQA）且有 8 个 KV 头（`n_kv_heads = 8`）时，4,096 词元 KV 缓存在 FP16 下约为 0.671 GB，在 FP8 下约为 0.336 GB。采用只有 1 个 KV 头的多查询注意力（MQA）时，FP16 下约为 0.084 GB。

> 计算时务必使用模型实际的 `n_layers`、`n_kv_heads`、`head_dim` 和 KV 精度，因为 FP8、FP4 会改变 `bytes_per_element`。

除了模型权重之外，仅 KV 缓存本身就可能使服务大量并发序列的 GPU 很快接近内存上限。因此，应确保解码工作节点使用经过优化的内存分配器，防止大量长序列同时运行时耗尽 GPU 内存。下面介绍解码工作节点高效管理 KV 内存的几种策略。

分页 GPU 内存分配器

vLLM 的 PagedAttention 机制是一个典型例子：它把 KV 缓存划分为固定大小的页。具体框架及配置还可以将非活动页换出到 CPU 内存。除 vLLM 外，SGLang 和 NVIDIA TensorRT-LLM 也实现了分页式 KV 内存管理器，NVIDIA Dynamo 则建立在这些技术之上。LMCache 等项目还可利用 DRAM 和 NVMe 构建外部 KV 存储层，并在重计算与 I/O 之间调度，以平衡带宽。

大显存 GPU 与自定义分配器

解码服务器通常使用 HBM 容量较大的 GPU（例如配备 180 GB HBM 的 Blackwell B200 和配备 288 GB HBM 的 Blackwell B300）存储 KV 缓存。此外，vLLM 和 NVIDIA TensorRT-LLM 等系统采用优化的内存管理器，以固定大小的页分配 KV 内存，从而减少碎片、支持跨请求复用前缀，并管理大量不同长度的序列。这有助于共享内存并减少碎片和浪费。

KV 缓存卸载（例如换出分页）

GPU 内存接近满载时，解码工作节点可以把较旧的 KV 块卸载到 CPU RAM，或 NVMe 等更冷的存储层。例如，某个序列生成了 1,000 个词元，而这些词元的 KV 并非都需要立即驻留时，可以把部分较早词元的 KV 移到主机内存。

需要时，再按需将其调回 GPU 内存。卸载数据在重新参与注意力计算时会产生延迟，因此应谨慎使用。解码服务器通常会尝试预取数据或将传输与计算重叠，以减轻 KV 数据换入对生成过程的影响。

上下文限制与压缩

有些部署会对解码词元数或输出序列长度设置硬上限。这是一种应用层取舍，可以限制 KV 缓存大小，避免其无限增长。KV 压缩也能减少每词元所需内存，例如用较低精度的 FP16、FP8 或 INT8 存储 KV；实际精度损失需针对模型和任务验证。

另一种方法是使用多查询注意力（MQA），让多个查询头共享每个词元的 KV 向量，从而显著缩小 KV 缓存。这属于直接降低 KV 占用的模型架构变化。分组查询注意力（GQA）和 DeepSeek 的多头潜在注意力（MLA）也有助于减小 KV 缓存。

解耦架构中的内存层级

解耦设计的另一个优势是，解码集群的 GPU 内存可以专门用于存储模型权重和 KV 缓存；它不需要同时处理大型提示词预填充计算，而在单体服务系统中，预填充会暂时消耗大量额外内存。

每块解码 GPU 通常都会加载完整模型权重（除非使用模型并行），剩余内存用于存储 KV。例如，GPU 总内存为 180 GB、模型权重占 70 GB 时，理论上约有 110 GB 可用于 KV（实际还需扣除运行时缓冲区等开销）。

这会直接影响该 GPU 能同时容纳的大致“词元数 × 序列数”。解耦并不会消除 KV 内存问题，但通过分离角色，可以选择在内存容量和内存带宽方面更适合解码的节点类型。

在这种集群配置下，需要决定何时把预填充卸载到预填充工作节点池，何时由解码工作节点本地执行。卸载会引入排队、网络传输等开销，因此只有确实有助于降低延迟时才应使用。下一节将介绍负责作出这一决定的路由策略。

解耦路由与调度策略

并非每个请求都需要卸载到预填充工作节点。没有必要时这样做只会增加开销，收益却很小。因此，解耦推理系统会使用路由策略进行条件解耦，仅在远程预填充可能有帮助时才采用该路径。表 17-1概述了包括 KV 感知和前缀感知在内的路由策略。

表 17-1. 高级路由策略，包括前缀感知和 KV 感知路由

路由策略	说明
Round robin	依次路由到每个节点
最少请求	路由到活动请求最少的工作节点
前缀感知	使用请求前缀选择工作节点
KV aware	路由到 KV 缓存与请求最匹配的工作节点

解耦路由器会在最初接收新请求的解码工作节点上，对每个新请求运行，并快速决定在本地预填充，还是在预填充工作节点池中远程预填充。

路由因素

是否把预填充卸载到预填充工作节点池，可依据请求和系统状态的多个因素决定。常见路由因素包括当前队列长度、GPU 内存可用量，以及硬件专门化程度，因为某些 GPU 更适合特定模型或提示词类型。

vLLM 的 KV 缓存感知路由器等高级路由器还会考虑缓存局部性，将请求路由到已经在缓存中持有其部分前缀的解码工作节点。图 17-6展示了一个 KV 缓存感知路由器如何根据工作节点发出的 KV 缓存事件数据，在系统中转发请求。

目标是通过路由最大化缓存命中并平衡负载。表 17-2总结了典型解耦设计中影响路由决策的一些关键因素。

表 17-2. 影响路由器决定是否卸载预填充的因素

因素	说明	对路由决策的影响
提示词长度	输入提示词中的词元数（应用前缀缓存之后）。	长提示词意味着更多计算：超过阈值时卸载预填充；短提示词则本地处理。
前缀缓存命中	解码工作节点 KV 缓存中已经存在的提示词部分（来自之前的请求）。	前缀命中较大时，预填充实际更短且更偏内存受限，应本地处理；没有命中时，全部是新词元，计算负担较重，卸载可能有益。
预填充队列长度	全局预填充队列中的待处理任务数（反映预填充工作节点的繁忙程度）。	队列较长、预填充节点落后时，避免卸载新请求，改在本地处理；队列为空或较短时，预填充节点有容量，满足其他条件即可卸载。
解码工作节点负载	本地解码工作节点当前负载（正在进行的解码任务等）。	解码 GPU 忙于许多解码流时，卸载可释放其繁重计算并提升并行性；解码大多空闲而预填充队列积压时，则本地预填充以利用可用容量。
延迟 SLO 紧迫性	请求的优先级或延迟要求的严格程度。	紧急低延迟要求时，可卸载以尽快计算提示词，尤其在本地解码繁忙时；要求宽松时可本地运行以节省资源（见“QoS 与提前拒绝策略”）。

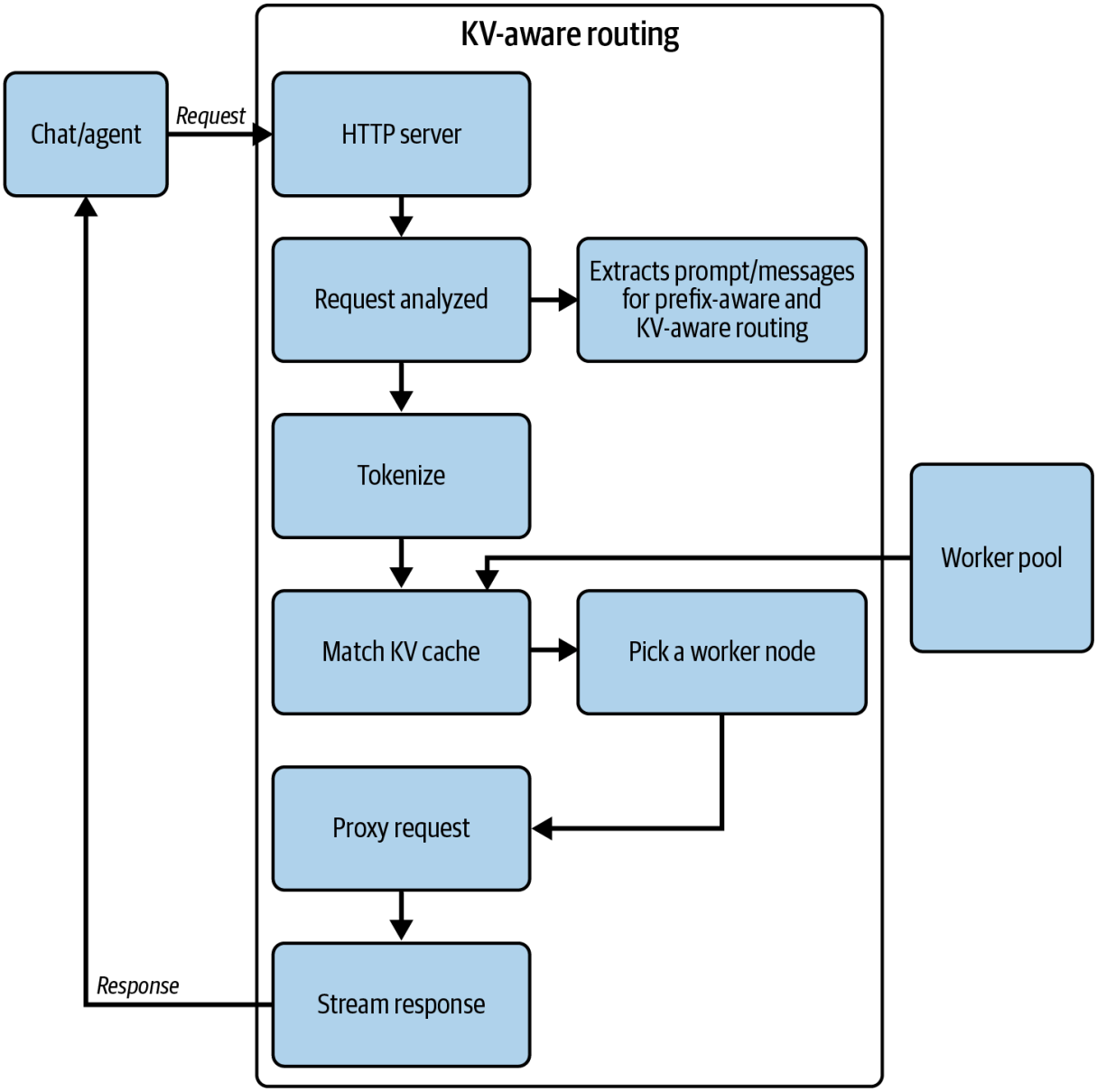


图 17-6. 基于从工作节点发出的 KV 缓存事件接收的数据进行 KV 缓存感知请求路由

这些因素会使系统只卸载能够从远程执行中获益的请求，例如长且计算密集的提示词。短提示词和缓存命中的提示词则在本地处理，以减少开销。阈值可以调节。表 17-2中的每个因素都对应一种取舍，下面分别说明。

提示词长度

不应把无关紧要的提示词卸载出去浪费时间。例如，五个词元的提示词即使远程执行开销很小，也不值得卸载，因为解码 GPU 自己很快就能处理。卸载应保留给那些会让解码 GPU 长时间被占用的繁重提示词。

前缀缓存

现代推理系统通常实现 KV 缓存，用于保存此前见过的上下文，例如对话早期轮次或重复出现的模板化系统提示词。如果新请求的提示词有很长前缀与已处理提示词重叠，解码工作节点可能已经在内存中保存了该前缀的 KV 数据；此时只需为未命中缓存的剩余部分执行计算。

如果剩余部分很短，卸载的收益就会降低。如果整个提示词都因完整前缀命中而已被缓存，则完全不需要预填充计算，可以直接利用缓存状态解码。路由器会将有效提示词长度视为 `prompt_length - prefix_cached_length`，从而考虑这一点。

较大的前缀命中不仅减少所需计算，还意味着如果把请求发送到预填充工作节点，就必须来回传输大量 KV 数据，这通常没有意义。因此，这类请求应保留在本地并利用已有缓存。

预填充队列长度

这个因素本质上衡量预填充工作节点集群的负载。如果大量任务正在等待，预填充工作节点已经不堪重负，再发送一个任务过去可能只会让请求继续排队，TTFT 反而变差。此时应暂时让解码工作节点自行执行更多工作。

这为预填充集群形成了自然的负载卸除机制：专用预填充层达到容量上限时，系统会平滑退化为更多本地计算；预填充队列重新缩短、工作节点追上进度后，则恢复卸载长提示词。这种动态平衡使解耦架构能够适应多种负载条件。

解码工作节点负载

虽然代码不一定总会显式表达这一点，但路由器本身就有助于分散工作负载。如果解码工作节点已经忙于解码许多流，仍可卸载新到达的提示词；否则，该 GPU 必须同时处理繁重计算和解码，可能让两者都变慢。

反过来，如果系统负载很低、解码 GPU 空闲，它们可以在本地处理更长的预填充。实践中，整体负载较低时，空闲解码 GPU 往往也对应较空的预填充队列，现有条件已间接覆盖这种情况；但某些实现也会直接检查本地 GPU 利用率，以决定是否需要卸载预填充。

延迟 SLO 与优先级

在混合 SLA 或优先级类别的系统中，可以调整路由来改善 QoS。对于必须获得最快 TTFT 的高优先级请求，可以绕过队列检查并立即卸载，使其尽快开始计算，即使解码 GPU 当前空闲也是如此；这样可把该解码工作节点留给其他高优先级解码任务。

对于低优先级请求，系统也可以完全不使用预填充集群资源，而是在解码工作节点本地执行，甚至推迟处理。“QoS 与提前拒绝策略”一节将再次讨论优先级请求；这里要点在于，基础路由器可以纳入这类因素。

路由策略所用的具体阈值和权重应通过实验确定。例如，在某种 GPU（如 Blackwell B200）上，少于 50 个词元的提示词可能在本地计算更快，而更长提示词则可能从卸载中获益。

随着新硬件发布，阈值也可能变化，因为新 GPU 通常有更高 FLOPS 和更大内存带宽。例如，单块 B200 可以在解码工作节点上处理更多词元，因此卸载的盈亏平衡提示词长度可能提高。

> 升级硬件后，应根据计算 FLOPS 和内存带宽的变化，通过实测调整 `PREFILL_LENGTH_THRESHOLD` 等阈值。

有效路由的结果是兼得两方面优势：短提示词在本地运行，TTFT 更快且没有额外开销；长提示词则因预填充和解码在不同 GPU 上并行而受益。同时，预填充工作节点池只在需要时使用，无法跟上负载时也不会继续被大量请求淹没。

这种条件策略是 DistServe 原型以及 vLLM、Dynamo 等现代推理服务器的重要组成部分，使它们能够提高延迟约束下的有效吞吐量（goodput），而不只是原始吞吐量。

动态路由策略代码示例

实践中，路由策略可以实现为简单的条件检查。下面的代码展示了这种路由逻辑的简化版本：

```
# Offload prefill decision running on a decode worker
# (B200/B300 tuned)
def should_offload_prefill(prompt_length: int,
                           prefix_cached_length: int,
                           prefill_queue_size: int,
                           decode_active_reqs: int,
                           ttft_slo_ms: int = 500) -> bool:
    # Effective prefill after prefix KV hits
    eff_len = max(0, prompt_length - prefix_cached_length)

    # Tunables (kept in config; shown here for clarity)
    PREFILL_LENGTH_THRESHOLD = 256  # see split_policy.prompt_length_threshold
    PREFILL_QUEUE_MAX        = 10   # see autoscale/prefill queue-length guidance
    DECODE_LOAD_THRESHOLD    = 8    # active decode streams

    long_prefill = (eff_len >= PREFILL_LENGTH_THRESHOLD)
    prefill_available = (prefill_queue_size < PREFILL_QUEUE_MAX)

    # Prefer remote when prefill is compute-heavy
    # and the pool has capacity.
    if long_prefill and prefill_available:
        return True

    # If local decode is busy and prefill is moderately long,
    # free decode via offload.
    if decode_active_reqs >= DECODE_LOAD_THRESHOLD and eff_len >= 64:
        return True

    # Otherwise keep prefill local
    # (lower overhead / better cache locality).
    return False
```

在这段伪代码中，`PREFILL_LENGTH_THRESHOLD` 是通过系统调优得到的参数，例如 50 或 100 个词元，用于定义什么是“长”提示词。`PREFILL_QUEUE_MAX` 是预填充工作节点池被视为饱和的阈值，表示待处理任务过多。

解码工作节点收到新请求后会立即调用 `should_offload_prefill()`。如果函数返回 `True`，解码工作节点就把提示词封装成消息，推送到全局预填充任务队列；等待 KV 缓存结果返回期间，它可以执行其他工作。

如果 `should_offload_prefill()` 返回 `False`，解码工作节点会立即自行执行预填充。这样，预填充工作节点开始落后时，新请求就会回退到本地计算，以避免排队延迟。这是一种在解码池与预填充池之间平衡负载的自适应路由。

动态路由策略配置示例

在生产部署中，路由策略应通过文件或界面配置，而不应硬编码。例如，NVIDIA Dynamo 允许在 JSON 或 YAML 配置中指定复杂的路由和自动扩缩容规则。下面是一个封装部分策略逻辑的简化 Dynamo Planner JSON 示例：

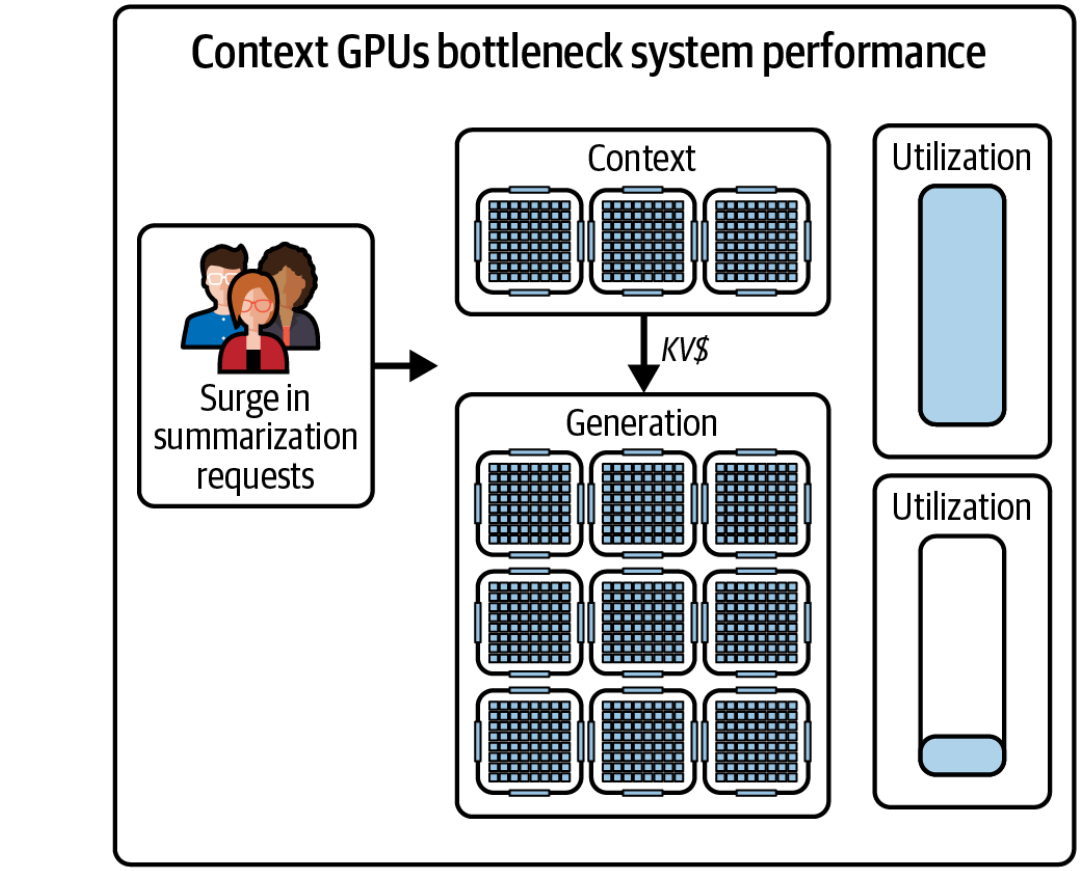
```
model: ...
split_policy:
  prompt_length_threshold: 256
  prefix_cache_weight: 10.0
  queue_length_weight: 1.5
  decode_load_weight: 0.5
  enable_hotspot_prevention: true
cache:
  reuse_prefix: true
  min_cache_hit_ratio: 0.75
autoscale:
  prefill:
    min_replicas: 4
    max_replicas: 12
    scale_up:   { queue_length: 8,  gpu_utilization: 80 }
    scale_down: { queue_length: 2,  gpu_utilization: 40 }
  decode:
    min_replicas: 8
    max_replicas: 24
    scale_up:   { queue_length: 16, kv_cache_usage: 75 }
    scale_down: { queue_length: 4,  kv_cache_usage: 30 }
qos:
  enable_early_rejection: true
  low_priority_threshold_ms: 500
  reject_on_slo_violation: true
```

该配置定义了一个 `split_policy`，其 `prompt_length_threshold` 为 256 个词元；还为缓存命中、队列长度、解码负载等因素指定权重，并为预填充和解码角色配置 `autoscale` 行为，包括根据队列长度、GPU 利用率和 KV 缓存使用量扩容或缩容。

此外，还可以应用提前拒绝等 QoS 规则，并调节将请求视为“慢请求”的阈值。实践中，Dynamo 路由器会在启动时读取该 JSON，或动态获取它，用于指导整个集群中各请求的路由决策。

容量感知路由

如前一节所述，NVIDIA Dynamo 支持动态路由策略。Dynamo GPU Planner 是这种能力的一种实现。Planner 使用 TTFT、TPOT 和 KV 缓存传输预计成本等指标，决定是否修改路由，甚至重新分配或扩展不同阶段的 GPU，以减少瓶颈并适应工作负载变化。如图 17-7所示，这有助于系统在需求激增时维持较高性能。


Context GPUs bottleneck system performance
Surge in summarization requests
Context
KV$
Generation
Utilization
Utilization

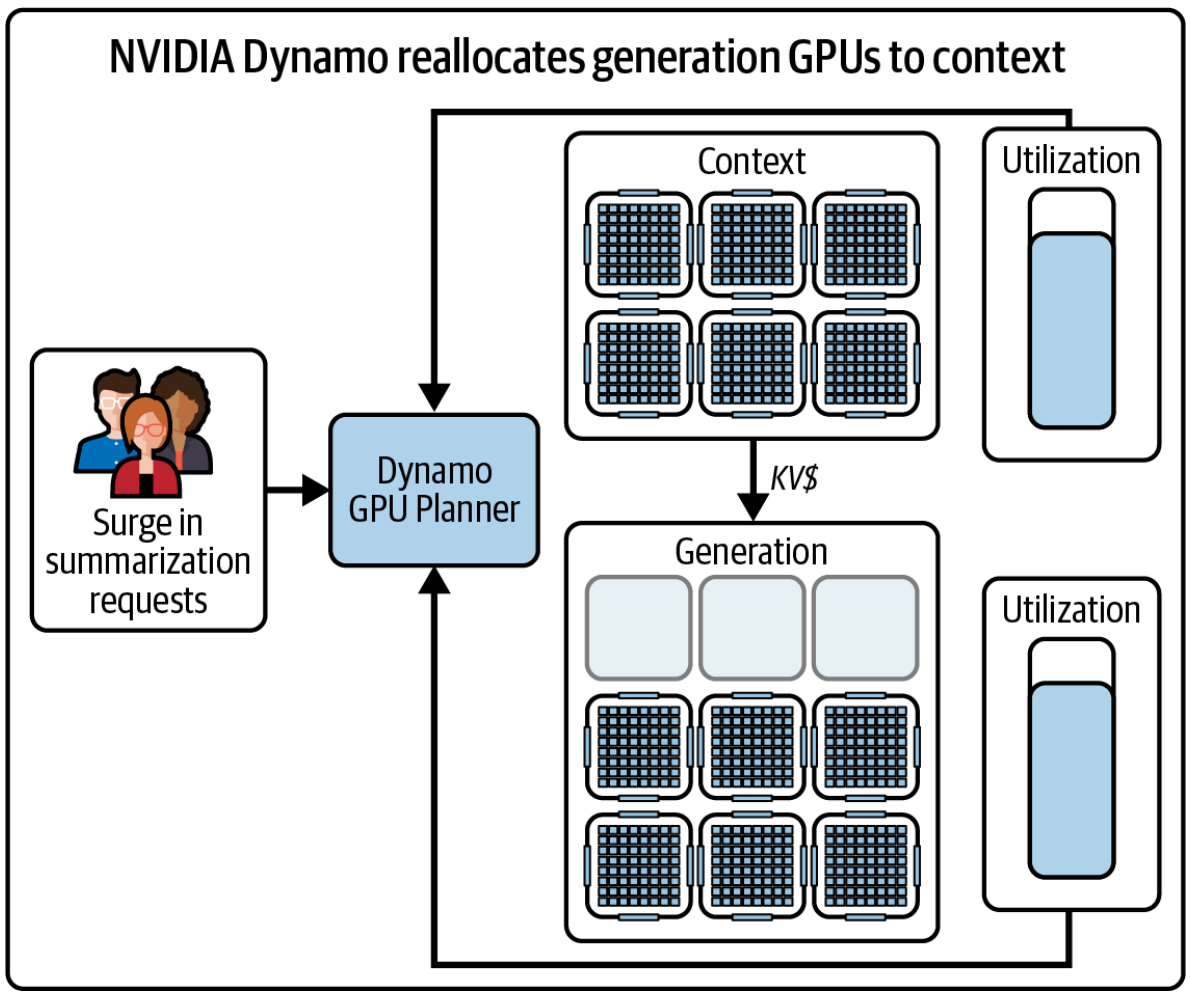

NVIDIA Dynamo reallocates generation GPUs to context
Surge in summarization requests
Dynamo GPU Planner
Context
KV$
Generation
Utilization
Utilization

图 17-7. NVIDIA 的 Dynamo GPU Planner 根据 GPU 利用率指标决定如何处理传入请求，并为预填充和解码分配 GPU 工作节点

这里，Dynamo 的 Planner 选择把更多 GPU 调配到预填充（上下文）阶段，因为大量摘要请求涌入：相对于解码（较短的摘要输出），它们需要大量预填充（较大的输入）。

相反，如果推理型请求大量涌入，Planner 可以把更多 GPU 重新分配给解码阶段，因为这类请求相对于输入词元往往会生成大量输出词元。在其他情况下，Planner 也可能选择传统单体方式，让预填充与解码在同一 GPU 工作节点上完成。

简而言之，NVIDIA Dynamo Planner 这类组件可以持续监控实时指标，并将其与 TTFT、TPOT 等应用层 SLO 对比，从而自动化这一决策过程。Planner 可以动态决定采用完全解耦、完全不解耦，还是介于两者之间的方案，并为预填充和解码阶段分配不同数量的 GPU；实际效果取决于部署和工作负载。

延迟感知路由

推理路由器可以超越简单阈值规则，使用更复杂的延迟评分模型为每个请求选择最佳工作节点。例如，它可以根据工作节点繁忙程度、内存使用量、是否存在相关缓存等实时指标，持续计算每个候选节点的延迟成本，然后将请求发送给延迟成本最低的节点。

假设延迟成本最低的工作节点最空闲、也最值得优先选择。一个简单的延迟成本函数如下所示：

```
# Lower cost is preferable
latency_cost = 0.7 * (occupancy_percent)
  + 0.3 * (active_req_count)
```

这个延迟成本函数更重视 GPU 占用率，因为它反映 GPU 当前正在处理多少工作；其次考虑该引擎当前正在处理的请求数。新请求会发送到延迟成本最低的引擎。

示例中的 0.7 和 0.3 权重可以根据实测数据调优。如果 KV 内存使用量因为大量换入换出或更高内存带宽占用而成为减速的重要预测因素，就应提高它的权重。

更先进的路由策略还可以纳入其他因素，例如将下面这些因素加入延迟成本函数：

精确缓存匹配的可用性

> 如果工作节点的缓存中已有所需前缀 KV，即发生前缀命中，那么它可以更快地服务请求。此时可以分配较大的负权重来降低延迟成本并优先选择该节点，因为在这里成本越低越好。

KV 占用率

> 更高的内存使用量意味着 GPU 更繁忙。系统会分配正权重来提高延迟成本，促使路由器避开该节点，因为此场景更偏好较低成本。

活动请求

> 并行请求越多，潜在的上下文切换开销越大，因此应增加延迟成本，避免选择该节点。

内存带宽利用率

如果 GPU 正因处理许多长序列而大量占用内存带宽，再增加工作只会使系统变慢。这应提高延迟成本，降低选择该节点的倾向。

最近的 KV 使用情况

如果某个工作引擎最近使用过完全相同的前缀，缓存处于“热”状态，那么 KV 可能仍在 L2 缓存中，或正从预填充工作节点传来，因此可能改善性能。此时可以加入小的负权重，降低延迟成本并优先选择最近见过该前缀的节点。

表 17-3总结了这种高级路由策略配置，列出若干因素及其示例成本。

表 17-3. 路由因素与相对成本

因素	含义	对延迟成本的影响
KV 占用率（%）（`occupancy_percent`）	越高表示内存压力越大	+3
活动请求数（`active_req_count`）	正在处理的请求越多，潜在排队越严重	+1
KV 缓存匹配（`cache_match_flag`）	引擎已经拥有所需的前缀 KV	−10（较大的负值）
内存带宽占用率（%）（`mem_bw_percent`）	越高表示内存总线越繁忙	+0.5
近期 KV 使用（`recent_prefix_flag`）	该引擎最近使用过这个前缀	−1

现在将这些额外指标加入延迟成本计算。路由器会计算如下复合延迟成本：

```
# Lower cost is preferable
def latency_cost(occupancy_percent: float,
                 active_reqs: int,
                 cache_match_flag: bool,
                 mem_bw_percent: float,
                 recent_prefix_flag: bool) -> float:
    return (
        3.0 * occupancy_percent
      + 1.0 * active_reqs
      - 10.0 * int(cache_match_flag)
      + 0.5 * mem_bw_percent
      - 1.0 * int(recent_prefix_flag)
    )
```

预填充集群和解码集群甚至可以使用略有不同的公式。对于预填充，缓存命中（例如前缀已经计算完成）可能更值得纳入；对于解码，内存带宽或可用 KV 空间可能更主导决策。

路由器需要从每个工作节点获取遥测数据，包括等待队列长度、KV 缓存使用量、内存利用率等指标，以便持续更新延迟成本。

这样，系统可以动态地把流量路由到对延迟影响最小的位置。低负载下所有工作节点的分数都较低，选择哪个节点可能并不重要，因为任何节点都能快速处理请求。

高负载下，路由器会把新工作发送给最空闲的节点，从而实现负载均衡；同时利用数据局部性（缓存命中）减少工作量，因此平均延迟和尾延迟都能降低。

如果某台服务器缓存了相关数据，即使它稍微繁忙一些，路由器仍可能选择它，从而更快服务该请求。评分函数定量刻画了这一取舍。

NVIDIA Dynamo 执行类似但方向相反的计算：分数越高越好。它会计算远程预填充分数；如果分数超过阈值，就把任务卸载到预填充工作节点池。下面的 YAML 片段配置了条件解耦：只有计算分数高于可配置的 `remote_prefill_min_score` 时才执行远程预填充。

```
disaggregated_router:
  enable: true
  policies:
    - metric: prefill_length   # Prompt length after prefix cache hit
      threshold: 256
      action: prefer_remote    # if prompt > 256 tokens, offload prefill
      weight: 0.7
    - metric: prefill_queue_depth  # requests queued for prefill workers
      threshold: 10
      action: prefer_local     # if queue > 10, lean toward prefill locally
      weight: 0.3
  remote_prefill_min_score: 0.5  # overall threshold decide remote prefill
```

这里，路由器根据 `prefill_length` 和 `prefill_queue_depth` 计算分数。如果提示词超过 256 个词元，就投票选择 `prefill_remote` 并卸载预填充；这部分计算的权重为 0.7。

但如果预填充队列很深，等待任务超过这里配置的 10 个，就会以 0.3 的权重投票选择 `prefill_local`。如果合并分数高于 `remote_prefill_min_score`（本例为 0.5），Dynamo 就会卸载预填充；否则保留本地预填充，不执行卸载。

多路径推理（竞速）

多路径推理是指将同一个请求发送到两种不同模型规模或两条不同路由，主要用于提高可靠性。本质上是竞速获取最快结果，因此常称为*竞速*。有报道称，Google 和 Meta 的生产系统会让模型竞速以降低尾延迟；这是一种成本高但可能有效的技术，具体是否采用取决于系统设计。

> 如果自行实现多路径推理，应确保请求具有幂等性，即两条路径都执行时不会造成副作用；还要及时取消较慢路径，以节省 GPU 周期。

跨工作节点的多分支并行推测解码

如第 15 章所述，一些先进推理服务器支持推测解码。推测解码会并行生成多个词元分支，并丢弃不够理想的分支。

推测解码虽然不是解耦本身的组成部分，却可以叠加在路由器的决策流程之上。例如，系统检测到某个推测分支不确定时，可以把多个推测请求并行发送给不同解码工作节点，生成多个候选的下一词元分支，以掩盖词元分支生成的不确定性。对于分支不可预测的生成任务，这相当于用额外计算换取更低延迟。

如果实现这种机制，路由器需要协调推测任务并合并结果。必须限制资源使用，避免压垮集群。多路径推理可以作为极端延迟敏感推理的潜在优化，但代价是额外消耗可能被浪费的计算和内存带宽资源。

总之，延迟感知路由器会同时考虑当前负载和缓存复用等潜在加速因素，把每个请求发送到合适的位置。它相当于分布式服务系统的大脑，与每块 GPU 上的底层调度协同工作；结合全局扩缩容和 KV 缓存策略，可以最大化有效吞吐量并降低延迟。

QoS 与提前拒绝策略

QoS 策略会对某些请求提高优先级或进行限流，以满足延迟目标。提前拒绝也称为*准入控制*，系统饱和时可以拒绝低优先级查询，从而为其他请求保留资源并维护其延迟等 SLO。

现代系统通常依据*队列等待时间阈值*执行这一策略。例如，如果请求在队列中等待时间超过 X 毫秒，与其让它严重违反 SLO，不如拒绝请求或将其卸载到低等级服务。这类 QoS 防护机制在任务关键型推理系统中越来越常见。

在超大规模推理系统中，需要通过 QoS 机制在重负载下维持尾延迟保证。即使解耦集群经过优化，只要负载超过容量，请求仍会开始排队，延迟也会升高。

设计良好的系统不会任由所有请求都违反延迟 SLO，而会选择平滑卸除负载。当某些请求显然会破坏其他请求的延迟保证时，可以拒绝或推迟它们，尤其是低优先级请求。

这类似于 Web 服务器在极端负载下返回HTTP 503“过载”错误。对于 LLM 服务，如果无法保证及时处理，可以主动拒绝请求或降低服务规格。下面介绍预填充—解码解耦场景中的几个 QoS 组成部分。

延迟 SLO 跟踪

> 系统应了解目标 TTFT 和 TPOT。例如，99% 的请求必须在 200 毫秒内返回首个词元。每个解码工作节点可以利用内部遥测数据，根据当前队列长度等信息，估算立即接受新请求时的 TTFT；也可以估算再增加一个解码流后的 TPOT。

准入控制（提前拒绝）

> 在完全接受并分配请求之前，系统可以检查接纳该请求是否会导致过载或违反 SLO。如果会，就立即以“服务器繁忙，请稍后再试”之类的响应拒绝请求，这称为*提前拒绝*。实践中，可以依据全局系统负载视图，也可以使用单个节点的启发式规则触发提前拒绝。
>
> 例如，公共 API 可能会在积压过高时返回错误，而不是继续接受请求并严重违反服务目标。有些提供商会在负载高峰动态降低最大生成长度，本质上是在回答完整性与延迟之间取舍。

优先级划分

请求并非同等重要。来自付费客户或关键服务的请求可能优先级较高，免费层用户或后台作业的请求可能较低。系统可以把优先级纳入调度决策：解码调度器可以先服务高优先级任务，预填充任务队列也可按优先级排序。系统繁忙时，低优先级任务可能等待更久，或被快速拒绝，以让位给高优先级任务。

优雅降级

系统开始过载时，可以不直接拒绝请求，而是降低较不重要请求的服务规格。例如，改用较小模型、截断提示词或限制输出词元数。一种简单做法是在高负载期间临时降低低等级请求允许的最大提示词长度或输出长度。

在解耦场景中，一种有趣的平滑降级方式是在系统承压时，暂时关闭低优先级请求的远程预填充。由于远程预填充会额外占用集群资源来优化延迟，可以让低优先级查询仅在解码工作节点本地执行预填充，只使用解码资源。这样这些请求的响应会变慢，却能为高优先级查询释放预填充工作节点。

例如，推理服务器可以为每个请求标记优先级；当预填充工作节点正忙于处理高优先级任务时，让路由器对低优先级请求忽略 `should_offload_prefill` 逻辑。

下面是一个提前拒绝策略示例。它可以在每个解码工作节点处理请求之前运行，也可以在更上游的 LLM 网关执行：

```
# Early rejection based on estimated latency and priority
from dataclasses import dataclass

class QoSController:
    def __init__(self, ttft_slo_ms: int = 500):
        self.ttft_slo_ms = ttft_slo_ms

    def admit_request(self, priority: str) -> bool:
        # The queue length and per-request averages are fed by the metrics system
        est_ttft = (get_current_prefill_queue_length()
                    * get_avg_prefill_time_per_req()
                    + get_current_decode_queue_length()
                    * get_avg_decode_time_per_req())

        if est_ttft > self.ttft_slo_ms and priority.lower() == "low":
            # reject low priority request to protect SLOs
            return False
        return True
```

这里通过预填充队列中的任务数估算 TTFT，因为每个任务都会增加一些延迟，同时考虑前方已有的解码任务数。请注意，解码任务通常会重叠，因此这只是粗略估计。如果估算 TTFT 超过允许的最大值（即 SLO），对于低优先级请求就拒绝它，并让 `should_offload_prefill` 函数返回 False。

对于高优先级请求，仍然接受它，必要时牺牲一些其他排队工作。更复杂的做法是抢占已经排队的低优先级请求，为新的高优先级请求腾出空间；这通常通过为每个优先级维护独立队列实现。

进一步看，前例中的 `avg_prefill_time_per_req()` 和 `avg_decode_time_per_req()` 函数会使用观测到的每请求预填充和解码时长的指数移动平均，动态计算这些值。它们分别按提

示词词元数（预填充）和生成词元数（解码）归一化；对于长度不同的输入，引擎会将每词元平均值乘以该请求的实际词元数进行外推。

`get_current_prefill_queue_length()` 和 `get_current_decode_queue_length()` 函数获取调度器内部队列跟踪的待处理预填充与解码任务数量。这些值由调度器维护。

这些逐词元时间估计会通过调度器循环实时刷新；默认情况下，调度器每秒从 `/metrics` 端点更新一次，从而捕获动态工作负载变化。

提前拒绝和优先级调度可以确保系统接近饱和时以可控方式失败，而不是崩溃。最重要请求的用户仍能在规定 SLO 内获得服务，同时卸除较不重要的流量。

许多生产部署需要与 LLM 网关层协调实现这一机制。网关可以向客户端返回特定错误码，或返回表示服务器过载的代码。关键是系统应保持在能够兑现已接受工作延迟承诺的状态，而不是过度接纳请求，导致所有请求都错过 SLO。

QoS 的另一个考虑是自适应生成上限。如果解码阶段可能运行过久并导致 TPOT SLO 违规，系统可以提前截断生成。例如，用户请求生成 1,000 个词元，但系统承压时，可能只允许生成 200 个词元，然后停止，为其他请求释放资源。QoS 策略通过提高优先级或限流满足延迟目标；系统饱和时，提前拒绝可以拒绝低优先级查询，以维护其他请求的 SLA。

总之，解耦可以减少固有干扰，但流量尖峰仍可能压垮任何固定容量。QoS 机制通过确保架构兑现延迟承诺来补充解耦；提前拒绝会拒绝或降低部分工作的优先级，以保护其余请求的延迟。

构建超大规模系统时，这些策略与核心性能优化同等重要。没有它们，请求洪峰会制造巨大的队列和减速，抵消所有性能改进。

预填充与解码解耦的可扩展性

可扩展性的另一个方面，是增加节点后性能能保持到什么程度。解耦方法在许多方面可以实现近似线性扩展：可以增加预填充节点来处理更多提示词吞吐量，也可以增加解码节点来处理更多词元生成吞吐量。实际扩展效率仍会受到网络、调度和负载分布等因素影响。

主要挑战在于平衡两者。规模越大，自适应调度器越重要，因为流量模式变化等失衡情况在大型系统中更容易发生。动态比例配置允许集群在运行中重新平衡预填充与解码的资源比例。

另一个考虑因素是多模型服务。如果多个模型的解码特性相近，解耦可以让它们共享解码能力。例如，模型 A 和模型 B 部署在同一集群时，可以让一部分解码工作节点处理两种模型。这在托管复用同一基础模型架构的不同 LoRA 适配器时尤其有用。

这样可以构建灵活的多租户、多模型、预填充—解码解耦推理服务器系统。虽然具体设计超出本章范围，但解耦的模块化确实提供了这种灵活性：多个模型可共享一个解码池，同时各自拥有预填充前端。

最后考察超大系统中的尾延迟。推理系统扩展到数千乃至更多 GPU 节点时，控制尾延迟会越来越困难，因为服务器越多，出现慢节点或故障节点的概率越高。

解耦在这里也能有所帮助：将预填充任务与解码任务隔离在不同节点上，某一侧的慢节点就不会大幅影响另一侧的任务。

本章讨论的提前拒绝、两级调度和 KV 缓存复用等技术，有助于缓解拖尾节点的影响。例如，缓存请求上下文的大部分内容后，一次慢操作不太会拖慢整个生成过程，因为许多计算已经完成。

总之，经过恰当配置和调优后，解耦系统在大规模下通常更具韧性。消除一种干扰因素并引入适应能力，能够改善尾延迟分布，至少可使其随负载增长而恶化得更慢；具体结果仍取决于工作负载、容量规划和故障处理机制。

要点回顾

本章涵盖的技术，从 KV 缓存高速 RDMA 传输到动态路由算法和 QoS 策略，都是大规模生产推理工作负载的重要组成部分。以下是关键要点：

消除预填充—解码干扰以改善延迟

分离预填充与解码阶段可以减少干扰和队头阻塞，避免长提示词拖延较短提示词，从而收紧延迟分布；这也有助于每个阶段更可靠地满足自身延迟 SLO。

分别优化各个阶段

预填充（提示词处理）受计算能力制约，受益于高并行度和高 FLOPS，通常可借助高算力 GPU 及 FP8/FP4 等低精度技术缩短 TTFT。解码（词元生成）受内存限制，受益于高内存带宽 GPU 和连续批处理等技术，以提高逐词元吞吐量。解耦允许两个阶段使用不同硬件和并行配置，而固定的“一刀切”配置难以同时做到这一点。

利用 KV 缓存

使用 KV 缓存避免重复预填充计算是常见做法。智能路由器可以决定新请求是卸载到预填充工作节点，还是直接由解码工作节点处理；路由策略应考虑提示词长度、缓存命中和集群负载。

智能路由

vLLM、SGLang 和 NVIDIA Dynamo 等现代推理服务器可使用多因素评分算法路由预填充与解码请求，以提高吞吐量并维持适当延迟。许多框架在调度器和仪表盘中也将 TPOT 称为*词元间延迟*（ITL）。建议分别跟踪预填充与解码节点的 TTFT（p50/p95/p99）和 ITL/TPOT（p50/p95/p99），以便在升级期间更容易调试性能回退。例如，当预填充服务器饱和或提示词很短时，可以跳过远程预填充。

使用 QoS 在大规模下维持 SLA

重负载下应实施准入控制和优先级调度。与其让所有请求的延迟失控，不如快速拒绝超额或低优先级请求。实际系统可以返回“繁忙”错误或降低响应长度来实现提前拒绝，以保护指定阈值下的第 99 百分位延迟。解耦与 QoS 结合可降低过载级联风险，具体效果取决于容量与策略配置。

结语

预填充—解码解耦已成为现代大规模 LLM 推理中常见的架构选择。Meta、Amazon、NVIDIA、Google 和 OpenAI 等主要 AI 提供商都公开讨论过或提供过与这种思路相关的技术；但完整生产部署架构通常并未公开。

虽然大多数公司不会公布完整生产架构，但已有许多开源实现，例如 vLLM、SGLang 和 NVIDIA 的 Dynamo，展示了这种方法及其潜在收益。与单体设计相比，解耦的预填充—解码架构可在合适的模型、硬件和负载条件下实现更高有效吞吐量（延迟约束下的有用吞吐量）和更好的成本效率。

下一章将继续探讨如何结合现代 GPU 和网络硬件，以及更先进的调度与缓存优化，为超大模型 LLM 服务更多用户，同时控制延迟和成本。

第 18 章　高级预填充—解码与 KV 缓存调优

本章以第 17 章为基础，深入讨论推理预填充和解码阶段的高级优化。我们将在高层扩展策略之上，介绍底层技术，包括单次解码“超级内核”、智能 KV 缓存调优与跨 GPU 共享、提示词状态的快速 GPU 到 GPU 传输、自适应资源调度，以及预填充与解码工作节点之间的动态路由。

还将介绍可提升性能和效率的软硬件创新。根据模型、硬件和工作负载恰当应用这些技术，可以降低解码延迟、提高单 GPU 吞吐量，并帮助在规模化场景中满足严格的延迟 SLO。

优化的解码内核

到目前为止，我们一直关注高层系统与集群优化策略。扩展超大规模推理时，另一组值得考虑的技术是底层内核和内存管理调优，尤其针对解码阶段。

解码阶段是分布式的，并且通常受内存限制。这促使研究人员和工程师尽可能加速解码，并针对特定硬件进行调优。该领域值得关注的技术包括 DeepSeek 的 FlashMLA、Stanford 的 ThunderMLA，以及 PyTorch 的 FlexDecoding；它们主要针对 LLM 工作负载常见的变长序列场景，提升 Transformer 多头注意力在解码时的效率。下面逐一介绍。

FlashMLA（DeepSeek）

Flash Multi-Latent Attention（FlashMLA）是 DeepSeek 推出的优化解码内核。它专注于单词元解码步骤，即用于生成下一个词元的 Transformer 层前向传播。FlashMLA 通过融合操作并更好地利用 GPU 内存层级来加速解码。

对推理而言，FlashMLA（解码）的作用可类比于 FlashAttention（预填充）对训练的作用：它减少内存访问开销和延迟。与标准内核相比，FlashMLA 在适配的模型、硬件和实现条件下可以显著缩短解码阶段延迟。

FlashMLA 将多个注意力操作融合为一个，以提高算术强度。这样可在一次融合内核启动中处理多个注意力头和多个时间步；即使批次较小，也能让计算单元保持忙碌，从而提高解码期间的 GPU 利用率。图 18-1展示了在 Hopper H100 GPU 上，MLA 相比于分组查询注意力（GQA）和多查询注意力（MQA）等实现的算术强度提升。（注：Blackwell 凭借更高 TFLOPs 和 HBM 带宽，会提高两条 Roofline 上限。）

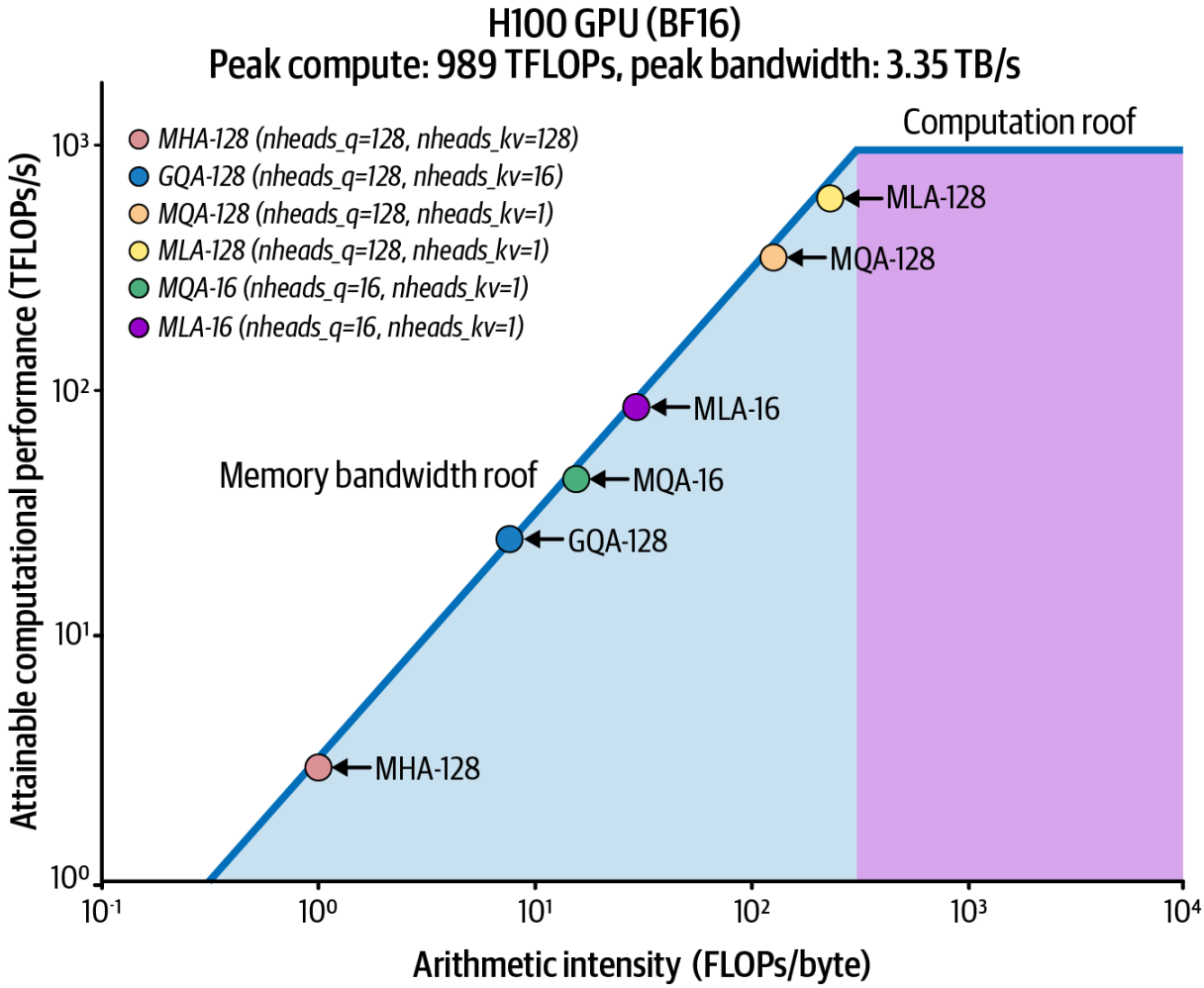


图 18-1. MLA 接近计算受限区域（在 NVIDIA Hopper H100 架构上测量）

FlashMLA 的意义在于，它表明解码阶段的内存带宽瓶颈和内核启动开销可以得到降低，即使在并非最优的 GPU 硬件上也是如此。它减少独立 GPU 内核启动次数并优化内存访问模式，尽可能从受限硬件中挖掘解码性能。

DeepSeek 已开源 FlashMLA 实现，SGLang 和 vLLM 都为 DeepSeek 模型提供支持。因此，可以评估 FlashMLA 是否能在不改变高层架构的前提下提高逐词元解码吞吐量。

由于 DeepSeek 的开源 FlashMLA 已被现代推理服务系统集成，可将其作为提高每个解码工作节点吞吐量或降低逐词元延迟的候选方案，同时无需进行高层架构变更。

ThunderMLA（Stanford）

在 FlashMLA 的基础上，Stanford 的研究人员提出了ThunderMLA：一种完全融合的注意力解码“超级内核”，侧重解码与调度，而非融合整个前馈块。这个“超级内核”将多个内核启动合并为一次，并整合中间内存写入，从而减少启动开销和尾部效应。
ThunderMLA 论文报告在不同工作负载下的解码吞吐量比 FlashMLA 快 20–35%；该数值取决于评测配置。

ThunderMLA 的核心思想是：解码不同长度的序列时，细粒度调度和融合操作可避免某些序列较早结束、其余序列导致 GPU 部分闲置的尾部效应。它通过动态打包并处理剩余流，使 GPU 在部分解码流提前完成后仍能保持忙碌。

在具有更大 L2 缓存和更快注意力原语的现代 GPU 上，这些收益可能更明显。现代 NVIDIA GPU 还通过 Transformer Engine 支持 FP8、FP4（以及 FP6；本书主要关注 FP8/FP4，因为 FP6 在现有 AI 框架和工具中尚未广泛使用）。结合更高内存带宽，Tensor Core 可使 ThunderMLA 等内核更接近硬件上限；但逐词元延迟的实际变化依赖模型、精度、内核和硬件配置。

FlexDecoding（PyTorch）

在第 14 章中，我们讨论过PyTorch 的 FlexAttention：无需编写自定义 CUDA，即可为局部窗口、块稀疏等任意注意力稀疏模式 JIT 编译融合内核。底层由 TorchInductor 和 OpenAI Triton 生成融合内核，只计算该模式允许的查询—键对。Triton 会在给定硬件上有利时自动使用 warp 专门化、异步复制等性能技术；也可以调优 `triton.Config`，例如配置 `num_consumer_groups` 进行进一步定制。

FlexDecoding 是 `torch.nn.attention.flex_attention` 的解码后端。它支持原地管理 KV，并像 FlexAttention 一样支持掩码和偏置。具体而言，FlexDecoding 为解码阶段（`Q_len=1`）编译专用内核，使其对不断增长的 KV 缓存执行注意力计算。

运行时，FlexDecoding 会选择专用解码内核并跨多个解码步骤复用它。形状和数据类型保持兼容时，这有助于减少开销，并可加速长序列 LLM 推理。

> 当重编译已受到控制时，对于稳定且延迟关键的解码，可优先采用 `torch.compile(mode="max-autotune")`。保持较窄的捕获边界（逐层或注意力块），以减少不规则批处理导致的图失效。预填充和解码可优先考虑 Transformer Engine FP8（MXFP8）；精度允许且性能确有提升时再考虑 FP4（NVFP4）。截至本书撰写时，FP4 支持仍在成熟中，短期内可能不如 8 位或 16 位格式。继续设置 `torch.set_float32_matmul_precision("high")`，以便对剩余 FP32 操作启用 TF32 回退。FlexAttention 的解码后端支持 GQA、PagedAttention 等常见性能增强。

FlexAttention 和 FlexDecoding 的一个关键特性，是支持嵌套锯齿布局张量（NJT）。它们允许在解码中对变长序列进行不规则批处理，这在 LLM 工作负载中很常见。图 18-2展示了多个序列的锯齿张量表示。

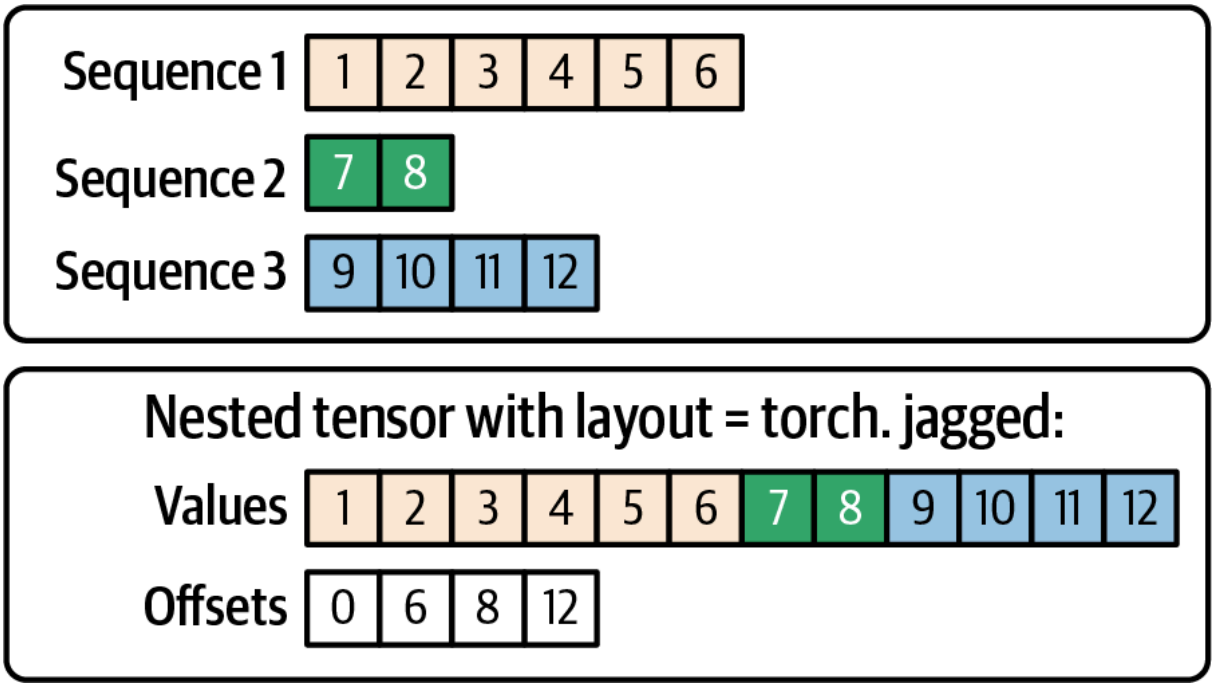


图 18-2. 将不规则批次表示为嵌套锯齿张量（偏移量）；三个序列（上）通过偏移量表示为单个嵌套锯齿张量（下）；解码时批处理可优先考虑 PyTorch NJT

此外，FlexDecoding 支持偏置项，并通过块掩码转换接口与PagedAttention 集成：该接口将逻辑块映射到物理缓存布局。如图 18-3所示，它将逻辑 KV 块分散到物理缓存布局中，而无需创建额外副本。

FlexDecoding 利用已捕获的张量在每次迭代中改变特定掩码或偏置值，而无需重新编译，并且可与 PagedAttention 集成。若要使用 vLLM LMCache 等全局 KV 缓存，可将缓存的页表映射到 FlexAttention 的 BlockMask，从而在线把逻辑 KV 页转换为物理内存地址。

借助 FlexDecoding，开发者可在 Python 层灵活定制注意力稀疏模式，这对 MoE 模型推理尤其有用。无需编写自定义 CUDA 内核，FlexDecoding 也能为任意注意力模式生成优化实现；实际性能接近程度仍需针对具体模式和硬件测量。随着新推理技术出现，这种能力会更有价值。

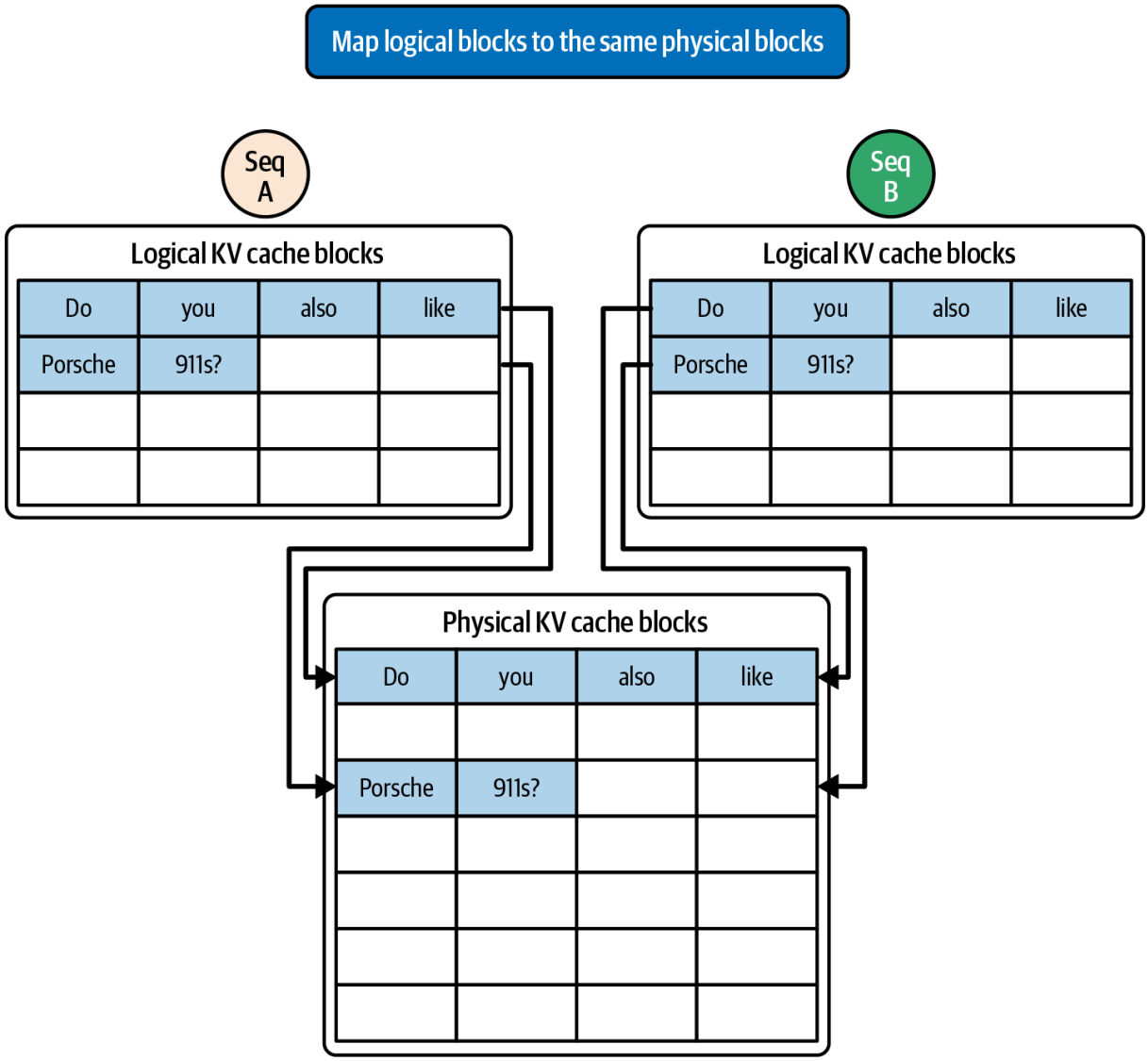


图 18-3. PagedAttention 将逻辑 KV 块分散到物理 KV 块中，以优化序列间的缓存复用；块大小应与 LMCache 页大小对齐——在解耦部署中，较大页（如 64–128 个词元）可能减少 RDMA 开销

这些能力中有许多——例如用于解码的融合注意力以及 PyTorch 嵌套锯齿张量（NJT）批处理支持——已在 PyTorch 核心库中提供。对于典型模式，这使自定义融合的必要性降低。

> 对 LLM 工作负载中常见的不规则序列进行批处理时，可优先考虑 NJT 布局。

这些内核级进展技术含量很高，能够充分利用 GPU、网络和内存。即使硬件不变，软件优化也可能显著改善解码性能。设计超大规模系统时，应在可行的情况下采用这些优化内核，并通过带硬件 CUDA 跟踪的 Nsight Systems 验证重叠情况和内核效率；另外使用 Nsight Compute 检查具体内存和互连指标。

启用其中一些高级内核可能需要安装自定义库或启用专用 CUDA 内核，尤其是新技术。PyTorch 和流行推理引擎通常会在后续版本中支持这些技术；即使需要安装额外组件，也应先基准测试验证收益，因为较低延迟或更少解码 GPU 的实际结果取决于具体工作负载。

调优 KV 缓存利用率与管理

解耦要求把KV 缓存视为跨集群的一等共享资源。由于 KV 缓存现在可以存活更久并在节点之间移动，高性能推理系统也改进了 KV 缓存的存储和共享方式。

尤其值得关注的是分布式 KV 缓存池和跨请求的前缀复用。此外，还应关注新一代 GPU 和 HBM 带来的内存带宽提升。下面在改进 KV 缓存性能的背景下讨论这些技术。

解耦的 KV 缓存池

解耦 KV 缓存池不再让每块 GPU 只保存其当前服务请求的KV，而是将 KV 存储从单个 GPU 解耦，并把数据分布到整个集群的 GPU 内存中。

该缓存池还可以卸载到 CPU 内存，包括 Grace Blackwell 和 Vera Rubin 平台的统一 CPU/GPU 内存，也可卸载到 NVMe SSD 等持久化存储。

使用解耦 KV 缓存池时，预填充为提示词计算 KV 张量，或解码扩展 KV 张量后，KV 块会以分布式方式存储在多个计算节点之间。图 18-4展示了这一机制，改编自关于解耦 KV 池的研究。

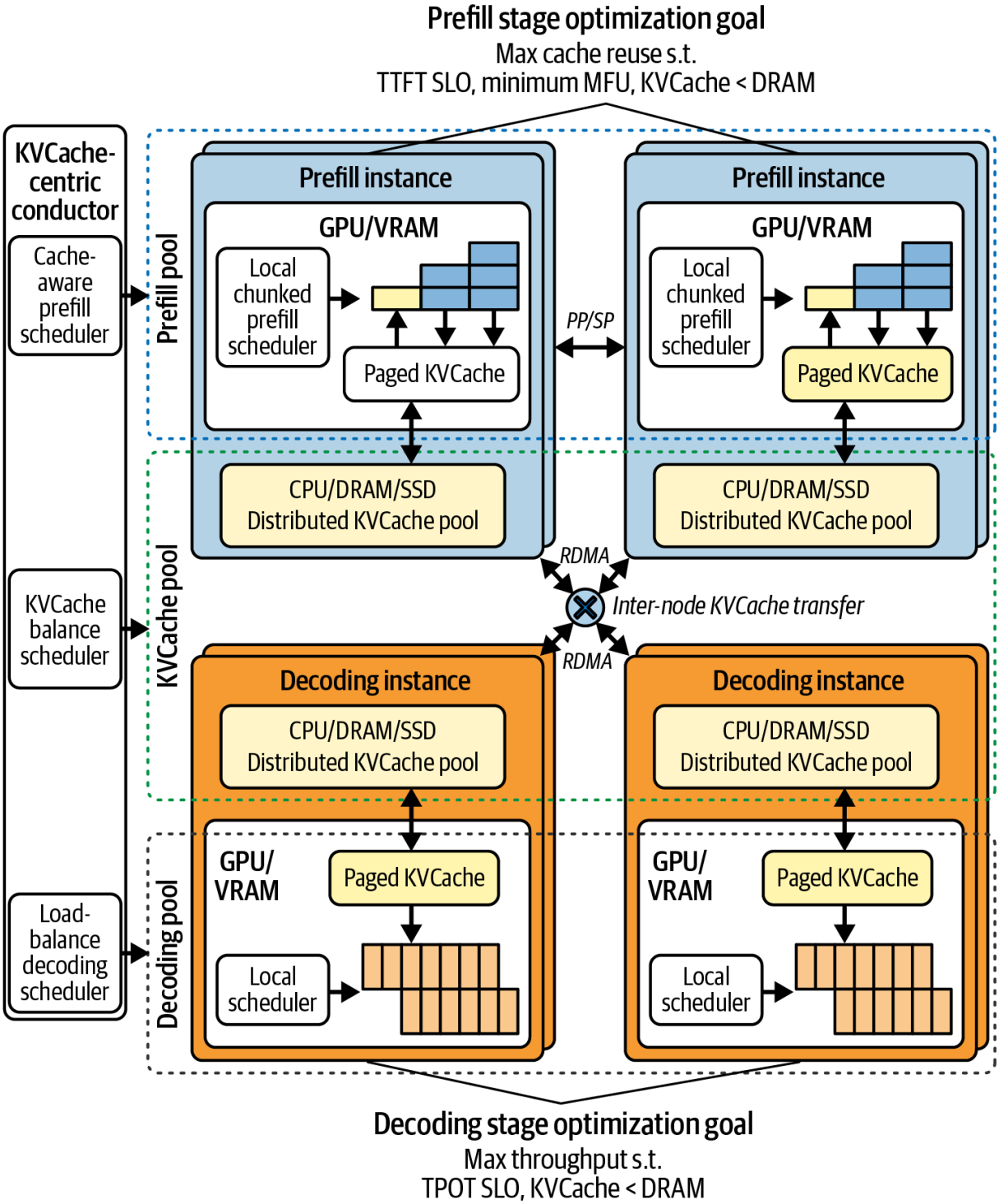


图 18-4. 解耦（分布式）KV 缓存池（来源：https://oreil.ly/2xtK-）

考虑一个极长的 250,000 词元上下文（例如多轮聊天会话），使用一个 700 亿参数、80 层、32 个注意力头且每个头维度为 128 的 Transformer 模型。这会产生极大的每词元 KV 缓存占用。

每个词元都会生成长度等于模型隐藏维度（`num_heads` × `head_dim`）的键和值向量。假设本例模型的隐藏维度为 4,096，则每层产生 8,192 个浮点数；累加 80 层后，每词元产生 655,360 个 KV 浮点数。假设使用 16 位精度，即每个浮点数 2 字节，那么每词元约需 1.31 MB，扩展到 250,000 个词元约为 328 GB——这还只是 KV 数据本身。

这些计算基于大模型的每词元 KV 大小，说明了为什么 vLLM 等引擎会采用 FP8 KV 缓存来减少占用并增加批处理机会。

假设将 KV 缓存量化到 FP8，并使用选择性层缓存等技术，这个 250,000 词元提示词的占用可降至约 100–150 GB；具体取决于模型结构和缓存策略。单块 GPU 很可能无法同时容纳所有词元的 KV、模型权重和其他运行时数据，尤其在多轮对话继续增长时。因此，系统可能需要截断上下文，或对早期词元执行代价高昂的 KV 重计算。

使用解耦 KV 池后，上下文中较旧部分的 KV 可以从 GPU 驱逐到分布在集群中的 KV 缓存池、CPU DRAM 或 NVMe 存储；需要时再调回 GPU 内存。

解耦 KV 缓存池实现了多层内存层级：GPU 设备内存保存活跃 KV 缓存，CPU 主机 RAM（或 NVMe 存储）作为溢出后备存储。现代推理引擎可选择把 KV 缓存卸载到 CPU 内存或 NVMe，这在概念上类似操作系统虚拟内存子系统对 GPU 内存进行扩展。

这一设计可通过在 GPU 内存和 KV 缓存池之间异步换页 KV 块，支持超长上下文；前提是通信与计算能良好重叠，否则换页仍可能使计算流水线停顿。

此外，将请求状态从单个 GPU 解耦后，系统可利用全局 KV 缓存池在集群多个节点之间动态分片 KV 数据，以自适应平衡负载。这可简化扩展并改善大型推理集群的故障隔离。

由于任何解码节点都能访问全局 KV 池，发生故障转移或负载均衡需要时，任何解码节点都可参与解码任意请求。这为调度器增加了灵活性，使其可以选择最接近相关 KV 缓存块的解码节点。

如果某个前缀的部分 KV 块缓存在服务器 A 的 DRAM 中，在服务器 A 上调度解码可能更快，因为它能迅速将数据调入本机 GPU。相比之下，服务器 B 必须经由网络获取这些 KV 块。

这体现了分布式系统的一项经典最佳实践：选择最接近待计算数据的计算节点，从而最小化昂贵的数据移动。

高效的 KV 缓存调度器可以结合网络拓扑，查看池中 KV 块的分布，并相应分配预填充和解码任务。因此，预填充节点可将 KV 数据放入实现为跨集群可访问分布式内存空间的缓存池。

当 KV 缓存位于集群侧共享内存空间时，任何解码节点都能检索数据，避免每次都必须安排预填充到解码的直接传输。

从缓存池检索 KV 数据会增加一次额外跳转的开销，但由于所有解码节点都能访问全部 KV 缓存数据，系统更加灵活。这也意味着未直接接收某次预填充数据的解码节点，仍可在需要时从池中访问相关 KV 数据。

如果解码节点崩溃，或请求因任何原因需要在生成中途迁移，KV 数据不会随节点丢失。数据保存在缓存池中，另一个节点可以接手并使用已保存的 KV 从中断处继续，这提高了容错能力。

全局 KV 缓存池还提供跨请求的缓存持久性。若两个请求共享某段前缀，该前缀的 KV 可以只计算一次并在集群中复用，即使两个请求最终落在不同解码服务器上。

总之，解耦 KV 缓存池以内存（或更冷存储）换取计算：通过保存更大的 KV 缓存，系统可在许多场景避免重算 KV 数据。其依据是，即使从 DRAM 或 SSD 复用数据，在不少情况下也比反复执行具有二次时间复杂度 $O(N^2)$ 的大型注意力矩阵乘法更便宜；具体取决于数据传输、缓存命中与重计算成本。

KV 缓存复用与前缀共享

如前所述，对于共享公共前缀的提示词，跨请求复用缓存 KV 数据很有价值。这种场景常见于多轮对话、共享系统提示词和附加文档。

系统无需为每个请求重算该前缀的 Transformer 注意力输出，而是存储前缀 KV 输出并直接复用。本质上，这跳过了对应输入部分的预填充计算，从而节省时间和 GPU 周期。

以 KV 缓存为中心的调度器在分配工作时，会考虑“前缀缓存命中长度”，即该提示词已有多少词元存在于缓存池中。实际中，如果新请求前 N 个词元与 KV 池中的某个缓存前缀完全匹配，系统可决定复用该 KV 数据。

vLLM 通过其 PagedAttention 机制，以 KV“页”为单位的全局哈希表实现自动前缀缓存。按本书所述配置，每个唯一的 16-token 上下文块都有一个哈希值。若新请求需要的前缀与已存储块按哈希匹配，系统即可直接复制对应的 KV 张量，无须重新计算。

当同一上下文再次出现时，系统可从内存中提供它。也就是说，它将上下文的 KV 视为可按内容经由哈希查找的可复用数据。实现通常维护一棵全局“提示词树”来管理这些缓存的上下文，并在必要时将其驱逐；这会优先优化最常被复用的前缀。

有效复用 KV 的关键，是识别相同或重叠的前缀。为简化实现，系统通常着重于精确匹配：若前 N 个 token 完全一致，便复用该片段。组合部分重叠的前缀更复杂，因为需要以并不总是直接的方式合并缓存。因此，典型做法是精确前缀缓存。

不过，这也存在权衡：无限期保存许多用户的 KV 缓存会消耗大量内存。系统须为 KV 块实现 LRU 等驱逐策略，丢弃不太可能再次使用的缓存，以腾出空间给新缓存。调度器也可以依复用概率决定保留哪些缓存；目标是在内存限制内最大化缓存命中。

若某个预填充节点已在本地 GPU 内存或本地 DRAM 缓存中保存了所需 KV 的一部分，将请求路由到该节点可能有利于减少数据传输。这是数据感知调度的例子：将计算送往数据所在处，而不是总把数据拉到任意有可用算力的位置。

这类似于分布式系统中的局部性感知调度。前面的路由讨论已提及这一点：如有可能，应将请求路由到生成其前缀的服务器，以提高缓存命中的概率。

在更广义的解耦架构中，前缀缓存依赖于跨众多请求的统一 KV 视图，并可将其存入全局池等可共享位置；这不同于按请求或按节点彼此隔离的方案。

这也能减少解耦架构本可能产生的重算开销：例如同一提示词在不同时间到达不同节点。通过全局 KV 存储或协调式缓存，即使某个用户的请求命中不同的 decode 服务器，也能受益于彼此已缓存的工作。

优化 KV 缓存内存布局

另一个底层创新方向是优化 KV 缓存的内存布局。KV 缓存保存每个序列中所有历史 token 的键和值；当并发 decode 流很多时，其容量可能非常大，因为每个流的内存用量大致与 `num_layers` × 2 × `sequence_length` × `d_head` 成正比。

分层缓存等技术很有用，因为并非所有 KV 对都须始终保留在 GPU 内存中。较旧的 KV 缓存片段可换出到 CPU，甚至进行压缩。

鉴于我们强调维持较低的 decode 延迟，多数设计会将活跃 KV 缓存保留在 GPU 内存中以便快速访问；在这种情况下，可调优内存的布局和访问方式。

DeepSeek 的 FlashMLA 会对 KV 缓存分页，并将缓存分配为固定大小的块（页），使活跃序列能够连续访问内存，从而减少缓存未命中和 DRAM 流量。

此外，若提示词前缀已不再被注意力机制访问，例如上下文窗口已移动，一些系统会实施前缀压缩。此时，KV 缓存管理器可能丢弃或压缩这些 KV 条目。这在上下文窗口会滑动的长对话中更相关；对极长序列而言，它可节省内存和带宽。

> 当模型采用滑动窗口或其他受限注意力模式时，这种驱逐/压缩技术是安全的；但对于仍会在完整内容窗口上执行全注意力的层（或检索钩子），未经谨慎评估不应采用它。

另一项称为 *POD-Attention* 的技术，也会重组注意力计算以降低 HBM 流量。具体而言，它采用 SM 感知的线程块（或协作线程阵列，cooperative thread array [CTA]）调度。它实现了*运行时操作绑定*，将运行在某个 SM 上的每个 CTA 动态分配为执行 prefill 或 decode 任务，如图 18-5所示。

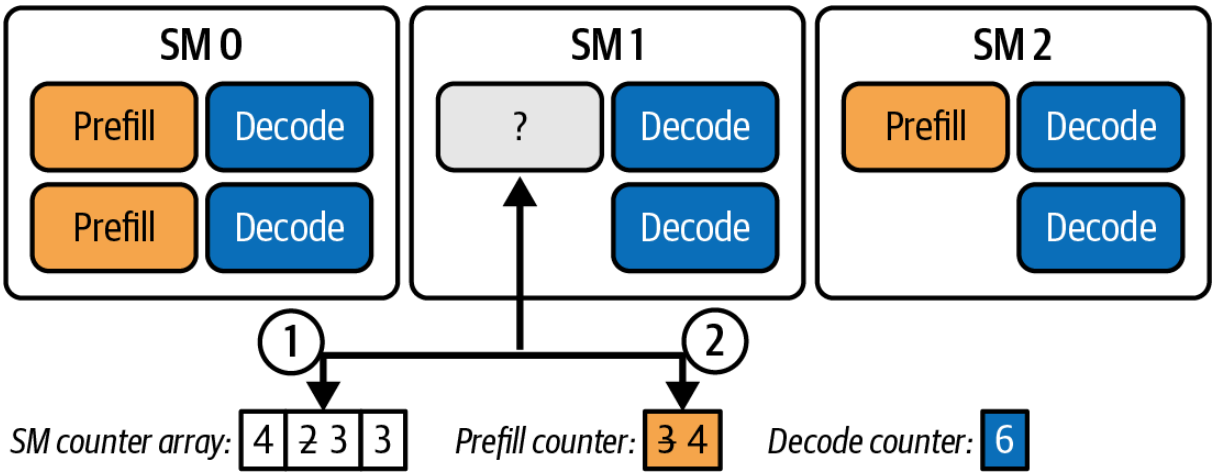


图 18-5. 将 SM 上的 prefill 任务与 decode 任务配对的 SM 感知线程块（CTA）调度，以减少内存移动

因此，系统不是为每个阶段静态启动独立内核，而是由单个内核启动足够的 CTA 来覆盖两类工作负载。运行时，每个 CTA 检查自己所在的 SM，并利用每个 SM 的计数器，根据该 SM 上正在运行的其他工作决定执行哪一种操作（prefill 或 decode）。

SM 感知调度逻辑会尝试在运行时配对 prefill 与 decode 操作，以避免孤立的内存流量突发，并平滑资源需求。

具体来说，POD-Attention 在同一 SM 上共置 prefill 与 decode 工作，使融合内核改善局部性、减少冗余 HBM 事务。这样可减少内存移动、提高带宽利用率，并在每个 SM 上平衡计算受限与内存受限的工作负载。相关论文在其 SM 感知 CTA 调度、同 SM 共置两类工作以充分重叠的评测条件下，报告注意力性能最高约提升 29%；实际结果取决于模型、内核、硬件和工作负载。

POD-Attention 的动态绑定将硬件的 CTA-SM 分配与软件的 CTA 角色分配（prefill 或 decode）解耦。这类创新体现出越来越重视通过软硬件协同设计来减少内存移动，并尽可能发挥系统性能。

GPU 与 CPU-GPU 超级芯片的改进

还应考虑新硬件中内存带宽的提升。更高的内存带宽和更大的 L2 缓存会直接改善内存受限的 decode 阶段性能。

NVIDIA 的 Grace Blackwell GB200 NVL72 系统是一个机架级平台，配有 36 颗 Grace CPU 和 72 块 Blackwell GPU，可使一个逻辑 decode 单元拥有数十 TB 的 KV 缓存内存。该硬件约有 30 TB 的统一内存，适合将非常大的上下文保留在内存中；具体可容纳的上下文可能达到数百万 token 的量级，但取决于模型 KV 大小、精度、缓存策略与可用资源。

此类平台的统一内存容量很大，但对于延迟关键的 decode，活跃键和值仍应驻留在 GPU HBM 中。因此，应将 Grace CPU 内存（LPDDR5X，并非 HBM）用作较低层级的缓存，或存放很旧的 token。即使在 NVL72 这样的系统上，当上下文超过可用 HBM 时，prefill 和键值卸载仍然重要。

简言之，宏观层面的解耦应与微观层面的优化相结合，才能充分发挥推理性能。FlashMLA/ThunderMLA 等高级 decode 内核、高效内存布局（如分页缓存）及最新 GPU 架构，共同有助于实现高效且可扩展的 decode。

在 Prefill 与 Decode 之间快速传输 KV 缓存

解耦式推理的一项关键要求，是将 KV 缓存从 prefill worker 快速、高效地传给 decode worker。若传输不够快，prefill 与 decode 并行所节省的时间可能都会耗在等待数据移动上。

本节讨论用于降低传输开销的技术，并说明系统如何借助高速互连、避免额外 KV 复制来实现这一交接。

KV 缓存大小

prefill 的输出主要是所有提示词 token 的 KV 缓存，数据量可能很大。设模型有 L 层，每层有 h 个维度为 d 的注意力头，提示词有 N 个 token，则 KV 缓存大小约为 2 × L × N × (h × d)，其中 2 代表键和值两者。

实际大小取决于精度（如 FP16 或 INT8）和模型细节，但通常很大。例如，一个有 40 层、16 个大小为 64 的头、提示词长度为 1,000 token 的模型，会产生约 40,000 个 KV 向量；其数据量可能达到数百 MB。若 token 数为 5,000，则约大 5 倍。

若采用朴素方式，经由网络传输这些数据会引入显著延迟。例如，先将 prefill worker 上的 KV 复制到 CPU 内存，再通过 TCP 发送，甚至写入磁盘供 decode 进程加载；对于大型提示词，这可能慢至数百毫秒量级。目标是将传输时间降至数毫秒，使 prefill 与 decode 能真正并行重叠。

> 要获得低延迟 KV 数据传输，通常须将小型 PagedAttention 块拼合成较大缓冲区，并通过基于 GPUDirect RDMA 的路径而非 CPU socket 传输。

零拷贝 GPU 到 GPU 传输

现代解耦式系统采用零拷贝 GPU 到 GPU 传输技术。实际中，这通常是通过高速网络使用远程直接内存访问（RDMA）。例如，可使用 InfiniBand 在机架/节点之间传输，也可在单节点多 GPU 平台内利用 NVLink/NVSwitch 直接写入 GPU 内存。这些方法在 GPU 间直接传送数据，不经由 CPU 内存复制。

NVIDIA 用于推理的高性能 GPU 到 GPU 传输库名为 *NVIDIA Inference Xfer Library*（NIXL）。NIXL 为零拷贝 GPU↔GPU 和 GPU↔存储的数据移动提供插件式架构，例如 NVLink、UCX fabric 和 GPUDirect Storage。

NIXL 简化了 RDMA 风格的传输，使一块 GPU 能通过可用高速 fabric（例如 InfiniBand 或基于 NVLink 的连接）直接写入另一块 GPU 的内存。换言之，prefill GPU 可将 KV 张量直接注入 decode worker GPU 的内存。

基于 RDMA 的协议会绕过 CPU，并充分利用 GPU 互连带宽。NVIDIA Dynamo，以及开源 vLLM 与 LMCache 的集成均可使用 NIXL；具体路径和是否启用取决于部署、版本与配置。现代 GPU 互连提供很高带宽；1 GB 的传输在链路类型与争用不同的情况下，可能需要数毫秒至数十毫秒。

在实践中，实现会将数据传输与计算重叠以获得较低传输时间。例如，使用 RDMA 时，prefill worker 可异步地把 KV 数据写入 decode GPU 的内存缓冲区，同时 decode GPU 继续为其他序列生成 token。按设计不同，prefill 可推送数据（RDMA write 的 push 模式），也可由 decode 拉取（RDMA read）；两种路径的数据面都不需要 CPU 参与。

快速 KV 传输的常见策略包括 prefill 端推送、decode 端拉取、共享内存（CUDA IPC）缓冲区、连接器/队列抽象及非阻塞重叠。下面逐一讨论：

Prefill 端推送

> prefill worker 完成提示词处理后，启动一次 RDMA 写入，将 KV 数据直接写入 decode worker GPU 上预留的缓冲区。这可以非阻塞地执行：prefill 启动传输后可继续其他工作，DMA 则在后台进行。

Decode 端拉取

> 或者，decode worker 准备开始 decode 时，可直接从 prefill GPU 内存执行 RDMA read。推送与拉取都能达到同一结果（无 CPU 复制）。有些实现偏好推送，以将协调工作交给发送端；另一些则偏好拉取，以便由接收端控制时机。

共享内存（IPC）缓冲区

若 prefill 和 decode 恰好位于同一机器（同一服务器中的不同 GPU），它们可使用 CUDA 进程间通信共享内存句柄，甚至共享 PCIe BAR，从而在同一主机上借助 NVLink 或 NVSwitch 进行有效传输。这是不经网络的零拷贝传输本地变体。

连接器/队列抽象

vLLM 的实现将传输机制抽象在逻辑接口（Pipe 或 LookupBuffer）之后。prefill 进程将 KV 放入该缓冲区或通知其可用，decode 端再取得它。底层可使用 RDMA，甚至使用高性能发布-订阅消息（Dynamo 中的 NATS 用于控制信号）。关键在于将逻辑交接与传输层解耦，以便插入 RDMA、共享内存等不同传输方式。

非阻塞重叠

如前所述，优化后的系统会将 KV 传输与正在进行的 decode 计算重叠。例如，Dynamo 的 decode worker 可在 prefill worker 为新请求把 KV 写入其 GPU 内存时，继续为其他请求生成 token；这会隐藏大部分传输延迟。因此，在合适的工作负载和重叠条件下，可将约 5 ms 的 KV 传输与 decode 计算重叠，对该请求首个生成 token 增加的净延迟很小。

采用这些方法时，KV 传输可能只需数毫秒量级，远低于 prefill worker 实际计算这些 KV 所需的数百毫秒。因此，prefill → 传输 → decode 流水线可获得良好并行性：prefill 完成后，decode 能几乎立即开始，无须长时间停顿。

发送 KV 缓存数据时，须注意避免碎片化和额外开销。例如，vLLM 的 PagedAttention 将 KV 缓存存为固定大小的 token 块，常见配置是每块 16 个 token。KV 块相对较小（但每块字节数随头数、头维度、层数和数据类型而变）。若通过 RDMA 朴素地发送数千个小 KV 页，会因每次传输都有固定延迟和协议开销而产生过多开销，导致带宽利用率低。

> 现代 LLM 引擎支持多种页大小，例如每块 8、16、32、64 或 128 个 token。经 RDMA 移动 KV 时，较大的页可降低传输开销，因为更大的拼合缓冲区和更少的工作队列元素（WQE）能提高持续链路吞吐。条件允许时，每次 RDMA 写入应拼合至少 128-token 的页；务必在专用 CUDA stream 上重叠传输，优先使用非阻塞 stream 和事件栅栏，并用 Nsight Systems 等工具验证重叠。LMCache 报告，在其 RDMA 评测中，拼合后 7.5k-token 的 KV 传输时间约从 20 ms 降至 8 ms。

LMCache 扩展会在传输前将 KV 页拼合为大块连续缓冲区，从而解决这一低效问题。实质上，它在 GPU 内存中把小块聚集成一个大块，然后一次传输该大缓冲区。

例如，若把一个 7,500-token 的 KV 缓存拆成 470 次小传输需要 20 ms，拼合为较大块（如 128-token 页）可将传输时间降至约 8 ms。这一简单的批处理优化能让网络链路保持满载，并降低每个数据包的开销。

下面展示系统如何配置快速 GPU 到 GPU KV 传输。这是 LMCache 的 prefill-decode 模式使用 NIXL 传输通道的一个配置示例：

```yaml
# Prefill server config (lmcache-prefiller-config.yaml)
enable_pd: true
transfer_channel: "nixl"
pd_role: "sender"              # this instance sends KV data
pd_proxy_host: "decode-host"   # PD proxy / decode coordinator
pd_proxy_port: 7500            # control-plane port on the proxy/decoder
# size the buffer to the KV you plan to transfer
# FP8/FP4 KV should shrink it significantly
pd_buffer_size: 1073741824     # 1 GiB transfer buffer size
pd_buffer_device: "cuda"       # buffer stays in GPU memory
```

这里，prefill server 被配置为 RDMA 发送端，使用为 KV 传输分配的 1 GB GPU 缓冲区，目标为 decode host 的 7500 端口。如下所示，decode server 被配置为该端口的接收端，并配有匹配的 1 GB GPU 缓冲区：

```yaml
# Decode server config (lmcache-decoder-config.yaml)
enable_pd: true
transfer_channel: "nixl"
pd_role: "receiver"            # this instance receives KV
pd_peer_host: "0.0.0.0"        # bind address for NIXL peer
pd_peer_init_port: 7300        # NIXL handshake/control port
pd_peer_alloc_port: 7400       # NIXL allocation/data port
pd_buffer_size: 1073741824     # 1 GiB (match sender unless you plan to shard)
pd_buffer_device: "cuda"       # keep buffer in GPU memory
nixl_backends: [UCX]           # UCX backend is sufficient for disagg
```

该配置使 prefill 可将 KV 缓存直接写入 decode GPU 内存，每次最多 1 GB，不需 CPU 干预。两端均将传输缓冲区保留在 GPU 内存中，以支持零拷贝操作。

确定传输缓冲区大小时，可从 `pd_buffer_size = 1 GB` 开始。对具有 700 亿参数、80 层、32 个头和 128 维头的模型，这大致对应 FP16 KV 缓存的约 4–8k token；若提示词超过约 7.5k token，可使用 2 GB。可按数据类型和头数缩放：字节数 $\approx 2 \times L \times N \times (H \times Dh) \times$ `bytes_per_val`。传输前应拼合页面，以避免小 I/O 的低效问题。

> 若将 KV 缓存量化为 FP8 或 FP4，固定 token 数所需的传输缓冲区会随每个 token 的字节数减少而变小。因此，既可在每个缓冲区传输更多 token，也可相应缩小缓冲区。1–2 GiB 缓冲区适用于许多部署，但应依上面的 KV 公式计算，并向上取整到 256 MB 边界。使用 FP8 或 FP4 KV 时可按比例缩小；务必以将要传输的最大拼合页组验证。为获得最佳链路利用率，优先使用 GPUDirect RDMA，并拼合至至少 128-token 的页。

实践中，可用以下 shell 脚本通过 CLI 启动 decode server。这会减少 eager 模式的碎片化，并促使较大缓冲区采用 rendezvous：

```shell
# Example decode worker
# (select device by index or UUID)

UCX_RNDV_THRESH=16384
UCX_MAX_EAGER_RAILS=1
UCX_TLS=cuda_ipc,rc,rdmacm,cuda_copy,cuda_ipc,tcp \
CUDA_VISIBLE_DEVICES=1 \
LMCACHE_CONFIG_FILE=lmcache-decoder-config.yaml \
python run_vllm_decoder.py --port 8200
```

同样可在另一块 GPU 上用其配置文件启动 prefill server。这些设置旨在使系统跨节点传输时使用 InfiniBand RDMA，或在节点内使用 NVLink peer-to-peer，而非通过标准 TCP socket 传输 KV。

> 对于单节点多 GPU 运行，应启用 CUDA IPC；跨节点运行时则优先使用 RDMA。LMCache/vLLM worker 的典型 UCX 配置是设置 `UCX_TLS=rc,rdmacm,cuda_copy,cuda_ipc,tcp`，并确保在 fabric 上应用 RoCE/IB 无损设置（ECN/PFC）。对于节点间 RDMA，可考虑 `UCX_RNDV_THRESH=16384`，使大型 KV 缓冲区使用 rendezvous，小型 KV 缓冲区使用 eager。务必用 `ucx_info -f` 验证。

部署 RDMA 与恰当缓冲后，交接延迟可处于个位数到数十毫秒的范围，具体取决于互连和页大小。例如，LMCache 在 7,500-token 上下文的评测中，许多小传输约需 20 ms，而拼合为较大块后约需 8 ms。具体而言，建议在 RDMA 前将 16-token 页拼合为至少 128-token 的 slab，以减少每数据包开销。

简言之，解耦式系统应使用高速互连和合理的数据拼合，使 prefill → decode 的切换顺畅且迅速。最小化交接时间至关重要，因为交接若过慢，便会抵消阶段并行化原本带来的收益。

> 在多进程运行中，可通过设置 `export PYTHONHASHSEED=0`，为 KV 块路由使用确定性的哈希。

连接器与数据路径设计

在零拷贝优化的基础上，下面考察 prefill 与 decode 节点如何端到端协调传输，而不只是移动比特。prefill 和 decode worker 往往经由调度器或路由器通信。实践中，调度器可实现为中心化组件（如 NVIDIA Dynamo），也可采用去中心化协调方式（如 SGLang）。

例如，NVIDIA Dynamo 实现了一个全局调度队列：decode worker 将新的提示词任务推入该队列，由 prefill worker 消费。在这一设计中，decode 节点将提示词处理请求入队，如图 18-6中的“Put RemovePrefillRequest”（第 6 步）所示。

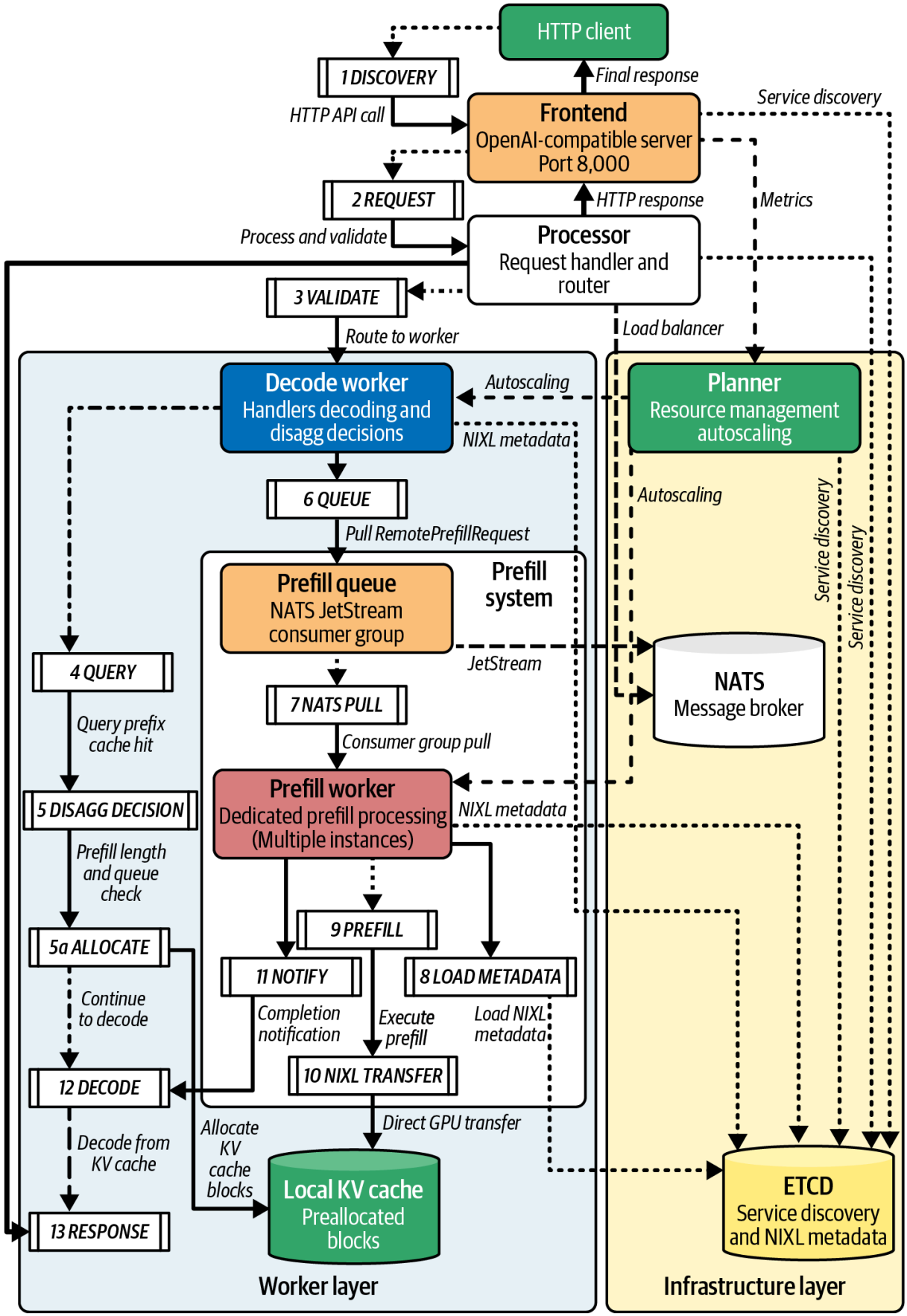


图 18-6. NVIDIA Dynamo 中的请求生命周期：decode 拉取提示词，prefill 使用 NIXL 推送 KV（改编自 NVIDIA Dynamo 架构文档）

prefill 节点取得该请求后，完成时会因请求携带了 decode 节点的 origin 或 reply-to ID，而准确知道应将结果发送给哪个 decode 节点。随后使用 NIXL RDMA 将 KV 直接传给该 decode worker 的 GPU。

vLLM + LMCache 的实现采用了更去中心化的方式。decode 和 prefill 进程会为每个请求的 KV 通过 pipe 或 buffer 建立直连通道；底层可能在请求开始时协商一对一 TCP 或 RDMA 连接。每个请求建立自己的传输通道，而不是使用全局队列。两种方式各有利弊：全局队列更便于负载均衡和故障处理；直连通道可减少排队。

决定采用哪种模式时，应考虑工作负载和基础设施约束。若需要健壮的多租户负载均衡和易于故障切换，并可接受少量排队延迟，通常更适合全局队列模型。反之，若对尾延迟要求严格，decode-prefill 配对相对稳定且拥有高速互连，则按请求的直连通道方案可减少跳数和抖动。

> 实践中，应在预期的请求混合下对两种设计都进行基准测试。改变提示词长度、并发程度和故障情景，找出最符合 SLO 的延迟-吞吐量权衡。

关键设计目标是使流水线非阻塞且具有高吞吐：一个请求正在 decode 时，另一个提示词可开始 prefill，同时还有第三个请求的 KV 正在传输。只要其他阶段仍有工作，任何阶段就不应空闲。这正是解耦能在大规模下提升整体吞吐的原因——所有阶段并行保持忙碌。

> 通常不会显式传输 prefill 得到的第一个生成 token 的 logits，因为 decode worker 可直接根据 KV 重算第一个 token 的概率。一些系统会传输第一个 token 的输出，以省去 decode worker 数百微秒的额外计算；另一些系统则只传输 KV，并让 decode worker 重算最后一层，以保持实现简单。

必须确保该流水线能抵御故障。若 decode 节点在生成中失败，前面讨论的全局 KV 缓存池可让另一节点利用保存的 KV 从中断处接续；若 prefill 节点在处理提示词时失败，也可在其他位置重试该提示词。连接器设计应妥善处理这些情况，避免单个节点的失败导致整个请求报错。

> 这些路由器通常使用心跳检查和超时机制；若 prefill 到 decode 的传输停滞，便可重新分配请求或安全中止。

用于 Prefill 和 Decode 的异构硬件与并行策略

解耦的一项强大优势，是可分别为 prefill 与 decode 集群选择最适合的不同硬件，甚至不同的模型并行配置。在统一的单体部署中，两个阶段通常只能使用同一种硬件和配置；解耦后，则可按阶段混搭硬件和策略，下面将作说明。

计算优化型与内存优化型硬件

prefill 阶段受益于具备高计算吞吐、大量 TFLOPS、专用 Tensor Core 和高时钟频率的 GPU。它也从充足的内存带宽中获益，但不一定需要超过提示词 KV 缓存需求的超大 HBM 容量。

相对地，decode 阶段要处理许多 token 对应的 KV，因此同时受益于大内存容量和高内存带宽。它不一定需要极高的算力，但更多算力仍可能有帮助。

这使得两个阶段可使用不同代际的 GPU。例如，一种设计可为 prefill 集群使用最新的高算力 GPU，而为 decode 集群保留具备足够内存带宽的较旧或更具成本效益的 GPU。

这样可避免将最新 GPU（例如最新 Tensor Core）浪费在无法充分发挥其潜力的 decode 任务上。prefill 往往会使 GPU 数学单元满载、功耗更高；decode 在同一 GPU 上的功耗通常低得多。

吞吐量和成本收益

将阶段分配到异构硬件可改善单位成本吞吐量和单位功耗吞吐量。在 Splitwise 研究中，使用阶段专用硬件的一项配置，相对同构基线实现了 1.4 倍吞吐量和低 20% 的成本。

在另一项固定成本/功耗预算下追求最高性能的配置中，研究报告以相同成本和功耗实现了 2.35 倍吞吐量。具体而言，该研究为 prefill 使用 4× H100（高算力），为 decode 使用 4× A100（大内存）；这种混合配置在其评测条件下达到同成本/功耗的 8-GPU 同构系统（全 H100 或全 A100）约 2.35 倍 RPS。

或者，为匹配基线吞吐量，研究发现可将 decode 卸载至更便宜的 GPU，从而让异构系统整体使用更少 GPU（如五或六块，而非八块）。这突出了节省成本的可能性，以及在最适合的地方使用每类 GPU 的价值。具体地说，可将计算受限工作置于每美元算力更高的 GPU（例如 Blackwell 或 Rubin 代），将内存受限工作交给具备合适内存带宽、更具成本效益的旧代 GPU（例如 Hopper 或 Ampere）；实际选择仍取决于价格、可用性和负载。

Splitwise 的评估计入了异构 GPU 间状态传输开销。其测试通过 NVSwitch fabric 传输 KV 数据，即使跨不同 GPU 代际，开销也很小。该结果表明，在相应测试条件下，NVSwitch 和 NVLink 等高带宽互连可支持 prefill/decode 解耦，即使混合 GPU 部署也可能只带来很小的性能影响。

另一个系统是 HexGen-2：它是一个分布式推理框架，将异构 GPU 上解耦式推理的资源分配视为优化问题。其调度器联合优化资源分配、每阶段并行策略和通信效率。

在 Llama 2 70B 等模型的实验中，HexGen-2 相比相同价格点的当时先进系统，报告服务吞吐最高提升 2 倍（平均约 1.3 倍）；同时，以约低 30% 的成本达到高端基线相近吞吐。这些改进来自混合 GPU 类型和优化工作划分，本质上是将 Splitwise 的概念做成自动化方案。结果受模型、硬件、价格假设和工作负载制约。

这些结果说明，解耦不仅关乎速度，也关乎效率，即用更少资源完成更多工作。在云环境部署推理时，对于服务数百万乃至数十亿终端用户的大型推理服务，若成本结构和利用率合适，潜在节省可非常可观。

例如，若某一实际负载可由 6 块 GPU（prefill 与 decode 混合）而非 8 块顶级 GPU 满足，服务的硬件成本可能节省约 25%。因此，解耦可让相同硬件服务更多用户；这在 GPU，尤其最新 GPU 供应有限时很重要。

鉴于包括美国在内的一些地区存在供电约束，能效也很重要。Splitwise 展示了将 decode 任务运行在低功耗 GPU 上可获得更佳能效，但速度会略有下降。

通过将 prefill 和 decode 分配给不同硬件类型，可分别决定每个阶段在何处、以何种方式运行，从而在性能与成本间优化；阶段相互独立正是解耦提供这种灵活性的原因。

简言之，Splitwise、HexGen-2 和相关异构部署研究的评测表明，解耦除追求纯速度外，也可用于成本优化。将硬件与工作负载相匹配，可在固定预算内降低单次查询成本并提高性能；实际收益需以目标集群和请求分布的测量为准。

对大规模服务而言，这对于保持经济可行性十分重要。代价是系统复杂度上升，因为必须管理多种 GPU；又因 GPU 能力不一致，集群配置的灵活性和在 prefill/decode 任务间动态重分配 GPU 的能力会受限。但在许多场景，按阶段使用不同硬件所获得的效率提升可能值得这些代价。

阶段专用的模型并行

异构性和按阶段专门化的另一种形式，是为每个阶段在 GPU 间选择不同的模型并行方式（如张量并行、流水线并行等）。这对因内存约束必须跨 GPU 切分的超大模型尤其相关。

在传统设置中，可能使用固定并行策略，并在 prefill 和 decode 两个阶段都用张量并行或流水线并行将模型切分到多个 GPU；但 prefill 的最优并行策略未必与 decode 相同。

例如，prefill 阶段是对 *N* 个提示词 token 的一次大型前向传播，受益于较高并行度。可跨多块 GPU 使用张量并行（TP）加快计算并降低 TTFT。

GPU 同步的开销会分摊到一次可处理的大量 token 上，从而降低该阶段的实际用时；这对 TTFT 至关重要。

还可使用流水线并行（PP）进一步加快 prefill 并提升吞吐量：它将模型层划分到多个 GPU，让提示词流经多个流水线阶段。

另一方面，decode 按步顺序执行且对延迟敏感。为一次 decode 使用过多 GPU 反而可能损害每输出 token 时间（TPOT，也称*词元间延迟* ITL），因为每一步 token 都会增加多 GPU 通信开销。由于每次仅有一个 token（采用推测解码时为少数 token）的计算可供切分，其加速潜力有限。

解耦使这些方法可混合使用：一个阶段使用 TP，另一个阶段使用 PP，或为两者设置不同的并行度。例如，可令 prefill 使用 `TP=8`、跨 8 块 GPU 以降低提示词延迟；再令 decode 使用 `TP=1`（单 GPU），以提高每 token 吞吐并降低步延迟。如此可分别调优每个阶段的吞吐和延迟，如图 18-7所示。

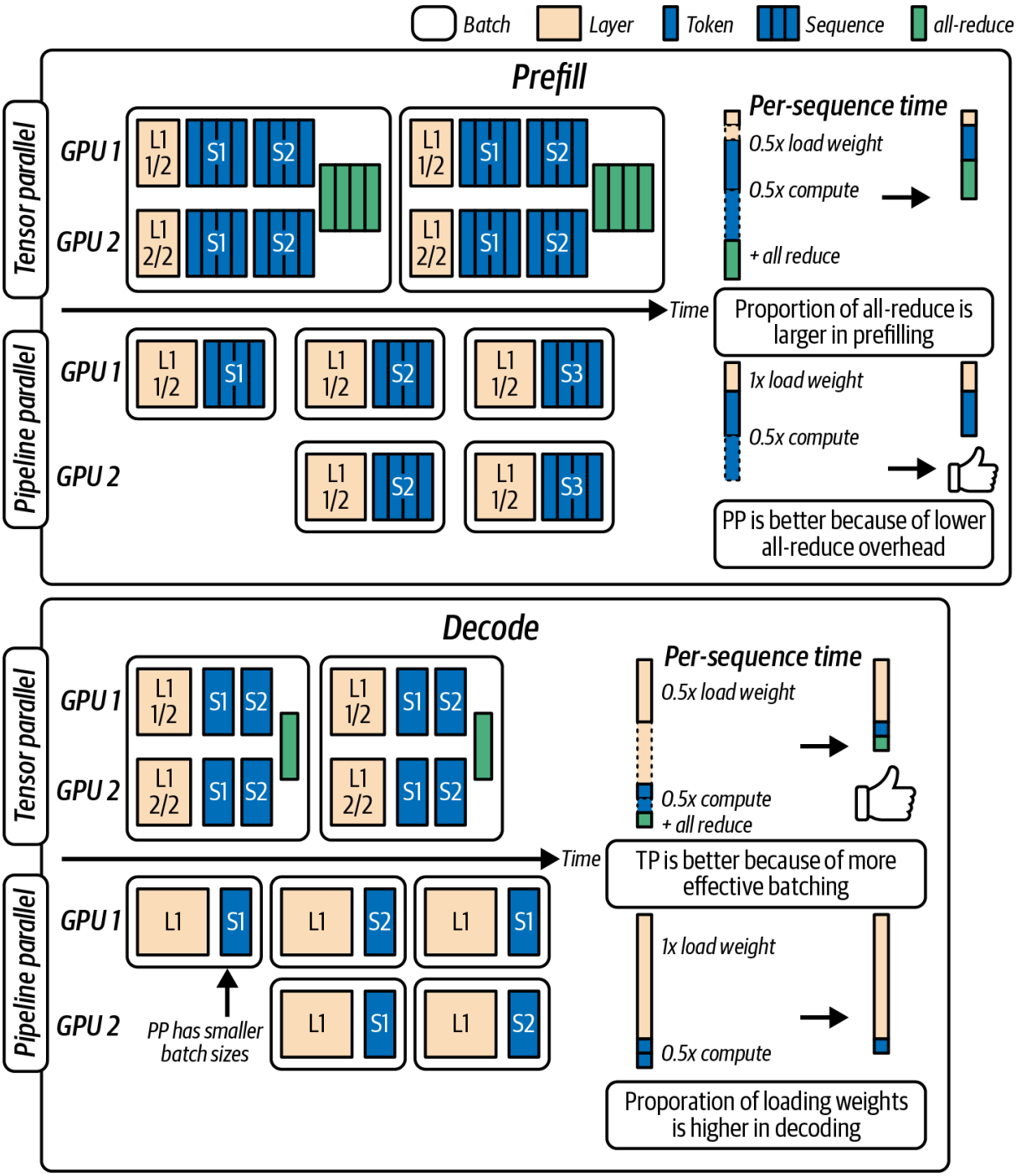


图 18-7. 为 prefill 和 decode 使用不同的并行策略（来源：https://oreil.ly/1-Ti0）

这里，张量并行额外的 all-reduce 通信开销在 prefill 阶段更显著，因为此时会并行处理大量 token。因此，此示例选择流水线并行，认为它对该 prefill 工作负载更高效。

> 张量并行和流水线并行都可能有效用于 prefill。下面的示例为 prefill 使用流水线并行；但在某些集群中，较大的张量并行度可降低 TTFT。最佳选择取决于网络带宽、集合通信延迟和模型形状。

但对 decode 阶段而言，流水线并行会使 token 在 GPU 间传递，产生更多、但更小的前向传播。仅生成一个 token 就要求大量数据进出 GPU。因此，本示例选择张量并行，认为它更适合该 decode 工作负载。

不过，这带来了一个复杂性：由于模型在 prefill 和 decode 间的并行方案不同，KV 缓存格式也必须不同。例如，若 prefill 阶段在四块 GPU 上使用 `TP = 1`（因为使用了 PP），四块 GPU 各自拥有完整大小的 KV 张量。

若 decode 阶段使用 `TP = 4`，每块 GPU 仅需要 1/4 的 KV 张量，因为数据应沿模型隐藏维度切分。为处理此问题，NVIDIA Dynamo 等系统可在传输期间进行 KV 转置或转换：将 KV 缓存从 `[TP_p parts]` 转换并重排为 `[TP_d parts]` 所需格式，其中 `TP_p` 是 prefill 并行度，`TP_d` 是 decode 并行度。

Dynamo 包含高性能内核，可在从 NIXL 读取后、写入 decode worker 内存前即时完成这一转置，使接收端 decode 得到其期望布局的 KV 缓存数据。

相较网络传输，这种转置的开销可能较小，尤其 NVLink 吞吐量足以快速处理此类数据重组时更是如此；在这种条件下，相对于为每个阶段选用优化并行策略节省的计算，额外开销往往是合理的。

下面考察一个阶段专用并行的示例。对于可采用张量（TP）、流水线（PP）、数据（DP）、序列（SP，即跨 GPU 切分序列）等并行方案的大模型，可为不同阶段选择独立并行配置。表 18-1展示了 prefill 阶段的一个并行配置示例。

表 18-1. Prefill 并行示例

并行策略	符号	取值	说明
张量并行	`TP_p`	2	将模型权重张量分到 2 块 GPU，以可控的通信开销使 prefill 延迟约减半。
流水线并行	`PP_p`	2	将模型层分为两个流水线阶段；对于深层模型，使微批次流经每个阶段。
序列并行	`SP_p`	1	除非处理极大上下文，否则不跨 GPU 切分输入序列（不进行序列分片）。
上下文并行	`CP`	1	优化后若能装入内存，则将完整上下文保留在一块 GPU 上（不进行上下文级切分）。
数据并行	`DP_p`	1 (or 2)	每块 GPU 使用一个模型副本；或通过权重复制使用两个副本，以加倍批处理提示词的吞吐量。

这些取值在使用多 GPU 加速大型提示词的同时，力求最小化 GPU 间开销。接着看一个 decode 并行策略示例，如表 18-2所示。

表 18-2. Decode 并行策略示例

并行策略	符号	取值	说明
张量并行	`TP_d`	1（默认） … *N* （GPU 数）	默认 `TP_d = 1`，以保持简单并最小化同步开销。小批次上的微小 GEMM，或单 GPU 无法容纳模型时，`TP_d = N` 块 GPU 可提高效率。
流水线并行	`PP_d`	1	流水线并行会为单 token decode 引入气泡，故 `PP_d = 1` 可避免空闲阶段。
序列并行	`SP_d`	1	跨 GPU 切分输出序列并不常见。 `SP_d = 1` 会让每条 decode 流保持本地，除非处理极长输出。
数据并行	`DP_d`	1	每个 decode 流在每块 GPU 上使用一个模型副本。 应使用独立副本处理并行请求，而非为单条流复制副本。

例如，若模型无法装入单块 Blackwell B200，decode 可优先使用 `TP_d` = 2 或 4，而非 PP；这有助于避免流水线气泡。

理想情况下，每条 decode 流在单块 GPU 上运行，避免跨 GPU 开销。这仅在模型可装入 GPU 内存时才可行。此时 `TP_d` = 1，即 decode 时不使用张量并行。若模型无法装入内存，可提高张量并行度（如 `TP_d` = *N*，其中 *N* 为 GPU 数）。

当系统要用微小 GEMM 处理小批次时，提高张量并行度也可能有用，因为将小型矩阵乘法分配到多个设备可将通信延迟隐藏在计算之后，并可能提高整体吞吐量。

这些是示例取值；关键在于，解耦允许在提示词侧独立配置资源以达到 TTFT 目标，同时在 decode 侧独立调整资源，达到向终端用户流式返回 token 所需的吞吐量和延迟目标。

这样，两种并行策略不会相互牵制。在统一系统中，若试图这样做，只能选择一个对两个阶段都非最优的折中策略。

Prefill 和 Decode 使用不同精度

一些推理引擎允许在 prefill 与 decode 间使用不同精度。例如，可用 FP8、INT8 或 FP4 等较低精度执行 prefill 来加速，同时若为获得更佳生成准确性而有需要，则以较高精度 decode。

通常两个阶段使用相同精度，以便 prefill 阶段计算的 KV 缓存能用于 decode 阶段。但也可应用类似上一节并行转换的转换：传输前将 KV 从 FP16 量化、压缩为 INT8/FP8/FP4，若需要再在接收端转回。

例如，可在网络上传输较低精度的数据来加快传输；也可根据可用 FLOPS 等，在发送端或接收端执行转换。

这些是高级技术，但它们说明：在解耦式设置中，几乎每个方面——硬件类型、GPU 数量和精度——都可针对每一阶段独立调优。

GPU-CPU 协作的混合 Prefill

到目前为止，我们假设 prefill 和 decode 阶段都在 GPU 上运行，且可能使用不同类型的 GPU。但在极端规模、极大模型或极长提示词下，值得评估 CPU 能否为 GPU 分担压力。

现代 CPU 执行神经网络计算远慢于 GPU，但也有其他优势：充足的 RAM、不与 GPU 争用内存带宽，以及可灵活处理 GPU 可能不那么擅长的任务，例如极长序列及分词、填充等非 Transformer 操作。

混合 prefill 策略会在 CPU 上完成一部分 prefill 计算。一种情景是为超长提示词使用 CPU 卸载：设想一个来自大型文档附件、包含数万 token 的提示词，即使是强大的 GPU 也可能因内存限制而难以处理。

对于极长提示词，系统可选择在拥有大量 RAM 的 CPU worker 上执行模型初始层以容纳长序列，之后再将中间结果流式传给 GPU；若延迟不重要，甚至可完全在 CPU 上执行 prefill。随后 decode GPU 从 CPU worker 接收巨大的 KV 缓存。

> 虽然在交互式推理中并不常见，但一些批处理或离线流水线，例如耗时较长的“Deep Research”任务，可对极长文本使用 CPU 预处理。

CPU 卸载的一项实际用途是处理后台或低优先级 prefill 任务。例如，LLM 服务可允许提交极大的提示词，以离线处理非交互请求；这些任务可分配给纯 CPU worker，最终再交给 decode GPU 快速生成 token。由于 CPU 较慢，延迟会较高；但对离线任务可能可以接受，而且这种配置释放了 GPU 资源供更交互式的工作负载使用。

混合 prefill 在 Grace Blackwell 等 CPU-GPU 超级芯片上更有意义，因为其芯片间互连很快，并可利用 CPU 内存相对 GPU 内存容量巨大的特点。

可以设想将巨大的 KV 缓存存放在 CPU 内存中，并让 GPU 快速访问。混合 prefill 使用 CPU 内存来缓冲或预处理输入 token，同时让 GPU 专注于开销大的 Transformer 层。

Grace Blackwell Superchip 可让 CPU 管理内存和初始层、GPU 对序列分块执行稠密注意力，从而处理巨大的上下文。Grace CPU 也可将无法装入 HBM 的 KV 缓存数据溢写到 CPU DDR 内存，这等效于扩展 GPU 可支持的上下文长度。

可通过跨设备切分 Transformer：先在 GPU 上运行前 N 层，在其中处理大部分 token，序列在一定程度上被压缩；再将后续 M 层卸载到拥有大量内存的 CPU，最后将剩余层带回 GPU 以生成最终输出。

这种按层切分技术会增加显著的数据移动和编排复杂度，仅应在极长上下文或 GPU 内存严重受限等少数情形下使用。尽管如此，它展示了在极端推理场景中如何突破硬件边界。

在解耦架构中，采用 CPU 意味着引入第三类 worker：CPU prefill worker。调度逻辑可在 GPU prefill worker、CPU prefill worker 或本地 decode GPU prefill 三种选项中选择；决策取决于提示词长度、优先级等因素。

例如，策略可规定：若 `prompt_length` > 5,000，则路由至 CPU prefill worker。虽然较慢，但它不会占用 GPU，且能使用较大内存；decode 阶段将等待更久才得到 KV。极端情况下，若任务确属离线，也可能在 CPU 上完成 decode。

一般而言，CPU 卸载会增加 TTFT，因此不用于普通的延迟敏感请求。它更像一种可扩展性与兜底功能。若系统频繁走该路径，应监控并考虑是否需要更多 GPU 容量或模型优化。

CPU 卸载也让系统能应对极长输入或 GPU 全忙时的突发负载：它选择回退到较慢的 CPU，而不是完全失败。但若处理过慢并超过 SLO 要求，仍应快速失败。

从成本看，CPU 核通常比 GPU 小时便宜，因此用 CPU 处理部分工作可能更经济。一些云提供商可能为 LLM 服务混合部署 GPU 和 CPU 实例：CPU 在使用 GPU 前执行输入预处理，甚至执行小模型推理。

简言之，尽管核心解耦逻辑聚焦于跨 GPU 分配工作，健壮的超大规模推理系统仍可在工作负载的特定部分创造性地使用 CPU。随着硬件走向 Grace Blackwell 等紧密耦合的 CPU-GPU 超级芯片设计，使用 GPU 与 CPU 的界线会愈发模糊。

高效调度应考虑所有可用计算资源，但指导原则不变：将 GPU 用于其擅长的任务，例如中等序列长度上的大规模并行计算；在 GPU 可能低效的场景使用 CPU，例如极长序列、内存密集任务或低优先级任务。

SLO 感知的请求管理与容错

在超大规模下，要达到 SLO 目标，仅高效扩缩容和调度还不够。有时还须拒绝或延后在当前负载下会违反 SLO 的工作。前文在 Mooncake 的早期拒绝中已涉及这一点。

早期拒绝（准入控制）

早期拒绝或准入控制已在第 17 章介绍。简言之，若系统预测无法在延迟目标内服务某请求，便返回错误或“请稍后再试”来快速失败。预测可基于当前队列长度、近期吞吐量，甚至预测响应时间的轻量级 ML 模型。

早期拒绝不同于先将请求排队、之后却错过其 SLO 截止时间。它通过确保推理系统实际服务的请求可满足承诺，来保持有效吞吐量（goodput）。

在 Mooncake 中，早期拒绝策略会针对 prefill 和 decode 集群的估计负载评估传入请求。例如，设一个 decode 集群正在处理大量长序列。

当新请求到达时，系统会根据其预期输出长度判断 decode 利用率是否将超过安全阈值；若会超过，则立即拒绝或延后该请求，以释放 GPU 节点资源，如图 18-8所示。

这样可避免请求滞留在队列中、耗时过长而超过延迟 SLO，并消耗稀缺的计算或内存带宽资源、拖慢所有其他请求。因此，快速返回“系统太忙”比悄悄接受请求后又无法兑现延迟保证更好。

这类似于 Web 服务器在极端过载下丢弃部分负载，以可接受的延迟继续服务其余请求——也就是令人不快的 HTTP 503 错误；这优于让所有请求都超时。

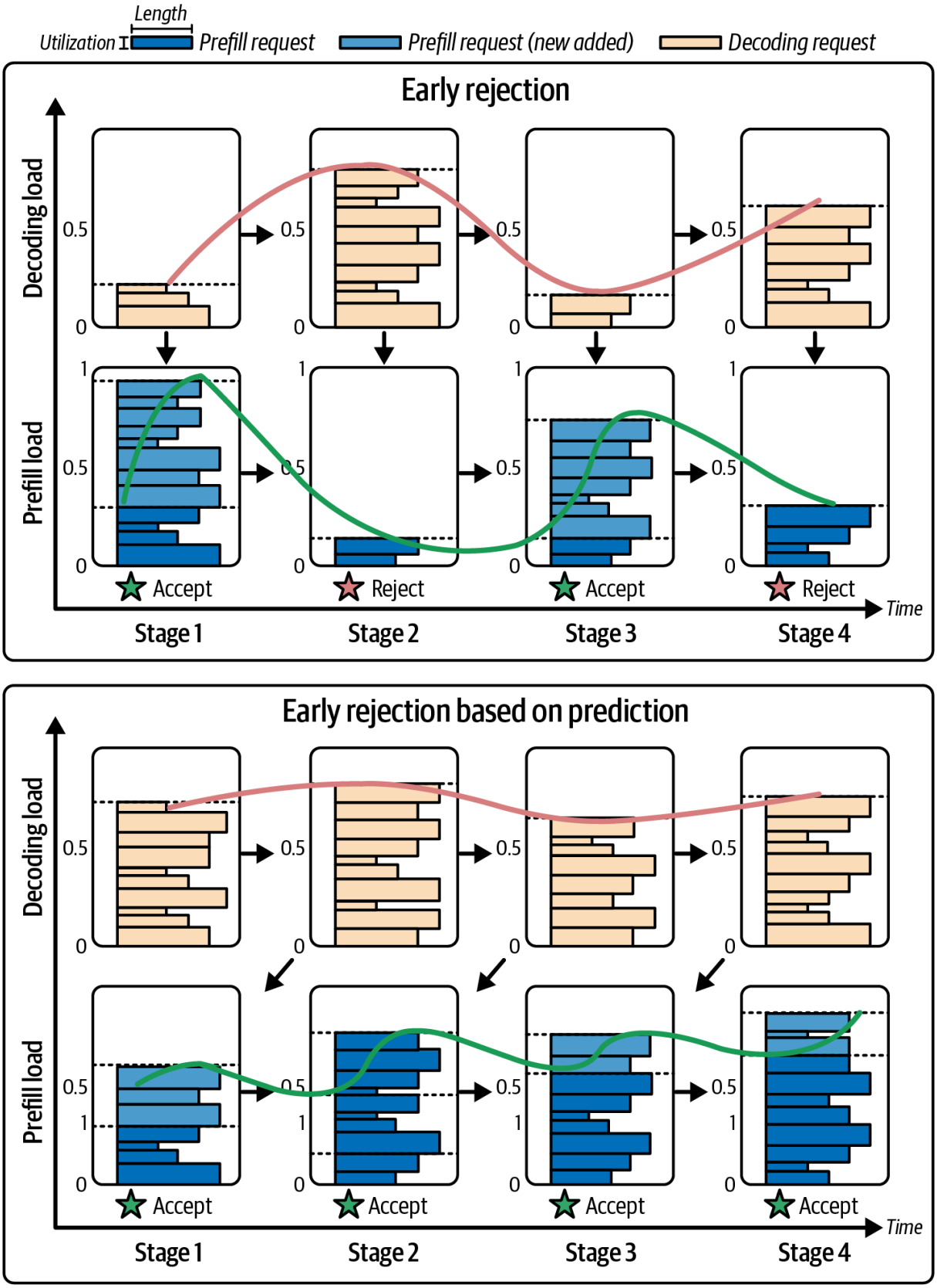


图 18-8. 应用早期拒绝时的实例负载（来源：https://oreil.ly/2xtK-）

在 LLM 服务中，请求大小和持续时间差异很大，因此明确的准入控制步骤有助于维持已接受请求的性能。良好的准入控制使系统处于能够为其接受负载同时满足 TTFT 和 TPOT 目标的工作区间。

它实际上为并发工作设定上限：若增加工作会破坏 SLO，系统便不尝试处理。这必须与扩缩容策略结合；例如，可扩容而非拒绝。不过扩容通常不是瞬时完成，或者系统可能已到最大容量，因此从长远看，拒绝通常是最安全、性能也最可预测的选项。

服务质量

SLO 感知管理的另一个方面是服务质量（QoS）差异化。一些请求带有优先级，例如付费客户优先于免费层客户，或交互式查询优先于离线批处理请求。

调度器可优先处理某些请求，以始终满足其 SLO，即使其他请求必须等待或被丢弃。解耦可通过将一部分 prefill 和 decode worker 专用于高优先级任务来提供帮助。例如，可将集群容量的 10% 专留给高级层请求，30% 专留给标准层请求。

这种分层方法为付费层（高级和标准）保证余量，确保这些请求不被低优先级（免费层）请求延迟。下面给出一个 NVIDIA Dynamo 的 QoS 配置示例，展示此类分层调度：

```
# configs/qos.yaml

scheduler:
  # Define QoS classes and their reserved capacity
  qos_classes:
    - name: premium
      reserved_fraction: 0.10   # reserve 10% of prefill/decode workers
      priority: 100
    - name: standard
      reserved_fraction: 0.30   # reserve 30% for standard tier
      priority: 50
    - name: free
      reserved_fraction: 0.60   # share remaining capacity (free tier)
      priority: 10

# Map incoming requests to QoS classes by label
request_router:
  routes:
    - match:
        header: "x-customer-tier: premium"
      qos: premium
    - match:
        header: "x-customer-tier: standard"
      qos: standard
    - match:
        header: "x-customer-tier: free"
      qos: free
    - match:
        # default fallback
      qos: free
```

这种分层方法允许高优先级请求利用预留容量被准入并以低延迟服务；若系统正忙于处理高优先级请求，低优先级请求将排队，或可能被拒绝。

延迟监控和反馈回路对这类系统很有用，可持续监测 TTFT 与 TPOT 的 p99、p99.9。若这些指标接近 SLO 限制，系统可触发扩容、拒绝新负载、通过限制最大输出长度降低请求

负荷，或暂时缩减负载等动作。

在解耦系统中，可分别观察 prefill 队列和 decode 队列，从而定位哪一端是瓶颈。利用这些信息，若 prefill 积压可停止接收新提示词；若 decode 饱和则可限制长输出的推理请求。

> 使用 ChatGPT 时可能遇到过“随机”错误，原因可能很多；其中一个可能原因是，系统高负载下这些“断路器”启动，对某些请求类型进行负载削减。

容错

容错是运行健壮推理系统的另一个方面。在解耦系统中，若 decode 节点在生成中失败，由于 KV 已保存于池中，前面讨论的 KV 缓存池可支持在另一节点恢复。这样另一 decode worker 能接续生成剩余 token，可能带有少量延迟。

若 KV 未被保存，系统只能在另一节点重新计算 prefill，再继续 decode。这也是 vLLM 等系统即使不严格需要解耦，也会定期对 KV 缓存做 checkpoint 或复制到池中的原因：以防范故障。

除了框架级 KV 快照，如第 5 章所述，承载 GPU worker 的 Linux 进程可借助 cuda-checkpoint 和 Checkpoint/Restore in Userspace（CRIU）暂停并创建快照。这样可将 checkpoint 恢复至具有相同 GPU 芯片类型的另一节点，以尽量减少抢占或故障时丢失的工作。

> 可通过 cuda-checkpoint 恢复预热的 GPU 内存，而非每次启动都重新编译图并加载权重，从而降低推理引擎冷启动延迟。

checkpointing 可能损害延迟，但至少请求能够完成。prefill 节点在完成计算并将 KV 数据发送给 decode worker 或 KV 缓存池后再失败，影响较小，因为 KV 数据已离开故障节点；但若 prefill worker 在处理提示词期间失败，该任务须在另一 prefill 节点重试。架构应优雅处理这些故障，避免一个节点失败便丢弃整个请求或造成大范围级联延迟。

简言之，SLO 感知请求管理确保即使在极端条件下，系统也能为它选择服务的负载维持性能保证。它通过智能决定哪些请求服务、哪些卸载、哪些延后实现这一点。早期拒绝是一个具体例子：它在准入时使用负载预测来防止过载，并在 SLO 约束内维持高吞吐量。

SLO 感知请求管理和容错与此前讨论的其他策略（如扩缩容、调度和缓存）结合，有助于在生产环境维持严格的 SLO 目标。

解耦之所以使这成为可能，是因为它提供了清晰的控制点。可分别观察 prefill 和 decode 指标，并在需要处施加准入控制：例如，prefill 集群积压时停止接收新提示词，或 decode 集群达到容量时停止允许长输出。

最终会得到更可预测、稳定且可适应的推理服务：当负载超过容量时，性能不会急剧劣化；相反，系统优雅地拒绝多余负载，并快速重新平衡以应对变化。

动态调度与负载均衡

开始时可确定多少资源专用于 prefill、多少专用于 decode。但随工作负载变化，应动态调整这种资源划分。

在纯静态配置中，若工作负载组合变化，需要更偏向提示词或更偏向生成的划分，固定比例就会变得非最优。prefill 与 decode 容量的最佳平衡会随时间变化：某一时刻，大量新请求到达，需要偏重 prefill；之后可能有大量持续较久的推理生成，需要偏重 decode。

自适应调度和负载均衡机制旨在持续调优系统，使任一阶段都不成为瓶颈，从而在变化条件下保持较高有效吞吐量。

vLLM、SGLang 和 NVIDIA Dynamo 等现代推理引擎将负载监控与动态 worker 分配纳入平台；许多云推理平台也拥有遵循类似原则的自定义自动扩缩器和路由器。

自适应资源调度与热点预防

解耦设置的一个问题是 prefill 与 decode 集群间的负载不均衡。若当前负载组合下 prefill 与 decode worker 的比例配置不当，一端可能饱和，另一端却利用不足。

例如，相对 prefill 而言 decode worker 不足时，decode 任务会排队。即使提示词处理很快，TPOT 仍会上升。反之，若 decode 过度配置而 prefill 配置不足，新请求可能等待开始，导致 TTFT 很高，同时许多 decode GPU 闲置。

理想状态是两端都接近容量运行但不至于被压垮：系统让 prefill 和 decode 均保持忙碌，同时任一端都没有不断增长的队列。

静态的 prefill-decode worker 分配在真实条件下并不充分，因为负载会在一天内大幅变化。在长期运行的大规模推理系统中，输入长度和输出长度的组合可能逐小时显著不同。

例如，一个小时内可能有大量长问题、短答案（如摘要），需要更多 prefill 资源和较少 decode 资源；下一小时可能出现短问题、长答案（如推理、Web 搜索），所需 prefill 较少但 decode 多得多。

换言之，对一种情景有效的静态 prefill:decode 比例未必适合另一种情景。因此，最佳 `X_p` 与 `Y_d` 配置依赖于工作负载，且会随时间变化。

为此，高级推理系统可采用自适应调度算法，动态重新分配负载，甚至即时改变实例用途。接下来讨论几种此类方法。

TetriInfer 的两级调度器

研究原型调度器 TetriInfer以两个粒度工作。首先，在单请求层面，它依据当前负载将传入请求分配到特定 prefill 与 decode 实例，这属于预期的常规路由。其次，它在全局集群层面监控资源利用率并预测瓶颈可能出现的位置，主动迁移工作以防止热点。

例如，调度器可能发现一个 decode 节点排队了许多极长序列。在问题变严重前，它会将其中一部分路由至更空闲的 decode 节点，即使常规情况下不会路由至该节点，从而平衡负载。图 18-9从执行时间线与架构角度比较了现有系统和 TetriInfer 的工作。

这里可以看到，TetriInfer 的两级调度器通过队列长度、GPU 利用率趋势等预测资源使用，平滑集群内负载，并防止任一节点过载。

TetriInfer 这个名称暗示像摆放俄罗斯方块一样“装箱”请求，使其填满 GPU 时间而彼此不干扰。

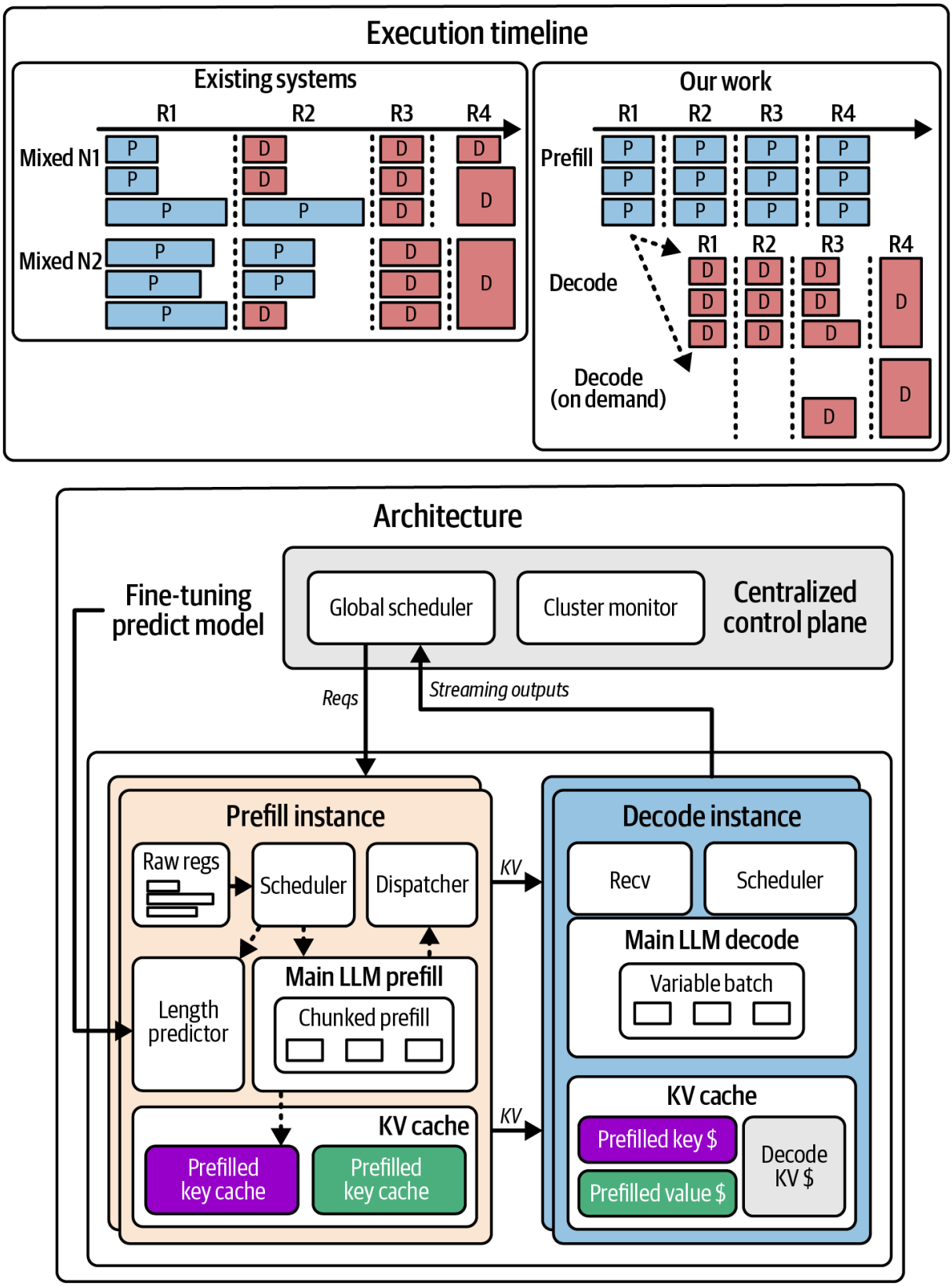


图 18-9. 现有系统与 TetriInfer 架构的比较（来源：https://oreil.ly/_3KGj）

Arrow 的自适应实例伸缩

动态资源调度的另一种技术是 Arrow（不要与流行的 Arrow 数据格式混淆）。这是一种自适应实例伸缩技术，利用了解耦系统对工作负载变化常有滞后响应这一事实。例如，输入与输出的分布变化后，静态的 prefill/decode worker 数量不会立刻调整；一端成为瓶颈，便会暂时损失有效吞吐量。

Arrow 通过测量输入 token 速率与输出 token 速率，以及集群中每个 worker 池的积压，持续分析工作负载；随后动态调整 worker 分配，如图 18-10所示。

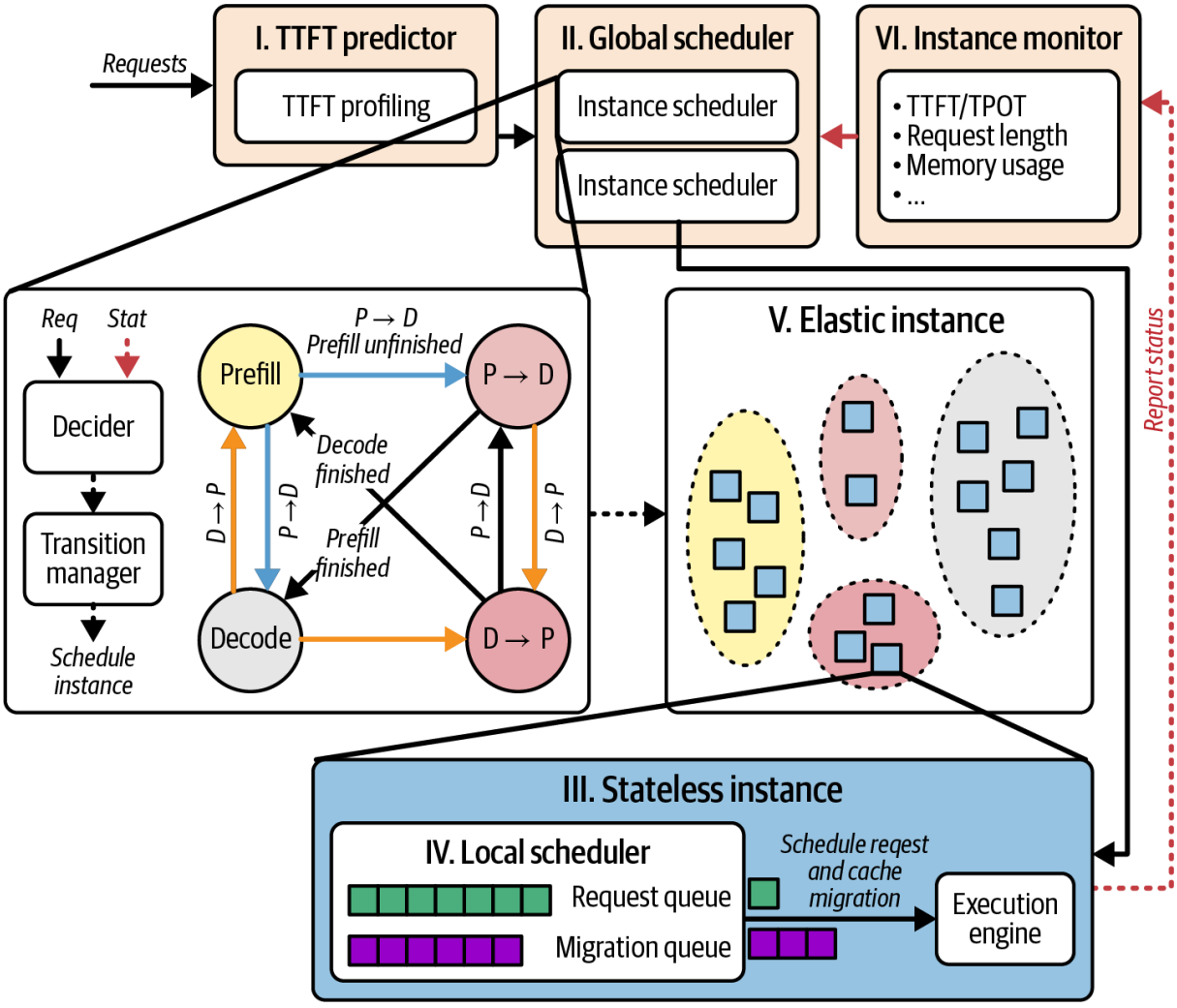


图 18-10. Arrow 架构

在云环境中，这可意味着输出负载增加时启动额外 decode 实例；若检测到输入密集型负载，也可缩减 decode、增加 prefill 实例。

Arrow 的设计同时包括请求调度（决定哪个节点处理哪个请求，类似其他调度技术）和实例调度（决定何时启动或关闭 prefill 或 decode 实例）。

Arrow 将 prefill 和 decode worker 数量视为可在接近实时的尺度上调整和伸缩的参数，将自动扩缩逻辑纳入 LLM 推理调度器。

例如，若系统收到大量输入密集、输出较少的请求，prefill 节点可能成为瓶颈。此时 Arrow 会检测到 prefill 排队时间增长或 TTFT 分位数上升，并决定将一部分 decode

worker 转为 prefill worker，或启动额外 prefill 实例以处理突发负载。

反过来，若出现输出密集、答案很长的请求浪潮（如推理链），TPOT 上升并形成很长的 decode 队列时，decode 集群可能开始滞后。此时 Arrow 可为 decode 阶段分配更多 GPU worker，例如在提示词到达放缓时暂时闲置一些 prefill 节点，或直接添加新的 decode 节点。

在 GPU 数量固定的静态本地集群中，动态伸缩会通过任务重新分配实现：指示若干原本分给 prefill 的 GPU 转换角色，暂时加入 decode 池。只有每个计算节点都配置为可同时充当 prefill worker 和 decode worker 时，这才可行。

角色切换可能有开销，因为模型可能须为不同并行策略或量化选择加载不同的模型分片。一些设计会在每个 GPU 上保持所有模型权重已加载，仅按需分派不同任务。这等效于将集群作为一个灵活池：任何时刻一部分 worker 做 prefill，另一部分做 decode。

这开始类似于时间共享的统一集群，但粒度较粗：一个节点先专门用一段时间执行 prefill 任务，之后切换为执行 decode 任务。

在云端部署中，动态伸缩还可表示与 Kubernetes Horizontal Pod Autoscaler、Cluster Autoscaler 等自动扩缩器对接，为各角色添加或移除 pod 与节点。例如，若 prefill 负载持续较高，Arrow 可触发启动新的 prefill pod，从而形成一套完全弹性的解耦 prefill/decode 推理集群，按需扩展或收缩每一端。

实际中，扩容新的 GPU pod 可能需要数十秒或更久。因此，在等待容量期间系统可能需要卸载负载。下节将讨论 Mooncake 如何处理这种情况。

Mooncake 的自适应策略

除 Arrow 外，Mooncake 系统也突出了自适应策略。Mooncake 提出了名为*早期拒绝*的预测式准入控制，它与管理供给的扩缩容在一定程度上互补，关注的是管理需求。

具体来说，若系统预测没有足够容量在 SLO 内处理新请求，就会拒绝该请求，而不是先接受又可能违反 SLO。这是动态*负载削减*而非扩容的一种形式，但目标是防止过载。

上一节已介绍 Mooncake 的早期拒绝方法。此处要点是：自适应既可发生在供给侧，即增加或重新分配资源；也可发生在需求侧，即限流或拒绝部分请求。

动态伸缩的关键收益是在负载变化中维持较高有效吞吐量。自动调节 prefill 与 decode 的比例，可避免一端 GPU 长时间闲置、另一端过载的失配状态。

理想情况下，两类节点得到均衡利用。这样可在瓶颈形成时尽快缓解以改善延迟，也可提高效率：无需在一端过载时继续为空闲的另一类 GPU 付费；系统会重新分配资源或转移任务以维持平衡。

从结果看，自适应系统可能给出类似结论：“应用自适应伸缩后，系统在静态系统无法覆盖的流量模式中维持了 > 90% 的 SLO 合规率，并通过快速迁移资源在高峰中多处理了 X% 的请求。”这类结果须结合具体基准场景解读。

具体而言，Arrow 的结果提到，在工作负载变化极剧烈的一个场景中，相比非自适应系统，请求服务速率最高可达 5.6 倍；典型改进较低但仍显著。确切提升取决于工作负载变化的剧烈程度及部署条件。

这些内容表明，解耦可消除阶段互相干扰问题；动态解耦则进一步消除固定分配带来的阶段不均衡约束。要实现这种自适应性，调度器需持续监控 prefill 队列长度、decode 队列长度与 TTFT/TPOT 分位数等指标。这些指标常通过 Prometheus 和 K8s 部署中的自定义控制器送入控制面板，算法随后可自动完成决策。

复杂的推理系统可使用 ARIMA 等预测模型及其他预测技术，根据时段模式预见突发负载和流量变化。若基于历史使用模式预测晚 9 点会出现长输出峰值，调度器可恰好提前分配更多 decode 容量。总体目标是在无需人工干预和重新配置的情况下，让尽可能高比例的请求在 SLO 内完成。

动态资源伸缩

在预测性调度的思想基础上，动态资源伸缩专指响应负载，改变 prefill 与 decode 间资源分布。例如，Arrow 的自适应实例调度持续调整 prefill 和 decode worker 的数量。以下是平衡推理系统负载的一些方法：

弹性实例

> 在 Kubernetes 或类似系统中，可为每个部署定义自动伸缩策略。例如，可指定将平均 prefill GPU 利用率维持在 70%；利用率更高时，系统向 prefill worker 池添加 pod 或节点；利用率降低且 decode 利用率仍高时，系统可将一个 pod 从 prefill 移至 decode。这可基于规则，也可基于算法，例如在每个时间间隔求解最优配置，选择新的 X 和 Y 数量。

实例“翻转”机制

> TetriInfer 论文描述了“实例翻转”：一些节点可按需切换角色。这要求模型以恰当方式加载、分片和量化，从而能处理两种角色。

面向弹性的无状态设计

> Arrow 利用无状态实例，因此 worker 上不保存长生命周期的会话状态，可以自由重新分配。活跃请求的 KV 缓存会使正在生成 token 的 decode 节点难以切换为 prefill；因此通常须等待 decode 完成任务后再切换角色。

稳定性与振荡

> 快速切换角色可能导致振荡和抖动，因此通常会规定角色的最短保持时间，以避免过于频繁地翻转。

除负载均衡外，还应考虑多租户或混合工作负载。若集群服务不同模型或任务，甚至可按需求在它们之间重新分配 GPU，而不局限于单一模型。

解耦的模块化配置允许用闲置的 decode GPU 执行另一个较小模型的推理任务。这在写作时尚未成为主流扩展，但随着推理引擎演进和设计更灵活，在概念上是可行的。

归根结底，超大规模推理系统需要一个反馈回路，持续使资源分配与当前工作负载匹配。TetriInfer、Arrow、Mooncake 等在使用此类反馈回路、适应负载变化时都报告了显著改进。这表明解耦消除干扰，而自适应解耦消除不平衡；在动态条件下取得最佳性能需要二者配合。

要点回顾

本章涵盖了统一巨型内核、高效内存分配、快速数据传输、prefill/decode 解耦、KV 缓存池、动态伸缩和持续 SLO 感知等多种技术。以下是一些要点：

加速 decode 阶段

采用 FlashMLA、ThunderMLA、FlexDecoding 等融合注意力内核，可显著加速 decode 阶段。这些内核可改善单 token 吞吐量和 GPU 利用率，实际效果依赖模型、硬件与实现。

将 KV 缓存视为一等资源

通过解耦在 GPU 间共享 KV 缓存，并复用前缀以避免冗余计算；全局缓存池和哈希可支持这些能力。

力求使 prefill 与 decode worker 间开销接近零

利用 GPUDirect RDMA 和 NIXL 进行高速 GPU 到 GPU 传输，并将计算与传输重叠，以尽可能降低 prefill 与 decode worker 间的开销。

为每个阶段采用专用硬件和并行策略

解耦 prefill 和 decode 后可为每阶段采用专用硬件与并行策略。例如，prefill 使用高算力 GPU 或多 GPU 节点，decode 使用内存充裕的 GPU 或单 GPU；目标是降低成本、提高吞吐，具体选择取决于负载。

用自适应与动态算法优化系统

自适应调度和 SLO 感知控制（如早期拒绝、动态伸缩）有助于在负载变化下维持延迟保证，并在不过载的前提下充分利用 GPU。

结论

超大规模 LLM 推理需要整体性方法，同时涵盖高层自适应资源管理以及底层内核和内存优化。组合本章技术可让高度优化的推理部署在现代硬件上追求更高吞吐，同时满足严格延迟保证；结果仍取决于模型、硬件、互连与实际负载。

随着硬件持续演进，GPU 获得更大内存和专用推理核心，软件框架也提供动态路由和灵活角色分配等更成熟能力，这些优化可相互叠加并更具威力，使推理引擎能更高效地处理更大模型、更长上下文和更多用户。

第 19 章. 动态与自适应推理引擎优化

现代硬件上的超大语言模型（LLM）推理需要在运行时动态适应，才能在不同条件下同时实现高吞吐量和低延迟。静态的“一刀切”模型服务优化方式已不再足够。

相反，先进的模型服务系统采用*自适应策略*，动态调整并行度、数值精度、CUDA 内核调度和内存使用。本章探讨这些高级技术，包括动态并行切换、精度伸缩、实时缓存管理及基于强化学习（RL）的调优。

本章为超大规模 LLM 推理提供最佳实践，说明如何编排一套能监测自身性能、实时适应并最大化效率的引擎。

自适应并行策略（TP、PP 与混合并行）

超大 LLM 需要采用张量并行、流水线并行或其混合等模型并行方式，将计算分布到多块 GPU。各种方法各有优缺点。表 19-1汇总了针对特定推理流量模式的推荐并行策略。

表 19-1. 常见推理流量模式及其推荐并行策略汇总

推理流量模式	推荐并行方式	理由
大量短请求（< 256 token，高 RPS）	数据并行/副本伸缩	最小化 GPU 间通信；每块 GPU 运行副本、处理独立请求（假设模型可装入单块 GPU 内存）
少量长请求（≥ 8k token，低并发）	流水线并行（配合微批次）	通过跨 GPU 切分层来降低单请求延迟
混合负载（短请求 + 一些长请求）	混合动态方式（自动切换）	在单 GPU 上运行短对话，并为长请求使用流水线，以满足延迟 SLA
超大模型（超过单块 GPU 内存）	张量 + 流水线混合并行	为容纳模型所必需；在两种维度上平衡计算与内存
MoE 模型推理（稀疏专家选择）	专家并行	将单个专家分布到多块 GPU；每个请求只调用部分专家，降低每设备的内存与计算负载

数据并行和副本伸缩策略会在每块 GPU 上复制完整模型，并在这些副本间对传入请求做负载均衡。由于每块 GPU 独立处理不同请求，单次推理无需 GPU 间同步。

这能以最小通信开销最大化大量小型或中型输入的吞吐量；但若模型无法装入单块 GPU 内存，数据并行便不可用。

张量并行（TP）是一种模型并行（不同于数据并行）方式：将模型矩阵（如权重、层内张量等）分到多个 GPU，以加速矩阵乘法。但它会引入额外的 all-reduce 通信，使 GPU 保持同步。

流水线并行（PP）也是一种切分模型的模型并行方式。但它不是切分各模型层内部的矩阵，而是将完整层分配给不同 GPU 以突破内存限制（前提是每层可装入单块 GPU）。PP 会以顺序阶段延迟的形式引入额外开销，称为*流水线气泡*，如图 19-1所示。

专家并行用于混合专家（MoE）模型架构，将每个专家子网络分配给相应 GPU。轻量级门控网络再将每个输入请求或 token 仅路由到 router 选定的 top-k 活跃专家；每块 GPU 只处理它托管的专家子集。

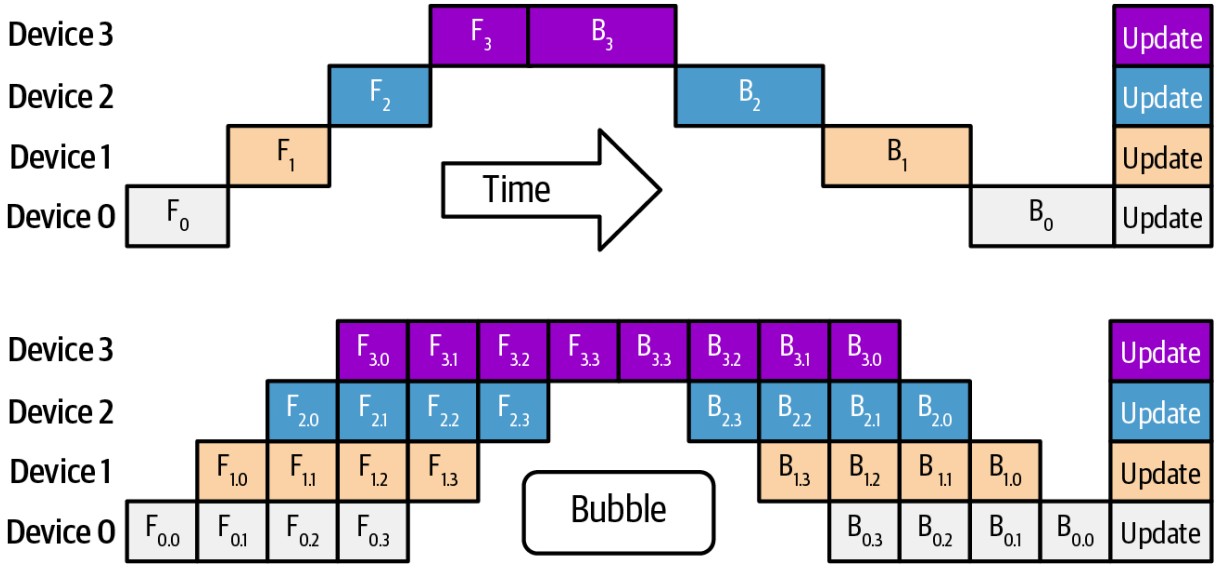


图 19-1. PP 导致的流水线气泡

每个输入仅激活少量专家，专家并行可为拥有大量专家（常称为*宽专家模型*）的模型降低单设备内存、推理时间和计算成本。随着增加更多专家，这种基于 router 的条件专家计算模式可高效扩展。例如，DeepSeek-R1 有 256 个专家，但推理时 router 只选择 top-9 专家（包括 1 个共享专家）。

传统上，并行策略（包括组合多种并行技术的混合策略）在加载模型时预先选定并固定。不过，为在动态工作负载下最大化性能，现代推理引擎可基于输入特征，在运行时选择不同的并行策略。

高性能自适应推理系统使用运行时指标选择 TP、PP 或混合方式。关键因素包括批大小、序列长度、内存利用率，以及响应延迟与吞吐量要求。例如，极长提示词可路由至 TP + PP 实例，以将层分布到多个 GPU、避免显存不足（OOM）。

与此同时，短而延迟敏感的请求可路由到仅使用 TP 的模型实例，以避免流水线阶段开销。为此，服务引擎维护多个预分片模型实例，每个实例针对不同负载特征优化，并将传入查询动态分发至并行策略最符合该任务 SLO 的模型实例。

也可使用不同数量的分片。图 19-2展示了在八块 GPU 上以两种不同 TP 分片数配置的两套混合 TP + PP 并行方式。

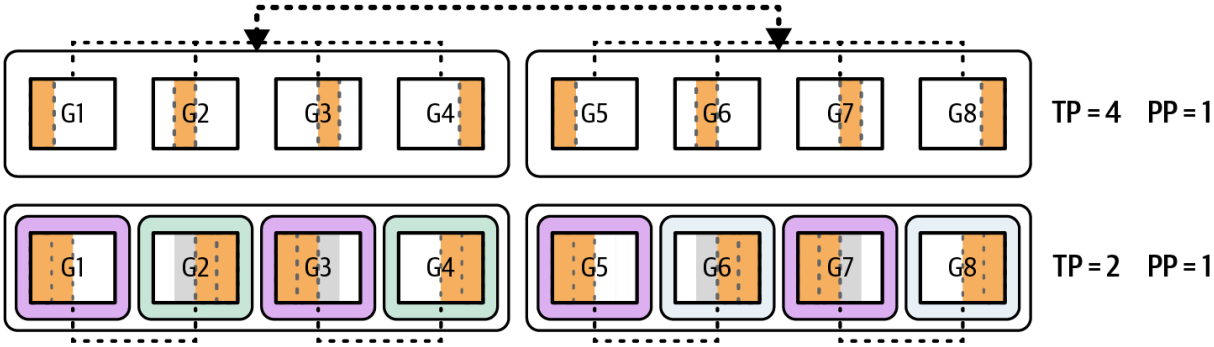


图 19-2. 为给定模型在八块 GPU 上预先配置两种不同的混合分片池（TP = 4、PP = 1 与 TP = 2、PP = 1）

为长序列输入使用 PP 池有助于避免大输入序列造成的 OOM。相反，短提示词和延迟敏感查询可路由到针对低延迟优化的张量并行模型实例，从而避免 PP 的开销。这里的“动态”通常是把请求路由至已预配的 PP/TP 池，而非对正在运行的实例即时重分片。

由于每个请求可使用不同并行策略，系统需要维护多个模型实例供推理调度器/路由器选择。其中一个实例可使用 TP 针对低延迟优化，另一个则使用 TP 与 PP 针对高吞吐和大输入序列优化。

需要维护多个模型实例，是因为即时重分片会严重扰动 GPU 缓存，并给内存和网络子系统施加过大压力，尤其对超大模型更是如此。

运行时，每个查询根据其长度和指定服务等级协议（SLA）分发到最匹配的模型实例（分片策略）。例如，DeepSeek-R1 是约 6,800 亿参数的稀疏 MoE 模型，每个 token 在专家中仅激活约 370 亿参数。

为支持不同负载特征，可将 GPU 组织成逻辑 worker 池，每个池预先采用一种特定并行策略，例如张量并行，或张量 + 流水线混合并行。

来看一个示例。若有一台配备 8× GPU Blackwell B200 的服务器，其 HBM 总量为 1,440 GB（1,440 GB = 每 GPU 180 GB × 8 GPU），可跨四块 GPU 以四路 TP 服务 DeepSeek-R1，另四块 GPU 则空闲。

若单个查询带有极长上下文（例如 > 100 万 token），调度器可使用两阶段流水线：阶段 1 跨 GPU 0–3，阶段 2 跨 GPU 4–7。这样每个阶段可获得约 720 GB 总 HBM（每 GPU 180 GB × 4 GPU），有助于处理大型输入时避免 OOM。实际可容纳的上下文长度还取决于模型的 KV 大小、KV 精度、权重/激活占用和缓存策略，不能仅由 HBM 总容量推断。

反之，当数十个短而延迟敏感的提示词并发到达时，系统只将其路由至张量并行实例。避免填充和排空流水线阶段产生的空闲期（流水线气泡）后，此配置可在可用 GPU 资源内提供较低的单请求延迟。

要实现动态并行切换，可实现一个决策函数，检查输入序列长度、GPU 内存使用率和当前负载等运行时指标，并用这些指标为每个请求选择最合适分片的模型实例，如下所示：

```
def choose_worker_pool(seq_len, gpu_mem_util, concurrent_reqs):
    # For long contexts or high memory pressure,
    # use hybrid pipeline + tensor parallelism
    # (example thresholds shown here)
    if seq_len > 4096 or gpu_mem_util > 0.8:
        return "tp_pp_hybrid"
    # For many simultaneous small requests, stick with tensor parallelism
    if concurrent_reqs > 4:
```

```
        return "tensor_parallel"
    # Fallback to tensor-parallel for typical workloads
    return "tensor_parallel"
```

可在 GPU 集群上预先启动多个模型副本，其中一些只按 TP 分片，另一些按 TP + PP 分片；路由器再根据输入和决策策略将每个查询送至恰当副本。这样，大型内存密集任务获得完整流水线支持，短而延迟敏感的调用则在仅 TP 的实例上运行，避免不必要的流水线开销。

建议使用模型与硬件遥测数据指导并行切换。可实时监测 GPU 内存利用率、计算利用率和互连（如 NVLink/NVSwitch）流量以作决定。若因长流水线气泡发现 GPU 闲置，且有额外内存余量，可将流水线收缩为更少阶段，使每块 GPU 执行更多工作并保持忙碌。反之，若某些阶段触及内存限制或计算瓶颈，可扩展为更多流水线阶段，或提高张量并行度，以将计算和内存占用分布到更多 GPU。

关键是动态调节张量切分与流水线切分的平衡，以保持 GPU 良好利用，同时遵守内存约束并达到延迟目标；静态的一刀切配置无法实现这种能力。

动态精度调整

Blackwell 等现代 GPU 引入了对 8 位和 4 位浮点（FP8/FP4）Tensor Core 运算单元的支持。这些低精度格式可带来显著加速和内存节省；质量损失是否很小则取决于模型、量化方案、内核与评测任务。

动态精度切换是一种高级技术：推理引擎根据模型置信度或资源压力在运行时调整数值精度，目标是在不显著损失质量的前提下提高吞吐。实践中，这可能意味着为效率以 FP8 或 FP4 执行模型的某些部分，并在需要稳定性时回退至更高精度（FP16/BF16）。这种按 token 或按步切换通常需要自定义内核、校准和回归验证，并非多数引擎的标准现成功能。

精度自适应的一个触发因素是*logit 尖锐度*，即模型输出置信度。例如，若模型对某个特定 token 的置信度很高，使下一 token 的概率分布呈现极尖峰，则低精度造成的小数值误差不太可能改变结果。

若下一 token 的生成可容忍低精度，引擎可在后续若干步使用 FP4 来获得速度；反之，若分布因不确定性高而更平坦，则应保持 FP8 或 FP16 以维持保真度。

推理引擎可通过计算词表上 softmax 分布的 Shannon 熵来量化不确定性。更低的熵表示更尖锐（更有信心）的预测。基于留出验证集调优的固定熵阈值，可决定何时降到 FP4、何时保持 FP8/FP16 以维持数值稳定性；目标是在延迟收益和准确率损失间取得平衡。

> 使用能维持准确率的最低精度；当以最大 softmax 概率衡量的模型置信度下降时，回退到更高精度。

这利用了一个事实：大型 LLM 在生成确定性续写（如闭合引号或完成列表）时往往更有把握，在这些情况下较低精度通常足够。

另一个因素是内存压力。若因极长上下文或众多并行请求使 GPU 内存使用接近上限，系统可动态将激活压缩至较低精度。

内存紧张时，可将注意力键/值张量以 INT4 而非 INT8 存储，理论上可将该部分内存占用降低 50%。但须确保 INT4 的量化误差不会在大量 decode 步中累积，建议定期重新评估输出质量。

例如，若推理达到 KV 缓存占用 90% 内存的程度，引擎可能决定将新的缓存条目从 INT8 量化到 INT4，甚至追溯压缩旧条目以释放空间。这可在不停止模型的情况下执行；后续注意力层只需读取 INT4 缓存值，但会引入量化误差。此类在线重新量化的成本、布局兼容性与精度影响必须针对具体引擎验证。

将 4 位权重量化与 8 位激活相结合可显著降低内存。对纯计算受限内核，FP8 激活在高带宽现代 GPU 上的特定配置中可实现最高约 2 倍吞吐；对混合或内存受限负载，约 1.5 倍可能可达。使用 FP4 激活还能进一步节省内存，但可能引入更高累积误差，需要谨慎进行逐层调优。

现代 GPU 提供原生 FP8 与 FP4 Tensor Core。不过，按本书写作时，PyTorch 的 AMP 支持（`torch.autocast`）仍只面向 FP16 和 BF16，不管理 FP8 或 FP4。虽然 PyTorch 存在 FP8 数据类型（如 `torch.float8_e4m3` 和 `torch.float8_e5m2`）及缩放计算路径，AMP 不负责管理它们。对于推理和训练，可在适用时使用 NVIDIA Transformer Engine（TE）及其 MXFP8、NVFP4；具体可用性需以所使用的 PyTorch、TE 和硬件版本为准。

> 对延迟关键的 decode，在 Blackwell 上使用 AMP 时可优先评估 BF16 而非 FP16。对 FP8 路径，Transformer Engine 的 MXFP8 是 Blackwell 上值得优先评估的默认格式；对 KV 缓存和轻量层可选择性尝试 NVFP4，并进行严格回归测试。务必针对特定工作负载逐层验证数值表现。

表 19-2汇总了一些精度配置示例及其权衡。较低精度通常会降低内存占用、提高吞吐量，但也可能带来质量退化。

表 19-2. LLM 推理中不同精度模式的近似权衡

精度模式	内存用量（相对值）	计算吞吐量	质量影响（准确率变化）
FP16（基线）	1.0×（100%）	1.0×（基线）	无影响（完整保真）
FP16 权重 + FP8 激活	~0.5×（50%）	~1.5×	可忽略（< 0.1%）
INT4 权重 + FP8 激活	~0.25×（25%）	~1.8×（计算与内存混合受限）	质量约下降 0.5%
INT4 权重 + FP4 激活	~0.2×（20%）	~3.5×	约下降 1%（需谨慎调优）

这里可看到，若只考虑激活位宽，FP8 激活相对 FP16 基线可将激活内存减少约 50%。表中 FP8 激活测得的质量损失可忽略（< 0.1%）；但低精度的质量影响取决于模型和内核，必须用自己的数据与工作负载验证。

当内存是主要瓶颈时，INT4 权重 + FP8 激活工作流在特定配置中可达到约 1.8 倍基线吞吐量。INT4 权重 + FP4 激活可将相关内存降至基线的约 20%，表中速度提升约 3.5 倍；这与相对 FP16 理论峰值约 4 倍的位宽比相符，但端到端实测通常受内存、通信、量化/反量化和内核效率限制。

动态精度切换的目标是在输出质量保持于可接受范围内时最大化性能。理想情况下，内核运行在最快可用精度（如 FP8 或 FP4），仅在必要时回退到更高精度（如 FP16）。实践中，PyTorch 的 NVIDIA Transformer Engine 等库可提供逐层精度控制，但其粒度、可用格式与行为取决于版本和所用模块。

线性层可默认使用 FP8，但运行时 hook 可根据层的角色将其精度提高至 FP16 或降至 FP4。例如，可将 FP4 应用于允许轻微准确率退化的输出投影等轻量层，而处理原始用户输入、受益于较高精度的早期层可使用 FP8 或 FP16。

除逐层混合精度控制外，还可使用更细粒度的按 token 精度调整策略。预测很有信心时，该策略让推理系统以 FP8 等最快模式运行；不确定性变高时，再回退至 FP16 等高精度模式。此处为概念性实现，须评估调度和同步开销是否抵消精度切换收益。

在实践中，模型可先以默认精度（如 FP16）生成当前 token，再根据输出熵、最大 softmax 概率或 logit 方差等运行时指标评估置信度。

若模型对预测非常有信心，下一个 token 可按较低精度处理；不确定性高时，系统回退至更稳定的格式以维持输出质量。下面的示例代码说明这一概念：

```
import contextlib
import torch

# ----------------------------
# Safe Transformer Engine (TE) FP8 autocast import
```

```python
# ----------------------------
try:
    # TE is only effective if your model actually uses TE-enabled layers
    # (e.g., Linear, LayerNorm wrappers).
    from transformer_engine.pytorch import fp8_autocast as _te_fp8_autocast
    # type: ignore
    _TE_AVAILABLE = True
except Exception:
    _TE_AVAILABLE = False
    # No-op stand-in so the code runs without TE installed. It never changes
    # numerical behavior.
    class _NullCtx(contextlib.ContextDecorator):
        def __init__(self, **_): pass
        def __enter__(self): return self
        def __exit__(self, *exc): return False
    def _te_fp8_autocast(**_):
        return _NullCtx()

# ----------------------------
# Helper: choose the precision context *for this step* safely
# ----------------------------
def _precision_context_cuda(use_fp8: bool,
                            prefer_bfloat16: bool,
                            enable_fp8: bool):
    """
    Enter exactly one precision context. If FP8 isn't enabled or TE is
    missing/unused, fall back to AMP (BF16/FP16).
    """
    if use_fp8 and enable_fp8 and _TE_AVAILABLE:
        # Note: fp8_autocast affects only TE-enabled modules. Non-TE modules
        # run at their native dtypes.
        return _te_fp8_autocast(enabled=True)
    amp_dtype = torch.bfloat16 if prefer_bfloat16 else torch.float16
    return torch.autocast(device_type="cuda", dtype=amp_dtype)

def _precision_context(device: torch.device, use_fp8: bool,
                       prefer_bfloat16: bool, enable_fp8: bool):
    return _precision_context_cuda(use_fp8, prefer_bfloat16,
                                   enable_fp8) if device.type == "cuda"
                                   else contextlib.nullcontext()

# ----------------------------
# Main decode loop with smoothed, hysteretic precision switching
# ----------------------------
@torch.no_grad()
def decode_with_dynamic_precision(
    model,
    tokens: torch.Tensor,
    max_steps: int,
    *,
    device: torch.device = torch.device("cuda"),
    prefer_bfloat16: bool = True,      # B200: prefer BF16 over FP16 for AMP
    enable_fp8: bool = True,           # Allow FP8 when TE present
    enter_fp8_threshold: float = 6.0,  # hysteresis upper bound
                                       # (logit margin average)
    exit_fp8_threshold: float = 3.0,   # hysteresis lower bound (avoid flapping)
    reeval_interval: int = 8,          # compute/inspect confidence every N steps
                                       # to avoid per-step sync
    topk_dim: int = -1,                # last dimension holds vocabulary logits
    eos_id: int | None = None,
):
    """
    Autoregressive decode loop that *smoothly* switches between AMP (BF16/FP16)
    and FP8 (TE) without per-step host sync. Works even when TE is not
```

```
    installed; in that case, runs AMP only.

    - Confidence signal: mean(top1 - top2) logits margin across the batch.
    - Smoothing: EMA + interval re-evaluation to minimize CPU-GPU sync pressure.
    - Hysteresis: separate enter/exit thresholds to avoid precision flapping.
    """
    assert exit_fp8_threshold <= enter_fp8_threshold,
        "Hysteresis requires exit <= enter threshold"

    model.eval()
    tokens = tokens.to(device, non_blocking=True)

    # Internal state
    use_fp8: bool = False  # start in AMP.
                           # Upgrade to FP8 when sustained confidence permits
    ema_conf: torch.Tensor | None = None  # stays on device;
                                          # host consults only at intervals
    alpha = 0.2  # EMA smoothing factor for confidence
# A tiny helper to update on-device EMA without host sync
    def _update_confidence_ema(logits: torch.Tensor) -> torch.Tensor:
        # logits: [B, vocab] or [B, T, vocab]. Use the last time-step if 3D.
        last = logits if logits.dim() == 2 else logits[:, -1, :]
        # Compute top-2 margin on-device
        top2 = torch.topk(last, k=2, dim=topk_dim).values  # [B, 2]
        margin = (top2[:, 0] - top2[:, 1]).mean()      # scalar tensor on device
        nonlocal ema_conf
        ema_conf = (1 - alpha)
                   * (ema_conf if ema_conf is not None else margin)+alpha*margin
        return ema_conf  # device scalar

    # Decode
    for step in range(max_steps):
        # 1) Precision context (exactly one).
        # No nested contexts, no leakage across iterations.
        with _precision_context(device, use_fp8, prefer_bfloat16, enable_fp8):
            # Forward pass (HF-style or plain)
            try:
                logits = model(input_ids=tokens)
                if hasattr(logits, "logits"):
                    logits = logits.logits
            except TypeError:
                logits = model(tokens)

            # 2) Pick next token from the *last* position
            last_step_logits = logits if logits.dim() == 2 else logits[:, -1, :]
            next_token = torch.argmax(last_step_logits, dim=-1,
                                      keepdim=True)  # [B, 1]
            tokens = torch.cat([tokens, next_token], dim=1)

        # 3) Update on-device EMA signal every step (no host sync yet)
        conf_dev = _update_confidence_ema(logits)


        # 4) Periodically re-evaluate precision choice on host
        # to avoid per-step sync
        if (step + 1) % reeval_interval == 0:
            conf_value = float(conf_dev)  # exactly one tiny sync every N steps
            if not use_fp8 and enable_fp8 and _TE_AVAILABLE
               and (conf_value > enter_fp8_threshold):
                use_fp8 = True
            elif use_fp8 and (conf_value < exit_fp8_threshold):
                use_fp8 = False

        # 5) EOS handling
```

```
            if eos_id is not None:
                if (tokens[:, -1] == eos_id).all():
                    break

        return tokens

# ----------------------------
# Example (commented):
# ----------------------------
# model = ...  # your TE-enabled model (or any torch.nn.Module)
# input_ids = torch.randint(0, vocab_size, (batch_size, seq_len))
# out = decode_with_dynamic_precision(model, input_ids, max_steps=128,
# eos_id=tokenizer.eos_token_id)
# print(out.shape)
```

这里可以看到，按本书写作时，PyTorch `autocast` 仅支持 FP16 和 BF16 这两种降精度路径。若要让受支持模块走 FP8 内核，需要使用 Transformer Engine 库；这也要求模型实际采用了 TE 支持的模块。

> 本示例使用的阈值（进入 = 6.0、退出 = 3.0）应使用有代表性的提示词在验证集上校准，避免延迟收益影响准确率。

该模式创建了弹性精度机制，旨在最大化吞吐量。当模型处于可预测区域（如低熵区域），例如生成标点或模板化续写时，可继续使用 FP8 以提高性能；当进入高熵片段，如含糊提示词或决策点时，则回到 FP16 以保持数值准确性。

结合现代 GPU 的低精度运算支持，按 token 动态精度切换可作为高吞吐、延迟敏感推理的一种自适应策略：仅在适合时使用低精度，降低计算开销，并力求在多种提示词条件下保持响应质量。其端到端收益必须与置信度计算、同步和切换成本一起实测评估。

Transformer 自注意力和 MLP 路径的内核自动调优

神经网络层在 GPU 上的性能会随线程块大小、tile 维度、循环展开和内存访问模式等底层参数而显著变化。对于固定尺寸模型，库通常只选择一次这些参数，往往基于通用启发式或离线调优。

但在在线推理服务中，序列长度、批大小等输入尺寸会随请求变化。内核自动调优指一种运行时机制：为当前负载选择最优内核变体，甚至 JIT 编译相应变体。

对于大型 Transformer 模型，推理的两个主要计算阶段是自注意力和前馈 MLP 层；两者的 GPU 内核都可从自动调优中受益。下面分别讨论。

考虑一个具有 H 个注意力头、处理长度为 L 的序列的注意力层。注意力有许多实现，包括标准注意力、优化的 FlashAttention 及其多个变体。

FlashAttention 及其变体凭借 tile 化、并行和内存访问改进，通常对长序列显著更快；但对很短的序列，其开销可能超过收益。动态引擎可依据序列长度 L，在 FlashAttention 内核与较简单内核间切换。

例如，若请求有 $L = 256$ 个 token，引擎可使用一次性通过全局内存读取计算注意力的直接内核启动，这对较小 L 已足够。若另一个请求有 $L = 2{,}048$，则可切换为 FlashAttention 的专用 tile 内核：它通过在共享内存复用数据、避免不必要的 HBM 读取，已知可更好地随大 L 扩展。下面用基于输入序列长度的条件语句说明：

```
// Note: example threshold shown here
if (seq_len < 256) {
    // global-memory version, best for small L
    attn_kernel = standard_attention_kernel;
} else {
    // tiled loads, best for large L
    attn_kernel = tiled_attention_kernel;
}
output = attn_kernel(Q, K, V, mask);
```

在底层，`attn_kernel` 会在完全不同的 CUDA 实现之间选择：一种使用默认注意力内核、针对小输入优化；另一种使用 tile 内核、针对大上下文优化。

理想的 tile 维度取决于 GPU 的共享内存容量和计算资源。CUTLASS、OpenAI Triton 等框架包含自动调优器：它们在初始化时，甚至自适应地在运行时，对一系列 `(TILE_Q, TILE_K)` 组合做基准测试，以选择最快变体。表 19-3展示不同 tile 大小在 Blackwell 级 GPU 上的性能示例。

表 19-3. tile 大小选择对共享内存占用、SM 占用率和实现吞吐量的示例影响（实际数值取决于线程块维度、时钟频率和其他微架构因素）

Tile 大小	共享内存（KB）	占用率（%）	吞吐量（GOPS）
64 × 64	48	85	8.2
128 × 64	64	78	10.5
128 × 128	96	72	9.8
256 × 128	128	60	11.3

根据输入在运行时选择恰当变体，可避免一刀切方案造成的大幅性能断崖。实践中可在目标硬件上做基准测试，发现 $L = 128$ 左右可能是盈亏平衡点；实际阈值应以模型、内核和硬件测量为准。

接下来在自动调优背景下分析前馈 MLP 内核。前馈层本质上是大型矩阵乘法，具体为两次线性投影，中间夹一个非线性激活。

PyTorch 等现代 AI 框架使用 cuBLAS、CUTLASS 等优化 CUDA 库提供的高性能 GEMM 内核。对于给定矩阵大小，这些库通常有多个算法变体，采用不同的 tile 策略、不同 Tensor Core 路径和独立后备路径。

例如，NVIDIA 的 cuBLAS 和 cuBLASLt 库可通过先尝试若干算法、再选择给定维度最快算法的方式自动调优 GEMM 内核。但这通常在首次遇到该形状的 GEMM 时发生，之后未必重新评估。

若 cuBLAS/cuBLASLt 或自定义内核可用，可通过 programmatic dependent launch（PDL）缩小启动间隙、改善稳态吞吐量。务必进行性能分析以确认重叠效果。

在会遇到多种批大小的推理服务器中，可显式调用这类自动调优机制，或维护最佳算法缓存。例如，对于 MLP 形状为 `[batch_size, hidden_dim] x [hidden_dim, 4*hidden_dim]` 的 GEMM，`batch_size = 1` 与 `batch_size = 16` 的最优内核可能不同。

引擎可检测新批大小，并使用 cuBLASLt 或自定义实现对候选内核执行快速微基准测试，选出最快内核；该批大小后续调用即可直接使用已选内核。

此外，一些推理框架和运行时使用 OpenAI 的 Triton GPU 内核领域专用语言（DSL），以自动调优的 tile 大小即时编译注意力和 MLP 内核。运行时会生成若干具有不同 tile 大小（如 128 × 128、64 × 256）的内核变体，并针对实际硬件与输入形状测量哪一个表现更佳。

可使用 Nsight Systems 等工具并排实测分析不同内核变体。

具体地说，Nsight Systems 提供详细 CUDA 时间线，包括 `memcpy` 和 NVLink 活动；Nsight Compute 提供内存工作负载分析，有助于将缓存与内存行为归因到具体内核位置。这对评估 tile 大小与共享内存权衡特别有用，也常能揭示 L2 缓存未命中等不明显的瓶颈，进一步指导调优决策。

由于 L2 缓存效应、内存 bank 冲突等原因，硬件在负载下有时难以完全预测，实证调优通常优于理论猜测；但以合理的理论值作为调优起点仍然有益。

动态 tile 切换会影响 GPU 占用率，因此选择 tile 大小时应考虑这一点。较大 tile 可提高复用、降低内核启动开销，但也会使用更多寄存器和共享内存，从而可能减少可并发运行的线程块数、降低占用率。合格的自动调优器会考虑这种权衡。

在注意力内核中，较大 tile（如 128 × 128）可最大化共享内存中的数据复用。它适合长序列：可发出更少的全局内存读取、摊销循环开销，并产生更高的持续吞吐量。

但对较短序列，同样的大 tile 可能消耗过多共享内存，限制每个 SM 并发运行的线程块数（即占用率）。为短序列减小 tile 大小（如 64 × 64）可释放共享内存，以并行调度更多块，从而提高 SM 占用率、降低单内核延迟。

依据输入序列长度自适应 tile 大小，可使内核在多数情况下达到接近最优的占用率。一些系统甚至在运行时查询 CUDA Occupancy API，动态选择线程块大小等内核启动参数。下面给出 C++ Occupancy API 示例，Python API 也可用：

```
// Pseudocode for occupancy-based launch configuration
int maxBlocks, bestThreads;
for (int threads = 64; threads <= 256; threads *= 2) {
    cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &maxBlocks, MyKernel, threads,
        sharedMemPerBlock(threads));
```

```
        // choose "threads" to maximize occupancy
        // (remember to not exceed the max threads per SM limit (e.g., 2,048)
        float occupancy = (float) maxBlocks * threads /
                                  hardwareMaxThreadsPerSM;
    }
```

这段伪 C++ 展示了如何为内核评估不同线程块大小。它根据共享内存用量检查每个 SM 可运行多少块，之后内核启动相应调整线程数或共享内存用量。高性能框架和推理引擎会在内部按以下步骤自动化这类逻辑：

1. 测量工作负载

检查下一次模型前向传播的当前输入维度（批大小 `B`、序列长度 `L` 等）。

2. 选择候选内核

确定各组件可用的内核实现，例如注意力阶段的标准注意力或 FlashAttention，以及 MLP 阶段合适的 GEMM 算法。

3. 估算或基准测试

在样本数据上让每个算法运行若干次，对各候选执行快速测试。

4. 选择最佳变体

根据测得维度，选择执行时间最短或吞吐量足够的内核。

5. 缓存结果

将选择结果存入以输入维度或工作负载签名为键的查找表。之后出现相似请求时，无须重复这些步骤即可获知最佳内核。

6. 执行

使用选定的内核实现运行模型层。

> 这类似于数据库查询优化器选择查询计划；此处的“计划”就是被选中的内核实现。

遵循该过程，推理运行时可持续自我调优。随着时间推移，系统为短/长提示词、小/大批次等场景构建优化路径库。可通过异步调优来保持即时调优开销较低——当前推理使用默认内核时，在独立 stream 测试新内核——或者在低流量时段进行，以免影响延迟。

建议在模型加载时纳入初始预热阶段，运行多种样本输入以触发自动调优；其中可包括最大序列长度、最大批大小等极端情况，使引擎预先为这些情况优化内核。

还应在运行时监测每一层的执行时间。若某层因输入特征变化突然成为瓶颈，就应重新评估内核选择。

一些高级框架甚至使用多臂老虎机算法，持续探索替代内核，并随条件变化更新内核选择。

简言之，自动调优将静态内核转化为自适应内核。它能针对每组输入尽可能挖掘 GPU 集群性能；实际收益取决于调优候选、缓存命中、调优开销和工作负载。

动态共享内存分配与占用率感知的内核选择

与内核调优紧密相关的是 GPU 共享内存和整体流式多处理器（SM）占用率的管理。现代 GPU 每个 SM 都配有大量共享内存。根据问题大小、当前共享内存利用率与占用率，在运行时动态分配线程，可显著改善整体 AI 系统性能。

动态共享内存分配会依据问题大小调整每个线程块使用的共享内存量。占用率感知的内核选择则会选择能最佳利用 SM 资源（包括寄存器、共享内存和 warp）的内核启动参数，使 GPU 保持忙碌。

选择 tile 化注意力算法时，应平衡数据复用与 SM 占用率。例如，将 `T` 视作每线程块以 token 计的 tile 宽度。按本书的概念化简化模型，由于自注意力具有二次特性，每个线程块为保存 query、key、value 块会在共享内存中预留约 $O(T^2)$ 个 float。实际 FlashAttention 类内核的共享内存公式还取决于头维度、流水化阶段和具体实现，不能一概而论。

较大 tile（如 `T=256`）只需从 DRAM 加载每个 key/value 块一次，之后供许多 query 复用，从而降低全局内存流量。但每线程块会使用较多共享内存，受硬件限制，每个 SM 同时只能运行较少线程块，因而降低占用率。例如，若 `T=256` 时每 SM 仅能运行 1 个块，而 `T=128` 时能运行 4 个块，使用 `T=256` 时可能仅有 30% SM 占用率。

较小 tile（如 `T=64`）使用的共享内存少得多，因此每个 SM 能容纳更多线程块，更好地隐藏延迟并提高利用率；但同一 key/value 数据会被更频繁地重新加载，增加 DRAM 访问。

最佳 tile 大小 `T` 取决于序列长度 `L`、GPU 共享内存容量和 SM 数等因素。tile 应足够大以摊销 DRAM 读取，又足够小以保持较高占用率，使许多线程块可在 SM 上并发活跃。

实践中，可手动选取 64、128、256 等候选 `T` 值，并在特定硬件上以代表数据集的序列长度 `L` 对每个值进行基准测试，选择总体吞吐量最佳的 `T`。也可不预先硬编码 `T`，而是在启动内核前计算，如下所示：

```
int T = choose_tile(L, gpu_shared_mem_per_block, num_sms);

// calculate shared memory in bytes based on the tile size
// (multiplying by 3 for Q, K, and V)

size_t shared_mem_bytes = 3 * T * T * sizeof(float);

numBlocks = ...

MyAttentionKernel<<<numBlocks, threadsPerBlock, shared_mem_bytes>>>(...);
```

这里，T 根据序列长度 L、每线程块的 GPU 共享内存上限 gpu_shared_mem_per_block 以及 SM 数 num_sms 计算。随后每线程块共享内存 shared_mem_bytes 依据计算出的 tile 大小 T 在运行时得出。

之后可使用共享内存参数 shared_mem_bytes 启动 CUDA 内核。内核本身包含下面的定义，以声明 extern __shared__ 数组，为每线程块分配大小为 shared_mem_bytes 的共享内存缓冲区：

```
// holds 3 tiles of T×T floats for Q, K, and V
extern __shared__ float smem[];

// Q tile: smem[0 ... T*T-1]
float* tile_q = smem;

// K tile: smem[T*T ... 2*T*T-1]
float* tile_k = smem + T*T;

// V tile: smem[2*T*T ... 3*T*T-1]
float* tile_v = smem + 2*T*T;
```

通过每次启动改变 shared_mem_bytes，同一内核二进制可使用不同 tile 大小运行。选择 T 后，可用 CUDA Occupancy API 查询占用率，查看每个 SM 能容纳多少块。

若占用率过低、每个 SM 只分配一个块，可减小 T；若 DRAM 出现抖动，则可增大 T。这可实现为自动反馈回路：利用 CUDA Occupancy API 或 NVIDIA Data Center GPU Manager（DCGM）测量占用率相关信号，并在后续迭代调整 T。注意，Occupancy API 主要给出基于资源限制的理论可驻留度，实际达到的占用率及瓶颈仍须由 profiler 验证。这样每个注意力层可根据当前序列长度 L 和硬件限制使用合适配置。

如第 6 章所见，优化 SM 占用率时还须考虑每线程寄存器用量。使用更多寄存器（如循环展开）可加快单线程性能，但因每个 SM 的寄存器文件有限，可能降低整体 SM 占用率。

若每个 warp 使用许多寄存器，可调度的 warp 就更少。动态运行时可检测内核是否因寄存器触及占用率限制，并切换至使用较少寄存器的版本，代价是更多指令。这些底层考量对自适应高性能推理服务器至关重要。

动态共享内存调优需要分析占用率与吞吐量的关系。NVIDIA Nsight Systems/Compute 和 CUDA Occupancy API 等工具可显示每个内核的占用率相关信息与执行效率；DCGM 则在系统层面提供实时 GPU 利用率和 SM 占用率指标。例如，自适应系统可借此发现：由于每线程块使用大量共享内存，序列长度为 2,048 的某个注意力内核可能只达到 30% 占用率。

此时，系统可动态切换至通过将注意力计算拆为两遍、降低每线程块共享内存的内核配置。若内存延迟是瓶颈，这将提高占用率，并可能提高吞吐量。

反之，若内核受内存限制且未使 ALU 饱和，使用更多共享内存即使降低占用率，也可能通过减少内存停顿提升有效吞吐量。理解这些权衡很重要，尤其占用率有时比其他指标更不直观。

建议设计允许在运行时调整共享内存和线程块大小的内核。系统随后可依据输入和硬件反馈，将调优配置适应于运行时条件。例如，可为 CUTLASS 等库使用的 tile 大小提供运行时参数和模板参数；这些库正因此提供可在运行时选择的内核变体。

还应持续监控 SM 利用率指标。例如，大量空闲 warp（如活跃 warp < 50%）或内存停顿周期很多（如 > 70% 停顿）都可能表示不平衡：要么占用率过低（warp 空闲），要么 tile 过小导致过多内存流量。因此，系统应相应调整以恢复平衡。

在推理服务中，常为不同问题大小维护一个最优线程块配置的小表。可将这种映射实现为 JSON 或配置文件，把序列长度区间映射到启动参数；随模型和硬件演进，这种方式便于更新。

例如，系统对序列长度 512 执行注意力时，可使用 128 threads/block 和 16 KB 共享内存；对序列长度 4,096，则可使用 256 threads/block 和 64 KB 共享内存等。这将自动调优概念延伸至资源分配。

请记住，现代 NVIDIA GPU 为 L1 数据缓存与共享内存提供统一的片上池；carveout 控制其中有多少保留给共享内存、多少给 L1。当内核需要更大 tile 时，可用 `cudaFuncSetAttribute` 调整 carveout，提高共享内存可用比例。

现代 NVIDIA GPU 的设备驱动允许设置 L1 缓存与共享内存的划分比例，即“carveout”百分比。因此，可按用例将 SM 配置为偏向更多共享内存或更多 L1 缓存。例如，当内核需要更大 tile 时，可增加分给共享内存的比例。

> carveout 是每内核属性，只是提示而非保证；它是用于平衡占用率与缓存行为的另一个可调旋钮。

复杂的运行时可在启动时对特定内核使用 `cudaFuncSetAttribute()` 和 `cudaFuncAttributePreferredSharedMemoryCarveout` 切换这一 carveout 百分比。例如，若注意力内核使用非常大的 tile、需要更多共享内存，可尝试将 L1 降至 25%、将共享内存提高至 75%（假设初始 carveout 值为 50%）。

> 共享内存与 L1 的 carveout 属性是提示而非保证。应始终将该设置视为提示并用性能分析验证效果，确认请求的设置确实影响了占用率和缓存行为。

简言之，动态共享内存和占用率感知技术旨在针对给定任务使每个 SM 尽可能保持忙碌，并让内核资源使用适配具体用例。对于一些层或批大小本会使 SM 利用不足的大模型，这类技术尤为重要。

用于加快 TTFT 的推测性 KV 预取

在实时场景服务 LLM 时，首 token 时间（TTFT）是一项关键指标，它衡量系统产生模型响应首个 token 的速度，并直接影响终端用户体验。

大型模型 TTFT 的一个主要构成是，在可开始生成 token 前建立模型内部状态（如键值 KV 缓存）所花的时间。正如前章所述，注意力 KV 缓存存放每层历史 token 的键和值投影。

推测性 KV 预取是一种优化：系统预测生成第一个 token 所需的数据，并提前将其加载到 GPU。这会将 KV 缓存准备与计算等其他步骤重叠，使 token 生成能更快开始。SpeCache 是推测性 KV 缓存的一个研究示例，如图 19-3所示。

在 SpeCache 中，KV 缓存被压缩（此处为 16 位），并按层移出 GPU，从而降低内存占用。生成第一个输出 token 后，会计算一个推测的“下一” token；同时，模型预取该首次 decode 步所需的相应降精度 KV 对。

在每个后续步骤中，模型并行 decode 两个 token，包括实际输出 token 和推测 token。两个结果都会输入下一步；每步前会预取推测路径最相关的 top-k 个 16 位 KV 对，使两条路径都准备好所需 KV 缓存。简言之，SpeCache 在其评测中通过预取降精度 KV 并与计算重叠，报告了 TTFT 改进。

> 只有在验证访问模式和存储层级后，才应集成推测性预取技术。

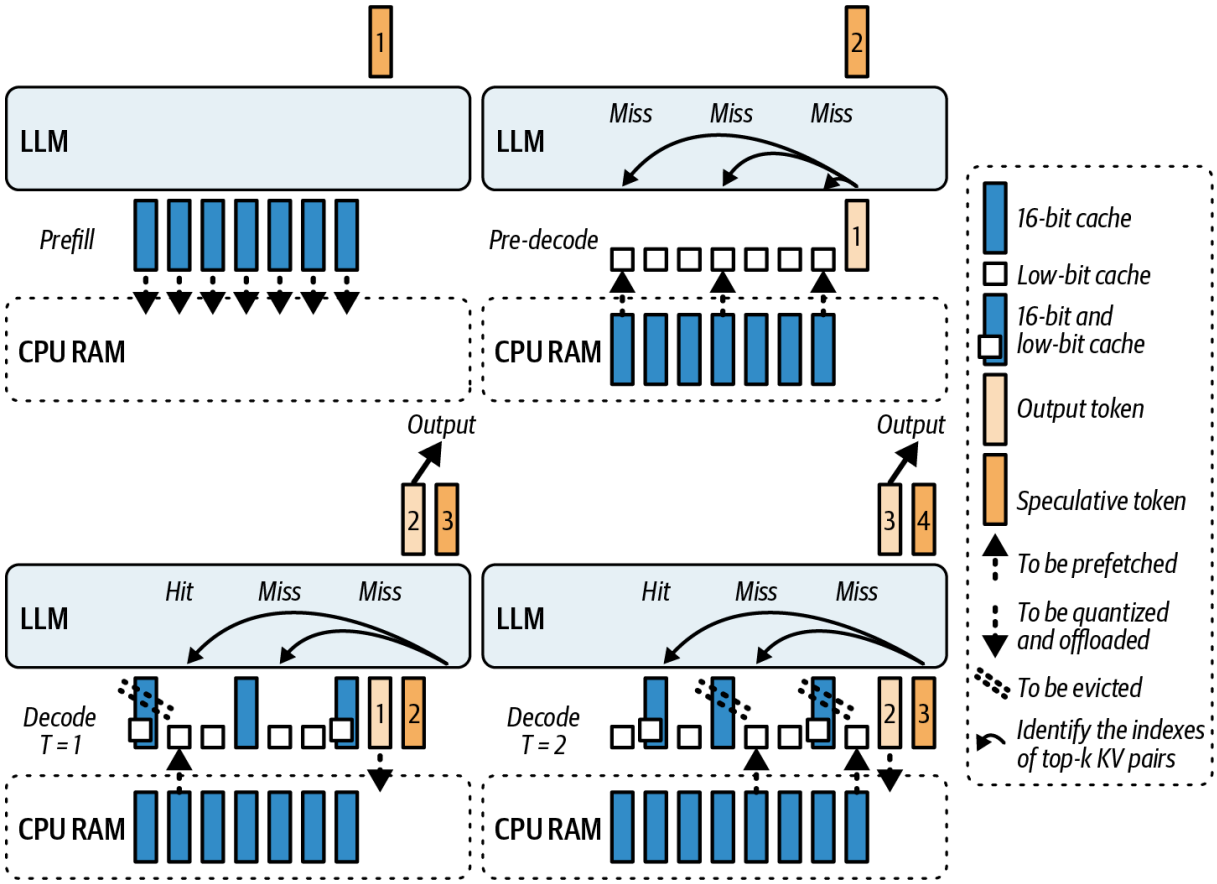


图 19-3. 使用 SpeCache 的推测解码（来源：https://oreil.ly/b21E5）

由于现代 LLM 的层数、不断扩大的上下文窗口以及现代“思考”模型生成的大量推理链，KV 缓存可能极大。现代推理系统常在 GPU、CPU 内存和 SSD 间换入换出 KV 缓存以更好地管理容量，尤其是无法装入 GPU 内存的极长上下文。

生成新 token 时，朴素方法是同步地从 KV 所在位置（部分在 GPU、部分在 CPU）取回数据，再计算下一个 token 的 decode。这会增加显著延迟，尤其缓存已被换出到 CPU 内

存或 NVMe 存储时。

KV 缓存预取通过提前开始 KV 数据传输来提供帮助。用户提示词一收到，服务器便可开始把所需 KV页以及模型权重直接复制到 GPU 内存。到模型在 prefill 阶段完成提示词计算时，生成第一个输出 token 所需的数据已经就位。

具体来说，该机制仅将当前层的 KV 保留在 GPU 内存，并将其他层 KV 卸载到 CPU。计算当前层时，它异步预取下一层缓存到 GPU，同时将上一层缓存写回 CPU。

通信与计算的这种重叠使 GPU 很少等待数据。因此，使用 CPU 内存中卸载的 KV 缓存对延迟的影响可能较小；但仍有传输开销。例如，在合适的重叠条件下，卸载后 tokens/s 吞吐量可能降低约 5%–10%，实际取决于 CPU DRAM 带宽、PCIe/NVLink、模型层数、批大小和序列长度。

> 重叠可掩盖 CPU 驻留 KV 的大部分延迟，但通常仍会因 CPU DRAM 带宽和 PCIe 开销而有吞吐损失。应针对自身的批大小和序列长度反复分析性能。

Hugging Face Transformers 库中 KV 缓存卸载的一个例子是 `OffloadedCache` 机制。调用 `generate(cache_implementation="offloaded")` 或 `generate(cache_implementation="offloaded_static")` 时可启用它，从而使用 Transformers 库生成 token，如下所示。它的接入成本较低，但收益大小依工作负载而定：

```
# Dynamic, variable-length serving and sliding layers
# (recommended default)
out = model.generate(..., cache_implementation="offloaded")

# Static shapes + torch.compile and CUDA Graphs
# (highest throughput with fixed shapes, use with torch.compile)
# out = model.generate(..., cache_implementation="offloaded_static")
```

在底层，生成开始时，`OffloadedCache` 会确保第 1 层的 KV 移入 GPU。第 1 层计算期间，`OffloadedCache` 为第 2 层的 KV 从 CPU 向 GPU 发起异步 DMA，依此类推；它始终提前一层进行预取。

当前向传播到达第 2 层时，其 KV 已在本地，从而减少每层同步复制本会造成的停顿。介绍完普通 KV 预取后，接下来讨论推测性 KV 预取。

推测性 KV 预取超越了常规 KV 预取仅提前一层的前瞻范围。设想一种推理服务器配置：它有多个模型副本，或具有多个可能路径，例如 MoE 模型中一个 token 可被路由到若干专家网络之一。

KV 预取有助于阶段边界处的衔接。理想情况下，到 prefill 阶段结束时，各层缓存已经位于 GPU 内存，或已排队等待移入 GPU 内存。这样生成开始后模型无须等待内存传输，因而可直接降低 TTFT。

建议使用 NVTX marker 等追踪工具持续监测 TTFT，并测量首 token 的 decode 时间，以精确量化 TTFT。若 decode 阶段刚开始就出现异常的空闲时间尖峰，说明可能错过了预取机会。

要在自己的栈中实现 KV 预取而不阻塞推理，可按第 11 章所述，使用 CUDA stream 来重叠传输与计算，使预取与主计算 stream 并发运行。之后仅在需要预取数据时使用 CUDA event 同步这些 stream，如下所示：

```
// kv_prefetch_overlap.cu

#include <cstdio>
#include <cuda_runtime.h>

// Example sizes
static constexpr size_t KV_BYTES =
  /* set to your chunk size */ 8ull<<20; // 8 MiB

__global__ void forward_kernel(/* ... */) {
  // compute logits for current token ...
}

__global__ void consume_prefetched_kv(/* use prefetch_dest */) {
  // consumes KV in prefetch_dest ...
}

int main() {
  // Allocate destination buffer on this GPU
  void* prefetch_dest = nullptr;
  cudaMalloc(&prefetch_dest, KV_BYTES);

  // Example: staging source on host. MUST be pinned for real overlap.
  void* kv_src_host = nullptr;
  cudaMallocHost(&kv_src_host, KV_BYTES);  // pinned (page-locked)
  // Fill kv_src_host with data for the first iteration...

  cudaStream_t compute_stream, prefetch_stream;
  cudaStreamCreateWithFlags(&compute_stream, cudaStreamNonBlocking);
  cudaStreamCreateWithFlags(&prefetch_stream, cudaStreamNonBlocking);
  cudaEvent_t kv_ready;
  cudaEventCreateWithFlags(&kv_ready, cudaEventDisableTiming);

  bool done = false;
  while (!done) {
    // 1) Launch compute for current token
    forward_kernel<<< /*grid*/1, /*block*/1, 0, compute_stream>>>();

    // 2) Asynchronously prefetch next KV chunk
    // If your source is another GPU, use cudaMemcpyPeerAsync
    // and enable peer access.
    cudaMemcpyAsync(prefetch_dest, kv_src_host, KV_BYTES,
                              cudaMemcpyHostToDevice, prefetch_stream);
    cudaEventRecord(kv_ready, prefetch_stream);

    // 3) Ensure consumer on compute_stream waits just-in-time
    cudaStreamWaitEvent(compute_stream, kv_ready, /*flags*/0);

    // 4) Launch work that consumes the prefetched KV
    consume_prefetched_kv<<< /*grid*/1, /*block*/1, 0, compute_stream>>>();
```

```
      // 5) ...advance state, update kv_src_host for next iteration, set `done`
      done = true; // demo
    }

    cudaEventDestroy(kv_ready);
    cudaStreamDestroy(prefetch_stream);
    cudaStreamDestroy(compute_stream);
    cudaFree(prefetch_dest);
    cudaFreeHost(kv_src_host);
    return 0;
}
```

在该设置中，`cudaMemcpyAsync` 在 `prefetch_stream` 上运行，而 `model.forward()` 使用 `compute_stream`。这让 CUDA 驱动可重叠数据传输与计算。仅当实际需要预取 KV 数据时，才在继续消费该数据的计算前等待 `kv_ready` event；该 event 在交接点实施即时同步。

> 务必让主机缓冲区固定（page-locked）；否则 `cudaMemcpyAsync` 可能串行化，无法获得期望的复制/计算重叠。若 KV 源在另一 GPU，使用 `cudaMemcpyPeerAsync` 并启用 peer access。若使用 Unified Memory（如 Grace Blackwell、Vera Rubin 超级芯片），可考虑使用 `cudaMemPrefetchAsync` 提前暂存页面。若模式可重复，还可用 CUDA Graphs 捕获该序列，从而在频繁预取时进一步降低内核启动开销。

使用独立 stream 可实现高效流水线。当一个 token 正在生成时，下一个 token 的 KV 缓存可被预取而不打断计算 stream。这样通过隐藏传输延迟、持续为计算单元提供数据来提高 GPU 利用率。现代 LLM 推理引擎可在分页 KV 缓存等机制中自动采用类似重叠。

最好将权重和 KV 缓存的数据移动视为整个推理流水线的一部分。正如应将计算操作流水化一样，也应将数据移动流水化：当前计算进行时，让下一份所需数据在传输途中。KV 缓存压缩是改善 KV 缓存层性能的另一选项，下面讨论。

实时 KV 缓存压缩与策略切换

LLM 在会话中生成越来越多 token 时，KV 缓存线性增长。对长对话、文档和推理链，KV 缓存会消耗大量 GPU 内存，且往往是主要内存消费者。

KV 缓存很适合压缩/量化。和任何压缩形式一样，KV 缓存压缩会降低其内存占用；实时执行则意味着在推理期间即时压缩。

策略切换是指压缩策略可根据当前上下文改变。目标是在需要时释放内存和网络带宽，同时尽量不影响模型准确率或减慢涉及 KV 缓存数据的计算。图 19-4展示了几种 KV 缓存压缩算法。

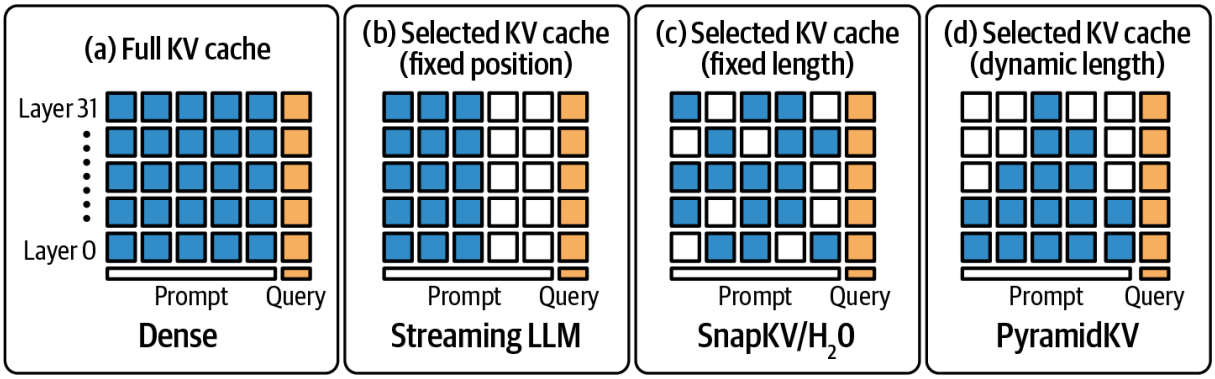


图 19-4. 不同 KV 缓存算法，包括无选择性缓存（如 Dense）

一种直接、简单的 KV 压缩方法是降低其精度。许多框架的 KV 缓存默认使用 FP16 或 BF16，因为模型激活通常使用 16 位。但键和值往往可压缩至 8 位甚至 4 位，而对输出质量的影响可能较小；这种影响仍取决于模型、上下文位置、量化方案和任务，必须验证。

Hugging Face Transformers 库支持用于 KV 内存的 `QuantizedCache`，包括 INT8 和 INT4。可通过指定 `cache_implementation="quantized"` 与特定位宽，用一行启用该特性。其代价是量化/反量化增加少量计算，换取显著内存节省；整体模型质量是否受损及受损程度依模型、后端、位宽和工作负载而定。

> 同时使用量化和 CPU offload 时，应确保主机缓冲区被固定（page-locked），以避免传输串行化，并有助于维持复制带宽（如 PCIe/NVLink）。

接下来讨论动态策略切换。一个策略示例是保留最后 128 个 token 的完整精度，而将其他 token 压缩为 4 位。这样，最近上下文通常可用更高精度保留，较早历史则以较低精度存储以节省空间；但“最近一定最重要”并不是所有任务、模型和注意力模式下都成立。

若模型突然需要关注较早 token，影响不一定会很严重，因为许多 LLM 确实表现出近因偏置，即偏向最近上下文；但长距离检索、代码和文档问答等任务可能高度依赖早期内容，因此应进行评测。

还可根据用户提示词长度调整该窗口。例如，极长提示词使用更大的全精度窗口，或在 GPU 内存使用超过阈值时更激进地压缩。

策略也可基于内存使用。例如，可规定 GPU 内存使用超过 80% 时，将整个 KV 缓存压缩为 8 位，以帮助避免长生成中的 OOM。策略可包含多层压缩：轻度压力时压缩至 8 位，极端压力时再改为 4 位。

真正动态、实时的策略切换可让引擎在生成 token 期间改用不同压缩方式。这种实现需要同时维护缓存的多个表示：例如，初始以 FP16 存储 KV，同时维护同一数据的 INT8 版本。

系统默认使用 FP16；若内存利用率跨过某阈值，可开始使用带合适缩放因子的 INT8 版本，并释放 FP16 内存以缓解压力。之后的注意力读取从 INT8 存储取得反量化值。

但这需要谨慎同步，确保压缩版本在需要时已更新完毕、可供使用。双缓冲和后台压缩线程等技术在此有用。

通常 CPU 线程可利用向量化 INT8 量化操作异步执行压缩，准备好后再将压缩块复制到 GPU 内存；是否合算取决于 CPU 算力和互连带宽。

> 应在安全点（如一次迭代结束）执行实时策略切换，以避免在计算中途切换，并可在后台 stream 执行重新量化来隐藏其延迟。

还存在无损压缩等其他技术，它们使用熵编码和聚类来压缩激活、做到逐位不丢失信息。但这些实现复杂，即使在 GPU 上也可能太慢，难以实时执行。

可考虑分块 ZFP（一种浮点压缩）等更简单机制，或通用的基于 CPU 的压缩；但截至本书写作时，最简单、通常最有效且支持最广的方法仍是量化。

> 按本书写作时，ZFP 等无损方法虽在离线和研究场景中被评估，但相对量化缓存受吞吐约束，在生产 LLM KV 缓存路径中仍不常见。因此，量化凭借速度与约 2–4 倍内存缩减的平衡，仍是首选方法。

为尽量减少质量影响，可尝试按 head 或按 token 缩放。KV 缓存量化是否以按 head、分组缩放优于按 token 缩放，取决于后端、模型与形状；Hugging Face `QuantizedCache` 的实现会按注意力 head 校准值范围。

具体来说，`QuantizedCache` 实现了逐通道、分组量化，具有可配置组大小和残差窗口；残差窗口将最近 token 保留为原始精度。通过设置 `cache_implementation="quantized"` 并将 `cache_config` 作为字典传入即可启用。可计算张量中的最大绝对值，并据此缩放 4 位或 8 位量化；本质上这是一种 min-max 或基于幅值的量化。

`QuantizedCache` 的一种有用实现是 半二次量化（Half-Quadratic Quantization，HQQ）后端。HQQ 提供免校准的即时量化器，支持 2 位、3 位、4 位和 8 位等多种低比特格式。它使用鲁棒优化来建模异常值和重尾误差分布，并可集成到 Hugging Face Transformers 的 KV 缓存实现中，提供 PyTorch 和自定义 CUDA 内核实现以支持快速推理；可用后端和位宽以所用 Transformers/HQQ 版本为准。

可实现一个动态策略，根据内存压力在 8 位量化与 4 位量化间切换。数值的尖锐度或分布也可指导决策：若缓存值大多很小、方差低，可能可更激进量化。实时切换压缩策略可与 Hugging Face `QuantizedCache` 机制结合，但需要评估重新量化成本与数值影响。

遗憾的是，Transformers 不支持原地改变已初始化缓存对象的位宽。为实现动态策略，代码可将 token 分成小块生成，并在存在内存压力时，以新的量化缓存配置启动下一块。这种实现类似于命中内存不足（OOM）错误后回退至 offloaded cache。代码如下：

```
# dynamic_quantized_cache.py
# Dynamic KV cache policy using Hugging Face Transformers' QuantizedCache (2025).
# Starts with int8 HQQ and drops to int4 when device memory is tight.
# Requires: transformers >= 4.55, hqq (for HQQ backend).
#
# This uses only public APIs:
#   - cache_implementation="quantized"
```

```python
#   - cache_config as a dict
#
# References:
#   - KV cache strategies docs (QuantizedCache, HQQ/Quanto backends)

from __future__ import annotations
from typing import Dict, Optional
import logging
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer


def make_cache_config(
    *,
    backend: str,
    nbits: int,
    device: torch.device,
    compute_dtype: torch.dtype = torch.float16,
    q_group_size: int = 64,
    residual_length: int = 128,
    axis_key: int = 1,
    axis_value: int = 1,
) -> Dict:
    """
    Build a cache_config dictionary accepted by Transformers' quantized cache.
    HQQ supports nbits in {2, 4, 8}; Quanto supports {2, 4}.
    axis_key/axis_value=1 are recommended for HQQ.
    """
    return {
        "backend": backend,                 # "HQQ" or "quanto"
        "nbits": int(nbits),
        "axis_key": axis_key,
        "axis_value": axis_value,
        "q_group_size": int(q_group_size),  # group size along head_dim
        "residual_length": int(residual_length), # recent tokens (orig precision)
        "compute_dtype": compute_dtype,     # dequantization compute dtype
    }


def _gpu_used_ratio() -> float:
    """
    Return fraction of device memory used as 1 - free/total.
    Uses CUDA driver info, which reflects true device state,
    not just the PyTorch allocator's reserved bytes.
    """
    free, total = torch.cuda.mem_get_info()
    return 1.0 - (free / total)


@torch.no_grad()
def generate_with_dynamic_quantized_cache(
    model: AutoModelForCausalLM,
    tokenizer: AutoTokenizer,
    prompt: str,
    *,
    max_new_tokens: int = 256,
    chunk_tokens: int = 32,
    memory_threshold: float = 0.90,   # switch policy if used_ratio >= threshold
    backend: str = "hqq",             # "hqq" or "quanto"
    start_bits: int = 8,              # initial cache bit-width
    fallback_bits: int = 4,           # lower bit-width on pressure
    residual_length: int = 128,
) -> str:
    """
```

```
Generate text in chunks while allowing mid-run policy changes.
The policy applies to each chunk by choosing cache_config for that chunk.
If memory is tight, we switch from int8 to int4 in subsequent chunks.
"""
backend = backend.lower()
assert backend in {"hqq", "quanto"}, "backend must be 'hqq' or 'quanto'"
if backend == "quanto":
    assert start_bits in {2, 4} and fallback_bits in {2, 4},
           "Quanto supports only 2 or 4 bits"
if backend == "hqq":
    assert start_bits in {2, 4, 8} and fallback_bits in {2, 4, 8},
           "HQQ supports 2, 4, or 8 bits"

device = model.device
inputs = tokenizer(prompt, return_tensors="pt").to(device)
generated_ids = inputs["input_ids"]   # [batch=1, seq_len]
tokens_remaining = int(max_new_tokens)
current_bits = int(start_bits)

# Use EOS if available to terminate early.
eos_id: Optional[int] = tokenizer.eos_token_id

while tokens_remaining > 0:
    # Decide policy for this chunk based on current memory pressure.
    if torch.cuda.is_available():
        # Smooth the signal to avoid oscillation
        # when multiple processes are active.

        if 'used_ratio' in locals():
            used_ratio = 0.8 * used_ratio + 0.2 * _gpu_used_ratio()
        else:
            used_ratio = _gpu_used_ratio()
        if used_ratio >= memory_threshold:
            current_bits = min(current_bits, fallback_bits) # drop bits
            logging.info(f"Current bits {current_bits}")


    cache_cfg = make_cache_config(
        backend=backend,
        nbits=current_bits,
        device=device,
        compute_dtype=torch.bfloat16,
        q_group_size=64,
        residual_length=residual_length,
        axis_key=1,
        axis_value=1,
    )

    # Generate a small chunk with the chosen cache policy.
    this_chunk = min(chunk_tokens, tokens_remaining)
    out = model.generate(
        input_ids=generated_ids,
        max_new_tokens=this_chunk,
        do_sample=False,       # deterministic for clarity; adjust as needed
        use_cache=True,
        cache_implementation="quantized",          # select QuantizedCache
        cache_config=cache_cfg,                    # pass backend + settings
        pad_token_id=eos_id,
        return_dict_in_generate=False,        # we only need the tokens here
    )
    # 'out' is [1, old_len + this_chunk]; slice out newly generated suffix
    new_tokens = out[:, generated_ids.shape[1]:]
    generated_ids = out
    tokens_remaining -= new_tokens.shape[1]
```

```
        # Early termination if the model emitted EOS.
        if eos_id is not None and int(new_tokens[0, -1].item()) == eos_id:
            break

    return tokenizer.decode(generated_ids[0], skip_special_tokens=True)


if __name__ == "__main__":
    # Example usage. Replace with a model that supports your hardware.
    ckpt = "meta-llama/Llama-3.1-8B-Instruct"
    tok = AutoTokenizer.from_pretrained(ckpt)
    mdl = AutoModelForCausalLM.from_pretrained(ckpt,
                                               torch_dtype=torch.float16).to("cuda")

    text = generate_with_dynamic_quantized_cache(
        mdl,
        tok,
        "Explain attention key-value caches in one paragraph.",
        max_new_tokens=120,
        chunk_tokens=32,
        memory_threshold=0.90,
        backend="hqq",         # or "quanto" if you installed Quanto
        start_bits=8,
        fallback_bits=4,
        residual_length=128,
    )
    print(text)
```

这里先以 INT8 HQQ 缓存进行适度压缩，在实际 GPU 可用内存降至阈值以下时切换到 INT4。这通过 `torch.cuda.mem_get_info()` 测量，它反映设备内存实际空闲量与总量，为策略选择提供信号。

随后以小块生成 token，因此可在块与块之间安全切换策略，而不必修改既有缓存实例。这避免访问私有属性或手动量化张量；缓存后端在模型前向传播内完成相应工作。

> 如示例所示，策略切换时建议记录事件或递增计数器，以便将压缩事件与准确率或输出异常关联起来。

同样，条件改善时可动态关闭压缩。假设长对话刚结束、下一个问题很短，系统可决定停止压缩，甚至将部分缓存恢复为更高精度，前提是这样会产生更高质量响应；差异可能很小，未必值得付出恢复成本。

避免压缩快速波动很重要，因为频繁开关压缩会造成性能抖动。为此，可在变化间引入有意延迟（即*滞回*和*冷却期*）。例如，切换到更高压缩策略后，保持该策略直至 GPU 内存降到远低于给定阈值，从而避免振荡和抖动。

这种灵活性很有用，例如服务有时为高级用户优先保证最高质量（不压缩），而为免费层用户优先保证最高吞吐量（更激进压缩）。策略也可根据用户订阅类型等请求元数据切换。

讨论缓存还必须考虑最近最少使用（LRU）等驱逐策略。若上下文长度过长，一些具有近因偏置或滑动窗口注意力的模型架构可能选择完全丢弃或下采样很旧的 token。图 19-5展示了滑动窗口注意力。

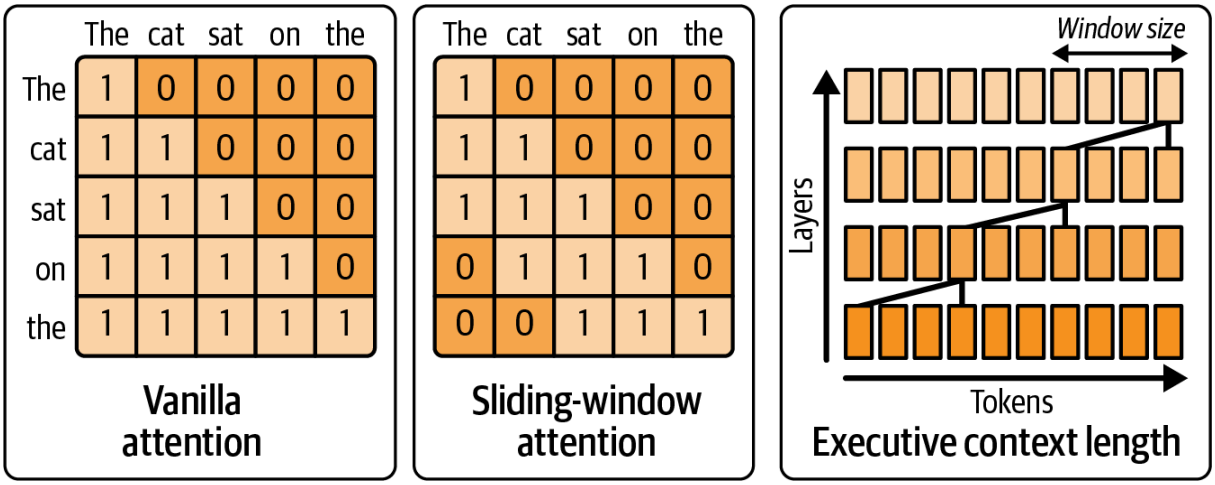


图 19-5. 滑动窗口注意力使用“最近 token 最有用”的直觉

从上下文中 LRU 驱逐较早 token 并不严格属于压缩，却是另一种可在运行时动态选择的策略。例如，系统可依据某些启发式或更小 LLM 判断：超过 2,048 token 后模型可能不再需要较早 token；这种判断必须与模型的实际注意力模式和任务需求相验证。

此时系统可开始丢弃这些旧 token，或定期将其压缩为较小摘要。这已进入模型与算法范畴，需要更多支持、维护和模型训练，但确实是高级服务引擎应考虑的一种动态上下文管理形式。

简言之，应考虑推理引擎提供的量化缓存机制，因为它们可处理维护量化缩放因子、与注意力内核接口及运行时监测 GPU 内存分配器等细节。至少当系统发现内存利用率接近限制时，应记录这一点，并评估在该点启用压缩策略能否避免 OOM 而不损害延迟。

> 实践中，可将 GPU 内存高水位阈值（如 80%）作为触发 8 位压缩的起点，以帮助避免生产环境 OOM；实际阈值须基于模型、并发和预留工作空间实测设置。

若动态压缩策略确有意义，可实现相应触发器。与任何量化和压缩策略一样，务必测试它们对所在领域模型输出的影响。许多生成任务可容忍激进压缩，但仍应验证 4 位相对 8 位不会引入错误或意外输出。

用于运行时调优 AI 系统的强化学习智能体

到目前为止讨论的许多技术，都涉及根据系统当前状态做决策。这些决策常须在速度与准确率、吞吐量与延迟间取舍。与其收集越来越多启发式规则，也可使用具有 RL 智能体、环境、策略和奖励的强化学习来调优推理系统。

> 这是一种前沿方法。应先以更简单的启发式规则作为基线，待基础系统稳定后再将 RL 作为渐进式改进。

具体来说，推理引擎观察服务器指标（环境），选择动作（策略）以在延迟低于目标时最大化吞吐量，并接收指导持续改进的反馈（奖励）。这样系统成为在线优化器，随条件变化不断完善决策。

例如，可构建一个环境，使每个 RL“步骤”对应一个推理请求；RL 动作可包括以下内容：

动作 1

选择并行模式：单卡、TP、PP 或混合。

动作 2

选择精度：全 FP8，或混合 FP8 和 FP4。

动作 3

调整批大小或批等待时间。

动作 4

启用或禁用缓存压缩。

动作 5

启用或禁用推测解码。

动作 6

为推测解码选择较小的 draft model。

动作 7

为推测解码选择较大的 draft model。

动作 8

启用或禁用推测性 KV 预取。

……更多动作……

智能体观察到的当前 RL 状态可包括 GPU 利用率、内存利用率、平均延迟、请求队列长度等。RL 奖励可通过捕获业务目标来定义，例如 `reward = throughput - λ * max(0, latency - SLA)`。λ 是该奖励函数中可调的惩罚权重，决定对延迟违约的惩罚有多严厉。

> 建议将状态特征归一化，使 RL 智能体无须自行学习量纲；这可加快训练收敛。例如，可用最大值缩放队列长度等。

较大的 λ 会使智能体优先保证 SLA，而非榨取额外吞吐量；较小的 λ 则允许它冒偶发延迟超标的风险来获得更高整体 token 速率。实质上，该奖励函数惩罚超过 SLA 的延迟，其他情况下则尽力提高吞吐量。

> 实践中，可从满足“λ × 典型延迟超标量 ≈ 愿意为此交换的吞吐收益”的 λ 开始。例如，若可接受多 10 ms 延迟以换取 100 tokens/sec，则可设置 λ 使 10 ms × λ ≈ 100。

经过多次迭代，RL 智能体可学习何时压缩缓存更有利，例如内存占用较高而延迟尚未立即受影响时；也可学习在 GPU 总体利用率低、但某一 GPU 过载时切换到流水线并行模式等。

这之所以有帮助，是因为 PP 将模型划分为跨多个设备的顺序阶段，把重工作从瓶颈设备重新分配出去，从而平滑利用率并避免单 GPU 热点。

智能体可能找到能带来更佳性能的非直观配置。例如，它可能学到：超过某长度的提示词应同时启用 PP 与 FP4 压缩以获得最佳 token 吞吐，而较短提示词仅使用 FP8 的纯张量并行。若将逻辑编码为静态规则，可能遗漏复杂交互。

RL 智能体可通过探索动作空间更容易发现最优组合，并在环境条件不变时利用最优配置；条件变化时，RL 会相应调整推理系统。生产环境应对探索动作施加安全约束、金丝雀发布和快速回滚，避免为探索牺牲 SLO。

可使用模拟器和 Hugging Face 的 Transformer RL（TRL）库等离线训练这种智能体。例如，可记录运行中系统在各种条件下的大量数据，然后在仿真中训练 RL 策略来预测结果。概念上，RL 奖励与更新循环类似以下伪代码：

```
# Pseudo structure for an RL-driven tuner

# This loop runs in separate thread/process alongside main inference service.

# e.g., {gpu_util:0.7, mem_util:0.9, avg_latency:120ms, req_queue:5}
state = get_system_state()

# e.g., 0 -> high precision, 1 -> low precision
action = rl_agent.select_action(state)

# Map action to actual parameter changes
if action == 0:
    precision_policy = "FP8"
else:
    precision_policy = "FP4"

# (We could have multiple actions, but single action for illustration)
apply_precision_policy(precision_policy)

# ... After the next token or set of tokens ...
new_state = get_system_state()
reward = compute_reward(old_state, new_state)
rl_agent.update(state, action, reward, new_state)
```

这里，该循环在推理服务器后台持续运行。`compute_reward` 函数综合吞吐量（例如自上一步以来的每秒 token 数）和延迟指标。由于需要平衡吞吐量与延迟，这属于同时优化多个目标的多目标优化问题。常见做法是用加权和将多个目标合并为一个目标。

> 若需要更大的灵活性，尤其在存在不确定性时，可将多目标优化问题（或 Pareto 前沿分析）建模为部分可观测决策过程。这样智能体能够学习吞吐量、延迟等目标之间自身的权衡策略；当单一加权奖励不足以表达需求时尤其有用。

这类多参数交互很难用基础网格搜索调好。因此，可用强化学习及近端策略优化（PPO）等优化方法来调节推理负载。PPO 在连续动作空间中的学习通常较稳定；它以渐进方式调整策略，适合实时环境的持续更新，并能避免剧烈振荡——这对推理稳定性很重要。我们不希望智能体针对每个请求都在不同决策间来回切换。

另一种抑制振荡的技术称为*阻尼*（damping）：要求一项动作至少保持一段时间或至少持续若干个请求。必要时可在严重违反 SLO 时覆盖阻尼，但应谨慎且少量使用。

需要认识到，RL 智能体在学习期间可能做出不安全或次优的动作。缓解方法是将动作空间约束在合理范围内。也建议以已确定的启发式方法构建一个良好的默认策略，再让智能体在该初始默认策略附近微调。

另一种方式是在包含探索阶段的真实系统上，以*影子模式*（shadow mode）在线训练智能体。探索期间，系统偶尔尝试随机或略有变化的策略以收集新数据；其他时间则利用当前最佳策略。

还可采用奖励塑形（reward shaping），使智能体避免违反关键约束。例如，当延迟超过硬性阈值，或不当动作导致 OOM 错误时，RL 系统可给出很大的负奖励。

此外，即使奖励函数建议这样做，也可将系统硬编码为拒绝不安全动作。这提供额外保护，避免智能体的自然探索造成灾难性故障；这是将 RL 与基于规则的护栏结合的实用方法。

设计恰当的奖励函数很重要。例如，若关心延迟上限约束下的吞吐量，奖励可写成如下代码：

```
reward = tokens_per_second - 1000 * (1 if latency > SLA else 0)
```

这里，延迟超过 SLA 时施加较大惩罚；否则不惩罚。另一种做法是使用与延迟超出目标 SLA 的幅度成比例的连续惩罚。一个简单的连续惩罚奖励可写为：

```
reward = tokens_per_second - λ * max(0.0, latency - SLA)
```

这里，λ 是惩罚权重，`max(0.0, latency - SLA)` 会随超出 SLA 的程度线性增长。这样，延迟超标越多，智能体得到的惩罚越平滑地增加，从而形成更平滑的梯度和更渐进的权衡决策。实践中，连续的“软”惩罚常比二元“硬”惩罚产生更稳定的策略，但仍需在目标工作负载中验证。

建议先用一组简单、静态的启发式规则调优。系统稳定后，再逐步引入 RL 智能体，处理启发式规则无法捕捉的复杂调节问题。

日志与可观测性十分重要。应持续记录 RL 智能体作出的决策及其结果。例如，使用结构化日志、计数器和遥测仪表盘，实时追踪“状态 → 动作 → 奖励”序列；若智能体行为异常，这些记录有助于调试。

还建议提供退出通道，即*紧急停用开关*（kill switch）。如果智能体持续恶化延迟等明显做坏事，可在诊断问题并离线重新训练策略期间，让系统回退到安全的静态配置。例如，启用智能体后 p95 延迟增长超过 50% 时，系统可自动禁用智能体动作并向值班人员告警；具体阈值应由 SLO 和基线波动决定。

截至写作时，基于 RL 的在线推理调优尚未成为现代推理服务引擎的主流能力，但已开始出现。随着技术成熟，预计会有更多推理平台提供自调优能力。这一点很重要，因为模型和系统正日益复杂；在快速变化的条件下，人工管理所有调优旋钮十分困难。

可实时自适应的智能体是系统优化的自然演进方向。已经开始出现能依据自身实时遥测指标自动实现专家级性能调优的自优化 AI 推理服务器；其成熟度和收益仍取决于模型、负载、护栏设计及严格的线上验证。

动态切换内存分配（slab、缓存与流序分配）

GPU 内存碎片和非最优的内存分配可能是隐蔽的性能杀手。推理服务器每秒会为 token、中间激活等许多对象分配和释放数千个张量。内存分配器所采用的策略会影响碎片程度和分配延迟。

动态切换内存分配器是指系统可在运行中改变分配内存的方式。例如，系统可针对某些分配大小使用 slab 分配器，或切换到 CUDA 的流序（`cudaMallocAsync`）分配器。决策取决于观测到的分配模式和预期内存碎片。

PyTorch 的默认 CUDA 内存管理机制随版本和后端而异，通常使用缓存分配器以复用较大的 GPU 内存块并减少相对缓慢、同步的 `cudaMalloc` 与 `cudaFree` 调用。将其笼统视为固定的 buddy/BFC 实现并不严谨；应以所用 PyTorch 版本的文档和实际测量为准。

伙伴（buddy）分配器将内存拆分为大小为 2 的幂的块。slab 分配器可构建在伙伴系统之上，高效管理小型、固定大小的对象：它预分配 slab（某类对象的集合），并在每个 slab 内维护空闲链表，如图 19-6所示。

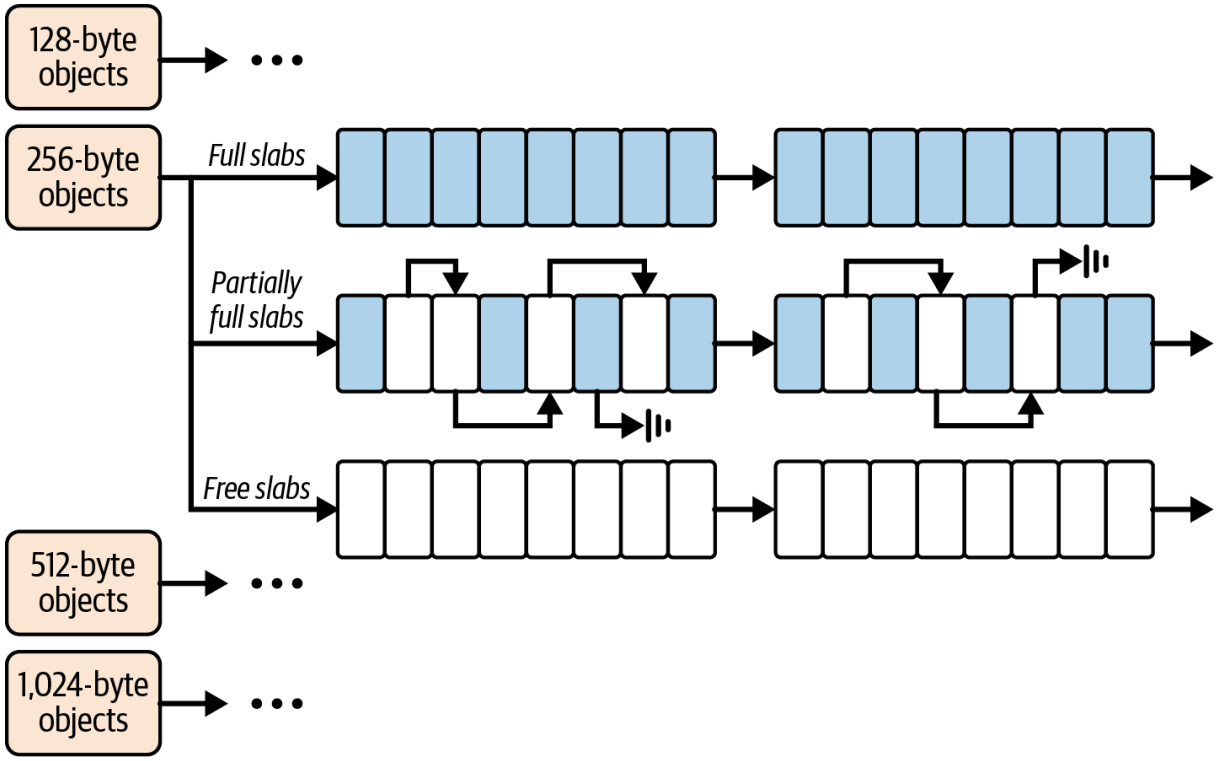


图 19-6. slab 分配器在每个预分配 slab 内维护内存对象的空闲链表

slab 分配器支持快速复用，并可避免不同对象大小混用造成的碎片；伙伴分配器处理粗粒度的页分配，slab 分配器则优化细粒度对象的复用。

PyTorch 缓存分配器对许多负载表现良好。然而，当大、小分配交替且模式差异很大时，可能出现碎片。处理不同类型请求的长运行服务器中，碎片可能逐渐积累。

此时虽然有大量空闲内存，却不够连续，无法满足大张量分配；即使从技术上看仍有可用内存，也可能导致 OOM。

PyTorch 提供 `torch.cuda.memory_summary()` 以协助评估内存使用情况，也在 `torch.profiler` 中提供内存分析（`profile_memory=True`）。可借此确定大量分配内存的操作。此外，NVIDIA Nsight Systems 可在时间线上跟踪 CUDA 内存事件和统一内存缺页，Nsight Compute 则提供内存工作负载分析。

这些工具结合起来，可观察分配行为和碎片效应随时间的变化。可在开发和测试阶段使用它们来制定内存分配器调优策略，包括接下来讨论的动态策略。

减少内存碎片的一种粗暴方法是定期重置分配器状态。另一种做法是使用 CUDA 流序缓存分配器 `cudaMallocAsync`，它会在内部按一定限制以内的分配大小分箱；slab 分配则更进一步，完全不混合大小。

`cudaMallocAsync` 在某些方面类似结合了 slab 与伙伴系统的机制，并由 CUDA 驱动管理。这可能以较低的自定义工作量获得部分自定义分配器收益，但并非无条件优于其他后端，须针对实际负载测量。

具体而言，`cudaMallocAsync` 使用流序内存池，在内存引用释放后自动复用内存；由于它知道内存释放的依赖顺序，可在后台处理已释放块。其行为和收益会因 CUDA 版本、工作负载和流同步模式而异。

在 PyTorch 中，分配器后端及配置键是否可用取决于 PyTorch 与 CUDA 版本。`max_split_size_mb` 属于原生缓存分配器的配置项；使用 `backend:cudaMallocAsync` 时，它通常不是可动态调节的等价控制项。应查阅目标版本的 `PYTORCH_ALLOC_CONF` 文档并实测验证。

例如，动态系统可能在预计出现大分配时调整原生缓存分配器的分割策略，以降低大块被切成小块的概率；处理许多小请求时又可能采用不同阈值以提高复用率。此类更改一般必须在 CUDA 初始化前生效，而不是在正在运行的 Python 进程中即时切换。

> 过小的分割大小会使分配器充满许多微小块，增加元数据开销和潜在碎片；过大的分割大小会减少块数及元数据，但若只释放块的一部分，可能留下无法利用的较大“空洞”。

设想服务检测到碎片，例如 PyTorch 内存快照显示了由碎片形成的空洞。在这种情况下，系统可在新启动的工作进程中尝试 `cudaMallocAsync` 后端，它可能改善某些负载的内存池复用；不能假定其一定会合并或消除现有进程中的碎片。

应使用内存监控工具跟踪并记录内存碎片。例如在 PyTorch 中，可使用 `torch.cuda.memory_reserved()` 和 `torch.cuda.memory_allocated()`：前者是分配器保留的 GPU 内存总量，后者是其中实际被张量使用的量。

保留量与已分配量之间很大的差距可能表示未使用缓存块较多，但不能仅凭该差距断定碎片。若差距随时间增长，可考虑定期清空缓存，将未使用块交还给驱动，或重启工作进程以完全重置分配器；这些方法具有侵入性，虽有时用于碎片严重的长运行生产进程。

> 清空和重启等侵入性“去碎片”方法应仅在维护窗口使用，或在整个集群中滚动重启以避免停机。若必须依赖这些破坏性机制，通常意味着还有需要定位和优化的更深层问题。

例如，要在 PyTorch 中实现动态分配策略，可先使用默认的原生分配器；发生 OOM 时，再在使用 `cudaMallocAsync` 后端的新进程中重试。

遗憾的是，首次导入 `torch` 或首次触及 CUDA 上下文时，CUDA 缓存分配器便已创建。Python 的 `importlib.reload` 不会卸载 C++ 扩展或销毁分配器。因此，运行中修改 `PYTORCH_ALLOC_CONF`（旧称 `PYTORCH_CUDA_ALLOC_CONF`）后重新加载 Python 模块，并不能在进程内重新配置分配器。

不过，可派生一个新进程，在导入 `torch` 前设置环境变量。下方示例捕获 OOM，在父进程释放可释放内存后，用设置好 `PYTORCH_ALLOC_CONF` 的干净子进程重试。该方法展示如何以不同分配器后端设置 `backend:cudaMallocAsync` 并重新运行相同调用；它较为权宜且代价高，下面是捕获 `torch.cuda.OutOfMemoryError` 时实施该策略的 PyTorch 示例：

```
# dynamic_memory_allocator.py
# Retry generation in fresh process with cudaMallocAsync if first attempt OOMs.
# This is the only reliable way to change the CUDA allocator at runtime.
```

```
import os
import sys
import gc
import pickle
import tempfile
import subprocess
import importlib
from typing import Callable, Any

def _resolve_factory(factory_path: str) -> Callable[[], Any]:
    """
    Resolves a factory string like "my_package.my_mod:build_model" to a callable.
    The callable must return ready-to-use model with .generate(request) method.
    """
    module_name, func_name = factory_path.split(":", 1)
    module = importlib.import_module(module_name)  # safe to import, no torch yet
    return getattr(module, func_name)

def generate_with_allocator_retry(
    model_factory_path: str,
    request_object: Any,
    *,
    allocator_conf: str = "backend:cudaMallocAsync"
) -> Any:
    """
    Attempts model.generate(request_object) in the current process.
    On torch.cuda.OutOfMemoryError, retries in a fresh subprocess with
    PYTORCH_ALLOC_CONF set to allocator_conf. request_object and the
    returned value must be picklable.
    """
    # Import torch only inside function; avoid importing at module import time.
    import torch

    model_factory = _resolve_factory(model_factory_path)
    model = model_factory()  # user-supplied function builds model, moves to GPU

    try:
        # First attempt uses whatever allocator current process started with.
        return model.generate(request_object)

    except torch.cuda.OutOfMemoryError:
        # Free as much as possible in the parent before spawning the child.
        # Avoids compounding pressure when two processes momentarily overlap.
        try:
            del model
        finally:
            gc.collect()
            torch.cuda.empty_cache()

        # Serialize request to temp file and ask fresh interpreter to do work.
        with tempfile.TemporaryDirectory() as td:
            req_path = os.path.join(td, "request.pkl")
            out_path = os.path.join(td, "output.pkl")
            with open(req_path, "wb") as f:
                pickle.dump(request_object, f)

            # In the child, we want torch to see allocator config at import.
            env = os.environ.copy()
            env["PYTORCH_ALLOC_CONF"] = allocator_conf

            # Re-run this module as a helper child. The child will import torch
            # only after PYTORCH_ALLOC_CONF is set in its environment.
            cmd = [
                sys.executable,
```

```
                __file__,
                "--child",
                "--factory", model_factory_path,
                "--request", req_path,
                "--output", out_path,
            ]
            completed = subprocess.run(cmd, env=env, capture_output=True,
                                       text=True)

            if completed.returncode != 0:
                # Bubble up child stderr to aid debugging
                raise RuntimeError(
                  f"Retry failed with exit code {completed.returncode}\n"
                  f"stdout:\n{completed.stdout}\n\nstderr:\n{completed.stderr}"
                )

            with open(out_path, "rb") as f:
                return pickle.load(f)

    def _child_main(factory_path: str, request_path: str, output_path: str) -> None:
        """
        Child entrypoint: assumes PYTORCH_ALLOC_CONF is already present in env.
        Imports torch only now, builds the model, runs generate, and pickles result.
        """
        # Import torch after env var set by the parent's subprocess.run(env=...).
        import torch

        model_factory = _resolve_factory(factory_path)
        model = model_factory()  # build the model inside the child process

        with open(request_path, "rb") as f:
            request_object = pickle.load(f)
    if __name__ == "__main__":
        import argparse
        parser = argparse.ArgumentParser()
        parser.add_argument("--child", action="store_true")
        parser.add_argument("--factory", type=str, default="")
        parser.add_argument("--request", type=str, default="")
        parser.add_argument("--output", type=str, default="")
        args = parser.parse_args()
        if args.child:
            _child_main(args.factory, args.request, args.output)
        else:
            print("This module is intended to be imported.")
```

这里通过工厂函数在子进程构造模型，确保 `PYTORCH_ALLOC_CONF` 环境变量生效前不导入任何 CUDA 相关内容。代码使用 `torch.cuda.empty_cache()` 清空未使用的缓存块并将其交还给驱动。该模式保证子进程导入 `torch` 前已应用分配器配置，也避免试图在运行时卸载 CPython 不支持卸载的原生扩展。注意，`empty_cache()` 不会释放仍被张量占用的显存，也不能修复真正的碎片；新进程重试开销很大，不应成为逐请求的常规策略。

在非 PyTorch 环境（例如基于 C++ 的 LLM 推理引擎）中，可实现能够按特定分配大小配置的纯 slab 分配器。此类分配器将内存预先划分为固定大小的“slab”，对重复的同尺寸分配非常高效；对该单一尺寸可显著减少碎片，但仍须处理不同尺寸、生命周期和总体内存利用率之间的权衡。

在 LLM 服务器中，一种常见思路是为每个 token 的中间缓冲区复用内存。例如生成一个 token 时，可能需要与该 token 激活相关的 `[layers, hidden_dim]` 张量。若此类分配每

次大小都相同（如 64 KB），为该精确大小设置 slab 会很合适；实际张量形状与缓冲策略则因引擎实现而异。

系统若检测到反复分配大量 64 KB 张量，可建立由专用 64 KB 块组成的“slab”。slab 分配器通常在整个 slab 都释放后，才将内存归还通用池。

> 现代 LLM 推理引擎通常会对生成 token 所需的缓冲区做复用，而不是每次释放再重新分配；后者会带来很高开销，尤其在 token 粒度上。具体复用层级和实现因引擎而异。

分配器切换可能意味着为系统不同部分使用完全不同的分配策略。例如，模型权重在推理期间很少释放，可采用适合大型长期分配的缓存分配器；短生命周期的逐 token 分配则可使用自定义分配器或专用内存池。

> 建议在架构上将模型权重等长生命周期分配与 token 缓冲区等短生命周期分配分开，这样更容易将它们导向不同分配器和内存池。

应始终为 OOM 实现回退策略。许多生产系统采用多层内存：发生 GPU OOM 时，可将部分状态卸载到 CPU 后重试，而不是直接使请求失败。例如可释放未使用缓存、将部分层卸载到 CPU，或压缩缓存。每种方法都会在延迟、吞吐、质量与可靠性间取舍，必须与服务的 SLO 相匹配。

总之，动态分配器管理和多层内存等技术可帮助系统在不同类型负载下长期运行，同时降低碎片与分配延迟尖峰的风险。它是用户不直接看见的后台优化，却可能对长运行、大规模推理服务器的稳健性很重要；效果不应假定，需用目标环境的监控和压测验证。

运行时内核性能改进与热替换实现

在快速演进的 GPU 硬件和算法创新领域，新的、更快的内核实现不断涌现，包括 FlashAttention 的新变体、巨型内核（megakernel）和硬件专用的软件优化。

运行时内核补丁是指无需完整重新部署或重新加载模型，便可将这些新实现集成到正在运行的系统中。本质上，是在运行中将较慢的内核函数热替换为更快的函数。

设想推理服务器的多头注意力使用默认的 PyTorch 缩放点积注意力（SDPA）内核，随后发现 FlashAttention-3 一类的新内核实现在某些长序列场景可提升约 20% 的速度。该数字取决于硬件、形状、精度及实现版本，须在目标系统中复测。

传统做法是安装更新后的库并重启服务器来使用新实现；借助运行时补丁，则可在不中断服务的情况下动态加载新内核并将调用重定向到它。

对重启或模型重载会造成过高延迟、甚至中断的 24/7 服务而言，零停机升级很关键。在 Python 中，可像下面这样进行简单的 monkey patch：

```
import new_flash_attn_lib

# Monkey-patch the model's attention forward to use the new library
old_attn_forward = model.transformer.self_attn.forward

def new_attn_forward(self, *args, **kwargs):
    return new_flash_attn_lib.forward(*args, **kwargs)

model.transformer.self_attn.forward =
new_attn_forward.__get__(
model.transformer.self_attn,
type(model.transformer.self_attn))
```

这段代码将注意力模块的 `forward` 方法替换为调用 `new_flash_attn_lib.forward` 的方法。通过绑定到实例（`__get__`）来模拟正确的方法绑定。补丁应用后，该注意力层的后续调用将转入新实现，因此实际上完成了内核的热替换。

> 务必确认 `new_flash_attn_lib.forward()` 可直接替换原实现，并经过充分测试，确保输出在可接受数值误差内一致，以避免模型质量回归。

另一种技术是 JIT 补丁：使用 PyTorch Inductor 或 OpenAI Triton 等即时编译器在运行时生成更快的内核，再接入推理流水线。如下可使用 PyTorch 的 `torch.compile` 优化函数并获得编译后的版本：

```
compiled_forward =
    torch.compile(model.transformer.self_attn.forward,
                  backend="inductor")

model.transformer.self_attn.forward = compiled_forward
```

此后每次调用 `forward` 都执行优化后的代码，其中可能融合多个操作等。若在模型初始化后执行，这就是一种热补丁，因为无需重新加载模型，只是替换函数引用。实际 `torch.compile` 可能带来编译延迟、图断裂或形状相关的重新编译，应预热并按真实输入分布验证。

在 CUDA 侧，可使用 CUDA Driver API 进行运行时模块加载。例如，为新内核编译自定义 PTX，并在不重置设备的情况下加载到上下文中。若函数签名匹配，系统可更新函数引用。NVIDIA Runtime Compilation（NVRTC）等工具能够在运行时从字符串编译 PTX，从而支持该工作流。

以 CUDA C++ 实现这一点较复杂。Python 的 monkey patch 或 PyTorch 可扩展后端等高层方案在长期内可能更实用、更易维护；但高性能推理引擎未必采用 Python/PyTorch 运行时。

> 进行此类热替换时，必须保证线程安全。例如，排空传入请求队列或使用屏障，确保没有其他线程正执行要替换的函数，以避免与运行中线程发生竞态。这类似操作系统内核加载模块：需要谨慎同步，但可避免完整重启。

使用定义清晰的模块边界（例如分离注意力和 MLP 内核）会更容易替换内部实现，因此模型代码宜保持较高模块化程度并提供这些替换点。单体模型实现更难打补丁，但可能性能更高，例如巨型内核。

当然，必须保证新内核产生与旧实现相同或足够接近的结果。优化内核通常设计为在相同精度（如 FP16）下数值等价，但并非保证；应进行正确性、稳定性与端到端质量验证。

热补丁也可用于快速修复。例如当前内核中特定操作序列触发已知 bug 时，可先快速补入修复，而不必等待完整更新。

例如，部署的 FP8 内核可能在未预料的边界输入上表现异常。若有更安全但稍慢的版本，可检测异常条件并在这些情况下热替换为安全内核。务必记录这些事件以便后续分析。

> 可采用分阶段发布：将少量流量路由到新内核（影子测试或金丝雀）。然后使用遥测数据与旧内核比较延迟、吞吐量和输出；新内核达到预期且结果正确后，再向全部用户发布。

为管理多个候选实现，引擎可维护注册表，例如 `attention_impl = "fast"` 或 `"safe"`，并据此分支。可将其实现为特性开关，无需改代码即可切换实现；这有利于通过改变开关快速回滚。要确保代码会主动读取特性开关系统的值，否则值静态不变，便失去开关意义。

设计良好的推理引擎还应测量新内核的运行时性能。若新内核在实际生产数据下不更快，便可回滚；原因可能是硬件略有不同、批次大小未经测试或负载条件不同。

这是上一节所述自动调优的一种形式。若自动调优器发现某实现超过阈值地优于当前默认实现，系统可触发“补丁”，将该更快实现提升为之后的默认选择。

这就闭合了自动调优器的循环：系统不仅动态找到更好的内核，也将其切换进来，形成自优化的内核选择机制。

还可将此策略与前述 RL 策略结合：同时加载多个实现，让 RL 智能体为不同类型请求选择其一。实际部署时需对选择器本身的开销、确定性与回滚路径施加约束。

> 注意在内存中维护多个实现的开销。额外代码体积不应将关键代码逐出指令缓存，也不应耗尽 GPU 内存。

例如，高度优化的内核可能最适合长序列，而更简单的实现对短序列已足够。运行时系统可按请求选择它们，相当于依据输入序列长度逐请求地打补丁。

总之，运行时内核补丁强调灵活性：今天最优的方案明天可能不再最优，而它提供了快速适应的手段，使服务基础设施跟上模型加速技术的快速发展。vLLM、SGLang、NVIDIA TensorRT-LLM 和 NVIDIA Dynamo 等推理引擎提供了不同程度的动态能力；例如 TensorRT-LLM 支持通过运行时插件加载优化内核。应根据所用版本的正式文档和兼容性限制利用这些能力。

使用时间序列预测持续预热 CUDA Graph 与缓存

在高吞吐量推理场景中，冷启动开销可能是请求—响应延迟的重要组成部分。第 12 章所述的 CUDA Graph 能够捕获一系列 GPU 操作，并以极小的启动开销重放该序列。

预热 CUDA Graph 是指在真正需要图之前先完成图的建立。若能预测某种请求类型或批次大小何时出现，预热好的 CUDA Graph 即可随时执行。

该技术依赖预测准确性。若预测不准，可能预热了不会被使用的图。应监控预测命中率，确认优化确实带来收益。

经预热后，请求实际到达时图可跳过昂贵的启动与初始化步骤。系统可使用 ARIMA、Prophet 等时间序列预测算法，预判未来工作负载模式（包括流量激增与批量大小变化），主动为快速执行准备图。

设想某推理服务观察到每日周期：由于时区相关流量模式，每天上午 9 点用户流量都会激增。了解这一规律后，系统可在 9 点前运行少量请求，以将模型载入 GPU 缓存、JIT 编译必要内核，并为预期批次大小捕获 CUDA Graph。9 点实际负载上升时，传入请求即可复用针对预期批次大小和使用模式的预热状态。

实践中，可由类似 cron 的作业或调度服务在预期流量激增前触发预热流程。务必为资源预配留出足够时间。

在 PyTorch 中，将模型计算包裹在 `torch.cuda.graph()` 上下文内，可捕获包含 GPU 内核启动的静态计算图。重放图时，PyTorch 绕过 Python 到 CUDA 的调度开销，并以一次 `cudaGraphLaunch` 提交整个工作负载。由于 CPU 参与极少，尤其对大型、重复的输入批次，这可能显著加快执行。

缺点是这些图是静态的，不易支持可变形状或长度。推理服务器却常因 GPU 内存限制和算法优化而处理离散的批次大小（1、2、4、8、16 等），这与图模型很契合。

为处理可变性，可为每个常用批次大小或序列长度维护预捕获图池，并通过持续预热策略准备这些图。例如高峰时段常见 16 个请求的批次，服务器可预先以批次大小 16 捕获模型前向传播图，再保存供后续使用。

下次同时到达 16 个请求时，推理系统将批处理输入送入预捕获图。图中的内核已经预热并针对图执行优化，无需逐个排队和启动许多操作，一次图启动即可完成。

请注意，图池中每个已捕获图的工作区都会额外占用 GPU 内存。存储许多图时应监控内存使用；内存紧张时可能需要从图池中逐出较少使用的图。

为避免图池带来的额外内存占用，可使用第 12 章讨论的图更新/补丁机制，对图节点做小幅尺寸调整。但实践中，预先捕获的图池往往有更可预测的性能；应权衡显存成本与命中率。

CUDA Graph 也能降低 CPU 开销：它只协调一次图启动，而不是协调许多单独的内核启动。这使 CPU 可以处理数据预处理等其他实际工作。

同样，可用 ARIMA、Prophet 等时间序列预测算法预测 RPS、平均提示词长度等指标。若模型预测批次大小将跃升，或将出现长输入序列等特定请求模式，系统可开始准备并预热相应的 CUDA Graph、缓存及其他资源。例如，主动调大连续批处理算法的批次上限、从磁盘预取模型权重到 GPU 内存，或分配额外 GPU 实例。预测与预热都应受到资源预算和限流保护。

> 应使用近期流量数据频繁重新训练和更新这些时间序列模型，因为新用户群体上线等因素会随时间改变使用模式。此外，意外的“假期”或活动也可能使负载曲线骤增。

与缓存和预热相关的另一点是预判 prefill 与 decode 工作器的扩容。若已知定时批处理任务或日报场景会在某时刻带来大量长提示词请求，系统可扩展 prefill 工作器，并以代表性样本数据执行前向传播，预热 CUDA Graph 和 KV 缓存。

扩容和预热事件也应覆盖 decode 工作器与 CUDA Graph。对于 decode，例如可预热并加载推测解码的草稿模型及其 KV 缓存到 GPU 内存。其他 decode 优化包括针对固定生成 token 数展开循环：普通解码为 1 个 token，推测解码时可能为多个 token。

还应考虑缓存 CUDA 内核本身。CUDA 通常会缓存已编译内核（C++ 代码 → PTX 指令 → SASS 汇编）以加快执行；内核首次运行时，可能产生即时 JIT 编译开销。

应将这种预热与集群自动扩缩容器协调：新 GPU 实例启动时，自动扩缩容器使用若干预热 API 调用执行快速推理。这会验证推理引擎、缓存 JIT 编译结果、分配内存池、准备 CUDA Graph 等，使引擎在加入实时流量池前达到可服务状态。

> 在受控预热阶段，应根据系统知识识别并调用尽可能多的不同路径，包括各种批次大小、CUDA Graph 变体等。

还应通过比较预测激增期间前几个请求在有、无预热时的延迟，监控预热是否真正有帮助，并按需调整时机和阈值。时间序列预测可能出错，务必确保预热任务在错误时间运行时不会影响整体推理性能。

例如，应尽量在 GPU 利用率较低时，以示例数据执行预热。CUDA 支持流优先级调度，可将预热流设置为低于推理流的优先级，使预热不与实时高流量请求竞争资源。

预热任务应使用低优先级 CUDA 流，以便在预热期间到达的高优先级实时流量能抢占资源。闲置 GPU 确实是可利用的机会，但只有不与真实工作冲突时，预热计算才近似“免费”。

Grace Blackwell 系统中 CPU 与 GPU 以较低延迟共享统一、一致性内存，因此可采用一些额外技巧。例如，可让 CPU 先将 GPU 计划使用的数据填充到统一内存，从而可能避免后续显式 GPU 拷贝；实际行为取决于内存放置、访问模式和平台软件栈。

由时间序列预测引导的持续预热可使延迟更可预测：推理引擎学习常见流量模式，自动准备数据和硬件，提前应对需求。它通过在可摊销的时段平滑编译、内存传输等昂贵操作，可能减少抖动与延迟尖峰。

这对一次性初始化成本很高的大模型尤为有价值。随着模型和上下文增长，自适应预加载在生产 LLM 系统中可能从“锦上添花”变成必需能力；但前提是预测质量、资源成本与用户体验收益均经过验证。

自适应批处理与分块预填充调度

在第 16 章中，我们讨论了现代推理服务器如何使用不同的请求批处理方式（例如连续批处理），以最大化吞吐量并尽量降低所有请求的延迟。但批处理会增加单个请求的延迟。可通过*自适应批处理*技术，随一天中条件变化动态处理这种取舍。

自适应批处理会根据负载以及请求推进情况，动态调整请求如何分组为批次。这种动态策略可随环境变化实时调整批次大小和阈值参数。

例如，在峰值负载期间，吞吐量至关重要，系统可使用较大批次（如 8 或 16）；在低负载期间，系统可缩小批次，让请求更快得到服务，从而优先延迟而非吞吐量。

决定批次大小时，可采用简单启发式规则，例如："GPU 利用率 > 80% 时允许更大批次；< 20% 时使用批次大小 1 以最小化延迟。"也可使用前文讨论的更复杂 RL 智能体或预测策略。固定百分比仅是起点，需结合排队时延、KV 容量与 SLO 校准。

prefill 与 decode 阶段的算术强度差异会导致执行时长不匹配。因此，通常最好将二者解耦，作为独立负载处理：单节点上可用独立线程/进程，多节点集群中可用独立工作器池，并分别调优。

通过解耦 prefill 与 decode，可把两个阶段看成独立操作，分别针对各自的计算和内存带宽需求优化，其中一项便是各阶段采用的批次大小。

实践中，vLLM 等现代推理引擎可为 prefill 和 decode 工作器形成独立批次，因此一批 prefill 请求可独立于一批 decode 请求执行。例如，decode 属于内存受限负载，较大批次可能提高算术强度；具体调度与解耦程度取决于引擎版本、部署架构和配置。

vLLM 等现代推理服务框架使用自适应调度循环，动态选择处理 prefill 批次还是 decode 批次。vLLM 支持分块预填充和以 decode 为优先的调度，以交错执行 prefill 与 decode 来提高利用率。这些技术可提升整体利用率和吞吐量，但对延迟的影响需随工作负载测量。

prefill 与 decode 的系统资源特性不匹配也会影响流水线并行（PP）。设想一个微批次对长序列执行 prefill，另一个微批次逐 token 执行 decode：二者持续时间不匹配，便会产生流水线气泡。

可将大型 prefill 请求切成小块，并在块之间捎带执行对延迟敏感的 decode 任务，使二者交错。这可保持各流水线阶段忙碌，并尽量减少 GPU 调度中的空闲"气泡"。

分块预填充（chunked prefill）是用于减少流水线气泡的常见模式。它将大任务（prefill）进行时间切片，为小任务（decode）在各分块制造的流水线间隙中执行腾出空间，如图 19-7所示。

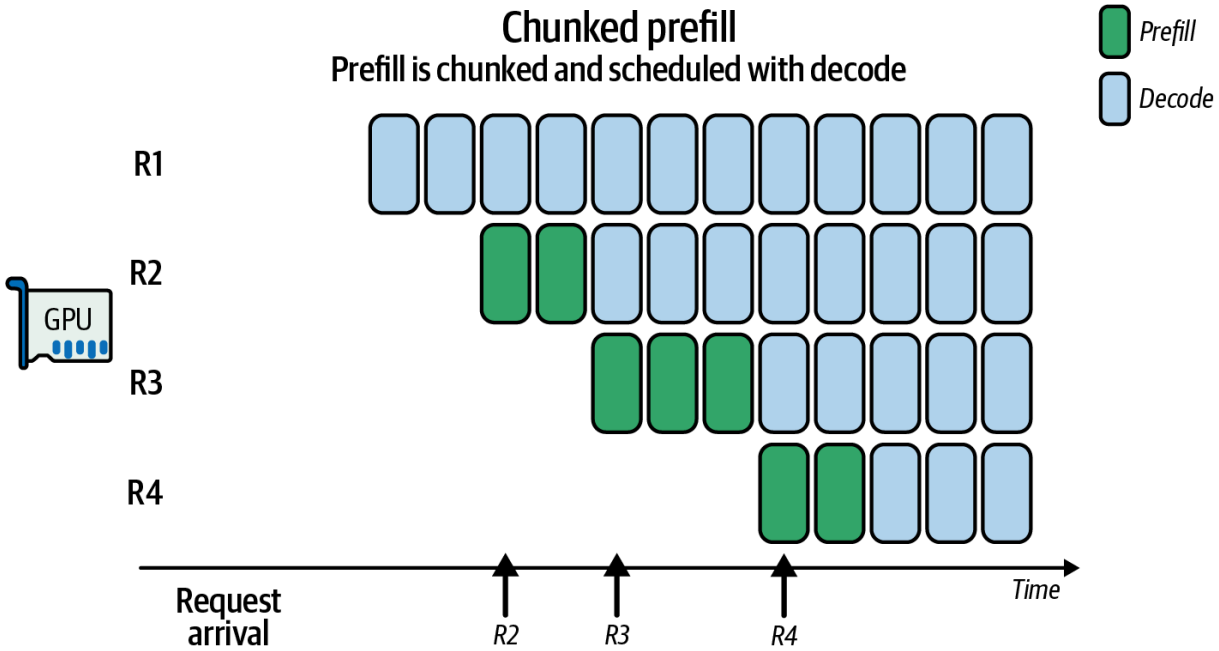


图 19-7. 跨四个请求采用分块预填充进行以 decode 为优先批处理的优势

SARATHI 论文表明，相比朴素调度，这类分块 prefill 与捎带执行方式在其评测配置中可帮助确定合适的*以 decode 为优先的批处理*程度、减少气泡，并将吞吐量提高约 1.3–1.9 倍。SARATHI 名称指能聪明地同时驾驭 prefill 和 decode 任务的御者。

例如，考虑一个不使用分块的 10,000 token prefill 请求。单次 prefill 会阻塞整个流水线，使 decode 任务在 prefill 完成前排队。

若使用分块 prefill，将这个 10,000 token 请求分为五个 2,000 token 块，便可在 prefill 块之间交错执行 decode 批次，让 GPU 同时推进两个阶段。这有助于挤出流水线气泡、提高吞吐量并平滑 GPU 利用率。

> 经验法则是选择使一个 prefill 块耗时约 50–100 ms 的分块大小，从而能频繁在其间调度 decode 批次。对应的 token 数可能是数千，取决于模型架构/规模和 GPU 硬件；应以 SLO 和实际测量为准。

vLLM 等现代推理引擎会基于 GPU 利用率和队列状态，通过自适应调度循环决定处理下一个 prefill 块还是执行 decode 批次。vLLM 的调度器明确支持分块 prefill 和以 decode 为优先的批处理；其执行器及分块 prefill 功能旨在让大型 prefill 与较小的交互式 decode 重叠。

自适应调度器在选择分块 prefill 大小时，需要考虑 GPU 共享内存限制和占用率。下面给出一个简单的自适应分块 prefill 实现；该代码动态调整块大小，以保持 GPU 上较高的 SM 利用率和占用率：

```
# Example adaptive scheduler for chunked prefill/decode

import cupy as cp
import torch
```

```
# Hardware constraints
SHMEM_LIMIT    = 256 * 1024
BLOCK_THREADS = 256
TARGET_UTIL    = 0.85
OCC_THRESHOLD = 0.5

# Cache for occupancy results and tile lookup
_occ_cache = {}
_tile_table = {}  # e.g., {L: optimal_T}

# 1) Precompute tile_table offline; here we lazy-initialize on first use
def get_optimal_tile(L):
    if L in _tile_table:
        return _tile_table[L]
    # compute block size by querying for an occupancy-based suggestion
    min_grid, max_grid, block_size = ... # left out for brevity

    T = min(block_size, L)
    T = max(32, (T // 32) * 32)
    _tile_table[L] = T
    return T

# 2) Cached occupancy query
def get_occupancy(threads, shared_bytes):
    key = (threads, shared_bytes)
    if key in _occ_cache:
        return _occ_cache[key]
    max_blocks = cp.cuda.runtime.\
        cudaOccupancyMaxActiveBlocksPerMultiprocessor(
            attention_kernel_ptr, threads, shared_bytes
        )
    props = torch.cuda.get_device_properties(0)
    warps_per_block = threads // props.warp_size
    max_warps = props.max_threads_per_multi_processor // props.warp_size
    occ = (max_blocks * warps_per_block) / max_warps
    _occ_cache[key] = occ
    return occ

def scheduler_loop():
    stream = cp.cuda.Stream(non_blocking=True)
    while True:
        pending = get_pending_requests()
        util = gpu_utilization()
        if util < TARGET_UTIL and any(r.phase=='prefill' for r in pending):
            req = select_heaviest_prefill(pending)
            L   = req.remaining_length()
            T   = get_optimal_tile(L)
            shared_bytes = 3 * T * T * 4
            occ = get_occupancy(BLOCK_THREADS,
              shared_bytes)
            if occ < OCC_THRESHOLD:
                T = max(32, T // 2)
                shared_bytes = 3 * T * T * 4
            chunk = req.next_prefill_chunk(T)
            # Launch with CuPy RawKernel on our stream
            attention_kernel((...grid...), (BLOCK_THREADS,),
               (chunk, ...), shared_mem=shared_bytes,
               stream=stream)
            # Record an event to know when done
            event = cp.cuda.Event()
            event.record(stream)
        elif any(r.phase=='decode' for r in pending):
            batch = form_decode_batch(pending, max_batch=16)
            # Trace the adaptive logic
```

```
            logger.info(f"T={T}, occ={occ:.2f},
                util={util:.2f}")
            launch_decode_kernel(batch, stream=stream)
            event = cp.cuda.Event()
            event.record(stream)
        else:
            # Poll the last event rather than sleep
            if not event.query():
                cp.cuda.get_current_stream().synchronize()  # or pass
            continue
```

这里，调度器调整分块大小，以尽可能使用共享内存，同时不牺牲并行度。

具体而言，调度器先计算 tile 宽度 `T`，使查询、键和值的三个共享内存缓冲区（代码将每个简化为需要 `T x T` 个浮点数）能放入 GPU 每个 SM 的动态共享内存上限。随后调用 CuPy API（Python），测量该 `T` 下每个 SM 可并发运行的线程块数。真实注意力内核的共享内存需求取决于 head dimension、布局、流水方式和内核实现，不能由此简化模型直接推断。

若占用率低于设定的 50% 阈值，便将 `T` 减半，释放共享内存以便更多线程块共驻，从而在数据复用（较少 DRAM 载入）与并行度之间取得平衡。

当整体 GPU 利用率低于 85% 且存在待处理 prefill 工作时，调度器选择剩余量最大的 prefill 请求，将其分成每块 `T` 个 token 的等长块，使每个块能通过流水线而不独占所有阶段。

辅助函数 `next_prefill_chunk` 不固定块大小，而是根据实时指标即时调整 `T`：占用率低时缩小块，DRAM 流量过大时增大块。实际策略需要同时考虑排队延迟、KV 容量、内核形状与 SLO，而不是仅依赖这些阈值。

> 务必为调度器埋点，记录所选 `T` 及得到的占用率/利用率，以分析和验证这种自适应方法是否持续保持较高 GPU 利用率。

在 prefill 块之间，调度器可使用*以 decode 为优先的批次*，通过 `form_decode_batch` 将所有已就绪的 decode 请求合并为一次启动。这样，短小且对延迟敏感的 token 生成可利用本会闲置的流水线间隙；短提示词用户的 decode 无需等待巨型 prefill 完成。

调度器持续监控 `gpu_utilization()`，决定处理下一个 prefill 块还是清空 decode 队列。不论何种选择，它都尝试填满 SM 槽位并最小化空闲时间。这称为*利用率最大化策略*，类似以接近 100% CPU 利用率为目标的操作系统调度器；但服务系统还须明确限制尾延迟与公平性。

这些机制共同降低大上下文 prefill 作业饿死交互式 decode 的风险，同时让小型、延迟关键请求得到及时服务。在现代 GPU 上，这有助于获得较高吞吐量而不明显损害终端用户体验；实际效果取决于调度参数与负载组合。

分块 prefill 让工作与 GPU 优势相匹配：大型 prefill 矩阵运算被批处理，小型 decode 运算被交错执行，从而力求同时改善整体吞吐量和延迟。

如上所示，调度器监控实时指标并即时自适应。分块调度降低单个请求在不同条件下阻塞整个阶段的可能性，有助于保持 GPU 活跃并减少流水线*气泡*。

应将 prefill 与 decode 视作各自具有 SLA 的独立队列。通常可先用专用时间片或 CUDA 流处理延迟敏感的 decode 任务，再使用剩余周期处理大型 prefill 作业。例如，为 decode 任务分配一定时间预算（如每次 decode 1–5 ms），使其得到更及时的关注；具体值需按模型和 SLO 调整。

优先处理面向用户的实时 decode 可降低感知到的推理滞后；同时，系统有剩余容量时仍可持续构建批量上下文。

许多高性能推理引擎采用*生产者—消费者*模型，为不同阶段使用独立线程。例如，vLLM 通过多线程让一个线程准备 decode 输入，另一个准备新的 prefill 等，再以优化顺序送入执行流。这是有效重叠工作的成熟模式，具体内部实现随版本变化。

如果有多个节点，可将 prefill 请求发送到一组节点，将 decode 请求发送到另一组。此时 prefill 和 decode 节点可采用针对各自任务专门选择的异构硬件。

例如，prefill 阶段往往更偏计算受限，计算节点可使用高 FLOPS 的 GPU；decode 往往更偏内存带宽受限，节点可能更受益于较高内存带宽。实际瓶颈会随模型、批量、上下文长度和并行策略改变。

需要注意，异构的 prefill/decode 工作器配置虽可能节约成本，却会使动态负载均衡更复杂、甚至受限。若 prefill/decode 比例意外改变，更多 prefill 工作不得不在为 decode 优化（FLOPS 较低）的工作器上执行，这些工作器可能成为瓶颈。

使用同构节点会简化调度。只有在负载比例可预测且系统能应对负载变化时，才适合进行硬件专用化；但资本投入、快速演进的 GPU 架构和成本预算未必允许这样做。

对于 prefill 与 decode 批处理，建议尽可能将 decode 调用合批，以最大化 GPU 吞吐量并减少启动开销。对于 prefill，应避免在同一批中混合极短和极长的提示词。

应采用按长度分桶，使每个批次具有相近序列长度。例如，形成批次前将传入请求归入最近的 512-token 桶，便可避免在过多填充上浪费计算。

实践中，大多数现代推理引擎实现 token 级调度器，在每一步生成时动态形成批次：它会等待数毫秒收集就绪 token，限制批次大小以符合内存和占用率约束，并使用轮转/最大延迟规则改善长、短提示词之间的公平性。应确认推理引擎配置中已启用所需功能。

总之，自适应批处理和 prefill/decode 解耦可提高 GPU 利用率与吞吐量，而不必牺牲过多延迟。它们也可能改善总体延迟，因为 GPU 空闲时间更少通常意味着任务整体更快完成；不过仍需分别监控 TTFT、ITL、尾延迟和公平性。

面向多 GPU 的拥塞感知与拓扑感知调度

Grace Blackwell GB200 NVL72（72 个 Blackwell B200 GPU、每个 180 GB HBM）和较新的 Grace Blackwell Ultra GB300 NVL72（72 个 B300 GPU、每个 288 GB）等现代多 GPU、多机架系统，使用单一高带宽 NVLink/NVSwitch 网络结构连接 72 个 GPU。此类架构形成统一的 72-GPU 域，单 GPU 聚合双向 NVLink 吞吐最高约 1.8 TB/s，NVSwitch 网络横截面聚合带宽超过 130 TB/s。具体规格应以所用系统、产品版本和 NVIDIA 官方资料为准。

不过，大规模推理要达到峰值性能，原始带宽还不够，还需要智能、自适应的通信调度。如图 19-8所示，拥塞感知和拓扑感知策略使数据传输能够实时避开瓶颈。

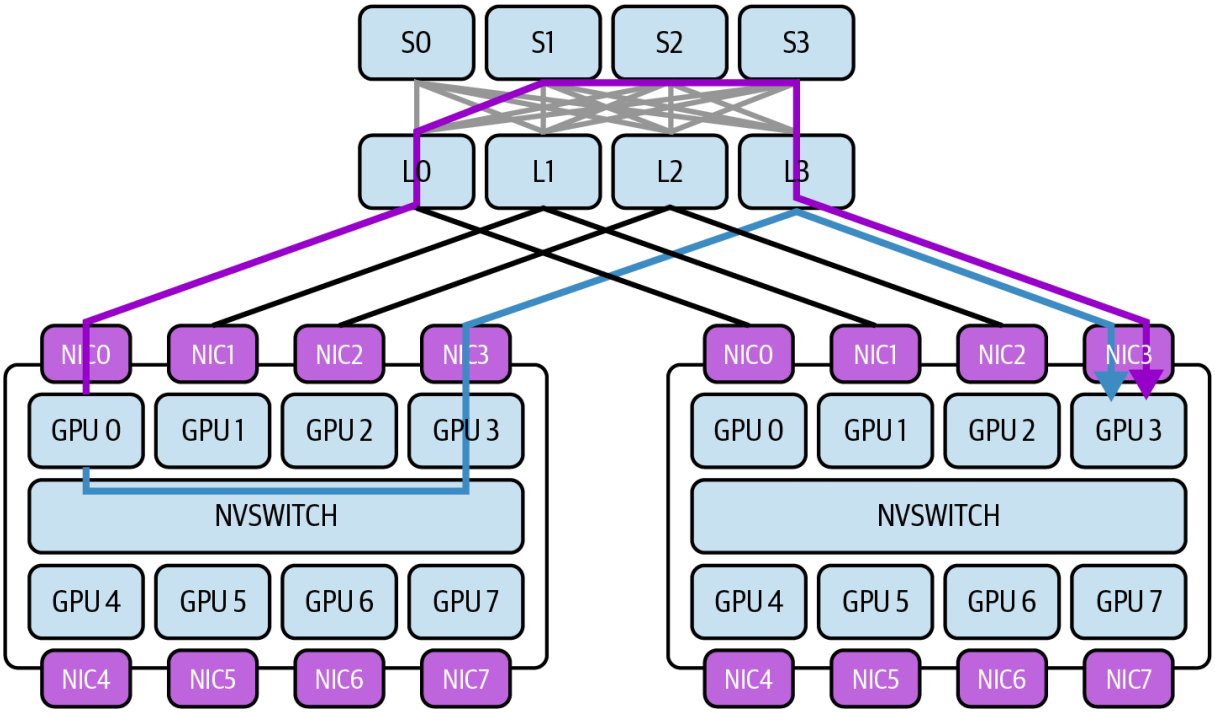


图 19-8. 使用拓扑感知路由避免 GPU 与多节点集群间的瓶颈

为处理这些瓶颈，下面讨论链路利用率遥测、动态消息路由和集合通信的波次调度。这些原则和技术可在保持低延迟、高吞吐量的同时，高效调度 GPU 间通信。为使讨论具体化，以下以 NVL72 机架环境为例。

NVLink/NVSwitch 拓扑与带宽约束

NVLink 在 GPU 之间，以及 Grace Blackwell Superchip 模块中 GPU 与 Grace CPU 之间提供高速点对点链路。NVSwitch 充当机架内网络交换机，将所有 GPU 连接为全互连拓扑。

在 NVL72 机架系统中，每个 GPU 配有多个接入一组 NVSwitch 芯片的 NVLink 端口（例如每 GPU 18 条 NVLink 链路）。该设计提供 GPU 对等连接，使任一 GPU 都可通过 NVSwitch 网络一次跳转/遍历到达其他 GPU。这样的拓扑使 72 个 GPU 可表现得像一块具有均匀连接的大型板卡。

聚合容量仍受每端口和每交换机限制；因此多对一流量模式可能使入口超额订阅。

尽管带宽很高，互连仍有有限容量。按该书所述规格，每个 NVLink 5 端口提供 100 GB/s 双向带宽（每方向 50 GB/s），每个 GB200/GB300 Superchip 为每 GPU 提供 18 个 NVLink 端口，单 GPU 双向吞吐最高 1.8 TB/s、每 Superchip 最高 3.6 TB/s。平衡负载下 NVSwitch 提供无阻塞全互连；但所有 GPU 向同一 GPU 发送等模式仍可能使链路超额订阅，因为每个 NVSwitch 芯片的总交换吞吐量有限。

过多 GPU 同时经同一交换机发送数据，或同时发往同一目的地时会发生拥塞，队列积累、有效传输吞吐下降。例如单个 GPU 理论上可实现 1.8 TB/s 双向通信，但多个对端同时瞄准该 GPU 时必须共享其 NVLink 入站带宽。同样，即使 NVSwitch 为平衡负载设计为全无阻塞，所有 GPU 同时交换数据等通信模式也可能造成超额订阅。

理解拓扑（哪些 GPU 共享 NVSwitch 组件或 NVLink 路径）对推理性能至关重要。在更小尺度上，同一板卡或托盘内的 GPU 相比不同托盘或跨节点 GPU，通常路径更快、更直接；务必依照实际机器拓扑验证。

避免凭空假设拓扑。使用 CUDA 的拓扑 API 和 NCCL 的拓扑提示以程序方式取得信息；可通过 NVML、DCGM 查询每个 NVLink 端口的计数器和远端端点，必要时结合 Fabric Manager 或 NVSwitch 工具完成交换机级映射。

在更大尺度上，跨节点/机架边界、离开 NVLink 域并使用 InfiniBand/Ethernet 的通信，比 NVL72 内部传输具有更高延迟和更低带宽。例如，InfiniBand NDR 每跳可能增加 5–10 µs 延迟，而 NVSwitch 跳转可能低于 1 µs；实际延迟与网卡、交换机、布线、负载和测量方法有关。

物理拓扑使部分通信路径比其他路径代价更低。拥塞感知调度器利用这一知识，优先选择更高带宽、更低延迟、拥塞更少的链路，以最大化性能。

实时链路遥测与监控

要管理拥塞，系统必须先*观测*它。NVIDIA 提供接口以实时监控链路利用率。NVIDIA Management Library（NVML）及具体的 `nvmlDeviceGetNvLinkUtilizationCounter` 可暴露 NVLink 的逐链路吞吐量计数器和利用率统计；可用指标和 API 支持取决于硬件、驱动与 NVML 版本。

启用 NVLink 计数器会引入开销。应采用合理的采样间隔，避免“监控性能”本身影响性能。

自适应推理服务系统可查询每个 NVLink 端口传输字节数、错误率、特定 GPU 对之间的流量负载等指标。例如，可通过 NVML 或 DCGM 查询吞吐量计数器、带宽统计及其他遥

测数据（如错误）。`nvidia-smi nvlink --status` 提供链路健康状态和配置；对于吞吐量等性能计数器，NVML 和 DCGM 通常是更合适的机制。

这有助于识别接近 100% 利用率而其他链路却未充分使用的热点链路。这些低层硬件计数器让调度器定位瓶颈出现的位置，包括特定 NVSwitch 上行链路或特定两个 GPU 间的链路。

除 NVML 外，Nsight Systems 等高级分析工具提供 GPU 活动（包括通信事件）的时间线视图。Nsight 可显示 NVLink/NVSwitch 上的数据传输何时发生及其耗时。

使用 Nsight Systems 对推理运行进行插桩，可视化多个传输是否重叠并造成延迟，或某些阶段是否等待通信。例如时间线可能显示，所有流水线阶段都在同一时刻通过同一链路发送激活，从而压垮*互连*。

建议使用 Prometheus 与 DCGM exporter 将这些指标集成到 Grafana 监控仪表盘中，以实时和历史方式查看链路利用率。识别到同步点等热点后，系统可插入轻微延迟、重新调度任务，或重新分配 GPU 角色，以缓解热点并平滑流量。

例如，系统可通过插入轻微延迟或以不同方式重叠工作来降低争用。实时遥测支持动态、自适应、反馈驱动的决策；如下节所述，调度器可对链路利用率尖峰即时作出反应，重路由流量或将任务重新调度到其他 GPU。

自适应进程—GPU 映射

一种强有力的策略是按拓扑放置计算进程，使大量通信尽量不跨越缓慢或拥塞的链路。例如，在不同 LLM 层（“进程”）驻留在不同 GPU 的多 GPU 推理流水线中，大型中间张量必须沿推理流水线传递。

这本质上是进程—GPU 放置优化问题：要将神经网络模型层的图映射到通信成本最小的 GPU 硬件上。若层/进程到 GPU 的初始分配过于朴素，这些张量可能经过长距离、代价高且拥塞的路径，包括多次 NVSwitch 跳转，甚至离开 NVLink 网络到另一机架或数据中心，从而降低吞吐量和整体性能。

采用自适应进程—GPU 映射时，系统动态分配进程到 GPU，使通信尽可能本地化且平衡。例如，将 LLM 层（进程）划分到 NVL72 机架的多个 GPU 后，若 GPU 0 上的层/进程 0 向 GPU 2 上的层/进程 2 供给数据，但两块 GPU 位于 NVSwitch 网络的相对两端，数据必须经过更多链路。此时，如图 19-9所示，将层/进程 2 移至 GPU 1 是更优映射；图示采用 NVIDIA Topology-Aware GPU Selection（NVTAGS）系统的语境。

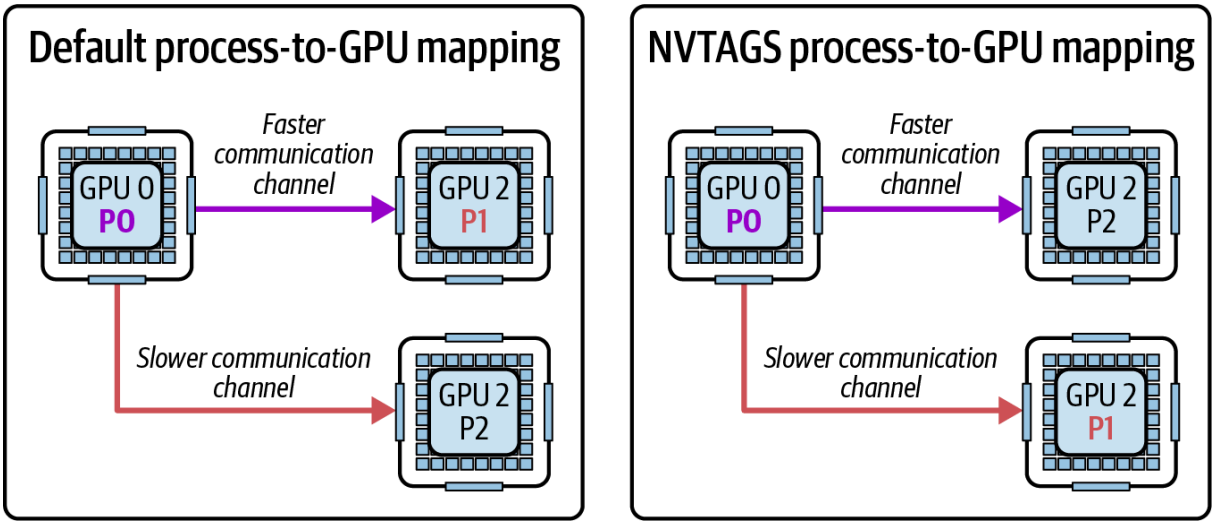


图 19-9. NVIDIA 拓扑感知 GPU 选择（NVTAGS）的进程到 GPU 映射

这里，NVTAGS 根据 GPU 间通信模式自动为进程分配 GPU 亲和性。NVTAGS 是 NVIDIA 的拓扑感知 GPU 选择框架，利用网络距离和链路指标自动完成进程—GPU 映射；它分析拓扑，并将进程分配到相互链路最快的 GPU。可用性与具体行为应以所用 NVTAGS 版本文档为准。

例如，若遥测显示该链路因激活张量很大而趋于饱和，调度器可将进程 2 重映射到与 GPU 0“更近”的 GPU，理想情况下该 GPU 共享高带宽连接或位于同一 NVSwitch 模块。对 LLM 推理而言，这种自适应重映射会动态重分配哪个 GPU 承载哪个模型层。

> 作为起点，若未使用 NVTAGS，可利用系统拓扑图识别在网络拓扑语境下哪些 GPU 分组“更近”。

这种重映射可在初始化时及两次推理之间进行。有些系统通过预先或周期性分析运行确定最优放置。若两个进程每秒交换数十 GB 数据，可能时应放在同一节点；反之，计算受限或相互数据传输很少的进程可以容忍更远距离。

当系统监控性能并随条件变化迭代改进映射时，重映射就是*自适应*的。例如，若一次运行后发现最高流量连接位于不同节点上的进程 3 与进程 4 之间，调度器可将其中一个进程与同节点的另一进程交换，使 3 与 4 聚在一起。

自适应重映射的结果是会响应已观测流量模式而演进的 GPU 分配方案。通过将数据交换限制在本地域，它可直接减少跨节点流量。

例如，重映射后原先 50 GB/s 的跨节点传输可能变为两条 25 GB/s 的节点内传输。这可能消除网络瓶颈并降低该通信的网络延迟，但绝不能一般性地保证“降低 50%”；收益取决于路径、重映射成本和并发流量。

可将重映射表述为使用图划分算法的优化问题：图的边权为链路上传输的数据量，求解最小割或包含容量、内存及计算约束的变体。

> 注意，移动层/进程意味着移动模型权重。若模型层包含数 GB 数据，不宜频繁执行。最好在大批次之间执行，或在映射至少会保持相对稳定一段时间时应用该策略。

在深度学习推理中，可将自适应映射思想应用到流水线、张量和专家并行技术的 GPU 间通信中。彼此通信最频繁的 GPU 应尽量通过最强、最不拥塞的连接相连。

使用 NCCL 优化集合通信

NVIDIA Collective Communications Library（NCCL）是管理 GPU 集合通信的标准库，提供适用于多 GPU 环境的多种算法和优化。

许多推理负载涉及集合通信模式，例如汇集多个专家输出、广播参数或执行 all-reduce，如图 19-10所示。此处 NCCL 通信（流 1）与 GEMM 计算（流 0）重叠。

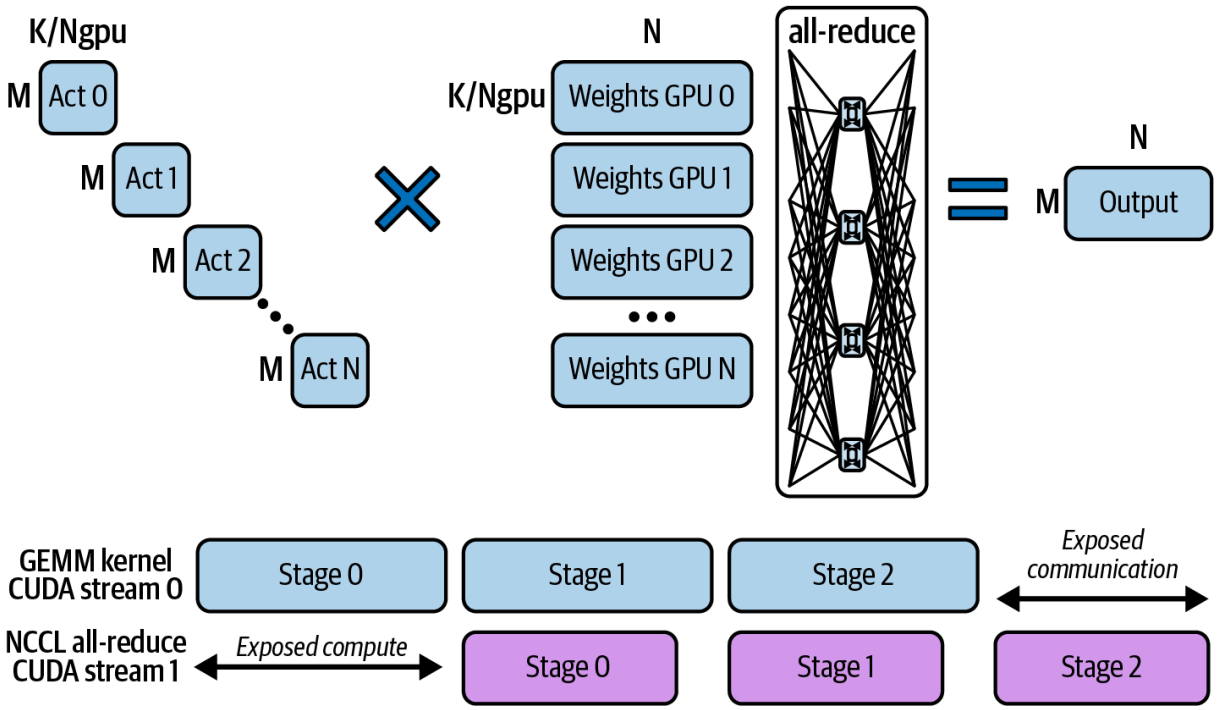


图 19-10. 使用多个 GPU 的分布式 GEMM 与跨 NVLink 的 all-reduce

在拥塞感知环境中，调度器可动态应用这些 NCCL 优化。根据拓扑和拥塞信息选择合适的集合通信算法并调优，能降低推理系统的通信开销。下面是调度器即时调优 NCCL 时可考虑的要点。

环形与树形 all-reduce

NCCL 可使用环形或树形算法执行归约及其他集合通信。在环形 all-reduce 中，每个 GPU 沿闭环传递数据，使每一部分数据依次经过所有 GPU。

环形方法通过保持所有链路忙碌，可能最大化 NVLink/NVSwitch 上的带宽利用率，但其延迟随 GPU 数量线性增长。例如，在 72-GPU 环中，完成一次归约的数据需要 71 跳。

相反，树形算法以对数方式归约或广播数据：GPU 被组织为逻辑二叉树，每一步参与者数量减半。但 GPU 的物理连接是逐链路的，可在逻辑上视为*树链*。图 19-11比较了 GPU 间的树形和环形通信。

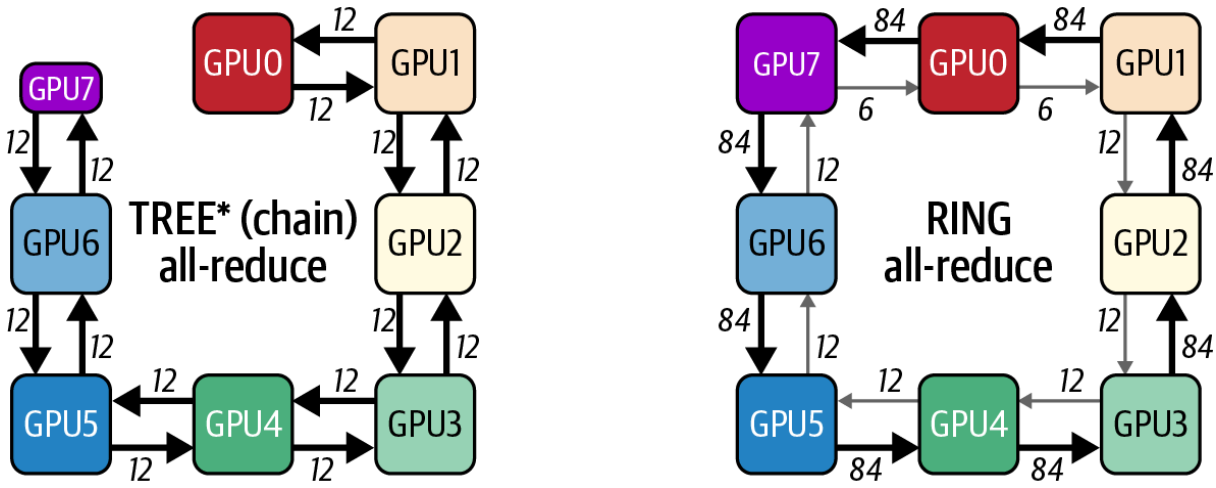


图 19-11. 树形（链式）通信与环形通信的比较

> 实践中，在树形 all-reduce 中，GPU 会先在每个节点内以简单 NVLink/NVSwitch“链”连接；跨节点时，以流水化的双二叉树拓扑连接。这使树形算法具有 `O(log N)` 级别的通信步数/延迟特征；具体 NCCL 算法与拓扑由版本、消息大小和系统配置决定。

树形算法以更少步骤完成（对于 72 个参与者，约为 $\log_2(72)$），可降低延迟，尤其适用于跨大量 GPU 的“长途”传输。树可利用 NVSwitch 的并行性，使多个独立归约流同时沿不同分支进行。代价是树未必在每一刻都充分饱和每条链路的带宽，但它避免长环路径使某一链路或 GPU 成为瓶颈。

> 对于小消息、延迟敏感的归约，树形算法通常更有优势；对于带宽主导的极大消息，网络未拥塞时环形算法通常更好。应按消息大小和拓扑选择，并考虑分层方案；以基准测试而非固定规则为准。

默认情况下，NCCL 会根据消息大小和拓扑以启发式方式选择环形、树形或分层变体。在基于 NVSwitch 的节点内路径上，环形算法常因带宽而被采用；跨节点时则常见分层和树形变体。即使在 NVL72 一类全连接单节点 NVSwitch 系统上，也不应一概强制树形处理大归约，应让 NCCL 默认选择或对候选算法进行实测比较。

例如，可用 `NCCL_ALGO` 环境变量请求使用树形算法，以避免单一 NVL72 机架中所有数据都经由跨 72 个 GPU 的长环路径，从而在某些负载下降低拥塞。树的约 6 个对数级步骤与环的 71 个顺序步骤不能直接等同于端到端耗时，因每步传输量、流水、通道数和链路利用率不同；必须测量后再强制设置。

调度器可使用动态 NCCL 调优参数，显式选择最适合当前拓扑和消息大小的算法。例如，GPU 数量很大时，可评估树形 all-reduce 以避免每次更新都环绕整个集群。

轮换环形端点

例如，当因简单且带宽效率高而选择环形算法时，一个担忧是相同链路可能持续承载最重负载，尤其是环回绕处特定 GPU 对之间的链路。

一种拥塞感知方法是在不同迭代或集合通信操作间轮换环的顺序。通过周期性改变哪一 GPU 是环的起点、以及哪些 GPU 对先通信，可将通信负载更均匀地分布到各条 NVLink 连接上。

NCCL 的“inside/outside”环机制已在连续调用间交替环方向；若负载持续不平衡，在步骤间进一步打乱 rank 可能有帮助，从而降低某条 NVLink 长期成为瓶颈的风险。具体行为依 NCCL 版本、算法和拓扑而定。

实践中，NCCL 的交替环机制可在内部实现此类轮换；也可由调度器针对不同集合通信，在通信器中重新编号 GPU。为此可周期性地以置换后的 rank 顺序调用 `ncclCommInitRank`。这样长期来看，没有单条链路或单个 GPU 总是每次集合通信的关键路径，从而平滑利用率；但重建通信器存在同步与初始化开销，不能频繁用于热路径。

集合通信的波次调度

波次调度不一次启动使用全部 GPU 的巨大集合通信操作，而是将通信拆分为分阶段的波次，以减少瞬时负载。例如，设想推理负载需在 72 个 GPU 间交换 embedding；这是 MoE 或某些集成方法中常见的 all-to-all 模式。

若把此交换作为单一整体步骤，每个 GPU 会同时向其他 71 个 GPU 发送数据，即 72 个 GPU × 71 条消息同时饱和各链路与 NVSwitch 端口。

72 个 GPU 同时交换会产生尖峰。可改为拆成 4 组、每组 18 个 GPU 的波次，以平滑流量。

这称为*波次调度*：将交换组织为一系列较小的 all-to-all，每个波次只使用部分 GPU；也可对更小的数据块做流水，使任一时刻只有部分流量在传输中。

用 NCCL 的术语说，这可能对应于在内部将一个大型 all-reduce 拆成多个切片。NCCL 确实会自动这样做，以便通过环形路径流水化数据；调度器也可使用 NCCL 编排一系列较小的集合通信。

错开各通信波的启动时间后，在下一波增加流量前，前一波的数据已部分通过网络结构，从而为网络留出余量。这称为*时间复用*，可避免 NVSwitch 网络过载；概念上类似对网络流量做节流以避免突发。

另一示例是重叠计算与通信。若按波次归约层输出，系统可让下一层计算与归约的后续波次重叠。

这在计算与通信间形成流水线：部分 GPU 正完成归约时，其他 GPU 已开始下一层计算。这种重叠实质上将部分通信移到计算单元原本可能空闲的时间，从而在不出现导致拥塞的巨大一次性尖峰时提高 NVLink 带宽利用率。

重要的是仔细优化集合通信、选择正确算法，并将通信组织为均衡波次。这对拓扑感知调度至关重要，有助于所有 GPU 更高效、公平、均衡地使用 NVLink/NVSwitch 网络。

使用 GPUDirect RDMA 的多节点、多机架通信

扩展到单一节点（如 NVL72 机架）之外时，会出现更多挑战，因为通信须经过相对较慢的 InfiniBand、Ethernet 等网络接口。此时 NVLink 和 NVSwitch 不再直接连接系统中所有 GPU；不同节点的 GPU 使用网卡和网络交换机交换数据。

为在多节点、多机架环境保持高性能，现代 AI 系统使用 GPUDirect RDMA。如第 4 章所述，GPUDirect RDMA 可让 GPU 直接向远程 GPU 内存发送/接收数据，而无需主机 CPU 参与，如图 19-12所示。

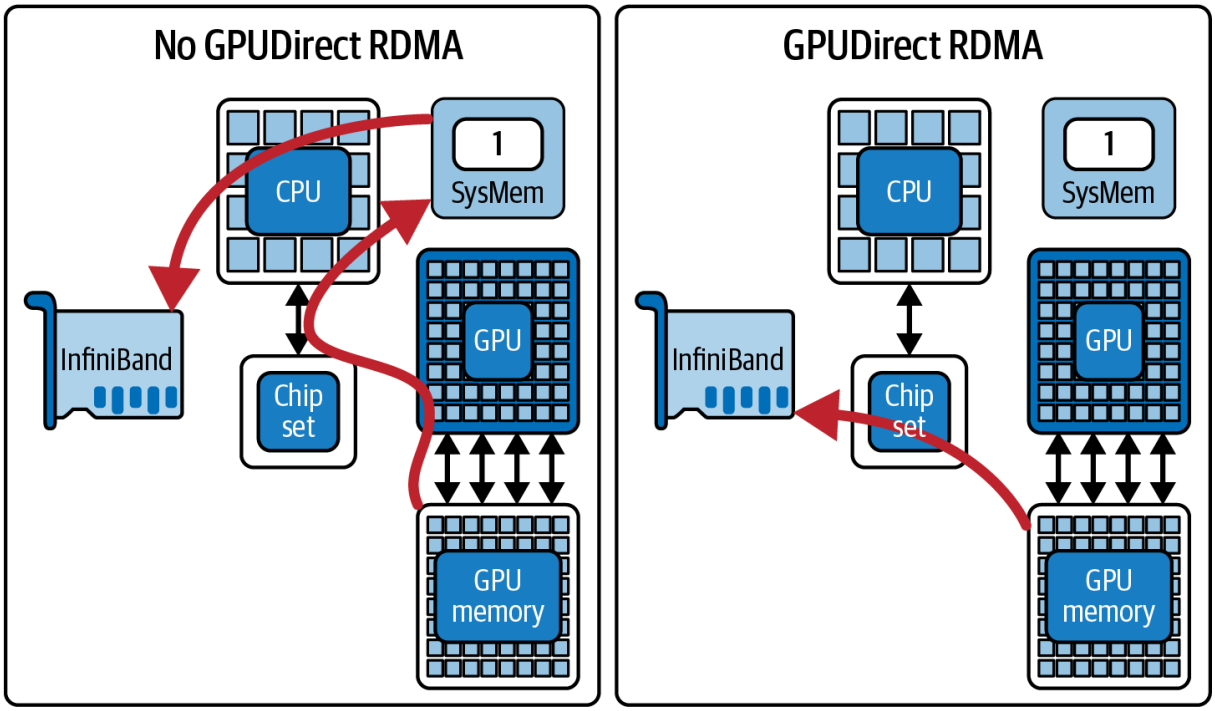


图 19-12. 借助 GPUDirect RDMA 的 GPU 到 GPU 直接内存传输，不涉及主机 CPU 内存（来源：https://oreil.ly/445a9）

即便 RDMA 效率很高，网络带宽通常仍低于、延迟仍高于节点内 NVLink。若没有动态自适应路由调度，集群网络结构中的拥塞会成为限制因素。前文所述的拥塞感知调度器除节点内流量外，也可智能路由并均衡节点间流量。

一项关键技术是利用多个网络接口，即*多轨（multirail）*。高端 GPU 服务器常有多个 NIC，例如每节点双 InfiniBand 端口，某些设计甚至每 GPU 一个 NIC。相对于单 NIC，每节点使用两个 NIC 的吞吐量在合适的多轨拓扑中可能接近翻倍；多 NIC 也有开销，实际增益须测量。

NCCL 支持使用多个 NIC 并行增加带宽，并可在这些 NIC 间拆分环和树。拓扑感知在此至关重要：如果每个 NIC 连接不同网络交换机，形成集群网络中独立“轨道”，每次集合通信使用多个 NIC 可降低任一网络路径的负载。

NCCL 环境变量 `NCCL_CROSS_NIC` 控制一次集合通信是否可在不同节点的同一环/树上使用不同 NIC。在设计良好的网络拓扑中启用它时，NCCL 可能将一部分 GPU 数据经 NIC1 发送，另一部分经 NIC2 发送，从而增加带宽并缓解单链路瓶颈；并不保证吞吐量恰好翻倍。

若 GPU 节点有多个 NIC，且所用 NCCL 版本支持 `NCCL_CROSS_NIC`，可在为多轨设计的拓扑上对大型集合通信启用它，使流量跨轨道条带化；先验证性能与网络设计是否匹配。

调度器可检测某个 NIC 或路径是否接近容量上限。若接近上限，可在具备可用替代接口或路由时，将部分 GPU 通信迁移过去重新分配流量。网络层自适应路由可进一步改善这一点；应用也可将特定 GPU 流量固定到利用率较低的 NIC。如何把 NCCL 通道映射到 NIC 取决于 NCCL 版本、拓扑文件及环境变量，并非简单设置即可安全完成。

在多节点环境中，重路由还意味着与网络的自适应路由功能协作。现代 InfiniBand 网络可将拥塞流自动移到网络结构中较不拥塞的路径。尽管由网络层处理，上层调度器仍可例如通过改变 GPU 传输使用的目标 IP/路由来影响它，或将传输拆成小块以便网络进行均衡。

此外，重要的是强制 NIC 亲和性：将每个 GPU 的通信绑定到最近的 NIC（如位于同一 PCIe 根复合体或同一 NVSwitch/CPU 复合体的 NIC），以减少本地争用。

要实施 GPU–NIC 亲和性，可使用 NVML 或 NCCL 映射 GPU ↔ NIC 的本地性。可向 NCCL 提供关联每个 GPU 与指定 NIC 的静态拓扑描述，使其遵守该映射（例如 GPU 0–3 ↔ NIC 0，GPU 4–7 ↔ NIC 1）。

设计良好的系统会对齐 GPU–NIC 配对，让每个 GPU 的数据以最短路径离开节点。若同一端口的多个重 GPU 流使某 NIC 拥塞，调度器可在下一轮通信时将一个 GPU 的流量重新分配到节点内另一端口。NCCL 自动调优会处理其中许多情况，但针对特定且持续的问题可能需要人工覆盖。

设想一个极端情况：大型、高负载推理集群托管巨型 MoE LLM。若系统调优不当，节点间大量通信会使网络成为主要瓶颈。

在这类极端情形中，调度器可决定在多个节点复制某些数据（如专家）以减少跨节点查询；也可分层执行操作，先在每个节点内聚合结果，再在节点间交换摘要，而非让所有 GPU 跨节点交换完整数据。

NVIDIA SHARP 可将某些聚合操作卸载到交换机硬件。在推理集群中，结合 SHARP 与自适应路由有助于减少通信瓶颈；是否支持及收益取决于网络硬件、软件栈和所用集合通信。

在多节点环境中，需要拥塞感知调度以避免任何单条网络链路饱和。这类调度要求审慎的路由/绑定决策、使用 GPUDirect RDMA 避免不必要的内存复制，并利用多轨 NIC 最大化带宽。

目标是将拓扑感知扩展到 NVSwitch 之外，理解集群网络完整拓扑（如 fat-tree、dragonfly 等），并让通信模式适应该拓扑。这样，即便在大规模 GPU 集群中，也能更均衡地编排节点间传输，降低单条慢链路拖慢整个分布式推理系统的风险。

> 应像对待 GPU、内存等一样，将网络视为可调度资源；应像其他动态分配资源那样规划并自适应管理。

MoE 专家重平衡与重新分组

大规模语言模型越来越多地采用 MoE 层，从而引入独特通信模式。在 MoE 模型中，神经网络的不同子集（即专家）驻留在不同 GPU；每个输入 token 被路由到少量专家网络处理。

例如，在推理中，这会产生 all-to-all 流量模式：token 被发送到承载所选专家的 GPU，再将结果汇集回来。若静态地朴素分配专家到 GPU，许多 token 经常路由到特定 GPU 的专家时，该 GPU 可能成为通信热点。若经常协作处理相似 token 的专家在拓扑中相距很远（例如一个在 GPU 0，另一个在网络另一侧 GPU 71），token 需要不断经过长路径。

专家重平衡是一种通过周期性调整每个专家所在 GPU 来本地化通信的策略。关键是利用工作负载中的偏斜或模式。例如，若专家 5 和专家 19 都接收同一批查询的片段，可能时将它们放在同一 GPU（或同一节点）是合理的，可减少这些操作的通信距离。

同样，若专家 7 非常热门并接收大量 token，其 GPU 可能承受大量入站流量。调度器可以把该专家移到通信较少的 GPU，或在系统允许时复制该专家以分摊负载。此类重平衡可在推理运行之间，或长运行服务的定期维护窗口进行。系统收集专家之间、或专家与专家门控节点之间的通信频率统计，再将专家重映射到可最小化高流量链路的 GPU。

实践中，实现 MoE 专家重平衡需要协调地重新分发模型参数，因为专家本质上是模型权重的子集。这是重操作，不应频繁执行；但偶发重平衡可能显著减少拥塞传输。

由于移动一个专家可能传输数 GB 权重，专家重平衡应在计划维护窗口进行。关键是利用记录的路由指标选择更好的运行时放置策略，同时计入迁移成本、KV/会话亲和性、容量和服务中断风险。

重平衡后，理想状态是每个 GPU 承载一组专家，使大多数 token 的路由停留在单 GPU 或至少本地 NVLink 组内；其余非本地通信更均匀地分散在 NVLink/NVSwitch 网络中，而非反复击中相同 GPU 对链路。

> 简言之，分散热门专家，并将经常相互通信的专家共置。

另一项相关优化是专家分桶或分组：将经常一起使用的专家安排到同一组 GPU（例如同一 NVSwitch 或同一服务器）上，以减少跨组流量。

调度器可将专家放置视为图划分问题。专家（或承载专家的 GPU）是图的节点，边权表示专家间以及路由器到专家的 token 流量。使用最小割划分图，尽量切过较少的重边。由此，MoE 通信成为拓扑感知的，遵循 NVLink/NVSwitch 边界，并将数据交换限制在这些边界内。实际方案还必须满足专家容量、负载均衡和模型精度约束。

MoE 专家重新分组是模型架构层面拥塞感知调度的例子：它调整工作负载以适应网络，而非调整网络以适应工作负载。

动态拥塞感知调度

前述技术虽可在系统启动或设计阶段配置，最稳健、先进的系统还会使用动态调度来响应正在发生的拥塞。动态拥塞感知调度是指系统利用前述遥测持续监控网络状况，并实时调整任务或通信调度。

拥塞感知调度与路由有助于减少瓶颈，并在动态条件下保持较高性能。这类似网络层的动态分组路由，如图 19-13所示。

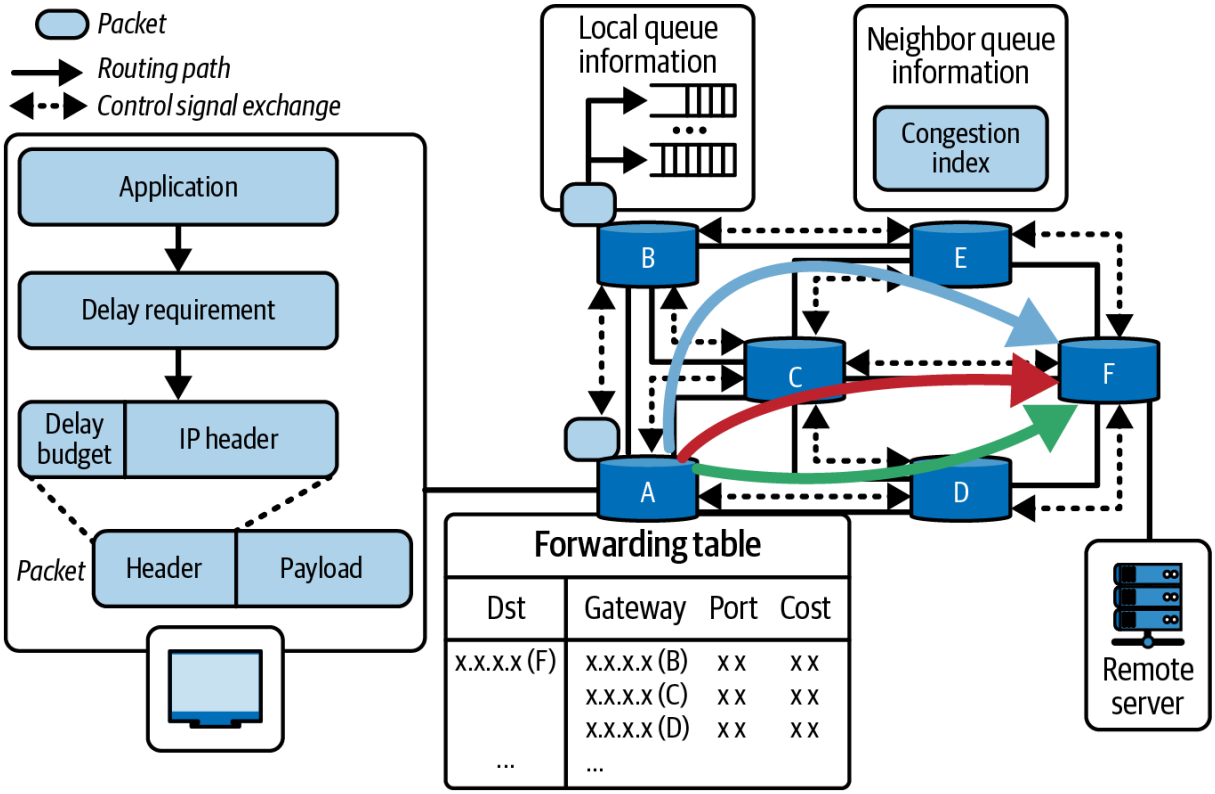


图 19-13. 避免拥塞的网络层分组路由

在多 GPU 推理中，动态策略包括依据拥塞反馈限流、重路由或重排操作。例如，调度器检测到连接两块特定 GPU 的 NVLink 0 号链路，正因大型流水线并行激活传输中的巨型张量而满载。

若另一个高优先级传输也计划使用同一链路，调度器可将第二个传输延后数毫秒，让第一个完成并清空队列。这称为*时间负载均衡*：插入微小间隔以防队列积累，类似网络交换机队列管理与 NIC 级反压。短暂排队往往优于过载，但具体策略须受应用 SLO、可靠性和传输语义约束。

反过来，若发现一个通常很大的传输因源 GPU 等待计算而空闲，调度器可利用该时间片经此链路发送较低优先级数据，填补空闲时间。这能利用可用带宽而不干扰关键数据传输。

另一种动态手段是软件层自适应路由：某路径拥塞时，若有替代路径则选择它。在具有多个 NVSwitch 平面或多个 NIC 轨道的网络中，自适应运行时可为下一次通信选择较空闲的平面。

多路径存在时 NCCL 会在内部处理部分此类工作，但高级调度器可维护映射到不同路径配置的多个 NCCL 通信器，再按拥塞情况从中选择。通信器创建、拓扑绑定和同步有成本，须避免在每个请求热路径中频繁更换。

动态选择替代路径要求调度器评估最佳路径；可借助不同虚拟通道，或由系统软件/硬件调节传输使用的 NVLink 路径。应用层通常不能任意选择单个 NVLink 端口，应依赖受支持的 NCCL、驱动与网络配置接口。

现代 NVSwitch 系统可能支持多个虚拟通道与硬件服务质量（HQOS）设置。若平台公开这些控制并已正确配置，调度器可将非紧急流量导向低优先级通道，以避免与紧急传输争用。

依赖负载的任务调度是动态拥塞管理的另一能力。若推理服务器处理许多共享资源的并发查询，调度器可暂时将部分查询排队或重排，以避免峰值重叠。这类似前文错开大型集合通信、避免它们同时运行的做法。

例如，调度器知道查询 A 下一步将涉及跨 GPU 的巨大 all-gather，又看到查询 B 刚开始且会同时加入另一个大型 all-gather。这时可短暂推迟启动查询 B 的该步骤，让查询 A 的通信在没有 B 的资源争用下完成。

这种细粒度调度可优化随时间变化的通信模式：重流被串行化或错开，而非并发启动；决策由最近遥测引导。若系统发现并行运行 8 个查询时 NVSwitch 利用率显著尖峰，可尝试第一波仅运行 4 个，紧接着再运行 4 个。系统因监控实时遥测并持续寻找避开拥塞的执行计划而具有自调优特性，但必须同时保持公平性和尾延迟护栏。

动态调度通常实现为监控所有 GPU 与网络链路的集中式调度器。它还可结合分布式协议：目的 GPU 的接收能力受限时，GPU 间互相发出拥塞或反压信号。

在该反压场景中，目的 GPU 通知源 GPU 暂停传入传输。智能调度器可在源 GPU 暂停时重新安排任务，使其在等待目的 GPU 解除暂停期间执行计算任务。实际可观测的反压机制和可控制粒度取决于硬件、驱动与通信库。

> 当接收端无法跟上时，NCCL 的进度会受到流控/反压影响。自定义调度器可通过观察通信操作进度受阻来利用这段时间执行其他有用工作；不要假定能直接访问每个接收端的 NVLink 缓冲状态。

随着时间推移，这类动态调整可在批次大小、输入数据分布和其他工作负载变化下维持较高通信效率。系统根据实时反馈修改调度，学习拥塞模式并快速适应；可使用前节讨论的 RL 智能体或一组启发式规则。对突发且不可预测的推理请求环境，这种能力尤为重要。

以精细调度协调 NVSwitch 传输

NVLink/NVSwitch 系统的核心是 NVSwitch 网络结构本身。它是处理大量并发 GPU 到 GPU 传输的集中式交叉开关。NVSwitch 具有极高带宽，并有自身内部调度算法，包括跨多个交换芯片与平面的自适应路由；具体实现细节并非总是向应用暴露。

不过，软件可利用对流水线、张量和专家级并行等应用层策略的了解来调度数据传输，从而提升其有效性。目标是在不过度订阅集群网络的前提下，编排哪些 GPU 对通信以及何时通信，以最大化并行性。

一种成熟技术是错开通信波次。它与前文集合通信的波次调度有关，但可更广泛地应用于任意重叠传输。

设想 NVL72 机架中的 72 个 GPU 都要向特定对端发送数据，例如由 GPU 0 收集所有 72 个 GPU 结果的中心参数服务器。若其他 71 个 GPU 精确同时发送，GPU 0 的 18 条 NVLink 链路以及连接它们的 NVSwitch 会遭受 71 路输入的大突发，如图 19-14所示。这可能超过该时刻可传送的带宽。

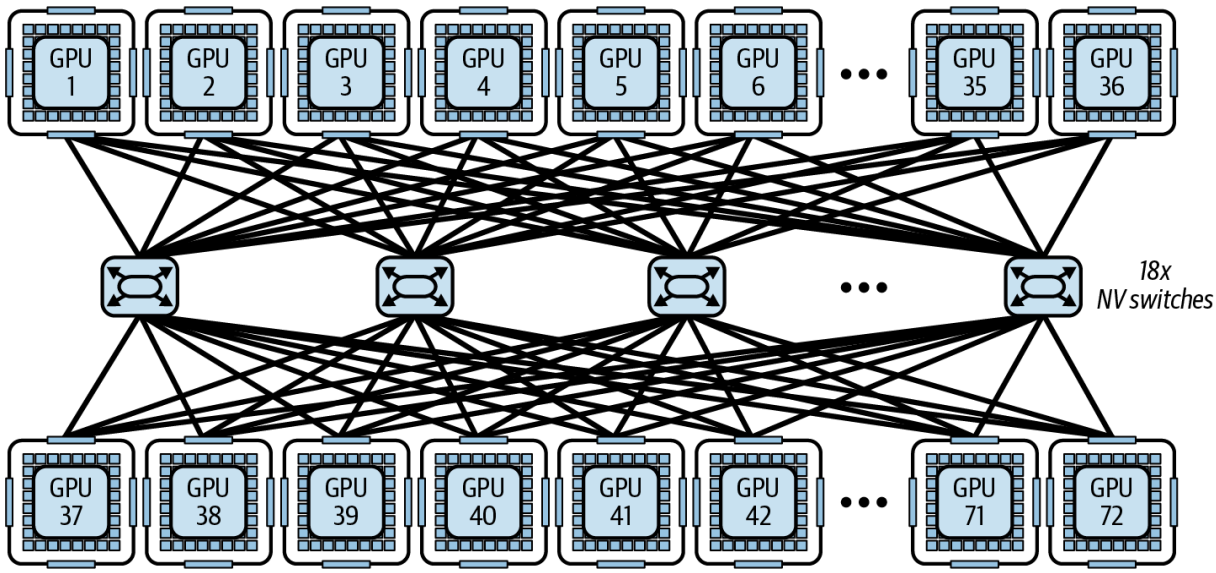


图 19-14. 全部 72 个 GPU 向中心参数服务器发送数据

在此情况下，NVSwitch 需要缓冲并串行化其中许多传输，导致延迟尖峰。替代的协调优化方法是将发送方分为四组：第 1 组（GPU 1–18）先发送，数微秒后第 2 组（GPU 19–36）发送，依此类推。波次大小和间隔只是示例，应根据消息大小、端口数、实际遥测和 SLO 调整。

从 GPU 0 视角看，它顺序接收四个较小的流量波。任一时刻约只有 18 个 GPU 向 GPU 0 发送，这与其 18 端口数量相称；但端口数量并不能保证完全无排队或链路不饱和，因为带宽、报文调度和其他流量也会影响结果。第 4 组完成时，GPU 0 收齐所有数据，流量在时间上得到平滑与均衡。

这种错开波次的方法可泛化到许多模式。all-to-all 交换可拆成按轮次轮换的成对交换，常称为*蝶形*（butterfly）或*洗牌模式*（shuffle pattern）。洗牌会安排每个时间步哪些 GPU 相互通信，使每个 NVSwitch 端口保持忙碌但不过载。

NVSwitch 传输调度器可采用时间片算法，为特定 GPU 对或 GPU 组分配通信槽位。它不启动一个自由竞争的大型批量传输，而是执行多个小型同步通信步骤，每个步骤分配特定时间槽。这类似前述时分复用，有助于更可预测地使用 NVSwitch 交叉开关，但是否无冲突取决于实际流量和硬件调度。

值得注意的是，NVSwitch 硬件自身会尝试降低网络争用。例如，多个流争用同一链路时，硬件可交错调度各流的报文以促进公平性；确切仲裁行为取决于平台实现。

若可用，它还可能自适应选择不同内部交叉开关路径。不过从软件视角，可将这些自适应技术纳入网络设计，尽量避免一开始就触及这些限制。

精细调度还包括并发控制：限制推理期间并行运行的重传输数量。例如，在多 GPU 推理流水线中，可避免同时启动所有跨 GPU 专家汇集或广播操作。这在设计上以少量并行性换取更少争用。

推理中跨 GPU 同时进行 2–4 个大型专家汇集或广播传输有时是合适的折中点；更多可能带来收益递减或拥塞。这个数不是普适规则，须通过目标模型、拓扑和负载的基准测试确定。

例如，与其同时触发 12 个有使 NVSwitch 和 NVLink 饱和风险的集合通信，调度器可错开它们：每次最多运行 4 个大容量传输，等待完成后再启动下一组。由于避免过多传输重叠导致的拥塞，这种更受控方式在某些负载上可能更快；是否更快必须用端到端测量验证。

协调 NVSwitch 传输，就是把通信网络视为可调度的共享资源，类似调度 GPU 内核和 CPU 线程。通过调度网络资源，系统让高优先级流量避开干扰，并用低优先级流量填补空闲间隙以维持较高利用率。

错开和分组通信等技术通过避免严重争用模式，可能提高 NVSwitch 有效吞吐量。这将带来更可预测、延迟更低的通信；对关注尾延迟（由网络拥塞造成的异常慢响应）的推理服务尤为重要。

总之，多 GPU 推理中的拥塞感知、拓扑感知调度，关键在于让通信模式与给定硬件布局智能匹配。高性能推理系统会实时监控链路使用情况并适应 NVLink/NVSwitch 拓扑，方法包括谨慎放置任务、优化集合通信算法配置、多节点路由调整、MoE 专家重分配、动态运行时调整和数据传输的细粒度协调。

其他自适应与动态优化技术

下面介绍一些补充核心调优策略的动态推理和运行时自适应技术。截至写作时，这些思路多为实验性且尚未广泛可用，但具有潜力，值得介绍。每项技术均附有简要描述和延伸阅读链接。

动态提前退出网络

提前退出模型允许 LLM 在达到足够置信度时自行截短生成，例如减少对简单输入的不必要计算。动态提前退出方法监控中间表示与 logit 熵，在每层或每个 token 决定是否停止计算并输出最终结果。

这类网络因在中间层添加辅助分类器，需要专门模型架构或训练。文献在特定推理任务和评测设置中报告了最高约 30%–50% 的推理加速，且无明显精度损失（见 *https://oreil.ly/dn3vl* 和 *https://oreil.ly/73AeE*）；不能直接推广到任意 LLM 或任务。

输入感知层跳过（DASH）

DASH 一类框架将推理建模为马尔可夫决策过程，依据输入特征逐 token 动态决定执行或跳过每个 Transformer 层。通过学习小型评分网络，DASH 在其设置中可为许多 token 跳过 20%–40% 的层。

DASH 通常需要修改模型，在每层加入门控。但它可能在保持 NLP 基准性能的同时显著降低推理成本（*https://oreil.ly/mz59T* 和 *https://oreil.ly/oL_Lr*）；生产采用前应针对目标

质量、安全性和延迟进行验证。

推测式 MoE 专家路由与通信削减

对于 MoE 模型，推测式专家路由预判后续 token 会激活哪些专家，并提前共同重排 token 与专家分配。

该技术会提前将 token 发送给预测专家。若预测错误，会浪费部分工作；预测良好时，整体通信会减少。相对静态专家并行（EP）+ 张量并行（TP）部署，相关研究在其评测设置中报告跨节点带宽用量最高可下降 30%；收益取决于路由可预测性与实现开销。

使用 LazyLLM 的动态 token 剪枝

LazyLLM 使用轻量评分函数判定重要 token，并选择性地仅为这些 token 计算 KV 缓存。它在 prefill 和 decode 中剪除影响较低的 token（如停用词和填充 token）。通过只将昂贵注意力计算用于相关 token，LazyLLM 在长上下文工作负载的评测中报告端到端延迟降低约 20%–30%；剪枝策略必须针对模型、任务和质量要求验证，因为被视为低影响的 token 也可能承载关键语义。

面向边缘的 MoE 内存预算

双路由和动态调度技术在边缘部署等受限环境中，会给专家权重带来潜在内存问题。实践中，可让一部分专家常驻 GPU 内存，其余专家按需从闪存调入，通常依据使用频率决定。调入延迟、闪存耐久性、预取准确率和回退路径会决定实际可行性。

通过动态调整哪些低比特专家驻留内存、哪些被卸载，推理系统可在较低内存使用量下保持较高专家激活率；效果依赖工作集稳定性、I/O 带宽与量化质量。

动态量化与激活范围调整

静态 PTQ 与 QAT 是广为人知的技术；在线量化策略可在推理期间实时调整激活量化参数。例如，可使用滑动窗口统计每隔 N 个 token 修改激活量化器的 observer 范围。

此类动态激活量化实时监控激活统计，并每隔 N 个 token 重新计算 observer 范围。因而可将 FP16 分给方差高的“热”层，将 FP8 分给方差低的“冷”层；低方差层受限于较窄动态范围。其目标是降低量化误差并提高吞吐量，但每层、每 token 的精度切换并非大多数推理框架的现成功能，测量与控制开销可能抵消收益。

低方差层产生的激活若紧密聚集于均值附近，FP8 有限的指数和尾数位（如 E4M3）可能足以准确表示其值，进而节约计算和内存；是否没有可察觉的精度退化必须按目标模型、任务、输入分布与量化格式实测验证。

可考虑采用混合 FP8 策略：前向传播（如激活和权重）使用 E4M3，反向传播（如梯度）使用 E5M2。下面是使用 Transformer Engine 延迟缩放机制的代码；纯推理通常不执行反向传播，实际可用格式与策略取决于硬件和 Transformer Engine 版本：

```
from transformer_engine.common.recipe import Format, DelayedScaling
recipe = DelayedScaling(
    # E4M3 fwd, E5M2 bwd
```

```
        fp8_format=Format.HYBRID,
        amax_history_len=1024,
        # delayed scaling window
        amax_compute_algo="max",
    )
```

这里用 `amax_history_len=1024` 设置延迟缩放窗口，这是一个常见默认值。除非收敛分析表明另有需要，否则通常保持 `amax_compute_algo='max'`；对部署环境应遵循所用 Transformer Engine 版本文档并进行数值验证。

同时，FP16 可保留给激活波动更大、即“热”的层。这些层需要更宽动态范围，以保留关键计算所需的数值保真度。

这种混合精度策略可用于某些无需离线校准的推理流水线，并按层调整数值保真度，从而权衡整体性能与模型精度。是否可用及收益需随模型和框架验证。

要点回顾

在现代 GPU 上，本章讨论的方法是从这些万亿参数级模型中挖掘每一分性能的实用途径。在真实部署中，动态运行时自适应技术可能决定高性能推理服务的成败。以下是本章的几个要点：

使用 `torch.compile` 的稳态推理

若可承担预热时间，优先使用 `mode="reduce-overhead"` 或自动调优模式。这有助于降低低延迟推理负载的运行时开销；支持情况与收益取决于 PyTorch 版本、模型图和输入形状。

内核级自动调优（kernel-level autotuning）

动态优化 GPU 内核和 tile 大小。可行时利用张量内存加速器（Tensor Memory Accelerator，TMA）进行异步内存预取。优先使用提供自动调优的 CUTLASS、Triton 等库和编译器；除非性能确有必要，避免手工调优。

自适应精度（adaptive precision）

推理中可在 8 位和 4 位浮点（FP8/FP4）间切换，以平衡速度与精度，并按需混用 16 位精度。PyTorch 中可使用 Transformer Engine 支持 FP8，因为 `torch.autocast()` 不直接支持 FP8。实际精度切换支持与质量应按硬件、框架和模型验证。

解耦式推理流水线（disaggregated inference pipeline）

将预填充（prefill，提示词处理）和解码（decode，生成）阶段分配到不同资源，并根据 KV 缓存命中、队列深度和负载等实时因素进行上下文感知请求路由。这有助于在长提示词下保持高吞吐量，同时避免拖慢短提示词响应；需权衡额外通信与调度开销。

动态并行策略（dynamic parallelism strategies）

根据输入/输出序列长度、模型结构（包括 MoE 路由）在数据并行、张量并行、流水线并行与混合执行组合间做运行时决策，包括复制或切分模型。生产环境通常将请求路由

到预部署的池，而非对活跃实例即时重分片。

自适应解码与调度（adaptive decoding and scheduling）

使用推测解码、运行中批次重塑和 token 级调度等技术改善吞吐量与延迟。vLLM、SGLang 和 NVIDIA TensorRT-LLM 等引擎实现了其中部分能力；这并不自动证明每种技术在所有生产负载下均有效，仍需测量。

内存管理与统一内存（Unified Memory）调优

可利用 Grace CPU 的统一寻址内存，将不常用 KV 缓存页卸载到 CPU 或 NVMe；使用 `cudaMemAdvise`、`cudaMemPrefetchAsync` 等 API 优化放置。若可用可考虑 GPUDirect Storage；它支持在适当的软件/硬件配置下绕过 CPU 内存路径在 GPU 与存储间传输数据，但并不等同于透明的按需分页。

分析驱动的优化（profiling-driven optimization）

使用 NVML、Nsight Systems/Compute、NVTX 插桩和 Prometheus 指标识别运行时瓶颈，并在验证后应用图与内核优化。还可使用 AI 分析遥测数据，以发现异常和优化机会。

结语

本章技术可将静态推理部署转变为可自优化、可自适应的引擎。通过监控运行时信号（延迟、利用率、内存、网络吞吐量），并应用动态并行、精度缩放、自动调优内核、主动缓存、基于 RL 的控制和智能调度等策略，可推动超大模型推理接近系统极限。

成功的推理服务可在支持超大模型规模的同时，处理大量用户、大型输入上下文、大量上传文档、复杂推理和严格延迟 SLA。若这些优化在目标负载中有效，还可能以更少硬件实现相同吞吐量，从而改善成本效益；这不是自动保证。

请记住，网络结构是系统协同设计的一部分，而非事后补充。目标是有效利用 NVLink 和 NVSwitch 集群网络。随着模型和 GPU 集群增长，在理想且均衡的条件下，这可能带来接近线性的吞吐扩展，同时保持低延迟。借助适当调度，NVLink/NVSwitch 网络中的 GPU 可表现为紧密耦合的加速器，从软件视角近似一块大型 GPU；实际扩展性受通信模式、算法与硬件限制。

每项策略都是 AI 系统性能工程师工具箱中的工具。最有效的方案通常会组合使用它们。例如，可训练 RL 策略决定何时切换并行模式或调整精度，或通过预热让连续批处理调度器保持就绪。

不必同时实现全部技术；即便一两项也可能带来可观改进。可先从最容易的缓存和批处理改进着手，再逐层加入其他技术。

这些技术的主题是灵活性与适应性。推理运行时应能够响应当前工作负载重新配置自身，从而将庞大 LLM 转化为调优良好、可扩展且成本有效的生产服务。

第 20 章. AI 辅助性能优化与迈向数百万 GPU 集群的扩展

本章汇集一系列案例研究与未来趋势，说明人类与 AI 如何协作优化 AI 系统性能。具体而言，AI 可协助微调底层 GPU 代码，产生比人工实现运行更快的内核。

从更广角度看，这些例子表明，即使是矩阵乘法等核心操作中的算法创新，也可能带来类似添置新硬件的性能收益。概念上，可考虑利用一系列强化学习（reinforcement learning，RL） rollout（例如迭代）的奖励反馈来搜索最适合目标环境的 GPU 内核代码，如图 20-1所示。

这些 AI 辅助方法可帮助提升性能、缩短训练时间并降低运营成本，也可能让较小系统更高效地部署更大模型，推动未来 AI 进展。换言之，这是 AI 帮助创造更好的 AI。

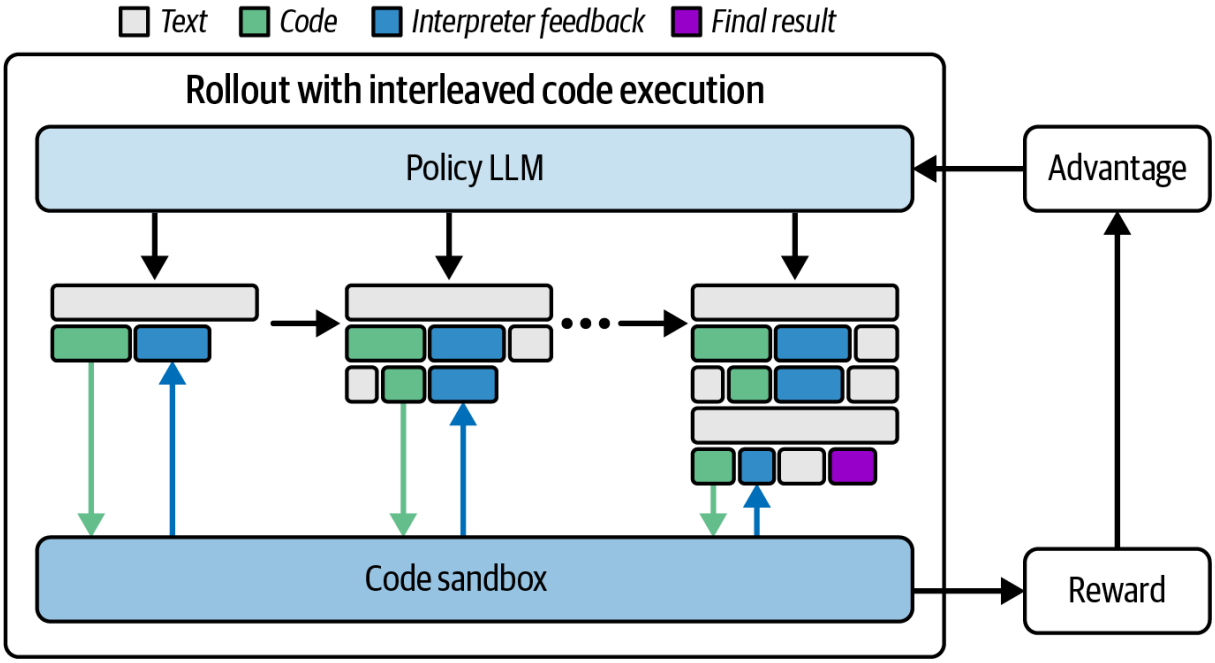


图 20-1. 使用强化学习为目标环境搜索最优 GPU 内核代码

AlphaTensor 发现的算法提升GPU 性能（Google DeepMind）

并非所有 AI 优化都发生在代码层面。有时，优化会深入算法和数学领域。一个开创性例子是 DeepMind 2022 年的 AlphaTensor 项目，其中 AI 被用于发现新的通用矩阵乘法（general matrix multiply，GEMM）技术。

GEMM 是支撑几乎所有模型训练和推理负载的核心操作。GEMM 效率的轻微改进都可能对整个 AI 领域产生巨大影响。AlphaTensor 将快速算法搜索形式化为单人博弈，使用强化学习探索大量不同可能性。

令人惊讶的结果是，它找到的矩阵乘法公式在当时的某些设置中优于已知人工推导方法。例如，如图 20-2所示，它重新发现了 Strassen 著名的 2 × 2 矩阵次二次算法，并针对更大矩阵尺寸进行了改进。

真正的验证来自这些算法在实际硬件上的测试。AlphaTensor 找到了一种面向 NVIDIA Volta V100 GPU 世代的方法：在其测试设置中，大矩阵乘法比当时 V100 世代的标准 NVIDIA cuBLAS 库快约 10%–20%。GEMM 性能提升 10%–20% 具有重要意义，近似于为模型前向和反向传播获得额外算力；但结果受矩阵形状、精度、库版本和硬件限制。

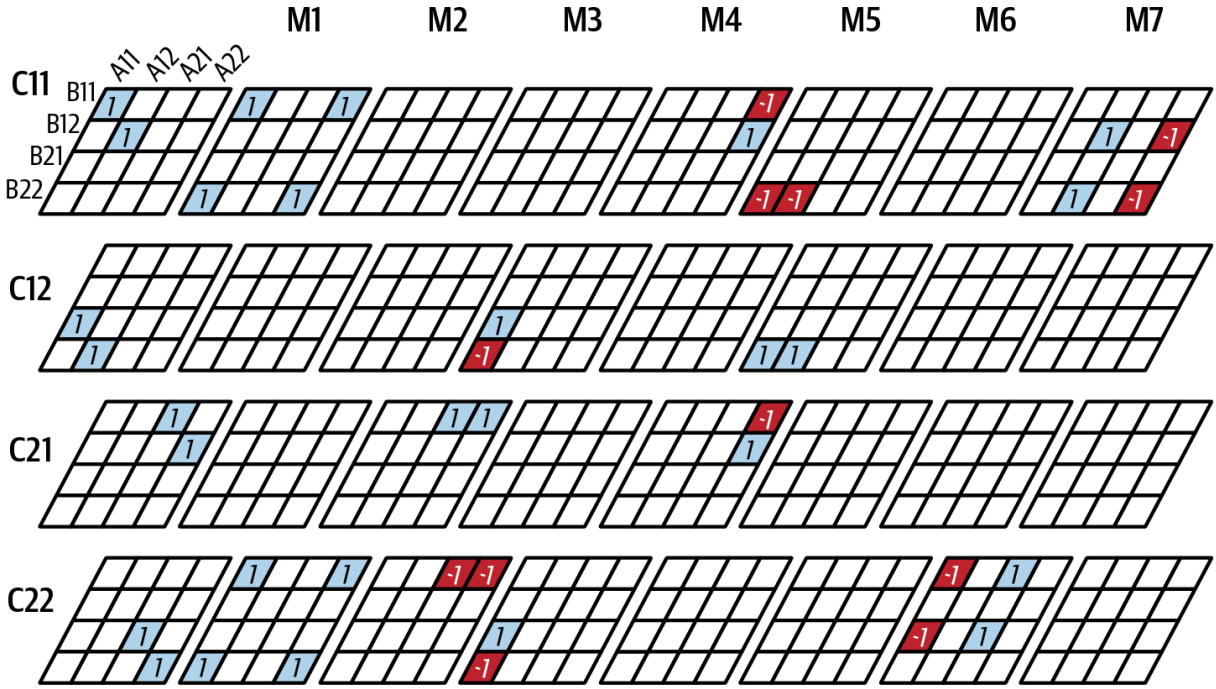


图 20-2. Strassen 用于乘法 2 × 2 矩阵的次二次算法（来源：https://oreil.ly/5jzLn）

这类收益通常来自新一代硬件，或数月的底层 CUDA 调优；但在此案例中，AI 在相对较短时间内从数学上找到了更优方法。

启示是：在人类工程师认为已很新颖的基础算法与数学操作中，可能仍有尚未发现的效率空间。AI 可以筛选数千乃至数百万种算法变体，这是人类无法在合理时间内逐一尝试的。对性能工程师而言，AlphaTensor 的成功表明算法创新并未结束。未来，AI 可能为卷积、排序或注意力等基础操作提供更快算法的新工具箱。

这里的投资回报（ROI）较为间接但影响很大。若将 AlphaTensor 的矩阵乘法算法纳入 GPU 库，兼容的大规模训练或推理负载可能立即获得速度提升，影响范围可从图形渲染、LLM 性能延伸到科学计算。AlphaTensor 的结果表明：在数百 GPU、数千次训练迭代中，约 15% 的速度改善可转化为大量时间与能耗节省。它无需额外硬件，仅凭更智能的软件；但仍需克服算法稳定性、数值误差、实现开销和通用性等问题。

对超大规模性能工程师的启示是：应对软件栈各层的 AI 驱动优化保持开放。即使 GEMM 这类最基础、已充分优化的操作也可能仍有改进空间。让 AI 不受人类偏见限制地探索优化空间，有可能显著缩短整体运行时间。

截至写作时，AlphaTensor 的矩阵乘法算法仍属实验性研究。cuBLAS 等主流 GPU 库尚未纳入这些技术，仍需进一步验证与泛化。

使用 DeepSeek-R1 的自动化 GPU 内核优化（NVIDIA）

底层 GPU 代码优化长期被视为只有“CUDA 高手”（*CUDA Ninjas*）才能掌握的技艺，但已有研究显示 AI 可执行这些专家任务。NVIDIA 工程师以强大的推理模型 DeepSeek-R1 进行实验，考察它能否为复杂注意力机制生成可与高性能手工调优实现竞争的 CUDA 内核。

作为推理模型，DeepSeek-R1 采用“推理时扩展”（inference-time scaling）策略：它不在生成响应前只快速经过模型一次，而是在一段时间内不断完善输出，通常可用时间越多，探索越充分。DeepSeek-R1 一类推理模型经过微调以进行更长思考并迭代答案，类似人类在回答前花时间推敲。

在该实验中，NVIDIA 在一块 H100 上部署 R1，并给它 15 分钟生成优化注意力内核代码。工程师在生成器循环中插入*验证器*（verifier）程序：R1 每提出一个内核，验证器就检查生成内核的正确性并测量效率。生成 → 验证 → 反馈 → 迭代循环类似下面的伪代码：

```
for iteration in range(max_iters):
    code = R1_model.generate_code(prompt)
    valid, runtime = verifier.verify(code)
    if valid and runtime < target_time:
        break  # Accept this kernel
    prompt = refine_prompt(prompt, verifier.feedback)
    ...
```

该反馈循环为下一次内核代码迭代提供改进提示词的指导，循环持续到代码满足给定标准，如图 20-3所示。

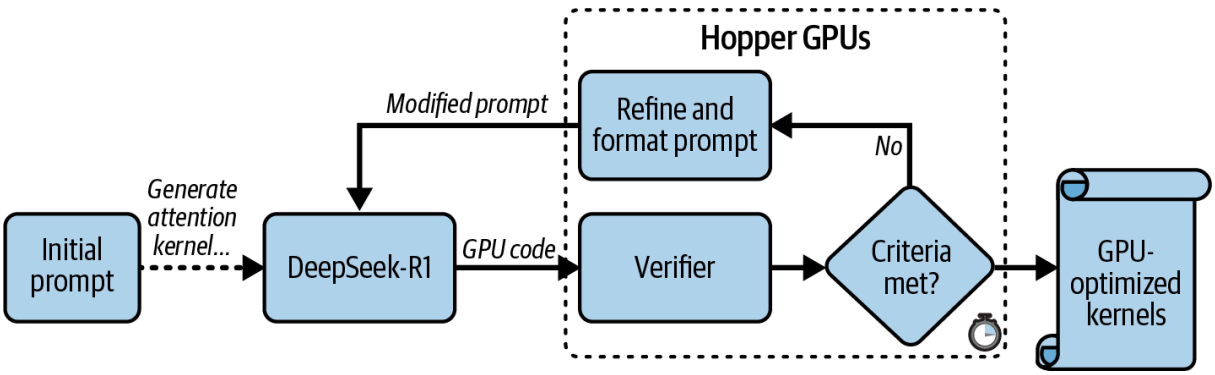


图 20-3. NVIDIA Hopper 平台上使用 DeepSeek-R1 的推理时扩展（来源：Automating GPU Kernel Generation with DeepSeek-R1 and Inference Time Scaling | NVIDIA Technical Blog）

实验使用了如下提示词：

Please write a GPU attention kernel to support relative position encodings. Implement the relative positional encoding on the fly within the kernel. The complete code should be returned, including the necessary modifications.

Use the following function to compute the relative positional encoding:

```
def relative_positional(score, b, h, q_idx, kv_idx):
    return score + (q_idx - kv_idx)
```

*When implementing the kernel, keep in mind that a constant scaling factor 1.44269504 should be applied to the relative positional encoding due to qk_scale = sm_scale * 1.44269504. The PyTorch reference does not need to scale the relative positional encoding, but in the GPU kernel, use:*

```
qk = qk * qk_scale + rel_pos * 1.44269504
```

Please provide the complete updated kernel code that incorporates these changes, ensuring that the relative positional encoding is applied efficiently within the kernel operations.

借助该提示词，AI 为注意力生成了功能正确的 CUDA 内核。（注意 `1.44269504 = 1/ln(2)`。形成 `qk` 时，提示词使用该值相应缩放相对位置项。）除正确性外，生成内核在 NVIDIA 所报告的评测中，相对内置 PyTorch FlexAttention API 获得了 1.1–2.1 倍加速。图 20-4展示了生成内核与 PyTorch 优化 FlexAttention 在各种注意力模式（包括因果掩码和长文档掩码）下的性能比较。结果取决于 GPU、输入形状、框架版本和基准方法。

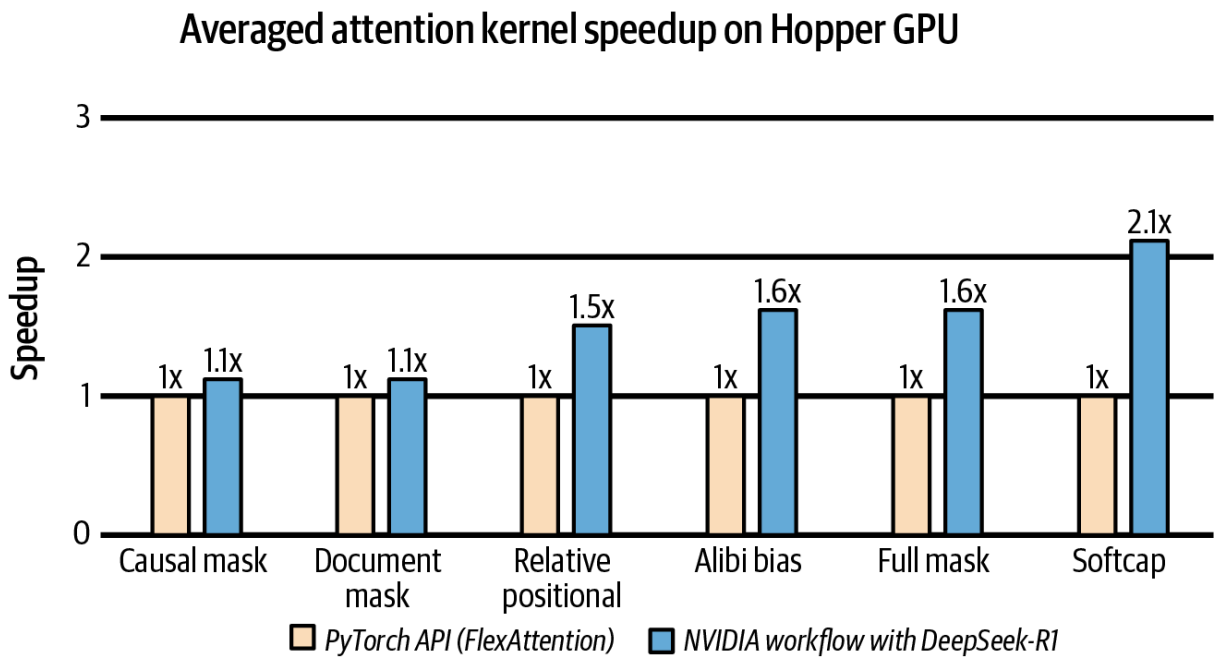


图 20-4. 自动生成的注意力内核相对 PyTorch FlexAttention 获得 1.1–2.1 倍加速（来源：Automating GPU Kernel Generation with DeepSeek-R1 and Inference Time Scaling | NVIDIA Technical Blog）

更令人印象深刻的是，使用斯坦福的 KernelBench 套件（注意力任务），AI 生成内核在基础测试用例（Level-1）上的可验证正确率为 100%，在复杂用例（Level-2）上为 96%。

这表明其在该基准中的正确性较高，但不足以单凭此断言与人类工程师在所有任务和边界条件下具有同等可靠性。

> 实践中，应像 KernelBench 那样，将此类验证器系统与健壮测试套件集成，避免罕见边界条件将错误引入生成代码；还应包含数值容差、内存安全、并发性、确定性和目标负载回归测试。

启示是：为 LLM 提供恰当工具来验证、批评和改进输出，可以提高代码质量。直观上，这一工作流等同于人类工程师反复分析、调试和改进自己的代码。生成 → 验证 → 改进循环让粗略代码草案在约 15 分钟内演进为该实验标准下的高质量注意力实现，展示了 AI 辅助性能调优的强大范式；生产可用性仍需独立评审和长期维护验证。

其 ROI 可能具有颠覆性，因为即使 NVIDIA 顶尖 CUDA 工程师也可能需要数小时或数天手工构建并测试新型注意力内核变体。借助 AI 辅助优化，AI 或许能以更短时间生成效率可比的底层 CUDA 内核，使工程师可聚焦于更高层 AI 系统优化机会，以及 AI 难以发现和修复的边界条件。实际成本还应包括模型推理、验证基础设施、人工审查与安全治理。

尽管仍需人类监督，该实验展示了一条可行路径：在获得显著运行时加速的同时，降低 GPU 优化软件的开发成本。对 AI 系统性能工程师而言，这类 AI 辅助暗示未来工作流可能是与 AI 副驾（copilot）协作，快速协同设计跨硬件、软件和算法的优化。AI 副驾可放大人类生产力；可将其视为经过预训练和微调、能凭借从现有代码库学到的 CUDA 技巧推理复杂问题的 AI 助手。

以强化学习生成优化 GPU 内核的方法（Predibase）

另一家创业公司 Predibase 采用略有不同的强化学习方法，展示了自动化 GPU 编程。它提出一个更大胆的问题：能否用大量 PyTorch 和 OpenAI Triton 代码示例，将 LLM 训练成高级 Triton 程序员?

OpenAI Triton 是一种类 Python 的 GPU 编程语言（及编译器），可简化 GPU 编程。任务是考察 AI 能否生成替代 PyTorch 代码的高效 Triton 代码，并在 NVIDIA GPU 上运行得比 PyTorch 的 TorchInductor 编译器（使用 Triton 生成 GPU 代码）更快。

在其实验中，Predibase 使用 H100 GPU 集群，并在一个规模适中的 320 亿参数 Qwen2.5-Coder-32B-Instruct LLM 上采用名为组相对偏好优化（Group Relative Preference Optimization，GRPO）的 RL 微调过程。Predibase 的 RL 调优模型能为全部 13 项任务生成正确 Triton 内核。值得注意的是，其环境针对正确性而非运行时性能优化。

为此，Predibase 构建奖励函数，以强化学习引导模型持续生成更好的代码。具体而言，LLM 先生成候选内核，系统自动编译并测试其正确性和速度。若图 20-5所示的内核无错误运行、产生正确结果且快于基线内核，模型便获得正奖励。

经过许多轮基于 RL 的试错，模型稳步提升。在数天训练内，AI 从近 0% 成功率提升到仅 5,000 个训练步后约 40% 的时间能生成可工作的内核。一些生成的 Triton 内核比基线快最高约 3 倍；模型还会随训练继续改进。这些数字来自该实验设置，不应直接外推到其他任务或性能基线。

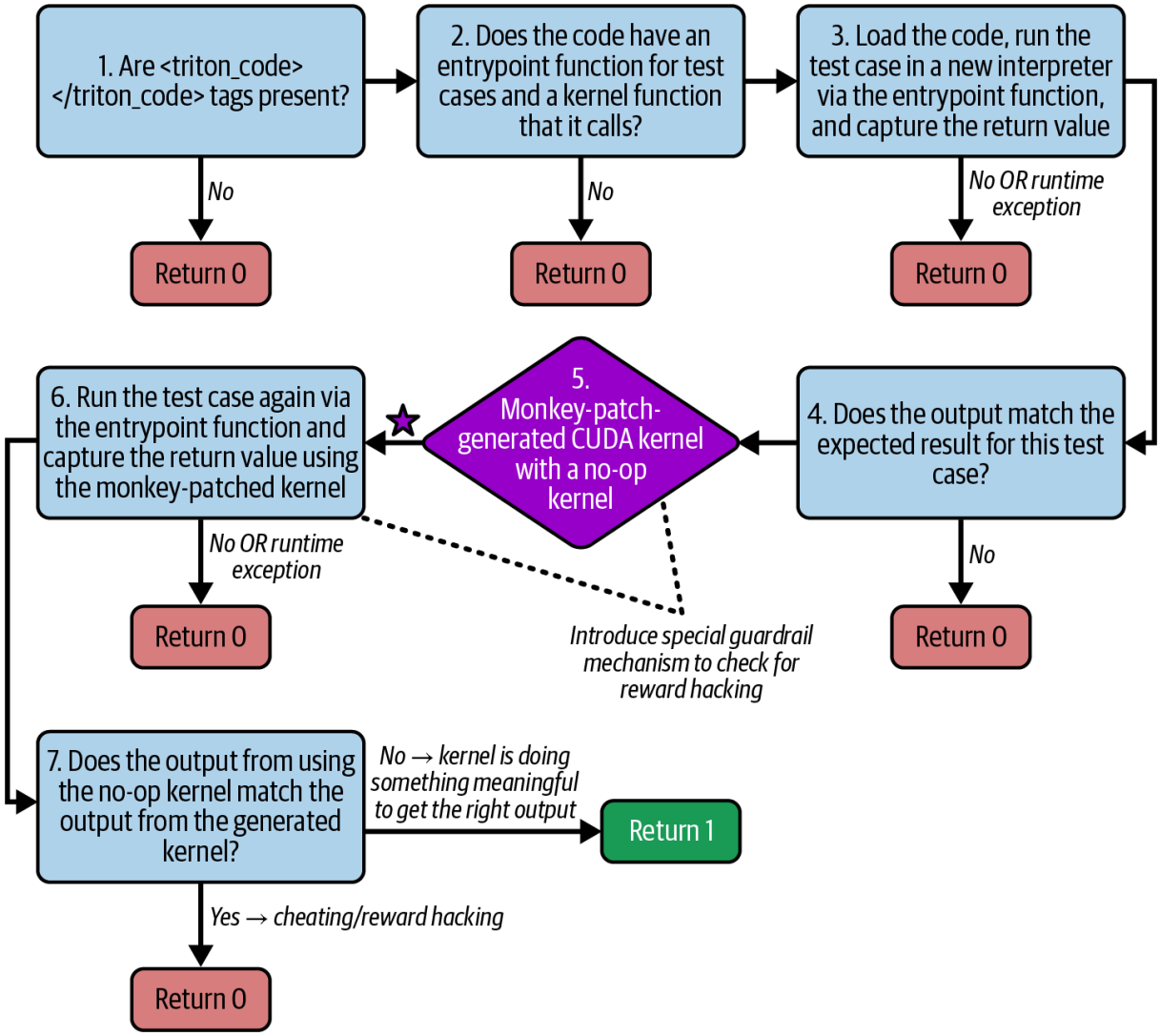


图 20-5. 为生成正确且高性能的 OpenAI Triton 代码（相对基线）分配基于 RL 的奖励（来源：https://oreil.ly/JBxdW）

该结果表明，AI 可通过测试、观察反馈和调整来优化代码，类似工程师迭代改进代码的方式。通过同时奖励正确性与速度，强化学习能使 AI 生成代码对齐真实世界性能指标，并促使 AI 探索 warp 级并行、减少全局内存访问等优化以提升整体性能。

Predibase 演示的启示与 ROI 在于：这种 AI 辅助可自动化内核代码级性能优化，可能减少手工调优需要。工程师无需为新模型手工创建自定义内核，训练好的 AI 助手可生成多个变体并选择最佳者。这可缩短开发周期，使工程师能聚焦探索新模型架构等工作；但要达到前沿性能仍取决于测试覆盖、硬件知识、人工审查和系统集成。

这种方法也暗示未来 Triton、Python 等高级语言和框架可能替代部分手工 CUDA 编程。它们降低 GPU 编程门槛，长期可能形成 AI 智能体持续编写和改进计算内核的自动化流水线，成为性能工程师的重要工具；低层 CUDA 专业知识仍对调试、验证及突破极限性能至关重要。

自我改进 AI 智能体（AI Futures Project）

到目前为止，这些案例研究为我们展示了真实世界超大规模 AI 优化的一个切面。展望未来，AI 系统性能工程师面对令人振奋的挑战与机遇。下一时代的 AI 模型将需要更大、更快的硬件，也需要更智能、更高效地使用硬件的方法。下面转向若干关键未来趋势，仍聚焦于对性能工程师有实用价值的洞见和最佳实践。

2025 年初，AI Futures Project 的一份报告描述了一系列里程碑和 AI 模型/智能体，用于设想未来数年技术进步、研究速度提升及对 AI 研发的变革性收益。报告称前沿 AI 实验室正在设计和建设史上最大的 AI 数据中心之一。这些超级集群若实现，将提供远超既有系统的计算量，并可能推动模型性能的大幅跃升。它是情景预测而非已验证的事实或时间表。

作为背景，训练 GPT-3 估计需要约 3×10^{23} FLOPS，GPT-4 约 2×10^{25} FLOPS。设想中的超大规模 AI 工厂目标是处理约 10^{27}–10^{28} FLOPS 的训练量，约为 GPT-4 所用计算量的 100 倍，如图 20-6所示。GPT-4 训练计算量不是官方公开定值，且这些未来规模均为报告情景中的估算。

研究者设想的 Agent-1 模型将用比前代模型多两个数量级的计算量训练。这可能带来更快训练运行与反馈循环，并形成高吞吐、高效率的平台，显著缩短研究周期、加速机器学习突破；但这是对未来能力的推演。

按 AI Futures Project 的情景设定，Agent-1 被设想为可实时生成和优化代码的自我改进模型。通过自动化从常规调试到复杂内核融合的编码任务，这一前沿 AI 系统可能缩短获得洞见的时间，并拓展全球研究工程师的创造空间。自动化编码可充当生产力放大器，实现快速迭代，让研究者以更少人工开销探索更宏大的想法。

这些巨型 AI 系统被设想为支持持续模型微调与改进。后续模型 Agent-2 可能是一个从不真正结束训练的持续学习 AI：它不再只检查点化并部署静态模型，而是基于新生成的合成数据每天更新权重。这同样是情景描述，持续训练还面临稳定性、成本、安全与评估挑战。

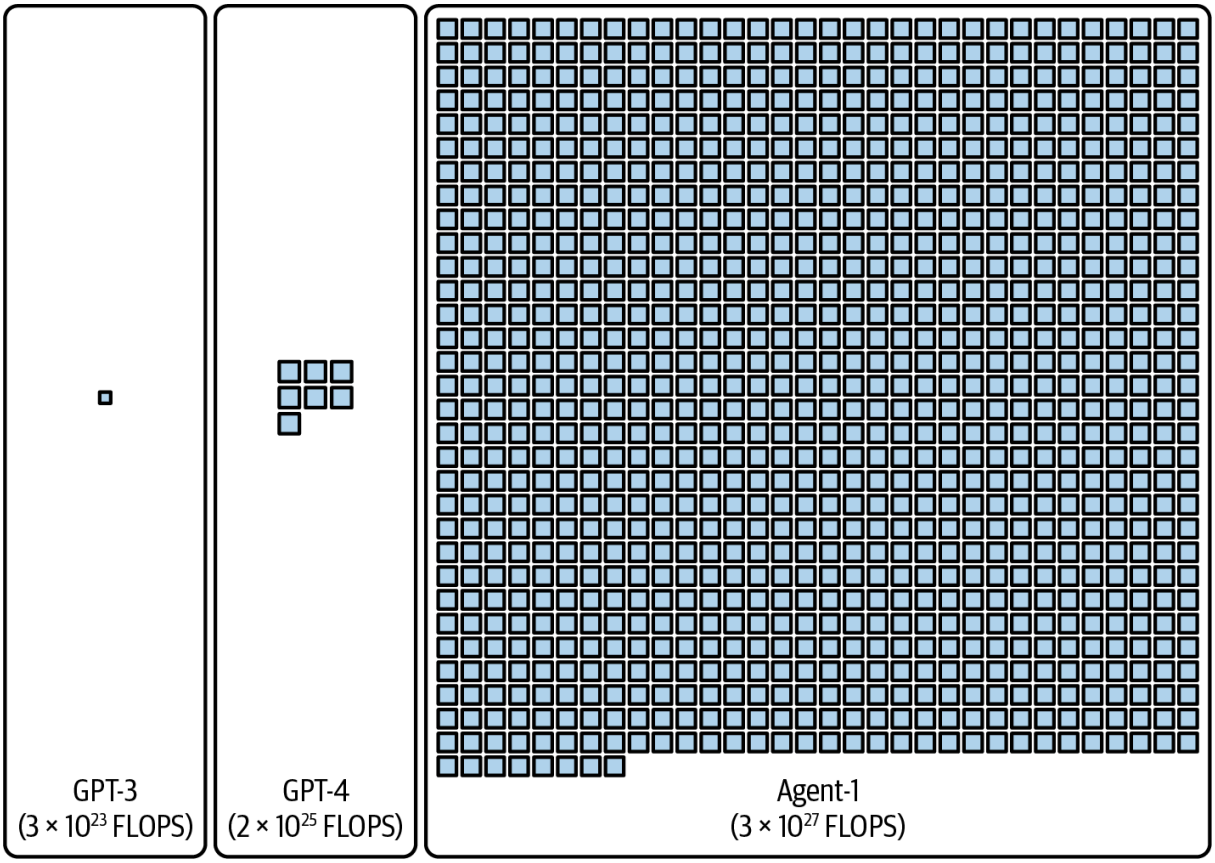


图 20-6. 训练 GPT-3、GPT-4 所需计算量，与 AI Futures Project 研究者设想的“下一代”模型 Agent-1 预期计算量的比较（来源：https://ai-2027.com）

这种永续或持续学习过程旨在通过不断改进性能、适应新信息，让系统保持前沿。若能实现，它将使我们从部署静态训练、静态微调模型的现有范式转变；是否能安全、稳定、经济地实现仍未确定。

> 这种持续再训练（Agent-2 的方法）仍是活跃研究领域，因为需要保持模型稳定性并避免灾难性遗忘（catastrophic forgetting）。灾难性遗忘是指模型在专门化新任务时，执行先前任务的能力下降。

报告将 Agent-3 描述为利用算法突破大幅提升编码效率的 AI 系统。通过整合先进神经草稿板（neural scratchpads）和迭代蒸馏与放大技术，设想中的 Agent-3 会成为快速、成本有效、超越人类能力的编程者。

在 AI Futures Project 提出的假设情景中，Agent-3 可并行运行 200,000 个副本，形成相当于数万名顶尖人类程序员、且速度快 30 倍的虚拟劳动力。这种大规模并行若实现，将加速研究周期，并推动先进 AI 算法与系统的设计和实现普及化。

> 这一预测远超当今实际限制；不过，它是关于未来 AI 生产力潜力的有趣思想实验。

加速研究可让新想法被迅速开发、测试与改进，R&D 的加速可能为 AI 性能大幅提升铺平道路。

报告设想，自我改进 AI 将达到能在研发任务上有效超越人类团队的水平。这些系统可持续运行，处理海量数据流，并以远超人类的速度改进算法；这是一种未来假设，并非对当前能力的描述。

在该设想中，不间断改进循环每天都会提升模型准确性和效率，精简 R&D 流水线、降低运营成本并实现此前难以想象的创新。人类团队将转向监督和高层战略角色，而 AI 处理大量执行工作并以更快节奏推动突破。实际部署仍需人类问责、安全验证与治理。

Agent-4 是一个假设的、自我改写且超越人类能力的研究者。这本质上对应一种通用人工智能（AGI）情景：AI 可重写自身代码来改进自己。Agent-4 在前代基础上发展，其设想中的区别是以最高效率自我改进并优化复杂研究任务。

在 Agent-4 情景中，问题求解被加速。它利用机制可解释性（mechanistic interpretability）澄清自身内部决策过程，以帮助理解底层算法与推理过程的内部工作机制。

实际而言，设想中的 Agent-4 性能使它能解决科学挑战、生成创新研究设计，并推动生成式 AI 模型能力边界；且速度远超人类。这将是 AI 研发的重大突破，形成发现与进步的良性循环。这些能力尚属假设。

AI Futures Project 展示了这些智能体的演进，包括 AI 系统基础设施、自动化编码、持续学习和自我改进模型的进展。情景中的每一代均提升研究生产力与创新能力。这些智能体共同强调：AI 系统性能与效率对迈向 AGI 和超级智能的进展至关重要。

智能编译器与自动化代码优化

我们正进入 AI 性能工具箱中高度智能化编译器与自动化的时代。性能工程师手工调优每个 CUDA 内核、摆弄每个底层旋钮的时代正在过去；高层工具乃至 AI 驱动系统越来越多地承担繁重工作，以榨取最后一点性能。

PyTorch、TensorFlow、JAX 等 AI 框架正快速演进，利用智能编译器和执行图优化器发挥最新 GPU 能力。这些框架可自动融合操作并利用 Tensor Core，也可借助张量内存加速器（Tensor Memory Accelerator，TMA）等现代 GPU 特性重叠计算与异步数据移动。

此外，OpenAI 的 Triton 编译器让开发者使用其基于 Python 的语言编写 GPU 内核。Triton 在底层将这些基于 Python 的内核编译为高效 CUDA 内核，同时对 Triton 用户抽象掉这部分复杂性。

这类工具正日益强大。OpenAI 与 NVIDIA 紧密合作，使 Triton 支持最新 GPU 架构并自动利用其专用特性；实际支持范围应以相应发布版本文档为准。

新一代 GPU 发布后，更新后的 Triton 编译器可暴露其新能力，使研究人员或工程师不必了解底层 C++ 或 PTX 汇编代码。他们编写高层 Python 代码，编译器则为特定 GPU 环境生成优化代码；在特殊形状、数值语义或极限性能场景，底层知识仍很重要。

许多过去需要手写的优化已由编译器自动化，且这一趋势正在加速。自动内核融合、内核启动参数自动调优，乃至部分数值精度决策都可交由编译器和 AI 助手完成；结果仍应以正确性和性能测试验证。

除内核生成外，现代框架对执行图和调度也愈发智能。图执行有助于降低 CPU–GPU 同步开销，并为跨整个图的全局优化打开空间。NVIDIA CUDA Graph 等技术可将一系列 GPU 操作及其依赖捕获为静态图，再使用 `cudaGraphInstantiate()` 和 `cudaGraphLaunch()` API 以极低 CPU 开销实例化和启动，如图 20-7所示。

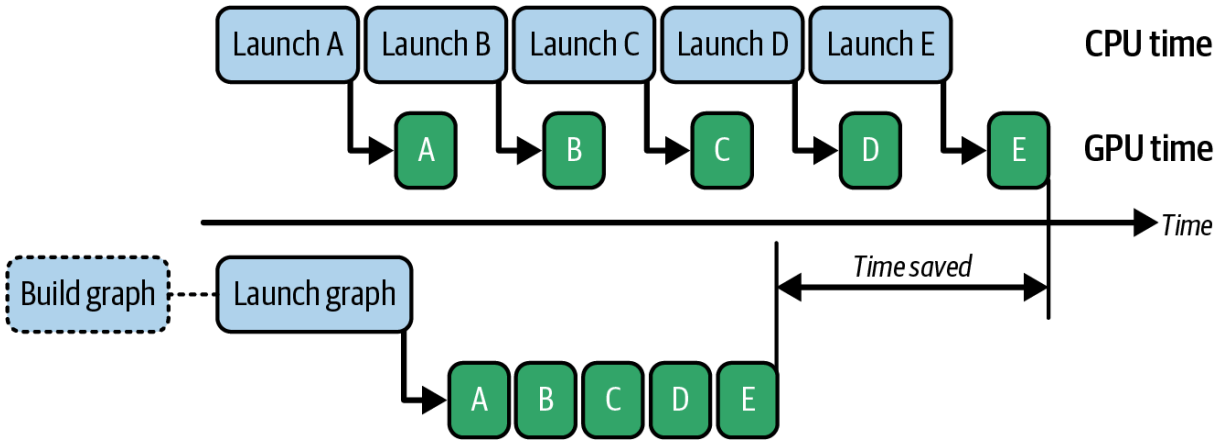


图 20-7. CUDA 中的图执行降低顺序启动多个内核的开销（来源：https://oreil.ly/kxSDm）

我们已经看到 AI 框架自动将训练循环和其他重复模式捕获为图以降低开销。即使执行图是动态而非静态，框架也可能先追踪一次，再反复运行该追踪结果；可捕获范围与重编译行为取决于图动态性和框架版本。

此外，通信与计算的重叠将越来越多地自动化。过去这需要人工安排，但系统可能分析模型后发现，例如 GPU 1 计算第 10 层时，GPU 2 可并行开始第 11 层计算，相当于在底层执行流水线并行。

> 截至写作时，完全自动的流水线并行仍是活跃研究领域。当前 AI 框架通常仍要求显式实现流水线并行，尚不能在没有用户指导的情况下透明地将顺序层分布到多 GPU。

我们已经了解如何实施 3D、4D、5D 并行（数据、张量、模型/流水线、专家、上下文/序列），以在训练和服务大模型时最大化 GPU 利用率。这类技术兼具艺术与科学，目前仍需要大量人类直觉和经验。它们目前被记录在Hugging Face Ultra-Scale Playbook等专家指南中；期望未来会被纳入编译器、库和框架。

归根结底，AI 框架应当理解这些模式，并安排工作以让分布式系统的各个部分保持忙碌；用户不必亲自分析、调试并优化每一条 GPU 流、每次内存传输和每个网络调用。例如，未来某一天，我们也许会拥有一位 AI 顾问：当你定义一个 5,000 亿参数模型时，它会立即建议：“每个节点采用八路张量并行（tensor parallelism），再在节点之间采用四路流水线并行；此外，为获得最佳效率，请使用这些层分组和块大小。”这仍是对未来工具能力的设想，实际配置必须以具体模型、硬件和测量结果为准。

对性能工程师而言，这可能带来巨大的生产力提升。与其无休止地尝试策略和配置，不如从一开始就向 AI 系统询问接近最优的方案。将人类洞见与编译器/AI 自动化结合，在较少投入下也许能取得过去所需大量试验才能得到的结果。这有点像再次从汇编语言迈向高级

语言：我们把更多责任交给工具。性能工程师的角色将更偏向于引导这些工具，并快速验证它们是否确实做得可靠，而不是缓慢地手工试验和验证每一项设置。

简言之，AI 软件栈正变得更加智能和自主。较好的实践是接纳并审慎评估这些工具。利用了解硬件能力和性能选项的高级编译器，例如 OpenAI 的 Triton；也应持续关注新兴的 AI 驱动优化服务。它们起初可能像黑盒，却凝结了大量来之不易的性能经验，仍需以可复现的基准测试验证其收益。

AI 辅助的实时系统优化与集群运维

自动化的推进不仅发生在代码层面，也扩展到系统和集群运维层面。未来，AI 系统可能日益自行管理和优化，特别是在任一时刻都有大量并发作业和请求在运行、因而需要复杂资源共享策略的大规模训练与推理集群中。

一个可能很快发展的方向是由 AI 驱动的自主调度和集群管理。今天的集群编排器（如 Kubernetes、SLURM）仍主要依赖静态启发式规则和简单的资源请求，但更具自适应性的调度机制正在兴起。设想一个智能代理观察整个集群的状态，并学习如何调度推理请求和训练作业以最大化总体吞吐量。

这个调度代理可能学会：某些请求或作业可以同置于一个节点而互不干扰，例如一个计算密集，另一个内存带宽密集。通过摄取 Kubernetes 集群的遥测数据（Pod 的 GPU 利用率、队列等待时间等），AI 调度器可以动态重新调度作业或调整 Pod 资源，以最大化总体吞吐量并尽量减少空闲时间。

从某种意义上说，集群开始像自动驾驶汽车一样，根据实时状况不断调整其“驾驶策略”（资源分配），而不是遵循固定路线。对性能工程师的好处是更高的资源利用率和更少的瓶颈。我们的工作将转向为 AI 调度器设定高层策略和目标，再让它确定具体做法。

例如，NVIDIA Dynamo 分布式推理框架可协调跨 GPU 和节点的请求调度、KV 缓存放置和数据移动；其与 Kubernetes 的集成和解耦式推理能力应以所使用版本的官方文档为准。在适用配置中，Dynamo 的调度器可将微批次分配到不同流水线阶段，并通过重新路由请求处理节点故障。

借助权重流式传输（weight streaming）和激活卸载（activation offloading）等技术，模型层可按需从主机内存流送到 GPU，仅在需要权重时（如解码阶段）加载；这一过程也可扩展到多个节点和 GPU。理论上，这能将一个 100 万亿参数模型的部分内容托管在成本较低的存储上，从而有助于扩展推理；实际效果取决于存储、网络、缓存命中率及延迟预算。

我们也可能看到面向系统运维人员的 AI 性能副驾（performance copilot）。大语言模型可作为支持角色成为基础设施的一部分。例如，性能工程师可以向 AI 助手询问“如何加速我的训练作业？”，并得到有依据的建议。听起来颇具想象力，但若这类助手能够从数千次历史运行、日志和调优记录中学习累积知识，便有实现的可能。

AI 性能副驾还可能识别出 GPU 内存利用率偏低并建议增大批大小，或发现梯度噪声尺度较高并建议调整学习率调度。这类代理可封装人类专家积累的部分经验，使这些知识能够随时获得；不过建议应先在隔离环境中验证。

同样，AI 助手可以监控训练作业和推理服务器，并标记异常。例如，如图 20-8所示，助手可在监控训练作业时提示：“训练早期的损失正在发散；请检查数据输入是否有问题，或降

低学习率。”

Splunk（现属 Cisco）和 PagerDuty 等公司已将 AI 模型用于系统日志数据，以预测故障并检测数据中心异常。建议将这些理念扩展到面向 AI 工作负载的专用遥测数据。

简言之，AI 为每个运行中的作业和每台推理服务器提供一双始终“新鲜”的眼睛：它能监控、提出建议并实时调整。传统利用率指标可能具有误导性；例如，GPU 因冗余数据传输而 100% 忙碌并不等于有效产出。AI 驱动调度器的目标则是最大化有效吞吐量（goodput），确保 GPU 忙于有用的神经网络计算，这会直接改善成本效率。

例如，在 AI 集群中，可以使用基于 Prometheus 的指标管道为基于 LLM 的助手提供数据：当 GPU 内存因潜在内存泄漏或数据停滞而突然下降时，它发出警报，甚至识别可能的根本原因。这类繁琐工作适合由 AI 辅助自动化，并可全天候持续运行；告警和归因仍应接受人工复核。

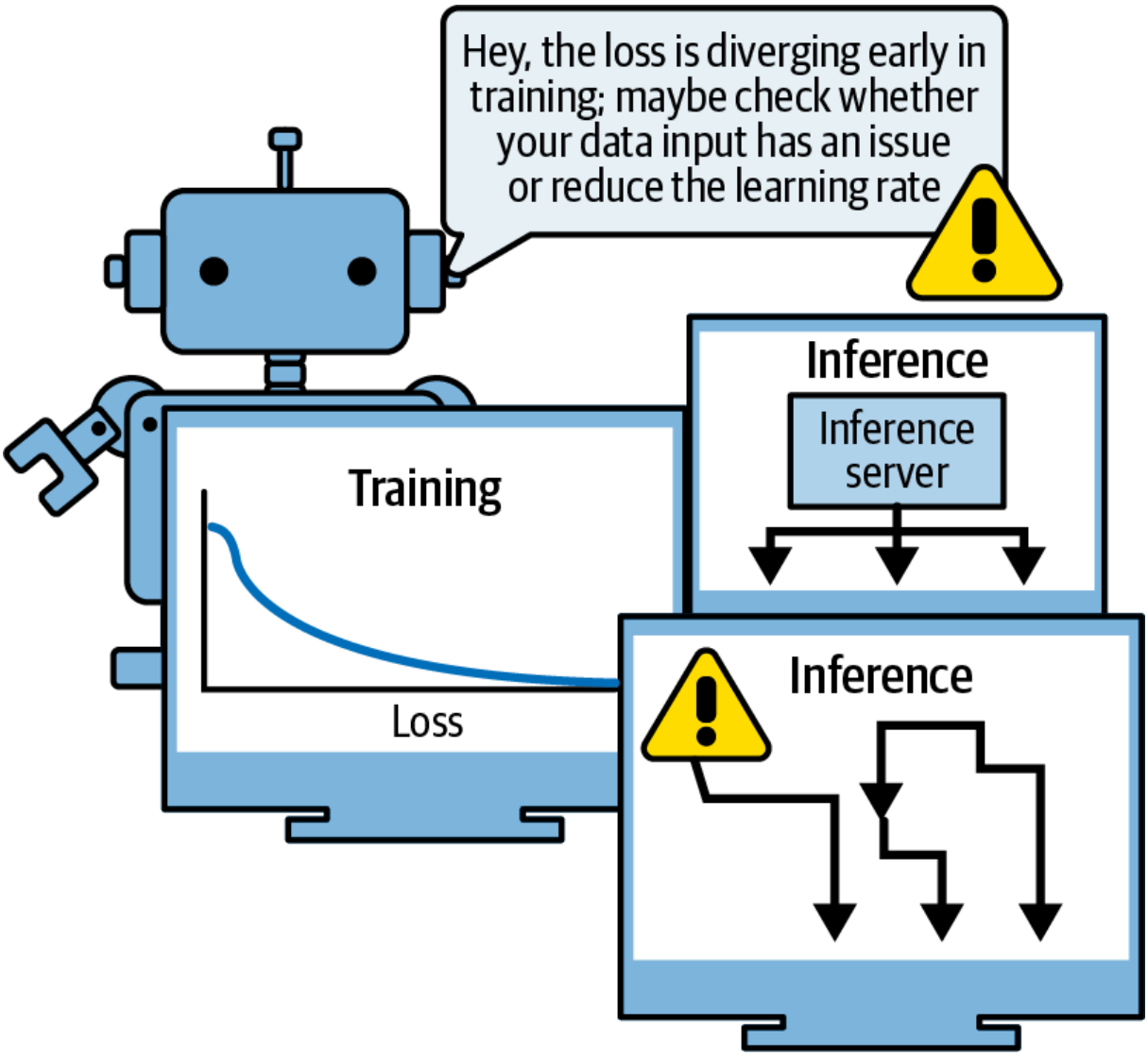


图 20-8. AI 助手监控长时间运行的训练作业，并建议修复异常的措施

AI 的另一项强大用途是 AI 系统的自动调试和故障分析。当训练作业在其为期三个月的运行中途失败时，人类必须通读错误日志、设备统计信息，甚至可能还要查看内存转储，才能找出问题所在：是硬件故障、数值溢出，还是网络短暂异常？

未来，AI 系统可消化包括日志、指标和告警在内的所有数据，并可能比今天更快地定位可能原因。它可能指出：“作业崩溃前，节点 42 出现了 5 次 ECC 内存错误，GPU 上的 HBM 内存器件或通道可能存在问题。”或者：“第 10,000 次迭代时损失变为 NaN，可能是梯度不稳定；可考虑梯度裁剪。”这类归因是候选假设，不应替代硬件诊断和复现测试。

通过从众多历史事件中学习，AI 排障工具可能为工程师节省大量排查时间。一些大型计算设施已在其事件数据库上训练模型，以预测故障并建议修复方法。

更进一步，强化学习（reinforcement learning，RL）可用于实时控制系统行为，其适应能力未必是固定算法能够轻易匹敌的。例如，可训练一个电源管理 RL 代理，在运行中的系统上持续调整频率和核心分配，以最大化每瓦性能（performance per watt）。该代理会通过实时分析系统来学习策略；投入生产前必须设置功耗、温度、可靠性和变更权限等硬性边界。

另一个例子是在 AI 模型中主动管理内存。AI 代理可以学习哪些张量应保留在 GPU 内存，哪些应换出到 CPU 或 NVMe，而不局限于“换出最久未使用项”这类静态规则。通过观察实时访问模式，AI 可更高效地管理缓存；当模式不明显或依赖工作负载时尤其可能有效。

已有前沿实践者使用 RL 优化缓存驱逐、网络拥塞控制等问题。超大规模系统拥有数百个相互作用的组件和资源，复杂性使其成为此类学习型控制的潜在适用对象。可调参数过多，人类难以及时找到最佳设置，更难使设置实时适应不同工作负载。

对性能工程师而言，AI 辅助运维代理的兴起意味着角色将更侧重于编排和监督 AI 驱动的流程，而非手工微调每一个参数。这有点像飞行员管理现代飞机的自动驾驶系统：仍需深厚知识和持续监督，但大量毫秒级控制已经自动化。Tesla 的完全自动驾驶（Full Self-Driving，FSD）模式也类似；驾驶者仍需要知识和直觉来避免困难情境、预防事故，而车辆控制由 FSD 软件辅助自动化。

为了引导 AI 助手高效管理集群，我们需要设定目标、提供安全与公平性护栏，并处理 AI 尚未见过的新情形。负载均衡、故障恢复和内存缓冲区调优等例行优化可交由 AI 协助处理。实际运维还应采用最小权限、人工审批、金丝雀发布和可验证回滚，并明确 SLO 与公平性约束；在此前提下，接纳这一范式将十分重要。

> 在如此复杂的 AI 系统中，坚持完全手工优化的人，可能会落后于善用 AI 辅助和自动调优的人。愿意采用 AI 自动化的团队可以把人力投入到新颖创新、复杂优化和创造性解决方案中——这正是人类在这个新兴 AI 世界最能创造价值的地方。让 AI 协助处理其余的例行事务，但应保留人工监督、审计和最终决策权。

迈向数百万 GPU 集群和100 万亿参数模型的扩展

最后，让我们重新审视迈向超大规模100 万亿参数模型的探索。我们已经跨过万亿参数门槛；接下来的问题是未来几年如何扩展到数十万亿乃至数百亿亿参数模型。这类模型会对系统提出什么要求，又需要哪些创新才能让训练成为可能？这里汇集了我们讨论过的所有要素：高效硬件、智能软件和巧妙算法。达到 100 万亿参数规模需要用尽已有技巧，甚至还需要尚未发现的技巧。以下是面向未来的推演，并非已实现能力。

在硬件方面，显而易见的需求是更多内存和更高带宽，最好直接位于 GPU 上。若要训练 100 万亿个参数，就需要高效存储和移动极其庞大的数据量。下一代内存技术将至关重

要。

高带宽内存（high-bandwidth memory，HBM）仍在持续演进。HBM3e 已用于 Blackwell 一代 GPU；Rubin 一代及 HBM4 的具体产品配置、供货时间和指标，应以 NVIDIA 与内存供应商届时公布的官方资料为准。书中提到 HBM4 每个堆叠的带宽可能再次翻倍至约 1.6 TB/s，每个模块的容量可能提升到 48 GB 或 64 GB；这些均应视为规格预期而非通用既成事实。

HBM 更高的容量和吞吐量意味着未来 GPU 或许会配备 8 或 16 个、每个 64 GB 的 HBM 堆叠，即单板总计 512 GB 或 1,024 GB 的超高速 HBM RAM。这样的本地 HBM 容量可在每张 GPU 上直接容纳大量模型参数，显著减少数据换入换出的需求；这只是设计空间示例，实际产品不一定采用该配置。

不难看出，这可能支持更大模型、更高带宽的训练运行和更低延迟的推理服务器。过去需要切分到 8 张 GPU 的内容或许可以放入一张；过去需要 100 张 GPU 的内容或许可以放入 10 张，依此类推。实际可缩减的倍数取决于精度、优化器状态、激活、批大小和模型结构。

除 Grace Blackwell Superchip 等多芯片架构外，多个 NVL72 机架可连接为一个大型集群，让数百个 GPU 共享统一的高速网络。从通信角度看，集群在一定程度上会表现得像一颗超级 GPU。这对扩展到 100 万亿参数很重要，因为它意味着我们可以继续增加 GPU 来获得更多总内存和算力，而不立即撞上通信瓶颈；这一推论假定 NVLink 或类似互连能继续扩展到该超大规模。

然而，仅有硬件无法解决 100 万亿参数挑战。软件和算法创新同样重要，甚至可能更重要。例如，用朴素数据并行训练这种规模的模型会极其缓慢且昂贵——设想优化器在每一步都要更新 100 万亿个权重。我们需要大量采用降低有效计算量的技术。一个重要方向是低数值精度：除 FP8 和 FP4 外，未来硬件或许会为网络某些部分支持更低的 1 位精度。混合方案很可能至关重要：模型的大部分使用低精度，敏感部分使用更高精度；可训练性和最终质量必须逐模型验证。

作为性能工程师，我们应关注这些新能力并准备加以利用。若要训练 100 万亿参数模型，为了效率，极可能需要低精度；否则工作负载可能慢到、贵到难以承受。

好消息是，硬件和库有望让这一转变相对顺畅。我们已在 CUDA 中看到 NVIDIA Transformer Engine（TE）和 Tensor Cores 对低精度算术的一流支持；PyTorch 和 OpenAI 的 Triton 也可充分利用 CUDA。可用的数据类型和性能仍随硬件、CUDA、驱动和框架版本而变化。

另一个关键方法是稀疏性和条件计算。我们已在稀疏专家混合（mixture of experts，MoE）等模型中采用稀疏激活：对于给定输入，只有模型参数的一小部分处于活跃状态。这一想法可以推广，使每次不必都使用完整的 100 万亿参数，而只使用所需部分。采用 MoE 架构的模型已显示出较强能力和效率；若 100 万亿参数模型出现，其中许多可能需要稀疏激活，这仍是一项预测。

对性能工程师而言，这意味着吞吐量不仅取决于矩阵乘法速度，也取决于 MoE 条件路由效率、专家输出缓存和稀疏数据交换的通信模式。这增加了复杂性，也带来机会：若能确保合适的专家在合适的时间位于合适的设备上，从而最小化通信，便可能显著加速这些大模型。

我们还应考虑算法效率改进。占用更少内存的优化器可能至关重要。传统 Adam 优化器变体通常为动量和方差估计各保留一份额外权重副本，这实际上使内存占用约增至三倍。因此，如果有 100 万亿参数权重，还需要额外 200 万亿个数值来保存优化器状态。Adafactor、Shampoo 等内存高效优化器有助于降低此开销，但其收敛行为和实现成本应在具体任务上评估。

激活检查点（activation checkpointing）等技术通过重新计算激活值而非存储它们，以计算换取内存。在 100 万亿参数规模下，几乎肯定需要积极采用检查点。一个更激进的想法是：也许不在每一步更新所有权重，而是轮换更新权重子集——像是不每天浇灌所有植物，而是轮流浇灌。若设计得当，模型仍能有效学习，同时每个参数的更新频率更低，从而降低系统总计算需求；这属于研究性设想，需要严格验证收敛和质量。

这些想法与算法设计领域交叉，但性能感知视角仍然有用。对于训练和推理的每个环节，我们都应追问：“真的需要如此频繁地、以如此精度执行*X*吗？”答案常常是可以找到仍然有效、却更便宜的近似方法。在 100 万亿参数规模下，这些近似可能节省数月时间或数百万美元，但前提是质量、可靠性和安全性目标不被牺牲。

超大规模训练中常被忽视的一个方面是基础设施和网络。当讨论让超过 10,000 个 GPU 协作训练一个模型的集群时，网络结构变得与 GPU 本身同样重要。以太网和 InfiniBand 技术在吞吐量和更智能的自适应路由等方面不断演进。NVIDIA Spectrum-X 是面向 AI 优化的以太网结构（例如包含 RoCE、自适应路由和高双向带宽），旨在降低大规模训练和推理工作负载中的拥塞；具体能力需参考部署版本及其配置。

性能工程师需要深入理解这些层级，并确保数据在正确时间位于正确位置。目标是模拟横跨 GPU 与 CPU 的巨大内存空间，使模型即便放不进一台机器，程序员也可在一定程度上透明地处理它。今天已有一部分能力可通过 Unified Memory 和按需分页系统实现，例如使用 `cudaMemPrefetchAsync()` 将页面预先放到目标设备上，以避免缺页停顿。但在 100 万亿参数规模下，这些功能将经受严峻考验。

前沿研究实验室和云服务商都在持续建设大规模算力集群；关于 xAI、OpenAI 或 Microsoft 正在建设“超过 1,000,000 张 GPU”集群的说法，不应在未获相应官方、可核实资料支持时视为已确认事实。在 100 万亿参数规模下，一个作业可能跨越整个数据中心的硬件，因此性能工程师必须从数据中心及多数据中心（全球）尺度思考问题。

最后，随着模型及其所需算力扩大，还会出现社会技术层面的趋势：任何单个团队，甚至单个公司，可能都难以独自训练最大的模型。我们希望 AI 社区会出现更多协作和资源共享来处理这些巨大项目，这类似于粒子物理实验等大型科学项目由众多机构参与。类似现已解散的非营利倡议 Open Collective Foundation 的项目，或可汇集 AI 计算资源以训练 100 万亿参数模型，再将其与世界共享；这只是组织模式的设想。

这将要求标准化检查点格式等内容、共同开发训练代码，并思考模型的多方所有权。虽然这本身并非性能问题，却会影响我们构建大型 AI 系统的方式。我们需要使系统更具容错性、更容易创建快照，以便共享部分结果。作为工程师，除了纯速度，还可能需要为可复现性和互操作性而优化，从而让不同团队顺畅高效地参与训练和推理工作流的不同环节。

达到 100 万亿参数模型将需要整体性的全栈创新。这个挑战没有单一解法，而是拼图的每一块都必须改进：硬件需要更快、能容纳更多数据；软件需要更强的自优化能力，并通过编译器、AI 助手和实时自适应更高效地使用资源；算法则需通过稀疏性、低精度和更好的优化器，巧妙地避免不必要的工作。

性能工程师的职责是将这些进展整合为连贯的工作流。这就像组装一辆高性能赛车：发动机、轮胎、空气动力学和驾驶技术都必须协同工作。如果做得好，眼下看似不可能的事情——例如以可承受成本训练 100 万亿参数——未来或许会成为现实。

不久之前，万亿参数模型听起来还很疯狂。如今，Moonshot AI 的开放权重模型 Kimi K2 等已展示了这一规模；其公开资料通常描述为 1 万亿参数 MoE、每个 token 激活 320 亿参数，具体数值及版本应以模型发布方资料为准。按照这样的进展速度，并借助 AI 辅助的人类创造力，我们或能在较短时间内跨越下一个数量级的里程碑。

要点回顾

以下各点概括本章案例研究和超大规模 AI 系统性能工程假设性未来状态中讨论的最佳实践与新兴趋势：

软硬件协同设计优化

大语言模型的性能提升，确实来自软硬件紧密集成的协同设计创新所带来的突破。

AI 辅助编程与性能优化

Google DeepMind、NVIDIA 和 Predibase 已展示 AI 辅助发现和优化矩阵乘法、注意力等核心内核。这些工作表明，AI 可以生成、测试并改进底层 GPU 代码，并在特定基准中以很少人工干预取得显著加速；具体收益仍取决于任务和测试条件。

100 万亿参数模型的策略

训练 100 万亿参数模型将需要结合激进量化、多维并行（数据、流水线、张量、专家及上下文/序列）和精细的跨机架通信编排。这强调未来 AI 扩展既依赖硬件能力，也依赖软件层调度的创造力；这是面向未来的技术判断。

指数级算力基础设施扩展

下一代 AI 数据中心正被设计为提供数量级增长的计算能力。这类设施可能以远超当前水平的计算预算训练 AI 模型，并支持使用当前系统 FLOPS 的 100 到 1,000 倍的训练运行；实际落地规模取决于建设、供电、网络与经济条件。

演进中的 AI 模型与代理

未来模型可能成为自我改进系统，能够生成和优化代码、利用新数据持续更新权重，甚至重写自身代码。这种持续学习和改进的循环有望缩短突破间隔，并形成在某些研究与研发任务上超越人类团队的虚拟劳动力；这属于前瞻性设想，需兼顾安全、可控性和评估。

AI 辅助实时故障排查

除调度外，AI 副驾还可监控系统日志和训练/推理工作负载，快速检测异常，包括精度损失突增或硬件错误数量上升。它们可辅助自动调试、进行故障分析，甚至通过强化学习学习较优配置，从而改善每瓦性能和单位时间性能；生产使用须有监督与验证。

每瓦性能：关键指标

所有这些协同设计工作最终都旨在最大化单位成本的吞吐量。具体来说，目标是每美元、每瓦电力每秒处理和生成更多 token。例如，Grace Blackwell NVL72 机架系统相对上一代 Hopper 的每瓦性能提升 25 倍是厂商在特定配置和指标下的宣称；实际结果及每 token 成本需按工作负载和部署条件测量。

结语

本书标志着 AI 系统性能工程领域的一个转折点。NVIDIA 将 CPU 和 GPU 紧密集成到 Grace Hopper、Grace Blackwell 等超级芯片模块中，带来了新的计算效率和规模；Vera Rubin 和 Feynman 的实际产品形态、时间表和能力仍应以官方发布为准。在底层，GPU 使用高度优化的 Tensor Cores，以及针对大语言模型基础计算优化的 Transformer Engine。

NVIDIA GB200/GB300 NVL72 等超级计算系统将 72 个 GPU 连接为一个处理单元（NVLink 域），并以 NVLink、NVSwitch 和 SHARP 等技术构成机架和数据中心通信基础。这些技术可为多万亿参数模型提供低延迟推理能力，实际延迟和吞吐量取决于模型、并行策略、网络及服务负载。

在软件方面，vLLM、SGLang、NVIDIA Dynamo 和 TensorRT-LLM 等工具可改善大型推理集群中的调度和资源使用。这包括飞行中批处理（in-flight batching）、分页 KV 缓存（paged KV cache），以及为提高效率将提示词预填充阶段与生成解码阶段分离到不同资源池的技术。这些方法有助于降低尾延迟并提高每瓦吞吐量，实际收益应按部署环境测量。

这些例子说明了协同设计的力量：硬件、软件和算法共同演进。这种以硬件特性理解（mechanical sympathy）为基础的协作，有助于缩短训练时间、改善推理性能并降低运营费用，是在当今快速发展且资本密集的 AI 系统中获得可衡量投资回报的必要条件。

此外，来自 Google DeepMind、NVIDIA 和 Predibase 的 AI 驱动编程与算法代理展示了 AI 如何帮助优化 AI。随着模型和系统变得过于复杂而难以手工调优，自动化可以处理例行优化，使人类工程师能专注于更高层的优化和系统设计。

我们正从蛮力扩展转向智能扩展：每个周期完成更多有用工作，充分挖掘新硬件特性，并让 AI 助手管理细节。性能工程师将向软件栈上层发展，成为在效率、可靠性和可持续性之间取得平衡的全球算力生态系统架构师。

作为 AI 系统性能工程师，我们的角色将从单节点内核扩展到系统级和设施级优化。我们将依靠直觉发现瓶颈——例如缓慢的 all-reduce 模式——再引导 AI 工具修复它们。同时必须不断学习，因为硬件和算法创新的速度只会加快。

总之，要保持相关性和竞争力，应夯实 AI 系统性能基础，保持好奇心，试验新的硬件和软件进展，审慎评估 AI 建议，并准备好适应包括量子计算在内不断变化的技术版图。设想一下：在研究更加普惠、可一键访问 AI 超级计算机、多万亿参数模型日益可及的时代，你可能成为下一次重大超级智能突破的推动者之一；这仍是鼓舞性的未来愿景，而非确定预测。

附录　AI 系统性能检查清单 （175+ 项）

这份详尽清单既涵盖面向流程的广泛最佳实践，也提供 AI 系统性能工程师所需的细致底层调优建议。每一项都是帮助你从 AI 系统中榨取最大性能和效率的实用提醒。

在调试、性能分析、诊断和调优 AI 系统时可使用本指南。系统性地应用这些建议——从底层 OS 和 CUDA 调整到集群规模优化——可使 AI 系统性能工程师借助 CUDA、PyTorch、OpenAI Triton、TensorFlow、Keras 和 JAX 等多种软件框架，在现代 NVIDIA GPU 硬件上同时获得极快执行速度与成本效益。这些原则也适用于未来几代 NVIDIA 硬件，包括 GPU、基于 ARM 的 CPU、CPU-GPU 超级芯片、网络设备和机架系统。

性能调优与成本优化思维

务实且有记录的闭环——先取得快速收益，再进行深入工作——可将工程时间转化为可衡量的投资回报率。先针对最大的运行时间和成本驱动因素，且始终在优化前后进行性能分析以验证影响。

结合自动调优、框架升级、云定价策略和利用率仪表板，获得高 ROI 收益；记录结果并优先选择简单、易维护的修复方案。在准确率允许时调节对吞吐量敏感的超参数。以下是性能调优与成本优化的思维建议：

优先优化最昂贵的部分

> 遵循 80/20 原则。找出运行时间的主要贡献者并集中处理。若 90% 的时间花在少数几个内核或某个通信阶段，深入优化它们远比微调只占 1% 时间的部分更有价值。各章技术应应用在最关键的地方。例如，若训练中数据加载占 40%、GPU 计算占 50%、通信占 10%，应先解决数据加载，因为可能将该开销减半；随后再研究 GPU 内核优化。

优化前后都进行性能分析

> 每次应用优化时都要测量影响。这听起来显而易见，但实践中许多调整基于理论，可能没有帮助，甚至造成损害。假设负载并不受内存限制，却决定为训练启用激活检查点：它可能通过增加计算来降低内存，反而使作业变慢。换言之，始终比较变更前后的吞吐量、延迟和利用率等关键指标。可使用内置分析器进行简单计时，例如统计 100 次迭代的平均时间。

采用自适应自动调优反馈闭环

> 实现利用实时性能反馈的高级自动调优框架，例如采用强化学习或贝叶斯优化动态调整系统参数。这使系统能响应不断变化的工作负载和运行条件，持续微调设置。

为优化时间编制预算

> 性能工程是一项迭代性投资，收益会递减。应先做容易取得的成果，例如启用 AMP 和数据预取，它们可能轻松带来 2 倍收益。编写自定义内核等较难优化的增益可能更小。

始终权衡工程时间与节省的运行时间和成本。对于训练旗舰模型等大规模重复作业，即使 5% 的收益也可能节省数百万成本，足以证明数周调优合理；对一次性或小型负载，则应务实地关注更大的收益。

持续了解框架改进

我们讨论的许多优化，如混合精度、融合内核和分布式算法，仍在深度学习框架和库中不断改进。升级到最新版 PyTorch 或 TensorFlow 有时可立即加速，因为其中包含新的融合操作或更好的启发式策略。应利用这些近乎免费的收益，并阅读发行说明中的性能相关变更。

与供应商和社区成员协同设计

与硬件供应商和更广泛的性能工程社区保持联系，使软件优化适配最新硬件架构。通过按新硬件能力定制算法，这种协同设计方法可发现显著性能机会。定期查阅供应商文档、参与论坛并测试驱动或框架的 Beta 版；这些互动常能发现可集成到系统的新优化机会和最佳实践。整合新的驱动优化、库更新和硬件专用建议，可带来额外且有时显著的性能提升。

利用云的灵活性控制成本

在云环境中运行时，应明智使用更便宜的竞价实例或预留实例。它们可大幅降低成本，但竞价实例可能在数分钟前通知后被收回。也应考虑实例类型：若工作负载不需要绝对最新硬件，成本仅为一小部分的稍旧 GPU 实例有时可提供更好的性价比。我们对 H800 与 H100 的讨论表明，投入足够工程工作，也能在次优硬件上完成出色工作；云中也存在类似权衡。应在不同实例配置上进行基准测试，评估 CPU 数量、CPU 内存、GPU 数量、GPU 内存、L1/L2 缓存、统一内存、NVLink/NVSwitch 互连、网络带宽与延迟、以及本地磁盘配置等因素的成本/性能。计算每美元吞吐量等指标，以指导优化决策。

监控利用率指标

持续监控 GPU 利用率、SM 效率、内存带宽使用率，以及多节点场景下的网络利用率。使用 DCGM exporter、Prometheus 等建立仪表板，以便发现任一资源利用不足。若 GPU 利用率仅为 50%，应深入查找原因：很可能是等待数据/停顿或同步通信缓慢。若网络仅使用 10%，GPU 却在等数据，问题可能是锁等其他因素。这些指标有助于找准应关注的子系统。

迭代调节超参数以提升吞吐量

可调节某些模型超参数（如 batch 大小、序列长度和 MoE 活跃专家数）以提升吞吐量，而不降低最终准确率。例如，较大 batch 可带来更好吞吐量，但可能需要调节学习率计划来维持准确率。不要害怕调整这些参数，以找到速度与准确率的最佳平衡。这也是性能工程的一部分：有时可调整模型或训练过程以提高效率，例如使用激活检查点，或为相同有效 batch 增加计算步骤；可相应调整训练学习率计划。

记录并复用

记录已应用的优化及其影响，并写入代码或内部 wiki 式共享知识库。这会为未来项目建立知识基础。许多建议都是可复用模式，例如启用重叠或设置有助于集群运行的特定环境变量。积累这些历史，在启动新工作或让新成员加入性能调优时可节省时间。

平衡优化与复杂性

目标应是以最简单方案达到所需性能。例如，若原生 PyTorch 配合 `torch.compile` 已满足速度目标，可能无需编写自定义 CUDA 内核，从而避免额外维护。使用高度定制代码过度优化会让系统变得脆弱。既快速又易维护的方案具有优雅性。因此，应先采用带来所需收益的侵入性最小优化，仅在必要时升级为更复杂的方法。

优化 AI 驱动的性能

利用机器学习模型分析历史遥测数据并预测系统瓶颈，使系统能实时自动调整参数，优化资源分配和吞吐量。

可复现性与文档最佳实践

性能收益若不能复现、版本化并持续检查，便无法持久，且会随时间悄然回退。应将文档、CI 基准测试和共享知识视为保留加速成果、加快成员入职和审计的粘合剂。

在源代码管理中锁定版本、配置和基准，以便重复实验并追踪回退。将性能检查引入 CI/CD，实施端到端监控与告警，并使优化配合安全措施和详尽文档，从而建立持久、可审计的实践。以下是提高可复现性和文档质量的建议：

严格的版本控制

对所有系统配置、框架/驱动版本、OS 设置、优化脚本和基准测试实施全面版本控制。使用 Git（或类似系统）追踪变更并标记发布版本。这样可精确复现实验，也能轻松识别性能回退。

用于性能回退的持续集成

将自动化性能基准和实时监控集成到 CI/CD 流水线。这可确保从代码更新到配置变更的每项变化，均以一组性能指标验证，帮助尽早发现回退并保持一致、可量化的性能提升。采用 MLPerf 等行业标准基准，建立可靠性能基线并长期跟踪改进。

端到端工作流优化

确保优化整体应用于整个 AI 流水线——从数据摄取和预处理，到训练和推理部署。协调的跨系统调优能发现孤立调整可能错过的协同效应，从而实现更显著的总体性能提升。

自动化监控与诊断

部署端到端监控方案，收集跨硬件、网络和应用层的实时指标。将其集成到 Prometheus/Grafana 等仪表板，并配置自动告警，以迅速发现 GPU 利用率突然下降或网络延迟激增等异常。

容错与自动恢复

通过分布式检查点、冗余硬件配置和动态作业重调度，将容错纳入系统设计。这一策略可最小化停机时间，并在硬件或网络故障时保持性能。

编译器与构建优化

在构建过程中利用激进的编译器标志和基于性能分析的优化，从代码中提取最大性能。定期更新和调优构建配置，并通过严格基准验证每项变更的影响，以确保最佳执行效果。

安全性、合规性与性能

整合并协同设计安全性、合规性与性能。定期审计配置，执行访问控制，并保持符合行业标准的保障措施，包括加密、安全数据通道、零信任网络、硬件安全模块（HSM）和安全隔离区。确保性能调优绝不损害系统安全；同样也要确保安全机制不会产生不必要的性能开销。

全面的文档与知识共享

详细记录所有优化步骤、系统配置和性能基准。建立内部知识库以促进团队协作和快速入职，确保最佳实践可在项目之间保留并复用。

面向未来与可扩展性规划

设计模块化、可适应的系统架构，以便轻松纳入新兴硬件和软件技术。持续评估可扩展性需求，并更新优化策略，使系统在工作负载增长时仍保持有竞争力的性能。

系统架构与硬件规划

硬件、互连和数据路径决定性能和成本效益的上限——软件调整无法弥补 GPU 长期缺料。应将加速器、CPU/DRAM/I/O 与散热/供电匹配到工作负载，按每美元/每瓦的有效吞吐量规划，从一开始就避免瓶颈。

具体而言，应面向有效吞吐量（每美元/每瓦的有用工作），而非仅面向原始 FLOPS 进行设计。使加速器和互连适配工作负载，适当配置 CPU/内存/I/O 以持续供给 GPU，保持数据本地，并规划电力和散热以使硬件维持峰值时钟。增加 GPU 前先评估扩展效率。以下是优化系统架构和提高硬件规划效率的建议：

面向有效吞吐量与效率设计

将有用吞吐量视为目标。规模化场景中，每一点性能提升都可转化为巨大的成本节省。应着力最大化每美元/每瓦的生产性工作，而非只关注原始 FLOPS。

选择合适的加速器

优先考虑具有更佳每瓦性能和内存容量的现代 GPU。较新架构提供原生 FP8、FP4 精度支持等特性，以及更快的互连；与旧代 GPU 和系统相比，这些能力可带来显著加速。

利用高带宽互连

对于多 GPU 工作负载，应优先使用具备 NVLink/NVSwitch 的系统（如 GB200/GB300 NVL72），而不是仅有 PCIe 连接的系统。NVLink 5 的 GPU 间双向带宽最高可达 1.8 TB/s（超过 PCIe Gen5 的 14 倍），可能实现近线性 GPU 扩展。NVLink Switch 域可通过二级交换机扩展，在一个 NVLink 域中连接最多 576 块 GPU。这样可实现分层集合通信，尽可能保持在 NVLink 内，再回退到机架间网络。

平衡 CPU/GPU 与内存配比

为每张 GPU 配置足够的 CPU 核、DRAM 和存储吞吐量。例如，可为每张 GPU 分配约一个快速 CPU 核来承担数据加载和网络任务。确保系统 RAM 和 I/O 能以所需速率（每 GPU 数百 MB/s 量级）供给 GPU，避免其饥饿。

规划数据局部性

若跨多个节点训练，应最小化节点外通信。尽可能将紧耦合工作负载留在同一 NVLink/NVSwitch 域内以利用完整带宽，并使用可获得的最高速互连。理想情况下，节点内和机架内通信使用 NVLink，机架间通信使用 InfiniBand。

避免链路中的瓶颈

找出最慢环节——CPU、内存、磁盘或网络——并扩展它。例如，若 I/O 导致 GPU 利用率低，应投资更快存储或缓存，而不是增加更多 GPU。端到端设计中各组件良好匹配，可避免浪费 GPU 周期。

选择合适的集群规模

增加 GPU 时要警惕边际收益递减。超过某个集群规模后，开销会增长，应确保加速收益足以证明成本合理。通常应先提高 N 张 GPU 的利用率（例如达到 95%），再扩展到 $2N$ 张 GPU。

面向散热与供电设计

确保数据中心能承受 GPU 的散热和供电需求。GB200/GB300 等高性能系统具有很高 TDP。应提供充足的散热（可能为液冷）和供电能力，使 GPU 能在不降频的情况下维持加速时钟。

统一 CPU-GPU“超级芯片”架构

统一内存和封装内链路能在将合适数据放在合适层级时容纳更大模型并降低拷贝开销。将 Grace 用于预处理、将 HBM 用于“热”张量，可使超级芯片成为停顿更少的紧耦合引擎。

在 Grace Blackwell 超级芯片上，应将 CPU 和 GPU 视为共享内存复合体。将热权重/激活保留在 HBM，将溢出或低频数据通过 NVLink-C2C 放入 Grace LPDDR。利用封装内 Grace CPU 进行预处理/编排，并采用预取或流水线管理内存来隐藏超大模型的延迟。可按以下方式利用超级芯片架构：

利用统一 CPU-GPU 内存

利用 Grace Blackwell（GB200/GB300）超级芯片的统一内存空间。两张 Blackwell GPU 与一颗 72 核 Grace CPU 通过 NVLink-C2C（900 GB/s）共享一致性内存池。可将 CPU 的大容量内存（例如 480 GB LPDDR5X）作为超大模型的扩展，同时将“热”数据保留在 GPU HBM 中以获得速度。

按局部性放置数据

即使采用统一内存，也应优先考虑数据放置。将模型权重、激活和其他频繁访问的数据放在具有更高本地带宽的 GPU HBM3e 中，让低频或溢出数据驻留在 CPU RAM。这样可避免 900 GB/s NVLink-C2C 链路成为关键数据的瓶颈。

在可用时利用 CPU-GPU 直接内存访问

在 GB200、GB300 等组合 CPU-GPU 超级芯片上，利用 GPU 直接访问 CPU 内存的能力。GPU 可经 NVLink-C2C 一致性地读写 Grace LPDDR 内存，无需经主机 PCIe 中转。其带宽和延迟仍逊于 HBM，因此应预取托管指针、分阶段准备数据并流水化传输以隐藏延迟。建议将热激活和 KV 缓存保留在 HBM，将 CPU 内存作为较低层级缓存并显式预取。

高效使用 Grace CPU

封装内 Grace CPU 提供 72 个高性能核心，应充分利用。将数据预处理、增强和其他适合 CPU 的任务卸载给这些核心。它们可通过 NVLink-C2C 快速供给 GPU，本质上充当 GPU 的超高速 I/O 和计算伙伴。

规划超大模型

对于超过 GPU 内存的万亿参数模型训练，GB200/GB300 系统允许将 CPU 内存作为模型内存池的一部分。优先使用框架的缓存分配器，并在自定义代码中使用 `cudaMallocAsync` 以减少碎片并启用图捕获。使用 CUDA Unified Memory 或托管内存 API 平稳处理溢出，并考虑显式预取即将使用的层（如 `cudaMemPrefetchAsync`），将数据从 CPU → GPU 内存分阶段迁移以隐藏延迟。

考虑针对超级芯片优化的算法

SuperOffload 是一组针对超级芯片优化的算法示例，重点提升卸载和张量转换/拷贝策略的效率。其创新包括先推测后验证（STV）、异构优化器计算和基于 ARM 的 CPU 优化器。SuperOffload 专为 NVIDIA 超级芯片（如 Grace Hopper、Grace Blackwell、Vera Rubin）设计；与传统卸载策略相比，可能提高 token 处理吞吐量和芯片利用率，具体收益取决于模型与实现。

多 GPU 扩展与互连优化

只有通信快速且具备拓扑感知时，扩展才有回报；否则新增 GPU 只会相互等待。应依托 NVLink/NVSwitch 带宽、现代集合通信和面向网络结构的放置，争取接近线性的加速。

具体而言，利用 NVLink/NVSwitch 域（如 NVL72）实现近线性扩展，并选择适配网络结构的并行策略。使用拓扑感知放置、更新的 NCCL 集合通信（如 PAT）和遥测数据，验证是否有效利用每 GPU 约 1.8 TB/s 的双向带宽；随规模扩展规划分层通信。以下是通过互连和拓扑优化利用多 GPU 扩展的建议：

面向高速全互连拓扑设计

例如，在拥有 72 张完全互连 GPU 的 NVL72 NVSwitch 集群上，任意 GPU 均可与其他 GPU 以完整 NVLink 5 速度通信。网络层面，NVLink Switch 域是无阻塞的；但应用层吞吐量会随并发流量和路径调度而变化，因此在假设每对 GPU 已饱和前，应使用 DCGM NVLink 计数器和 Nsight Systems 轨迹验证。可采用数据并行、张量并行和流水线并行等在较弱互连上可能受限的策略来利用该拓扑。

采用拓扑感知调度

在可能时，始终将多 GPU 作业共同放置在一个 NVLink Switch 域中。将作业的全部 GPU 保留在 NVL72 网络内，可使通信密集型负载接近线性扩展。跨 NVLink 域或标准网络混用 GPU 会引入瓶颈，紧耦合任务应避免这种做法。

利用高带宽

应认识到，NVLink 5 每张 GPU 每个方向可达 900 GB/s，相比前代使单 GPU 带宽翻倍。一个 NVL72 机架提供约 130 TB/s 的机架内总聚合带宽。这可显著降低通信等待时间；在 1.8 TB/s 条件下，即使数十 GB 梯度数据也可能在数毫秒内完成 all-reduce。应设计梯度同步、参数分片等训练算法，充分利用这种相对充裕的通信预算；实际带宽仍应以工作负载测量为准。

采用现代集合通信算法

使用针对 NVSwitch 优化的最新 NVIDIA NCCL 库。具体而言，可启用为 NVLink Switch 拓扑引入的并行聚合树（PAT）算法。它可利用 NVL72 拓扑，比其他树形/环形算法更高效地执行归约，从而进一步降低同步时间。

考虑细粒度并行

具备全带宽全互连时，可考虑此前不可行的细粒度模型并行。例如，当每张 GPU 与其他 GPU 都有 1.8 TB/s 双向吞吐量时，跨多 GPU 的逐层并行或张量并行可能很高效。过去可能会避免过多跨 GPU 通信，而 NVL72 允许更激进地划分工作且不轻易触及网络限制；实际效果仍受模型和流量模式影响。

监控是否饱和

尽管 NVL72 极快，性能分析时仍应关注链路利用率。若应用因极端 all-to-all 操作使 NVSwitch 饱和，可能需通过聚合梯度等方式限制通信。使用 NVIDIA 工具或 NVSwitch 遥测确认通信是否在 NVLink 容量范围内，并在必要时调整模式；例如可错开 all-to-all 交换以避免网络争用。DCGM 提供 NVLink 计数器，可帮助验证链路平衡并发现集合通信中的热点。

规划未来扩展

注意 NVLink Switch 可扩展到单机架之外：利用二级交换机，一个连接域最多可达 576 张 GPU。若以该超大规模运行，应规划分层通信，先使用本地 NVL72 的机架间集合通信，仅在必要时再使用机架间互连。这有助于优先最大化机架内 NVLink 使用，确保先利用最快链路，再退回机架间 InfiniBand 跳转。

识别联邦与分布式优化机会

对于跨异构环境的部署（如多云或边缘到云的设置），应采用自适应通信协议和动态负载均衡策略。这可在分布式系统中最小化延迟并最大化吞吐量，即使资源能力和容量不一致也能保持稳健性能。

操作系统与驱动优化

OS 抖动、NUMA 未命中和驱动不匹配会悄然吞噬吞吐量并造成难以通过调优消除的波动。强化软件栈（大页、亲和性、一致的 CUDA/驱动、持久化）可建立稳定、高性能的基线。

运行精简、面向 HPC 调优的 Linux。设置 NUMA/IRQ 亲和性，启用 THP 和较高 memlock 限制；保持各节点 NVIDIA 驱动/CUDA 一致；隔离系统抖动，调优 CPU 库和存储，正确设置容器限制，并及时更新 BIOS/固件/NVSwitch 网络以获得可预测吞吐量。以下是应在环境中探索的主机、OS 和容器优化：

使用针对 HPC 调优的 Linux 内核

确保 GPU 服务器运行配置为高性能计算的近期稳定 Linux 内核。禁用消耗 CPU 或 I/O 的非必要后台服务。使用“performance”CPU governor，而非“on-demand”或“power-save”，使 CPU 核保持高时钟以供给 GPU。

为性能关键负载禁用 swap

在训练服务器上禁用 swap 以避免页面抖动；若必须保留 swap，则使用 `mlock` 或 `cudaHostAlloc` 锁定关键缓冲区，确保其驻留在 RAM 中。

通过积极预分配避免内存碎片

为频繁使用的张量预分配大块连续内存，以减少运行时分配开销和碎片。这种主动策略可在长时间训练中实现更稳定、高效的内存管理。

调优 CPU 库环境变量

微调 `OMP_NUM_THREADS`、`MKL_NUM_THREADS` 等参数，使其更匹配硬件配置。调整这些变量可减少线程争用并提高 CPU 受限操作的并行效率。

面向 NUMA 感知设计

对于多 NUMA 服务器，将 GPU 进程/线程固定到本地 NUMA 节点的 CPU。使用 `numactl` 或 taskset 等工具，将每个训练进程绑定到离其分配 GPU 最近的 CPU；同样，将内存分配绑定到本地 NUMA 节点（`numactl --membind`），使 GPU DMA 的主机

内存来自最近的 RAM。这可避免代价高昂的跨 NUMA 内存流量，后者可能使有效 PCIe/NVLink 带宽减半。

为网络和 GPU 任务使用 IRQ 亲和性

将 NIC 中断显式绑定到与 NIC 同一 NUMA 节点的 CPU 核；同样将 GPU 驱动线程固定到专用核心，包括 `nvidia-persistence` 服务守护进程等长期运行服务的线程。该策略可最小化跨 NUMA 流量，并在高负载下稳定性能。

启用透明大页

以 always 或 `madvise` 模式启用透明大页（THP），使大内存分配使用 2 MB 页面。这可在框架分配数十或数百 GB 主机内存时减少 TLB 抖动和内核开销。检查 `/sys/kernel/mm/transparent_hugepage/enabled` 以确认 THP 是否激活。启用 THP 后，进程会为大分配使用大页；若负载对延迟敏感且观察到抖动，优先使用 `madvise` 模式。

提高最大锁定内存

配置 OS 以允许大量页锁定内存分配。GPU 应用常为更快传输锁定内存——设置 `ulimit -l unlimited` 或较高值，使数据加载器可分配页锁定缓冲区而不触及 OS 限制。这可避免失败或回退到可分页内存，后者会拖慢 GPU DMA。

使用最新的 NVIDIA 驱动和 CUDA 栈

在所有节点上保持 NVIDIA 驱动和 CUDA runtime 处于经过测试的稳定版本范围内的新版本。新驱动可带来性能改进，也是新 GPU 计算能力所必需。确保所有节点具有相同驱动/CUDA 版本，以避免多节点作业发生不匹配。启动时为 GPU 启用持久化模式（`nvidia-smi -pm 1`），使驱动保持加载并避免 GPU 重新初始化延迟。在所有节点更新 NVIDIA 驱动和工具包，以获取 bug 修复和性能改进。

使用 MIG 配置时启用 GPU 持久化

启用持久化模式后，GPU 保持“预热”并可随时使用，可降低作业启动延迟。若使用 Multi-Instance GPU（MIG）分区，这尤其关键：没有持久化时，MIG 配置会在每个作业后重置；保持驱动活跃则能保留切片。使用 MIG 时应始终配置持久化模式。

隔离系统任务

在每台服务器上专门保留一个或少量核心用于 OS 日常任务，如中断处理和后台守护进程。这样，负责供给 GPU 的主 CPU 线程不会被中断。可通过 CPU isolation 或 cgroup 固定来实现。消除 OS 抖动可确保吞吐量一致。

优化系统 I/O 设置

若负载执行大量日志记录或检查点，应使用有利于吞吐量的选项挂载文件系统。可为数据磁盘使用 `noatime`，并为流式读取增加文件系统预读。确保为 NVMe SSD 正确设置磁盘调度器，使用 `mq-deadline` 或 `noop` 以降低延迟波动。

执行定期维护

保持 BIOS/固件更新以获得性能修复。某些 BIOS 更新可改善 PCIe 带宽，或修复 GPU 的输入输出内存管理单元（IOMMU）问题。还应定期检查 NIC 和适用的 NVSwitch/Fabric 固件更新，以及 NVIDIA 提供的 Fabric Manager 升级等。微小的固件调整有时可解决隐蔽瓶颈或可靠性问题。

调优 Docker 和 Kubernetes 配置以获得最高性能

在容器中运行时，添加用于共享内存的 `--ipc=host` 等选项，并设置 `--ulimit memlock=-1` 以避免内存锁定问题。这可确保容器化进程访问内存时不受 OS 施加的限制。

GPU 资源管理与调度

更智能的放置与分区可在不购买新硬件的情况下提高利用率，并为混合负载保持可预测性。应尊重拓扑，在合适场景使用 MPS/MIG，并控制时钟/功耗以减少争用和尾延迟。

调度时考虑 GPU/NUMA/NVLink 拓扑；对较小作业使用 MPS 或 MIG 提高利用率，同时保留 ECC 和持久化以保证可靠性。需要稳定性时锁定时钟或设定功耗上限，避免 CPU 过度订阅，并智能地打包作业以在无争用条件下最大化 ROI。以下是 GPU 资源管理与调度建议：

拓扑感知的作业调度

确保 Kubernetes、SLURM 等编排器将容器调度到遵守 NUMA 和 NVLink 边界的节点上，以最小化跨 NUMA 和跨 NVLink 域的内存访问。这种对齐可降低延迟并改善总体吞吐量。

多进程服务（Multi-Process Service，MPS）

在单张 GPU 上运行多个进程时启用 NVIDIA MPS 以提高利用率。MPS 允许来自不同进程的内核在 GPU 上并发执行，而非时间片轮转。当单个作业未充分饱和 GPU 时这很有用——例如，在一张 GPU 上使用 MPS 运行 4 个训练任务可使其工作重叠并提升总体吞吐量。

多实例 GPU（Multi-Instance GPU，MIG）

使用 MIG 将高端 GPU 分割为更小实例，供多个作业使用。若有小模型推理或大量实验等轻负载，可将 GPU 切片，为每个作业保证资源。例如，现代 GPU 可分成多个 MIG 切片（最多 7 个）。不要对紧耦合并行作业使用 MIG，因为它们受益于完整 GPU 访问。当作业规模小于整张 GPU 时，可部署 MIG 实现隔离并最大化 GPU ROI。

MIG 的持久化

保持持久化模式开启，以在作业之间保留 MIG 分区。这可避免重新分区开销，并确保后续作业无需延迟即可看到预期 GPU 切片。应在集群启动时配置 MIG 并保持启用，使调度可预测；因为动态修改 MIG 配置需要重置 GPU，可能中断正在运行的作业。需规

划维护窗口，因为 GPU 重启后不会保留 MIG 设备分区。使用 NVIDIA MIG Manager 可在启动时自动重建所需布局。

GPU 时钟与功耗设置

若需要运行间一致性，可考虑使用 `nvidia-smi -lgc/-lmc` 将 GPU 时钟锁定到固定高频。默认情况下 GPU 使用自动加速，通常是最优选择；但固定时钟可避免瞬态降频。在功耗受限场景，可略微降频或设置功耗上限，让 GPU 保持在稳定的热/功耗包络内；若偶发降频是问题，这可能获得更一致的性能。

ECC 内存

除非有明确理由禁用，否则应在数据中心 GPU 上保持 ECC 启用以确保可靠性。其性能代价很小——内存带宽和容量可能损失几个百分点——但 ECC 可捕获否则会损坏长时间训练作业的内存错误。大多数服务器 GPU 出厂时默认启用 ECC；应保持开启以保护多周训练。

作业调度器感知

将 GPU 拓扑集成到 SLURM、Kubernetes 等作业调度器。需要低延迟耦合时，将调度器配置为在同一节点或同一 NVSwitch 组分配作业。使用 Kubernetes 设备插件或 SLURM Gres 为小作业调度 MIG 切片。GPU 感知调度器可避免单个作业跨越相距很远的 GPU 并遭受带宽问题。

CPU 过度订阅

调度作业时，应考虑每个 GPU 任务所需的 CPU 资源，如数据加载线程等。不要在一个节点上打包超过 CPU 承受能力的 GPU 作业。与其过载 CPU 导致所有 GPU 供给不足，不如让一张 GPU 空闲。监控每个 GPU 作业的 CPU 利用率，为调度决策提供依据。

为 NVSwitch 使用 NVIDIA Fabric Manager

在具备 NVSwitch 的系统（如 GB200/GB300 NVL72 机架）上，确保 NVIDIA Fabric Manager 正在运行。它管理 NVSwitch 拓扑与路由。没有它，多 GPU 通信可能无法充分优化，甚至在大型作业中失败。Fabric Manager 服务通常在配备 NVSwitch 的服务器上默认运行，但仍应确认已启用且正在运行，特别是在驱动更新后。

面向利用率的作业打包

通过智能打包作业最大化利用率。例如，在一个 4-GPU 节点上，若有两个 CPU 使用不多的 2-GPU 作业，在同一节点共同运行可节省资源；若位于同一计算节点或支持 NVLink 的机架中，还可使用更快的 NVLink 通信。相反，应避免共同放置后合计超过节点内存或 I/O 容量的作业。目标是在无争用条件下实现高硬件利用率。

I/O 优化

若数据跟不上，GPU 就会空闲——通常最大、最便宜的加速来自修复输入，而非数学计算。并行化、页锁定内存、异步传输和快速存储可确保模型持续得到供给。

通过并行化数据加载器、使用页锁定内存和异步传输、并将数据存储在快速 NVMe（最好配合 GPUDirect Storage）上来持续供给 GPU。合理进行条带化、缓存和压缩。测量端到端吞吐量，使 I/O 随集群规模扩展；异步写入检查点/日志。以下是数据流水线 I/O 优化建议：

并行加载数据

使用多个 worker/线程为 GPU 加载和预处理数据。默认的一到两个数据加载 worker 可能不足。通过 PyTorch `DataLoader(num_workers=N)` 等方式进行性能分析并增加数据加载进程/线程数，直至数据输入不再是瓶颈。高核心数 CPU 的存在就是为了供给 GPU，务必充分利用。

为 I/O 锁定主机内存

为数据传输缓冲区启用页锁定内存。许多框架提供类似 PyTorch DataLoader 的 `pin_memory=True` 选项，用于分配 GPU 可直接 DMA 访问的主机内存。使用页锁定内存可显著提高 H2D 拷贝吞吐量；结合异步传输可让数据加载与计算重叠。

重叠计算与数据传输

将输入数据流水化。GPU 正在计算 batch N 时，在 CPU 上加载和准备 batch N+1，并使用 CUDA stream 与非阻塞 `cudaMemcpyAsync` 在后台传输。这种双缓冲隐藏延迟——理想情况下 GPU 永不等待数据。确保训练循环使用异步传输；例如在 PyTorch 中，可用 `non_blocking=True` 将张量拷贝到 GPU。异步传输使 CPU 能在后台数据传输期间继续运行，从而通过重叠计算和数据传输提高性能。

使用快速存储（NVMe/SSD）

将训练数据存于快速本地 NVMe SSD 或高性能并行文件系统。机械磁盘会严重限制吞吐量。若可用，启用 GPUDirect Storage（GDS），使 GPU 能从 NVMe 或网络存储直接流式读取数据，绕过 CPU。这会在读取大型数据集时进一步降低 I/O 延迟和 CPU 负载。对于大型数据集，可让每个节点拥有数据的本地副本或分片。若使用网络存储，优先采用支持条带化的 Lustre 等分布式文件系统，或能并行服务许多客户端的对象存储。

调优 I/O 并发与条带化

避免单文件访问造成的瓶颈。若所有 worker 使用同一个大文件，可跨多个存储目标对其进行条带化，或将其拆分为块以供多台服务器服务。例如，将数据集拆为多个文件，让每个数据加载 worker 同时读取不同文件；这可最大化存储系统的聚合带宽。

优化小文件访问

若数据集由数百万小文件组成，应减轻元数据开销。每秒打开过多小文件会压垮文件系统的元数据服务器。解决方式包括将小文件打包进 `tar` 或 `RecordIO` 等较大容器，使用

批量读取的数据摄取库，或确保客户端启用元数据缓存。这可降低每文件开销并加快 epoch 启动时间。

可用时使用客户端缓存

利用任何可用缓存层。若使用 NFS，可增加客户端缓存大小和持续时间；对于分布式文件系统，可考虑缓存守护进程，甚至手动将部分数据集缓存到本地磁盘。目标是避免从慢速源反复读取相同数据。若各节点在不同时间处理相同文件，本地缓存可大幅减少冗余 I/O。

明智地压缩数据

若 I/O 是瓶颈，可压缩存储数据集，但应使用 LZ4 或 Zstd 快速模式等轻量压缩。这会牺牲少量 CPU 以减少 I/O 数据量。若解压使 CPU 成为瓶颈，可考虑多线程解压或卸载到加速器。还可让一个线程读取压缩数据、另一个线程并行解压，使解压与读取重叠。现代 GPU 搭配 GPUDirect Storage 和 `cuFile` I/O 栈时，可利用 GPU 计算资源（或图像/视觉数据专用解码器）即时解压数据。

测量吞吐量并消除瓶颈

持续监控数据流水线吞吐量。若 GPU 利用率未接近 100%，且怀疑存在输入延迟，应测量磁盘读取 MB/s 以及数据加载核心的繁忙程度。`dstat`、NVIDIA DCGM 等工具可揭示 GPU 是否在等待数据。系统性地调优每个组件，例如增加预取缓冲区、网络缓冲区大小、优化磁盘 RAID 设置等，直到输入流水线供给数据的速度与 GPU 消耗速度一致。消除 I/O 停顿后，这些优化常能在相同硬件上将 GPU 利用率从约 70% 提升到 >95%。

为多节点扩展 I/O

在集群规模下，确保存储系统可承受聚合吞吐量。例如，8 张 GPU 每张消耗 200 MB/s，则每节点为 1.6 GB/s；100 个节点需要 160 GB/s。极少有中心化文件系统能持续提供该性能。可通过跨存储服务器分片数据、使用每节点缓存或将数据预加载到各节点本地磁盘来缓解。以存储空间换取吞吐量（如保存多份数据）通常值得，可避免昂贵 GPU 缺料。

最小化检查点与日志开销

高效写入检查点和日志。若可行，对检查点使用异步写入，或先写入本地磁盘再复制到网络存储，以避免训练停顿。压缩检查点或使用稀疏存储格式以减小大小。通过聚合迭代统计数据并仅每 N 次迭代记录一次，而非每次记录，限制逐步日志频率；这会大幅降低 I/O 开销。

还可使用 `cuda-checkpoint` 和 Checkpoint/Restore in Userspace（CRIU）挂起运行中的 GPU 进程，以持久化进程映像。准备恢复时，CUDA 驱动可恢复设备内存和 CUDA状态，甚至恢复到同一设备类型的其他 GPU。应将其视为模型 state-dict 或分片检查点文件的补充，而非替代品。

数据处理流水线

数据格式、布局和局部性决定流水线在规模化运行时是否顺畅。二进制格式、分片、缓存和高优先级线程能将 I/O 从瓶颈转为稳定的数据流。

将数据集转换为二进制或内存映射格式，跨存储和节点分片；提高线程优先级或将简单增强移到 GPU 以防止停顿。缓存热数据/KV 状态，积极预取和缓冲，并设置合适 batch 大小，使流水线从磁盘到设备始终顺畅。以下是改进数据处理的建议：

使用二进制数据格式

将数据集转换为 TFRecords、LMDB 或内存映射数组等二进制格式。这可减少处理数百万小文件的开销，并加速数据摄取。

调优文件系统

除使用 `noatime` 挂载文件系统并增加预读外，还应考虑跨多个存储节点分片数据，以分摊 I/O 负载并避免单台服务器成为瓶颈。

为 CPU 受限负载禁用超线程

对于高度 CPU 受限的数据流水线，禁用超线程可减少资源争用并带来更一致的性能。这对单线程性能关键的系统尤其有益。

提高线程优先级

使用 `chrt` 或 `pthread_setschedparam` 等工具提高数据加载和预处理 CPU 线程的调度优先级。赋予这些线程更高优先级，可确保以最低延迟向 GPU 供给数据，降低流水线停顿概率。

缓存频繁使用的数据

利用操作系统页面缓存或专用 RAM 磁盘缓存频繁访问的数据。这对 NLP 等应用尤其有益，其中某些 token 或短语会被反复访问，可减少冗余处理和 I/O 开销。

预取和缓冲数据

始终在需要某次迭代前提前加载数据。使用后台数据加载线程或进程，例如带 `prefetch_factor` 的 PyTorch DataLoader。对于分布式训练，使用 `DistributedSampler` 确保每个进程获得唯一数据，避免冗余 I/O。

并行化数据转换

若图像增强、文本 token 化等 CPU 预处理负担很重，应将其分发到多个 worker 线程/进程。进行性能分析以确保 GPU 等待时 CPU 不是瓶颈。若是，可增加 worker，或将部分转换移至 GPU；NVIDIA DALI 等库可在 GPU 上异步执行图像操作。

缓存模型状态与输出

使用 LLM 推理时，缓存频繁出现 token 的嵌入和 V 缓存有助于避免反复重新计算。同样，若 LLM 训练作业多次复用相同数据集（称为 *epoch*），应利用 OS 页面缓存或

RAM 存储热数据。

跨节点分片数据

在多节点训练中，为每个节点分配数据子集，避免所有节点从同一源读取完整数据集，从而横向扩展 I/O。可使用分布式文件系统，或手动分配分片使各节点读取不同文件。这不仅加速，还天然符合数据并行，因为每个节点处理自己的数据分片。DeepSeek 的 Fire-Flyer File System（3FS）是分布式数据集分片文件系统示例。其通过将数据集分片分布在每个节点的 NVMe SSD 上、同时尽量减少传统缓存，达到每秒数 TB 吞吐量；该设计以本地高速数据供给每张 GPU，避免 I/O 瓶颈。

监控流水线并调整 batch 大小

有时增大 batch 大小会将更多工作推给 GPU、降低 I/O 频率，从而提高总体利用率；但会影响收敛，因此只能增加到一定程度。相反，若 GPU 经常等待数据且无法加快 I/O，可能应减小 batch 以缩短每次迭代、减少空闲时间，或对较小 batch 做梯度累积，使数据读取更连续。应找到让 GPU 几乎始终忙碌的平衡点。

在 GPU 上应用数据增强

若增强操作简单但应用于海量数据，如添加噪声或归一化，可能值得在 GPU 上执行以避免饱和 CPU。数据加载期间 GPU 往往未充分利用，因此加载后用小型 CUDA 内核增强数据可能高效；但需注意不要将流水线串行化。使用 stream，在 batch N 训练时重叠 batch N+1 的增强。利用 NVIDIA DALI 等 GPU 加速库异步执行这些任务，有助于保持平滑、高吞吐的数据流水线。

关注端到端吞吐量（如每秒 token 数）

请记住，若数据流水线将吞吐量减半，加速模型计算并无帮助。始终进行端到端分析，而非只孤立分析训练循环。使用 Nsight Systems、Nsight Compute 测量内核时间线和停顿，或使用 PyTorch 分析器进行框架层归因。随后比较合成数据与真实数据的迭代时间，了解数据加载引入多少开销。目标是使开销低于理想值的 10%；若超过，应投入时间优化流水线，因为这常能为训练带来巨大的“免费”加速。

性能分析、调试与监控

无法优化未曾测量的内容；性能分析可揭示负载究竟受计算、内存、I/O 还是网络限制，从而针对正确问题修复。随着代码、驱动和数据演进，持续遥测和回归测试可防止性能收益流失。

具体而言，使用带 NVTX 的 Nsight Systems/Compute 和框架分析器，确定负载是否受计算、内存、I/O 或通信限制。减少 Python 开销、观察利用率空隙、平衡各 rank 的工作、追踪内存/网络/磁盘健康状况，并用性能回归测试和告警作为变更门槛。可按以下建议分析、监控和调试 AI 工作负载性能：

性能分析以发现瓶颈并进行根因分析

定期对训练/推理作业运行分析器。使用 NVIDIA Nsight Systems 获取 CPU 和 GPU 活动时间线；还可用 Nsight Compute 或 PyTorch 分析器深入查看内核效率。确定作

业是计算受限、内存受限，还是在等待 I/O/通信，并据此优化。例如，内存受限负载应专注减少内存流量，而非实施计算受限型优化。还可结合机器学习驱动的分析预测并预防性能瓶颈，帮助实时自动化微调。使用 GPUDirect Storage 时，启用 GDS 跟踪以关联 `cuFile` 活动与内核空隙。

消除 Python 开销

分析训练脚本，识别过度循环或日志记录等 Python 瓶颈，并用向量化操作或优化库调用替换。最小化 Python 开销有助于确保 CPU 不会成为整体系统性能中的隐藏瓶颈。

测量 GPU 利用率与空闲间隙

持续监控 GPU 利用率、SM 效率、内存带宽使用等。若发现利用率周期性下降，应将其与事件关联。例如，每 5 分钟一次的利用率下降可能与保存检查点同步。这类模式指向错开检查点、使用异步刷新等优化机会。使用 DCGM 或守护模式的 `nvidia-smi` 随时间记录这些指标。

使用 NVTX 标记

使用 NVTX 区间或框架性能分析 API 为代码插桩，标记数据加载、前向传播、反向传播等不同阶段。这些标记会出现在 Nsight Systems 或 Perfetto 时间线上，帮助将 GPU 空闲时间或延迟归因到流水线的具体部分，也便于向开发者说明哪段代码需要关注。对于 PyTorch，可使用 `torch.profiler.record_function()`。

除 PyTorch 分析器外使用内核分析工具

对于性能关键内核，使用 Nsight Compute 检查 occupancy、吞吐量等内核级指标，或使用 Nsight Systems 分析 GPU/CPU 时间线及重叠情况。检查实际 occupancy、内存吞吐量和指令吞吐量，寻找内存带宽接近硬件最大值等内存瓶颈迹象，以识别内存受限负载。分析器的“Issues”部分通常会直接指出内核是否受内存或计算限制及其原因。使用这些反馈指导代码变更，例如全局加载效率低时改善内存合并访问。

检查 warp 分歧

使用分析器查看 warp 是否发生分歧，它可显示分支效率和分歧分支指标。分歧意味着 warp 中部分线程因分支而不活跃，会损害吞吐量。若影响显著，应重新审视内核代码，重构条件逻辑或数据分配以最小化 warp 内分歧，并确保每个 warp 处理一致的工作。

验证负载均衡

在多 GPU 作业中跨各 rank 进行分析。有时一张 GPU（rank 0）负责汇总统计和收集数据等额外工作，常会成为瓶颈。监控每张 GPU 的时间线；若某张 GPU 持续落后，应分散这些额外工作。例如，可让非零 rank 分担 I/O 和日志职责。确保所有 GPU/rank 负载相近，避免最慢 rank 拖累其余部分。

监控内存使用

随时间跟踪 GPU 内存分配与使用。确保不接近 OOM，否则框架可能意外将张量换出至主机，造成巨大减速。若内存使用逐次迭代上升，很可能发现了泄漏；此时使用

`torch.cuda.memory_summary()`、Nsight Systems 的 GPU 内存跟踪等工具分析详细分配。在 CPU 侧监控分页，因为进程常驻内存（RES）不应显著超过物理 RAM。若观察到分页，应减少数据集预加载量或增加 RAM。

监控网络与磁盘

对于分布式作业，使用 OS 工具监控网络和磁盘吞吐量，确保实际吞吐量符合预期。例如，若 100 Gbps 链路完全利用，应看到 12.5 GB/s（12.5 GB/s = 100 Gb/s ÷ 每字节 8 bit）。若达不到，网络可能是瓶颈或配置错误。同样监控训练节点的磁盘 I/O；若看到磁盘利用率 100% 的尖峰同时 GPU 空闲，可能需要更好地缓冲或缓存数据。

为异常设置告警

在生产或长时间训练场景中，为 ECC 错误、设备过热等 GPU 错误事件设置自动告警或日志，有助于识别异常缓慢的迭代。例如，NVIDIA DCGM 可监测健康指标；若 GPU 开始降频或发生错误，可触发操作。这样能立即捕获散热故障导致降频等性能问题，而非等作业结束后才发现。

执行回归测试

维护一组基准任务，在更改软件（包括 CUDA 驱动、CUDA 版本、AI 框架版本甚至训练代码）时运行。将性能与先前运行比较，尽早捕获回退。驱动更新或代码改动无意中降低吞吐量并不罕见；在标准负载上快速分析即可凸显问题以便调查。例如，某个内核可能意外不再使用 Tensor Core，这显然需要排查。

GPU 编程与 CUDA 调优优化

让内核与内存层次结构及硬件特性相匹配，才能取得显著且持久的收益。融合、Tensor Core、CUDA Graph 以及编译器路径（如 `torch.compile` 和 OpenAI Triton）可将启动开销转化为有效计算。

围绕内存层次结构优化：合并全局内存加载、分块载入共享内存、管理寄存器与占用率，并将传输（如 `cp.async`/TMA）与计算重叠。优先采用经过调优的库和 CUDA Graph，借助 `torch.compile` 与 OpenAI Triton 实现融合，并通过屋顶线分析和 PTX/SASS 检查验证可扩展性。以下是一些 GPU 与 CUDA 编程优化提示和技术：

理解 GPU 内存层次结构

请牢记 GPU 的分层内存结构：每线程的寄存器、每个块/SM 的共享内存与 L1 缓存、跨 SM 的 L2 缓存，以及全局 HBM。在较高层级中尽可能复用数据。例如，使用寄存器和共享内存复用数值，并减少对较慢全局内存的访问。好的内核会确保绝大多数数据要么位于寄存器中，要么通过合并访问和缓存从 HBM 高效加载。

合并全局内存访问

确保同一 warp 内的线程访问连续内存地址，使硬件能以尽可能少的事务完成服务。warp 线程的跨步或离散内存访问会使每个 warp 产生多次内存事务，从而实际浪费带宽。应重组数据布局或索引计算，使 warp 每次加载数据时都能通过一次宽内存事务完成。

使用共享内存复用数据

共享内存类似带宽极高、由开发者手动管理的缓存。将频繁使用的数据（如矩阵分块）载入共享内存，让线程在继续处理前多次操作这些分块。这种常见的分块技术可大幅减少全局内存流量。注意共享内存的 bank 冲突：应组织共享内存访问模式或填充数据，确保线程不会竞争同一 memory bank，否则访问会被串行化并降低性能。

优化内存对齐

尽可能将数据结构按 128 字节对齐，尤其是批量内存复制或向量化加载时。即使理论上已合并，未对齐访问仍可能迫使硬件执行多次事务。对全局内存 I/O 使用 float2、float4 等向量类型可让每条指令加载或存储多个值，但必须确保数据指针正确对齐到该向量大小。

尽量减少内存传输

仅在必要时向 GPU 传输数据，并尽量以大块传输。若可以，应将许多小传输合并为一次大传输。例如，如果每次迭代都要发送许多小数组，可将它们打包到一个缓冲区中一次发送。小而频繁的 `cudaMemcpy` 会成为瓶颈。若使用统一内存（Unified Memory），请使用显式预取（`cudaMemPrefetchAsync`）在需要前将数据调入 GPU，以避免关键计算阶段发生按需缺页。

避免过多的临时分配

频繁分配和释放 GPU 内存会损害性能。例如，在内核中频繁使用 `cudaMalloc/cudaFree` 或设备端 malloc 会带来额外开销。应复用内存缓冲区，或使用 PyTorch 等大多数深度学习框架提供的内存池（其实现了 GPU 缓存分配器）。若编写自定义 CUDA 代码，可考虑将 `cudaMallocAsync` 与内存池配合使用，或自行管理临时内存池，以避免反复分配/释放的开销。

平衡线程与资源使用

实现良好的占用率—资源平衡。增加线程以提高占用率有助于隐藏内存延迟，但若每线程使用过多寄存器或共享内存，占用率便会下降。调优内核启动参数（包括每块线程数），以确保有足够多正在执行的 warp 来掩盖延迟，但也不能多到使每线程缺少寄存器或共享内存。对于指令级并行性（ILP）高的内核，为提高占用率而减少寄存器使用反而可能损害性能。最佳点通常位于占用率区间中部，因为最大占用率并不总是理想选择。可使用 NVIDIA Nsight Compute 占用率计算器（Occupancy Calculator） 试验各种配置。

监测寄存器和共享内存用量

使用 Nsight Compute 等剖析工具持续监测每线程的寄存器和共享内存消耗。若观察到占用率低于 25%，可考虑增加每块线程数，以更充分利用可用硬件资源。不过，应通过详细的占用率报告和内核执行指标，确认此调整不会造成过多寄存器溢出。寄存器溢出会带来额外内存流量并降低整体性能。

将内存传输与计算重叠

尽可能将内存传输与计算重叠。在多个 CUDA 流中使用 `cudaMemcpyAsync`，在内核运行时进行预取。批量移动到共享内存时优先使用 Tensor Memory Accelerator；细粒度的分阶段复制和预取则使用 `cp.async`。这些方法通过把数据传输与计算重叠，可有效掩盖全局内存延迟，确保 GPU 核心无需等待内存操作完成而保持充分利用。

可行时使用批量预取

对于可预测的模式，可通过 PTX `cp.async.bulk.prefetch.tensor.[1-5]d.L2.global*`（或 `prefetch.global.L2` 系列）预取到 L2，并使用 TMA（例如 `cp.async.bulk.tensor`）将分块暂存到共享内存。也可用 cp.async 异步地将全局内存暂存至共享内存，使复制与计算重叠；还可在使用前显式将数据加载到寄存器中。这些主动方法可减少全局内存访问造成的延迟，确保关键数据恰好在需要时位于寄存器或共享内存等更快、延迟更低的存储中，从而最小化执行停顿并提高整体内核效率。

利用协作组

使用 CUDA 协作组，在部分线程之间实现高效的局部同步，而不是强制整个线程块的屏障。这项技术能对同步进行更细粒度控制，减少不必要的等待时间和开销。将共享数据或执行相关计算的线程编成组后，只需同步确实需要协调的线程，从而形成更高效的执行模式并提升整体吞吐量。

优化 warp 分歧

应组织代码，使一个 warp 内的线程尽量遵循相同执行路径。分歧可能使该 warp 的执行时间翻倍，例如半个 warp（16 个线程）走一个分支，另半个 warp（16 个线程）走另一个分支。若某些分支只被少量数据触发，可考虑对数据“排序”或分组，让 warp 处理一致的情况，使其全部为真或全部为假。使用 ballot、shuffle 等 warp 级原语可为某些问题构建无分支方案。将 warp 视为工作单元，力求全部 32 个线程以锁步方式执行相同工作，以获得最高效率。

利用 warp 级操作

适当时使用 CUDA 的 warp 内置函数，让线程无需经由共享内存即可通信。例如，使用 `__shfl_sync` 向 warp 中全部线程广播一个值，或执行 warp 级归约（如对整个 warp 的寄存器求和），而非让每个线程写入共享内存。这些内置函数绕过较慢的内存，可加速能在 warp 内完成的归约、扫描等算法。在 warp 内处理这些任务，便可避免共享内存和整块同步相关的延迟。

使用 CUDA 流实现并发

在单个进程/GPU 内，若独立内核未占满所有资源，可在不同 CUDA 流中启动它们以重叠执行。可让计算与计算重叠，例如一个流计算模型的一部分，另一个流启动独立内核（如 GPU 上的数据预处理）或异步 `memcpy`。注意依赖关系，并在需要时用 CUDA 事件同步。正确使用流可避免任何资源闲置，从而提高 GPU 利用率，尤其适用于较轻的内核。

优先使用库函数

尽可能使用 NVIDIA 针对核心数学和集合通信操作优化的库，如 cuBLAS、cuDNN、Thrust 和 NCCL。在分布式推理中进行点对点 GPU 数据移动时，可用时请使用 NIXL；需要 GPU 发起的细粒度传输时也可使用 NVSHMEM。这些库针对每代 GPU 架构都经过深度优化，性能通常接近理论“光速”峰值，因此无需自行重复实现。例如，除非具有非常特殊的模式，否则应使用 cuBLAS GEMM 完成矩阵乘法而不是自定义内核。库还会透明地处理新硬件特性。PyTorch（及其编译器）等 AI 框架在底层就使用了这些优化库。

为重复启动使用 CUDA Graph

若静态训练循环会启动数千次，可考虑使用 CUDA Graph 捕获操作序列并以图形式启动。这能显著降低每次迭代的 CPU 启动开销，尤其是在多 GPU 场景中：启动大量内核和 `memcpy` 会给 CPU 带来额外压力和延迟。

检查可扩展性限制

优化内核时，应定期检查其随问题规模增长以及跨架构的扩展表现。一个内核可能在小输入上拥有优异占用率和性能，却难以扩展到大输入，因为此时可能开始频繁扰动 L2 缓存或遭遇内存缓存驱逐。使用屋顶线分析，将实际 FLOPS 和带宽与硬件上限比较，确保没有遗漏可获得的性能。

检查 PTX 和 SASS 以进行高级内核分析

对于性能关键的自定义 CUDA 内核，请使用 Nsight Compute 检查生成的 PTX 和 SASS。此类深入分析可揭示内存 bank 冲突、冗余计算等问题，并引导你进行有针对性的底层优化。

使用 PyTorch 编译器

利用 PyTorch 的 `torch.compile`，通过 TorchInductor 将 Python 层操作融合为优化内核。编译器还可通过整合 CUDA Graph 降低启动开销。优化预热后，通常可获得约 10%–40% 的提升。这消除了解释器开销，并解锁编译器级优化。

实践中，启用 `torch.compile` 会自动组合内核，并更高效地利用 NVIDIA GPU 硬件（如 Tensor Core），已为许多模型带来显著加速（例如 20%–50%）。务必在你的模型上测试编译模式。它虽可大幅提高吞吐量，但部署前应确保兼容性和正确性。图稳定后，启用 CUDA Graph 以降低每次迭代的 CPU 开销；同时维持静态内存池，以满足指针稳定性约束。

为动态形状做好规划

若输入大小会变化，可用 `torch._dynamo.mark_dynamic()` 标注动态维度，或用 `torch.export()` 导出形状多态图，再进行编译。使用 `torch.compiler.set_stance()` 配合 `"fail_on_recompile"`，以及 `torch._dynamo.error_on_graph_break()`，控制重新编译行为，并在测试和 CI 中暴露有问题的形状频繁变化。可行时使用静态形状，以启用 CUDA Graph 并降低每次迭代的 CPU 开销。

通过 `torch.compile` 利用 Triton 内核

若 PyTorch 未能很好地融合某项操作，可考虑用 Triton 编写并整合自定义 GPU 内核。PyTorch 可通过 `torch.library.triton_op` 轻松注册自定义 GPU 内核。

可用时使用自动调优

启用库的自动调优功能以最大化底层性能。例如，在输入大小固定时设置 `torch.backends.cudnn.benchmark=True`。这会让 NVIDIA cuDNN 库尝试多种卷积算法，并为你的硬件选择最快的一种。一次性开销会换来可加速训练和推理的优化内核。若不要求严格可复现性，可关闭 `cudnn.deterministic` 以允许非确定性算法，从而解锁这些更快的实现。

利用只读路径

将常用常量或系数标记为只读，以便 GPU 将其缓存在专用的 L1 只读缓存中。在 CUDA C++ 中，可使用 `const __restrict__` 指针提示数据不可变。在现代 GPU 架构上，编译器会为带 `const __restrict__` 限定符的指针生成带缓存的全局加载。使用 AI 框架和库时，应确保查找表或静态权重位于设备上并被视为常量。此优化可减少这些值的全局内存流量和延迟，因为每个 SM 可从缓存快速读取，而非反复访问较慢的 DRAM。

内核调度与执行优化

启动开销和不必要的同步会造成闲置间隙，严重损害吞吐量。融合小内核并使用持久化/动态策略，能让设备保持忙碌并隐藏延迟。

通过最小化同步、融合小内核，并在反复启动相同工作时使用持久化内核，使设备保持忙碌。对于不规则任务，可考虑 GPU 动态并行，但应审慎使用以免增加开销。以下是改善内核调度和执行的提示：

最小化 GPU 同步调用

避免使 GPU 进度停滞的不必要全局同步。过度使用 `cudaDeviceSynchronize()` 或阻塞式 GPU 操作（如同步内存复制），会插入 CPU 与 GPU 均无法有效工作的空闲间隙。只在绝对必要时同步，例如传输最终结果或调试时。让异步操作排队，可保持 GPU 忙碌，并让 CPU 腾出时间准备后续工作，从而形成更连续的执行流水线。

融合小内核以摊销启动开销

若有许多微小 GPU 内核连续启动，应尽可能将其操作合并到一个内核中运行。每次内核启动都有数十微秒量级的固定成本；通过手动 CUDA 内核融合、XLA 融合，或借助 NVIDIA CUTLASS/Triton 等工具处理自定义操作，可提升吞吐量。融合内核将更多时间用于实际工作，较少时间耗在启动开销或内存往返上。这尤其适合可一次执行一串逐元素操作的推理或预处理流水线。先尝试 `torch.compile(mode="reduce-overhead")`：编译器能融合操作链，并将稳定区域封装到 CUDA Graph 中，从而降低 CPU 启动开销。对于未融合的热点，可考虑迁移到 Triton 内核，并在适用时使用异步 TMA 和自动 warp 专用化。

利用 GPU 动态并行进行 GPU 内部工作调度

使用 CUDA 动态并行，让 GPU 内核可直接从 GPU 启动其他内核而无需返回 CPU。在不可预测或迭代性工作中，例如算法需根据中间结果生成额外任务时，动态并行通过移除 CPU 启动瓶颈来降低延迟。例如，父内核可在设备上直接划分并启动子内核进行进一步处理。这使整个工作流留在 GPU 上，避免 CPU 介入并实现更好的重叠和利用率。不过应审慎使用，过度使用会引入自身开销。

为重复工作负载使用持久化内核

当工作负载涉及快速连续启动相同内核时（如处理工作队列，或处理具有相同计算的流式批次），可采用持久化内核策略。持久化内核只启动一次并保持活跃，在循环中复用线程处理许多工作单元，而不是为每个单元重新启动内核。这种方式以更复杂的内核设计换取显著更低的调度开销。保持内核存活可避免反复启动成本，并获得更高的持续占用率。高性能分布式训练和推理系统经常使用该技术，以最大化吞吐量并最小化迭代任务延迟。

评估线程块集群

使用线程块集群以保持数据接近并减少重复启动开销。在 Blackwell 上，最多 16 个线程块可组成一个集群（提高非可移植限制后）。在持久化风格的设计中，使用集群感知同步和共享内存驻留来改善局部性。借助 Nsight Compute 等内核级剖析工具，分析占用率与驻留之间的权衡。

算术优化与低精度/混合精度

低精度和稀疏性让你以比特数换取显著的速度和内存收益，且通常对精度影响微乎其微。混合精度、TF32/FP8/INT8 和融合缩放可利用硬件计算路径，提高单位成本吞吐量。

具体而言，使用混合精度（BF16/FP16）和 Tensor Core 可获得大幅收益；采用 TF32 可轻松加速 FP32；在质量允许时评估 FP8/FP4。利用结构化稀疏性、低精度梯度/通信和用于推理的 INT8/INT4 量化，并融合缩放/激活操作以保持精度。以下优化技术适用于提高算术计算性能并利用低精度/混合精度：

使用混合精度训练

训练时利用 FP16 或 BF16 加速数学运算并减少内存使用。现代 GPU 的 Tensor Core 可大幅加速 FP16/BF16 矩阵操作。为保持数值稳定性，应将最终累加、权重副本等关键部分保留为 FP32，而将大部分计算以半精度运行。这通常可带来约 1.5–3.5 倍加速（取决于模型和内核组合，`matmul` 密集型工作负载收益更大），精度损失很小；如今这已是大多数框架自动混合精度（AMP）的标准做法。

采用梯度累积与激活检查点

使用梯度累积可在不额外增加内存使用的情况下有效扩大批量大小；对于非常深的网络，可考虑激活检查点以减少内存占用。当训练的模型接近或超过 GPU 内存限制时，这些技术至关重要。

在较新硬件上优先选用 BF16 而非 FP16

若可用，应使用 BF16 而不是 FP16，因为它具有更大的指数范围且不需要损失缩放。现代 GPU 对 BF16 Tensor Core 的支持速度与 FP16 相同。BF16 可避免上溢/下溢问题，从而简化训练，同时仍享有半精度的性能优势。

利用 FP8、新型精度和缩放技术

在现代 GPU 上，FP8 Tensor Core 对计算受限内核提供约为 FP16 或 BF16 两倍的计算吞吐量，同时降低激活值和权重带宽。此外，FP4（NVFP4）Tensor Core 的吞吐量又是 FP8 的两倍；它结合微张量缩放（一种保持精度的误差校正技术）用于推理，可提高 token 吞吐量。训练时使用 FP8 与 NVIDIA Transformer Engine，并在需要时保持 FP16 或 FP32 累加器。推理时先评估 FP8，只有在校准显示质量满足任务要求后才采用 NVFP4。训练建议使用混合 FP8：前向激活值/权重使用 E4M3，梯度使用 E5M2；通常使用 256–1024 的延迟缩放窗口也有益。TE 可集成到 PyTorch，并受现代 GPU 硬件支持。应优先采用框架的 TE 内核，而不是临时编写的 FP8 自定义操作。端到端加速取决于内核组合、内存带宽和校准，因此需在你的模型和工作负载上验证精度与性能。

利用 Tensor Core 和 Tensor Memory Accelerator（TMA）

若可能，应确保自定义 CUDA 内核使用 Tensor Core 执行矩阵操作；为简化实现，可使用 CUTLASS 模板。借助 Tensor Core 和 TMA 将张量异步移动至共享内存，可显著加速 GEMM、卷积及其他张量操作，性能常接近 GPU 的峰值 FLOPS。确保数据按需要采用 FP16/BF16/TF32，且对齐到 Tensor Core 分块维度（8 或 16 的倍数）。

使用 TF32 轻松加速

对于 32 位矩阵乘法，可设置 `torch.set_float32_matmul_precision("high")`，为 PyTorch 中数值安全的操作启用 TF32（快速 FP32）。cuBLAS、cuDNN 等库会在现代 GPU 硬件上自动选择最佳 Tensor Core 代码路径。若以“highest”（而非“high”）强制完整精度 FP32，请务必了解其性能影响。

利用结构化稀疏性

现代 NVIDIA GPU 支持矩阵乘法的 2:4 结构化稀疏性，即按结构化模式将 50% 的权重置零，使硬件吞吐量翻倍。可通过剪枝模型来利用此特性。若能将权重剪枝至符合 2:4 稀疏模式，则这些层的 GEMM 可快约 2 倍。使用 NVIDIA SDK 或库支持应用结构化稀疏性，并确保采用稀疏 Tensor Core 路径。若模型能容忍或能在这种稀疏性下训练，便可获得近乎免费的加速；这通常需要使用稀疏正则化重新训练。

可行时降低梯度和激活值精度

即使保留较高精度的权重，也可考虑将梯度或激活值压缩至较低精度。例如，对梯度使用 FP16/BF16 或 FP8 通信；许多框架支持 FP16 梯度 all-reduce。同样，在激活检查点中以 16 位而非 FP32 存储激活值可节省内存。针对 FP8、FP4 优化器和量化梯度的研究仍在继续。这些方法可在降低内存和带宽成本的同时保持模型质量。在带宽受限环境中，梯度压缩尤具颠覆性。DeepSeek 曾通过压缩梯度在受限 GPU 上训练来展示这一点。

为推理使用定制量化

部署时应尽可能使用 INT8 量化。GPU 上的 INT8 推理极快且内存效率高。使用 NVIDIA TensorRT 或量化工具将模型量化为 INT8 并完成校准。许多神经网络（如 Transformer）可用 INT8 运行，精度下降可忽略不计；相对 FP16 的加速可达 2–4 倍。在最新 GPU 上，还应探索和评估某些模型的 FP8 或 INT4，以进一步提高推理吞吐量。

可行时融合缩放与计算操作

使用低精度时，记得融合操作以保留精度。例如，Blackwell 的 FP4“微缩放”建议为每组数值保留一个缩放因子。应在一次遍历中完成缩放和计算，而不是分多次遍历，以免造成精度损失。现有库已处理许多此类操作，应直接使用它们而不要从头实现。

高级调优策略与算法技巧

算法层面的改变通常能以减少工作量的方式，在投资回报率（ROI）上胜过硬件升级，而非只是更快地执行同样工作。自动调优、FlashAttention、通信/计算重叠和分片可在减少浪费的同时释放扩展能力。

具体而言，自动调优内核与层参数，替换为融合/FlashAttention 内核，并在分布式训练中将通信与计算重叠。使用流水线/张量并行和 ZeRO 分片扩展深度模型，并考虑异步更新或剪枝/稀疏性，以少量精度代价换取显著吞吐量收益。以下是一些高级性能优化和算法技巧：

自动调优内核参数

针对目标 GPU 自动调优自定义 CUDA 内核。正确的块大小、分块大小、展开因子等都会影响性能，且最佳设置常因 GPU 世代而异，如 Ampere、Hopper、Blackwell 及后续架构。使用自动调优脚本或 OpenAI Triton 等框架，甚至在预处理步骤中进行暴力搜索，以找到最佳启动配置。这可轻松带来静态“合理”设置会错失的 20%–30% 提升。在自动调优循环中使用 Triton 功能，例如设置 `num_warps` 和 `num_stages`、启用自动 warp 专用化，并测试异步 TMA 布局。共享内存暂存优先使用 tensor map descriptor API。迁移到不同硬件时应重新基准测试分块形状，因为最佳选择会随 GPU 世代变化。

在机器学习工作负载中使用内核融合

利用深度学习库提供的融合内核。例如，启用融合优化器会融合权重更新、动量等逐元素操作；还可使用融合多头注意力实现和融合归一化内核。NVIDIA 库以及 Transformer Engine、FasterTransformer 等开源项目为常见模式提供融合操作，如融合的 LayerNorm + dropout。这些操作可减少启动开销并更高效地使用内存。

使用 FlashAttention 等高内存效率注意力机制

为 Transformer 模型集成 FlashAttention 等高级算法。FlashAttention 采用分块、流式方式计算注意力，避免实体化大型中间矩阵，从而显著减少内存使用并提高速度，

尤其适用于长序列。以 FlashAttention 替换标准注意力可同时改善吞吐量和内存占用，使相同硬件能够支持更大的批量或序列长度。

重叠通信与计算

在分布式训练中，应尽可能将网络通信与 GPU 计算重叠。例如，在梯度 all-reduce 中，每层梯度一就绪便异步启动 all-reduce，同时下一层仍在计算反向传播。若实现得当，这种流水线可完全隐藏 all-reduce 延迟。使用异步 NCCL 调用或 PyTorch Distributed Data Parallel（DDP）等原生支持重叠的框架库，确保 GPU 不会闲置等待网络。

为深度模型使用流水线并行

当模型大小迫使你使用张量并行或流水线并行跨 GPU 划分时，可使用足够多的微批次以保持所有流水线阶段忙碌。利用 NVLink/NVSwitch 在阶段间快速传送激活值；通过交错调度重叠并减少流水线气泡。一些框架会自动完成这类调度。NVL72 互连在此尤其有帮助：即使通信密集型流水线阶段也可按数 TB/s 的速度交换数据，最大限度减少流水线停顿。

利用分布式优化器分片

使用 Zero Redundancy Optimizer（ZeRO）等节省内存的优化策略：它将优化器状态、梯度等张量分片到各 GPU，而不是复制到每个 GPU。通过分散内存和通信负载，这种方式可扩展到极大模型。它通过降低每 GPU 内存压力、避免交换到 CPU，并在分块处理时减少通信量来提高吞吐量。DeepSpeed、Megatron-LM 等许多框架提供此类分片。对大模型应加以利用，以在不发生 OOM 或因交换而减速的情况下保持高速度。

可行时异步训练

若适用，可考虑异步更新。例如，可使用陈旧随机梯度下降（SGD），其中工作器并不总是等待彼此共享更新。这种方法可提高吞吐量，但需要仔细调优以免影响收敛。若实现得当，异步训练能带来巨大的性能收益。

引入稀疏性与剪枝

大模型通常存在冗余。训练时使用剪枝技术引入稀疏性，可在推理时利用；若受支持，在训练中也可部分利用。现代 GPU 硬件支持加速的稀疏矩阵乘法（2:4），未来 GPU 很可能会扩展这一能力。即使保持稠密训练、仅为推理剪枝，较小模型也会运行得更快且占用更少内存，从而提高模型部署的成本效益。探索彩票假设、蒸馏或结构化剪枝，在缩减模型大小的同时保持精度。

分布式训练与网络优化

在集群规模下，网络会成为限制因素。若不加处理，网络会破坏线性扩展并推高成本。RDMA/巨型帧、分层集合通信、亲和性和压缩可保护带宽并控制延迟。

可用时使用 RDMA（InfiniBand/RoCE）；若使用以太网，则调优 TCP 缓冲区、启用巨型帧，并选择现代拥塞控制。对齐 NIC/CPU 亲和性，调整 NCCL 线程/缓冲区（以及受支持

时的 SHARP/CollNet），压缩或累积梯度，并测试网络以发现丢包或错误配置。请按以下指导优化多 GPU、多节点模型训练等分布式环境的网络：

可用时使用 RDMA 网络

为多节点集群配备 InfiniBand 或 RoCE，以获得低延迟和高吞吐量。确保 NCCL 和 MPI 在训练中使用 RDMA；NCCL 会自动检测 InfiniBand，并在可用时使用 GPUDirect RDMA。RDMA 绕过内核网络栈，相比传统 TCP 可显著降低延迟。若只有以太网，则在支持 RDMA 的 NIC 上启用 RoCE，以获取类似 RDMA 的性能。在 NVLink 域系统（NVL72、GB200/GB300 等）上，可行时让集合通信保持在互连网络内；主机网络留给跨岛链路。使 NCCL 拓扑提示与 NVLink/NVSwitch 域对齐。

使用以太网时调优 TCP/IP 协议栈

对于基于 TCP 的集群，增大网络缓冲区。提高 `/proc/sys/net/core/{r,w}mem_max` 和自动调优限制（`net.ipv4.tcp_{r,w}mem`），以允许更大的发送/接收缓冲区。这有助于跑满 10/40/100 GbE 链路。在所有节点和交换机上启用巨型帧（MTU 9000），以降低每包开销、提高吞吐量并减少 CPU 使用率。对广域或拥塞网络，还可考虑 BBR 等现代 TCP 拥塞控制。

为 NIC 分配 CPU 亲和性

将网络中断和线程固定到与 NIC 位于同一 NUMA 节点的 CPU 核心。这可避免网络流量的跨 NUMA 代价，并使网络栈的内存访问保持本地化。检查 `/proc/interrupts` 并使用 `irqaffinity` 设置，确保例如 NUMA 节点 0 上的 NIC 由 NUMA 节点 0 的核心处理。这可提高网络性能和一致性，尤其是在高包速率下。

针对环境优化 NCCL 环境变量

针对大型多节点作业试验 NCCL 参数。例如，将每 GPU 的 NCCL CPU 线程数 `NCCL_NTHREADS` 从默认的 4 提高到 8 或 16，可在增加 CPU 使用的代价下驱动更高带宽。将每 GPU 的缓冲区大小 `NCCL_BUFFSIZE` 从默认 1 MB 提高到 4 MB 或更多，可改善大消息吞吐量。若集群使用支持 SHARP 的交换机，请安装 NCCL SHARP 插件并设置 `NCCL_COLLNET_ENABLE=1` 启用 CollNet，然后按文档使用 `SHARP_COLL_LOCK_ON_COMM_INIT=1`、`SHARP_COLL_NUM_COLL_GROUP_RESOURCE_ALLOC_THRESHOLD=0` 等 SHARP 插件变量。只有归约足够大且网络支持 SHARP 卸载时，才应期待加速。

为慢速网络使用梯度累积

若由于扩展了过多由中等性能互连连接的节点而使网络成为瓶颈，可使用梯度累积，执行更少但更大的 all-reduce 操作。在同步前跨若干小批次累积梯度，使 N 个批次只通信一次，而非每批次一次。这以少量额外内存和模型精度调优为代价，显著降低网络开销；当增加更多 GPU 因通信成本而产生递减收益时尤其有帮助。

优化 all-reduce 拓扑

确保为集群拓扑使用最佳 all-reduce 算法。NCCL 会自动选择环形或树形算法；但在混合互连上（每节点 GPU 通过 NVLink 连接、节点间通过 InfiniBand 或以太网连

接），分层 all-reduce 可能有益。分层 all-reduce 会先在节点内执行，再跨节点执行。大多数框架默认会进行基于 NCCL 的分层聚合，但应通过剖析确认。在传统 MPI 设置中，也可考虑手动执行相同的两级归约：先节点内，再节点间。

避免网络过度订阅

在多 GPU 服务器上，应确保 GPU 的合计流量不会使 NIC 过度订阅。例如，八个 GPU 在 all-reduce 期间很容易产生超过 200 Gbps 的流量，只有一张 100 Gbps NIC 会形成限制。若扩展到每节点许多 GPU，可考虑每节点多张 NIC 和 200/400 Gbps InfiniBand。同样，若 NIC 与 GPU 共享同一 PCIe 根复合体，还需注意 PCIe 带宽限制。

压缩通信

与单节点内存相同，可考虑压缩用于网络传输的数据。技术包括 16 位或 8 位梯度压缩、量化跨节点流水线传输的激活值，甚至 sketching 等更特殊的方法。若网络是最慢的组件，那么为压缩/解压数据而增加少量计算成本是值得的。NVIDIA NCCL 原生不压缩，但可在框架中集成压缩（如 Horovod 的梯度压缩或 PyTorch 的自定义 AllReduce hook）。这也是 DeepSeek 成功的一个关键：压缩梯度以应对受限的节点间带宽。

监测网络健康状况

确保没有隐性问题妨碍分布式训练。检查丢包（会表现为重试或超时：在 InfiniBand 上使用重发计数器，在以太网上检查 TCP 重传）。即使少量丢包，因触发拥塞控制也可能严重降低吞吐量。使用带外网络测试（如 iPerf 或 NCCL 测试）验证是否获得预期带宽和延迟；否则，应调查交换机配置、NIC 固件或 CPU 亲和性。

高效推理与服务

服务是成本与延迟之间的博弈：利用率的提高来自编排和批处理，而不仅是更大的 GPU。专用运行时、KV 缓存策略和预热可在不违反 SLO 的情况下保持高吞吐量。

借助自动扩缩容、微服务和动态/连续批处理按需编排，使 GPU 保持繁忙而不违反延迟 SLO。使用专用运行时（vLLM、SGLang、TensorRT-LLM），利用 NIXL 和 KV 缓存卸载实现解耦式服务，预热模型，并隔离资源以控制尾部延迟。请采用以下技术提高模型推理效率和性能：

高效编排动态资源

集成先进的容器编排平台，例如配合自定义性能指标的 Kubernetes。这可根据实时使用模式和吞吐量目标动态扩缩容并平衡工作负载。

为推理采用无服务器架构

探索用于推理工作负载的无服务器架构和微服务设计：它们可高效处理突发流量，并在需求较低时缩容以减少闲置资源开销。

优化批处理与并发

针对推理工作负载，选择合适的批处理策略。优先采用动态或连续批处理，自动将传入请求组成批次。较大的批量通过保持 GPU 忙碌来提高吞吐量，但过大也会增加延迟。若一个流未占用全部 GPU 资源，还可并行运行多个推理流，例如两个并发推理批次以充分使用 GPU SM 和 Tensor Core。

在分布式推理中利用 NIXL

跨 GPU 或节点服务大模型时，使用 NVIDIA Inference Xfer Library 经由 RDMA 在预填充与解码工作器之间流式传输 KV 缓存。对于 NIXL，基于 Transformer 的大型 KV 缓存会在节点之间传输。NIXL 提供高吞吐、低延迟的 API，可在解耦式 LLM 推理集群中将 KV 缓存从预填充 GPU 流式传输到解码 GPU。它使用 GPUDirect RDMA 和最优路径完成此事，不涉及 CPU，从而降低跨节点解耦预填充/解码服务的尾部延迟。

必要时卸载 KV 缓存

若 LLM 的注意力 KV 缓存增长到超出 GPU 内存，请使用分层卸载。NVIDIA Dynamo 的 Distributed KV Cache Manager 可将访问较少的 KV 页卸载到 CPU 内存、SSD 或网络存储；TensorRT-LLM、vLLM 等推理引擎支持分页和量化 KV 缓存。复用缓存可降低内存压力和首 token 延迟。应验证端到端影响，因为卸载后的缓存未命中会引入额外 I/O 延迟。借助快速 NVMe 和计算—I/O 重叠，此法可在原本会超过 GPU 内存的序列上推理，且性能损失很小。若预期超长提示词或聊天，应确保推理服务器已配置使用此机制。卸载到磁盘总比完全失败更好。

高效服务模型

使用经过优化的模型推理系统（如 vLLM、SGLang、NVIDIA Dynamo 和 NVIDIA TensorRT-LLM）以低延迟、高吞吐量服务大模型。它们应实现量化、低精度格式、融合、高度优化的注意力内核及其他技巧，以最大化推理期间的 GPU 利用率。这些库还应处理张量并行、流水线并行、专家并行、上下文并行、推测解码、分块预填充、解耦预填充/解码、动态请求批处理等众多高性能功能。

监测并调优尾部延迟

在实时服务中，平均延迟和（长）尾部延迟（第 99 百分位）都很重要。剖析推理延迟分布；若尾部很高，找出离群值的原因，如意外的 CPU 介入、垃圾回收（GC）暂停或过多上下文切换。将推理服务器进程绑定到特定核心、与嘈杂邻居隔离，并在必要时使用实时调度，以获得更一致的延迟。

预热以避免冷启动延迟

将模型加载到 GPU 并运行几次虚拟推理，以预热 GPU。这样可避免第一条真实请求进入推理服务器时遭遇一次性的冷启动延迟。

为服务质量（QoS）高效划分资源

若在同一基础设施上运行训练和推理等混合异构工作负载，或运行不同架构的模型，应考虑划分资源，以确保延迟敏感的推理得到优先级。这可能意味着将部分 GPU 完全专用于推理，或在服务不需完整 GPU、但需要可预测延迟时，使用 MIG 为推理服务提供

有保障的 GPU 分片。若可能，应将推理与训练部署在不同节点上，因为训练时繁重 I/O 或突发通信可能引入抖动。

利用 Grace CPU 进行推理预处理

在 Grace Blackwell 系统中，服务器级 CPU 可在与 GPU 相同的内存空间内极快地处理分词、批次整理等预处理任务。将此类任务卸载到 CPU，由它在 GPU 可直接使用的共享内存中准备数据。这样可减少缓冲区重复，并利用强大的 CPU 处理推理流水线的一部分，使 GPU 专注于计算更密集的神经网络计算。

为边缘 AI 和延迟关键部署谨慎调优

借助专用边缘加速器，并优化中心服务器与边缘设备之间的数据传输协议，将性能调优延伸至边缘。这有助于为时间敏感型应用实现超低延迟。

多节点推理与服务

将预填充/解码解耦并分片模型，可凭借更高占用率处理更大的上下文和更多用户。连续批处理及分层内存/卸载即使在长提示词和高并发下也能维持流动。

具体而言，在设备间解耦预填充与解码，持续跨请求汇集 token，并通过张量/流水线并行分片超大模型。为超长上下文添加分层内存/卸载，以少量延迟换取大幅更高的容量，从而在不发生 OOM 的情况下服务更多请求。以下性能提示适用于多节点推理与服务：

解耦推理流水线

将推理工作流划分为不同阶段：包括使输入提示词通过全部模型层的“预填充（prefill）”阶段，以及逐 token 生成输出的迭代“解码（decode）”阶段。将这些阶段分配给不同资源，以便独立扩展。这种两阶段方法可避免较快任务受较慢任务瓶颈限制。对大语言模型，一种策略是运行完整模型以编码提示词，然后按阶段处理自回归解码，并可能为每阶段配置专用工作器。通过解耦流水线，可确保 GPU 持续处理它最擅长的任务部分，避免队头阻塞，即一次长生成任务阻塞其后其他任务的情况。

为 LLM 使用连续批处理

超越简单的请求批处理，使用连续批处理策略以最大化高负载下的吞吐量。传统动态批处理将传入请求分组并作为批次处理，以提高 GPU 利用率；连续批处理更进一步，会实时动态合并和拆分跨请求的 token 序列。vLLM 等系统实现 token 池化：任何线程一准备好生成下一个 token，便与其他就绪线程组成新批次。这种方法始终保持 GPU 高占用率，并大幅减少空闲期，尤其在服务许多具有不同序列长度的并发用户时，可获得显著更高的 token 吞吐量和更好延迟一致性。

跨 GPU 和节点高效分片模型

对于大到无法放入单张 GPU 内存的模型，可通过将模型划分到多个 GPU 甚至多台服务器上，采用模型并行推理技术。这可通过张量并行实现：将每层权重和计算划分到各设备；或通过流水线并行实现：将模型层划分为托管在不同 GPU 上的片段，按顺序让数据流经它们。虽然模型分片会因数据在分片间流动而引入通信开销和一些延迟，却可部署原本无法服务的万亿参数模型。确保 GPU 间具有 NVLink、InfiniBand 等高速互

连，并尽可能使通信与计算重叠。关键在于平衡负载，使所有设备并行工作，且没有单一阶段成为瓶颈。

为扩展上下文卸载内存

使用分层内存策略，以支持所需内存超过 GPU 可用内存的推理工作负载。服务超大模型或长序列上下文（如很长的多轮对话和大型文档）时，应加入内存卸载。GPU 内存紧张时，可将访问频率较低的数据（如旧的注意力 KV 缓存条目或不常访问的模型权重）移到 CPU RAM 甚至 NVMe 存储。现代推理框架可自动换出这些张量，并在需要时动态换回。虽然缓存未命中会引入额外延迟，但可防止内存不足错误并处理极端情况。合理地卸载和预取数据，是以少量速度换取服务大工作集请求的能力，从而在内存约束下实现更好的整体吞吐量。

功耗与散热管理

每瓦性能是一等指标：热或功耗降频会抹去调优收益并缩短硬件寿命。功率上限、高效装箱和主动冷却可稳定时钟频率，同时降低能源开销。

除速度外，还应跟踪每瓦性能和温度：为内存受限工作负载限制功率或降低时钟，可在吞吐量损失很小的情况下提高效率。主动管理冷却，整合任务以使 GPU 接近满载运行，监测每 GPU 功耗，并在可降低成本时按能源价格/可再生能源安排作业。以下是管理 AI 系统功耗和散热特性的提示：

可行时使用高效且环保的能源

在跟踪性能的同时跟踪并优化能耗。除管理功率和温度限制外，还应监测能源使用指标，并考虑同时改善性能和可持续性的技术。例如，根据可再生能源可用性实施动态功率上限或迁移工作负载，可降低运营成本和碳足迹。这种双重关注可降低运营成本，并支持负责任、环保的 AI 部署。

监测温度与时钟频率

运行期间密切关注 GPU 温度和时钟频率。若 GPU 接近温度限制（某些情况下为 85°C），它们可能开始降低时钟频率，从而降低性能。使用 `nvidia-smi dmon` 或遥测数据查看时钟是否从最大值下降。若检测到降频，应改善冷却、提高风扇速度、改善气流，或略微降低功率限制以保持稳定的热包络。目标是在不因温度导致波动的情况下实现一致性能。

使用感知能源的动态功耗管理

现代数据中心日益采用感知能源的调度，根据实时能源成本和可再生能源可用性调整工作负载。结合自适应功率上限和动态时钟缩放，可在降低运营成本和碳足迹的同时优化每瓦吞吐量。

针对每瓦性能进行优化

在功率预算受限（或能源成本高）的多 GPU 部署中，应考虑为效率调优。许多工作负载，尤其内存受限的工作负载，可在略微降低 GPU 时钟时运行，性能损失可忽略，功耗却明显降低。例如，若内核受内存限制，将 GPU 锁定在较低时钟可节电而不损害运

行时间，从而提高每瓦吞吐量。使用 `nvidia-smi -pl` 测试若干功率限制，查看每瓦吞吐量是否改善。对某些模型，将功率限制从 100% 降至 80%，可在少用 20% 电力的情况下获得几乎相同的速度。

使用自适应冷却策略

若运行环境的冷却或能源可用性存在变化，应与集群管理集成以调整工作负载。例如，若成本受此影响，可将重型作业安排在一天中较凉爽的时段或可再生能源供应充足时运行。一些场所实施策略，将非紧急作业排队到电价较低的夜间运行。这不会改变单个作业的性能，却能显著降低成本。

整合工作负载

应让 GPU 高利用率运行，而不是让许多 GPU 低利用率运行。就每瓦完成的工作而言，繁忙 GPU 比闲置或轻度使用的 GPU 更节能，因为其基础功耗得到了更好摊销。除非要优化最短墙钟时间，否则让一个 GPU 以 90% 利用率先后运行作业，可能优于两张 GPU 各以 45% 利用率并行运行。应规划调度，在不使用时关闭或闲置整个节点，而非让大量硬件以低利用率运行。

高效配置冷却

对于风冷系统，重负载运行时可考虑将 GPU 风扇设为较高的固定速度，以预先冷却 GPU。一些数据中心始终以最大转速运行风扇以改善一致性。确保数据中心进风温度符合规格；定期检查服务器 GPU 是否有灰尘或阻塞物，堵塞的散热片会大幅降低冷却效率。对于水冷系统，确保流量最优且水温受控。

谨慎监测功耗

使用工具监测每 GPU 功耗。`nvidia-smi` 会报告瞬时功耗，有助于了解工作负载的功率特征。功耗尖峰可能与特定阶段相关：例如，all-reduce 阶段的计算负载和功耗可能较低，而稠密层会使负载和功耗测量出现尖峰。了解这一点后，可调整作业顺序以平滑功耗。在受限供电线路上运行集群时，这一点很重要。在功率受限场景中，可能需避免在同一节点同时运行多个功耗尖峰型作业，以免触发功率限制。

提高长时间运行作业的韧性

若运行数月之久的训练作业或 7×24 推理作业，应考虑温度对硬件寿命的影响。持续以 100% 功率和温度上限运行，随时间推移可能略微增加故障风险。实际中，数据中心 GPU 就是为此类韧性设计的；但若希望更加稳妥，将功率目标设为 90% 可在减速很小的情况下减少组件压力。这是在更长训练时间与更少硬件磨损之间的权衡，尤其当硬件将长期复用于多个项目时。

结语

将这份清单视为可重复使用的行动手册：剖析性能，在正确层级调优正确的瓶颈，并在横向扩展前验证收益。系统地应用这些实践——从操作系统和内核到分布式通信和服务——无论规模大小，你都能构建快速、成本高效且可靠的 AI 系统。

这份清单虽全面，却并非穷尽。随着硬件、软件和算法不断演进，AI 系统性能工程领域也将持续发展。这里列出的并非每项最佳实践都适用于所有情形；但总体而言，它们覆盖了 AI 系统性能工程场景的广泛范围。这些提示凝结了多年优化 AI 系统性能所积累的大量实务智慧。

调优 AI 系统时，应系统地浏览本章列出的每个相关类别，并逐项检查清单。例如，应确保操作系统已调优，确认 GPU 内核高效，检查是否正确使用库，监控数据流水线，优化训练循环，调优推理策略，并平稳地横向扩展。遵循这些最佳实践，你可诊断和解决大多数性能问题，并从 AI 系统中提取最大性能。

还请记住：在大幅扩展集群前，应先在较少节点上进行剖析并识别潜在的规模瓶颈。例如，若在 8 张 GPU 上已看到一次 all-reduce 集合操作占用单次迭代的 20%，那么在更大规模下情况只会更糟，尤其当规模超过单个计算节点或数据中心机架系统（如 Grace Blackwell GB200 和 GB300 NVL72、Vera Rubin VR200 和 VR300 NVL 系统）的容量时。

请随手保留这份清单，并在发现新技巧时不断补充。将这些提示和最佳实践与前文的深入理解结合，你将能设计并运行高效、可扩展、可维护、成本有效且可靠的 AI 系统。

现在就行动起来，把你最宏大的想法变为现实。祝你优化愉快！

索引

符号

#pragma unroll, Exposing Instruction-Level Parallelism, Loop Unrolling, Interleaving, and Compiler Hinting

– unrolling too many iterations, Profiling and Mitigating Register Pressure

100 万亿参数模型, 迈向 100 万亿参数模型

– 机架级 NVIDIA 超级计算机, NVIDIA 的“机架级 AI 超级计算机”-NVIDIA 的“机架级 AI 超级计算机”

– 讨论原因, 迈向 100 万亿参数模型

– 扩展至, AI 辅助性能优化与迈向数百万 GPU 集群的扩展-AI 辅助性能优化与迈向数百万 GPU 集群的扩展

3FS（参见 Fire-Flyer 文件系统（3FS；DeepSeek））

<<< >>> (chevron syntax)

– CUDA 特定启动参数, CUDA 编程回顾

– 线程数量, CUDA 编程回顾

__global__, CUDA Programming Refresher

__launch_bounds__, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds, Techniques for Occupancy Tuning

– occupancy optimization, Compiler Hints to Optimize Occupancy

__ldg(), Read-Only Data Caches

__shared__ regions, Distributed Shared Memory

__shfl_sync, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization-Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization

__syncthreads(), Cooperative Groups

约 6800 亿参数模型, 在美国对华硬件出口限制下，DeepSeek 仍扩展至约 6800 亿参数模型-在美国对华硬件出口限制下，DeepSeek 仍扩展至约 6800 亿参数模型

A

实际占用率, 保持高占用率和 GPU 利用率, 检查实际占用率和 GPU 利用率

- 检查, 检查实际占用率和 GPU 利用率-优化内核, 使用原子工作队列进行动态调度
  - 内核计算吞吐量与 GPU FLOPS 对比, 内核计算吞吐量与 GPU 峰值 FLOPS 对比
  - 内核内存吞吐量与 HBM 对比, 内核内存吞吐量与峰值 HBM 内存带宽对比
- 提高并行度, 保持高占用率和 GPU 利用率

激活检查点, 用于节省内存的激活检查点

- FSDP 自动检查点, FSDP 自动检查点与卸载-FSDP 自动检查点与卸载

自适应批处理, 自适应批处理与分块预填充调度-自适应批处理与分块预填充调度

自适应路由, 自适应专家路由与实时监控, 使用 GPUDirect RDMA 的多节点、多机架通信

准入控制（参见 早期拒绝）

自我改进的 AI 智能体, 自我改进 AI 智能体（AI Futures Project）-自我改进 AI 智能体（AI Futures Project）

通过 NVL72 构建块构建 AI 工厂, 共封装光学器件：网络硬件的未来

AI Futures Project, Self-Improving AI Agents (AI Futures Project)-Self-Improving AI Agents (AI Futures Project)

AI 优化 AI 系统, AI 辅助的实时系统优化与集群运维-AI 辅助的实时系统优化与集群运维

AI supercomputers in a rack (NVIDIA), NVIDIA’s “AI Supercomputer in a Rack”-NVIDIA’s “AI Supercomputer in a Rack”

- about, AI System Hardware Overview
- cache coherency, The CPU and GPU Superchip
- CPU + GPU superchip, The CPU and GPU Superchip-Streaming Multiprocessor, Threads, and Warps
- GPU Tensor Cores and Transformer Engine, NVIDIA GPU Tensor Cores and Transformer Engine-NVIDIA GPU Tensor Cores and Transformer Engine
- many GPUs as one, Ultrascale Networking Treating Many GPUs as One-Co-Packaged Optics: Future of Networking Hardware
- multi-GPU programming, Multi-GPU Programming-Multi-GPU Programming
- multirack and storage communication, Multirack and Storage Communication
- NVIDIA Blackwell dual-die GPU, NVIDIA Blackwell “Dual-Die” GPU-NVIDIA Blackwell “Dual-Die” GPU
- NVIDIA Grace CPU, NVIDIA Grace CPU
- NVL72, Ultrascale Networking Treating Many GPUs as One

- NVLink and NVSwitch, NVLink and NVSwitch-NVLink and NVSwitch
- SHARP for in-network aggregations, In-Network Aggregations with NVIDIA SHARP-In-Network Aggregations with NVIDIA SHARP
- streaming multiprocessors, threads, warps, Streaming Multiprocessor, Threads, and Warps-Streaming Multiprocessor, Threads, and Warps

AI 系统性能工程师, AI 系统性能工程师
- 基准测试与性能分析, 基准测试与性能分析
- 跨团队协作, 跨团队协作
- AI 的未来能力, AI 辅助性能优化与迈向数百万 GPU 集群的扩展
- 产生远高于成本的回报, 衡量“Goodput”：有效吞吐量
- 高效管理资源, 高效管理资源
- 概览, AI 系统性能工程师
- 性能监控, 实际性能监控与利用率
  - GPU 接近 100% 利用率, 实际性能监控与利用率
  - NVLink 使用情况, 实际性能监控与利用率
- 扩展分布式训练与推理, 扩展分布式训练与推理
- 透明度与可复现性, 透明度与可复现性

air cooling versus liquid cooling, Liquid Cooling Versus Air Cooling-Liquid Cooling Versus Air Cooling

all-to-all communication, Expert Communication Optimization
- mixture-of-experts models, Expert Communication Optimization

AlphaTensor (DeepMind), AlphaTensor AI-Discovered Algorithms Boosting GPU Performance (Google DeepMind)

Amazon FSx for Lustre, Distributed, Parallel Filesystems and Object Stores
- performance improvement verification, Distributed, Parallel Filesystems and Object Stores

Amazon S3, Distributed, Parallel Filesystems and Object Stores

AMP（参见 自动混合精度（AMP；PyTorch））

AOT Autograd, AOT Autograd Fusion for Forward and Backward Passes
- FX Graph, TorchDynamo for Bytecode Capture and Graph Extraction, AOT Autograd Fusion for Forward and Backward Passes

  - PyTorch compiler pipeline, PyTorch Compiler Deep Dive

AOTInductor (PyTorch), TorchInductor Backend Code Generation

application performance management (APM), Monitoring System Metrics and Counters

application-level inference optimizations, Application-Level Optimizations-Token Output Limits and Timeouts

  - debouncing and request coalescing, Debouncing and Request Coalescing
  - model cascading and tiered deployment, Model Cascading and Tiered Model Deployment-Model Cascading and Tiered Model Deployment
  - prefix caching, Prefix Caching-Prefix Caching
  - prompt cleansing, Prompt Cleansing
  - prompt compression, Prompt Compression
  - streaming responses, Streaming Responses-Streaming Responses
  - token output limits and timeouts, Token Output Limits and Timeouts

Apptainer, Reduce Image Size for Faster Container Startup

architecture of GPUs, Understanding GPU Architecture-CUDA GPU Backward and Forward Compatibility Model

  - maximum threads per SM and block, Tuning Occupancy with Launch Bounds
  - memory hierarchy, Understanding GPU Memory Hierarchy-Understanding GPU Memory Hierarchy
  - parallelism, Understanding GPU Architecture
    - cuTile, C++ and Python CUDA Libraries
    - Data Parallel versus Distributed Data Parallel, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
    - fully sharded data parallel, Distributed Data Parallel Strategies
    - PTX, Inline PTX and SASS Tuning for Microoptimizations
  - Special Function Unit, Understanding GPU Architecture
  - streaming multiprocessors, Understanding GPU Architecture
    - warps, Streaming Multiprocessor, Threads, and Warps, Understanding GPU Architecture-Understanding GPU Architecture
  - thread block clusters, Threads, Warps, Blocks, and Grids

- thread blocks, Threads, Warps, Blocks, and Grids-Threads, Warps, Blocks, and Grids
  - thread block size, Choosing Threads-per-Block and Blocks-per-Grid Sizes
- threads, Threads, Warps, Blocks, and Grids-Threads, Warps, Blocks, and Grids
- threads-per-block and blocks-per-grid, Choosing Threads-per-Block and Blocks-per-Grid Sizes-Choosing Threads-per-Block and Blocks-per-Grid Sizes

arithmetic intensity, Roofline Model: Compute-Bound or Memory-Bound Workloads, Increasing CUDA Kernel Efficiency and Arithmetic Intensity
- batching increasing, Structured Sparsity
- CUTLASS for optimal, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance-Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance
- data reuse increasing, Multilevel Microtiling and Software Prefetching
- distributed shared memory increasing, Distributed Shared Memory
- increasing, Increasing CUDA Kernel Efficiency and Arithmetic Intensity, Kernel Fusion-Kernel Fusion
- kernel fusion increasing, Kernel Fusion-Kernel Fusion
- low versus high arithmetic intensity, Increasing CUDA Kernel Efficiency and Arithmetic Intensity
- prefill and decode phase differences, Adaptive Batching and Chunked Prefill Scheduling
- pruning increasing, Structured Sparsity
- PyTorch, PyTorch and Arithmetic Intensity
- recomputation of cheap expressions increasing, Recomputation Versus Memory Trade-Off
- Roofline performance model, Increasing CUDA Kernel Efficiency and Arithmetic Intensity-Increasing CUDA Kernel Efficiency and Arithmetic Intensity, Roofline-Guided Scheduling and Orchestration Decisions
  - lower precision with Tensor Cores, Mixed Precision and Utilizing Tensor Cores
- software prefetching, Multilevel Microtiling and Software Prefetching
- tiling increasing, Tiling and Data Reuse Using Shared Memory, Multilevel Microtiling and Software Prefetching

    - multilevel tiling, Multilevel Microtiling and Software Prefetching
    - software prefetching, Multilevel Microtiling and Software Prefetching

asynchronous execution with streams, Asynchronous Execution with Streams

asynchronous memory transfers with CUTLASS, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance

AsyncTP (asynchronous tensor parallel), TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP

aten operations

- FX Graph, TorchDynamo for Bytecode Capture and Graph Extraction
- PrimTorch IR decomposing, PrimTorch IR (Prims) Simplified Operator Set
- PyTorch compiler pipeline, PyTorch Compiler Deep Dive

atomic work queues, Dynamic Scheduling with Atomic Work Queues-Atomic Queues

- atomic counters, Atomic Counters
    - Nsight Compute, Atomic Counters-Atomic Counters
- atomic queues, Atomic Queues-Atomic Queues
    - dynamic work allocation, Atomic Counters
- CUDA Graphs device initiated, Atomic Queues and Device-Initiated CUDA Graphs for In-Kernel Persistent Scheduling

attention

- AI-produced CUDA kernel for attention, Automated GPU Kernel Optimizations with DeepSeek-R1 (NVIDIA)-Automated GPU Kernel Optimizations with DeepSeek-R1 (NVIDIA)
- bottleneck fixed by FlashAttention, Mechanical Sympathy: Hardware-Software Codesign
- context parallelism communicating across partitions, Context (Sequence) Parallelism
- PyTorch optimized attention mechanisms, PyTorch Optimized Attention Mechanisms

automated performance tests, Benchmarking and Profiling

- AI optimizing AI systems, AI-Assisted Real-Time System Optimizations and Cluster Operations-AI-Assisted Real-Time System Optimizations and Cluster Operations

automatic mixed precision (AMP; PyTorch), TF32 and Automatic Mixed Precision (PyTorch)-TF32 and Automatic Mixed Precision (PyTorch)

AutoParallel (PyTorch), TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP

autotuning

- AI-assisted autotuning, AI-Assisted Real-Time System Optimizations and Cluster Operations-AI-Assisted Real-Time System Optimizations and Cluster Operations
- static kernels transformed into adaptive, Kernel Autotuning for Transformer Self-Attention and MLP Paths
- TorchInductor, Using the PyTorch Compiler, Autotuning with TorchInductor-Autotuning with TorchInductor
- transformer self-attention and MLP paths, Kernel Autotuning for Transformer Self-Attention and MLP Paths-Kernel Autotuning for Transformer Self-Attention and MLP Paths
- Triton kernels, Autotuning Triton Kernels

B

B200 (NVIDIA Blackwell), Introduction and AI System Overview, NVIDIA Blackwell “Dual-Die” GPU, Choosing Threads-per-Block and Blocks-per-Grid Sizes, Understanding GPU Memory Hierarchy

B300 (NVIDIA Blackwell Ultra), Toward 100-Trillion-Parameter Models, Blackwell Ultra and Grace Blackwell Ultra

CPU-GPU 内存数据传输的 bandwidthTest, NUMA 友好的内存分配与锁页内存

批处理

- 自适应批处理, 自适应批处理与分块预填充调度-自适应批处理与分块预填充调度
- 提高算术强度, 结构化稀疏性
- 比较静态、动态与连续批处理, 连续批处理
- 连续批处理, 连续批处理
- 动态批处理, 动态批处理-动态批处理
- 静态批处理, 动态批处理

基准测试

- AI 系统性能工程师, 基准测试与性能分析
- bandwidthTest, NUMA-Friendly Memory Allocation and Memory Pinning
- CUDA Graph 的影响, 使用 CUDA 流捕获 CUDA Graph

- 数据流水线, 使用 NVIDIA DALI 进行多模态数据处理
  - I/O 持续性能分析与调优, 持续性能分析与调优工作流-持续性能分析与调优工作流
- 动态并行, 动态并行
- GPU 时钟, GPU 时钟频率与 ECC
- TorchInductor 生成的内核, 编译器性能问题的性能分析与调试
- MLPerf, Transparency and Reproducibility
- 性能基准测试, 持续集成与性能基准测试-性能基准与 MLPerf 日志记录
  - PyTorch 性能平视显示器, 持续集成与性能基准测试-PyTorch HUD 性能仪表板

带合并的最佳适配（BFC）, 动态切换内存分配（slab、缓存与流序分配）

BF16, BF16/FP16, FP8, and FP4 Reduced Precision

Blackwell GPU (NVIDIA)

- 100 万亿参数模型, 迈向 100 万亿参数模型
- 架构
  - B200, Choosing Threads-per-Block and Blocks-per-Grid Sizes, Understanding GPU Memory Hierarchy
  - B300, Blackwell Ultra and Grace Blackwell Ultra
  - FP32 与 INT32 流水线合并, Warp 调度与双发射指令
- Blackwell B200 GPU, NVIDIA Blackwell “Dual-Die” GPU, Choosing Threads-per-Block and Blocks-per-Grid Sizes, Understanding GPU Memory Hierarchy
  - appearing as one GPU, NVIDIA Blackwell “Dual-Die” GPU
  - chiplet approach, NVIDIA Blackwell “Dual-Die” GPU
- Data Center GPU Manager, Performance Monitoring and Utilization in Practice
  - I/O monitoring, Monitoring Storage I/O
  - inference monitoring, Monitoring System Metrics and Counters
- 双裸片 GPU, NVIDIA Blackwell“双裸片”GPU-NVIDIA Blackwell“双裸片”GPU
  - 显示为一张 GPU, NVIDIA Blackwell“双裸片”GPU
  - 小芯片方法, NVIDIA Blackwell“双裸片”GPU
  - 远程内存访问延迟, NUMA 感知与 CPU 绑核
- FP32 80 TFLOP theoretical, Tiling and Data Reuse Using Shared Memory

- 全局内存访问, 向量化内存访问
- ILP 配置, ILP 与占用率
- 将多 GPU 视作一体, 将多块 GPU 视作一体的超大规模网络-共封装光学器件：网络硬件的未来
  - 多 GPU 编程, 多 GPU 编程-多 GPU 编程
    - （另请参见 多块 GPU）
  - 多机架与存储通信, 多机架与存储通信
  - 用于网络内聚合的 SHARP, 使用 NVIDIA SHARP 进行网络内聚合-使用 NVIDIA SHARP 进行网络内聚合
- 内存, CPU 与 GPU 超级芯片
- NVL72, Ultrascale Networking Treating Many GPUs as One
  - NVLink and NVSwitch, NVLink and NVSwitch-NVLink and NVSwitch
- 屋顶线模型, 屋顶线模型：计算受限或内存受限的工作负载-屋顶线模型：计算受限或内存受限的工作负载
- 具有 32 个 bank 的共享内存, 避免共享内存 bank 冲突
- 流式多处理器、线程与 warp, 流式多处理器、线程与 Warp-流式多处理器、线程与 Warp
- TFLOPS, Tiling and Data Reuse Using Shared Memory

Blackwell HBM3e, NVIDIA Blackwell “Dual-Die” GPU

Blackwell NVL72（参见 NVL72）

Blackwell Ultra B300

- 100 万亿参数模型, 迈向 100 万亿参数模型
- NVL72 直接替换升级, Blackwell Ultra 与 Grace Blackwell Ultra

blocksPerGrid, CUDA Programming Refresher, CUDA Programming Refresher

- launch parameter, Configuring Launch Parameters: Blocks per Grid and Threads per Block-Configuring Launch Parameters: Blocks per Grid and Threads per Block

BlueField-3 DPU (NVIDIA), Multirack and Storage Communication

在线图书补充材料, 使用代码示例

图书网页, 如何联系我们

瓶颈

- 经 PyTorch 优化的注意力机制, PyTorch 优化的注意力机制
- 通信与计算对比, 诊断通信受限与计算受限的工作负载
- MoE 中的专家过载, 专家并行
- GPU bottlenecks, Profiling and Diagnosing GPU Bottlenecks-Profiler-Guided Analysis
  - about, Profiling and Diagnosing GPU Bottlenecks
  - data pipeline profiling and tuning, Profiling and Tuning the Data Pipeline
  - iteratively profiling, Iteratively Profiling and Determining the Kernel Bottleneck-Iteratively Profiling and Determining the Kernel Bottleneck
  - kernel optimization, Optimizing the Kernel-Optimizing the Kernel
  - Nsight Compute and Roofline analysis, Nsight Compute and Roofline Analysis
  - Nsight Systems timeline view, Profiling and Diagnosing GPU Bottlenecks
  - PyTorch profiler via Kineto, PyTorch Profiler and Visualization Tools-Profiler-Guided Analysis
  - warp stall reasons, Analyzing Warp Stall Reasons with Nsight Compute-Other Stall Reasons
- 识别瓶颈, 基准测试与性能分析
- I/O continuous profiling and tuning, Continuous Profiling and Tuning Workflow-Continuous Profiling and Tuning Workflow
- NFS servers, Distributed, Parallel Filesystems and Object Stores
  - tuning parameters, Distributed, Parallel Filesystems and Object Stores
- NVIDIA 机械同理心, 机械同理心：软硬件协同设计
- 性能分析
  - example MoE model, Profiling PyTorch to Identify Bottlenecks-CPU and GPU Profiling with Linux perf
  - Linux perf on CPU and GPU, CPU and GPU Profiling with Linux perf-CPU and GPU Profiling with Linux perf
  - Nsight Systems and NVTX Timelines, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines
  - NVTX markers in profiling tools, NVTX Markers and Profiling Tools-NVTX Markers and Profiling Tools

    - tools for profiling, NVTX Markers and Profiling Tools-NVTX Markers and Profiling Tools
  - 共享文件系统, 分布式并行文件系统与对象存储
  - 通过 SHARP 与自适应路由最小化, 使用 GPUDirect RDMA 的多节点、多机架通信

分桶（PyTorch）, 降低通信频率与数据量

蝶形调度, 专家通信优化

C

C++

- CUDA C++ Pipeline API, Asynchronous Memory Prefetching and Tensor Memory Accelerator-Asynchronous Memory Prefetching and Tensor Memory Accelerator
  - hardware-software codesign, Asynchronous Memory Prefetching and Tensor Memory Accelerator
- libraries in CUDA Toolkit, C++ and Python CUDA Libraries

C 状态与 CPU 频率, CPU 频率与 C-states

缓存一致性, CPU 与 GPU 超级芯片

- NVLink-C2C, CPU 与 GPU 超级芯片, 多 GPU 编程

缓存机制

- 缓存 CUDA 内核, 使用时间序列预测持续预热 CUDA Graph 与缓存
- 缓存命中与未命中指标, 推理性能的性能分析、调试与调优
- 前缀缓存, 前缀缓存-前缀缓存
- 预热缓存, 使用时间序列预测持续预热 CUDA Graph 与缓存-使用时间序列预测持续预热 CUDA Graph 与缓存
- read-only data caches, Read-Only Data Caches-Read-Only Data Caches
  - const __restrict__, Read-Only Data Caches, Read-Only Data Caches, Read-Only Data Caches
  - pitfall of not stating read-only, Read-Only Data Caches
- vLLM LMCache, Profiling, Debugging, and Tuning Inference Performance, Zero-Copy GPU-to-GPU Transfer

GPU 与 CPU 之间的回调函数, 使用事件与回调实现细粒度同步

CC（计算能力）与 CUDA Toolkit 版本, CUDA 工具包与运行时

CDU (Coolant Distribution Unit), Liquid Cooling Versus Air Cooling

CFQ（完全公平队列；Linux）已废弃, 为吞吐量调优 NVMe 与文件系统

CFS（完全公平调度器；Linux）, 调度器与中断亲和性

ChatGPT（OpenAI）有状态对话, 前缀缓存

AI 系统性能检查清单

- 高级调优策略, 高级调优策略与算法技巧-高级调优策略与算法技巧
- 算法技巧, 高级调优策略与算法技巧-高级调优策略与算法技巧
- 算术优化, 算术优化与低精度/混合精度-算术优化与低精度/混合精度
- 成本优化, 性能调优与成本优化思维-性能调优与成本优化思维
- CUDA 调优优化, GPU 编程与 CUDA 调优优化-GPU 编程与 CUDA 调优优化
- 数据处理流水线, 数据处理流水线-数据处理流水线
- GPU 编程, GPU 编程与 CUDA 调优优化-GPU 编程与 CUDA 调优优化
- GPU 资源管理与调度, GPU 资源管理与调度-GPU 资源管理与调度
- I/O 优化, I/O 优化-I/O 优化
- 推理与服务, 高效推理与服务-高效推理与服务
    - 多节点, 多节点推理与服务-多节点推理与服务
- 内核调度与执行优化, 内核调度与执行优化-内核调度与执行优化
- 多 GPU 扩展与互连优化, 多 GPU 扩展与互连优化-多 GPU 扩展与互连优化
- 网络优化, 分布式训练与网络优化-分布式训练与网络优化
- operating system and driver optimizations, Operating System and Driver Optimizations-Operating System and Driver Optimizations
- performance profiling, debugging, and monitoring, Performance Profiling, Debugging, and Monitoring-Performance Profiling, Debugging, and Monitoring
- power management, Power and Thermal Management-Power and Thermal Management
- reduced/mixed precision, Arithmetic Optimizations and Reduced/Mixed Precision-Arithmetic Optimizations and Reduced/Mixed Precision
- reproducibility and documentation, Reproducibility and Documentation Best Practices-Reproducibility and Documentation Best Practices
- system architecture and hardware planning, System Architecture and Hardware Planning-System Architecture and Hardware Planning

- thermal management, Power and Thermal Management-Power and Thermal Management
- unified CPU-GPU “Superchip” architecture, Unified CPU-GPU “Superchip” Architecture-Unified CPU-GPU “Superchip” Architecture

checkpoints written to disk, Filesystem Caching and Write-Back

chevron syntax (<<< >>>)

- CUDA-specific launch parameters, CUDA Programming Refresher
- how many threads, CUDA Programming Refresher

子内核启动上限（CUDA）, 动态并行

双裸片 GPU 的小芯片方法, NVIDIA Blackwell“双裸片”GPU

- 显示为一张 GPU, NVIDIA Blackwell“双裸片”GPU

使用 Perfetto 查看器进行 Chrome 跟踪, NVTX 标记与性能分析工具, NVTX 标记与性能分析工具

分块预填充, 无停顿调度（分块预填充）, 自适应批处理与分块预填充调度-自适应批处理与分块预填充调度

CI（参见 持续集成（CI））

云存储缓存, 分布式并行文件系统与对象存储

- 性能改进验证, 分布式并行文件系统与对象存储

集群池, 解耦的预填充与解码集群池-KV 缓存的内存管理

集群调度器, 共享与调度

可移植的最大集群大小, 分布式共享内存

共封装光学器件（CPO）, 共封装光学器件：网络硬件的未来

CollNet 通信算法, NCCL 通信算法

通信（参见 分布式网络通信）

编译器

- 常用编译器, NVIDIA 软件栈
- 编译与编写自定义内核的比较, 编译还是编写自定义内核
- CUDA 未提供的 float8, 向量化内存访问
- __global__, CUDA Programming Refresher
- 暴露指令级并行性, 暴露指令级并行性-分析并缓解寄存器压力
  - 编译器提示, 暴露指令级并行性

- __launch_bounds__, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds, Techniques for Occupancy Tuning
  - occupancy optimization, Compiler Hints to Optimize Occupancy
- nvcc CUDA 编译器, CUDA Toolkit and Runtime
  - （另请参见 nvcc CUDA 编译器）
- 谓词化, 避免 Warp 分歧的技术
- PTX, CUDA GPU Backward and Forward Compatibility Model, TorchInductor Backend Code Generation
  - CUDA compatibility across GPU generations, CUDA Forward and Backward Compatibility Across GPU Hardware Generations, CUDA GPU Backward and Forward Compatibility Model
  - DeepSeek memory allocation optimization, DeepSeek’s Use of Inline PTX for Memory Allocation Optimization
  - inline PTX code, Inline PTX and SASS Tuning for Microoptimizations-Inline PTX and SASS Tuning for Microoptimizations
  - JIT compiling, CUDA Forward and Backward Compatibility Across GPU Hardware Generations
  - streaming assembler, Inline PTX and SASS Tuning for Microoptimizations-Inline PTX and SASS Tuning for Microoptimizations
  - streaming assembler changing over generations, Inline PTX and SASS Tuning for Microoptimizations
  - TorchInductor, TorchInductor Backend Code Generation
  - Triton, Triton Programming Model
- PyTorch 编译器, PyTorch and Higher-Level AI Frameworks, PyTorch Compiler (torch.compile)
  - （另请参见 PyTorch 编译器（torch.compile））
- 智能编译器, 智能编译器与自动化代码优化-智能编译器与自动化代码优化
- Triton, C++ and Python CUDA Libraries
  - （另请参见 Triton（OpenAI））
- 向量化内存访问, 向量化内存访问-向量化内存访问
  - 向量化内存操作, 向量化内存访问

完全公平队列（CFQ；Linux）已废弃, 为吞吐量调优 NVMe 与文件系统

完全公平调度器（CFS；Linux）, 调度器与中断亲和性

压缩数据, 调优、复制与压缩数据

计算能力（CC）与 CUDA Toolkit 版本, CUDA 工具包与运行时

Compute Sanitizer (NVIDIA), Debugging Functional Correctness with NVIDIA Compute Sanitizer

- compute-sanitizer CLI, Debugging Functional Correctness with NVIDIA Compute Sanitizer
  - NVIDIA Tools Extension support, Debugging Functional Correctness with NVIDIA Compute Sanitizer
- concurrency and synchronization, Using CUDA Streams with MoE Models
- four primary tools, Debugging Functional Correctness with NVIDIA Compute Sanitizer
- inference correctness, Debugging Correctness Issues

计算受限导致的 GPU 停顿, 迭代分析并确定内核瓶颈-迭代分析并确定内核瓶颈

- 内核优化, 优化内核-优化内核

compute-sanitizer CLI, Debugging Functional Correctness with NVIDIA Compute Sanitizer

- NVIDIA Tools Extension support, Debugging Functional Correctness with NVIDIA Compute Sanitizer

使用 CUDA 流实现并发, 使用 CUDA 流实现并发-在 MoE 模型中使用 CUDA 流

- 计算与数据传输重叠, 使用流使计算与数据传输重叠-使用流使计算与数据传输重叠
  - 并发, 通信与计算重叠-通信与计算重叠
- 在目标流上启动, 在 MoE 模型中使用 CUDA 流
- 混合专家模型, 在 MoE 模型中使用 CUDA 流
- NVIDIA Compute Sanitizer, 在 MoE 模型中使用 CUDA 流
- 引入并发时进行性能分析, 在 MoE 模型中使用 CUDA 流
- 使用事件进行流同步, 使用事件进行流同步-使用事件进行流同步

const __restrict__, Read-Only Data Caches, Read-Only Data Caches, Read-Only Data Caches

“Contact” (Phish), Acknowledgments

配合 NVIDIA Container Toolkit 的 containerd, NVIDIA 容器运行时

containers, Container Runtime Optimizations for GPUs

- NVIDIA Container Toolkit, Container Runtime Optimizations for GPUs

    - CUDA compatibility, NVIDIA Container Toolkit and CUDA Compatibility
    - GPU performance, Container Runtime Optimizations for GPUs
  - runtime optimizations for GPUs, Container Runtime Optimizations for GPUs-Reduce Image Size for Faster Container Startup
    - container overlay filesystem overhead, Avoiding Container Overlay Filesystem Overhead
    - NVIDIA container runtime injecting libraries, NVIDIA Container Runtime
    - NVIDIA Container Toolkit and CUDA compatibility, NVIDIA Container Toolkit and CUDA Compatibility
    - reducing image size, Reduce Image Size for Faster Container Startup

上下文并行（CP）, 服务超大 MoE 模型的并行策略, 上下文（序列）并行
  - 上下文并行（PyTorch）, PyTorch 优化的注意力机制
  - 混合并行, 混合并行

连续批处理, 连续批处理

持续集成（CI）
  - Computer Sanitizer integrated, Debugging Functional Correctness with NVIDIA Compute Sanitizer
  - 性能基准测试, 持续集成与性能基准测试-性能基准与 MLPerf 日志记录
    - 未捕获完整图时强制报错, 图中断与 TorchDynamo explain()
    - PyTorch 性能平视显示器, 持续集成与性能基准测试-PyTorch HUD 性能仪表板

持续预热, 使用时间序列预测持续预热 CUDA Graph 与缓存-使用时间序列预测持续预热 CUDA Graph 与缓存

连续调度, 连续调度

冷却液分配单元（CDU）, 液冷与风冷对比

风冷与液冷对比, 液冷与风冷对比-液冷与风冷对比

cooperative groups (CGs), Cooperative Groups-When to Combine Persistent Kernels and Cooperative Groups
  - Cooperative Groups API, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization, Cooperative Groups
    - coordinating thread block clusters, Coordinating Thread Block Clusters with Cooperative Groups API-Coordinating Thread Block Clusters with Cooperative Groups API

  - described, Thread Block Clusters and Distributed Shared Memory
  - launching a kernel in cooperative mode, Cooperative Groups
    - grid size for launch, Cooperative Groups
    - reserving all SMs in the grid, When to Combine Persistent Kernels and Cooperative Groups
  - persistent kernels via grid sync, Cooperative Grid Synchronization and Persistent Kernels-Cooperative Grid Synchronization and Persistent Kernels
    - when to combine, When to Combine Persistent Kernels and Cooperative Groups
  - thread block clusters contrasted with, Thread Block Clusters and Distributed Shared Memory

Cooperative Groups API (CUDA), Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization, Cooperative Groups
  - coordinating thread block clusters, Coordinating Thread Block Clusters with Cooperative Groups API-Coordinating Thread Block Clusters with Cooperative Groups API

协作线程阵列（CTA）, Threads, Warps, Blocks, and Grids
  - （另请参见 线程块）
  - 运行时操作绑定, Optimized KV Cache Memory Layout

推理中的正确性问题, 调试正确性问题-调试正确性问题

成本
  - AI 系统性能工程师创造回报, 衡量“Goodput”：有效吞吐量
  - 成本优化概览, 性能调优与成本优化思维-性能调优与成本优化思维
  - 模型参数数量爆炸, 引言与 AI 系统概览
    - 100 万亿参数模型, 迈向 100 万亿参数模型
  - 训练
    - DeepSeek-R1, Introduction and AI System Overview
    - DeepSeek-V3, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China
    - Google Gemini Ultra, Introduction and AI System Overview
    - OpenAI GPT-4, Introduction and AI System Overview

CP（参见 上下文并行（CP））

CPO（共封装光学，co-packaged optics）, Co-Packaged Optics: Future of Networking Hardware

CPU + GPU 超级芯片, The CPU and GPU Superchip-Streaming Multiprocessor, Threads, and Warps

- NVIDIA Blackwell 双裸片 GPU, NVIDIA Blackwell “Dual-Die” GPU-NVIDIA Blackwell “Dual-Die” GPU
  - 呈现为一个 GPU, NVIDIA Blackwell “Dual-Die” GPU
- NVIDIA GPU Tensor Core 和 Transformer Engine, NVIDIA GPU Tensor Cores and Transformer Engine-NVIDIA GPU Tensor Cores and Transformer Engine
- NVIDIA Grace CPU, NVIDIA Grace CPU
- NVLink-C2C, The CPU and GPU Superchip
- 流式多处理器、线程和 warp, Streaming Multiprocessor, Threads, and Warps-Streaming Multiprocessor, Threads, and Warps

CPU 和操作系统配置, Configuring the CPUs and OS for GPU Environments-Tune Host CPU Memory Allocator

- 概述, Configuring the CPUs and OS for GPU Environments
- CPU 频率和 C 状态, CPU Frequency and C-states
- 文件系统缓存和回写, Filesystem Caching and Write-Back
- NUMA 感知和 CPU 绑定, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
  - CPU + GPU 超级芯片的 CPU 到 GPU 数据传输, NUMA Awareness and CPU Pinning
  - CPU 绑定, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
  - NUMA 节点, NUMA Awareness and CPU Pinning
- NUMA 友好的内存分配和固定, NUMA-Friendly Memory Allocation and Memory Pinning-NUMA-Friendly Memory Allocation and Memory Pinning
  - 固定内存的效率, NUMA-Friendly Memory Allocation and Memory Pinning
  - 最大锁定内存设置, Transparent Hugepages
  - 操作系统对固定内存的限制, NUMA-Friendly Memory Allocation and Memory Pinning
  - 数据加载器的固定内存, NUMA-Friendly Memory Allocation and Memory Pinning

- 调度器和中断亲和性, Scheduler and Interrupt Affinity
  - irqbalance 守护进程, Scheduler and Interrupt Affinity
- 透明大页, Transparent Hugepages
- 调优主机 CPU 内存分配器, Tune Host CPU Memory Allocator
- 虚拟内存和交换, Virtual Memory and Swapping

CPU DRAM

- CPU pinning, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
- CPU-GPU memory data-transfer bandwidth, NUMA-Friendly Memory Allocation and Memory Pinning, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
  - NIXL GPU-to-GPU direct transfer, Separate Prefill and Decode Inference Stages
- CUDA programming flow, Understanding GPU Architecture
- CUDA Unified Memory, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
  - programming CUDA, Unified Memory-Unified Memory
  - Unified CPU-GPU Memory, The CPU and GPU Superchip
- GDS, NUMA-Friendly Memory Allocation and Memory Pinning, Using NVIDIA GDS-Measuring GDS with gdsio, Offloading Parameters to CPU and NVMe
- host CPU memory allocator tuned, Tune Host CPU Memory Allocator
- large slower extension of GPU memory, The CPU and GPU Superchip
  - superchips’ high-speed extension of GPU memory, Topology Awareness in NCCL
- NIXL offloading KV cache to, NVIDIA’s NIXL and Disaggregated Inference, KV Cache Offloading with NIXL-KV Cache Offloading with NIXL
- parameters offloaded to, Offloading Parameters to CPU and NVMe
- peer-to-peer DMA avoiding, Enabling Peer-to-Peer DMA and UCX
- swapping, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling

CPU host and GPU device communication via CUDA events, Fine-Grained Synchronization with Events and Callbacks

  - callback function, Fine-Grained Synchronization with Events and Callbacks

CPython Frame Evaluation API, TorchDynamo for Bytecode Capture and Graph Extraction

CTA（协作线程阵列）, Threads, Warps, Blocks, and Grids

  - （另请参见 线程块）

CTRL approach to prompt trimming, Prompt Cleansing

CUBIN, CUDA Forward and Backward Compatibility Across GPU Hardware Generations

cuBLAS

  - CUDA Toolkit, CUDA Toolkit and Runtime, PyTorch and Higher-Level AI Frameworks
  - PyTorch delegating tasks to, PyTorch and Higher-Level AI Frameworks

CUDA

  - 子内核启动上限, Dynamic Parallelism
  - 跨 GPU 世代的兼容性, CUDA Forward and Backward Compatibility Across GPU Hardware Generations, CUDA GPU Backward and Forward Compatibility Model
  - 启动 CUDA Graph, Capturing a CUDA Graph with a CUDA Stream-Capturing a CUDA Graph with a CUDA Stream
  - 未提供 float8, Vectorized Memory Access
  - 在 Python 中创建的内核, NVIDIA Software Stack
  - 异步执行的内核, CUDA Programming Refresher
    - 非法操作的全局故障标志, CUDA Programming Refresher
  - 编程
    - 异步内存分配, Asynchronous Memory Allocation and Memory Pools-Asynchronous Memory Allocation and Memory Pools
    - CUDA 流（stream）, Asynchronous Memory Allocation and Memory Pools
    - CPU 与 GPU 间的流程, Understanding GPU Architecture
    - __global__, CUDA Programming Refresher
    - GPU 内存层级, Understanding GPU Memory Hierarchy-Understanding GPU Memory Hierarchy

  - 高占用率和 GPU 利用率, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
  - 可用的最高层级和最新 API, Asynchronous Memory Prefetching and Tensor Memory Accelerator
  - 内核错误, CUDA Programming Refresher
  - 一维内核输入, 2D and 3D Kernel Inputs
  - 二维和三维内核输入, 2D and 3D Kernel Inputs
  - 用于并行工作的内核, CUDA Programming Refresher-CUDA Programming Refresher
  - 启动参数, Configuring Launch Parameters: Blocks per Grid and Threads per Block-Configuring Launch Parameters: Blocks per Grid and Threads per Block
  - 最大 threadsPerBlock 编译时参数, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
  - 内存池, Asynchronous Memory Allocation and Memory Pools-Asynchronous Memory Allocation and Memory Pools
  - 每个 SM 上最少常驻线程块数, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
  - NVIDIA Compute Sanitizer, Debugging Functional Correctness with NVIDIA Compute Sanitizer
  - 使用 __launch_bounds__ 调优占用率, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
  - 屋顶线模型（roofline model）, Roofline Model: Compute-Bound or Memory-Bound Workloads-Roofline Model: Compute-Bound or Memory-Bound Workloads
  - 统一内存（Unified Memory）, Unified Memory-Unified Memory
- 基于 Python 的框架, PyTorch and Higher-Level AI Frameworks
- 以 Python 为中心的库, Writing Custom Kernels with OpenAI Triton
- 统一内存（Unified Memory）, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
  - CUDA 编程, Unified Memory-Unified Memory
  - 统一 CPU-GPU 内存, The CPU and GPU Superchip
- 容器库与主机驱动程序的版本, NVIDIA Container Toolkit and CUDA Compatibility

  - NVIDIA Container Toolkit, NVIDIA Container Toolkit and CUDA Compatibility

CUDA C++ Pipeline API, Asynchronous Memory Prefetching and Tensor Memory Accelerator-Asynchronous Memory Prefetching and Tensor Memory Accelerator

  - hardware-software codesign, Asynchronous Memory Prefetching and Tensor Memory Accelerator

CUDA 编译器（参见 nvcc CUDA 编译器）

CUDA events

  - 从 GPU 设备到 CPU 主机的通信, Fine-Grained Synchronization with Events and Callbacks
  - CUDA 流同步, Fine-Grained Synchronization with Events and Callbacks, Stream Synchronization with Events-Stream Synchronization with Events
    - 跨流同步, Using CUDA Events for Cross-Stream Synchronization
  - 多 GPU 计算与数据的重叠, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - 用于分析, Using CUDA Events for Cross-Stream Synchronization

CUDA Graphs, CUDA Graphs-Conditional Graph Nodes

  - about CUDA Graphs, Techniques for Occupancy Tuning, CUDA Graphs
  - benchmarking, Capturing a CUDA Graph with a CUDA Stream
  - best practices, Best Practices for CUDA Graphs-Best Practices for CUDA Graphs
  - capturing a CUDA Graph, Capturing a CUDA Graph with a CUDA Stream-Capturing a CUDA Graph with a CUDA Stream, Capturing a CUDA Graph and Preallocating Memory-Capturing a CUDA Graph and Preallocating Memory
    - capturing only a portion of pipeline, Capturing a CUDA Graph with a CUDA Stream
    - device-initiated launch, Device-Initiated CUDA Graph Launch-Device-Initiated CUDA Graph Launch
    - dynamic graph update, Dynamic Graph Update
    - warm-up pass necessary, Capturing a CUDA Graph with a CUDA Stream
  - compiler modes that trigger, Autotuning with TorchInductor
    - static shapes required, Autotuning with TorchInductor, Dynamic Shapes and Variable Sequence Lengths

– conditional graph nodes, Conditional Graph Nodes-Conditional Graph Nodes
  – nested, Conditional Graph Nodes
– CUDA streams, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  – capturing a CUDA Graph, Capturing a CUDA Graph with a CUDA Stream-Capturing a CUDA Graph with a CUDA Stream
  – PyTorch, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
– device-initiated launch, Device-Initiated CUDA Graph Launch-Device-Initiated CUDA Graph Launch
  – dependency management, Device-Initiated CUDA Graph Launch
  – example of use, Device-Initiated CUDA Graph Launch
  – persistent scheduling, Atomic Queues and Device-Initiated CUDA Graphs for In-Kernel Persistent Scheduling
– dynamic graph update, Dynamic Graph Update
– inference engines, PyTorch, Inference Engines, and CUDA Graphs
– kernel launch overhead reduced, Reducing Kernel Launch Overhead with CUDA Graphs-CUDA Graph Trees (PyTorch Compiler Internal)
– kernels that use NVSHMEM, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM
– launching a CUDA Graph, Capturing a CUDA Graph with a CUDA Stream-Capturing a CUDA Graph with a CUDA Stream
  – device-initiated launch, Device-Initiated CUDA Graph Launch-Device-Initiated CUDA Graph Launch
– memory management, Memory Pools for CUDA Graphs
– memory pools, Replaying the Graph
– multiple-GPU collectives captured, Capturing Multi-GPU Collectives with NCCL and CUDA Graphs-Capturing Multi-GPU Collectives with NCCL and CUDA Graphs
– pitfalls, Capturing a CUDA Graph with a CUDA Stream
– prewarming, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction-Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction

- PyTorch, PyTorch, Inference Engines, and CUDA Graphs, Capturing a CUDA Graph with a CUDA Stream
  - CUDA Graph Trees, CUDA Graph Trees (PyTorch Compiler Internal)
  - static memory pools, Memory Pools for CUDA Graphs
- replaying the graph, Replaying the Graph-Replaying the Graph
  - any stream for replay, Capturing a CUDA Graph and Preallocating Memory
- TorchInductor, Autotuning with TorchInductor
- when to use dynamic parallelism instead, Dynamic Parallelism

CUDA 托管内存（Managed Memory）（参见 CUDA 统一内存）

CUDA 内存分配器, Tuning the CUDA Memory Allocator

CUDA Memory Inspector, Profiling and Tuning Memory in PyTorch

CUDA Ninjas, Automated GPU Kernel Optimizations with DeepSeek-R1 (NVIDIA)

CUDA 占用率 API, Techniques for Occupancy Tuning, Techniques for Occupancy Tuning, Determine Optimal Launch Configuration with the Occupancy API

- 协作式网格, Cooperative Grid Synchronization and Persistent Kernels

CUDA Pipeline API

- 双缓冲, Cooperative Tiling and Double-Buffering with the CUDA Pipeline API-Cooperative Tiling and Double-Buffering with the CUDA Pipeline API
- 与 TMA 结合, Asynchronous Memory Prefetching and Tensor Memory Accelerator
- warp 专门化, Using CUDA Pipeline API for Warp Specialization-Using CUDA Pipeline API for Warp Specialization
  - PyTorch, PyTorch, CUDA Pipeline API, and Warp Specialization

CUDA 分析工具接口（CUPTI）, PyTorch Profiler and Visualization Tools

CUDA 程序计数器采样（Nsight Compute）, Profiling with Nsight Systems and Nsight Compute

CUDA Python 库（NVIDIA）, NVIDIA Software Stack

CUDA Runtime API 回调函数, Fine-Grained Synchronization with Events and Callbacks

CUDA Runtime 库, CUDA Toolkit and Runtime

CUDA streams, Overlapping Kernel Execution with CUDA Streams

– asynchronous execution, Asynchronous Execution with Streams
– compute overlapping with data transfers, Using Streams to Overlap Compute with Data Transfers-Using Streams to Overlap Compute with Data Transfers
  – concurrency, Overlapping Communication and Computation-Overlapping Communication and Computation
  – multiple GPUs, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
– concurrency, Concurrency with CUDA Streams-Using CUDA Streams with MoE Models
  – launching on intended stream, Using CUDA Streams with MoE Models
  – profiling when introducing concurrency, Using CUDA Streams with MoE Models
– creating in PyTorch, Concurrency with CUDA Streams
  – profiling when adding streams, Overlapping Communication and Computation
– CUDA Graph captured, Capturing a CUDA Graph with a CUDA Stream-Capturing a CUDA Graph with a CUDA Stream
– default streams
  – default versus explicit streams, Default Versus Explicit (Nondefault) Streams
  – legacy default stream, Legacy Default Stream
  – per-thread default streams, Modern Per-Thread Default Stream
  – using default streams, Best Practices for Default Stream Usage-Best Practices for Default Stream Usage
– kernel execution overlapping with, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
  – launching five kernels on two streams, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
  – warp specialization replaced with CUDA streams, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
– mixture-of-experts models, Using CUDA Streams with MoE Models

- multi-GPU overlap of compute and data, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - multiple streams per GPU, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
- Programmatic Dependent Launch, Programmatic Dependent Launch-Programmatic Dependent Launch
- stream-ordered memory allocator, Stream-Ordered Memory Allocator-Stream-Ordered Memory Allocator
  - multiple GPUs, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - using, Using CUDA Streams and Stream-Ordered Memory Allocator with LLMs-Using CUDA Streams and Stream-Ordered Memory Allocator with LLMs
- synchronization with events and callbacks, Fine-Grained Synchronization with Events and Callbacks, Stream Synchronization with Events-Stream Synchronization with Events
  - cross-stream synchronization, Using CUDA Events for Cross-Stream Synchronization
- warp specialization, Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)-Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)
  - thread block clusters, Warp Specialization with Thread Block Clusters and CUDA Streams

CUDA Toolkit

- nvcc CUDA compiler, CUDA Toolkit and Runtime
- NVIDIA Compute Sanitizer, Debugging Functional Correctness with NVIDIA Compute Sanitizer
- NVIDIA software stack, CUDA Toolkit and Runtime
- optimized libraries included, CUDA Toolkit and Runtime
  - C++ and Python libraries, C++ and Python CUDA Libraries

CUDA Unified Memory, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling

- programming CUDA, Unified Memory-Unified Memory

– Unified CPU-GPU Memory, The CPU and GPU Superchip

cudaMemPool, Monitoring System Metrics and Counters

cudart (CUDA Runtime library), CUDA Toolkit and Runtime

CUDA_DEVICE_ORDER, Pitfall #4: Verify CPU-GPU NUMA-node affinity for NCCL threads

CUDA_VISIBLE_DEVICES, Pitfall #4: Verify CPU-GPU NUMA-node affinity for NCCL threads

cuDNN, CUDA Toolkit and Runtime

CUPTI (CUDA Profiling Tools Interface), PyTorch Profiler and Visualization Tools

CuPy, C++ and Python CUDA Libraries, Adaptive Batching and Chunked Prefill Scheduling

cuPyNumeric, C++ and Python CUDA Libraries

典型数据中心使用的电流, Compute Density and Power Requirements

– 液冷, Liquid Cooling Versus Air Cooling

使用 OpenAI Triton 的自定义内核, Writing Custom Kernels with OpenAI Triton-Profiling with Triton Proton Profiler

– CUDA C++ 自定义内核, Registering Custom Kernels with PyTorch

– 在 PyTorch 中注册, Registering Custom Kernels with PyTorch-Registering Custom Kernels with PyTorch

– 调优内核启动参数, Tuning Kernel-Launch Parameters

cuTile, C++ and Python CUDA Libraries

– CUDA kernels in Python, NVIDIA Software Stack

– CUDA Toolkit, C++ and Python CUDA Libraries

CUTLASS (NVIDIA)

– arithmetic intensity and Tensor Core performance, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance-Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance

– CUDA kernels in Python, NVIDIA Software Stack

– CUDA Toolkit, C++ and Python CUDA Libraries

– thread block pairs, Thread Block Pair

– TMEM, Feeding Tensor Cores with TMEM and TMA

  - warp specialization, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance

D

DALI（数据加载库，NVIDIA）, Multimodal Data Processing with NVIDIA DALI, Optimizing the Data Input Pipeline

DASH 框架, Input-Aware Layer Skipping (DASH)

数据中心 GPU 管理器（DCGM，NVIDIA）, Performance Monitoring and Utilization in Practice

  - I/O 监控, Monitoring Storage I/O
  - 推理监控, Monitoring System Metrics and Counters
  - 监控拖尾节点或进程, Pitfall #5: Straggler nodes or processes

数据中心

  - 典型数据中心使用的电流, Compute Density and Power Requirements
  - 液冷, Liquid Cooling Versus Air Cooling

数据压缩, Tuning, Replicating, and Compressing Data

数据输入管线优化, Optimizing the Data Input Pipeline-Optimizing the Data Input Pipeline

数据并行（DP）, Parallelism Strategies for Serving Massive MoE Models, Data Parallelism

  - 数据并行的定义, Distributed Data Parallel Strategies, Data Parallelism
  - 分布式数据并行与数据并行, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
    - 分布式数据并行的定义, Distributed Data Parallel Strategies
  - 在 GPU 间拆分模型权重和数据, Parallelism Strategies for Serving Massive MoE Models
  - 与 TP、PP 和混合并行的比较, Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)-Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)

数据管线（参见 管线）

NVL72 中的数据处理单元（DPU）, Multirack and Storage Communication

通过共享内存复用数据, Tiling and Data Reuse Using Shared Memory-Tiling and Data Reuse Using Shared Memory

  - 提高算术强度, Multilevel Microtiling and Software Prefetching

DataLoader (PyTorch)
- 每个工作进程的 CPU 亲和性, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
- 数据输入管线优化, Optimizing the Data Input Pipeline-Optimizing the Data Input Pipeline
- 数据加载和预处理, Efficient Data Loading and Preprocessing-Efficient Data Loading and Preprocessing
- 多个工作进程, Tuning NVMe and Filesystem for Throughput, Efficient Data Loading and Preprocessing, Scaling Out Workers as You Scale Out Number of GPUs, Optimizing the Data Input Pipeline
- 持久化工作进程, Efficient Data Loading and Preprocessing
- 固定内存标志, NUMA-Friendly Memory Allocation and Memory Pinning, Using Streams to Overlap Compute with Data Transfers, Optimizing the Data Input Pipeline
- 预取因子, Optimizing the Data Input Pipeline

通过 NeMo Curator 构建的训练数据集, Creating High-Quality LLM Datasets with NVIDIA NeMo Curator

DCGM（参见 数据中心 GPU 管理器（DCGM，NVIDIA））

DDP（参见 分布式数据并行（DDP，PyTorch））

去抖, Debouncing and Request Coalescing

调试
- 编译器性能问题, Profiling and Debugging Compiler Performance Issues
- DWARF, CPU and GPU Profiling with Linux perf
- 图中断（graph breaks）, Graph Breaks and TorchDynamo explain(), Debugging Compiler Phases, Graph Breaks, and Performance
  - 监控图重新编译, TorchDynamo for Bytecode Capture and Graph Extraction, Minimize Graph Recompilations
- 推理正确性问题, Debugging Correctness Issues-Debugging Correctness Issues
- NCCL, Profiling and Debugging NCCL
  - debugging enabled, Profiling and Debugging NCCL, Debugging Correctness Issues
  - NCCL logs, In-Network SHARP Aggregation, Debugging Correctness Issues

    - NCCL_DEBUG, Topology Awareness in NCCL, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables, Profiling and Debugging NCCL, Debugging Correctness Issues
    - test suite, Debugging Correctness Issues
  - numerical correctness and accuracy, Debugging Numerical Correctness and Accuracy-Debugging Numerical Correctness and Accuracy
  - NVIDIA Compute Sanitizer, Debugging Functional Correctness with NVIDIA Compute Sanitizer
  - PyTorch compiler, Performance Hints and Debugging Generated Code

使用带属性记录格式（DWARF）调试, CPU and GPU Profiling with Linux perf

解码（decode）

  - 解码阶段概述, Scaling Disaggregated Prefill and Decode for Inference
  - 受限解码, Constrained Decoding Performance Implications
  - 与预填充解耦
    - 关于解耦, Multinode Inference, Parallelism, Decoding, and Routing Optimizations, Prefill-Decode Interference, Scaling Disaggregated Prefill and Decode for Inference
    - 架构, Disaggregated Prefill and Decode Architecture
    - 批大小优化, Adaptive Batching and Chunked Prefill Scheduling
    - 集群池, Disaggregated Prefill and Decode Cluster Pools-Memory management for the KV cache
    - 使用 Kubernetes 部署, Deploying Disaggregated Prefill and Decode with Kubernetes-Deploying Disaggregated Prefill and Decode with Kubernetes, Full-Stack Inference Optimizations
    - 延迟和吞吐量, Impact on Latency (TTFT) and Throughput (TPOT)
    - 负载均衡, Dynamic Scheduling and Load Balancing-Dynamic resource scaling
    - 从预填充到解码的数据传输, KV Cache Data Transfer and NIXL, Fast KV Cache Transfer Between Prefill and Decode-Zero-Copy GPU-to-GPU Transfer
    - 扩展, Scaling Prefill and Worker Nodes Independently, Scalability of Disaggregated Prefill and Decode
    - 分离预填充和解码阶段, Separate Prefill and Decode Inference Stages-Separate Prefill and Decode Inference Stages, Disaggregated Prefill and Decode Architecture

- disaggregated routing policies, Disaggregated Routing and Scheduling Policies-QoS and early rejection policies
  - capacity-aware routing, Capacity-aware routing-Capacity-aware routing
  - example dynamic routing policy configuration, Example dynamic routing policy configuration
  - example dynamic routing policy in code, Example dynamic routing policy in code
  - latency-aware routing, Latency-aware routing-Latency-aware routing
  - multipath inference (racing), Multipath inference (racing)
  - QoS and early rejection policies, QoS and early rejection policies-QoS and early rejection policies
  - routing factors, Routing factors-Routing factors
  - speculative decoding, Multibranch, parallel speculative decoding across workers
- hardware and parallelism, Heterogeneous Hardware and Parallelism Strategies for Prefill and Decode-Different precision for prefill and decode
- KV cache transfer from prefill, Fast KV Cache Transfer Between Prefill and Decode-Zero-Copy GPU-to-GPU Transfer
  - connector and data path design, Connector and Data Path Design-Connector and Data Path Design
  - KV cache size, KV Cache Size
  - zero-copy GPU-to-GPU, Zero-Copy GPU-to-GPU Transfer-Zero-Copy GPU-to-GPU Transfer
- LLM decode phase as memory bound, Maintaining High Occupancy and GPU Utilization
- optimized decode kernels, Optimized Decode Kernels-FlexDecoding (PyTorch)
  - FlashMLA, FlashMLA (DeepSeek)
  - FlexDecoding, FlexDecoding (PyTorch)-FlexDecoding (PyTorch)
  - ThunderMLA, ThunderMLA (Stanford)
- scale-out of workers, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction
- speculative decoding, Disaggregated Prefill and Decode Architecture, Speculative Decoding and Parallel Token Generation Techniques-Combining

Decoding Techniques and Evaluating Complexity

- combining techniques, Combining Decoding Techniques and Evaluating Complexity
- EAGLE algorithm, Two-Model, Draft-Based Speculative Decoding and EAGLE
- interleaving decode steps, Interleaving Decode Steps from Multiple Requests
- Medusa multiple heads, Speculative Decoding and Parallel Token Generation Techniques
- Medusa multiple heads multitoken, Multitoken Decoding with Medusa’s Multiple Heads-Multitoken Decoding with Medusa’s Multiple Heads
- single-model self-speculative decoding, Single-Model Self-Speculative Decoding
- two-model draft-based, Two-Model, Draft-Based Speculative Decoding and EAGLE-Two-Model, Draft-Based Speculative Decoding and EAGLE

- why disaggregation, Why Prefill-Decode Disaggregation?-Scalability of Disaggregated Prefill and Decode
  - advantages of disaggregation, Advantages of Disaggregation-Phase-specific optimizations
  - cluster pools, Disaggregated Prefill and Decode Cluster Pools-Memory management for the KV cache
  - routing and scheduling policies, Disaggregated Routing and Scheduling Policies-QoS and early rejection policies
  - scalability, Scaling Prefill and Worker Nodes Independently, Scalability of Disaggregated Prefill and Decode

DeepEP（Deep Experts Parallelism；DeepSeek）, Transparency and Reproducibility

- 内联 PTX 代码, DeepSeek’s Use of Inline PTX for Memory Allocation Optimization

DeepGEMM（DeepSeek）, Transparency and Reproducibility

DeepMind AlphaTensor, AlphaTensor AI-Discovered Algorithms Boosting GPU Performance (Google DeepMind)

DeepSeek Fire-Flyer 文件系统（3FS）, Transparency and Reproducibility, DeepSeek’s Fire-Flyer File System-DeepSeek’s Fire-Flyer File System

DeepSeek 多头潜在注意力（MLA）, Mechanical Sympathy: Hardware-Software Codesign

DeepSeek-R1 (NVIDIA)

- 自动化内核优化, Automated GPU Kernel Optimizations with DeepSeek-R1 (NVIDIA)-Automated GPU Kernel Optimizations with DeepSeek-R1 (NVIDIA)
- 思维链推理, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China
- 自定义内核、高级优化和性能较低的 GPU, Introduction and AI System Overview
- 效率提升, Introduction and AI System Overview, Introduction and AI System Overview
- MoE 效率, Toward 100-Trillion-Parameter Models
- 训练成本, Introduction and AI System Overview

DeepSeek-V3, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China-DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China

- MoE 效率, Toward 100-Trillion-Parameter Models

DeepSeek.AI

- 出口限制, Introduction and AI System Overview, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China-DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China
- 开源 GitHub 仓库, Transparency and Reproducibility
  - 开源周, Transparency and Reproducibility, Transparency and Reproducibility

部署

- 解耦式推理系统, Deploying Disaggregated Prefill and Decode with Kubernetes-Deploying Disaggregated Prefill and Decode with Kubernetes
- 推理优化, Full-Stack Inference Optimizations
- 作为预集成机架设备的 NVL72, Preintegrated Rack Appliance
  - Kubernetes, Preintegrated Rack Appliance
  - NVIDIA Base Command Manager, Preintegrated Rack Appliance
  - 用于资源管理的简易 Linux 工具, Preintegrated Rack Appliance
- 分层部署, Model Cascading and Tiered Model Deployment

直接内存访问（DMA）

- NVIDIA GDS, Using NVIDIA GDS
- O_DIRECT, Using NVIDIA GDS
  - GDS, Using NVIDIA GDS
- 点对点 DMA, Enabling Peer-to-Peer DMA and UCX

解耦式预填充和解码

- 概述, Multinode Inference, Parallelism, Decoding, and Routing Optimizations, Prefill-Decode Interference, Scaling Disaggregated Prefill and Decode for Inference
- 架构, Disaggregated Prefill and Decode Architecture
  - 分离预填充和解码, Separate Prefill and Decode Inference Stages-Separate Prefill and Decode Inference Stages
  - 分离预填充和解码阶段, Disaggregated Prefill and Decode Architecture
  - 推测解码, Disaggregated Prefill and Decode Architecture
- 批大小优化, Adaptive Batching and Chunked Prefill Scheduling
- 使用 Kubernetes 部署, Deploying Disaggregated Prefill and Decode with Kubernetes-Deploying Disaggregated Prefill and Decode with Kubernetes, Full-Stack Inference Optimizations
- 硬件和并行策略, Heterogeneous Hardware and Parallelism Strategies for Prefill and Decode-Different precision for prefill and decode
- 从预填充到解码的 KV 缓存传输, Fast KV Cache Transfer Between Prefill and Decode-Zero-Copy GPU-to-GPU Transfer
- 延迟和吞吐量, Impact on Latency (TTFT) and Throughput (TPOT)
- 负载均衡, Dynamic Scheduling and Load Balancing
- NIXL, NVIDIA’s NIXL and Disaggregated Inference-NCCL Versus NIXL
  - about NIXL, NVIDIA’s NIXL and Disaggregated Inference-NVIDIA’s NIXL and Disaggregated Inference
  - asynchronous API with callbacks, NIXL Asynchronous API with Callbacks-NIXL Asynchronous API with Callbacks
  - Dynamo throughput improved, NIXL and High-Performance Inference Systems Like NVIDIA Dynamo
  - intelligent interconnect routing, Intelligent Interconnect Routing for KV Cache Transfers, KV Cache Data Transfer and NIXL

  - KV cache offloading with NIXL, KV Cache Offloading with NIXL-KV Cache Offloading with NIXL
  - prefill to decode data transfers, KV Cache Data Transfer and NIXL
  - separate prefill and decode inference stages, Separate Prefill and Decode Inference Stages-Separate Prefill and Decode Inference Stages
- 从预填充到解码的数据传输, KV Cache Data Transfer and NIXL
  - 连接器和数据路径设计, Connector and Data Path Design-Connector and Data Path Design
  - KV 缓存, Fast KV Cache Transfer Between Prefill and Decode-Zero-Copy GPU-to-GPU Transfer
  - KV 缓存大小, KV Cache Size
  - 零拷贝 GPU 到 GPU, Zero-Copy GPU-to-GPU Transfer-Zero-Copy GPU-to-GPU Transfer
- 预填充—解码干扰, Prefill-Decode Interference
- 路由和调度策略, Disaggregated Routing and Scheduling Policies-QoS and early rejection policies
  - 容量感知路由, Capacity-aware routing-Capacity-aware routing
  - 动态路由策略配置示例, Example dynamic routing policy configuration
  - 代码中的动态路由策略示例, Example dynamic routing policy in code
  - 延迟感知路由, Latency-aware routing-Latency-aware routing
  - 多路径推理（竞争）, Multipath inference (racing)
  - QoS 和提前拒绝策略, QoS and early rejection policies-QoS and early rejection policies
  - 路由因素, Routing factors-Routing factors
  - 推测解码, Multibranch, parallel speculative decoding across workers
- 扩展, Scaling Prefill and Worker Nodes Independently
  - 横向扩展工作进程, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction
- 为何解耦, Why Prefill-Decode Disaggregation?-Scalability of Disaggregated Prefill and Decode
  - 解耦的优势, Advantages of Disaggregation-Phase-specific optimizations, Heterogeneous Hardware and Parallelism Strategies for Prefill and Decode

    - 集群池, Disaggregated Prefill and Decode Cluster Pools-Memory management for the KV cache
    - 路由和调度策略, Disaggregated Routing and Scheduling Policies-QoS and early rejection policies
    - 可扩展性, Scalability of Disaggregated Prefill and Decode

分布式数据并行（DDP；PyTorch）

- 分桶, Reducing Communication Frequency and Volume
- 与数据并行的比较, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
    - 作为并行策略的数据并行, Parallelism Strategies for Serving Massive MoE Models, Data Parallelism
    - 数据并行的定义, Distributed Data Parallel Strategies, Data Parallelism
    - 分布式数据并行的定义, Distributed Data Parallel Strategies
- 通过 DDP 与 torch.compile 扩展, DDP with torch.compile

分布式网络通信

- 多 GPU 通信, NCCL for Distributed Multi-GPU Communication-In-Network SHARP Aggregation
    - 拥塞感知和拓扑感知调度, Congestion-Aware and Topology-Aware Scheduling with Multiple GPUs-Coordinating NVSwitch Transfers with Fine-Tuned Scheduling
    - 数据并行的定义, Distributed Data Parallel Strategies
    - 分布式数据并行的定义, Distributed Data Parallel Strategies
    - 分布式数据并行策略, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
    - NCCL 通信算法, NCCL Communication Algorithms-NCCL Communication Algorithms
    - NCCL 拓扑感知, Topology Awareness in NCCL-Topology Awareness in NCCL
    - 分析和调试 NCCL, Profiling and Debugging NCCL
- 多节点通信陷阱, Multinode Communication Pitfalls-Pitfall #6: GPU memory fragmentation under UCX/RDMA
    - 使用 GLOO 后端而非 NCCL, Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL-Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL

  - UCX/RDMA 下的 GPU 内存碎片, Pitfall #6: GPU memory fragmentation under UCX/RDMA-Pitfall #6: GPU memory fragmentation under UCX/RDMA
  - 带宽不足或 NIC 配置错误, Pitfall #4: Insufficient network bandwidth or misconfigured NICs
  - NCCL 版本不匹配, Pitfall #2: Mismatched NCCL versions
  - 拖尾节点或进程, Pitfall #5: Straggler nodes or processes
  - 总结, Pitfall #6: GPU memory fragmentation under UCX/RDMA
  - NCCL 引导期间 TCP 端口耗尽, Pitfall #3: TCP port exhaustion during NCCL bootstrap
- 已调优的多节点连通性, Tuning Multinode Connectivity-Tuning Multinode Connectivity
- NCCL 陷阱和注意事项, NCCL Communicator Lifecycle and Environment Gotchas-Pitfall #6: NCCL communicator hangs, errors, or shuts down completely
  - 通信器挂起、报错或关闭, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely-Pitfall #6: NCCL communicator hangs, errors, or shuts down completely
  - NCCL 线程的 CPU-GPU NUMA 节点, Pitfall #4: Verify CPU-GPU NUMA-node affinity for NCCL threads
  - 创建和销毁通信器, Pitfall #2: Do not create and destroy NCCL communicators on every iteration
  - 过于频繁地创建通信器, Pitfall #1: Creating NCCL communicators too often-Pitfall #1: Creating NCCL communicators too often
  - 忽略警告和错误, Pitfall #5: Resist the temptation to ignore NCCL warnings and errors
  - 通过环境变量覆盖或禁用功能, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables-NCCL_SHARP_DISABLE
- NVIDIA Magnum IO, NVIDIA Magnum IO Optimization Stack
- 针对 Kubernetes 优化, Optimizing Network Communication for Kubernetes
- 通信和计算重叠, Overlapping Communication and Computation (Pipelining)-Achieving Maximal Overlap in Practice
  - 无重叠与重叠的比较, Achieving Maximal Overlap in Practice-Achieving Maximal Overlap in Practice

  - 分布式数据并行策略, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
  - 降低频率和数据量, Reducing Communication Frequency and Volume
  - 用于异步执行的流, Asynchronous Execution with Streams
- RDMA, High-Speed, Low-Overhead Data Transfers with RDMA-Pitfall #6: GPU memory fragmentation under UCX/RDMA

分布式共享内存（DSMEM）, Tiling with Thread Block Clusters

- 在线程块之间共享状态, Distributed Shared Memory
- 线程块集群, Thread Block Clusters and Distributed Shared Memory-Thread Block Pair
  - 工作负载并行化算法, Designing Efficient Algorithms with Thread Block Clusters-Designing Efficient Algorithms with Thread Block Clusters
  - 协调线程块集群, Coordinating Thread Block Clusters with Cooperative Groups API-Coordinating Thread Block Clusters with Cooperative Groups API
  - 说明, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters
  - 降低全局内存流量, Reducing Global Memory Traffic with Thread Block Clusters-Reducing Global Memory Traffic with Thread Block Clusters
  - 启动线程块集群, Launching a Thread Block Cluster, Designing Efficient Algorithms with Thread Block Clusters
  - 暂存内存, Scratch Memory
  - 线程块对, Thread Block Pair-Thread Block Pair
  - 线程块重排, Thread Block Swizzling
  - 线程访问彼此的共享内存, Threads, Warps, Blocks, and Grids, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters, Thread Block Clusters and Distributed Shared Memory
  - 带线程块集群的 warp 专门化, Warp Specialization with Thread Block Clusters-Warp Specialization with Thread Block Clusters

DistServe, Scaling Disaggregated Prefill and Decode for Inference

DMA（参见 直接内存访问（DMA））

Docker

- 禁用交换并强制 NUMA 亲和性, Virtual Memory and Swapping
- NVIDIA Container Toolkit, Container Runtime Optimizations for GPUs

- RDMA, High-Speed, Low-Overhead Data Transfers with RDMA
- 运行时优化
  - 容器概述, Container Runtime Optimizations for GPUs
  - 容器覆盖文件系统开销, Avoiding Container Overlay Filesystem Overhead
  - NVIDIA 容器运行时注入库, NVIDIA Container Runtime
  - NVIDIA Container Toolkit 和 CUDA 兼容性, NVIDIA Container Toolkit and CUDA Compatibility
  - 减小镜像大小, Reduce Image Size for Faster Container Startup
- 面向 GPU 的运行时优化, Container Runtime Optimizations for GPUs-Reduce Image Size for Faster Container Startup

使用 CUTLASS 进行双缓冲, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance

DP（参见 数据并行（DP）；动态并行（DP））

NVL72 中的 DPU（数据处理单元）, Multirack and Storage Communication

作为共享内存的 DRAM, Tiling and Data Reuse Using Shared Memory
- （另见 CPU DRAM）

DSMEM（参见 分布式共享内存（DSMEM））

DTensor (PyTorch), TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP

双裸片 GPU, NVIDIA Blackwell “Dual-Die” GPU
- 呈现为一个 GPU, NVIDIA Blackwell “Dual-Die” GPU
- GPC“分区”, Thread Block Clusters and Distributed Shared Memory
- 编程, Understanding GPU Memory Hierarchy
- 远程内存访问延迟, NUMA Awareness and CPU Pinning

DualPipe（DeepSeek）, Transparency and Reproducibility
- 混合专家模型 DeepSeek-V3, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China

DWARF (Debugging With Attributed Record Formats), CPU and GPU Profiling with Linux perf

动态激活量化, Dynamic Quantization and Activation Range Adjustment

动态批处理, Dynamic Batching-Dynamic Batching
- 自适应批处理延迟, Dynamic Batching

  - 与静态批处理的比较, Dynamic Batching

动态更新 CUDA Graph, Dynamic Graph Update

动态早退网络, Dynamic Early-Exit Networks

动态并行（DP）, Dynamic Parallelism-Dynamic Parallelism

  - 基准测试, Dynamic Parallelism
    - 变更前后进行分析, Dynamic Parallelism
  - 子内核启动上限, Dynamic Parallelism
  - 监控设备内存使用量, Dynamic Parallelism
  - 避免“栈溢出”错误, Dynamic Parallelism
  - 何时改为捕获 CUDA Graph, Dynamic Parallelism

动态路由

  - 延迟感知调度, Latency-Aware Scheduling and Dynamic Routing-Latency-Aware Scheduling and Dynamic Routing
  - MoE 推理策略, Dynamic Routing Strategies for MoE Inference-Adaptive Expert Routing and Real-Time Monitoring
    - 专家通信优化, Expert Communication Optimization-Expert Communication Optimization
    - 专家复制, Load Balancing, Capacity Factor, and Expert Replication
    - 负载均衡和容量因子, Load Balancing, Capacity Factor, and Expert Replication

动态调度

  - 原子工作队列, Dynamic Scheduling with Atomic Work Queues-Atomic Queues
    - 原子计数器, Atomic Counters
    - 原子队列, Atomic Queues-Atomic Queues
    - 动态工作分配, Atomic Counters
    - Nsight Compute, Atomic Counters-Atomic Counters
  - 动态拥塞感知调度, Dynamic Congestion-Aware Scheduling-Dynamic Congestion-Aware Scheduling
  - 负载均衡, Dynamic Scheduling and Load Balancing-Dynamic resource scaling
    - 自适应资源调度和热点预防, Adaptive Resource Scheduling and Hotspot Prevention-Dynamic resource scaling

  - Arrow 的自适应实例扩缩, Arrow’s adaptive instance scaling-Arrow’s adaptive instance scaling
  - 动态资源扩缩, Dynamic resource scaling
  - Mooncake 自适应策略, Mooncake adaptive strategies
  - TetriInfer 的两级调度器, TetriInfer’s two-level scheduler

动态形状（PyTorch）

- TorchDynamo, Profiling and Debugging Compiler Performance Issues, TorchDynamo for Bytecode Capture and Graph Extraction
- 用于可变长度序列, Dynamic Shapes and Variable Sequence Lengths-Dynamic Shapes and Variable Sequence Lengths
  - 分析动态形状与填充的差异, Dynamic Shapes and Variable Sequence Lengths
- XLA, PyTorch XLA Backend

动态共享内存分配, Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection

Dynamo（参见 NVIDIA Dynamo；TorchDynamo（PyTorch））

E

提前拒绝, Early Rejection (Admission Control)-Early Rejection (Admission Control)

- QoS, QoS and early rejection policies-QoS and early rejection policies

GPU 内存的 ECC 模式, GPU Clock Speeds and ECC

EGM（扩展 GPU 内存，Extended GPU Memory）, The CPU and GPU Superchip

环境变量

- 显式设置而非使用默认值, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely
- CUDA_DEVICE_ORDER, Pitfall #4: Verify CPU-GPU NUMA-node affinity for NCCL threads
- CUDA_VISIBLE_DEVICES, Pitfall #4: Verify CPU-GPU NUMA-node affinity for NCCL threads
- 调试编译器阶段, Debugging Compiler Phases, Graph Breaks, and Performance
- MALLOC_CONF, Tune Host CPU Memory Allocator

– 覆盖或禁用 NCCL 功能, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables-NCCL_SHARP_DISABLE

– 在线 NCCL 信息, NCCL Communicator Lifecycle and Environment Gotchas

– NCCL_ALGO, Topology Awareness in NCCL, NCCL Communication Algorithms, Ring versus tree all-reduce

– NCCL_ASYNC_ERROR_HANDLING, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely, Profiling and Debugging NCCL

– NCCL_BUFFSIZE, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables

– NCCL_CROSS_NIC, Multinode and Multirack Communication with GPUDirect RDMA

– NCCL_DEBUG, Topology Awareness in NCCL, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables, Profiling and Debugging NCCL, Debugging Correctness Issues

– NCCL_IGNORE_CPU_AFFINITY, Pitfall #4: Verify CPU-GPU NUMA-node affinity for NCCL threads

– NCCL_MAX_NCHANNELS, NCCL_MIN_NCHANNELS and NCCL_MAX_NCHANNELS

– NCCL_MIN_NCHANNELS, NCCL_MIN_NCHANNELS and NCCL_MAX_NCHANNELS

– NCCL_NSOCKS_PERTHREAD, Tuning Multinode Connectivity, Pitfall #4: Insufficient network bandwidth or misconfigured NICs, NCCL_NSOCKS_PERTHREAD and NCCL_SOCKET_NTHREADS (Socket/TCP transports)

– NCCL_P2P_DISABLE, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables

– NCCL_PROFILER_PLUGIN, Profiling and Debugging NCCL

– NCCL_SHARP_DISABLE, NCCL_SHARP_DISABLE

– NCCL_SHM_DISABLE, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables

– NCCL_SOCKET_IFNAME, Optimizing Network Communication for Kubernetes, Pitfall #5: Resist the temptation to ignore NCCL warnings and errors

– NCCL_SOCKET_NTHREADS, Tuning Multinode Connectivity, Pitfall #4: Insufficient network bandwidth or misconfigured NICs,

NCCL_NSOCKS_PERTHREAD and NCCL_SOCKET_NTHREADS (Socket/TCP transports)

- NCCL_TOPO_DUMP_FILE, Topology Awareness in NCCL
- NCCL_TOPO_FILE, NCCL_TOPO_FILE
- PYTORCH_CUDA_ALLOC_CONF, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling, Stream-Ordered Memory Allocator
- TCMALLOC_MAX_TOTAL_THREAD_CACHE_BYTES, Tune Host CPU Memory Allocator
- TCMALLOC_RELEASE_RATE, Tune Host CPU Memory Allocator
- TORCHDYNAMO_REPRO_*, Debugging Compiler Phases, Graph Breaks, and Performance
- TORCHINDUCTOR_BENCHMARK_KERNEL, Profiling and Debugging Compiler Performance Issues
- TORCHINDUCTOR_UNIQUE_KERNEL_NAMES, Profiling and Debugging Compiler Performance Issues
- TORCH_COMPILE_DEBUG, Debugging Compiler Phases, Graph Breaks, and Performance
- TORCH_LOGS, Profiling and Debugging Compiler Performance Issues
  - compiler debugging, Debugging Compiler Phases, Graph Breaks, and Performance
  - logging options summary, Debugging Compiler Phases, Graph Breaks, and Performance
  - performance hints, Performance Hints and Debugging Generated Code
- 将调试值更新为生产值, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables

EP（参见 专家并行（EP））

EPLB（专家并行负载均衡器；DeepSeek）, Transparency and Reproducibility

错误处理

- CUDA 内核, CUDA Programming Refresher
- 推理系统, Error Handling
- NCCL, Profiling and Debugging NCCL
- NCCL_ASYNC_ERROR_HANDLING, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely

ExecuTorch, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools

专家共置, Expert Communication Optimization

专家并行（EP）, Parallelism Strategies for Serving Massive MoE Models, Expert Parallelism-Expert Parallelism

- 混合并行, Hybrid Parallelism
- 负载均衡, Expert Parallelism
- 在 GPU 间拆分模型权重和数据, Parallelism Strategies for Serving Massive MoE Models
- 与 TP、PP 和混合并行的比较, Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)-Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)

专家并行负载均衡器（EPLB；DeepSeek）, Transparency and Reproducibility

专家复制, Load Balancing, Capacity Factor, and Expert Replication

指数计算单元, Mechanical Sympathy: Hardware-Software Codesign

扩展 GPU 内存（EGM）, The CPU and GPU Superchip

F

故障

- 文件系统缓存和回写, Filesystem Caching and Write-Back
- Meta 报告的大规模 GPU 故障, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely
- NCCL 通信器挂起、报错或关闭, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely-Pitfall #6: NCCL communicator hangs, errors, or shuts down completely

fatbinary 模型, CUDA Forward and Backward Compatibility Across GPU Hardware Generations

容错, Fault Tolerance

前馈神经网络（FFN）激活检查点, Activation Checkpointing for Memory Savings

Feynman GPU, Feynman GPU (2028) and Doubling Something Every Year

文件系统

- 缓存和回写, Filesystem Caching and Write-Back
- DeepSeek Fire-Flyer 文件系统, DeepSeek’s Fire-Flyer File System-DeepSeek’s Fire-Flyer File System

– FUSE 无法使用 GDS, DeepSeek’s Fire-Flyer File System

– 必须针对大规模并发 I/O 优化, Sequential Versus Random Read Patterns

– NFS, Distributed, Parallel Filesystems and Object Stores

– tuning parameters, Distributed, Parallel Filesystems and Object Stores

– 并行文件系统, Distributed, Parallel Filesystems and Object Stores

– 共享文件系统, Distributed, Parallel Filesystems and Object Stores

– 调优, Tuning, Replicating, and Compressing Data

Fire-Flyer 文件系统（3FS；DeepSeek）, Transparency and Reproducibility, DeepSeek’s Fire-Flyer File System-DeepSeek’s Fire-Flyer File System

FireworksAI, Replaying the Graph

先进先出（FIFO）调度器, Latency-Aware Scheduling and Dynamic Routing-Latency-Aware Scheduling and Dynamic Routing

FlashAttention, Mechanical Sympathy: Hardware-Software Codesign

FlashMLA（快速多头潜在注意力；DeepSeek）, Transparency and Reproducibility, FlashMLA (DeepSeek)

– 软硬件协同设计, Mechanical Sympathy: Hardware-Software Codesign

FlexAttention (PyTorch), PyTorch Optimized Attention Mechanisms, Autotuning with TorchInductor, FlexDecoding (PyTorch)

– nested jagged-layout tensors, FlexDecoding (PyTorch)

– PagedAttention, FlexDecoding (PyTorch)

FlexDecoding (PyTorch), PyTorch Optimized Attention Mechanisms, FlexDecoding (PyTorch)-FlexDecoding (PyTorch)

– nested jagged-layout tensors, FlexDecoding (PyTorch)

CUDA 未提供 float8, Vectorized Memory Access

FP4、FP8、FP16, NVIDIA GPU Tensor Cores and Transformer Engine, BF16/FP16, FP8, and FP4 Reduced Precision

– 动态精度变更, Dynamic Precision Changes-Dynamic Precision Changes

– 推理量化, Reducing Precision from FP16 to FP8 and FP4

– 仅权重量化, Weight-Only Quantization (GPTQ, AWQ)

FSDP（完全分片数据并行；PyTorch）, Distributed Data Parallel Strategies, FSDP Automatic Checkpointing and Offloading-FSDP Automatic Checkpointing and Offloading

- 自动检查点, Activation Checkpointing for Memory Savings, FSDP Automatic Checkpointing and Offloading-FSDP Automatic Checkpointing and Offloading
- 通过 FSDP 与 torch.compile 扩展, FSDP with torch.compile-FSDP with torch.compile
- SimpleFSDP, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
- TorchTitan, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP

FUSE 无法使用 GDS, DeepSeek’s Fire-Flyer File System

融合内核（参见 内核融合）

NVIDIA 的未来硬件, A Glimpse into the Future: NVIDIA’s Roadmap-Feynman GPU (2028) and Doubling Something Every Year
- Blackwell Ultra 和 Grace Blackwell Ultra, Blackwell Ultra and Grace Blackwell Ultra
- 每年翻倍, Feynman GPU (2028) and Doubling Something Every Year
- Feynman GPU, Feynman GPU (2028) and Doubling Something Every Year
- 跨世代 GPU 编程, CUDA Forward and Backward Compatibility Across GPU Hardware Generations, CUDA GPU Backward and Forward Compatibility Model
- Rubin Ultra 和 Vera Rubin Ultra, Rubin Ultra and Vera Rubin Ultra (2027)
- Vera Rubin 超级芯片, Vera Rubin Superchip (2026)

FX Graph
- AOT Autograd, TorchDynamo for Bytecode Capture and Graph Extraction, AOT Autograd Fusion for Forward and Backward Passes
- AutoParallel, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
- PrimTorch IR, PrimTorch IR (Prims) Simplified Operator Set
- TorchDynamo, TorchDynamo for Bytecode Capture and Graph Extraction
  - dumping graph after each stage, Debugging Compiler Phases, Graph Breaks, and Performance
- TorchInductor, AOT Autograd Fusion for Forward and Backward Passes

G

GB200（参见 NVIDIA GB200 NVL72）

GB300（参见 NVIDIA GB300 NVL72）

GDS（GPUDirect Storage；NVIDIA）, NUMA-Friendly Memory Allocation and Memory Pinning, Using NVIDIA GDS-Measuring GDS with gdsio, Offloading Parameters to CPU and NVMe

Gemini Ultra（Google）训练成本, Introduction and AI System Overview

GEMM（通用矩阵乘法，general matrix multiply）

- AI 发现的算法, AlphaTensor AI-Discovered Algorithms Boosting GPU Performance (Google DeepMind)-AlphaTensor AI-Discovered Algorithms Boosting GPU Performance (Google DeepMind)
- GEMM 内核, Structured Sparsity
  - CUTLASS, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance
  - GEMM 管线, Combining PDL and Thread Block Clusters with Warp Specialization-Combining PDL and Thread Block Clusters with Warp Specialization
  - PyTorch 编译器 TorchInductor, Using the PyTorch Compiler
  - 屋顶线分析, Kernel Roofline Analysis for General Matrix Multiply (GEMM)-Kernel Roofline Analysis for General Matrix Multiply (GEMM)
  - 线程块集群, Designing Efficient Algorithms with Thread Block Clusters

GitHub

- DeepSeek 仓库, Transparency and Reproducibility
- 报告问题, Debugging Numerical Correctness and Accuracy
- 基于 TorchBench 的性能回归, Continuous Integration and Performance Benchmarking

GLaM (Google), Expert Parallelism

全局内存

- 缓存加载, Read-Only Data Caches
- 降低流量, Reducing Global Memory Traffic with Thread Block Clusters-Reducing Global Memory Traffic with Thread Block Clusters

全局内存

- 算术强度, Roofline Model: Compute-Bound or Memory-Bound Workloads
- 合并访问与非合并访问, Coalesced Versus Uncoalesced Global Memory Access-Coalesced Versus Uncoalesced Global Memory Access
- GPU 内存层级, Understanding GPU Memory Hierarchy-Understanding GPU Memory Hierarchy

- NVIDIA Compute Sanitizer 的 Initcheck, Debugging Functional Correctness with NVIDIA Compute Sanitizer
- 使加载匹配原生事务大小, Vectorized Memory Access
- 反复读取同一数据的陷阱, Tiling and Data Reuse Using Shared Memory

__global__, CUDA Programming Refresher

应使用 NCCL 而非 Gloo 后端, Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL-Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL

作为吞吐量指标的有效吞吐量（goodput）, Measuring “Goodput” Useful Throughput

Google Cloud TPU 和 XLA, TorchInductor Backend Code Generation, PyTorch XLA Backend

Google Gemini Ultra 训练成本, Introduction and AI System Overview

Google GLaM, Expert Parallelism

Google Switch Transformer 的 MoE 效率, Toward 100-Trillion-Parameter Models

GPC（GPU 处理集群）, Thread Block Clusters and Distributed Shared Memory

GPT-4（OpenAI）训练成本, Introduction and AI System Overview

GPU Boost（NVIDIA）, GPU Clock Speeds and ECC
- 将功率上限设为低于 TDP, GPU Clock Speeds and ECC
- 降频以减少发热, GPU Clock Speeds and ECC

GPU 驱动程序
- 保持 GPU 驱动程序已加载, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
- nvidia-smi, GPU Driver
- 用于性能的持久化模式, GPU Persistence Mode
- 软件栈, GPU Driver

GPU 处理集群（GPC）, Thread Block Clusters and Distributed Shared Memory

GPUDirect RDMA, Multi-GPU Programming, High-Speed, Low-Overhead Data Transfers with RDMA
- 多节点、多机架通信, Multinode and Multirack Communication with GPUDirect RDMA-Multinode and Multirack Communication with GPUDirect RDMA
- PyTorch MemPool 中的 NCCL 原生分配器, Pluggable Memory Allocators and Cross-GPU Data Transfers

- 验证真正的 GPUDirect RDMA, High-Speed, Low-Overhead Data Transfers with RDMA

GPUDirect Storage（参见 GDS（GPUDirect Storage；NVIDIA））

GPU

- 架构, Understanding GPU Architecture-CUDA GPU Backward and Forward Compatibility Model
  - 每个 SM 和线程块的最大线程数, Tuning Occupancy with Launch Bounds
  - 内存层级, Understanding GPU Memory Hierarchy-Understanding GPU Memory Hierarchy
  - 特殊函数单元, Understanding GPU Architecture
  - 流式多处理器, Understanding GPU Architecture
  - 线程块集群, Threads, Warps, Blocks, and Grids
  - 线程块大小, Choosing Threads-per-Block and Blocks-per-Grid Sizes
  - 线程块, Threads, Warps, Blocks, and Grids-Threads, Warps, Blocks, and Grids
  - 线程, Threads, Warps, Blocks, and Grids-Threads, Warps, Blocks, and Grids
  - 每块线程数和每网格线程块数, Choosing Threads-per-Block and Blocks-per-Grid Sizes-Choosing Threads-per-Block and Blocks-per-Grid Sizes
  - 统一 CPU-GPU 内存, The CPU and GPU Superchip
  - warp, Streaming Multiprocessor, Threads, and Warps, Understanding GPU Architecture-Understanding GPU Architecture
- 分析和诊断瓶颈, Profiling and Diagnosing GPU Bottlenecks-Profiler-Guided Analysis
  - 概述, Profiling and Diagnosing GPU Bottlenecks
  - 实际占用率和 GPU 利用率, Inspecting Achieved Occupancy and GPU Utilization-Optimizing the Kernel
  - 数据管线分析和调优, Profiling and Tuning the Data Pipeline
  - 迭代分析, Iteratively Profiling and Determining the Kernel Bottleneck-Iteratively Profiling and Determining the Kernel Bottleneck
  - 内核优化, Optimizing the Kernel-Optimizing the Kernel
  - 生产环境监控, Monitoring System Metrics and Counters
  - Nsight Compute 和屋顶线分析, Nsight Compute and Roofline Analysis

  - Nsight Systems 时间线视图, Profiling and Diagnosing GPU Bottlenecks, Nsight Compute and Roofline Analysis
  - 通过 Kineto 使用 PyTorch 分析器, PyTorch Profiler and Visualization Tools-Profiler-Guided Analysis
  - warp 停顿原因, Analyzing Warp Stall Reasons with Nsight Compute-Other Stall Reasons
- 压缩数据, Tuning, Replicating, and Compressing Data
- 跨所有 SM 的最大并发内核数, Stream-Ordered Memory Allocator
- 容器运行时优化, Container Runtime Optimizations for GPUs-Reduce Image Size for Faster Container Startup
  - 容器概述, Container Runtime Optimizations for GPUs
  - 容器覆盖文件系统开销, Avoiding Container Overlay Filesystem Overhead
  - NVIDIA 容器运行时注入库, NVIDIA Container Runtime
  - NVIDIA Container Toolkit, Container Runtime Optimizations for GPUs
  - NVIDIA Container Toolkit 和 CUDA 兼容性, NVIDIA Container Toolkit and CUDA Compatibility
  - 减小镜像大小, Reduce Image Size for Faster Container Startup
- CPU-GPU 内存数据传输带宽, NUMA-Friendly Memory Allocation and Memory Pinning, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
- 用于与 CPU 主机通信的 CUDA 事件, Fine-Grained Synchronization with Events and Callbacks
  - 回调函数, Fine-Grained Synchronization with Events and Callbacks
- 使用 OpenAI Triton 的自定义内核, Writing Custom Kernels with OpenAI Triton-Profiling with Triton Proton Profiler
  - 在 PyTorch 中注册, Registering Custom Kernels with PyTorch-Registering Custom Kernels with PyTorch
  - 调优内核启动参数, Tuning Kernel-Launch Parameters
- 分布式多 GPU 通信, NCCL for Distributed Multi-GPU Communication-In-Network SHARP Aggregation
- FLOPs 增长快于内存带宽, Maintaining High Occupancy and GPU Utilization
- Meta 报告的大规模 GPU 故障, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely

- 强制 GPU-NIC 亲和性, Multinode and Multirack Communication with GPUDirect RDMA
- 使用 NVSHMEM 进行 GPU 到 GPU 内存共享, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM-Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM
- 空闲时间测量, Monitoring Storage I/O
- 通过 NVIDIA 设备插件接入 Kubernetes 调度器, Kubernetes for Topology-Aware Container Orchestration and Networking
- 内存访问模式
  - 避免共享内存, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization-Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization
  - 避免共享内存银行冲突, Avoid Shared-Memory Bank Conflicts-Avoid Shared-Memory Bank Conflicts
  - 合并与非合并全局内存访问, Coalesced Versus Uncoalesced Global Memory Access-Coalesced Versus Uncoalesced Global Memory Access
  - 只读数据缓存, Read-Only Data Caches-Read-Only Data Caches
  - 只读数据缓存的陷阱, Read-Only Data Caches
  - 对称内存, PyTorch Symmetric Memory
  - 用于分块获取的 Tensor Memory Accelerator, Asynchronous Memory Prefetching and Tensor Memory Accelerator-Asynchronous Memory Prefetching and Tensor Memory Accelerator
  - 通过共享内存进行分块和数据复用, Tiling and Data Reuse Using Shared Memory-Tiling and Data Reuse Using Shared Memory
  - 向量化内存访问, Vectorized Memory Access-Vectorized Memory Access
  - warp shuffle 内建函数, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization
- UCX/RDMA 下的内存碎片, Pitfall #6: GPU memory fragmentation under UCX/RDMA-Pitfall #6: GPU memory fragmentation under UCX/RDMA
- 多 GPU 编排, Orchestrate Across Multiple GPUs and Cluster Nodes (NVSHMEM)-Pattern for N-GPU Scaling
  - （另见 多 GPU）
  - GPU 到 GPU 内存共享, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM-Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM

  - NVIDIA SHMEM 库, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM-Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM
- 并行性
  - 顺序和并行地相加两个向量, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
  - GPU 架构, Understanding GPU Architecture
  - 子内核启动上限, Dynamic Parallelism
  - cuTile, C++ and Python CUDA Libraries
  - 数据并行与分布式数据并行的比较, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
  - 动态并行, Dynamic Parallelism-Dynamic Parallelism
  - 完全分片数据并行, Distributed Data Parallel Strategies
  - PTX, Inline PTX and SASS Tuning for Microoptimizations
  - 线程块集群算法, Designing Efficient Algorithms with Thread Block Clusters-Designing Efficient Algorithms with Thread Block Clusters
- 功率限制, Performance Monitoring and Utilization in Practice, Power and Thermal Constraints
- 进程-GPU 映射, Adaptive Process-GPU Mapping-Adaptive Process-GPU Mapping
- CUDA 编程, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
  - 内存层级, Understanding GPU Memory Hierarchy-Understanding GPU Memory Hierarchy
- 直接从存储设备读取数据, Using NVIDIA GDS-Measuring GDS with gdsio
- 用于性能的运行时设置, GPU Driver and Runtime Settings for Performance-GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
  - 内存 ECC 模式, GPU Clock Speeds and ECC
  - GPU 时钟频率, GPU Clock Speeds and ECC
  - GPU 内存碎片和超额订阅, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling-GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
  - GPU 持久化模式, GPU Persistence Mode
  - 保持 GPU 驱动程序已加载, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling

    - MIG, MIG-MIG
    - MPS, MPS-MPS
  - 利用率与延迟的权衡, Maximizing GPU Utilization and Throughput Versus Latency Trade-Offs
  - 通过多实例 GPU 进行虚拟化, Managing Resources Efficiently, Slicing a GPU with MIG

重点为 Grace Blackwell，但不限于此, NVIDIA’s “AI Supercomputer in a Rack”

  - 通过 NVLink-C2C 连接的 CPU 和 GPU, NUMA Awareness and CPU Pinning, NUMA Awareness and CPU Pinning

Grace Blackwell GB200 NVL72, GPU and CPU-GPU Superchip Improvements, NVLink/NVSwitch Topology and Bandwidth Constraints

Grace Blackwell GB200 Superchip, The CPU and GPU Superchip

Grace Blackwell 超级芯片, NVIDIA’s “AI Supercomputer in a Rack”

  - 内存, The CPU and GPU Superchip, Topology Awareness in NCCL
  - NIXL 将 KV 缓存卸载到 CPU 内存, KV Cache Offloading with NIXL-KV Cache Offloading with NIXL
  - 使用 CPU-GPU 统一内存的技巧, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction

Grace Blackwell Ultra GB300 NVL72, Blackwell Ultra and Grace Blackwell Ultra, NVLink/NVSwitch Topology and Bandwidth Constraints

Grace CPU, NVIDIA Grace CPU

  - Grace Blackwell Superchip, NVIDIA’s “AI Supercomputer in a Rack”
  - memory, The CPU and GPU Superchip
  - NVL72, Ultrascale Networking Treating Many GPUs as One

Grace Hopper GH200, The CPU and GPU Superchip

Grace Hopper 超级芯片：NIXL 卸载 KV 缓存, KV Cache Offloading with NIXL-KV Cache Offloading with NIXL

梯度检查点, Activation Checkpointing for Memory Savings

  - （另见 激活检查点）

Grafana 仪表板

  - 推理监控, Monitoring System Metrics and Counters, Monitoring System Metrics and Counters

– NVLink/NVSwitch 监控, Real-Time Link Telemetry and Monitoring

graph breaks, Explaining and Minimizing Graph Breaks-Tips for Handling Graph Breaks

– avoiding whenever possible, TorchDynamo for Bytecode Capture and Graph Extraction, Graph Breaks and TorchDynamo explain()
  – conditionals as tensor operations, Graph Breaks and TorchDynamo explain()
  – data-dependent branches, Graph Breaks and TorchDynamo explain()
  – marking functions and code blocks as safe, Mark Functions and Code Blocks as Safe with allow_in_graph
  – minimizing graph recompilations, Minimize Graph Recompilations
– causes, Tips for Handling Graph Breaks-Tips for Handling Graph Breaks
  – sequence not captured into a single graph, Graph Breaks and TorchDynamo explain()
  – shape mismatches, Profiling and Debugging Compiler Performance Issues
  – tips for handling common causes, Tips for Handling Graph Breaks-Tips for Handling Graph Breaks
  – unsupported operations in model, Profiling and Debugging Compiler Performance Issues
– complex graphs requiring graph breaks, Graph Breaks and TorchDynamo explain()
– debugging, Graph Breaks and TorchDynamo explain(), Debugging Compiler Phases, Graph Breaks, and Performance
  – monitoring graph recompilations, TorchDynamo for Bytecode Capture and Graph Extraction, Minimize Graph Recompilations
– forcing an error when full graph not captured, Graph Breaks and TorchDynamo explain()
– PyTorch compiler trace, Using PyTorch Profiler
– TorchDynamo, TorchDynamo for Bytecode Capture and Graph Extraction
  – explain() output, Profiling and Debugging Compiler Performance Issues, FSDP with torch.compile
  – forcing error to be raised at compile, TorchDynamo for Bytecode Capture and Graph Extraction

网格（grid）, Threads, Warps, Blocks, and Grids-Threads, Warps, Blocks, and Grids

- 每网格线程块数, Choosing Threads-per-Block and Blocks-per-Grid Sizes-Choosing Threads-per-Block and Blocks-per-Grid Sizes
- blocksPerGrid 参数, CUDA Programming Refresher, Configuring Launch Parameters: Blocks per Grid and Threads per Block-Configuring Launch Parameters: Blocks per Grid and Threads per Block
- 协作组
  - 协作式内核启动网格大小, Cooperative Groups, Cooperative Grid Synchronization and Persistent Kernels
  - 通过网格同步实现持久化内核, Cooperative Grid Synchronization and Persistent Kernels-Cooperative Grid Synchronization and Persistent Kernels
- 最大维度, Choosing Threads-per-Block and Blocks-per-Grid Sizes

H

H100 与 NVL72 的比较, Multi-GPU Programming

半精度, BF16/FP16, FP8, and FP4 Reduced Precision

硬件

- 盒装 AI 超级计算机概述, AI System Hardware Overview
  - 缓存一致性, The CPU and GPU Superchip
  - CPU + GPU 超级芯片, The CPU and GPU Superchip-Streaming Multiprocessor, Threads, and Warps
  - GPU Tensor Core 和 Transformer Engine, NVIDIA GPU Tensor Cores and Transformer Engine-NVIDIA GPU Tensor Cores and Transformer Engine
  - 将多个 GPU 视为一个, Ultrascale Networking Treating Many GPUs as One-Co-Packaged Optics: Future of Networking Hardware
  - 多 GPU 编程, Multi-GPU Programming-Multi-GPU Programming
  - 多机架和存储通信, Multirack and Storage Communication
  - NVIDIA Blackwell 双裸片 GPU, NVIDIA Blackwell “Dual-Die” GPU-NVIDIA Blackwell “Dual-Die” GPU
  - NVIDIA Grace CPU, NVIDIA Grace CPU
  - NVL72, Ultrascale Networking Treating Many GPUs as One
    - （另见 NVL72）

    - 流式多处理器、线程和 warp, Streaming Multiprocessor, Threads, and Warps-Streaming Multiprocessor, Threads, and Warps
- 计算优化型与内存优化型的比较, Compute-Optimized Versus Memory-Optimized Hardware-Different precision for prefill and decode
- DeepSeek.AI 的出口限制, Introduction and AI System Overview, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China-DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China
- 解耦式预填充和解码, Heterogeneous Hardware and Parallelism Strategies for Prefill and Decode-Different precision for prefill and decode
- 未来演进, Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models-Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models
- future hardware from NVIDIA, A Glimpse into the Future: NVIDIA’s Roadmap-Feynman GPU (2028) and Doubling Something Every Year
    - Blackwell Ultra and Grace Blackwell Ultra, Blackwell Ultra and Grace Blackwell Ultra
    - doubling every year, Feynman GPU (2028) and Doubling Something Every Year
    - Feynman GPU, Feynman GPU (2028) and Doubling Something Every Year
    - Rubin Ultra and Vera Rubin Ultra, Rubin Ultra and Vera Rubin Ultra (2027)
    - Vera Rubin Superchip, Vera Rubin Superchip (2026)
- 跨世代 GPU 编程, CUDA Forward and Backward Compatibility Across GPU Hardware Generations
- GPU 接入 Kubernetes 调度器, Kubernetes for Topology-Aware Container Orchestration and Networking
- 软硬件协同设计, Mechanical Sympathy: Hardware-Software Codesign-Mechanical Sympathy: Hardware-Software Codesign
    - CUDA Pipeline API 加 TMA, Asynchronous Memory Prefetching and Tensor Memory Accelerator
    - DeepSeek MLA, Mechanical Sympathy: Hardware-Software Codesign
- 采用同构硬件以避免拖尾节点或进程, Pitfall #5: Straggler nodes or processes
- 限制占用率的因素, Choosing Threads-per-Block and Blocks-per-Grid Sizes, Choosing Threads-per-Block and Blocks-per-Grid Sizes

- 优化开源, Transparency and Reproducibility
- Page Migration Engine, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
- 升级的投资回报率（ROI）, ROI of Upgrading Your Hardware
- 屋顶线模型, Roofline Model: Compute-Bound or Memory-Bound Workloads-Roofline Model: Compute-Bound or Memory-Bound Workloads

HBM（高带宽内存，high-bandwidth memory）, Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models

隐藏延迟（参见 延迟隐藏）

高占用率, Threads, Warps, Blocks, and Grids
- 维持高占用率, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
  - 占用率的影响, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
- CUDA 编程, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization

高带宽内存（HBM）, Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models

整体追踪分析（HTA；PyTorch）, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools
- 多 GPU 分析, Multi-GPU Profiling with HTA

Hopper H100, Introduction and AI System Overview

HTA（参见 整体追踪分析（HTA；PyTorch））

大页, Transparent Hugepages
- 透明大页, Transparent Hugepages

Hugging Face Transformers 库
- CTRL, Prompt Cleansing
- KV 缓存卸载, Speculative KV Prefetching for Faster TTFT
- KV 缓存量化, Real-Time KV Cache Compression and Policy Switching
  - 半二次量化实现, Real-Time KV Cache Compression and Policy Switching

Hugging Face Ultra-Scale Playbook, Smart Compilers and Automated Code Optimizations

混合并行, Hybrid Parallelism-Hybrid Parallelism
- 与 TP、PP 的比较, Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)-Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)

超线程, NUMA Awareness and CPU Pinning

I

IB 网络（参见 InfiniBand（IB）网络）

IBTA（InfiniBand 行业协会）, Multi-GPU Programming

ILP（参见 指令级并行（ILP））

飞行中批处理, Continuous Batching

Inductor（参见 TorchInductor（PyTorch））

推理
- 批处理
  - 提高算术强度, Structured Sparsity
  - 比较静态、动态和连续批处理, Continuous Batching
  - 连续批处理, Continuous Batching
  - 动态批处理, Dynamic Batching-Dynamic Batching
  - 延迟感知调度, Latency-Aware Scheduling and Dynamic Routing-Latency-Aware Scheduling and Dynamic Routing
  - 无停顿调度, Stall-Free Scheduling (Chunked Prefill)
  - 静态批处理, Dynamic Batching
- 连续调度, Continuous Scheduling
- CUDA Graph 和推理引擎, PyTorch, Inference Engines, and CUDA Graphs
- 解耦式预填充和解码, Multinode Inference, Parallelism, Decoding, and Routing Optimizations
  - （另见 解耦式预填充和解码）
- MoE 模型的动态路由策略, Dynamic Routing Strategies for MoE Inference-Adaptive Expert Routing and Real-Time Monitoring
  - 专家通信优化, Expert Communication Optimization-Expert Communication Optimization
  - 专家复制, Load Balancing, Capacity Factor, and Expert Replication

    - 负载均衡和容量因子, Load Balancing, Capacity Factor, and Expert Replication
- NIXL 提升 Dynamo 吞吐量, NIXL and High-Performance Inference Systems Like NVIDIA Dynamo
- 大规模推理集群, Data Parallelism
- 巨型内核, Megakernels for Inference
- 多节点推理, Multinode Inference, Parallelism, Decoding, and Routing Optimizations
    - 解耦式推理（参见 解耦式预填充和解码）
    - 使用 Kubernetes 部署, Deploying Disaggregated Prefill and Decode with Kubernetes-Deploying Disaggregated Prefill and Decode with Kubernetes, Full-Stack Inference Optimizations
- NIXL 解耦式推理, NVIDIA’s NIXL and Disaggregated Inference-NCCL Versus NIXL
    - 关于 NIXL, NCCL for Distributed Multi-GPU Communication, NVIDIA’s NIXL and Disaggregated Inference-NVIDIA’s NIXL and Disaggregated Inference
    - 带回调的异步 API, NIXL Asynchronous API with Callbacks-NIXL Asynchronous API with Callbacks
    - 智能互连路由, Intelligent Interconnect Routing for KV Cache Transfers
    - 使用 NIXL 卸载 KV 缓存, KV Cache Offloading with NIXL-KV Cache Offloading with NIXL
    - 分离预填充和解码推理阶段, Separate Prefill and Decode Inference Stages-Separate Prefill and Decode Inference Stages
- 优化
    - 超大语言模型推理概述, Dynamic and Adaptive Inference Engine Optimizations
    - 自适应批处理, Adaptive Batching and Chunked Prefill Scheduling-Adaptive Batching and Chunked Prefill Scheduling
    - 自适应并行策略, Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)-Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)
    - 分块预填充调度, Adaptive Batching and Chunked Prefill Scheduling-Adaptive Batching and Chunked Prefill Scheduling
    - 动态激活量化, Dynamic Quantization and Activation Range Adjustment
    - 动态早退网络, Dynamic Early-Exit Networks

  - 动态内存分配切换, Dynamic Memory-Allocation Switching (Slab Versus Caching Versus Stream-Ordered)-Dynamic Memory-Allocation Switching (Slab Versus Caching Versus Stream-Ordered)
  - 动态精度变更, Dynamic Precision Changes-Dynamic Precision Changes
  - 动态共享内存分配, Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection-Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection
  - 动态 token 剪枝, Dynamic Token Pruning with LazyLLM
  - 面向边缘的 MoE 内存预算, Edge-Oriented MoE Memory Budgeting
  - 输入感知层跳过, Input-Aware Layer Skipping (DASH)
  - 内核自动调优, Kernel Autotuning for Transformer Self-Attention and MLP Paths-Kernel Autotuning for Transformer Self-Attention and MLP Paths
  - KV 缓存压缩和策略切换, Real-Time KV Cache Compression and Policy Switching-Real-Time KV Cache Compression and Policy Switching
  - MoE 推测专家路由, Speculative MoE Expert Routing and Communication Reduction
  - 多 GPU 拥塞感知和拓扑感知调度, Congestion-Aware and Topology-Aware Scheduling with Multiple GPUs-Coordinating NVSwitch Transfers with Fine-Tuned Scheduling
  - 占用率感知内核选择, Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection-Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection
  - 预热图和缓存, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction-Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction
  - 运行时内核修补, Runtime Kernel Performance Improvements and Hot-Swappable Implementations-Runtime Kernel Performance Improvements and Hot-Swappable Implementations
  - 推测性 KV 预取, Speculative KV Prefetching for Faster TTFT-Speculative KV Prefetching for Faster TTFT
  - 通过 RL 智能体在运行时调优 AI, Reinforcement Learning Agents for Tuning AI Systems at Runtime
- 应用级优化, Application-Level Optimizations-Token Output Limits and Timeouts
  - 去抖和请求合并, Debouncing and Request Coalescing

  - 模型级联和分层部署, Model Cascading and Tiered Model Deployment-Model Cascading and Tiered Model Deployment
  - 前缀缓存, Prefix Caching-Prefix Caching
  - 提示词清洗, Prompt Cleansing
  - 提示词压缩, Prompt Compression
  - 流式响应, Streaming Responses-Streaming Responses
  - token 输出限制和超时, Token Output Limits and Timeouts
- optimizations at the systems level, Systems-Level Optimizations
  - GPU utilization versus latency, Maximizing GPU Utilization and Throughput Versus Latency Trade-Offs
  - KV cache, KV Cache Offloading and Memory Pool Allocation
  - memory, Memory
  - overlapping communication and computation, Overlapping Communication and Computation-Overlapping Communication and Computation
  - power and thermal constraints, Power and Thermal Constraints
- 实时推理量化, Quantization Approaches for Real-Time Inference-Fusing Quantization-Dequantization Steps into the Execution Graph
  - 量化概述, Quantization Approaches for Real-Time Inference
  - 激活量化, Activation Quantization
  - 结合权重量化和激活量化, Combining Weight and Activation Quantization
  - 训练后量化流程, Post-Training Quantization Workflow
  - 将量化-反量化步骤融合到执行图, Fusing Quantization-Dequantization Steps into the Execution Graph
  - 降低精度, Reducing Precision from FP16 to FP8 and FP4
  - 仅权重量化, Weight-Only Quantization (GPTQ, AWQ)
- 用于推理工作负载的结构化稀疏, Structured Sparsity
- 性能调优
  - 正确性问题调试, Debugging Correctness Issues-Debugging Correctness Issues
  - 全栈优化, Full-Stack Inference Optimizations-Full-Stack Inference Optimizations

- 监控系统指标和计数器, Monitoring System Metrics and Counters-Monitoring System Metrics and Counters
- 观察-假设-调优循环, Profiling, Debugging, and Tuning Inference Performance-Profiling, Debugging, and Tuning Inference Performance
- 使用 Nsight 进行性能分析, Profiling with Nsight Systems and Nsight Compute-Profiling with Nsight Systems and Nsight Compute
- 通过强化学习智能体进行运行时调优, Reinforcement Learning Agents for Tuning AI Systems at Runtime
- 系统级优化, Systems-Level Optimizations
- 故障排除, Inference Troubleshooting Recipes

InfiniBand（IB）网络

- 容器访问 InfiniBand 设备, High-Speed, Low-Overhead Data Transfers with RDMA
- 利用节点计数器发现拖尾节点, Pitfall #5: Straggler nodes or processes
- 启用 GPUDirect RDMA, Optimizing Network Communication for Kubernetes
- 主机通道适配器, Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL
- RDMA 支持, High-Speed, Low-Overhead Data Transfers with RDMA
- SHARP, In-Network SHARP Aggregation

InfiniBand 贸易协会（IBTA）, Multi-GPU Programming

输入

- 变长序列, Dynamic Shapes and Variable Sequence Lengths-Dynamic Shapes and Variable Sequence Lengths
  - 动态形状, Dynamic Shapes and Variable Sequence Lengths-Dynamic Shapes and Variable Sequence Lengths
  - 填充, Dynamic Shapes and Variable Sequence Lengths
  - 分析动态形状与填充的性能, Dynamic Shapes and Variable Sequence Lengths
- 超长输入
  - 上下文并行, Context (Sequence) Parallelism
  - 提示词压缩, Prompt Compression
  - 无停顿调度, Stall-Free Scheduling (Chunked Prefill)

使用 PTX 进行指令调度, Inline PTX and SASS Tuning for Microoptimizations

指令级并行（ILP）

- 利用
  - 编译器提示, Exposing Instruction-Level Parallelism, Loop Unrolling, Interleaving, and Compiler Hinting
  - 交错执行独立操作, Loop Unrolling, Interleaving, and Compiler Hinting
  - 循环展开, Loop Unrolling, Interleaving, and Compiler Hinting
- 暴露, Exposing Instruction-Level Parallelism-Profiling and Mitigating Register Pressure
  - 编译器提示, Exposing Instruction-Level Parallelism, Loop Unrolling, Interleaving, and Compiler Hinting
  - 占用率和 ILP, Exposing Instruction-Level Parallelism, ILP and Occupancy, Loop Unrolling, Interleaving, and Compiler Hinting
  - warp 调度器和双发射指令, Warp Scheduling and Dual Issue Instructions-Warp Scheduling and Dual Issue Instructions
- 为延迟受限 GPU 提升 ILP, Iteratively Profiling and Determining the Kernel Bottleneck
- 使用 PTX 提升 ILP, Inline PTX and SASS Tuning for Microoptimizations
- “执行更多操作”这一误解, Exposing Instruction-Level Parallelism
- PyTorch 编译器不保证 ILP, Using Predication to Minimize Divergence
- 寄存器压力, Profiling and Mitigating Register Pressure
- warp 停顿原因, Other Stall Reasons, Other Stall Reasons

INT8, INT8 Reduced Precision and DP4A Instructions for Inference

核间流水线

- 核间流水线概述, Inter-Kernel Pipelining, Synchronization, and CUDA Stream-Ordered Memory Allocations
- 通过流使计算与数据传输重叠, Using Streams to Overlap Compute with Data Transfers-Using Streams to Overlap Compute with Data Transfers
  - 并发, Overlapping Communication and Computation-Overlapping Communication and Computation
  - 多 GPU, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams

- CUDA 流有序内存分配器, Stream-Ordered Memory Allocator-Stream-Ordered Memory Allocator
  - 使用, Using CUDA Streams and Stream-Ordered Memory Allocator with LLMs-Using CUDA Streams and Stream-Ordered Memory Allocator with LLMs
  - 与多 GPU 一起使用, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
- 默认流
  - 默认流与显式流, Default Versus Explicit (Nondefault) Streams
  - 旧式默认流, Legacy Default Stream
  - 每线程默认流, Modern Per-Thread Default Stream
  - 使用默认流, Best Practices for Default Stream Usage-Best Practices for Default Stream Usage
- 使用 CUDA 流重叠内核执行, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
  - 在两个流上启动五个内核, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
  - 以 CUDA 流替代 warp 专用化, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
- 多 GPU 计算与数据传输重叠, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
- 程序化依赖启动（Programmatic Dependent Launch）, Programmatic Dependent Launch-Programmatic Dependent Launch
  - 线程块集群和 warp 专用化, Combining PDL and Thread Block Clusters with Warp Specialization-Combining PDL and Thread Block Clusters with Warp Specialization
- 使用事件和回调同步, Fine-Grained Synchronization with Events and Callbacks
  - 跨流同步, Using CUDA Events for Cross-Stream Synchronization
- warp 专用化
  - CUDA 流, Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)-Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)

    - 线程块集群和 CUDA 流, Warp Specialization with Thread Block Clusters and CUDA Streams

中断请求（IRQ）

- irqbalance 守护进程, Scheduler and Interrupt Affinity
- 调度器和中断亲和性, Scheduler and Interrupt Affinity

核内流水线, Intra-Kernel Pipelining Techniques

- 核内流水线概述, Intra-Kernel Pipelining Techniques
- 比较技术, Intra-Kernel Pipelining Techniques, Using CUDA Pipeline API for Warp Specialization, Warp Specialization with Thread Block Clusters-Warp Specialization with Thread Block Clusters
- 协作组, Cooperative Groups-When to Combine Persistent Kernels and Cooperative Groups
    - Cooperative Groups API, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization, Cooperative Groups, Coordinating Thread Block Clusters with Cooperative Groups API-Coordinating Thread Block Clusters with Cooperative Groups API
    - 说明, Thread Block Clusters and Distributed Shared Memory
    - 以协作模式启动内核, Cooperative Groups, When to Combine Persistent Kernels and Cooperative Groups
    - 通过网格同步使用持久化内核, Cooperative Grid Synchronization and Persistent Kernels-Cooperative Grid Synchronization and Persistent Kernels
    - 线程块集群与……的比较, Thread Block Clusters and Distributed Shared Memory
    - 何时与持久化内核结合, When to Combine Persistent Kernels and Cooperative Groups
- 使用 CUDA Pipeline API 进行双缓冲, Cooperative Tiling and Double-Buffering with the CUDA Pipeline API-Cooperative Tiling and Double-Buffering with the CUDA Pipeline API
- 持久化内核, Persistent Kernels and Megakernels-Persistent Kernels and Warp Specialization
    - 适用的常见工作负载, Common Workloads for Persistent Kernels
    - 通过网格同步使用协作组, Cooperative Grid Synchronization and Persistent Kernels-Cooperative Grid Synchronization and Persistent Kernels
    - 巨型内核, Megakernels for Inference

    - 占用率, Persistent Kernels and Megakernels
    - warp 专用化, Persistent Kernels and Warp Specialization
    - 何时与协作组结合, When to Combine Persistent Kernels and Cooperative Groups
  - 线程块集群, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters
    - (参见 线程块集群)
  - warp 专用化, Warp Specialization and the Producer-Consumer Model-Warp Specialization and the Producer-Consumer Model
    - CUDA Pipeline API, Using CUDA Pipeline API for Warp Specialization-Using CUDA Pipeline API for Warp Specialization
    - PyTorch 中的 CUDA Pipeline API, PyTorch, CUDA Pipeline API, and Warp Specialization
    - CUDA 流, Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)-Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)
    - 线程块集群, Warp Specialization with Thread Block Clusters-Warp Specialization with Thread Block Clusters
  - 何时使用 warp 专用化或双缓冲, Warp Specialization and the Producer-Consumer Model

输入/输出（I/O）

  - 通信与计算瓶颈, Diagnosing Communication- Versus Compute-Bound Workloads
  - 持续性能分析和调优工作流, Continuous Profiling and Tuning Workflow-Continuous Profiling and Tuning Workflow
  - 数据管线调优, Tuning the Data Pipeline
    - 数据加载和预处理, Efficient Data Loading and Preprocessing-Efficient Data Loading and Preprocessing
    - 用于训练数据集的 NeMo Curator, Creating High-Quality LLM Datasets with NVIDIA NeMo Curator
    - NVIDIA 数据加载库, Multimodal Data Processing with NVIDIA DALI
    - 随 GPU 数扩展工作进程, Scaling Out Workers as You Scale Out Number of GPUs
  - Kubernetes 中的 I/O 隔离, Dealing with I/O Isolation

- 监控, Monitoring Storage I/O-Monitoring Storage I/O
- NVIDIA Magnum IO, NVIDIA Magnum IO Optimization Stack
- 优化基于 GPU 的存储
  - 云存储缓存, Distributed, Parallel Filesystems and Object Stores
  - 压缩数据, Tuning, Replicating, and Compressing Data
  - DeepSeek Fire-Flyer File System, DeepSeek’s Fire-Flyer File System-DeepSeek’s Fire-Flyer File System
  - 快速存储和数据局部性, Fast Storage and Data Locality
  - 优化文件系统, Sequential Versus Random Read Patterns
  - 多队列块 I/O 调度器, Tuning NVMe and Filesystem for Throughput
  - NFS 服务器, Distributed, Parallel Filesystems and Object Stores
  - NFS 服务器调优参数, Distributed, Parallel Filesystems and Object Stores
  - NVIDIA GDS, Using NVIDIA GDS-Measuring GDS with gdsio
  - 针对吞吐量调优 NVMe 和文件系统, Tuning NVMe and Filesystem for Throughput
  - 并行文件系统, Distributed, Parallel Filesystems and Object Stores
  - 预读, Tuning NVMe and Filesystem for Throughput
  - 调优读取大小, Sequential Versus Random Read Patterns
  - 顺序读取与随机读取, Sequential Versus Random Read Patterns-Sequential Versus Random Read Patterns
  - 共享文件系统, Distributed, Parallel Filesystems and Object Stores
  - 文件条带化, Distributed, Parallel Filesystems and Object Stores
  - 调优文件系统, Tuning, Replicating, and Compressing Data

irqbalance 守护进程, Scheduler and Interrupt Affinity

迭代级调度, Continuous Batching

J

jemalloc, Tune Host CPU Memory Allocator

使用 Kubernetes 和 SLURM 的作业调度, Kubernetes 与 Slurm 作业调度

K

K8s (参见 Kubernetes（K8s))

内核自动调优 (参见 自动调优)

内核缓存, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction

内核融合

- 提高算术强度, Kernel Fusion
- 使用 PyTorch 后较少需要自定义融合, FlexDecoding (PyTorch)
- FlexAttention, Autotuning with TorchInductor, FlexDecoding (PyTorch)
- 内存受限工作负载, Roofline-Guided Scheduling and Orchestration Decisions
- TorchInductor, Tuning Occupancy with PyTorch, Kernel Fusion, TorchDynamo for Bytecode Capture and Graph Extraction
  - fused attention kernels, PyTorch, CUDA Pipeline API, and Warp Specialization
  - GEMM kernel prologue and epilogue fusion, Using the PyTorch Compiler

使用 CUDA Graphs 降低内核启动开销, Reducing Kernel Launch Overhead with CUDA Graphs-CUDA Graph Trees (PyTorch Compiler Internal)

- CUDA Graphs 最佳实践, Best Practices for CUDA Graphs-Best Practices for CUDA Graphs
- 捕获 CUDA Graph, Capturing a CUDA Graph and Preallocating Memory-Capturing a CUDA Graph and Preallocating Memory
- 重放图, Replaying the Graph-Replaying the Graph
  - 使用任意流进行重放, Capturing a CUDA Graph and Preallocating Memory

内核优化, Optimizing the Kernel-Optimizing the Kernel

- 自动化, Automated GPU Kernel Optimizations with DeepSeek-R1 (NVIDIA)-Automated GPU Kernel Optimizations with DeepSeek-R1 (NVIDIA)
- 强化学习方法, Reinforcement Learning Approach to Generating Optimized GPU Kernels (Predibase)-Reinforcement Learning Approach to Generating Optimized GPU Kernels (Predibase)
- 运行时内核修补, Runtime Kernel Performance Improvements and Hot-Swappable Implementations-Runtime Kernel Performance Improvements and Hot-Swappable Implementations

Kimi K2 (Moonshot AI), Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models

Kineto（PyTorch 性能分析器）, PyTorch Profiler and Visualization Tools, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools

- NCCL 性能分析器插件, Profiling and Debugging NCCL

Kubernetes (K8s)

- 概述, Kubernetes for Topology-Aware Container Orchestration and Networking
- 集群调度器, Sharing and Scheduling
- 部署解耦式推理系统, Deploying Disaggregated Prefill and Decode with Kubernetes-Deploying Disaggregated Prefill and Decode with Kubernetes, Full-Stack Inference Optimizations
- 应调优主机, Dealing with I/O Isolation
- I/O 隔离, Dealing with I/O Isolation
- 改善资源保障, Improving Resource Guarantees
- 作业调度, Job Scheduling with Kubernetes and SLURM
- 内存隔离, Memory Isolation and Avoiding the OOM Killer
- 内存限制, Memory Isolation and Avoiding the OOM Killer
- 设备插件中的 MIG 设备, MIG
- MIG 模式, Slicing a GPU with MIG
- 禁用交换并强制 NUMA 亲和性, Virtual Memory and Swapping
- NVIDIA Container Toolkit, Container Runtime Optimizations for GPUs
- NVIDIA device plugin for Kubernetes, Kubernetes for Topology-Aware Container Orchestration and Networking
- NVIDIA Kubernetes GPU Operator, Kubernetes for Topology-Aware Container Orchestration and Networking
- NVL72 部署, Preintegrated Rack Appliance
- 优化网络通信, Optimizing Network Communication for Kubernetes
- 使用 Topology Manager 编排容器, Orchestrating Containers with Kubernetes Topology Manager
- 降低编排抖动, Reducing Kubernetes Orchestration Jitter
- RDMA, High-Speed, Low-Overhead Data Transfers with RDMA
- 运行时优化
  - 容器概述, Container Runtime Optimizations for GPUs
  - 容器覆盖文件系统开销, Avoiding Container Overlay Filesystem Overhead
  - NVIDIA 容器运行时注入库, NVIDIA Container Runtime

    - NVIDIA Container Toolkit 与 CUDA 兼容性, NVIDIA Container Toolkit and CUDA Compatibility
    - 减小镜像大小, Reduce Image Size for Faster Container Startup
  - 面向 GPU 的运行时优化, Container Runtime Optimizations for GPUs-Reduce Image Size for Faster Container Startup
  - 拓扑感知, Kubernetes for Topology-Aware Container Orchestration and Networking
  - Topology Manager, Orchestrating Containers with Kubernetes Topology Manager

Kubernetes Topology Manager (NVIDIA), Orchestrating Containers with Kubernetes Topology Manager

KV 缓存, NVIDIA’s NIXL and Disaggregated Inference

  - 压缩和策略切换, Real-Time KV Cache Compression and Policy Switching-Real-Time KV Cache Compression and Policy Switching
  - 推理调优, KV Cache Offloading and Memory Pool Allocation
  - NIXL 简化传输, NVIDIA’s NIXL and Disaggregated Inference
    - 用于传输的智能互连, Intelligent Interconnect Routing for KV Cache Transfers
    - 卸载到 CPU 内存, NVIDIA’s NIXL and Disaggregated Inference, KV Cache Offloading with NIXL-KV Cache Offloading with NIXL
    - 预填充和解码推理阶段, Separate Prefill and Decode Inference Stages-Separate Prefill and Decode Inference Stages
    - 预填充到解码的数据传输, KV Cache Data Transfer and NIXL, Fast KV Cache Transfer Between Prefill and Decode-Zero-Copy GPU-to-GPU Transfer
  - 预填充阶段, Scaling Disaggregated Prefill and Decode for Inference
  - 前缀合并事件, Profiling, Debugging, and Tuning Inference Performance
  - 推测性 KV 预取, Speculative KV Prefetching for Faster TTFT-Speculative KV Prefetching for Faster TTFT
    - KV 缓存卸载, Speculative KV Prefetching for Faster TTFT
  - 超级芯片内存带宽, GPU and CPU-GPU Superchip Improvements
  - 从预填充到解码的传输, KV Cache Data Transfer and NIXL, Fast KV Cache Transfer Between Prefill and Decode-Zero-Copy GPU-to-GPU Transfer
    - 连接器和数据路径设计, Connector and Data Path Design-Connector and Data Path Design

    - KV 缓存大小, KV Cache Size
    - 零拷贝 GPU 到 GPU, Zero-Copy GPU-to-GPU Transfer-Zero-Copy GPU-to-GPU Transfer
  - 调优, Tuning KV Cache Utilization and Management-GPU and CPU-GPU Superchip Improvements
    - 解耦式 KV 缓存池, Disaggregated KV Cache Pool-Disaggregated KV Cache Pool
    - 内存布局, Optimized KV Cache Memory Layout
    - 复用和前缀共享, KV Cache Reuse and Prefix Sharing-KV Cache Reuse and Prefix Sharing
  - vLLM PagedAttention, Continuous Scheduling, KV Cache Offloading and Memory Pool Allocation

KV Cache Manager (NVIDIA Dynamo), KV Cache Offloading with NIXL-KV Cache Offloading with NIXL

L

延迟

- 预填充和解码解耦, Impact on Latency (TTFT) and Throughput (TPOT)
- GPU 利用率与……的权衡, Maximizing GPU Utilization and Throughput Versus Latency Trade-Offs
- 延迟感知调度和动态路由, Latency-Aware Scheduling and Dynamic Routing-Latency-Aware Scheduling and Dynamic Routing
- 延迟受限 GPU 停顿, Iteratively Profiling and Determining the Kernel Bottleneck-Iteratively Profiling and Determining the Kernel Bottleneck
  - 内核优化, Optimizing the Kernel-Optimizing the Kernel
- 对延迟敏感的训练工作流, Filesystem Caching and Write-Back
- 动态批处理的最大延迟, Monitoring System Metrics and Counters
- 启用流式处理时进行性能分析, Streaming Responses

延迟隐藏, Streaming Multiprocessor, Threads, and Warps, Maintaining High Occupancy and GPU Utilization

- 在内核之间或 GPU 与主机之间 (参见 核间流水线)
- 用更高占用率更好地隐藏延迟, Tuning Occupancy
  - 高占用率加高 ILP, Loop Unrolling, Interleaving, and Compiler Hinting

  - 实际达到的占用率低, Inspecting Achieved Occupancy and GPU Utilization
  - 在内核内 (参见 核内流水线)

__launch_bounds__, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds, Techniques for Occupancy Tuning

  - occupancy optimization, Compiler Hints to Optimize Occupancy

LazyLLM, Dynamic Token Pruning with LazyLLM

__ldg(), Read-Only Data Caches

Linux

  - 将 CPU DRAM 视为 CPU NUMA 内存, NUMA Awareness and CPU Pinning
  - GPU 驱动程序, GPU Driver
  - GPU 服务器通常使用 Linux, Operating System
  - 避免内存交换, Virtual Memory and Swapping
  - NUMA 平衡, NUMA Awareness and CPU Pinning
  - numactl
    - CPU 绑定, NUMA Awareness and CPU Pinning, NUMA Awareness and CPU Pinning
    - 强制从特定 NUMA 节点分配内存, NUMA Awareness and CPU Pinning, NUMA-Friendly Memory Allocation and Memory Pinning
    - 启动时应用进程 CPU 和内存策略, NUMA-Friendly Memory Allocation and Memory Pinning
  - NVMe
    - DeepSeek Fire-Flyer File System, DeepSeek’s Fire-Flyer File System-DeepSeek’s Fire-Flyer File System
    - 快速存储和数据局部性, Fast Storage and Data Locality
    - GPUDirect Storage, NUMA-Friendly Memory Allocation and Memory Pinning, Using NVIDIA GDS-Measuring GDS with gdsio
    - 参数卸载到, Offloading Parameters to CPU and NVMe
    - 顺序读取与随机读取, Sequential Versus Random Read Patterns-Sequential Versus Random Read Patterns
    - 针对吞吐量调优, Tuning NVMe and Filesystem for Throughput
    - 优化 XFS, Sequential Versus Random Read Patterns
  - perf, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools

    - example MoE model, CPU and GPU Profiling with Linux perf-CPU and GPU Profiling with Linux perf
    - NVIDIA Performance Monitoring Unit, CPU and GPU Profiling with Linux perf
  - 调度器
    - 完全公平队列已过时, Tuning NVMe and Filesystem for Throughput
    - 完全公平调度器, Scheduler and Interrupt Affinity
    - 多队列块 I/O 调度器, Tuning NVMe and Filesystem for Throughput
    - none 调度器, Tuning NVMe and Filesystem for Throughput
    - 设置, Tuning NVMe and Filesystem for Throughput
  - Simple Linux Utility for Resource Management, Preintegrated Rack Appliance
    - cluster scheduler, Sharing and Scheduling

液冷与风冷, Liquid Cooling Versus Air Cooling-Liquid Cooling Versus Air Cooling

Little Red Whiting Hood, Acknowledgments

LLM 请求不均匀性, Monitoring System Metrics and Counters

LMCache (vLLM), Profiling, Debugging, and Tuning Inference Performance
  - disaggregated prefill and decode, Scaling Disaggregated Prefill and Decode for Inference, Zero-Copy GPU-to-GPU Transfer

负载均衡
  - 动态调度, Dynamic Scheduling and Load Balancing-Dynamic resource scaling
    - 自适应资源调度和热点预防, Adaptive Resource Scheduling and Hotspot Prevention-Dynamic resource scaling
    - Arrow 的自适应实例扩缩容, Arrow’s adaptive instance scaling-Arrow’s adaptive instance scaling
    - 动态资源扩缩容, Dynamic resource scaling
    - Mooncake 自适应策略, Mooncake adaptive strategies
    - TetriInfer 的两级调度器, TetriInfer’s two-level scheduler
  - 专家并行, Expert Parallelism
  - MoE 专家重平衡, MoE Expert Rebalancing and Regrouping
  - 时序负载均衡, Dynamic Congestion-Aware Scheduling

日志记录

- 推理监控, Monitoring System Metrics and Counters
- 用于性能基准测试的 MLPerf, Performance Benchmarks and MLPerf Logging-Performance Benchmarks and MLPerf Logging
- NCCL 日志, In-Network SHARP Aggregation, Debugging Correctness Issues
- PyTorch 编译器, Debugging Compiler Phases, Graph Breaks, and Performance
- TORCH_LOGS, Profiling and Debugging Compiler Performance Issues, Debugging Compiler Phases, Graph Breaks, and Performance
  - logging options summary, Debugging Compiler Phases, Graph Breaks, and Performance
  - performance hints, Performance Hints and Debugging Generated Code

循环展开, Loop Unrolling, Interleaving, and Compiler Hinting, Kernel Fusion

M

Magnum IO (NVIDIA), NVIDIA Magnum IO Optimization Stack, CPU and GPU Profiling with Linux perf

MALLOC_CONF, Tune Host CPU Memory Allocator

超大模型

- 推理集群, Data Parallelism
- 服务的并行策略, Parallelism Strategies for Serving Massive MoE Models-Hybrid Parallelism
  - 混合并行, Hybrid Parallelism-Hybrid Parallelism

矩阵乘法（matmul）

- AsyncTP, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
- 针对 FP8 优化的 DeepGEMM 库, Transparency and Reproducibility
- 持久化 Triton 内核, Tiled and Persistent GEMM Kernel (Triton)-Tiled and Persistent GEMM Kernel (Triton)

硬件亲和性, Mechanical Sympathy: Hardware-Software Codesign-Mechanical Sympathy: Hardware-Software Codesign

Medusa 多头推测解码, Speculative Decoding and Parallel Token Generation Techniques

- 多 token 解码, Multitoken Decoding with Medusa’s Multiple Heads-Multitoken Decoding with Medusa’s Multiple Heads

巨型内核, Megakernels for Inference

Mellanox, In-Network Aggregations with NVIDIA SHARP

内存

- 使用 CUTLASS 的异步内存传输, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance
- Blackwell GPU, The CPU and GPU Superchip
- 靠近 CPU、靠近 GPU, NUMA-Friendly Memory Allocation and Memory Pinning
- 超级芯片上的 CPU-GPU 互连, Topology Awareness in NCCL
  - 性能, The CPU and GPU Superchip
- CUDA 流有序内存分配器, Stream-Ordered Memory Allocator-Stream-Ordered Memory Allocator
  - 多 GPU, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - 与 CUDA 流一起使用, Using CUDA Streams and Stream-Ordered Memory Allocator with LLMs-Using CUDA Streams and Stream-Ordered Memory Allocator with LLMs
- CUDA 统一内存, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
  - CUDA 编程, Unified Memory-Unified Memory
  - 统一 CPU-GPU 内存, The CPU and GPU Superchip
- 分布式共享内存 (参见 分布式共享内存（DSMEM）)
- 动态内存分配切换, Dynamic Memory-Allocation Switching (Slab Versus Caching Versus Stream-Ordered)-Dynamic Memory-Allocation Switching (Slab Versus Caching Versus Stream-Ordered)
- GPU 内存的 ECC 模式, GPU Clock Speeds and ECC
- 面向边缘的 MoE 内存预算, Edge-Oriented MoE Memory Budgeting
- 监控碎片化, Monitoring System Metrics and Counters
  - 动态内存分配切换, Dynamic Memory-Allocation Switching (Slab Versus Caching Versus Stream-Ordered)-Dynamic Memory-Allocation Switching (Slab Versus Caching Versus Stream-Ordered)
- GDS, NUMA-Friendly Memory Allocation and Memory Pinning, Using NVIDIA GDS-Measuring GDS with gdsio, Offloading Parameters to CPU and

NVMe

- 减少全局内存流量, Reducing Global Memory Traffic with Thread Block Clusters-Reducing Global Memory Traffic with Thread Block Clusters
  - (另见 全局内存)
- GPU FLOPs 增长快于内存带宽, Maintaining High Occupancy and GPU Utilization
- GPU 内存访问模式
  - 避免共享内存, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization-Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization
  - 避免共享内存 bank 冲突, Avoid Shared-Memory Bank Conflicts-Avoid Shared-Memory Bank Conflicts
  - 合并与非合并全局内存访问, Coalesced Versus Uncoalesced Global Memory Access-Coalesced Versus Uncoalesced Global Memory Access
  - 只读数据缓存, Read-Only Data Caches-Read-Only Data Caches
  - 只读数据缓存陷阱, Read-Only Data Caches
  - 对称内存, PyTorch Symmetric Memory
  - 用于获取分块的 Tensor Memory Accelerator, Asynchronous Memory Prefetching and Tensor Memory Accelerator-Asynchronous Memory Prefetching and Tensor Memory Accelerator
  - 通过共享内存分块和数据复用, Tiling and Data Reuse Using Shared Memory-Tiling and Data Reuse Using Shared Memory
  - 向量化内存访问, Vectorized Memory Access-Vectorized Memory Access
- GPU 内存碎片化和超额订阅, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling-GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
- UCX/RDMA 下的 GPU 内存碎片化, Pitfall #6: GPU memory fragmentation under UCX/RDMA-Pitfall #6: GPU memory fragmentation under UCX/RDMA
- 使用 NVSHMEM 进行 GPU 到 GPU 内存共享, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM-Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM
- Grace Blackwell Superchip, The CPU and GPU Superchip, Topology Awareness in NCCL
- Grace CPU, The CPU and GPU Superchip

- 高带宽内存, Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models
- 大页, Transparent Hugepages
  - 透明大页, Transparent Hugepages
- 推理调优, Memory
- 内核内存吞吐量与 HBM 内存带宽, Kernel Memory Throughput Versus Peak HBM Memory Bandwidth
- Kubernetes
  - 内存隔离, Memory Isolation and Avoiding the OOM Killer
  - 内存限制, Memory Isolation and Avoiding the OOM Killer
- KV 缓存
  - 优化内存布局, Optimized KV Cache Memory Layout
  - 卸载到 CPU 内存, KV Cache Offloading with NIXL-KV Cache Offloading with NIXL
- 内存受限工作负载示例, Maintaining High Occupancy and GPU Utilization
- NUMA 友好的分配和固定, NUMA-Friendly Memory Allocation and Memory Pinning-NUMA-Friendly Memory Allocation and Memory Pinning
  - 固定内存的效率, NUMA-Friendly Memory Allocation and Memory Pinning
  - 最大锁定内存设置, Transparent Hugepages, Virtual Memory and Swapping
  - 操作系统对固定内存的限制, NUMA-Friendly Memory Allocation and Memory Pinning
  - 数据加载器的固定内存, NUMA-Friendly Memory Allocation and Memory Pinning
- 内存不足（OOM）错误
  - CPU + GPU 架构, The CPU and GPU Superchip
  - 最大锁定内存设置, Transparent Hugepages
  - 内存碎片化或过度缓存, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
  - 监控 GPU 内存使用量, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
- 通过 PyTorch 性能分析器进行性能分析和调优, Profiling and Tuning Memory in PyTorch-Enabling Peer-to-Peer DMA and UCX

  - 激活检查点, Activation Checkpointing for Memory Savings
  - 调优 CUDA 内存分配器, Tuning the CUDA Memory Allocator
  - FSDP 自动检查点, FSDP Automatic Checkpointing and Offloading-FSDP Automatic Checkpointing and Offloading
  - FSDP 与张量并行和流水线并行, Combining FSDP with Tensor Parallel and Pipeline Parallel
  - 多节点拓扑上的 NCCL 和 UCX, Enabling Peer-to-Peer DMA and UCX
  - 将参数卸载到 CPU 和 NVMe, Offloading Parameters to CPU and NVMe
  - 点对点 DMA, Enabling Peer-to-Peer DMA and UCX
  - 可插拔内存分配器, Pluggable Memory Allocators and Cross-GPU Data Transfers-Pluggable Memory Allocators and Cross-GPU Data Transfers
- CUDA 编程
  - 异步内存分配, Asynchronous Memory Allocation and Memory Pools-Asynchronous Memory Allocation and Memory Pools
  - CUDA 流, Asynchronous Memory Allocation and Memory Pools
  - GPU 内存层次结构, Understanding GPU Memory Hierarchy-Understanding GPU Memory Hierarchy
  - 内存池, Asynchronous Memory Allocation and Memory Pools-Asynchronous Memory Allocation and Memory Pools
- PyTorch 内存分配器插件, Pluggable Memory Allocators and Cross-GPU Data Transfers-Pluggable Memory Allocators and Cross-GPU Data Transfers
- 重计算与内存, Recomputation Versus Memory Trade-Off
- 暂存内存, Scratch Memory
  - DSMEM, Scratch Memory
    - (另见 分布式共享内存（DSMEM))
- 禁用共享内存交换, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables
  - (另见 共享内存（SMEM))
- 避免交换, Virtual Memory and Swapping
  - 最大锁定内存设置, Virtual Memory and Swapping
- 对称内存, PyTorch Symmetric Memory
- 调优主机 CPU 内存分配器, Tune Host CPU Memory Allocator

- 统一 CPU-GPU 内存, The CPU and GPU Superchip

内存受限的 GPU 停顿, Iteratively Profiling and Determining the Kernel Bottleneck-Iteratively Profiling and Determining the Kernel Bottleneck

- 内核优化, Optimizing the Kernel-Optimizing the Kernel

MIG (Multi-Instance GPU; NVIDIA)

- GPU 驱动程序运行时设置, MIG-MIG
- GPU 虚拟化, Managing Resources Efficiently
- Kubernetes MIG 模式, Slicing a GPU with MIG
- 建议为 GPU 启用持久化模式, Slicing a GPU with MIG
- 配置文件命名, MIG
- 将单个 GPU 切分为更小 GPU, Sharing and Scheduling
- 何时使用, MIG

混合精度

- 自动混合精度, TF32 and Automatic Mixed Precision (PyTorch)-TF32 and Automatic Mixed Precision (PyTorch)
- 降低精度, BF16/FP16, FP8, and FP4 Reduced Precision
- Tensor Cores, Mixed Precision and Utilizing Tensor Cores-Transformer Engine and TMEM in Depth

混合专家（MoE）模型

- MoE 模型概述, Profiling PyTorch to Identify Bottlenecks
- 所有专家同时激活, Expert Parallelism
- 全对全通信, Expert Communication Optimization
- CUDA 流, Using CUDA Streams with MoE Models
- DeepEP 通信库, Transparency and Reproducibility
- DeepSeek-V3, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China-DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China
  - DualPipe parallelism algorithm, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China
- 用于推理的动态路由策略, Dynamic Routing Strategies for MoE Inference-Adaptive Expert Routing and Real-Time Monitoring

    - 专家通信优化, Expert Communication Optimization-Expert Communication Optimization
    - 专家复制, Load Balancing, Capacity Factor, and Expert Replication
    - 负载均衡和容量因子, Load Balancing, Capacity Factor, and Expert Replication
  - 面向边缘的内存预算, Edge-Oriented MoE Memory Budgeting
  - 专家并行, Parallelism Strategies for Serving Massive MoE Models, Expert Parallelism-Expert Parallelism
    - 负载均衡, Expert Parallelism
    - 模型权重和数据跨 GPU 切分, Parallelism Strategies for Serving Massive MoE Models
  - 专家重平衡和重新分组, MoE Expert Rebalancing and Regrouping
  - 混合并行, Hybrid Parallelism-Hybrid Parallelism
  - MoE 效率, Toward 100-Trillion-Parameter Models
  - 需要多节点推理, Multinode Inference, Parallelism, Decoding, and Routing Optimizations
  - 并行策略, Parallelism Strategies for Serving Massive MoE Models-Hybrid Parallelism
  - 性能分析, Profiling PyTorch to Identify Bottlenecks-CPU and GPU Profiling with Linux perf
    - 在 CPU 和 GPU 上使用 Linux perf, CPU and GPU Profiling with Linux perf-CPU and GPU Profiling with Linux perf
    - Nsight Systems 和 NVTX 时间线, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines
    - PyTorch 性能分析器, Using PyTorch Profiler-Using PyTorch Profiler
    - GEMM 内核的屋顶线分析, Kernel Roofline Analysis for General Matrix Multiply (GEMM)-Kernel Roofline Analysis for General Matrix Multiply (GEMM)
  - 稀疏模型减少计算需求, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China, Toward 100-Trillion-Parameter Models
  - 推测性专家路由, Speculative MoE Expert Routing and Communication Reduction

MLA (参见 DeepSeek 多头潜在注意力（MLA))

MLPerf（MLCommons）, Transparency and Reproducibility
- 每 GPU 结果的注意事项, Transparency and Reproducibility
- 日志记录, Performance Benchmarks and MLPerf Logging-Performance Benchmarks and MLPerf Logging
- MLPerf 推理, Transparency and Reproducibility
- MLPerf 训练, Transparency and Reproducibility

MMA API (CUDA), Transformer Engine and TMEM in Depth-Transformer Engine and TMEM in Depth

模型级联, Model Cascading and Tiered Model Deployment-Model Cascading and Tiered Model Deployment

MoE (参见 混合专家（MoE）模型)

监控
- CPU 利用率, Scaling Out Workers as You Scale Out Number of GPUs
- 训练期间的文件系统 I/O, Tuning, Replicating, and Compressing Data
- GPU 内存使用量, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
- 图重新编译, TorchDynamo for Bytecode Capture and Graph Extraction, Minimize Graph Recompilations
- 推理系统指标和计数器, Monitoring System Metrics and Counters-Monitoring System Metrics and Counters
  - 正确性问题调试, Debugging Correctness Issues-Debugging Correctness Issues
  - 生产环境, Inference Troubleshooting Recipes
- 内存碎片化, Monitoring System Metrics and Counters
- 模型跑题, Token Output Limits and Timeouts
- 多 GPU NVLink 利用率, Real-Time Link Telemetry and Monitoring
- 网络吞吐量, Pitfall #4: Insufficient network bandwidth or misconfigured NICs
- 性能监控, Performance Monitoring and Utilization in Practice
  - 通信与计算瓶颈, Diagnosing Communication- Versus Compute-Bound Workloads
  - GPU 利用率接近 100%, Performance Monitoring and Utilization in Practice

  - I/O 持续性能分析和调优, Continuous Profiling and Tuning Workflow-Continuous Profiling and Tuning Workflow
  - NVLink 使用情况, Performance Monitoring and Utilization in Practice
- 功耗监控, Performance Monitoring and Utilization in Practice
- 存储 I/O, Monitoring Storage I/O-Monitoring Storage I/O
- 拖尾节点或进程, Pitfall #5: Straggler nodes or processes

Mooncake
- 自适应策略, Mooncake adaptive strategies
- 提前拒绝, Early Rejection (Admission Control)

Moonshot AI Kimi K2, Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models

MPS (NVIDIA), MPS-MPS

多 GPU (参见 多个 GPU)

多实例 GPU (参见 MIG（Multi-Instance GPU；NVIDIA))

多裸片 GPU GPC“分区”, Thread Block Clusters and Distributed Shared Memory
- (另见 双裸片 GPU)

多级分块, Multilevel Microtiling and Software Prefetching

调优多节点连接性, Tuning Multinode Connectivity-Tuning Multinode Connectivity
- 多节点通信陷阱, Multinode Communication Pitfalls-Pitfall #6: GPU memory fragmentation under UCX/RDMA
  - 使用 GLOO 后端而非 NCCL, Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL-Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL
  - UCX/RDMA 下的 GPU 内存碎片化, Pitfall #6: GPU memory fragmentation under UCX/RDMA-Pitfall #6: GPU memory fragmentation under UCX/RDMA
  - 带宽不足或 NIC 配置错误, Pitfall #4: Insufficient network bandwidth or misconfigured NICs
  - NCCL 版本不匹配, Pitfall #2: Mismatched NCCL versions
  - 拖尾节点或进程, Pitfall #5: Straggler nodes or processes
  - 摘要, Pitfall #6: GPU memory fragmentation under UCX/RDMA
  - NCCL 引导期间 TCP 端口耗尽, Pitfall #3: TCP port exhaustion during NCCL bootstrap

- NCCL 与 UCX, Enabling Peer-to-Peer DMA and UCX

多节点推理, Multinode Inference, Parallelism, Decoding, and Routing Optimizations

- 解耦式推理 (参见 解耦式预填充和解码)
- 使用 Kubernetes 部署, Deploying Disaggregated Prefill and Decode with Kubernetes-Deploying Disaggregated Prefill and Decode with Kubernetes, Full-Stack Inference Optimizations

multiple GPUs

- 通过 NCCL 和 CUDA Graphs 捕获集合通信, Capturing Multi-GPU Collectives with NCCL and CUDA Graphs-Capturing Multi-GPU Collectives with NCCL and CUDA Graphs
- 拥塞感知和拓扑感知调度, Congestion-Aware and Topology-Aware Scheduling with Multiple GPUs-Coordinating NVSwitch Transfers with Fine-Tuned Scheduling
  - 自适应进程-GPU 映射, Adaptive Process-GPU Mapping-Adaptive Process-GPU Mapping
  - 动态拥塞感知调度, Dynamic Congestion-Aware Scheduling-Dynamic Congestion-Aware Scheduling
  - GPUDirect RDMA, Multinode and Multirack Communication with GPUDirect RDMA-Multinode and Multirack Communication with GPUDirect RDMA
  - MoE 专家重平衡和重新分组, MoE Expert Rebalancing and Regrouping
  - 多节点和多机架通信, Multinode and Multirack Communication with GPUDirect RDMA-Multinode and Multirack Communication with GPUDirect RDMA
  - NCCL 集合通信, Optimizing Collective Communication with NCCL-Wave scheduling of collectives
  - NVLink/NVSwitch 拓扑和约束, NVLink/NVSwitch Topology and Bandwidth Constraints-NVLink/NVSwitch Topology and Bandwidth Constraints
  - NVSwitch 与精细调度, Coordinating NVSwitch Transfers with Fine-Tuned Scheduling-Coordinating NVSwitch Transfers with Fine-Tuned Scheduling
  - 遥测和监控, Real-Time Link Telemetry and Monitoring
- 将多个 GPU 视为一个, Ultrascale Networking Treating Many GPUs as One-Co-Packaged Optics: Future of Networking Hardware

- 数百万 GPU 集群, Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models-Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models
- N-GPU 扩展模式, Pattern for N-GPU Scaling
- 用于分布式通信的 NCCL, NCCL for Distributed Multi-GPU Communication-In-Network SHARP Aggregation
- 跨设备和节点编排, Orchestrate Across Multiple GPUs and Cluster Nodes (NVSHMEM)-Pattern for N-GPU Scaling
  - GPU 到 GPU 内存共享, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM-Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM
  - NVIDIA SHMEM 库, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM-Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM
  - 对称内存, PyTorch Symmetric Memory
- 计算与数据重叠, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - 每个 GPU 多个流, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - PyTorch, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
- peer-to-peer DMA, Enabling Peer-to-Peer DMA and UCX
- 使用 HTA 进行性能分析, Multi-GPU Profiling with HTA
- 多 GPU 编程, Multi-GPU Programming-Multi-GPU Programming
- PyTorch 可插拔内存分配器, Pluggable Memory Allocators and Cross-GPU Data Transfers-Pluggable Memory Allocators and Cross-GPU Data Transfers
- 流有序内存分配器, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams

multirack and storage communication, Multirack and Storage Communication

N

N-GPU 扩展模式, Pattern for N-GPU Scaling

Nagle 算法, Streaming Responses

NCCL (NVIDIA Collective Communications Library)

- NCCL 概述, Scaling Distributed Training and Inference, NCCL for Distributed Multi-GPU Communication, Optimizing Collective Communication with

NCCL

- 使用 NCCL 后端而非 GLOO, Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL-Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL
- 引导 TCP 端口耗尽, Pitfall #3: TCP port exhaustion during NCCL bootstrap
- CUDA Toolkit, CUDA Toolkit and Runtime
- 调试, Profiling and Debugging NCCL
  - 推理正确性问题, Debugging Correctness Issues
  - 启用 NCCL 调试, Debugging Correctness Issues
  - NCCL 日志, In-Network SHARP Aggregation, Debugging Correctness Issues
  - 测试套件, Debugging Correctness Issues
- 用于网络传输的专用高优先级流, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - 环形或树形算法, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams, Ring versus tree all-reduce-Ring versus tree all-reduce
- 分布式多 GPU 通信, NCCL for Distributed Multi-GPU Communication-In-Network SHARP Aggregation
  - 概述, NCCL for Distributed Multi-GPU Communication
  - 并行中的 all-reduce, Distributed Data Parallel Strategies
  - CollNet 通信算法, NCCL Communication Algorithms
  - 通信算法, NCCL Communication Algorithms-NCCL Communication Algorithms
  - 多对多集合通信, NVIDIA’s NIXL and Disaggregated Inference, NVIDIA’s NIXL and Disaggregated Inference, NCCL Versus NIXL
  - NCCL_ALGO 环境变量, Topology Awareness in NCCL, NCCL Communication Algorithms
  - 优化, Optimizing Collective Communication with NCCL-Wave scheduling of collectives
  - 并行聚合树通信算法, NCCL Communication Algorithms
  - 环形通信算法, NCCL Communication Algorithms
  - 拓扑感知, Topology Awareness in NCCL-Topology Awareness in NCCL
  - 树形通信算法, NCCL Communication Algorithms

  - 集合通信的波次调度, Wave scheduling of collectives
- 分布式多 GPU 通信
  - 环形或树形算法, Ring versus tree all-reduce-Ring versus tree all-reduce
  - 轮转环形端点, Rotating ring endpoints
- 强制 GPU-NIC 亲和性, Multinode and Multirack Communication with GPUDirect RDMA
- 每个通信器的最大 TCP 套接字连接数, NCCL_NSOCKS_PERTHREAD and NCCL_SOCKET_NTHREADS (Socket/TCP transports)
- PyTorch 中的内存分配器, Pluggable Memory Allocators and Cross-GPU Data Transfers
- 并行使用多个 NIC, Multinode and Multirack Communication with GPUDirect RDMA
- 捕获多 GPU 集合通信, Capturing Multi-GPU Collectives with NCCL and CUDA Graphs-Capturing Multi-GPU Collectives with NCCL and CUDA Graphs
- 与 NIXL 对比, NCCL Versus NIXL
  - 由 NCCL 执行的多对多集合通信, NVIDIA’s NIXL and Disaggregated Inference, NVIDIA’s NIXL and Disaggregated Inference, NCCL Versus NIXL
- 持久化用户缓冲区, Persistent NCCL User Buffers and Zero-Copy Registration
- 陷阱和注意事项, NCCL Communicator Lifecycle and Environment Gotchas-Pitfall #6: NCCL communicator hangs, errors, or shuts down completely
  - 通信器挂起、报错或关闭, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely-Pitfall #6: NCCL communicator hangs, errors, or shuts down completely
  - NCCL 线程的 CPU-GPU NUMA 节点, Pitfall #4: Verify CPU-GPU NUMA-node affinity for NCCL threads
  - 创建和销毁通信器, Pitfall #2: Do not create and destroy NCCL communicators on every iteration
  - 过于频繁地创建通信器, Pitfall #1: Creating NCCL communicators too often-Pitfall #1: Creating NCCL communicators too often
  - 忽略警告和错误, Pitfall #5: Resist the temptation to ignore NCCL warnings and errors
  - 通过环境变量覆盖或禁用功能, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables-NCCL_SHARP_DISABLE
  - 版本不匹配, Pitfall #2: Mismatched NCCL versions

- profiling, Profiling and Debugging NCCL
  - NCCL logs, In-Network SHARP Aggregation
- RDMA 容器访问设备, High-Speed, Low-Overhead Data Transfers with RDMA
- SHARP and, In-Network SHARP Aggregation
- 用于多节点拓扑的 UCX, Enabling Peer-to-Peer DMA and UCX
- 用户缓冲区注册, Persistent NCCL User Buffers and Zero-Copy Registration
- 版本不匹配, Pitfall #2: Mismatched NCCL versions

NCCL_ALGO, Topology Awareness in NCCL, NCCL Communication Algorithms, Ring versus tree all-reduce

NCCL_ASYNC_ERROR_HANDLING, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely, Profiling and Debugging NCCL

NCCL_BUFFSIZE, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables

NCCL_CROSS_NIC, Multinode and Multirack Communication with GPUDirect RDMA

NCCL_DEBUG, Topology Awareness in NCCL, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables, Profiling and Debugging NCCL, Debugging Correctness Issues

- NCCL logs, In-Network SHARP Aggregation, Debugging Correctness Issues

NCCL_IGNORE_CPU_AFFINITY, Pitfall #4: Verify CPU-GPU NUMA-node affinity for NCCL threads

NCCL_MAX_NCHANNELS, NCCL_MIN_NCHANNELS and NCCL_MAX_NCHANNELS

NCCL_MIN_NCHANNELS, NCCL_MIN_NCHANNELS and NCCL_MAX_NCHANNELS

NCCL_NSOCKS_PERTHREAD, Tuning Multinode Connectivity, Pitfall #4: Insufficient network bandwidth or misconfigured NICs, NCCL_NSOCKS_PERTHREAD and NCCL_SOCKET_NTHREADS (Socket/TCP transports)

NCCL_P2P_DISABLE, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables

NCCL_PROFILER_PLUGIN, Profiling and Debugging NCCL

NCCL_SHARP_DISABLE, NCCL_SHARP_DISABLE

NCCL_SHM_DISABLE, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables

NCCL_SOCKET_IFNAME, Optimizing Network Communication for Kubernetes, Pitfall #5: Resist the temptation to ignore NCCL warnings and errors

NCCL_SOCKET_NTHREADS, Tuning Multinode Connectivity, Pitfall #4: Insufficient network bandwidth or misconfigured NICs, NCCL_NSOCKS_PERTHREAD and NCCL_SOCKET_NTHREADS (Socket/TCP transports)

NCCL_TOPO_DUMP_FILE, Topology Awareness in NCCL

NCCL_TOPO_FILE, NCCL_TOPO_FILE

用于 Nsight Compute 的 ncu

- 性能分析器, Profiling and Diagnosing GPU Bottlenecks, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools
- 屋顶线分析, Nsight Compute and Roofline Analysis, Kernel Roofline Analysis for General Matrix Multiply (GEMM)-Kernel Roofline Analysis for General Matrix Multiply (GEMM)

嵌套锯齿布局张量（NJT）, FlexDecoding (PyTorch)

嵌套张量（PyTorch）, PyTorch and Arithmetic Intensity

网络通信 (参见 分布式网络通信)

NFS 服务器, Distributed, Parallel Filesystems and Object Stores

- 调优参数, Distributed, Parallel Filesystems and Object Stores

强制 NIC-GPU 亲和性, Multinode and Multirack Communication with GPUDirect RDMA

NIXL (NVIDIA Inference Xfer Library)

- NIXL 概述, Scaling Distributed Training and Inference, NCCL for Distributed Multi-GPU Communication, NVIDIA’s NIXL and Disaggregated Inference-NVIDIA’s NIXL and Disaggregated Inference
- 用于网络传输的专用高优先级流, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
- 解耦式推理, NVIDIA’s NIXL and Disaggregated Inference-NCCL Versus NIXL
  - NIXL 概述, NVIDIA’s NIXL and Disaggregated Inference-NVIDIA’s NIXL and Disaggregated Inference
  - 带回调的异步 API, NIXL Asynchronous API with Callbacks-NIXL Asynchronous API with Callbacks

    - NIXL 提升 Dynamo 吞吐量, NIXL and High-Performance Inference Systems Like NVIDIA Dynamo
    - 智能互连路由, Intelligent Interconnect Routing for KV Cache Transfers, KV Cache Data Transfer and NIXL
    - 使用 NIXL 卸载 KV 缓存, KV Cache Offloading with NIXL-KV Cache Offloading with NIXL
    - 预填充到解码的数据传输, KV Cache Data Transfer and NIXL
    - 分离预填充和解码推理阶段, Separate Prefill and Decode Inference Stages-Separate Prefill and Decode Inference Stages
  - 与 NCCL 对比, NCCL Versus NIXL
    - NIXL 执行的一对一传输, Distributed Data Parallel Strategies, NVIDIA’s NIXL and Disaggregated Inference, NVIDIA’s NIXL and Disaggregated Inference, NCCL Versus NIXL
  - NVIDIA Dynamo 核心组件, NVIDIA’s NIXL and Disaggregated Inference
    - 提升 Dynamo 吞吐量, NIXL and High-Performance Inference Systems Like NVIDIA Dynamo

Nsight Compute

  - 实际占用率, Inspecting Achieved Occupancy and GPU Utilization-Optimizing the Kernel
    - 内核计算吞吐量与 GPU FLOPS, Kernel Compute Throughput Versus Peak GPU FLOPS
    - 内核内存吞吐量与 HBM 内存带宽, Kernel Memory Throughput Versus Peak HBM Memory Bandwidth
    - 利用不足, Dynamic Scheduling with Atomic Work Queues
  - ALU 管线忙碌, Execution Unit Contention
  - 命令行 ncu
    - 性能分析器, Profiling and Diagnosing GPU Bottlenecks, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools
    - 屋顶线分析, Nsight Compute and Roofline Analysis, Kernel Roofline Analysis for General Matrix Multiply (GEMM)-Kernel Roofline Analysis for General Matrix Multiply (GEMM)
  - CUDA Program Counter Sampling, Profiling with Nsight Systems and Nsight Compute
  - ILP 配置, ILP and Occupancy

- 并行与串行操作的影响, Maintaining High Occupancy and GPU Utilization
- 推理性能分析, Profiling with Nsight Systems and Nsight Compute
  - CUDA 程序计数器采样, Profiling with Nsight Systems and Nsight Compute
- 内核从内存受限转为计算受限, Feeding Tensor Cores with TMEM and TMA
- 启动统计信息, Designing Efficient Algorithms with Thread Block Clusters
- 内存工作负载分析
  - 原子事务计数器, Atomic Counters-Atomic Counters
  - 合并与非合并内存访问, Coalesced Versus Uncoalesced Global Memory Access
- 内存工作负载分析, Kernel Autotuning for Transformer Self-Attention and MLP Paths
- 占用率
  - 受寄存器限制, Profiling and Mitigating Register Pressure
  - 实际 FLOPS 低, Kernel Compute Throughput Versus Peak GPU FLOPS
  - 占用率计算器, Techniques for Occupancy Tuning
  - 占用率限制因素, Inspecting Achieved Occupancy and GPU Utilization
- 性能分析的并行策略, Hybrid Parallelism
- 每内核指标, Read-Only Data Caches
- 每线程寄存器数和占用率指标, Tuning Occupancy with Launch Bounds
- 屋顶线分析, Nsight Compute and Roofline Analysis
  - MoE 模型示例, Kernel Roofline Analysis for General Matrix Multiply (GEMM)-Kernel Roofline Analysis for General Matrix Multiply (GEMM)
- 源代码计数器
  - 实际 FLOPS 低, Kernel Compute Throughput Versus Peak GPU FLOPS
  - 对 warp 发散进行性能分析, Profiling and Detecting Warp Divergence
- Speed of Light 分析, Mixed Precision and Utilizing Tensor Cores
- 停顿
  - 计算单元忙碌, Execution Unit Contention
  - 执行依赖, Execution-Dependency Stalls, Profiling and Mitigating Register Pressure
  - 空闲, Other Stall Reasons

    - 长记分牌, Memory-Related Stalls
    - 数学管线限速, Execution Unit Contention
    - 内存依赖, Other Stall Reasons
    - 内存限速, Memory-Related Stalls
    - 无合格 warp, Other Stall Reasons
    - 未被选中, Memory-Related Stalls
    - 其他原因, Other Stall Reasons-Other Stall Reasons
    - 短记分牌, Memory-Related Stalls
    - 纹理限速, Other Stall Reasons
  - Triton 内核支持, Profiling with Triton Proton Profiler
  - 利用不足指标, Dynamic Scheduling with Atomic Work Queues
  - 更新到最新 GPU, Profiler-Guided Analysis
  - Warp 状态, Profiling and Detecting Warp Divergence

Nsight Systems

  - 概述, Profiling and Diagnosing GPU Bottlenecks
  - 命令行 nsys
    - 性能分析器, Profiling and Diagnosing GPU Bottlenecks, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines
    - 时间线视图, Nsight Systems Timeline View, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines
  - 通信与计算瓶颈, Diagnosing Communication- Versus Compute-Bound Workloads
  - I/O 监控, Monitoring Storage I/O
  - 并行与串行操作的影响, Maintaining High Occupancy and GPU Utilization
  - 推理性能分析, Profiling with Nsight Systems and Nsight Compute-Profiling with Nsight Systems and Nsight Compute
  - NVLink 性能分析, Topology Awareness in NCCL, Real-Time Link Telemetry and Monitoring
  - 性能分析的并行策略, Hybrid Parallelism

- Python 回溯采样, PyTorch Profiler and Visualization Tools
- PyTorch 聚焦模式, PyTorch Profiler and Visualization Tools
- 时间线视图, Read-Only Data Caches, Nsight Systems Timeline View, Nsight Compute and Roofline Analysis, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines, Kernel Autotuning for Transformer Self-Attention and MLP Paths
  - NVTX 标记, Nsight Systems Timeline View, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines
- Triton 内核支持, Profiling with Triton Proton Profiler
- 更新到最新 GPU, Profiler-Guided Analysis

作为 Nsight Systems 命令行工具的 nsys

- 性能分析器, Profiling and Diagnosing GPU Bottlenecks, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines
- 时间线视图, Nsight Systems Timeline View, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines

NUMA 节点, NUMA Awareness and CPU Pinning

- NUMA 感知和 CPU 绑定, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
  - CPU 绑定, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
- NUMA 友好的内存分配和固定, NUMA-Friendly Memory Allocation and Memory Pinning-NUMA-Friendly Memory Allocation and Memory Pinning

numactl

- CPU 绑定, NUMA Awareness and CPU Pinning, NUMA Awareness and CPU Pinning
- 强制从特定 NUMA 节点分配内存, NUMA Awareness and CPU Pinning, NUMA-Friendly Memory Allocation and Memory Pinning
- 启动时应用进程 CPU 和内存策略, NUMA-Friendly Memory Allocation and Memory Pinning

数值正确性和精度调试, Debugging Numerical Correctness and Accuracy-Debugging Numerical Correctness and Accuracy

NumPy 替代品 cuPyNumeric, C++ and Python CUDA Libraries

NV-HBI (High-Bandwidth Interface), NVIDIA Blackwell “Dual-Die” GPU

nvcc CUDA 编译器

- 跨 GPU 世代的兼容性, CUDA Forward and Backward Compatibility Across GPU Hardware Generations, CUDA GPU Backward and Forward Compatibility Model
- -cubin option, CUDA Forward and Backward Compatibility Across GPU Hardware Generations
- 软件栈中的 CUDA Toolkit, CUDA Toolkit and Runtime
- fatbinary, CUDA Forward and Backward Compatibility Across GPU Hardware Generations
- 未提供 float8, Vectorized Memory Access
- __global__, CUDA Programming Refresher
- __launch_bounds__, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
- 针对占用率优化, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
- 启用每线程默认流, Modern Per-Thread Default Stream
- 谓词化, Techniques to Avoid Warp Divergence

NVIDIA

- 机架中的 AI 超级计算机, NVIDIA’s “AI Supercomputer in a Rack”-NVIDIA’s “AI Supercomputer in a Rack”
  - CPU + GPU 超级芯片, The CPU and GPU Superchip-Streaming Multiprocessor, Threads, and Warps
  - GPU Tensor Cores 和 Transformer Engine, NVIDIA GPU Tensor Cores and Transformer Engine-NVIDIA GPU Tensor Cores and Transformer Engine
  - 将多个 GPU 视为一个, Ultrascale Networking Treating Many GPUs as One-Co-Packaged Optics: Future of Networking Hardware
  - 多 GPU 编程, Multi-GPU Programming-Multi-GPU Programming
  - 多机架和存储通信, Multirack and Storage Communication
  - NVIDIA Blackwell 双裸片 GPU, NVIDIA Blackwell “Dual-Die” GPU-NVIDIA Blackwell “Dual-Die” GPU
  - NVIDIA Grace CPU, NVIDIA Grace CPU

    - NVL72, Ultrascale Networking Treating Many GPUs as One
        - (另见 NVL72)
    - NVLink 和 NVSwitch, NVLink and NVSwitch-NVLink and NVSwitch
    - 用于网络内聚合的 SHARP, In-Network Aggregations with NVIDIA SHARP-In-Network Aggregations with NVIDIA SHARP
    - 流式多处理器、线程、warp, Streaming Multiprocessor, Threads, and Warps-Streaming Multiprocessor, Threads, and Warps
  - 未来硬件, A Glimpse into the Future: NVIDIA’s Roadmap-Feynman GPU (2028) and Doubling Something Every Year
    - Blackwell Ultra 和 Grace Blackwell Ultra, Blackwell Ultra and Grace Blackwell Ultra
    - 每年翻倍, Feynman GPU (2028) and Doubling Something Every Year
    - Feynman GPU, Feynman GPU (2028) and Doubling Something Every Year
    - Rubin Ultra 和 Vera Rubin Ultra, Rubin Ultra and Vera Rubin Ultra (2027)
    - Vera Rubin 超级芯片, Vera Rubin Superchip (2026)
  - 因 DeepSeek-R1 成本而出现的股价下跌, Introduction and AI System Overview

NVIDIA Base Command Manager, Preintegrated Rack Appliance

NVIDIA Blackwell B200, Introduction and AI System Overview, NVIDIA Blackwell “Dual-Die” GPU, Choosing Threads-per-Block and Blocks-per-Grid Sizes, Understanding GPU Memory Hierarchy

NVIDIA Blackwell B300 Ultra, Toward 100-Trillion-Parameter Models, Blackwell Ultra and Grace Blackwell Ultra

NVIDIA Blackwell 双裸片 GPU, NVIDIA Blackwell “Dual-Die” GPU-NVIDIA Blackwell “Dual-Die” GPU
  - 远程内存访问延迟, NUMA Awareness and CPU Pinning

NVIDIA 集合通信库 (参见 NCCL（NVIDIA 集合通信库))

NVIDIA Compute Sanitizer, Debugging Functional Correctness with NVIDIA Compute Sanitizer
  - compute-sanitizer CLI, Debugging Functional Correctness with NVIDIA Compute Sanitizer
    - NVIDIA Tools Extension 支持, Debugging Functional Correctness with NVIDIA Compute Sanitizer
  - 并发与同步, Using CUDA Streams with MoE Models

- 四种主要工具, Debugging Functional Correctness with NVIDIA Compute Sanitizer
- 推理正确性, Debugging Correctness Issues

NVIDIA Container Toolkit, Container Runtime Optimizations for GPUs

NVIDIA CPU + GPU 超级芯片, The CPU and GPU Superchip-Streaming Multiprocessor, Threads, and Warps

- GPU Tensor Cores 和 Transformer Engine, NVIDIA GPU Tensor Cores and Transformer Engine-NVIDIA GPU Tensor Cores and Transformer Engine
- NVIDIA Blackwell 双裸片 GPU, NVIDIA Blackwell “Dual-Die” GPU-NVIDIA Blackwell “Dual-Die” GPU
  - 呈现为一个 GPU, NVIDIA Blackwell “Dual-Die” GPU
  - 远程内存访问延迟, NUMA Awareness and CPU Pinning
- NVIDIA Grace CPU, NVIDIA Grace CPU
- NVLink-C2C, The CPU and GPU Superchip
- 流式多处理器、线程、warp, Streaming Multiprocessor, Threads, and Warps-Streaming Multiprocessor, Threads, and Warps

NVIDIA 数据中心 GPU 管理器（参见 数据中心 GPU 管理器（DCGM；NVIDIA））

NVIDIA Data Loading Library (DALI), Multimodal Data Processing with NVIDIA DALI, Optimizing the Data Input Pipeline

NVIDIA DCGM（参见 数据中心 GPU 管理器（DCGM；NVIDIA））

以 NIXL 为核心组件的 NVIDIA Dynamo, NVIDIA’s NIXL and Disaggregated Inference

- 提高吞吐量, NIXL and High-Performance Inference Systems Like NVIDIA Dynamo

NVIDIA GB200 NVL72, NVIDIA’s “AI Supercomputer in a Rack”, Ultrascale Networking Treating Many GPUs as One, GPU and CPU-GPU Superchip Improvements

- P2P-capable NVLink, Enabling Peer-to-Peer DMA and UCX
- training throughput per GPU, Transparency and Reproducibility

NVIDIA GB300 NVL72, NVIDIA’s “AI Supercomputer in a Rack”, Ultrascale Networking Treating Many GPUs as One, NVLink/NVSwitch Topology and Bandwidth Constraints

- Grace Blackwell Ultra 超级芯片, Blackwell Ultra and Grace Blackwell Ultra

NVIDIA GDS（参见 GDS（NVIDIA GPUDirect Storage））

NVIDIA Grace Blackwell GB200 and GB300, NVIDIA’s “AI Supercomputer in a Rack”-NVIDIA’s “AI Supercomputer in a Rack”

NVIDIA Grace CPU, NVIDIA Grace CPU

NVIDIA H800 GPU, Introduction and AI System Overview, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China-DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China

NVIDIA 推理 Xfer 库（参见 NIXL（NVIDIA 推理 Xfer 库））

NVIDIA Kubernetes GPU Operator, Kubernetes for Topology-Aware Container Orchestration and Networking

NVIDIA Magnum IO, NVIDIA Magnum IO Optimization Stack, CPU and GPU Profiling with Linux perf

NVIDIA 管理库（NVML）

- 强制 GPU-NIC 亲和性, Multinode and Multirack Communication with GPUDirect RDMA
- NVLink 遥测与监控, Real-Time Link Telemetry and Monitoring
- 功率限制监控, Kernel Compute Throughput Versus Peak GPU FLOPS

NVIDIA NeMo, Creating High-Quality LLM Datasets with NVIDIA NeMo Curator

- NeMo Curator, Creating High-Quality LLM Datasets with NVIDIA NeMo Curator

NVIDIA NIXL（参见 NIXL（NVIDIA 推理 Xfer 库））

NVIDIA Performance Monitoring Unit (PMU), CPU and GPU Profiling with Linux perf

NVIDIA SHMEM（NVSHMEM）, Multi-GPU Programming, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM-Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM

- CUDA Graphs, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM
- 使用时机, Fine-Grained GPU-to-GPU Memory Sharing with NVSHMEM

NVIDIA 软件栈, NVIDIA Software Stack

- 概述, PyTorch and Higher-Level AI Frameworks
- 在 Python 中创建的 CUDA 内核, NVIDIA Software Stack
- CUDA Runtime 和 Toolkit, CUDA Toolkit and Runtime

    - Toolkit 的 C++ 和 Python 库, C++ and Python CUDA Libraries
  - GPU 驱动程序, GPU Driver
    - nvidia-smi, GPU Driver
  - PyTorch 简介, PyTorch and Higher-Level AI Frameworks

NVIDIA 工具扩展（参见 NVTX（NVIDIA 工具扩展））

NVIDIA Topology-Aware GPU Selection (NVTAGS), Adaptive Process-GPU Mapping

NVIDIA Transformer Engine（参见 Transformer Engine（TE；NVIDIA））

NVIDIA Warp
  - CUDA kernels in Python, NVIDIA Software Stack
  - CUDA Toolkit, C++ and Python CUDA Libraries

nvidia-peermem driver, Multi-GPU Programming

nvidia-persistenced daemon, GPU Persistence Mode

nvidia-smi, GPU Driver

nvJPEG, Tuning, Replicating, and Compressing Data

NVL72, NVIDIA’s “AI Supercomputer in a Rack”, Ultrascale Networking Treating Many GPUs as One
  - AI 工厂构建块, Co-Packaged Optics: Future of Networking Hardware
  - Blackwell Ultra 和 Grace Blackwell Ultra 升级, Blackwell Ultra and Grace Blackwell Ultra
  - 作为预集成机架设备交付, Preintegrated Rack Appliance
    - Kubernetes, Preintegrated Rack Appliance
    - NVIDIA Base Command Manager, Preintegrated Rack Appliance
    - Simple Linux Utility for Resource Management, Preintegrated Rack Appliance
  - GPU 到 GPU 带宽, Multi-GPU Programming, Orchestrating Containers with Kubernetes Topology Manager
  - 与 H100 的比较, Multi-GPU Programming
  - 每 GPU 推理吞吐量, Transparency and Reproducibility
  - 多 GPU 编程, Multi-GPU Programming-Multi-GPU Programming
  - 多机架与存储通信, Multirack and Storage Communication

    - 数据处理单元（DPU）, Multirack and Storage Communication
  - NVLink 和 NVSwitch, NVLink and NVSwitch-NVLink and NVSwitch, Co-Packaged Optics: Future of Networking Hardware
  - 性能监控, Performance Monitoring and Utilization in Practice
  - 功耗要求, Compute Density and Power Requirements
    - 液冷与风冷, Liquid Cooling Versus Air Cooling-Liquid Cooling Versus Air Cooling
  - 共享与调度工作负载, Sharing and Scheduling
  - 拓扑与带宽约束, NVLink/NVSwitch Topology and Bandwidth Constraints-NVLink/NVSwitch Topology and Bandwidth Constraints
    - 遥测与监控, Real-Time Link Telemetry and Monitoring

NVLink

  - 专用流上的 NCCL, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - 名称中的 NVL, NVIDIA’s “AI Supercomputer in a Rack”
  - NVL72, Ultrascale Networking Treating Many GPUs as One, Co-Packaged Optics: Future of Networking Hardware
  - NVSwitch, NVLink and NVSwitch-NVLink and NVSwitch
    - 精细调优的调度及, Coordinating NVSwitch Transfers with Fine-Tuned Scheduling-Coordinating NVSwitch Transfers with Fine-Tuned Scheduling
    - Magnum IO 支持, NVIDIA Magnum IO Optimization Stack
    - NVIDIA SHARP, In-Network Aggregations with NVIDIA SHARP-In-Network Aggregations with NVIDIA SHARP
    - 机架中的 NVIDIA 超级计算机, NVIDIA’s “AI Supercomputer in a Rack”
    - NVL72, Co-Packaged Optics: Future of Networking Hardware
    - 遥测与监控, Real-Time Link Telemetry and Monitoring
    - 拓扑与带宽约束, NVLink/NVSwitch Topology and Bandwidth Constraints-NVLink/NVSwitch Topology and Bandwidth Constraints
  - P2P-capable, Enabling Peer-to-Peer DMA and UCX
  - single domain for data tensor pipeline, NVIDIA’s “AI Supercomputer in a Rack”

- topology and bandwidth constraints, NVLink/NVSwitch Topology and Bandwidth Constraints-NVLink/NVSwitch Topology and Bandwidth Constraints
  - telemetry and monitoring, Real-Time Link Telemetry and Monitoring
- Transformer Engine in modern GPUs, Mechanical Sympathy: Hardware-Software Codesign

NVLink-C2C (chip-to-chip), The CPU and GPU Superchip
- cache coherency, The CPU and GPU Superchip, Multi-GPU Programming
- Grace Blackwell CPU and GPU linked, NUMA Awareness and CPU Pinning, NUMA Awareness and CPU Pinning
- Linux treatment of memory, NUMA Awareness and CPU Pinning

NVMe (Linux)
- DeepSeek Fire-Flyer File System, DeepSeek’s Fire-Flyer File System-DeepSeek’s Fire-Flyer File System
- fast storage and data locality, Fast Storage and Data Locality
- GPUDirect Storage, NUMA-Friendly Memory Allocation and Memory Pinning, Using NVIDIA GDS-Measuring GDS with gdsio
- parameters offloaded to, Offloading Parameters to CPU and NVMe
- sequential versus random reads, Sequential Versus Random Read Patterns-Sequential Versus Random Read Patterns
- tuning for throughput, Tuning NVMe and Filesystem for Throughput
- XFS optimized, Sequential Versus Random Read Patterns

NVMe over Fabrics (NVMe-oF), Multirack and Storage Communication

NVML（参见 NVIDIA 管理库（NVML））

NVSHMEM（参见 NVIDIA SHMEM（NVSHMEM））

NVSwitch, NVLink and NVSwitch-NVLink and NVSwitch
- fine-tuned scheduling and, Coordinating NVSwitch Transfers with Fine-Tuned Scheduling-Coordinating NVSwitch Transfers with Fine-Tuned Scheduling
- Magnum IO support, NVIDIA Magnum IO Optimization Stack
- NVIDIA SHARP, In-Network Aggregations with NVIDIA SHARP-In-Network Aggregations with NVIDIA SHARP

  - NVIDIA supercomputer on a rack, NVIDIA’s “AI Supercomputer in a Rack”
  - NVL72, Co-Packaged Optics: Future of Networking Hardware
  - telemetry and monitoring, Real-Time Link Telemetry and Monitoring
  - topology and bandwidth constraints, NVLink/NVSwitch Topology and Bandwidth Constraints-NVLink/NVSwitch Topology and Bandwidth Constraints

NVTAGS (NVIDIA Topology-Aware GPU Selection), Adaptive Process-GPU Mapping

NVTX (NVIDIA Tools Extension)

  - compute-sanitizer CLI supporting, Debugging Functional Correctness with NVIDIA Compute Sanitizer
  - markers, NVTX Markers and Profiling Tools
    - Holistic Trace Analysis, Multi-GPU Profiling with HTA
    - profiling tools that use markers, NVTX Markers and Profiling Tools-NVTX Markers and Profiling Tools
  - Nsight Systems
    - example MoE model, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines
    - timeline view, Nsight Systems Timeline View, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines

O

对象存储目标（OST）, Distributed, Parallel Filesystems and Object Stores

占用率

  - 实际占用率与 GPU 利用率, Inspecting Achieved Occupancy and GPU Utilization-Optimizing the Kernel
    - 实际占用率的定义, Inspecting Achieved Occupancy and GPU Utilization
    - 内核计算吞吐量与 GPU FLOPS, Kernel Compute Throughput Versus Peak GPU FLOPS
    - 内核内存吞吐量与 HBM 内存带宽, Kernel Memory Throughput Versus Peak HBM Memory Bandwidth
  - 配置启动参数, Configuring Launch Parameters: Blocks per Grid and Threads per Block

- CUDA Occupancy API, Techniques for Occupancy Tuning, Techniques for Occupancy Tuning
  - occupancy optimization, Determine Optimal Launch Configuration with the Occupancy API
- 占用率的定义, Maintaining High Occupancy and GPU Utilization, Tuning Occupancy
- 硬件限制, Choosing Threads-per-Block and Blocks-per-Grid Sizes, Choosing Threads-per-Block and Blocks-per-Grid Sizes
- 高占用率, Threads, Warps, Blocks, and Grids
  - 较高 ILP 下的, Loop Unrolling, Interleaving, and Compiler Hinting
  - 保持高占用率, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
- ILP 与, Exposing Instruction-Level Parallelism, ILP and Occupancy, Loop Unrolling, Interleaving, and Compiler Hinting
- 占用率的影响, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
- 提高占用率与每线程资源, Tuning Occupancy with Launch Bounds, Tuning Occupancy, Compiler Hints to Optimize Occupancy
- 感知占用率的内核选择, Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection-Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection
- 持久化内核, Persistent Kernels and Megakernels
- 寄存器权衡, Tuning Occupancy with Launch Bounds
- 调优, Tuning Occupancy
  - 每个调度器的活跃 warp, Find the Right Occupancy for Your Workload
  - CUDA Occupancy API, Techniques for Occupancy Tuning, Determine Optimal Launch Configuration with the Occupancy API
  - 每周期合格 warp, Find the Right Occupancy for Your Workload
  - __launch_bounds__, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds, Techniques for Occupancy Tuning, Compiler Hints to Optimize Occupancy
  - 适合工作负载的占用率, Find the Right Occupancy for Your Workload
  - 技术, Techniques for Occupancy Tuning

在线资源 (参见 在线资源)

OOM（内存不足）错误

- CPU + GPU 架构, The CPU and GPU Superchip
- 最大锁定内存设置, Transparent Hugepages
- 内存碎片化或过度缓存, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
- 监控 GPU 内存使用情况, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling

OOM 杀手（Linux）, Memory Isolation and Avoiding the OOM Killer

Open Collective Foundation, Scaling Toward Multimillion GPU Clusters and 100-Trillion-Parameter Models

Open Infra Index (DeepSeek), Transparency and Reproducibility

Open-Source Week (DeepSeek), Transparency and Reproducibility, Transparency and Reproducibility

OpenAI

- ChatGPT 有状态对话, Prefix Caching
- GPT-4 训练成本, Introduction and AI System Overview
- Triton, Triton Programming Model
  - 访问共享内存, Accessing Shared Memory in Triton
  - 高级内核实现, Advanced Triton Kernel Implementations-Software Pipelining and Double Buffering with Triton
  - 在 Python 中的 CUDA 内核, NVIDIA Software Stack
  - 自定义内核, Writing Custom Kernels with OpenAI Triton-Profiling with Triton Proton Profiler
  - 双缓冲流水线, Software Pipelining and Double Buffering with Triton-Software Pipelining and Double Buffering with Triton
  - 语言和编译器, C++ and Python CUDA Libraries
  - Nsight Compute 和 Systems 支持, Profiling with Triton Proton Profiler
  - 开源, C++ and Python CUDA Libraries
  - 持久化矩阵乘法内核, Tiled and Persistent GEMM Kernel (Triton)-Tiled and Persistent GEMM Kernel (Triton)
  - 使用 Triton Proton Profiler 分析, Profiling with Triton Proton Profiler
  - 向 PyTorch 注册自定义内核, TorchInductor Backend Code Generation, Registering Custom Kernels with PyTorch-Registering Custom Kernels

with PyTorch
  - 智能编译器, Smart Compilers and Automated Code Optimizations-Smart Compilers and Automated Code Optimizations
  - TorchInductor 基础, PyTorch and Higher-Level AI Frameworks, Using the PyTorch Compiler
  - 调优内核启动参数, Tuning Kernel-Launch Parameters
  - warp 专用化, Warp Specialization with Triton
- Triton Proton Profiler, Profiling with Triton Proton Profiler

操作系统, Operating System

运算强度 (参见 算术强度)

光网络, Co-Packaged Optics: Future of Networking Hardware

使用 AI 优化 AI 系统, AI-Assisted Real-Time System Optimizations and Cluster Operations-AI-Assisted Real-Time System Optimizations and Cluster Operations

OS 和 CPU 配置, Configuring the CPUs and OS for GPU Environments-Tune Host CPU Memory Allocator
- 概述, Configuring the CPUs and OS for GPU Environments
- CPU 频率和 C 状态, CPU Frequency and C-states
- 文件系统缓存与回写, Filesystem Caching and Write-Back
- NUMA 感知与 CPU 绑定, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
  - CPU + GPU 超级芯片的 CPU 到 GPU 数据传输, NUMA Awareness and CPU Pinning
  - CPU 绑定, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
  - NUMA 节点, NUMA Awareness and CPU Pinning
- NUMA 友好的内存分配与固定, NUMA-Friendly Memory Allocation and Memory Pinning-NUMA-Friendly Memory Allocation and Memory Pinning
  - 固定内存的效率, NUMA-Friendly Memory Allocation and Memory Pinning
  - 最大锁定内存设置, Transparent Hugepages
  - OS 对固定内存的限制, NUMA-Friendly Memory Allocation and Memory Pinning
  - 数据加载器的固定内存, NUMA-Friendly Memory Allocation and Memory Pinning

- 调度器和中断亲和性, Scheduler and Interrupt Affinity
  - irqbalance 守护进程, Scheduler and Interrupt Affinity
- 透明大页, Transparent Hugepages
- 调优主机 CPU 内存分配器, Tune Host CPU Memory Allocator
- 虚拟内存与交换, Virtual Memory and Swapping

OST（对象存储目标）, Distributed, Parallel Filesystems and Object Stores

内存不足（OOM）错误

- CPU + GPU 架构, The CPU and GPU Superchip
- 最大锁定内存设置, Transparent Hugepages
- 内存碎片化或过度缓存, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling
- 监控 GPU 内存使用情况, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling

输出

- 流式响应, Streaming Responses-Streaming Responses
- 结构化输出, Constrained Decoding Performance Implications
- Token 输出限制与超时, Token Output Limits and Timeouts

通信与计算重叠

- 自动化, Smart Compilers and Automated Code Optimizations
- CUDA 流, Using Streams to Overlap Compute with Data Transfers-Using Streams to Overlap Compute with Data Transfers
  - 并发, Overlapping Communication and Computation-Overlapping Communication and Computation
  - 多个 GPU, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
- 分布式网络通信, Overlapping Communication and Computation (Pipelining)-Achieving Maximal Overlap in Practice
  - 分布式数据并行策略, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
- 推理调优, Overlapping Communication and Computation-Overlapping Communication and Computation

– NCCL 通信与 GEMM 计算, Optimizing Collective Communication with NCCL
– 卸载的 KV 缓存, Speculative KV Prefetching for Faster TTFT
– PyTorch 模式, Overlapping Communication and Computation

O_DIRECT
– 绕过页缓存, Filesystem Caching and Write-Back, Using NVIDIA GDS
– GDS, Using NVIDIA GDS

P

P2P（点对点）DMA, Enabling Peer-to-Peer DMA and UCX
– P2P（点对点）GPU 复制, Pitfall #3: Avoid overtuning or disabling NCCL features with environment variables

填充共享数组, Avoid Shared-Memory Bank Conflicts
– 作为替代方案的 swizzling, Avoid Shared-Memory Bank Conflicts

页缓存
– 绕过 DeepSeek Fire-Flyer 文件系统, DeepSeek’s Fire-Flyer File System
– 对延迟敏感的训练工作流, Filesystem Caching and Write-Back
– O_DIRECT 绕过, Filesystem Caching and Write-Back, Using NVIDIA GDS

Page Migration Engine (PME; NVIDIA), GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling

页缓存大小, Filesystem Caching and Write-Back

页锁定内存, NUMA-Friendly Memory Allocation and Memory Pinning

PagedAttention (vLLM), Continuous Scheduling, KV Cache Offloading and Memory Pool Allocation, Memory management for the KV cache
– FlexDecoding, FlexDecoding (PyTorch)

并行聚合树（PAT）, NCCL Communication Algorithms

并行文件系统, Distributed, Parallel Filesystems and Object Stores

并行线程执行 (参见 PTX（并行线程执行))

并行性
– 超大规模推理的自适应策略, Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)-Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)
– 顺序和并行地相加两个向量, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization

- GPU 架构, Understanding GPU Architecture
- cuTile, C++ and Python CUDA Libraries
- 数据并行与分布式数据并行的比较, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
- 动态并行, Dynamic Parallelism-Dynamic Parallelism
  - 基准测试, Dynamic Parallelism
  - 子内核启动限制, Dynamic Parallelism
  - 监控设备内存使用情况, Dynamic Parallelism
  - 在变更前后进行性能分析, Dynamic Parallelism
  - 避免“栈溢出”错误, Dynamic Parallelism
  - 何时改为捕获 CUDA Graph, Dynamic Parallelism
- 全分片数据并行, Distributed Data Parallel Strategies
- PTX, Inline PTX and SASS Tuning for Microoptimizations
  - （另请参见 PTX（并行线程执行））
- 推测解码, Disaggregated Prefill and Decode Architecture, Speculative Decoding and Parallel Token Generation Techniques-Combining Decoding Techniques and Evaluating Complexity
- 策略, Parallelism Strategies for Serving Massive MoE Models-Hybrid Parallelism
  - 上下文并行, Parallelism Strategies for Serving Massive MoE Models, Context (Sequence) Parallelism
  - 数据并行, Parallelism Strategies for Serving Massive MoE Models, Data Parallelism
  - 专家并行, Parallelism Strategies for Serving Massive MoE Models, Expert Parallelism-Expert Parallelism
  - 混合并行, Hybrid Parallelism-Hybrid Parallelism
  - 跨 GPU 切分模型权重和数据, Parallelism Strategies for Serving Massive MoE Models
  - 流水线并行, Parallelism Strategies for Serving Massive MoE Models, Pipeline Parallelism
  - 性能分析, Hybrid Parallelism
  - 汇总表, Parallelism Strategies for Serving Massive MoE Models

    - 张量并行, Parallelism Strategies for Serving Massive MoE Models, Tensor Parallelism
  - 线程块集群算法, Designing Efficient Algorithms with Thread Block Clusters-Designing Efficient Algorithms with Thread Block Clusters

卸载到 CPU 和 NVMe 的参数, Offloading Parameters to CPU and NVMe

分区全局地址空间（PGAS）, Multi-GPU Programming

PAT（并行聚合树）, NCCL Communication Algorithms

PCIe, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams

PDU（配电单元）, Compute Density and Power Requirements

点对点 (参见 P2P（点对点）DMA)

每线程默认流（PTDS）, Modern Per-Thread Default Stream

perf (Linux), NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools

  - example MoE model, CPU and GPU Profiling with Linux perf-CPU and GPU Profiling with Linux perf
  - NVIDIA Performance Monitoring Unit, CPU and GPU Profiling with Linux perf

Perfetto Trace Viewer, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools, Multi-GPU Profiling with HTA

性能提升

  - 压缩数据, Tuning, Replicating, and Compressing Data
  - CUDA Graphs, Capturing a CUDA Graph with a CUDA Stream
    - conditional graph nodes, Conditional Graph Nodes
  - 解耦预填充与解码, Prefill-Decode Interference
  - 动态批处理, Dynamic Batching-Dynamic Batching
  - 高 ILP 下的高占用率, Loop Unrolling, Interleaving, and Compiler Hinting
  - 内核优化, Optimizing the Kernel-Optimizing the Kernel
  - max-autotune 编译模式, PyTorch Compiler Deep Dive
  - 内存固定, NUMA-Friendly Memory Allocation and Memory Pinning
  - 消除网络读取, Tuning, Replicating, and Compressing Data
  - 反复读取同一全局内存数据的陷阱, Tiling and Data Reuse Using Shared Memory

- 并发运行多个 warp, Maintaining High Occupancy and GPU Utilization
- 避免流 0 的旧式默认流, Legacy Default Stream
- TORCH_LOGS 性能提示, Performance Hints and Debugging Generated Code
- 多节点拓扑中与 NCCL 配合的 UCX, Enabling Peer-to-Peer DMA and UCX

性能工程师（参见 AI 系统性能工程师）

性能指标

- 性能调优的最终目标, Maximizing GPU Utilization and Throughput Versus Latency Trade-Offs
- 自动化性能测试, Benchmarking and Profiling
- 基准测试与性能分析, Benchmarking and Profiling
  - (另请参见 基准测试；性能分析)
- 缓存命中与未命中, Profiling, Debugging, and Tuning Inference Performance
- 通信与计算瓶颈的比较, Diagnosing Communication- Versus Compute-Bound Workloads
- 预填充与解码的解耦, Impact on Latency (TTFT) and Throughput (TPOT)
- 作为吞吐量指标的有效吞吐量, Measuring “Goodput” Useful Throughput
- GPU 空闲时间测量, Monitoring Storage I/O
- GPU 利用率百分比, Maximizing GPU Utilization and Throughput Versus Latency Trade-Offs
- 使用 NVIDIA Container Toolkit 的 GPU, Container Runtime Optimizations for GPUs
- 推理调优
  - 错误处理, Error Handling
  - 全栈优化, Full-Stack Inference Optimizations-Full-Stack Inference Optimizations
  - 监控系统指标与计数器, Monitoring System Metrics and Counters-Monitoring System Metrics and Counters
  - 观察—假设—调优循环, Profiling, Debugging, and Tuning Inference Performance-Profiling, Debugging, and Tuning Inference Performance
  - 使用 Nsight 进行性能分析, Profiling with Nsight Systems and Nsight Compute-Profiling with Nsight Systems and Nsight Compute
  - 故障排查, Inference Troubleshooting Recipes

- I/O 持续性能分析与调优, Continuous Profiling and Tuning Workflow-Continuous Profiling and Tuning Workflow
- 作为重要工具的延迟隐藏, Streaming Multiprocessor, Threads, and Warps
- MLPerf Logging, Performance Benchmarks and MLPerf Logging-Performance Benchmarks and MLPerf Logging
- 性能基准测试, Continuous Integration and Performance Benchmarking-Performance Benchmarks and MLPerf Logging
  - 未捕获完整图时强制报错, Graph Breaks and TorchDynamo explain()
- 性能监控, Performance Monitoring and Utilization in Practice
  - GPU 接近 100% 利用率, Performance Monitoring and Utilization in Practice
  - NVLink 使用情况, Performance Monitoring and Utilization in Practice
- 前缀合并指标, Profiling, Debugging, and Tuning Inference Performance
- PyTorch 性能平视显示器, Continuous Integration and Performance Benchmarking-PyTorch HUD Performance Dashboard
- 屋顶线模型, Roofline Model: Compute-Bound or Memory-Bound Workloads-Roofline Model: Compute-Bound or Memory-Bound Workloads
  - Nsight Compute 屋顶线分析, Nsight Compute and Roofline Analysis
- RPS (requests per second), Dynamic Batching
- 线程块大小, Choosing Threads-per-Block and Blocks-per-Grid Sizes
- TPOT (time per output token)
  - 预填充与解码的解耦, Impact on Latency (TTFT) and Throughput (TPOT)
  - 预填充与解码阶段, Scaling Disaggregated Prefill and Decode for Inference
- TTFT (time to first token)
  - constrained outputs, Constrained Decoding Performance Implications
  - context parallelism, Context (Sequence) Parallelism
  - disaggregation of prefill and decode, Impact on Latency (TTFT) and Throughput (TPOT)
  - prefill and decode phases, Scaling Disaggregated Prefill and Decode for Inference

performance regression continuous integration, Continuous Integration and Performance Benchmarking-Performance Benchmarks and MLPerf Logging

persistence mode for GPU, GPU Persistence Mode
- nvidia-persistenced daemon, GPU Persistence Mode
- recommended when using MIG, Slicing a GPU with MIG

persistent kernels, Persistent Kernels and Megakernels-Persistent Kernels and Warp Specialization
- common workloads for, Common Workloads for Persistent Kernels
- cooperative groups via grid sync, Cooperative Grid Synchronization and Persistent Kernels-Cooperative Grid Synchronization and Persistent Kernels
  - when to combine, When to Combine Persistent Kernels and Cooperative Groups
- megakernels, Megakernels for Inference
- occupancy, Persistent Kernels and Megakernels
- warp specialization, Persistent Kernels and Warp Specialization

持久化线程 (参见 持久化内核)

PGAS (partitioned global address space), Multi-GPU Programming

流水线并行（PP）, Parallelism Strategies for Serving Massive MoE Models, Pipeline Parallelism
- 与 FSDP 配合, Combining FSDP with Tensor Parallel and Pipeline Parallel
- 混合并行, Hybrid Parallelism
- 与 PyTorch 编译器配合, Tensor and Pipeline Parallelism with torch.compile
- 与张量并行的比较, Pipeline Parallelism
- 与 TP、混合并行的比较, Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)-Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)

流水线
- 无流水线与流水线的比较, Achieving Maximal Overlap in Practice-Achieving Maximal Overlap in Practice
- CUDA C++ Pipeline API, Asynchronous Memory Prefetching and Tensor Memory Accelerator-Asynchronous Memory Prefetching and Tensor Memory Accelerator
  - hardware-software codesign, Asynchronous Memory Prefetching and Tensor Memory Accelerator
- CUDA Graphs, Capturing a CUDA Graph with a CUDA Stream

    - （另请参见 CUDA Graphs）
- 数据压缩, Tuning, Replicating, and Compressing Data
- data pipeline tuning, Tuning the Data Pipeline
    - data loading and preprocessing, Efficient Data Loading and Preprocessing-Efficient Data Loading and Preprocessing
    - NeMo Curator for training datasets, Creating High-Quality LLM Datasets with NVIDIA NeMo Curator
    - NVIDIA Data Loading Library, Multimodal Data Processing with NVIDIA DALI
    - scaling out workers as GPUs scaled, Scaling Out Workers as You Scale Out Number of GPUs
- 分布式数据并行策略, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
- 内核间流水线
    - 内核间流水线概述, Inter-Kernel Pipelining, Synchronization, and CUDA Stream-Ordered Memory Allocations
    - 通过流使计算与数据传输重叠, Using Streams to Overlap Compute with Data Transfers-Using Streams to Overlap Compute with Data Transfers, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams, Overlapping Communication and Computation-Overlapping Communication and Computation
    - 跨流同步, Using CUDA Events for Cross-Stream Synchronization
    - CUDA 流有序内存分配器, Stream-Ordered Memory Allocator-Stream-Ordered Memory Allocator
    - 使用 CUDA 流有序内存分配器, Using CUDA Streams and Stream-Ordered Memory Allocator with LLMs-Using CUDA Streams and Stream-Ordered Memory Allocator with LLMs
    - 多 GPU 下的 CUDA 流有序内存分配器, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
    - 默认流用法, Best Practices for Default Stream Usage-Best Practices for Default Stream Usage
    - 默认流与显式流的比较, Default Versus Explicit (Nondefault) Streams
    - 通过 CUDA 流重叠内核执行, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams

- 在两个 CUDA 流上启动五个内核, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
- 旧式默认流, Legacy Default Stream
- 多 GPU 的计算与数据重叠, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams-Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
- 结合线程块集群和 warp 专用化的 PDL, Combining PDL and Thread Block Clusters with Warp Specialization-Combining PDL and Thread Block Clusters with Warp Specialization
- 每线程默认流, Modern Per-Thread Default Stream
- 程序化依赖启动, Programmatic Dependent Launch-Programmatic Dependent Launch
- 使用事件和回调同步, Fine-Grained Synchronization with Events and Callbacks
- warp 专用化与 CUDA 流, Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)-Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)
- 以 CUDA 流替代 warp 专用化, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
- warp 专用化、线程块集群和 CUDA 流, Warp Specialization with Thread Block Clusters and CUDA Streams

- 内核内流水线
  - 内核内流水线概述, Intra-Kernel Pipelining Techniques
  - 技术比较, Intra-Kernel Pipelining Techniques, Using CUDA Pipeline API for Warp Specialization, Warp Specialization with Thread Block Clusters-Warp Specialization with Thread Block Clusters
  - 协作组, Thread Block Clusters and Distributed Shared Memory
    - （另请参见 协作组（CG））
  - 使用 Pipeline API 的双缓冲, Cooperative Tiling and Double-Buffering with the CUDA Pipeline API-Cooperative Tiling and Double-Buffering with the CUDA Pipeline API
  - 持久化内核, Persistent Kernels and Megakernels-Persistent Kernels and Warp Specialization
  - PyTorch, PyTorch, CUDA Pipeline API, and Warp Specialization

  - 线程块集群, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters
    - (另请参见 线程块集群)
  - warp 专用化, Warp Specialization and the Producer-Consumer Model-Warp Specialization and the Producer-Consumer Model
  - warp 专用化与 CUDA 流, Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)-Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)
  - 使用 CUDA Pipeline API 的 warp 专用化, Using CUDA Pipeline API for Warp Specialization-Using CUDA Pipeline API for Warp Specialization
  - 持久化内核中的 warp 专用化, Persistent Kernels and Warp Specialization
  - 何时结合持久化内核与协作组, When to Combine Persistent Kernels and Cooperative Groups
  - 何时使用 warp 专用化而非双缓冲, Warp Specialization and the Producer-Consumer Model
- 监控 I/O, Monitoring Storage I/O-Monitoring Storage I/O
- 通信与计算重叠, Overlapping Communication and Computation (Pipelining)-Achieving Maximal Overlap in Practice
- 降低频率和数据量, Reducing Communication Frequency and Volume
- 用于异步执行的流, Asynchronous Execution with Streams
- Triton 双缓冲, Software Pipelining and Double Buffering with Triton-Software Pipelining and Double Buffering with Triton

PMU（NVIDIA 性能监控单元）, CPU and GPU Profiling with Linux perf

PoC（一致性点）, Understanding GPU Memory Hierarchy

POD-Attention, Optimized KV Cache Memory Layout

使用 NVIDIA Container Toolkit 的 Podman, NVIDIA Container Runtime

一致性点（PoC）, Understanding GPU Memory Hierarchy

可移植集群大小上限, Distributed Shared Memory

推理的功率和散热约束, Power and Thermal Constraints

配电单元（PDU）, Compute Density and Power Requirements

功耗监控, Performance Monitoring and Utilization in Practice

- NVIDIA 管理库, Kernel Compute Throughput Versus Peak GPU FLOPS

- 功率限制, Performance Monitoring and Utilization in Practice, Power and Thermal Constraints
  - 设置为略低于 TDP, GPU Clock Speeds and ECC

功耗要求

- GPU 持久化模式, GPU Persistence Mode
- NVL72, Compute Density and Power Requirements
  - liquid cooling versus air cooling, Liquid Cooling Versus Air Cooling-Liquid Cooling Versus Air Cooling
  - power monitoring, Performance Monitoring and Utilization in Practice

CPU 节能模式, CPU Frequency and C-states

PP (参见 流水线并行（PP))

#pragma unroll, Exposing Instruction-Level Parallelism, Loop Unrolling, Interleaving, and Compiler Hinting

- 展开过多迭代, Profiling and Mitigating Register Pressure

精度

- 动态精度变更, Dynamic Precision Changes-Dynamic Precision Changes
- FP4, FP8, FP16, NVIDIA GPU Tensor Cores and Transformer Engine, BF16/FP16, FP8, and FP4 Reduced Precision
- 推理量化, Reducing Precision from FP16 to FP8 and FP4
  - 仅权重量化, Weight-Only Quantization (GPTQ, AWQ)
- 混合精度
  - 自动混合精度, TF32 and Automatic Mixed Precision (PyTorch)-TF32 and Automatic Mixed Precision (PyTorch)
  - 降精度, BF16/FP16, FP8, and FP4 Reduced Precision
  - Tensor Cores, Mixed Precision and Utilizing Tensor Cores-Transformer Engine and TMEM in Depth
- 量化（参见 量化）

Predibase, Reinforcement Learning Approach to Generating Optimized GPU Kernels (Predibase)-Reinforcement Learning Approach to Generating Optimized GPU Kernels (Predibase)

谓词化, Techniques to Avoid Warp Divergence

- 避免 warp 发散, Techniques to Avoid Warp Divergence

- 最小化 warp 发散, Using Predication to Minimize Divergence-Using Predication to Minimize Divergence

预填充

- 预填充阶段概述, Scaling Disaggregated Prefill and Decode for Inference
- 分块预填充, Stall-Free Scheduling (Chunked Prefill), Adaptive Batching and Chunked Prefill Scheduling-Adaptive Batching and Chunked Prefill Scheduling
- 上下文并行, Parallelism Strategies for Serving Massive MoE Models, Context (Sequence) Parallelism
  - 上下文并行（PyTorch）, PyTorch Optimized Attention Mechanisms
- 与解码解耦
  - 解耦概述, Multinode Inference, Parallelism, Decoding, and Routing Optimizations, Prefill-Decode Interference, Scaling Disaggregated Prefill and Decode for Inference
  - 架构, Disaggregated Prefill and Decode Architecture
  - 批大小优化, Adaptive Batching and Chunked Prefill Scheduling
  - 集群池, Disaggregated Prefill and Decode Cluster Pools-Memory management for the KV cache
  - 使用 Kubernetes 部署, Deploying Disaggregated Prefill and Decode with Kubernetes-Deploying Disaggregated Prefill and Decode with Kubernetes, Full-Stack Inference Optimizations
  - 延迟与吞吐量, Impact on Latency (TTFT) and Throughput (TPOT)
  - 负载均衡, Dynamic Scheduling and Load Balancing-Dynamic resource scaling
  - 预填充到解码的数据传输, KV Cache Data Transfer and NIXL, Fast KV Cache Transfer Between Prefill and Decode-Zero-Copy GPU-to-GPU Transfer
  - 扩展, Scaling Prefill and Worker Nodes Independently
  - 分离预填充与解码阶段, Separate Prefill and Decode Inference Stages-Separate Prefill and Decode Inference Stages, Disaggregated Prefill and Decode Architecture
- 解耦式路由策略, Disaggregated Routing and Scheduling Policies-QoS and early rejection policies
  - 容量感知路由, Capacity-aware routing-Capacity-aware routing
  - 动态路由策略配置示例, Example dynamic routing policy configuration

    - 代码中的动态路由策略示例, Example dynamic routing policy in code
    - 延迟感知路由, Latency-aware routing-Latency-aware routing
    - 多路径推理（竞速）, Multipath inference (racing)
    - QoS 与提前拒绝策略, QoS and early rejection policies-QoS and early rejection policies
    - 路由因素, Routing factors-Routing factors
    - 推测解码, Multibranch, parallel speculative decoding across workers
  - FlashAttention, Mechanical Sympathy: Hardware-Software Codesign
  - 硬件与并行性, Heterogeneous Hardware and Parallelism Strategies for Prefill and Decode-Different precision for prefill and decode
  - GPU-CPU 协作的混合预填充, Hybrid Prefill with GPU-CPU Collaboration-Hybrid Prefill with GPU-CPU Collaboration
  - 向解码传输 KV 缓存, Fast KV Cache Transfer Between Prefill and Decode-Zero-Copy GPU-to-GPU Transfer
    - 连接器和数据路径设计, Connector and Data Path Design-Connector and Data Path Design
    - KV 缓存大小, KV Cache Size
    - 零拷贝 GPU 到 GPU, Zero-Copy GPU-to-GPU Transfer-Zero-Copy GPU-to-GPU Transfer
  - 借助 FlexDecoding 优化内核, FlexDecoding (PyTorch)-FlexDecoding (PyTorch)
  - 工作节点横向扩展, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction
  - 为何解耦, Why Prefill-Decode Disaggregation?-Scalability of Disaggregated Prefill and Decode
    - 解耦的优势, Advantages of Disaggregation-Phase-specific optimizations
    - 预填充工作节点, Prefill workers design-Latency-aware scheduling and batching
    - 路由和调度策略, Disaggregated Routing and Scheduling Policies-QoS and early rejection policies
    - 可扩展性, Scalability of Disaggregated Prefill and Decode

预填充—解码干扰, Prefill-Decode Interference

前缀缓存, Prefix Caching-Prefix Caching

  - SGLang RadixAttention, Prefix Caching-Prefix Caching

预热图和缓存, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction-Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction

PrimTorch 中间表示（IR）, PrimTorch IR (Prims) Simplified Operator Set

- FX 图输出, PrimTorch IR (Prims) Simplified Operator Set
- PyTorch 编译器流水线, PyTorch Compiler Deep Dive

进程—GPU 映射, Adaptive Process-GPU Mapping-Adaptive Process-GPU Mapping

profiling, Benchmarking and Profiling

- achieved occupancy, Inspecting Achieved Occupancy and GPU Utilization-Optimizing the Kernel
  - kernel compute throughput versus GPU FLOPS, Kernel Compute Throughput Versus Peak GPU FLOPS
  - kernel memory throughput versus HBM memory bandwidth, Kernel Memory Throughput Versus Peak HBM Memory Bandwidth
- before and after a change, Dynamic Parallelism
- compiler performance issues, Profiling and Debugging Compiler Performance Issues
- CUDA events for, Using CUDA Events for Cross-Stream Synchronization
- DataLoader, Efficient Data Loading and Preprocessing
- example mixture-of-experts model, Profiling PyTorch to Identify Bottlenecks-CPU and GPU Profiling with Linux perf
  - about MoE models, Profiling PyTorch to Identify Bottlenecks
  - Linux perf on CPU and GPU, CPU and GPU Profiling with Linux perf-CPU and GPU Profiling with Linux perf
  - Nsight Systems and NVTX Timelines, System Profiling with Nsight Systems and NVTX Timelines-System Profiling with Nsight Systems and NVTX Timelines
  - PyTorch profiler, Using PyTorch Profiler-Using PyTorch Profiler
  - roofline analysis for GEMM kernels, Kernel Roofline Analysis for General Matrix Multiply (GEMM)-Kernel Roofline Analysis for General Matrix Multiply (GEMM)
- GPU bottlenecks, Profiling and Diagnosing GPU Bottlenecks-Profiler-Guided Analysis

  - achieved occupancy and GPU utilization, Inspecting Achieved Occupancy and GPU Utilization-Optimizing the Kernel
  - CUDA Profiling Tools Interface, PyTorch Profiler and Visualization Tools
  - iteratively profiling, Iteratively Profiling and Determining the Kernel Bottleneck-Iteratively Profiling and Determining the Kernel Bottleneck
  - kernel optimization, Optimizing the Kernel-Optimizing the Kernel
  - Nsight Compute, Profiling and Diagnosing GPU Bottlenecks
  - Nsight Compute and Roofline analysis, Nsight Compute and Roofline Analysis
  - Nsight Systems, Profiling and Diagnosing GPU Bottlenecks
  - Nsight Systems timeline view, Nsight Systems Timeline View
  - PyTorch profiler via Kineto, PyTorch Profiler and Visualization Tools-Profiler-Guided Analysis
  - warp stall reasons, Analyzing Warp Stall Reasons with Nsight Compute-Other Stall Reasons
- graph break recompilations monitored, TorchDynamo for Bytecode Capture and Graph Extraction, Minimize Graph Recompilations
- Holistic Trace Analysis, Multi-GPU Profiling with HTA
- inference, Profiling with Nsight Systems and Nsight Compute-Profiling with Nsight Systems and Nsight Compute
  - production environment, Inference Troubleshooting Recipes
- I/O continuous profiling and tuning, Continuous Profiling and Tuning Workflow-Continuous Profiling and Tuning Workflow
  - communication versus compute bottlenecks, Diagnosing Communication- Versus Compute-Bound Workloads
- kernels generated by TorchInductor, Profiling and Debugging Compiler Performance Issues
- latency with streaming enabled, Streaming Responses
- memory via PyTorch profiler, Profiling and Tuning Memory in PyTorch-Enabling Peer-to-Peer DMA and UCX
  - activation checkpointing, Activation Checkpointing for Memory Savings
  - CUDA memory allocator tuned, Tuning the CUDA Memory Allocator
  - FSDP automatic checkpointing, FSDP Automatic Checkpointing and Offloading-FSDP Automatic Checkpointing and Offloading

    - FSDP with tensor parallel and pipeline parallel, Combining FSDP with Tensor Parallel and Pipeline Parallel
    - NCCL with UCX over multinode topologies, Enabling Peer-to-Peer DMA and UCX
    - offloading parameters, Offloading Parameters to CPU and NVMe
    - peer-to-peer DMA, Enabling Peer-to-Peer DMA and UCX
    - pluggable memory allocators, Pluggable Memory Allocators and Cross-GPU Data Transfers-Pluggable Memory Allocators and Cross-GPU Data Transfers
- multi-GPU profiling
    - HTA, Multi-GPU Profiling with HTA
    - Nsight Systems, Real-Time Link Telemetry and Monitoring
- NCCL, Profiling and Debugging NCCL
    - NCCL logs, In-Network SHARP Aggregation
- Nsight Compute occupancy limiters, Inspecting Achieved Occupancy and GPU Utilization
    - Limited by Registers, Profiling and Mitigating Register Pressure
- Nsight Compute warp stall reasons, Analyzing Warp Stall Reasons with Nsight Compute-Other Stall Reasons
    - Stall: Compute Unit Busy, Execution Unit Contention
    - Stall: Exec Dependency, Execution-Dependency Stalls, Profiling and Mitigating Register Pressure
    - Stall: Idle, Other Stall Reasons
    - Stall: Long Scoreboard, Memory-Related Stalls
    - Stall: Math Pipe Throttle, Execution Unit Contention
    - Stall: Memory Dependency, Other Stall Reasons
    - Stall: Memory Throttle, Memory-Related Stalls
    - Stall: No Eligible, Other Stall Reasons
    - Stall: Not Selected, Memory-Related Stalls
    - Stall: Short Scoreboard, Memory-Related Stalls
    - Stall: Texture Throttle, Other Stall Reasons
    - stalling for other reasons, Other Stall Reasons-Other Stall Reasons

- NVTX markers, NVTX Markers and Profiling Tools
  - profiling tools that use markers, NVTX Markers and Profiling Tools-NVTX Markers and Profiling Tools
- occupancy tuning, Find the Right Occupancy for Your Workload
- parallelism strategies, Hybrid Parallelism
- tools for profiling, NVTX Markers and Profiling Tools-NVTX Markers and Profiling Tools
- Triton Proton Profiler, Profiling with Triton Proton Profiler
- warp divergence, Profiling and Detecting Warp Divergence
- workload under insufficient bandwidth, Pitfall #4: Insufficient network bandwidth or misconfigured NICs

程序化依赖启动（PDL；NVIDIA）, Programmatic Dependent Launch-Programmatic Dependent Launch

- 结合线程块集群和 warp 专用化, Combining PDL and Thread Block Clusters with Warp Specialization-Combining PDL and Thread Block Clusters with Warp Specialization

编程

- 自动化代码优化, Smart Compilers and Automated Code Optimizations-Smart Compilers and Automated Code Optimizations
- CUDA
  - 异步内存分配, Asynchronous Memory Allocation and Memory Pools-Asynchronous Memory Allocation and Memory Pools
  - CUDA 流, Asynchronous Memory Allocation and Memory Pools
  - CPU 与 GPU 之间的流程, Understanding GPU Architecture
  - __global__, CUDA Programming Refresher
  - GPU 内存层次结构, Understanding GPU Memory Hierarchy-Understanding GPU Memory Hierarchy
  - 高占用率和 GPU 利用率, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
  - 可用的最高层级和最新 API, Asynchronous Memory Prefetching and Tensor Memory Accelerator
  - 内核错误, CUDA Programming Refresher
  - 一维内核输入, 2D and 3D Kernel Inputs

  - 二维和三维内核输入, 2D and 3D Kernel Inputs
  - 用于并行工作的内核, CUDA Programming Refresher-CUDA Programming Refresher
  - 启动参数, Configuring Launch Parameters: Blocks per Grid and Threads per Block-Configuring Launch Parameters: Blocks per Grid and Threads per Block
  - 最大 threadsPerBlock 编译时参数, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
  - 内存池, Asynchronous Memory Allocation and Memory Pools-Asynchronous Memory Allocation and Memory Pools
  - 使用 __launch_bounds__ 调优占用率, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
  - 屋顶线模型, Roofline Model: Compute-Bound or Memory-Bound Workloads-Roofline Model: Compute-Bound or Memory-Bound Workloads
  - Unified Memory, Unified Memory-Unified Memory
- 指令级并行
  - 编译器提示, Loop Unrolling, Interleaving, and Compiler Hinting
  - 交错独立操作, Loop Unrolling, Interleaving, and Compiler Hinting
  - 循环展开, Loop Unrolling, Interleaving, and Compiler Hinting, Kernel Fusion
- 内核融合, Kernel Fusion-Kernel Fusion
- 循环展开, Loop Unrolling, Interleaving, and Compiler Hinting, Kernel Fusion
- MMA API, Transformer Engine and TMEM in Depth-Transformer Engine and TMEM in Depth
- 多个 GPU, Multi-GPU Programming-Multi-GPU Programming
- NVIDIA Compute Sanitizer, Debugging Functional Correctness with NVIDIA Compute Sanitizer
- NVLink-C2C 的 CPU 到 GPU 内存访问, NUMA Awareness and CPU Pinning
- 使用线程块集群并行化工作负载, Designing Efficient Algorithms with Thread Block Clusters-Designing Efficient Algorithms with Thread Block Clusters
- PTX, Feeding Tensor Cores with TMEM and TMA, Inline PTX and SASS Tuning for Microoptimizations

    - code from compiler, CUDA Forward and Backward Compatibility Across GPU Hardware Generations, CUDA GPU Backward and Forward Compatibility Model
    - compatibility across GPU generations, CUDA Forward and Backward Compatibility Across GPU Hardware Generations, CUDA GPU Backward and Forward Compatibility Model
    - inline PTX code, Inline PTX and SASS Tuning for Microoptimizations-Inline PTX and SASS Tuning for Microoptimizations
    - inline PTX code in DeepSeek DeepEP, DeepSeek’ s Use of Inline PTX for Memory Allocation Optimization
    - streaming assembler, Inline PTX and SASS Tuning for Microoptimizations-Inline PTX and SASS Tuning for Microoptimizations
    - streaming assembler changing across generations, Inline PTX and SASS Tuning for Microoptimizations
    - Triton, Triton Programming Model
  - Python-centric CUDA libraries, Writing Custom Kernels with OpenAI Triton
  - smart compilers, Smart Compilers and Automated Code Optimizations-Smart Compilers and Automated Code Optimizations
  - Triton programming model, Triton Programming Model

Prometheus

  - 推理监控, Monitoring System Metrics and Counters, Monitoring System Metrics and Counters
    - 告警管理器, Debugging Correctness Issues
    - Data Center GPU Manager 指标, Monitoring System Metrics and Counters
  - NVLink/NVSwitch 监控, Real-Time Link Telemetry and Monitoring

提示词清理, Prompt Cleansing

  - CTRL 方法, Prompt Cleansing

提示词压缩, Prompt Compression

提示词服务器, Prefill workers design

提示词极长

  - 上下文并行, Context (Sequence) Parallelism
  - 提示词压缩, Prompt Compression
  - 无停顿调度, Stall-Free Scheduling (Chunked Prefill)

剪枝, Structured Sparsity

- 动态 Token 剪枝, Dynamic Token Pruning with LazyLLM
- TorchAO, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning

PTDS（每线程默认流）, Modern Per-Thread Default Stream

PTX（并行线程执行）, Inline PTX and SASS Tuning for Microoptimizations, TorchInductor Backend Code Generation

- 来自编译器的代码, CUDA Forward and Backward Compatibility Across GPU Hardware Generations, CUDA GPU Backward and Forward Compatibility Model, TorchInductor Backend Code Generation
  - 跨 GPU 世代的 CUDA 兼容性, CUDA Forward and Backward Compatibility Across GPU Hardware Generations, CUDA GPU Backward and Forward Compatibility Model
  - JIT 编译, CUDA Forward and Backward Compatibility Across GPU Hardware Generations
- 内联 PTX 代码, Inline PTX and SASS Tuning for Microoptimizations-Inline PTX and SASS Tuning for Microoptimizations
  - DeepSeek 内存分配优化, DeepSeek’s Use of Inline PTX for Memory Allocation Optimization
- 流式汇编器, Inline PTX and SASS Tuning for Microoptimizations-Inline PTX and SASS Tuning for Microoptimizations
  - 随 GPU 架构世代变化, Inline PTX and SASS Tuning for Microoptimizations
- TMEM 和 CUTLASS, Feeding Tensor Cores with TMEM and TMA
- TorchInductor, TorchInductor Backend Code Generation
- Triton, Triton Programming Model

ptxas warning, Tuning Occupancy with Launch Bounds

Python

- 将条件语句作为张量操作, TorchDynamo for Bytecode Capture and Graph Extraction
- 创建 CUDA 内核, NVIDIA Software Stack
- CUDA 库, Writing Custom Kernels with OpenAI Triton
- CUDA Toolkit 库, C++ and Python CUDA Libraries
  - NumPy 替代品 cuPyNumeric, C++ and Python CUDA Libraries

- 构建在 CUDA 之上的框架, PyTorch and Higher-Level AI Frameworks
  - (另请参见 PyTorch)
- Nsight Systems 回溯采样, PyTorch Profiler and Visualization Tools
- Triton 语言和编译器, C++ and Python CUDA Libraries

PyTorch

- 算术强度, PyTorch and Arithmetic Intensity
- 优化的注意力机制, PyTorch Optimized Attention Mechanisms
- 自动混合精度, TF32 and Automatic Mixed Precision (PyTorch)-TF32 and Automatic Mixed Precision (PyTorch)
- 后端支持, Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL
  - 验证使用的后端, Pitfall #1: Using a CPU-bound Gloo backend instead of NCCL
- 编译器 (参见 PyTorch 编译器)
- CUDA Graphs, PyTorch, Inference Engines, and CUDA Graphs, Capturing a CUDA Graph with a CUDA Stream
  - static memory pools, Memory Pools for CUDA Graphs
- 自定义内核注册, Registering Custom Kernels with PyTorch-Registering Custom Kernels with PyTorch
- DataLoader
  - 每个工作进程的 CPU 亲和性, NUMA Awareness and CPU Pinning-NUMA Awareness and CPU Pinning
  - 数据输入流水线优化, Optimizing the Data Input Pipeline-Optimizing the Data Input Pipeline
  - 数据加载与预处理, Efficient Data Loading and Preprocessing-Efficient Data Loading and Preprocessing
  - 多个工作进程, Tuning NVMe and Filesystem for Throughput, Efficient Data Loading and Preprocessing, Scaling Out Workers as You Scale Out Number of GPUs, Optimizing the Data Input Pipeline
  - 持久工作进程, Efficient Data Loading and Preprocessing
  - 固定内存标志, NUMA-Friendly Memory Allocation and Memory Pinning, Using Streams to Overlap Compute with Data Transfers, Optimizing the Data Input Pipeline
  - 预取因子, Optimizing the Data Input Pipeline

- 分布式数据并行通信
  - 分桶, Reducing Communication Frequency and Volume
  - 数据并行, Distributed Data Parallel Strategies
  - 数据并行与分布式数据并行的比较, Distributed Data Parallel Strategies-Distributed Data Parallel Strategies
  - 数据并行的定义, Distributed Data Parallel Strategies
  - 分布式数据并行的定义, Distributed Data Parallel Strategies
  - 模型并行, Distributed Data Parallel Strategies
- DistributedSampler, Fast Storage and Data Locality
- 文件系统缓存, Filesystem Caching and Write-Back
- GPU 空闲时间测量, Monitoring Storage I/O
- 内核内流水线, PyTorch, CUDA Pipeline API, and Warp Specialization
- 简介, PyTorch and Higher-Level AI Frameworks
  - 编译器栈, PyTorch and Higher-Level AI Frameworks, PyTorch Compiler (torch.compile), PyTorch Compiler, OpenAI Triton, and XLA Backends
  - 抽象掉 CUDA 复杂性, PyTorch and Higher-Level AI Frameworks
  - 使用 GPU 的张量操作, PyTorch and Higher-Level AI Frameworks
  - TorchInductor 后端, PyTorch and Higher-Level AI Frameworks
- 带合并的最佳适配内存分配器, Dynamic Memory-Allocation Switching (Slab Versus Caching Versus Stream-Ordered)
- 内存分配器插件, Pluggable Memory Allocators and Cross-GPU Data Transfers-Pluggable Memory Allocators and Cross-GPU Data Transfers
- 内存缓存分配器, Asynchronous Memory Allocation and Memory Pools
- 内存性能分析器, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools
- 在专用流上运行的 NCCL 集合通信, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
- 嵌套张量, PyTorch and Arithmetic Intensity
- NVLink 域, NVIDIA’s “AI Supercomputer in a Rack”
- 持久化内核和巨型内核, Common Workloads for Persistent Kernels
- 性能分析器 (参见 PyTorch 性能分析器)
- 从训练后的稠密模型生成的稀疏格式, Structured Sparsity

- 流
  - 创建, Concurrency with CUDA Streams
  - 默认流与显式流的比较, Default Versus Explicit (Nondefault) Streams
  - 多流下的多个 GPU, Multi-GPU Compute and Data Transfer Overlap with CUDA Streams
  - 计算与数据传输重叠, Overlapping Communication and Computation
  - 添加流时进行性能分析, Overlapping Communication and Computation
- 对称内存, PyTorch Symmetric Memory

PyTorch 架构优化（torchao）, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning

- 剪枝, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning
- 量化, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning
  - INT8 量化, Transformer Engine and TMEM in Depth
- 稀疏性, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning

PyTorch compiler (torch.compile), PyTorch Compiler (torch.compile)-Profiling and Debugging Compiler Performance Issues

- about the compiler, PyTorch and Higher-Level AI Frameworks, PyTorch Compiler (torch.compile)
- arithmetic intensity, PyTorch and Arithmetic Intensity
- AsyncTP enabled, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
- compilation pipeline, PyTorch Compiler Deep Dive-Debugging Numerical Correctness and Accuracy
  - about the compilation pipeline, PyTorch Compiler Deep Dive
  - AOT Autograd, AOT Autograd Fusion for Forward and Backward Passes
  - PrimTorch IR, PrimTorch IR (Prims) Simplified Operator Set
  - TorchDynamo, TorchDynamo for Bytecode Capture and Graph Extraction-TorchDynamo for Bytecode Capture and Graph Extraction
  - TorchInductor, TorchInductor Backend Code Generation
- compiler modes, Compilation Modes and Trade-Offs in Speed, Memory, and Compile Time-Compilation Modes and Trade-Offs in Speed, Memory, and

Compile Time
  - CUDA Graphs triggered, Autotuning with TorchInductor
  - eager mode, Disabling the PyTorch Compiler and Reverting Back to Eager Mode
  - max-autotune mode, PyTorch Compiler Deep Dive
  - static shapes required for CUDA Graphs, Autotuning with TorchInductor, Dynamic Shapes and Variable Sequence Lengths
- compiler stack, PyTorch and Higher-Level AI Frameworks, PyTorch Compiler (torch.compile), PyTorch Compiler, OpenAI Triton, and XLA Backends
- compiling versus writing custom kernels, Compiling Versus Writing Custom Kernels
- CUDA Graph Trees, CUDA Graph Trees (PyTorch Compiler Internal)
- debugging, Performance Hints and Debugging Generated Code, Debugging Numerical Correctness and Accuracy, Debugging Compiler Phases, Graph Breaks, and Performance
  - numerical correctness and accuracy, Debugging Numerical Correctness and Accuracy-Debugging Numerical Correctness and Accuracy
  - TORCH_LOGS, Profiling and Debugging Compiler Performance Issues, Performance Hints and Debugging Generated Code
- disabling, Disabling the PyTorch Compiler and Reverting Back to Eager Mode
  - eager mode, Disabling the PyTorch Compiler and Reverting Back to Eager Mode
- dynamic shapes, Dynamic Shapes and Variable Sequence Lengths-Dynamic Shapes and Variable Sequence Lengths
- export, TorchInductor Backend Code Generation
- kernel fusion, Tuning Occupancy with PyTorch, Kernel Fusion, TorchDynamo for Bytecode Capture and Graph Extraction
- launch overhead reduced via CUDA Graphs, PyTorch, Inference Engines, and CUDA Graphs
- Mega-Cache, Using the PyTorch Compiler
- per-thread default stream enabled, Modern Per-Thread Default Stream
- performance hints, Performance Hints and Debugging Generated Code
- pipeline parallel with, Tensor and Pipeline Parallelism with torch.compile

- profiling and debugging compiler performance, Profiling and Debugging Compiler Performance Issues
- quantization with torchao, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning
- scaling with DDP, DDP with torch.compile
- scaling with FSDP, FSDP with torch.compile-FSDP with torch.compile
- SimpleFSDP, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
- smart compilers, Smart Compilers and Automated Code Optimizations-Smart Compilers and Automated Code Optimizations
- tensor parallel with, Tensor and Pipeline Parallelism with torch.compile
- TorchInductor, Coalesced Versus Uncoalesced Global Memory Access, Using the PyTorch Compiler, Using the PyTorch Compiler
  - profiling and benchmarking kernels generated, Profiling and Debugging Compiler Performance Issues
- Transformer Engine, Autotuning with TorchInductor
- using, Using the PyTorch Compiler-Using the PyTorch Compiler
- vectorized memory operations, Vectorized Memory Access

PyTorch DataLoader（参见 DataLoader（PyTorch））

PyTorch Distributed Data Parallel（参见 分布式数据并行（DDP；PyTorch））

PyTorch Fully Sharded Data Parallel（参见 FSDP（完全分片数据并行；PyTorch））

PyTorch Holistic Trace Analysis（参见 Holistic Trace Analysis（HTA；PyTorch））

PyTorch Kineto, PyTorch Profiler and Visualization Tools, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools
- NCCL profiler plugin, Profiling and Debugging NCCL

PyTorch Mega-Cache, Using the PyTorch Compiler

PyTorch profiler
- CUDA Performance Tools Interface, PyTorch Profiler and Visualization Tools
- example mixture-of-experts model, Using PyTorch Profiler-Using PyTorch Profiler
- GPU bottlenecks, PyTorch Profiler and Visualization Tools-Profiler-Guided Analysis
- Kineto library, PyTorch Profiler and Visualization Tools, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools

    - NCCL profiler plugin, Profiling and Debugging NCCL
  - memory profiler, NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools
  - memory profiling and tuning, Profiling and Tuning Memory in PyTorch-Enabling Peer-to-Peer DMA and UCX
    - activation checkpointing, Activation Checkpointing for Memory Savings
    - CUDA memory allocator tuned, Tuning the CUDA Memory Allocator
    - FSDP automatic checkpointing, FSDP Automatic Checkpointing and Offloading-FSDP Automatic Checkpointing and Offloading
    - FSDP with tensor parallel and pipeline parallel, Combining FSDP with Tensor Parallel and Pipeline Parallel
    - NCCL with UCX over multinode topologies, Enabling Peer-to-Peer DMA and UCX
    - offloading parameters, Offloading Parameters to CPU and NVMe
    - peer-to-peer DMA, Enabling Peer-to-Peer DMA and UCX
    - pluggable memory allocators, Pluggable Memory Allocators and Cross-GPU Data Transfers-Pluggable Memory Allocators and Cross-GPU Data Transfers
  - NCCL profiler plugin, Profiling and Debugging NCCL
  - Nsight Systems PyTorch focused mode, PyTorch Profiler and Visualization Tools

PyTorch TorchDynamo（参见 TorchDynamo（PyTorch））

PyTorch TorchEval（参见 TorchEval（PyTorch））

PyTorch TorchInductor（参见 TorchInductor（PyTorch））

PYTORCH_CUDA_ALLOC_CONF, GPU Memory Oversubscription, Fragmentation, and Out-of-Memory Handling, Stream-Ordered Memory Allocator

Q

QoS（服务质量）, 服务质量

  - 动态路由策略, 动态路由策略配置示例
  - 早期拒绝策略, QoS 与早期拒绝策略-QoS 与早期拒绝策略

四裸片 Rubin GPU, Rubin Ultra and Vera Rubin Ultra (2027)

  - GPC“分区”, Thread Block Clusters and Distributed Shared Memory

服务质量（参见 QoS（服务质量））

量化

- 动态激活量化, 动态量化与激活范围调整
- 推理优化, 实时推理的量化方法-将量化—反量化步骤融合进执行图
  - 关于量化, 实时推理的量化方法
  - 激活量化, 激活量化
  - 联合使用权重与激活量化, 联合使用权重与激活量化
  - 训练后量化工作流, 训练后量化工作流
  - 将量化—反量化步骤置入执行图, 将量化—反量化步骤融合进执行图
  - 降低精度, 将精度从 FP16 降至 FP8 和 FP4
  - 仅权重量化, 仅权重量化（GPTQ、AWQ）
- KV 压缩, 实时 KV 缓存压缩与策略切换
- TorchAO, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning

Quantum InfiniBand（NVIDIA）, Co-Packaged Optics: Future of Networking Hardware

R

R300 Rubin Ultra GPU, Rubin Ultra and Vera Rubin Ultra (2027)

RadixAttention（SGLang）, Continuous Scheduling

- 前缀缓存, Prefix Caching

随机读取与顺序读取的比较, Sequential Versus Random Read Patterns-Sequential Versus Random Read Patterns

RDMA（远程直接内存访问）, Multi-GPU Programming, High-Speed, Low-Overhead Data Transfers with RDMA-Pitfall #6: GPU memory fragmentation under UCX/RDMA

- DeepSeek Fire-Flyer 文件系统, DeepSeek’s Fire-Flyer File System
- GPUDirect RDMA, Multi-GPU Programming, High-Speed, Low-Overhead Data Transfers with RDMA
  - 多节点、多机架通信, Multinode and Multirack Communication with GPUDirect RDMA-Multinode and Multirack Communication with GPUDirect RDMA

    - PyTorch MemPool 中的 NCCL 原生分配器, Pluggable Memory Allocators and Cross-GPU Data Transfers
    - 验证真正的 GPUDirect RDMA, High-Speed, Low-Overhead Data Transfers with RDMA
  - InfiniBand 支持, High-Speed, Low-Overhead Data Transfers with RDMA
  - 零拷贝 GPU 到 GPU 传输, Zero-Copy GPU-to-GPU Transfer-Zero-Copy GPU-to-GPU Transfer

RDMA over Converged Ethernet (RoCE), Multi-GPU Programming

只读数据缓存, Read-Only Data Caches-Read-Only Data Caches

  - const __restrict__, Read-Only Data Caches, Read-Only Data Caches, Read-Only Data Caches
  - 未声明只读的陷阱, Read-Only Data Caches

读取数据

  - DeepSeek Fire-Flyer 文件系统, DeepSeek’s Fire-Flyer File System-DeepSeek’s Fire-Flyer File System
  - 消除网络读取, Tuning, Replicating, and Compressing Data
  - 预读, Tuning NVMe and Filesystem for Throughput
  - 顺序与随机模式的比较, Sequential Versus Random Read Patterns-Sequential Versus Random Read Patterns
    - 经过优化的文件系统, Sequential Versus Random Read Patterns
    - 调优读取大小, Sequential Versus Random Read Patterns

重计算与内存的权衡, Recomputation Versus Memory Trade-Off

降精度, BF16/FP16, FP8, and FP4 Reduced Precision

寄存器

  - GPUDirect RDMA, Multi-GPU Programming
  - 内存层次结构, Streaming Multiprocessor, Threads, and Warps
  - 占用率权衡, Tuning Occupancy with Launch Bounds
  - 来自 ILP 的寄存器压力, Profiling and Mitigating Register Pressure
  - 寄存器溢出, Tuning Occupancy with Launch Bounds

强化学习（RL）

  - 在运行时调优 AI 的智能体, Reinforcement Learning Agents for Tuning AI Systems at Runtime

  - 借助强化学习进行 GPU 内核优化, Reinforcement Learning Approach to Generating Optimized GPU Kernels (Predibase)-Reinforcement Learning Approach to Generating Optimized GPU Kernels (Predibase)

远程直接内存访问 (参见 RDMA（远程直接内存访问))

可复现性, Transparency and Reproducibility, Reproducibility and Documentation Best Practices-Reproducibility and Documentation Best Practices

请求合并, Debouncing and Request Coalescing

每秒请求数（RPS）, Dynamic Batching

在线资源

  - 书籍补充材料, Using Code Examples
  - 书籍网页, How to Contact Us
  - FireworksAI 的 CUDA Graphs 文章, Replaying the Graph
  - Meta 报告的大规模 GPU 故障, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely
  - NCCL 环境变量, NCCL Communicator Lifecycle and Environment Gotchas
  - PyTorch 中计算与数据传输重叠, Overlapping Communication and Computation

环形通信算法, NCCL Communication Algorithms

RL (参见 强化学习（RL))

RoCE（融合以太网上的 RDMA）, Multi-GPU Programming

  - 启用 GPUDirect RDMA, Optimizing Network Communication for Kubernetes

屋顶线分析（Nsight Compute）, Nsight Compute and Roofline Analysis

  - MoE 模型示例, Kernel Roofline Analysis for General Matrix Multiply (GEMM)-Kernel Roofline Analysis for General Matrix Multiply (GEMM)

屋顶线模型, Roofline Model: Compute-Bound or Memory-Bound Workloads-Roofline Model: Compute-Bound or Memory-Bound Workloads

  - 屋顶线引导的调度, Roofline-Guided Scheduling and Orchestration Decisions

屋顶线性能模型, Increasing CUDA Kernel Efficiency and Arithmetic Intensity

  - 算术强度, Increasing CUDA Kernel Efficiency and Arithmetic Intensity-Increasing CUDA Kernel Efficiency and Arithmetic Intensity, Roofline-Guided Scheduling and Orchestration Decisions
    - 使用 Tensor Cores 的较低精度, Mixed Precision and Utilizing Tensor Cores

RPS（每秒请求数）, Dynamic Batching

Rubin Ultra GPU (R300), Rubin Ultra and Vera Rubin Ultra (2027)

运行时优化, Full-Stack Inference Optimizations

S

SASS（参见 流式汇编器（SASS））

可扩展分层聚合与归约协议（参见 SHARP（NVIDIA 可扩展分层聚合与归约协议））

缩放点积注意力（SDPA；PyTorch）, PyTorch Optimized Attention Mechanisms

scaling, Scaling Distributed Training and Inference

- disaggregated prefill and decode, Scalability of Disaggregated Prefill and Decode
  - prefill and decode workers, Continuous Prewarming of CUDA Graphs and Caches Using Time-Series Prediction
- n-GPU scaling pattern, Pattern for N-GPU Scaling
- PyTorch distributed libraries, Scaling with PyTorch Distributed-Multi-GPU Profiling with HTA
  - DDP with torch.compile, DDP with torch.compile
  - FSDP with torch.compile, FSDP with torch.compile-FSDP with torch.compile

调度器和中断亲和性, Scheduler and Interrupt Affinity

scheduling

- 蝶形调度, Expert Communication Optimization
- 集群调度器, Sharing and Scheduling
- congestion- and topology-aware with multiple GPUs, Congestion-Aware and Topology-Aware Scheduling with Multiple GPUs-Coordinating NVSwitch Transfers with Fine-Tuned Scheduling
  - adaptive process-GPU mapping, Adaptive Process-GPU Mapping-Adaptive Process-GPU Mapping
  - dynamic congestion-aware scheduling, Dynamic Congestion-Aware Scheduling-Dynamic Congestion-Aware Scheduling
  - GPUDirect RDMA, Multinode and Multirack Communication with GPUDirect RDMA-Multinode and Multirack Communication with GPUDirect RDMA
  - MoE expert rebalancing and regrouping, MoE Expert Rebalancing and Regrouping

  - multinode and multirack communication, Multinode and Multirack Communication with GPUDirect RDMA
  - NCCL collective communication, Optimizing Collective Communication with NCCL-Wave scheduling of collectives
  - NVSwitch and fine-tuned scheduling, Coordinating NVSwitch Transfers with Fine-Tuned Scheduling-Coordinating NVSwitch Transfers with Fine-Tuned Scheduling
  - telemetry and monitoring, Real-Time Link Telemetry and Monitoring
- 连续调度, Continuous Scheduling
  - 与动态批处理合并, Continuous Scheduling
- 动态调度与负载均衡, Dynamic Scheduling and Load Balancing-Dynamic resource scaling
- 拥塞感知动态调度, Dynamic Congestion-Aware Scheduling-Dynamic Congestion-Aware Scheduling
- 使用原子工作队列的动态调度, Dynamic Scheduling with Atomic Work Queues-Atomic Queues
  - 原子计数器, Atomic Counters
  - 原子队列, Atomic Queues-Atomic Queues
  - 允许动态工作分配的原子队列, Atomic Counters
  - Nsight Compute, Atomic Counters-Atomic Counters
- 先进先出（FIFO）调度器, Latency-Aware Scheduling and Dynamic Routing-Latency-Aware Scheduling and Dynamic Routing
- 核函数内持久调度, Atomic Queues and Device-Initiated CUDA Graphs for In-Kernel Persistent Scheduling
- 迭代级调度, Continuous Batching
- Kubernetes 作业调度, Job Scheduling with Kubernetes and SLURM
- 延迟感知调度和动态路由, Latency-Aware Scheduling and Dynamic Routing-Latency-Aware Scheduling and Dynamic Routing
- Linux schedulers
  - 完全公平排队已废弃, Tuning NVMe and Filesystem for Throughput
  - 完全公平调度器, Scheduler and Interrupt Affinity
  - 多队列块 I/O 调度器, Tuning NVMe and Filesystem for Throughput
  - none 调度器, Tuning NVMe and Filesystem for Throughput

    - 设置, Tuning NVMe and Filesystem for Throughput
  - 屋顶线分析引导的调度, Roofline-Guided Scheduling and Orchestration Decisions
  - 共享工作负载, Sharing and Scheduling
  - 无停顿调度, Stall-Free Scheduling (Chunked Prefill)
  - 线程块调度, Thread Block Clusters and Distributed Shared Memory
  - NCCL 集合通信的波次调度, Wave scheduling of collectives

暂存内存, Scratch Memory

SDPA（缩放点积注意力；PyTorch）, PyTorch Optimized Attention Mechanisms

序列并行（参见 上下文并行（CP））

顺序读取与随机读取, Sequential Versus Random Read Patterns-Sequential Versus Random Read Patterns

SFU（参见 特殊函数单元（SFU））

SGLang
  - 动态批处理与连续调度合并, Continuous Scheduling
    - RadixAttention, Continuous Scheduling
  - 巨型核函数解码吞吐量, Megakernels for Inference
  - RadixAttention
    - 动态批处理与连续调度合并, Continuous Scheduling
    - 前缀缓存, Prefix Caching-Prefix Caching
  - 推测解码, Two-Model, Draft-Based Speculative Decoding and EAGLE

分片
  - 将小文件合并为大分片, Sequential Versus Random Read Patterns
  - DeepSeek Fire-Flyer 文件系统分片元数据, DeepSeek’s Fire-Flyer File System
  - 跨节点分片数据, Fast Storage and Data Locality

共享文件系统, Distributed, Parallel Filesystems and Object Stores

共享内存（SMEM）
  - 避免共享内存, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization-Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization

- 避免共享内存库冲突, Avoid Shared-Memory Bank Conflicts-Avoid Shared-Memory Bank Conflicts
  - 填充共享数组, Avoid Shared-Memory Bank Conflicts
- Blackwell GPU 共享内存, Avoid Shared-Memory Bank Conflicts
- 数据复用, Tiling and Data Reuse Using Shared Memory-Tiling and Data Reuse Using Shared Memory
- 分布式共享内存, Tiling with Thread Block Clusters
- 作为共享内存的 DRAM, Tiling and Data Reuse Using Shared Memory
- 动态共享内存分配, Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection-Dynamic Shared-Memory Allocation and Occupancy-Aware Kernel Selection
- 占用率与资源使用量的平衡, Tuning Occupancy
- PyTorch, PyTorch and Arithmetic Intensity
- 分块, Tiling and Data Reuse Using Shared Memory-Tiling and Data Reuse Using Shared Memory, Multilevel Microtiling and Software Prefetching
  - CUTLASS, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance
- Triton 编程, Accessing Shared Memory in Triton

__shared__ 区域, Distributed Shared Memory

工作负载共享和调度, Sharing and Scheduling

SHARP（NVIDIA 可扩展分层聚合与归约协议）, In-Network Aggregations with NVIDIA SHARP-In-Network Aggregations with NVIDIA SHARP
- NCCL 和, In-Network SHARP Aggregation
- 将聚合操作卸载到交换机, Multinode and Multirack Communication with GPUDirect RDMA

__shfl_sync, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization

移位全互连调度, Expert Communication Optimization

硅光子学, Co-Packaged Optics: Future of Networking Hardware

用于资源管理的简单 Linux 实用工具（参见 SLURM）

SimpleFSDP, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP

SIMT（单指令多线程）模型, Streaming Multiprocessor, Threads, and Warps, Triton Programming Model

同步多线程（SMT）, NUMA Awareness and CPU Pinning

单指令多线程（SIMT）模型, Streaming Multiprocessor, Threads, and Warps, Triton Programming Model

单核函数分块, Tiled and Persistent GEMM Kernel (Triton)-Tiled and Persistent GEMM Kernel (Triton)

单程序多数据（SPMD）, Triton Programming Model

Singularity, Reduce Image Size for Faster Container Startup

CPU 睡眠状态, CPU Frequency and C-states

划分 GPU（参见 MIG（NVIDIA 多实例 GPU））

Slinky, Job Scheduling with Kubernetes and SLURM

SLO 感知请求管理

- 提前拒绝, Early Rejection (Admission Control)-Early Rejection (Admission Control)
- 容错, Fault Tolerance
- 服务质量, Quality of Service

SLURM（用于资源管理的简单 Linux 实用工具）, Preintegrated Rack Appliance

- 集群调度器, Sharing and Scheduling
- 作业调度, Job Scheduling with Kubernetes and SLURM

SM 驻留限制, Choosing Threads-per-Block and Blocks-per-Grid Sizes

SmoothQuant, Activation Quantization

SM（流式多处理器）, Streaming Multiprocessor, Threads, and Warps-Streaming Multiprocessor, Threads, and Warps

- 所有 SM 上并发核函数的最大数, Stream-Ordered Memory Allocator
- GPU 架构, Understanding GPU Architecture-Understanding GPU Architecture
  - 每个 SM 的线程和寄存器, Choosing Threads-per-Block and Blocks-per-Grid Sizes
  - warp, Understanding GPU Architecture-Understanding GPU Architecture
- 高占用率, Threads, Warps, Blocks, and Grids
  - CUDA 编程, Maintaining High Occupancy and GPU Utilization-Maintaining High Occupancy and GPU Utilization
- 最少驻留线程块数, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds

- SM 驻留限制, Choosing Threads-per-Block and Blocks-per-Grid Sizes
- Tensor Cores, NVIDIA GPU Tensor Cores and Transformer Engine-NVIDIA GPU Tensor Cores and Transformer Engine
- 按 warp 执行的线程, Streaming Multiprocessor, Threads, and Warps
  - 延迟隐藏, Streaming Multiprocessor, Threads, and Warps
- warp 调度器, Warp Scheduling and Dual Issue Instructions

SMT（同步多线程）, NUMA Awareness and CPU Pinning

软件

- 软硬件协同设计, Mechanical Sympathy: Hardware-Software Codesign-Mechanical Sympathy: Hardware-Software Codesign
  - CUDA Pipeline API 加 TMA, Asynchronous Memory Prefetching and Tensor Memory Accelerator
  - DeepSeek 多头潜在注意力, Mechanical Sympathy: Hardware-Software Codesign
- NVIDIA Kubernetes GPU Operator 自动化管理, Kubernetes for Topology-Aware Container Orchestration and Networking

软件栈, NVIDIA Software Stack

- 概述, PyTorch and Higher-Level AI Frameworks
- 在 Python 中创建 CUDA 核函数, NVIDIA Software Stack
- CUDA Runtime 和 Toolkit, CUDA Toolkit and Runtime
  - Toolkit 的 C++ 和 Python 库, C++ and Python CUDA Libraries
- GPU 驱动程序, GPU Driver
  - nvidia-smi, GPU Driver
- PyTorch 简介, PyTorch and Higher-Level AI Frameworks

稀疏模型

- 作为混合专家模型, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China, Toward 100-Trillion-Parameter Models
- 降低有效计算需求, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China, Toward 100-Trillion-Parameter Models

稀疏性

- 优化的稀疏 GEMM 核函数, Structured Sparsity
- 剪枝, Structured Sparsity
  - 提高算术强度, Structured Sparsity
- 结构化稀疏性, Structured Sparsity-Structured Sparsity
  - 推理工作负载, Structured Sparsity
- TorchAO, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning

SpeCache, Speculative KV Prefetching for Faster TTFT

特殊函数单元（SFU）, Understanding GPU Architecture

Spectrum-X800 Ethernet, Co-Packaged Optics: Future of Networking Hardware

推测解码, Disaggregated Prefill and Decode Architecture, Speculative Decoding and Parallel Token Generation Techniques-Combining Decoding Techniques and Evaluating Complexity
- 组合技术, Combining Decoding Techniques and Evaluating Complexity
- 交错解码步骤, Interleaving Decode Steps from Multiple Requests
- Medusa 多头, Speculative Decoding and Parallel Token Generation Techniques
  - 多 token 解码, Multitoken Decoding with Medusa’s Multiple Heads-Multitoken Decoding with Medusa’s Multiple Heads
- 单模型自推测解码, Single-Model Self-Speculative Decoding
- 双模型草稿式, Two-Model, Draft-Based Speculative Decoding and EAGLE-Two-Model, Draft-Based Speculative Decoding and EAGLE
  - EAGLE 算法, Two-Model, Draft-Based Speculative Decoding and EAGLE

推测性 KV 预取, Speculative KV Prefetching for Faster TTFT

推测性 MoE 专家路由, Speculative MoE Expert Routing and Communication Reduction

SPMD（单程序多数据）, Triton Programming Model

无停顿调度, Stall-Free Scheduling (Chunked Prefill)

静态批处理, Dynamic Batching

用原子驱动工作队列替代静态索引, Atomic Queues

存储通信, Multirack and Storage Communication

存储 I/O 优化

- 云存储缓存, Distributed, Parallel Filesystems and Object Stores
- 通信与计算瓶颈的比较, Diagnosing Communication- Versus Compute-Bound Workloads
- 压缩数据, Tuning, Replicating, and Compressing Data
- 持续分析与调优工作流, Continuous Profiling and Tuning Workflow-Continuous Profiling and Tuning Workflow
- data pipeline tuning, Tuning the Data Pipeline
  - data loading and preprocessing, Efficient Data Loading and Preprocessing-Efficient Data Loading and Preprocessing
  - NeMo Curator for training datasets, Creating High-Quality LLM Datasets with NVIDIA NeMo Curator
  - NVIDIA Data Loading Library, Multimodal Data Processing with NVIDIA DALI
  - scaling out workers as GPUs scaled, Scaling Out Workers as You Scale Out Number of GPUs
- DeepSeek Fire-Flyer File System, DeepSeek’s Fire-Flyer File System-DeepSeek’s Fire-Flyer File System
- fast storage and data locality, Fast Storage and Data Locality
- monitoring I/O, Monitoring Storage I/O-Monitoring Storage I/O
- NFS servers, Distributed, Parallel Filesystems and Object Stores
  - tuning parameters, Distributed, Parallel Filesystems and Object Stores
- NVIDIA GDS, Using NVIDIA GDS-Measuring GDS with gdsio
- NVMe and filesystem tuned for throughput, Tuning NVMe and Filesystem for Throughput
  - multiqueue block I/O scheduler, Tuning NVMe and Filesystem for Throughput
- object storage, Distributed, Parallel Filesystems and Object Stores
- sequential versus random reads, Sequential Versus Random Read Patterns-Sequential Versus Random Read Patterns
  - filesystem optimized, Sequential Versus Random Read Patterns
  - read size tuned, Sequential Versus Random Read Patterns
- shared filesystems, Distributed, Parallel Filesystems and Object Stores
- striping files, Distributed, Parallel Filesystems and Object Stores

– tuning filesystems, Tuning, Replicating, and Compressing Data

straggler effect MoE models, Expert Parallelism

stream 0, Legacy Default Stream

流式汇编器（SASS）, Inline PTX and SASS Tuning for Microoptimizations-Inline PTX and SASS Tuning for Microoptimizations

– 随 GPU 架构代际变化, Inline PTX and SASS Tuning for Microoptimizations

流式多处理器（参见 SM（流式多处理器））

流式响应, Streaming Responses-Streaming Responses

用于异步执行的流, Asynchronous Execution with Streams

– （另请参见 CUDA 流）

跨步索引, Coalesced Versus Uncoalesced Global Memory Access

条带化文件, Distributed, Parallel Filesystems and Object Stores

结构化输出, Constrained Decoding Performance Implications

结构化稀疏性, Structured Sparsity-Structured Sparsity

表面对象 API, Read-Only Data Caches

重排, Avoid Shared-Memory Bank Conflicts

– 线程块重排, Thread Block Swizzling

对称内存, PyTorch Symmetric Memory

SymPy, Dynamic Shapes and Variable Sequence Lengths

– 关于动态形状, Dynamic Shapes and Variable Sequence Lengths

__syncthreads(), Cooperative Groups

systemctl, GPU Persistence Mode

系统性能工程师（参见 AI 系统性能工程师）

用于调优推理的系统级优化, Systems-Level Optimizations

– 错误处理, Error Handling

– GPU 利用率与延迟, Maximizing GPU Utilization and Throughput Versus Latency Trade-Offs

– KV 缓存, KV Cache Offloading and Memory Pool Allocation

– 内存, Memory

- 通信与计算重叠, Overlapping Communication and Computation-Overlapping Communication and Computation
- 功率和散热约束, Power and Thermal Constraints

T

tcmalloc, Tune Host CPU Memory Allocator

TCMALLOC_MAX_TOTAL_THREAD_CACHE_BYTES, Tune Host CPU Memory Allocator

TCMALLOC_RELEASE_RATE, Tune Host CPU Memory Allocator

TDP (thermal design power), GPU Clock Speeds and ECC

temporal load balancing, Dynamic Congestion-Aware Scheduling

Tensor Cores, NVIDIA GPU Tensor Cores and Transformer Engine-NVIDIA GPU Tensor Cores and Transformer Engine
- CUTLASS, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance-Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance
- mixed precision, Mixed Precision and Utilizing Tensor Cores-Transformer Engine and TMEM in Depth
- MMA API, Transformer Engine and TMEM in Depth-Transformer Engine and TMEM in Depth
- TF32, TF32 and Automatic Mixed Precision (PyTorch)
  - enabling, TF32 and Automatic Mixed Precision (PyTorch)
- theoretical throughput of NVL72 rack, NVIDIA GPU Tensor Cores and Transformer Engine
- TMEM, Feeding Tensor Cores with TMEM and TMA-Feeding Tensor Cores with TMEM and TMA
- Transformer Engine, Transformer Engine and TMEM in Depth
- very high active utilization, Execution Unit Contention

张量内存加速器（TMA）, Feeding Tensor Cores with TMEM and TMA
- 功能, Asynchronous Memory Prefetching and Tensor Memory Accelerator
- 将分块从全局内存取入共享内存, Asynchronous Memory Prefetching and Tensor Memory Accelerator-Asynchronous Memory Prefetching and Tensor Memory Accelerator

  - 减少全局内存流量, Reducing Global Memory Traffic with Thread Block Clusters-Reducing Global Memory Traffic with Thread Block Clusters
  - 软硬件协同设计, Asynchronous Memory Prefetching and Tensor Memory Accelerator
  - 启动 TMA 传输, Asynchronous Memory Prefetching and Tensor Memory Accelerator

张量并行（TP）, Parallelism Strategies for Serving Massive MoE Models, Tensor Parallelism
  - AsyncTP, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
  - 配合 FSDP, Combining FSDP with Tensor Parallel and Pipeline Parallel
  - 混合并行, Hybrid Parallelism
  - 与流水线并行比较, Pipeline Parallelism
  - 配合 PyTorch 编译器, Tensor and Pipeline Parallelism with torch.compile
  - 与 PP 和混合并行比较, Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)-Adaptive Parallelism Strategies (TP Versus PP Versus Hybrid)

TensorBoard 跟踪查看器, Multi-GPU Profiling with HTA

TensorRT-LLM (NVIDIA), PyTorch, Inference Engines, and CUDA Graphs

TetriInfer 的两级调度器, TetriInfer’ s two-level scheduler

纹理对象 API, Read-Only Data Caches

TF32, TF32 and Automatic Mixed Precision (PyTorch)
  - 启用, TF32 and Automatic Mixed Precision (PyTorch)

TFLOPS (teraFLOPS), Tiling and Data Reuse Using Shared Memory

推理的散热约束, Power and Thermal Constraints

热设计功耗（TDP）, GPU Clock Speeds and ECC

Thompson, Martin, Mechanical Sympathy: Hardware-Software Codesign

THP（透明大页）, Transparent Hugepages

线程块集群, Threads, Warps, Blocks, and Grids, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters
  - 与协作组的比较, Thread Block Clusters and Distributed Shared Memory
  - 说明, Thread Block Clusters and Distributed Shared Memory
  - 分布式共享内存, Thread Block Clusters and Distributed Shared Memory-Thread Block Pair

  - 并行化工作负载的算法, Designing Efficient Algorithms with Thread Block Clusters-Designing Efficient Algorithms with Thread Block Clusters
  - 协调线程块集群, Coordinating Thread Block Clusters with Cooperative Groups API-Coordinating Thread Block Clusters with Cooperative Groups API
  - 减少全局内存流量, Reducing Global Memory Traffic with Thread Block Clusters-Reducing Global Memory Traffic with Thread Block Clusters
  - 启动线程块集群, Launching a Thread Block Cluster, Designing Efficient Algorithms with Thread Block Clusters
  - 暂存内存, Scratch Memory
  - 在线程块之间共享状态, Distributed Shared Memory
  - 线程块集群说明, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters
  - 线程块对, Thread Block Pair-Thread Block Pair
  - 线程块重排, Thread Block Swizzling
  - 线程访问彼此的共享内存, Threads, Warps, Blocks, and Grids, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters, Thread Block Clusters and Distributed Shared Memory
  - 使用线程块集群和 DSMEM 的线程, Thread Block Clusters and Distributed Shared Memory
  - 结合线程块集群的 warp 专用化, Warp Specialization with Thread Block Clusters-Warp Specialization with Thread Block Clusters
- 核函数内流水线与核函数间重叠, Combining PDL and Thread Block Clusters with Warp Specialization-Combining PDL and Thread Block Clusters with Warp Specialization
- 启动线程块集群, Launching a Thread Block Cluster, Designing Efficient Algorithms with Thread Block Clusters
- 在线程块之间共享状态, Distributed Shared Memory
- 线程块对, Launching a Thread Block Cluster, Thread Block Pair-Thread Block Pair
- 分块, Tiling with Thread Block Clusters-Tiling with Thread Block Clusters
- 最多 16 个线程块, Tiling with Thread Block Clusters

线程块, Threads, Warps, Blocks, and Grids-Threads, Warps, Blocks, and Grids

- 协作模式, Cooperative Groups

- 在同一 GPC 上协同调度, Thread Block Clusters and Distributed Shared Memory
- 分布式共享内存, Thread Block Clusters and Distributed Shared Memory-Thread Block Pair
  - 在线程块之间共享状态, Distributed Shared Memory
  - 线程块集群说明, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters
  - 线程访问彼此的共享内存, Threads, Warps, Blocks, and Grids, Thread Block Clusters and Distributed Shared Memory
  - 使用线程块集群和 DSMEM 的线程, Thread Block Clusters and Distributed Shared Memory
- 分布式共享内存和
  - 线程访问彼此的共享内存, Intra-Kernel Pipelining, Warp Specialization, and Cooperative Thread Block Clusters
- CUDA 编程
  - blocksPerGrid, CUDA Programming Refresher, CUDA Programming Refresher
  - 作为启动参数的 blocksPerGrid, Configuring Launch Parameters: Blocks per Grid and Threads per Block-Configuring Launch Parameters: Blocks per Grid and Threads per Block
  - 每个 SM 上最少驻留线程块数, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
  - threadsPerBlock, CUDA Programming Refresher, CUDA Programming Refresher
  - 作为启动参数的 threadsPerBlock, Configuring Launch Parameters: Blocks per Grid and Threads per Block-Configuring Launch Parameters: Blocks per Grid and Threads per Block
- 在线程块之间共享状态, Distributed Shared Memory
- 线程块对, Launching a Thread Block Cluster, Thread Block Pair-Thread Block Pair
  - （另请参见 线程块集群）
- 线程块大小, Choosing Threads-per-Block and Blocks-per-Grid Sizes
- 分块, Tiling and Data Reuse Using Shared Memory

线程

- 合并与非合并全局内存访问, Coalesced Versus Uncoalesced Global Memory Access-Coalesced Versus Uncoalesced Global Memory Access
- 协作组, Cooperative Groups-When to Combine Persistent Kernels and Cooperative Groups
- 分布式共享内存, Thread Block Clusters and Distributed Shared Memory
- GPU 架构, Threads, Warps, Blocks, and Grids-Threads, Warps, Blocks, and Grids
- warp 内通信, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization-Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization
- CUDA 编程
  - 尖括号语法（<<< >>>）, CUDA Programming Refresher
  - 最大 threadsPerBlock 编译时参数, Tuning Occupancy with Launch Bounds-Tuning Occupancy with Launch Bounds
  - threadsPerBlock, CUDA Programming Refresher, CUDA Programming Refresher
  - 作为启动参数的 threadsPerBlock, Configuring Launch Parameters: Blocks per Grid and Threads per Block-Configuring Launch Parameters: Blocks per Grid and Threads per Block
- 分块, Tiling and Data Reuse Using Shared Memory
- warp, Streaming Multiprocessor, Threads, and Warps, Understanding GPU Architecture-Understanding GPU Architecture
  - warp 发散, Threads, Warps, Blocks, and Grids

threadsPerBlock, CUDA Programming Refresher, CUDA Programming Refresher

- launch parameter, Configuring Launch Parameters: Blocks per Grid and Threads per Block-Configuring Launch Parameters: Blocks per Grid and Threads per Block

吞吐量指标

- 预填充和解码解耦, Impact on Latency (TTFT) and Throughput (TPOT)
- FP4, FP8, FP16, NVIDIA GPU Tensor Cores and Transformer Engine
- “有效吞吐量”, Measuring “Goodput” Useful Throughput
- NVL72 rack Tensor Core throughput, NVIDIA GPU Tensor Cores and Transformer Engine

- NVMe and filesystem tuned for throughput, Tuning NVMe and Filesystem for Throughput

ThunderMLA (Stanford), ThunderMLA (Stanford)

TileIR, Tiling and Data Reuse Using Shared Memory

分块

- 提高算术强度, Tiling and Data Reuse Using Shared Memory, Multilevel Microtiling and Software Prefetching
  - 多级分块, Multilevel Microtiling and Software Prefetching
- 双缓冲协作分块, Cooperative Tiling and Double-Buffering with the CUDA Pipeline API-Cooperative Tiling and Double-Buffering with the CUDA Pipeline API
- 多级分块, Multilevel Microtiling and Software Prefetching
- PyTorch, PyTorch and Arithmetic Intensity
- 用于分块的共享内存, Tiling and Data Reuse Using Shared Memory-Tiling and Data Reuse Using Shared Memory, Multilevel Microtiling and Software Prefetching
  - CUTLASS, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance
- 单核函数分块, Tiled and Persistent GEMM Kernel (Triton)-Tiled and Persistent GEMM Kernel (Triton)
- 软件预取, Multilevel Microtiling and Software Prefetching
- 线程块集群, Tiling with Thread Block Clusters-Tiling with Thread Block Clusters
- 分块, Tiling and Data Reuse Using Shared Memory
- TMA 将数据从全局内存取入共享内存, Asynchronous Memory Prefetching and Tensor Memory Accelerator-Asynchronous Memory Prefetching and Tensor Memory Accelerator

每个输出 token 的时间（参见 TPOT（每个输出 token 的时间））

首 token 时间（参见 TTFT（首 token 时间））

TLB（参见 转换后备缓冲区（TLB））

TMA（参见 张量内存加速器（TMA））

TMEM, Feeding Tensor Cores with TMEM and TMA-Feeding Tensor Cores with TMEM and TMA

- double buffering with CUTLASS, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance
- Transformer Engine, Transformer Engine and TMEM in Depth

工具包（参见 CUDA Toolkit）

torch.compile（参见 PyTorch 编译器（torch.compile））

torch.profiler（参见 PyTorch 分析器）

TorchAO, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning

- 剪枝, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning
- 量化, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning
  - INT8 量化, Transformer Engine and TMEM in Depth
- 稀疏性, PyTorch Architecture Optimization (torchao), Quantization, Sparsity, and Pruning

TorchBench, Continuous Integration and Performance Benchmarking

TorchDynamo（PyTorch）, Using the PyTorch Compiler, TorchDynamo for Bytecode Capture and Graph Extraction-TorchDynamo for Bytecode Capture and Graph Extraction

- 调试数值准确性, Debugging Numerical Correctness and Accuracy
- 动态形状, Profiling and Debugging Compiler Performance Issues, TorchDynamo for Bytecode Capture and Graph Extraction
- explain() 输出, Profiling and Debugging Compiler Performance Issues, Graph Breaks and TorchDynamo explain()
  - 与 all-reduce 相关的警告, Tensor and Pipeline Parallelism with torch.compile
  - 图中断, Profiling and Debugging Compiler Performance Issues, FSDP with torch.compile
  - 已调试的图中断, Graph Breaks and TorchDynamo explain()
- FX Graph 输出, TorchDynamo for Bytecode Capture and Graph Extraction
  - 在每个阶段后转储图, Debugging Compiler Phases, Graph Breaks, and Performance
- 图中断, TorchDynamo for Bytecode Capture and Graph Extraction

    - 将编译器条件分支作为张量操作, TorchDynamo for Bytecode Capture and Graph Extraction
    - 强制在编译时抛出错误, TorchDynamo for Bytecode Capture and Graph Extraction
  - 最小化工具, Debugging Numerical Correctness and Accuracy
  - 分析与调试编译器问题, Profiling and Debugging Compiler Performance Issues
  - PyTorch 编译器流水线, PyTorch Compiler Deep Dive
  - PyTorch 编译器栈, PyTorch and Higher-Level AI Frameworks, PyTorch Compiler (torch.compile), PyTorch Compiler, OpenAI Triton, and XLA Backends
  - 错误或回退的策略, TorchDynamo for Bytecode Capture and Graph Extraction
  - TORCH_LOGS, Profiling and Debugging Compiler Performance Issues

TORCHDYNAMO_REPRO_*, Debugging Compiler Phases, Graph Breaks, and Performance

TorchEval (PyTorch), NVTX Markers and Profiling Tools, NVTX Markers and Profiling Tools, Optimizing the Data Input Pipeline

TorchInductor (PyTorch), TorchInductor Backend Code Generation

  - arithmetic intensity increased, Increasing CUDA Kernel Efficiency and Arithmetic Intensity
  - automatic mixed precision, TF32 and Automatic Mixed Precision (PyTorch)-TF32 and Automatic Mixed Precision (PyTorch)
  - autotuning, Using the PyTorch Compiler, Autotuning with TorchInductor-Autotuning with TorchInductor
    - max-autotune compile mode, Using the PyTorch Compiler, PyTorch Compiler Deep Dive, Autotuning with TorchInductor
  - CUDA Graphs, Autotuning with TorchInductor
  - debug mode, Performance Hints and Debugging Generated Code, Debugging Compiler Phases, Graph Breaks, and Performance
    - inspecting TorchInductor code, Performance Hints and Debugging Generated Code
  - FX Graph, AOT Autograd Fusion for Forward and Backward Passes
  - introduction, PyTorch and Higher-Level AI Frameworks
  - IR, TorchInductor Backend Code Generation

- kernel fusion, Tuning Occupancy with PyTorch, Kernel Fusion, TorchDynamo for Bytecode Capture and Graph Extraction
  - fused attention kernels, PyTorch, CUDA Pipeline API, and Warp Specialization
  - GEMM kernel prologue and epilogue, Using the PyTorch Compiler
- minifier tool, Debugging Numerical Correctness and Accuracy
- optimizations versus writing custom kernels, Compiling Versus Writing Custom Kernels
- PrimTorch IR simplifying operations, PrimTorch IR (Prims) Simplified Operator Set
- profiling and benchmarking kernels generated, Profiling and Debugging Compiler Performance Issues
- PyTorch compiler pipeline, PyTorch Compiler Deep Dive
- PyTorch compiler stack, PyTorch and Higher-Level AI Frameworks, PyTorch Compiler (torch.compile), PyTorch Compiler, OpenAI Triton, and XLA Backends
- SimpleFSDP, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
- symbolic shapes, TorchInductor Backend Code Generation
- Triton based, PyTorch and Higher-Level AI Frameworks, Using the PyTorch Compiler, TorchInductor Backend Code Generation, Writing Custom Kernels with OpenAI Triton

TORCHINDUCTOR_BENCHMARK_KERNEL, Profiling and Debugging Compiler Performance Issues

TORCHINDUCTOR_UNIQUE_KERNEL_NAMES, Profiling and Debugging Compiler Performance Issues

TorchTitan (PyTorch), TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
- AsyncTP, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP
- SimpleFSDP reducing memory usage, TorchTitan, AsyncTP, AutoParallel, and SimpleFSDP

TORCH_COMPILE_DEBUG, Debugging Compiler Phases, Graph Breaks, and Performance

TORCH_LOGS, Profiling and Debugging Compiler Performance Issues
- compiler debugging, Debugging Compiler Phases, Graph Breaks, and Performance

  - logging options summary, Debugging Compiler Phases, Graph Breaks, and Performance
  - performance hints, Performance Hints and Debugging Generated Code

TP（参见 张量并行（TP））

TPOT (time per output token)

  - disaggregation of prefill and decode, Impact on Latency (TTFT) and Throughput (TPOT)
  - prefill and decode phases, Scaling Disaggregated Prefill and Decode for Inference

training

  - 100-trillion parameter models, Toward 100-Trillion-Parameter Models
    - loading training data, Toward 100-Trillion-Parameter Models
  - costs
    - DeepSeek-R1, Introduction and AI System Overview
    - DeepSeek-V3, DeepSeek Scales to ~680-Billion Parameter Models Despite US Export Hardware Restrictions in China
    - Google Gemini Ultra, Introduction and AI System Overview
    - OpenAI GPT-4, Introduction and AI System Overview
  - datasets via NVIDIA NeMo Curator, Creating High-Quality LLM Datasets with NVIDIA NeMo Curator

Transformer Engine (TE; NVIDIA), NVIDIA GPU Tensor Cores and Transformer Engine-NVIDIA GPU Tensor Cores and Transformer Engine

  - balancing accuracy and precision, NVIDIA GPU Tensor Cores and Transformer Engine
  - installing only if calling TE modules directly, Autotuning with TorchInductor
  - mechanical sympathy, Mechanical Sympathy: Hardware-Software Codesign
  - modern GPUs using, Mechanical Sympathy: Hardware-Software Codesign
  - PyTorch compiler, Autotuning with TorchInductor
  - TMEM, Transformer Engine and TMEM in Depth

translation lookaside buffer (TLB), Transparent Hugepages

transparency

  - AI systems performance engineers, Transparency and Reproducibility

– DeepSeek Open-Source Week, Transparency and Reproducibility

transparent hugepages (THP), Transparent Hugepages

tree communication algorithm, NCCL Communication Algorithms

tree parallelism communication algorithm, NCCL Communication Algorithms

Triton (OpenAI)

– advanced kernel implementations, Advanced Triton Kernel Implementations-Software Pipelining and Double Buffering with Triton
  – double-buffering pipelining, Software Pipelining and Double Buffering with Triton-Software Pipelining and Double Buffering with Triton
  – persistent matmul kernel, Tiled and Persistent GEMM Kernel (Triton)-Tiled and Persistent GEMM Kernel (Triton)
  – warp specialization, Warp Specialization with Triton
– CUDA kernels in Python, NVIDIA Software Stack
– custom kernels, Writing Custom Kernels with OpenAI Triton-Profiling with Triton Proton Profiler
– language and compiler, C++ and Python CUDA Libraries
– open source, C++ and Python CUDA Libraries
– programming
  – accessing shared memory, Accessing Shared Memory in Triton
  – autotuning Triton kernels, Autotuning Triton Kernels
  – custom kernels, Writing Custom Kernels with OpenAI Triton-Profiling with Triton Proton Profiler
  – Nsight Compute and Systems support, Profiling with Triton Proton Profiler
  – profiling with Triton Proton Profiler, Profiling with Triton Proton Profiler
  – programming model, Triton Programming Model
  – registering custom kernels with PyTorch, TorchInductor Backend Code Generation, Registering Custom Kernels with PyTorch-Registering Custom Kernels with PyTorch
  – tuning kernel launch parameters, Tuning Kernel-Launch Parameters
– Proton Profiler, Profiling with Triton Proton Profiler
– smart compilers, Smart Compilers and Automated Code Optimizations-Smart Compilers and Automated Code Optimizations

  - TorchInductor basis, PyTorch and Higher-Level AI Frameworks, Using the PyTorch Compiler, TorchInductor Backend Code Generation, Writing Custom Kernels with OpenAI Triton

troubleshooting inference, Inference Troubleshooting Recipes

TTFT (time to first token)

  - constrained outputs, Constrained Decoding Performance Implications
  - context parallelism, Context (Sequence) Parallelism
  - disaggregation of prefill and decode, Impact on Latency (TTFT) and Throughput (TPOT)
  - prefill and decode phases, Scaling Disaggregated Prefill and Decode for Inference
  - speculative KV prefetching, Speculative KV Prefetching for Faster TTFT-Speculative KV Prefetching for Faster TTFT

U

UCX (Unified Communication X; NVIDIA), Enabling Peer-to-Peer DMA and UCX

GPU 利用不足导致的停顿, 迭代分析并确定内核瓶颈-迭代分析并确定内核瓶颈

  - 内核优化, 优化内核-优化内核
  - Nsight Compute 指标, 迭代分析并确定内核瓶颈

统一通信 X（UCX；NVIDIA）, Enabling Peer-to-Peer DMA and UCX

统一 CPU-GPU 内存, CPU 与 GPU 超级芯片

升级与更新

  - 可用的最高层级和最新 API, 异步内存预取与 Tensor Memory Accelerator
  - NCCL release notes, Pitfall #6: NCCL communicator hangs, errors, or shuts down completely
  - Nsight Systems and Nsight Compute with latest GPUs, Profiler-Guided Analysis
  - NVIDIA 持续更新库, 使用 CUTLASS 实现最佳算术强度与 Tensor Core 性能
  - 升级硬件的投资回报率, 升级硬件的投资回报率（ROI）
  - 更新但要验证, 持续性能分析与调优工作流

利用率最大化策略, 自适应批处理与分块预填充调度

V

通过动态形状支持可变长度序列, 动态形状与可变序列长度-动态形状与可变序列长度

向量化内存访问, 向量化内存访问-向量化内存访问

Vera Rubin 超级芯片（VR200）, Vera Rubin Superchip (2026)

vLLM

- 受约束解码, Constrained Decoding Performance Implications
- CUDA Graphs, PyTorch, Inference Engines, and CUDA Graphs
- 解耦的预填充, KV Cache Data Transfer and NIXL, Deploying Disaggregated Prefill and Decode with Kubernetes
- 动态批处理与连续调度合并, Continuous Scheduling
  - PagedAttention, Continuous Scheduling
- LMCache, Profiling, Debugging, and Tuning Inference Performance
  - disaggregated prefill and decode, Scaling Disaggregated Prefill and Decode for Inference, Zero-Copy GPU-to-GPU Transfer
- 巨型核函数解码吞吐量, Megakernels for Inference
- NIXL-vLLM 集成, KV Cache Data Transfer and NIXL
- NVLink 域, NVIDIA’s “AI Supercomputer in a Rack”
- PagedAttention, Continuous Scheduling, KV Cache Offloading and Memory Pool Allocation, Memory management for the KV cache
- 并行解码, Interleaving Decode Steps from Multiple Requests
- 前缀缓存, Prefix Caching, KV Cache Reuse and Prefix Sharing
- 推测解码, Two-Model, Draft-Based Speculative Decoding and EAGLE
- 推理故障排查, Inference Troubleshooting Recipes

VR200（Vera Rubin 超级芯片）, Vera Rubin Superchip (2026)

W

Warp（参见 NVIDIA Warp）

warp 停顿原因, Analyzing Warp Stall Reasons with Nsight Compute-Other Stall Reasons

warps, Streaming Multiprocessor, Threads, and Warps

– efficiency via PyTorch, PyTorch Considerations for Warp-Level Efficiency-PyTorch Considerations for Warp-Level Efficiency
– efficient intrawarp communication, Efficient Intrawarp Communication with Warp Intrinsics
– execution efficiency improved, Improving Warp Execution Efficiency (Warp Divergence)-PyTorch Considerations for Warp-Level Efficiency
– GPU architecture, Understanding GPU Architecture-Understanding GPU Architecture
– high occupancy, Threads, Warps, Blocks, and Grids
– ILP increased for latency-bound GPUs, Iteratively Profiling and Determining the Kernel Bottleneck
– latency hiding, Streaming Multiprocessor, Threads, and Warps
– maintaining high occupancy, Maintaining High Occupancy and GPU Utilization
– shuffle intrinsics avoiding shared memory, Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization-Warp Shuffle Intrinsics: Avoid Shared Memory and Explicit Synchronization
– stalls or frequent pauses, Analyzing Warp Stall Reasons with Nsight Compute-Other Stall Reasons
  – execution-dependency stalls, Execution-Dependency Stalls
  – memory-related stalls, Memory-Related Stalls
– warp divergence, Threads, Warps, Blocks, and Grids, Improving Warp Execution Efficiency (Warp Divergence)
  – avoiding, Techniques to Avoid Warp Divergence
  – causes of, Causes of Warp Divergence
  – efficient intrawarp communication, Efficient Intrawarp Communication with Warp Intrinsics
  – improving warp execution efficiency, Improving Warp Execution Efficiency (Warp Divergence)-PyTorch Considerations for Warp-Level Efficiency
  – predication to minimize, Using Predication to Minimize Divergence-Using Predication to Minimize Divergence
  – profiling and detecting, Profiling and Detecting Warp Divergence
  – PyTorch considerations for efficiency, PyTorch Considerations for Warp-Level Efficiency-PyTorch Considerations for Warp-Level Efficiency

- warp specialization
  - CUDA streams and warp specialization, Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)-Pipelining with Warp Specialization (Intra-Kernel) and CUDA Streams (Inter-Kernel)
  - CUTLASS, Using CUTLASS for Optimal Arithmetic Intensity and Tensor Core Performance
  - inter-kernel pipelining with CUDA streams, Overlapping Kernel Execution with CUDA Streams-Overlapping Kernel Execution with CUDA Streams
  - inter-kernel pipelining with CUDA streams and thread block clusters, Warp Specialization with Thread Block Clusters and CUDA Streams
  - intra-kernel pipelining, Warp Specialization and the Producer-Consumer Model-Warp Specialization and the Producer-Consumer Model
  - intra-kernel pipelining with CUDA Pipeline API, Using CUDA Pipeline API for Warp Specialization-Using CUDA Pipeline API for Warp Specialization
  - intra-kernel pipelining with inter-kernel overlap, Combining PDL and Thread Block Clusters with Warp Specialization-Combining PDL and Thread Block Clusters with Warp Specialization
  - intra-kernel pipelining with persistent kernels, Persistent Kernels and Warp Specialization
  - intra-kernel pipelining with thread block clusters, Warp Specialization with Thread Block Clusters-Warp Specialization with Thread Block Clusters
  - Triton, Warp Specialization with Triton

本书网页, How to Contact Us

权重切分并行策略, Parallelism Strategies for Serving Massive MoE Models

工作负载

- 动态工作队列, Dynamic Scheduling with Atomic Work Queues-Atomic Queues
  - 原子计数器, Atomic Counters
  - 原子队列, Atomic Counters, Atomic Queues-Atomic Queues
  - Nsight Compute, Atomic Counters-Atomic Counters
- 内存受限工作负载示例, Maintaining High Occupancy and GPU Utilization
- 持久核函数, Common Workloads for Persistent Kernels

  - 带宽不足时的性能分析, Pitfall #4: Insufficient network bandwidth or misconfigured NICs
  - 工作负载的合适占用率, Find the Right Occupancy for Your Workload
  - 共享和调度, Sharing and Scheduling
  - 用线程块集群并行化工作负载, Designing Efficient Algorithms with Thread Block Clusters-Designing Efficient Algorithms with Thread Block Clusters

回写, Filesystem Caching and Write-Back

X

XFS

  - 优化, Sequential Versus Random Read Patterns
  - 为吞吐量调优, Tuning NVMe and Filesystem for Throughput

XLA, PyTorch XLA Backend

  - 激活, PyTorch XLA Backend
  - Google Cloud TPU, TorchInductor Backend Code Generation, PyTorch XLA Backend
  - PyTorch 编译器流水线, PyTorch Compiler Deep Dive, TorchDynamo for Bytecode Capture and Graph Extraction, AOT Autograd Fusion for Forward and Backward Passes

Z

FSDP 中的 ZeRO Stage-3, FSDP Automatic Checkpointing and Offloading

零拷贝 GPU 到 GPU 传输, Zero-Copy GPU-to-GPU Transfer-Zero-Copy GPU-to-GPU Transfer

零拷贝注册, Persistent NCCL User Buffers and Zero-Copy Registration

关于作者

Chris Fregly 是一位性能工程师和 AI 产品负责人，曾在 Netflix、Databricks、Amazon Web Services（AWS）及多家初创公司推动创新。他曾领导以性能为核心的工程团队，构建 AI/ML 产品、扩展市场推广项目，并降低大规模生成式 AI 和分析工作负载的成本。Chris 与人合著了 *Data Science on AWS* 和 *Generative AI on AWS*（O’Reilly），并创作了 O’Reilly 课程“High-Performance AI in Production with NVIDIA GPUs”。他的工作涵盖内核级调优、编译器驱动的加速、分布式训练和高吞吐量推理。Chris 组织全球性的 AI Performance Engineering meetup，该社群在全球拥有超过 10 万名成员。

版本说明

AI Systems Performance Engineering 封面上的动物是东北非猎豹（*Acinonyx jubatus soemmeringii*），又称苏丹猎豹。这一猎豹亚种主要分布于东北非，包括埃塞俄比亚、索马里和南苏丹等地区。

东北非猎豹与所有猎豹一样，具有标志性的纤细身躯和长肢，这使它能以惊人的速度奔跑（最高约每小时 65–75 英里）。与南方近亲相比，它的泪痕（从眼角延伸至嘴部的黑色条纹）变化更多，毛皮也更厚。

猎豹以瞪羚、斑马和黑斑羚等中型有蹄动物为食，是技艺高超的猎手。它们的捕猎策略结合了隐蔽接近与爆发性的短距离冲刺。为了避免被能制服它们的狮子、鬣狗等其他捕食者抢走食物，它们会迅速进食。不同于许多其他大型猫科动物，猎豹不会咆哮，而是用呼噜声、啁啾声、嘶声和低吼声等多种声音交流。它们在交配季节或雌豹与幼崽交流时尤其爱发声。一胎通常有三到五只幼崽，幼崽在出生后头一年半与母亲共同生活。雄性猎豹往往更具社交性，会组成松散联盟，彼此梳理毛发并一同捕猎。

由于栖息地丧失和偷猎，IUCN 将大多数猎豹物种列为易危或濒危。O’Reilly 封面上的许多动物都面临灭绝威胁；它们对世界都很重要。

封面插画由 José Marzan Jr. 创作，基于为 O’Reilly Media 绘制的原始插图。丛书设计由 Edie Freedman、Ellie Volckhausen 和 Karen Montgomery 完成。封面字体为 Gilroy Semibold 和 Guardian Sans；正文字体为 Adobe Minion Pro，标题字体为 Adobe Myriad Condensed，代码字体为 Dalton Maag 的 Ubuntu Mono。